FarmVerse

Complete Source Code & Project Files

Formatted source listing from the Higgsfield website project checkout

576 text source/config/documentation files included

Website: https://farmverse.higgsfield.app

Notes: Git history, installed dependencies, generated build outputs, lockfiles, binary assets and environment files are excluded. Images and other binary assets are not printed as source text. This PDF is a readable reference; use the ZIP project archive for editing and running the website.

Contents — Project files

.github/workflows/ci.yml
.gitignore
app/.gitignore
app/.prettierignore
app/.prettierrc
app/AGENTS.md
app/app.manifest.json
app/bunfig.toml
app/components.json
app/eslint.config.js
app/migrations/0001_init.sql
app/package.json
app/packages/.snapshot.json
app/packages/app-landing/package.json
app/packages/app-landing/src/index.ts
app/packages/app-landing/src/landing-page.tsx
app/packages/app-landing/src/schema.ts
app/packages/app-landing/tsconfig.json
app/packages/fnf-react/AGENTS.md
app/packages/fnf-react/ai/AGENTS.md
app/packages/fnf-react/package.json
app/packages/fnf-react/README.md
app/packages/fnf-react/src/__tests__/attachments.test.ts
app/packages/fnf-react/src/__tests__/character-query.test.ts
app/packages/fnf-react/src/__tests__/generation-cache.test.ts
app/packages/fnf-react/src/__tests__/generation-fold.test.ts
app/packages/fnf-react/src/__tests__/generation-run.test.ts
app/packages/fnf-react/src/__tests__/live-feed-generations.test.ts
app/packages/fnf-react/src/__tests__/profile-query.test.ts
app/packages/fnf-react/src/__tests__/provider-wire.test.ts
app/packages/fnf-react/src/__tests__/queries.test.ts
app/packages/fnf-react/src/__tests__/realtime-query.test.ts
app/packages/fnf-react/src/__tests__/realtime.test.ts
app/packages/fnf-react/src/__tests__/reference-query.test.ts
app/packages/fnf-react/src/__tests__/test-utils.ts
app/packages/fnf-react/src/attachments-hook.ts
app/packages/fnf-react/src/attachments.ts
app/packages/fnf-react/src/character-query.ts
app/packages/fnf-react/src/cost-query.ts
app/packages/fnf-react/src/external-store-hook.ts
app/packages/fnf-react/src/external-store.ts
app/packages/fnf-react/src/generation-cache.ts
app/packages/fnf-react/src/generation-fold.ts
app/packages/fnf-react/src/generation-query.ts
app/packages/fnf-react/src/generation-run-hook.ts

app/packages/fnf-react/src/generation-run.ts
app/packages/fnf-react/src/index.ts
app/packages/fnf-react/src/job-set-query.ts
app/packages/fnf-react/src/jobs-feed-query.ts
app/packages/fnf-react/src/keys.ts
app/packages/fnf-react/src/live-feed-generations-hook.ts
app/packages/fnf-react/src/pool.ts
app/packages/fnf-react/src/profile-query.ts
app/packages/fnf-react/src/provider.ts
app/packages/fnf-react/src/realtime-query.ts
app/packages/fnf-react/src/realtime.ts
app/packages/fnf-react/src/reference-query.ts
app/packages/fnf-react/src/wire-preview.ts
app/packages/fnf-react/src/workspace-switch.ts
app/packages/fnf-react/tsconfig.json
app/packages/fnf-react/vitest.config.ts
app/packages/fnf/.gitignore
app/packages/fnf/ai/AGENTS.md
app/packages/fnf/CLAUDE.md
app/packages/fnf/package.json
app/packages/fnf/README.md
app/packages/fnf/scripts/make-publish-package.mjs
app/packages/fnf/scripts/smoke-publish.mjs
app/packages/fnf/src/__tests__/define-job.test.ts
app/packages/fnf/src/__tests__/errors.test.ts
app/packages/fnf/src/__tests__/group.test.ts
app/packages/fnf/src/__tests__/llm-client.test.ts
app/packages/fnf/src/__tests__/llm-errors.test.ts
app/packages/fnf/src/__tests__/normalize.test.ts
app/packages/fnf/src/__tests__/selectors.test.ts
app/packages/fnf/src/__tests__/spec.test.ts
app/packages/fnf/src/__tests__/types.test.ts
app/packages/fnf/src/__tests__/z.test.ts
app/packages/fnf/src/backend.ts
app/packages/fnf/src/characters/__tests__/characters.test.ts
app/packages/fnf/src/characters/client.ts
app/packages/fnf/src/characters/codec.ts
app/packages/fnf/src/characters/index.ts
app/packages/fnf/src/characters/types.ts
app/packages/fnf/src/client/__tests__/submit.test.ts
app/packages/fnf/src/client/adjust.ts
app/packages/fnf/src/client/cancel.ts
app/packages/fnf/src/client/context.ts
app/packages/fnf/src/client/cost.ts
app/packages/fnf/src/client/get-set.ts
app/packages/fnf/src/client/get.ts
app/packages/fnf/src/client/index.ts

app/packages/fnf/src/client/list.ts
app/packages/fnf/src/client/poll.ts
app/packages/fnf/src/client/submit-input.ts
app/packages/fnf/src/client/submit.ts
app/packages/fnf/src/client/wait.ts
app/packages/fnf/src/define-job.ts
app/packages/fnf/src/errors.ts
app/packages/fnf/src/group.ts
app/packages/fnf/src/groups/__tests__/media-meta.test.ts
app/packages/fnf/src/groups/__tests__/media.test.ts
app/packages/fnf/src/groups/__tests__/prompt.test.ts
app/packages/fnf/src/groups/media.ts
app/packages/fnf/src/groups/prompt.ts
app/packages/fnf/src/headers.ts
app/packages/fnf/src/index.ts
app/packages/fnf/src/jobs/__tests__/jobs.test.ts
app/packages/fnf/src/jobs/__tests__/product-parity.test.ts
app/packages/fnf/src/jobs/__tests__/seedance-2-5.test.ts
app/packages/fnf/src/jobs/checks.ts
app/packages/fnf/src/jobs/dimensions.ts
app/packages/fnf/src/jobs/gpt-image-2.ts
app/packages/fnf/src/jobs/grok-imagine.ts
app/packages/fnf/src/jobs/happy-horse.ts
app/packages/fnf/src/jobs/image-helpers.ts
app/packages/fnf/src/jobs/index.ts
app/packages/fnf/src/jobs/kling-3-motion-control.ts
app/packages/fnf/src/jobs/kling-3.ts
app/packages/fnf/src/jobs/kling.ts
app/packages/fnf/src/jobs/nano-banana-2-upscale.ts
app/packages/fnf/src/jobs/nano-banana-2.ts
app/packages/fnf/src/jobs/nano-banana-flash.ts
app/packages/fnf/src/jobs/nano-banana-shared.ts
app/packages/fnf/src/jobs/recraft-v4-1.ts
app/packages/fnf/src/jobs/seedance-2-0.ts
app/packages/fnf/src/jobs/seedance-2-5.ts
app/packages/fnf/src/jobs/seedream-v4-5.ts
app/packages/fnf/src/jobs/soul-v2.ts
app/packages/fnf/src/jobs/text2image-soul.ts
app/packages/fnf/src/jobs/todo.txt
app/packages/fnf/src/jobs/topaz-image-upscale.ts
app/packages/fnf/src/jobs/upscale-helpers.ts
app/packages/fnf/src/jobs/veo-3-1-lite.ts
app/packages/fnf/src/jobs/video-helpers.ts
app/packages/fnf/src/jobs/video-upscale.ts
app/packages/fnf/src/jobs/wan-2-7.ts
app/packages/fnf/src/llm/backend.ts
app/packages/fnf/src/llm/client.ts

app/packages/fnf/src/llm/errors.ts
app/packages/fnf/src/media-meta.ts
app/packages/fnf/src/media/__tests__/blob-uploader.test.ts
app/packages/fnf/src/media/__tests__/media.test.ts
app/packages/fnf/src/media/blob-uploader.ts
app/packages/fnf/src/media/client.ts
app/packages/fnf/src/media/context.ts
app/packages/fnf/src/media/dom-meta-resolver.ts
app/packages/fnf/src/media/errors.ts
app/packages/fnf/src/media/get.ts
app/packages/fnf/src/media/index.ts
app/packages/fnf/src/media/list.ts
app/packages/fnf/src/media/mime.ts
app/packages/fnf/src/media/resolve.ts
app/packages/fnf/src/media/types.ts
app/packages/fnf/src/media/upload.ts
app/packages/fnf/src/normalize.ts
app/packages/fnf/src/observability/__tests__/observability.test.ts
app/packages/fnf/src/observability/index.ts
app/packages/fnf/src/profile/client.ts
app/packages/fnf/src/profile/context.ts
app/packages/fnf/src/profile/credits.ts
app/packages/fnf/src/profile/get.ts
app/packages/fnf/src/profile/index.ts
app/packages/fnf/src/profile/mappers.ts
app/packages/fnf/src/profile/switch.ts
app/packages/fnf/src/profile/types.ts
app/packages/fnf/src/realtime/__tests__/realtime.test.ts
app/packages/fnf/src/realtime/client.ts
app/packages/fnf/src/realtime/codec.ts
app/packages/fnf/src/realtime/index.ts
app/packages/fnf/src/realtime/types.ts
app/packages/fnf/src/references/__tests__/references.test.ts
app/packages/fnf/src/references/client.ts
app/packages/fnf/src/references/codec.ts
app/packages/fnf/src/references/index.ts
app/packages/fnf/src/references/types.ts
app/packages/fnf/src/registry.ts
app/packages/fnf/src/selectors.ts
app/packages/fnf/src/spec.ts
app/packages/fnf/src/transport.ts
app/packages/fnf/src/typed-api.test-d.ts
app/packages/fnf/src/types.ts
app/packages/fnf/src/workflow-platform/__tests__/fetch-transport.test.ts
app/packages/fnf/src/workflow-platform/__tests__/workflow-platform-adapter.test.ts
app/packages/fnf/src/workflow-platform/fetch-transport.ts
app/packages/fnf/src/workflow-platform/index.ts

app/packages/fnf/src/workflow-platform/job-response-normalize.ts
app/packages/fnf/src/workflow-platform/workflow-platform-adapter.ts
app/packages/fnf/src/z.ts
app/packages/fnf/tsconfig.json
app/packages/fnf/vite.config.ts
app/packages/fnf/vitest.config.ts
app/packages/quanta/.gitignore
app/packages/quanta/ai/AGENTS.md
app/packages/quanta/CLAUDE.md
app/packages/quanta/package.json
app/packages/quanta/README.md
app/packages/quanta/src/components/accordion/accordion.css
app/packages/quanta/src/components/accordion/accordion.test.tsx
app/packages/quanta/src/components/accordion/accordion.tsx
app/packages/quanta/src/components/accordion/index.ts
app/packages/quanta/src/components/autocomplete/autocomplete.css
app/packages/quanta/src/components/autocomplete/autocomplete.test.tsx
app/packages/quanta/src/components/autocomplete/autocomplete.tsx
app/packages/quanta/src/components/autocomplete/index.ts
app/packages/quanta/src/components/avatar/avatar.css
app/packages/quanta/src/components/avatar/avatar.test.tsx
app/packages/quanta/src/components/avatar/avatar.tsx
app/packages/quanta/src/components/avatar/index.ts
app/packages/quanta/src/components/badge/badge.css
app/packages/quanta/src/components/badge/badge.test.tsx
app/packages/quanta/src/components/badge/badge.tsx
app/packages/quanta/src/components/badge/index.ts
app/packages/quanta/src/components/button-group/button-group.css
app/packages/quanta/src/components/button-group/button-group.test.tsx
app/packages/quanta/src/components/button-group/button-group.tsx
app/packages/quanta/src/components/button-group/index.ts
app/packages/quanta/src/components/button/button.css
app/packages/quanta/src/components/button/button.test.tsx
app/packages/quanta/src/components/button/button.tsx
app/packages/quanta/src/components/button/index.ts
app/packages/quanta/src/components/card/card.css
app/packages/quanta/src/components/card/card.test.tsx
app/packages/quanta/src/components/card/card.tsx
app/packages/quanta/src/components/card/index.ts
app/packages/quanta/src/components/checkbox/checkbox.css
app/packages/quanta/src/components/checkbox/checkbox.test.tsx
app/packages/quanta/src/components/checkbox/checkbox.tsx
app/packages/quanta/src/components/checkbox/index.ts
app/packages/quanta/src/components/chip/chip.css
app/packages/quanta/src/components/chip/chip.test.tsx
app/packages/quanta/src/components/chip/chip.tsx
app/packages/quanta/src/components/chip/index.ts

app/packages/quanta/src/components/close-button/close-button.css
app/packages/quanta/src/components/close-button/close-button.test.tsx
app/packages/quanta/src/components/close-button/close-button.tsx
app/packages/quanta/src/components/close-button/index.ts
app/packages/quanta/src/components/cmdk/cmdk.css
app/packages/quanta/src/components/cmdk/cmdk.test.tsx
app/packages/quanta/src/components/cmdk/cmdk.tsx
app/packages/quanta/src/components/cmdk/index.ts
app/packages/quanta/src/components/divider/divider.css
app/packages/quanta/src/components/divider/divider.test.tsx
app/packages/quanta/src/components/divider/divider.tsx
app/packages/quanta/src/components/divider/index.ts
app/packages/quanta/src/components/dot/dot.css
app/packages/quanta/src/components/dot/dot.test.tsx
app/packages/quanta/src/components/dot/dot.tsx
app/packages/quanta/src/components/dot/index.ts
app/packages/quanta/src/components/dropdown/dropdown.css
app/packages/quanta/src/components/dropdown/dropdown.test.tsx
app/packages/quanta/src/components/dropdown/dropdown.tsx
app/packages/quanta/src/components/dropdown/index.ts
app/packages/quanta/src/components/glass/glass.css
app/packages/quanta/src/components/glass/glass.test.tsx
app/packages/quanta/src/components/glass/glass.tsx
app/packages/quanta/src/components/glass/index.ts
app/packages/quanta/src/components/grid/grid-gap.ts
app/packages/quanta/src/components/grid/grid.css
app/packages/quanta/src/components/grid/grid.test.tsx
app/packages/quanta/src/components/grid/grid.tsx
app/packages/quanta/src/components/grid/index.ts
app/packages/quanta/src/components/grid/use-grid-virtualizer.ts
app/packages/quanta/src/components/grid/virtual-grid.test.tsx
app/packages/quanta/src/components/grid/virtual-grid.tsx
app/packages/quanta/src/components/icon/icon.css
app/packages/quanta/src/components/icon/icon.test.tsx
app/packages/quanta/src/components/icon/icon.tsx
app/packages/quanta/src/components/icon/index.ts
app/packages/quanta/src/components/input/index.ts
app/packages/quanta/src/components/input/input.css
app/packages/quanta/src/components/input/input.test.tsx
app/packages/quanta/src/components/input/input.tsx
app/packages/quanta/src/components/kbd/index.ts
app/packages/quanta/src/components/kbd/kbd.css
app/packages/quanta/src/components/kbd/kbd.test.tsx
app/packages/quanta/src/components/kbd/kbd.tsx
app/packages/quanta/src/components/loader/index.ts
app/packages/quanta/src/components/loader/loader.css
app/packages/quanta/src/components/loader/loader.test.tsx

app/packages/quanta/src/components/loader/loader.tsx
app/packages/quanta/src/components/media/index.ts
app/packages/quanta/src/components/media/media.css
app/packages/quanta/src/components/media/media.test.tsx
app/packages/quanta/src/components/media/media.tsx
app/packages/quanta/src/components/menu/menu.css
app/packages/quanta/src/components/modal/index.ts
app/packages/quanta/src/components/modal/modal.css
app/packages/quanta/src/components/modal/modal.test.tsx
app/packages/quanta/src/components/modal/modal.tsx
app/packages/quanta/src/components/navigation-menu/index.ts
app/packages/quanta/src/components/navigation-menu/navigation-menu.css
app/packages/quanta/src/components/navigation-menu/navigation-menu.test.tsx
app/packages/quanta/src/components/navigation-menu/navigation-menu.tsx
app/packages/quanta/src/components/not-found/index.ts
app/packages/quanta/src/components/not-found/not-found.css
app/packages/quanta/src/components/not-found/not-found.test.tsx
app/packages/quanta/src/components/not-found/not-found.tsx
app/packages/quanta/src/components/progress/index.ts
app/packages/quanta/src/components/progress/progress.css
app/packages/quanta/src/components/progress/progress.test.tsx
app/packages/quanta/src/components/progress/progress.tsx
app/packages/quanta/src/components/radio/index.ts
app/packages/quanta/src/components/radio/radio.css
app/packages/quanta/src/components/radio/radio.test.tsx
app/packages/quanta/src/components/radio/radio.tsx
app/packages/quanta/src/components/select/index.ts
app/packages/quanta/src/components/select/select.css
app/packages/quanta/src/components/select/select.test.tsx
app/packages/quanta/src/components/select/select.tsx
app/packages/quanta/src/components/sidebar/index.ts
app/packages/quanta/src/components/sidebar/sidebar.css
app/packages/quanta/src/components/sidebar/sidebar.test.tsx
app/packages/quanta/src/components/sidebar/sidebar.tsx
app/packages/quanta/src/components/slider/index.ts
app/packages/quanta/src/components/slider/slider.css
app/packages/quanta/src/components/slider/slider.test.tsx
app/packages/quanta/src/components/slider/slider.tsx
app/packages/quanta/src/components/sonner/index.ts
app/packages/quanta/src/components/sonner/sonner.css
app/packages/quanta/src/components/sonner/sonner.test.tsx
app/packages/quanta/src/components/sonner/sonner.tsx
app/packages/quanta/src/components/switch/index.ts
app/packages/quanta/src/components/switch/switch.css
app/packages/quanta/src/components/switch/switch.test.tsx
app/packages/quanta/src/components/switch/switch.tsx
app/packages/quanta/src/components/tabs/index.ts

app/packages/quanta/src/components/tabs/tabs.css
app/packages/quanta/src/components/tabs/tabs.test.tsx
app/packages/quanta/src/components/tabs/tabs.tsx
app/packages/quanta/src/components/tag/index.ts
app/packages/quanta/src/components/tag/tag.css
app/packages/quanta/src/components/tag/tag.test.tsx
app/packages/quanta/src/components/tag/tag.tsx
app/packages/quanta/src/components/textarea/index.ts
app/packages/quanta/src/components/textarea/textarea.css
app/packages/quanta/src/components/textarea/textarea.test.tsx
app/packages/quanta/src/components/textarea/textarea.tsx
app/packages/quanta/src/components/toggle/index.ts
app/packages/quanta/src/components/toggle/toggle.css
app/packages/quanta/src/components/toggle/toggle.test.tsx
app/packages/quanta/src/components/toggle/toggle.tsx
app/packages/quanta/src/components/tooltip/index.ts
app/packages/quanta/src/components/tooltip/tooltip.css
app/packages/quanta/src/components/tooltip/tooltip.test.tsx
app/packages/quanta/src/components/tooltip/tooltip.tsx
app/packages/quanta/src/components/typography/index.ts
app/packages/quanta/src/components/typography/typography.css
app/packages/quanta/src/components/typography/typography.test.tsx
app/packages/quanta/src/components/typography/typography.tsx
app/packages/quanta/src/components/utils/cx.test.tsx
app/packages/quanta/src/components/utils/cx.ts
app/packages/quanta/src/components/utils/slot.ts
app/packages/quanta/src/components/utils/use-flip.ts
app/packages/quanta/src/components/utils/use-in-view.ts
app/packages/quanta/src/components/vault/index.ts
app/packages/quanta/src/components/vault/vault.css
app/packages/quanta/src/components/vault/vault.test.tsx
app/packages/quanta/src/components/vault/vault.tsx
app/packages/quanta/src/css/components/component.css
app/packages/quanta/src/css/primitives/border-width.css
app/packages/quanta/src/css/primitives/breakpoint.css
app/packages/quanta/src/css/primitives/color.css
app/packages/quanta/src/css/primitives/gradient.css
app/packages/quanta/src/css/primitives/icon.css
app/packages/quanta/src/css/primitives/index.css
app/packages/quanta/src/css/primitives/motion.css
app/packages/quanta/src/css/primitives/opacity.css
app/packages/quanta/src/css/primitives/radius.css
app/packages/quanta/src/css/primitives/shadow.css
app/packages/quanta/src/css/primitives/spacing.css
app/packages/quanta/src/css/primitives/typography.css
app/packages/quanta/src/css/primitives/z-index.css
app/packages/quanta/src/css/tailwind/border-width.css

app/packages/quanta/src/css/tailwind/breakpoint.css
app/packages/quanta/src/css/tailwind/color.css
app/packages/quanta/src/css/tailwind/components.css
app/packages/quanta/src/css/tailwind/gradient.css
app/packages/quanta/src/css/tailwind/icon.css
app/packages/quanta/src/css/tailwind/index.css
app/packages/quanta/src/css/tailwind/motion.css
app/packages/quanta/src/css/tailwind/opacity.css
app/packages/quanta/src/css/tailwind/radius.css
app/packages/quanta/src/css/tailwind/shadow.css
app/packages/quanta/src/css/tailwind/slot.css
app/packages/quanta/src/css/tailwind/spacing.css
app/packages/quanta/src/css/tailwind/typography.css
app/packages/quanta/src/css/tailwind/z-index.css
app/packages/quanta/src/css/theme/color.css
app/packages/quanta/src/css/theme/fonts.css
app/packages/quanta/src/css/theme/fonts/ibm-plex-mono/OFL.txt
app/packages/quanta/src/css/theme/fonts/inter/OFL.txt
app/packages/quanta/src/css/theme/fonts/space-grotesk/OFL.txt
app/packages/quanta/src/css/theme/index.css
app/packages/quanta/src/css/theme/typography.css
app/packages/quanta/src/runtime/bootstrap.ts
app/packages/quanta/src/runtime/controller.test.ts
app/packages/quanta/src/runtime/controller.ts
app/packages/quanta/src/runtime/define-theme.test.ts
app/packages/quanta/src/runtime/define-theme.ts
app/packages/quanta/src/runtime/index.ts
app/packages/quanta/src/runtime/storage-keys.ts
app/packages/quanta/src/runtime/storage.test.ts
app/packages/quanta/src/runtime/storage.ts
app/packages/quanta/src/test/setup-components.ts
app/packages/quanta/tsconfig.json
app/packages/quanta/vitest.config.ts
app/packages/quanta/wrangler.jsonc
app/packages/README.md
app/public/assets/landing/template-preview.svg
app/public/farmverse-farm.svg
app/scripts/check-adaptation.mjs
app/scripts/check-ui.mjs
app/scripts/verify-install.mjs
app/src/app-meta.json
app/src/assets/icon-sparkles-soft.svg
app/src/cloudflare-workers.d.ts
app/src/components/AGENTS.md
app/src/components/asset-library.tsx
app/src/components/before-after-compare/before-after-compare.tsx
app/src/components/before-after-compare/index.ts

app/src/components/composer/composer.css
app/src/components/composer/composer.tsx
app/src/components/composer/index.ts
app/src/components/custom-ui/custom-ui.tsx
app/src/components/custom-ui/index.ts
app/src/components/dropzone/dropzone-preview.tsx
app/src/components/dropzone/dropzone.tsx
app/src/components/dropzone/index.ts
app/src/components/gallery/demo-data.ts
app/src/components/gallery/density-control.tsx
app/src/components/gallery/gallery-tile.tsx
app/src/components/gallery/gallery.css
app/src/components/gallery/index.ts
app/src/components/gallery/justified-engine.ts
app/src/components/gallery/justified-gallery.tsx
app/src/components/gallery/types.ts
app/src/components/gallery/use-justified-gallery.ts
app/src/components/gallery/use-reduced-motion.ts
app/src/components/generation-card/card-actions.tsx
app/src/components/generation-card/generation-card.css
app/src/components/generation-card/generation-card.tsx
app/src/components/generation-card/generation-tile.tsx
app/src/components/generation-card/index.ts
app/src/components/generation-detail.tsx
app/src/components/history-grid.tsx
app/src/components/icon-tile/icon-tile.css
app/src/components/icon-tile/icon-tile.tsx
app/src/components/icon-tile/index.ts
app/src/components/media-card/index.ts
app/src/components/media-card/media-card.css
app/src/components/media-card/media-card.tsx
app/src/components/prompt-box/index.ts
app/src/components/prompt-box/prompt-box.css
app/src/components/prompt-box/prompt-box.tsx
app/src/components/rail-footer/index.ts
app/src/components/rail-footer/rail-footer.tsx
app/src/components/screen-empty-state.tsx
app/src/components/scroll-scrub/scroll-scrub.css
app/src/components/scroll-scrub/scroll-scrub.tsx
app/src/components/setting-trigger/index.ts
app/src/components/setting-trigger/setting-trigger.css
app/src/components/setting-trigger/setting-trigger.tsx
app/src/components/step-rail/index.ts
app/src/components/step-rail/step-rail.tsx
app/src/components/StructuredData.tsx
app/src/components/template-modal/index.ts
app/src/components/template-modal/template-modal.tsx

app/src/components/template-picker.tsx
app/src/components/three-farm-scene.tsx
app/src/components/ui/accordion.tsx
app/src/components/ui/alert-dialog.tsx
app/src/components/ui/alert.tsx
app/src/components/ui/aspect-ratio.tsx
app/src/components/ui/attachment.tsx
app/src/components/ui/avatar.tsx
app/src/components/ui/badge.tsx
app/src/components/ui/breadcrumb.tsx
app/src/components/ui/bubble.tsx
app/src/components/ui/button-group.tsx
app/src/components/ui/button.tsx
app/src/components/ui/calendar.tsx
app/src/components/ui/card.tsx
app/src/components/ui/carousel.tsx
app/src/components/ui/chart.tsx
app/src/components/ui/checkbox.tsx
app/src/components/ui/close-button.tsx
app/src/components/ui/collapsible.tsx
app/src/components/ui/combobox.tsx
app/src/components/ui/command.tsx
app/src/components/ui/context-menu.tsx
app/src/components/ui/dialog.tsx
app/src/components/ui/direction.tsx
app/src/components/ui/drawer.tsx
app/src/components/ui/dropdown-menu.tsx
app/src/components/ui/empty.tsx
app/src/components/ui/field.tsx
app/src/components/ui/hover-card.tsx
app/src/components/ui/input-group.tsx
app/src/components/ui/input-otp.tsx
app/src/components/ui/input.tsx
app/src/components/ui/item.tsx
app/src/components/ui/kbd.tsx
app/src/components/ui/label.tsx
app/src/components/ui/marker.tsx
app/src/components/ui/marketing-button.tsx
app/src/components/ui/menubar.tsx
app/src/components/ui/message-scroller.tsx
app/src/components/ui/message.tsx
app/src/components/ui/native-select.tsx
app/src/components/ui/navigation-menu.tsx
app/src/components/ui/pagination.tsx
app/src/components/ui/popover.tsx
app/src/components/ui/progress.tsx
app/src/components/ui/radio-group.tsx

app/src/components/ui/resizable.tsx
app/src/components/ui/scroll-area.tsx
app/src/components/ui/select.tsx
app/src/components/ui/separator.tsx
app/src/components/ui/sheet.tsx
app/src/components/ui/sidebar.tsx
app/src/components/ui/skeleton.tsx
app/src/components/ui/slider.tsx
app/src/components/ui/sonner.tsx
app/src/components/ui/spinner.tsx
app/src/components/ui/switch.tsx
app/src/components/ui/table.tsx
app/src/components/ui/tabs.tsx
app/src/components/ui/textarea.tsx
app/src/components/ui/toggle-group.tsx
app/src/components/ui/toggle.tsx
app/src/components/ui/tooltip.tsx
app/src/components/upload-field/index.ts
app/src/components/upload-field/upload-field.tsx
app/src/components/user-generations/index.ts
app/src/components/user-generations/user-generations.tsx
app/src/hooks/use-mobile.ts
app/src/landing-content.ts
app/src/layouts/AGENTS.md
app/src/layouts/custom.tsx
app/src/lib/api/example.functions.ts
app/src/lib/bindings.server.ts
app/src/lib/config.server.ts
app/src/lib/download-media.ts
app/src/lib/error-capture.ts
app/src/lib/error-page.ts
app/src/lib/higgsfield-error-reporting.ts
app/src/lib/higgsfield-generation-results.ts
app/src/lib/quanta-icons.ts
app/src/lib/security-headers.server.ts
app/src/lib/utils.ts
app/src/module/design-inspector/protocol.ts
app/src/module/design-inspector/README.md
app/src/module/design-inspector/registry.ts
app/src/module/design-inspector/runtime.ts
app/src/module/design-inspector/vite.ts
app/src/router.tsx
app/src/routes/__root.tsx
app/src/routes/app.tsx
app/src/routes/index.tsx
app/src/routes/README.md
app/src/routes/robots[.]txt.ts

app/src/routes/sitemap[.]xml.ts
app/src/routeTree.gen.ts
app/src/scroll-scrub-scenes.ts
app/src/server.ts
app/src/start.ts
app/src/styles.css
app/src/svg.d.ts
app/tests/history-performance.test.ts
app/tests/landing-contract.test.ts
app/tests/security-headers.test.ts
app/tsconfig.json
app/tsr.config.json
app/vite.config.ts
app/wrangler.jsonc

.github/workflows/ci.yml

```
name: Template CI

on:
  push:
    branches:
      - main
  workflow_dispatch:

jobs:
  validate:
    runs-on: arc-runners-frontend
    name: Validate template
    # The app project lives in app/ (ci/ holds the deploy config, AGENTS.md +
    # skills/ stay at root). Every build step runs there.
    defaults:
      run:
        working-directory: app
    steps:
      - uses: actions/checkout@v6

      - uses: oven-sh/setup-bun@v2

      - name: Install dependencies
        run: bun install --frozen-lockfile

      - name: Typecheck
        run: bun run typecheck

      - name: Lint
        run: bun run lint

      - name: Test
        run: bun run test

      # Deploys always ship the design inspector: apps-ci / apps-ci-dev export
      # HF_DESIGN_INSPECTOR=1 before `bun run build`. Validate that exact
      # artifact here — one build, not a separate plain + design pair.
      - name: Build (design inspector on, as deploys ship it)
        env:
          HF_DESIGN_INSPECTOR: "1"
        run: bun run build
```

.gitignore

```
node_modules
```

app/.gitignore

```
# Logs
logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*
lerna-debug.log*

node_modules
dist
dist-ssr
.output
.vinxi
.tanstack/**
.nitro
*.local

# Wrangler / Cloudflare
.wrangler/
.dev.vars

# Editor directories and files
.vscode/*
!.vscode/extensions.json
.idea
.DS_Store
*.suo
*.ntvs*
*.njsproj
*.sln
*.sw?
```

app/.prettierignore

```
node_modules
dist
.output
.vinxi
pnpm-lock.yaml
package-lock.json
bun.lock
routeTree.gen.ts
packages
```

app/.prettierrc

```
{
  "printWidth": 100,
  "semi": true,
  "singleQuote": false,
  "trailingComma": "all"
}
```

app/AGENTS.md

```
# Scroll-scrub website

A **`type: "website"`** template — a standalone brand, NOT a Higgsfield app.
The visitor's scroll plays a generated film while semantic chapters read over
it. The scrub engine is already built; your job is the film, the scenes, and
the brand.

The authoritative workflow is the `website-builder-flow` skill
(`references/scroll-scrub.md`). This file is the in-repo contract.

## The engine is done — do not rewrite it

`src/components/scroll-scrub/scroll-scrub.tsx` + `.css` ship the full runtime:
Blob-backed seeking, seek coalescing, lazy nearby-segment loading, desktop and
mobile sources, exact-frame posters held until a real painted frame, iOS
gesture priming, `prefers-reduced-motion`, reverse scroll, and complete
teardown (aborts, listeners, RAF, revoked Blob URLs).

Do NOT rebuild the controller, drive per-frame values through React state, or
attach a second scroll timeline to the same video elements. Adapt composition
and scene data instead. The engine intentionally ships **no** header, no shared
button system, no scroll hint, and no per-scene eyebrow — compose the site's
own nav and bespoke CTAs around it.

## What you fill in

| File | What you do |
|---|---|
| `src/scroll-scrub-scenes.ts` | Brand tokens + the scene array. Every `<...>` placeholder gets real copy. |
| `src/routes/index.tsx` | Compose the page: nav, chapters, CTAs, and the content sections after the journey. |
| `public/assets/world/` | The encoded clips and their exact-frame posters. |
| `src/styles.css` | The site's own token layer from the design brief. |
| `src/app-meta.json` | `og_title`, `og_description`, `favicon_url`, `og_image_url`, `marketplace_cover_url`. |

## Hard rules

- **This is a website, not an app.** Never import `@higgsfield/quanta/*`, never
  use q-prefixed tokens, never add "Sign in with Higgsfield", the fnf SDK, or
  any runtime generation. No "Powered by Higgsfield" anywhere in page content.
  The user's brand is the only brand on the page.
- **Every `poster` is the exact first frame of the clip beside it.** Never a
  design board, never an imagined destination still, never the next scene's
  image. Generate posters from the ENCODED clip, after encoding.
- **Provide `mobilePoster` whenever `mobileClip` is set.**
- **Keep `scrollScrubScenes` a module constant.** Changing its identity on
  every render rebuilds the controller.
- **CSP:** the controller assigns Blob URLs to `<video src>`, so `media-src`
  must include `blob:` and clip fetches stay same-origin (`connect-src 'self'`).
- **Byte budget:** start at ≤32 MiB for all desktop clips and ≤16 MiB for all
  mobile clips. Visited clips are retained for smooth reverse scroll, so
  oversized clips cost memory as well as transfer.
- **SSR safety:** no browser global at module top level or during render. A
  top-level `window` reference crashes SSR.
- **No placeholders may ship.** Every `<...>` token is replaced before deploy.

## Vendored packages and legacy scaffold files

`app/packages/` holds managed snapshots vendored from the Higgsfield web app
(`@higgsfield/fnf`, `fnf-react`, `quanta`, `app-landing`). Websites never use
them, but do NOT edit or delete them, and leave the Quanta Tailwind entry wired
in `src/styles.css` — the build depends on it.

**This template is a byte-for-byte copy of the `custom` app scaffold**, plus the
three scroll-scrub files and a `/` that renders the journey. Keeping it in
lockstep with `custom` is deliberate: the platform's build, deploy and design
inspector all assume that tree, and every past divergence has been a bug.

So it also carries the whole Higgsfield **app** surface — `src/layouts/`, the
```

`/app` route, `src/landing-content.ts`, the fnf/Quanta components (composer, generation-card, gallery, asset-library, user-generations), the shadcn kit, and demo assets under `public/presets/`. For a website these are inert raw material, NOT part of the product:

- Do NOT wire them into the site, and do NOT import from `@higgsfield/quanta/*` or the fnf SDK by way of them.
- Delete NOTHING. Unused files simply stay unreferenced. Removing them is what makes this template drift from `custom`.
- The shadcn primitives in `@/components/ui` may be restyled to the design brief as ordinary React components when genuinely useful.

Required completion checks

```bash
cd app
bun run typecheck
bun run build   # runs check:ui first, same as every app template
```

`check:ui` scans `src/layouts`, `src/routes`, `src/components/custom-ui`, `src/features` and `src/widgets` for raw colour literals and `dark:` variants. It does NOT scan `src/components/scroll-scrub` or `src/scroll-scrub-scenes.ts`, so the site's own palette lives in those (and in `src/styles.css`) without tripping the gate. Keep raw hex out of route files.

app/app.manifest.json

```
{
  "db": false,
  "r2": false,
  "kv": false,
  "durableObject": null
}
```

app/bunfig.toml

```
[install]
# 24h supply-chain guard: skip package versions published less than a day ago.
minimumReleaseAge = 86400
```

app/components.json

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "radix-nova",
  "rsc": false,
  "tsx": true,
  "tailwind": {
    "config": "",
    "css": "src/styles.css",
    "baseColor": "neutral",
    "cssVariables": true,
    "prefix": ""
  },
  "iconLibrary": "lucide",
  "rtl": false,
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "menuColor": "default",
  "menuAccent": "subtle",
  "registries": {}
}
```

app/eslint.config.js

```
import js from "@eslint/js";
import globals from "globals";
import reactHooks from "eslint-plugin-react-hooks";
import reactRefresh from "eslint-plugin-react-refresh";
import tseslint from "typescript-eslint";

export default tseslint.config(
  // Formatting is prettier's job (`bun run format`), NOT a lint error — lint
  // reports real code problems only. Vendored packages/ and generated files
  // are not ours to lint.
  { ignores: ["dist", ".output", ".vinxi", "packages", "scripts", "src/routeTree.gen.ts"] },
  {
    extends: [js.configs.recommended, ...tseslint.configs.recommended],
    files: ["**/*.{ts,tsx}"],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
    },
    plugins: {
      "react-hooks": reactHooks,
      "react-refresh": reactRefresh,
    },
    rules: {
      ...reactHooks.configs.recommended.rules,
      "no-restricted-imports": [
        "error",
        {
          paths: [
            {
              name: "server-only",
              message:
                "TanStack Start does not use the Next.js `server-only` package. Rename the module to `*.server.ts` or mark it with `@tanstack/react-start/server-only`.",
            },
          ],
        },
      ],
      // Route files export `Route` and shared components export helpers/data
      // alongside components BY DESIGN here — this rule is all noise in this
      // codebase (HMR just full-reloads those modules).
      "react-refresh/only-export-components": "off",
      // `catch {}` is a deliberate ignore pattern in shipped code.
      "no-empty": ["error", { allowEmptyCatch: true }],
      "@typescript-eslint/no-unused-vars": "off",
    },
  },
  {
    // The justified-gallery engine deliberately reads refs during its
    // render-time measure pass (bespoke out-of-React layout/windowing engine).
    // Exempt from the compiler-strict refs rule — do NOT imitate this pattern
    // in app code.
    files: ["src/components/gallery/**"],
    rules: { "react-hooks/refs": "off" },
  },
);
```

app/migrations/0001_init.sql

```
-- D1 schema. Applied by the platform on deploy (only when app.manifest.json
-- sets "db": true). ONE database is shared by preview + prod — keep every
-- change additive (CREATE TABLE IF NOT EXISTS / ADD COLUMN); a destructive
-- change hits production data. Bound as env.DB (see src/lib/bindings.server.ts).
--
-- Example:
-- CREATE TABLE IF NOT EXISTS items (
--   id INTEGER PRIMARY KEY AUTOINCREMENT,
--   title TEXT NOT NULL,
--   created_at TEXT NOT NULL DEFAULT (datetime('now'))
-- );
```

app/package.json

```
{
  "name": "tanstack_start_ts",
  "private": true,
  "sideEffects": false,
  "type": "module",
  "workspaces": [
    "packages/*"
  ],
  "scripts": {
    "dev": "vite dev",
    "dev:design": "HF_DESIGN_INSPECTOR=1 vite dev --mode design",
    "build": "bun run check:ui && tsr generate && tsc --noEmit && vite build",
    "build:dev": "vite build --mode development",
    "preview": "vite preview",
    "postinstall": "node scripts/verify-install.mjs",
    "lint": "eslint .",
    "format": "prettier --write .",
    "typecheck": "tsr generate && tsc --noEmit",
    "check:ui": "node scripts/check-ui.mjs",
    "check:adapted": "bun scripts/check-adaptation.mjs",
    "test": "bun test tests/*.test.ts"
  },
  "dependencies": {
    "@base-ui/react": "1.5.0",
    "@higgsfield/app-landing": "workspace:*",
    "@higgsfield/fnf": "workspace:*",
    "@higgsfield/fnf-react": "workspace:*",
    "@higgsfield/quanta": "workspace:*",
    "@phosphor-icons/react": "^2.1.10",
    "@shadcn/react": "^0.2.1",
    "@tailwindcss/vite": "^4.3.2",
    "@tanstack/react-query": "^5.101.2",
    "@tanstack/react-router": "^1.170.17",
    "@tanstack/react-start": "^1.168.27",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
    "date-fns": "^4.4.0",
    "embla-carousel-react": "^8.6.0",
    "input-otp": "^1.4.2",
    "lucide-react": "^1.23.0",
    "next-themes": "^0.4.6",
    "radix-ui": "^1.6.2",
    "react": "^19.2.7",
    "react-day-picker": "^10.0.1",
    "react-dom": "^19.2.7",
    "react-resizable-panels": "^4.12.1",
    "recharts": "^3.8.0",
    "shadcn": "^4.13.0",
    "sonner": "^2.0.7",
    "tailwind-merge": "^3.6.0",
    "tailwindcss": "^4.3.2",
    "three": "^0.186.1",
    "tw-animate-css": "^1.4.0",
    "vaul": "^1.1.2",
    "zod": "^4.4.3"
  },
  "devDependencies": {
    "@cloudflare/workers-types": "^5.20260708.1",
    "@eslint/js": "^10.0.1",
    "@tanstack/router-cli": "^1.167.18",
    "@types/node": "^26.1.0",
    "@types/react": "^19.2.15",
    "@types/react-dom": "^19.2.3",
    "@types/three": "^0.186.0",
    "@vitejs/plugin-react": "^5.0.4",
    "eslint": "^10.6.0",
    "eslint-plugin-react-hooks": "^7.1.1",
```

```
    "eslint-plugin-react-refresh": "^0.5.3",
    "globals": "^17.7.0",
    "nitro": "3.0.260429-beta",
    "prettier": "^3.9.4",
    "typescript": "^6.0.3",
    "typescript-eslint": "^8.63.0",
    "vite": "^8.1.3",
    "vite-plugin-svgr": "^5.2.0"
  },
  "overrides": {
    "@types/react": "19.2.15"
  }
}
```

app/packages/.snapshot.json


```
{
  "sourceRepository": "git@github.com:higgsfield-ai/fnf-web.git",
  "fnfRef": "b8f86ca2cffa0576a3e85a42e18c78b4f8225de6",
  "fnfCommit": "b8f86ca2cffa0576a3e85a42e18c78b4f8225de6",
  "quantaRef": "b8f86ca2cffa0576a3e85a42e18c78b4f8225de6",
  "quantaCommit": "b8f86ca2cffa0576a3e85a42e18c78b4f8225de6",
  "bundleName": "manual-package-sync-b8f86ca2cffa.zip",
  "bundleSha256": "not-generated-manual-sync",
  "syncedBy": "nurdaulet@higgsfield.com",
  "syncMode": "manual-package-copy",
  "generatedAt": "2026-07-01T13:13:14Z",
  "managedPackages": [
    "@higgsfield/fnf",
    "@higgsfield/fnf-react",
    "@higgsfield/quanta"
  ],
  "templateLayout": "app-project",
  "notes": "quanta's @higgsfield-ai/icons (private Nexus) is demoted to an optionalDependency; @higgsfield-ai/icons/* is aliased to src/lib/quanta-lucide-icons.ts (lucide-react) in vite.config.ts + ts
}
```

app/packages/app-landing/package.json

```
{
  "name": "@higgsfield/app-landing",
  "version": "0.0.0",
  "private": true,
  "description": "Typed landing-page content contract and Quanta renderer for generated Higgsfield apps",
  "type": "module",
  "exports": {
    ".": "./src/index.ts"
  },
  "files": [
    "src"
  ],
  "scripts": {
    "typecheck": "tsc --noEmit -p tsconfig.json"
  },
  "dependencies": {
    "@higgsfield/quanta": "workspace:*",
    "lucide-react": "^1.23.0",
    "zod": "^4.4.3"
  },
  "peerDependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "devDependencies": {
    "@types/react": "^19.0.0",
    "@types/react-dom": "^19.0.0",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "typescript": "^5.8.3"
  }
}
```

app/packages/app-landing/src/index.ts

```
export {
  landingContentSchema,
  landingCtaSchema,
  landingIconSchema,
  landingMediaSchema,
  landingPreviewSchema,
  landingStepActionPreviewSchema,
  landingStepInstructionPreviewSchema,
  landingStepPreviewSchema,
  landingStepResultPreviewSchema,
  parseLandingContent,
} from "./schema";
export type {
  LandingContent,
  LandingIconName,
  LandingMedia,
  LandingStepPreview,
} from "./schema";
export { LandingPage, LandingSections } from "./landing-page";
export type { LandingPageProps, LandingSectionsProps } from "./landing-page";
```

app/packages/app-landing/src/landing-page.tsx

```
"use client";

import type { ReactNode } from "react";
import { useState } from "react";
import {
  Check,
  ChevronLeft,
  ChevronRight,
  Clock3,
  Crop,
  Image as ImageIcon,
  Layers3,
  ShieldCheck,
  SlidersHorizontal,
  Sparkles,
  Video,
  WandSparkles,
} from "lucide-react";
import { Button } from "@higgsfield/quanta/button";
import { Icon } from "@higgsfield/quanta/icon";
import { Media } from "@higgsfield/quanta/media";
import { Typography } from "@higgsfield/quanta/typography";
import type {
  LandingContent,
  LandingIconName,
  LandingMedia as LandingMediaContent,
} from "./schema";

const ICONS = {
  check: Check,
  clock: Clock3,
  frame: Crop,
  image: ImageIcon,
  layers: Layers3,
  shield: ShieldCheck,
  sliders: SlidersHorizontal,
  sparkles: Sparkles,
  video: Video,
  wand: WandSparkles,
} satisfies Record<LandingIconName, typeof Check>;

const STUDIO_HERO_GLOW =
  "radial-gradient(60% 80% at 50% 0%, rgba(160,164,170,0.14) 0%, rgba(160,164,170,0.05) 42%, transparent 72%)";
const STUDIO_HERO_DOTS = "radial-gradient(rgba(255,255,255,0.2) 1px, transparent 1px)";
const STUDIO_HERO_DOTS_MASK =
  "radial-gradient(55% 70% at 50% 0%, #000 0%, rgba(0,0,0,0.35) 45%, transparent 75%)";

function classes(...values: Array<string | false | null | undefined>) {
  return values.filter(Boolean).join(" ");
}

function LandingMedia({
  media,
  eager = false,
  className,
}: {
  media: LandingMediaContent;
  eager?: boolean;
  className?: string;
}) {
  return (
    <Media ratio="video" rounded="lg" className={classes("h-full w-full", className)}>
      {media.kind === "image" ? (
        <Media.Image
          src={media.src}
          alt={media.alt}
          loading={eager ? "eager" : "lazy"}
          fetchPriority={eager ? "high" : "auto"}
```

```
        />
      ) : (
        <Media.Video
          src={media.src}
          poster={media.poster}
          aria-label={media.alt}
          autoPlayInView
          loop
        />
      )}
    </Media>
  );
}

function SectionHeading({
  title,
  description,
  align = "center",
}: {
  title: string;
  description?: string;
  align?: "center" | "start";
}) {
  return (
    <header
      className={classes(
        "flex max-w-3xl flex-col gap-3",
        align === "center" ? "mx-auto items-center text-center" : "items-start text-left",
      )}
    >
      <Typography
        as="h2"
        variant="accent-xl-bold"
        color="primary"
        className="text-balance uppercase md:text-q-accent-2xl-bold"
      >
        {title}
      </Typography>
      {description != null ? (
        <Typography
          as="p"
          variant="body-md-regular"
          color="secondary"
          className="max-w-2xl text-pretty"
        >
          {description}
        </Typography>
      ) : null}
    </header>
  );
}

function Hero({ content }: { content: LandingContent["hero"] }) {
  return (
    <section className="flex flex-col items-center gap-6 pb-4 pt-10 text-center md:pb-8 md:pt-16">
      {content.eyebrow != null ? (
        <Typography
          as="p"
          variant="label-sm-semi-bold"
          color="brand"
          className="uppercase tracking-wider"
        >
          {content.eyebrow}
        </Typography>
      ) : null}
      <div className="flex max-w-4xl flex-col items-center gap-4">
        <Typography
          as="h1"
          variant="accent-3xl-bold"
          color="primary"
```

```
          className="text-balance uppercase md:text-q-accent-5xl-bold"
        >
          {content.title}
        </Typography>
        <Typography
          as="p"
          variant="body-lg-regular"
          color="secondary"
          className="max-w-2xl text-pretty"
        >
          {content.description}
        </Typography>
      </div>
      <div className="flex flex-wrap justify-center gap-3">
        <Button as="a" href={content.primaryCta.href} variant="marketingPrimary" size="lg">
          {content.primaryCta.label}
        </Button>
        {content.secondaryCta != null ? (
          <Button as="a" href={content.secondaryCta.href} variant="marketingSecondary" size="lg">
            {content.secondaryCta.label}
          </Button>
        ) : null}
      </div>
    </section>
  );
}

function AppPreview({
  content,
  inlinePreview,
}: {
  content: LandingContent["preview"];
  inlinePreview?: ReactNode;
}) {
  if (content.kind === "inline") {
    return inlinePreview == null ? null : (
      <section aria-label={content.title} className="overflow-hidden rounded-q-600">
        {inlinePreview}
      </section>
    );
  }

  return (
    <section aria-label={content.title}>
      <div className="group relative overflow-hidden rounded-q-600 border border-q-border-subtle bg-q-background-secondary shadow-q-raised">
        <div className="aspect-video min-h-72 w-full md:min-h-[540px]">
          {content.kind === "route" ? (
            <iframe
              title={content.title}
              src={content.src}
              loading="lazy"
              tabIndex={-1}
              className="pointer-events-none h-full w-full border-0"
            />
          ) : (
            <LandingMedia media={content.media} eager className="rounded-q-0" />
          )}
        </div>
        <a
          href={content.openHref}
          aria-label={content.openLabel}
          className="absolute inset-0 flex cursor-pointer items-end justify-center bg-q-transparent-dark-20 p-5 opacity-100 transition-colors hover:bg-q-transparent-dark-40 focus-visible:outline focus
        >
          <span className="rounded-q-full bg-q-background-primary px-5 py-3 text-q-body-sm-semi-bold text-q-text-primary shadow-q-raised-sm">
            {content.openLabel}
          </span>
        </a>
      </div>
    </section>
  );
```

```
}

function StepPreview({
  preview,
}: {
  preview: LandingContent["steps"]["items"][number]["preview"];
}) {
  if (preview.kind === "instruction") {
    const Glyph = ICONS[preview.icon];
    return (
      <div className="flex h-full flex-col items-center justify-center gap-3 rounded-q-300 border border-dashed border-q-border-subtle px-8">
        <Icon as={Glyph} size="md" color="secondary" />
        <div className="flex flex-col items-center gap-1 text-center">
          <Typography as="span" variant="body-sm-semi-bold" color="primary" className="uppercase">
            {preview.title}
          </Typography>
          {preview.description != null ? (
            <Typography as="span" variant="caption-xs-regular" color="secondary">
              {preview.description}
            </Typography>
          ) : null}
        </div>
      </div>
    );
  }

  if (preview.kind === "action") {
    return (
      <div className="flex h-full items-center justify-center">
        <Button
          variant="marketingPrimary"
          size="lg"
          start={<Icon as={Sparkles} size="sm" />}
        >
          {preview.label}
        </Button>
      </div>
    );
  }

  return (
    <div className="flex h-full items-center justify-center p-6">
      <LandingMedia media={preview.media} className="max-w-full rounded-q-300" />
    </div>
  );
}

function Steps({ content }: { content: LandingContent["steps"] }) {
  return (
    <section id="how-it-works" className="flex scroll-mt-8 flex-col gap-8">
      <SectionHeading title={content.title} description={content.description} align="start" />
      <div className="grid gap-10 md:grid-cols-3">
        {content.items.map((item, index) => (
          <article key={item.title} className="flex min-w-0 flex-col gap-4">
            <div className="h-60 overflow-hidden rounded-q-400 bg-q-transparent-light-05">
              <StepPreview preview={item.preview} />
            </div>
            <div className="flex flex-col gap-2">
              <Typography
                as="h3"
                variant="accent-xs-bold"
                color="primary"
                className="uppercase"
              >
                {`${index + 1}. ${item.title}`}
              </Typography>
              <Typography as="p" variant="body-sm-regular" color="secondary">
                {item.description}
              </Typography>
            </div>
```

```
          </article>
        ))}
      </div>
    </section>
  );
}

function Features({ content }: { content: LandingContent["features"] }) {
  return (
    <section className="flex flex-col gap-8">
      <SectionHeading title={content.title} description={content.description} />
      <div className="grid gap-3 md:grid-cols-3">
        {content.items.map((item) => {
          const Glyph = ICONS[item.icon];
          return (
            <article
              key={item.title}
              className="flex min-h-48 flex-col gap-5 rounded-q-500 border border-q-border-subtle bg-q-background-secondary p-5"
            >
              <span className="flex size-11 items-center justify-center rounded-q-300 bg-q-transparent-light-10 text-q-icon-primary">
                <Icon as={Glyph} size="md" />
              </span>
              <div className="flex flex-col gap-2">
                <Typography as="h3" variant="body-lg-semi-bold" color="primary">
                  {item.title}
                </Typography>
                <Typography as="p" variant="body-sm-regular" color="secondary">
                  {item.description}
                </Typography>
              </div>
            </article>
          );
        })}
      </div>
    </section>
  );
}

function Showcase({ content }: { content: LandingContent["showcase"] }) {
  const [selectedIndex, setSelectedIndex] = useState(0);
  const selected = content.items[selectedIndex] ?? content.items[0];
  if (selected == null) return null;

  return (
    <section className="flex flex-col gap-8">
      <SectionHeading title={content.title} description={content.description} />
      <div className="flex flex-col items-center gap-4">
        <div className="aspect-video w-full overflow-hidden rounded-q-600 border border-q-border-subtle bg-q-background-secondary">
          <LandingMedia media={selected.media} eager />
        </div>
        {content.items.length > 1 ? (
          <div className="flex max-w-full items-center gap-2">
            <Button
              variant="ghost"
              size="sm"
              iconOnly
              className="!border-0 !bg-transparent !shadow-none hover:!bg-transparent"
              aria-label="Previous showcase example"
              onClick={() =>
                setSelectedIndex(
                  (current) => (current - 1 + content.items.length) % content.items.length,
                )
              }
              start={<Icon as={ChevronLeft} size="sm" />}
            />
            <div
              role="tablist"
              aria-label="Showcase examples"
              className="flex min-w-0 items-center gap-2 overflow-x-auto rounded-q-full bg-q-background-secondary p-1.5"
            >
```

```
              {content.items.map((item, index) => (
                <button
                  key={`${item.label}-${index}`}
                  type="button"
                  role="tab"
                  aria-selected={selectedIndex === index}
                  aria-label={`Show ${item.label}`}
                  onClick={() => setSelectedIndex(index)}
                  className={classes(
                    "relative aspect-[3/2] h-12 shrink-0 cursor-pointer overflow-hidden rounded-q-full border-2 transition-opacity focus-visible:outline focus-visible:outline-2 focus-visible:outline-o
                    selectedIndex === index
                      ? "border-q-border-focus opacity-100"
                      : "border-transparent opacity-60 hover:opacity-100",
                  )}
                >
                  {item.media.kind === "image" || item.media.poster != null ? (
                    <img
                      src={item.media.kind === "image" ? item.media.src : item.media.poster}
                      alt=""
                      loading="lazy"
                      className="h-full w-full object-cover"
                    />
                  ) : (
                    <span className="flex h-full w-full items-center justify-center bg-q-background-tertiary">
                      <Icon as={Video} size="sm" color="secondary" />
                    </span>
                  )}
                </button>
              ))}
            </div>
            <Button
              variant="ghost"
              size="sm"
              iconOnly
              className="!border-0 !bg-transparent !shadow-none hover:!bg-transparent"
              aria-label="Next showcase example"
              onClick={() =>
                setSelectedIndex((current) => (current + 1) % content.items.length)
              }
              start={<Icon as={ChevronRight} size="sm" />}
            />
          </div>
        ) : null}
      </div>
    </section>
  );
}

function FinalCta({ content }: { content: LandingContent["finalCta"] }) {
  return (
    <section className="relative isolate overflow-hidden rounded-q-600 border border-q-border-subtle bg-q-background-secondary px-5 py-16 md:px-12 md:py-24">
      {content.backgroundMedia != null ? (
        <div className="absolute inset-0 -z-10 opacity-30">
          <LandingMedia media={content.backgroundMedia} className="rounded-q-0" />
        </div>
      ) : null}
      <div
        aria-hidden
        className="absolute inset-0 -z-10"
        style={{
          backgroundImage: STUDIO_HERO_DOTS,
          backgroundSize: "14px 14px",
          maskImage: STUDIO_HERO_DOTS_MASK,
          WebkitMaskImage: STUDIO_HERO_DOTS_MASK,
        }}
      />
      <div
        aria-hidden
        className="absolute inset-0 -z-10"
        style={{ backgroundImage: STUDIO_HERO_GLOW }}
```

```
      />
      <div className="mx-auto flex max-w-3xl flex-col items-center gap-5 text-center">
        <Typography
          as="h2"
          variant="accent-2xl-bold"
          color="primary"
          className="text-balance uppercase md:text-q-accent-3xl-bold"
        >
          {content.title}
        </Typography>
        <Typography as="p" variant="body-md-regular" color="secondary" className="text-pretty">
          {content.description}
        </Typography>
        <Button as="a" href={content.action.href} variant="marketingPrimary" size="lg">
          {content.action.label}
        </Button>
      </div>
    </section>
  );
}

export interface LandingSectionsProps {
  content: LandingContent;
  className?: string;
}

export function LandingSections({ content, className }: LandingSectionsProps) {
  return (
    <div className={classes("flex flex-col gap-20 md:gap-28", className)}>
      <Steps content={content.steps} />
      <Features content={content.features} />
      <Showcase content={content.showcase} />
      <FinalCta content={content.finalCta} />
    </div>
  );
}

export interface LandingPageProps {
  content: LandingContent;
  inlinePreview?: ReactNode;
}

export function LandingPage({ content, inlinePreview }: LandingPageProps) {
  return (
    <main className="relative min-h-dvh overflow-hidden bg-q-background-primary">
      <div
        aria-hidden
        className="pointer-events-none fixed inset-x-0 top-0 h-[600px]"
        style={{
          backgroundImage: STUDIO_HERO_DOTS,
          backgroundSize: "14px 14px",
          maskImage: STUDIO_HERO_DOTS_MASK,
          WebkitMaskImage: STUDIO_HERO_DOTS_MASK,
        }}
      />
      <div
        aria-hidden
        className="pointer-events-none fixed inset-x-0 top-0 h-[600px]"
        style={{ backgroundImage: STUDIO_HERO_GLOW }}
      />

      <div className="relative mx-auto flex w-full max-w-7xl flex-col gap-16 px-4 pb-8 md:gap-24 md:px-8 md:pb-12">
        <Hero content={content.hero} />
        <AppPreview content={content.preview} inlinePreview={inlinePreview} />
        <LandingSections content={content} />
      </div>
    </main>
  );
}
```

app/packages/app-landing/src/schema.ts

```
import { z } from "zod";

const nonEmptyText = z.string().trim().min(1);
const assetPath = nonEmptyText.refine(
  (value) => value.startsWith("/") || value.startsWith("https://"),
  "Media paths must be root-relative or HTTPS URLs.",
);

export const landingCtaSchema = z.object({
  label: nonEmptyText,
  href: nonEmptyText,
});

export const landingMediaSchema = z.discriminatedUnion("kind", [
  z.object({
    kind: z.literal("image"),
    src: assetPath,
    alt: nonEmptyText,
  }),
  z.object({
    kind: z.literal("video"),
    src: assetPath,
    alt: nonEmptyText,
    poster: assetPath.optional(),
  }),
]);

export const landingPreviewSchema = z.discriminatedUnion("kind", [
  z.object({
    kind: z.literal("inline"),
    title: nonEmptyText,
  }),
  z.object({
    kind: z.literal("route"),
    title: nonEmptyText,
    src: nonEmptyText,
    openHref: nonEmptyText,
    openLabel: nonEmptyText,
  }),
  z.object({
    kind: z.literal("media"),
    title: nonEmptyText,
    media: landingMediaSchema,
    openHref: nonEmptyText,
    openLabel: nonEmptyText,
  }),
]);

export const landingIconSchema = z.enum([
  "check",
  "clock",
  "frame",
  "image",
  "layers",
  "shield",
  "sliders",
  "sparkles",
  "video",
  "wand",
]);

const landingStepBaseSchema = z.object({
  title: nonEmptyText,
  description: nonEmptyText,
});

export const landingStepInstructionPreviewSchema = z.object({
  kind: z.literal("instruction"),
```

```
  icon: landingIconSchema,
  title: nonEmptyText,
  description: nonEmptyText.optional(),
});

export const landingStepActionPreviewSchema = z.object({
  kind: z.literal("action"),
  label: nonEmptyText,
});

export const landingStepResultPreviewSchema = z.object({
  kind: z.literal("result"),
  media: landingMediaSchema,
});

export const landingStepPreviewSchema = z.discriminatedUnion("kind", [
  landingStepInstructionPreviewSchema,
  landingStepActionPreviewSchema,
  landingStepResultPreviewSchema,
]);

export const landingContentSchema = z.object({
  hero: z.object({
    eyebrow: nonEmptyText.optional(),
    title: nonEmptyText,
    description: nonEmptyText,
    primaryCta: landingCtaSchema,
    secondaryCta: landingCtaSchema.optional(),
  }),
  preview: landingPreviewSchema,
  steps: z.object({
    title: nonEmptyText,
    description: nonEmptyText.optional(),
    items: z.tuple([
      landingStepBaseSchema.extend({ preview: landingStepInstructionPreviewSchema }),
      landingStepBaseSchema.extend({ preview: landingStepActionPreviewSchema }),
      landingStepBaseSchema.extend({ preview: landingStepResultPreviewSchema }),
    ]),
  }),
  features: z.object({
    title: nonEmptyText,
    description: nonEmptyText.optional(),
    items: z
      .array(
        z.object({
          icon: landingIconSchema,
          title: nonEmptyText,
          description: nonEmptyText,
        }),
      )
      .length(3),
  }),
  showcase: z.object({
    title: nonEmptyText,
    description: nonEmptyText.optional(),
    items: z
      .array(
        z.object({
          label: nonEmptyText,
          media: landingMediaSchema,
        }),
      )
      .min(1)
      .max(6),
  }),
  finalCta: z.object({
    title: nonEmptyText,
    description: nonEmptyText,
    action: landingCtaSchema,
    backgroundMedia: landingMediaSchema.optional(),
```

```
  }),
});

export type LandingContent = z.infer<typeof landingContentSchema>;
export type LandingIconName = z.infer<typeof landingIconSchema>;
export type LandingMedia = z.infer<typeof landingMediaSchema>;
export type LandingStepPreview = z.infer<typeof landingStepPreviewSchema>;

export function parseLandingContent(input: unknown): LandingContent {
  return landingContentSchema.parse(input);
}
```

app/packages/app-landing/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "noEmit": true,
    "isolatedModules": true,
    "allowImportingTsExtensions": true,
    "skipLibCheck": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules"]
}
```

app/packages/fnf-react/AGENTS.md

```
# `@higgsfield/fnf-react` Agent Guide

The canonical AI agent guide lives at [`ai/AGENTS.md`](./ai/AGENTS.md).

Read that file before generating React integration code with
`@higgsfield/fnf-react`.
```

app/packages/fnf-react/ai/AGENTS.md

````
# Instructions for AI agents using `@higgsfield/fnf-react`

Canonical AI guide for React apps that consume `@higgsfield/fnf-react`.
This package is the React integration layer over `@higgsfield/fnf`: TanStack
Query options for pull-shaped reads, external-store controllers for push-shaped
lifecycles (submit runs, uploads), stable SDK clients from one provider, and no
UI/product policy. All generation cache writes go through one door so job,
job-set, and feed entries stay consistent and terminal statuses never regress.

For model contracts, routes, settings, media validation, profile types, errors,
credits, and adapter behavior, read `../../fnf/ai/AGENTS.md` first. This guide
only explains the React integration layer. For generated UI, pair this package
with `@higgsfield/quanta` and read `../../quanta/ai/AGENTS.md`; `fnf-react`
does not provide components, styles, layout rules, or product copy.

## TL;DR rules

1. **Use both providers.** Host apps provide TanStack's `QueryClientProvider`;
   this package provides `FnfProvider`.
2. **Create SDK clients through `FnfProvider` in React code.** Use
   `createFnfReactClients()` only for tests or advanced non-provider hosts.
3. **Keep adapter, jobs, uploader, resolver, and observability references
   stable.** Controllers are stateful and bind to the first client/options they
   receive.
4. **Always use `scopeKey` for user/workspace caches.** Recommended shape:
   `${userId}:${workspaceId}`. Pass `{ scopeKey }` to query helpers and cache
   door helpers.
5. **Use query-option factories.** Use `generationQueryOptions`,
   `jobSetQueryOptions`, `jobsFeedQueryOptions`, character/reference/profile
   query options, and `costQueryOptions`. Never inline TanStack query-key
   arrays.
6. **Use the cache door for generation writes.** `applyGenerations`,
   `prependGenerations`, and `removeGenerationQueries` are the only supported
   generation cache mutation helpers.
7. **Resume live feed items.** Call `useLiveFeedGenerations` with the flattened
   feed so queued/in-progress jobs restored after a refresh keep polling.
8. **Use controllers for push lifecycles.** `useGenerationRun` handles submit
   to terminal state. `useAttachments` handles file upload and submit-ready
   refs.
9. **Errors are state, not UI policy.** `run.start()` does not reject; read
   `run.error`, `run.failed`, and generation statuses. This package does not
   show toasts, redirect, gate plans, or write product copy.
10. **Workspace switching is only SDK/backend context.** Host apps still update
    Clerk/session metadata, adapter header state, routing, and copy.
11. **Observability is safe metadata-only.** Never emit prompts, raw params,
    headers, tokens, URLs, filenames, emails, workspace names, or file bytes.
12. **Use Quanta for the actual interface.** Build forms, feeds, profile panels,
     drawers, command palettes, toasts, and status surfaces with Quanta
     components in host apps; keep this package headless.

## Provider setup

```tsx
import { createFnfWebAdapter } from '@higgsfield/fnf-adapters'
import { seedance2_0 } from '@higgsfield/fnf/jobs'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { useMemo } from 'react'
import {
  FnfProvider,
  useFnf,
  useFnfCharacterClient,
  useFnfJobClient,
  useFnfJobs,
  useFnfLlmClient,
  useFnfMediaClient,
  useFnfObservability,
  useFnfProfileClient,
  useFnfReferenceClient,
````
````

````
  useFnfScopeKey,
} from '@higgsfield/fnf-react'

const queryClient = new QueryClient()

function Root() {
  const adapter = useMemo(
    () => createFnfWebAdapter({
      baseUrl,
      getToken,
      workspaceId: activeWorkspaceId,
    }),
    [activeWorkspaceId, baseUrl, getToken],
  )
  const scopeKey = `${userId}:${activeWorkspaceId}`

  return (
    <QueryClientProvider client={queryClient}>
      <FnfProvider
        adapter={adapter}
        jobs={[seedance2_0]}
        scopeKey={scopeKey}
        observability={observability}
      >
        <App />
      </FnfProvider>
    </QueryClientProvider>
  )
}

function App() {
  const fnf = useFnf()
  const characters = useFnfCharacterClient()
  const jobs = useFnfJobClient()
  const media = useFnfMediaClient()
  const profile = useFnfProfileClient()
  const references = useFnfReferenceClient()
  const registry = useFnfJobs()
  const scopeKey = useFnfScopeKey()
  const observability = useFnfObservability()

  void fnf
  void characters
  void jobs
  void media
  void profile
  void references
  void registry
  void scopeKey
  void observability
}
```

`FnfProvider` accepts:

- `adapter`: required jobs backend. The fnf-web adapter also satisfies media
  and profile ports. UI hosts that submit generations must pass the adapter's
  `confirm` option — see "Submission confirmation gate" in
  `../../fnf/ai/AGENTS.md`.
- `jobs`: required SDK model registry. Model/settings autocomplete flows from
  this array.
- `mediaAdapter`, `profileAdapter`: optional overrides when jobs/media/profile
  use different ports.
- `characterAdapter`, `referenceAdapter`: optional overrides for
  custom-reference training and the user's Elements library.
- `blobUploader`, `resolveJob`: optional media-client dependencies.
- `llm`: optional browser-safe LLM chat backend. Read it with
  `useFnfLlmClient()` — the hook throws if no `llm` was provided. Generated
  apps keep the raw zero-token `createLlmClient` instance Worker-side on
  `https://fnf.internal/llm`, discover current ids there with `listModels()`,
````
````

````
  and proxy app-safe operations through authenticated server functions or
  routes. Never pass the raw client or a hardcoded catalog to browser code (see
  "LLM (chat completions)" in `../../fnf/ai/AGENTS.md`).
- `scopeKey`: optional cache isolation key; use it for user/workspace apps.
- `observability`: optional safe SDK observer config propagated into created
  job, media, profile clients and React controllers.

Use `createFnfReactClients(config)` for unit tests or non-provider hosts. Normal
components should read clients from provider hooks.

Important stability rule: do not create adapters or job arrays inline if that
causes a new reference every render. Keep them at module scope, in state, or in
memoized host infrastructure. Controllers intentionally bind to the first client
they receive.

## Browser/server boundary

`fnf-react` can run in browser React, but auth policy belongs to the host.

- Product apps may use a browser adapter only when the host intentionally
  provides a safe user-scoped token source.
- Generated app templates with server-only credentials should use browser
  adapters that call app-local `/api/*` routes, not direct fnf backend routes.
- Never expose dev user ids, bearer tokens, or privileged workspace headers to
  arbitrary visitors.

This package should not import environment variables or decide where secrets
live.

## Query options: pull side

Always use the exported query-option factories. They are the public contract for
polling, stale-time, key shapes, and scope behavior.

```tsx
import { useInfiniteQuery, useQuery } from '@tanstack/react-query'
import {
  characterQueryOptions,
  costQueryOptions,
  flattenFeedPages,
  generationQueryOptions,
  jobSetQueryOptions,
  jobsFeedQueryOptions,
  profileSnapshotQueryOptions,
  referenceQueryOptions,
  referencesQueryOptions,
  useLiveFeedGenerations,
} from '@higgsfield/fnf-react'

const scope = { scopeKey }

const generation = useQuery(generationQueryOptions(jobClient, jobId, scope))
const character = useQuery(characterQueryOptions(characterClient, characterId, scope))
const element = useQuery(referenceQueryOptions(referenceClient, elementId, scope))
const elements = useQuery(referencesQueryOptions(
  referenceClient,
  { category: 'character' },
  scope,
))
const jobSet = useQuery(jobSetQueryOptions(jobClient, jobSetId, scope))

const feed = useInfiniteQuery({
  ...jobsFeedQueryOptions(jobClient, { type: 'video', size: 20 }, scope),
  select: flattenFeedPages,
})
useLiveFeedGenerations(jobClient, feed.data ?? [], scope)

const snapshot = useQuery(profileSnapshotQueryOptions(profileClient, scope))
const cost = useQuery(costQueryOptions(jobClient, input, {
  ...scope,
````
````

```
  enabled: wirePreview.ok,
}))
```

Defaults:

- `generationQueryOptions` polls every `DEFAULT_POLL_INTERVAL_MS` while
  non-terminal, in background, and stops once terminal.
- `jobSetQueryOptions` polls one whole job set per tick and stops when every
  member is terminal.
- `jobsFeedQueryOptions` is door-driven with `staleTime: Infinity` by default.
  Use `invalidateQueries` for hard refreshes or spread your own `staleTime` if
  a host wants focus refetches.
- `flattenFeedPages` converts paginated feed data into a deduplicated list.
- `characterQueryOptions` polls character training until `completed` or
  `failed`; `referenceQueryOptions` polls processing Elements, while
  `referencesQueryOptions` lists the user's Elements library.

Query keys and scope

Use `fnfKeys` factories only. Unscoped key shapes are backward-compatible public
contracts:

```ts
fnfKeys.job('g1')              // ['fnf', 'job', 'g1']
fnfKeys.jobSet('set1')         // ['fnf', 'job-set', 'set1']
fnfKeys.jobs({ type: 'video' }) // ['fnf', 'jobs', { type: 'video' }]
```

Scoped variants are additive:

```ts
const scope = { scopeKey: `${userId}:${workspaceId}` }

fnfKeys.job('g1', scope)
fnfKeys.jobSet('set1', scope)
fnfKeys.jobs({ type: 'video' }, scope)
fnfKeys.cost(input, scope)
fnfKeys.character('character1', scope)
fnfKeys.reference('element1', scope)
fnfKeys.references({ category: 'character' }, scope)
fnfKeys.profileSnapshot(scope)
fnfKeys.profileCredits({ ...scope, includeOnDemand: false })
```

Do not write `['fnf', ...]` arrays by hand in app code.

After character training completes, invalidate the matching
`fnfKeys.references(...)` query so the new Character Element appears. The
Element id and `reference.characterId` are distinct; pass `characterId` to
Soul generation as `settings.customReferenceId`.

Cache door rules

Use these helpers for generation cache mutations:

```tsx
import {
  applyGenerations,
  prependGenerations,
  removeGenerationQueries,
} from '@higgsfield/fnf-react'

applyGenerations(queryClient, liveGenerations, { scopeKey })
prependGenerations(queryClient, { type: 'video', size: 20 }, submittedGenerations, { scopeKey })
removeGenerationQueries(queryClient, { scopeKey })
```

Rules:

- `applyGenerations` updates existing caches only. It folds fresh snapshots into
  job, job-set, and feed entries that already contain those ids.
- `prependGenerations` inserts fresh submit results into one explicit feed. The
  host chooses which feeds should show optimistic new work.
- `removeGenerationQueries` clears generation, job-set, feed, and cost caches
  for a scope without clearing profile caches.
- `foldGeneration` prevents stale non-terminal snapshots from reopening settled
  generations and preserves object references when nothing changed.
- Direct `queryClient.setQueryData(...)` for generation snapshots is an
  anti-pattern outside tested package helpers.

Submit lifecycle controller

Use `useGenerationRun` in components and `GenerationRun` directly in tests or
framework adapters.

```tsx
import {
  prependGenerations,
  useGenerationRun,
} from '@higgsfield/fnf-react'

const run = useGenerationRun(jobClient, { scopeKey })

async function submit(input: Parameters<typeof jobClient.submit>[0]) {
  const generations = await run.start(input)
  prependGenerations(queryClient, { type: 'video', size: 20 }, generations, { scopeKey })
}
```

Lifecycle:

- `idle -> submitting -> generating -> completed | failed | aborted`
- `run.start(input)` resolves with settled generations and never rejects.
- `run.error` holds submit/poll run-level errors.
- `run.failed` and `run.warning` hold partial fan-out submit failures.
- Each generation's own `status` remains the per-job verdict surface.
- Starting a new run aborts the previous polling loop and late results are
  dropped.
- Unmounting the hook aborts polling. It does not cancel backend jobs; call the
  SDK job client's `cancel(id)` for server-side cancellation.
- Every controller commit folds `run.generations` through `applyGenerations`.
  It does not decide which feed should show fresh submits; call
  `prependGenerations` explicitly.

The job client passed to `useGenerationRun` must ultimately reach a server-side
SDK call. If a host wraps SDK calls in TanStack server functions or `/api/*`
routes, submit and cost must be `POST`, not `GET`. A 405 / "Method Not Allowed"
on generation is usually a route/function method mismatch below the React
controller, an old `operation`-envelope WFP adapter snapshot, or a backend route
contract mismatch. It is not a `useGenerationRun` state problem.

Do not send raw browser files through React Query variables or server-function
JSON inputs. For server-only fnf.internal hosts, upload files via multipart
`FormData` to an app-local `POST /api/media/upload` route and store only the
returned `MediaRef` in React state. `useGenerationRun` should receive
prompt/settings/refs, never `File`, `Blob`, `ArrayBuffer`, base64, or byte
arrays.

`useGenerationRun` receives provider observability by default. Pass
`observability` in hook options only when a specific run needs a different
observer.

Attachments controller

Use `useAttachments` for file inputs. It uploads immediately, keeps previews,
and returns submit-ready refs.

```tsx
import { useAttachments } from '@higgsfield/fnf-react'
```

```
const attachments = useAttachments(mediaClient, {
  upload: { forceIpCheck: true },
})

function onFiles(files: File[]) {
  attachments.add(files, { role: 'start_image' })
}

async function submit() {
  const refs = await attachments.settled()
  await jobClient.submit({
    model: 'seedance_2_0',
    media: { start_image: refs },
    prompt,
    settings,
  })
}
```

Lifecycle:

- File items start as `uploading`, then become `ready`, `blocked`, or `failed`.
- Already-uploaded `MediaRef` values are added as `ready`.
- `blocked` is moderation-as-state; it is not thrown and does not appear in
  `attachments.refs`.
- `failed` items can be retried with `attachments.retry(key)`.
- `remove(key)` aborts an in-flight upload and revokes local preview URLs.
- `clear()` aborts all in-flight uploads and removes every item.
- `settled()` waits for in-flight uploads and returns ready refs.
- `move(from, to)` is presentational ordering.
- Intrinsic image/video/audio meta is measured by default from local files and
  attached to `MediaRef.meta`. Set `measure: false` to disable it. Measurement
  is best-effort by contract: a failed measurement never fails the upload.
- Use `useAttachments` only when its media client can upload in the browser
  safely. If the adapter/token must stay server-side, build a multipart upload
  route and feed the returned `MediaRef` into the same UI state shape.

The controller does not enforce job media counts, roles, dimensions, or
duration limits. The SDK job declaration validates those when building/submitting
the job.

Profile and workspace

Use profile query options for account/workspace reads.

```tsx
import { useQuery } from '@tanstack/react-query'
import {
  profileCreditsQueryOptions,
  profileCurrentWorkspaceQueryOptions,
  profileSnapshotQueryOptions,
  profileUserQueryOptions,
  profileWalletQueryOptions,
  profileWorkspacesQueryOptions,
  setProfileSnapshot,
} from '@higgsfield/fnf-react'

const profile = useFnfProfileClient()
const scope = { scopeKey }

const snapshot = useQuery(profileSnapshotQueryOptions(profile, scope))
const user = useQuery(profileUserQueryOptions(profile, scope))
const workspaces = useQuery(profileWorkspacesQueryOptions(profile, scope))
const current = useQuery(profileCurrentWorkspaceQueryOptions(profile, scope))
const wallet = useQuery(profileWalletQueryOptions(profile, scope))
const credits = useQuery(profileCreditsQueryOptions(profile, {
  ...scope,
  includeOnDemand: true,
}))
```

```
setProfileSnapshot(queryClient, snapshot.data!, scope)
```

Workspace switching:

```tsx
import { useSwitchWorkspaceMutation } from '@higgsfield/fnf-react'

const switchWorkspace = useSwitchWorkspaceMutation({
  scopeKey,
  nextScopeKey: snapshot => `${snapshot.user?.id}:${snapshot.currentWorkspace?.id}`,
  onWorkspaceChanged: async snapshot => {
    setActiveWorkspaceId(snapshot.currentWorkspace?.id ?? '')
    await updateHostSession(snapshot)
  },
})

await switchWorkspace.mutateAsync({ workspaceId })
```

On success, the mutation helper:

- calls `profile.switchWorkspace({ workspaceId })`
- removes generation/cost caches for the old scope
- removes old scoped profile caches when the scope changes
- writes the returned snapshot into profile caches for the next scope
- calls `onWorkspaceChanged(snapshot)`

It does not update Clerk/session metadata, mutate adapter header state, reload
the page, redirect, or show copy. The host must do those things.

Request preview and cost

Use local wire previews to show validation/build-wire state without submitting.

```tsx
import {
  costQueryOptions,
  getWirePreview,
  useFnfWirePreview,
} from '@higgsfield/fnf-react'

const preview = useFnfWirePreview(input)
const manual = getWirePreview(input, jobs)

const cost = useQuery(costQueryOptions(jobClient, input, {
  scopeKey,
  enabled: preview.ok,
}))

if (preview.ok) {
  preview.jobSetType
  preview.outputType
  preview.params
}
else {
  preview.error.code
}
```

`cost.data?.credits` is already a display-credit value from the core SDK.
Profile wallet raw balances are credit-cents, but profile credits returned by
the SDK profile domain are normalized for display.

Observability

`FnfProvider` accepts the core SDK observability options and passes them into
created clients. `useGenerationRun` and `useAttachments` also inherit provider
observability unless hook options override it.

```
Safe React event names include:

- `fnf.react.generation_run.start`
- `fnf.react.generation_run.submitted`
- `fnf.react.generation_run.progress`
- `fnf.react.generation_run.completed`
- `fnf.react.generation_run.failed`
- `fnf.react.generation_run.aborted`
- `fnf.react.generation_run.reset`
- `fnf.react.attachments.add`
- `fnf.react.attachments.upload_start`
- `fnf.react.attachments.ready`
- `fnf.react.attachments.blocked`
- `fnf.react.attachments.failed`
- `fnf.react.attachments.retry`
- `fnf.react.attachments.remove`
- `fnf.react.attachments.clear`
- `fnf.react.attachments.settled`

Observers are metadata-only; see the observability privacy rule in
`../../fnf/ai/AGENTS.md`.

## Realtime helpers

`Realtime` and `RefCountPool` are low-level helpers for shared live channels.
They are transport-agnostic and optional.

- Subscribe by job set when a transport has a real-time channel.
- Reuse one connection per job set through ref counting; the pool keeps the
  connection open for `freeGraceMs` (default 1000 ms) after the last
  unsubscribe so quick re-subscribes reuse it.
- Let event callbacks invalidate `fnfKeys.jobSet(jobSetId, { scopeKey })`.
- If a transport returns `undefined` or throws, treat it as "no channel" and
  rely on polling query options.

## When in doubt

Read `../../fnf/ai/AGENTS.md` for SDK/domain behavior, use `src/index.ts` as
the public API list, and keep host policy in the host app, not in `fnf-react`.
```

app/packages/fnf-react/package.json

```
{
  "name": "@higgsfield/fnf-react",
  "version": "0.0.0",
  "private": true,
  "description": "React bindings for @higgsfield/fnf — TanStack Query options + cache door, and the stateful presenters (attachments, submit run)",
  "author": "support@higgsfield.ai",
  "license": "UNLICENSED",
  "type": "module",
  "exports": {
    ".": "./src/index.ts",
    "./package.json": "./package.json"
  },
  "files": ["src"],
  "scripts": {
    "test": "vitest run",
    "typecheck": "tsc --noEmit -p tsconfig.json"
  },
  "peerDependencies": {
    "@higgsfield/fnf": "0.0.0",
    "@tanstack/react-query": "^5.40.0",
    "react": ">=18"
  },
  "devDependencies": {
    "@higgsfield/fnf": "0.0.0",
    "@tanstack/react-query": "^5.74.4",
    "@types/node": "^20.17.30",
    "@types/react": "^19.0.0",
    "react": "^19.0.0",
    "typescript": "^5.8.3",
    "vitest": "^4.0.17"
  }
}
```

app/packages/fnf-react/README.md

```
# @higgsfield/fnf-react

React bindings for `@higgsfield/fnf` — the layer that makes building a new
Higgsfield-style frontend fast, without leaking the SDK's internals or
inventing restrictions the SDK doesn't have.

The split is by shape: pull-shaped reads (feeds, job sets, single
generations) go through TanStack Query via `queryOptions` factories and a
cache door — you bring the `QueryClient`. Push-driven processes (submit runs,
upload lists) are framework-agnostic `ExternalStore` controllers with thin
`useSyncExternalStore` hooks. Clients come from one provider (`FnfProvider`);
the package has no auth, billing, routing, copy, or UI opinions.

An optional LLM backend may be provided to `FnfProvider` and read with
`useFnfLlmClient()`. In generated apps, it must be a browser-safe proxy: keep
the raw `createLlmClient` instance Worker-side, discover current model ids
there with `listModels()`, and expose app-safe operations through authenticated
server functions or routes. Never bundle the raw client or a hardcoded catalog.

## The cache door

TanStack is a non-normalizing document cache — one generation lives as
copies under many keys. The door makes that safe: every write goes through
`foldGeneration` (terminal anti-regress — a stale read can never roll back a
settled generation; reference stability — an unchanged snapshot keeps the
previous object, so memoized tiles don't re-render). The door **updates, it
never seeds**: unknown ids are ignored, membership belongs to fetches and
`prependGenerations`. Which feeds should show a fresh submit is product
policy — the fan-out stays your explicit call.

Feed queries are intentionally stable after loading. Mount
`useLiveFeedGenerations(client, flattenedFeed, { scopeKey })` beside a visible
feed so non-terminal jobs restored after refresh keep polling; the hook writes
fresh snapshots through the same cache door and stops observing terminal jobs.

## What this package deliberately is NOT

- Not a second cache — TanStack owns caching; the package owns the domain
  rules of writing into it.
- Not a UI kit and not product policy — no toasts, no plan gates, no copy.
- Not a second SDK — everything it exposes is the SDK's own domain model
  (`Generation`, `MediaRef`, `ApiJobError`).

All rules, code idioms, and API usage live in `ai/AGENTS.md`.
```

app/packages/fnf-react/src/__tests__/attachments.test.ts

```
import type { UploadInput, UploadResult } from '@higgsfield/fnf/media'
import type { FnfObservationEvent } from '@higgsfield/fnf/observability'
import { describe, expect, it } from 'vitest'
import { AttachmentsController } from '../attachments'

function fakeMedia(respond?: (input: UploadInput) => Partial<UploadResult> | Error) {
  const uploads: UploadInput[] = []
  let release: (() => void) | undefined
  const gate = new Promise<void>(resolve => (release = resolve))
  const media = {
    async upload(input: UploadInput): Promise<UploadResult> {
      uploads.push(input)
      await gate
      const out = respond?.(input)
      if (out instanceof Error)
        throw out
      return {
        ref: { id: `m${uploads.length}`, type: 'media_input', url: `https://cdn/m${uploads.length}.png`, ...(input.role ? { role: input.role } : {}) },
        mediaId: `m${uploads.length}`,
        status: 'uploaded',
        type: 'image',
        contentType: 'image/png',
        filename: String(input.filename ?? 'x.png'),
        ...out,
      }
    },
  }
  return { media, uploads, release: () => release?.() }
}

const file = (name = 'cat.png') => new File([new Uint8Array([1, 2, 3])], name, { type: 'image/png' })

describe('AttachmentsController', () => {
  it('files preview immediately, upload in background, and become submit-ready refs', async () => {
    const { media, uploads, release } = fakeMedia()
    const attachments = new AttachmentsController(media, { upload: { forceIpCheck: true }, measure: false })

    const [key] = attachments.add(file(), { role: 'start_image' })
    expect(attachments.items[0]).toMatchObject({ key, status: 'uploading' })
    expect(attachments.isUploading).toBe(true)
    expect(attachments.refs).toEqual([]) // not ready yet

    release()
    const refs = await attachments.settled()

    expect(uploads[0]).toMatchObject({ filename: 'cat.png', role: 'start_image', forceIpCheck: true, throwOnModeration: false })
    expect(refs).toHaveLength(1)
    expect(attachments.items[0]).toMatchObject({ status: 'ready', previewUrl: 'https://cdn/m1.png' })
  })

  it('already-uploaded refs are ready as-is; order is presentational (move)', () => {
    const { media } = fakeMedia()
    const attachments = new AttachmentsController(media)
    attachments.add({ id: 'r1', type: 'media_input', url: 'https://cdn/r1.png' })
    attachments.add({ id: 'r2', type: 'media_input', url: 'https://cdn/r2.png' })

    attachments.move(1, 0)

    expect(attachments.refs.map(r => r.id)).toEqual(['r2', 'r1'])
  })

  it('a moderation verdict becomes a blocked item, not a throw', async () => {
    const { media, release } = fakeMedia(() => ({ status: 'ip_detected', moderation: { status: 'ip_detected' } }))
    const attachments = new AttachmentsController(media, { measure: false })
    attachments.add(file())
    release()

    const refs = await attachments.settled()
```

```
  expect(refs).toEqual([]) // blocked items never feed a submit
  expect(attachments.items[0]).toMatchObject({ status: 'blocked', moderation: { status: 'ip_detected' } })
})

it('retry keeps the original role and options', async () => {
  let attempts = 0
  const { media, uploads, release } = fakeMedia(() => (attempts++ === 0 ? new Error('boom') : {}))
  const attachments = new AttachmentsController(media, { measure: false })
  const [key] = attachments.add(file(), { role: 'start_image', forceIpCheck: true })
  release()
  await attachments.settled()

  attachments.retry(key)
  await attachments.settled()

  expect(uploads[1]).toMatchObject({ role: 'start_image', forceIpCheck: true }) // not dropped on retry
  expect(attachments.items[0].role).toBe('start_image')
})

it('removing an in-flight item aborts its upload signal', async () => {
  let seenSignal: AbortSignal | undefined
  const { media, release } = fakeMedia()
  const wrapped = {
    async upload(input: UploadInput) {
      seenSignal = input.signal
      return media.upload(input)
    },
  }
  const attachments = new AttachmentsController(wrapped, { measure: false })
  const [key] = attachments.add(file())
  attachments.remove(key)
  release()
  await attachments.settled()

  expect(seenSignal?.aborted).toBe(true)
})

it('a failed upload carries the typed error and is retryable', async () => {
  let attempts = 0
  const { media, release } = fakeMedia(() => (attempts++ === 0 ? new Error('socket hang up') : {}))
  const attachments = new AttachmentsController(media, { measure: false })
  const [key] = attachments.add(file())
  release()
  await attachments.settled()
  expect(attachments.items[0]).toMatchObject({ status: 'failed', error: { code: 'unexpected', message: 'socket hang up' } })

  attachments.retry(key)
  const refs = await attachments.settled()

  expect(refs).toHaveLength(1)
  expect(attachments.items[0].status).toBe('ready')
})

it('removing an item mid-upload drops the late result', async () => {
  const { media, release } = fakeMedia()
  const attachments = new AttachmentsController(media, { measure: false })
  const [key] = attachments.add(file())
  attachments.remove(key)
  release()

  const refs = await attachments.settled()

  expect(refs).toEqual([])
  expect(attachments.items).toEqual([])
})

it('emits safe observability events for attachment lifecycle', async () => {
  const events: FnfObservationEvent[] = []
  const { media, release } = fakeMedia()
  const attachments = new AttachmentsController(media, {
```

```
      measure: false,
      observability: {
        observer: (event) => {
          events.push(event)
        },
      },
    })

    attachments.add(file('private-cat.png'), { role: 'start_image' })
    release()
    await attachments.settled()
    attachments.clear()

    expect(events.map(event => event.name)).toEqual(expect.arrayContaining([
      'fnf.react.attachments.add',
      'fnf.react.attachments.upload_start',
      'fnf.react.attachments.ready',
      'fnf.react.attachments.settled',
      'fnf.react.attachments.clear',
    ]))
    expect(JSON.stringify(events)).not.toContain('private-cat.png')
    expect(JSON.stringify(events)).not.toContain('https://cdn/')
  })
})

describe('AttachmentsController — the measure path (default on)', () => {
  /** Stub the structural DOM the SDK resolver probes; `loads` scripts the attempts. */
  function stubDom(loads: boolean[]) {
    const attempts = { count: 0 }
    class FakeImage {
      naturalWidth = 800
      naturalHeight = 600
      onload: (() => void) | null = null
      onerror: (() => void) | null = null
      #src = ''

      get src() {
        return this.#src
      }

      set src(value: string) {
        this.#src = value
        const ok = loads[Math.min(attempts.count++, loads.length - 1)]
        queueMicrotask(() => (ok ? this.onload?.() : this.onerror?.()))
      }
    }
    ;(globalThis as Record<string, unknown>).Image = FakeImage
    ;(globalThis as Record<string, unknown>).document = { createElement: () => ({}) }
    return {
      attempts,
      restore: () => {
        delete (globalThis as Record<string, unknown>).Image
        delete (globalThis as Record<string, unknown>).document
      },
    }
  }

  it('measured intrinsic size lands on the submit-ready ref', async () => {
    const dom = stubDom([true])
    try {
      const { media, release } = fakeMedia()
      const attachments = new AttachmentsController(media)
      attachments.add(file())
      release()

      const refs = await attachments.settled()

      expect(refs[0].meta).toEqual({ width: 800, height: 600 })
    }
    finally {
```

```
      dom.restore()
    }
  })

  it('a failed measurement NEVER fails a successful upload — ready, just without meta', async () => {
    const dom = stubDom([false]) // e.g. a PDF mislabeled as image, a broken codec
    try {
      const { media, release } = fakeMedia()
      const attachments = new AttachmentsController(media)
      attachments.add(file())
      release()

      const refs = await attachments.settled()

      expect(attachments.items[0].status).toBe('ready')
      expect(refs).toHaveLength(1)
      expect(refs[0].meta).toBeUndefined()
    }
    finally {
      dom.restore()
    }
  })

  it('retry after a failed upload re-measures instead of replaying a cached failure', async () => {
    const dom = stubDom([false, true])
    try {
      let uploadAttempts = 0
      const { media, release } = fakeMedia(() => (uploadAttempts++ === 0 ? new Error('boom') : {}))
      const attachments = new AttachmentsController(media)
      const [key] = attachments.add(file())
      release()
      await attachments.settled()
      expect(attachments.items[0].status).toBe('failed')

      attachments.retry(key)
      const refs = await attachments.settled()

      expect(attachments.items[0].status).toBe('ready')
      expect(refs[0].meta).toEqual({ width: 800, height: 600 }) // the second measurement, not the poisoned first
    }
    finally {
      dom.restore()
    }
  })
})
```

app/packages/fnf-react/src/__tests__/character-query.test.ts

```
import type { Character } from '@higgsfield/fnf/characters'
import { QueryClient } from '@tanstack/react-query'
import { describe, expect, it, vi } from 'vitest'
import { characterQueryOptions } from '../character-query'
import { fnfKeys } from '../keys'

function evaluate<T>(fn: unknown, data: T): number | false | undefined {
  return (fn as (query: { state: { data?: T } }) => number | false | undefined)({ state: { data } })
}

const queued: Character = {
  id: '11111111-1111-4111-8111-111111111111',
  type: 'soul_2',
  status: 'queued',
}

describe('character query options', () => {
  it('uses a scoped key and stops polling when character training settles', async () => {
    const client = { get: vi.fn(async () => queued) }
    const options = characterQueryOptions(client, queued.id, { scopeKey: 'u:w' })
    const queryClient = new QueryClient()

    await expect(queryClient.fetchQuery(options)).resolves.toEqual(queued)
    expect(queryClient.getQueryData(fnfKeys.character(queued.id, { scopeKey: 'u:w' }))).toEqual(queued)
    expect(queryClient.getQueryData(fnfKeys.character(queued.id))).toBeUndefined()
    expect(evaluate(options.refetchInterval, queued)).toBe(5000)
    expect(evaluate(options.refetchInterval, { ...queued, status: 'completed' as const })).toBe(false)
    expect(evaluate(options.refetchInterval, { ...queued, status: 'failed' as const })).toBe(false)
  })
})
```

app/packages/fnf-react/src/__tests__/generation-cache.test.ts

```
import type { Generation, ListResult } from '@higgsfield/fnf/client'
import type { InfiniteData } from '@tanstack/react-query'
import { QueryClient } from '@tanstack/react-query'
import { describe, expect, it } from 'vitest'
import { applyGenerations, prependGenerations, removeGenerationQueries } from '../generation-cache'
import { fnfKeys } from '../keys'

function gen(id: string, status: Generation['status'], extra?: Partial<Generation>): Generation {
  return { id, model: 'demo', type: 'image', status, input: { model: 'demo', settings: {} }, ...extra }
}

function feedData(pages: Generation[][]): InfiniteData<ListResult> {
  return {
    pages: pages.map((items, at) => ({ items, ...(at < pages.length - 1 ? { cursor: at } : {}) })),
    pageParams: pages.map((_p, at) => (at === 0 ? undefined : at - 1)),
  }
}

describe('applyGenerations — the single write door', () => {
  it('folds one snapshot into EVERY cache entry that holds it: job, job set, all feeds', () => {
    const qc = new QueryClient()
    qc.setQueryData(fnfKeys.job('a'), gen('a', 'queued'))
    qc.setQueryData(fnfKeys.jobSet('set-1'), [gen('a', 'queued', { jobSetId: 'set-1' }), gen('b', 'queued', { jobSetId: 'set-1' })])
    qc.setQueryData(fnfKeys.jobs({ type: 'image' }), feedData([[gen('a', 'queued')]]))
    qc.setQueryData(fnfKeys.jobs({}), feedData([[gen('z', 'completed'), gen('a', 'queued')]]))

    const fresh = gen('a', 'completed', { jobSetId: 'set-1', results: { rawUrl: 'https://x/a.png' } })
    applyGenerations(qc, [fresh])

    expect(qc.getQueryData<Generation>(fnfKeys.job('a'))?.status).toBe('completed')
    expect(qc.getQueryData<Generation[]>(fnfKeys.jobSet('set-1'))?.map(g => g.status)).toEqual(['completed', 'queued'])
    expect(qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({ type: 'image' }))?.pages[0].items[0].status).toBe('completed')
    expect(qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({}))?.pages[0].items[1].status).toBe('completed')
  })

  it('updates, never seeds: ids nothing holds are ignored, absent keys are not created', () => {
    const qc = new QueryClient()
    qc.setQueryData(fnfKeys.jobs({}), feedData([[gen('a', 'queued')]]))

    applyGenerations(qc, [gen('stranger', 'completed', { jobSetId: 'set-9' })])

    expect(qc.getQueryData(fnfKeys.job('stranger'))).toBeUndefined()
    expect(qc.getQueryData(fnfKeys.jobSet('set-9'))).toBeUndefined()
    expect(qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({}))?.pages[0].items.map(g => g.id)).toEqual(['a'])
  })

  it('the terminal anti-regress holds at the door: a stale tick cannot roll back a feed tile', () => {
    const qc = new QueryClient()
    qc.setQueryData(fnfKeys.jobs({}), feedData([[gen('a', 'completed', { results: { rawUrl: 'https://x/a.png' } })]]))

    applyGenerations(qc, [gen('a', 'in_progress')])

    expect(qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({}))?.pages[0].items[0].status).toBe('completed')
  })

  it('a write that changes nothing bails out — references stay stable for memoization', () => {
    const qc = new QueryClient()
    qc.setQueryData(fnfKeys.jobs({}), feedData([[gen('a', 'in_progress')]]))
    const before = qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({}))

    applyGenerations(qc, [gen('a', 'in_progress')]) // a tick with no observable change

    expect(qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({}))).toBe(before)
  })

  it('updates only the named scope when scoped', () => {
    const qc = new QueryClient()
    qc.setQueryData(fnfKeys.job('a'), gen('a', 'queued'))
```

```
    qc.setQueryData(fnfKeys.job('a', { scopeKey: 'u:w' }), gen('a', 'queued'))
    qc.setQueryData(fnfKeys.jobs({}, { scopeKey: 'u:w' }), feedData([[gen('a', 'queued')]]))

    applyGenerations(qc, [gen('a', 'completed', { results: { rawUrl: 'https://x/a.png' } })], { scopeKey: 'u:w' })

    expect(qc.getQueryData<Generation>(fnfKeys.job('a'))?.status).toBe('queued')
    expect(qc.getQueryData<Generation>(fnfKeys.job('a', { scopeKey: 'u:w' }))?.status).toBe('completed')
    expect(qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({}, { scopeKey: 'u:w' }))?.pages[0].items[0].status).toBe('completed')
  })

  it('removes generation queries for one scope without touching profile or other scopes', () => {
    const qc = new QueryClient()
    qc.setQueryData(fnfKeys.job('a', { scopeKey: 'old' }), gen('a', 'queued'))
    qc.setQueryData(fnfKeys.jobs({}, { scopeKey: 'old' }), feedData([[gen('a', 'queued')]]))
    qc.setQueryData(fnfKeys.job('b', { scopeKey: 'new' }), gen('b', 'queued'))
    qc.setQueryData(fnfKeys.profileSnapshot({ scopeKey: 'old' }), { user: null, workspaces: [], currentWorkspace: null, wallet: null, credits: null })

    removeGenerationQueries(qc, { scopeKey: 'old' })

    expect(qc.getQueryData(fnfKeys.job('a', { scopeKey: 'old' }))).toBeUndefined()
    expect(qc.getQueryData(fnfKeys.jobs({}, { scopeKey: 'old' }))).toBeUndefined()
    expect(qc.getQueryData(fnfKeys.job('b', { scopeKey: 'new' }))).toBeDefined()
    expect(qc.getQueryData(fnfKeys.profileSnapshot({ scopeKey: 'old' }))).toBeDefined()
  })
})

describe('prependGenerations — explicit optimistic insert', () => {
  it('inserts fresh submits at the head of the named feed, deduplicated', () => {
    const qc = new QueryClient()
    qc.setQueryData(fnfKeys.jobs({ type: 'video' }), feedData([[gen('old', 'completed')]]))

    prependGenerations(qc, { type: 'video' }, [gen('new', 'queued'), gen('old', 'completed')])

    const items = qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({ type: 'video' }))?.pages[0].items
    expect(items?.map(g => g.id)).toEqual(['new', 'old'])
  })

  it('a feed that was never fetched is left alone — its first fetch includes the new jobs anyway', () => {
    const qc = new QueryClient()

    prependGenerations(qc, { type: 'video' }, [gen('new', 'queued')])

    expect(qc.getQueryData(fnfKeys.jobs({ type: 'video' }))).toBeUndefined()
  })

  it('targets ONLY the named feed — fan-out across feeds is the app\'s explicit policy', () => {
    const qc = new QueryClient()
    qc.setQueryData(fnfKeys.jobs({ type: 'video' }), feedData([[]]))
    qc.setQueryData(fnfKeys.jobs({}), feedData([[]]))

    prependGenerations(qc, { type: 'video' }, [gen('new', 'queued')])

    expect(qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({ type: 'video' }))?.pages[0].items).toHaveLength(1)
    expect(qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({}))?.pages[0].items).toHaveLength(0)
  })

  it('can optimistically insert into a scoped feed', () => {
    const qc = new QueryClient()
    qc.setQueryData(fnfKeys.jobs({ type: 'video' }, { scopeKey: 'u:w' }), feedData([[]]))

    prependGenerations(qc, { type: 'video' }, [gen('new', 'queued')], { scopeKey: 'u:w' })

    expect(qc.getQueryData<InfiniteData<ListResult>>(fnfKeys.jobs({ type: 'video' }, { scopeKey: 'u:w' }))?.pages[0].items).toHaveLength(1)
    expect(qc.getQueryData(fnfKeys.jobs({ type: 'video' }))).toBeUndefined()
  })
})
```

app/packages/fnf-react/src/__tests__/generation-fold.test.ts

```
import type { Generation } from '@higgsfield/fnf/client'
import { describe, expect, it } from 'vitest'
import { foldGeneration } from '../generation-fold'

function gen(id: string, status: Generation['status'], extra?: Partial<Generation>): Generation {
  return { id, model: 'demo', type: 'image', status, input: { model: 'demo', settings: {} }, ...extra }
}

describe('foldGeneration', () => {
  it('progress folds forward', () => {
    const prev = gen('a', 'queued')
    const next = gen('a', 'in_progress')
    expect(foldGeneration(prev, next)).toBe(next)
  })

  it('a stale snapshot can NOT reopen a settled generation (terminal anti-regress)', () => {
    const settled = gen('a', 'completed', { results: { rawUrl: 'https://x/a.png' } })
    const stale = gen('a', 'in_progress')
    expect(foldGeneration(settled, stale)).toBe(settled)
  })

  it('terminal → terminal still folds (a late fail reason, a fresher result url)', () => {
    const prev = gen('a', 'completed', { results: { rawUrl: 'https://x/a.png' } })
    const next = gen('a', 'completed', { results: { rawUrl: 'https://x/a.png', minUrl: 'https://x/a-min.png' } })
    expect(foldGeneration(prev, next)).toBe(next)
  })

  it('nothing observable changed → the PREVIOUS reference survives (memoization keeps working)', () => {
    const prev = gen('a', 'in_progress')
    const tick = gen('a', 'in_progress') // a poll tick re-parsed into a fresh object
    expect(foldGeneration(prev, tick)).toBe(prev)
  })

  it('no previous (or a different id) → the next snapshot wins', () => {
    const next = gen('a', 'queued')
    expect(foldGeneration(undefined, next)).toBe(next)
    expect(foldGeneration(gen('b', 'completed'), next)).toBe(next)
  })
})
```

app/packages/fnf-react/src/__tests__/generation-run.test.ts

```
import type { Generation } from '@higgsfield/fnf/client'
import type { FnfObservationEvent } from '@higgsfield/fnf/observability'
import { createJobClient } from '@higgsfield/fnf/client'
import { ApiJobError } from '@higgsfield/fnf/errors'
import { defineJob, z } from '@higgsfield/fnf/jobs'
import { describe, expect, it } from 'vitest'
import { GenerationRun } from '../generation-run'
import { createMemoryBackend } from './test-utils'

const demo = defineJob({
  jobSetType: 'demo',
  outputType: 'image',
  params: { prompt: true, settings: { aspectRatio: z.aspectRatio(['1:1', '16:9']) } },
})

function statuses<I>(run: GenerationRun<I>) {
  const seen: string[] = []
  run.subscribe(() => {
    if (seen[seen.length - 1] !== run.status)
      seen.push(run.status)
  })
  return seen
}

describe('GenerationRun', () => {
  it('walks idle → submitting → generating → completed against the memory backend', async () => {
    const client = createJobClient({ adapter: createMemoryBackend(), jobs: [demo] })
    const run = new GenerationRun(client)
    const seen = statuses(run)

    const done = await run.start({ model: 'demo', prompt: { instruction: 'x' }, settings: { aspectRatio: '1:1' } })

    expect(run.status).toBe('completed')
    expect(done.length).toBeGreaterThan(0)
    expect(run.generations).toEqual(done)
    expect(seen).toEqual(['submitting', 'generating', 'completed'])
  })

  it('a failed submit becomes state, not a rejection', async () => {
    const client = createJobClient({ adapter: createMemoryBackend(), jobs: [demo] })
    const run = new GenerationRun(client)

    const done = await run.start({ model: 'unknown_model' } as never)

    expect(done).toEqual([])
    expect(run.status).toBe('failed')
    expect(run.error?.code).toBe('unknown_model')
  })

  it('a rejected confirmation becomes an aborted run, not an error', async () => {
    const run = new GenerationRun({
      submit: async () => { throw new ApiJobError('confirmation_rejected', 'Generation cancelled') },
      wait: async generations => generations,
    })

    const done = await run.start({} as never)

    expect(done).toEqual([])
    expect(run.status).toBe('aborted')
    expect(run.error).toBeUndefined()
  })

  it('abort mid-poll lands on status aborted (no error)', async () => {
    let release: (() => void) | undefined
    const pending: Generation = { id: 'g1', model: 'demo', type: 'image', status: 'queued', input: { model: 'demo', settings: {} } }
    const run = new GenerationRun({
      submit: async () => ({ generations: [pending] }),
      wait: (_gens, opts) => new Promise((_resolve, reject) => {
```

```
      // mimic the real client: abort surfaces as the typed JobAbortedError
      release = () => reject(new ApiJobError('aborted', 'Operation aborted'))
      opts?.signal?.addEventListener('abort', () => release?.())
    }),
  })

  const started = run.start({} as never)
  await Promise.resolve() // let submit settle
  await Promise.resolve()
  run.abort()
  await started

  expect(run.status).toBe('aborted')
  expect(run.error).toBeUndefined()
})

it('starting again supersedes the previous run (late results are dropped)', async () => {
  let resolveFirst: ((gens: Generation[]) => void) | undefined
  const first: Generation = { id: 'old', model: 'demo', type: 'image', status: 'queued', input: { model: 'demo', settings: {} } }
  const second: Generation = { id: 'new', model: 'demo', type: 'image', status: 'completed', input: { model: 'demo', settings: {} } }
  let calls = 0
  const run = new GenerationRun({
    submit: async () => ({ generations: calls++ === 0 ? [first] : [second] }),
    wait: async (gens) => {
      if (gens[0].id === 'old')
        return new Promise(resolve => (resolveFirst = resolve))
      return gens
    },
  })

  const firstStart = run.start({} as never)
  await Promise.resolve()
  await Promise.resolve()
  const secondStart = run.start({} as never)
  resolveFirst?.([{ ...first, status: 'completed' }]) // the superseded run settles late
  await Promise.all([firstStart, secondStart])

  expect(run.generations.map(g => g.id)).toEqual(['new']) // old run never touched state again
  expect(run.status).toBe('completed')
})

it('abort during the SUBMIT phase still lands on aborted (no stuck submitting)', async () => {
  let releaseSubmit: (() => void) | undefined
  const run = new GenerationRun({
    submit: () => new Promise((resolve) => {
      releaseSubmit = () => resolve({ generations: [] })
    }),
    wait: async gens => gens,
  })

  const started = run.start({} as never)
  run.abort() // mid-submit — before any generation exists
  releaseSubmit?.()
  await started

  expect(run.status).toBe('aborted')
  expect(run.isRunning).toBe(false) // a Generate button must un-brick
})

it('abort during submit keeps a handle on the generations the submit created', async () => {
  const created: Generation = { id: 'g1', model: 'demo', type: 'image', status: 'queued', input: { model: 'demo', settings: {} } }
  let releaseSubmit: (() => void) | undefined
  const run = new GenerationRun({
    submit: () => new Promise((resolve) => {
      releaseSubmit = () => resolve({ generations: [created], warning: 'partial' })
    }),
    wait: async gens => gens,
  })

  const started = run.start({} as never)
```

```
    run.abort()
    releaseSubmit?.() // the submit DID happen on the backend
    await started

    expect(run.status).toBe('aborted')
    // the UI can still render them and call client.cancel(id)
    expect(run.generations.map(g => g.id)).toEqual(['g1'])
    expect(run.warning).toBe('partial')
  })

  it('reset returns to idle and clears state', async () => {
    const client = createJobClient({ adapter: createMemoryBackend(), jobs: [demo] })
    const run = new GenerationRun(client)
    await run.start({ model: 'demo', prompt: { instruction: 'x' }, settings: { aspectRatio: '1:1' } })

    run.reset()

    expect(run.status).toBe('idle')
    expect(run.generations).toEqual([])
    expect(run.error).toBeUndefined()
  })

  it('emits safe observability events for lifecycle transitions', async () => {
    const events: FnfObservationEvent[] = []
    const client = createJobClient({ adapter: createMemoryBackend(), jobs: [demo] })
    const run = new GenerationRun(client, {
      observability: {
        observer: (event) => {
          events.push(event)
        },
      },
    })

    await run.start({ model: 'demo', prompt: { instruction: 'private prompt' }, settings: { aspectRatio: '1:1' } })
    run.reset()

    expect(events.map(event => event.name)).toEqual(expect.arrayContaining([
      'fnf.react.generation_run.start',
      'fnf.react.generation_run.submitted',
      'fnf.react.generation_run.progress',
      'fnf.react.generation_run.completed',
      'fnf.react.generation_run.reset',
    ]))
    expect(JSON.stringify(events)).not.toContain('private prompt')
  })
})
```

app/packages/fnf-react/src/__tests__/live-feed-generations.test.ts

```
import type { Generation } from '@higgsfield/fnf/client'
import { describe, expect, it } from 'vitest'
import { liveFeedPollTargets } from '../live-feed-generations-hook'

function gen(id: string, status: Generation['status']): Generation {
  return { id, model: 'demo', type: 'image', status, input: { model: 'demo', settings: {} } }
}

describe('liveFeedPollTargets', () => {
  it('keeps one snapshot per non-terminal id and skips terminal generations', () => {
    const targets = liveFeedPollTargets([
      gen('a', 'queued'),
      gen('b', 'completed'),
      gen('a', 'in_progress'),
      gen('c', 'failed'),
    ])

    expect(targets).toEqual([gen('a', 'in_progress')])
  })

  it('does not reopen a terminal duplicate from a stale pending snapshot', () => {
    expect(liveFeedPollTargets([
      gen('a', 'completed'),
      gen('a', 'in_progress'),
    ])).toEqual([])
  })

  it('bounds restored polling and advances through the feed as earlier targets settle', () => {
    const restored = Array.from({ length: 25 }, (_, index) => gen(`job-${index}`, 'queued'))

    expect(liveFeedPollTargets(restored).map(generation => generation.id))
      .toEqual(Array.from({ length: 20 }, (_, index) => `job-${index}`))

    const afterHeadSettles = [gen('job-0', 'completed'), ...restored.slice(1)]
    expect(liveFeedPollTargets(afterHeadSettles).map(generation => generation.id))
      .toEqual(Array.from({ length: 20 }, (_, index) => `job-${index + 1}`))
  })
})
```

app/packages/fnf-react/src/__tests__/profile-query.test.ts

```
import { createProfileClient } from '@higgsfield/fnf/profile'
import { QueryClient } from '@tanstack/react-query'
import { describe, expect, it, vi } from 'vitest'
import { fnfKeys } from '../keys'
import {
  profileCreditsQueryOptions,
  profileCurrentWorkspaceQueryOptions,
  profileSnapshotQueryOptions,
  profileUserQueryOptions,
  profileWalletQueryOptions,
  profileWorkspacesQueryOptions,
  setProfileSnapshot,
} from '../profile-query'
import { switchWorkspaceMutationOptions } from '../workspace-switch'
import { createMemoryProfileAdapter, gen } from './test-utils'

describe('profile query options', () => {
  it('fetches profile pieces through the SDK profile client into scoped keys', async () => {
    const client = createProfileClient({
      profileAdapter: createMemoryProfileAdapter({
        user: { id: 'u1', workspace_id: 'w1', workspace_type: 'private', workspace_role: 'owner' },
        workspaces: [{ id: 'w1', name: 'Personal', type: 'private', user_role: 'owner' }],
        wallet: { subscription_balance: 500, total_credits: 1000, credits_balance: 200 },
      }),
    })
    const qc = new QueryClient()
    const scope = { scopeKey: 'u1:w1' }

    await expect(qc.fetchQuery(profileUserQueryOptions(client, scope))).resolves.toMatchObject({ id: 'u1', workspaceId: 'w1' })
    await expect(qc.fetchQuery(profileWorkspacesQueryOptions(client, scope))).resolves.toHaveLength(1)
    await expect(qc.fetchQuery(profileCurrentWorkspaceQueryOptions(client, scope))).resolves.toMatchObject({ id: 'w1' })
    await expect(qc.fetchQuery(profileWalletQueryOptions(client, scope))).resolves.toMatchObject({ subscriptionBalance: 500 })
    await expect(qc.fetchQuery(profileCreditsQueryOptions(client, scope))).resolves.toMatchObject({ totalAvailableCredits: 7 })
    await expect(qc.fetchQuery(profileSnapshotQueryOptions(client, scope))).resolves.toMatchObject({ user: { id: 'u1' }, wallet: { id: 'w1' } })

    expect(qc.getQueryData(fnfKeys.profileUser(scope))).toMatchObject({ id: 'u1' })
    expect(qc.getQueryData(fnfKeys.profileUser())).toBeUndefined()
    expect(fnfKeys.profileCredits({ ...scope, includeOnDemand: false })).toEqual(['fnf', 'scope', 'u1:w1', 'profile', 'credits', { includeOnDemand: false }])
    expect(fnfKeys.profileSnapshot({ ...scope, includeOnDemand: false })).toEqual(['fnf', 'scope', 'u1:w1', 'profile', 'snapshot', { includeOnDemand: false }])
  })

  it('writes a composed snapshot into every profile cache leaf', () => {
    const qc = new QueryClient()
    const snapshot = {
      user: { id: 'u1' },
      workspaces: [{ id: 'w1' }],
      currentWorkspace: { id: 'w1' },
      wallet: { id: 'w1' },
      credits: { totalAvailableCredits: 12 },
    } as never

    setProfileSnapshot(qc, snapshot, { scopeKey: 'u1:w1' })

    expect(qc.getQueryData(fnfKeys.profileSnapshot({ scopeKey: 'u1:w1' }))).toBe(snapshot)
    expect(qc.getQueryData(fnfKeys.profileUser({ scopeKey: 'u1:w1' }))).toEqual({ id: 'u1' })
    expect(qc.getQueryData(fnfKeys.profileWorkspaces({ scopeKey: 'u1:w1' }))).toEqual([{ id: 'w1' }])
    expect(qc.getQueryData(fnfKeys.profileCurrentWorkspace({ scopeKey: 'u1:w1' }))).toEqual({ id: 'w1' })
    expect(qc.getQueryData(fnfKeys.profileWallet({ scopeKey: 'u1:w1' }))).toEqual({ id: 'w1' })
    expect(qc.getQueryData(fnfKeys.profileCredits({ scopeKey: 'u1:w1' }))).toEqual({ totalAvailableCredits: 12 })
  })
})

describe('workspace switch mutation options', () => {
  it('switches context, clears old scoped generation caches, writes next profile snapshot, and notifies the host', async () => {
    const qc = new QueryClient()
    qc.setQueryData(fnfKeys.job('old', { scopeKey: 'u:w1' }), gen('old', 'queued'))
    qc.setQueryData(fnfKeys.jobs({}, { scopeKey: 'u:w1' }), { pages: [{ items: [gen('old', 'queued')] }], pageParams: [undefined] })
    qc.setQueryData(fnfKeys.realtimeCustomStyles({}, { scopeKey: 'u:w1' }), { items: [{ id: 'old-style' }] })
```

```
    qc.setQueryData(fnfKeys.job('new', { scopeKey: 'u:w2' }), gen('new', 'queued'))
    qc.setQueryData(fnfKeys.realtimeCustomStyles({}, { scopeKey: 'u:w2' }), { items: [{ id: 'new-style' }] })
    const onWorkspaceChanged = vi.fn()
    const snapshot = {
      user: { id: 'u', workspaceId: 'w2' },
      workspaces: [],
      currentWorkspace: { id: 'w2' },
      wallet: null,
      credits: null,
    } as never
    const client = {
      switchWorkspace: vi.fn(async () => snapshot),
    }
    const mutation = switchWorkspaceMutationOptions(client, qc, {
      scopeKey: 'u:w1',
      nextScopeKey: snap => `u:${snap.currentWorkspace?.id ?? 'none'}`,
      onWorkspaceChanged,
    })

    const result = await mutation.mutationFn({ workspaceId: 'w2' })
    await mutation.onSuccess(result)

    expect(client.switchWorkspace).toHaveBeenCalledWith({ workspaceId: 'w2' })
    expect(qc.getQueryData(fnfKeys.job('old', { scopeKey: 'u:w1' }))).toBeUndefined()
    expect(qc.getQueryData(fnfKeys.jobs({}, { scopeKey: 'u:w1' }))).toBeUndefined()
    expect(qc.getQueryData(fnfKeys.realtimeCustomStyles({}, { scopeKey: 'u:w1' }))).toBeUndefined()
    expect(qc.getQueryData(fnfKeys.job('new', { scopeKey: 'u:w2' }))).toBeDefined()
    expect(qc.getQueryData(fnfKeys.realtimeCustomStyles({}, { scopeKey: 'u:w2' }))).toBeDefined()
    expect(qc.getQueryData(fnfKeys.profileSnapshot({ scopeKey: 'u:w2' }))).toBe(snapshot)
    expect(onWorkspaceChanged).toHaveBeenCalledWith(snapshot)
  })
})
```

app/packages/fnf-react/src/__tests__/provider-wire.test.ts

```
import type { CharacterBackend, GenerationInput, RealtimeBackend, ReferenceBackend } from '@higgsfield/fnf'
import type { FnfObservationEvent } from '@higgsfield/fnf/observability'
import { defineJob, z } from '@higgsfield/fnf/jobs'
import { QueryClient } from '@tanstack/react-query'
import { describe, expect, it, vi } from 'vitest'
import { costQueryOptions } from '../cost-query'
import { fnfKeys } from '../keys'
import { createFnfReactClients } from '../provider'
import { getWirePreview } from '../wire-preview'
import { createMemoryBackend, createMemoryMediaAdapter, createMemoryProfileAdapter } from './test-utils'

const demo = defineJob({
  jobSetType: 'demo',
  outputType: 'image',
  params: {
    prompt: true,
    settings: {
      aspectRatio: z.wire('aspect_ratio', z.aspectRatio(['1:1', '16:9'])),
    },
  },
})

describe('FnfProvider and client creation', () => {
  it('creates stable SDK clients from explicit adapters', async () => {
    const characterAdapter: CharacterBackend = {
      createCharacter: vi.fn(),
      getCharacter: vi.fn(async () => ({ id: 'character-1', type: 'soul_2', status: 'completed' })),
    }
    const referenceAdapter: ReferenceBackend = {
      getReferenceElement: vi.fn(async () => ({
        id: 'element-1',
        name: 'Ada',
        category: 'character',
        status: 'completed',
        medias: [],
        video_medias: [],
        character_id: 'character-1',
      })),
      listReferenceElements: vi.fn(async () => ({ items: [] })),
    }
    const clients = createFnfReactClients({
      adapter: createMemoryBackend(),
      characterAdapter,
      referenceAdapter,
      mediaAdapter: createMemoryMediaAdapter(),
      profileAdapter: createMemoryProfileAdapter({
        user: { id: 'u1', workspace_id: 'w1' },
        workspaces: [{ id: 'w1', name: 'Personal', type: 'private', user_role: 'owner' }],
        currentWorkspaceId: 'w1',
      }),
      jobs: [demo],
      scopeKey: 'u1:w1',
    })

    expect(clients.scopeKey).toBe('u1:w1')
    expect(clients.jobs).toEqual([demo])
    expect(clients.realtimeClient).toBeUndefined()
    await expect(clients.characterClient.get('character-1')).resolves.toMatchObject({ id: 'character-1' })
    await expect(clients.referenceClient.get('element-1')).resolves.toMatchObject({ characterId: 'character-1' })
    await expect(clients.profileClient.getUser()).resolves.toMatchObject({ id: 'u1', workspaceId: 'w1' })
  })

  it('creates the optional realtime client from an explicit adapter', async () => {
    const jobId = '44444444-4444-4444-8444-444444444444'
    const jobSetId = '33333333-3333-4333-8333-333333333333'
    const adapter = createMemoryBackend()
    vi.spyOn(adapter, 'getJob').mockResolvedValue({
      id: jobId,
```

```
      job_set_id: jobSetId,
      job_set_type: 'flux_klein_edit',
      status: 'completed',
      result_url: 'https://cdn.example/edit.png',
    })
    const realtimeAdapter: RealtimeBackend = {
      editRealtimeChain: vi.fn(),
      estimateRealtimeChainCost: vi.fn(async () => ({ credits: 1, credits_exact: 0.5 })),
      finalizeRealtimeChain: vi.fn(),
      listRealtimeCustomStyles: vi.fn(async () => ({ items: [], next_cursor: null })),
      createRealtimeCustomStyle: vi.fn(),
      updateRealtimeCustomStyle: vi.fn(),
      deleteRealtimeCustomStyle: vi.fn(),
    }
    const clients = createFnfReactClients({ adapter, realtimeAdapter, jobs: [demo] })

    await expect(clients.realtimeClient?.estimateChainCost({
      resolution: '1k',
      aspectRatio: '1:1',
    })).resolves.toEqual({ credits: 1, creditsExact: 0.5 })
    await expect(clients.realtimeClient?.getEditJob(jobId)).resolves.toMatchObject({
      id: jobId,
      model: 'flux_klein_edit',
      results: { rawUrl: 'https://cdn.example/edit.png' },
    })
  })

  it('passes shared observability into created clients', async () => {
    const events: FnfObservationEvent[] = []
    const clients = createFnfReactClients({
      adapter: createMemoryBackend({ cost: 5 }),
      mediaAdapter: createMemoryMediaAdapter(),
      profileAdapter: createMemoryProfileAdapter(),
      jobs: [demo],
      observability: {
        observer: (event) => {
          events.push(event)
        },
        traceId: 'react-trace',
      },
    })

    await clients.jobClient.cost({ model: 'demo', prompt: { instruction: 'private' }, settings: { aspectRatio: '1:1' } })
    await clients.profileClient.getUser()

    expect(clients.observability?.traceId).toBe('react-trace')
    expect(events.map(event => event.name)).toEqual(expect.arrayContaining(['fnf.job.cost', 'fnf.profile.get_user']))
    expect(events.every(event => event.traceId === 'react-trace')).toBe(true)
    expect(JSON.stringify(events)).not.toContain('private')
  })
})

describe('request helpers', () => {
  it('builds a local wire preview or returns a typed SDK error', () => {
    const input: GenerationInput = { model: 'demo', prompt: { instruction: 'hello' }, settings: { aspectRatio: '16:9' } }
    const preview = getWirePreview(input, [demo])

    expect(preview.ok).toBe(true)
    if (preview.ok) {
      expect(preview.jobSetType).toBe('demo')
      expect(preview.params).toMatchObject({ prompt: 'hello', aspect_ratio: '16:9' })
    }

    const bad = getWirePreview({ model: 'missing', settings: {} } as never, [demo])
    expect(bad.ok).toBe(false)
    if (!bad.ok)
      expect(bad.error.code).toBe('unknown_model')
  })

  it('caches cost estimates under scoped request keys', async () => {
```

```
    const input = { model: 'demo', prompt: { instruction: 'hello' }, settings: { aspectRatio: '1:1' } } as const
    const client = {
      cost: vi.fn(async () => ({ credits: 3 })),
    }
    const qc = new QueryClient()

    await expect(qc.fetchQuery(costQueryOptions(client, input, { scopeKey: 'u1:w1' }))).resolves.toEqual({ credits: 3 })

    expect(client.cost).toHaveBeenCalledWith(input)
    expect(qc.getQueryData(fnfKeys.cost(input, { scopeKey: 'u1:w1' }))).toEqual({ credits: 3 })
    expect(qc.getQueryData(fnfKeys.cost(input))).toBeUndefined()
  })
})
```

app/packages/fnf-react/src/__tests__/queries.test.ts

```
import type { Generation, ListOptions, ListResult } from '@higgsfield/fnf/client'
import { QueryClient } from '@tanstack/react-query'
import { describe, expect, it } from 'vitest'
import { generationQueryOptions } from '../generation-query'
import { jobSetQueryOptions } from '../job-set-query'
import { flattenFeedPages, jobsFeedQueryOptions } from '../jobs-feed-query'
import { fnfKeys } from '../keys'

function gen(id: string, status: Generation['status'], extra?: Partial<Generation>): Generation {
  return { id, model: 'demo', type: 'image', status, input: { model: 'demo', settings: {} }, ...extra }
}

/** Evaluate a refetchInterval/staleTime function the way the query core does. */
function evalWith<T>(fn: unknown, data: T): number | false | undefined {
  return (fn as (query: { state: { data?: T } }) => number | false | undefined)({ state: { data } })
}

describe('generationQueryOptions', () => {
  it('keeps old unscoped key shapes and adds scoped variants additively', () => {
    expect(fnfKeys.job('g1')).toEqual(['fnf', 'job', 'g1'])
    expect(fnfKeys.jobSet('set-1')).toEqual(['fnf', 'job-set', 'set-1'])
    expect(fnfKeys.jobs({ type: 'video' })).toEqual(['fnf', 'jobs', { type: 'video' }])
    expect(fnfKeys.job('g1', { scopeKey: 'u:w' })).toEqual(['fnf', 'scope', 'u:w', 'job', 'g1'])
    expect(fnfKeys.jobSet('set-1', { scopeKey: 'u:w' })).toEqual(['fnf', 'scope', 'u:w', 'job-set', 'set-1'])
    expect(fnfKeys.jobs({ type: 'video' }, { scopeKey: 'u:w' })).toEqual(['fnf', 'scope', 'u:w', 'jobs', { type: 'video' }])
  })

  it('fetches through the client into the contract key', async () => {
    const qc = new QueryClient()
    const data = await qc.fetchQuery(generationQueryOptions({ get: async id => gen(id, 'completed') }, 'g1'))
    expect(data.id).toBe('g1')
    expect(qc.getQueryData(fnfKeys.job('g1'))).toBe(data)
  })

  it('polls at the 5s product cadence while pending, stops at terminal', () => {
    const opts = generationQueryOptions({ get: async id => gen(id, 'queued') }, 'g1')
    expect(evalWith(opts.refetchInterval, undefined)).toBe(5000) // nothing known yet — keep asking
    expect(evalWith(opts.refetchInterval, gen('g1', 'in_progress'))).toBe(5000)
    expect(evalWith(opts.refetchInterval, gen('g1', 'completed'))).toBe(false)
    expect(evalWith(opts.staleTime, gen('g1', 'completed'))).toBe(Number.POSITIVE_INFINITY) // settled = immutable
    expect(evalWith(opts.staleTime, gen('g1', 'queued'))).toBe(0)
  })

  it('the cadence is a default, not a law: intervalMs overrides, false disables', () => {
    const client = { get: async (id: string) => gen(id, 'queued') }
    const custom = generationQueryOptions(client, 'g1', { intervalMs: 1000 })
    expect(evalWith(custom.refetchInterval, gen('g1', 'queued'))).toBe(1000)
    expect(evalWith(custom.refetchInterval, gen('g1', 'completed'))).toBe(false) // stop-at-terminal survives the override
    const off = generationQueryOptions(client, 'g1', { intervalMs: false })
    expect(evalWith(off.refetchInterval, gen('g1', 'queued'))).toBe(false)
  })

  it('uses scoped keys when provided', async () => {
    const qc = new QueryClient()
    const data = await qc.fetchQuery(generationQueryOptions({ get: async id => gen(id, 'completed') }, 'g1', { scopeKey: 'u:w' }))
    expect(qc.getQueryData(fnfKeys.job('g1'))).toBeUndefined()
    expect(qc.getQueryData(fnfKeys.job('g1', { scopeKey: 'u:w' }))).toBe(data)
  })
})

describe('jobSetQueryOptions', () => {
  it('one request per tick for the whole batch; stops when EVERY member settles', async () => {
    let reads = 0
    const client = {
      getSet: async (jobSetId: string) => {
        reads++
        return [gen('a', 'completed', { jobSetId }), gen('b', 'in_progress', { jobSetId })]
      },
```

```
    }
    const qc = new QueryClient()
    const opts = jobSetQueryOptions(client, 'set-1')
    const members = await qc.fetchQuery(opts)

    expect(reads).toBe(1)
    expect(members.map(g => g.id)).toEqual(['a', 'b'])
    expect(evalWith(opts.refetchInterval, members)).toBe(5000) // b is still running
    expect(evalWith(opts.refetchInterval, [gen('a', 'completed'), gen('b', 'failed')])).toBe(false)
  })
})

describe('jobsFeedQueryOptions', () => {
  function listClient(pages: Record<string, ListResult>) {
    const calls: Array<ListOptions | undefined> = []
    return {
      calls,
      list: async (opts?: ListOptions) => {
        calls.push(opts)
        return pages[String(opts?.cursor)]
      },
    }
  }

  it('walks the cursor: the query is the identity, the cursor is the page param (verbatim to list)', async () => {
    const client = listClient({
      undefined: { items: [gen('a', 'completed')], cursor: 'c1' },
      c1: { items: [gen('b', 'completed')] },
    })
    const qc = new QueryClient()
    const data = await qc.fetchInfiniteQuery({ ...jobsFeedQueryOptions(client, { type: 'video', size: 2 }), pages: 2 })

    expect(client.calls[0]).toEqual({ type: 'video', size: 2 })
    expect(client.calls[1]).toEqual({ type: 'video', size: 2, cursor: 'c1' })
    expect(data.pages.map(p => p.items[0].id)).toEqual(['a', 'b'])
    expect(qc.getQueryData(fnfKeys.jobs({ type: 'video', size: 2 }))).toBe(data)
  })

  it('the last page (no cursor) ends the walk', async () => {
    const client = listClient({ undefined: { items: [gen('a', 'completed')] } })
    const qc = new QueryClient()
    const data = await qc.fetchInfiniteQuery({ ...jobsFeedQueryOptions(client), pages: 5 })
    expect(data.pages).toHaveLength(1) // no next cursor — stop, whatever `pages` asked
  })

  it('keeps scoped feeds separate from unscoped feeds', async () => {
    const client = listClient({ undefined: { items: [gen('a', 'completed')] } })
    const qc = new QueryClient()
    const data = await qc.fetchInfiniteQuery({ ...jobsFeedQueryOptions(client, { type: 'image' }, { scopeKey: 'u:w' }), pages: 1 })

    expect(qc.getQueryData(fnfKeys.jobs({ type: 'image' }))).toBeUndefined()
    expect(qc.getQueryData(fnfKeys.jobs({ type: 'image' }, { scopeKey: 'u:w' }))).toBe(data)
  })
})

describe('flattenFeedPages', () => {
  it('deduplicates across pages: FIRST position, LATEST-fetched value', () => {
    const flat = flattenFeedPages({
      pages: [
        { items: [gen('a', 'queued'), gen('b', 'completed')] },
        { items: [gen('a', 'completed'), gen('c', 'completed')] }, // 'a' slid onto page 2, fetched later
      ],
      pageParams: [undefined, 'c1'],
    })
    expect(flat.map(g => g.id)).toEqual(['a', 'b', 'c']) // position from the first sighting
    expect(flat[0].status).toBe('completed') // value from the latest fetch
  })
})
```

app/packages/fnf-react/src/__tests__/realtime-query.test.ts

```
import { QueryClient } from '@tanstack/react-query'
import { describe, expect, it, vi } from 'vitest'
import { fnfKeys } from '../keys'
import { realtimeChainCostQueryOptions, realtimeCustomStylesQueryOptions } from '../realtime-query'

describe('realtime query options', () => {
  it('keeps chain-cost caches isolated by user/workspace scope and input', async () => {
    const input = { resolution: '2k' as const, aspectRatio: '16:9' as const }
    const client = {
      estimateChainCost: vi.fn(async () => ({ credits: 1, creditsExact: 0.75 })),
    }
    const queryClient = new QueryClient()

    await expect(queryClient.fetchQuery(realtimeChainCostQueryOptions(client, input, {
      scopeKey: 'u:w',
    }))).resolves.toEqual({ credits: 1, creditsExact: 0.75 })

    expect(fnfKeys.realtimeChainCost(input)).toEqual(['fnf', 'realtime', 'chain-cost', input])
    expect(fnfKeys.realtimeChainCost(input, { scopeKey: 'u:w' })).toEqual([
      'fnf',
      'scope',
      'u:w',
      'realtime',
      'chain-cost',
      input,
    ])
    expect(queryClient.getQueryData(fnfKeys.realtimeChainCost(input, { scopeKey: 'u:w' }))).toEqual({
      credits: 1,
      creditsExact: 0.75,
    })
    expect(queryClient.getQueryData(fnfKeys.realtimeChainCost(input))).toBeUndefined()
  })

  it('caches a custom-style cursor page under its exact query', async () => {
    const page = { items: [], nextCursor: 123.5 }
    const client = { listCustomStyles: vi.fn(async () => page) }
    const query = { cursor: 120, size: 20 }
    const queryClient = new QueryClient()

    await expect(queryClient.fetchQuery(realtimeCustomStylesQueryOptions(client, query))).resolves.toEqual(page)
    expect(client.listCustomStyles).toHaveBeenCalledWith(query)
    expect(queryClient.getQueryData(fnfKeys.realtimeCustomStyles(query))).toEqual(page)
  })
})
```

app/packages/fnf-react/src/__tests__/realtime.test.ts

```
import type { Generation } from '@higgsfield/fnf/client'
import { afterEach, beforeEach, describe, expect, it, vi } from 'vitest'
import { RefCountPool } from '../pool'
import { Realtime } from '../realtime'

function gen(id: string, jobSetId?: string): Generation {
  return { id, model: 'demo', type: 'image', status: 'queued', input: { model: 'demo', settings: {} }, ...(jobSetId ? { jobSetId } : {}) }
}

describe('Realtime (refcounted channels)', () => {
  beforeEach(() => vi.useFakeTimers())
  afterEach(() => vi.useRealTimers())

  it('shares ONE connection per job set across subscribers; the last unsubscribe closes it', () => {
    let opened = 0
    let closed = 0
    const live = new Realtime(() => {
      opened++
      return () => closed++
    }, { freeGraceMs: 0 })

    const a = live.subscribe(gen('a', 'set-1'), () => {})
    const b = live.subscribe(gen('b', 'set-1'), () => {})
    expect(opened).toBe(1) // one channel for the whole set

    a?.()
    expect(closed).toBe(0) // b still listening
    b?.()
    expect(closed).toBe(1)
  })

  it('events fan out to every listener on the channel', () => {
    let fire: (() => void) | undefined
    const live = new Realtime((_g, emit) => {
      fire = emit
      return () => {}
    })
    const seen: string[] = []
    live.subscribe(gen('a', 'set-1'), () => seen.push('a'))
    live.subscribe(gen('b', 'set-1'), () => seen.push('b'))

    fire?.()

    expect(seen.sort()).toEqual(['a', 'b'])
  })

  it('the grace window survives quick re-subscribes without reconnecting', () => {
    let opened = 0
    let closed = 0
    const live = new Realtime(() => {
      opened++
      return () => closed++
    }, { freeGraceMs: 1000 })

    const first = live.subscribe(gen('a', 'set-1'), () => {})
    first?.() // effect re-run: unsubscribe...
    live.subscribe(gen('a', 'set-1'), () => {}) // ...and resubscribe within the grace

    vi.advanceTimersByTime(2000)
    expect(opened).toBe(1) // the live connection was reused
    expect(closed).toBe(0)

    // and the grace DOES close an abandoned channel
    const second = live.subscribe(gen('z', 'set-2'), () => {})
    second?.()
    vi.advanceTimersByTime(2000)
    expect(closed).toBe(1)
  })
```

```
  it('returns undefined when the transport has no channel (poll those instead)', () => {
    const live = new Realtime(g => (g.jobSetId === 'covered' ? () => {} : undefined))

    expect(live.subscribe(gen('a', 'covered'), () => {})).toBeDefined()
    expect(live.subscribe(gen('b', 'legacy-type'), () => {})).toBeUndefined()
  })

  it('a THROWING transport means "no channel", not a crash in the caller', () => {
    const live = new Realtime(() => {
      throw new Error('new WebSocket(...) blew up')
    }, { freeGraceMs: 0 })

    expect(live.subscribe(gen('a', 'set-1'), () => {})).toBeUndefined()
    // and the failed open didn't leak a refcount on the channel key
    expect(live.subscribe(gen('a', 'set-1'), () => {})).toBeUndefined()
  })

  it('unsubscribe is idempotent — extra calls cannot close a channel under another subscriber', () => {
    let closed = 0
    let fire: (() => void) | undefined
    const live = new Realtime((_g, emit) => {
      fire = emit
      return () => closed++
    }, { freeGraceMs: 0 })

    const seen: string[] = []
    const a = live.subscribe(gen('a', 'set-1'), () => seen.push('a'))
    live.subscribe(gen('b', 'set-1'), () => seen.push('b'))

    a?.()
    a?.() // a double cleanup (a buggy consumer) must be a no-op
    fire?.()

    expect(closed).toBe(0) // b's channel survived
    expect(seen).toEqual(['b'])
  })
})

describe('RefCountPool', () => {
  it('allocate shares an instance; the last free disposes it', () => {
    const freed: string[] = []
    const pool = new RefCountPool(
      (id: string) => id,
      id => ({ id, free: () => freed.push(id) }),
    )

    const one = pool.allocate('x')
    const two = pool.allocate('x')
    expect(two).toBe(one)

    pool.free('x')
    expect(freed).toEqual([])
    pool.free('x')
    expect(freed).toEqual(['x'])
    expect(pool.has('x')).toBe(false)
  })

  it('ensure gets-or-creates without taking a reference (render-phase safe)', () => {
    const freed: string[] = []
    const pool = new RefCountPool(
      (id: string) => id,
      id => ({ id, free: () => freed.push(id) }),
    )

    const ensured = pool.ensure('x')
    expect(pool.allocate('x')).toBe(ensured)
    pool.free('x') // releases the ONE allocate reference
    expect(freed).toEqual(['x'])
  })
```

```
  it('extra frees are a no-op — the count never underflows a live instance', () => {
    const freed: string[] = []
    const pool = new RefCountPool(
      (id: string) => id,
      id => ({ id, free: () => freed.push(id) }),
    )

    pool.free('never-allocated') // must not throw, must not dispose anything
    expect(freed).toEqual([])

    pool.allocate('x')
    pool.free('x')
    pool.free('x') // double free after full release
    expect(freed).toEqual(['x'])

    // the key is fully reusable afterwards
    pool.allocate('x')
    expect(pool.has('x')).toBe(true)
    pool.free('x')
    expect(freed).toEqual(['x', 'x'])
  })

  it('a factory throw does not leak a reference', () => {
    const freed: string[] = []
    let boom = true
    const pool = new RefCountPool(
      (id: string) => id,
      (id) => {
        if (boom) {
          boom = false
          throw new Error('factory blew up')
        }
        return { id, free: () => freed.push(id) }
      },
    )

    expect(() => pool.allocate('x')).toThrow('factory blew up')

    pool.allocate('x') // works now — and holds the ONLY reference
    pool.free('x') // would NOT dispose here if the throw above had leaked a ref
    expect(freed).toEqual(['x'])
  })
})
```

app/packages/fnf-react/src/__tests__/reference-query.test.ts

```
import type { Reference } from '@higgsfield/fnf/references'
import { QueryClient } from '@tanstack/react-query'
import { describe, expect, it, vi } from 'vitest'
import { referenceQueryOptions, referencesQueryOptions } from '../reference-query'
import { fnfKeys } from '../keys'

function evaluate<T>(fn: unknown, data: T): number | false | undefined {
  return (fn as (query: { state: { data?: T } }) => number | false | undefined)({ state: { data } })
}

const processing: Reference = {
  id: '11111111-1111-4111-8111-111111111111',
  name: 'Ada',
  category: 'character',
  status: 'processing',
  description: null,
  medias: [],
  videoMedias: [],
}

describe('reference query options', () => {
  it('uses scoped keys and stops polling at a terminal status', async () => {
    const client = { get: vi.fn(async () => processing) }
    const options = referenceQueryOptions(client, processing.id, { scopeKey: 'u:w' })
    const queryClient = new QueryClient()

    await expect(queryClient.fetchQuery(options)).resolves.toEqual(processing)
    expect(queryClient.getQueryData(fnfKeys.reference(processing.id, { scopeKey: 'u:w' }))).toEqual(processing)
    expect(evaluate(options.refetchInterval, processing)).toBe(5000)
    expect(evaluate(options.refetchInterval, { ...processing, status: 'completed' as const })).toBe(false)
  })

  it('caches the authenticated user reference library', async () => {
    const page = { items: [processing] }
    const client = { list: vi.fn(async () => page) }
    const queryClient = new QueryClient()

    await expect(queryClient.fetchQuery(referencesQueryOptions(client, { size: 20 }))).resolves.toEqual(page)
    expect(queryClient.getQueryData(fnfKeys.references({ size: 20 }))).toEqual(page)
  })
})
```

app/packages/fnf-react/src/__tests__/test-utils.ts

```
import type { GenerationBackend, MediaBackend, ProfileBackend, SwitchWorkspaceRequest } from '@higgsfield/fnf'
import type { Generation } from '@higgsfield/fnf/client'
import { REF_TYPE_BY_UPLOAD } from '@higgsfield/fnf/media'

export function gen(id: string, status: Generation['status'], extra?: Partial<Generation>): Generation {
  return { id, model: 'demo', type: 'image', status, input: { model: 'demo', settings: {} }, ...extra }
}

// In-memory port stubs, inlined from @higgsfield/fnf-adapters' memory adapters.
// fnf-react tests must depend only on @higgsfield/fnf so the vendored snapshot
// (template app/packages/) stays self-sufficient without the adapters package.

export function createMemoryBackend(options: { cost?: number } = {}): GenerationBackend {
  const jobs = new Map<string, Record<string, unknown>>()
  const cost = options.cost ?? 1
  let seq = 0

  return {
    async createJobs({ jobSetType, params }) {
      const id = `mem_${++seq}`
      jobs.set(id, {
        id,
        job_set_type: jobSetType,
        status: 'completed',
        params,
        result_url: `memory://${jobSetType}/${id}.out`,
        created_at: seq,
      })
      return [id]
    },
    async getJob(id) {
      return jobs.get(id) ?? { id, status: 'failed', fail_reason: 'not found' }
    },
    async listJobs() {
      return { items: [...jobs.values()], next_cursor: null }
    },
    async estimateCost() {
      return { credits: cost }
    },
  }
}

export function createMemoryMediaAdapter(): MediaBackend {
  let seq = 0
  return {
    async getMedia({ id }) {
      return { id, type: 'media_input' }
    },
    async listMedia() {
      return { items: [], next_cursor: null }
    },
    async getUploadUrl({ type }) {
      const id = `mem_media_${++seq}`
      return { id, type: REF_TYPE_BY_UPLOAD[type], url: `memory://${type}/${id}`, upload_url: `memory://upload/${id}` }
    },
    async confirmMedia({ mediaId, type }) {
      return { id: mediaId, status: 'uploaded', type: REF_TYPE_BY_UPLOAD[type], url: `memory://${type}/${mediaId}` }
    },
  }
}

export interface MemoryProfileAdapterOptions {
  user?: Record<string, unknown>
  workspaces?: Record<string, unknown>[]
  wallet?: Record<string, unknown>
  currentWorkspaceId?: string
}
```

```
const DEFAULT_USER = {
  id: 'mem_user',
  email: 'memory@example.com',
  workspace_id: 'mem_private',
  workspace_type: 'private',
  workspace_role: 'owner',
  workspace_membership_exists: true,
  plan_type: 'free',
  credits: 0,
  billing_period: 'monthly',
  total_plan_credits: 0,
  package_credits: 0,
  subscription_credits: 0,
}

const DEFAULT_WORKSPACES = [
  {
    id: 'mem_private',
    clerk_organization_id: '',
    name: 'Personal',
    type: 'private',
    user_role: 'owner',
  },
]

const DEFAULT_WALLET = {
  workspace_id: 'mem_private',
  subscription_balance: 0,
  total_credits: 0,
  credits_balance: 0,
  on_demand_credits: 0,
  wallet_created_at: null,
  next_credit_allocation_date: null,
}

export function createMemoryProfileAdapter(options: MemoryProfileAdapterOptions = {}): ProfileBackend {
  const user = { ...DEFAULT_USER, ...options.user }
  const workspaces = options.workspaces?.map(item => ({ ...item })) ?? DEFAULT_WORKSPACES.map(item => ({ ...item }))
  const wallet = { ...DEFAULT_WALLET, ...options.wallet }
  let currentWorkspaceId = options.currentWorkspaceId
    ?? (typeof user.workspace_id === 'string' ? user.workspace_id : undefined)
    ?? (typeof workspaces[0]?.id === 'string' ? workspaces[0].id : 'mem_private')

  const currentWorkspace = () => workspaces.find(workspace => workspace.id === currentWorkspaceId) ?? workspaces[0] ?? null

  return {
    async getUser() {
      const workspace = currentWorkspace()
      return {
        ...user,
        ...(workspace
          ? {
              workspace_id: workspace.id,
              workspace_type: workspace.type,
              workspace_role: workspace.user_role,
            }
          : {}),
      }
    },
    async listWorkspaces() {
      return workspaces
    },
    async getCurrentWorkspace() {
      return currentWorkspace()
    },
    async getWorkspaceWallet() {
      return {
        ...wallet,
        workspace_id: currentWorkspaceId,
      }
```

```
    },
    async switchWorkspace({ workspaceId }: SwitchWorkspaceRequest) {
      currentWorkspaceId = workspaceId
      return {}
    },
  }
}
```

app/packages/fnf-react/src/attachments-hook.ts

```
'use client'

import type { AttachmentsMediaClient, AttachmentsOptions } from './attachments'
import { useEffect, useState } from 'react'
import { AttachmentsController } from './attachments'
import { useStore } from './external-store-hook'
import { useOptionalFnfObservability } from './provider'

/**
 * An attachments presenter bound to the component: previews render
 * immediately, uploads run in the background, `refs`/`settled()` feed the
 * submit. Object URLs are revoked on unmount. For an attachments list that
 * must outlive the component (route changes, pools), construct
 * `AttachmentsController` yourself and bind it with `useStore`.
 *
 *   const attachments = useAttachments(media, { upload: { forceIpCheck: true } })
 *   <input type="file" onChange={e => attachments.add([...e.target.files ?? []])} />
 *   {attachments.items.map(a => <Thumb key={a.key} src={a.previewUrl} state={a.status} />)}
 *   <button onClick={async () => submit(await attachments.settled())} />
 */
export function useAttachments(media: AttachmentsMediaClient, opts?: AttachmentsOptions): AttachmentsController {
  const providerObservability = useOptionalFnfObservability()
  const observability = opts?.observability ?? providerObservability
  // useState, not useMemo: items live in the controller; a cache-discard
  // would wipe them. `media` and `opts` are read once — both must be stable.
  const [controller] = useState(() => new AttachmentsController(media, { ...opts, ...(observability ? { observability } : {}) }))
  useEffect(() => () => controller.dispose(), [controller])
  return useStore(controller)
}
```

app/packages/fnf-react/src/attachments.ts

```
import type { MediaRef, UploadInput, UploadModeration, UploadResult } from '@higgsfield/fnf/media'
import type { FnfObservabilityContext, FnfObservabilityOptions } from '@higgsfield/fnf/observability'
import { ApiJobError } from '@higgsfield/fnf/errors'
import { createDomMediaMetaResolver, isBlockedModeration } from '@higgsfield/fnf/media'
import { createObservabilityContext, observeEvent } from '@higgsfield/fnf/observability'
import { ExternalStore } from './external-store'

/** What the presenter needs from a media client — structural on purpose. */
export interface AttachmentsMediaClient {
  upload: (input: UploadInput) => Promise<UploadResult>
}

/**
 * Everything `media.upload` accepts except the byte source — forwarded
 * VERBATIM. The presenter only fills defaults (filename/contentType from the
 * File, `throwOnModeration: false` so verdicts become item state, a
 * per-item cancellation signal); any of them can be overridden here.
 */
export type AttachmentUploadOptions = Omit<UploadInput, 'source'>

export type AttachmentStatus = 'uploading' | 'ready' | 'blocked' | 'failed'

export interface Attachment {
  /** Stable local identity — survives the upload, safe as a React key. */
  readonly key: string
  /** Present for file-born attachments (absent when a ready ref was added). */
  readonly file?: File
  /** The media role this attachment was added for (drives the submit slot). */
  readonly role?: string
  /** The submit-ready ref — present once `status` is `ready` (or `blocked`). */
  readonly ref?: MediaRef
  /** Renderable immediately: a local object URL while uploading, the remote url after. */
  readonly previewUrl?: string
  readonly status: AttachmentStatus
  /** The typed upload failure when `status` is `failed`. */
  readonly error?: ApiJobError
  /** The moderation verdict when the backend blocked the upload. */
  readonly moderation?: UploadModeration
}

export interface AttachmentsOptions {
  /** Controller-wide upload options (per-`add` options override them). */
  upload?: AttachmentUploadOptions
  /**
   * Measure intrinsic size/duration into `MediaRef.meta` from the local file
   * (default true; a no-op outside a DOM). Meta drives the SDK's media rules
   * and the product-parity 'auto' aspect-ratio resolution.
   */
  measure?: boolean
  observability?: FnfObservabilityOptions
}

/**
 * The attachments presenter: files in → previews now, uploads in flight,
 * submit-ready `MediaRef`s out. The frontend counterpart of fnf-web's
 * `InputMediaController`, rebuilt on the SDK media client:
 *
 *   const refs = await attachments.settled()       // wait out in-flight uploads
 *   client.submit({ media: { image: refs }, ... })
 *
 * Per-item lifecycle: `uploading` → `ready` | `blocked` (moderation verdict,
 * kept visible instead of thrown) | `failed` (typed error, `retry`-able).
 * Removing an in-flight item aborts its upload; object URLs are revoked when
 * the remote url takes over and on remove/clear/dispose. No counts, types,
 * or roles are restricted here — the job declarations validate media; the
 * presenter only presents.
 */
export class AttachmentsController extends ExternalStore {
```

```
private _items: Attachment[] = []
private readonly inFlight = new Set<Promise<void>>()
private readonly localUrls = new Map<string, string>()
private readonly aborts = new Map<string, AbortController>()
private readonly uploadOpts = new Map<string, AttachmentUploadOptions | undefined>()
private readonly measure = createDomMediaMetaResolver()
private readonly observability: FnfObservabilityContext
private seq = 0

constructor(
  private readonly media: AttachmentsMediaClient,
  private readonly opts: AttachmentsOptions = {},
) {
  super()
  this.observability = createObservabilityContext(opts.observability)
}

get items(): Attachment[] {
  return this._items
}

/** Submit-ready refs (ready items only, in display order). */
get refs(): MediaRef[] {
  return this._items.flatMap(item => item.status === 'ready' && item.ref ? [item.ref] : [])
}

get isUploading(): boolean {
  return this._items.some(item => item.status === 'uploading')
}

/**
 * Add files (uploaded immediately, preview available at once) or
 * already-uploaded refs (ready as-is). `opts` are upload options for THESE
 * entries, merged over the controller-wide ones (`role` lives here too).
 * Returns the new items' keys.
 */
add(input: File | MediaRef | Array<File | MediaRef>, opts?: AttachmentUploadOptions): string[] {
  const entries = Array.isArray(input) ? input : [input]
  const keys: string[] = []
  observeEvent(this.observability, 'fnf.react.attachments.add', {
    item_count: entries.length,
    file_count: entries.filter(entry => entry instanceof File).length,
    ref_count: entries.filter(entry => !(entry instanceof File)).length,
    ...(opts?.role ?? this.opts.upload?.role ? { role: opts?.role ?? this.opts.upload?.role ?? null } : {}),
  })
  for (const entry of entries) {
    const key = `att-${++this.seq}`
    keys.push(key)
    if (entry instanceof File) {
      const previewUrl = this.createLocalUrl(key, entry)
      this._items = [...this._items, { key, file: entry, role: opts?.role ?? this.opts.upload?.role, previewUrl, status: 'uploading' }]
      this.uploadOpts.set(key, opts)
      this.track(this.upload(key, entry, opts))
    }
    else {
      this._items = [...this._items, { key, ref: entry, role: entry.role ?? opts?.role, previewUrl: entry.url, status: 'ready' }]
    }
  }
  this.commit()
  return keys
}

/** Re-upload a failed file-born attachment with its original options. */
retry(key: string): void {
  const item = this._items.find(i => i.key === key)
  if (!item || item.status !== 'failed' || !item.file)
    return
  observeEvent(this.observability, 'fnf.react.attachments.retry', { attachment_key: key })
  this.patch(key, { status: 'uploading', error: undefined })
  this.track(this.upload(key, item.file, this.uploadOpts.get(key)))
```

```
}

/** Remove the item; an in-flight upload is aborted. */
remove(key: string): void {
  const item = this._items.find(entry => entry.key === key)
  observeEvent(this.observability, 'fnf.react.attachments.remove', {
    attachment_key: key,
    ...(item ? { status: item.status } : {}),
  })
  this.aborts.get(key)?.abort()
  this.aborts.delete(key)
  this.uploadOpts.delete(key)
  this.revokeLocalUrl(key)
  this._items = this._items.filter(item => item.key !== key)
  this.commit()
}

/** Reorder (drag-and-drop): move the item at `from` to position `to`. */
move(from: number, to: number): void {
  if (from === to || from < 0 || to < 0 || from >= this._items.length || to >= this._items.length)
    return
  const next = [...this._items]
  const [moved] = next.splice(from, 1)
  next.splice(to, 0, moved)
  this._items = next
  this.commit()
}

clear(): void {
  observeEvent(this.observability, 'fnf.react.attachments.clear', { item_count: this._items.length })
  for (const controller of this.aborts.values()) controller.abort()
  this.aborts.clear()
  this.uploadOpts.clear()
  for (const key of [...this.localUrls.keys()]) this.revokeLocalUrl(key)
  this._items = []
  this.commit()
}

/** `clear` + drop in-flight bookkeeping — call when the owner unmounts. */
dispose(): void {
  this.clear()
  this.inFlight.clear()
}

/**
 * Wait for every in-flight upload to finish, then return the submit-ready
 * refs — the "user hit Generate while uploads are running" path. Failures
 * don't reject; they stay visible as `failed`/`blocked` items.
 */
async settled(): Promise<MediaRef[]> {
  while (this.inFlight.size > 0)
    await Promise.allSettled([...this.inFlight])
  observeEvent(this.observability, 'fnf.react.attachments.settled', {
    item_count: this._items.length,
    ready_count: this.refs.length,
  })
  return this.refs
}

private async upload(key: string, file: File, opts?: AttachmentUploadOptions): Promise<void> {
  const abort = new AbortController()
  this.aborts.set(key, abort)

  // Measure from the local preview in parallel with the upload. The File's
  // mime drives the kind (a blob URL has no extension to sniff); the
  // resolver caches per url and resolves undefined outside a DOM. Best
  // effort by contract: a measurement failure must never fail the upload.
  const previewUrl = this.localUrls.get(key)
  const kind = file.type.startsWith('video/') ? 'video' : file.type.startsWith('audio/') ? 'audio' : 'image'
  observeEvent(this.observability, 'fnf.react.attachments.upload_start', {
```

```
    attachment_key: key,
    media_type: kind,
    ...(opts?.role ?? this.opts.upload?.role ? { role: opts?.role ?? this.opts.upload?.role ?? null } : {}),
  })
  const measured = this.opts.measure === false || !previewUrl
    ? Promise.resolve(undefined)
    : this.measure({ id: key, type: kind, url: previewUrl }).catch(() => undefined)

  try {
    const result = await this.media.upload({
      source: file,
      filename: file.name,
      contentType: file.type || undefined,
      throwOnModeration: false, // verdicts become item state, not throws
      signal: abort.signal,
      ...this.opts.upload,
      ...opts,
    })
    const meta = await measured
    const ref: MediaRef = { ...result.ref, ...(meta ? { meta } : {}) }
    if (result.moderation && isBlockedModeration(result.moderation.status)) {
      // keep the local preview — a blocked upload has no usable remote url
      observeEvent(this.observability, 'fnf.react.attachments.blocked', {
        attachment_key: key,
        media_id: result.mediaId,
        moderation_status: result.moderation.status,
      })
      this.patch(key, { ref, status: 'blocked', moderation: result.moderation })
      return
    }
    if (ref.url)
      this.revokeLocalUrl(key)
    observeEvent(this.observability, 'fnf.react.attachments.ready', {
      attachment_key: key,
      media_id: result.mediaId,
      media_type: result.type,
      status: result.status,
    })
    this.patch(key, { ref, status: 'ready', previewUrl: ref.url ?? this.localUrls.get(key), moderation: result.moderation })
  }
  catch (err) {
    const error = err instanceof ApiJobError ? err : new ApiJobError('unexpected', err instanceof Error ? err.message : String(err))
    observeEvent(this.observability, 'fnf.react.attachments.failed', {
      attachment_key: key,
      error_code: error.code,
      ...(error.status !== undefined ? { error_status: error.status } : {}),
    })
    this.patch(key, { status: 'failed', error })
  }
  finally {
    if (this.aborts.get(key) === abort)
      this.aborts.delete(key)
  }
}

private track(promise: Promise<void>): void {
  this.inFlight.add(promise)
  void promise.finally(() => this.inFlight.delete(promise))
}

private patch(key: string, changes: Partial<Attachment>): void {
  const at = this._items.findIndex(item => item.key === key)
  if (at < 0)
    return // removed while uploading — drop the result
  this._items = [...this._items.slice(0, at), { ...this._items[at], ...changes }, ...this._items.slice(at + 1)]
  this.commit()
}

private createLocalUrl(key: string, file: File): string | undefined {
  if (typeof URL === 'undefined' || typeof URL.createObjectURL !== 'function')
```

```
      return undefined
    const url = URL.createObjectURL(file)
    this.localUrls.set(key, url)
    return url
  }

  private revokeLocalUrl(key: string): void {
    const url = this.localUrls.get(key)
    if (url === undefined)
      return
    this.localUrls.delete(key)
    URL.revokeObjectURL(url)
  }
}
```

app/packages/fnf-react/src/character-query.ts

```
import type { Character, CharacterClient } from '@higgsfield/fnf/characters'
import type { FnfScopeOptions } from './keys'
import { queryOptions } from '@tanstack/react-query'
import { DEFAULT_POLL_INTERVAL_MS } from './generation-query'
import { fnfKeys } from './keys'

export type CharacterQueryClient = Pick<CharacterClient, 'get'>

export interface CharacterQueryOptions extends FnfScopeOptions {
  intervalMs?: number | false
}

function isTerminal(character: Character | undefined): boolean {
  return character?.status === 'completed' || character?.status === 'failed'
}

export function characterQueryOptions(
  client: CharacterQueryClient,
  id: string,
  opts?: CharacterQueryOptions,
) {
  return queryOptions({
    queryKey: fnfKeys.character(id, opts),
    queryFn: () => client.get(id),
    refetchInterval: (query) => {
      if (opts?.intervalMs === false)
        return false
      return isTerminal(query.state.data)
        ? false
        : opts?.intervalMs ?? DEFAULT_POLL_INTERVAL_MS
    },
    refetchIntervalInBackground: true,
    staleTime: query => isTerminal(query.state.data) ? Number.POSITIVE_INFINITY : 0,
  })
}
```

app/packages/fnf-react/src/cost-query.ts

```
import type { CostEstimate, GenerationInput } from '@higgsfield/fnf/client'
import type { FnfScopeOptions } from './keys'
import { queryOptions } from '@tanstack/react-query'
import { fnfKeys } from './keys'

export interface CostQueryClient<Input = GenerationInput> {
  cost: (input: Input) => Promise<CostEstimate>
}

export interface CostQueryOptions extends FnfScopeOptions {
  enabled?: boolean
}

export function costQueryOptions<Input>(client: CostQueryClient<Input>, input: Input, opts?: CostQueryOptions) {
  return queryOptions({
    queryKey: fnfKeys.cost(input, opts),
    queryFn: () => client.cost(input),
    enabled: opts?.enabled,
  })
}
```

app/packages/fnf-react/src/external-store-hook.ts

```
'use client'

import type { ExternalStore } from './external-store'
import { useSyncExternalStore } from 'react'

/**
 * Bind any controller from this package (or your own `ExternalStore`
 * subclass) to a component: re-renders on every `commit()`, returns the
 * controller itself. This is the escape hatch the `use*` hooks are built on —
 * construct a controller yourself (module scope, context, a pool) and bind it
 * wherever it's read.
 */
export function useStore<T extends ExternalStore>(store: T): T {
  useSyncExternalStore(store.subscribe, store.snapshot, store.snapshot)
  return store
}
```

app/packages/fnf-react/src/external-store.ts

```
/**
 * The minimal external-store base every controller in this package extends —
 * the same shape as fnf-web's `SyncExternalStorage` (the InputMediaController
 * pattern): the controller itself is the stable object React holds, and the
 * snapshot is a version counter that bumps on every `commit()`. Reads go
 * through the controller's getters, so renders never chase object identity.
 *
 * Framework-agnostic on purpose: controllers are fully usable (and testable)
 * without React; `useStore` is the one-line binding.
 */
export class ExternalStore {
  private version = 0
  private readonly listeners = new Set<() => void>()

  readonly snapshot = (): number => this.version

  readonly subscribe = (listener: () => void): (() => void) => {
    this.listeners.add(listener)
    return () => this.listeners.delete(listener)
  }

  protected commit(): void {
    this.version++
    for (const listener of this.listeners) listener()
  }
}
```

app/packages/fnf-react/src/generation-cache.ts

```
import type { Generation, ListResult } from '@higgsfield/fnf/client'
import type { InfiniteData, QueryClient } from '@tanstack/react-query'
import type { FnfScopeOptions, JobsQuery } from './keys'
import { foldGeneration } from './generation-fold'
import { fnfKeys } from './keys'

/**
 * The single write door over the QueryClient: fold fresh generation snapshots
 * into EVERY fnf-owned cache entry that holds them — per-job entries, job-set
 * entries, and all feed page lists — in one call. Poll ticks, realtime
 * re-reads and run progress all enter here, so no source can race another
 * past `foldGeneration`'s guards.
 *
 * The door UPDATES, it never seeds: ids no cache holds are ignored (feed
 * membership belongs to fetches and `prependGenerations`), and absent keys
 * are not created. Writes that change nothing bail out, keeping every
 * untouched reference stable.
 *
 * This is the lesson of fnf-web's `AssetCache` made structural: TanStack is a
 * non-normalizing document cache, so one entity lives as copies under many
 * keys — the difference is that here ALL the keys belong to the package and
 * all the surgery lives behind one tested function.
 */
export function applyGenerations(queryClient: QueryClient, generations: Generation[], opts?: FnfScopeOptions): void {
  if (generations.length === 0)
    return
  const byId = new Map(generations.map(g => [g.id, g]))

  for (const g of generations) {
    if (queryClient.getQueryState(fnfKeys.job(g.id, opts))) {
      queryClient.setQueryData<Generation>(fnfKeys.job(g.id, opts), (prev) => {
        const folded = foldGeneration(prev, g)
        return folded === prev ? undefined : folded // undefined bails out — no write, stable reference
      })
    }
    if (g.jobSetId !== undefined && queryClient.getQueryState(fnfKeys.jobSet(g.jobSetId, opts))) {
      queryClient.setQueryData<Generation[]>(fnfKeys.jobSet(g.jobSetId, opts), (prev) => {
        if (!prev)
          return undefined
        let changed = false
        const next = prev.map((member) => {
          const fresh = byId.get(member.id)
          if (!fresh)
            return member
          const folded = foldGeneration(member, fresh)
          if (folded !== member)
            changed = true
          return folded
        })
        return changed ? next : undefined
      })
    }
  }

  queryClient.setQueriesData<InfiniteData<ListResult>>({ queryKey: jobsRootKey(opts) }, (prev) => {
    if (!prev)
      return undefined
    let changed = false
    const pages = prev.pages.map((page) => {
      let pageChanged = false
      const items = page.items.map((item) => {
        const fresh = byId.get(item.id)
        if (!fresh)
          return item
        const folded = foldGeneration(item, fresh)
        if (folded !== item)
          pageChanged = true
        return folded
```

```
      })
      if (!pageChanged)
        return page
      changed = true
      return { ...page, items }
    })
    return changed ? { ...prev, pages } : undefined
  })
}

/**
 * Optimistic head insert after a submit — into the ONE feed the caller names.
 * Which feeds should show a fresh submit is product policy, so the fan-out
 * stays an explicit app decision (call it once per feed); ids the feed
 * already holds are skipped. A feed that was never fetched is left alone —
 * its first fetch will include the new jobs anyway.
 */
export function prependGenerations(queryClient: QueryClient, query: JobsQuery, generations: Generation[], opts?: FnfScopeOptions): void {
  queryClient.setQueryData<InfiniteData<ListResult>>(fnfKeys.jobs(query, opts), (prev) => {
    if (!prev || prev.pages.length === 0)
      return undefined
    const seen = new Set(prev.pages.flatMap(page => page.items.map(item => item.id)))
    const fresh = generations.filter(g => !seen.has(g.id))
    if (fresh.length === 0)
      return undefined
    const [head, ...rest] = prev.pages
    return { ...prev, pages: [{ ...head, items: [...fresh, ...head.items] }, ...rest] }
  })
}

export function removeGenerationQueries(queryClient: QueryClient, opts?: FnfScopeOptions): void {
  if (opts?.scopeKey) {
    queryClient.removeQueries({ queryKey: [...fnfKeys.scope(opts.scopeKey), 'job'] })
    queryClient.removeQueries({ queryKey: [...fnfKeys.scope(opts.scopeKey), 'job-set'] })
    queryClient.removeQueries({ queryKey: fnfKeys.scopedJobsRoot(opts.scopeKey) })
    queryClient.removeQueries({ queryKey: [...fnfKeys.scope(opts.scopeKey), 'cost'] })
    return
  }
  queryClient.removeQueries({ queryKey: ['fnf', 'job'] })
  queryClient.removeQueries({ queryKey: ['fnf', 'job-set'] })
  queryClient.removeQueries({ queryKey: fnfKeys.jobsRoot })
  queryClient.removeQueries({ queryKey: ['fnf', 'cost'] })
}

function jobsRootKey(opts?: FnfScopeOptions) {
  return opts?.scopeKey ? fnfKeys.scopedJobsRoot(opts.scopeKey) : fnfKeys.jobsRoot
}
```

app/packages/fnf-react/src/generation-fold.ts

```
import type { Generation } from '@higgsfield/fnf/client'
import { isTerminalJobStatus } from '@higgsfield/fnf/client'

/**
 * The single write rule for folding a fresh generation snapshot over a cached
 * one — every cache write in this package (`applyGenerations`) goes through
 * it, so poll ticks, set reads and run progress all obey the same two domain
 * guards no matter which source raced which:
 *
 * - **terminal anti-regress**: a stale read (an out-of-order response, a
 *   lagging endpoint) can never roll a settled generation back to pending —
 *   the guard fnf-web hand-rolls as `effectiveStatus` in its job polling.
 * - **identity stability**: when nothing observable changed, the PREVIOUS
 *   object is returned, so reference-equality memoization downstream
 *   (`React.memo` tiles, selectors) survives polling.
 *
 * Pure and tanstack-free on purpose — usable by any cache, not just ours.
 */
export function foldGeneration(prev: Generation | undefined, next: Generation): Generation {
  if (!prev || prev.id !== next.id)
    return next
  if (isTerminalJobStatus(prev.status) && !isTerminalJobStatus(next.status))
    return prev // a stale snapshot can't reopen a settled generation
  if (
    prev.status === next.status
    && prev.results?.rawUrl === next.results?.rawUrl
    && prev.results?.minUrl === next.results?.minUrl
    && prev.failReason === next.failReason
  ) {
    return prev // nothing observable changed — keep the reference stable
  }
  return next
}
```

app/packages/fnf-react/src/generation-query.ts

```
import type { Generation } from '@higgsfield/fnf/client'
import type { FnfScopeOptions } from './keys'
import { isTerminalJobStatus } from '@higgsfield/fnf/client'
import { queryOptions } from '@tanstack/react-query'
import { fnfKeys } from './keys'

/** What the generation query needs from a client — structural on purpose. */
export interface GenerationQueryClient {
  get: (id: string) => Promise<Generation>
}

/**
 * The product's job-polling cadence (fnf-web `use-job-status-polling`:
 * react-query `refetchInterval: 5000`, in background too).
 */
export const DEFAULT_POLL_INTERVAL_MS = 5000

export interface LiveQueryOptions extends FnfScopeOptions {
  /**
   * Poll cadence while non-terminal (default `DEFAULT_POLL_INTERVAL_MS`);
   * `false` turns polling off without losing the stop-at-terminal rule for
   * anything you keep. For full control, spread your own `refetchInterval`
   * over the returned options instead.
   */
  intervalMs?: number | false
}

/**
 * One generation, live until it settles: polls at `intervalMs` while the
 * status is non-terminal (in background too — uploads/long videos outlive a
 * focused tab), then stops and becomes immutable (`staleTime: Infinity` once
 * terminal). Every default is overridable by spreading over the result:
 *
 *   useQuery(generationQueryOptions(client, id))
 *   useSuspenseQuery({ ...generationQueryOptions(client, id), refetchInterval: false })
 */
export function generationQueryOptions(client: GenerationQueryClient, id: string, opts?: LiveQueryOptions) {
  return queryOptions({
    queryKey: fnfKeys.job(id, opts),
    queryFn: () => client.get(id),
    refetchInterval: (query) => {
      if (opts?.intervalMs === false)
        return false
      const generation = query.state.data
      return generation !== undefined && isTerminalJobStatus(generation.status)
        ? false
        : opts?.intervalMs ?? DEFAULT_POLL_INTERVAL_MS
    },
    refetchIntervalInBackground: true,
    staleTime: (query) => {
      const generation = query.state.data
      return generation !== undefined && isTerminalJobStatus(generation.status) ? Number.POSITIVE_INFINITY : 0
    },
  })
}
```

app/packages/fnf-react/src/generation-run-hook.ts

```
'use client'

import type { FnfObservabilityOptions } from '@higgsfield/fnf/observability'
import type { GenerationRunClient } from './generation-run'
import type { FnfScopeOptions } from './keys'
import { useQueryClient } from '@tanstack/react-query'
import { useEffect, useState } from 'react'
import { useStore } from './external-store-hook'
import { applyGenerations } from './generation-cache'
import { GenerationRun } from './generation-run'
import { useOptionalFnfObservability } from './provider'

export type GenerationRunHookOptions = FnfScopeOptions & {
  observability?: FnfObservabilityOptions
}

/**
 * A submit-to-terminal lifecycle bound to the component (requires a
 * `QueryClientProvider` above). Every run commit folds the live snapshots
 * into the shared query cache through `applyGenerations` — pending tiles
 * tick in any feed/job-set view that holds them, no extra wiring. Which feed
 * should SHOW a fresh submit stays your explicit call: `prependGenerations`.
 * Polling is aborted on unmount (the backend job keeps running — cancel
 * server-side via `client.cancel` if that's the intent).
 *
 * `client` must be referentially stable (module scope / context / useState) —
 * the controller binds to the first one; an inline-created client would be
 * silently ignored after mount.
 *
 *   const run = useGenerationRun(client, { scopeKey })
 *   useEffect(() => { // optimistic tiles: the submit landed, polling begins
 *     if (run.status === 'generating')
 *       prependGenerations(queryClient, { type: 'video' }, run.generations, { scopeKey })
 *   }, [run.status])
 *   <button onClick={() => run.start(input)} disabled={run.isRunning}>
 *   {run.error && <ErrorNote code={run.error.code} />}
 */
export function useGenerationRun<Input>(client: GenerationRunClient<Input>, opts?: GenerationRunHookOptions): GenerationRun<Input> {
  const providerObservability = useOptionalFnfObservability()
  const observability = opts?.observability ?? providerObservability
  // useState, not useMemo: a controller holds state, and useMemo is a cache
  // React may discard — recreation would silently wipe the run.
  const [run] = useState(() => new GenerationRun(client, { observability }))
  const queryClient = useQueryClient()
  useEffect(() => {
    const unsubscribe = run.subscribe(() => applyGenerations(queryClient, run.generations, opts))
    return () => {
      unsubscribe()
      run.abort()
    }
  }, [run, queryClient, opts?.scopeKey])
  return useStore(run)
}
```

app/packages/fnf-react/src/generation-run.ts

```
import type { Generation, SubmitResult, WaitOptions } from '@higgsfield/fnf/client'
import type { ApiJobErrorJSON } from '@higgsfield/fnf/errors'
import type { FnfObservabilityContext, FnfObservabilityOptions } from '@higgsfield/fnf/observability'
import { ApiJobError } from '@higgsfield/fnf/errors'
import { createObservabilityContext, observeEvent } from '@higgsfield/fnf/observability'
import { ExternalStore } from './external-store'

/**
 * The slice of a job client a run needs — structural, so the real
 * `JobClient`, a context-bound pair of free functions, or an app wrapper all
 * fit. `Input` is inferred from `submit`, so model autocomplete flows through.
 */
export interface GenerationRunClient<Input> {
  submit: (input: Input) => Promise<SubmitResult>
  wait: (generations: Generation[], opts?: WaitOptions) => Promise<Generation[]>
}

export type GenerationRunStatus = 'idle' | 'submitting' | 'generating' | 'completed' | 'failed' | 'aborted'

export interface GenerationRunOptions {
  observability?: FnfObservabilityOptions
}

/**
 * One submit-to-terminal lifecycle as observable state: submit the input,
 * then poll the batch live until every generation settles.
 *
 *   idle → submitting → generating → completed
 *                     ■ failed / aborted
 *
 * Errors are STATE, not throws (`start` never rejects — the safeSubmit
 * philosophy): read `error` for the run-level failure, `failed`/`warning`
 * for partial fan-out failures, and each generation's own `status` for
 * per-job verdicts (failed/nsfw/ip_detected resolve, they don't throw).
 * Starting again aborts the previous run.
 */
export class GenerationRun<Input> extends ExternalStore {
  private active: AbortController | null = null
  private _status: GenerationRunStatus = 'idle'
  private _generations: Generation[] = []
  private _failed: ApiJobErrorJSON[] = []
  private _warning: string | undefined
  private _error: ApiJobError | undefined
  private readonly observability: FnfObservabilityContext

  constructor(private readonly client: GenerationRunClient<Input>, opts: GenerationRunOptions = {}) {
    super()
    this.observability = createObservabilityContext(opts.observability)
  }

  get status(): GenerationRunStatus {
    return this._status
  }

  /** Live snapshots — updated on every poll tick while generating. */
  get generations(): Generation[] {
    return this._generations
  }

  /** Per-job errors from a `count > 1` fan-out where some jobs failed to submit. */
  get failed(): ApiJobErrorJSON[] {
    return this._failed
  }

  get warning(): string | undefined {
    return this._warning
  }
```

```
/** The run-level failure (submit rejected, polling died) — undefined otherwise. */
get error(): ApiJobError | undefined {
  return this._error
}

get isRunning(): boolean {
  return this._status === 'submitting' || this._status === 'generating'
}

/**
 * Submit and poll to terminal. Resolves with the settled generations —
 * `[]` when the submit itself failed (the failure is in `error`).
 */
async start(input: Input): Promise<Generation[]> {
  this.active?.abort()
  const run = new AbortController()
  this.active = run
  observeEvent(this.observability, 'fnf.react.generation_run.start', inputAttributes(input))
  this._status = 'submitting'
  this._generations = []
  this._failed = []
  this._warning = undefined
  this._error = undefined
  this.commit()

  try {
    const submitted = await this.client.submit(input)
    observeEvent(this.observability, 'fnf.react.generation_run.submitted', {
      generation_count: submitted.generations.length,
      failed_count: submitted.failed?.length ?? 0,
    })
    if (run.signal.aborted) {
      // Two cases share this signal: superseded by a newer start() (it owns
      // the state) and a plain abort() (nobody else will ever write — land
      // the machine ourselves or `submitting` sticks forever). The submit
      // DID happen: the created generations go into state too, so the UI
      // keeps a handle on them (to show, to `client.cancel`) even though
      // the documented hook pattern discards `start()`'s promise.
      if (run === this.active) {
        this._generations = submitted.generations
        this._failed = submitted.failed ?? []
        this._warning = submitted.warning
        this._status = 'aborted'
        observeEvent(this.observability, 'fnf.react.generation_run.aborted', {
          generation_count: submitted.generations.length,
        })
        this.commit()
      }
      return submitted.generations
    }
    this._generations = submitted.generations
    this._failed = submitted.failed ?? []
    this._warning = submitted.warning
    this._status = 'generating'
    this.commit()

    const done = await this.client.wait(submitted.generations, {
      signal: run.signal,
      onProgress: g => this.upsert(run, g),
    })
    if (run !== this.active)
      return done // superseded while settling — the newer run owns the state
    this._generations = done
    this._status = 'completed'
    observeEvent(this.observability, 'fnf.react.generation_run.completed', {
      generation_count: done.length,
    })
    this.commit()
    return done
  }
```

```
    catch (err) {
      if (run !== this.active)
        return [] // a newer start() owns the state now
      if (err instanceof ApiJobError && (err.code === 'aborted' || err.code === 'confirmation_rejected')) {
        this._status = 'aborted'
        observeEvent(this.observability, 'fnf.react.generation_run.aborted', {
          generation_count: this._generations.length,
        })
      }
      else {
        this._error = err instanceof ApiJobError ? err : new ApiJobError('unexpected', err instanceof Error ? err.message : String(err))
        this._status = 'failed'
        observeEvent(this.observability, 'fnf.react.generation_run.failed', {
          generation_count: this._generations.length,
          error_code: this._error.code,
          ...(this._error.status !== undefined ? { error_status: this._error.status } : {}),
        })
      }
      this.commit()
      return this._generations
    }
  }

  /** Stop polling (the backend job keeps running — see `client.cancel`). */
  abort(): void {
    this.active?.abort()
  }

  /** Abort if running and return to `idle` with empty state. */
  reset(): void {
    this.active?.abort()
    this.active = null
    this._status = 'idle'
    this._generations = []
    this._failed = []
    this._warning = undefined
    this._error = undefined
    observeEvent(this.observability, 'fnf.react.generation_run.reset')
    this.commit()
  }

  private upsert(run: AbortController, g: Generation): void {
    if (run !== this.active)
      return
    const at = this._generations.findIndex(existing => existing.id === g.id)
    this._generations = at >= 0
      ? [...this._generations.slice(0, at), g, ...this._generations.slice(at + 1)]
      : [...this._generations, g]
    observeEvent(this.observability, 'fnf.react.generation_run.progress', generationAttributes(g))
    this.commit()
  }
}

function inputAttributes(input: unknown): Record<string, string | number | boolean | null> {
  if (typeof input !== 'object' || input === null)
    return {}
  const record = input as { model?: unknown, count?: unknown }
  return {
    ...(typeof record.model === 'string' ? { model: record.model } : {}),
    ...(typeof record.count === 'number' ? { count: record.count } : {}),
  }
}

function generationAttributes(g: Generation): Record<string, string | number | boolean | null> {
  return {
    generation_id: g.id,
    model: g.model,
    type: g.type,
    status: g.status,
    ...(g.jobSetId ? { job_set_id: g.jobSetId } : {}),
```

```
  }
}
```

app/packages/fnf-react/src/index.ts

```
// Attachments — the input-media presenter: files in, submit-ready refs out
export { AttachmentsController } from './attachments'
export type { Attachment, AttachmentsMediaClient, AttachmentsOptions, AttachmentStatus, AttachmentUploadOptions } from './attachments'
export { useAttachments } from './attachments-hook'

export { characterQueryOptions } from './character-query'
export type { CharacterQueryClient, CharacterQueryOptions } from './character-query'
export { costQueryOptions } from './cost-query'
export type { CostQueryClient, CostQueryOptions } from './cost-query'
export { referenceQueryOptions, referencesQueryOptions } from './reference-query'
export type { ReferenceQueryClient, ReferenceQueryOptions, ReferencesQueryClient, ReferencesQueryOptions } from './reference-query'
export { realtimeChainCostQueryOptions, realtimeCustomStylesQueryOptions } from './realtime-query'
export type { RealtimeQueryClient, RealtimeQueryOptions } from './realtime-query'
// The base the controllers share — extend it for your own presenters
export { ExternalStore } from './external-store'
export { useStore } from './external-store-hook'
// Cache door — the ONE way generation snapshots enter the query cache
export { applyGenerations, prependGenerations, removeGenerationQueries } from './generation-cache'
export { foldGeneration } from './generation-fold'
// Query factories — pull-shaped reads as TanStack queryOptions
export { DEFAULT_POLL_INTERVAL_MS, generationQueryOptions } from './generation-query'

export type { GenerationQueryClient, LiveQueryOptions } from './generation-query'
// Feed bridge — resume polling jobs restored from history after a remount.
export { useLiveFeedGenerations } from './live-feed-generations-hook'
// Submit — one submit-to-terminal lifecycle as observable state
export { GenerationRun } from './generation-run'
export type { GenerationRunClient, GenerationRunOptions, GenerationRunStatus } from './generation-run'
export { useGenerationRun } from './generation-run-hook'
export type { GenerationRunHookOptions } from './generation-run-hook'
export { jobSetQueryOptions } from './job-set-query'
export type { JobSetQueryClient } from './job-set-query'
export { flattenFeedPages, jobsFeedQueryOptions } from './jobs-feed-query'
export type { JobsFeedQueryClient } from './jobs-feed-query'
// Keys — the public query-key contract (build keys ONLY through these)
export { fnfKeys } from './keys'

export type { FnfScopeOptions, JobsQuery } from './keys'
export { RefCountPool } from './pool'
export type { PoolEntry } from './pool'
// Profile queries — account/workspace/wallet reads as TanStack queryOptions
export {
  profileCreditsQueryOptions,
  profileCurrentWorkspaceQueryOptions,
  profileSnapshotQueryOptions,
  profileUserQueryOptions,
  profileWalletQueryOptions,
  profileWorkspacesQueryOptions,
  setProfileSnapshot,
} from './profile-query'

export type { ProfileCreditsQueryOptions, ProfileQueryClient, ProfileQueryOptions } from './profile-query'
// Provider — stable SDK clients from one React context
export { createFnfReactClients, FnfProvider, useFnf, useFnfCharacterClient, useFnfJobClient, useFnfJobs, useFnfLlmClient, useFnfMediaClient, useFnfObservability, useFnfProfileClient, useFnfRealtimeCli
export type { FnfProviderProps, FnfReactClients, FnfReactClientsConfig } from './provider'

export { Realtime } from './realtime'
export type { RealtimeTransport } from './realtime'
export { getWirePreview, useFnfWirePreview } from './wire-preview'
export type { WirePreviewResult } from './wire-preview'
export { switchWorkspaceMutationOptions, useSwitchWorkspaceMutation } from './workspace-switch'
export type { SwitchWorkspaceClient, SwitchWorkspaceMutationConfig } from './workspace-switch'
```

app/packages/fnf-react/src/job-set-query.ts

```
import type { Generation } from '@higgsfield/fnf/client'
import type { LiveQueryOptions } from './generation-query'
import { isTerminalJobStatus } from '@higgsfield/fnf/client'
import { queryOptions } from '@tanstack/react-query'
import { DEFAULT_POLL_INTERVAL_MS } from './generation-query'
import { fnfKeys } from './keys'

/** What the job-set query needs from a client — structural on purpose. */
export interface JobSetQueryClient {
  getSet: (jobSetId: string) => Promise<Generation[]>
}

/**
 * One job set (a batch), live until EVERY member settles — one request per
 * tick for the whole batch (`client.getSet`), the same economy `wait`'s
 * set-aware polling has, expressed as a query. Realtime glue is one line:
 * an event for the set is `queryClient.invalidateQueries({ queryKey:
 * fnfKeys.jobSet(id) })` — TanStack dedupes and cancels racing refetches.
 * Defaults are overridable by spreading over the result.
 */
export function jobSetQueryOptions(client: JobSetQueryClient, jobSetId: string, opts?: LiveQueryOptions) {
  return queryOptions({
    queryKey: fnfKeys.jobSet(jobSetId, opts),
    queryFn: () => client.getSet(jobSetId),
    refetchInterval: (query) => {
      if (opts?.intervalMs === false)
        return false
      const members = query.state.data
      return members !== undefined && members.every(g => isTerminalJobStatus(g.status))
        ? false
        : opts?.intervalMs ?? DEFAULT_POLL_INTERVAL_MS
    },
    refetchIntervalInBackground: true,
    staleTime: (query) => {
      const members = query.state.data
      return members !== undefined && members.every(g => isTerminalJobStatus(g.status)) ? Number.POSITIVE_INFINITY : 0
    },
  })
}
```

app/packages/fnf-react/src/jobs-feed-query.ts

```
import type { Generation, ListOptions, ListResult } from '@higgsfield/fnf/client'
import type { InfiniteData } from '@tanstack/react-query'
import type { FnfScopeOptions, JobsQuery } from './keys'
import { infiniteQueryOptions } from '@tanstack/react-query'
import { fnfKeys } from './keys'

/** What the feed query needs from a client — structural on purpose. */
export interface JobsFeedQueryClient {
  list: (opts?: ListOptions) => Promise<ListResult>
}

/**
 * A cursor feed as an infinite query. The `query` (filters/size, forwarded to
 * `client.list` VERBATIM) is the cache identity; the cursor is the page param.
 *
 * The feed is door-driven, so pages are fresh forever by default
 * (`staleTime: Infinity`): live updates fold in through `applyGenerations`,
 * fresh submits enter through `prependGenerations`, and the explicit hard
 * refresh is `queryClient.invalidateQueries({ queryKey: fnfKeys.jobs(query) })`.
 * Mount/focus refetches would race those writes — spread your own `staleTime`
 * over the result if you want them back.
 *
 *   const feed = useInfiniteQuery({
 *     ...jobsFeedQueryOptions(client, { type: 'video' }),
 *     select: flattenFeedPages,
 *   })
 *   feed.data?.map(g => <Tile key={g.id} generation={g} />)
 */
export function jobsFeedQueryOptions(client: JobsFeedQueryClient, query: JobsQuery = {}, opts?: FnfScopeOptions) {
  return infiniteQueryOptions({
    queryKey: fnfKeys.jobs(query, opts),
    queryFn: ({ pageParam }) => client.list({ ...query, ...(pageParam !== undefined ? { cursor: pageParam } : {}) }),
    initialPageParam: undefined as string | number | undefined,
    getNextPageParam: lastPage => lastPage.cursor ?? undefined,
    staleTime: Number.POSITIVE_INFINITY,
  })
}

/**
 * Pages → one deduplicated list (a `select` for the feed query). An id seen
 * on several pages keeps its FIRST position (head order — where the user
 * already saw it) with the LATEST-fetched value — the fnf-web `MapPool`
 * semantics.
 */
export function flattenFeedPages(data: InfiniteData<ListResult>): Generation[] {
  const seen = new Map<string, Generation>()
  for (const page of data.pages) {
    for (const item of page.items) seen.set(item.id, item)
  }
  return [...seen.values()]
}
```

app/packages/fnf-react/src/keys.ts

```
import type { ListOptions } from '@higgsfield/fnf/client'

/** The feed page query (filters/size) — the cache owns the cursor. */
export type JobsQuery = Omit<ListOptions, 'cursor'>

export interface FnfScopeOptions {
  scopeKey?: string
}

export interface FnfCreditsKeyOptions extends FnfScopeOptions {
  includeOnDemand?: boolean
}

function scoped(scopeKey: string, segment: string): readonly ['fnf', 'scope', string, string] {
  return ['fnf', 'scope', scopeKey, segment] as const
}

/**
 * The package's query-key namespace — a PUBLIC CONTRACT. Apps invalidate and
 * read by these keys, so their shape is versioned deliberately: always build
 * keys through these factories, never inline the literals. Object segments
 * hash order-independently (TanStack's `hashKey` sorts object keys), so two
 * `jobs({ type: 'video', size: 50 })` calls always hit the same entry.
 */
export const fnfKeys = {
  /** Every fnf-owned cache entry — invalidate this to drop them all. */
  root: ['fnf'] as const,
  /** Every fnf-owned cache entry for one user/workspace scope. */
  scope: (scopeKey: string) => ['fnf', 'scope', scopeKey] as const,
  /** One generation by job id (`client.get`). */
  job: (id: string, opts?: FnfScopeOptions) => opts?.scopeKey ? [...scoped(opts.scopeKey, 'job'), id] as const : ['fnf', 'job', id] as const,
  /** All members of one job set (`client.getSet`). */
  jobSet: (jobSetId: string, opts?: FnfScopeOptions) => opts?.scopeKey ? [...scoped(opts.scopeKey, 'job-set'), jobSetId] as const : ['fnf', 'job-set', jobSetId] as const,
  /** Every feed — the `setQueriesData` target `applyGenerations` patches. */
  jobsRoot: ['fnf', 'jobs'] as const,
  /** Every feed inside one user/workspace scope. */
  scopedJobsRoot: (scopeKey: string) => scoped(scopeKey, 'jobs'),
  /** One feed (a page list) for this query. */
  jobs: (query: JobsQuery = {}, opts?: FnfScopeOptions) => opts?.scopeKey ? [...scoped(opts.scopeKey, 'jobs'), query] as const : ['fnf', 'jobs', query] as const,
  cost: (input: unknown, opts?: FnfScopeOptions) => opts?.scopeKey ? [...scoped(opts.scopeKey, 'cost'), input] as const : ['fnf', 'cost', input] as const,
  character: (id: string, opts?: FnfScopeOptions) => opts?.scopeKey ? [...scoped(opts.scopeKey, 'character'), id] as const : ['fnf', 'character', id] as const,
  reference: (id: string, opts?: FnfScopeOptions) => opts?.scopeKey ? [...scoped(opts.scopeKey, 'reference'), id] as const : ['fnf', 'reference', id] as const,
  references: (query: unknown = {}, opts?: FnfScopeOptions) => opts?.scopeKey ? [...scoped(opts.scopeKey, 'references'), query] as const : ['fnf', 'references', query] as const,
  realtimeRoot: ['fnf', 'realtime'] as const,
  realtime: (opts?: FnfScopeOptions) => opts?.scopeKey ? scoped(opts.scopeKey, 'realtime') : ['fnf', 'realtime'] as const,
  realtimeChainCost: (input: unknown, opts?: FnfScopeOptions) => [...fnfKeys.realtime(opts), 'chain-cost', input] as const,
  realtimeCustomStyles: (query: unknown = {}, opts?: FnfScopeOptions) => [...fnfKeys.realtime(opts), 'custom-styles', query] as const,
  profileRoot: ['fnf', 'profile'] as const,
  profile: (opts?: FnfScopeOptions) => opts?.scopeKey ? scoped(opts.scopeKey, 'profile') : ['fnf', 'profile'] as const,
  profileSnapshot: (opts?: FnfCreditsKeyOptions) => opts?.includeOnDemand === undefined
    ? [...fnfKeys.profile(opts), 'snapshot'] as const
    : [...fnfKeys.profile(opts), 'snapshot', { includeOnDemand: opts.includeOnDemand }] as const,
  profileUser: (opts?: FnfScopeOptions) => [...fnfKeys.profile(opts), 'user'] as const,
  profileWorkspaces: (opts?: FnfScopeOptions) => [...fnfKeys.profile(opts), 'workspaces'] as const,
  profileCurrentWorkspace: (opts?: FnfScopeOptions) => [...fnfKeys.profile(opts), 'current-workspace'] as const,
  profileWallet: (opts?: FnfScopeOptions) => [...fnfKeys.profile(opts), 'wallet'] as const,
  profileCredits: (opts?: FnfCreditsKeyOptions) => opts?.includeOnDemand === undefined
    ? [...fnfKeys.profile(opts), 'credits'] as const
    : [...fnfKeys.profile(opts), 'credits', { includeOnDemand: opts.includeOnDemand }] as const,
}
```

app/packages/fnf-react/src/live-feed-generations-hook.ts

```
'use client'

import type { Generation } from '@higgsfield/fnf/client'
import type { GenerationQueryClient, LiveQueryOptions } from './generation-query'
import { isTerminalJobStatus } from '@higgsfield/fnf/client'
import { useQueries, useQueryClient } from '@tanstack/react-query'
import { useEffect, useMemo } from 'react'
import { applyGenerations } from './generation-cache'
import { foldGeneration } from './generation-fold'
import { generationQueryOptions } from './generation-query'

// Restored history can contain many stale non-terminal jobs. Keep observer,
// timer, and request fan-out bounded while still draining targets in feed order.
const MAX_LIVE_FEED_POLL_TARGETS = 20

/**
 * Keep non-terminal generations restored from a feed moving after remount or
 * refresh. Up to 20 unique ids at a time get the normal generation query
 * (seeded from the feed snapshot), and fresh reads re-enter every feed cache
 * through the shared cache door. Feed order is preserved, so later pending
 * items enter as earlier ones settle. Terminal items never allocate observers.
 */
export function useLiveFeedGenerations(
  client: GenerationQueryClient,
  generations: readonly Generation[],
  opts?: LiveQueryOptions,
): void {
  const queryClient = useQueryClient()
  const targets = useMemo(() => liveFeedPollTargets(generations), [generations])
  const snapshots = useQueries({
    queries: targets.map(generation => ({
      ...generationQueryOptions(client, generation.id, opts),
      initialData: generation,
    })),
    combine: combineSnapshots,
  })

  useEffect(() => {
    applyGenerations(
      queryClient,
      snapshots,
      opts?.scopeKey ? { scopeKey: opts.scopeKey } : undefined,
    )
  }, [queryClient, snapshots, opts?.scopeKey])
}

/** Exported from this module for focused tests; the package barrel exposes the hook only. */
export function liveFeedPollTargets(generations: readonly Generation[]): Generation[] {
  const unique = new Map<string, Generation>()
  for (const generation of generations)
    unique.set(generation.id, foldGeneration(unique.get(generation.id), generation))

  const targets: Generation[] = []
  for (const generation of unique.values()) {
    if (isTerminalJobStatus(generation.status))
      continue
    targets.push(generation)
    if (targets.length === MAX_LIVE_FEED_POLL_TARGETS)
      break
  }
  return targets
}

function combineSnapshots(results: Array<{ data?: Generation }>): Generation[] {
  return results.flatMap(result => result.data ? [result.data] : [])
}
```

app/packages/fnf-react/src/pool.ts

```
/**
 * Refcounted instance pool — the fnf-web `RefCountClassPool` pattern
 * (shared/lib/ref-count-class-pool.ts), dependency-free. Share one stateful
 * instance (a controller, a realtime channel) between consumers: `allocate`
 * gets-or-creates and increments, `free` decrements and disposes the instance
 * (calls its `free()`) when the last consumer leaves — after `freeGraceMs`,
 * so quick unmount/remount cycles reuse the live instance instead of
 * rebuilding it.
 */
export interface PoolEntry {
  free: () => void
}

export class RefCountPool<Id, Instance extends PoolEntry> {
  private readonly instances = new Map<string, Instance>()
  private readonly refs = new Map<string, number>()
  private readonly freeTimers = new Map<string, ReturnType<typeof setTimeout>>()
  private readonly freeGraceMs: number

  constructor(
    private readonly hash: (id: Id) => string,
    private readonly factory: (id: Id) => Instance,
    options?: { freeGraceMs?: number },
  ) {
    this.freeGraceMs = options?.freeGraceMs ?? 0
  }

  /** Get-or-create AND take a reference (pair every allocate with a free). */
  allocate(id: Id): Instance {
    const key = this.hash(id)
    const instance = this.ensure(id) // a factory throw must not leak a ref
    const pending = this.freeTimers.get(key)
    if (pending) {
      clearTimeout(pending)
      this.freeTimers.delete(key)
    }
    this.refs.set(key, (this.refs.get(key) ?? 0) + 1)
    return instance
  }

  /** Release a reference; the last one disposes the instance (after the grace). Extra frees are a no-op — never underflow a live instance. */
  free(id: Id): void {
    const key = this.hash(id)
    const current = this.refs.get(key)
    if (current === undefined)
      return // never allocated, or already fully freed (a double-free)
    if (current > 1) {
      this.refs.set(key, current - 1)
      return
    }
    this.refs.delete(key)
    if (this.freeGraceMs > 0) {
      this.freeTimers.set(key, setTimeout(() => {
        this.freeTimers.delete(key)
        this.dispose(key)
      }, this.freeGraceMs))
    }
    else {
      this.dispose(key)
    }
  }

  /** Get-or-create WITHOUT touching the ref count — safe in a render phase. */
  ensure(id: Id): Instance {
    const key = this.hash(id)
    const existing = this.instances.get(key)
    if (existing)
      return existing
```

```
    const created = this.factory(id)
    this.instances.set(key, created)
    return created
  }

  get(id: Id): Instance | undefined {
    return this.instances.get(this.hash(id))
  }

  has(id: Id): boolean {
    return this.instances.has(this.hash(id))
  }

  private dispose(key: string): void {
    this.instances.get(key)?.free()
    this.instances.delete(key)
  }
}
```

app/packages/fnf-react/src/profile-query.ts

```
import type { ProfileClient, ProfileCredits, ProfileCreditsOptions, ProfileSnapshot, ProfileUser, ProfileWorkspace, ProfileWorkspaceWallet } from '@higgsfield/fnf/profile'
import type { QueryClient } from '@tanstack/react-query'
import type { FnfScopeOptions } from './keys'
import { queryOptions } from '@tanstack/react-query'
import { fnfKeys } from './keys'

export interface ProfileQueryClient {
  getUser: () => Promise<ProfileUser | null>
  listWorkspaces: () => Promise<ProfileWorkspace[]>
  getCurrentWorkspace: () => Promise<ProfileWorkspace | null>
  getWallet: () => Promise<ProfileWorkspaceWallet | null>
  getCredits: (options?: ProfileCreditsOptions) => Promise<ProfileCredits | null>
  getSnapshot: (options?: ProfileCreditsOptions) => Promise<ProfileSnapshot>
}

export type ProfileQueryOptions = FnfScopeOptions
export type ProfileCreditsQueryOptions = ProfileCreditsOptions & FnfScopeOptions

export function profileSnapshotQueryOptions(client: ProfileQueryClient, opts?: ProfileCreditsQueryOptions) {
  return queryOptions({
    queryKey: fnfKeys.profileSnapshot(opts),
    queryFn: () => client.getSnapshot(opts),
  })
}

export function profileUserQueryOptions(client: Pick<ProfileQueryClient, 'getUser'>, opts?: ProfileQueryOptions) {
  return queryOptions({
    queryKey: fnfKeys.profileUser(opts),
    queryFn: () => client.getUser(),
  })
}

export function profileWorkspacesQueryOptions(client: Pick<ProfileQueryClient, 'listWorkspaces'>, opts?: ProfileQueryOptions) {
  return queryOptions({
    queryKey: fnfKeys.profileWorkspaces(opts),
    queryFn: () => client.listWorkspaces(),
  })
}

export function profileCurrentWorkspaceQueryOptions(client: Pick<ProfileQueryClient, 'getCurrentWorkspace'>, opts?: ProfileQueryOptions) {
  return queryOptions({
    queryKey: fnfKeys.profileCurrentWorkspace(opts),
    queryFn: () => client.getCurrentWorkspace(),
  })
}

export function profileWalletQueryOptions(client: Pick<ProfileQueryClient, 'getWallet'>, opts?: ProfileQueryOptions) {
  return queryOptions({
    queryKey: fnfKeys.profileWallet(opts),
    queryFn: () => client.getWallet(),
  })
}

export function profileCreditsQueryOptions(client: Pick<ProfileQueryClient, 'getCredits'>, opts?: ProfileCreditsQueryOptions) {
  return queryOptions({
    queryKey: fnfKeys.profileCredits(opts),
    queryFn: () => client.getCredits(opts),
  })
}

export function setProfileSnapshot(queryClient: QueryClient, snapshot: ProfileSnapshot, opts?: FnfScopeOptions): void {
  queryClient.setQueryData<ProfileSnapshot>(fnfKeys.profileSnapshot(opts), snapshot)
  queryClient.setQueryData<ProfileUser | null>(fnfKeys.profileUser(opts), snapshot.user)
  queryClient.setQueryData<ProfileWorkspace[]>(fnfKeys.profileWorkspaces(opts), snapshot.workspaces)
  queryClient.setQueryData<ProfileWorkspace | null>(fnfKeys.profileCurrentWorkspace(opts), snapshot.currentWorkspace)
  queryClient.setQueryData<ProfileWorkspaceWallet | null>(fnfKeys.profileWallet(opts), snapshot.wallet)
  queryClient.setQueryData<ProfileCredits | null>(fnfKeys.profileCredits(opts), snapshot.credits)
}
```

```
export type { ProfileClient }
```

app/packages/fnf-react/src/provider.ts

```
'use client'

import type {
  BinaryUploader,
  CharacterBackend,
  CharacterClient,
  FnfObservabilityContext,
  FnfObservabilityOptions,
  GenerationBackend,
  JobClient,
  JobEntry,
  LlmBackend,
  MediaBackend,
  MediaClient,
  ProfileBackend,
  ProfileClient,
  ReferenceBackend,
  ReferenceClient,
  RealtimeBackend,
  RealtimeClient,
  ResolveJobRef,
} from '@higgsfield/fnf'
import type { ReactNode } from 'react'
import { createCharacterClient, createJobClient, createMediaClient, createObservabilityContext, createProfileClient, createRealtimeClient, createReferenceClient } from '@higgsfield/fnf'
import { createContext, createElement, use, useMemo } from 'react'

export interface FnfReactClientsConfig<Jobs extends readonly JobEntry[] = readonly JobEntry[]> {
  adapter: GenerationBackend
  jobs: Jobs
  characterAdapter?: CharacterBackend
  referenceAdapter?: ReferenceBackend
  realtimeAdapter?: RealtimeBackend
  mediaAdapter?: MediaBackend
  profileAdapter?: ProfileBackend
  blobUploader?: BinaryUploader
  resolveJob?: ResolveJobRef
  scopeKey?: string
  observability?: FnfObservabilityOptions
  /**
   * Optional LLM chat/text backend (`createLlmClient`). Worker-side only in
   * generated apps — construct it server-side and pass it through; never mint
   * it in browser code.
   */
  llm?: LlmBackend
}

export interface FnfReactClients<Jobs extends readonly JobEntry[] = readonly JobEntry[]> {
  jobClient: JobClient<Jobs>
  characterClient: CharacterClient
  referenceClient: ReferenceClient
  realtimeClient?: RealtimeClient
  mediaClient: MediaClient
  profileClient: ProfileClient
  llmClient?: LlmBackend
  jobs: Jobs
  scopeKey?: string
  observability?: FnfObservabilityOptions
}

export type FnfProviderProps<Jobs extends readonly JobEntry[] = readonly JobEntry[]> =
  FnfReactClientsConfig<Jobs> & {
    children: ReactNode
  }

const FnfContext = createContext<FnfReactClients | null>(null)

export function createFnfReactClients<const Jobs extends readonly JobEntry[]>(
  config: FnfReactClientsConfig<Jobs>,
```

```
): FnfReactClients<Jobs> {
  const mediaAdapter = config.mediaAdapter ?? (config.adapter as unknown as MediaBackend)
  const profileAdapter = config.profileAdapter ?? (config.adapter as unknown as ProfileBackend)
  const characterAdapter = config.characterAdapter ?? (config.adapter as unknown as CharacterBackend)
  const referenceAdapter = config.referenceAdapter ?? (config.adapter as unknown as ReferenceBackend)
  const realtimeAdapter = config.realtimeAdapter ?? asRealtimeBackend(config.adapter)
  const observability = config.observability ? observabilityOptionsFromContext(createObservabilityContext(config.observability)) : undefined
  return {
    jobClient: createJobClient({ adapter: config.adapter, jobs: config.jobs, ...(observability ? { observability } : {}) }),
    characterClient: createCharacterClient({ adapter: characterAdapter, ...(observability ? { observability } : {}) }),
    referenceClient: createReferenceClient({ adapter: referenceAdapter, ...(observability ? { observability } : {}) }),
    ...(realtimeAdapter
      ? {
          realtimeClient: createRealtimeClient({
            adapter: realtimeAdapter,
            jobAdapter: config.adapter,
            ...(config.adapter.confirm ? { confirm: config.adapter.confirm } : {}),
            ...(observability ? { observability } : {}),
          }),
        }
      : {}),
    mediaClient: createMediaClient({
      mediaAdapter,
      ...(config.blobUploader ? { blobUploader: config.blobUploader } : {}),
      ...(config.resolveJob ? { resolveJob: config.resolveJob } : {}),
      ...(observability ? { observability } : {}),
    }),
    profileClient: createProfileClient({ profileAdapter, ...(observability ? { observability } : {}) }),
    ...(config.llm ? { llmClient: config.llm } : {}),
    jobs: config.jobs,
    ...(config.scopeKey ? { scopeKey: config.scopeKey } : {}),
    ...(observability ? { observability } : {}),
  }
}

export function FnfProvider<const Jobs extends readonly JobEntry[]>(props: FnfProviderProps<Jobs>) {
  const {
    children,
    adapter,
    jobs,
    characterAdapter,
    referenceAdapter,
    realtimeAdapter,
    mediaAdapter,
    profileAdapter,
    blobUploader,
    resolveJob,
    scopeKey,
    observability,
    llm,
  } = props
  const value = useMemo(
    () => createFnfReactClients({
      adapter,
      jobs,
      ...(characterAdapter ? { characterAdapter } : {}),
      ...(referenceAdapter ? { referenceAdapter } : {}),
      ...(realtimeAdapter ? { realtimeAdapter } : {}),
      ...(mediaAdapter ? { mediaAdapter } : {}),
      ...(profileAdapter ? { profileAdapter } : {}),
      ...(blobUploader ? { blobUploader } : {}),
      ...(resolveJob ? { resolveJob } : {}),
      ...(scopeKey ? { scopeKey } : {}),
      ...(observability ? { observability } : {}),
      ...(llm ? { llm } : {}),
    }),
    [adapter, jobs, characterAdapter, referenceAdapter, realtimeAdapter, mediaAdapter, profileAdapter, blobUploader, resolveJob, scopeKey, observability, llm],
  )
  return createElement(FnfContext.Provider, { value }, children)
}
```

```
export function useFnf<Jobs extends readonly JobEntry[] = readonly JobEntry[]>(): FnfReactClients<Jobs> {
  const value = use(FnfContext)
  if (!value)
    throw new Error('useFnf must be used inside <FnfProvider>')
  return value as FnfReactClients<Jobs>
}

export function useFnfJobClient<Jobs extends readonly JobEntry[] = readonly JobEntry[]>(): JobClient<Jobs> {
  return useFnf<Jobs>().jobClient
}

export function useFnfReferenceClient(): ReferenceClient {
  return useFnf().referenceClient
}

export function useFnfCharacterClient(): CharacterClient {
  return useFnf().characterClient
}

export function useFnfRealtimeClient(): RealtimeClient {
  const client = useFnf().realtimeClient
  if (!client)
    throw new Error('useFnfRealtimeClient requires a realtime-capable adapter passed to <FnfProvider>')
  return client
}

export function useFnfMediaClient(): MediaClient {
  return useFnf().mediaClient
}

export function useFnfProfileClient(): ProfileClient {
  return useFnf().profileClient
}

export function useFnfLlmClient(): LlmBackend {
  const client = useFnf().llmClient
  if (!client)
    throw new Error('useFnfLlmClient requires an `llm` client passed to <FnfProvider>')
  return client
}

export function useFnfJobs<Jobs extends readonly JobEntry[] = readonly JobEntry[]>(): Jobs {
  return useFnf<Jobs>().jobs
}

export function useFnfScopeKey(): string | undefined {
  return useFnf().scopeKey
}

export function useFnfObservability(): FnfObservabilityOptions | undefined {
  return useFnf().observability
}

export function useOptionalFnfObservability(): FnfObservabilityOptions | undefined {
  return use(FnfContext)?.observability
}

function observabilityOptionsFromContext(ctx: FnfObservabilityContext): FnfObservabilityOptions {
  return {
    ...(ctx.observer ? { observer: ctx.observer } : {}),
    traceId: ctx.traceId,
    ...(ctx.parentId ? { parentId: ctx.parentId } : {}),
    attributes: ctx.attributes,
    ...(ctx.onObserverError ? { onObserverError: ctx.onObserverError } : {}),
    now: ctx.now,
    idFactory: ctx.idFactory,
  }
}

function asRealtimeBackend(adapter: GenerationBackend): RealtimeBackend | undefined {
```

```
  const candidate = adapter as unknown as Partial<Record<keyof RealtimeBackend, unknown>>
  const methods: Array<keyof RealtimeBackend> = [
    'editRealtimeChain',
    'estimateRealtimeChainCost',
    'finalizeRealtimeChain',
    'listRealtimeCustomStyles',
    'createRealtimeCustomStyle',
    'updateRealtimeCustomStyle',
    'deleteRealtimeCustomStyle',
  ]
  return methods.every(method => typeof candidate[method] === 'function')
    ? adapter as unknown as RealtimeBackend
    : undefined
}
```

app/packages/fnf-react/src/realtime-query.ts

```
import type {
  RealtimeChainCostEstimate,
  RealtimeChainCostInput,
  RealtimeClient,
  RealtimeCustomStyleListQuery,
  RealtimeCustomStyleListResult,
} from '@higgsfield/fnf/realtime'
import type { FnfScopeOptions } from './keys'
import { queryOptions } from '@tanstack/react-query'
import { fnfKeys } from './keys'

export type RealtimeQueryClient = Pick<RealtimeClient, 'estimateChainCost' | 'listCustomStyles'>

export interface RealtimeQueryOptions extends FnfScopeOptions {
  enabled?: boolean
}

/** Cache one explicit realtime chain-cost estimate. */
export function realtimeChainCostQueryOptions(
  client: Pick<RealtimeQueryClient, 'estimateChainCost'>,
  input: RealtimeChainCostInput,
  opts?: RealtimeQueryOptions,
) {
  return queryOptions<RealtimeChainCostEstimate>({
    queryKey: fnfKeys.realtimeChainCost(input, opts),
    queryFn: () => client.estimateChainCost(input),
    enabled: opts?.enabled,
  })
}

/** Cache one cursor page from the authenticated user's saved custom styles. */
export function realtimeCustomStylesQueryOptions(
  client: Pick<RealtimeQueryClient, 'listCustomStyles'>,
  query: RealtimeCustomStyleListQuery = {},
  opts?: RealtimeQueryOptions,
) {
  return queryOptions<RealtimeCustomStyleListResult>({
    queryKey: fnfKeys.realtimeCustomStyles(query, opts),
    queryFn: () => client.listCustomStyles(query),
    enabled: opts?.enabled,
  })
}
```

app/packages/fnf-react/src/realtime.ts

```
import type { Generation } from '@higgsfield/fnf/client'
import { RefCountPool } from './pool'

/**
 * The connection port — inject whatever the app has. fnf-web today: SSE per
 * job set (`GET /sse/{type}?job_set_id=` — wrap `createSseConnection` /
 * the app's `Realtime.pool` here in a few lines); tomorrow: a websocket.
 * Return the close function, or `undefined` when this generation has NO
 * realtime channel (not every job type has an SSE route) — the caller falls
 * back to polling for those.
 *
 * Event payloads deliberately stop at this boundary: an event means
 * "something changed", and the fresh state is re-read through the client —
 * no wire shapes leak up.
 */
export type RealtimeTransport = (
  generation: Generation,
  emit: () => void,
) => (() => void) | undefined

/** One live channel (a job set): a single connection, N listeners. */
class RealtimeChannel {
  private readonly listeners = new Set<() => void>()
  private close: (() => void) | undefined

  /** Whether the transport actually opened a connection for this channel. */
  connected = false

  open(transport: RealtimeTransport, generation: Generation): void {
    // A throwing transport means "no channel", not a crash in the caller's
    // React effect — errors are state here too; the caller polls instead.
    try {
      this.close = transport(generation, () => {
        for (const listener of this.listeners) listener()
      })
    }
    catch {
      this.close = undefined
    }
    this.connected = this.close !== undefined
  }

  subscribe(listener: () => void): () => void {
    this.listeners.add(listener)
    return () => this.listeners.delete(listener)
  }

  free(): void {
    this.close?.()
    this.close = undefined
    this.listeners.clear()
  }
}

/**
 * Live updates over refcounted channels — the fnf-web `Realtime` +
 * `RefCountClassPool` shape, transport-agnostic. One connection per job set
 * no matter how many subscribers (a feed, a tile, a toast); the last
 * unsubscribe closes it after `freeGraceMs`, so quick re-subscribes (a feed
 * effect re-running) reuse the live connection instead of reconnecting.
 */
export class Realtime {
  private readonly channels: RefCountPool<Generation, RealtimeChannel>

  constructor(transport: RealtimeTransport, options?: { freeGraceMs?: number }) {
    this.channels = new RefCountPool(
      generation => generation.jobSetId ?? generation.id,
      (generation) => {
```

```
        const channel = new RealtimeChannel()
        channel.open(transport, generation)
        return channel
      },
      { freeGraceMs: options?.freeGraceMs ?? 1000 },
    )
  }

  /**
   * Listen for this generation's channel events. Returns the unsubscribe
   * (idempotent — extra calls are a no-op, they can't close a channel under
   * another subscriber), or `undefined` when the transport has no channel
   * for it (poll instead).
   */
  subscribe(generation: Generation, listener: () => void): (() => void) | undefined {
    const channel = this.channels.allocate(generation)
    if (!channel.connected) {
      this.channels.free(generation)
      return undefined
    }
    const unsubscribe = channel.subscribe(listener)
    let done = false
    return () => {
      if (done)
        return
      done = true
      unsubscribe()
      this.channels.free(generation)
    }
  }
}
```

app/packages/fnf-react/src/reference-query.ts

```
import type { Reference, ReferenceClient, ReferenceListQuery } from '@higgsfield/fnf/references'
import type { FnfScopeOptions } from './keys'
import { queryOptions } from '@tanstack/react-query'
import { DEFAULT_POLL_INTERVAL_MS } from './generation-query'
import { fnfKeys } from './keys'

export type ReferenceQueryClient = Pick<ReferenceClient, 'get'>
export type ReferencesQueryClient = Pick<ReferenceClient, 'list'>

export interface ReferenceQueryOptions extends FnfScopeOptions {
  intervalMs?: number | false
}

export interface ReferencesQueryOptions extends FnfScopeOptions {
  enabled?: boolean
}

function isTerminal(reference: Reference | undefined): boolean {
  return reference !== undefined && reference.status !== 'processing' && reference.status !== 'ip_checking'
}

export function referenceQueryOptions(
  client: ReferenceQueryClient,
  id: string,
  opts?: ReferenceQueryOptions,
) {
  return queryOptions({
    queryKey: fnfKeys.reference(id, opts),
    queryFn: () => client.get(id),
    refetchInterval: (query) => {
      if (opts?.intervalMs === false)
        return false
      return isTerminal(query.state.data)
        ? false
        : opts?.intervalMs ?? DEFAULT_POLL_INTERVAL_MS
    },
    refetchIntervalInBackground: true,
    staleTime: query => isTerminal(query.state.data) ? Number.POSITIVE_INFINITY : 0,
  })
}

export function referencesQueryOptions(
  client: ReferencesQueryClient,
  query: ReferenceListQuery = {},
  opts?: ReferencesQueryOptions,
) {
  return queryOptions({
    queryKey: fnfKeys.references(query, opts),
    queryFn: () => client.list(query),
    enabled: opts?.enabled,
  })
}
```

app/packages/fnf-react/src/wire-preview.ts

```
import type { GenerationInput, JobEntry } from '@higgsfield/fnf'
import { ApiJobError, buildRegistry, buildWireParams } from '@higgsfield/fnf'
import { useMemo } from 'react'
import { useFnfJobs } from './provider'

export type WirePreviewResult =
  | {
    ok: true
    jobSetType: string
    outputType: JobEntry['outputType']
    params: Record<string, unknown>
  }
  | {
    ok: false
    error: { code: string, message: string, status?: number, data?: unknown }
  }

export function getWirePreview(input: GenerationInput, jobs: readonly JobEntry[]): WirePreviewResult {
  try {
    const entry = buildRegistry(jobs).get(input.model)
    if (!entry)
      throw new ApiJobError('unknown_model', `Unknown model: ${input.model}`)
    return {
      ok: true,
      jobSetType: entry.jobSetType,
      outputType: entry.outputType,
      params: buildWireParams(input, entry),
    }
  }
  catch (err) {
    if (err instanceof ApiJobError)
      return { ok: false, error: err.toJSON() }
    return { ok: false, error: { code: 'unexpected', message: err instanceof Error ? err.message : String(err) } }
  }
}

export function useFnfWirePreview(input: GenerationInput): WirePreviewResult {
  const jobs = useFnfJobs()
  return useMemo(() => getWirePreview(input, jobs), [input, jobs])
}
```

app/packages/fnf-react/src/workspace-switch.ts

```
'use client'

import type { ProfileSnapshot, SwitchWorkspaceInput } from '@higgsfield/fnf/profile'
import type { QueryClient } from '@tanstack/react-query'
import type { FnfScopeOptions } from './keys'
import { useMutation, useQueryClient } from '@tanstack/react-query'
import { removeGenerationQueries } from './generation-cache'
import { fnfKeys } from './keys'
import { setProfileSnapshot } from './profile-query'
import { useFnfProfileClient, useFnfScopeKey } from './provider'

export interface SwitchWorkspaceClient {
  switchWorkspace: (input: SwitchWorkspaceInput) => Promise<ProfileSnapshot>
}

export interface SwitchWorkspaceMutationConfig extends FnfScopeOptions {
  nextScopeKey?: (snapshot: ProfileSnapshot) => string | undefined
  onWorkspaceChanged?: (snapshot: ProfileSnapshot) => void | Promise<void>
}

export function switchWorkspaceMutationOptions(
  client: SwitchWorkspaceClient,
  queryClient: QueryClient,
  opts: SwitchWorkspaceMutationConfig = {},
) {
  return {
    mutationFn: (input: SwitchWorkspaceInput) => client.switchWorkspace(input),
    onSuccess: async (snapshot: ProfileSnapshot) => {
      removeGenerationQueries(queryClient, opts)
      queryClient.removeQueries({ queryKey: fnfKeys.realtime(opts) })
      const nextScopeKey = opts.nextScopeKey?.(snapshot) ?? opts.scopeKey
      if (opts.scopeKey && opts.scopeKey !== nextScopeKey)
        queryClient.removeQueries({ queryKey: fnfKeys.profile({ scopeKey: opts.scopeKey }) })
      setProfileSnapshot(queryClient, snapshot, nextScopeKey ? { scopeKey: nextScopeKey } : undefined)
      await opts.onWorkspaceChanged?.(snapshot)
    },
  }
}

export function useSwitchWorkspaceMutation(opts: SwitchWorkspaceMutationConfig = {}) {
  const client = useFnfProfileClient()
  const queryClient = useQueryClient()
  const providerScopeKey = useFnfScopeKey()
  return useMutation(
    switchWorkspaceMutationOptions(client, queryClient, {
      ...opts,
      scopeKey: opts.scopeKey ?? providerScopeKey,
    }),
  )
}
```

app/packages/fnf-react/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2021",
    "lib": ["ES2021", "DOM"],
    "types": ["node"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "verbatimModuleSyntax": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules"]
}
```

app/packages/fnf-react/vitest.config.ts

```
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    globals: true,
    environment: 'node',
    include: ['src/**/*.test.ts'],
  },
})
```

app/packages/fnf/.gitignore

```
# Local scratch scripts stay untracked; the publish tooling is allowlisted
# because the build/publish workflow (and package.json scripts) depend on it.
scripts/*
!scripts/make-publish-package.mjs
!scripts/smoke-publish.mjs
```

app/packages/fnf/ai/AGENTS.md

```
# Instructions for AI agents using `@higgsfield/fnf`

Canonical AI guide for code that consumes or extends the fnf SDK; This file is
the package-level source of truth for agents; `packages/fnf/CLAUDE.md` points
here. The README explains the full SDK shape, while this guide is the strict
operational contract: what to import, what to avoid, how to wire jobs/media/
profile/observability, and how to keep model parity correct.

For SDK implementation details, read `README.md`, source JSDoc, and tests. The
tests are the executable spec.

## TL;DR rules


1. **Use public subpath imports only**: `@higgsfield/fnf/client`, `/jobs`,
   `/media`, `/profile`, `/characters`, `/references`,
   `/observability`, `/workflow-platform`, `/errors`.
   Never deep-import `src/` paths. `@higgsfield/fnf` ships the backend
   ports/interfaces plus exactly ONE bundled adapter — the Workflow Platform
   adapter at `@higgsfield/fnf/workflow-platform` (so fnf-only hosts are
   self-sufficient for generated apps). Every other concrete adapter (fnf-web,
   dev, apps-marketplace, memory) lives in the separate
   `@higgsfield/fnf-adapters` package.
2. **Never hand-build backend wire params or routes.** `createJobClient`,
   `createMediaClient`, and `createProfileClient` own validation, codecs,
   snake_case mapping, derived fields, route quirks, polling, and typed errors.
3. **All normal SDK configuration is constructor input.** Do not add hidden
   `process.env` reads, hardcoded tokens, user ids, or base URLs in job/media/
   profile code. The one intentional exception is the server-only
   `createAppsMarketplaceAdapter` secret fallback (`FNF_APPS_MARKETPLACE_SECRET`
   by default), because that adapter is a trusted-service integration.
4. **Settings are public camelCase.** Jobs map backend names with `z.wire`, for
   example `batchSize -> batch_size`, `aspectRatio -> aspect_ratio`. Unknown
   settings are stripped by schema parsing; deliberate raw wire fields belong
   in `extra`.
5. **`count` and `batchSize` are different.** `count` is SDK fan-out into N
   separate submit calls. `batchSize` is a model setting sent inside one job.
   Total outputs are `count * batchSize`. Because host confirmation tokens are
   single-use, adapters with a `confirm` gate reject `count > 1`; use a model's
   `batchSize` setting for multiple outputs from one confirmed request.
6. **Upload media before submit.** `media.upload(...)` returns a `MediaRef`;
   pass it to `client.submit({ media: { role: ref } })`. Seedance media uploads
   that need IP checks must set `forceIpCheck: true`.
7. **Job submission requires a host confirmation gate.** Every generation
   adapter factory accepts a `confirm` option (`ConfirmSubmit`); if the host
   submits generations on behalf of a user, you must implement it. Embedded
   apps delegate the exact `jobSetType` + wire `params` to the platform's
   injected approval API and must not add a second dialog. Standalone UI hosts
   own their approval modal. `submit` calls the gate once per submission after
   validation, before any network call. Resolve to proceed — a resolved string
   token rides the submit body as `confirmation_token` — reject/throw to abort
   with the typed
   `confirmation_rejected` error. Only tests and non-interactive service flows
   may omit it. See "Submission confirmation gate" below.
8. **Attach or resolve `MediaRef.meta` when local validation depends on size or
   duration.** Unknown meta is skipped locally and remains backend-authoritative.
   `aspectRatio: 'auto'` uses first-image meta where model logic supports it.
9. **Display credits are divided by 100.** Wallet fields from `getWallet()` are
   backend credit-cents. `profile.getCredits()`, `profile.getSnapshot().credits`,
   and `client.cost(input).credits` are display credits.
10. **Use safe APIs across iframe/worker boundaries.** Prefer `safeSubmit` and
    `safeUploadMedia`; branch on `error.code`. `instanceof` is not reliable after
    JSON/postMessage/Comlink serialization.
11. **Observability is metadata-only.** Observers may receive model ids, status,
    durations, counts, and safe backend ids. Never emit prompts, auth headers,
    tokens, request/response bodies, upload URLs, result URLs, filenames, emails,
    workspace names, or file bytes. The submit confirmation token is a token —
    it never goes into observer attributes.
```

Public entry points

```ts
import { createJobClient } from '@higgsfield/fnf/client'
import { createCharacterClient } from '@higgsfield/fnf/characters'
import { nanoBanana2, seedance2_0 } from '@higgsfield/fnf/jobs'
import { createMediaClient } from '@higgsfield/fnf/media'
import { createProfileClient } from '@higgsfield/fnf/profile'
import { createReferenceClient } from '@higgsfield/fnf/references'
import { createConsoleObserver } from '@higgsfield/fnf/observability'
import { createWorkflowPlatformAdapter } from '@higgsfield/fnf/workflow-platform'
import { createAppsMarketplaceAdapter, createDevFnfWebAdapter, createFnfWebAdapter } from '@higgsfield/fnf-adapters'
import { errorFromJSON } from '@higgsfield/fnf/errors'
```

The root barrel also exports these APIs, but subpaths are preferred because
agents and app bundlers can pull only the domain they need.

Adapters and auth

One full adapter can satisfy jobs, media, and profile plus the optional
Elements and character-training ports. Reuse the same adapter when a host wants
a single auth/workspace context.

Scope rule: the `https://fnf.internal` requirement is a Supercomputer generated
app/template policy, not a universal SDK limitation. If the task explicitly
targets another approved web app, product surface, dev sandbox, service
integration, or custom endpoint, choose or implement the adapter for that host's
backend contract. Do not force generated-app rules onto non-generated SDK
consumers.

Adapter	Auth	Use
`createFnfWebAdapter({ baseUrl, getToken, workspaceId? })`	Clerk Bearer token plus optional `hf-workspace-id`	product/in-app flows
`createDevFnfWebAdapter({ userId, workspaceId? })`	`hf-dev-user-id` plus optional `hf-workspace-id`; base URL defaults to dev	dev backend smoke tests and local sandboxes only
`createAppsMarketplaceAdapter({ userId, workspaceId? })`	`fnf-apps-marketplace-secret` from `secret` or `FNF_APPS_MARKETPLACE_SECRET` + `hf-user-id`; base URL defaults to `https://dev-fnf.higgsfie
`createWorkflowPlatformAdapter({ baseUrl, observability? })` — BUNDLED in `@higgsfield/fnf/workflow-platform`	platform-attached server identity; no bearer token	generated apps and Supercomputer
`createMemoryBackend()` / `createMemoryMediaAdapter()` / `createMemoryProfileAdapter()`	none	tests, examples, offline demos

Every generation adapter factory above also accepts a `confirm` option — the
host's submission confirmation gate (see "Submission confirmation gate" under
Job recipes). Hosts that submit generations for a user must pass it.

```ts
import { createJobClient } from '@higgsfield/fnf/client'
import { createMediaClient } from '@higgsfield/fnf/media'
import { createProfileClient } from '@higgsfield/fnf/profile'
import { createWorkflowPlatformAdapter } from '@higgsfield/fnf/workflow-platform'
import { seedance2_0 } from '@higgsfield/fnf/jobs'

const adapter = createWorkflowPlatformAdapter({
  baseUrl: 'https://fnf.internal',
})

const jobs = createJobClient({ adapter, jobs: [seedance2_0] })
const media = createMediaClient({ mediaAdapter: adapter })
const profile = createProfileClient({ profileAdapter: adapter })
```

Generated apps must put this fnf.internal adapter server-side behind app-local
server functions or `/api/*` routes. Other hosts may use their matching adapter
or a custom backend-port implementation. Do not expose dev user ids, bearer
tokens, or workspace secrets to arbitrary visitors.

Apps Marketplace uses a different trusted-service header contract from the
normal dev adapter. The backend mount is intentionally dev-only for now, and
the service secret must never be shipped to browser code. This adapter is for
SDK smoke tests and trusted service experiments, not generated apps:

```ts
```

````
import { createAppsMarketplaceAdapter } from '@higgsfield/fnf-adapters'

const adapter = createAppsMarketplaceAdapter({
  userId: actingUserId,
  workspaceId,
})
```

By default, the adapter reads `FNF_APPS_MARKETPLACE_SECRET` from the server
environment. Pass `secret` for tests/custom secret loading, or `secretEnv` when
the host uses a different env var name.

The adapter returns the same `FnfAdapter` shape as `createFnfWebAdapter`, so pass
it to `createJobClient`, `createMediaClient`, and `createProfileClient` exactly
the same way. Do not duplicate route/body logic for `/apps-marketplace`; it
mirrors the existing fnf routers behind different auth headers.

For generated apps, use the Workflow Platform adapter against
`https://fnf.internal`. It sends static, versionable routes under `/user`,
`/workspaces/*`, `/reference-elements/*`, `/custom-references/*`, and
`/jobs/*`; the platform behind fnf.internal resolves the final internal
endpoints and body quirks. The SDK still handles model
validation, camelCase settings, `z.wire` serialization, media refs, profile
mapping, costs, polling, and typed errors.

```ts
import { createWorkflowPlatformAdapter } from '@higgsfield/fnf/workflow-platform'

const adapter = createWorkflowPlatformAdapter({
  baseUrl: 'https://fnf.internal',
  observability,
})
```

In generated apps, do not use env-controlled backend URL selectors, public fnf
URLs, dev fnf URLs, apps marketplace URLs, `getToken`, `Authorization`,
`hf-dev-user-id`, or service secrets. The platform attaches identity
automatically to server-side `https://fnf.internal` egress. In non-generated
hosts, follow that host's explicit adapter/auth contract instead.

fnf.internal method contract:

- Submit goes through `POST /jobs/submit`.
- Cost goes through `POST /jobs/cost`.
- Cancel goes through `POST /jobs/{job_id}/cancel`.
- Media presign goes through `POST /jobs/media/presign`.
- Media confirm goes through `POST /jobs/media/{media_id}/confirm`.
- Job reads go through `GET /jobs/{job_id}`, `GET /jobs/sets/{job_set_id}`,
  and the feed route `GET /jobs?gen_type=<image|video|...>`.
- Media reads go through `GET /jobs/media/{media_id}` and `GET /jobs/media`.
- Element reads go through `GET /reference-elements/{element_id}` and
  `GET /reference-elements`.
- Character training goes through `POST /custom-references`; lifecycle reads
  go through `GET /custom-references/{character_id}`.
- User/profile reads go through `GET /user`.
- Workspace reads go through `GET /workspaces`, `GET /workspaces/current`,
  and `GET /workspaces/wallet`.
- Workspace switch goes through `POST /workspaces/switch`.

Do not hand-write `fetch('/jobs')` with an `operation` body,
`fetch('/jobs?operation=submit')`, or model-specific fnf routes in generated
apps. Use `createJobClient`, `createMediaClient`, and `createProfileClient`
with `createWorkflowPlatformAdapter`. If an app wraps SDK calls in TanStack
server functions or `/api/*` routes, submit/cost/media routes must be `POST`,
not `GET`. A 405 / "Method Not Allowed" on generation is usually this method or
stale SDK snapshot mismatch.

## Elements and custom-reference character recipes

Elements are the user's reusable asset library; Characters are one Element
````
````

kind. Character training is a separate multi-image operation that creates a processing Character Element. Do not wrap Elements in a character-only abstraction and do not add presets to this flow.

```ts
import { createCharacterClient } from '@higgsfield/fnf/characters'
import { createJobClient } from '@higgsfield/fnf/client'
import { soulV2Image } from '@higgsfield/fnf/jobs'
import { createReferenceClient } from '@higgsfield/fnf/references'

const references = createReferenceClient({ adapter })
const characters = createCharacterClient({ adapter })
const jobs = createJobClient({ adapter, jobs: [soulV2Image] })

const existing = await references.list({ category: 'character', size: 50 })

const pending = await characters.create({
  name: 'My character',
  type: 'soul_2',
  images, // 1–100 MediaRef values; upload browser Files first
})
const character = await characters.wait(pending)

if (character.status === 'completed') {
  await jobs.submit({
    model: 'text2image_soul_v2',
    prompt: { instruction: 'Candid flash photo at a late-night diner' },
    settings: { customReferenceId: character.id, batchSize: 4 },
  })
}
```

Rules:

- Supported types are `soul`, `soul_2` (default), and `soul_cinematic`; use the generic custom-reference route for all of them.
- `soul` accepts uploaded `media_input` refs only. `soul_2` and `soul_cinematic` also accept compatible image job refs ending in `_job`.
- Poll to `completed` or `failed`, then refresh the Elements query.
- Existing Character Elements expose `reference.characterId`; generation uses that value as `settings.customReferenceId`. Element id and character id are different.
- Use `references.list/get` and `characters.create/get/wait`; never hand-write the underlying routes.

LLM (chat completions) — same zero-token rail

Generated apps reach the platform LLM through the SAME zero-token egress rail as `fnf.internal` jobs — construct the client with the `/llm` mount and NO `getToken`/`userId`; the platform attaches the signed-in visitor's identity on egress, and spend is billed to that visitor's credits:

```ts
import { createLlmClient } from '@higgsfield/fnf'

// Worker-side only (server functions / `/api/*` routes) — never browser code.
const llm = createLlmClient({ baseUrl: 'https://fnf.internal/llm' })

const [model] = await llm.listModels()
if (!model) throw new Error('No LLM models are currently available')

const res = await llm.complete({
  model,
  messages: [{ role: 'user', content: 'One-line haiku about renders.' }],
})

for await (const chunk of llm.stream({
  model,
  messages: [{ role: 'user', content: 'Stream me a haiku.' }],
})) {
```

```
  // chunk.choices[0].delta.content — SSE deltas, OpenAI-compatible shape
}
```

Rules: do NOT pass `getToken`, `userId`, or any secret on this rail (the README's external `fnf-api-gw` + `getToken` example is for NON-generated first-party hosts only); Worker-side only. Model availability is runtime gateway configuration: call `listModels()` server-side, use exact returned ids, handle an empty list, and never hardcode a catalog. A React model picker may receive those ids through an authenticated app-local server function or route. Pass only a browser-safe proxy backend to `FnfProvider` as `llm`; never pass the raw internal client to client-rendered code (see `@higgsfield/fnf-react`'s `ai/AGENTS.md`).

Apps Marketplace uses a different trusted-service header contract from the normal dev adapter. The backend mount is intentionally dev-only for now, and the service secret must never be shipped to browser code. This adapter is for SDK smoke tests and trusted service experiments, not generated apps:

```ts
import { createAppsMarketplaceAdapter } from '@higgsfield/fnf-adapters'

const adapter = createAppsMarketplaceAdapter({
  userId: actingUserId,
  workspaceId,
})
```

By default, the adapter reads `FNF_APPS_MARKETPLACE_SECRET` from the server environment. Pass `secret` for tests/custom secret loading, or `secretEnv` when the host uses a different env var name.

The adapter returns the same `FnfAdapter` shape as `createFnfWebAdapter`, so pass it to `createJobClient`, `createMediaClient`, and `createProfileClient` exactly the same way. Do not duplicate route/body logic for `/apps-marketplace`; it mirrors the existing fnf routers behind different auth headers.

For generated apps, use the Workflow Platform adapter against `https://fnf.internal`. It sends static, versionable routes under `/user`, `/workspaces/*`, and `/jobs/*`; the platform behind fnf.internal resolves the final internal endpoints and body quirks. The SDK still handles model validation, camelCase settings, `z.wire` serialization, media refs, profile mapping, costs, polling, and typed errors.

```ts
import { createWorkflowPlatformAdapter } from '@higgsfield/fnf/workflow-platform'

const adapter = createWorkflowPlatformAdapter({
  baseUrl: 'https://fnf.internal',
  observability,
})
```

In generated apps, do not use env-controlled backend URL selectors, public fnf URLs, dev fnf URLs, apps marketplace URLs, `getToken`, `Authorization`, `hf-dev-user-id`, or service secrets. The platform attaches identity automatically to server-side `https://fnf.internal` egress. In non-generated hosts, follow that host's explicit adapter/auth contract instead.

fnf.internal method contract:

- Submit goes through `POST /jobs/submit`.
- Cost goes through `POST /jobs/cost`.
- Cancel goes through `POST /jobs/{job_id}/cancel`.
- Media presign goes through `POST /jobs/media/presign`.
- Media confirm goes through `POST /jobs/media/{media_id}/confirm`.
- Job reads go through `GET /jobs/{job_id}`, `GET /jobs/sets/{job_set_id}`, and the feed route `GET /jobs?gen_type=<image|video|...>`.
- Media reads go through `GET /jobs/media/{media_id}` and `GET /jobs/media`.
- User/profile reads go through `GET /user`.
- Workspace reads go through `GET /workspaces`, `GET /workspaces/current`,

````
  and `GET /workspaces/wallet`.
- Workspace switch goes through `POST /workspaces/switch`.

Do not hand-write `fetch('/jobs')` with an `operation` body,
`fetch('/jobs?operation=submit')`, or model-specific fnf routes in generated
apps. Use `createJobClient`, `createMediaClient`, and `createProfileClient`
with `createWorkflowPlatformAdapter`. If an app wraps SDK calls in TanStack
server functions or `/api/*` routes, submit/cost/media routes must be `POST`,
not `GET`. A 405 / "Method Not Allowed" on generation is usually this method or
stale SDK snapshot mismatch.

## Job recipes

Register only the jobs the host can actually use. The `jobs: [...]` registry is
what gives `model`, `settings`, and media roles their TypeScript narrowing.

```ts
import { createJobClient } from '@higgsfield/fnf/client'
import { seedance2_0 } from '@higgsfield/fnf/jobs'

const client = createJobClient({ adapter, jobs: [seedance2_0] })

const { generations } = await client.submit({
  model: 'seedance_2_0',
  prompt: { instruction: 'cinematic lightning over the ocean' },
  settings: {
    mode: 'std',
    duration: 5,
    aspectRatio: '16:9',
    resolution: '720p',
    batchSize: 1,
  },
})

const [done] = await client.wait(generations, {
  signal,
  onProgress: generation => {
    console.log(generation.status)
  },
})

console.log(done.results?.rawUrl)
```

Useful client calls:

- `submit(input)` submits immediately and returns generations.
- `safeSubmit(input)` returns `{ ok: true, ... } | { ok: false, error }`.
- `adjust(input, ['near-aspect-ratio', 'near-duration'])` explicitly snaps
  values; `submit` itself does not normalize.
- `cost(input)` returns display credits for UI previews.
- `get(id)`, `getSet(jobSetId)`, `poll(id)`, and `wait(generations)` read and
  poll generation state.
- `cancel(id)` cancels queued jobs when the adapter supports it.
- `list(opts)` reads the generation feed; handle typed `not_supported` for
  filters the backend route cannot support.

When regenerating, submit the parsed read-model input back through the same
client and registry:

```ts
await client.submit(done.input as Parameters<typeof client.submit>[0])
```

### Submission confirmation gate

If the host submits generations on behalf of a user, implement the `confirm`
gate — pass it to the adapter factory (all generation adapters accept it).
Embedded apps must call the platform's injected approval API with the exact
`jobSetType` and final wire `params`, return its token unchanged, and must not
````

````
open an additional browser or custom dialog. Standalone UI hosts own their
approval modal. `submit` calls the gate after validation and wire-param
building, before any network call. Because confirmation tokens are single-use,
`submit` rejects `count > 1` before invoking the gate; use the model's
`batchSize` setting when it supports multiple outputs in one request.

Contract (`ConfirmSubmit` from `@higgsfield/fnf`):

- Resolve to proceed. Resolving with a string sends it as top-level
  `confirmation_token` on the create request (e.g. a token minted by the
  host's own confirm endpoint or modal flow).
- Resolve with nothing only when the standalone host's approval flow is tokenless.
- Reject/throw to abort: `submit` fails with the typed
  `confirmation_rejected` error and no request is sent. Branch on
  `error.code === 'confirmation_rejected'` — user declined is not a failure.

```ts
const adapter = createFnfWebAdapter({
  baseUrl,
  getToken,
  // An embedded host owns the security UI and mints the intent token.
  confirm: ({ jobSetType, params }) =>
    platformApproval.requestGeneration(jobSetType, params),
})
```

Do not auto-confirm (`confirm: async () => {}` in a real UI host defeats the
gate), and never log or emit the token through observability.

## Media recipes

Media is a separate client. Upload and resolve APIs produce `MediaRef` objects
that job media codecs know how to serialize.

Strict upload rule for generated apps and server bridges: binary files do not go
through JSON. Never pass browser `File`, `Blob`, `ArrayBuffer`, `Uint8Array`,
base64 strings, or byte arrays through `createServerFn` input or a JSON request
body. Use multipart `FormData` to an app-local `POST /api/media/upload` route,
read `await file.arrayBuffer()` server-side, call `media.upload({ source:
bytes, ... })`, and return only the submit-ready `MediaRef`. The later
generation request should contain prompt/settings/`MediaRef` only.

Bad patterns:

```ts
await generate({ data: { file } })                // bad: File through JSON
await generate({ data: { bytes: Array.from(u8) } }) // bad: huge JSON
String.fromCharCode(...u8)                        // bad: stack overflow
```

Good shape:

```txt
browser File -> POST multipart /api/media/upload -> media.upload -> MediaRef
MediaRef + prompt/settings -> POST /api/generate -> jobs.submit
```

```ts
import { createMediaClient, createDomMediaMetaResolver, resolveMediaMeta } from '@higgsfield/fnf/media'

const media = createMediaClient({ mediaAdapter: adapter })

const { ref } = await media.upload({
  source: fileBytes,
  filename: 'frame.png',
  type: 'image',
  forceIpCheck: true,
})

const input = {
````
````

````
  model: 'seedance_2_0' as const,
  media: { start_image: [ref] },
  prompt: { instruction: 'animate this frame' },
  settings: { duration: 5, aspectRatio: 'auto' as const },
}

const measured = await resolveMediaMeta(input, createDomMediaMetaResolver())
await jobs.submit(measured)
```

Notes:

- `media.upload({ source })` accepts only `Blob`, `ArrayBuffer`, `Uint8Array`, or
  a lazy `{ read() }` returning one of those. Invalid JSON-shaped sources fail
  early with `invalid_media_source`.
- `MediaRef.meta` never goes to the backend; it only powers local validation and
  local aspect/size derivation.
- `uploadFromUrl` is convenience, not SSRF protection. For server-side
  untrusted URLs, inject a hardened uploader/fetcher with private-IP, redirect,
  byte-size, and content-type policy.
- Moderation/IP results are data. Use `safeUploadMedia` when crossing process
  boundaries.

## Profile and workspace recipes

The profile client is the SDK account/workspace domain. It is not the app's
creator-profile entity.

```ts
import { createProfileClient } from '@higgsfield/fnf/profile'

const profile = createProfileClient({ profileAdapter: adapter })

const user = await profile.getUser()
const workspaces = await profile.listWorkspaces()
const currentWorkspace = await profile.getCurrentWorkspace()
const wallet = await profile.getWallet()
const credits = await profile.getCredits({ includeOnDemand: true })
const snapshot = await profile.getSnapshot()

await profile.switchWorkspace({ workspaceId: workspaces[0].id })
```

Public fields are camelCase:

- user: `workspaceId`, `workspaceType`, `workspaceRole`, `planType`,
  `billingPeriod`, `totalPlanCredits`
- workspace: `clerkOrganizationId`, `avatarUrl`, `isOwner`
- wallet: `subscriptionBalance`, `subscriptionTotal`, `creditsBalance`,
  `onDemandCredits`

Credit display rule:

```ts
wallet?.subscriptionBalance          // raw backend credit-cents; not for UI
credits?.totalAvailableCredits       // display credits; safe for UI
credits?.raw.totalAvailableCredits   // raw value for debug/accounting only
```

`profile.getCredits()` returns a `ProfileCredits` object, not a number. Never
write `typeof credits === 'number'`; use one of the display fields above.

`switchWorkspace()` posts backend context and returns a fresh snapshot. Host
apps still own Clerk/session metadata, React Query invalidation, and adapter
header state.

## Observability recipes

Observability is a vendor-neutral observer callback. The SDK does not send
telemetry anywhere by itself.
````
````

```ts
import { createJobClient } from '@higgsfield/fnf/client'
import { createConsoleObserver } from '@higgsfield/fnf/observability'

const observer = createConsoleObserver()

const client = createJobClient({
  adapter,
  jobs: [seedance2_0],
  observability: {
    observer,
    traceId: requestId,
    attributes: { surface: 'sandbox' },
  },
})
```

Expected event names include:

- jobs: `fnf.job.submit`, `fnf.job.cost`, `fnf.job.get`, `fnf.job.get_set`,
  `fnf.job.list`, `fnf.job.cancel`, `fnf.job.wait`, `fnf.job.poll`
- media: `fnf.media.upload`, `fnf.media.presign`, `fnf.media.transfer`,
  `fnf.media.confirm`, `fnf.media.get`, `fnf.media.list`, `fnf.media.resolve`
- profile: `fnf.profile.get_user`, `fnf.profile.list_workspaces`,
  `fnf.profile.get_current_workspace`, `fnf.profile.get_wallet`,
  `fnf.profile.get_credits`, `fnf.profile.get_snapshot`,
  `fnf.profile.switch_workspace`
- transport/backend wrappers: `fnf.transport.request` and
  `fnf.backend.*` wrapper spans

Observer errors are caught and never change SDK behavior. Async observer
promises are intentionally not awaited.

Privacy rule: never add prompts, request bodies, raw params, auth headers,
tokens, upload/result URLs, filenames, emails, workspace names, or file bytes to
event attributes. Bridge the safe observer events to Sentry, Datadog,
OpenTelemetry, console logs, or product analytics in host code.

Current SDK catalog

Import catalog entries from `@higgsfield/fnf/jobs` and register them with
`createJobClient({ jobs: [...] })`.

Image:

- `soulV2Image` -> `text2image_soul_v2`
- `soulCinemaImage` -> `soul_cinematic`
- `gptImage2` -> `gpt_image_2`
- `seedreamV4_5`
- `nanoBanana2` -> `nano_banana_2`
- `nanoBananaFlash` -> `nano_banana_flash`
- `recraftV41Image`

Video:

- `seedance2_0` with `settings.mode: 'std' | 'fast'`
- `seedance2_5` with `settings.model: 'default' | 'video_edit' |
  'video_extension'` — one job set type, three shapes. `default`
  generates from prompt + references; `video_edit` rewrites the ONE
  attached `video` ref (its length and box come from that clip, so
  `duration`/`aspectRatio` are ignored); `video_extension` continues a
  clip and additionally requires `settings.extensionMode:
  'backward' | 'forward'`. Takes up to 30 images, 10 videos and 10
  audios (50 refs total, counting `<<<element>>>`/`@uuid` mentions in
  the prompt); reference videos and audios are each 2-30s.
- `kling3_0`
- `kling3MotionControl`
- `happyHorse`
- `grokImagine`
- `grokImagineV15`

- `veo3_1Lite`
- `wan27`

Upscale:

- `topazImageUpscale`
- `topazImageGenerativeUpscale`
- `nanoBanana2Upscale`
- `topazVideoUpscale`
- `higgsfieldVideoUpscale`
- `soraEnhanceVideo`
- `bytedanceVideoUpscale`

Legacy/currently supported baseline:

- `klingVideo`
- `textToImageSoul`

Errors and boundaries

All SDK errors extend `ApiJobError` with stable `code`, optional `status`, and
typed `data`. `toJSON()` is uniform; `errorFromJSON` restores SDK error classes
when needed.

Common codes: `validation`, `out_of_credits`, `rate_limit`,
`minimum_plan_required`, `prompt_nsfw`, `ip_detected`,
`ip_check_rate_limit_reached`, `job_failed`, `timeout`, `aborted`,
`not_supported`, `media_too_large`, `request_timeout`,
`confirmation_rejected` (the host's `confirm` gate declined the submission —
a user choice, not a failure).

For UI and worker/iframe boundaries, branch on `error.code`:

```ts
const result = await client.safeSubmit(input)

if (!result.ok) {
  if (result.error.code === 'out_of_credits') {
    // show billing/credits UI
  }
}
```

Avoid:

```ts
try {
  await client.submit(input)
}
catch (error) {
  if (error instanceof OutOfCreditsError) {
    // fragile across postMessage/Comlink/JSON boundaries
  }
}
```

Anti-patterns

Do not fetch product job routes directly

```ts
await fetch(`${baseUrl}/jobs/v2/seedance_2_0`, { body: JSON.stringify(params) }) // bad
await client.submit({ model: 'seedance_2_0', prompt, settings })                 // good
```

Direct fetches bypass validation, route/body special cases, typed errors,
cost logic, restore/finalize round-trips, and observability.

Generated apps must use `createWorkflowPlatformAdapter` against
`https://fnf.internal` rather than direct fnf product routes. The adapter may

call only `/user`, `/workspaces/*`, and `/jobs/*`; do not add `/jobs/v2/*`,
`/job-sets/*`, `/media/*`, `/video`, `/audio`, or `/workspaces/details` paths to
generated app code.

Do not use backend snake_case in public settings

```ts
settings: { aspect_ratio: '9:16', batch_size: 2 } // bad; stripped by schema
settings: { aspectRatio: '9:16', batchSize: 2 }   // good
extra: { experimental_backend_flag: true }        // good only when deliberate
```

Do not display raw wallet balances

```ts
wallet.subscriptionBalance                  // bad for UI; raw credit-cents
snapshot.credits?.monthlyRemaining          // good display credits
snapshot.credits?.raw.monthlyRemaining      // good only for debug/accounting
await client.cost(seedanceInput)            // good model preview; 2250 cents -> 23 credits
```

Do not leak private payloads into observability

```ts
attributes: { prompt: input.prompt.instruction } // bad
attributes: { model: input.model }               // good
```

Do not assume all adapters support all optional calls

Generic tools should handle `error.code === 'not_supported'` for operations
such as unsupported list filters, unavailable media listing, missing cancel
support, or backend-only costs.

Extending the SDK catalog

Use `defineJob({ jobSetType, outputType, params, validate, credits, finalize,
restore })`. The public settings schema should be camelCase; wire names belong
in `z.wire`.

Non-negotiables:

1. Ground product constants in fnf-web and fnf-api behavior; cite provenance in
   comments where existing jobs do.
2. Reuse helpers in `jobs/checks.ts`, `jobs/dimensions.ts`,
   `jobs/video-helpers.ts`, `jobs/image-helpers.ts`, and
   `jobs/upscale-helpers.ts` before adding one-off logic.
3. If `finalize` deletes, renames, or derives settings, add `restore` so
   get-then-resubmit round-trips parsed generations correctly.
4. Export new jobs from `src/jobs/index.ts` and the root barrel.
5. Update `createFnfWebAdapter` route/body/cost behavior (in
   `@higgsfield/fnf-adapters`) when the product route is not the generic
   `/jobs/{jobSetType}` shape.
6. Add tests for defaults, validations, media role/count/meta rules, cost,
   adapter route/body shape, parse/build round-trips, and typed API coverage.
7. For docs/type tests, use `expectType<T>(value)` instead of bare property
   access expressions; ESLint treats bare expressions as unused.

Focused package checks:

```sh
yarn workspace @higgsfield/fnf test
yarn workspace @higgsfield/fnf typecheck
yarn workspace @higgsfield/fnf-adapters test
yarn workspace @higgsfield/fnf-adapters typecheck
```

The client/media/profile wire-parity tests (exact adapter routes and bodies)
live in `packages/fnf-adapters/src/__tests__`; `@higgsfield/fnf` keeps the
port-level unit tests only.

```
Do not run the full app build, `next build`, or `ci:build` in this workspace.

## When in doubt

1. Read the job declaration in `src/jobs/<model>.ts`.
2. Read `README.md` for the deeper SDK lifecycle and adapter model.
3. Read the nearest product-parity test in `src/jobs/__tests__/`.
4. Check `src/errors.ts` before branching on an error.
5. Do not guess backend wire behavior; confirm it in product/backend source and
   encode it through the SDK surface.
```

app/packages/fnf/CLAUDE.md

```
@./ai/AGENTS.md
```

app/packages/fnf/package.json

```
{
  "name": "@higgsfield/fnf",
  "version": "0.0.0",
  "private": true,
  "description": "Isomorphic generation + media client for Higgsfield (fnf)",
  "author": "support@higgsfield.ai",
  "license": "UNLICENSED",
  "sideEffects": [
    "./src/media/errors.ts"
  ],
  "type": "module",
  "exports": {
    ".": "./src/index.ts",
    "./jobs": "./src/jobs/index.ts",
    "./client": "./src/client/index.ts",
    "./characters": "./src/characters/index.ts",
    "./references": "./src/references/index.ts",
    "./realtime": "./src/realtime/index.ts",
    "./media": "./src/media/index.ts",
    "./profile": "./src/profile/index.ts",
    "./observability": "./src/observability/index.ts",
    "./workflow-platform": "./src/workflow-platform/index.ts",
    "./errors": "./src/errors.ts",
    "./package.json": "./package.json"
  },
  "files": [
    "src"
  ],
  "scripts": {
    "build": "vite build",
    "publish:prepare": "node scripts/make-publish-package.mjs",
    "test": "vitest run",
    "typecheck": "tsc --noEmit -p tsconfig.json"
  },
  "peerDependencies": {
    "zod": "^4.0.0"
  },
  "devDependencies": {
    "@types/node": "^20.17.30",
    "typescript": "^5.8.3",
    "vite": "^8.1.3",
    "vite-plugin-dts": "^5.0.3",
    "vitest": "^4.0.17",
    "zod": "^4.0.0"
  }
}
```

app/packages/fnf/README.md

````
# @higgsfield/fnf

Typed SDK for Higgsfield generation jobs, media, and profiles.
The package owns validation, wire serialization, polling, uploads, workspace
reads, and typed errors. Network behavior is supplied through backend adapters.

The full operational contract for agents lives in [`ai/AGENTS.md`](./ai/AGENTS.md).

## Public API

- `@higgsfield/fnf/client` — submit, inspect, poll, wait for, cancel, list,
  and estimate the cost of generation jobs.
- `@higgsfield/fnf/jobs` — built-in model definitions.
- `@higgsfield/fnf/media` — resolve and upload media.
- `@higgsfield/fnf/profile` — user, workspace, wallet, and credit reads.
- `@higgsfield/fnf/observability` — metadata-only SDK telemetry.
- `@higgsfield/fnf/workflow-platform` — the bundled Workflow Platform
  adapter.
- `@higgsfield/fnf/errors` — serializable typed errors.
- `@higgsfield/fnf` — root exports, including the job-definition helpers.

## Generation

```ts
import { createJobClient } from '@higgsfield/fnf/client'
import { nanoBanana2 } from '@higgsfield/fnf/jobs'
import { createWorkflowPlatformAdapter } from '@higgsfield/fnf/workflow-platform'

const adapter = createWorkflowPlatformAdapter({
  baseUrl: 'https://fnf.internal',
})

const jobs = createJobClient({
  adapter,
  jobs: [nanoBanana2],
})

const { generations } = await jobs.submit({
  model: 'nano_banana_2',
  prompt: { instruction: 'A blue cat' },
  settings: {
    aspectRatio: '1:1',
    resolution: '2k',
    batchSize: 1,
  },
})

const [result] = await jobs.wait(generations)
console.log(result.results?.rawUrl)
```

Generated apps should call the Workflow Platform adapter from their server
code. Hosts submitting on behalf of a user must also provide the adapter's
`confirm` gate. Confirmation intents are single-use, so confirmed submissions
must keep the top-level `count` at `1`; use a model's `settings.batchSize` when
it supports multiple outputs in one approved request.

## Defining a job

```ts
import { defineJob, z } from '@higgsfield/fnf'

export const myModel = defineJob({
  jobSetType: 'my_model',
  outputType: 'image',
  params: {
    prompt: true,
    media: {
````
````

```
      field: 'input_images',
      format: 'unwrapped',
      roles: ['image'],
    },
    settings: {
      aspectRatio: z.wire(
        'aspect_ratio',
        z.aspectRatio(['1:1', '16:9']),
      ),
      resolution: z._default(z.enum(['1k', '2k']), '1k'),
    },
  },
})
```

Public settings use camelCase. Use `z.wire` when the backend field has a
different name. Job tests are the executable specification for validation,
serialization, and model parity.

LLM gateway

`createLlmClient` supports non-streaming completion, streaming completion,
tool calls, and model listing. Keep its platform token on the server.

```ts
import { createLlmClient } from '@higgsfield/fnf'

const llm = createLlmClient({
  baseUrl: process.env.LLM_GATEWAY_URL!,
  getToken: () => getPlatformToken(),
  workspaceId: () => getActiveWorkspaceId(),
  appId: process.env.APP_SLUG,
})

const [model] = await llm.listModels()
if (!model) throw new Error('No LLM models are currently available')

const result = await llm.complete({
  model,
  messages: [{ role: 'user', content: 'Write a one-line haiku.' }],
})

console.log(result.content)
```

`baseUrl` must be the gateway's LLM mount without a trailing `/v1`; the
client appends `/v1/chat/completions` and `/v1/models`.

The gateway's model catalog is runtime configuration. Discover it with
`listModels()` on the server, use exact returned ids, and handle an empty
list. A UI model picker should receive those ids through an authenticated
app-local route; if a selected id is no longer returned, refresh the list and
surface an unavailable-model error instead of silently changing models.

app/packages/fnf/scripts/make-publish-package.mjs

```
#!/usr/bin/env node
/**
 * Assembles the publishable package in packages/fnf/publish/.
 *
 * The workspace package (@higgsfield/fnf, private, version 0.0.0, exports
 * pointing at TypeScript source) is never touched — in-repo consumers keep
 * resolving raw source through yarn workspaces. This script derives a
 * publish-only package.json for external consumption from GitHub Packages,
 * where the scope must match the org: @higgsfield-ai/fnf.
 *
 * Usage: node scripts/make-publish-package.mjs <version>
 * Expects `vite build` to have populated dist/ first.
 */
import { cpSync, existsSync, mkdirSync, readFileSync, rmSync, writeFileSync } from 'node:fs'
import { dirname, join } from 'node:path'
import { fileURLToPath } from 'node:url'

const PUBLISH_NAME = '@higgsfield-ai/fnf'
const REGISTRY = 'https://npm.pkg.github.com'

const packageRoot = dirname(fileURLToPath(new URL('.', import.meta.url)))
const distDir = join(packageRoot, 'dist')
const publishDir = join(packageRoot, 'publish')

const version = process.argv[2]
if (!version || !/^\d+\.\d+\.\d+(-[0-9A-Za-z-]+(\.[0-9A-Za-z-]+)*)?$/.test(version)) {
  console.error(`Usage: make-publish-package.mjs <semver>. Got: ${JSON.stringify(version)}`)
  process.exit(1)
}

if (!existsSync(join(distDir, 'index.js'))) {
  console.error('dist/index.js not found — run `yarn workspace @higgsfield/fnf build` first.')
  process.exit(1)
}

const pkg = JSON.parse(readFileSync(join(packageRoot, 'package.json'), 'utf8'))

const exports = {}
for (const [subpath, target] of Object.entries(pkg.exports)) {
  if (!target.endsWith('.ts'))
    continue
  const entryName = subpath === '.' ? 'index' : subpath.slice(2)
  exports[subpath] = {
    // d.ts files mirror the src/ layout (dist/jobs/index.d.ts); js entries are flat.
    types: target.replace('./src', './dist').replace(/\.ts$/, '.d.ts'),
    default: `./dist/${entryName}.js`,
  }
}
exports['./package.json'] = './package.json'

// Deliberately minimal: no repository, author, or any other field that points
// back at the source repo or people. Only what consumers need to install and
// resolve the package.
const publishPkg = {
  name: PUBLISH_NAME,
  version,
  description: pkg.description,
  license: pkg.license,
  type: pkg.type,
  // src/media/errors.ts registers media error codes at import time. After
  // bundling it lives in a shared chunk (or the media entry), so mark both
  // patterns side-effectful to keep tree-shakers from dropping the registrations.
  sideEffects: ['./dist/chunks/*.js', './dist/media.js'],
  exports,
  files: ['dist'],
  // No peerDependencies: zod is bundled into dist by the vite build.
  publishConfig: {
    registry: REGISTRY,
```

```
    access: 'restricted',
  },
}

rmSync(publishDir, { recursive: true, force: true })
mkdirSync(publishDir, { recursive: true })
writeFileSync(join(publishDir, 'package.json'), `${JSON.stringify(publishPkg, null, 2)}\n`)
cpSync(distDir, join(publishDir, 'dist'), { recursive: true })

// Published docs must reference the published name, not the workspace name.
const readme = readFileSync(join(packageRoot, 'README.md'), 'utf8')
writeFileSync(join(publishDir, 'README.md'), readme.replaceAll('@higgsfield/fnf', PUBLISH_NAME))

console.log(`Prepared ${PUBLISH_NAME}@${version} in ${publishDir}`)
```

app/packages/fnf/scripts/smoke-publish.mjs

```
/**
 * Smoke test for the packed tarball. Runs in a scratch project where the
 * tarball has been extracted to node_modules/@higgsfield-ai/fnf and nothing
 * else is installed — verifies every public subpath resolves in plain Node
 * ESM, that zod was bundled in (no runtime deps), and that import-time side
 * effects (media error-code registration) survived bundling.
 */
/* eslint-disable import/no-unresolved -- @higgsfield-ai/fnf only exists in the smoke scratch project */
import assert from 'node:assert/strict'

const { createJobClient } = await import('@higgsfield-ai/fnf/client')
const jobs = await import('@higgsfield-ai/fnf/jobs')
const { createMediaClient } = await import('@higgsfield-ai/fnf/media')
const { createProfileClient } = await import('@higgsfield-ai/fnf/profile')
const { createConsoleObserver } = await import('@higgsfield-ai/fnf/observability')
const workflowPlatform = await import('@higgsfield-ai/fnf/workflow-platform')
const { errorFromJSON, ApiJobError } = await import('@higgsfield-ai/fnf/errors')
const root = await import('@higgsfield-ai/fnf')

assert.equal(typeof createJobClient, 'function', 'client subpath broken')
assert.equal(typeof createMediaClient, 'function', 'media subpath broken')
assert.equal(typeof createProfileClient, 'function', 'profile subpath broken')
assert.equal(typeof createConsoleObserver, 'function', 'observability subpath broken')
// The ONE bundled adapter (other adapters live in @higgsfield/fnf-adapters).
assert.equal(typeof workflowPlatform.createWorkflowPlatformAdapter, 'function', 'workflow-platform subpath broken')
assert.equal(typeof workflowPlatform.createFetchTransport, 'function', 'fetch transport missing from workflow-platform subpath')
assert.equal(root.createWorkflowPlatformAdapter, workflowPlatform.createWorkflowPlatformAdapter, 'workflow-platform adapter duplicated across entrypoints')
assert.equal(typeof errorFromJSON, 'function', 'errors subpath broken')
assert.equal(typeof root.createJobClient, 'function', 'root barrel broken')
assert.ok(Object.keys(jobs).length > 0, 'jobs subpath exported nothing')

// Class identity must be shared across subpaths — a per-entry copy of the
// error hierarchy would break instanceof after bundling.
assert.equal(root.ApiJobError, ApiJobError, 'ApiJobError duplicated across entrypoints')

// Media error codes are registered as an import side effect of the media
// module; if tree-shaking/bundling dropped them, rehydration falls back to
// the base class.
const rehydrated = errorFromJSON({ code: 'invalid_media_source', message: 'smoke' })
assert.equal(rehydrated.code, 'invalid_media_source', 'media error code lost in build')
assert.ok(rehydrated instanceof ApiJobError, 'rehydrated error lost its class hierarchy')

// Functional round-trip through an inline in-memory backend (the memory
// adapters live in @higgsfield/fnf-adapters, which this tarball must not
// need): cost, submit, wait to a completed result, and a bad setting rejected
// by the bundled zod pipeline.
const memoryJobs = new Map()
let seq = 0
const client = createJobClient({
  adapter: {
    async createJobs({ jobSetType, params }) {
      const id = `mem_${++seq}`
      memoryJobs.set(id, { id, job_set_type: jobSetType, status: 'completed', params, result_url: `memory://${jobSetType}/${id}.out`, created_at: seq })
      return [id]
    },
    async getJob(id) {
      return memoryJobs.get(id) ?? { id, status: 'failed', fail_reason: 'not found' }
    },
    async listJobs() {
      return { items: [...memoryJobs.values()], next_cursor: null }
    },
    async estimateCost() {
      return { credits: 500 }
    },
  },
  jobs: [jobs.nanoBanana2, jobs.seedance2_0],
})
```

```
const cost = await client.cost({
  model: 'nano_banana_2',
  prompt: { instruction: 'smoke lighthouse' },
  settings: { batchSize: 2 },
})
assert.equal(typeof cost.credits, 'number', 'cost estimate broken')

const { generations } = await client.submit({
  model: 'nano_banana_2',
  prompt: { instruction: 'smoke lighthouse' },
  settings: { batchSize: 1 },
})
assert.ok(generations.length >= 1, 'submit returned no generations')
const [done] = await client.wait(generations)
assert.equal(done.status, 'completed', `unexpected status: ${done.status}`)
assert.ok(done.results?.rawUrl?.startsWith('memory://'), `bad result url: ${done.results?.rawUrl}`)

const bad = await client.safeSubmit({
  model: 'seedance_2_0',
  prompt: { instruction: 'smoke' },
  settings: { duration: 999 },
})
assert.equal(bad.ok, false, 'validation should have rejected duration 999')
assert.equal(bad.error.code, 'validation', `expected validation error, got: ${bad.error.code}`)

console.log('smoke ok: subpaths import, single error hierarchy, media codes registered, memory-backend round-trip works')
```

app/packages/fnf/src/__tests__/define-job.test.ts

```
import { describe, expect, it } from 'vitest'
import { defineJob } from '../define-job'
import { buildRegistry } from '../registry'
import { z } from '../z'

const job = defineJob({
  jobSetType: 'demo_video',
  outputType: 'video',
  params: {
    media: { field: 'medias', format: 'wrapped', roles: ['start_image'] },
    settings: {
      seed: z.optional(z.number()),
      duration: z.duration({ values: [5, 10] }),
      aspectRatio: z.aspectRatio(['1:1', '16:9']),
      resolution: z._default(z.enum(['720p', '1080p']), '720p'),
    },
  },
  finalize: wire => ({ ...wire, use_chain: true }),
})

describe('defineJob', () => {
  it('keeps identity fields and media config', () => {
    expect(job.jobSetType).toBe('demo_video')
    expect(job.outputType).toBe('video')
    expect(job.media?.format).toBe('wrapped')
  })

  it('builds a settings schema that validates and applies defaults', () => {
    expect(job.settingsSchema.parse({ duration: 5, aspectRatio: '1:1' })).toEqual({
      duration: 5,
      aspectRatio: '1:1',
      resolution: '720p',
    })
  })

  it('collects normalizers for tagged settings fields only', () => {
    expect(job.normalizers).toEqual({
      duration: { kind: 'duration', values: [5, 10] },
      aspectRatio: { kind: 'aspectRatio', options: ['1:1', '16:9'] },
    })
  })

  it('exposes the finalize hook (last touch on the assembled wire body)', () => {
    expect(job.finalize?.({ existing: 1 }, { model: 'demo_video', settings: {} })).toEqual({ existing: 1, use_chain: true })
  })

  it('rejects single/unwrapped media with multiple roles', () => {
    expect(() => defineJob({
      jobSetType: 'bad',
      outputType: 'image',
      params: {
        media: { field: 'input_images', format: 'unwrapped', roles: ['image', 'audio'] },
        settings: {},
      },
    })).toThrow(/exactly one role/)
  })
})

describe('defineJob wire-collision guard', () => {
  it('rejects two settings mapping to one wire key', () => {
    expect(() => defineJob({
      jobSetType: 'clash',
      outputType: 'image',
      params: {
        settings: { a: z.wire('same_key', z.number()), b: z.wire('same_key', z.string()) },
      },
    })).toThrow(/both serialize to wire key 'same_key'/)
  })
```

```
  it('rejects a settings key shadowing a prompt/media wire key', () => {
    expect(() => defineJob({
      jobSetType: 'clash2',
      outputType: 'image',
      params: {
        prompt: true,
        settings: { prompt: z.string() }, // 'prompt' is the prompt group's wire key
      },
    })).toThrow(/collides with the prompt wire key/)
  })
})

describe('tagged schemas survive z.optional / z._default wrappers', () => {
  it('keeps the normalizer and wire name of a wrapped schema', () => {
    const wrapped = defineJob({
      jobSetType: 'wrapped',
      outputType: 'image',
      params: {
        settings: {
          aspectRatio: z.optional(z.wire('aspect_ratio', z.aspectRatio(['1:1', '16:9']))),
          duration: z._default(z.duration({ values: [5, 10] }), 5),
        },
      },
    })
    expect(wrapped.normalizers.aspectRatio).toEqual({ kind: 'aspectRatio', options: ['1:1', '16:9'] })
    expect(wrapped.normalizers.duration).toEqual({ kind: 'duration', values: [5, 10] })
    expect(wrapped.wireNames).toEqual({ aspectRatio: 'aspect_ratio' })
  })
})

describe('buildRegistry', () => {
  it('indexes jobs by jobSetType', () => {
    const registry = buildRegistry([job])
    expect(registry.get('demo_video')).toBe(job)
    expect(registry.get('missing')).toBeUndefined()
  })

  it('throws on a duplicate jobSetType instead of last-write-wins', () => {
    expect(() => buildRegistry([job, job])).toThrow(/duplicate jobSetType 'demo_video'/)
  })
})
```

app/packages/fnf/src/__tests__/errors.test.ts

```
import { describe, expect, it } from 'vitest'
import {
  AccountSuspendedError,
  ApiJobError,
  ApiMessageError,
  AutoTopUpSuspendedError,
  BatchRateLimitError,
  BillingError,
  errorFromJSON,
  errorFromResponse,
  GraceDailyLimitError,
  IpCheckRateLimitError,
  IpDetectedError,
  JobAbortedError,
  JobInProgressError,
  JobTimeoutError,
  KybVerificationRequiredError,
  MinimumPlanError,
  NotEnoughBoostCreditsError,
  OutOfCreditsError,
  PromptNsfwError,
  RateLimitError,
  UnknownSubmitResponseError,
  ValidationError,
  VpnDetectedError,
  WorkspaceMemberSpendPausedError,
  WorkspaceSelectionRequiredError,
} from '../errors'
import { MediaModerationError, UploadTransferError } from '../media/errors'

describe('errorFromResponse', () => {
  it('returns null for a successful response', () => {
    expect(errorFromResponse(200, { ok: true })).toBeNull()
  })

  it('maps not_enough_credits to OutOfCreditsError carrying the real HTTP status', () => {
    // Both backend not_enough_credits sites emit HTTP 403 (fnf-api
    // src/exceptions/auth.py:71-88, src/exceptions/workspace.py:267-290) —
    // the class must not pin 402 over it.
    const err = errorFromResponse(403, {
      detail: { error_type: 'not_enough_credits', plan_type: 'pro', billing_period: 'monthly', workspace: { type: 'shared', role: 'owner' } },
    })
    expect(err).toBeInstanceOf(OutOfCreditsError)
    expect(err?.code).toBe('out_of_credits')
    expect(err?.status).toBe(403)
    expect((err as OutOfCreditsError).data).toEqual({ plan: 'pro', billingPeriod: 'monthly', team: { workspaceType: 'Shared', userRole: 'Owner' } })

    // the stamped status survives a toJSON round-trip
    const round = errorFromJSON(JSON.parse(JSON.stringify(err!.toJSON())))
    expect(round).toBeInstanceOf(OutOfCreditsError)
    expect(round.status).toBe(403)
  })

  it('lowercases plan_type before casting (shared workspaces send "Team")', () => {
    // fnf-api src/exceptions/workspace.py:260,287 send TeamPlanFactory.team().name = 'Team'
    // (src/types_/team_subscription.py:149-153) — outside the lowercase union as-is.
    const err = errorFromResponse(403, {
      detail: { error_type: 'not_enough_credits', plan_type: 'Team', billing_period: 'monthly', workspace: { type: 'shared', role: 'member' } },
    })
    expect((err as OutOfCreditsError).data?.plan).toBe('team')
  })

  it('maps rate_limit_reached with maxCount + isUnlimited', () => {
    const err = errorFromResponse(429, { detail: { error_type: 'rate_limit_reached', plan_type: 'basic', concurrent_jobs_limit: 3, use_unlim: true } })
    expect(err).toBeInstanceOf(RateLimitError)
    expect((err as RateLimitError).data).toMatchObject({ plan: 'basic', maxCount: 3, isUnlimited: true })
  })
```

```
it('maps both nsfw (real wire type) and prompt_nsfw (alias) to PromptNsfwError', () => {
  expect(errorFromResponse(400, { detail: { error_type: 'nsfw' } })).toBeInstanceOf(PromptNsfwError)
  expect(errorFromResponse(400, { detail: { error_type: 'prompt_nsfw' } })).toBeInstanceOf(PromptNsfwError)
})

it('maps the 422 ip_detected error-body shape to IpDetectedError (not the ValidationError fallback)', () => {
  const err = errorFromResponse(422, { detail: { error_type: 'ip_detected' } })
  expect(err).toBeInstanceOf(IpDetectedError)
  expect(err?.code).toBe('ip_detected')
  expect(err?.status).toBe(422)
  expect(err?.message).toBe('Protected content is not allowed')
  // the backend's message wins when present
  expect(errorFromResponse(422, { detail: { error_type: 'ip_detected', message: 'IP found in media' } })?.message).toBe('IP found in media')
})

it('maps ip_check_rate_limit_reached (presign/confirm) to IpCheckRateLimitError', () => {
  const err = errorFromResponse(422, { detail: { error_type: 'ip_check_rate_limit_reached' } })
  expect(err).toBeInstanceOf(IpCheckRateLimitError)
  expect(err?.code).toBe('ip_check_rate_limit_reached')
  expect(err?.message).toMatch(/eligibility check limit/)
})

it('collapses the minimum-plan error_types into MinimumPlanError with plan/feature in data', () => {
  // Plans are the backend-named (legacy) values, matching the product's
  // native mapping (fnf-web src/entities/job/error/plan.ts).
  expect((errorFromResponse(402, { detail: { error_type: 'ultimate_plan_required' } }) as MinimumPlanError).data).toEqual({ plan: 'ultimate' })
  expect((errorFromResponse(402, { detail: { error_type: 'job_minimum_pro_plan_required' } }) as MinimumPlanError).data).toEqual({ plan: 'pro' })
  expect((errorFromResponse(402, { detail: { error_type: 'avatar_minimum_pro_plan_required' } }) as MinimumPlanError).data).toEqual({ plan: 'pro', feature: 'avatar' })
  expect((errorFromResponse(402, { detail: { error_type: 'job_minimum_creator_plan_required' } }) as MinimumPlanError).data).toEqual({ plan: 'creator' })
  expect((errorFromResponse(402, { detail: { error_type: 'train_minimum_basic_plan_required' } }) as MinimumPlanError).data).toEqual({ plan: 'basic', feature: 'train' })
})

it('maps the remaining typed billing/plan error_types', () => {
  expect(errorFromResponse(402, { detail: { error_type: 'non_eligible_vpn_detected' } })).toBeInstanceOf(VpnDetectedError)
  expect(errorFromResponse(402, { detail: { error_type: 'not_enough_boost_credits' } })).toBeInstanceOf(NotEnoughBoostCreditsError)
  expect(errorFromResponse(402, { detail: { error_type: 'batch_rate_limit_reached', concurrent_batches_limit: 2 } })).toBeInstanceOf(BatchRateLimitError)
  expect(errorFromResponse(402, { detail: { error_type: 'auto_top_up_suspended' } })).toBeInstanceOf(AutoTopUpSuspendedError)
  expect(errorFromResponse(402, { detail: { error_type: 'grace_daily_limit_reached', data: { type: 'pay_less_notice' } } })).toBeInstanceOf(GraceDailyLimitError)
  expect(errorFromResponse(402, { detail: { error_type: 'workspace_member_spend_paused', workspace_id: 'w1', user_id: 'u1' } })).toBeInstanceOf(WorkspaceMemberSpendPausedError)
  expect(errorFromResponse(402, { detail: { error_type: 'account_blocked' } })).toBeInstanceOf(AccountSuspendedError)
})

it('maps media_too_large to a base ApiJobError keeping status 413 and maxSizeMb', () => {
  // fnf-api src/utils/media_size.py:16-22 — HTTP 413, detail { error_type, max_size_mb }.
  const err = errorFromResponse(413, { detail: { error_type: 'media_too_large', max_size_mb: 50 } })
  expect(err).toBeInstanceOf(ApiJobError)
  expect(err?.code).toBe('media_too_large')
  expect(err?.status).toBe(413)
  expect(err?.data).toEqual({ maxSizeMb: 50 })

  const round = errorFromJSON(JSON.parse(JSON.stringify(err!.toJSON())))
  expect(round.code).toBe('media_too_large')
  expect(round.status).toBe(413)
  expect(round.data).toEqual({ maxSizeMb: 50 })
})

it('maps request_timeout (content-check timeout) with the backend text as message', () => {
  // fnf-api src/handlers/media.py:282-290 — HTTP 408, detail { error_type, text }.
  const text = 'Content check is taking longer than expected. Please try again in a moment.'
  const err = errorFromResponse(408, { detail: { error_type: 'request_timeout', text } })
  expect(err).toBeInstanceOf(ApiJobError)
  expect(err?.code).toBe('request_timeout')
  expect(err?.status).toBe(408)
  expect(err?.message).toBe(text)

  const round = errorFromJSON(JSON.parse(JSON.stringify(err!.toJSON())))
  expect(round.code).toBe('request_timeout')
  expect(round.message).toBe(text)
})
```

```
it('maps unlim_battery_rate_limit_reached with billing context + concurrentBatchesLimit', () => {
  // fnf-api src/exceptions/auth.py:145-170 — HTTP 429, detail carries
  // workspace/plan_type/billing_period/concurrent_batches_limit.
  const err = errorFromResponse(429, {
    detail: { error_type: 'unlim_battery_rate_limit_reached', plan_type: 'Team', billing_period: 'monthly', workspace: { type: 'shared', role: 'member' }, concurrent_batches_limit: 4 },
  })
  expect(err).toBeInstanceOf(ApiJobError)
  expect(err?.code).toBe('unlim_battery_rate_limit_reached')
  expect(err?.status).toBe(429)
  expect(err?.data).toEqual({ plan: 'team', billingPeriod: 'monthly', team: { workspaceType: 'Shared', userRole: 'Member' }, concurrentBatchesLimit: 4 })

  const round = errorFromJSON(JSON.parse(JSON.stringify(err!.toJSON())))
  expect(round.code).toBe('unlim_battery_rate_limit_reached')
  expect(round.data).toEqual(err?.data)
})

it('maps enterprise_group_credit_limit_reached with text + camelCased limit fields', () => {
  // fnf-api src/exceptions/credit_limit.py:30-56 — HTTP 429,
  // detail { text, limit_amount, period, spent_in_period, cost }.
  const text = 'You\'ve reached your weekly workspace group credit limit. Ask your workspace admin to increase the group limit or wait until it resets.'
  const err = errorFromResponse(429, {
    detail: { error_type: 'enterprise_group_credit_limit_reached', text, limit_amount: 1000, period: 'every_week', spent_in_period: 990, cost: 20 },
  })
  expect(err).toBeInstanceOf(ApiJobError)
  expect(err?.code).toBe('enterprise_group_credit_limit_reached')
  expect(err?.status).toBe(429)
  expect(err?.message).toBe(text)
  expect(err?.data).toEqual({ limitAmount: 1000, period: 'every_week', spentInPeriod: 990, cost: 20 })

  const round = errorFromJSON(JSON.parse(JSON.stringify(err!.toJSON())))
  expect(round.code).toBe('enterprise_group_credit_limit_reached')
  expect(round.message).toBe(text)
  expect(round.data).toEqual(err?.data)
})

it('maps the bare 403 gate codes to base ApiJobErrors preserving the wire code', () => {
  // fnf-api src/exceptions/job.py:70-75 and :86-91 — HTTP 403, bare { error_type }.
  for (const code of ['free_generation_not_allowed', 'generation_not_available']) {
    const err = errorFromResponse(403, { detail: { error_type: code } })
    expect(err).toBeInstanceOf(ApiJobError)
    expect(err?.code).toBe(code)
    expect(err?.status).toBe(403)

    const round = errorFromJSON(JSON.parse(JSON.stringify(err!.toJSON())))
    expect(round.code).toBe(code)
    expect(round.status).toBe(403)
    expect(round.message).toBe(err?.message)
  }
})

it('matches KYB by detail.message', () => {
  expect(errorFromResponse(403, { detail: { message: 'KYB verification required' } })).toBeInstanceOf(KybVerificationRequiredError)
})

it('maps 409 workspace_selection_required, carrying workspaces in data', () => {
  const err = errorFromResponse(409, { detail: { error_type: 'workspace_selection_required', workspaces: [{ id: 'w1', name: null, type: 'private' }] } })
  expect(err).toBeInstanceOf(WorkspaceSelectionRequiredError)
  expect((err as WorkspaceSelectionRequiredError).data?.workspaces).toEqual([{ id: 'w1', name: null, type: 'private' }])
})

it('maps 409 job_in_progress (by error_type) to JobInProgressError', () => {
  const err = errorFromResponse(409, { detail: { error_type: 'job_in_progress' } })
  expect(err).toBeInstanceOf(JobInProgressError)
  expect(err?.status).toBe(409)
})

it('maps a bare 409 (no error_type) to UnknownSubmitResponseError, not JobInProgressError', () => {
  const err = errorFromResponse(409, { some: 'body' })
  expect(err).toBeInstanceOf(UnknownSubmitResponseError)
  expect(err?.status).toBe(409)
```

```
    expect((err as UnknownSubmitResponseError).data?.responseBody).toEqual({ some: 'body' })
  })

  it('maps 422 to ValidationError with the first detail message', () => {
    const err = errorFromResponse(422, { detail: [{ msg: 'prompt too long' }] })
    expect(err).toBeInstanceOf(ValidationError)
    expect(err?.message).toBe('prompt too long')
    expect(err?.status).toBe(422)
  })

  it('maps a string detail to ApiMessageError', () => {
    const err = errorFromResponse(400, { detail: 'something went wrong' })
    expect(err).toBeInstanceOf(ApiMessageError)
    expect(err?.message).toBe('something went wrong')
  })

  it('falls back to UnknownSubmitResponseError (code unknown) for unmapped failures', () => {
    const err = errorFromResponse(500, { detail: { weird: true } })
    expect(err).toBeInstanceOf(UnknownSubmitResponseError)
    expect(err?.code).toBe('unknown')
    expect(err?.status).toBe(500)
  })

  it('every produced error is a ApiJobError', () => {
    for (const type of ['not_enough_credits', 'rate_limit_reached', 'nsfw', 'non_eligible_vpn_detected', 'workspace_selection_required']) {
      expect(errorFromResponse(402, { detail: { error_type: type } })).toBeInstanceOf(ApiJobError)
    }
  })
})

describe('toJSON + errorFromJSON round-trip', () => {
  it('emits a uniform { code, message, status?, data? } shape', () => {
    // No pinned status — OutOfCreditsError carries whatever errorFromResponse
    // stamped (the backend emits 403; the class itself stays status-agnostic).
    const err = new OutOfCreditsError({ plan: 'pro', billingPeriod: 'monthly' })
    expect(err.toJSON()).toEqual({ code: 'out_of_credits', message: 'Not enough credits', data: { plan: 'pro', billingPeriod: 'monthly' } })
  })

  it('rehydrates the right class + data from JSON (survives a Comlink boundary)', () => {
    const cases = [
      new OutOfCreditsError({ plan: 'pro', team: { workspaceType: 'Shared', userRole: 'Member' } }),
      new RateLimitError({ plan: 'basic', maxCount: 5, isUnlimited: false }),
      new MinimumPlanError({ plan: 'plus', feature: 'avatar' }),
      new BatchRateLimitError(4),
      new AccountSuspendedError('account_blocked'),
      new VpnDetectedError(),
      new WorkspaceSelectionRequiredError([{ id: 'w1', name: 'Team', type: 'shared' }]),
    ]
    for (const original of cases) {
      const round = errorFromJSON(JSON.parse(JSON.stringify(original.toJSON())))
      expect(round).toBeInstanceOf(original.constructor as new (...a: any[]) => ApiJobError)
      expect(round.code).toBe(original.code)
      expect(round.data).toEqual(original.data)
    }
  })

  it('rehydrates ip_detected and ip_check_rate_limit_reached, preserving the message', () => {
    const ip = errorFromJSON(JSON.parse(JSON.stringify(new IpDetectedError('IP found in media').toJSON())))
    expect(ip).toBeInstanceOf(IpDetectedError)
    expect(ip.code).toBe('ip_detected')
    expect(ip.message).toBe('IP found in media')

    const limit = errorFromJSON(JSON.parse(JSON.stringify(new IpCheckRateLimitError().toJSON())))
    expect(limit).toBeInstanceOf(IpCheckRateLimitError)
    expect(limit.code).toBe('ip_check_rate_limit_reached')
    expect(limit.message).toMatch(/eligibility check limit/)
  })

  it('rehydrates code unknown to UnknownSubmitResponseError with status/body/url intact', () => {
    const original = new UnknownSubmitResponseError(500, { weird: true }, 'https://api/jobs/demo')
```

```
    const round = errorFromJSON(JSON.parse(JSON.stringify(original.toJSON())))
    expect(UnknownSubmitResponseError.is(round)).toBe(true) // .is() works again after the round-trip
    expect(round.status).toBe(500)
    expect((round as UnknownSubmitResponseError).data?.responseBody).toEqual({ weird: true })
    expect((round as UnknownSubmitResponseError).data?.responseUrl).toBe('https://api/jobs/demo')
  })

  it('rehydrates an unknown code to a generic ApiJobError', () => {
    const round = errorFromJSON({ code: 'some_future_code', message: 'hi', data: { x: 1 } })
    expect(round).toBeInstanceOf(ApiJobError)
    expect(round.code).toBe('some_future_code')
    expect(round.data).toEqual({ x: 1 })
  })

  it('rehydrates the client-lifecycle codes, preserving the timeout message and generation', () => {
    const aborted = errorFromJSON(new JobAbortedError().toJSON())
    expect(aborted).toBeInstanceOf(JobAbortedError)

    const gen = { id: 'j1', model: 'demo', type: 'image', status: 'in_progress', input: { model: 'demo', settings: {} } } as const
    const original = new JobTimeoutError('j1', 60_000, gen as never)
    const round = errorFromJSON(JSON.parse(JSON.stringify(original.toJSON())))
    expect(round).toBeInstanceOf(JobTimeoutError)
    expect(round.message).toBe(original.message) // not rebuilt as "within 0ms"
    expect((round as JobTimeoutError).data?.generation?.id).toBe('j1')
  })

  it('rehydrates media-pipeline errors to their classes (registered from the media half)', () => {
    const moderation = errorFromJSON(JSON.parse(JSON.stringify(new MediaModerationError('nsfw').toJSON())))
    expect(moderation).toBeInstanceOf(MediaModerationError)
    expect(moderation.code).toBe('media_moderation_blocked')
    expect((moderation as MediaModerationError).data?.status).toBe('nsfw')

    const transfer = errorFromJSON(new UploadTransferError('Upload failed (503)', 503).toJSON())
    expect(transfer).toBeInstanceOf(UploadTransferError)
    expect(transfer.status).toBe(503)
  })

  it('does not crash on a data-less payload for any registered code', () => {
    // A different SDK version (or hand-built payload) may legitimately omit data.
    for (const code of ['minimum_plan_required', 'out_of_credits', 'rate_limit', 'workspace_subscription_inactive']) {
      expect(() => errorFromJSON({ code, message: 'x' })).not.toThrow()
    }
  })

  it('preserves the serialized status when the factory drops it', () => {
    const round = errorFromJSON({ code: 'rate_limit', message: 'x', status: 429 })
    expect(round.status).toBe(429)
  })
})

describe('errorFromResponse stamps the HTTP status', () => {
  it('mapped errors without a hard-coded status carry the response status', () => {
    const err = errorFromResponse(429, { detail: { error_type: 'rate_limit_reached' } })
    expect(err?.code).toBe('rate_limit')
    expect(err?.status).toBe(429)

    const auth = errorFromResponse(401, { detail: 'Not authenticated' })
    expect(auth?.code).toBe('api_message')
    expect(auth?.status).toBe(401)
  })

  it('keeps the deliberately hard-coded statuses', () => {
    const err = errorFromResponse(400, { detail: { error_type: 'job_in_progress' } })
    expect(err?.status).toBe(409) // JobInProgressError pins 409
  })

  it('stamps the real status on out_of_credits instead of a pinned 402', () => {
    // Backend emits 403 at both not_enough_credits sites (fnf-api
    // src/exceptions/auth.py:71-88, src/exceptions/workspace.py:267-290).
    const err = errorFromResponse(403, { detail: { error_type: 'not_enough_credits' } })
```

```
    expect(err?.status).toBe(403)
  })
})

describe('.is() type-guard sugar', () => {
  const oc = new OutOfCreditsError({ plan: 'pro' })

  it('narrows by leaf, mid-tier, and base', () => {
    expect(OutOfCreditsError.is(oc)).toBe(true)
    expect(RateLimitError.is(oc)).toBe(false)
    expect(BillingError.is(oc)).toBe(true) // mid-tier groups all billing errors
    expect(ApiJobError.is(oc)).toBe(true)
  })

  it('returns false for non-errors and plain JSON', () => {
    expect(OutOfCreditsError.is(new Error('x'))).toBe(false)
    expect(OutOfCreditsError.is(oc.toJSON())).toBe(false) // serialized form is not an instance
    expect(ApiJobError.is(undefined)).toBe(false)
  })
})
```

app/packages/fnf/src/__tests__/group.test.ts

```
import { describe, expect, it } from 'vitest'
import { field, group } from '../group'

const codec = group({
  instruction: field('prompt'),
  enhance: field('enhance_prompt'),
})

describe('group codec', () => {
  it('serializes structured keys to wire keys, dropping undefined', () => {
    expect(codec.serialize({ instruction: 'cat', enhance: true })).toEqual({
      prompt: 'cat',
      enhance_prompt: true,
    })
    expect(codec.serialize({ instruction: 'cat' })).toEqual({ prompt: 'cat' })
  })

  it('parses wire keys back to structured keys', () => {
    expect(codec.parse({ prompt: 'cat', enhance_prompt: true })).toEqual({
      instruction: 'cat',
      enhance: true,
    })
  })

  it('exposes the wire keys it claims', () => {
    expect(codec.wireKeys.sort()).toEqual(['enhance_prompt', 'prompt'])
  })

  it('round-trips: parse(serialize(x)) === x', () => {
    const x = { instruction: 'dog', enhance: false }
    expect(codec.parse(codec.serialize(x))).toEqual(x)
  })
})
```

app/packages/fnf/src/__tests__/llm-client.test.ts

```
import type { Transport } from '../transport'
import { describe, expect, it, vi } from 'vitest'
import { createLlmClient } from '../llm/client'

function completionBody(content: string) {
  return {
    id: 'chatcmpl-1',
    object: 'chat.completion',
    model: 'claude-sonnet-4-6',
    choices: [{ index: 0, message: { role: 'assistant', content, tool_calls: [{ id: 't1', type: 'function', function: { name: 'f', arguments: '{}' } }] }, finish_reason: 'stop' }],
    usage: { prompt_tokens: 10, completion_tokens: 2, total_tokens: 12 },
  }
}

describe('createLlmClient.complete', () => {
  it('shapes the request and normalizes the completion', async () => {
    const transport = vi.fn<Transport>(async () => ({ status: 200, body: completionBody('hi') }))
    const client = createLlmClient({ baseUrl: 'https://gw.example', transport })

    const res = await client.complete({
      model: 'claude-sonnet-4-6',
      messages: [{ role: 'user', content: 'hello' }],
      maxTokens: 64,
      temperature: 0.2,
      toolChoice: { name: 'f' },
      tools: [{ name: 'f', parameters: { type: 'object' } }],
    })

    const sent = transport.mock.calls[0][0]
    expect(sent.method).toBe('POST')
    expect(sent.path).toBe('/v1/chat/completions')
    const body = sent.body as Record<string, unknown>
    expect(body.model).toBe('claude-sonnet-4-6')
    expect(body.stream).toBeUndefined()
    expect(body.max_tokens).toBe(64)
    expect(body.temperature).toBe(0.2)
    expect(body.tool_choice).toEqual({ type: 'function', function: { name: 'f' } })
    expect(body.tools).toEqual([{ type: 'function', function: { name: 'f', description: undefined, parameters: { type: 'object' } } }])

    expect(res.content).toBe('hi')
    expect(res.toolCalls).toEqual([{ id: 't1', name: 'f', arguments: '{}' }])
    expect(res.finishReason).toBe('stop')
    expect(res.usage).toEqual({ promptTokens: 10, completionTokens: 2 })
  })

  it('maps replayed assistant tool_calls to the OpenAI wire shape', async () => {
    const transport = vi.fn<Transport>(async () => ({ status: 200, body: completionBody('ok') }))
    const client = createLlmClient({ baseUrl: 'https://gw.example', transport })

    await client.complete({
      model: 'm',
      messages: [
        { role: 'assistant', content: '', tool_calls: [{ id: 't1', name: 'f', arguments: '{"a":1}' }] },
        { role: 'tool', content: 'result', tool_call_id: 't1' },
      ],
    })

    const body = transport.mock.calls[0][0].body as { messages: Array<Record<string, unknown>> }
    expect(body.messages[0].tool_calls).toEqual([{ id: 't1', type: 'function', function: { name: 'f', arguments: '{"a":1}' } }])
    expect(body.messages[1]).toEqual({ role: 'tool', content: 'result', tool_call_id: 't1' })
  })

  it('throws the mapped catalog error on 402', async () => {
    const transport: Transport = async () => ({ status: 402, body: { error: { message: 'no credits', code: 'insufficient_credits' } } })
    const client = createLlmClient({ baseUrl: 'https://gw.example', transport })
    await expect(client.complete({ model: 'm', messages: [{ role: 'user', content: 'x' }] }))
      .rejects
      .toMatchObject({ code: 'out_of_credits', status: 402 })
```

```
  })
})

describe('createLlmClient.listModels', () => {
  it('unwraps the OpenAI list shape', async () => {
    const transport: Transport = async () => ({ status: 200, body: { object: 'list', data: [{ id: 'a' }, { id: 'b' }] } })
    const client = createLlmClient({ baseUrl: 'https://gw.example', transport })
    expect(await client.listModels()).toEqual(['a', 'b'])
  })
})

function sseResponse(events: string[]): Response {
  const stream = new ReadableStream<Uint8Array>({
    start(controller) {
      for (const e of events)
        controller.enqueue(new TextEncoder().encode(`data: ${e}\n\n`))
      controller.close()
    },
  })
  return new Response(stream, { status: 200, headers: { 'Content-Type': 'text/event-stream' } })
}

/**
 * Enqueue raw byte chunks verbatim — lets a test control exactly where the
 * transport boundaries fall (mid-frame splits, multiple frames per read),
 * which `sseResponse` (one frame per chunk) cannot.
 */
function rawByteResponse(chunks: string[]): Response {
  const stream = new ReadableStream<Uint8Array>({
    start(controller) {
      for (const c of chunks)
        controller.enqueue(new TextEncoder().encode(c))
      controller.close()
    },
  })
  return new Response(stream, { status: 200, headers: { 'Content-Type': 'text/event-stream' } })
}

describe('createLlmClient.stream', () => {
  it('yields deltas and the final usage chunk, then completes', async () => {
    const chunks = [
      JSON.stringify({ choices: [{ index: 0, delta: { content: 'he' }, finish_reason: null }] }),
      JSON.stringify({ choices: [{ index: 0, delta: { tool_calls: [{ index: 0, id: 'c1', function: { name: 'f', arguments: '' } }] }, finish_reason: null }] }),
      JSON.stringify({ choices: [{ index: 0, delta: { tool_calls: [{ index: 0, function: { arguments: '{"a":1}' } }] }, finish_reason: null }] }),
      JSON.stringify({ choices: [{ index: 0, delta: {}, finish_reason: 'stop' }], usage: { prompt_tokens: 5, completion_tokens: 1 } }),
      '[DONE]',
    ]
    const fetchMock = vi.fn(async () => sseResponse(chunks))
    const client = createLlmClient({ baseUrl: 'https://gw.example', fetch: fetchMock, getToken: async () => 'tok' })

    const seen: unknown[] = []
    for await (const d of client.stream({ model: 'm', messages: [{ role: 'user', content: 'x' }] }))
      seen.push(d)

    expect(seen[0]).toEqual({ content: 'he' })
    expect(seen[1]).toEqual({ toolCallDelta: { index: 0, id: 'c1', name: 'f', argumentsDelta: '' } })
    expect(seen[2]).toEqual({ toolCallDelta: { index: 0, argumentsDelta: '{"a":1}' } })
    expect(seen[3]).toEqual({ finishReason: 'stop', usage: { promptTokens: 5, completionTokens: 1 } })

    const [url, init] = fetchMock.mock.calls[0] as unknown as [string, RequestInit]
    expect(url).toBe('https://gw.example/v1/chat/completions')
    expect(new Headers(init.headers).get('Authorization')).toBe('Bearer tok')
    expect(JSON.parse(init.body as string).stream).toBe(true)
  })

  it('throws the mapped error when the stream endpoint rejects', async () => {
    const fetchMock = vi.fn(async () => new Response(JSON.stringify({ error: { message: 'nope', code: 'rate_limited' } }), { status: 429 }))
    const client = createLlmClient({ baseUrl: 'https://gw.example', fetch: fetchMock })
    const iterate = async () => {
      for await (const _d of client.stream({ model: 'm', messages: [{ role: 'user', content: 'x' }] })) { /* drain */ }
```

```
    }
    await expect(iterate()).rejects.toMatchObject({ code: 'rate_limit', status: 429 })
  })

  it('throws the mapped error on a MID-stream error event, after earlier deltas', async () => {
    const chunks = [
      JSON.stringify({ choices: [{ index: 0, delta: { content: 'he' }, finish_reason: null }] }),
      JSON.stringify({ error: { message: 'stream blew up', code: 'upstream_error' } }),
    ]
    const fetchMock = vi.fn(async () => sseResponse(chunks))
    const client = createLlmClient({ baseUrl: 'https://gw.example', fetch: fetchMock })

    const seen: unknown[] = []
    const iterate = async () => {
      for await (const d of client.stream({ model: 'm', messages: [{ role: 'user', content: 'x' }] }))
        seen.push(d)
    }
    await expect(iterate()).rejects.toMatchObject({ code: 'upstream_error', message: 'stream blew up' })
    expect(seen).toEqual([{ content: 'he' }])
  })

  it('aborts via the request signal', async () => {
    const fetchMock = vi.fn(async (_url: unknown, init?: RequestInit) => {
      const signal = init?.signal as AbortSignal
      return new Response(new ReadableStream<Uint8Array>({
        start(controller) {
          controller.enqueue(new TextEncoder().encode(`data: ${JSON.stringify({ choices: [{ index: 0, delta: { content: 'x' }, finish_reason: null }] })}\n\n`))
          signal.addEventListener('abort', () => controller.error(new DOMException('aborted', 'AbortError')))
        },
      }), { status: 200 })
    })
    const client = createLlmClient({ baseUrl: 'https://gw.example', fetch: fetchMock })
    const ac = new AbortController()
    const iterate = async () => {
      for await (const d of client.stream({ model: 'm', messages: [{ role: 'user', content: 'x' }], signal: ac.signal })) {
        if (d.content === 'x')
          ac.abort()
      }
    }
    await expect(iterate()).rejects.toThrow(/abort/i)
  })

  it('reassembles a frame split across read() boundaries', async () => {
    // The `data: {json}\n\n` frame arrives in two reads mid-payload — the
    // reader must buffer across reads and parse exactly one delta.
    const frame = `data: ${JSON.stringify({ choices: [{ index: 0, delta: { content: 'split' }, finish_reason: null }] })}\n\n`
    const cut = 10 // mid-JSON, well before the `\n\n`
    const fetchMock = vi.fn(async () => rawByteResponse([frame.slice(0, cut), frame.slice(cut)]))
    const client = createLlmClient({ baseUrl: 'https://gw.example', fetch: fetchMock })

    const seen: unknown[] = []
    for await (const d of client.stream({ model: 'm', messages: [{ role: 'user', content: 'x' }] }))
      seen.push(d)

    expect(seen).toEqual([{ content: 'split' }])
  })

  it('parses two complete frames packed into a single read(), in order', async () => {
    const one = `data: ${JSON.stringify({ choices: [{ index: 0, delta: { content: 'a' }, finish_reason: null }] })}\n\n`
    const two = `data: ${JSON.stringify({ choices: [{ index: 0, delta: { content: 'b' }, finish_reason: null }] })}\n\n`
    const fetchMock = vi.fn(async () => rawByteResponse([one + two])) // both frames in ONE chunk
    const client = createLlmClient({ baseUrl: 'https://gw.example', fetch: fetchMock })

    const seen: unknown[] = []
    for await (const d of client.stream({ model: 'm', messages: [{ role: 'user', content: 'x' }] }))
      seen.push(d)

    expect(seen).toEqual([{ content: 'a' }, { content: 'b' }])
  })
})
```

```
describe('createLlmClient.stream observability', () => {
  it('emits a start event before the fetch', async () => {
    const events: Array<{ name: string, phase: string }> = []
    const fetchMock = vi.fn(async () => sseResponse([JSON.stringify({ choices: [{ index: 0, delta: { content: 'hi' }, finish_reason: null }] }), '[DONE]']))
    const client = createLlmClient({
      baseUrl: 'https://gw.example',
      fetch: fetchMock,
      observability: { observer: e => void events.push({ name: e.name, phase: e.phase }) },
    })

    for await (const _d of client.stream({ model: 'm', messages: [{ role: 'user', content: 'x' }] })) { /* drain */ }

    expect(events).toContainEqual({ name: 'fnf.llm.stream', phase: 'start' })
    // Lazy iterator: deliberately no success/duration span (see stream() JSDoc).
    expect(events.some(e => e.phase === 'success')).toBe(false)
  })

  it('emits an error event on a pre-stream HTTP rejection', async () => {
    const events: Array<{ phase: string, code?: string }> = []
    const fetchMock = vi.fn(async () => new Response(JSON.stringify({ error: { message: 'nope', code: 'rate_limited' } }), { status: 429 }))
    const client = createLlmClient({
      baseUrl: 'https://gw.example',
      fetch: fetchMock,
      observability: { observer: e => void events.push({ phase: e.phase, code: e.error?.code }) },
    })
    const iterate = async () => {
      for await (const _d of client.stream({ model: 'm', messages: [{ role: 'user', content: 'x' }] })) { /* drain */ }
    }
    await expect(iterate()).rejects.toMatchObject({ code: 'rate_limit' })
    expect(events).toContainEqual({ phase: 'error', code: 'rate_limit' })
  })

  it('emits an error event on a mid-stream error event', async () => {
    const events: Array<{ phase: string, code?: string }> = []
    const chunks = [
      JSON.stringify({ choices: [{ index: 0, delta: { content: 'he' }, finish_reason: null }] }),
      JSON.stringify({ error: { message: 'stream blew up', code: 'upstream_error' } }),
    ]
    const fetchMock = vi.fn(async () => sseResponse(chunks))
    const client = createLlmClient({
      baseUrl: 'https://gw.example',
      fetch: fetchMock,
      observability: { observer: e => void events.push({ phase: e.phase, code: e.error?.code }) },
    })
    const iterate = async () => {
      for await (const _d of client.stream({ model: 'm', messages: [{ role: 'user', content: 'x' }] })) { /* drain */ }
    }
    await expect(iterate()).rejects.toMatchObject({ code: 'upstream_error' })
    expect(events).toContainEqual({ phase: 'error', code: 'upstream_error' })
  })
})
```

app/packages/fnf/src/__tests__/llm-errors.test.ts

```
import { describe, expect, it } from 'vitest'
import { ApiJobError, OutOfCreditsError, RateLimitError } from '../errors'
import { errorFromLlmResponse, stampLlmStatus } from '../llm/errors'

describe('errorFromLlmResponse', () => {
  it('maps 402 to the existing out_of_credits error, gateway detail on data', () => {
    const err = errorFromLlmResponse(402, { error: { message: 'not enough credits', type: 'billing_error', code: 'insufficient_credits' } })
    expect(err).toBeInstanceOf(OutOfCreditsError)
    expect(err!.status).toBe(402)
    // Canonical class message stays; the gateway's verbatim detail rides data.
    expect(err!.message).toBe('Not enough credits')
    expect((err as OutOfCreditsError).data).toMatchObject({
      gateway: { message: 'not enough credits', code: 'insufficient_credits' },
    })
  })

  it('maps 429 to the existing rate_limit error, gateway detail on data', () => {
    const err = errorFromLlmResponse(429, { error: { message: 'too many requests', type: 'rate_limit_error', code: 'rate_limited' } })
    expect(err).toBeInstanceOf(RateLimitError)
    expect((err as RateLimitError).data).toMatchObject({
      gateway: { message: 'too many requests', code: 'rate_limited' },
    })
  })

  it('never overwrites a class-pinned status (stamp fills null only)', () => {
    class PinnedStubError extends ApiJobError {
      constructor() { super('pinned_stub', 'pinned', { status: 409 }) }
    }
    expect(stampLlmStatus(new PinnedStubError(), 502).status).toBe(409)
    // and the fill half of the contract, for symmetry
    expect(stampLlmStatus(new ApiJobError('unpinned', 'x'), 502).status).toBe(502)
  })

  it('carries the gateway code for other errors', () => {
    const err = errorFromLlmResponse(404, { error: { message: 'model x not available', type: 'invalid_request_error', code: 'model_not_found' } })
    expect(err!.code).toBe('model_not_found')
    expect(err!.status).toBe(404)
    expect(err!.message).toContain('model x')
  })

  it('survives a non-OpenAI-shaped body', () => {
    const err = errorFromLlmResponse(502, 'bad gateway')
    expect(err!.code).toBe('llm_gateway_error')
    expect(err!.status).toBe(502)
  })

  it('returns null on 2xx', () => {
    expect(errorFromLlmResponse(200, {})).toBeNull()
  })
})
```

app/packages/fnf/src/__tests__/normalize.test.ts

```
import { describe, expect, it } from 'vitest'
import { type AdjustKind, clampDuration, closestAspectRatio, normalizeSettings } from '../normalize'

describe('closestAspectRatio', () => {
  it('returns the value unchanged when it is an allowed option', () => {
    expect(closestAspectRatio('16:9', ['1:1', '16:9', '9:16'])).toBe('16:9')
  })

  it('returns the closest option by aspect when not allowed', () => {
    expect(closestAspectRatio('1920:1081', ['1:1', '16:9', '9:16'])).toBe('16:9')
    expect(closestAspectRatio('100:101', ['1:1', '16:9'])).toBe('1:1')
  })
})

describe('clampDuration', () => {
  it('snaps to the nearest allowed value', () => {
    expect(clampDuration(7, { kind: 'duration', values: [5, 10] })).toBe(5)
    expect(clampDuration(8, { kind: 'duration', values: [5, 10] })).toBe(10)
  })

  it('clamps to a min/max range', () => {
    expect(clampDuration(2, { kind: 'duration', min: 4, max: 15 })).toBe(4)
    expect(clampDuration(20, { kind: 'duration', min: 4, max: 15 })).toBe(15)
    expect(clampDuration(8, { kind: 'duration', min: 4, max: 15 })).toBe(8)
  })
})

describe('normalizeSettings', () => {
  const ALL = new Set<AdjustKind>(['near-aspect-ratio', 'near-duration'])

  it('snaps enabled kinds and records the adjustments', () => {
    const out = normalizeSettings(
      { aspectRatio: '1920:1081', duration: 7, resolution: '720p' },
      {
        aspectRatio: { kind: 'aspectRatio', options: ['1:1', '16:9'] },
        duration: { kind: 'duration', values: [5, 10] },
      },
      ALL,
    )
    expect(out.settings).toEqual({ aspectRatio: '16:9', duration: 5, resolution: '720p' })
    expect(out.adjustments).toEqual([
      { field: 'aspectRatio', from: '1920:1081', to: '16:9' },
      { field: 'duration', from: 7, to: 5 },
    ])
  })

  it('leaves a kind untouched when it is not enabled (passthrough)', () => {
    const out = normalizeSettings(
      { aspectRatio: '1920:1081', duration: 7 },
      {
        aspectRatio: { kind: 'aspectRatio', options: ['1:1', '16:9'] },
        duration: { kind: 'duration', values: [5, 10] },
      },
      new Set<AdjustKind>(['near-aspect-ratio']), // duration not enabled
    )
    expect(out.settings).toEqual({ aspectRatio: '16:9', duration: 7 })
    expect(out.adjustments).toEqual([{ field: 'aspectRatio', from: '1920:1081', to: '16:9' }])
  })

  it('skips a normalizer when the value is absent', () => {
    const out = normalizeSettings(
      { resolution: '720p' },
      { aspectRatio: { kind: 'aspectRatio', options: ['1:1'] } },
      ALL,
    )
    expect(out.settings).toEqual({ resolution: '720p' })
    expect(out.adjustments).toEqual([])
  })
```

```
})
```

app/packages/fnf/src/__tests__/selectors.test.ts

```
import type { Generation } from '../types'
import { describe, expect, it } from 'vitest'
import { seedance2_0 } from '../jobs/seedance-2-0'
import { getJobPhase, getMediaType, getPreviewUrl, getRawUrl, hasResult, isCompleted, isFailed, isFailedJobStatus, isFromJob, isGenerating, isTerminalJobStatus } from '../selectors'

function gen(overrides: Partial<Generation>): Generation {
  return { id: 'j1', model: 'demo', type: 'image', status: 'completed', input: { model: 'demo', settings: {} }, ...overrides }
}

describe('url selectors', () => {
  it('getRawUrl returns the full-quality url, getPreviewUrl prefers min → thumbnail → raw', () => {
    const full = gen({ results: { rawUrl: 'https://x/raw.png', minUrl: 'https://x/min.webp' } })
    expect(getRawUrl(full)).toBe('https://x/raw.png')
    expect(getPreviewUrl(full)).toBe('https://x/min.webp')

    const video = gen({ type: 'video', results: { rawUrl: 'https://x/o.mp4', thumbnailUrl: 'https://x/t.webp' } })
    expect(getPreviewUrl(video)).toBe('https://x/t.webp')

    expect(getPreviewUrl(gen({ results: { rawUrl: 'https://x/only.png' } }))).toBe('https://x/only.png')
    expect(getPreviewUrl(gen({}))).toBeUndefined()
  })
})

describe('status selectors', () => {
  it('buckets statuses into progress / failed / completed', () => {
    expect(getJobPhase('queued')).toBe('progress')
    expect(getJobPhase('in_progress')).toBe('progress')
    expect(getJobPhase('ip_detect')).toBe('progress')
    expect(getJobPhase('completed')).toBe('completed')
    for (const status of ['failed', 'nsfw', 'canceled', 'ip_detected'] as const)
      expect(getJobPhase(status)).toBe('failed')
    // unknown future statuses are still in flight, not failures
    expect(getJobPhase('warming_up' as never)).toBe('progress')
    expect(getJobPhase(gen({ status: 'failed' }))).toBe('failed')
  })

  it('isTerminalJobStatus / isFailedJobStatus', () => {
    expect(isTerminalJobStatus('completed')).toBe(true)
    expect(isTerminalJobStatus('nsfw')).toBe(true)
    expect(isTerminalJobStatus('queued')).toBe(false)
    expect(isFailedJobStatus('completed')).toBe(false)
    expect(isFailedJobStatus('canceled')).toBe(true)
  })
})

describe('predicates', () => {
  it('isCompleted / isFailed / isGenerating branch on the read model directly', () => {
    expect(isCompleted(gen({}))).toBe(true)
    expect(isFailed(gen({ status: 'nsfw' }))).toBe(true)
    expect(isGenerating(gen({ status: 'in_progress' }))).toBe(true)
    expect(isGenerating(gen({ status: 'failed' }))).toBe(false)
  })

  it('hasResult narrows results to non-optional', () => {
    const done = gen({ results: { rawUrl: 'https://x/o.png' } })
    expect(hasResult(done)).toBe(true)
    if (hasResult(done))
      expect(done.results.rawUrl).toBe('https://x/o.png') // no ?. needed — narrowed
    expect(hasResult(gen({}))).toBe(false)
  })

  it('isFromJob narrows input to the model typed shape', () => {
    const seedance = gen({ model: 'seedance_2_0' })
    expect(isFromJob(seedance, seedance2_0)).toBe(true)
    if (isFromJob(seedance, seedance2_0)) {
      // typed access: duration is the declared settings field, not unknown
      const duration: number | undefined = seedance.input.settings.duration
      expect(duration).toBeUndefined()
```

```
    }
    expect(isFromJob(gen({ model: 'demo' }), seedance2_0)).toBe(false)
  })
})

describe('getMediaType', () => {
  it('answers from a Generation, a MediaRef url, or a bare url; undefined otherwise', () => {
    expect(getMediaType(gen({ type: 'video' }))).toBe('video')
    expect(getMediaType('https://cdn/x/clip.MP4?sig=1')).toBe('video')
    expect(getMediaType('https://cdn/x/pic.webp#frag')).toBe('image')
    expect(getMediaType({ id: 'm1', type: 'media_input', url: 'https://cdn/a.heic' })).toBe('image')
    expect(getMediaType({ id: 'm1', type: 'media_input' })).toBeUndefined()
    expect(getMediaType('https://cdn/x/file.bin')).toBeUndefined()
    expect(getMediaType(undefined)).toBeUndefined()
  })
})
```

app/packages/fnf/src/__tests__/spec.test.ts

```
import { describe, expect, it } from 'vitest'
import { defineJob } from '../define-job'
import { buildWireParams, parseGeneration } from '../spec'
import { z } from '../z'

const job = defineJob({
  jobSetType: 'seedance_2_0',
  outputType: 'video',
  params: {
    prompt: true,
    media: { field: 'medias', format: 'wrapped', roles: ['start_image'] },
    settings: {
      duration: z.duration({ values: [5, 10] }),
      aspectRatio: z.aspectRatio(['1:1', '16:9']),
      resolution: z._default(z.enum(['720p', '1080p']), '720p'),
    },
  },
  finalize: wire => ({ ...wire, use_chain: true }),
})

describe('buildWireParams', () => {
  it('flattens prompt, media, settings (no normalization), finalize and extra', () => {
    const wire = buildWireParams(
      {
        model: 'seedance_2_0',
        prompt: { instruction: 'a cat', enhance: true },
        media: { start_image: [{ id: 'a', type: 'media_input', url: 'https://x/a' }] },
        settings: { duration: 7, aspectRatio: '1920:1081' },
        extra: { sample_shift: 0.3 },
      },
      job,
    )
    // settings pass through untouched — submit does not snap; `adjust()` does.
    expect(wire).toEqual({
      prompt: 'a cat',
      enhance_prompt: true,
      medias: [{ role: 'start_image', data: { id: 'a', type: 'media_input', url: 'https://x/a' } }],
      duration: 7,
      aspectRatio: '1920:1081',
      resolution: '720p',
      sample_shift: 0.3,
      use_chain: true,
    })
  })
})

describe('parseGeneration', () => {
  it('parses a job response into structured Generation, routing unknown keys to extra', () => {
    const gen = parseGeneration(
      {
        id: 'job-1',
        job_set_id: 'set-1',
        status: 'completed',
        result_url: 'https://x/out.mp4',
        params: {
          prompt: 'a cat',
          enhance_prompt: true,
          medias: [{ role: 'start_image', data: { id: 'a', type: 'media_input', url: 'https://x/a' } }],
          duration: 5,
          aspectRatio: '16:9',
          resolution: '720p',
          sample_shift: 0.3,
        },
      },
      job,
    )
    expect(gen.id).toBe('job-1')
    expect(gen.jobSetId).toBe('set-1')
```

```
    expect(gen.type).toBe('video')
    expect(gen.status).toBe('completed')
    expect(gen.input.prompt).toEqual({ instruction: 'a cat', enhance: true })
    expect(gen.input.settings).toEqual({ duration: 5, aspectRatio: '16:9', resolution: '720p' })
    expect(gen.input.extra).toEqual({ sample_shift: 0.3 })
    expect(gen.results?.rawUrl).toBe('https://x/out.mp4')
  })

  it('round-trips: buildWireParams(parseGeneration(job).input) reproduces params', () => {
    const params = {
      prompt: 'dog',
      enhance_prompt: false,
      medias: [{ role: 'start_image', data: { id: 'b', type: 'media_input', url: 'https://x/b' } }],
      duration: 10,
      aspectRatio: '1:1',
      resolution: '1080p',
      sample_shift: 0.5,
    }
    const gen = parseGeneration({ id: 'j', status: 'queued', params }, job)
    // finalize (use_chain) is deterministically injected on every submit; round-trip
    // losslessness covers prompt/media/settings/extra, plus the job's finalize fields.
    expect(buildWireParams(gen.input, job)).toEqual({ ...params, use_chain: true })
  })
})
```

app/packages/fnf/src/__tests__/types.test.ts

```
import { describe, expect, it } from 'vitest'
import { isTerminal, TERMINAL_STATUSES } from '../types'

describe('status helpers', () => {
  it('marks completed/failed/nsfw/canceled/ip_detected as terminal', () => {
    expect(isTerminal('completed')).toBe(true)
    expect(isTerminal('failed')).toBe(true)
    expect(isTerminal('nsfw')).toBe(true)
    expect(isTerminal('canceled')).toBe(true)
    expect(isTerminal('ip_detected')).toBe(true)
  })

  it('marks in-flight statuses as not terminal', () => {
    expect(isTerminal('pending')).toBe(false)
    expect(isTerminal('queued')).toBe(false)
    expect(isTerminal('in_progress')).toBe(false)
  })

  it('exposes exactly five terminal statuses', () => {
    expect(TERMINAL_STATUSES.size).toBe(5)
  })
})
```

app/packages/fnf/src/__tests__/z.test.ts

```
import { describe, expect, it } from 'vitest'
import { aspectRatio, duration, getNormalize, z } from '../z'

describe('z helpers', () => {
  it('re-exports zod/mini primitives', () => {
    expect(typeof z.object).toBe('function')
    expect(typeof z.string).toBe('function')
    expect(typeof z.number).toBe('function')
  })

  it('aspectRatio validates against its options and carries a normalize tag', () => {
    const schema = aspectRatio(['1:1', '16:9'])
    expect(schema.parse('16:9')).toBe('16:9')
    expect(getNormalize(schema)).toEqual({ kind: 'aspectRatio', options: ['1:1', '16:9'] })
  })

  it('duration validates a number and carries a normalize tag with values', () => {
    const schema = duration({ values: [5, 10] })
    expect(schema.parse(10)).toBe(10)
    expect(getNormalize(schema)).toEqual({ kind: 'duration', values: [5, 10] })
  })

  it('duration supports a min/max range tag', () => {
    const schema = duration({ min: 4, max: 15 })
    expect(getNormalize(schema)).toEqual({ kind: 'duration', min: 4, max: 15 })
  })
})
```

app/packages/fnf/src/backend.ts

```
/**
 * The transport-agnostic ports the client cores depend on. Operations are
 * expressed as intent ("create these jobs", "get this media") — NOT as HTTP
 * requests. An HTTP/REST adapter (`createDevFnfWebAdapter`)
 * is one implementation; a websocket, a different service, or in-process code
 * can implement the same port without the core knowing or caring.
 *
 * Domain ports stay independent so SDK entry points bundle independently:
 * jobs, media, profile, reusable Elements, and custom-reference character training.
 *
 * Each method resolves the raw response payload, or throws an `ApiJobError`
 * (the adapter maps its own failures — HTTP status codes via `errorFromResponse`,
 * socket errors, etc. — onto the typed error catalog).
 */

import type { OutputType } from './types'
import type {
  CreateRealtimeCustomStyleWireRequest,
  RealtimeChainCostWireRequest,
  RealtimeChainEditWireRequest,
  RealtimeChainFinalizeWireRequest,
  RealtimeCustomStyleListQuery,
  UpdateRealtimeCustomStyleWireRequest,
} from './realtime/types'

export interface JobListQuery {
  // Derive from OutputType (not a duplicated literal) so this port stays in sync
  // with the public `ListOptions.type` (also OutputType). Hardcoding the union
  // here drifts the moment OutputType grows (e.g. "audio"), breaking the
  // ListOptions -> JobListQuery assignment in client/list.ts with TS2322.
  type?: OutputType
  cursor?: string | number
  size?: number
  /** List only the derived children of this job set (e.g. its upscales). */
  parentId?: string
  /** Only jobs in these wire statuses (e.g. 'queued', 'completed'). Repeatable. */
  status?: string | string[]
  /** Only these job set types (registry `jobSetType` strings). Repeatable. */
  model?: string | string[]
}

export interface MediaListQuery {
  type: 'image' | 'video' | 'audio'
  cursor?: string | number
  size?: number
}

export interface MediaGetQuery {
  id: string
  type: 'image' | 'video' | 'audio'
}

export interface SwitchWorkspaceRequest {
  workspaceId: string
}

/**
 * What the host's confirmation gate sees before a generation goes out: the
 * resolved job type and the final wire params (validation has already passed).
 */
export interface ConfirmSubmitRequest {
  jobSetType: string
  params: Record<string, unknown>
}

/**
 * Host-injected confirmation gate, run by `submit` AFTER validation/wire-
 * building and BEFORE any network call. Confirmed `count > 1` fan-out is
```

```
 * rejected before this gate because confirmation tokens are single-use.
 * Resolve to proceed —
 * optionally with an opaque confirmation token, which the adapter forwards on
 * the create request (`confirmationToken`). Reject (or throw) to abort the
 * submit; non-`ApiJobError` rejections surface as the typed
 * `confirmation_rejected` error. Embedded apps should delegate to their host's
 * approval API and preserve its token; standalone hosts may own their own modal.
 */
export type ConfirmSubmit = (req: ConfirmSubmitRequest) => Promise<string | void>

/**
 * THE combined adapter shape for a full fnf backend — one object that satisfies
 * the core job/media/profile ports and can optionally expose Elements and
 * character training.
 * Concrete implementations live in `@higgsfield/fnf-adapters`.
 */
export interface FnfAdapter extends GenerationBackend, MediaBackend, ProfileBackend, Partial<ReferenceBackend>, Partial<CharacterBackend>, Partial<RealtimeBackend> {}

/** Starts supported custom-reference character training and reads its lifecycle. */
export interface CharacterBackend {
  createCharacter: (req: {
    name: string
    type: 'soul' | 'soul_2' | 'soul_cinematic'
    input_images: Array<{ id: string, type: string }>
  }) => Promise<unknown>
  getCharacter: (id: string) => Promise<unknown>
}

/** The authenticated user's reusable Elements library. */
export interface ReferenceBackend {
  getReferenceElement: (id: string) => Promise<unknown>
  listReferenceElements: (query: { cursor?: number, size?: number, category?: string }) => Promise<unknown>
}

/** Stateful Flux Klein edit chains and the authenticated user's custom styles. */
export interface RealtimeBackend {
  editRealtimeChain: (req: RealtimeChainEditWireRequest) => Promise<unknown>
  estimateRealtimeChainCost: (req: RealtimeChainCostWireRequest) => Promise<unknown>
  finalizeRealtimeChain: (req: RealtimeChainFinalizeWireRequest) => Promise<unknown>
  listRealtimeCustomStyles: (query: RealtimeCustomStyleListQuery) => Promise<unknown>
  createRealtimeCustomStyle: (req: CreateRealtimeCustomStyleWireRequest) => Promise<unknown>
  updateRealtimeCustomStyle: (styleId: string, req: UpdateRealtimeCustomStyleWireRequest) => Promise<unknown>
  deleteRealtimeCustomStyle: (styleId: string) => Promise<unknown>
}

/** The jobs port: create/read/list generations and estimate cost. */
export interface GenerationBackend {
  createJobs: (req: { jobSetType: string, params: Record<string, unknown>, confirmationToken?: string }) => Promise<unknown>
  getJob: (id: string) => Promise<unknown>
  /**
   * OPTIONAL — fetch ALL jobs of a job set in one request, normalized to the
   * same shape as `getJob`. When present, `wait`/`poll` group generations by
   * `jobSetId` and poll per SET instead of per job — one request per tick for
   * a whole batch — and gate fields (the fnf `ip_check_finished` IP gate) are
   * seen every tick even on adapters whose per-job read lacks them (the
   * fnf-web adapter carries the gate on both reads).
   */
  getJobSet?: (id: string) => Promise<unknown>
  listJobs: (query: JobListQuery) => Promise<unknown>
  estimateCost: (req: { jobSetType: string, params: Record<string, unknown> }) => Promise<unknown>
  /**
   * OPTIONAL — cancel a running job server-side (the client-side `signal` only
   * stops polling; the backend job keeps burning credits without this). Adapters
   * whose backend has no cancel route omit it; `cancelGeneration` then throws
   * `cancel_not_supported`.
   */
  cancelJob?: (id: string) => Promise<unknown>
  /**
   * OPTIONAL — the host-injected confirmation gate (see `ConfirmSubmit`).
   * Adapters don't implement this themselves; they surface the callback the
```

```
   * host passed at construction (`confirm` option). Absent → submits proceed
   * unconfirmed.
   */
  confirm?: ConfirmSubmit
}

export interface UploadUrlRequest {
  type: 'image' | 'video' | 'audio'
  filename?: string
  contentType?: string
  extra?: Record<string, unknown>
}

export interface ConfirmMediaRequest {
  mediaId: string
  type: 'image' | 'video' | 'audio'
  filename?: string
  jobId?: string
  forceIpCheck?: boolean
  forceNsfwCheck?: boolean
  startSeconds?: number
  endSeconds?: number
  extra?: Record<string, unknown>
}

/**
 * The media port: get/list media, plus the JSON control plane of uploads —
 * `getUploadUrl` (presign) and `confirmMedia`. These two are OPTIONAL so a
 * read-only media adapter need not implement them; `uploadMedia` throws
 * `UploadNotSupportedError` when they're absent. The binary transfer itself is
 * NOT a method here — it's raw bytes to a storage host (not JSON), handled by a
 * separate injected `BinaryUploader`.
 */
export interface MediaBackend {
  getMedia: (query: MediaGetQuery) => Promise<unknown>
  listMedia: (query: MediaListQuery) => Promise<unknown>
  getUploadUrl?: (req: UploadUrlRequest) => Promise<unknown>
  confirmMedia?: (req: ConfirmMediaRequest) => Promise<unknown>
}

/**
 * The profile port: account/workspace/wallet reads plus backend workspace
 * context switching. Host apps that also mirror workspace choice into an
 * identity provider (Clerk unsafeMetadata/session reload in fnf-web) should do
 * that outside this SDK port.
 */
export interface ProfileBackend {
  getUser: () => Promise<unknown>
  listWorkspaces: () => Promise<unknown>
  getCurrentWorkspace: () => Promise<unknown>
  getWorkspaceWallet: () => Promise<unknown>
  switchWorkspace: (req: SwitchWorkspaceRequest) => Promise<unknown>
}
```

app/packages/fnf/src/characters/__tests__/characters.test.ts

```
import type { CharacterBackend } from '../../backend'
import type { MediaRef } from '../../types'
import { describe, expect, it, vi } from 'vitest'
import { createCharacterClient } from '../client'
import { buildCharacterCreateRequest, parseCharacter } from '../codec'

function image(index: number): MediaRef {
  return {
    id: `00000000-0000-4000-8000-${String(index).padStart(12, '0')}`,
    type: 'media_input',
    url: `https://private.example/${index}.jpg`,
  }
}

function jobImage(index: number): MediaRef {
  return {
    id: `00000000-0000-4000-8001-${String(index).padStart(12, '0')}`,
    type: 'image_job',
  }
}

function adapter(overrides: Partial<CharacterBackend> = {}): CharacterBackend {
  return {
    createCharacter: vi.fn(async req => ({
      id: '11111111-1111-4111-8111-111111111111',
      type: req.type,
      status: 'queued',
      name: 'Ada',
    })),
    getCharacter: vi.fn(),
    ...overrides,
  }
}

const input = {
  name: ' Ada ',
  images: Array.from({ length: 5 }, (_, index) => image(index + 1)),
}

describe('custom-reference character training workflow', () => {
  it('defaults to Soul 2 and removes media URLs from the wire request', () => {
    expect(buildCharacterCreateRequest(input)).toEqual({
      name: 'Ada',
      type: 'soul_2',
      input_images: input.images.map(({ id }) => ({ id, type: 'media_input' })),
    })
  })

  it.each(['soul', 'soul_2', 'soul_cinematic'] as const)('supports the %s custom-reference type', (type) => {
    expect(buildCharacterCreateRequest({ ...input, type })).toMatchObject({ type })
  })

  it.each(['soul_2', 'soul_cinematic'] as const)('preserves job image references for %s', (type) => {
    const images = [jobImage(1)]
    expect(buildCharacterCreateRequest({ type, images })).toEqual({
      name: 'Untitled character',
      type,
      input_images: images.map(({ id }) => ({ id, type: 'image_job' })),
    })
  })

  it('uses a sensible default name', () => {
    expect(buildCharacterCreateRequest({ images: input.images })).toMatchObject({
      name: 'Untitled character',
    })
  })

  it('creates a character and parses its custom-reference lifecycle record', async () => {
```

```
  const backend = adapter()
  const client = createCharacterClient({ adapter: backend })

  await expect(client.create(input)).resolves.toEqual({
    id: '11111111-1111-4111-8111-111111111111',
    name: 'Ada',
    type: 'soul_2',
    status: 'queued',
  })
  expect(backend.createCharacter).toHaveBeenCalledWith(buildCharacterCreateRequest(input))
})

it('maps the linked Element id when the backend provides it', () => {
  expect(parseCharacter({
    id: '11111111-1111-4111-8111-111111111111',
    element_id: '22222222-2222-4222-8222-222222222222',
    type: 'soul_cinematic',
    status: 'completed',
  })).toMatchObject({
    id: '11111111-1111-4111-8111-111111111111',
    elementId: '22222222-2222-4222-8222-222222222222',
    type: 'soul_cinematic',
    status: 'completed',
  })
})

it('polls the character training record until it completes', async () => {
  const statuses = ['in_progress', 'completed']
  const backend = adapter({
    getCharacter: vi.fn(async () => ({
      id: '11111111-1111-4111-8111-111111111111',
      type: 'soul_2',
      status: statuses.shift(),
    })),
  })
  const client = createCharacterClient({ adapter: backend, scheduler: { sleep: async () => {} } })
  const completed = await client.wait({
    id: '11111111-1111-4111-8111-111111111111',
    type: 'soul_2',
    status: 'queued',
  })

  expect(completed.status).toBe('completed')
  expect(backend.getCharacter).toHaveBeenCalledTimes(2)
})

it.each([
  ['no images', []],
  ['more than one hundred', Array.from({ length: 101 }, (_, index) => image(index))],
  ['a non-image media reference', [{ id: 'video-1', type: 'video_input' }]],
])('rejects %s before any request', async (_label, images) => {
  const backend = adapter()
  const client = createCharacterClient({ adapter: backend })

  await expect(client.create({ images })).rejects.toMatchObject({ code: 'validation' })
  expect(backend.createCharacter).not.toHaveBeenCalled()
})

it('rejects job image references for the original Soul type', async () => {
  const backend = adapter()
  const client = createCharacterClient({ adapter: backend })

  await expect(client.create({ type: 'soul', images: [jobImage(1)] }))
    .rejects.toMatchObject({ code: 'validation' })
  expect(backend.createCharacter).not.toHaveBeenCalled()
})

it('rejects a name longer than one hundred characters before any request', async () => {
  const backend = adapter()
  const client = createCharacterClient({ adapter: backend })
```

```
    await expect(client.create({ name: 'a'.repeat(101), images: input.images }))
      .rejects.toMatchObject({ code: 'validation' })
    expect(backend.createCharacter).not.toHaveBeenCalled()
  })
})
```

app/packages/fnf/src/characters/client.ts

```
import type { CharacterBackend } from '../backend'
import type { FnfObservabilityOptions } from '../observability'
import type { Character, CharacterWaitOptions, CreateCharacterInput } from './types'
import { ApiJobError, JobAbortedError, JobTimeoutError } from '../errors'
import { createObservabilityContext, observeAsync } from '../observability'
import { buildCharacterCreateRequest, parseCharacter } from './codec'

export interface CharacterClientConfig {
  adapter: CharacterBackend
  poll?: { intervalMs?: number, timeoutMs?: number }
  scheduler?: { sleep?: (milliseconds: number) => Promise<void> }
  observability?: FnfObservabilityOptions
}

export interface CharacterClient {
  /** Trains a supported custom-reference character. The backend immediately creates its processing Character Element. */
  create: (input: CreateCharacterInput) => Promise<Character>
  get: (id: string) => Promise<Character>
  wait: (character: Character, options?: CharacterWaitOptions) => Promise<Character>
}

const DEFAULT_INTERVAL_MS = 2000
const DEFAULT_TIMEOUT_MS = 600_000

export function createCharacterClient(config: CharacterClientConfig): CharacterClient {
  const observability = createObservabilityContext(config.observability)
  const intervalMs = config.poll?.intervalMs ?? DEFAULT_INTERVAL_MS
  const timeoutMs = config.poll?.timeoutMs ?? DEFAULT_TIMEOUT_MS
  const sleep = config.scheduler?.sleep ?? (milliseconds => new Promise(resolve => setTimeout(resolve, milliseconds)))

  const get = (id: string): Promise<Character> => observeAsync(
    observability,
    'fnf.character.get',
    { character_id: id },
    async () => {
      if (!config.adapter.getCharacter)
        throw new ApiJobError('not_supported', 'Character reads are not supported by this adapter')
      return parseCharacter(await config.adapter.getCharacter(id))
    },
    { successAttributes: character => ({ character_id: character.id, status: character.status }) },
  )

  return {
    create: input => observeAsync(observability, 'fnf.character.create', {}, async () => {
      if (!config.adapter.createCharacter)
        throw new ApiJobError('not_supported', 'Character creation is not supported by this adapter')
      return parseCharacter(await config.adapter.createCharacter(buildCharacterCreateRequest(input)))
    }, { successAttributes: character => ({ character_id: character.id, status: character.status }) }),
    get,
    wait: async (character, options = {}) => {
      if (character.status === 'completed' || character.status === 'failed')
        return character
      const startedAt = Date.now()
      const waitIntervalMs = options.intervalMs ?? intervalMs
      const waitTimeoutMs = options.timeoutMs ?? timeoutMs
      let current = character
      while (Date.now() - startedAt < waitTimeoutMs) {
        if (options.signal?.aborted)
          throw new JobAbortedError()
        await sleep(waitIntervalMs)
        if (options.signal?.aborted)
          throw new JobAbortedError()
        current = await get(character.id)
        options.onProgress?.(current)
        if (current.status === 'completed' || current.status === 'failed')
          return current
      }
      throw new JobTimeoutError(character.id, waitTimeoutMs)
```

```
    },
  }
}
```

app/packages/fnf/src/characters/codec.ts

```
import type { Character, CharacterCreateRequest, CharacterStatus, CharacterType, CreateCharacterInput } from './types'
import { ValidationError } from '../errors'

const MIN_CHARACTER_IMAGES = 1
const MAX_CHARACTER_IMAGES = 100
const CHARACTER_TYPES = new Set<CharacterType>(['soul', 'soul_2', 'soul_cinematic'])

function isCharacterImageType(type: unknown): type is string {
  return typeof type === 'string' && (type === 'media_input' || type.endsWith('_job'))
}

export function buildCharacterCreateRequest(input: CreateCharacterInput): CharacterCreateRequest {
  const issues: Array<{ loc: string[], msg: string }> = []
  const name = input.name?.trim()
  const type = input.type ?? 'soul_2'

  if (!CHARACTER_TYPES.has(type))
    issues.push({ loc: ['type'], msg: 'type must be soul, soul_2, or soul_cinematic' })

  if (name && name.length > 100)
    issues.push({ loc: ['name'], msg: 'name must be at most 100 characters' })

  if (!Array.isArray(input.images) || input.images.length < MIN_CHARACTER_IMAGES || input.images.length > MAX_CHARACTER_IMAGES) {
    issues.push({ loc: ['images'], msg: `images must contain between ${MIN_CHARACTER_IMAGES} and ${MAX_CHARACTER_IMAGES} items` })
  }
  else {
    input.images.forEach((image, index) => {
      if (!image || typeof image.id !== 'string' || image.id.trim() === '')
        issues.push({ loc: ['images', String(index), 'id'], msg: 'image id is required' })
      if (!image || !isCharacterImageType(image.type))
        issues.push({ loc: ['images', String(index), 'type'], msg: 'character images must be image references' })
      else if (type === 'soul' && image.type !== 'media_input')
        issues.push({ loc: ['images', String(index), 'type'], msg: 'soul character images must be uploaded image references' })
    })
  }

  if (issues.length > 0)
    throw new ValidationError('Invalid character input', issues)

  return {
    name: name || 'Untitled character',
    type,
    input_images: input.images.map(image => ({ id: image.id, type: image.type })),
  }
}

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value)
}

function optionalString(value: unknown): string | undefined {
  return typeof value === 'string' && value.length > 0 ? value : undefined
}

function normalizeStatus(value: unknown): CharacterStatus {
  if (value === 'completed' || value === 'showcase_completed')
    return 'completed'
  if (value === 'failed')
    return 'failed'
  if (value === 'in_progress' || value === 'training')
    return 'in_progress'
  return 'queued'
}

export function parseCharacter(body: unknown): Character {
  const value = isRecord(body) && isRecord(body.data) ? body.data : body
  if (!isRecord(value))
    throw new ValidationError('Invalid character response')
```

```
  const id = optionalString(value.id) ?? optionalString(value.id_)
  if (!id)
    throw new ValidationError('Invalid character response', [{ loc: ['id'], msg: 'character id is required' }])

  const elementId = optionalString(value.element_id)
  const name = optionalString(value.name)
  const failReason = optionalString(value.fail_reason)
  const thumbnailUrl = optionalString(value.thumbnail_url)
  const type = optionalString(value.type)
  if (!type || !CHARACTER_TYPES.has(type as CharacterType))
    throw new ValidationError('Invalid character response', [{ loc: ['type'], msg: 'unsupported character type' }])

  return {
    id,
    type: type as CharacterType,
    status: normalizeStatus(value.status),
    ...(elementId ? { elementId } : {}),
    ...(name ? { name } : {}),
    ...(failReason ? { failReason } : {}),
    ...(thumbnailUrl ? { thumbnailUrl } : {}),
    ...(typeof value.created_at === 'number' ? { createdAt: value.created_at } : {}),
  }
}
```

app/packages/fnf/src/characters/index.ts

```
export { createCharacterClient } from './client'
export type { CharacterClient, CharacterClientConfig } from './client'
export { buildCharacterCreateRequest, parseCharacter } from './codec'
export type { Character, CharacterCreateRequest, CharacterStatus, CharacterType, CharacterWaitOptions, CreateCharacterInput } from './types'
```

app/packages/fnf/src/characters/types.ts

```
import type { MediaRef } from '../types'

export type CharacterStatus = 'queued' | 'in_progress' | 'completed' | 'failed'
export type CharacterType = 'soul' | 'soul_2' | 'soul_cinematic'

export interface Character {
  /** Custom-reference id used by compatible Soul generation models. */
  id: string
  /** Linked Character Element id when the backend includes it. */
  elementId?: string
  name?: string
  type: CharacterType
  status: CharacterStatus
  failReason?: string
  thumbnailUrl?: string
  createdAt?: number
}

export interface CreateCharacterInput {
  name?: string
  /** Defaults to `soul_2`. */
  type?: CharacterType
  images: MediaRef[]
}

export interface CharacterCreateRequest {
  name: string
  type: CharacterType
  input_images: Array<{ id: string, type: string }>
}

export interface CharacterWaitOptions {
  signal?: AbortSignal
  intervalMs?: number
  timeoutMs?: number
  onProgress?: (character: Character) => void
}
```

app/packages/fnf/src/client/__tests__/submit.test.ts

```
import type { GenerationBackend } from '../../backend'
import { describe, expect, it, vi } from 'vitest'
import { defineJob } from '../../define-job'
import { createJobClient } from '../index'

const demo = defineJob({
  jobSetType: 'demo',
  outputType: 'image',
  params: { settings: {} },
})

function backend(confirm?: GenerationBackend['confirm']): GenerationBackend {
  let id = 0
  return {
    createJobs: vi.fn(async () => [`job-${++id}`]),
    getJob: vi.fn(async jobId => ({ id: jobId, status: 'completed' })),
    listJobs: vi.fn(async () => ({ items: [] })),
    estimateCost: vi.fn(async () => ({ credits: 1 })),
    ...(confirm ? { confirm } : {}),
  }
}

describe('submit confirmation fan-out', () => {
  it('rejects confirmed count fan-out before opening approval or creating jobs', async () => {
    const confirm = vi.fn(async () => 'single-use-token')
    const adapter = backend(confirm)
    const client = createJobClient({ adapter, jobs: [demo] })

    await expect(client.submit({ model: 'demo', settings: {}, count: 2 })).rejects.toMatchObject({
      code: 'validation',
      message: expect.stringContaining('settings.batchSize'),
    })
    expect(confirm).not.toHaveBeenCalled()
    expect(adapter.createJobs).not.toHaveBeenCalled()
  })

  it('preserves count fan-out for direct adapters without confirmation', async () => {
    const adapter = backend()
    const client = createJobClient({ adapter, jobs: [demo] })

    const result = await client.submit({ model: 'demo', settings: {}, count: 2 })

    expect(result.generations.map(generation => generation.id)).toEqual(['job-1', 'job-2'])
    expect(adapter.createJobs).toHaveBeenCalledTimes(2)
  })

  it('passes one confirmed request its opaque token unchanged', async () => {
    const confirm = vi.fn(async () => 'single-use-token')
    const adapter = backend(confirm)
    const client = createJobClient({ adapter, jobs: [demo] })

    await client.submit({ model: 'demo', settings: {} })

    expect(confirm).toHaveBeenCalledOnce()
    expect(adapter.createJobs).toHaveBeenCalledWith(expect.objectContaining({
      confirmationToken: 'single-use-token',
    }))
  })
})
```

app/packages/fnf/src/client/adjust.ts

```
import type { AdjustKind, Adjustment } from '../normalize'
import type { GenerationInput } from '../types'
import type { GenerationContext } from './context'
import { normalizeSettings } from '../normalize'
import { entryFor } from './context'

/** Result of the standalone `adjust()` step: a normalized input plus the changes made. */
export interface AdjustResult<Input> {
  input: Input
  adjustments: Adjustment[]
}

/**
 * Snap the requested setting kinds (e.g. `'near-aspect-ratio'`) to their nearest
 * allowed values, returning a NEW input plus the list of changes. This is the
 * one place that normalizes — `submit()` deliberately does not, so callers opt in
 * explicitly (UI preview, agent canonicalization) and the submit path stays pure.
 *
 *   const { input, adjustments } = adjust(ctx, raw, ['near-aspect-ratio'])
 *   await submit(ctx, input)
 */
export function adjust(ctx: GenerationContext, input: GenerationInput, kinds: readonly AdjustKind[]): AdjustResult<GenerationInput> {
  const entry = entryFor(ctx, input.model)
  const { settings, adjustments } = normalizeSettings(
    (input.settings ?? {}) as Record<string, unknown>,
    entry.normalizers,
    new Set(kinds),
  )
  return { input: { ...input, settings }, adjustments }
}
```

app/packages/fnf/src/client/cancel.ts

```
import type { GenerationContext } from './context'
import { ApiJobError } from '../errors'
import { observeAsync } from '../observability'

/**
 * Cancel a running job SERVER-SIDE. This is the counterpart to the client-side
 * `signal` on poll/wait: aborting only stops polling — the backend job keeps
 * running (and burning credits) until cancelled here.
 *
 * Requires the adapter to implement the optional `cancelJob` port method;
 * otherwise throws the typed `cancel_not_supported` error.
 */
export async function cancelGeneration(ctx: GenerationContext, id: string): Promise<void> {
  return observeAsync(ctx.observability, 'fnf.job.cancel', { generation_id: id }, async () => {
    if (!ctx.adapter.cancelJob)
      throw new ApiJobError('cancel_not_supported', 'This backend adapter does not support job cancellation')
    await ctx.adapter.cancelJob(id)
  })
}
```

app/packages/fnf/src/client/context.ts

```
import type { GenerationBackend } from '../backend'
import type { JobEntry } from '../define-job'
import type { FnfObservabilityContext, FnfObservabilityOptions } from '../observability'
import type { Registry } from '../registry'
import { ApiJobError } from '../errors'
import { createObservabilityContext } from '../observability'
import { buildRegistry } from '../registry'

export interface ClientConfig<Jobs extends readonly JobEntry[] = readonly JobEntry[]> {
  /** The transport-agnostic jobs adapter. Use one from `@higgsfield/fnf-adapters`, or your own. */
  adapter: GenerationBackend
  /** Registered jobs — the source of `model`/`settings` autocomplete on `submit`. */
  jobs: Jobs
  poll?: { intervalMs?: number, timeoutMs?: number }
  scheduler?: { sleep?: (ms: number) => Promise<void>, isActive?: () => boolean }
  observability?: FnfObservabilityOptions
}

/**
 * The single shared dependency every job operation needs. Build one with
 * `createContext(config)` and pass it to any operation — you do not need the
 * full client to call `submit(ctx, …)` or `listGenerations(ctx, …)`.
 */
export interface GenerationContext {
  adapter: GenerationBackend
  registry: Registry
  poll: { intervalMs: number, timeoutMs: number }
  scheduler: { sleep: (ms: number) => Promise<void>, isActive?: () => boolean }
  observability: FnfObservabilityContext
}

const DEFAULT_INTERVAL_MS = 2000
const DEFAULT_TIMEOUT_MS = 600_000

/** Resolve user config into the shared context every operation consumes. */
export function createContext(config: ClientConfig<readonly JobEntry[]>): GenerationContext {
  return {
    adapter: config.adapter,
    registry: buildRegistry(config.jobs),
    poll: {
      intervalMs: config.poll?.intervalMs ?? DEFAULT_INTERVAL_MS,
      timeoutMs: config.poll?.timeoutMs ?? DEFAULT_TIMEOUT_MS,
    },
    scheduler: {
      sleep: config.scheduler?.sleep ?? defaultSleep,
      isActive: config.scheduler?.isActive,
    },
    observability: createObservabilityContext(config.observability),
  }
}

export function entryFor(ctx: GenerationContext, model: string): JobEntry {
  const entry = ctx.registry.get(model)
  if (!entry)
    throw new ApiJobError('unknown_model', `Unknown model: ${model}`)
  return entry
}

function defaultSleep(ms: number): Promise<void> {
  return new Promise(resolve => setTimeout(resolve, ms))
}
```

app/packages/fnf/src/client/cost.ts

```
import type { GenerationInput } from '../types'
import type { GenerationContext } from './context'
import { observeAsync } from '../observability'
import { buildWireParams, parseSettings } from '../spec'
import { entryFor } from './context'

export interface CostEstimate {
  credits: number
}

export async function estimateCost(ctx: GenerationContext, input: GenerationInput): Promise<CostEstimate> {
  return observeAsync(ctx.observability, 'fnf.job.cost', { model: input.model }, async () => {
    const entry = entryFor(ctx, input.model)
    // Local per-model calculator first (the fnf-web pattern) — instant price for
    // UI previews; the backend is only asked when the model can't price locally.
    // Settings are parsed first so the calculator prices what would actually be
    // submitted (defaults applied, garbage rejected) — same input the backend
    // path prices via buildWireParams.
    if (entry.credits) {
      const local = entry.credits({ ...input, settings: parseSettings(input, entry) })
      if (typeof local === 'number')
        return { credits: local }
    }
    const params = buildWireParams(input, entry)
    const body = (await ctx.adapter.estimateCost({ jobSetType: entry.jobSetType, params }) ?? {}) as {
      credits?: number
      credits_exact?: number
    }
    return { credits: body.credits ?? body.credits_exact ?? 0 }
  }, {
    successAttributes: estimate => ({ credits: estimate.credits }),
  })
}
```

app/packages/fnf/src/client/get-set.ts

```
import type { JobResponse } from '../spec'
import type { Generation } from '../types'
import type { GenerationContext } from './context'
import { ApiJobError } from '../errors'
import { observeAsync } from '../observability'
import { parseGeneration } from '../spec'

/**
 * Fetch ALL jobs of one job set in a single request — the one-shot read
 * counterpart of `pollJobSetGroup`'s tick (same adapter route, same gate
 * semantics, no loop). For callers that schedule reads themselves — a query
 * cache's `refetchInterval`, a realtime-triggered re-read — instead of
 * holding the SDK's poll loop open.
 *
 * Requires an adapter with `getJobSet`; throws the typed `not_supported`
 * otherwise (same contract as `pollJobSetGroup`).
 */
export async function getJobSetGenerations(ctx: GenerationContext, jobSetId: string): Promise<Generation[]> {
  return observeAsync(ctx.observability, 'fnf.job.get_set', { job_set_id: jobSetId }, async () => {
    const getJobSet = ctx.adapter.getJobSet
    if (!getJobSet)
      throw new ApiJobError('not_supported', 'getSet requires an adapter with getJobSet')
    const body = await getJobSet(jobSetId) as (JobResponse & { job_set_type?: string })[]
    return (Array.isArray(body) ? body : []).map((job) => {
      const entry = job.job_set_type ? ctx.registry.get(job.job_set_type) : undefined
      // Fail fast like the singles path (`getGeneration`): an unresolvable type
      // is a local configuration error.
      if (!entry)
        throw new ApiJobError('unknown_model', `Cannot resolve job type for job ${job.id} in set ${jobSetId}: '${job.job_set_type ?? 'unknown'}' is not registered`)
      return parseGeneration(job, entry)
    })
  }, {
    successAttributes: generations => ({ generation_count: generations.length }),
  })
}
```

app/packages/fnf/src/client/get.ts

```
import type { JobEntry } from '../define-job'
import type { JobResponse } from '../spec'
import type { Generation } from '../types'
import type { GenerationContext } from './context'
import { ApiJobError } from '../errors'
import { observeAsync } from '../observability'
import { parseGeneration } from '../spec'

export async function getGeneration(ctx: GenerationContext, id: string, fallbackEntry?: JobEntry): Promise<Generation> {
  return observeAsync(ctx.observability, 'fnf.job.get', { generation_id: id }, async () => {
    const body = await ctx.adapter.getJob(id) as JobResponse & { job_set_type?: string }
    const entry = (body.job_set_type ? ctx.registry.get(body.job_set_type) : undefined) ?? fallbackEntry
    if (!entry)
      throw new ApiJobError('unknown_model', `Cannot resolve job type for job ${id}`)
    return parseGeneration(body, entry)
  }, {
    successAttributes: generation => ({ model: generation.model, status: generation.status }),
  })
}
```

app/packages/fnf/src/client/index.ts

```
import type { JobEntry } from '../define-job'
import type { AdjustKind } from '../normalize'
import type { Generation, GenerationInput } from '../types'
import type { AdjustResult } from './adjust'
import type { ClientConfig } from './context'
import type { CostEstimate } from './cost'
import type { ListOptions, ListResult } from './list'
import type { PollOptions } from './poll'
import type { SafeSubmitResult, SubmitResult } from './submit'
import type { SubmitInputFor } from './submit-input'
import type { WaitOptions } from './wait'
import { adjust as adjustInput } from './adjust'
import { cancelGeneration } from './cancel'
import { createContext } from './context'
import { estimateCost } from './cost'
import { getGeneration } from './get'
import { getJobSetGenerations } from './get-set'
import { listGenerations } from './list'
import { pollGeneration } from './poll'
import { safeSubmit, submit } from './submit'
import { waitGenerations } from './wait'

export interface JobClient<Jobs extends readonly JobEntry[] = readonly JobEntry[]> {
  submit: (input: SubmitInputFor<Jobs>) => Promise<SubmitResult>
  safeSubmit: (input: SubmitInputFor<Jobs>) => Promise<SafeSubmitResult>
  /**
   * Opt-in client-side normalization: snap the requested setting kinds to their
   * nearest allowed values, returning a new input + the changes. Submit does not.
   */
  adjust: (input: SubmitInputFor<Jobs>, kinds: readonly AdjustKind[]) => AdjustResult<SubmitInputFor<Jobs>>
  get: (id: string) => Promise<Generation>
  /** All jobs of one job set in a single request (requires adapter `getJobSet` support — typed `not_supported` otherwise). */
  getSet: (jobSetId: string) => Promise<Generation[]>
  poll: (id: string, opts?: Pick<PollOptions, 'signal' | 'onProgress'>) => Promise<Generation>
  wait: (generations: Generation[], opts?: WaitOptions) => Promise<Generation[]>
  /** Cancel a job server-side (requires adapter `cancelJob` support). */
  cancel: (id: string) => Promise<void>
  list: (opts?: ListOptions) => Promise<ListResult>
  cost: (input: SubmitInputFor<Jobs>) => Promise<CostEstimate>
}

/**
 * Compose the job operations into a client. This is sugar over the free
 * functions — every method just binds the shared context. If you only need one
 * capability, build a context and call the operation directly:
 *
 *   const ctx = createContext(config)
 *   await submit(ctx, input)
 *
 * Media is a separate concern: use `createMediaClient` (it bundles
 * independently, so a jobs-only frontend never pulls the media code).
 */
export function createJobClient<const Jobs extends readonly JobEntry[]>(
  config: ClientConfig<Jobs>,
): JobClient<Jobs> {
  const ctx = createContext(config)
  return {
    submit: input => submit(ctx, input as GenerationInput),
    safeSubmit: input => safeSubmit(ctx, input as GenerationInput),
    adjust: (input, kinds) => {
      const r = adjustInput(ctx, input as GenerationInput, kinds)
      return { input: r.input as typeof input, adjustments: r.adjustments }
    },
    get: id => getGeneration(ctx, id),
    getSet: jobSetId => getJobSetGenerations(ctx, jobSetId),
    poll: (id, opts) => pollGeneration(ctx, id, opts),
    wait: (generations, opts) => waitGenerations(ctx, generations, opts),
    cancel: id => cancelGeneration(ctx, id),
```

```
    list: opts => listGenerations(ctx, opts),
    cost: input => estimateCost(ctx, input as GenerationInput),
  }
}

// The core types this entry's own signatures return — a client-only consumer
// must not need the root barrel (which pulls in the media half). The model
// catalog itself lives behind "@higgsfield/fnf/jobs".
export type { JobEntry } from '../define-job'
export type { AdjustKind, Adjustment } from '../normalize'
// Read-model selectors — the derivations the production layer needs per render.
export { getJobPhase, getMediaType, getPreviewUrl, getRawUrl, hasResult, isCompleted, isFailed, isFailedJobStatus, isFromJob, isGenerating, isTerminalJobStatus } from '../selectors'
export type { JobPhase } from '../selectors'
export type { Generation, GenerationInput, GenerationResults, GenerationStatus, MediaInput, MediaRef, OutputType, PromptInput } from '../types'
// Composable operations + context — usable on their own, no full client required.
export { adjust } from './adjust'
export type { AdjustResult } from './adjust'
export { cancelGeneration } from './cancel'
export { createContext, entryFor } from './context'
export type { ClientConfig, GenerationContext } from './context'
export { estimateCost } from './cost'
export type { CostEstimate } from './cost'
export { getGeneration } from './get'
export { getJobSetGenerations } from './get-set'
export { listGenerations } from './list'
export type { ListOptions, ListResult } from './list'
export { pollGeneration, pollJobSetGroup } from './poll'

export type { PollOptions } from './poll'
export { generationsFromBody, safeSubmit, submit } from './submit'
export type { SafeSubmitResult, SubmitResult } from './submit'
export type { BaseSubmitFields, SubmitInputFor } from './submit-input'
export { waitGenerations } from './wait'
export type { WaitOptions } from './wait'
```

app/packages/fnf/src/client/list.ts

```
import type { Generation, GenerationResults, GenerationStatus, OutputType } from '../types'
import type { GenerationContext } from './context'
import { observeAsync } from '../observability'
import { parseGeneration } from '../spec'

export interface ListOptions {
  /** Generation kind bucket (the backend expands it into per-type sets). */
  type?: OutputType
  /** Filter by terminal/progress statuses (repeatable). */
  status?: GenerationStatus | GenerationStatus[]
  /** Filter by model (`jobSetType`, repeatable). */
  model?: string | string[]
  cursor?: string | number
  size?: number
  /** List only the derived children of this job set (e.g. its upscales). */
  parentId?: string
}

export interface ListResult {
  items: Generation[]
  cursor?: string | number
}

export async function listGenerations(ctx: GenerationContext, opts: ListOptions = {}): Promise<ListResult> {
  return observeAsync(ctx.observability, 'fnf.job.list', {
    ...(opts.type ? { type: opts.type } : {}),
    ...(opts.size !== undefined ? { size: opts.size } : {}),
    ...(opts.cursor !== undefined ? { has_cursor: true } : {}),
    ...(opts.parentId ? { parent_id: opts.parentId } : {}),
    ...(opts.model !== undefined ? { model_count: Array.isArray(opts.model) ? opts.model.length : 1 } : {}),
    ...(opts.status !== undefined ? { status_count: Array.isArray(opts.status) ? opts.status.length : 1 } : {}),
  }, async () => {
    const body = (await ctx.adapter.listJobs(opts) ?? {}) as {
      items?: RawListItem[]
      jobs?: RawListItem[]
      next_cursor?: string | number | null
      cursor?: string | number | null
    }
    const items = (body.items ?? body.jobs ?? []).map(item => parseListItem(ctx, item, opts.type))
    const cursor = body.next_cursor ?? body.cursor ?? undefined
    return { items, ...(cursor != null ? { cursor } : {}) }
  }, {
    successAttributes: result => ({ item_count: result.items.length, has_cursor: result.cursor !== undefined }),
  })
}

interface RawListItem {
  id: string
  job_set_type?: string
  job_set_id?: string
  job_set_parent_id?: string | null
  status?: string
  result_url?: string | null
  min_result_url?: string | null
  thumbnail_url?: string | null
  params?: Record<string, unknown>
  created_at?: number
  fail_reason?: string | null
}

function parseListItem(ctx: GenerationContext, item: RawListItem, fallbackType?: OutputType): Generation {
  const entry = item.job_set_type ? ctx.registry.get(item.job_set_type) : undefined
  if (entry) {
    return parseGeneration(
      {
        id: item.id,
        job_set_id: item.job_set_id,
        job_set_parent_id: item.job_set_parent_id,
```

```
        status: item.status ?? 'pending',
        result_url: item.result_url,
        min_result_url: item.min_result_url,
        thumbnail_url: item.thumbnail_url,
        params: item.params,
        created_at: item.created_at,
        fail_reason: item.fail_reason,
      },
      entry,
    )
  }
  // Unregistered job type: keep raw params in `extra` so nothing is dropped.
  const completed = item.status === 'completed' && Boolean(item.result_url)
  const results = completed ? buildUnknownListResults(item, fallbackType) : undefined
  return {
    id: item.id,
    ...(item.job_set_id ? { jobSetId: item.job_set_id } : {}),
    ...(item.job_set_parent_id ? { parentJobSetId: item.job_set_parent_id } : {}),
    model: item.job_set_type ?? 'unknown',
    type: fallbackType ?? 'image',
    status: (item.status ?? 'pending') as GenerationStatus,
    input: { model: item.job_set_type ?? 'unknown', settings: {}, ...(item.params ? { extra: item.params } : {}) },
    ...(results ? { results } : {}),
    ...(item.fail_reason ? { failReason: item.fail_reason } : {}),
    ...(item.created_at !== undefined ? { createdAt: item.created_at } : {}),
  }
}

function buildUnknownListResults(item: RawListItem, fallbackType?: OutputType): GenerationResults {
  const results: GenerationResults = { rawUrl: item.result_url as string }
  if (fallbackType === 'video') {
    if (item.thumbnail_url || item.min_result_url)
      results.thumbnailUrl = item.thumbnail_url ?? item.min_result_url ?? undefined
    return results
  }
  if (item.min_result_url)
    results.minUrl = item.min_result_url
  else if (item.thumbnail_url)
    results.thumbnailUrl = item.thumbnail_url
  return results
}
```

app/packages/fnf/src/client/poll.ts

```
import type { JobEntry } from '../define-job'
import type { JobResponse } from '../spec'
import type { Generation } from '../types'
import type { GenerationContext } from './context'
import { ApiJobError, JobTimeoutError, throwIfAborted } from '../errors'
import { observeAsync, observeEvent } from '../observability'
import { parseGeneration } from '../spec'
import { isTerminal } from '../types'
import { getGeneration } from './get'

/**
 * The Nth consecutive failed tick is rethrown (the counter resets on any
 * success) — at the default 2s interval that tolerates ~a minute of continuous
 * backend unavailability. The product polls through outages indefinitely
 * (react-query keeps interval-refetching an errored query), so this is a
 * diagnosability backstop against a hard-down backend, not parity: it surfaces
 * the real 5xx/network error instead of a 10-minute JobTimeoutError.
 */
const MAX_CONSECUTIVE_TRANSIENT_FAILURES = 30

/** Options for `pollGeneration` (the per-job inner loop of `wait`). */
export interface PollOptions {
  /** Fallback entry when the job response carries no `job_set_type`. */
  entry?: JobEntry
  /** Cancels the poll loop with `JobAbortedError` at the next checkpoint. */
  signal?: AbortSignal
  /** Fires after every fetch with the current generation (any status). */
  onProgress?: (g: Generation) => void
}

/**
 * Poll one job to a terminal status.
 *
 * - `opts.onProgress` fires after EVERY fetch — intermediate, unknown, and
 *   terminal statuses alike — so novel backend statuses are observable instead
 *   of silently polling to the deadline.
 * - While the scheduler reports inactive (backgrounded tab/plugin) the loop
 *   sleeps WITHOUT consuming the timeout: the deadline shifts by the paused
 *   duration. Use `opts.signal` to cancel a poll that may pause indefinitely.
 * - Cancellation is cooperative: the abort is honored at the next checkpoint
 *   (loop top / sleep), throwing `JobAbortedError`.
 * - Transient fetch failures (network blips, 5xx/429) are missed ticks, not
 *   verdicts: the loop sleeps the normal interval and retries — still
 *   consuming deadline time — up to a consecutive-failure backstop, after
 *   which the last error is rethrown. Deterministic errors stay fatal.
 */
export async function pollGeneration(ctx: GenerationContext, id: string, opts: PollOptions = {}): Promise<Generation> {
  return observeAsync(ctx.observability, 'fnf.job.poll', { generation_id: id }, async () => {
    const { sleep, isActive } = ctx.scheduler
    let deadline = Date.now() + ctx.poll.timeoutMs
    let transientFailures = 0
    let lastKnown: Generation | undefined
    while (true) {
      throwIfAborted(opts.signal)
      if (isActive && !isActive()) {
        const pausedAt = Date.now()
        await sleepUnlessAborted(sleep, ctx.poll.intervalMs, opts.signal)
        deadline += Date.now() - pausedAt // pause does not consume the timeout
        continue
      }
      let gen: Generation | undefined
      try {
        gen = await getGeneration(ctx, id, opts.entry)
        transientFailures = 0
      }
      catch (err) {
        if (!isTransientPollError(err) || ++transientFailures >= MAX_CONSECUTIVE_TRANSIENT_FAILURES)
          throw err
```

```
      }
      if (gen) {
        lastKnown = gen
        observeEvent(ctx.observability, 'fnf.job.poll.progress', generationAttributes(gen))
        opts.onProgress?.(gen)
        if (isTerminal(gen.status))
          return gen
      }
      if (Date.now() >= deadline)
        throw new JobTimeoutError(id, ctx.poll.timeoutMs, lastKnown)
      await sleepUnlessAborted(sleep, ctx.poll.intervalMs, opts.signal)
    }
  }, {
    successAttributes: generation => ({ status: generation.status, model: generation.model }),
  })
}

/**
 * Poll the members of ONE job set to terminal statuses via the adapter's
 * `getJobSet` — one request per tick for the whole batch, instead of one per
 * job. Gate fields like fnf's `ip_check_finished` (a `completed` job whose IP
 * check hasn't settled maps to the non-terminal `ip_detect` and keeps
 * polling) reach the client through whichever read carries them: on the
 * fnf-web adapter BOTH `getJob` and `getJobSet` payloads do — the mapping
 * lives in the adapter's `mapJob` — so the set path is an efficiency win,
 * not the only gate-aware read.
 *
 * Same pause/abort/timeout/transient-failure semantics as `pollGeneration`.
 * `opts.onProgress` fires per MEMBER per fetch. Members missing from a set
 * payload keep their last-known state (and keep the loop alive until they
 * settle or time out).
 */
export async function pollJobSetGroup(
  ctx: GenerationContext,
  jobSetId: string,
  members: Generation[],
  opts: Omit<PollOptions, 'entry'> = {},
): Promise<Generation[]> {
  return observeAsync(ctx.observability, 'fnf.job.poll', {
    job_set_id: jobSetId,
    generation_count: members.length,
  }, async () => {
    const getJobSet = ctx.adapter.getJobSet
    if (!getJobSet)
      throw new ApiJobError('not_supported', 'pollJobSetGroup requires an adapter with getJobSet')
    const current = new Map(members.map(g => [g.id, g]))
    const { sleep, isActive } = ctx.scheduler
    let deadline = Date.now() + ctx.poll.timeoutMs
    let transientFailures = 0

    while (true) {
      throwIfAborted(opts.signal)
      if (isActive && !isActive()) {
        const pausedAt = Date.now()
        await sleepUnlessAborted(sleep, ctx.poll.intervalMs, opts.signal)
        deadline += Date.now() - pausedAt // pause does not consume the timeout
        continue
      }
      let body: (JobResponse & { job_set_type?: string })[] | undefined
      try {
        body = await getJobSet(jobSetId) as (JobResponse & { job_set_type?: string })[]
        transientFailures = 0
      }
      catch (err) {
        if (!isTransientPollError(err) || ++transientFailures >= MAX_CONSECUTIVE_TRANSIENT_FAILURES)
          throw err
      }
      for (const job of Array.isArray(body) ? body : []) {
        const known = current.get(job.id)
        if (!known)
```

```
          continue // the caller isn't waiting on this set member
        const entry = (job.job_set_type ? ctx.registry.get(job.job_set_type) : undefined) ?? ctx.registry.get(known.model)
        // Fail fast like the singles path (`getGeneration`): an unresolvable type
        // is a local configuration error no amount of further polling can fix.
        if (!entry)
          throw new ApiJobError('unknown_model', `Cannot resolve job type for job ${job.id} in set ${jobSetId}: '${job.job_set_type ?? known.model}' is not registered`)
        const gen = parseGeneration(job, entry)
        current.set(job.id, gen)
        observeEvent(ctx.observability, 'fnf.job.poll.progress', generationAttributes(gen))
        opts.onProgress?.(gen)
      }
      if ([...current.values()].every(g => isTerminal(g.status)))
        return members.map(m => current.get(m.id) as Generation)
      if (Date.now() >= deadline) {
        const pending = [...current.values()].find(g => !isTerminal(g.status)) as Generation
        throw new JobTimeoutError(pending.id, ctx.poll.timeoutMs, pending)
      }
      await sleepUnlessAborted(sleep, ctx.poll.intervalMs, opts.signal)
    }
  }, {
    successAttributes: generations => ({ terminal_count: generations.filter(g => isTerminal(g.status)).length }),
  })
}

/**
 * Is this fetch failure worth retrying on the next tick? The product never
 * lets one blip kill a multi-minute wait: fnf-web's job polling
 * (`use-job-status-polling`) rides react-query's `refetchInterval: 5000` with
 * default retries, so a 502 or a dropped socket just means the next tick.
 * Transient = the adapter's `network` code (a thrown fetch) or a retryable
 * HTTP status (any 5xx, 429). Everything else — 404/401/422, `unknown_model`,
 * aborts, timeouts — is deterministic and rethrown immediately.
 */
function isTransientPollError(err: unknown): boolean {
  if (err instanceof ApiJobError && err.code === 'network')
    return true
  const status = (err as { status?: unknown } | null)?.status
  return typeof status === 'number' && (status >= 500 || status === 429)
}

function generationAttributes(g: Generation): Record<string, string | number | boolean | null> {
  return {
    generation_id: g.id,
    model: g.model,
    type: g.type,
    status: g.status,
    ...(g.jobSetId ? { job_set_id: g.jobSetId } : {}),
  }
}

/** Sleep that wakes early on abort and then throws the typed `JobAbortedError`. */
async function sleepUnlessAborted(sleep: (ms: number) => Promise<void>, ms: number, signal?: AbortSignal): Promise<void> {
  if (!signal) {
    await sleep(ms)
    return
  }
  throwIfAborted(signal)
  let onAbort: (() => void) | undefined
  const aborted = new Promise<void>((resolve) => {
    onAbort = resolve
    signal.addEventListener('abort', onAbort, { once: true })
  })
  try {
    await Promise.race([sleep(ms), aborted])
  }
  finally {
    if (onAbort)
      signal.removeEventListener('abort', onAbort)
  }
  throwIfAborted(signal)
```

```
}
```

app/packages/fnf/src/client/submit-input.ts

```
import type { JobEntry } from '../define-job'
import type { GenerationInput } from '../types'

/**
 * Fields shared by every submit input, independent of the chosen model.
 * `prompt`/`media` are deliberately NOT here — they are part of a job's
 * envelope and appear only when the job declares them (see `Envelope`).
 */
export interface BaseSubmitFields {
  count?: number
  folderId?: string
  /** Parent job set id — derived jobs (upscale/outpaint/…) link to their source via wire `parent_id`. */
  parentId?: string
  extra?: Record<string, unknown>
}

type InputOf<E> = E extends JobEntry<infer Type, infer Settings, infer Env>
  // A weakly-typed entry (Type = string) widens to the open input WITHOUT
  // collapsing the typed entries next to it in the union.
  ? string extends Type ? GenerationInput : BaseSubmitFields & Env & { model: Type, settings: Settings }
  : never

/**
 * Submit input discriminated by `model`, derived from the registered jobs:
 * `model` autocompletes to a registered jobSetType and `settings` is typed to
 * that model's schema. A dynamically-built entry contributes the open
 * `GenerationInput` branch (autocomplete survives; strictness is per-entry).
 */
export type SubmitInputFor<Jobs extends readonly JobEntry[]> = InputOf<Jobs[number]>
```

app/packages/fnf/src/client/submit.ts

```
import type { JobEntry } from '../define-job'
import type { ApiJobErrorJSON } from '../errors'
import type { JobResponse } from '../spec'
import type { Generation, GenerationInput } from '../types'
import type { GenerationContext } from './context'
import { ApiJobError, ConfirmationRejectedError, UnknownSubmitResponseError, ValidationError } from '../errors'
import { observeAsync } from '../observability'
import { buildWireParams, parseGeneration } from '../spec'
import { entryFor } from './context'

export interface SubmitResult {
  generations: Generation[]
  /** Per-job errors from a `count > 1` fan-out where some (but not all) jobs failed. */
  failed?: ApiJobErrorJSON[]
  /** Human-readable summary of a partial failure. */
  warning?: string
}

export type SafeSubmitResult =
  | { ok: true, generations: Generation[], failed?: ApiJobErrorJSON[], warning?: string }
  | { ok: false, error: { code: string, message: string, status?: number, data?: unknown } }

export async function submit(ctx: GenerationContext, input: GenerationInput): Promise<SubmitResult> {
  return observeAsync(ctx.observability, 'fnf.job.submit', {
    model: input.model,
    count: Math.max(1, input.count ?? 1),
  }, async () => {
    const entry = entryFor(ctx, input.model)
    // Parse once (throws ValidationError on bad settings) — the same wire params
    // are reused for every job in the fan-out. Normalization is NOT done here; it
    // is the separate, opt-in `adjust()` step.
    const params = buildWireParams(input, entry)
    const count = Math.max(1, input.count ?? 1)

    // Confirmation tokens are single-use. Reusing one across the client-side
    // fan-out would let only the first request pass, so reject before opening
    // the host's approval UI. Model batchSize remains one confirmed request.
    if (ctx.adapter.confirm && count > 1) {
      throw new ValidationError(
        'Confirmed submissions cannot use count greater than 1 because confirmation tokens are single-use. Use the model settings.batchSize option instead.',
      )
    }

    // The host's confirmation gate runs after validation/wire-building and
    // before any network call. A rejection aborts the whole submit with the
    // typed `confirmation_rejected` error.
    const confirmationToken = await confirmSubmission(ctx, entry.jobSetType, params)

    // `count` is a client-side fan-out: N independent job submissions, orthogonal
    // to the per-model `batch_size` param (already in `params`). Total outputs =
    // count × batch_size. allSettled so one transient failure doesn't discard the
    // jobs that did succeed.
    const settled = await Promise.allSettled(
      Array.from({ length: count }, () => ctx.adapter.createJobs({
        jobSetType: entry.jobSetType,
        params,
        ...(confirmationToken !== undefined ? { confirmationToken } : {}),
      })),
    )
    const generations = settled.flatMap(r => (r.status === 'fulfilled' ? generationsFromBody(r.value, entry, input) : []))
    const failed = settled.flatMap(r => (r.status === 'rejected' ? [asError(r.reason)] : []))

    if (generations.length === 0) {
      // Everything fulfilled but no response shape was recognized (see
      // generationsFromBody) — keep the first body on the error so the
      // unrecognized shape survives toJSON/Comlink for diagnosis.
      const fulfilled = settled.find((r): r is PromiseFulfilledResult<unknown> => r.status === 'fulfilled')
      throw failed[0] ?? new UnknownSubmitResponseError(undefined, fulfilled?.value)
```

```
    }

    const result: SubmitResult = { generations }
    if (failed.length > 0) {
      result.failed = failed.map(e => e.toJSON())
      const ranOut = failed.some(e => e.code === 'out_of_credits')
      result.warning = `Submitted ${generations.length} of ${count} requested jobs${ranOut ? ', then ran out of credits' : ''}.`
    }
    return result
  }, {
    successAttributes: result => ({
      generation_count: result.generations.length,
      failed_count: result.failed?.length ?? 0,
    }),
  })
}

async function confirmSubmission(ctx: GenerationContext, jobSetType: string, params: Record<string, unknown>): Promise<string | undefined> {
  if (!ctx.adapter.confirm)
    return undefined
  let token: string | void
  try {
    token = await ctx.adapter.confirm({ jobSetType, params })
  }
  catch (err) {
    // Typed throws (a host may raise its own ApiJobError) pass through; anything
    // else — a plain throw from a dismissed modal — becomes the stable code.
    throw err instanceof ApiJobError ? err : new ConfirmationRejectedError(err instanceof Error ? err.message : undefined)
  }
  return typeof token === 'string' ? token : undefined
}

export async function safeSubmit(ctx: GenerationContext, input: GenerationInput): Promise<SafeSubmitResult> {
  try {
    const result = await submit(ctx, input)
    return {
      ok: true,
      generations: result.generations,
      ...(result.failed ? { failed: result.failed } : {}),
      ...(result.warning ? { warning: result.warning } : {}),
    }
  }
  catch (err) {
    if (err instanceof ApiJobError)
      return { ok: false, error: err.toJSON() }
    return { ok: false, error: { code: 'unexpected', message: err instanceof Error ? err.message : String(err) } }
  }
}

/**
 * Normalize a create response into Generations. Backends differ: some return a
 * bare `string[]` of created job ids, some `{ id, status, … }`,
 * some `{ job_ids: [...] }`. Id-only responses become pending generations that
 * carry the submitted input (so `wait`/`get` can refresh them).
 */
export function generationsFromBody(body: unknown, entry: JobEntry, input: GenerationInput): Generation[] {
  if (Array.isArray(body)) {
    // An array of full job objects (adapters that normalize { job_sets } fan out
    // to these) parses completely; an array of bare ids becomes pendings.
    if (body.length > 0 && body.every(item => isObject(item) && typeof item.id === 'string'))
      return body.map(job => parseGeneration(job as JobResponse, entry))
    return body.filter(isString).map(id => pendingGeneration(id, entry, input))
  }
  if (body && typeof body === 'object') {
    const obj = body as Record<string, unknown>
    const ids = obj.job_ids ?? obj.ids
    if (Array.isArray(ids))
      return ids.filter(isString).map(id => pendingGeneration(id, entry, input))
    if (typeof obj.id === 'string')
      return [parseGeneration(body as JobResponse, entry)]
```

```
  }
  return []
}

function pendingGeneration(id: string, entry: JobEntry, input: GenerationInput): Generation {
  return { id, model: entry.jobSetType, type: entry.outputType, status: 'queued', input }
}

function isString(x: unknown): x is string {
  return typeof x === 'string'
}

function isObject(x: unknown): x is Record<string, unknown> {
  return typeof x === 'object' && x !== null
}

// 'unexpected' (a local throw), NOT 'unknown' (UnknownSubmitResponseError's code
// for unrecognized backend responses) — consumers handle the two differently.
function asError(err: unknown): ApiJobError {
  return err instanceof ApiJobError ? err : new ApiJobError('unexpected', err instanceof Error ? err.message : String(err))
}
```

app/packages/fnf/src/client/wait.ts

```
import type { Generation } from '../types'
import type { GenerationContext } from './context'
import { ApiJobError } from '../errors'
import { observeAsync, observeEvent } from '../observability'
import { isTerminal } from '../types'
import { pollGeneration, pollJobSetGroup } from './poll'

export interface WaitOptions {
  /**
   * Fires on EVERY poll tick with the freshly fetched generation (including
   * intermediate and unknown statuses), and once for already-terminal inputs.
   */
  onProgress?: (g: Generation) => void
  /**
   * Reject when any generation — including an already-terminal input — lands
   * on a failure status (`failed`/`nsfw`/`ip_detected`); `canceled` resolves.
   */
  throwOnFail?: boolean
  /** Cancels the wait: pollers stop at the next checkpoint with `JobAbortedError`. */
  signal?: AbortSignal
}

/**
 * Poll a batch to terminal statuses. Generations sharing a `jobSetId` are
 * polled as ONE set (when the adapter implements `getJobSet`): one request per
 * tick per set, and set-only gate fields (fnf's `ip_check_finished`) apply.
 * Generations without a set id — or on adapters without `getJobSet` — fall
 * back to per-job polling.
 *
 * On the first rejection (a `throwOnFail` failure, a timeout, the caller's
 * `signal`, or a network error) the remaining pollers are aborted — no
 * orphaned loops keep hitting the backend after the wait itself has thrown.
 */
export async function waitGenerations(ctx: GenerationContext, generations: Generation[], opts?: WaitOptions): Promise<Generation[]> {
  return observeAsync(ctx.observability, 'fnf.job.wait', { generation_count: generations.length }, async () => {
    const controller = new AbortController()
    const onCallerAbort = (): void => controller.abort()
    if (opts?.signal?.aborted)
      controller.abort()
    else opts?.signal?.addEventListener('abort', onCallerAbort, { once: true })

    const settled = new Map<string, Generation>()
    const checkFail = (done: Generation): Generation => {
    // 'canceled' is deliberately exempt: it's a user action, not a failure.
      if (opts?.throwOnFail && (done.status === 'failed' || done.status === 'nsfw' || done.status === 'ip_detected'))
        throw new ApiJobError('job_failed', done.failReason ?? `Job ${done.id} ${done.status}`, { data: { generation: done } })
      return done
    }

    const onProgress = (g: Generation): void => {
      observeEvent(ctx.observability, 'fnf.job.wait.progress', generationAttributes(g))
      opts?.onProgress?.(g)
    }

    // Partition: already-terminal inputs pass through; pending ones group by
    // job set when the adapter can poll sets, else poll individually.
    const canPollSets = typeof ctx.adapter.getJobSet === 'function'
    const setGroups = new Map<string, Generation[]>()
    const singles: Generation[] = []

    try {
      for (const g of generations) {
        if (isTerminal(g.status)) {
          onProgress(g) // already terminal: observed once, never polled
          checkFail(g) // judged the same as a generation that fails one tick later
          continue
        }
        if (canPollSets && g.jobSetId) {
```

```
          const group = setGroups.get(g.jobSetId) ?? []
          group.push(g)
          setGroups.set(g.jobSetId, group)
        }
        else {
          singles.push(g)
        }
      }

      await Promise.all([
        ...[...setGroups.entries()].map(async ([jobSetId, members]) => {
          const done = await pollJobSetGroup(ctx, jobSetId, members, {
            signal: controller.signal,
            // Judge members as they land, not when the whole set settles — a
            // failed member must reject the wait (and abort the set poller)
            // while siblings are still running, matching the singles path.
            onProgress: (g) => {
              onProgress(g)
              if (isTerminal(g.status))
                checkFail(g)
            },
          })
          for (const gen of done) settled.set(gen.id, checkFail(gen))
        }),
        ...singles.map(async (g) => {
          const done = await pollGeneration(ctx, g.id, {
            entry: ctx.registry.get(g.model),
            signal: controller.signal,
            onProgress,
          })
          settled.set(done.id, checkFail(done))
        }),
      ])
    }
    catch (err) {
      controller.abort() // stop the surviving pollers
      throw err
    }
    finally {
      opts?.signal?.removeEventListener('abort', onCallerAbort)
    }

    // Input order preserved; already-terminal inputs pass through unchanged.
    return generations.map(g => settled.get(g.id) ?? g)
  }, {
    successAttributes: done => ({ terminal_count: done.filter(g => isTerminal(g.status)).length }),
  })
}

function generationAttributes(g: Generation): Record<string, string | number | boolean | null> {
  return {
    generation_id: g.id,
    model: g.model,
    type: g.type,
    status: g.status,
    ...(g.jobSetId ? { job_set_id: g.jobSetId } : {}),
  }
}
```

app/packages/fnf/src/define-job.ts

```
import type * as zm from 'zod/mini'
import type { MediaConfig, MediaIssue, MediaRule } from './groups/media'
import type { GenerationInput, MediaFormat, MediaRef, OutputType, PromptInput } from './types'
import type { Normalize, NORMALIZE_TYPE } from './z'
import { promptCodec } from './groups/prompt'
import { getNormalize, getWireName, z } from './z'

type SettingsMap = Record<string, unknown>

/**
 * The input type of one settings field. Normalized fields (`z.aspectRatio` /
 * `z.duration`) carry a phantom literal union under `NORMALIZE_TYPE` — prefer it
 * so callers get autocomplete on the canonical values; otherwise fall back to
 * the schema's zod input type.
 */
type FieldInput<T> = T extends { readonly [NORMALIZE_TYPE]: infer U } ? U : zm.input<T>

/**
 * The settings shape a caller passes for a job: each field typed from
 * `FieldInput`, and made optional when that input admits `undefined`
 * (i.e. `z.optional(...)` / `z._default(...)`).
 */
export type SettingsInput<M extends SettingsMap> = [keyof M] extends [never]
  ? Record<string, never> // a zero-settings job takes {}, not "anything" (bare {} accepts 42)
  : & { [K in keyof M as undefined extends FieldInput<M[K]> ? never : K]: FieldInput<M[K]> }
    & { [K in keyof M as undefined extends FieldInput<M[K]> ? K : never]?: FieldInput<M[K]> }

/**
 * A job's media declaration, with its allowed roles captured as literals.
 * `roles` is the ONLY inference site for the literal union (`NoInfer` below):
 * `counts` keys and `MediaRule<Roles>` hold `Roles` in partial/contravariant
 * positions, and letting them contribute candidates (subsets from `maxTotal`
 * etc.) makes newer TS reconcile the conflict by widening to `string` —
 * which silently breaks the `media` key narrowing on `submit`.
 */
export interface MediaConfigFor<Roles extends string> {
  field: string
  format: MediaFormat
  roles: readonly Roles[]
  /** Per-role cardinality: `min >= 1` makes the role required; `max` caps it. */
  counts?: Partial<Record<NoInfer<Roles>, { min?: number, max?: number }>>
  /** Cross-role rules (`requiresOneOf`/`atLeastOneOf`, or any custom `MediaRule`). */
  rules?: readonly MediaRule<NoInfer<Roles>>[]
}

/**
 * The non-settings input fields a job exposes, derived entirely from what the
 * job declares — no universal assumptions. A job that declares no `media` gets
 * no `media` field in its input type; one that declares no `prompt` gets none.
 * When `media` is declared its key is narrowed to the job's roles.
 */
type MediaField<Roles extends string> = [Roles] extends [never]
  ? { media?: never } // `never`, not `unknown`: rejects non-fresh objects too, not just literals
  : { media?: Partial<Record<Roles, MediaRef | MediaRef[]>> }
type PromptField<P extends boolean> = P extends true ? { prompt?: PromptInput } : { prompt?: never }
export type Envelope<Roles extends string, P extends boolean> = MediaField<Roles> & PromptField<P>

/**
 * A job's params declaration — it mirrors the backend wire `params` object:
 * the `prompt`/`media` envelopes (serialized by their codecs) plus the scalar
 * `settings` fields (zod schemas; tag a wire name with `z.wire`).
 */
export interface JobParams<M extends SettingsMap, Roles extends string, P extends boolean> {
  /** Set `true` to expose the standard prompt group. Omit for a job that takes no prompt. */
  prompt?: P
  /** Declare media support — its `roles` become the allowed `media` keys. Omit for a job that takes no media. */
  media?: MediaConfigFor<Roles>
  /** The scalar wire fields. */
```

```
  settings: M
}

export interface DefineJobConfig<Type extends string, M extends SettingsMap, Roles extends string, P extends boolean> {
  jobSetType: Type
  outputType: OutputType
  params: JobParams<M, Roles, P>
  /**
   * Optional local per-job credit calculator (the fnf-web `credits(ctx)` pattern):
   * lets `cost()` answer without a network round-trip. `input.settings` is typed
   * to THIS model's schema; `estimateCost` parses settings first, so schema
   * defaults are already applied when the calculator runs. Return null when the
   * inputs aren't sufficient to price locally — `estimateCost` then falls back
   * to the backend. Price ONE job (the `count` fan-out multiplies upstream).
   */
  credits?: (input: GenerationInput<SettingsInput<M>>) => number | null
  /**
   * Cross-field validation across the WHOLE input (prompt × media × settings) —
   * the rules `counts`/`rules` can't express (e.g. "prompt is required unless
   * media is attached"). Runs in the submit path after parse + media checks;
   * return pydantic-shaped issues, or null/[] when valid. Sync and pure.
   */
  validate?: (input: GenerationInput<SettingsInput<M>>) => MediaIssue[] | null
  /**
   * The job-level body hooks — ONE serialization lifecycle, documented once:
   *
   *   submit: input → codecs + settings assemble the wire body → `finalize(wire, input)` → POST
   *   get:    stored params → `restore(params)` overlay → codecs + settings parse → input
   *
   * `finalize` is the job's last touch on the assembled body: derive fields
   * (seedance folds `mode` into the wire `model`), compute width/height, inject
   * the constants the product surface always sends (soul's `version: 3`).
   * Omit it when the declared fields map 1:1.
   */
  finalize?: (wire: Record<string, unknown>, input: GenerationInput) => Record<string, unknown>
  /**
   * Inverse of `finalize`, required whenever finalize DELETES or renames a
   * settings key: without it `parseGeneration` can't see the original setting
   * and get-then-resubmit silently falls back to the schema default — a fetched
   * fast generation would resubmit as std. Return the wire-keyed entries to
   * overlay on the stored params before settings extraction. Omit otherwise.
   */
  restore?: (wire: Record<string, unknown>) => Record<string, unknown>
}

export interface JobEntry<Type extends string = string, Settings = Record<string, unknown>, Env = unknown> {
  jobSetType: Type
  outputType: OutputType
  /** Present only when the job declared media. */
  media?: MediaConfig
  /** Whether the standard prompt group is active for this job. */
  prompt: boolean
  settingsMap: SettingsMap
  settingsSchema: ReturnType<typeof z.object>
  normalizers: Record<string, Normalize>
  /** Explicit wire-name overrides for settings (local key → wire key), from `z.wire`. */
  wireNames: Record<string, string>
  /** Local per-job credit calculator; null = can't price locally (backend fallback). */
  credits?: (input: GenerationInput) => number | null
  /** Cross-field validation (prompt × media × settings); pydantic-shaped issues. */
  validate?: (input: GenerationInput) => MediaIssue[] | null
  /** Last touch on the assembled wire body — see DefineJobConfig.finalize. */
  finalize?: (wire: Record<string, unknown>, input: GenerationInput) => Record<string, unknown>
  /** Inverse of `finalize` (wire-keyed overlay) — see DefineJobConfig.restore. */
  restore?: (wire: Record<string, unknown>) => Record<string, unknown>
  /** Phantom: the typed settings-input shape. Carries types only, never set at runtime. */
  readonly __settings?: Settings
  /** Phantom: the typed envelope (media/prompt) the job exposes. Types only. */
  readonly __envelope?: Env
}
```

```
export function defineJob<
  const Type extends string,
  M extends SettingsMap,
  const Roles extends string = never,
  const P extends boolean = false,
>(
  config: DefineJobConfig<Type, M, Roles, P>,
): JobEntry<Type, SettingsInput<M>, Envelope<Roles, P>> {
  const { prompt, media, settings } = config.params
  // Non-`wrapped` media formats carry no role tag on the wire, so they can only
  // represent one role — guard the invariant the types alone can't (callers may
  // bypass with `as`). See mediaCodec serialize/parse.
  if (media && media.format !== 'wrapped' && media.roles.length !== 1) {
    throw new Error(
      `defineJob('${config.jobSetType}'): media format '${media.format}' supports exactly one role, got ${media.roles.length} [${media.roles.join(', ')}] — use format 'wrapped' for multi-role media`,
    )
  }
  const wireNames = collectWireNames(settings)
  assertNoWireCollisions(config.jobSetType, settings, wireNames, prompt ?? false, media)

  return {
    jobSetType: config.jobSetType,
    outputType: config.outputType,
    media,
    prompt: prompt ?? false,
    settingsMap: settings,
    settingsSchema: z.object(settings as Record<string, never>),
    normalizers: collectNormalizers(settings),
    wireNames,
    // Type-erased on the entry (the registry is heterogeneous); the runtime
    // value IS the caller's typed input — same erasure as settingsSchema.
    credits: config.credits as JobEntry['credits'],
    validate: config.validate as JobEntry['validate'],
    finalize: config.finalize,
    restore: config.restore,
  }
}

/**
 * Duplicate effective wire keys (two settings → one key via z.wire / a shared
 * tagged schema instance, or a settings key shadowing a prompt/media wire key)
 * would serialize last-write-wins and parse back ambiguously — fail at
 * definition time instead.
 */
function assertNoWireCollisions(jobSetType: string, settings: SettingsMap, wireNames: Record<string, string>, prompt: boolean, media?: MediaConfig): void {
  const reserved = new Set<string>([...(prompt ? promptCodec.wireKeys : []), ...(media ? [media.field] : [])])
  const seen = new Map<string, string>()
  for (const key of Object.keys(settings)) {
    const wireKey = wireNames[key] ?? key
    const clash = seen.get(wireKey)
    if (clash !== undefined)
      throw new Error(`defineJob('${jobSetType}'): settings '${clash}' and '${key}' both serialize to wire key '${wireKey}'`)
    if (reserved.has(wireKey))
      throw new Error(`defineJob('${jobSetType}'): settings key '${key}' collides with the ${prompt && promptCodec.wireKeys.includes(wireKey) ? 'prompt' : 'media'} wire key '${wireKey}'`)
    seen.set(wireKey, key)
  }
}

function collectNormalizers(settings: SettingsMap): Record<string, Normalize> {
  const out: Record<string, Normalize> = {}
  for (const [key, schema] of Object.entries(settings)) {
    const n = getNormalize(schema)
    if (n)
      out[key] = n
  }
  return out
}

function collectWireNames(settings: SettingsMap): Record<string, string> {
  const out: Record<string, string> = {}
```

```
  for (const [key, schema] of Object.entries(settings)) {
    const name = getWireName(schema)
    if (name !== undefined && name !== key)
      out[key] = name
  }
  return out
}
```

app/packages/fnf/src/errors.ts

```
import type { Generation } from './types'

/**
 * Error taxonomy for the fnf SDK.
 *
 * `ApiJobError` is the generic superclass every SDK error extends. Each
 * error carries a stable `code` (the cross-boundary discriminator — survives
 * `toJSON`/Comlink where `instanceof` does not) and an optional typed `data`
 * payload. Serialization is uniform: `toJSON()` emits `{ code, message, status?,
 * data? }` at the base, so subclasses never need their own override and nothing
 * is silently dropped across a worker/iframe boundary. Rehydrate with
 * `errorFromJSON`.
 */

// ■■ shared value types (mirror the fnf/mcp backend detail vocabulary) ■■
// Superset of fnf-web's SubscriptionPlanTypeEnum (subscriptions.ts) — the
// legacy 'team-plan'/'enterprise-plan' spellings are kept for compatibility.
export type SubscriptionPlanType
  = 'free' | 'basic' | 'pro' | 'ultimate' | 'creator' | 'team-plan' | 'enterprise-plan' | 'starter' | 'plus' | 'ultra' | 'max' | 'team' | 'scale' | 'enterprise'
// Mirrors the backend's wire enum (fnf-api src/types_/subscription.py:198-203).
export type BillingPeriod = 'monthly' | 'annual' | 'quarterly' | 'eighteen_month' | 'two_year'
export interface TeamErrorDetails { workspaceType: 'Shared' | 'Private', userRole: 'Owner' | 'Member', isEnterprise?: boolean }
export type AutoTopUpErrorType = 'auto_top_up_suspended' | 'auto_top_up_charge_failed' | 'auto_top_up_in_progress'
export type MinimumPlanFeature = 'avatar' | 'train'
export type GraceLimitReachedType = 'downgrade_notice' | 'pay_less_notice' | 'unlock_full_access_notice'

/**
 * Plan/billing context shared by the billing errors (structured, not flattened).
 * All fields are optional — the backend doesn't guarantee every one on every
 * error, so the shape stays honest rather than casting absent values.
 */
export interface BillingContext {
  plan?: SubscriptionPlanType
  billingPeriod?: BillingPeriod
  team?: TeamErrorDetails
}

// ■■ base ■■
export interface ApiJobErrorJSON<D = unknown> {
  code: string
  message: string
  status?: number
  data?: D
}

export class ApiJobError<D = unknown> extends Error {
  readonly code: string
  readonly status?: number
  readonly data?: D

  constructor(code: string, message: string, opts?: { status?: number, data?: D }) {
    super(message)
    this.name = code
    this.code = code
    this.status = opts?.status
    this.data = opts?.data
  }

  toJSON(): ApiJobErrorJSON<D> {
    return {
      code: this.code,
      message: this.message,
      ...(this.status != null ? { status: this.status } : {}),
      ...(this.data !== undefined ? { data: this.data } : {}),
    }
  }

  /**
```

```
   * Type-guard sugar: `OutOfCreditsError.is(err)` narrows `err` to that class —
   * and `BillingError.is(err)` / `ApiJobError.is(err)` narrow to a tier.
   * In-process only (it's `instanceof`); across a serialization boundary switch
   * on `code`, or rehydrate with `errorFromJSON` first.
   */
  static is<T extends ApiJobError>(this: new (...args: never[]) => T, value: unknown): value is T {
    return value instanceof this
  }
}

// ■■ billing mid-tier (instanceof grouping; data is a BillingContext) ■■
export class BillingError<D extends BillingContext = BillingContext> extends ApiJobError<D> {}

export class OutOfCreditsError extends BillingError {
  // No pinned status: both backend not_enough_credits sites emit HTTP 403
  // (fnf-api src/exceptions/auth.py:71-88 private, src/exceptions/workspace.py:267-290
  // shared) — a hard-coded status here would survive errorFromResponse's
  // stamping (it only fills a null status) and misreport the wire. The real
  // status arrives via stamping, and errorFromJSON restores the serialized one.
  constructor(data: BillingContext) { super('out_of_credits', 'Not enough credits', { data }) }
}

export class RateLimitError extends BillingError<BillingContext & { maxCount?: number, isUnlimited?: boolean }> {
  constructor(data: BillingContext & { maxCount?: number, isUnlimited?: boolean }) {
    super('rate_limit', 'Rate limit reached', { data })
  }
}

export class SharedTeamSubscriptionInactiveError extends BillingError {
  constructor(data: BillingContext) { super('workspace_subscription_inactive', 'Team subscription is inactive', { data }) }
}

export class MinimumPlanError extends BillingError<BillingContext & { feature?: MinimumPlanFeature }> {
  constructor(data: BillingContext & { feature?: MinimumPlanFeature }) {
    const plan = data.plan ? `${data.plan} plan or higher` : 'a higher plan'
    super('minimum_plan_required', `Requires ${plan}${data.feature ? ` for ${data.feature}` : ''}`, { data })
  }
}

// ■■ non-billing submission errors ■■
export class PromptNsfwError extends ApiJobError {
  constructor() { super('prompt_nsfw', 'Prompt flagged as unsafe') }
}

/**
 * The protected-content (IP) check blocked the media. Current fnf-api never
 * emits `ip_detected` as an HTTP `detail.error_type` (zero sites) — it exists
 * only as a 2xx confirm-body status (`status: 'ip_detected'`, the media half's
 * MediaModerationError) and a job status. This class covers the legacy HTTP 422
 * error-body shape fnf-web still defends against at presign/confirm
 * (fnf-web src/entities/input-media/api/image-api.ts, video-api.ts → its
 * IpDetectedError). Lives here, not media/errors.ts: errors.ts owns the wire
 * error_type catalog and must not import from media/.
 */
export class IpDetectedError extends ApiJobError {
  constructor(message = 'Protected content is not allowed') { super('ip_detected', message) }
}

/**
 * `detail.error_type: 'ip_check_rate_limit_reached'` at presign/confirm — the
 * eligibility (IP) check quota, not a generation rate limit (so no billing
 * context). Default message mirrors fnf-web's CONFIRM-side class
 * (src/shared/lib/ip-check/ip-check-rate-limit-error.ts); the product shows a
 * differently-worded upload variant at presign (ip-check-upload-rate-limit-
 * error.ts: 'Eligibility check upload limit reached…') that the SDK does not
 * distinguish — same code, one message.
 */
export class IpCheckRateLimitError extends ApiJobError {
  constructor(message = 'You\'ve reached the eligibility check limit. Please try again later.') {
    super('ip_check_rate_limit_reached', message)
```

```
  }
}

export class VpnDetectedError extends ApiJobError {
  constructor() { super('vpn_detected', 'VPN detected — this model is not available over a VPN') }
}

export class KybVerificationRequiredError extends ApiJobError {
  constructor() { super('kyb_verification_required', 'KYB verification required') }
}

export class NotEnoughBoostCreditsError extends ApiJobError {
  constructor() { super('not_enough_boost_credits', 'Not enough boost credits') }
}

export class SubscriptionRenewalFailedError extends ApiJobError {
  constructor() { super('subscription_renewal_failed', 'Subscription renewal failed') }
}

export class JobInProgressError extends ApiJobError {
  constructor() { super('job_in_progress', 'Another job is already in progress', { status: 409 }) }
}

export class UnlimitedGenerationNotAllowedError extends ApiJobError {
  constructor() { super('unlimited_generation_not_allowed', 'Unlimited generation is not allowed for this request') }
}

export class BatchRateLimitError extends ApiJobError<{ concurrentBatchesLimit?: number }> {
  constructor(concurrentBatchesLimit?: number) {
    super('batch_rate_limit', 'Batch rate limit reached', { data: { concurrentBatchesLimit } })
  }
}

export class BeatFitLimitError extends ApiJobError<{ concurrentBatchesLimit?: number }> {
  constructor(concurrentBatchesLimit?: number) {
    super('beat_fit_limit', 'Beat-fit batch limit reached', { data: { concurrentBatchesLimit } })
  }
}

export class GraceDailyLimitError extends ApiJobError<{ graceType: GraceLimitReachedType, graceData?: unknown }> {
  constructor(graceType: GraceLimitReachedType, graceData?: unknown) {
    super('grace_daily_limit', 'Daily generation limit reached during grace period', { data: { graceType, graceData } })
  }
}

export class AutoTopUpSuspendedError extends ApiJobError<{ autoTopUpType: AutoTopUpErrorType }> {
  constructor(autoTopUpType: AutoTopUpErrorType) {
    super('auto_top_up_suspended', 'Auto top-up is suspended', { data: { autoTopUpType } })
  }
}

export class WorkspaceMemberSpendPausedError extends ApiJobError<{
  workspaceId?: string
  userId?: string
  pausedAt?: string | null
  pausedByUserId?: string | null
}> {
  constructor(data: { workspaceId?: string, userId?: string, pausedAt?: string | null, pausedByUserId?: string | null }) {
    super('workspace_member_spend_paused', 'Spending is paused for this workspace member', { data })
  }
}

export class AccountSuspendedError extends ApiJobError<{ suspendedCode: 'account_suspended' | 'account_blocked' }> {
  constructor(suspendedCode: 'account_suspended' | 'account_blocked') {
    super('account_suspended', suspendedCode === 'account_blocked' ? 'Account is blocked' : 'Account is suspended', { data: { suspendedCode } })
  }
}

export class WorkspaceSelectionRequiredError extends ApiJobError<{ workspaces: Array<{ id: string, name: string | null, type: 'private' | 'shared' }> }> {
  constructor(workspaces: Array<{ id: string, name: string | null, type: 'private' | 'shared' }> = []) {
```

```
    super('workspace_selection_required', 'Workspace selection required', { status: 409, data: { workspaces } })
  }
}

export class ApiMessageError extends ApiJobError {
  constructor(message: string) { super('api_message', message) }
}

export class ValidationError extends ApiJobError<{ issues?: unknown }> {
  constructor(message: string, issues?: unknown) { super('validation', message, { status: 422, data: { issues } }) }
}

export class UnknownSubmitResponseError extends ApiJobError<{ responseUrl?: string, responseBody?: unknown }> {
  constructor(status?: number, responseBody?: unknown, responseUrl?: string) {
    super('unknown', `Request failed${status != null ? ` with status ${status}` : ''}`, { status, data: { responseUrl, responseBody } })
  }
}

// ■■ client-side lifecycle errors (share the base; not submission responses) ■■
export class JobAbortedError extends ApiJobError {
  constructor(message = 'Operation aborted') { super('aborted', message) }
}

/** The host's `confirm` gate rejected — the user declined the submission. */
export class ConfirmationRejectedError extends ApiJobError {
  constructor(message = 'Submission was not confirmed') { super('confirmation_rejected', message) }
}

/** Throw the typed `JobAbortedError` when the signal is already aborted. */
export function throwIfAborted(signal?: AbortSignal): void {
  if (signal?.aborted)
    throw new JobAbortedError()
}

export class JobTimeoutError extends ApiJobError<{ generation?: Generation }> {
  readonly generation?: Generation
  constructor(id: string, timeoutMs: number, generation?: Generation) {
    super('timeout', `Polling job ${id} did not reach a terminal status within ${timeoutMs}ms`, { data: { generation } })
    this.generation = generation
  }
}

// ■■ response → typed error ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
interface ErrorBody { detail?: unknown }

function detailObject(body: ErrorBody): Record<string, unknown> | undefined {
  return typeof body.detail === 'object' && body.detail !== null && !Array.isArray(body.detail)
    ? (body.detail as Record<string, unknown>)
    : undefined
}

// Runtime-checked readers for the loosely-typed backend `detail` (no schema lib;
// just enough to avoid blind `as` casts on primitives).
function str(v: unknown): string | undefined {
  return typeof v === 'string' ? v : undefined
}
function num(v: unknown): number | undefined {
  return typeof v === 'number' ? v : undefined
}
function record(v: unknown): Record<string, unknown> | undefined {
  return typeof v === 'object' && v !== null && !Array.isArray(v) ? (v as Record<string, unknown>) : undefined
}

/**
 * Map the backend `workspace` detail to a `TeamErrorDetails`. Field names
 * (`type`/`role`/`is_enterprise`) mirror the product's ApiWorkspaceErrorDetails
 * (fnf-web src/module/workspace/shared/error/api-team-error-details.ts).
 */
function mapWorkspace(w: unknown): TeamErrorDetails | undefined {
  const o = record(w)
```

```
  if (!o)
    return undefined
  // Only return a team detail when BOTH fields are recognized values — never
  // fabricate Owner/Private from an absent/unknown field (that would mislabel a
  // Member as Owner). An unrecognized shape means "no team info", not a default.
  const type = str(o.type)
  const role = str(o.role)
  if ((type !== 'shared' && type !== 'private') || (role !== 'member' && role !== 'owner'))
    return undefined
  return {
    workspaceType: type === 'shared' ? 'Shared' : 'Private',
    userRole: role === 'member' ? 'Member' : 'Owner',
    // Team-plan (enterprise) users have a separate credit-purchase flow in the
    // product — the bridge needs this to route them correctly.
    ...(typeof o.is_enterprise === 'boolean' ? { isEnterprise: o.is_enterprise } : {}),
  }
}

function billingFrom(d: Record<string, unknown>): BillingContext {
  return {
    // Shared-workspace billing errors send plan_type 'Team' (capital T —
    // fnf-api src/exceptions/workspace.py:260,287 via TeamPlanFactory.team().name,
    // src/types_/team_subscription.py:149-153); every SubscriptionPlanType
    // member is lowercase, so normalize case before casting.
    plan: str(d.plan_type)?.toLowerCase() as SubscriptionPlanType | undefined,
    billingPeriod: str(d.billing_period) as BillingPeriod | undefined,
    team: mapWorkspace(d.workspace),
  }
}

// Wire error_type → the plan the backend names. Mirrors the product's native
// mapping (fnf-web src/entities/job/error/plan.ts, MinimumPlanError.assertResponse)
// exactly — the sdk-bridge forwards `data.plan` verbatim into the app's
// MinimumPlanError, so both paths must agree. The SDK reports what the backend
// means; translating legacy plan names to the new lineup (starter/plus/ultra)
// is the app/display layer's job.
const PLAN_TIERS: Record<string, { plan: SubscriptionPlanType, feature?: MinimumPlanFeature }> = {
  // Bare spelling kept defensively — current fnf-api emits only the job_-prefixed
  // 'job_ultimate_plan_required' (src/consts/response.py:17; zero bare emit sites).
  ultimate_plan_required: { plan: 'ultimate' },
  job_ultimate_plan_required: { plan: 'ultimate' },
  minimum_pro_plan_required: { plan: 'pro' },
  job_minimum_pro_plan_required: { plan: 'pro' },
  job_minimum_creator_plan_required: { plan: 'creator' },
  avatar_minimum_pro_plan_required: { plan: 'pro', feature: 'avatar' },
  minimum_basic_plan_required: { plan: 'basic' },
  job_minimum_basic_plan_required: { plan: 'basic' },
  train_minimum_basic_plan_required: { plan: 'basic', feature: 'train' },
}

const ERROR_TYPE_MAP: Record<string, (d: Record<string, unknown>) => ApiJobError> = {
  not_enough_credits: d => new OutOfCreditsError(billingFrom(d)),
  rate_limit_reached: d => new RateLimitError({ ...billingFrom(d), maxCount: num(d.concurrent_jobs_limit), isUnlimited: d.use_unlim === true }),
  batch_rate_limit_reached: d => new BatchRateLimitError(num(d.concurrent_batches_limit)),
  beat_fit_rate_limit_reached: d => new BeatFitLimitError(num(d.concurrent_batches_limit)),
  // HTTP 429 — unlim-battery concurrent-batch quota; detail carries workspace/
  // plan_type/billing_period/concurrent_batches_limit (fnf-api src/exceptions/auth.py:145-170).
  unlim_battery_rate_limit_reached: d => new ApiJobError('unlim_battery_rate_limit_reached', 'Unlimited battery rate limit reached', { data: { ...billingFrom(d), concurrentBatchesLimit: num(d.concurre
  // HTTP 429 — enterprise workspace group credit limit; detail { text, limit_amount,
  // period, spent_in_period, cost } (fnf-api src/exceptions/credit_limit.py:30-56).
  enterprise_group_credit_limit_reached: d => new ApiJobError('enterprise_group_credit_limit_reached', str(d.text) ?? 'Workspace group credit limit reached', { data: { limitAmount: num(d.limit_amount)
  workspace_subscription_inactive: d => new SharedTeamSubscriptionInactiveError(billingFrom(d)),
  workspace_member_spend_paused: d => new WorkspaceMemberSpendPausedError({
    workspaceId: str(d.workspace_id),
    userId: str(d.user_id),
    pausedAt: str(d.paused_at) ?? null,
    pausedByUserId: str(d.paused_by_user_id) ?? null,
  }),
  subscription_renewal_failed: () => new SubscriptionRenewalFailedError(),
  unlimited_generation_not_allowed: () => new UnlimitedGenerationNotAllowedError(),
```

```
  // HTTP 403, bare { error_type } (fnf-api src/exceptions/job.py:70-75).
  free_generation_not_allowed: () => new ApiJobError('free_generation_not_allowed', 'Free generation is not allowed for this request'),
  // HTTP 403, bare { error_type } (fnf-api src/exceptions/job.py:86-91).
  generation_not_available: () => new ApiJobError('generation_not_available', 'Generation is not available'),
  grace_daily_limit_reached: d => new GraceDailyLimitError((str(record(d.data)?.type) as GraceLimitReachedType) ?? 'unlock_full_access_notice', d.data),
  // 'nsfw' is the only emitted wire type — 400 submit sites (fnf-api
  // src/handlers/job.py:160 et al.) and 422 media sites with detail.text
  // 'Restricted content detected' (src/handlers/media.py:274-281).
  // 'prompt_nsfw' is never emitted by current fnf-api (zero sites) — kept as a
  // defensive alias only.
  nsfw: () => new PromptNsfwError(),
  prompt_nsfw: () => new PromptNsfwError(),
  // Defensive: current fnf-api never emits 'ip_detected' as an HTTP error_type
  // (zero sites — it exists only as a 2xx confirm-body status and a job status).
  // Kept for the legacy 422 error-body shape fnf-web still defends against
  // (image-api/video-api read detail.error_type at presign/confirm). Backend
  // message wins when present.
  ip_detected: d => new IpDetectedError(str(d.message) ?? str(d.text)),
  ip_check_rate_limit_reached: d => new IpCheckRateLimitError(str(d.message) ?? str(d.text)),
  // HTTP 413 — detail { error_type, max_size_mb } (fnf-api src/utils/media_size.py:16-22).
  media_too_large: d => new ApiJobError('media_too_large', 'Media is too large', { data: { maxSizeMb: num(d.max_size_mb) } }),
  // HTTP 408 — content (IP) check timed out, retryable; detail { error_type, text }
  // (fnf-api src/handlers/media.py:282-290).
  request_timeout: d => new ApiJobError('request_timeout', str(d.text) ?? 'Request timed out — please try again'),
  non_eligible_vpn_detected: () => new VpnDetectedError(),
  not_enough_boost_credits: () => new NotEnoughBoostCreditsError(),
  auto_top_up_suspended: d => new AutoTopUpSuspendedError((str(d.error_type) as AutoTopUpErrorType) ?? 'auto_top_up_suspended'),
  auto_top_up_charge_failed: () => new AutoTopUpSuspendedError('auto_top_up_charge_failed'),
  account_suspended: () => new AccountSuspendedError('account_suspended'),
  account_blocked: () => new AccountSuspendedError('account_blocked'),
  job_in_progress: () => new JobInProgressError(),
  workspace_selection_required: d => new WorkspaceSelectionRequiredError(Array.isArray(d.workspaces) ? d.workspaces as Array<{ id: string, name: string | null, type: 'private' | 'shared' }> : []),
  other: d => new ApiMessageError(str(d.text) ?? 'Request failed'),
}

function errorType(body: ErrorBody): string | undefined {
  const d = detailObject(body)
  return typeof d?.error_type === 'string' ? d.error_type : undefined
}

function validationMessage(body: ErrorBody): string {
  const detail = body.detail
  if (Array.isArray(detail) && detail.length > 0) {
    const msg = (detail[0] as { msg?: unknown }).msg
    if (typeof msg === 'string')
      return msg
  }
  return 'Validation failed'
}

export function errorFromResponse(status: number, body: unknown): ApiJobError | null {
  const error = mapResponse(status, body)
  // Most factories receive only the detail object — stamp the HTTP status so
  // retry/telemetry keyed on err.status works (preserves any hard-coded status).
  if (error && error.status == null)
    (error as { status?: number }).status = status
  return error
}

function mapResponse(status: number, body: unknown): ApiJobError | null {
  if (status >= 200 && status < 300)
    return null
  const safe = (typeof body === 'object' && body !== null ? body : {}) as ErrorBody

  // A bare string `detail` is a server message.
  if (typeof safe.detail === 'string')
    return new ApiMessageError(safe.detail)

  const detail = detailObject(safe)
  if (detail?.message === 'KYB verification required')
```

```
    return new KybVerificationRequiredError()

  // Minimum-plan: many error_types collapse to one class (plan/feature in data).
  const type = errorType(safe)
  if (type && PLAN_TIERS[type])
    return new MinimumPlanError(PLAN_TIERS[type])
  if (type && ERROR_TYPE_MAP[type])
    return ERROR_TYPE_MAP[type](detail ?? {})

  if (status === 422)
    return new ValidationError(validationMessage(safe), safe.detail)

  // No blanket 409 → JobInProgressError: 409 is also workspace_selection_required.
  // job_in_progress is reachable only via its error_type (see ERROR_TYPE_MAP);
  // an unrecognized 409 (or any other status) keeps its body for diagnosis.
  return new UnknownSubmitResponseError(status, body)
}

// ■■ JSON → typed error (rehydrate across a worker/Comlink boundary by `code`) ■■
const CODE_TO_CLASS: Record<string, (j: ApiJobErrorJSON<any>) => ApiJobError> = {
  out_of_credits: j => new OutOfCreditsError(j.data),
  rate_limit: j => new RateLimitError(j.data),
  workspace_subscription_inactive: j => new SharedTeamSubscriptionInactiveError(j.data ?? {}),
  minimum_plan_required: j => new MinimumPlanError(j.data ?? {}),
  prompt_nsfw: () => new PromptNsfwError(),
  ip_detected: j => new IpDetectedError(j.message),
  ip_check_rate_limit_reached: j => new IpCheckRateLimitError(j.message),
  vpn_detected: () => new VpnDetectedError(),
  kyb_verification_required: () => new KybVerificationRequiredError(),
  not_enough_boost_credits: () => new NotEnoughBoostCreditsError(),
  subscription_renewal_failed: () => new SubscriptionRenewalFailedError(),
  job_in_progress: () => new JobInProgressError(),
  unlimited_generation_not_allowed: () => new UnlimitedGenerationNotAllowedError(),
  batch_rate_limit: j => new BatchRateLimitError(j.data?.concurrentBatchesLimit),
  beat_fit_limit: j => new BeatFitLimitError(j.data?.concurrentBatchesLimit),
  grace_daily_limit: j => new GraceDailyLimitError(j.data?.graceType, j.data?.graceData),
  auto_top_up_suspended: j => new AutoTopUpSuspendedError(j.data?.autoTopUpType ?? 'auto_top_up_suspended'),
  workspace_member_spend_paused: j => new WorkspaceMemberSpendPausedError(j.data ?? {}),
  account_suspended: j => new AccountSuspendedError(j.data?.suspendedCode ?? 'account_suspended'),
  workspace_selection_required: j => new WorkspaceSelectionRequiredError(j.data?.workspaces ?? []),
  api_message: j => new ApiMessageError(j.message),
  validation: j => new ValidationError(j.message, j.data?.issues),
  // 'unknown' must rehydrate to the class — UnknownSubmitResponseError.is()
  // has to keep working across a Comlink boundary, body and URL intact.
  unknown: j => new UnknownSubmitResponseError(j.status, j.data?.responseBody, j.data?.responseUrl),
  aborted: j => new JobAbortedError(j.message),
  confirmation_rejected: j => new ConfirmationRejectedError(j.message),
  timeout: (j) => {
    // Preserve the serialized message — the constructor would otherwise rebuild
    // it as "within 0ms" (the original timeoutMs isn't carried in the payload).
    const e = new JobTimeoutError(j.data?.generation?.id ?? 'unknown', 0, j.data?.generation)
    if (j.message)
      e.message = j.message
    return e
  },
}

/**
 * Register a rehydrator for an error code. Used by the media half (and custom
 * error subclasses) so `errorFromJSON` restores `instanceof`/`.is()` identity
 * without a circular import into this module.
 */
export function registerErrorCode(code: string, make: (j: ApiJobErrorJSON<any>) => ApiJobError): void {
  CODE_TO_CLASS[code] = make
}

export function errorFromJSON(json: ApiJobErrorJSON<any>): ApiJobError {
  const make = CODE_TO_CLASS[json.code]
  const error = make ? make(json) : new ApiJobError(json.code, json.message, { status: json.status, data: json.data })
  // Factories rebuild from `data` and may drop the serialized status — restore it.
```

```
  if (error.status == null && json.status != null)
    (error as { status?: number }).status = json.status
  return error
}
```

app/packages/fnf/src/group.ts

```
export interface FieldDef {
  wire: string
}

export function field(wire: string): FieldDef {
  return { wire }
}

export interface Codec<T> {
  serialize: (value: T) => Record<string, unknown>
  parse: (wire: Record<string, unknown>) => T
  wireKeys: string[]
}

export function group<M extends Record<string, FieldDef>>(
  map: M,
): Codec<{ [K in keyof M]?: unknown }> {
  const entries = Object.entries(map)
  const wireKeys = entries.map(([, def]) => def.wire)

  return {
    wireKeys,
    serialize(value) {
      const out: Record<string, unknown> = {}
      for (const [key, def] of entries) {
        const v = (value as Record<string, unknown> | null | undefined)?.[key]
        if (v !== undefined)
          out[def.wire] = v
      }
      return out
    },
    parse(wire) {
      const out: Record<string, unknown> = {}
      for (const [key, def] of entries) {
        if (wire[def.wire] !== undefined)
          out[key] = wire[def.wire]
      }
      return out as { [K in keyof M]?: unknown }
    },
  }
}
```

app/packages/fnf/src/groups/__tests__/media-meta.test.ts

```
import type { MediaRef } from '../../types'
import { describe, expect, it } from 'vitest'
import { resolveMediaMeta } from '../../media-meta'
import { checkMedia, dimensionsWithin, durationsWithin } from '../media'

function ref(id: string, meta?: MediaRef['meta']): MediaRef {
  return { id, type: 'media_input', url: `https://cdn/${id}.png`, ...(meta ? { meta } : {}) }
}

const CFG = {
  field: 'medias',
  format: 'wrapped' as const,
  roles: ['image', 'start_image', 'video', 'audio'],
  rules: [
    dimensionsWithin(['image', 'start_image'], { minSide: 300, maxSide: 6000, ratio: [0.4, 2.5] }),
    dimensionsWithin(['video'], { minSide: 300, maxSide: 6000, minPixels: 409_600, ratio: [0.4, 2.5] }),
    durationsWithin(['video', 'audio'], { each: [2, 15], total: 15 }),
  ],
}

describe('dimensionsWithin', () => {
  it('flags too-small / too-large / off-ratio refs with human labels', () => {
    const issues = checkMedia(CFG, {
      image: [ref('ok', { width: 1024, height: 1024 }), ref('tiny', { width: 100, height: 400 })],
      start_image: [ref('wide', { width: 3000, height: 1000 })],
    })
    expect(issues.map(i => i.msg)).toEqual([
      'Image 2 is too small — the minimum dimension is 300px',
      'Start image aspect ratio must be between 0.4 and 2.5',
    ])
  })

  it('checks the pixel floor only where declared (videos, not images)', () => {
    // 500×500 = 250k pixels: fine as an image, below the 409.6k video floor
    expect(checkMedia(CFG, { image: [ref('a', { width: 500, height: 500 })] })).toEqual([])
    const issues = checkMedia(CFG, { video: [ref('v', { width: 500, height: 500 })] })
    expect(issues[0].msg).toContain('resolution is too low')
  })

  it('skips refs without meta — local knowledge is optional', () => {
    expect(checkMedia(CFG, { image: [ref('unknown')] })).toEqual([])
  })
})

describe('durationsWithin', () => {
  it('flags per-ref bounds and the combined budget', () => {
    const issues = checkMedia(CFG, {
      image: [ref('i', { width: 1024, height: 1024 })],
      video: [ref('v1', { width: 1024, height: 1024, durationSec: 1 }), ref('v2', { width: 1024, height: 1024, durationSec: 9 })],
      audio: [ref('a1', { durationSec: 8 })],
    })
    expect(issues.map(i => i.msg)).toEqual([
      'Video 1 must be between 2 and 15 seconds',
      'combined video + audio duration must be at most 15s, got 18s',
    ])
  })

  it('unknown durations do not trip the bounds or the budget', () => {
    expect(checkMedia(CFG, { video: [ref('v')], audio: [ref('a')] })).toEqual([])
  })
})

describe('resolveMediaMeta', () => {
  const resolver = async (r: MediaRef) => (r.id === 'fails' ? Promise.reject(new Error('boom')) : { width: 640, height: 640 })

  it('fills only refs missing meta, in a NEW input (no mutation)', async () => {
    const input = {
      model: 'demo',
```

```
      media: {
        image: [ref('a'), ref('b', { width: 1, height: 1 })],
        start_image: ref('single'), // non-array values stay non-array
      },
      settings: {},
    }
    const out = await resolveMediaMeta(input, resolver)

    expect(out).not.toBe(input)
    expect((out.media.image as MediaRef[])[0].meta).toEqual({ width: 640, height: 640 })
    expect((out.media.image as MediaRef[])[1].meta).toEqual({ width: 1, height: 1 }) // untouched
    expect((out.media.start_image as MediaRef).meta).toEqual({ width: 640, height: 640 })
    expect((input.media.image as MediaRef[])[0].meta).toBeUndefined() // original intact
  })

  it('a failing resolver leaves that ref as-is instead of failing the step', async () => {
    const out = await resolveMediaMeta({ media: { image: [ref('fails'), ref('ok')] } }, resolver)
    const [failed, ok] = out.media!.image as MediaRef[]
    expect(failed.meta).toBeUndefined()
    expect(ok.meta).toEqual({ width: 640, height: 640 })
  })

  it('passes through inputs without media', async () => {
    const input = { model: 'demo', settings: {}, media: undefined }
    expect(await resolveMediaMeta(input, resolver)).toBe(input)
  })
})

describe('integration: the seedance-2-0 declaration', () => {
  it('meta violations fail buildWireParams as one aggregated ValidationError', async () => {
    const { buildWireParams } = await import('../../spec')
    const { seedance2_0 } = await import('../../jobs/seedance-2-0')
    const input = {
      model: 'seedance_2_0',
      prompt: { instruction: 'x' },
      media: {
        start_image: [ref('frame', { width: 100, height: 100 })], // below 300px
        audio: [ref('a', { durationSec: 20 })], // above 15s
      },
      settings: { duration: 5 as const, aspectRatio: 'auto' as const },
    }
    expect(() => buildWireParams(input, seedance2_0)).toThrowError(/minimum dimension is 300px/)
    expect(() => buildWireParams(input, seedance2_0)).toThrowError(/between 2 and 15 seconds/)
  })

  it('meta never reaches the wire — the codec sends only id/type/url', async () => {
    const { buildWireParams } = await import('../../spec')
    const { seedance2_0 } = await import('../../jobs/seedance-2-0')
    const wire = buildWireParams({
      model: 'seedance_2_0',
      prompt: { instruction: 'a cube' },
      media: { start_image: [ref('frame', { width: 1024, height: 1024 })] },
      settings: { duration: 5 as const, aspectRatio: 'auto' as const },
    }, seedance2_0)
    const items = wire.medias as Array<{ role: string, data: Record<string, unknown> }>
    expect(items[0].data).toEqual({ id: 'frame', type: 'media_input', url: 'https://cdn/frame.png' })
  })
})

describe('createDomMediaMetaResolver outside a DOM', () => {
  it('resolves to undefined instead of throwing (safe to wire unconditionally)', async () => {
    const { createDomMediaMetaResolver } = await import('../../media/dom-meta-resolver')
    expect(await createDomMediaMetaResolver()(ref('x'))).toBeUndefined()
  })
})

describe('createDomMediaMetaResolver in a DOM', () => {
  /** Stub the structural DOM the resolver probes; `loads[n]` scripts attempt n. */
  function stubDom(loads: boolean[]) {
    const attempts = { count: 0 }
```

```
  class FakeImage {
    naturalWidth = 800
    naturalHeight = 600
    onload: (() => void) | null = null
    onerror: (() => void) | null = null
    #src = ''

    get src() {
      return this.#src
    }

    set src(value: string) {
      this.#src = value
      const ok = loads[Math.min(attempts.count++, loads.length - 1)]
      queueMicrotask(() => (ok ? this.onload?.() : this.onerror?.()))
    }
  }
  ;(globalThis as Record<string, unknown>).Image = FakeImage
  ;(globalThis as Record<string, unknown>).document = { createElement: () => ({}) }
  return {
    attempts,
    restore: () => {
      delete (globalThis as Record<string, unknown>).Image
      delete (globalThis as Record<string, unknown>).document
    },
  }
}

function imageRef(): MediaRef {
  return { id: 'frame', type: 'image', url: 'blob:frame' }
}

it('a failed measurement resolves undefined (the MediaMetaResolver contract), never rejects', async () => {
  const dom = stubDom([false])
  try {
    const { createDomMediaMetaResolver } = await import('../../media/dom-meta-resolver')
    expect(await createDomMediaMetaResolver()(imageRef())).toBeUndefined()
  }
  finally {
    dom.restore()
  }
})

it('failures are not cached — a retry of the same url re-measures', async () => {
  const dom = stubDom([false, true])
  try {
    const { createDomMediaMetaResolver } = await import('../../media/dom-meta-resolver')
    const resolve = createDomMediaMetaResolver()
    expect(await resolve(imageRef())).toBeUndefined()
    expect(await resolve(imageRef())).toEqual({ width: 800, height: 600 })
    expect(dom.attempts.count).toBe(2)
  }
  finally {
    dom.restore()
  }
})

it('successes ARE cached — one measurement per url', async () => {
  const dom = stubDom([true])
  try {
    const { createDomMediaMetaResolver } = await import('../../media/dom-meta-resolver')
    const resolve = createDomMediaMetaResolver()
    expect(await resolve(imageRef())).toEqual({ width: 800, height: 600 })
    expect(await resolve(imageRef())).toEqual({ width: 800, height: 600 })
    expect(dom.attempts.count).toBe(1)
  }
  finally {
    dom.restore()
  }
})
```

```
})
```

app/packages/fnf/src/groups/__tests__/media.test.ts

```
import { describe, expect, it } from 'vitest'
import { atLeastOneOf, checkMedia, mediaCodec, requiresOneOf } from '../media'

const ref = (id: string, role: string) => ({ id, type: 'media_input', url: `https://x/${id}`, role })

describe('mediaCodec', () => {
  it('unwrapped: emits bare data array under the field', () => {
    const codec = mediaCodec({ field: 'input_images', format: 'unwrapped', roles: ['image'] })
    const wire = codec.serialize({ image: [ref('a', 'image'), ref('b', 'image')] })
    expect(wire).toEqual({ input_images: [{ id: 'a', type: 'media_input', url: 'https://x/a' }, { id: 'b', type: 'media_input', url: 'https://x/b' }] })
  })

  it('wrapped: emits {role,data} entries under the field', () => {
    const codec = mediaCodec({ field: 'medias', format: 'wrapped', roles: ['start_image', 'end_image'] })
    const wire = codec.serialize({ start_image: ref('a', 'start_image'), end_image: ref('b', 'end_image') })
    expect(wire).toEqual({
      medias: [
        { role: 'start_image', data: { id: 'a', type: 'media_input', url: 'https://x/a' } },
        { role: 'end_image', data: { id: 'b', type: 'media_input', url: 'https://x/b' } },
      ],
    })
  })

  it('single: emits one data object under the field', () => {
    const codec = mediaCodec({ field: 'input_image', format: 'single', roles: ['image'] })
    expect(codec.serialize({ image: ref('a', 'image') })).toEqual({
      input_image: { id: 'a', type: 'media_input', url: 'https://x/a' },
    })
  })

  it('emits nothing when no media is present', () => {
    const codec = mediaCodec({ field: 'input_images', format: 'unwrapped', roles: ['image'] })
    expect(codec.serialize({})).toEqual({})
  })

  it('wrapped round-trips role and id', () => {
    const codec = mediaCodec({ field: 'medias', format: 'wrapped', roles: ['start_image', 'end_image'] })
    const input = { start_image: [ref('a', 'start_image')], end_image: [ref('b', 'end_image')] }
    const parsed = codec.parse(codec.serialize(input))
    expect(parsed.start_image).toEqual([{ id: 'a', type: 'media_input', url: 'https://x/a', role: 'start_image' }])
    expect(parsed.end_image).toEqual([{ id: 'b', type: 'media_input', url: 'https://x/b', role: 'end_image' }])
  })

  it('single: rejects multiple refs instead of silently dropping all but the first', () => {
    const codec = mediaCodec({ field: 'input_image', format: 'single', roles: ['image'] })
    expect(() => codec.serialize({ image: [ref('a', 'image'), ref('b', 'image')] }))
      .toThrow(/exactly one ref, got 2/)
  })

  it('rejects an undeclared role instead of silently dropping it (typos, get-then-resubmit)', () => {
    const codec = mediaCodec({ field: 'medias', format: 'wrapped', roles: ['image'] })
    expect(() => codec.serialize({ style_ref: ref('a', 'style_ref') })).toThrow(/not declared by this job: style_ref/)
  })
})

describe('checkMedia (cardinality + cross-role rules)', () => {
  const cfg = {
    field: 'medias',
    format: 'wrapped' as const,
    roles: ['image', 'start_image', 'audio'],
    counts: { start_image: { min: 1, max: 1 }, image: { max: 2 } },
    rules: [requiresOneOf('audio', ['image'])],
  }

  it('passes a valid combination', () => {
    expect(checkMedia(cfg, { start_image: ref('s', 'start_image'), image: [ref('a', 'image')], audio: ref('m', 'audio') })).toEqual([])
  })
```

```
  it('reports min/max violations with pydantic-shaped issues', () => {
    const issues = checkMedia(cfg, { image: [ref('a', 'image'), ref('b', 'image'), ref('c', 'image')] })
    expect(issues).toEqual([
      { loc: ['media', 'start_image'], msg: expect.stringMatching(/at least 1.*got 0/) },
      { loc: ['media', 'image'], msg: expect.stringMatching(/at most 2.*got 3/) },
    ])
  })

  it('requiresOneOf fires only when the dependent role is present', () => {
    const rule = requiresOneOf('audio', ['image', 'video'])
    expect(rule({ audio: 0, image: 0, video: 0 }, {})).toBeNull() // no audio — nothing required
    expect(rule({ audio: 1, image: 0, video: 0 }, {})).toMatch(/requires one of: image, video/)
    expect(rule({ audio: 1, image: 1, video: 0 }, {})).toBeNull()
  })

  it('atLeastOneOf requires the group', () => {
    const rule = atLeastOneOf(['start_image', 'end_image'])
    expect(rule({ start_image: 0, end_image: 0 }, {})).toMatch(/at least one of/)
    expect(rule({ start_image: 1, end_image: 0 }, {})).toBeNull()
  })
})
```

app/packages/fnf/src/groups/__tests__/prompt.test.ts

```
import { describe, expect, it } from 'vitest'
import { promptCodec } from '../prompt'

describe('promptCodec', () => {
  it('maps structured prompt to flat wire keys', () => {
    expect(promptCodec.serialize({ instruction: 'a cat', enhance: true, negative: 'blur' })).toEqual({
      prompt: 'a cat',
      enhance_prompt: true,
      negative_prompt: 'blur',
    })
  })

  it('parses flat wire keys back to structured prompt', () => {
    expect(promptCodec.parse({ prompt: 'a cat', enhance_prompt: true, system_prompt: 'be brief' })).toEqual({
      instruction: 'a cat',
      enhance: true,
      system: 'be brief',
    })
  })

  it('round-trips losslessly', () => {
    const x = { instruction: 'dog', enhance: false, negative: 'noise', system: 'x' }
    expect(promptCodec.parse(promptCodec.serialize(x))).toEqual(x)
  })
})
```

app/packages/fnf/src/groups/media.ts

```
import type { Codec } from '../group'
import type { MediaFormat, MediaInput, MediaRef } from '../types'
import { ValidationError } from '../errors'

/**
 * A cross-role rule: gets the per-role ref counts (absent role = 0) plus the
 * refs themselves (for rules over `MediaRef.meta`), and returns one or more
 * problem messages, or null when satisfied. Ship combinators cover the known
 * product rules — cardinality (`requiresOneOf`, `atLeastOneOf`, `maxTotal`)
 * and meta (`dimensionsWithin`, `durationsWithin`); a custom rule is just a
 * function with this signature.
 */
export type MediaRule<Roles extends string = string> = (
  counts: Record<Roles, number>,
  refs: Partial<Record<Roles, MediaRef[]>>,
) => string | string[] | null

/** Pydantic-shaped issue — the same shape backend 422 details use. */
export interface MediaIssue {
  loc: string[]
  msg: string
}

export interface MediaConfig {
  field: string
  format: MediaFormat
  roles: readonly string[]
  /** Per-role cardinality: `min >= 1` makes the role required; `max` caps it. */
  counts?: Partial<Record<string, { min?: number, max?: number }>>
  /** Cross-role rules, checked on submit after cardinality. */
  rules?: readonly MediaRule[]
}

/** Cross-role combinator: `role`, when present, needs at least one of `anyOf`. */
export function requiresOneOf<R extends string>(role: R, anyOf: readonly R[]): MediaRule<R> {
  return (counts) => {
    if ((counts[role] ?? 0) === 0)
      return null
    return anyOf.some(dep => (counts[dep] ?? 0) > 0)
      ? null
      : `media role '${role}' requires one of: ${anyOf.join(', ')}`
  }
}

/** Cross-role combinator: at least one of `roles` must be present. */
export function atLeastOneOf<R extends string>(roles: readonly R[]): MediaRule<R> {
  return counts => roles.some(role => (counts[role] ?? 0) > 0)
    ? null
    : `at least one of media roles is required: ${roles.join(', ')}`
}

/** Cross-role combinator: the refs across `roles` may not exceed `max` in total. */
export function maxTotal<R extends string>(roles: readonly R[], max: number): MediaRule<R> {
  return (counts) => {
    const total = roles.reduce((sum, role) => sum + (counts[role] ?? 0), 0)
    return total <= max ? null : `media roles ${roles.join('+')} take at most ${max} ref(s) in total, got ${total}`
  }
}

// ■■ meta rules ■■
//
// These judge `MediaRef.meta` (intrinsic size/duration). A ref whose meta is
// missing is SKIPPED — meta is local knowledge, not a requirement; the backend
// re-validates regardless. Opt in to `resolveMediaMeta` before checking when
// you want every ref measured.

export interface DimensionLimits {
  /** Both sides must be at least this many pixels. */
```

```
  minSide?: number
  /** Neither side may exceed this many pixels. */
  maxSide?: number
  /** Total pixel floor (width × height) — fnf applies it to video inputs. */
  minPixels?: number
  /** Allowed width/height ratio window, e.g. [0.4, 2.5] (2:5 … 5:2). */
  ratio?: readonly [number, number]
}

/** Meta combinator: refs in `roles` whose known size violates `limits`. */
export function dimensionsWithin<R extends string>(roles: readonly R[], limits: DimensionLimits): MediaRule<R> {
  return (_counts, refs) => {
    const problems: string[] = []
    for (const { label, ref } of labeled(roles, refs)) {
      const { width, height } = ref.meta ?? {}
      if (width == null || height == null)
        continue // unknown size — nothing to judge locally
      // The else-if chain below is DELIBERATE — one issue per ref, first
      // violated constraint wins, mirroring fnf-web's getMediaConstraintError
      // (built.ts early-returns in the same order). Don't flatten into
      // independent ifs: issue counts/messages would diverge from the product.
      if (limits.minSide != null && (width < limits.minSide || height < limits.minSide))
        problems.push(`${label} is too small — the minimum dimension is ${limits.minSide}px`)
      else if (limits.maxSide != null && (width > limits.maxSide || height > limits.maxSide))
        problems.push(`${label} is too large — the maximum dimension is ${limits.maxSide}px`)
      else if (limits.minPixels != null && width * height < limits.minPixels)
        problems.push(`${label} resolution is too low — at least ${limits.minPixels} pixels are required`)
      else if (limits.ratio && (width / height < limits.ratio[0] || width / height > limits.ratio[1]))
        problems.push(`${label} aspect ratio must be between ${limits.ratio[0]} and ${limits.ratio[1]}`)
    }
    return problems.length > 0 ? problems : null
  }
}

/** Meta combinator: per-ref duration bounds and/or a combined budget across `roles`. */
export function durationsWithin<R extends string>(
  roles: readonly R[],
  bounds: { each?: readonly [number, number], total?: number },
): MediaRule<R> {
  return (_counts, refs) => {
    const problems: string[] = []
    let combined = 0
    for (const { label, ref } of labeled(roles, refs)) {
      const seconds = ref.meta?.durationSec
      if (seconds == null)
        continue // unknown duration — nothing to judge locally
      combined += seconds
      if (bounds.each && (seconds < bounds.each[0] || seconds > bounds.each[1]))
        problems.push(`${label} must be between ${bounds.each[0]} and ${bounds.each[1]} seconds`)
    }
    if (bounds.total != null && combined > bounds.total)
      problems.push(`combined ${roles.join(' + ')} duration must be at most ${bounds.total}s, got ${round1(combined)}s`)
    return problems.length > 0 ? problems : null
  }
}

/** Flatten role buckets into human-labeled refs: ('start_image', 2nd) → "Start image 2". */
function* labeled<R extends string>(roles: readonly R[], refs: Partial<Record<R, MediaRef[]>>): Generator<{ label: string, ref: MediaRef }> {
  for (const role of roles) {
    const bucket = refs[role] ?? []
    for (let i = 0; i < bucket.length; i++) {
      const name = role.replace(/_/g, ' ')
      yield { label: `${name.charAt(0).toUpperCase()}${name.slice(1)}${bucket.length > 1 ? ` ${i + 1}` : ''}`, ref: bucket[i] }
    }
  }
}

function round1(value: number): number {
  return Math.round(value * 10) / 10
}
```

```
/**
 * Validate cardinality + cross-role rules against a submit's media input.
 * Pure — returns ALL problems (not first-throw); the codec stays a dumb mapper.
 */
export function checkMedia(cfg: MediaConfig, media: MediaInput | undefined): MediaIssue[] {
  const counts: Record<string, number> = {}
  const refs: Record<string, MediaRef[]> = {}
  for (const role of cfg.roles) {
    refs[role] = asArray(media?.[role])
    counts[role] = refs[role].length
  }

  const issues: MediaIssue[] = []
  for (const [role, limit] of Object.entries(cfg.counts ?? {})) {
    const count = counts[role] ?? 0
    if (limit?.min != null && count < limit.min)
      issues.push({ loc: ['media', role], msg: `media role '${role}' requires at least ${limit.min} ref(s), got ${count}` })
    if (limit?.max != null && count > limit.max)
      issues.push({ loc: ['media', role], msg: `media role '${role}' takes at most ${limit.max} ref(s), got ${count}` })
  }
  for (const rule of cfg.rules ?? []) {
    const problems = rule(counts, refs)
    for (const msg of problems == null ? [] : Array.isArray(problems) ? problems : [problems])
      issues.push({ loc: ['media'], msg })
  }
  return issues
}

interface WireData {
  id: string
  type: string
  url?: string
}

function toData(ref: MediaRef): WireData {
  const data: WireData = { id: ref.id, type: ref.type }
  if (ref.url !== undefined)
    data.url = ref.url
  return data
}

function asArray(value: MediaRef | MediaRef[] | undefined): MediaRef[] {
  if (!value)
    return []
  return Array.isArray(value) ? value : [value]
}

function pushRole(media: MediaInput, role: string, ref: MediaRef): void {
  const existing = media[role]
  if (Array.isArray(existing))
    existing.push(ref)
  else media[role] = [ref]
}

export function mediaCodec(cfg: MediaConfig): Codec<MediaInput> {
  return {
    wireKeys: [cfg.field],
    serialize(media) {
      // Loud, not lossy: a role outside the declaration would be silently
      // dropped by the role loop below (typos, or a ref parsed from a newer
      // backend) — reject it instead so get-then-resubmit can't lose media.
      const undeclared = Object.keys(media ?? {}).filter(role => media[role] !== undefined && !cfg.roles.includes(role))
      if (undeclared.length > 0)
        throw new ValidationError(`Media role(s) not declared by this job: ${undeclared.join(', ')} (allowed: ${cfg.roles.join(', ')})`)

      const items: { role: string, ref: MediaRef }[] = []
      for (const role of cfg.roles) {
        for (const ref of asArray(media?.[role])) items.push({ role, ref })
      }
      if (items.length === 0)
```

```
        return {}
      if (cfg.format === 'single') {
        if (items.length > 1)
          throw new ValidationError(`Media format 'single' takes exactly one ref, got ${items.length}`)
        return { [cfg.field]: toData(items[0].ref) }
      }
      if (cfg.format === 'wrapped') {
        return { [cfg.field]: items.map(i => ({ role: i.role, data: toData(i.ref) })) }
      }
      return { [cfg.field]: items.map(i => toData(i.ref)) }
    },
    parse(wire) {
      const media: MediaInput = {}
      const raw = wire[cfg.field]
      if (raw == null)
        return media
      const arr = Array.isArray(raw) ? raw : [raw]
      for (const entry of arr) {
        if (cfg.format === 'wrapped') {
          const w = entry as { role?: string, data?: WireData }
          const role = w.role ?? cfg.roles[0]
          const data = w.data ?? ({} as WireData)
          pushRole(media, role, { ...data, role })
        }
        else {
          const data = entry as WireData
          const role = cfg.roles[0]
          pushRole(media, role, { ...data, role })
        }
      }
      return media
    },
  }
}
```

app/packages/fnf/src/groups/prompt.ts

```
import { field, group } from '../group'

export const promptCodec = group({
  instruction: field('prompt'),
  enhance: field('enhance_prompt'),
  negative: field('negative_prompt'),
  system: field('system_prompt'),
})
```

app/packages/fnf/src/headers.ts

```
/** Shared auth and identity header construction for HTTP adapters. */

type MaybePromise<T> = T | Promise<T>
/** Header value that must resolve to a string: static, or a lazy/async accessor. */
export type RequiredHeaderSource = string | (() => MaybePromise<string>)
/** Header value that may be absent: static string, or a lazy/async accessor returning null/undefined to omit the header. */
export type OptionalHeaderSource = string | (() => MaybePromise<string | null | undefined>)

export async function authorizationHeader(getToken: (() => Promise<string | null>) | undefined): Promise<Record<string, string | undefined>> {
  const token = await getToken?.()
  return token ? { Authorization: `Bearer ${token}` } : {}
}

export async function optionalNamedHeader(name: string, source: OptionalHeaderSource | undefined): Promise<Record<string, string | undefined>> {
  const value = typeof source === 'function' ? await source() : source
  return typeof value === 'string' && value.trim() !== '' ? { [name]: value } : {}
}

export function cleanHeaders(headers: Record<string, string | undefined>): Record<string, string> {
  const out: Record<string, string> = {}
  for (const [key, value] of Object.entries(headers)) {
    if (value)
      out[key] = value
  }
  return out
}
```

app/packages/fnf/src/index.ts

```
// Backend ports (interfaces — every concrete adapter lives in
// @higgsfield/fnf-adapters, EXCEPT the one bundled below)
export type { CharacterBackend, ConfirmMediaRequest, ConfirmSubmit, ConfirmSubmitRequest, FnfAdapter, GenerationBackend, JobListQuery, MediaBackend, MediaGetQuery, MediaListQuery, ProfileBackend, Real
// Custom-reference character training — also available at "@higgsfield/fnf/characters".
export { buildCharacterCreateRequest, createCharacterClient, parseCharacter } from './characters'
export type { Character, CharacterClient, CharacterClientConfig, CharacterCreateRequest, CharacterStatus, CharacterType, CharacterWaitOptions, CreateCharacterInput } from './characters'
// Elements library — also available at "@higgsfield/fnf/references".
export { createReferenceClient, parseReference } from './references'
export type { Reference, ReferenceCategory, ReferenceClient, ReferenceClientConfig, ReferenceListQuery, ReferenceListResult, ReferenceMedia, ReferenceStatus, ReferenceWaitOptions } from './references'
// Stateful realtime image editing and saved custom styles — also available at
// "@higgsfield/fnf/realtime".
export {
  buildCreateRealtimeCustomStyleRequest,
  buildRealtimeChainCostRequest,
  buildRealtimeChainEditRequest,
  buildRealtimeChainFinalizeRequest,
  buildUpdateRealtimeCustomStyleRequest,
  createRealtimeClient,
  parseRealtimeChainCostEstimate,
  parseRealtimeChainEditResult,
  parseRealtimeChainFinalizeResult,
  parseRealtimeCustomStyle,
  parseRealtimeCustomStyleList,
  validateRealtimeCustomStyleListQuery,
} from './realtime'
export type {
  CreateRealtimeCustomStyleInput,
  RealtimeAspectRatio,
  RealtimeChainCostEstimate,
  RealtimeChainCostInput,
  RealtimeChainEditInput,
  RealtimeChainEditOptions,
  RealtimeChainEditResult,
  RealtimeChainFinalizeInput,
  RealtimeChainFinalizeResult,
  RealtimeClient,
  RealtimeClientConfig,
  RealtimeCustomStyle,
  RealtimeCustomStyleListQuery,
  RealtimeCustomStyleListResult,
  RealtimeCustomStyleReference,
  RealtimeEditParams,
  RealtimeEditPollOptions,
  RealtimeImageRef,
  RealtimeImageType,
  RealtimeMode,
  RealtimeReferenceMode,
  RealtimeResolution,
  RealtimeSettings,
  RealtimeTarget,
  UpdateRealtimeCustomStyleInput,
} from './realtime'
// Client half — also available at "@higgsfield/fnf/client" (models: "@higgsfield/fnf/jobs")
export { createContext, createJobClient, entryFor } from './client'
export { adjust, cancelGeneration, estimateCost, generationsFromBody, getGeneration, listGenerations, pollGeneration, pollJobSetGroup, safeSubmit, submit, waitGenerations } from './client'
export type {
  AdjustResult,
  BaseSubmitFields,
  ClientConfig,
  CostEstimate,
  GenerationContext,
  JobClient,
  ListOptions,
  ListResult,
  PollOptions,
  SafeSubmitResult,
  SubmitInputFor,
  SubmitResult,
```

```
  WaitOptions,
} from './client'

export { defineJob } from './define-job'

export type { JobEntry, JobParams, SettingsInput } from './define-job'
export {
  AccountSuspendedError,
  ApiJobError,
  ApiMessageError,
  AutoTopUpSuspendedError,
  BatchRateLimitError,
  BeatFitLimitError,
  BillingError,
  ConfirmationRejectedError,
  errorFromJSON,
  errorFromResponse,
  GraceDailyLimitError,
  IpCheckRateLimitError,
  IpDetectedError,
  JobAbortedError,
  JobInProgressError,
  JobTimeoutError,
  KybVerificationRequiredError,
  MinimumPlanError,
  NotEnoughBoostCreditsError,
  OutOfCreditsError,
  PromptNsfwError,
  RateLimitError,
  registerErrorCode,
  SharedTeamSubscriptionInactiveError,
  SubscriptionRenewalFailedError,
  throwIfAborted,
  UnknownSubmitResponseError,
  UnlimitedGenerationNotAllowedError,
  ValidationError,
  VpnDetectedError,
  WorkspaceMemberSpendPausedError,
  WorkspaceSelectionRequiredError,
} from './errors'
export type {
  ApiJobErrorJSON,
  AutoTopUpErrorType,
  BillingContext,
  BillingPeriod,
  GraceLimitReachedType,
  MinimumPlanFeature,
  SubscriptionPlanType,
  TeamErrorDetails,
} from './errors'

export { field, group } from './group'
export type { Codec, FieldDef } from './group'

export { atLeastOneOf, checkMedia, dimensionsWithin, durationsWithin, maxTotal, mediaCodec, requiresOneOf } from './groups/media'
export type { DimensionLimits, MediaConfig, MediaIssue, MediaRule } from './groups/media'
export { promptCodec } from './groups/prompt'
export { countRefs, intRange, oneOf, promptMax, promptRequired, randomSeed } from './jobs/checks'
export { aspectRatioDimensions, closestRatioBySize, firstSizeMeta, lookupSize, simplifyRatio } from './jobs/dimensions'
export type { AspectRatioDimensions } from './jobs/dimensions'
export { gptImage2, GptImage2AspectRatio } from './jobs/gpt-image-2'
export { grokImagine, GrokImagineAspectRatio, GrokImagineResolution, grokImagineV15 } from './jobs/grok-imagine'
export { happyHorse, HappyHorseAspectRatio, HappyHorseResolution } from './jobs/happy-horse'
export { KLING_DEFAULT_MOTION_ID, KlingModel, klingVideo } from './jobs/kling'

export { kling3_0, Kling3AspectRatio, Kling3Mode, Kling3Sound } from './jobs/kling-3'
export { kling3MotionControl, Kling3MotionControlMode, Kling3MotionControlOrientation } from './jobs/kling-3-motion-control'
export { nanoBanana2, NanoBanana2AspectRatio } from './jobs/nano-banana-2'
export { nanoBanana2Upscale, NanoBanana2UpscaleAspectRatio } from './jobs/nano-banana-2-upscale'
export { nanoBananaFlash, NanoBananaFlashAspectRatio } from './jobs/nano-banana-flash'
```

```
export { RecraftV41AspectRatio, recraftV41Image, RecraftV41Model, RecraftV41ModelType } from './jobs/recraft-v4-1'
export { seedance2_0, Seedance2AspectRatio } from './jobs/seedance-2-0'
export { getSeedance25MediaDurationIssues, getSeedance25MediaLimitIssues, seedance2_5, SEEDANCE_2_5_MAX_PROMPT_LENGTH, SEEDANCE25_MAX_AUDIO_DURATION, SEEDANCE25_MAX_AUDIOS, SEEDANCE25_MAX_IMAGES, SEED
export type { Seedance25DurationRef, Seedance25MediaCounts, Seedance25MediaDurationCode, Seedance25MediaLimitCode, Seedance25ModelValue, Seedance25ValidationCode, Seedance25ValidationIssue } from './j
export { seedreamV4_5, SeedreamV4_5AspectRatio } from './jobs/seedream-v4-5'
export { DEFAULT_SOUL_CINEMA_OLZHAS_STYLE_ID, DEFAULT_SOUL_CINEMA_STYLE_ID, DEFAULT_SOUL_V2_STYLE_ID, soulCinemaImage, SoulV2AspectRatio, soulV2Image } from './jobs/soul-v2'
export { DEFAULT_SOUL_STYLE_ID, textToImageSoul } from './jobs/text2image-soul'
export { topazImageGenerativeUpscale, TopazImageModel, topazImageUpscale, TopazImageUpscaleFactor } from './jobs/topaz-image-upscale'
export { veo3_1Lite, Veo31LiteAspectRatio, Veo31LiteResolution } from './jobs/veo-3-1-lite'
export { batch, extractAngleRefIds, extractReferenceElementIds, firstMetaDuration, firstMetaSize, integerRange, refsFor, requiredPromptOrRole } from './jobs/video-helpers'
export type { Size } from './jobs/video-helpers'
export { bytedanceVideoUpscale, BytedanceVideoUpscalePreset, BytedanceVideoUpscaleResolution, higgsfieldVideoUpscale, soraEnhanceVideo, TopazVideoEnhancementModel, TopazVideoFocusFix, TopazVideoModel,
export { wan27, Wan27AspectRatio, Wan27Quality } from './jobs/wan-2-7'
// Media half — also available at "@higgsfield/fnf/media"
export {
  ConfirmError,
  confirmMedia,
  createDomMediaMetaResolver,
  createFetchUploader,
  createMediaClient,
  createMediaContext,
  createMemoryUploader,
  defaultFilenameForContentType,
  getMedia,
  getUploadUrl,
  inferContentType,
  inferUploadType,
  listMedia,
  MediaModerationError,
  PresignError,
  resolveMedia,
  safeUploadMedia,
  transferBytes,
  uploadMedia,
  uploadMediaFromUrl,
  UploadNotSupportedError,
  UploadTransferError,
  UrlIngestError,
} from './media'
export type {
  BinaryUploader,
  MediaBytes,
  MediaClient,
  MediaClientConfig,
  MediaContext,
  MediaGetOptions,
  MediaListOptions,
  MediaListResult,
  MediaReference,
  MediaSource,
  ResolveJobRef,
  SafeUploadResult,
  UploadFromUrlInput,
  UploadInput,
  UploadModeration,
  UploadResult,
  UploadSlot,
  UploadType,
} from './media'
export { resolveMediaMeta } from './media-meta'
export type { MediaMetaResolver } from './media-meta'
export type { AdjustKind, Adjustment } from './normalize'
export {
  composeObservers,
  createConsoleObserver,
  createNoopObserver,
  createObservabilityContext,
  observeAsync,
  observeEvent,
  withObservedGenerationBackend,
```

```
  withObservedMediaBackend,
  withObservedProfileBackend,
  withObservedTransport,
  withObservedUploader,
} from './observability'

export type {
  FnfObservabilityContext,
  FnfObservabilityOptions,
  FnfObservationAttributes,
  FnfObservationAttributeValue,
  FnfObservationError,
  FnfObservationEvent,
  FnfObservationPhase,
  FnfObserver,
  ObserveAsyncOptions,
} from './observability'
// Profile/account half — also available at "@higgsfield/fnf/profile"
export {
  calculateProfileCredits,
  createProfileClient,
  createProfileContext,
  getCurrentProfileWorkspace,
  getProfileCredits,
  getProfileSnapshot,
  getProfileUser,
  getProfileWallet,
  listProfileWorkspaces,
  mapProfileUser,
  mapProfileWallet,
  mapProfileWorkspace,
  mapProfileWorkspaces,
  switchProfileWorkspace,
} from './profile'

export type {
  ProfileClient,
  ProfileClientConfig,
  ProfileContext,
  ProfileCredits,
  ProfileCreditsOptions,
  ProfileCreditsRaw,
  ProfileSnapshot,
  ProfileUser,
  ProfileWorkspace,
  ProfileWorkspaceBlock,
  ProfileWorkspaceRole,
  ProfileWorkspaceType,
  ProfileWorkspaceWallet,
  SwitchWorkspaceInput,
} from './profile'
export { buildRegistry } from './registry'

export type { Registry } from './registry'
export { getJobPhase, getMediaType, getPreviewUrl, getRawUrl, hasResult, isCompleted, isFailed, isFailedJobStatus, isFromJob, isGenerating, isTerminalJobStatus } from './selectors'

export type { JobPhase } from './selectors'
export { buildWireParams, parseGeneration, parseSettings } from './spec'
export type { JobResponse } from './spec'
export type { Transport, TransportRequest, TransportResponse } from './transport'
export { isTerminal, TERMINAL_STATUSES } from './types'

export type { Generation, GenerationInput, GenerationResults, GenerationStatus, MediaInput, MediaMeta, MediaRef, OutputType, PromptInput } from './types'
// LLM access (spec §5.2) — Worker-side only; the platform token must never
// reach the browser bundle.
export type {
  LlmBackend,
  LlmCompleteRequest,
  LlmCompletion,
  LlmDelta,
```

```
  LlmMessage,
  LlmMessagePart,
  LlmToolCall,
  LlmToolCallDelta,
  LlmToolDef,
  LlmUsage,
} from './llm/backend'
export { createLlmClient } from './llm/client'
export type { LlmClientOptions } from './llm/client'
export { errorFromLlmResponse } from './llm/errors'

// The ONE bundled adapter implementation — the Workflow Platform adapter for
// https://fnf.internal (+ its fetch transport), so generated-app hosts that
// vendor only this package are self-sufficient. Also at
// "@higgsfield/fnf/workflow-platform".
export { createFetchTransport, createWorkflowPlatformAdapter } from './workflow-platform'
export type { FetchTransportOptions, WorkflowPlatformAdapterOptions } from './workflow-platform'

export { aspectRatio, duration, getNormalize, getWireName, wire, z } from './z'
export type { Normalize } from './z'
```

app/packages/fnf/src/jobs/__tests__/jobs.test.ts

```
import { describe, expect, it } from 'vitest'
import { buildWireParams, parseGeneration } from '../../spec'
import { nanoBanana2 } from '../nano-banana-2'
import { seedance2_0 } from '../seedance-2-0'

describe('nanoBanana2 (image, unwrapped input_images)', () => {
  it('builds wire params with bare image data array and canonical wire names', () => {
    const wire = buildWireParams(
      {
        model: 'nano_banana_2',
        prompt: { instruction: 'a blue cat' },
        media: { image: [{ id: 'u1', type: 'media_input', url: 'https://x/u1' }] },
        settings: { aspectRatio: '1:1', resolution: '2k', batchSize: 3 },
      },
      nanoBanana2,
    )
    expect(wire.prompt).toBe('a blue cat')
    // typed camelCase keys → backend's canonical snake_case wire keys (z.wire)
    expect(wire.aspect_ratio).toBe('1:1')
    expect(wire.batch_size).toBe(3)
    expect(wire).not.toHaveProperty('aspectRatio')
    expect(wire.resolution).toBe('2k')
    expect(wire.input_images).toEqual([{ id: 'u1', type: 'media_input', url: 'https://x/u1' }])
  })

  it('round-trips wire-named settings: parseGeneration maps aspect_ratio back to aspectRatio', () => {
    const gen = parseGeneration(
      { id: 'j1', status: 'completed', result_url: 'https://x/o.png', params: { prompt: 'cat', aspect_ratio: '1:1', resolution: '2k' } },
      nanoBanana2,
    )
    expect(gen.input.settings).toEqual({ aspectRatio: '1:1', resolution: '2k' })
    expect(gen.input.extra).toBeUndefined() // not misrouted to extra
    expect(buildWireParams(gen.input, nanoBanana2)).toMatchObject({ prompt: 'cat', aspect_ratio: '1:1', resolution: '2k' })
  })

  it('round-trips folder_id/parent_id and surfaces job_set_parent_id as parentJobSetId', () => {
    const gen = parseGeneration(
      {
        id: 'j2',
        job_set_id: 'set-2',
        job_set_parent_id: 'set-1',
        status: 'completed',
        result_url: 'https://x/o.png',
        params: { prompt: 'cat', aspect_ratio: '1:1', folder_id: 'f1', parent_id: 'set-1' },
      },
      nanoBanana2,
    )
    expect(gen.parentJobSetId).toBe('set-1')
    expect(gen.input.folderId).toBe('f1')
    expect(gen.input.parentId).toBe('set-1')
    expect(gen.input.extra).toBeUndefined() // targeting fields are not misrouted to extra
    expect(buildWireParams(gen.input, nanoBanana2)).toMatchObject({ folder_id: 'f1', parent_id: 'set-1' })
  })
})

describe('seedance2_0 (video, wrapped medias)', () => {
  it('passes settings through unchanged (no normalization) and wraps media with roles', () => {
    const wire = buildWireParams(
      {
        model: 'seedance_2_0',
        prompt: { instruction: 'demo' },
        media: { image: [{ id: 's1', type: 'image_job', url: 'https://x/s1' }] },
        settings: { duration: 8, aspectRatio: '16:9' },
      },
      seedance2_0,
    )
    expect(wire.duration).toBe(8) // not snapped — submit does not normalize; adjust() does
    expect(wire.aspect_ratio).toBe('16:9') // z.wire mapping
```

```
    expect(wire.generate_audio).toBe(true) // defaulted wire param
    expect(wire.medias).toEqual([{ role: 'image', data: { id: 's1', type: 'image_job', url: 'https://x/s1' } }])
  })

  it('enforces cardinality: a second start_image is rejected before any I/O', () => {
    const two = [{ id: 'a', type: 'media_input' }, { id: 'b', type: 'media_input' }]
    expect(() => buildWireParams(
      { model: 'seedance_2_0', media: { start_image: two }, settings: { duration: 8, aspectRatio: 'auto' } },
      seedance2_0,
    )).toThrow(/start_image.*at most 1.*got 2/)
  })

  it('enforces the cross-role rule: audio alone (no visuals) is rejected', () => {
    expect(() => buildWireParams(
      { model: 'seedance_2_0', media: { audio: { id: 'm', type: 'media_input' } }, settings: { duration: 8, aspectRatio: 'auto' } },
      seedance2_0,
    )).toThrow(/audio.*requires one of/)
  })

  it('validate hook: a frame with a forced concrete ratio is rejected (the live-failure trap)', () => {
    expect(() => buildWireParams(
      {
        model: 'seedance_2_0',
        prompt: { instruction: 'x' },
        media: { start_image: { id: 's', type: 'media_input' } },
        settings: { duration: 8, aspectRatio: '16:9' },
      },
      seedance2_0,
    )).toThrow(/aspectRatio must be 'auto'/)
  })

  it('validate hook: prompt is required when no media is attached (seedance + nano)', () => {
    expect(() => buildWireParams(
      { model: 'seedance_2_0', settings: { duration: 8, aspectRatio: 'auto' } },
      seedance2_0,
    )).toThrow(/Prompt is required/)
    expect(() => buildWireParams({ model: 'nano_banana_2', settings: { aspectRatio: 'auto' } }, nanoBanana2))
      .toThrow(/Prompt is required/)
  })

  it('validate hook: aggregates ALL issues at once', () => {
    try {
      buildWireParams(
        { model: 'seedance_2_0', media: { start_image: { id: 's', type: 'media_input' } }, settings: { duration: 8, aspectRatio: '16:9', batchSize: 9 } },
        seedance2_0,
      )
      throw new Error('should have thrown')
    }
    catch (err) {
      const issues = (err as { data?: { issues?: unknown[] } }).data?.issues ?? []
      expect(issues.length).toBeGreaterThanOrEqual(2) // ratio lock + batchSize range
    }
  })

  it('group cap: 10 reference images across image-like roles are rejected', () => {
    const refs = Array.from({ length: 9 }, (_, i) => ({ id: `r${i}`, type: 'media_input' }))
    expect(() => buildWireParams(
      {
        model: 'seedance_2_0',
        prompt: { instruction: 'x' },
        media: { image: refs, start_image: { id: 's', type: 'media_input' } },
        settings: { duration: 8, aspectRatio: 'auto' },
      },
      seedance2_0,
    )).toThrow(/at most 9.*got 10/)
  })
})
```

app/packages/fnf/src/jobs/__tests__/product-parity.test.ts

```
import type { MediaRef } from '../../types'
import { describe, expect, it } from 'vitest'
import { ValidationError } from '../../errors'
import { buildWireParams, parseGeneration } from '../../spec'
import { lookupSize } from '../dimensions'
import { KLING_DEFAULT_MOTION_ID, klingVideo } from '../kling'
import { nanoBanana2 } from '../nano-banana-2'
import { seedance2_0 } from '../seedance-2-0'
import { DEFAULT_SOUL_STYLE_ID, textToImageSoul } from '../text2image-soul'

function ref(id: string, meta?: MediaRef['meta']): MediaRef {
  return { id, type: 'media_input', url: `https://cdn/${id}.png`, ...(meta ? { meta } : {}) }
}

function issuesOf(fn: () => unknown): string[] {
  try {
    fn()
    return []
  }
  catch (err) {
    const issues = (err as { data?: { issues?: Array<{ msg: string }> } }).data?.issues ?? []
    return issues.map(i => i.msg)
  }
}

// ■■ restore: lossy finalize hooks round-trip (the get-then-resubmit contract) ■■

describe('seedance restore round-trip', () => {
  it('a fast generation resubmits as fast (mode restored from wire model)', () => {
    const input = {
      model: 'seedance_2_0',
      prompt: { instruction: 'a cube' },
      settings: { duration: 8 as const, aspectRatio: '16:9' as const, mode: 'fast' as const },
    }
    const wire = buildWireParams(input, seedance2_0)
    expect(wire.model).toBe('seedance_2_0_fast')
    expect(wire.resolution).toBe('720p') // fast product default

    const gen = parseGeneration({ id: 'j1', status: 'completed', result_url: 'https://x/v.mp4', params: wire }, seedance2_0)
    expect(gen.input.settings.mode).toBe('fast')

    const rewire = buildWireParams(gen.input, seedance2_0)
    expect(rewire.model).toBe('seedance_2_0_fast') // NOT the 'std' schema default
    expect(rewire.width).toBe(wire.width)
    expect(rewire.height).toBe(wire.height)
  })

  it('a frame-led auto generation resubmits wire-identically and passes validate', () => {
    const input = {
      model: 'seedance_2_0',
      prompt: { instruction: 'animate' },
      media: { start_image: [ref('f', { width: 720, height: 1280 })] },
      settings: { duration: 5 as const, aspectRatio: 'auto' as const },
    }
    const wire = buildWireParams(input, seedance2_0)
    // product behavior: auto + measured portrait frame → concrete 9:16 + table dims
    expect(wire.aspect_ratio).toBe('9:16')
    expect(wire.resolution).toBe('1080p') // std product default
    expect([wire.width, wire.height]).toEqual([1080, 1920])

    const gen = parseGeneration({ id: 'j2', status: 'completed', result_url: 'https://x/v.mp4', params: wire }, seedance2_0)
    // frames lock the INPUT ratio to 'auto' — restore maps the resolved
    // concrete ratio back so the parsed input passes the same validate
    expect(gen.input.settings.aspectRatio).toBe('auto')

    const rewire = buildWireParams(gen.input, seedance2_0) // must not throw the ratio lock
    // 'auto' re-resolves from the round-tripped dims (the parsed refs have no
    // meta), so the resubmitted wire is product-shaped and identical
```

```
    expect(rewire.aspect_ratio).toBe('9:16')
    expect([rewire.width, rewire.height]).toEqual([1080, 1920])
  })

  it('changing resolution on a parsed input re-derives dims instead of resubmitting stale ones', () => {
    const wire = buildWireParams({
      model: 'seedance_2_0',
      prompt: { instruction: 'animate' },
      media: { start_image: [ref('f', { width: 720, height: 1280 })] },
      settings: { duration: 5 as const, aspectRatio: 'auto' as const },
    }, seedance2_0)
    const gen = parseGeneration({ id: 'j3', status: 'completed', result_url: 'https://x/v.mp4', params: wire }, seedance2_0)

    const rewire = buildWireParams({
      ...gen.input,
      settings: { ...gen.input.settings, resolution: '480p' },
    } as never, seedance2_0)
    expect(rewire.resolution).toBe('480p')
    expect([rewire.width, rewire.height]).toEqual([480, 854]) // fresh table lookup, not 1080×1920
  })
})

describe('kling restore round-trip', () => {
  const portrait = {
    model: 'kling',
    prompt: { instruction: 'pan up' },
    media: { input_image: ref('start', { width: 720, height: 1280 }) },
    settings: { duration: 5 as const, aspectRatio: '9:16' },
  }

  it('wire dims come from the start image; the backend-enum ratio rides along', () => {
    const wire = buildWireParams(portrait, klingVideo)
    expect([wire.width, wire.height]).toEqual([720, 1280])
    expect(wire.aspect_ratio).toBe('9:16') // in the backend enum (16:9|9:16|1:1) → kept
  })

  it('off-enum ratios are dropped from the wire (the backend accepts only 16:9|9:16|1:1)', () => {
    const wire = buildWireParams(
      { ...portrait, settings: { duration: 5 as const, aspectRatio: '4:3' } },
      klingVideo,
    )
    expect(wire).not.toHaveProperty('aspect_ratio')
  })

  it('a 9:16 generation resubmits as 9:16 (no silent 16:9 flip)', () => {
    const wire = buildWireParams(portrait, klingVideo)
    const gen = parseGeneration({ id: 'k1', status: 'completed', result_url: 'https://x/v.mp4', params: wire }, klingVideo)
    expect(gen.input.settings.aspectRatio).toBe('9:16') // restored via simplifyRatio

    const rewire = buildWireParams(gen.input, klingVideo)
    expect([rewire.width, rewire.height]).toEqual([720, 1280])
    expect(rewire.model).toBe(wire.model)
  })
})

// ■■ kling product rules ■■

describe('kling product parity', () => {
  const base = {
    model: 'kling',
    media: { input_image: ref('start') },
  }

  it('enhance_prompt defaults to true and respects the default-preset sentinel', () => {
    // sentinel motion_id (the default) → the caller's choice, defaulting true
    const wire = buildWireParams({ ...base, settings: { duration: 5 as const } }, klingVideo)
    expect(wire.motion_id).toBe(KLING_DEFAULT_MOTION_ID)
    expect(wire.enhance_prompt).toBe(true)

    const declined = buildWireParams(
```

```
      { ...base, prompt: { enhance: false }, settings: { duration: 5 as const } },
      klingVideo,
    )
    expect(declined.enhance_prompt).toBe(false) // sentinel = "no preset" → user choice respected

    const preset = buildWireParams(
      { ...base, prompt: { enhance: false }, settings: { duration: 5 as const, motionId: 'real-preset-uuid' } },
      klingVideo,
    )
    expect(preset.enhance_prompt).toBe(true) // a real preset forces it on
  })

  it('always wires a prompt string and never the dead seed/cfg_scale knobs', () => {
    const wire = buildWireParams({ ...base, settings: { duration: 5 as const } }, klingVideo)
    expect(wire.prompt).toBe('') // the product always sends a string ('' when promptless)
    expect(wire).not.toHaveProperty('seed') // backend KlingParamsSchema drops it silently
    expect(wire).not.toHaveProperty('cfg_scale')
  })

  it('text-only kling works only at v2-5-turbo + 1080p (the one combination the backend allows)', () => {
    const wire = buildWireParams(
      { model: 'kling', prompt: { instruction: 'a storm rolls in' }, settings: { duration: 5 as const, resolution: '1080p' as const, aspectRatio: '9:16' } },
      klingVideo,
    )
    expect([wire.width, wire.height]).toEqual([720, 1280]) // box-derived dims
    expect(wire.mode).toBe('pro')

    // default resolution (720p → std) requires a start frame server-side
    expect(issuesOf(() => buildWireParams(
      { model: 'kling', prompt: { instruction: 'a storm rolls in' }, settings: { duration: 5 as const } },
      klingVideo,
    ))).toContain('input image is required for this model and mode')
  })

  it('legacy params with motion_id null coalesce to the sentinel and respect a stored enhance_prompt: false', () => {
    // shape stored by the previous SDK (motionId defaulted to null)
    const legacy = {
      model: 'kling-v2-5-turbo',
      prompt: 'pan',
      input_image: { id: 'start', type: 'media_input', url: 'https://cdn/start.png' },
      enhance_prompt: false,
      motion_id: null,
      cfg_scale: 0.5,
      resolution: '720p',
      duration: 5,
      seed: 7,
      use_unlim: false,
      width: 720,
      height: 1280,
      mode: 'std',
    }
    const gen = parseGeneration({ id: 'k-old', status: 'completed', result_url: 'https://x/v.mp4', params: legacy }, klingVideo)
    const rewire = buildWireParams(gen.input, klingVideo)
    expect(rewire.motion_id).toBe(KLING_DEFAULT_MOTION_ID) // product wire never carries null
    expect(rewire.enhance_prompt).toBe(false) // NOT force-flipped to true
  })

  it('rejects out-of-set durations with a typed issue (VideoKlingDuration is 5 | 10)', () => {
    expect(issuesOf(() => buildWireParams(
      { ...base, settings: { duration: 7 as unknown as 5 } },
      klingVideo,
    ))).toContain('duration must be one of: 5, 10')
  })
})

// ■■ auto-ratio resolution from measured media (product parity) ■■

describe('auto aspect ratio resolves from the first image like the product', () => {
  it('nano: auto + landscape image meta → 16:9 with table dims (not 3:4)', () => {
    const wire = buildWireParams(
```

```
      {
        model: 'nano_banana_2',
        prompt: { instruction: 'restyle this' },
        media: { image: [ref('i', { width: 1920, height: 1080 })] },
        settings: { aspectRatio: 'auto' as const },
      },
      nanoBanana2,
    )
    expect(wire.aspect_ratio).toBe('16:9')
    expect([wire.width, wire.height]).toEqual([1376, 768])
  })

  it('nano: auto without local size knowledge falls back to 3:4', () => {
    const wire = buildWireParams(
      { model: 'nano_banana_2', prompt: { instruction: 'a cat' }, settings: { aspectRatio: 'auto' as const } },
      nanoBanana2,
    )
    expect(wire.aspect_ratio).toBe('3:4')
    expect([wire.width, wire.height]).toEqual([896, 1200])
  })

  it('seedance: auto without meta keeps auto on the wire with the 16:9 dims', () => {
    const wire = buildWireParams(
      {
        model: 'seedance_2_0',
        prompt: { instruction: 'x' },
        media: { image: [ref('i')] },
        settings: { duration: 5 as const, aspectRatio: 'auto' as const },
      },
      seedance2_0,
    )
    expect(wire.aspect_ratio).toBe('auto')
    expect([wire.width, wire.height]).toEqual([1920, 1080])
  })
})

// ■■ validation gaps closed (typed issues, not TypeErrors / backend 422s) ■■

describe('seedance validate', () => {
  const withPrompt = (settings: Record<string, unknown>) => () => buildWireParams(
    { model: 'seedance_2_0', prompt: { instruction: 'x' }, settings: settings as never },
    seedance2_0,
  )

  it('enforces duration 4–15 (built.ts hard reject)', () => {
    expect(issuesOf(withPrompt({ duration: 30, aspectRatio: 'auto' }))).toContain('duration must be between 4 and 15')
  })

  it('rejects out-of-enum ratios with a typed issue instead of a TypeError', () => {
    expect(issuesOf(withPrompt({ duration: 8, aspectRatio: '2:1' }))[0]).toMatch(/aspectRatio must be one of/)
  })

  it('rejects fast + 1080p (the product fast config never offers it)', () => {
    expect(issuesOf(withPrompt({ duration: 8, aspectRatio: 'auto', mode: 'fast', resolution: '1080p' })))
      .toContain('resolution \'1080p\' is not available in fast mode')
  })

  it('defaults resolution per mode: std → 1080p, fast → 720p', () => {
    const std = buildWireParams({ model: 'seedance_2_0', prompt: { instruction: 'x' }, settings: { duration: 8 as const, aspectRatio: '16:9' as const } }, seedance2_0)
    expect(std.resolution).toBe('1080p')
    expect([std.width, std.height]).toEqual([1920, 1080])
    const fast = buildWireParams({ model: 'seedance_2_0', prompt: { instruction: 'x' }, settings: { duration: 8 as const, aspectRatio: '16:9' as const, mode: 'fast' as const } }, seedance2_0)
    expect(fast.resolution).toBe('720p')
    expect([fast.width, fast.height]).toEqual([1280, 720])
  })
})

describe('nano validate', () => {
  it('caps input images at 14 (NANO_BANANA_2_IMAGE_UPLOAD_LIMIT)', () => {
    const refs = Array.from({ length: 15 }, (_, i) => ref(`r${i}`))
```

```
    expect(() => buildWireParams(
      { model: 'nano_banana_2', prompt: { instruction: 'x' }, media: { image: refs }, settings: { aspectRatio: '1:1' as const } },
      nanoBanana2,
    )).toThrow(/image.*at most 14.*got 15/)
  })

  it('flags images under the 128px product minimum when meta is known', () => {
    expect(() => buildWireParams(
      { model: 'nano_banana_2', prompt: { instruction: 'x' }, media: { image: [ref('tiny', { width: 64, height: 64 })] }, settings: { aspectRatio: '1:1' as const } },
      nanoBanana2,
    )).toThrow(/minimum dimension is 128px/)
  })

  it('rejects on RAW prompt length >= 15000 (validatePrompt parity)', () => {
    const padded = `${'x'.repeat(14_000)}${' '.repeat(1_000)}` // trimmed 14k, raw 15k
    expect(() => buildWireParams(
      { model: 'nano_banana_2', prompt: { instruction: padded }, settings: { aspectRatio: '1:1' as const } },
      nanoBanana2,
    )).toThrow(/too long/)
  })
})

// ■■ soul product parity ■■

describe('soul product parity', () => {
  const minimal = { model: 'text2image_soul', prompt: { instruction: 'portrait' }, settings: {} }

  it('default submit matches the /ai/image strategy: style, steps 50, sampler fields, portrait table dims', () => {
    const wire = buildWireParams(minimal, textToImageSoul)
    expect(wire.style_id).toBe(DEFAULT_SOUL_STYLE_ID) // never null/null
    expect(wire.steps).toBe(50)
    expect(wire.sample_shift).toBe(4) // 1080p default
    expect(wire.sample_guide_scale).toBe(4)
    expect(wire.negative_prompt).toBe('')
    expect(wire.enhance_prompt).toBe(true)
    expect(wire.aspect_ratio).toBe('3:4') // the product soul form defaults to PORTRAIT
    expect([wire.width, wire.height]).toEqual([1536, 2048]) // SOUL_RESOLUTION_MAP 1080p 3:4
    expect(wire.seed).toBeGreaterThanOrEqual(1)
    expect(wire.seed).toBeLessThanOrEqual(1_000_000)
  })

  it('snaps an off-table ratio to the closest table key instead of the portrait fallback', () => {
    const wire = buildWireParams({ ...minimal, settings: { aspectRatio: '1920:1080' } }, textToImageSoul)
    expect(wire.aspect_ratio).toBe('16:9') // a landscape request stays landscape
    expect([wire.width, wire.height]).toEqual([2048, 1152])
  })

  it('720p quality switches the table row and sample_shift', () => {
    const wire = buildWireParams({ ...minimal, settings: { quality: '720p' as const, aspectRatio: '16:9' } }, textToImageSoul)
    expect(wire.sample_shift).toBe(3)
    expect([wire.width, wire.height]).toEqual([1696, 960])
  })

  it('explicit styleId: null fails locally with a typed issue (dev backend 422 parity)', () => {
    expect(issuesOf(() => buildWireParams({ ...minimal, settings: { styleId: null } }, textToImageSoul)))
      .toContain('styleId is required (or send fashion_factory_id via extra)')
  })

  it('an image reference forces enhance_prompt off; a non-default style forces it on', () => {
    const referenced = buildWireParams(
      { ...minimal, media: { image_reference: ref('r') } },
      textToImageSoul,
    )
    expect(referenced.enhance_prompt).toBe(false)
    const styled = buildWireParams({ ...minimal, prompt: { instruction: 'x', enhance: false }, settings: { styleId: 'custom-style' } }, textToImageSoul)
    expect(styled.enhance_prompt).toBe(true)
  })
})

describe('lookupSize', () => {
```

```
  it('throws the typed ValidationError on unknown keys, not a TypeError', () => {
    const map = { '1k': { '1:1': [10, 10] as const } }
    expect(() => lookupSize(map, '1k', '7:3')).toThrowError(ValidationError)
    expect(() => lookupSize(map, '1k', '7:3')).toThrow(/not supported/)
  })
})
```

app/packages/fnf/src/jobs/__tests__/seedance-2-5.test.ts

```
import type { MediaRef } from '../../types'
import { describe, expect, it } from 'vitest'
import { buildWireParams, parseGeneration } from '../../spec'
import { seedance2_5, SEEDANCE25_MAX_IMAGES, SEEDANCE25_MAX_VIDEOS } from '../seedance-2-5'

function ref(id: string, meta?: MediaRef['meta']): MediaRef {
  return { id, type: 'media_input', url: `https://cdn/${id}`, ...(meta ? { meta } : {}) }
}

function issuesOf(fn: () => unknown): string[] {
  try {
    fn()
    return []
  }
  catch (err) {
    const issues = (err as { data?: { issues?: Array<{ msg: string }> } }).data?.issues ?? []
    return issues.map(i => i.msg)
  }
}

function wireFor(input: Parameters<typeof buildWireParams>[0]): Record<string, unknown> {
  return buildWireParams(input, seedance2_5)
}

describe('seedance2_5 wire shape', () => {
  it('builds the default-model wire: canonical names, table dims, wrapped medias', () => {
    const wire = wireFor({
      model: 'seedance_2_5',
      prompt: { instruction: 'a cat walking' },
      media: { image: [ref('i1')], audio: ref('a1', { durationSec: 10 }) },
      settings: { duration: 8, resolution: '1080p', aspectRatio: '9:16', batchSize: 2 },
    })

    expect(wire.prompt).toBe('a cat walking')
    expect(wire.duration).toBe(8)
    expect(wire.resolution).toBe('1080p')
    expect(wire.aspect_ratio).toBe('9:16')
    expect(wire.batch_size).toBe(2)
    expect(wire.model).toBe('default')
    // camelCase settings never leak to the wire
    expect(wire).not.toHaveProperty('aspectRatio')
    expect(wire).not.toHaveProperty('batchSize')
    // 1080p × 9:16 from the size table
    expect(wire.width).toBe(1080)
    expect(wire.height).toBe(1920)
    expect(wire.medias).toEqual([
      { role: 'image', data: { id: 'i1', type: 'media_input', url: 'https://cdn/i1' } },
      { role: 'audio', data: { id: 'a1', type: 'media_input', url: 'https://cdn/a1' } },
    ])
  })

  it('applies the product defaults (720p / 16:9 / high bitrate / audio on)', () => {
    const wire = wireFor({ model: 'seedance_2_5', prompt: { instruction: 'x' }, settings: {} })
    expect(wire.resolution).toBe('720p')
    expect(wire.aspect_ratio).toBe('16:9')
    expect(wire.bitrate_mode).toBe('high')
    expect(wire.generate_audio).toBe(true)
    expect(wire.duration).toBe(5)
    expect(wire.batch_size).toBe(1)
    expect(wire.width).toBe(1280)
    expect(wire.height).toBe(720)
  })

  it('video_edit sends neither duration nor aspect_ratio (both come from the source clip)', () => {
    const wire = wireFor({
      model: 'seedance_2_5',
      prompt: { instruction: 'make it night' },
      media: { video: ref('v1', { durationSec: 120 }) },
```

```
      settings: { model: 'video_edit', duration: 8, aspectRatio: '9:16' },
    })
    expect(wire.model).toBe('video_edit')
    expect(wire).not.toHaveProperty('duration')
    expect(wire).not.toHaveProperty('aspect_ratio')
    expect(wire).not.toHaveProperty('extension_mode')
    // the box still ships, derived from resolution × ratio
    expect(wire.width).toBe(720)
    expect(wire.height).toBe(1280)
  })

  it('video_extension sends duration + extension_mode but no aspect_ratio', () => {
    const wire = wireFor({
      model: 'seedance_2_5',
      prompt: { instruction: 'keep going' },
      media: { video: ref('v1', { durationSec: 6 }) },
      settings: { model: 'video_extension', extensionMode: 'forward', duration: 10 },
    })
    expect(wire.model).toBe('video_extension')
    expect(wire.duration).toBe(10)
    expect(wire.extension_mode).toBe('forward')
    expect(wire).not.toHaveProperty('aspect_ratio')
  })
})

describe('seedance2_5 validate', () => {
  const base = { model: 'seedance_2_5', prompt: { instruction: 'x' } } as const

  it('requires a prompt', () => {
    expect(issuesOf(() => wireFor({ model: 'seedance_2_5', media: { image: ref('i1') }, settings: {} })))
      .toContain('Prompt is required')
  })

  it('rejects a duration outside 4–30 and a batch outside 1–4', () => {
    const issues = issuesOf(() => wireFor({ ...base, settings: { duration: 45, batchSize: 9 } }))
    expect(issues).toContain('duration must be an integer between 4 and 30')
    expect(issues).toContain('batchSize must be an integer between 1 and 4')
  })

  it('video_edit needs exactly one video; video_extension needs a video and a mode', () => {
    expect(issuesOf(() => wireFor({ ...base, settings: { model: 'video_edit' } })))
      .toContain('Seedance 2.5 video edit requires exactly one video reference')
    expect(issuesOf(() => wireFor({ ...base, media: { video: [ref('v1'), ref('v2')] }, settings: { model: 'video_edit' } })))
      .toContain('Seedance 2.5 video edit requires exactly one video reference')

    const extension = issuesOf(() => wireFor({ ...base, settings: { model: 'video_extension' } }))
    expect(extension).toContain('Seedance 2.5 video extension requires a video reference')
    expect(extension).toContain('extensionMode is required for video extension')
  })

  it('video_extension caps duration at 30 with its own message', () => {
    expect(issuesOf(() => wireFor({
      ...base,
      media: { video: ref('v1') },
      settings: { model: 'video_extension', extensionMode: 'backward', duration: 40 },
    }))).toContain('Seedance 2.5 video extension supports at most 30 seconds')
  })

  it('video_edit skips the duration checks the reference budget would apply', () => {
    // a 2-minute source clip is the SUBJECT of an edit, not a reference
    expect(issuesOf(() => wireFor({
      ...base,
      media: { video: ref('v1', { durationSec: 120 }) },
      settings: { model: 'video_edit', duration: 99 },
    }))).toEqual([])
    // the same clip attached to a default generation is over the budget
    expect(issuesOf(() => wireFor({ ...base, media: { video: ref('v1', { durationSec: 120 }) }, settings: {} })))
      .toContain('Each reference video must be at least 2 seconds and no more than 30 seconds')
  })
```

```
  it('enforces the per-role media caps', () => {
    const videos = Array.from({ length: SEEDANCE25_MAX_VIDEOS + 1 }, (_, i) => ref(`v${i}`, { durationSec: 2 }))
    expect(issuesOf(() => wireFor({ ...base, media: { video: videos }, settings: {} })))
      .toContain(`You can add a maximum of ${SEEDANCE25_MAX_VIDEOS} videos`)
  })

  it('counts prompt reference elements against the image budget', () => {
    const images = Array.from({ length: SEEDANCE25_MAX_IMAGES }, (_, i) => ref(`i${i}`))
    // 30 images alone is exactly the cap
    expect(issuesOf(() => wireFor({ ...base, media: { image: images }, settings: {} }))).toEqual([])
    // one element mentioned in the prompt pushes it over
    expect(issuesOf(() => wireFor({
      model: 'seedance_2_5',
      prompt: { instruction: 'with <<<element_abc-1>>>' },
      media: { image: images },
      settings: {},
    }))).toContain(`You can add a maximum of ${SEEDANCE25_MAX_IMAGES} images`)
  })

  it('bounds each audio reference and the combined audio budget', () => {
    expect(issuesOf(() => wireFor({ ...base, media: { audio: ref('a1', { durationSec: 45 }) }, settings: {} })))
      .toContain('Each reference audio file must be at least 2 seconds and no more than 30 seconds')
    expect(issuesOf(() => wireFor({
      ...base,
      media: { audio: [ref('a1', { durationSec: 20 }), ref('a2', { durationSec: 20 })] },
      settings: {},
    }))).toContain('The total audio duration limit is 30 seconds')
  })
})

describe('seedance2_5 round-trip', () => {
  it('a default generation resubmits to the same wire', () => {
    const wire = wireFor({
      model: 'seedance_2_5',
      prompt: { instruction: 'a cat' },
      media: { start_image: ref('s1') },
      settings: { duration: 12, resolution: '480p', aspectRatio: '4:3', generateAudio: false, bitrateMode: 'standard' },
    })
    const gen = parseGeneration({ id: 'j1', status: 'completed', result_url: 'https://x/v.mp4', params: wire }, seedance2_5)
    expect(gen.input.settings).toMatchObject({
      duration: 12,
      resolution: '480p',
      aspectRatio: '4:3',
      generateAudio: false,
      bitrateMode: 'standard',
      model: 'default',
    })
    expect(buildWireParams(gen.input, seedance2_5)).toEqual(wire)
  })

  it('restore recovers a video_edit ratio from the stored dims, not the 16:9 default', () => {
    const wire = wireFor({
      model: 'seedance_2_5',
      prompt: { instruction: 'recolor' },
      media: { video: ref('v1', { durationSec: 9 }) },
      settings: { model: 'video_edit', resolution: '720p', aspectRatio: '9:16' },
    })
    expect(wire).not.toHaveProperty('aspect_ratio') // finalize dropped it

    const gen = parseGeneration({ id: 'j2', status: 'completed', result_url: 'https://x/v.mp4', params: wire }, seedance2_5)
    expect(gen.input.settings).toMatchObject({ model: 'video_edit', aspectRatio: '9:16' })
    // the resubmitted box matches the original, not 1280×720
    expect(buildWireParams(gen.input, seedance2_5)).toEqual(wire)
  })
})
```

app/packages/fnf/src/jobs/checks.ts

```
import type { MediaIssue } from '../groups/media'
import type { MediaInput, PromptInput } from '../types'

/**
 * Issue builders shared by the jobs' `validate` hooks, so the catalog's models
 * emit uniform pydantic-shaped issues instead of each declaration hand-rolling
 * (and slowly drifting) the same snippets. Consumers key on these shapes.
 */

/** `value` (when set) must sit in [min, max] — the batchSize/steps/seed pattern. */
export function intRange(field: string, value: number | null | undefined, min: number, max: number): MediaIssue[] {
  if (value == null || (value >= min && value <= max))
    return []
  return [{ loc: ['settings', field], msg: `${field} must be between ${min} and ${max}` }]
}

/** `value` (when set) must be one of `options` — membership for permissive runtime schemas. */
export function oneOf(field: string, value: unknown, options: readonly unknown[]): MediaIssue[] {
  if (value == null || options.includes(value))
    return []
  return [{ loc: ['settings', field], msg: `${field} must be one of: ${options.join(', ')}` }]
}

/** `prompt.instruction` must contain at least `min` non-whitespace chars. */
export function promptRequired(prompt: PromptInput | undefined, min = 1): MediaIssue[] {
  const raw = prompt?.instruction ?? ''
  if (raw.trim().length >= min)
    return []
  return [{ loc: ['prompt'], msg: 'Prompt is required' }]
}

/** `prompt.instruction` raw length must be below/within `max`, matching product validators. */
export function promptMax(prompt: PromptInput | undefined, max: number, opts: { inclusive?: boolean } = {}): MediaIssue[] {
  const raw = prompt?.instruction ?? ''
  const tooLong = opts.inclusive ? raw.length >= max : raw.length > max
  if (!tooLong)
    return []
  return [{ loc: ['prompt'], msg: `Prompt is too long (max ${max} characters)` }]
}

/** Refs attached under `role` (single or array), absent role = 0. */
export function countRefs(media: MediaInput | undefined, role: string): number {
  const value = media?.[role]
  return Array.isArray(value) ? value.length : value ? 1 : 0
}

/**
 * The product's seed source (fnf-web `randomSeed`): 1…1,000,000 inclusive.
 * Meant as a `z._default(z.number(), randomSeed)` so the wire always carries a
 * seed (prod always sends one) without burdening every caller to invent it.
 */
export function randomSeed(): number {
  return Math.floor(Math.random() * 1_000_000) + 1
}
```

app/packages/fnf/src/jobs/dimensions.ts

```
import type { MediaInput, MediaRef } from '../types'
import { ValidationError } from '../errors'

export interface AspectRatioDimensions {
  width: number
  height: number
  /** The parsed `w:h`, or the 16:9 fallback when the input is malformed. */
  normalized: string
}

/**
 * Dimensions with a fixed short side, snapped to even numbers — the derivation
 * fnf-web's submit strategies (soul/kling/seedance/nano) all hand-roll. Meant
 * for a job's `finalize` hook: `{ ...wire, ...aspectRatioDimensions(ratio, 1080) }`.
 */
export function aspectRatioDimensions(aspectRatio: string | undefined, shortSide: number): AspectRatioDimensions {
  const match = /^(\d+):(\d+)$/.exec((aspectRatio ?? '').trim())
  const w = match ? Number(match[1]) : 16
  const h = match ? Number(match[2]) : 9
  const normalized = match ? `${w}:${h}` : '16:9'

  const even = (value: number) => Math.round(value / 2) * 2
  if (w >= h)
    return { width: even(shortSide * (w / h)), height: shortSide, normalized }
  return { width: shortSide, height: even(shortSide * (h / w)), normalized }
}

/**
 * Guarded lookup into a `resolution × ratio → [width, height]` size table —
 * the shape every table-based model (nano-banana-2, seedance, soul, and most
 * of the fnf-web catalog) carries. The settings schemas are deliberately
 * permissive at runtime (see z.aspectRatio), so an unknown ratio/resolution
 * from a JS caller must surface as the typed ValidationError, not as a
 * `undefined is not iterable` TypeError from a bare destructure.
 */
export function lookupSize<Res extends string, Ratio extends string>(
  map: Record<Res, Partial<Record<Ratio, readonly [number, number]>>>,
  resolution: Res,
  ratio: string,
): { width: number, height: number } {
  const row = map[resolution]
  if (!row) {
    throw new ValidationError(`resolution '${resolution}' is not supported (allowed: ${Object.keys(map).join(', ')})`, [
      { loc: ['settings', 'resolution'], msg: `resolution '${resolution}' is not supported` },
    ])
  }
  const size = row[ratio as Ratio]
  if (!size) {
    throw new ValidationError(`aspect ratio '${ratio}' is not supported (allowed: ${Object.keys(row).join(', ')})`, [
      { loc: ['settings', 'aspectRatio'], msg: `aspect ratio '${ratio}' is not supported` },
    ])
  }
  return { width: size[0], height: size[1] }
}

/**
 * The closest allowed ratio for an intrinsic size, by LINEAR distance on w/h —
 * the exact rule fnf-web's `getSeedance2_0ClosestAspectRatio` /
 * `getNanoBanana2ClosestAspectRatio` use to resolve 'auto' from the first
 * attached image. (Not log-distance: `adjust()`'s closestAspectRatio snaps
 * user-typed ratios, this mirrors the product's auto-resolution.)
 */
export function closestRatioBySize<R extends string>(ratios: readonly R[], size: { width: number, height: number }): R {
  const target = size.width / size.height
  let best = ratios[0]
  let bestDiff = Infinity
  for (const option of ratios) {
    const [w, h] = option.split(':').map(Number)
```

```
    if (!Number.isFinite(w) || !Number.isFinite(h) || h === 0)
      continue
    const diff = Math.abs(target - w / h)
    if (diff < bestDiff) {
      bestDiff = diff
      best = option
    }
  }
  return best
}

/**
 * The first ref across `roles` (in order) whose meta carries a known size —
 * the product resolves 'auto' from the FIRST attached image. Populate meta
 * from app data or `resolveMediaMeta`; undefined means no local knowledge.
 */
export function firstSizeMeta(media: MediaInput | undefined, roles: readonly string[]): { width: number, height: number } | undefined {
  for (const role of roles) {
    const value = media?.[role]
    const refs: MediaRef[] = Array.isArray(value) ? value : value ? [value] : []
    for (const ref of refs) {
      const { width, height } = ref.meta ?? {}
      if (width != null && height != null && height > 0)
        return { width, height }
    }
  }
  return undefined
}

/** Reduce a pixel size to its smallest integer `w:h` (720×1280 → '9:16'). */
export function simplifyRatio(width: number, height: number): string {
  const gcd = (a: number, b: number): number => (b === 0 ? a : gcd(b, a % b))
  const div = gcd(Math.round(width), Math.round(height)) || 1
  return `${Math.round(width) / div}:${Math.round(height) / div}`
}
```

app/packages/fnf/src/jobs/gpt-image-2.ts

```
import { defineJob } from '../define-job'
import { dimensionsWithin } from '../groups/media'
import { z } from '../z'
import { intRange, oneOf, promptRequired } from './checks'
import { firstSizeMeta } from './dimensions'
import { dimensionsFromRatios } from './image-helpers'

export const GptImage2AspectRatio = {
  auto: 'auto',
  r1x1: '1:1',
  r3x2: '3:2',
  r2x3: '2:3',
  r16x9: '16:9',
  r9x16: '9:16',
  r4x3: '4:3',
  r3x4: '3:4',
  r21x9: '21:9',
} as const

export type GptImage2AspectRatioValue = typeof GptImage2AspectRatio[keyof typeof GptImage2AspectRatio]
type ConcreteRatio = Exclude<GptImage2AspectRatioValue, 'auto'>
type Quality = 'low' | 'medium' | 'high'
type Resolution = '1k' | '2k' | '4k'

const CONCRETE_RATIOS: Record<ConcreteRatio, readonly [number, number]> = {
  '1:1': [1, 1],
  '3:2': [3, 2],
  '2:3': [2, 3],
  '16:9': [16, 9],
  '9:16': [9, 16],
  '4:3': [4, 3],
  '3:4': [3, 4],
  '21:9': [21, 9],
}

const BASE_SIZE: Record<Resolution, number> = {
  '1k': 1024,
  '2k': 2048,
  '4k': 4096,
}

const CREDITS_PER_IMAGE: Record<Quality, Record<Resolution, number>> = {
  low: { '1k': 0.5, '2k': 0.75, '4k': 1 },
  medium: { '1k': 2, '2k': 3, '4k': 6 },
  high: { '1k': 4, '2k': 7, '4k': 12 },
}

const SUB_MODELS = ['videotape-alpha', 'cassettetape-alpha', 'electricaltape-alpha', 'tidepool-alpha'] as const

function dimensionsFor(aspectRatio: GptImage2AspectRatioValue, resolution: Resolution) {
  const base = BASE_SIZE[resolution]
  if (aspectRatio === 'auto')
    return { width: base, height: base }
  return dimensionsFromRatios(CONCRETE_RATIOS, aspectRatio, base)
}

export const gptImage2 = defineJob({
  jobSetType: 'gpt_image_2',
  outputType: 'image',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['image'],
      counts: { image: { max: 16 } },
      rules: [dimensionsWithin(['image'], { minSide: 300, ratio: [0.4, 2.5] })],
    },
```

```
    settings: {
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(Object.values(GptImage2AspectRatio)), 'auto')),
      quality: z._default(z.enum(['low', 'medium', 'high']), 'high'),
      resolution: z._default(z.enum(['1k', '2k', '4k']), '2k'),
      subModel: z.wire('sub_model', z._default(z.enum(SUB_MODELS), 'videotape-alpha')),
      batchSize: z.wire('batch_size', z._default(z.number(), 1)),
    },
  },
  credits: ({ settings }) => {
    const quality = settings.quality ?? 'high'
    const resolution = settings.resolution ?? '2k'
    return (settings.batchSize ?? 1) * CREDITS_PER_IMAGE[quality][resolution]
  },
  validate: ({ prompt, settings }) => [
    ...promptRequired(prompt),
    ...intRange('batchSize', settings.batchSize, 1, 4),
    ...oneOf('aspectRatio', settings.aspectRatio, Object.values(GptImage2AspectRatio)),
  ],
  finalize: (wire, input) => {
    const aspectRatio = wire.aspect_ratio as GptImage2AspectRatioValue
    const resolution = (wire.resolution ?? '2k') as Resolution
    const size = aspectRatio === 'auto'
      ? firstSizeMeta(input.media, ['image']) ?? dimensionsFor(aspectRatio, resolution)
      : dimensionsFor(aspectRatio, resolution)

    return {
      ...wire,
      model: 'gpt_image_2',
      width: size.width,
      height: size.height,
    }
  },
})
```

app/packages/fnf/src/jobs/grok-imagine.ts

```
import type { GenerationInput } from '../types'
import { defineJob } from '../define-job'
import { z } from '../z'
import { intRange, oneOf, promptMax, promptRequired } from './checks'
import { closestRatioBySize, lookupSize } from './dimensions'
import { firstMetaSize, integerRange } from './video-helpers'

export const GrokImagineAspectRatio = {
  auto: 'auto',
  r16x9: '16:9',
  r9x16: '9:16',
  r4x3: '4:3',
  r3x4: '3:4',
  r3x2: '3:2',
  r2x3: '2:3',
  r1x1: '1:1',
} as const

export const GrokImagineResolution = {
  r480: '480p',
  r720: '720p',
} as const

const ASPECT_RATIOS = Object.values(GrokImagineAspectRatio)
const CONCRETE_RATIOS = ['16:9', '4:3', '1:1', '3:4', '9:16', '3:2', '2:3'] as const
const RESOLUTIONS = Object.values(GrokImagineResolution)

type Ratio = typeof CONCRETE_RATIOS[number]
type Resolution = typeof RESOLUTIONS[number]

const SIZE_MAP: Record<Resolution, Record<Ratio, [number, number]>> = {
  '480p': {
    '16:9': [854, 480],
    '4:3': [640, 480],
    '1:1': [480, 480],
    '3:4': [480, 640],
    '9:16': [480, 854],
    '3:2': [720, 480],
    '2:3': [480, 720],
  },
  '720p': {
    '16:9': [1280, 720],
    '4:3': [960, 720],
    '1:1': [720, 720],
    '3:4': [720, 960],
    '9:16': [720, 1280],
    '3:2': [1080, 720],
    '2:3': [720, 1080],
  },
}

function resolveGrokRatio(input: GenerationInput, aspectRatio: string | undefined): Ratio {
  if (aspectRatio && aspectRatio !== 'auto')
    return aspectRatio as Ratio
  const measured = firstMetaSize(input.media, ['start_image'])
  return measured ? closestRatioBySize(CONCRETE_RATIOS, measured) : '16:9'
}

export const grokImagine = defineJob({
  jobSetType: 'grok_video',
  outputType: 'video',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['start_image'],
      counts: { start_image: { max: 1 } },
```

```
    },
    settings: {
      duration: z._default(z.duration({ min: 1, max: 15 }), 6),
      resolution: z._default(z.enum(RESOLUTIONS), '720p'),
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(ASPECT_RATIOS), 'auto')),
    },
  },
  credits: ({ settings }) => Math.ceil((settings.duration ?? 6) * 1.5),
  validate: ({ prompt, settings }) => [
    ...promptRequired(prompt),
    ...promptMax(prompt, 2500),
    ...intRange('duration', settings.duration, 1, 15),
    ...integerRange('duration', settings.duration, 1, 15),
    ...oneOf('resolution', settings.resolution, RESOLUTIONS),
    ...oneOf('aspectRatio', settings.aspectRatio, ASPECT_RATIOS),
  ],
  finalize: (wire, input) => {
    const ratio = resolveGrokRatio(input, wire.aspect_ratio as string)
    const { width, height } = lookupSize(SIZE_MAP, wire.resolution as Resolution, ratio)
    return {
      ...wire,
      aspect_ratio: ratio,
      width,
      height,
      medias: wire.medias ?? [],
    }
  },
})

export const grokImagineV15 = defineJob({
  jobSetType: 'grok_video_v15',
  outputType: 'video',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['start_image'],
      counts: { start_image: { min: 1, max: 1 } },
    },
    settings: {
      duration: z._default(z.duration({ min: 2, max: 15 }), 6),
      resolution: z._default(z.enum(RESOLUTIONS), '720p'),
    },
  },
  credits: ({ settings }) => {
    const duration = Math.max(settings.duration ?? 6, 2)
    return duration * (settings.resolution === '720p' ? 4.5 : 2.5)
  },
  validate: ({ prompt, settings }) => [
    ...promptRequired(prompt),
    ...promptMax(prompt, 2500),
    ...intRange('duration', settings.duration, 2, 15),
    ...integerRange('duration', settings.duration, 2, 15),
    ...oneOf('resolution', settings.resolution, RESOLUTIONS),
  ],
  finalize: (wire, input) => {
    const ratio = resolveGrokRatio(input, 'auto')
    const { width, height } = lookupSize(SIZE_MAP, wire.resolution as Resolution, ratio)
    return {
      ...wire,
      duration: Math.max(wire.duration as number, 2),
      aspect_ratio: 'auto',
      width,
      height,
      medias: wire.medias ?? [],
    }
  },
})
```

app/packages/fnf/src/jobs/happy-horse.ts

```
import { defineJob } from '../define-job'
import { z } from '../z'
import { intRange, oneOf, promptMax, randomSeed } from './checks'
import { lookupSize } from './dimensions'
import { firstMetaSize, integerRange, requiredPromptOrRole } from './video-helpers'

export const HappyHorseAspectRatio = {
  r16x9: '16:9',
  r9x16: '9:16',
  r1x1: '1:1',
  r4x3: '4:3',
  r3x4: '3:4',
} as const

export const HappyHorseResolution = {
  r720: '720p',
  r1080: '1080p',
} as const

const ASPECT_RATIOS = Object.values(HappyHorseAspectRatio)
const RESOLUTIONS = Object.values(HappyHorseResolution)

type Ratio = typeof ASPECT_RATIOS[number]
type Resolution = typeof RESOLUTIONS[number]

const SIZE_MAP: Record<Resolution, Record<Ratio, [number, number]>> = {
  '720p': { '16:9': [1280, 720], '9:16': [720, 1280], '1:1': [720, 720], '4:3': [960, 720], '3:4': [720, 960] },
  '1080p': { '16:9': [1920, 1080], '9:16': [1080, 1920], '1:1': [1080, 1080], '4:3': [1440, 1080], '3:4': [1080, 1440] },
}

export const happyHorse = defineJob({
  jobSetType: 'happy_horse_video',
  outputType: 'video',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['start_image'],
      counts: { start_image: { max: 1 } },
    },
    settings: {
      resolution: z._default(z.enum(RESOLUTIONS), '720p'),
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(ASPECT_RATIOS), '16:9')),
      duration: z._default(z.duration({ min: 3, max: 15 }), 5),
      batchSize: z.wire('batch_size', z._default(z.number(), 1)),
      seed: z.optional(z.number()),
    },
  },
  validate: input => [
    ...requiredPromptOrRole(input, 'start_image', 'Prompt is required when no image is provided'),
    ...promptMax(input.prompt, 4000),
    ...oneOf('resolution', input.settings.resolution, RESOLUTIONS),
    ...oneOf('aspectRatio', input.settings.aspectRatio, ASPECT_RATIOS),
    ...intRange('duration', input.settings.duration, 3, 15),
    ...integerRange('duration', input.settings.duration, 3, 15),
    ...intRange('batchSize', input.settings.batchSize, 1, 4),
    ...integerRange('batchSize', input.settings.batchSize, 1, 4),
  ],
  finalize: (wire, input) => {
    const measured = firstMetaSize(input.media, ['start_image'])
    const table = lookupSize(SIZE_MAP, wire.resolution as Resolution, wire.aspect_ratio as Ratio)
    return {
      ...wire,
      prompt: wire.prompt ?? '',
      seed: wire.seed ?? randomSeed(),
      width: measured?.width ?? table.width,
      height: measured?.height ?? table.height,
```

```
      medias: wire.medias ?? [],
    }
  },
})
```

app/packages/fnf/src/jobs/image-helpers.ts

```
import type { MediaRef } from '../types'
import { ValidationError } from '../errors'
import { z } from '../z'

export interface WireMediaData {
  id: string
  type: string
  url?: string
}

export const mediaRefSchema = z.custom<MediaRef>(isMediaRef)

export function isMediaRef(value: unknown): value is MediaRef {
  if (!value || typeof value !== 'object')
    return false
  const ref = value as Partial<MediaRef>
  return typeof ref.id === 'string' && typeof ref.type === 'string'
}

export function toWireMediaData(value: unknown): WireMediaData | undefined {
  if (!isMediaRef(value))
    return undefined
  return {
    id: value.id,
    type: value.type,
    ...(value.url !== undefined ? { url: value.url } : {}),
  }
}

export function dimensionsFromRatios<Ratio extends string>(
  ratios: Record<Ratio, readonly [number, number]>,
  aspectRatio: Ratio,
  base: number,
): { width: number, height: number } {
  const ratio = ratios[aspectRatio]
  if (!ratio) {
    throw new ValidationError(`aspect ratio '${aspectRatio}' is not supported (allowed: ${Object.keys(ratios).join(', ')})`, [
      { loc: ['settings', 'aspectRatio'], msg: `aspect ratio '${aspectRatio}' is not supported` },
    ])
  }

  const [wRatio, hRatio] = ratio
  const maxRatio = Math.max(wRatio, hRatio)
  return {
    width: Math.round(base * (wRatio / maxRatio)),
    height: Math.round(base * (hRatio / maxRatio)),
  }
}
```

app/packages/fnf/src/jobs/index.ts

```
/**
 * The generation catalog — model declarations and the toolkit to author them.
 * This entry is intentionally separate from `@higgsfield/fnf/client`: a bundle
 * that only declares/ships models never pulls the client (and vice versa).
 */
export { defineJob } from '../define-job'
export type { DefineJobConfig, Envelope, JobEntry, JobParams, MediaConfigFor, SettingsInput } from '../define-job'
// Cross-role rule combinators for media declarations: cardinality
// (requiresOneOf/atLeastOneOf/maxTotal) and meta (dimensionsWithin/durationsWithin).
export { atLeastOneOf, checkMedia, dimensionsWithin, durationsWithin, maxTotal, requiresOneOf } from '../groups/media'
export type { DimensionLimits, MediaIssue, MediaRule } from '../groups/media'

// The settings-schema toolkit model authors need (z.wire / z.aspectRatio / z.duration).
export { aspectRatio, duration, getNormalize, getWireName, wire, z } from '../z'
export type { Normalize } from '../z'

// Validate-hook issue builders (uniform pydantic-shaped issues across models).
export { countRefs, intRange, oneOf, promptMax, promptRequired, randomSeed } from './checks'
export { aspectRatioDimensions, closestRatioBySize, firstSizeMeta, lookupSize, simplifyRatio } from './dimensions'
export type { AspectRatioDimensions } from './dimensions'
export { gptImage2, GptImage2AspectRatio } from './gpt-image-2'
export { grokImagine, GrokImagineAspectRatio, GrokImagineResolution, grokImagineV15 } from './grok-imagine'
export { happyHorse, HappyHorseAspectRatio, HappyHorseResolution } from './happy-horse'
export { mediaRefSchema, toWireMediaData } from './image-helpers'
export { KLING_DEFAULT_MOTION_ID, KlingModel, klingVideo } from './kling'
export { kling3_0, Kling3AspectRatio, Kling3Mode, Kling3Sound } from './kling-3'
export { kling3MotionControl, Kling3MotionControlMode, Kling3MotionControlOrientation } from './kling-3-motion-control'
export { nanoBanana2, NanoBanana2AspectRatio } from './nano-banana-2'
export { nanoBanana2Upscale, NanoBanana2UpscaleAspectRatio } from './nano-banana-2-upscale'
export { nanoBananaFlash, NanoBananaFlashAspectRatio } from './nano-banana-flash'
export { RecraftV41AspectRatio, recraftV41Image, RecraftV41Model, RecraftV41ModelType } from './recraft-v4-1'
export { seedance2_0, Seedance2AspectRatio } from './seedance-2-0'
export { getSeedance25MediaDurationIssues, getSeedance25MediaLimitIssues, seedance2_5, SEEDANCE_2_5_MAX_PROMPT_LENGTH, SEEDANCE25_MAX_AUDIO_DURATION, SEEDANCE25_MAX_AUDIOS, SEEDANCE25_MAX_IMAGES, SEED
export type { Seedance25DurationRef, Seedance25MediaCounts, Seedance25MediaDurationCode, Seedance25MediaLimitCode, Seedance25ModelValue, Seedance25ValidationCode, Seedance25ValidationIssue } from './s
export { seedreamV4_5, SeedreamV4_5AspectRatio } from './seedream-v4-5'
export { DEFAULT_SOUL_CINEMA_OLZHAS_STYLE_ID, DEFAULT_SOUL_CINEMA_STYLE_ID, DEFAULT_SOUL_V2_STYLE_ID, soulCinemaImage, SoulV2AspectRatio, soulV2Image } from './soul-v2'
export { DEFAULT_SOUL_STYLE_ID, textToImageSoul } from './text2image-soul'
export { topazImageGenerativeUpscale, TopazImageModel, topazImageUpscale, TopazImageUpscaleFactor } from './topaz-image-upscale'
export { veo3_1Lite, Veo31LiteAspectRatio, Veo31LiteResolution } from './veo-3-1-lite'
export { batch, extractAngleRefIds, extractReferenceElementIds, firstMetaDuration, firstMetaSize, integerRange, refsFor, requiredPromptOrRole } from './video-helpers'
export type { Size } from './video-helpers'
export { bytedanceVideoUpscale, BytedanceVideoUpscalePreset, BytedanceVideoUpscaleResolution, higgsfieldVideoUpscale, soraEnhanceVideo, TopazVideoEnhancementModel, TopazVideoFocusFix, TopazVideoModel,
export { wan27, Wan27AspectRatio, Wan27Quality } from './wan-2-7'
```

app/packages/fnf/src/jobs/kling-3-motion-control.ts

```
import { defineJob } from '../define-job'
import { dimensionsWithin, durationsWithin } from '../groups/media'
import { z } from '../z'
import { countRefs, promptMax } from './checks'
import { firstMetaDuration, firstMetaSize, integerRange } from './video-helpers'

export const Kling3MotionControlMode = {
  std: 'std',
  pro: 'pro',
} as const

export const Kling3MotionControlOrientation = {
  image: 'image',
  video: 'video',
} as const

const MODES = Object.values(Kling3MotionControlMode)
const ORIENTATIONS = Object.values(Kling3MotionControlOrientation)
const DEFAULT_WIDTH = 1280
const DEFAULT_HEIGHT = 720

export const kling3MotionControl = defineJob({
  jobSetType: 'kling3_0_motion_control',
  outputType: 'video',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['image', 'video'],
      counts: { image: { min: 1, max: 4 }, video: { min: 1, max: 1 } },
      rules: [
        dimensionsWithin(['image'], { minSide: 300, ratio: [0.4, 2.5] }),
        dimensionsWithin(['video'], { minSide: 340, maxSide: 3850, ratio: [1 / 3, 3] }),
        durationsWithin(['video'], { each: [3, 30] }),
      ],
    },
    settings: {
      mode: z._default(z.enum(MODES), 'std'),
      characterOrientation: z.wire('character_orientation', z._default(z.enum(ORIENTATIONS), 'image')),
      duration: z.optional(z.duration({ min: 3, max: 30 })),
      seed: z.optional(z.number()),
      isChain: z._default(z.boolean(), false),
      backgroundSource: z.wire('background_source', z.optional(z.string())),
    },
  },
  credits: ({ settings, media }) => {
    const duration = settings.duration ?? firstMetaDuration(media, ['video'])
    if (duration == null)
      return null
    const perSecond = settings.mode === 'pro' ? 2.5 : 1.5
    return Math.ceil(perSecond * duration + (settings.isChain ? 2 : 0))
  },
  validate: ({ prompt, media, settings }) => {
    const issues = [...promptMax(prompt, 2500)]
    if (countRefs(media, 'video') === 0)
      issues.push({ loc: ['media', 'video'], msg: 'Input Video Required' })
    if (countRefs(media, 'image') === 0)
      issues.push({ loc: ['media', 'image'], msg: 'Input Image Required' })

    const duration = settings.duration ?? firstMetaDuration(media, ['video'])
    if (duration != null)
      issues.push(...integerRange('duration', duration, 3, 30))
    return issues
  },
  finalize: (wire, input) => {
    const size = firstMetaSize(input.media, ['video', 'image'])
    const duration = wire.duration ?? firstMetaDuration(input.media, ['video'])
```

```
    return {
      ...wire,
      prompt: wire.prompt ?? '',
      width: typeof wire.width === 'number' ? wire.width : size?.width ?? DEFAULT_WIDTH,
      height: typeof wire.height === 'number' ? wire.height : size?.height ?? DEFAULT_HEIGHT,
      ...(duration != null ? { duration } : {}),
    }
  },
})
```

app/packages/fnf/src/jobs/kling-3.ts

```
import { defineJob } from '../define-job'
import { dimensionsWithin } from '../groups/media'
import { z } from '../z'
import { countRefs, intRange, oneOf, promptMax } from './checks'
import { closestRatioBySize, lookupSize } from './dimensions'
import { extractAngleRefIds, firstMetaSize, integerRange } from './video-helpers'

export const Kling3AspectRatio = {
  r16x9: '16:9',
  r9x16: '9:16',
  r1x1: '1:1',
} as const

export const Kling3Mode = {
  std: 'std',
  pro: 'pro',
  k4: '4k',
} as const

export const Kling3Sound = {
  on: 'on',
  off: 'off',
} as const

const ASPECT_RATIOS = Object.values(Kling3AspectRatio)
const MODES = Object.values(Kling3Mode)
const SOUNDS = Object.values(Kling3Sound)
const SHOT_PROMPT_MAX = 512
const NORMAL_PROMPT_MAX = 2500
const MAX_SHOTS = 6
const MAX_ELEMENTS = 3
const MIN_SHOT_DURATION = 1
const MIN_TOTAL_DURATION = 3

type Kling3Ratio = typeof ASPECT_RATIOS[number]
type Kling3ModeValue = typeof MODES[number]

const SIZE_MAP: Record<Kling3ModeValue, Record<Kling3Ratio, [number, number]>> = {
  'std': { '16:9': [1280, 720], '9:16': [720, 1280], '1:1': [720, 720] },
  'pro': { '16:9': [1920, 1080], '9:16': [1080, 1920], '1:1': [1080, 1080] },
  '4k': { '16:9': [3840, 2160], '9:16': [2160, 3840], '1:1': [2160, 2160] },
}

const multiPromptSchema = z.array(z.object({
  prompt: z.string(),
  duration: z.number(),
}))

export const kling3_0 = defineJob({
  jobSetType: 'kling3_0',
  outputType: 'video',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['start_image', 'end_image'],
      counts: { start_image: { max: 1 }, end_image: { max: 1 } },
      rules: [dimensionsWithin(['start_image', 'end_image'], { minSide: 300, ratio: [0.4, 2.5] })],
    },
    settings: {
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(ASPECT_RATIOS), '16:9')),
      mode: z._default(z.enum(MODES), 'std'),
      sound: z._default(z.enum(SOUNDS), 'off'),
      duration: z.duration({ min: 3, max: 15 }),
      multiShots: z.wire('multi_shots', z._default(z.boolean(), false)),
      multiShotMode: z.wire('multi_shot_mode', z._default(z.enum(['auto', 'custom']), 'auto')),
      multiPrompt: z.wire('multi_prompt', z.optional(multiPromptSchema)),
```

```
    klingElementIds: z.wire('kling_element_ids', z.optional(z.array(z.string()))),
  },
},
credits: ({ settings }) => {
  if (settings.mode !== '4k')
    return null
  const duration = settings.multiShots && settings.multiShotMode === 'custom' && settings.multiPrompt?.length
    ? settings.multiPrompt.reduce((sum, shot) => sum + (shot.duration || 0), 0)
    : settings.duration
  return Math.ceil(duration * 6)
},
validate: ({ prompt, media, settings }) => {
  const issues = [
    ...oneOf('aspectRatio', settings.aspectRatio, ASPECT_RATIOS),
    ...oneOf('mode', settings.mode, MODES),
    ...oneOf('sound', settings.sound, SOUNDS),
    ...intRange('duration', settings.duration, 3, 15),
    ...integerRange('duration', settings.duration, 3, 15),
    ...promptMax(prompt, NORMAL_PROMPT_MAX),
  ]

  const text = prompt?.instruction ?? ''
  const hasStart = countRefs(media, 'start_image') > 0
  const hasEnd = countRefs(media, 'end_image') > 0
  const multiShots = settings.multiShots ?? false
  const multiPrompt = settings.multiPrompt ?? []
  const elementIds = settings.klingElementIds ?? []
  const canSkipPrompt = (prompt?.enhance ?? true) && hasStart && !multiShots
  const promptRequired = !multiShots || settings.multiShotMode === 'auto'

  if (/<<<[^>]+>>>/.test(text) && !hasStart)
    issues.push({ loc: ['media', 'start_image'], msg: 'Start frame is required when prompt contains element references' })
  if (hasEnd && !hasStart)
    issues.push({ loc: ['media', 'start_image'], msg: 'Start frame is required when end frame is provided' })
  if (hasStart && hasEnd && multiShots)
    issues.push({ loc: ['media', 'end_image'], msg: 'End frame cannot be used with multi-shots' })
  if (elementIds.length > 0 && !hasStart)
    issues.push({ loc: ['media', 'start_image'], msg: 'Start frame is required for elements' })
  if (promptRequired && !canSkipPrompt && text.length === 0)
    issues.push({ loc: ['prompt'], msg: 'Prompt is required' })

  if (multiShots && settings.multiShotMode === 'custom') {
    if (multiPrompt.length === 0)
      issues.push({ loc: ['settings', 'multiPrompt'], msg: 'Multi-shot prompt is required when multi-shot mode is custom' })
    if (multiPrompt.length > MAX_SHOTS)
      issues.push({ loc: ['settings', 'multiPrompt'], msg: `Multi-shot supports up to ${MAX_SHOTS} shots` })

    let totalDuration = 0
    for (const shot of multiPrompt) {
      const shotPrompt = shot.prompt ?? ''
      totalDuration += shot.duration || 0
      if (shotPrompt.length === 0)
        issues.push({ loc: ['settings', 'multiPrompt'], msg: 'Each shot must have a prompt' })
      if (shotPrompt.length > SHOT_PROMPT_MAX)
        issues.push({ loc: ['settings', 'multiPrompt'], msg: `Shot prompt is too long (max ${SHOT_PROMPT_MAX})` })
      if (!shot.duration || shot.duration < MIN_SHOT_DURATION)
        issues.push({ loc: ['settings', 'multiPrompt'], msg: `Each shot must have at least ${MIN_SHOT_DURATION}s duration` })
    }
    if (multiPrompt.length > 0 && totalDuration < MIN_TOTAL_DURATION)
      issues.push({ loc: ['settings', 'multiPrompt'], msg: `Total duration must be at least ${MIN_TOTAL_DURATION}s` })
    if (totalDuration > 15)
      issues.push({ loc: ['settings', 'multiPrompt'], msg: 'Total duration must be at most 15s' })
    if (!hasStart && multiPrompt.some(shot => /<<<[^>]+>>>/.test(shot.prompt || '')))
      issues.push({ loc: ['media', 'start_image'], msg: 'Start frame is required when prompt contains element references' })
  }

  const refs = multiShots && multiPrompt.length
    ? multiPrompt.flatMap(shot => extractAngleRefIds(shot.prompt || ''))
    : extractAngleRefIds(text)
  if (new Set(refs).size > MAX_ELEMENTS || elementIds.length > MAX_ELEMENTS)
```

```
      issues.push({ loc: ['settings', 'klingElementIds'], msg: `Too many elements (max ${MAX_ELEMENTS}). Please remove some elements from your prompt.` })

    return issues
  },
  finalize: (wire, input) => {
    const measured = firstMetaSize(input.media, ['start_image'])
    const ratio = measured ? closestRatioBySize(ASPECT_RATIOS, measured) : wire.aspect_ratio as Kling3Ratio
    const { width, height } = lookupSize(SIZE_MAP, wire.mode as Kling3ModeValue, ratio)
    return {
      ...wire,
      prompt: wire.multi_shots && wire.multi_shot_mode === 'custom' ? '' : wire.prompt ?? '',
      aspect_ratio: ratio,
      width,
      height,
      enhance_prompt: wire.multi_shots ? false : wire.enhance_prompt,
      kling_element_ids: wire.kling_element_ids ?? [],
      duration: wire.multi_shots && wire.multi_shot_mode === 'custom' && Array.isArray(wire.multi_prompt) && wire.multi_prompt.length > 0
        ? wire.multi_prompt.reduce((sum, shot) => sum + (((shot as { duration?: number }).duration) || 0), 0)
        : wire.duration,
    }
  },
})
```

app/packages/fnf/src/jobs/kling.ts

```
import { defineJob } from '../define-job'
import { z } from '../z'
import { countRefs, oneOf } from './checks'
import { aspectRatioDimensions, firstSizeMeta, simplifyRatio } from './dimensions'

/** Kling model versions as a named, refactorable object enum (erased at runtime). */
export const KlingModel = {
  v2_1: 'kling-v2-1',
  v2_1Master: 'kling-v2-1-master',
  v2_5Turbo: 'kling-v2-5-turbo',
} as const

/**
 * The product's "no motion preset" sentinel (fnf-web defaultKlingPresetId):
 * the form always puts a motion_id on the wire, and THIS id means "none" —
 * KlingVideoJob.submit forces enhance_prompt only for ids different from it.
 */
export const KLING_DEFAULT_MOTION_ID = '7077cde8-7947-46d6-aea2-dbf2ff9d441c'

const MAX_PROMPT_CHARACTERS = 2000 // fnf-web submit/kling.ts: throws at >= 2000

// The only aspect_ratio values the backend accepts (fnf-api types_/kling.py).
const KLING_WIRE_RATIOS = new Set(['16:9', '9:16', '1:1'])

/**
 * Kling video — the /ai/video product model. Grounded in fnf-web's
 * VideoKlingSubmitParams + KlingVideoJob.submit (entities/job/model/submit/
 * kling.ts) and mapKlingParams: prompt is OPTIONAL (< 2000 chars when present —
 * motion presets run promptless), `mode` is NOT an input (derived from
 * model/resolution), enhance_prompt is forced on for any real motion preset
 * (motion_id !== the default sentinel). I2V via `input_image`; left optional
 * because the surfaces disagree — the /ai/video form hard-requires a start
 * frame ('Start frame required') but the app-viewer host advertises text-only
 * kling and the wire contract admits input_image: null. `input_image_end` is
 * not a declared role yet — but when a caller passes it via `extra`, finalize
 * still applies the product's enhance rule (an end frame forces it off).
 * NOTE: `parentId` is NOT honored by POST /jobs/kling — the backend's
 * CreateKlingJobSchema has no parent_id field, so it is silently dropped.
 */
export const klingVideo = defineJob({
  jobSetType: 'kling',
  outputType: 'video',
  params: {
    prompt: true,
    /** First frame to animate (I2V), as an uploaded `/media` input image. */
    media: { field: 'input_image', format: 'single', roles: ['input_image'] },
    settings: {
      model: z._default(z.enum(Object.values(KlingModel)), KlingModel.v2_5Turbo),
      // NOTE: the product also wires seed and cfg_scale, but the backend's
      // KlingParamsSchema declares neither (fnf-api schemas/job/kling.py) and
      // pydantic silently drops them — kling runs are not seeded or
      // CFG-tunable, so the SDK doesn't advertise the knobs.
      resolution: z._default(z.enum(['720p', '1080p']), '720p'),
      cameraControl: z.wire('camera_control', z._default(z.nullable(z.string()), null)),
      // The wire always carries a motion_id; the sentinel means "no preset"
      // (mapKlingParams: preset_id ?? defaultKlingPresetId).
      motionId: z.wire('motion_id', z._default(z.nullable(z.string()), KLING_DEFAULT_MOTION_ID)),
      duration: z.duration({ values: [5, 10] }),
      /** Fallback for width/height when the start image's size is unknown locally. */
      aspectRatio: z.wire('aspect_ratio', z._default(z.string(), '16:9')),
    },
  },
  validate: ({ prompt, media, settings }) => {
    // RAW length, like the product (KlingVideoJob.submit checks the untrimmed
    // string — the same form the codec puts on the wire).
    const raw = prompt?.instruction ?? ''
    const issues = raw.length >= MAX_PROMPT_CHARACTERS
      ? [{ loc: ['prompt'], msg: `Prompt is too long (max ${MAX_PROMPT_CHARACTERS} characters)` }]
```

```
      : []
    issues.push(...oneOf('duration', settings.duration, [5, 10]))
    // The backend's check_input_image (fnf-api schemas/job/kling.py): a start
    // frame is required for every model/mode EXCEPT v2-5-turbo at 1080p (pro) —
    // the only text-only combination that exists server-side. Fail locally with
    // the backend's own wording instead of a 422.
    const textOnlyAllowed = settings.model === KlingModel.v2_5Turbo && settings.resolution === '1080p'
    if (countRefs(media, 'input_image') === 0 && !textOnlyAllowed)
      issues.push({ loc: ['media', 'input_image'], msg: 'input image is required for this model and mode' })
    return issues
  },
  finalize: (wire, input) => {
    const model = wire.model as string
    const resolution = wire.resolution as string
    // The product never wires a null motion_id — mapKlingParams coalesces to
    // the sentinel BEFORE the enhance check. Legacy SDK params (and explicit
    // null callers) coalesce the same way, so a stored enhance_prompt: false
    // is respected on resubmit instead of being force-flipped.
    const motionId = wire.motion_id ?? KLING_DEFAULT_MOTION_ID
    // Wire width/height are the start image's intrinsic size in the product
    // (mapKlingParams measures it). Precedence: the ref's MediaRef.meta
    // (resolveMediaMeta — wins so a swapped image re-measures) → an existing
    // wire value (a round-tripped generation's params, via extra) → the
    // aspect-ratio box as a last resort.
    const measured = firstSizeMeta(input.media, ['input_image'])
    const box = aspectRatioDimensions(wire.aspect_ratio as string | undefined, 720)
    const width = measured?.width ?? (typeof wire.width === 'number' ? wire.width : box.width)
    const height = measured?.height ?? (typeof wire.height === 'number' ? wire.height : box.height)
    // aspect_ratio IS an optional backend param, but only as the enum
    // 16:9 | 9:16 | 1:1 (fnf-api types_/kling.py) — pass it through when it
    // fits, drop anything else (the wire's real geometry is width/height).
    const { aspect_ratio, ...rest } = wire
    const wireRatio = KLING_WIRE_RATIOS.has(aspect_ratio as string) ? { aspect_ratio } : {}
    return {
      ...rest,
      ...wireRatio,
      prompt: wire.prompt ?? '', // the product always sends a string ('' when promptless)
      motion_id: motionId,
      width,
      height,
      // Derivations from prod KlingVideoJob.submit:
      mode: model !== KlingModel.v2_1Master ? (resolution === '1080p' ? 'pro' : 'std') : 'std',
      // The product's exact precedence (KlingVideoJob.submit): a real preset
      // forces it on; an end frame (reaching the wire via extra — the role is
      // not declared yet) forces it off; otherwise the caller's choice,
      // defaulting to true like the product form (enhance ?? true).
      enhance_prompt: motionId !== KLING_DEFAULT_MOTION_ID
        ? true
        : wire.input_image_end ? false : wire.enhance_prompt ?? true,
    }
  },
  restore: (wire) => {
    // finalize keeps aspect_ratio only for the backend's 16:9|9:16|1:1 enum —
    // when the stored params carry none, derive it from width/height so the
    // parsed input reads naturally; the round-tripped width/height in extra
    // still win in finalize, so resubmits keep the exact original dims.
    if (wire.aspect_ratio == null && typeof wire.width === 'number' && typeof wire.height === 'number' && wire.height > 0)
      return { aspect_ratio: simplifyRatio(wire.width, wire.height) }
    return {}
  },
})
```

app/packages/fnf/src/jobs/nano-banana-2-upscale.ts

```
import { defineJob } from '../define-job'
import { atLeastOneOf } from '../groups/media'
import { z } from '../z'
import { oneOf } from './checks'
import {
  getNanoBananaDimensions,
  NANO_BANANA_ASPECT_RATIO_VALUES,
  NanoBananaAspectRatio,
  type NanoBananaResolution,
  resolveNanoBananaRatio,
} from './nano-banana-shared'

export { NanoBananaAspectRatio as NanoBanana2UpscaleAspectRatio }

const RESOLUTIONS = ['2k', '4k'] as const
type Resolution = typeof RESOLUTIONS[number]

const CREDITS: Record<Resolution, number> = {
  '2k': 2,
  '4k': 4,
}

export const nanoBanana2Upscale = defineJob({
  jobSetType: 'nano_banana_2_upscale',
  outputType: 'image',
  params: {
    media: {
      field: 'input_images',
      format: 'unwrapped',
      roles: ['image'],
      counts: { image: { min: 1 } },
      rules: [atLeastOneOf(['image'])],
    },
    settings: {
      resolution: z._default(z.enum(RESOLUTIONS), '4k'),
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(NANO_BANANA_ASPECT_RATIO_VALUES), NanoBananaAspectRatio.auto)),
    },
  },
  credits: ({ settings }) => CREDITS[settings.resolution ?? '4k'],
  validate: ({ settings }) => [
    ...oneOf('resolution', settings.resolution, RESOLUTIONS),
    ...oneOf('aspectRatio', settings.aspectRatio, NANO_BANANA_ASPECT_RATIO_VALUES),
  ],
  finalize: (wire, input) => {
    const ratio = resolveNanoBananaRatio(input, ['image'], wire.aspect_ratio as never)
    const { width, height } = getNanoBananaDimensions(wire.resolution as NanoBananaResolution, ratio)
    return {
      ...wire,
      aspect_ratio: ratio,
      width,
      height,
    }
  },
})
```

app/packages/fnf/src/jobs/nano-banana-2.ts

```
import { defineJob } from '../define-job'
import { dimensionsWithin } from '../groups/media'
import { z } from '../z'
import { intRange, oneOf, promptMax, promptRequired } from './checks'
import {
  getNanoBananaDimensions,
  NANO_BANANA_ASPECT_RATIO_VALUES,
  NanoBananaAspectRatio,
  type NanoBananaResolution,
  resolveNanoBananaRatio,
} from './nano-banana-shared'

export { NanoBananaAspectRatio as NanoBanana2AspectRatio }

const MAX_PROMPT_CHARACTERS = 15_000

const CREDITS_PER_IMAGE: Record<NanoBananaResolution, number> = {
  '1k': 2,
  '2k': 2,
  '4k': 4,
}

/**
 * Nano Banana Pro. Grounded in fnf-web's `job-image-nano-banana-2` module.
 * The app submits `/jobs/nano-banana-2` with `input_images` kept as a bare
 * array. Public settings carry ONLY user generation input — the product's
 * surface/billing markers (application_slug, is_draw/is_ugc/…, use_unlim,
 * use_seedream_bonus) are not part of the SDK surface; deliberate raw wire
 * fields belong in `extra`.
 */
export const nanoBanana2 = defineJob({
  jobSetType: 'nano_banana_2',
  outputType: 'image',
  params: {
    prompt: true,
    media: {
      field: 'input_images',
      format: 'unwrapped',
      roles: ['image'],
      counts: { image: { max: 14 } },
      rules: [dimensionsWithin(['image'], { minSide: 128 })],
    },
    settings: {
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(NANO_BANANA_ASPECT_RATIO_VALUES), '3:4')),
      resolution: z._default(z.enum(['1k', '2k', '4k']), '1k'),
      batchSize: z.wire('batch_size', z._default(z.number(), 1)),
    },
  },
  credits: ({ settings }) => (settings.batchSize ?? 1) * CREDITS_PER_IMAGE[settings.resolution ?? '1k'],
  validate: ({ prompt, settings }) => [
    ...promptRequired(prompt, 2),
    ...promptMax(prompt, MAX_PROMPT_CHARACTERS, { inclusive: true }),
    ...intRange('batchSize', settings.batchSize, 1, 4),
    ...oneOf('aspectRatio', settings.aspectRatio, NANO_BANANA_ASPECT_RATIO_VALUES),
  ],
  finalize: (wire, input) => {
    const ratio = resolveNanoBananaRatio(input, ['image'], wire.aspect_ratio as never)
    const { width, height } = getNanoBananaDimensions((wire.resolution ?? '1k') as NanoBananaResolution, ratio)

    return {
      ...wire,
      aspect_ratio: ratio,
      width,
      height,
      input_images: wire.input_images ?? [],
    }
  },
})
```

app/packages/fnf/src/jobs/nano-banana-flash.ts

```
import { defineJob } from '../define-job'
import { dimensionsWithin } from '../groups/media'
import { z } from '../z'
import { intRange, oneOf, promptMax, promptRequired } from './checks'
import {
  getNanoBananaDimensions,
  NANO_BANANA_ASPECT_RATIO_VALUES,
  NanoBananaAspectRatio,
  type NanoBananaResolution,
  resolveNanoBananaRatio,
} from './nano-banana-shared'

export { NanoBananaAspectRatio as NanoBananaFlashAspectRatio }

const MAX_PROMPT_CHARACTERS = 15_000

const CREDITS_PER_IMAGE: Record<NanoBananaResolution, number> = {
  '1k': 1.5,
  '2k': 2,
  '4k': 3,
}

/**
 * Nano Banana 2. Grounded in fnf-web's `job-image-nano-banana-flash` module.
 * This is separate from Nano Banana Pro: it posts to `/jobs/v2/nano_banana_flash`
 * and uses wrapped `medias` rather than bare `input_images`.
 */
export const nanoBananaFlash = defineJob({
  jobSetType: 'nano_banana_flash',
  outputType: 'image',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['image'],
      counts: { image: { max: 14 } },
      rules: [dimensionsWithin(['image'], { minSide: 128 })],
    },
    settings: {
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(NANO_BANANA_ASPECT_RATIO_VALUES), '3:4')),
      resolution: z._default(z.enum(['1k', '2k', '4k']), '1k'),
      batchSize: z.wire('batch_size', z._default(z.number(), 1)),
    },
  },
  credits: ({ settings }) => (settings.batchSize ?? 1) * CREDITS_PER_IMAGE[settings.resolution ?? '1k'],
  validate: ({ prompt, settings }) => [
    ...promptRequired(prompt, 2),
    ...promptMax(prompt, MAX_PROMPT_CHARACTERS, { inclusive: true }),
    ...intRange('batchSize', settings.batchSize, 1, 4),
    ...oneOf('aspectRatio', settings.aspectRatio, NANO_BANANA_ASPECT_RATIO_VALUES),
  ],
  finalize: (wire, input) => {
    const ratio = resolveNanoBananaRatio(input, ['image'], wire.aspect_ratio as never)
    const { width, height } = getNanoBananaDimensions((wire.resolution ?? '1k') as NanoBananaResolution, ratio)
    return {
      ...wire,
      aspect_ratio: ratio,
      width,
      height,
    }
  },
})
```

app/packages/fnf/src/jobs/nano-banana-shared.ts

```
import type { GenerationInput } from '../types'
import { closestRatioBySize, firstSizeMeta, lookupSize } from './dimensions'

export const NanoBananaAspectRatio = {
  auto: 'auto',
  r1x1: '1:1',
  r3x2: '3:2',
  r2x3: '2:3',
  r4x3: '4:3',
  r3x4: '3:4',
  r4x5: '4:5',
  r5x4: '5:4',
  r9x16: '9:16',
  r16x9: '16:9',
  r21x9: '21:9',
} as const

export type NanoBananaAspectRatioValue = typeof NanoBananaAspectRatio[keyof typeof NanoBananaAspectRatio]
export type NanoBananaConcreteRatio = Exclude<NanoBananaAspectRatioValue, 'auto'>
export type NanoBananaResolution = '1k' | '2k' | '4k'

export const NANO_BANANA_ASPECT_RATIO_VALUES = ['auto', '1:1', '3:2', '2:3', '4:3', '3:4', '4:5', '5:4', '9:16', '16:9', '21:9'] as const

// Iteration order mirrors fnf-web's getNanoBanana2ClosestAspectRatio.
export const NANO_BANANA_CONCRETE_RATIOS = ['1:1', '2:3', '3:2', '3:4', '4:3', '4:5', '5:4', '9:16', '16:9', '21:9'] as const

export const NANO_BANANA_SIZE_MAP: Record<NanoBananaResolution, Record<NanoBananaConcreteRatio, [number, number]>> = {
  '1k': { '1:1': [1024, 1024], '2:3': [848, 1264], '3:2': [1264, 848], '3:4': [896, 1200], '4:3': [1200, 896], '4:5': [928, 1152], '5:4': [1152, 928], '9:16': [768, 1376], '16:9': [1376, 768], '21:9':
  '2k': { '1:1': [2048, 2048], '2:3': [1696, 2528], '3:2': [2528, 1696], '3:4': [1792, 2400], '4:3': [2400, 1792], '4:5': [1856, 2304], '5:4': [2304, 1856], '9:16': [1536, 2752], '16:9': [2752, 1536],
  '4k': { '1:1': [4096, 4096], '2:3': [3392, 5056], '3:2': [5056, 3392], '3:4': [3584, 4800], '4:3': [4800, 3584], '4:5': [3712, 4608], '5:4': [4608, 3712], '9:16': [3072, 5504], '16:9': [5504, 3072],
}

export function resolveNanoBananaRatio(input: GenerationInput, roles: readonly string[], ratio: NanoBananaAspectRatioValue): NanoBananaConcreteRatio {
  if (ratio !== 'auto')
    return ratio
  const size = firstSizeMeta(input.media, roles)
  return size ? closestRatioBySize(NANO_BANANA_CONCRETE_RATIOS, size) : '3:4'
}

export function getNanoBananaDimensions(resolution: NanoBananaResolution, ratio: NanoBananaConcreteRatio): { width: number, height: number } {
  return lookupSize(NANO_BANANA_SIZE_MAP, resolution, ratio)
}
```

app/packages/fnf/src/jobs/recraft-v4-1.ts

```
import { defineJob } from '../define-job'
import { z } from '../z'
import { intRange, oneOf, promptMax, promptRequired } from './checks'
import { dimensionsFromRatios } from './image-helpers'

export const RecraftV41Model = {
  standard: 'recraft-v4-1',
  vector: 'recraft-v4-1-vector',
  utility: 'recraft-v4-1-utility',
  utilityVector: 'recraft-v4-1-utility-vector',
} as const

export const RecraftV41ModelType = {
  standard: 'standard',
  vector: 'vector',
  utility: 'utility',
  utilityVector: 'utility_vector',
} as const

export const RecraftV41AspectRatio = {
  r1x1: '1:1',
  r3x4: '3:4',
  r4x3: '4:3',
  r4x5: '4:5',
  r5x4: '5:4',
  r3x2: '3:2',
  r2x3: '2:3',
  r16x9: '16:9',
  r9x16: '9:16',
} as const

type RecraftModel = typeof RecraftV41Model[keyof typeof RecraftV41Model]
type RecraftModelType = typeof RecraftV41ModelType[keyof typeof RecraftV41ModelType]
type RecraftAspectRatio = typeof RecraftV41AspectRatio[keyof typeof RecraftV41AspectRatio]
type Resolution = '1k' | '2k'

const MODEL_TO_TYPE: Record<RecraftModel, RecraftModelType> = {
  'recraft-v4-1': 'standard',
  'recraft-v4-1-vector': 'vector',
  'recraft-v4-1-utility': 'utility',
  'recraft-v4-1-utility-vector': 'utility_vector',
}

const TYPE_TO_MODEL: Record<RecraftModelType, RecraftModel> = {
  standard: 'recraft-v4-1',
  vector: 'recraft-v4-1-vector',
  utility: 'recraft-v4-1-utility',
  utility_vector: 'recraft-v4-1-utility-vector',
}

const ASPECT_RATIOS: Record<RecraftAspectRatio, readonly [number, number]> = {
  '1:1': [1, 1],
  '3:4': [3, 4],
  '4:3': [4, 3],
  '4:5': [4, 5],
  '5:4': [5, 4],
  '3:2': [3, 2],
  '2:3': [2, 3],
  '16:9': [16, 9],
  '9:16': [9, 16],
}

const BASE_SIZE: Record<Resolution, number> = {
  '1k': 1024,
  '2k': 2048,
}

const CREDITS_PER_IMAGE: Record<RecraftModelType, Record<Resolution, number>> = {
```

```
  standard: { '1k': 1.25, '2k': 8 },
  utility: { '1k': 1.25, '2k': 8 },
  vector: { '1k': 2.5, '2k': 10 },
  utility_vector: { '1k': 2.5, '2k': 10 },
}

const MAX_PROMPT_CHARACTERS = 3_000
const MAX_PALETTE_COLORS = 10

function modelTypeFor(model: RecraftModel, explicit?: RecraftModelType): RecraftModelType {
  return explicit ?? MODEL_TO_TYPE[model]
}

export const recraftV41Image = defineJob({
  jobSetType: 'recraft_v4_1',
  outputType: 'image',
  params: {
    prompt: true,
    settings: {
      model: z._default(z.enum(Object.values(RecraftV41Model)), 'recraft-v4-1'),
      modelType: z.wire('model_type', z.optional(z.enum(Object.values(RecraftV41ModelType)))),
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(Object.values(RecraftV41AspectRatio)), '1:1')),
      resolution: z._default(z.enum(['1k', '2k']), '1k'),
      batchSize: z.wire('batch_size', z._default(z.number(), 1)),
      colors: z.optional(z.array(z.string())),
      backgroundColor: z.wire('background_color', z.optional(z.nullable(z.string()))),
    },
  },
  credits: ({ settings }) => {
    const model = settings.model ?? 'recraft-v4-1'
    const modelType = modelTypeFor(model, settings.modelType)
    const resolution = settings.resolution ?? '1k'
    return (settings.batchSize ?? 1) * CREDITS_PER_IMAGE[modelType][resolution]
  },
  validate: ({ prompt, settings }) => {
    const model = settings.model ?? 'recraft-v4-1'
    const modelType = modelTypeFor(model, settings.modelType)
    const resolution = settings.resolution ?? '1k'
    const issues = [
      ...promptRequired(prompt),
      ...promptMax(prompt, MAX_PROMPT_CHARACTERS),
      ...intRange('batchSize', settings.batchSize, 1, 4),
      ...oneOf('aspectRatio', settings.aspectRatio, Object.values(RecraftV41AspectRatio)),
    ]
    if (settings.modelType && settings.modelType !== MODEL_TO_TYPE[model]) {
      issues.push({
        loc: ['settings', 'modelType'],
        msg: `modelType '${settings.modelType}' does not match model '${model}'`,
      })
    }
    if ((settings.colors?.length ?? 0) > MAX_PALETTE_COLORS) {
      issues.push({
        loc: ['settings', 'colors'],
        msg: `Color palette can include up to ${MAX_PALETTE_COLORS} colors`,
      })
    }
    if (!CREDITS_PER_IMAGE[modelType]?.[resolution]) {
      issues.push({
        loc: ['settings', 'modelType'],
        msg: `modelType '${modelType}' is not supported`,
      })
    }
    return issues
  },
  finalize: (wire) => {
    const model = (wire.model ?? 'recraft-v4-1') as RecraftModel
    const modelType = modelTypeFor(model, wire.model_type as RecraftModelType | undefined)
    const aspectRatio = (wire.aspect_ratio ?? '1:1') as RecraftAspectRatio
    const resolution = (wire.resolution ?? '1k') as Resolution
    const { width, height } = dimensionsFromRatios(ASPECT_RATIOS, aspectRatio, BASE_SIZE[resolution])
```

```
    const rest = { ...wire }
    delete rest.model

    return {
      ...rest,
      model_type: modelType,
      width,
      height,
    }
  },
  restore: (wire) => {
    const modelType = typeof wire.model_type === 'string' ? wire.model_type as RecraftModelType : undefined
    return modelType && TYPE_TO_MODEL[modelType]
      ? { model: TYPE_TO_MODEL[modelType] }
      : {}
  },
})
```

app/packages/fnf/src/jobs/seedance-2-0.ts

```
import { defineJob } from '../define-job'
import { dimensionsWithin, durationsWithin, maxTotal, requiresOneOf } from '../groups/media'
import { z } from '../z'
import { countRefs, intRange, oneOf } from './checks'
import { closestRatioBySize, firstSizeMeta, lookupSize } from './dimensions'

/** Allowed aspect ratios as a named, refactorable object enum (erased at runtime). */
export const Seedance2AspectRatio = {
  auto: 'auto',
  r21x9: '21:9',
  r16x9: '16:9',
  r4x3: '4:3',
  r1x1: '1:1',
  r3x4: '3:4',
  r9x16: '9:16',
} as const

const MAX_PROMPT_CHARACTERS = 40_000 // fnf-web SEEDANCE_2_0_MAX_PROMPT_CHARACTERS
const FRAME_ROLES = ['start_image', 'end_image'] as const
const IMAGE_ROLES = ['image', 'start_image', 'end_image'] as const
// Iteration order mirrors the product's ASPECT_RATIO_VALUES (dimentions.ts) —
// closestRatioBySize is first-wins on ties, so order is part of the contract.
const CONCRETE_RATIOS = ['21:9', '16:9', '4:3', '1:1', '3:4', '9:16'] as const

type ConcreteRatio = typeof CONCRETE_RATIOS[number]
type Resolution = '480p' | '720p' | '1080p'

/**
 * Mirrors fnf-web's seedance-2-0 dimension table (job-video-seedance2-0/model/
 * dimentions.ts). The wire wants explicit width/height; with 'auto' and no
 * measurable image the app uses the 16:9 entry and keeps 'auto' on the wire.
 */
const SIZE_MAP: Record<Resolution, Record<ConcreteRatio, [number, number]>> = {
  '480p': { '21:9': [896, 384], '16:9': [854, 480], '4:3': [640, 480], '1:1': [480, 480], '3:4': [480, 640], '9:16': [480, 854] },
  '720p': { '21:9': [1344, 576], '16:9': [1280, 720], '4:3': [960, 720], '1:1': [720, 720], '3:4': [720, 960], '9:16': [720, 1280] },
  '1080p': { '21:9': [2016, 864], '16:9': [1920, 1080], '4:3': [1440, 1080], '1:1': [1080, 1080], '3:4': [1080, 1440], '9:16': [1080, 1920] },
}

const COST_PER_SECOND: Record<'std' | 'fast', Partial<Record<Resolution, number>>> = {
  std: { '480p': 3, '720p': 4.5, '1080p': 9 },
  fast: { '720p': 3.5 },
}

// Rules grounded in fnf-web's submit validation (job-video-seedance2-0/model/built.ts)
// + the gen-panel submit strategy (gen-panel-model-video-seedance-2-0/submit-strategy.ts)
// + the gen-panel aspect-ratio lock (gen-panel-model-video-seedance-2-0/fields.ts).
// Wire surface: POST /jobs/v2/seedance_2_0 with model/width/height/generate_audio.
// NOTE: the backend rejects input media whose IP check hasn't finished — upload
// inputs with `forceIpCheck: true` (the check then runs synchronously on confirm).
// NOTE: the app also counts reference elements embedded in the prompt text toward
// the 9-image budget; the SDK has no reference-elements concept, so prompts
// carrying that syntax share the cap server-side only.
export const seedance2_0 = defineJob({
  jobSetType: 'seedance_2_0',
  outputType: 'video',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['image', 'start_image', 'end_image', 'video', 'audio'],
      counts: { start_image: { max: 1 }, end_image: { max: 1 }, video: { max: 3 }, audio: { max: 3 } },
      rules: [
        // audio-only is not supported — it needs a visual to score
        requiresOneOf('audio', ['image', 'video', 'start_image', 'end_image']),
        // reference images (incl. frames) are capped at 9 in total
        maxTotal(['image', 'start_image', 'end_image'], 9),
        // Intrinsic-size/duration constraints from fnf-web's built.ts
```

```
      // (getMediaConstraintError + the per-media duration loops). They judge
      // MediaRef.meta when present — populate it from app data or via
      // resolveMediaMeta; refs without meta are left to the backend.
      dimensionsWithin(['image', 'start_image', 'end_image'], { minSide: 300, maxSide: 6000, ratio: [0.4, 2.5] }),
      dimensionsWithin(['video'], { minSide: 300, maxSide: 6000, minPixels: 409_600, ratio: [0.4, 2.5] }),
      durationsWithin(['video'], { each: [2, 15] }),
      durationsWithin(['audio'], { each: [2, 15] }),
      // videos + audios share one 15-second budget
      durationsWithin(['video', 'audio'], { total: 15 }),
    ],
  },
  settings: {
    // Per-model job param (seedance-2-0 honors batch_size in fnf-web). Orthogonal
    // to the client-side `count` fan-out: total outputs = count × batchSize.
    batchSize: z.wire('batch_size', z._default(z.number(), 1)),
    // Continuous 4–15s (the product slider), not a discrete set. Enforced in
    // validate (built.ts hard-rejects out-of-range), not just adjust()-snapped.
    duration: z.duration({ min: 4, max: 15 }),
    aspectRatio: z.wire('aspect_ratio', z.aspectRatio(Object.values(Seedance2AspectRatio))),
    // Per-mode product default applied in finalize: std → 1080p, fast → 720p
    // (gen-panel fields.ts; fast never offers 1080p).
    resolution: z.optional(z.enum(['480p', '720p', '1080p'])),
    // Public knob; maps to the wire `model` (seedance_2_0 / seedance_2_0_fast).
    mode: z._default(z.enum(['std', 'fast']), 'std'),
    generateAudio: z.wire('generate_audio', z._default(z.boolean(), true)),
    bitrateMode: z.wire('bitrate_mode', z._default(z.enum(['standard', 'high']), 'standard')),
  },
},
credits: ({ settings }) => {
  const mode = settings.mode ?? 'std'
  const resolution = settings.resolution ?? (mode === 'fast' ? '720p' : '1080p')
  const perSecond = COST_PER_SECOND[mode][resolution]
  if (perSecond === undefined)
    return null
  return Math.ceil((settings.duration ?? 4) * perSecond) * (settings.batchSize ?? 1)
},
validate: ({ prompt, media, settings }) => {
  const issues = []
  const text = (prompt?.instruction ?? '').trim()
  const visuals = countRefs(media, 'image') + countRefs(media, 'start_image')
    + countRefs(media, 'end_image') + countRefs(media, 'video')

  if (text.length > MAX_PROMPT_CHARACTERS)
    issues.push({ loc: ['prompt'], msg: `Prompt is too long (max ${MAX_PROMPT_CHARACTERS} characters)` })
  if (!text && visuals === 0)
    issues.push({ loc: ['prompt'], msg: 'Prompt is required when no media is provided' })
  // The gen-panel locks the ratio to 'auto' once frames are attached; a forced
  // concrete ratio with a frame fails the generation on the backend.
  if (FRAME_ROLES.some(role => countRefs(media, role) > 0) && settings.aspectRatio !== 'auto')
    issues.push({ loc: ['settings', 'aspectRatio'], msg: 'aspectRatio must be \'auto\' when a start/end frame is attached' })
  issues.push(...oneOf('aspectRatio', settings.aspectRatio, Object.values(Seedance2AspectRatio)))
  // built.ts: !duration || duration < 4 || duration > 15 is a hard reject.
  issues.push(...intRange('duration', settings.duration, 4, 15))
  issues.push(...intRange('batchSize', settings.batchSize, 1, 4))
  // The product's fast config offers 480p/720p only — fast@1080p has no cost
  // table entry and is unreachable from the app.
  if (settings.mode === 'fast' && settings.resolution === '1080p')
    issues.push({ loc: ['settings', 'resolution'], msg: 'resolution \'1080p\' is not available in fast mode' })
  return issues
},
finalize: (wire, input) => {
  const mode = wire.mode
  const resolution = (wire.resolution ?? (mode === 'fast' ? '720p' : '1080p')) as Resolution
  // 'auto' resolves like the product: measure the FIRST attached image-like
  // ref and snap to the closest concrete ratio (submit-strategy.ts). Without
  // meta, a round-tripped generation's stored width/height (riding in extra)
  // stand in for the measurement — restore mapped 'auto' back for the frame
  // lock, and the stored dims encode exactly what the frame resolved to.
  // Only a fresh 'auto' submit with no size knowledge at all keeps 'auto'
  // on the wire with the 16:9 dims (the product's no-image branch).
```

```
    let ratio = wire.aspect_ratio as string
    if (ratio === 'auto') {
      const size = firstSizeMeta(input.media, IMAGE_ROLES)
        ?? (typeof wire.width === 'number' && typeof wire.height === 'number' && wire.height > 0
          ? { width: wire.width, height: wire.height }
          : undefined)
      if (size)
        ratio = closestRatioBySize(CONCRETE_RATIOS, size)
    }
    // Dims always come fresh from the table — a parsed input whose resolution
    // or media changed re-derives instead of resubmitting stale extra dims.
    const { width, height } = lookupSize(SIZE_MAP, resolution, ratio === 'auto' ? '16:9' : ratio)
    return {
      ...wire,
      // `model` is the operative backend field (cost/plan/queue all key off
      // it); `mode` also stays on the wire — the backend schema declares it
      // and echoes it in stored params, so product UIs read the right label.
      model: mode === 'fast' ? 'seedance_2_0_fast' : 'seedance_2_0',
      prompt: wire.prompt ?? '', // required string on the wire; '' is valid with media
      aspect_ratio: ratio,
      resolution,
      width,
      height,
    }
  },
  restore: (wire) => {
    // Pre-mode-on-wire params only carry `model` — bring `mode` back so a
    // fetched fast generation resubmits as fast, not the 'std' schema default.
    const restored: Record<string, unknown> = {
      mode: wire.model === 'seedance_2_0_fast' ? 'fast' : 'std',
    }
    // With a frame attached the input ratio is necessarily 'auto' (validate
    // locks it); the concrete wire ratio came from the frame's measured size.
    // Map it back so the parsed input passes the same validate on resubmit.
    const medias = Array.isArray(wire.medias) ? wire.medias as Array<{ role?: string }> : []
    if (medias.some(m => m?.role === 'start_image' || m?.role === 'end_image'))
      restored.aspect_ratio = 'auto'
    return restored
  },
})
```

app/packages/fnf/src/jobs/seedance-2-5.ts

```
import type { MediaIssue } from '../groups/media'
import type { MediaInput } from '../types'
import { defineJob } from '../define-job'
import { z } from '../z'
import { oneOf, promptMax, promptRequired } from './checks'
import { closestRatioBySize, lookupSize } from './dimensions'
import { extractReferenceElementIds, integerRange, refsFor } from './video-helpers'

/**
 * Seedance 2.5 — the multi-reference video model. Ported from fnf-web's
 * `packages/fnf/src/jobs/seedance-2-5.ts` (+ `seedance-2-5-base.ts`, which
 * upstream extracted only because CS4.0 and the two edit jobs share the
 * tables; this catalog carries the one job, so the tables live here).
 *
 * Wire surface: POST /jobs/v2/seedance_2_5 with wrapped `medias`, explicit
 * width/height, and a `model` discriminator that changes which fields ship —
 * see `finalize`.
 *
 * Not ported deliberately: upstream's `business` group (`use_unlim`,
 * `use_free_gens`, `use_generation_seconds`). Those spend a Higgsfield user's
 * unlimited/free-generation grants, which app-built surfaces do not own — the
 * backend defaults them to false. Same call as `seedance2_0` in this catalog.
 */

/** Allowed aspect ratios as a named, refactorable object enum (erased at runtime). */
export const Seedance25AspectRatio = {
  r21x9: '21:9',
  r16x9: '16:9',
  r4x3: '4:3',
  r1x1: '1:1',
  r3x4: '3:4',
  r9x16: '9:16',
} as const

export const Seedance25Resolution = {
  r480: '480p',
  r720: '720p',
  r1080: '1080p',
} as const

/**
 * The wire `model` discriminator — one job set type, three shapes:
 * `default` generates, `video_edit` rewrites one attached video, and
 * `video_extension` continues one backward/forward.
 */
export const Seedance25Model = {
  default: 'default',
  videoEdit: 'video_edit',
  videoExtension: 'video_extension',
} as const

export const Seedance25ExtensionMode = {
  backward: 'backward',
  forward: 'forward',
} as const

export const Seedance25BitrateMode = {
  standard: 'standard',
  high: 'high',
} as const

const ASPECT_RATIOS = Object.values(Seedance25AspectRatio)
const RESOLUTIONS = Object.values(Seedance25Resolution)
const MODELS = Object.values(Seedance25Model)
const EXTENSION_MODES = Object.values(Seedance25ExtensionMode)
const BITRATE_MODES = Object.values(Seedance25BitrateMode)
const MEDIA_ROLES = ['image', 'start_image', 'end_image', 'video', 'audio'] as const
const IMAGE_ROLES = ['image', 'start_image', 'end_image'] as const
```

```
export const SEEDANCE_2_5_MAX_PROMPT_LENGTH = 60_000
export const SEEDANCE25_MAX_IMAGES = 30
export const SEEDANCE25_MAX_VIDEOS = 10
export const SEEDANCE25_MAX_AUDIOS = 10
export const SEEDANCE25_MAX_INPUT_MEDIA = 50
export const SEEDANCE25_MIN_MEDIA_DURATION = 2
export const SEEDANCE25_MAX_AUDIO_DURATION = 30
export const SEEDANCE25_MAX_TOTAL_VIDEO_DURATION = 30
export const SEEDANCE25_MAX_TOTAL_AUDIO_DURATION = 30

type AspectRatio = typeof ASPECT_RATIOS[number]
type Resolution = typeof RESOLUTIONS[number]
type Model = typeof MODELS[number]
type ExtensionMode = typeof EXTENSION_MODES[number]
type BitrateMode = typeof BITRATE_MODES[number]
export type Seedance25ModelValue = Model

/**
 * The render boxes, keyed `resolution × ratio`. Mirrors fnf-web's
 * SEEDANCE_2_5_SIZE_MAP; 1080p rows follow the same ratios at 1080 base.
 */
const SIZE_MAP: Record<Resolution, Record<AspectRatio, [number, number]>> = {
  '480p': { '21:9': [896, 384], '16:9': [854, 480], '4:3': [640, 480], '1:1': [480, 480], '3:4': [480, 640], '9:16': [480, 854] },
  '720p': { '21:9': [1344, 576], '16:9': [1280, 720], '4:3': [960, 720], '1:1': [720, 720], '3:4': [720, 960], '9:16': [720, 1280] },
  '1080p': { '21:9': [2016, 864], '16:9': [1920, 1080], '4:3': [1440, 1080], '1:1': [1080, 1080], '3:4': [1080, 1440], '9:16': [1080, 1920] },
}

// ■■ media budgets ■■
//
// These live in `validate` rather than in the media declaration's
// `counts`/`rules` because both inputs the product counts are outside a
// MediaRule's reach: reference elements embedded in the PROMPT eat the image
// and total budgets, and the video-duration rules are switched off in
// `video_edit` mode (the attached video is the subject, not a reference).

export type Seedance25MediaLimitCode = 'seedance-2-5-media-limit' | 'seedance-2-5-image-limit' | 'seedance-2-5-video-limit' | 'seedance-2-5-audio-limit'
export type Seedance25MediaDurationCode = 'seedance-2-5-audio-duration' | 'seedance-2-5-video-duration' | 'seedance-2-5-video-duration-budget' | 'seedance-2-5-audio-duration-budget'
export type Seedance25ValidationCode = Seedance25MediaLimitCode | Seedance25MediaDurationCode

export interface Seedance25MediaCounts {
  imageCount: number
  videoCount: number
  audioCount: number
  totalCount: number
  referenceElementCount?: number
}

/**
 * Pydantic-shaped issue with the product's stable `code` alongside — surfaces
 * key on the code to render a targeted message; `checkMedia`/`validate`
 * consumers that only read `loc`/`msg` are unaffected.
 */
export interface Seedance25ValidationIssue extends MediaIssue {
  code: Seedance25ValidationCode
}

export function getSeedance25MediaLimitIssues({
  imageCount,
  videoCount,
  audioCount,
  totalCount,
  referenceElementCount = 0,
}: Seedance25MediaCounts): Seedance25ValidationIssue[] {
  const issues: Seedance25ValidationIssue[] = []
  if (imageCount + referenceElementCount > SEEDANCE25_MAX_IMAGES)
    issues.push({ code: 'seedance-2-5-image-limit', loc: ['media', 'image'], msg: `You can add a maximum of ${SEEDANCE25_MAX_IMAGES} images` })
  if (videoCount > SEEDANCE25_MAX_VIDEOS)
    issues.push({ code: 'seedance-2-5-video-limit', loc: ['media', 'video'], msg: `You can add a maximum of ${SEEDANCE25_MAX_VIDEOS} videos` })
  if (audioCount > SEEDANCE25_MAX_AUDIOS)
    issues.push({ code: 'seedance-2-5-audio-limit', loc: ['media', 'audio'], msg: `You can add a maximum of ${SEEDANCE25_MAX_AUDIOS} audios` })
```

```
  if (totalCount + referenceElementCount > SEEDANCE25_MAX_INPUT_MEDIA)
    issues.push({ code: 'seedance-2-5-media-limit', loc: ['media'], msg: `You can add a maximum of ${SEEDANCE25_MAX_INPUT_MEDIA} input media items, including Elements` })
  return issues
}

export interface Seedance25DurationRef {
  durationSec?: number | null
}

/**
 * Per-ref and combined duration bounds. A ref whose duration is unknown is
 * skipped — meta is local knowledge (populate it from app data or
 * `resolveMediaMeta`), and the backend re-validates regardless.
 */
export function getSeedance25MediaDurationIssues(
  model: Model,
  videos: readonly Seedance25DurationRef[],
  audios: readonly Seedance25DurationRef[],
): Seedance25ValidationIssue[] {
  const issues: Seedance25ValidationIssue[] = []
  for (const audio of audios) {
    const duration = audio.durationSec
    if (duration != null && duration > 0 && (duration < SEEDANCE25_MIN_MEDIA_DURATION || duration > SEEDANCE25_MAX_AUDIO_DURATION))
      issues.push({ code: 'seedance-2-5-audio-duration', loc: ['media', 'audio'], msg: `Each reference audio file must be at least ${SEEDANCE25_MIN_MEDIA_DURATION} seconds and no more than ${SEEDANCE2
  }

  // In video_edit the attached video IS the subject, priced on its own
  // duration — the reference-video budget does not apply to it.
  if (model !== Seedance25Model.videoEdit) {
    for (const video of videos) {
      const duration = video.durationSec
      if (duration != null && duration > 0 && (duration < SEEDANCE25_MIN_MEDIA_DURATION || duration > SEEDANCE25_MAX_TOTAL_VIDEO_DURATION))
        issues.push({ code: 'seedance-2-5-video-duration', loc: ['media', 'video'], msg: `Each reference video must be at least ${SEEDANCE25_MIN_MEDIA_DURATION} seconds and no more than ${SEEDANCE25_M
    }

    const totalVideoDuration = videos.reduce((total, ref) => total + (ref.durationSec ?? 0), 0)
    if (totalVideoDuration > SEEDANCE25_MAX_TOTAL_VIDEO_DURATION)
      issues.push({ code: 'seedance-2-5-video-duration-budget', loc: ['media', 'video'], msg: `The total video duration limit is ${SEEDANCE25_MAX_TOTAL_VIDEO_DURATION} seconds` })
  }

  const totalAudioDuration = audios.reduce((total, ref) => total + (ref.durationSec ?? 0), 0)
  if (totalAudioDuration > SEEDANCE25_MAX_TOTAL_AUDIO_DURATION)
    issues.push({ code: 'seedance-2-5-audio-duration-budget', loc: ['media', 'audio'], msg: `The total audio duration limit is ${SEEDANCE25_MAX_TOTAL_AUDIO_DURATION} seconds` })
  return issues
}

function durationRefs(media: MediaInput | undefined, role: string): Seedance25DurationRef[] {
  return refsFor(media, role).map(ref => ({ durationSec: ref.meta?.durationSec }))
}

export const seedance2_5 = defineJob({
  jobSetType: 'seedance_2_5',
  outputType: 'video',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: MEDIA_ROLES,
      // Budgets are prompt-aware and mode-aware — see the note above `validate`.
    },
    settings: {
      duration: z._default(z.duration({ min: 4, max: 30 }), 5),
      resolution: z._default(z.enum(RESOLUTIONS), '720p'),
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(ASPECT_RATIOS), '16:9')),
      generateAudio: z.wire('generate_audio', z._default(z.boolean(), true)),
      // The product store's default ('high') wins over the backend schema
      // default ('standard') — every product surface sends it explicitly.
      bitrateMode: z.wire('bitrate_mode', z._default(z.enum(BITRATE_MODES), 'high')),
      // Per-model job param. Orthogonal to the client-side `count` fan-out:
```

```
    // total outputs = count × batchSize.
    batchSize: z.wire('batch_size', z._default(z.number(), 1)),
    model: z._default(z.enum(MODELS), Seedance25Model.default),
    // Required for (and only meaningful in) video_extension.
    extensionMode: z.wire('extension_mode', z.optional(z.enum(EXTENSION_MODES))),
  },
},
validate: ({ prompt, media, settings }) => {
  const model = (settings.model ?? Seedance25Model.default) as Model
  const videos = refsFor(media, 'video')
  const images = IMAGE_ROLES.flatMap(role => refsFor(media, role))
  const audioCount = refsFor(media, 'audio').length
  const mediaCount = MEDIA_ROLES.reduce((count, role) => count + refsFor(media, role).length, 0)
  const referenceElementCount = extractReferenceElementIds(prompt?.instruction ?? '').length

  return [
    ...promptRequired(prompt),
    ...promptMax(prompt, SEEDANCE_2_5_MAX_PROMPT_LENGTH),
    ...oneOf('resolution', settings.resolution, RESOLUTIONS),
    ...oneOf('bitrateMode', settings.bitrateMode, BITRATE_MODES),
    ...oneOf('model', model, MODELS),
    // The edit modes render into the source video's box; only `default`
    // takes a ratio from the caller.
    ...(model === Seedance25Model.default
      ? oneOf('aspectRatio', settings.aspectRatio, ASPECT_RATIOS)
      : []),
    // video_edit takes its duration from the input video, so nothing to check.
    ...(model === Seedance25Model.videoEdit
      ? []
      : model === Seedance25Model.videoExtension
        && typeof settings.duration === 'number'
        && settings.duration > 30
        ? [{ loc: ['settings', 'duration'], msg: 'Seedance 2.5 video extension supports at most 30 seconds' }]
        : integerRange('duration', settings.duration, 4, 30)),
    ...integerRange('batchSize', settings.batchSize, 1, 4),
    // Exactly one VIDEO — the subject of the edit. Images, audio and
    // reference elements are ordinary references here, so the total media
    // count is deliberately not constrained: requiring one media in total
    // rejected every edit that carried any reference at all.
    ...(model === Seedance25Model.videoEdit && videos.length !== 1
      ? [{ loc: ['media', 'video'], msg: 'Seedance 2.5 video edit requires exactly one video reference' }]
      : []),
    ...(model === Seedance25Model.videoExtension && videos.length === 0
      ? [{ loc: ['media', 'video'], msg: 'Seedance 2.5 video extension requires a video reference' }]
      : []),
    ...(model === Seedance25Model.videoExtension && !EXTENSION_MODES.includes(settings.extensionMode as ExtensionMode)
      ? [{ loc: ['settings', 'extensionMode'], msg: 'extensionMode is required for video extension' }]
      : []),
    ...getSeedance25MediaLimitIssues({
      imageCount: images.length,
      videoCount: videos.length,
      audioCount,
      totalCount: mediaCount,
      referenceElementCount,
    }),
    ...getSeedance25MediaDurationIssues(model, durationRefs(media, 'video'), durationRefs(media, 'audio')),
  ]
},
finalize: (wire) => {
  const resolution = wire.resolution as Resolution
  const aspectRatio = wire.aspect_ratio as AspectRatio
  // Dims always come fresh from the table — a parsed input whose resolution
  // or ratio changed re-derives instead of resubmitting stale extra dims.
  const { width, height } = lookupSize(SIZE_MAP, resolution, aspectRatio)
  const model = wire.model as Model

  const common: Record<string, unknown> = {
    ...wire,
    prompt: wire.prompt ?? '',
    width,
```

```
      height,
    }
    // The three shapes: video_edit inherits the source clip's length and box,
    // video_extension sets its own length but still inherits the box.
    delete common.aspect_ratio
    delete common.duration
    delete common.extension_mode

    if (model === Seedance25Model.videoEdit)
      return common

    if (model === Seedance25Model.videoExtension)
      return { ...common, duration: wire.duration, extension_mode: wire.extension_mode }

    return { ...common, duration: wire.duration, aspect_ratio: aspectRatio }
  },
  restore: (wire) => {
    // `finalize` drops `aspect_ratio` in both edit modes, so a fetched edit
    // would re-derive its render box from the '16:9' schema default. The
    // stored width/height encode what the box actually was — map them back.
    if (typeof wire.aspect_ratio === 'string')
      return {}
    const { width, height } = wire
    if (typeof width !== 'number' || typeof height !== 'number' || height <= 0)
      return {}
    return { aspect_ratio: closestRatioBySize(ASPECT_RATIOS, { width, height }) }
  },
})
```

app/packages/fnf/src/jobs/seedream-v4-5.ts

```
import { defineJob } from '../define-job'
import { z } from '../z'
import { intRange, oneOf, promptMax, promptRequired, randomSeed } from './checks'
import { firstSizeMeta } from './dimensions'

export const SeedreamV4_5AspectRatio = {
  r1x1: '1:1',
  r4x3: '4:3',
  r16x9: '16:9',
  r3x2: '3:2',
  r21x9: '21:9',
  r3x4: '3:4',
  r9x16: '9:16',
  r2x3: '2:3',
} as const

type Quality = 'basic' | 'high'

const MAX_PROMPT_CHARACTERS = 3_000

/**
 * Seedream 4.5. Grounded in fnf-web's `job-image-seedream-v4-5` module.
 * The app derives width/height from the first input image when present,
 * otherwise it submits a 1024x1024 default.
 */
export const seedreamV4_5 = defineJob({
  jobSetType: 'seedream_v4_5',
  outputType: 'image',
  params: {
    prompt: true,
    media: {
      field: 'input_images',
      format: 'unwrapped',
      roles: ['image'],
      counts: { image: { max: 14 } },
    },
    settings: {
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(Object.values(SeedreamV4_5AspectRatio)), '3:4')),
      quality: z._default(z.enum(['basic', 'high']), 'basic'),
      batchSize: z.wire('batch_size', z._default(z.number(), 1)),
      // The backend requires params.seed (422 'Field required' without it) —
      // default a random one like the product form does.
      seed: z._default(z.number(), randomSeed),
    },
  },
  credits: ({ settings }) => settings.batchSize ?? 1,
  validate: ({ prompt, settings }) => [
    ...promptRequired(prompt),
    ...promptMax(prompt, MAX_PROMPT_CHARACTERS),
    ...intRange('batchSize', settings.batchSize, 1, 4),
    ...intRange('seed', settings.seed, 1, 1_000_000),
    ...oneOf('aspectRatio', settings.aspectRatio, Object.values(SeedreamV4_5AspectRatio)),
  ],
  finalize: (wire, input) => {
    const size = firstSizeMeta(input.media, ['image']) ?? { width: 1024, height: 1024 }
    return {
      ...wire,
      model: 'seedream_v4_5',
      width: size.width,
      height: size.height,
      input_images: wire.input_images ?? [],
    }
  },
})

export type SeedreamV4_5Quality = Quality
```

app/packages/fnf/src/jobs/soul-v2.ts

```
import { defineJob } from '../define-job'
import { dimensionsWithin } from '../groups/media'
import { z } from '../z'
import { intRange, oneOf, randomSeed } from './checks'
import { lookupSize } from './dimensions'

export const DEFAULT_SOUL_V2_STYLE_ID = '3db34ab5-3439-4317-9e03-08dc30852e69'
export const DEFAULT_SOUL_CINEMA_STYLE_ID = '5fbabfac-d27b-4751-b550-fea356ed55ac'
export const DEFAULT_SOUL_CINEMA_OLZHAS_STYLE_ID = '08307202-efcd-4ac3-b0d5-6f29c63d0a1b'

export const SoulV2AspectRatio = {
  r9x16: '9:16',
  r3x4: '3:4',
  r2x3: '2:3',
  r1x1: '1:1',
  r4x3: '4:3',
  r16x9: '16:9',
  r3x2: '3:2',
  r21x9: '21:9',
} as const

type SoulV2AspectRatioValue = typeof SoulV2AspectRatio[keyof typeof SoulV2AspectRatio]
type Quality = '720p' | '1080p'

const SIZE_MAP: Record<Quality, Record<SoulV2AspectRatioValue, [number, number]>> = {
  '1080p': { '9:16': [1152, 2048], '16:9': [2048, 1152], '4:3': [2048, 1536], '3:4': [1536, 2048], '1:1': [2048, 2048], '2:3': [1344, 2016], '3:2': [2016, 1344], '21:9': [2528, 1088] },
  '720p': { '9:16': [960, 1696], '1:1': [1536, 1536], '4:3': [1536, 1152], '16:9': [1696, 960], '3:4': [1152, 1536], '2:3': [1088, 1632], '3:2': [1632, 1088], '21:9': [1680, 720] },
}

const DENOISE_DEFAULT = 0.83

function getSoulV2Dimensions(quality: Quality, aspectRatio: SoulV2AspectRatioValue) {
  return lookupSize(SIZE_MAP, quality, aspectRatio)
}

// Public settings carry ONLY user generation input. The product's internal
// pipeline constants (use_refiner/use_green/lora/chain_enhancer/model_version)
// are injected in `finalize` below with the exact values the product always
// sends; surface/billing markers (use_unlim, is_custom, full_name,
// is_marketing_studio_avatar, use_noise, use_green_aidar) are not part of the
// SDK surface — deliberate raw wire fields belong in `extra`.
function soulV2BaseSettings(defaultStyleId: string, defaultAspectRatio: SoulV2AspectRatioValue, customReferenceStrength: number | null) {
  return {
    styleId: z.wire('style_id', z._default(z.string(), defaultStyleId)),
    styleStrength: z.wire('style_strength', z._default(z.number(), 1)),
    customReferenceId: z.wire('custom_reference_id', z.optional(z.string())),
    customReferenceStrength: z.wire('custom_reference_strength', z._default(z.nullable(z.number()), customReferenceStrength)),
    quality: z._default(z.enum(['720p', '1080p']), '1080p'),
    aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(Object.values(SoulV2AspectRatio)), defaultAspectRatio)),
    seed: z._default(z.number(), randomSeed),
    batchSize: z.wire('batch_size', z._default(z.number(), 1)),
    colorPresetId: z.wire('color_preset_id', z.optional(z.nullable(z.string()))),
  }
}

// The pipeline constants the product submits on every soul request — kept on
// the wire (NOT public settings) so the backend pipeline selection does not
// silently change. Values mirror fnf-web's soul submitters byte-for-byte.
const SOUL_PIPELINE_WIRE = {
  use_refiner: false,
  use_green: true,
  lora: null,
  chain_enhancer: null,
  model_version: 'fast',
} as const

function commonSoulV2Validate(settings: { batchSize?: number, seed?: number, aspectRatio?: string }) {
  return [
```

```
    ...intRange('batchSize', settings.batchSize, 1, 4),
    ...intRange('seed', settings.seed, 1, 1_000_000),
    ...oneOf('aspectRatio', settings.aspectRatio, Object.values(SoulV2AspectRatio)),
  ]
}

export const soulV2Image = defineJob({
  jobSetType: 'text2image_soul_v2',
  outputType: 'image',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['image'],
      counts: { image: { max: 1 } },
      rules: [dimensionsWithin(['image'], { minSide: 128 })],
    },
    settings: soulV2BaseSettings(DEFAULT_SOUL_V2_STYLE_ID, '3:4', 1),
  },
  credits: ({ settings }) => (settings.batchSize ?? 1) * 0.125,
  validate: ({ settings }) => commonSoulV2Validate(settings),
  finalize: (wire) => {
    const quality = (wire.quality ?? '1080p') as Quality
    const aspectRatio = (wire.aspect_ratio ?? '3:4') as SoulV2AspectRatioValue
    const { width, height } = getSoulV2Dimensions(quality, aspectRatio)
    return {
      ...SOUL_PIPELINE_WIRE,
      ...wire,
      model: 'soul_v2',
      prompt: wire.prompt ?? '',
      width,
      height,
      aspect_ratio: aspectRatio,
      medias: wire.medias ?? [],
      enhance_prompt: wire.enhance_prompt ?? false,
      negative_prompt: wire.negative_prompt ?? '',
    }
  },
})

export const soulCinemaImage = defineJob({
  jobSetType: 'soul_cinematic',
  outputType: 'image',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['image'],
      counts: { image: { max: 1 } },
      rules: [dimensionsWithin(['image'], { minSide: 128 })],
    },
    settings: {
      ...soulV2BaseSettings(DEFAULT_SOUL_CINEMA_STYLE_ID, '16:9', null),
      cinematicVariant: z._default(z.enum(['default', 'olzhas', 'sultan', 'aidar']), 'default'),
    },
  },
  credits: ({ settings }) => (settings.batchSize ?? 1) * 0.125,
  validate: ({ settings }) => commonSoulV2Validate(settings),
  finalize: (wire) => {
    const quality = (wire.quality ?? '1080p') as Quality
    const aspectRatio = (wire.aspect_ratio ?? '16:9') as SoulV2AspectRatioValue
    // The backend types custom_reference_strength as float (not nullable) —
    // omit it entirely when there is no custom reference.
    if (wire.custom_reference_strength == null)
      delete wire.custom_reference_strength
    const { width, height } = getSoulV2Dimensions(quality, aspectRatio)
    const batchSize = typeof wire.batch_size === 'number' ? wire.batch_size : 1
    const denoise = Array(batchSize).fill(DENOISE_DEFAULT)
```

```
    const cinematicVariant = wire.cinematicVariant
    const hasImageReference = Array.isArray(wire.medias) && wire.medias.length > 0
    const rest = { ...wire }
    delete rest.cinematicVariant

    return {
      ...SOUL_PIPELINE_WIRE,
      ...rest,
      model: 'soul_cinematic',
      prompt: hasImageReference ? '' : wire.prompt ?? '',
      width,
      height,
      aspect_ratio: aspectRatio,
      medias: wire.medias ?? [],
      enhance_prompt: wire.enhance_prompt ?? true,
      negative_prompt: wire.negative_prompt ?? '',
      model_version: 'fast',
      chain_enhancer: null,
      time_denoise_from: denoise,
      ...(cinematicVariant === 'olzhas' ? { use_olzhas: true } : {}),
      ...(cinematicVariant === 'sultan' ? { use_sultan: true } : {}),
      ...(cinematicVariant === 'aidar' ? { use_aidar: true } : {}),
    }
  },
})
```

app/packages/fnf/src/jobs/text2image-soul.ts

```
import { defineJob } from '../define-job'
import { z } from '../z'
import { intRange, randomSeed } from './checks'
import { closestRatioBySize } from './dimensions'

/**
 * The product's default Soul style (fnf-web defaultSoulStyleId, soul-values.tsx).
 * The app NEVER submits style_id null on this surface — it falls back to this id
 * even in image-reference mode (null style_id exists only in fashion-factory
 * flows, where fashion_factory_id is sent instead — via `extra` in the SDK).
 */
export const DEFAULT_SOUL_STYLE_ID = '464ea177-8d40-4940-8d9d-b438bab269c7'

type Quality = '720p' | '1080p'

/**
 * fnf-web SOUL_RESOLUTION_MAP (entities/job/image/config/soul-values.tsx):
 * quality × ratio → [width, height], all 32/64-aligned. Unknown ratios fall
 * back exactly like the product's getSoulResolutionFromAspectRatio.
 */
const SIZE_MAP: Record<Quality, Record<string, readonly [number, number]>> = {
  '1080p': { '9:16': [1152, 2048], '16:9': [2048, 1152], '4:3': [2048, 1536], '3:4': [1536, 2048], '1:1': [2048, 2048], '2:3': [1344, 2016], '3:2': [2016, 1344], '21:9': [2528, 1088] },
  '720p': { '9:16': [960, 1696], '1:1': [1536, 1536], '4:3': [1536, 1152], '16:9': [1696, 960], '3:4': [1152, 1536], '2:3': [1088, 1632], '3:2': [1632, 1088], '21:9': [1680, 720] },
}
const FALLBACK_SIZE: Record<Quality, readonly [number, number]> = {
  '1080p': [768, 1024],
  '720p': [1152, 1536],
}

/**
 * Soul text-to-image — the /ai/image product model. Grounded in fnf-web's
 * soul submit strategy (gen-panel-model-image-soul/submit-strategy.ts) and
 * getDefaultSoulExpandedForm: steps 50, sample_guide_scale 4, sample_shift
 * 3 (720p) / 4 (1080p), negative_prompt '' (via `prompt.negative`), and
 * width/height looked up in SOUL_RESOLUTION_MAP from quality × aspect ratio.
 * enhance_prompt mirrors resolveEnhancePrompt's production behavior: false
 * with an image reference, forced true for a non-default style, otherwise the
 * caller's choice (default true).
 */
export const textToImageSoul = defineJob({
  jobSetType: 'text2image_soul',
  outputType: 'image',
  params: {
    prompt: true,
    /** An uploaded `/media` input image to edit/transform. */
    media: { field: 'image_reference', format: 'single', roles: ['image_reference'] },
    settings: {
      /** The Soul style; the product always sends one (default style when unset). */
      styleId: z.wire('style_id', z._default(z.nullable(z.string()), DEFAULT_SOUL_STYLE_ID)),
      customReferenceId: z.wire('custom_reference_id', z.optional(z.nullable(z.string()))),
      customReferenceStrength: z.wire('custom_reference_strength', z.optional(z.nullable(z.number()))),
      steps: z._default(z.number(), 50),
      batchSize: z.wire('batch_size', z._default(z.number(), 1)),
      /** Always on the wire (prod sends `seed || randomSeed()`); randomized when omitted. */
      seed: z._default(z.number(), randomSeed),
      quality: z._default(z.enum(['720p', '1080p']), '1080p'),
      // The product soul form defaults to PORTRAIT (createSoulForm: '3:4').
      aspectRatio: z.wire('aspect_ratio', z._default(z.string(), '3:4')),
      /** Diffusion sampler shift; the product derives 3 (720p) / 4 (1080p) when unset. */
      sampleShift: z.wire('sample_shift', z.optional(z.number())),
      sampleGuideScale: z.wire('sample_guide_scale', z._default(z.number(), 4)),
    },
  },
  // Ranges from the SoulSubmitParams contract comments (steps 1–50, batch 1–4,
  // seed 1–1M). No prompt rules: prod's soul submit has none (style-only runs).
  validate: ({ settings }) => {
    const issues = [
      ...intRange('steps', settings.steps, 1, 50),
```

```
      ...intRange('batchSize', settings.batchSize, 1, 4),
      ...intRange('seed', settings.seed, 1, 1_000_000),
    ]
    // The backend 422s when style_id AND fashion_factory_id are both absent
    // ("'style_id' or 'fashion_factory_id' must be provided") — reachable only
    // by passing styleId: null explicitly, since the default is the product's
    // default style (a fashion-factory flow would ride fashion_factory_id via
    // `extra`).
    if (settings.styleId == null)
      issues.push({ loc: ['settings', 'styleId'], msg: 'styleId is required (or send fashion_factory_id via extra)' })
    return issues
  },
  finalize: (wire) => {
    const quality = (wire.quality ?? '1080p') as Quality
    const row = SIZE_MAP[quality]
    // The product UI can only emit the 8 table ratios; the SDK's schema is an
    // open string, so snap a parseable off-table ratio ('1920:1080') to the
    // closest table key instead of silently shipping the portrait fallback
    // for a landscape request. Unparseable input keeps the product behavior:
    // raw ratio on the wire + the DEFAULT_RESOLUTIONS fallback dims.
    let ratio = wire.aspect_ratio as string
    if (!row[ratio]) {
      const parsed = /^(\d+(?:\.\d+)?):(\d+(?:\.\d+)?)$/.exec(ratio?.trim() ?? '')
      if (parsed)
        ratio = closestRatioBySize(Object.keys(row), { width: Number(parsed[1]), height: Number(parsed[2]) })
    }
    const [width, height] = row[ratio] ?? FALLBACK_SIZE[quality]
    // resolveEnhancePrompt (production branch): image reference → false;
    // a non-default style forces true; otherwise the caller's choice (true
    // when unset, like the product form's `enhance ?? true`).
    const styleId = wire.style_id ?? DEFAULT_SOUL_STYLE_ID
    const enhance = wire.image_reference != null
      ? false
      : styleId !== DEFAULT_SOUL_STYLE_ID ? true : wire.enhance_prompt ?? true
    return {
      ...wire,
      prompt: wire.prompt ?? '', // SoulSubmitParams types prompt as required ('' for style-only runs)
      aspect_ratio: ratio,
      width,
      height,
      enhance_prompt: enhance,
      sample_shift: wire.sample_shift ?? (quality === '720p' ? 3 : 4),
      // The product sends '' here, which SUPPRESSES the backend's substantive
      // anti-artifact default (fnf-api consts/text2image_soul.py) — kept for
      // byte parity with the product, pending a product decision.
      negative_prompt: wire.negative_prompt ?? '',
    }
  },
})
```

app/packages/fnf/src/jobs/todo.txt

```
PRE-SUBMIT VALIDATION — TODO
============================
Bring fnf-web's client-side, pre-network generation validation into the SDK.
Grounded in the real fnf code (refs below) + a design pass; this file is the
spec for the work. Paths are relative to the fnf-web repo root unless marked SDK.

STATUS (2026-06-11): sections B and C are SHIPPED — defineJob has the
cross-field `validate` hook, media declarations carry counts/rules
(checkMedia + the cardinality/meta combinators in src/groups/media.ts), and
buildWireParams aggregates everything into one typed ValidationError before
any I/O. What remains from this spec is section A only: the injected
entitlement/plan policy (EntitlementPolicy, PreflightError) — deferred; the
backend is authoritative and rejects with the typed billing errors the
catalog already maps.


ERROR / VALIDATION CATEGORIES
=============================

A) ENTITLEMENT / PLAN GUARDS  ->  NOT in SDK core. Injected policy (caller-supplied).
   Reads user context the SDK structurally lacks: plan_type, cohort,
   workspace_type, upsell grants. Bypassed for workspace==='shared'. Gated by a
   `withGuards` flag in the submit path.
   - VideoModelGuardError       guard.ts:26-37 (assertSubscriptionPlan, videoModelPlans);
                                invoked: abstract.ts:115-119 (ensureSubmitRequest),
                                guard/video.ts:40-106 (useVideoModelGuard.mustPass/canPass),
                                module/job/job/model/use-submit-job-set-mutation.ts:104.
   - VideoUseCaseGuardError     guard.ts:54-62 (videoUseCasePlans: videoStartEndFrame,
                                mixedMedia, voiceElement, vibeChat, viral, veoInstructCanvas...);
                                submit/video.ts:48, mix.ts:48, veo.ts:37, sora2-video.ts:50,
                                minimax.ts:81; hooks guard/video.ts:108-176.
   - ChatModelGuardError        module/assist-chat/entities/chat/error/guard.ts:23-29;
                                hooks/useChatFetch.ts:16-22 (pre-fetch).
   - MobileGuardError           entities/image/model/use-image-page-submit.ts:21-26,116;
                                entities/job/model/plan/image-product-plans.ts:244-295.
   - UpscaleUseCaseGuardError   entities/upscale/error/guard.ts; entities/upscale/model/guard/upscale.ts:36,61.
   Context source: model/submit/types/user-payload.ts:3-14 (UserPayload),
                   SubscriptionWeights.isEqualOrHigherPlans subscription-weights.ts:20-77.
   On fail: throws a GuardError subclass carrying (model|useCase, current plan);
            UI listener shows an upgrade modal (dispatchVideoModelEvent, error-event-target.ts:76).

B) REQUIRED / CROSS-FIELD / CONDITIONAL PARAM RULES  ->  SDK defineJob `validate(input) -> Issue[]`.
   Live in each model's submit() body; ALWAYS run (not guard-gated).
   - required prompt (conditional):
       sora2 unless is_sketch         submit/sora2-video.ts:60-62 (+ sketch-fields.ts:3-6)
       wan2-5 derived mutual-excl.    submit/wan2-5-video.ts:54-63 (prompt optional if image+draw, or enhance+image)
       veo3-1 mode + media branches   submit/veo3-1-video.ts:66-85
   - prompt length (min/max):
       veo3-1  3..4000 chars          submit/veo3-1-video.ts:54-64
       kling   < 2000 when present    submit/kling.ts:54-56
   - required media:
       veo  start frame always        submit/veo.ts:46-48 (ToastError 'Start frame required')
       minimax  start-OR-end          submit/minimax.ts:57-74 (skip for minimax-2.3)
       veo3-1  end requires start     submit/veo3-1-video.ts:68-72
   - cross-field required:
       aspect_ratio required w/o image  submit/wan2-5-video.ts:65-67, sora2-video.ts:64-66
   - cross-field lock (coerce in UI; SDK = reject or opt-in coerce):
       aspectRatio -> 'auto' when start/end frame present
       module/.../gen-panel-model-video-seedance-2-0/fields.ts:34-54
   On fail: throws ToastError / ToastMinimaxError (minimax.ts:49,53,64) /
            ToastInfiniteTalkError (infinite-talk.ts:41) — first-throw-wins.

C) MEDIA ROLE / COUNT  ->  SDK declarative MediaConfig.requiredRoles + counts:{role:{min,max}};
                           cross-role rules go in the validate hook (B).
   - max images per add        entities/job/model/.../api-submit-model-media-helpers.ts:122-124,193-195
   - "audio requires image|video" (cross-role)   seedance 2.0 form (video-form-seedance-2-0.tsx)
   - per-image size / aspect (ASYNC, network getImageSize -> keep in media pipeline, not job validate):
       minimax >=300px, ratio 2:5..5:2   submit/minimax.ts:44-55; is-aspect-ratio-in-range.ts:1-8
```

```
   - audio format WAV-only       submit/infinite-talk.ts:36-46  (candidate: MediaConfig accept? OR upload policy)
   - upload-time policy (size/ratio/dims/contentType)  entities/input-media/model/input-media-policy.ts

D) DERIVED / COERCED PARAMS  ->  NOT errors. fnf silently coerces; SDK keeps normalize() or opt-in coerce.
   - model normalization        submit/veo.ts:51-53, minimax.ts:99-100
   - end-frame nullification     submit/kling.ts:84-92 (isEndFrameWorks, end-frame-policy.ts:18-36), minimax.ts:98
   - enhance_prompt derivation   submit/veo.ts:56-61, kling.ts:69-75

E) CONTENT / NSFW  ->  media-upload pipeline (already in SDK media half), NOT job submit.
   - image NSFW (async, forceNsfwCheck on upload)  features/video/ui/video-form.tsx:847-851;
       nsfwFlags disable submit  video-form.tsx:1046; video-form-nsfw-store.ts:5-11
   - IP / ownership check         video-form.tsx:805-845 (uploadImageWithIpCheckMutation)
   - prompt moderation            SERVER-SIDE only (no client check). prompt_nsfw -> PromptNsfwError via errorFromResponse.

F) FORM-LEVEL SUBMIT GATING (/ai/video)  ->  caller/UI concern, informs (A)+(B) above.
   - Generate button disabled ONLY on NSFW-in-image or in-flight
       features/video/ui/video-form.tsx:1046; entities/.../job-generate-btn.tsx:156
   - everything else aborts-with-toast on click   handleSubmit video-form.tsx:887-933
   - enum values constrained by option lists (AspectRatio/Duration/ResolutionSelect)
   - aspect/resolution -> width/height mappers toast on invalid combo
       forms/video-form-seedance-1-5.tsx:152-155, seedance-2-0.tsx:704-707, grok-video.tsx:127-128
   - guest validation   entities/guest/model/guest-form-validation.ts:120-242


SDK IMPLEMENTATION PLAN
=======================
SDK files relative to packages/fnf/src/.

1. ERROR MODEL (errors.ts)
   - NEW: class PreflightError extends ApiJobError<{ issues: Issue[] }>
       code 'preflight', status 422, data:{issues}. Aggregates ALL issues (vs fnf first-throw).
       Register in CODE_TO_CLASS + export from index.ts + ./errors subpath.
   - Issue = { path?: string; code: 'required'|'too_long'|'too_short'|'conflict'|'too_few'|'too_many'|'invalid'; message: string }
   - Entitlement verdicts REUSE MinimumPlanError / RateLimitError / OutOfCreditsError (no new class).
   - ValidationError (existing) stays the BACKEND-422 mapper; PreflightError is the LOCAL one.

2. defineJob (define-job.ts)
   - DefineJobConfig + JobEntry gain:  validate?: (input: GenerationInput) => Issue[] | void   (SYNC, returns issues)
   - MediaConfigFor<Roles> + MediaConfig gain (optional, additive):
       requiredRoles?: readonly Roles[]
       counts?: Partial<Record<Roles, { min?: number; max?: number }>>

3. media (groups/media.ts)
   - NEW pure fn checkMedia(cfg, media) -> Issue[] (requiredRoles + counts). Codec stays a dumb mapper.

4. spec.ts
   - parseSettings(input, entry) already exists and already wraps the zod throw
     into ValidationError. DECIDE: does runPreflight re-wrap/aggregate that into
     PreflightError, or does a local parse failure stay a ValidationError?
     (todo item 1 says ValidationError = the BACKEND-422 mapper — resolve the tension.)

5. client/preflight.ts (NEW)
   - runPreflight(ctx, entry, input): parseSettings -> checkMedia -> entry.validate
       (aggregate -> throw one PreflightError) -> await ctx.entitlements?.(...) -> throw verdict.error.

6. client/submit.ts
   - Insert `await runPreflight(ctx, entry, input)` between entryFor and buildWireParams,
     BEFORE the count fan-out (all-or-nothing; never a partial batch).
   - safeSubmit UNCHANGED (PreflightError is a ApiJobError, maps via toJSON).

7. ClientConfig + GenerationContext (client/types.ts) + createContext (client/context.ts)
   - entitlements?: EntitlementPolicy   (optional; omitted => no-op)
       EntitlementContext = { model, jobSetType, settings, media?, count }
       EntitlementVerdict  = { ok:true } | { ok:false, error: ApiJobError }
       EntitlementPolicy   = (ctx) => Verdict | Promise<Verdict>   (ASYNC-capable)
   - Caller (fnf-web) closes over UserPayload + SubscriptionWeights + upsell; returns MinimumPlanError etc.

8. estimateCost (client/cost.ts)
   - Run the STRUCTURAL validation (runPreflight w/o entitlements); skip plan-gating for cost previews.
```

9. Tests
   - required/conditional/cross-field via validate; media counts/required; PreflightError aggregation +
     toJSON/errorFromJSON round-trip; entitlement policy {ok:false} -> error surfaces via safeSubmit;
     entitlement omitted => no-op.

OPEN QUESTIONS
==============
- validate hook is SYNC. fnf's per-image dimension/aspect check is async (network getImageSize,
  minimax.ts:44-55) -> keep in media-upload pipeline (dims known at confirm), NOT job validate. Confirm.
- Cross-field lock (aspectRatio->auto, fields.ts:34-54): SDK headless can REJECT (PreflightError) or
  SILENTLY COERCE (like normalize). Add opt-in coerce mode for MCP/agent callers?
- WAV-only (infinite-talk.ts:36-46): MediaConfig.accept? here, or upload policy (where the real file is)?
  Leaning upload policy.
- New 422 code 'preflight' alongside 'validation' — OK for existing error consumers? (recommend: yes, distinct.)

PROVENANCE
==========
Grounded via workflow analysis of fnf guard/submit/ai-video layers + SDK surface; design
adversarially verified (architecture sound; entitlement-needs-user-context confirmed).
Decisive boundary: A) entitlement = injected caller policy; B)+C) structural = declarative in SDK.

app/packages/fnf/src/jobs/topaz-image-upscale.ts

```
import type { MediaIssue } from '../groups/media'
import type { GenerationInput } from '../types'
import { defineJob } from '../define-job'
import { z } from '../z'
import { intRange, oneOf } from './checks'
import {
  imageFactorIssues,
  range,
  requirePositiveSize,
  scaleImageSize,
  stripUndefined,
  TOPAZ_IMAGE_MODELS,
  TopazImageModel,
  type TopazImageModelValue,
  UPSCALE_IMAGE_FACTORS,
  UpscaleImageFactor,
  type UpscaleImageFactorValue,
} from './upscale-helpers'
import { firstMetaSize } from './video-helpers'

function sourceSize(input: GenerationInput): { width: number, height: number } | undefined {
  const meta = firstMetaSize(input.media, ['image'])
  if (meta)
    return meta
  const settings = input.settings as { sourceWidth?: number, sourceHeight?: number }
  return settings.sourceWidth != null && settings.sourceHeight != null
    ? { width: settings.sourceWidth, height: settings.sourceHeight }
    : undefined
}

function finalizeTopazImage(wire: Record<string, unknown>, input: GenerationInput, model: TopazImageModelValue | 'Redefine'): Record<string, unknown> {
  const size = sourceSize(input)
  const factor = (wire.factor ?? 'x1') as UpscaleImageFactorValue
  const output = wire.output_width != null && wire.output_height != null
    ? { width: wire.output_width as number, height: wire.output_height as number }
    : size
      ? scaleImageSize(size, factor)
      : undefined
  const rest = { ...wire }
  delete rest.factor
  return stripUndefined({
    ...rest,
    model,
    width: size?.width,
    height: size?.height,
    output_width: output?.width,
    output_height: output?.height,
  })
}

function baseValidate(input: GenerationInput): MediaIssue[] {
  const settings = input.settings as {
    sourceWidth?: number
    sourceHeight?: number
    outputWidth?: number
    outputHeight?: number
    factor?: string
    denoise?: number
    sharpen?: number
    faceEnhancementCreativity?: number
    faceEnhancementStrength?: number
  }
  const size = sourceSize(input)
  return [
    ...requirePositiveSize('sourceSize', size),
    ...oneOf('factor', settings.factor, UPSCALE_IMAGE_FACTORS),
    ...imageFactorIssues(input.media, settings.factor),
    ...range('sourceWidth', settings.sourceWidth, 1, Number.MAX_SAFE_INTEGER),
```

```
    ...range('sourceHeight', settings.sourceHeight, 1, Number.MAX_SAFE_INTEGER),
    ...range('outputWidth', settings.outputWidth, 1, Number.MAX_SAFE_INTEGER),
    ...range('outputHeight', settings.outputHeight, 1, Number.MAX_SAFE_INTEGER),
    ...range('denoise', settings.denoise, 0, 1),
    ...range('sharpen', settings.sharpen, 0, 1),
    ...range('faceEnhancementCreativity', settings.faceEnhancementCreativity, 0, 1),
    ...range('faceEnhancementStrength', settings.faceEnhancementStrength, 0, 1),
  ]
}

const baseSettings = {
  sourceWidth: z.wire('width', z.optional(z.number())),
  sourceHeight: z.wire('height', z.optional(z.number())),
  outputWidth: z.wire('output_width', z.optional(z.number())),
  outputHeight: z.wire('output_height', z.optional(z.number())),
  factor: z._default(z.enum(UPSCALE_IMAGE_FACTORS), UpscaleImageFactor.x1),
  denoise: z._default(z.number(), 0.2),
  sharpen: z._default(z.number(), 0.3),
  faceEnhancement: z.wire('face_enhancement', z._default(z.boolean(), false)),
  faceEnhancementCreativity: z.wire('face_enhancement_creativity', z._default(z.number(), 0)),
  faceEnhancementStrength: z.wire('face_enhancement_strength', z._default(z.number(), 0.8)),
}

export { TopazImageModel, UpscaleImageFactor as TopazImageUpscaleFactor }

export const topazImageUpscale = defineJob({
  jobSetType: 'topaz_image',
  outputType: 'image',
  params: {
    media: {
      field: 'input_image',
      format: 'single',
      roles: ['image'],
      counts: { image: { min: 1, max: 1 } },
    },
    settings: {
      ...baseSettings,
      model: z._default(z.enum(TOPAZ_IMAGE_MODELS), TopazImageModel.standardV2),
    },
  },
  validate: input => [
    ...baseValidate(input),
    ...oneOf('model', input.settings.model, TOPAZ_IMAGE_MODELS),
  ],
  finalize: (wire, input) => finalizeTopazImage(wire, input, (wire.model ?? TopazImageModel.standardV2) as TopazImageModelValue),
})

export const topazImageGenerativeUpscale = defineJob({
  jobSetType: 'topaz_image_generative',
  outputType: 'image',
  params: {
    prompt: true,
    media: {
      field: 'input_image',
      format: 'single',
      roles: ['image'],
      counts: { image: { min: 1, max: 1 } },
    },
    settings: {
      ...baseSettings,
      detail: z._default(z.number(), 1),
      texture: z._default(z.number(), 1),
      creativity: z._default(z.number(), 1),
      autoprompt: z._default(z.boolean(), true),
    },
  },
  validate: input => [
    ...baseValidate(input),
    ...range('detail', input.settings.detail, 0, 1),
    ...intRange('texture', input.settings.texture, 1, 5),
```

```
    ...intRange('creativity', input.settings.creativity, 1, 6),
  ],
  finalize: (wire, input) => finalizeTopazImage(wire, input, 'Redefine'),
})
```

app/packages/fnf/src/jobs/upscale-helpers.ts

```
import type { MediaIssue } from '../groups/media'
import type { GenerationInput, MediaInput } from '../types'
import { firstMetaSize, refsFor, type Size } from './video-helpers'

export const UpscaleImageFactor = {
  x1: 'x1',
  x2: 'x2',
  x4: 'x4',
  x8: 'x8',
  x16: 'x16',
} as const

export const UPSCALE_IMAGE_FACTORS = Object.values(UpscaleImageFactor)
export type UpscaleImageFactorValue = typeof UPSCALE_IMAGE_FACTORS[number]

export const TopazImageModel = {
  standardV2: 'Standard V2',
  lowResolutionV2: 'Low Resolution V2',
  cgi: 'CGI',
  highFidelityV2: 'High Fidelity V2',
  textRefine: 'Text Refine',
} as const

export const TOPAZ_IMAGE_MODELS = Object.values(TopazImageModel)
export type TopazImageModelValue = typeof TOPAZ_IMAGE_MODELS[number]

type UpscaleQuality = 'largest' | '8K' | '4K' | '2K' | '1080p' | '720p' | '500p'

const SCALES_BY_IMAGE_QUALITY: Record<UpscaleQuality, UpscaleImageFactorValue[]> = {
  'largest': [],
  '8K': ['x1'],
  '4K': ['x1', 'x2'],
  '2K': ['x1', 'x2', 'x4'],
  '1080p': ['x1', 'x2', 'x4', 'x8'],
  '720p': ['x1', 'x2', 'x4', 'x8'],
  '500p': ['x1', 'x2', 'x4', 'x8', 'x16'],
}

export function imageQuality(size: Size): UpscaleQuality {
  const longest = Math.max(size.width, size.height)
  if (longest >= 9000)
    return 'largest'
  if (longest >= 7680)
    return '8K'
  if (longest >= 3840)
    return '4K'
  if (longest >= 2048)
    return '2K'
  if (longest >= 1920)
    return '1080p'
  if (longest >= 1280)
    return '720p'
  return '500p'
}

export function availableImageFactors(size: Size): UpscaleImageFactorValue[] {
  return SCALES_BY_IMAGE_QUALITY[imageQuality(size)]
}

export function scaleImageSize(size: Size, factor: UpscaleImageFactorValue): Size {
  const n = Number(factor.replace('x', ''))
  return { width: size.width * n, height: size.height * n }
}

export function firstMediaSizeOrSettings(input: GenerationInput, roles: readonly string[], wire: Record<string, unknown>): Size | undefined {
  const meta = firstMetaSize(input.media, roles)
  if (meta)
    return meta
```

```
  const width = typeof wire.width === 'number' ? wire.width : undefined
  const height = typeof wire.height === 'number' ? wire.height : undefined
  return width != null && height != null ? { width, height } : undefined
}

export function requirePositiveSize(field: string, size: Size | undefined): MediaIssue[] {
  if (size && size.width > 0 && size.height > 0)
    return []
  return [{ loc: ['settings', field], msg: `${field} requires positive width and height` }]
}

export function range(field: string, value: number | null | undefined, min: number, max: number): MediaIssue[] {
  if (value == null || (value >= min && value <= max))
    return []
  return [{ loc: ['settings', field], msg: `${field} must be between ${min} and ${max}` }]
}

export function oneOfString(field: string, value: unknown, options: readonly string[]): MediaIssue[] {
  if (typeof value !== 'string' || options.includes(value))
    return []
  return [{ loc: ['settings', field], msg: `${field} must be one of: ${options.join(', ')}` }]
}

export function imageFactorIssues(media: MediaInput | undefined, factor: string | undefined): MediaIssue[] {
  const size = firstMetaSize(media, ['image'])
  if (!size || !factor)
    return []
  const available = availableImageFactors(size)
  return available.includes(factor as UpscaleImageFactorValue)
    ? []
    : [{ loc: ['settings', 'factor'], msg: `factor must be one of: ${available.join(', ') || 'none'} for this image size` }]
}

export const VideoScaleFactor = {
  original: 'Original',
  fullHd: 'FULL_HD',
  r2k: '2k',
  r4k: '4k',
} as const

export const VIDEO_SCALE_FACTORS = Object.values(VideoScaleFactor)
export type VideoScaleFactorValue = typeof VIDEO_SCALE_FACTORS[number]

const VIDEO_RESOLUTIONS: Record<Exclude<VideoScaleFactorValue, 'Original'>, Size> = {
  'FULL_HD': { width: 1920, height: 1080 },
  '2k': { width: 2048, height: 1080 },
  '4k': { width: 3840, height: 2160 },
}

export function scaleVideoSize(factor: VideoScaleFactorValue, inputSize: Size): Size {
  if (factor === 'Original' || inputSize.width <= 0 || inputSize.height <= 0)
    return inputSize
  const target = VIDEO_RESOLUTIONS[factor]
  const targetLongestSide = Math.max(target.width, target.height)
  const aspect = inputSize.width / inputSize.height
  if (inputSize.width >= inputSize.height) {
    const width = targetLongestSide
    return { width, height: Math.round(width / aspect) }
  }
  const height = targetLongestSide
  return { width: Math.round(height * aspect), height }
}

export function topazFixedVideoSize(resolution: '1080p' | '2160p'): Size {
  return resolution === '2160p' ? { width: 3840, height: 2160 } : { width: 1920, height: 1080 }
}

export function firstDuration(media: MediaInput | undefined, roles: readonly string[]): number | undefined {
  for (const role of roles) {
    for (const ref of refsFor(media, role)) {
```

```
      const duration = ref.meta?.durationSec
      if (duration != null)
        return duration
    }
  }
  return undefined
}

export function localBytedanceVideoCredits(resolution: '1080p' | '2k' | '4k', duration: number | undefined, fps: number): number | null {
  if (duration == null)
    return null
  const rates = fps > 30
    ? { '1080p': 0.04, '2k': 0.08, '4k': 0.16 }
    : { '1080p': 0.02, '2k': 0.04, '4k': 0.08 }
  return Math.ceil(duration * rates[resolution])
}

export function stripUndefined<T extends Record<string, unknown>>(value: T): T {
  const out: Record<string, unknown> = {}
  for (const [key, item] of Object.entries(value)) {
    if (item !== undefined)
      out[key] = item
  }
  return out as T
}
```

app/packages/fnf/src/jobs/veo-3-1-lite.ts

```
import { defineJob } from '../define-job'
import { z } from '../z'
import { countRefs, oneOf, promptMax, promptRequired, randomSeed } from './checks'
import { lookupSize } from './dimensions'
import { firstMetaSize } from './video-helpers'

export const Veo31LiteAspectRatio = {
  auto: 'auto',
  r16x9: '16:9',
  r9x16: '9:16',
} as const

export const Veo31LiteResolution = {
  r720: '720p',
  r1080: '1080p',
} as const

const ASPECT_RATIOS = Object.values(Veo31LiteAspectRatio)
const RESOLUTIONS = Object.values(Veo31LiteResolution)

type ConcreteRatio = Exclude<typeof Veo31LiteAspectRatio[keyof typeof Veo31LiteAspectRatio], 'auto'>
type Resolution = typeof RESOLUTIONS[number]

const SIZE_MAP: Record<Resolution, Record<ConcreteRatio, [number, number]>> = {
  '720p': { '16:9': [1280, 720], '9:16': [720, 1280] },
  '1080p': { '16:9': [1920, 1080], '9:16': [1080, 1920] },
}
const AUTO_FALLBACK = { width: 1920, height: 1080 }

export const veo3_1Lite = defineJob({
  jobSetType: 'veo3_1_lite',
  outputType: 'video',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['start_image', 'end_image'],
      counts: { start_image: { max: 1 }, end_image: { max: 1 } },
    },
    settings: {
      duration: z._default(z.duration({ values: [4, 6, 8] }), 8),
      resolution: z._default(z.enum(RESOLUTIONS), '720p'),
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(ASPECT_RATIOS), 'auto')),
      generateAudio: z.wire('generate_audio', z._default(z.boolean(), true)),
      seed: z.optional(z.number()),
    },
  },
  credits: ({ settings }) => {
    const perSecond = settings.resolution === '1080p'
      ? (settings.generateAudio ? 2 : 1.5)
      : (settings.generateAudio ? 1.5 : 1)
    return Math.ceil((settings.duration ?? 8) * perSecond)
  },
  validate: ({ prompt, media, settings }) => {
    const issues = [
      ...promptRequired(prompt),
      ...promptMax(prompt, 4000),
      ...oneOf('duration', settings.duration, [4, 6, 8]),
      ...oneOf('resolution', settings.resolution, RESOLUTIONS),
      ...oneOf('aspectRatio', settings.aspectRatio, ASPECT_RATIOS),
    ]
    const hasFLF = countRefs(media, 'start_image') > 0 && countRefs(media, 'end_image') > 0
    if (hasFLF && settings.duration !== 8)
      issues.push({ loc: ['settings', 'duration'], msg: 'Duration must be 8 seconds when both first and last frames are provided' })
    if (settings.resolution === '1080p' && settings.duration !== 8)
      issues.push({ loc: ['settings', 'duration'], msg: 'Duration must be 8 seconds for 1080p resolution' })
    return issues
```

```
  },
  finalize: (wire, input) => {
    const measured = firstMetaSize(input.media, ['start_image'])
    const table = wire.aspect_ratio === 'auto'
      ? AUTO_FALLBACK
      : lookupSize(SIZE_MAP, wire.resolution as Resolution, wire.aspect_ratio as ConcreteRatio)
    return {
      ...wire,
      seed: wire.seed ?? randomSeed(),
      width: measured?.width ?? table.width,
      height: measured?.height ?? table.height,
      medias: wire.medias ?? [],
    }
  },
})
```

app/packages/fnf/src/jobs/video-helpers.ts

```
import type { MediaIssue } from '../groups/media'
import type { GenerationInput, MediaInput, MediaRef } from '../types'

export interface Size {
  width: number
  height: number
}

export function refsFor(media: MediaInput | undefined, role: string): MediaRef[] {
  const value = media?.[role]
  return Array.isArray(value) ? value : value ? [value] : []
}

export function firstMetaSize(media: MediaInput | undefined, roles: readonly string[]): Size | undefined {
  for (const role of roles) {
    for (const ref of refsFor(media, role)) {
      const { width, height } = ref.meta ?? {}
      if (width != null && height != null && height > 0)
        return { width, height }
    }
  }
  return undefined
}

export function firstMetaDuration(media: MediaInput | undefined, roles: readonly string[]): number | undefined {
  for (const role of roles) {
    for (const ref of refsFor(media, role)) {
      const seconds = ref.meta?.durationSec
      if (seconds != null)
        return seconds
    }
  }
  return undefined
}

export function integerRange(field: string, value: number | null | undefined, min: number, max: number): MediaIssue[] {
  if (value == null)
    return []
  if (Number.isInteger(value) && value >= min && value <= max)
    return []
  return [{ loc: ['settings', field], msg: `${field} must be an integer between ${min} and ${max}` }]
}

export function requiredPromptOrRole(input: GenerationInput, role: string, message: string): MediaIssue[] {
  const prompt = (input.prompt?.instruction ?? '').trim()
  if (prompt.length > 0 || refsFor(input.media, role).length > 0)
    return []
  return [{ loc: ['prompt'], msg: message }]
}

export function extractAngleRefIds(text: string): string[] {
  const ids = new Set<string>()
  for (const match of text.matchAll(/<<<([^>]+)>>>/g)) {
    const value = match[1]?.trim()
    // `movement:<uuid>` are camera-movement tokens the backend expands itself —
    // not reference elements, so they must not eat a model's media budget
    // (parity with fnf-web's isInternalNonElementMention). `movement--` is the
    // pre-rename namespace still found in old prompt strings.
    if (!value || /^(image|video|audio|cast)_/.test(value) || value.startsWith('movement:') || value.startsWith('movement--'))
      continue
    if (value.startsWith('element_')) {
      const id = value.slice('element_'.length)
      if (id && !/^\d+$/.test(id))
        ids.add(id)
      continue
    }
    ids.add(value)
  }
```

```
  return [...ids]
}

/**
 * Every reference element a prompt mentions: the `<<<...>>>` angle form plus
 * the bare `@<uuid>` mention the product's prompt editor also emits. Models
 * that count elements against a media budget (seedance 2.5) must use this,
 * not `extractAngleRefIds` alone, or an @-mentioned element slips the cap.
 */
export function extractReferenceElementIds(text: string): string[] {
  const ids = new Set(extractAngleRefIds(text))
  for (const match of text.matchAll(/(^|[^\w-])@([0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12})(?=$|[^\w-])/gi))
    ids.add(match[2])
  return [...ids]
}

export function batch(settings: Record<string, unknown>, fallback = 1): number {
  return typeof settings.batchSize === 'number' ? settings.batchSize : fallback
}
```

app/packages/fnf/src/jobs/video-upscale.ts

```
import type { MediaIssue } from '../groups/media'
import type { GenerationInput } from '../types'
import { defineJob } from '../define-job'
import { z } from '../z'
import { intRange, oneOf } from './checks'
import {
  firstDuration,
  localBytedanceVideoCredits,
  range,
  requirePositiveSize,
  scaleVideoSize,
  stripUndefined,
  topazFixedVideoSize,
  VIDEO_SCALE_FACTORS,
  VideoScaleFactor,
  type VideoScaleFactorValue,
} from './upscale-helpers'
import { firstMetaSize, integerRange, type Size } from './video-helpers'

export const TopazVideoModel = {
  proteus: 'prob-4',
  starlightCreative: 'slc-1',
  starlightFast: 'slf-1',
  starlightPrecise: 'slp-2.5',
} as const

export const TopazVideoEnhancementModel = {
  proteus: 'prob-4',
  artemis: 'ahq-12',
  iris: 'iris-3',
  rhea: 'rhea-1',
  gaia: 'ghq-5',
  theia: 'thd-3',
  starlightCreative: 'slc-1',
  starlightFast: 'slf-1',
  starlightPrecise: 'slp-2.5',
} as const

export const TopazVideoFocusFix = {
  normal: 'Normal',
  strong: 'Strong',
} as const

export const TopazVideoParameters = {
  auto: 'auto',
  manual: 'manual',
} as const

export const TopazVideoResolution = {
  r1080: '1080p',
  r2160: '2160p',
} as const

export const BytedanceVideoUpscaleResolution = {
  r1080: '1080p',
  r2k: '2k',
  r4k: '4k',
} as const

export const BytedanceVideoUpscalePreset = {
  common: 'common',
  aigc: 'aigc',
  shortSeries: 'short_series',
  ugc: 'ugc',
  oldFilm: 'old_film',
} as const

const TOPAZ_MODELS = Object.values(TopazVideoModel)
```

```
const TOPAZ_ENHANCEMENT_MODELS = Object.values(TopazVideoEnhancementModel)
const FOCUS_FIXES = Object.values(TopazVideoFocusFix)
const PARAMETER_MODES = Object.values(TopazVideoParameters)
const TOPAZ_RESOLUTIONS = Object.values(TopazVideoResolution)
const BYTEDANCE_RESOLUTIONS = Object.values(BytedanceVideoUpscaleResolution)
const BYTEDANCE_PRESETS = Object.values(BytedanceVideoUpscalePreset)

type TopazModel = typeof TOPAZ_MODELS[number]
type EnhancementModel = typeof TOPAZ_ENHANCEMENT_MODELS[number]
type TopazResolution = typeof TOPAZ_RESOLUTIONS[number]

function videoSourceSize(input: GenerationInput, roles: readonly string[] = ['video']): Size | undefined {
  const meta = firstMetaSize(input.media, roles)
  if (meta)
    return meta
  const settings = input.settings as { sourceWidth?: number, sourceHeight?: number }
  return settings.sourceWidth != null && settings.sourceHeight != null
    ? { width: settings.sourceWidth, height: settings.sourceHeight }
    : undefined
}

function videoSourceIssues(input: GenerationInput): MediaIssue[] {
  const settings = input.settings as {
    sourceWidth?: number
    sourceHeight?: number
    outputWidth?: number
    outputHeight?: number
    scaleFactor?: string
  }
  return [
    ...requirePositiveSize('sourceSize', videoSourceSize(input)),
    ...range('sourceWidth', settings.sourceWidth, 1, Number.MAX_SAFE_INTEGER),
    ...range('sourceHeight', settings.sourceHeight, 1, Number.MAX_SAFE_INTEGER),
    ...range('outputWidth', settings.outputWidth, 1, Number.MAX_SAFE_INTEGER),
    ...range('outputHeight', settings.outputHeight, 1, Number.MAX_SAFE_INTEGER),
    ...oneOf('scaleFactor', settings.scaleFactor, VIDEO_SCALE_FACTORS),
  ]
}

function resolveVideoOutput(wire: Record<string, unknown>, source: Size): Size {
  if (wire.output_width != null && wire.output_height != null)
    return { width: wire.output_width as number, height: wire.output_height as number }
  const factor = wire.scaleFactor as VideoScaleFactorValue | undefined
  if (factor && factor !== VideoScaleFactor.original)
    return scaleVideoSize(factor, source)
  const resolution = wire.resolution as TopazResolution | undefined
  if (resolution)
    return topazFixedVideoSize(resolution)
  const model = wire.model as string | undefined
  if (model === 'slc-1' || model === 'slf-1' || model === 'slp-2.5')
    return topazFixedVideoSize('2160p')
  return source
}

function topazVideoSettings() {
  return {
    sourceWidth: z.wire('width', z.optional(z.number())),
    sourceHeight: z.wire('height', z.optional(z.number())),
    outputWidth: z.wire('output_width', z.optional(z.number())),
    outputHeight: z.wire('output_height', z.optional(z.number())),
    scaleFactor: z._default(z.enum(VIDEO_SCALE_FACTORS), VideoScaleFactor.original),
    resolution: z.optional(z.enum(TOPAZ_RESOLUTIONS)),
    model: z._default(z.enum(TOPAZ_MODELS), TopazVideoModel.starlightPrecise),
    enhancement: z._default(z.boolean(), true),
    enhancementModel: z._default(z.enum(TOPAZ_ENHANCEMENT_MODELS), TopazVideoEnhancementModel.starlightPrecise),
    focusFix: z.wire('focus_fix_level', z._default(z.enum(FOCUS_FIXES), TopazVideoFocusFix.normal)),
    parameters: z._default(z.enum(PARAMETER_MODES), TopazVideoParameters.auto),
    compression: z.optional(z.number()),
    details: z.optional(z.number()),
    preblur: z.optional(z.number()),
```

```
    blur: z.optional(z.number()),
    noise: z.optional(z.number()),
    halo: z.optional(z.number()),
    grainEnabled: z._default(z.boolean(), false),
    grainStrength: z._default(z.number(), 0.04),
    grainSize: z._default(z.number(), 0.02),
    frameInterpolation: z._default(z.boolean(), false),
    frameInterpolationFps: z._default(z.number(), 24),
    slowMotion: z._default(z.number(), 1),
  }
}

function stripTopazControlFields(wire: Record<string, unknown>): Record<string, unknown> {
  const rest = { ...wire }
  for (const key of [
    'scaleFactor',
    'resolution',
    'enhancement',
    'enhancementModel',
    'focus_fix_level',
    'parameters',
    'compression',
    'details',
    'preblur',
    'blur',
    'noise',
    'halo',
    'grainEnabled',
    'grainStrength',
    'grainSize',
    'frameInterpolation',
    'frameInterpolationFps',
    'slowMotion',
  ]) delete rest[key]
  return rest
}

function manualParams(wire: Record<string, unknown>) {
  if (wire.parameters !== TopazVideoParameters.manual)
    return null
  return {
    compression: wire.compression ?? -0.6,
    details: wire.details ?? -0.4,
    preblur: wire.preblur ?? -1,
    blur: wire.blur ?? -0.8,
    noise: wire.noise ?? -0.7,
    halo: wire.halo ?? -1,
    grain: wire.grainEnabled === true
      ? {
          strength: wire.grainStrength ?? 0.04,
          size: wire.grainSize ?? 0.02,
        }
      : null,
  }
}

export const topazVideoUpscale = defineJob({
  jobSetType: 'topaz_video',
  outputType: 'video',
  params: {
    media: {
      field: 'input_video',
      format: 'single',
      roles: ['video'],
      counts: { video: { min: 1, max: 1 } },
    },
    settings: topazVideoSettings(),
  },
  validate: (input) => {
    const settings = input.settings as Record<string, number | string | boolean | undefined>
```

```
    return [
      ...videoSourceIssues(input),
      ...oneOf('model', settings.model, TOPAZ_MODELS),
      ...oneOf('enhancementModel', settings.enhancementModel, TOPAZ_ENHANCEMENT_MODELS),
      ...oneOf('focusFix', settings.focusFix, FOCUS_FIXES),
      ...oneOf('parameters', settings.parameters, PARAMETER_MODES),
      ...oneOf('resolution', settings.resolution, TOPAZ_RESOLUTIONS),
      ...range('compression', settings.compression as number | undefined, -1, 1),
      ...range('details', settings.details as number | undefined, -1, 1),
      ...range('preblur', settings.preblur as number | undefined, -1, 1),
      ...range('blur', settings.blur as number | undefined, -1, 1),
      ...range('noise', settings.noise as number | undefined, -1, 1),
      ...range('halo', settings.halo as number | undefined, -1, 1),
      ...range('grainStrength', settings.grainStrength as number | undefined, 0, 0.1),
      ...range('grainSize', settings.grainSize as number | undefined, 0, 0.1),
      ...(settings.frameInterpolation === true
        ? [
            ...intRange('frameInterpolationFps', settings.frameInterpolationFps as number | undefined, 15, 240),
            ...integerRange('frameInterpolationFps', settings.frameInterpolationFps as number | undefined, 15, 240),
            ...intRange('slowMotion', settings.slowMotion as number | undefined, 1, 16),
            ...integerRange('slowMotion', settings.slowMotion as number | undefined, 1, 16),
          ]
        : []),
    ]
  },
  finalize: (wire, input) => {
    const source = videoSourceSize(input)!
    const output = resolveVideoOutput(wire, source)
    const rest = stripTopazControlFields(wire)
    return stripUndefined({
      ...rest,
      width: source.width,
      height: source.height,
      output_width: output.width,
      output_height: output.height,
      model: (wire.model ?? TopazVideoModel.starlightPrecise) as TopazModel,
      enhancement: wire.enhancement === false
        ? null
        : {
            model: (wire.enhancementModel ?? TopazVideoEnhancementModel.starlightPrecise) as EnhancementModel,
            focus_fix_level: wire.focus_fix_level ?? TopazVideoFocusFix.normal,
            params: manualParams(wire),
          },
      frame_interpolation: wire.frameInterpolation === true
        ? { model: 'apo-8', fps: wire.frameInterpolationFps ?? 24, slowmo: wire.slowMotion ?? 1 }
        : null,
    })
  },
})

function simpleVideoUpscale(jobSetType: 'video_upscale' | 'video_deflicker') {
  return defineJob({
    jobSetType,
    outputType: 'video',
    params: {
      media: {
        field: 'input_video',
        format: 'single',
        roles: ['video'],
        counts: { video: { min: 1, max: 1 } },
      },
      settings: {
        sourceWidth: z.wire('width', z.optional(z.number())),
        sourceHeight: z.wire('height', z.optional(z.number())),
        outputWidth: z.wire('output_width', z.optional(z.number())),
        outputHeight: z.wire('output_height', z.optional(z.number())),
        scaleFactor: z._default(z.enum(VIDEO_SCALE_FACTORS), VideoScaleFactor.original),
      },
    },
    credits: () => 2,
```

```
    validate: videoSourceIssues,
    finalize: (wire, input) => {
      const source = videoSourceSize(input)!
      const output = wire.output_width != null && wire.output_height != null
        ? { width: wire.output_width as number, height: wire.output_height as number }
        : scaleVideoSize((wire.scaleFactor ?? VideoScaleFactor.original) as VideoScaleFactorValue, source)
      const rest = { ...wire }
      delete rest.scaleFactor
      return stripUndefined({
        ...rest,
        width: source.width,
        height: source.height,
        output_width: output.width,
        output_height: output.height,
      })
    },
  })
}

export const higgsfieldVideoUpscale = simpleVideoUpscale('video_upscale')
export const soraEnhanceVideo = simpleVideoUpscale('video_deflicker')

export const bytedanceVideoUpscale = defineJob({
  jobSetType: 'bytedance_video_upscale',
  outputType: 'video',
  params: {
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['video'],
      counts: { video: { min: 1, max: 1 } },
    },
    settings: {
      sourceWidth: z.wire('width', z.optional(z.number())),
      sourceHeight: z.wire('height', z.optional(z.number())),
      resolution: z._default(z.enum(BYTEDANCE_RESOLUTIONS), BytedanceVideoUpscaleResolution.r2k),
      preset: z._default(z.enum(BYTEDANCE_PRESETS), BytedanceVideoUpscalePreset.common),
      fps: z._default(z.number(), 24),
    },
  },
  credits: ({ settings, media }) => localBytedanceVideoCredits(settings.resolution ?? '2k', firstDuration(media, ['video']), settings.fps ?? 24),
  validate: (input) => {
    const settings = input.settings as { resolution?: string, preset?: string, fps?: number }
    return [
      ...videoSourceIssues(input),
      ...oneOf('resolution', settings.resolution, BYTEDANCE_RESOLUTIONS),
      ...oneOf('preset', settings.preset, BYTEDANCE_PRESETS),
      ...intRange('fps', settings.fps, 1, 240),
      ...integerRange('fps', settings.fps, 1, 240),
    ]
  },
  finalize: (wire, input) => {
    const source = videoSourceSize(input)!
    return stripUndefined({
      ...wire,
      width: source.width,
      height: source.height,
    })
  },
})
```

app/packages/fnf/src/jobs/wan-2-7.ts

```
import { defineJob } from '../define-job'
import { z } from '../z'
import { intRange, oneOf, promptRequired, randomSeed } from './checks'
import { lookupSize } from './dimensions'
import { integerRange } from './video-helpers'

export const Wan27AspectRatio = {
  r16x9: '16:9',
  r9x16: '9:16',
  r4x3: '4:3',
  r3x4: '3:4',
  r1x1: '1:1',
} as const

export const Wan27Quality = {
  r720: '720p',
  r1080: '1080p',
} as const

const ASPECT_RATIOS = Object.values(Wan27AspectRatio)
const QUALITIES = Object.values(Wan27Quality)

type Ratio = typeof ASPECT_RATIOS[number]
type Quality = typeof QUALITIES[number]

const SIZE_MAP: Record<Quality, Record<Ratio, [number, number]>> = {
  '720p': { '16:9': [1280, 720], '9:16': [720, 1280], '4:3': [960, 720], '3:4': [720, 960], '1:1': [720, 720] },
  '1080p': { '16:9': [1920, 1080], '9:16': [1080, 1920], '4:3': [1440, 1080], '3:4': [1080, 1440], '1:1': [1080, 1080] },
}

export const wan27 = defineJob({
  jobSetType: 'wan2_7',
  outputType: 'video',
  params: {
    prompt: true,
    media: {
      field: 'medias',
      format: 'wrapped',
      roles: ['start_image', 'end_image'],
      counts: { start_image: { max: 1 }, end_image: { max: 1 } },
    },
    settings: {
      seed: z.optional(z.number()),
      quality: z._default(z.enum(QUALITIES), '720p'),
      duration: z._default(z.duration({ min: 2, max: 15 }), 5),
      aspectRatio: z.wire('aspect_ratio', z._default(z.aspectRatio(ASPECT_RATIOS), '16:9')),
    },
  },
  credits: ({ settings }) => Math.ceil((settings.duration ?? 5) * (settings.quality === '1080p' ? 2.5 : 1.5)),
  validate: ({ prompt, settings }) => [
    ...promptRequired(prompt),
    ...oneOf('quality', settings.quality, QUALITIES),
    ...oneOf('aspectRatio', settings.aspectRatio, ASPECT_RATIOS),
    ...intRange('duration', settings.duration, 2, 15),
    ...integerRange('duration', settings.duration, 2, 15),
    ...intRange('seed', settings.seed, 1, 2_147_483_646),
    ...integerRange('seed', settings.seed, 1, 2_147_483_646),
  ],
  finalize: (wire) => {
    const quality = wire.quality as Quality
    const { width, height } = lookupSize(SIZE_MAP, quality, wire.aspect_ratio as Ratio)
    return {
      ...wire,
      seed: wire.seed ?? randomSeed(),
      width,
      height,
      input_images: [],
      resolution: quality,
```

```
      medias: wire.medias ?? [],
    }
  },
  restore: wire => ({
    ...(typeof wire.resolution === 'string' && wire.quality == null ? { quality: wire.resolution } : {}),
  }),
})
```

app/packages/fnf/src/llm/backend.ts

```
/**
 * LLM port (spec §5.1): the transport-agnostic surface template apps consume.
 * Wire shape is the gateway's OpenAI-compatible subset; these types stay
 * deliberately narrower than openai's own (YAGNI: only what the gateway accepts).
 */

export interface LlmMessagePart { type: 'text' | 'image_url', text?: string, image_url?: { url: string } }
export interface LlmToolCall { id: string, name: string, arguments: string }
export interface LlmToolCallDelta { index: number, id?: string, name?: string, argumentsDelta?: string }
export interface LlmMessage {
  role: 'system' | 'user' | 'assistant' | 'tool'
  content: string | LlmMessagePart[]
  tool_call_id?: string
  tool_calls?: LlmToolCall[]
}
export interface LlmToolDef {
  name: string
  description?: string
  parameters: Record<string, unknown> // JSON Schema
}
export interface LlmCompleteRequest {
  model: string
  messages: LlmMessage[]
  maxTokens?: number
  temperature?: number
  tools?: LlmToolDef[]
  toolChoice?: 'auto' | 'none' | { name: string }
  signal?: AbortSignal
}
export interface LlmUsage { promptTokens: number, completionTokens: number }
export interface LlmCompletion {
  content: string
  toolCalls: LlmToolCall[]
  finishReason: string
  usage: LlmUsage
}
export interface LlmDelta {
  content?: string
  toolCallDelta?: LlmToolCallDelta
  finishReason?: string
  usage?: LlmUsage
}

export interface LlmBackend {
  complete: (req: LlmCompleteRequest) => Promise<LlmCompletion>
  stream: (req: LlmCompleteRequest) => AsyncIterable<LlmDelta>
  listModels: () => Promise<string[]>
}
```

app/packages/fnf/src/llm/client.ts

```
import type { OptionalHeaderSource, RequiredHeaderSource } from '../headers'
import type { FnfObservabilityOptions } from '../observability'
import type { Transport } from '../transport'
import type { LlmBackend, LlmCompleteRequest, LlmCompletion, LlmDelta, LlmMessage, LlmToolCall } from './backend'
import { ApiJobError } from '../errors'
import { authorizationHeader, cleanHeaders, optionalNamedHeader } from '../headers'
import { observeEvent } from '../observability'
import { createFetchTransport } from '../workflow-platform/fetch-transport'
import { errorFromLlmResponse } from './errors'

export interface LlmClientOptions {
  /**
   * Gateway LLM mount WITHOUT the `/v1` suffix — the client appends
   * `/v1/chat/completions` and `/v1/models` itself. For CF-Worker apps this is
   * the EXTERNAL gateway, e.g. `https://fnf-api-gw.higgsfield.ai/llm`
   * (dev: `https://dev-fnf-api-gw.higgsfield.ai/llm`). Trailing slash trimmed.
   */
  baseUrl: string
  /** Same token source the WFP adapter uses; sent as `Authorization: Bearer`. */
  getToken?: () => Promise<string | null>
  /** Sent as `hf-user-id` (internal-gateway identity; ignored externally). */
  userId?: RequiredHeaderSource
  /** Sent as `hf-workspace-id`. */
  workspaceId?: OptionalHeaderSource
  /** Sent as `hf-app-id` (observability tag). */
  appId?: OptionalHeaderSource
  fetch?: typeof globalThis.fetch
  /**
   * Inject a transport (tests / custom envs). Covers `complete`/`listModels`
   * ONLY — `stream` always uses `fetch` because the Transport shape is
   * JSON-request/response by design and SSE is not.
   */
  transport?: Transport
  /**
   * Observes the JSON paths (`complete`/`listModels`) as full start→success/
   * error spans via the observed Transport. The SSE `stream` path bypasses the
   * Transport (fetch, not a Transport call), so it emits `fnf.llm.stream`
   * `start`/`error` EVENTS only — deliberately NO success/duration span: the
   * iterator is lazy, so there is no natural completion point to close a span
   * without misattributing consumer time. Full stream telemetry (a
   * span closed on stream end) is a follow-up.
   */
  observability?: FnfObservabilityOptions
}

export function createLlmClient(options: LlmClientOptions): LlmBackend {
  const headers = async () => cleanHeaders({
    ...(await authorizationHeader(options.getToken)),
    ...(await optionalNamedHeader('hf-user-id', options.userId)),
    ...(await optionalNamedHeader('hf-workspace-id', options.workspaceId)),
    ...(await optionalNamedHeader('hf-app-id', options.appId)),
  })
  const transport = options.transport ?? createFetchTransport({
    baseUrl: options.baseUrl,
    headers,
    fetch: options.fetch,
    observability: options.observability,
  })

  async function send(method: 'GET' | 'POST', path: string, body?: unknown, signal?: AbortSignal): Promise<unknown> {
    const res = await transport({ method, path, body, ...(signal ? { signal } : {}) })
    const err = errorFromLlmResponse(res.status, res.body)
    if (err)
      throw err
    return res.body
  }

  return {
```

```
    async complete(req: LlmCompleteRequest): Promise<LlmCompletion> {
      const body = (await send('POST', '/v1/chat/completions', wireBody(req, false), req.signal) ?? {}) as CompletionWire
      const choice = body.choices?.[0]
      return {
        content: choice?.message?.content ?? '',
        toolCalls: (choice?.message?.tool_calls ?? []).map(toToolCall),
        finishReason: choice?.finish_reason ?? '',
        usage: {
          promptTokens: body.usage?.prompt_tokens ?? 0,
          completionTokens: body.usage?.completion_tokens ?? 0,
        },
      }
    },

    async* stream(req: LlmCompleteRequest): AsyncIterable<LlmDelta> {
      const doFetch = options.fetch ?? globalThis.fetch.bind(globalThis)
      // Start/error events only — no success span (lazy iterator; see the
      // `observability` option JSDoc). One `try` covers BOTH throw paths:
      // pre-stream HTTP envelope and a mid-stream `data:{"error"}` (raised by
      // `chunkToDeltas`) / abort.
      observeEvent(options.observability, 'fnf.llm.stream', { model: req.model }, { phase: 'start' })
      try {
        const res = await doFetch(`${options.baseUrl.replace(/\/$/, '')}/v1/chat/completions`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json', ...(await headers()) },
          body: JSON.stringify(wireBody(req, true)),
          ...(req.signal ? { signal: req.signal } : {}),
        })
        // Pre-stream failures are plain HTTP error envelopes — thrown before the
        // iterator yields anything.
        if (!res.ok || !res.body) {
          let parsed: unknown
          try {
            parsed = await res.json()
          }
          catch {
            parsed = undefined
          }
          throw errorFromLlmResponse(res.status, parsed)
            ?? new ApiJobError('llm_gateway_error', `LLM gateway error (HTTP ${res.status})`, { status: res.status })
        }
        for await (const data of sseData(res.body, req.signal)) {
          if (data === '[DONE]')
            return
          let chunk: ChunkWire
          try {
            chunk = JSON.parse(data) as ChunkWire
          }
          catch {
            continue // tolerate keep-alive noise; real protocol errors surface as stream end
          }
          yield* chunkToDeltas(chunk)
        }
      }
      catch (error) {
        observeEvent(options.observability, 'fnf.llm.stream', { model: req.model }, { phase: 'error', error })
        throw error
      }
    },

    async listModels(): Promise<string[]> {
      const body = (await send('GET', '/v1/models') ?? {}) as { data?: Array<{ id?: string }> }
      return (body.data ?? []).map(m => m.id ?? '').filter(Boolean)
    },
  }
}

// ■■ wire shaping ■■

interface CompletionWire {
```

```
  choices?: Array<{ message?: { content?: string | null, tool_calls?: ToolCallWire[] }, finish_reason?: string }>
  usage?: { prompt_tokens?: number, completion_tokens?: number }
}
interface ToolCallWire { id?: string, function?: { name?: string, arguments?: string } }
interface ChunkWire {
  choices?: Array<{
    delta?: { content?: string, tool_calls?: Array<{ index?: number, id?: string, function?: { name?: string, arguments?: string } }> }
    finish_reason?: string | null
  }>
  usage?: { prompt_tokens?: number, completion_tokens?: number }
  error?: { message?: string, type?: string, code?: string }
}

function toToolCall(tc: ToolCallWire): LlmToolCall {
  return { id: tc.id ?? '', name: tc.function?.name ?? '', arguments: tc.function?.arguments ?? '' }
}

function wireBody(req: LlmCompleteRequest, stream: boolean): Record<string, unknown> {
  const body: Record<string, unknown> = { model: req.model, messages: req.messages.map(wireMessage) }
  if (stream)
    body.stream = true
  if (req.maxTokens !== undefined)
    body.max_tokens = req.maxTokens
  if (req.temperature !== undefined)
    body.temperature = req.temperature
  if (req.tools)
    body.tools = req.tools.map(t => ({ type: 'function', function: { name: t.name, description: t.description, parameters: t.parameters } }))
  if (req.toolChoice !== undefined)
    body.tool_choice = typeof req.toolChoice === 'string' ? req.toolChoice : { type: 'function', function: { name: req.toolChoice.name } }
  return body
}

/**
 * The SDK's `LlmMessage.tool_calls` is the FLAT `LlmToolCall` shape (id/name/
 * arguments — what `complete()` hands back), so replaying an assistant turn
 * into the next request must re-nest it into the OpenAI wire shape.
 */
function wireMessage(m: LlmMessage): unknown {
  if (!m.tool_calls)
    return m
  return {
    ...m,
    tool_calls: m.tool_calls.map(tc => ({ id: tc.id, type: 'function', function: { name: tc.name, arguments: tc.arguments } })),
  }
}

/**
 * One wire chunk → zero or more `LlmDelta`s, throwing on an in-band error
 * event (`data: {"error":{...}}` — the gateway's mid-stream failure shape;
 * status 502 is synthetic since the HTTP exchange already succeeded).
 * A chunk's `delta.tool_calls` is an ARRAY of fragments — `LlmDelta` carries
 * one `toolCallDelta`, so each fragment becomes its own delta (content first,
 * finish/usage last).
 */
function chunkToDeltas(chunk: ChunkWire): LlmDelta[] {
  if (chunk.error)
    throw errorFromLlmResponse(502, { error: chunk.error })
  const choice = chunk.choices?.[0]
  const out: LlmDelta[] = []
  if (choice?.delta?.content)
    out.push({ content: choice.delta.content })
  for (const tc of choice?.delta?.tool_calls ?? []) {
    out.push({
      toolCallDelta: {
        index: tc.index ?? 0,
        ...(tc.id !== undefined ? { id: tc.id } : {}),
        ...(tc.function?.name !== undefined ? { name: tc.function.name } : {}),
        ...(tc.function?.arguments !== undefined ? { argumentsDelta: tc.function.arguments } : {}),
      },
    })
```

```
  }
  const tail: LlmDelta = {}
  if (choice?.finish_reason)
    tail.finishReason = choice.finish_reason
  if (chunk.usage)
    tail.usage = { promptTokens: chunk.usage.prompt_tokens ?? 0, completionTokens: chunk.usage.completion_tokens ?? 0 }
  if (tail.finishReason !== undefined || tail.usage !== undefined)
    out.push(tail)
  return out
}

// ■■ SSE reader: yields the payload after each `data: ` line ■■

async function* sseData(body: ReadableStream<Uint8Array>, signal?: AbortSignal): AsyncGenerator<string> {
  const reader = body.getReader()
  const decoder = new TextDecoder()
  let buf = ''
  try {
    for (;;) {
      if (signal?.aborted)
        throw new DOMException('aborted', 'AbortError')
      const { done, value } = await reader.read()
      if (done)
        return
      buf += decoder.decode(value, { stream: true })
      for (;;) {
        const nl = buf.indexOf('\n\n')
        if (nl < 0)
          break
        const frame = buf.slice(0, nl)
        buf = buf.slice(nl + 2)
        for (const line of frame.split('\n')) {
          if (line.startsWith('data: '))
            yield line.slice(6)
        }
      }
    }
  }
  finally {
    // Cancel (not just release) so early exits — [DONE], abort, consumer
    // break — actually release the underlying connection.
    try {
      await reader.cancel()
    }
    catch {
      // already errored/closed — nothing to release
    }
    reader.releaseLock()
  }
}
```

app/packages/fnf/src/llm/errors.ts

```
import type { BillingContext } from '../errors'
import { ApiJobError, OutOfCreditsError, RateLimitError } from '../errors'

interface OpenAIErrorBody { error?: { message?: string, type?: string, code?: string } }

/** The gateway's verbatim error detail, carried on billing errors as `data.gateway`. */
export interface LlmGatewayErrorDetail { message?: string, code?: string }

/** `data` shape of the billing errors produced by `errorFromLlmResponse`. */
export type LlmBillingErrorData = BillingContext & { gateway?: LlmGatewayErrorDetail }

/**
 * Map a gateway (OpenAI-shaped) error response onto the SDK's ApiJobError
 * catalog. The gateway emits `{"error":{message,type,code}}` bodies — NOT the
 * fnf `detail` shape `errorFromResponse` parses — so the LLM client gets its
 * own thin mapper here rather than routing through `errorFromResponse`.
 *
 * 402/429 reuse the existing billing classes so app-level handling
 * (`err.code === 'out_of_credits'`) works identically for generations and LLM
 * calls; everything else carries the gateway's own `code` verbatim (falling
 * back to `llm_gateway_error` when the body isn't OpenAI-shaped at all, e.g.
 * a raw string/HTML error page from an intermediary proxy).
 *
 * Message-loss trade-off, resolved: the billing classes keep their canonical
 * messages ('Not enough credits' / 'Rate limit reached') so app copy keyed on
 * them stays stable — the gateway's verbatim message/code are NOT dropped but
 * ride on `data.gateway` (`LlmGatewayErrorDetail`), the catalog's usual
 * context carrier, so 402/429 callers can still surface them.
 *
 * Never throws on a malformed/non-JSON body — worst case it falls through to
 * the generic `ApiJobError('llm_gateway_error', ...)` with the raw status.
 */
export function errorFromLlmResponse(status: number, body: unknown): ApiJobError | null {
  if (status >= 200 && status < 300)
    return null
  const detail = (typeof body === 'object' && body !== null ? (body as OpenAIErrorBody).error : undefined) ?? {}
  const message = detail.message ?? `LLM gateway error (HTTP ${status})`
  let err: ApiJobError
  if (status === 402)
    err = attachGatewayDetail(new OutOfCreditsError({}), detail)
  else if (status === 429)
    err = attachGatewayDetail(new RateLimitError({}), detail)
  else
    err = new ApiJobError(detail.code ?? 'llm_gateway_error', message)
  return stampLlmStatus(err, status)
}

/**
 * Stamp the wire status onto the error — but only fill a null one, never
 * overwrite a status the class pinned in its constructor (same contract as
 * `errorFromResponse`'s stamping). Exported as the seam for the
 * status-preservation contract test — a synthetic pre-set-status class has no
 * other way into the mapper.
 */
export function stampLlmStatus<E extends ApiJobError>(err: E, status: number): E {
  if (err.status == null)
    (err as { status?: number }).status = status
  return err
}

/**
 * Preserve the gateway's verbatim message/code on the billing classes, whose
 * constructors fix `message` and take only a `BillingContext`. `data` is the
 * catalog's context carrier and `readonly`, so this is a post-construction
 * attach (mirroring the catalog's own status-stamp cast) rather than an
 * untyped cast into the constructor. Rides `toJSON()` for free via `data`.
 */
function attachGatewayDetail<E extends ApiJobError<BillingContext>>(err: E, detail: LlmGatewayErrorDetail): E {
```

```
  if (detail.message === undefined && detail.code === undefined)
    return err
  const gateway: LlmGatewayErrorDetail = {
    ...(detail.message !== undefined ? { message: detail.message } : {}),
    ...(detail.code !== undefined ? { code: detail.code } : {}),
  }
  ;(err as { data?: LlmBillingErrorData }).data = { ...err.data, gateway }
  return err
}
```

app/packages/fnf/src/media-meta.ts

```
import type { MediaInput, MediaMeta, MediaRef } from './types'

/**
 * Measures one media ref — answers its intrinsic `MediaMeta`, or undefined
 * when it can't (unknown kind, no url, measurement failed). The browser
 * implementation is `createDomMediaMetaResolver` (media entry point); a server
 * could back this with a probe service. Resolution is a capability the caller
 * injects, so the core stays zero-I/O.
 */
export type MediaMetaResolver = (ref: MediaRef) => Promise<MediaMeta | undefined>

/**
 * Fill in missing `meta` on every media ref of a submit input — the opt-in
 * async step in front of the sync meta rules (`dimensionsWithin`,
 * `durationsWithin`), mirroring how `adjust` fronts submit:
 *
 *   const measured = await resolveMediaMeta(input, resolver)
 *   await jobs.submit(measured) // meta rules now judge every ref
 *
 * Pure with respect to its arguments: returns a NEW input; refs that already
 * carry meta are untouched (the app usually knows sizes already — fnf's
 * InputImageMedia does); a resolver failure leaves that ref as-is rather than
 * failing the whole step. All refs resolve in parallel.
 */
export async function resolveMediaMeta<Input extends { media?: MediaInput }>(
  input: Input,
  resolve: MediaMetaResolver,
): Promise<Input> {
  const media = input.media
  if (!media)
    return input

  const out: MediaInput = {}
  await Promise.all(Object.entries(media).map(async ([role, value]) => {
    if (value === undefined)
      return
    out[role] = Array.isArray(value)
      ? await Promise.all(value.map(ref => withMeta(ref, resolve)))
      : await withMeta(value, resolve)
  }))
  return { ...input, media: out }
}

async function withMeta(ref: MediaRef, resolve: MediaMetaResolver): Promise<MediaRef> {
  if (ref.meta)
    return ref
  try {
    const meta = await resolve(ref)
    return meta ? { ...ref, meta } : ref
  }
  catch {
    return ref // measurement is best-effort; the backend re-validates anyway
  }
}
```

app/packages/fnf/src/media/__tests__/blob-uploader.test.ts

```
import { describe, expect, it } from 'vitest'
import { JobAbortedError } from '../../errors'
import { createFetchUploader } from '../blob-uploader'
import { UploadTransferError, UrlIngestError } from '../errors'

type FetchLike = typeof globalThis.fetch

function uploader(fetch: FetchLike, retries = 2) {
  return createFetchUploader({ fetch, retries })
}

const PUT = { uploadUrl: 'https://s3/put/m1', bytes: new Uint8Array([1]), contentType: 'image/png' }

describe('createFetchUploader.transfer', () => {
  it('retries 5xx then succeeds', async () => {
    let calls = 0
    const up = uploader(async () => new Response('', { status: ++calls < 2 ? 503 : 200 }))
    await up.transfer(PUT)
    expect(calls).toBe(2)
  })

  it('bails on 4xx with the status (no retry)', async () => {
    let calls = 0
    const up = uploader(async () => {
      calls++
      return new Response('denied', { status: 403 })
    })
    await expect(up.transfer(PUT)).rejects.toMatchObject({ code: 'upload_failed', status: 403 })
    expect(calls).toBe(1)
  })

  it('an abort mid-PUT throws JobAbortedError without burning the retry budget', async () => {
    const controller = new AbortController()
    let calls = 0
    const up = uploader(async () => {
      calls++
      controller.abort() // abort lands while the PUT is in flight
      const err = new Error('The operation was aborted')
      err.name = 'AbortError'
      throw err
    })
    await expect(up.transfer({ ...PUT, signal: controller.signal })).rejects.toBeInstanceOf(JobAbortedError)
    expect(calls).toBe(1) // not retried with the aborted signal
  })

  it('a pre-aborted signal throws before any request', async () => {
    const controller = new AbortController()
    controller.abort()
    let calls = 0
    const up = uploader(async () => {
      calls++
      return new Response('', { status: 200 })
    })
    await expect(up.transfer({ ...PUT, signal: controller.signal })).rejects.toBeInstanceOf(JobAbortedError)
    expect(calls).toBe(0)
  })

  it('exhausted network retries surface the last cause in the message', async () => {
    const up = uploader(async () => {
      throw new TypeError('Failed to fetch')
    }, 1)
    await expect(up.transfer(PUT)).rejects.toThrow(/after 2 attempts: Failed to fetch/)
  })
})

describe('createFetchUploader.fetchBytes', () => {
  it('returns bytes + content type', async () => {
    const up = uploader(async () => new Response(new Uint8Array([1, 2, 3]), { status: 200, headers: { 'content-type': 'image/png' } }))
```

```
    const out = await up.fetchBytes!('https://x/a.png')
    expect(out.contentType).toBe('image/png')
    expect((out.bytes as Uint8Array).length).toBe(3)
  })

  it('rejects early via content-length and late via byteLength when over maxBytes', async () => {
    const declared = uploader(async () => new Response('', { status: 200, headers: { 'content-length': '10' } }))
    await expect(declared.fetchBytes!('https://x/a', { maxBytes: 5 })).rejects.toBeInstanceOf(UrlIngestError)

    const undeclared = uploader(async () => new Response(new Uint8Array(10), { status: 200 }))
    await expect(undeclared.fetchBytes!('https://x/a', { maxBytes: 5 })).rejects.toBeInstanceOf(UrlIngestError)
  })

  it('an abort during the download throws JobAbortedError, not UrlIngestError', async () => {
    const controller = new AbortController()
    const up = uploader(async () => {
      controller.abort()
      const err = new Error('aborted')
      err.name = 'AbortError'
      throw err
    })
    await expect(up.fetchBytes!('https://x/a', { signal: controller.signal })).rejects.toBeInstanceOf(JobAbortedError)
  })

  it('a non-OK response throws UploadTransferError-free UrlIngestError with the status', async () => {
    const up = uploader(async () => new Response('', { status: 404 }))
    await expect(up.fetchBytes!('https://x/a')).rejects.toThrow(/404/)
    await expect(up.fetchBytes!('https://x/a')).rejects.toBeInstanceOf(UrlIngestError)
    expect(UploadTransferError.is(await up.fetchBytes!('https://x/a').catch(e => e))).toBe(false)
  })
})
```

app/packages/fnf/src/media/__tests__/media.test.ts

```
import { describe, expect, it } from 'vitest'
import { createMediaClient } from '../client'

describe('submit-ready media refs', () => {
  it('normalizes raw upload-plane labels while preserving job discriminators', async () => {
    const media = createMediaClient({
      mediaAdapter: {
        async getMedia() {
          return { id: 'video-1', type: 'video', url: 'https://cdn/video.mp4' }
        },
        async listMedia() {
          return {
            items: [
              { id: 'image-1', type: 'image', url: 'https://cdn/image.png' },
              { id: 'job-1', type: 'image_job', url: 'https://cdn/job.png' },
            ],
          }
        },
      },
    })

    await expect(media.get('video-1', 'video')).resolves.toMatchObject({ type: 'video_input' })
    await expect(media.list({ type: 'image' })).resolves.toMatchObject({
      items: [{ type: 'media_input' }, { type: 'image_job' }],
    })
  })
})
```

app/packages/fnf/src/media/blob-uploader.ts

```
import type { BinaryUploader, MediaBytes } from './types'
import { throwIfAborted } from '../errors'
import { UploadTransferError, UrlIngestError } from './errors'

// The return type spells out `Uint8Array<ArrayBuffer>`: this file is compiled
// under EVERY consumer's tsconfig (packages ship source), and DOM-lib
// consumers (fnf-react) have the post-TS-5.7 `BodyInit` that rejects
// `ArrayBufferLike`-backed views — node-lib alone wouldn't catch it.
function toBodyInit(bytes: MediaBytes): Blob | Uint8Array<ArrayBuffer> {
  if (bytes instanceof ArrayBuffer)
    return new Uint8Array(bytes) // a bare ArrayBuffer isn't a valid fetch body in every runtime; wrap it
  if (bytes instanceof Uint8Array) {
    // A SharedArrayBuffer-backed view is not a legal fetch body — copy out;
    // the normal ArrayBuffer-backed view re-wraps zero-copy.
    return bytes.buffer instanceof ArrayBuffer
      ? new Uint8Array(bytes.buffer, bytes.byteOffset, bytes.byteLength)
      : new Uint8Array(bytes)
  }
  return bytes
}

/**
 * Default isomorphic `BinaryUploader` (Node / browser / Figma webview / CEP).
 * Does the raw presigned PUT with the file's Content-Type and NO auth header;
 * retries on 5xx / network error and bails on 4xx (presigned PUTs are
 * idempotent). An abort is a cancel, not a retryable failure — it surfaces as
 * the typed `JobAbortedError`. Adobe UXP — where webviews forbid a direct
 * host-bridge fetch but allow a cross-origin PUT — should inject its own uploader.
 *
 * `fetchBytes` is a basic reader for `uploadMediaFromUrl`. NOTE: it is NOT
 * SSRF-hardened — in a server/untrusted-URL context inject a uploader whose
 * `fetchBytes` enforces private-IP rejection / redirect / byte caps.
 */
export function createFetchUploader(opts: { fetch?: typeof globalThis.fetch, retries?: number } = {}): BinaryUploader {
  // Lazy lookup (not a captured reference): keeps `this` off the call (CEP/polyfills
  // throw Illegal invocation on a detached fetch) and picks up late-installed polyfills.
  const doFetch = opts.fetch ?? ((...args: Parameters<typeof globalThis.fetch>) => globalThis.fetch(...args))
  const retries = opts.retries ?? 2

  return {
    async transfer({ uploadUrl, bytes, contentType, signal }) {
      const body = toBodyInit(bytes)
      let lastStatus: number | undefined
      let lastError: unknown
      for (let attempt = 0; attempt <= retries; attempt++) {
        throwIfAborted(signal)
        let res: Response
        try {
          res = await doFetch(uploadUrl, { method: 'PUT', headers: { 'Content-Type': contentType }, body, signal })
        }
        catch (err) {
          throwIfAborted(signal) // an aborted PUT is a cancel, not a retryable network error
          lastError = err
          continue
        }
        if (res.ok)
          return
        if (res.status < 500) {
          const text = await res.text().catch(() => '')
          throw new UploadTransferError(`Upload failed (${res.status})${text ? `: ${text}` : ''}`, res.status) // 4xx — bail
        }
        lastStatus = res.status // 5xx — retry
      }
      const cause = lastError instanceof Error ? `: ${lastError.message}` : ''
      throw new UploadTransferError(`Upload failed after ${retries + 1} attempts${cause}`, lastStatus)
    },

    async fetchBytes(url, fetchOpts) {
```

```
      let res: Response
      try {
        res = await doFetch(url, { signal: fetchOpts?.signal })
      }
      catch (err) {
        throwIfAborted(fetchOpts?.signal)
        throw new UrlIngestError(`Failed to fetch ${url}: ${err instanceof Error ? err.message : String(err)}`)
      }
      if (!res.ok)
        throw new UrlIngestError(`Failed to fetch ${url} (${res.status})`)
      // Cheap early reject when the server declares the size; the post-download
      // check below stays authoritative (Content-Length may be absent or encoded size).
      const declared = Number(res.headers.get('content-length'))
      if (fetchOpts?.maxBytes != null && declared > fetchOpts.maxBytes)
        throw new UrlIngestError(`Media exceeds maxBytes (${declared} > ${fetchOpts.maxBytes})`)
      const buf = await res.arrayBuffer().catch((err) => {
        throwIfAborted(fetchOpts?.signal)
        throw new UrlIngestError(`Failed to read ${url}: ${err instanceof Error ? err.message : String(err)}`)
      })
      if (fetchOpts?.maxBytes != null && buf.byteLength > fetchOpts.maxBytes)
        throw new UrlIngestError(`Media exceeds maxBytes (${buf.byteLength} > ${fetchOpts.maxBytes})`)
      return { bytes: new Uint8Array(buf), contentType: res.headers.get('content-type') ?? 'application/octet-stream' }
    },
  }
}

/** A no-op uploader for tests / offline pipelines (pairs with `createMemoryMediaAdapter`). */
export function createMemoryUploader(): BinaryUploader {
  return {
    async transfer() {},
    async fetchBytes() { return { bytes: new Uint8Array(), contentType: 'application/octet-stream' } },
  }
}
```

app/packages/fnf/src/media/client.ts

```
import type { MediaClient, MediaClientConfig } from './types'
import { createMediaContext } from './context'
import { getMedia } from './get'
import { listMedia } from './list'
import { resolveMedia } from './resolve'
import { confirmMedia, getUploadUrl, uploadMedia, uploadMediaFromUrl } from './upload'

/**
 * Compose the media operations into a client. Sugar over the free functions —
 * every method binds the shared media context. Build a context and call the
 * operations directly if you only need one:
 *
 *   const ctx = createMediaContext(config)
 *   await getMedia(ctx, id, 'image')
 */
export function createMediaClient(config: MediaClientConfig): MediaClient {
  const ctx = createMediaContext(config)
  return {
    get: (id, type) => getMedia(ctx, id, type),
    list: opts => listMedia(ctx, opts),
    resolve: refs => resolveMedia(ctx, refs),
    upload: input => uploadMedia(ctx, input),
    uploadFromUrl: input => uploadMediaFromUrl(ctx, input),
    getUploadUrl: req => getUploadUrl(ctx, req),
    confirm: req => confirmMedia(ctx, req),
  }
}
```

app/packages/fnf/src/media/context.ts

```
import type { MediaClientConfig, MediaContext } from './types'
import { createObservabilityContext } from '../observability'
import { createFetchUploader } from './blob-uploader'

/** Resolve media config into the shared context every media operation consumes. */
export function createMediaContext(config: MediaClientConfig): MediaContext {
  return {
    mediaAdapter: config.mediaAdapter,
    blobUploader: config.blobUploader ?? createFetchUploader(),
    resolveJob: config.resolveJob,
    observability: createObservabilityContext(config.observability),
  }
}
```

app/packages/fnf/src/media/dom-meta-resolver.ts

```
import type { MediaMetaResolver } from '../media-meta'
import type { MediaMeta, MediaRef } from '../types'
import { getMediaType } from '../selectors'

/**
 * Browser-backed `MediaMetaResolver`: measures images via `Image` and
 * video/audio via detached media elements — the same technique fnf-web's
 * `getImageSize`/`getMediaMetaVideo` use. Outside a DOM (Node, workers) every
 * ref resolves to undefined, so callers can wire it unconditionally. A failed
 * measurement (decode error, unsupported codec) resolves undefined too — the
 * `MediaMetaResolver` contract — and is NOT cached, so a retry of the same
 * url re-measures instead of replaying the failure. Successes are cached per
 * url for the resolver's lifetime.
 *
 * The package core compiles without `lib: dom`, so the handful of DOM shapes
 * used here are declared structurally instead of imported.
 */
export function createDomMediaMetaResolver(): MediaMetaResolver {
  const cache = new Map<string, Promise<MediaMeta | undefined>>()

  return async (ref: MediaRef) => {
    if (!dom() || !ref.url)
      return undefined
    const url = ref.url
    let pending = cache.get(url)
    if (!pending) {
      const kind = ref.type === 'image' || ref.type === 'video' || ref.type === 'audio' ? ref.type : undefined
      pending = measure(url, kind).catch(() => {
        cache.delete(url) // don't poison future retries of this url
        return undefined
      })
      cache.set(url, pending)
    }
    return pending
  }
}

// ■■ minimal structural DOM (the package core compiles without lib: dom) ■■

interface DomImage {
  naturalWidth: number
  naturalHeight: number
  onload: (() => void) | null
  onerror: (() => void) | null
  src: string
}

interface DomMediaElement {
  preload: string
  duration: number
  videoWidth: number
  videoHeight: number
  onloadedmetadata: (() => void) | null
  onerror: (() => void) | null
  src: string
}

interface DomGlobals {
  document?: { createElement: (tag: string) => DomMediaElement }
  Image?: new () => DomImage
}

function dom(): DomGlobals | null {
  const candidate = globalThis as DomGlobals
  return candidate.document && candidate.Image ? candidate : null
}

// `getMediaType` covers the image/video output kinds; audio inputs are its
```

```
// blind spot, so sniff those extensions here.
const AUDIO_URL = /\.(?:mp3|wav|m4a|aac|ogg|flac)(?:[?#]|$)/i

async function measure(url: string, kind?: string): Promise<MediaMeta | undefined> {
  // A blob/object URL carries no extension — callers that know the kind
  // (e.g. from File.type) pass it as the ref's literal type.
  if (kind === 'audio' || AUDIO_URL.test(url))
    return measureElement('audio', url)
  if (kind === 'video')
    return measureElement('video', url)
  if (kind === 'image')
    return measureImage(url)
  switch (getMediaType(url)) {
    case 'image':
      return measureImage(url)
    case 'video':
      return measureElement('video', url)
    default:
      // Unknown extension: an <img> probe is cheap and covers extensionless
      // CDN urls, which are images in this product far more often than not.
      return measureImage(url).catch(() => undefined)
  }
}

function measureImage(url: string): Promise<MediaMeta> {
  return new Promise((resolve, reject) => {
    const image = new (dom() as DomGlobals).Image!()
    image.onload = () => resolve({ width: image.naturalWidth, height: image.naturalHeight })
    image.onerror = () => reject(new Error(`Failed to load image: ${url}`))
    image.src = url
  })
}

function measureElement(tag: 'video' | 'audio', url: string): Promise<MediaMeta> {
  return new Promise((resolve, reject) => {
    const element = (dom() as DomGlobals).document!.createElement(tag)
    element.preload = 'metadata'
    element.onloadedmetadata = () => {
      const meta: MediaMeta = { durationSec: element.duration }
      if (tag === 'video') {
        meta.width = element.videoWidth
        meta.height = element.videoHeight
      }
      element.src = '' // release the network connection
      resolve(meta)
    }
    element.onerror = () => reject(new Error(`Failed to load ${tag} metadata: ${url}`))
    element.src = url
  })
}
```

app/packages/fnf/src/media/errors.ts

```
import { ApiJobError, registerErrorCode } from '../errors'

/** Presign step (`getUploadUrl`) failed or returned no upload URL. */
export class PresignError extends ApiJobError {
  constructor(message = 'Failed to create an upload URL') { super('presigned_failed', message) }
}

/** The binary PUT to the presigned URL failed. */
export class UploadTransferError extends ApiJobError {
  constructor(message = 'Upload transfer failed', status?: number) { super('upload_failed', message, { status }) }
}

/** Upload source is not a valid binary container. Usually caused by JSON-serializing File/Blob/bytes. */
export class InvalidMediaSourceError extends ApiJobError {
  constructor(message = 'Media upload source must be Blob, ArrayBuffer, Uint8Array, or a read() function returning one') {
    super('invalid_media_source', message)
  }
}

/** The confirm step failed. */
export class ConfirmError extends ApiJobError {
  constructor(message = 'Failed to confirm upload') { super('confirm_failed', message) }
}

/** `uploadMediaFromUrl` could not fetch the remote URL (no `fetchBytes`, or fetch failed). */
export class UrlIngestError extends ApiJobError {
  constructor(message = 'Failed to ingest media from URL') { super('url_ingest_failed', message) }
}

/** The media adapter doesn't implement `getUploadUrl`/`confirmMedia`. */
export class UploadNotSupportedError extends ApiJobError {
  constructor() { super('upload_not_supported', 'This media adapter does not support uploads') }
}

/**
 * Confirm reported the media was blocked. One stable code; the specific wire
 * verdict ('ip_detected' | 'nsfw' | 'ip_check_rate_limit_reached') lives in
 * `data.status` — same one-code-one-class pattern as AccountSuspendedError.
 */
export class MediaModerationError extends ApiJobError<{ status: string }> {
  constructor(status: string) {
    super('media_moderation_blocked', `Media blocked by moderation: ${status}`, { data: { status } })
  }
}

// Rehydration entries for the media codes (registered here — not in errors.ts —
// to avoid a circular import between the two halves). These run at IMPORT time,
// which is why this file is listed in package.json `sideEffects` — any future
// module that registers codes on import must be added there too, or production
// tree-shaking silently drops the registrations and cross-boundary rehydration
// degrades to the base ApiJobError.
registerErrorCode('presigned_failed', j => new PresignError(j.message))
registerErrorCode('upload_failed', j => new UploadTransferError(j.message, j.status))
registerErrorCode('invalid_media_source', j => new InvalidMediaSourceError(j.message))
registerErrorCode('confirm_failed', j => new ConfirmError(j.message))
registerErrorCode('url_ingest_failed', j => new UrlIngestError(j.message))
registerErrorCode('upload_not_supported', () => new UploadNotSupportedError())
registerErrorCode('media_moderation_blocked', j => new MediaModerationError((j.data as { status?: string } | undefined)?.status ?? 'blocked'))
```

app/packages/fnf/src/media/get.ts

```
import type { MediaRef } from '../types'
import type { MediaContext, MediaGetOptions } from './types'
import { observeAsync } from '../observability'
import { normalizeMediaRefType } from './types'

interface RawMedia {
  id?: string
  url?: string
  type?: string
}

/**
 * Get a single media item by id. The backend route is per-type (image/video/
 * audio), so the type must be supplied — the adapter routes on it. Normalizes
 * the raw payload to a `MediaRef` usable as a job input: the backend's own
 * job discriminator wins (fetched media can carry a type like
 * `nano_banana_job`, valid on the wire); raw `image`/`video`/`audio` plane
 * labels and missing types normalize to the same input vocabulary upload uses.
 */
export async function getMedia(ctx: MediaContext, id: string, type: MediaGetOptions['type']): Promise<MediaRef> {
  return observeAsync(ctx.observability, 'fnf.media.get', { media_id: id, media_type: type }, async () => {
    const raw = (await ctx.mediaAdapter.getMedia({ id, type }) ?? {}) as RawMedia
    return {
      id: raw.id ?? id,
      type: normalizeMediaRefType(raw.type, type),
      ...(raw.url ? { url: raw.url } : {}),
    }
  }, {
    successAttributes: ref => ({ ref_type: ref.type }),
  })
}
```

app/packages/fnf/src/media/index.ts

```
// The opt-in measurement step for the meta rules (dimensionsWithin/durationsWithin):
// fill MediaRef.meta before validating. The DOM resolver is the browser capability;
// resolveMediaMeta is the pure orchestration over any resolver.
export { resolveMediaMeta } from '../media-meta'

export type { MediaMetaResolver } from '../media-meta'
// MediaClient.get/list/resolve return MediaRef — export it so a /media-only
// consumer doesn't need the root barrel.
export type { MediaMeta, MediaRef } from '../types'
export { createFetchUploader, createMemoryUploader } from './blob-uploader'
export { createMediaClient } from './client'
export { createMediaContext } from './context'
export { createDomMediaMetaResolver } from './dom-meta-resolver'
export {
  ConfirmError,
  InvalidMediaSourceError,
  MediaModerationError,
  PresignError,
  UploadNotSupportedError,
  UploadTransferError,
  UrlIngestError,
} from './errors'
export { getMedia } from './get'
export { listMedia } from './list'
export { defaultFilenameForContentType, inferContentType, inferUploadType } from './mime'
export { resolveMedia } from './resolve'
// Upload kind → product input-media discriminator (media_input/video_input/
// audio_input) — the vocabulary media adapters answer presigns with.
export { REF_TYPE_BY_UPLOAD } from './types'
export type {
  BinaryUploader,
  MediaBytes,
  MediaClient,
  MediaClientConfig,
  MediaContext,
  MediaGetOptions,
  MediaListOptions,
  MediaListResult,
  MediaReference,
  MediaSource,
  ResolveJobRef,
  SafeUploadResult,
  UploadInput,
  UploadModeration,
  UploadResult,
  UploadSlot,
  UploadType,
} from './types'

export { confirmMedia, getUploadUrl, isBlockedModeration, safeUploadMedia, transferBytes, uploadMedia, uploadMediaFromUrl } from './upload'
export type { UploadFromUrlInput } from './upload'
```

app/packages/fnf/src/media/list.ts

```
import type { MediaRef } from '../types'
import type { MediaContext, MediaListOptions, MediaListResult } from './types'
import { observeAsync } from '../observability'
import { normalizeMediaRefType } from './types'

interface RawMediaItem {
  id: string
  url?: string
  type?: string
  created_at?: number
}

export async function listMedia(ctx: MediaContext, opts: MediaListOptions): Promise<MediaListResult> {
  return observeAsync(ctx.observability, 'fnf.media.list', {
    media_type: opts.type,
    ...(opts.size !== undefined ? { size: opts.size } : {}),
    ...(opts.cursor !== undefined ? { has_cursor: true } : {}),
  }, async () => {
    const body = (await ctx.mediaAdapter.listMedia(opts) ?? {}) as {
      items?: RawMediaItem[]
      medias?: RawMediaItem[]
      next_cursor?: string | number | null
      cursor?: string | number | null
    }
    const items: MediaRef[] = (body.items ?? body.medias ?? []).map(m => ({
      id: m.id,
      // Preserve job discriminators, but never leak raw upload-plane labels.
      type: normalizeMediaRefType(m.type, opts.type),
      ...(m.url ? { url: m.url } : {}),
    }))
    const cursor = body.next_cursor ?? body.cursor ?? undefined
    return { items, ...(cursor != null ? { cursor } : {}) }
  }, {
    successAttributes: result => ({ item_count: result.items.length, has_cursor: result.cursor !== undefined }),
  })
}
```

app/packages/fnf/src/media/mime.ts

```
import type { UploadType } from './types'

// Ported from fnf-mcp-server src/tools/media/mime.ts so the SDK derives
// content-type / upload-type the same way the server does.

export function inferUploadType(contentType: string): UploadType {
  if (contentType.startsWith('video/'))
    return 'video'
  if (contentType.startsWith('audio/'))
    return 'audio'
  return 'image'
}

export function inferContentType(filename: string | undefined, contentType: string | undefined): string {
  const normalized = contentType?.trim()
  if (normalized)
    return normalized

  const ext = filename?.split('.').pop()?.toLowerCase()
  switch (ext) {
    case 'jpg':
    case 'jpeg':
      return 'image/jpeg'
    case 'webp':
      return 'image/webp'
    case 'gif':
      return 'image/gif'
    case 'mp4':
    case 'm4v':
      return 'video/mp4'
    case 'mov':
      return 'video/quicktime'
    case 'webm':
      return 'video/webm'
    case 'mp3':
      return 'audio/mpeg'
    case 'm4a':
      return 'audio/mp4'
    case 'wav':
      return 'audio/wav'
    case 'ogg':
      return 'audio/ogg'
    case 'png':
    default:
      return 'image/png'
  }
}

export function defaultFilenameForContentType(contentType: string, type: UploadType): string {
  switch (contentType) {
    case 'image/jpeg':
      return 'upload.jpg'
    case 'image/webp':
      return 'upload.webp'
    case 'image/gif':
      return 'upload.gif'
    case 'video/mp4':
      return 'upload.mp4'
    case 'video/quicktime':
      return 'upload.mov'
    case 'video/webm':
      return 'upload.webm'
    case 'audio/mpeg':
      return 'upload.mp3'
    case 'audio/mp4':
      return 'upload.m4a'
    case 'audio/wav':
      return 'upload.wav'
```

```
    case 'audio/ogg':
      return 'upload.ogg'
    default:
      return type === 'video' ? 'upload.mp4' : type === 'audio' ? 'upload.mp3' : 'upload.png'
  }
}
```

app/packages/fnf/src/media/resolve.ts

```
import type { MediaRef } from '../types'
import type { MediaContext, MediaReference } from './types'
import { JobAbortedError } from '../errors'
import { observeAsync } from '../observability'
import { getMediaType } from '../selectors'

const HTTPS_RE = /^https:\/\//i
// getMediaType covers image/video output kinds; audio extensions are its blind
// spot (same sniff as the DOM meta resolver).
const AUDIO_URL = /\.(?:mp3|wav|m4a|aac|ogg|flac)(?:[?#]|$)/i

/** The plane discriminator for a bare URL — extension-sniffed, image-default. */
function typeForUrl(url: string): string {
  if (AUDIO_URL.test(url))
    return 'audio_input'
  return getMediaType(url) === 'video' ? 'video_input' : 'media_input'
}

async function resolveOne(ctx: MediaContext, ref: MediaReference): Promise<MediaRef> {
  // Already structured — pass through unchanged.
  if (typeof ref !== 'string')
    return ref
  // An HTTPS URL is usable directly as a media input.
  if (HTTPS_RE.test(ref))
    return { id: ref, type: typeForUrl(ref), url: ref }
  // Otherwise it's an id. If a job resolver is wired (e.g. from a job client),
  // try resolving it to a completed generation; else fall back to a media input.
  if (ctx.resolveJob) {
    try {
      const resolved = await ctx.resolveJob(ref)
      if (resolved)
        return resolved
    }
    catch (err) {
      // A cancel is not "not a job" — propagate it; anything else falls through
      // to the media-input fallback (the resolver is best-effort by contract).
      if (err instanceof JobAbortedError)
        throw err
    }
  }
  return { id: ref, type: 'media_input' }
}

export async function resolveMedia(ctx: MediaContext, refs: MediaReference[]): Promise<MediaRef[]> {
  return observeAsync(ctx.observability, 'fnf.media.resolve', { ref_count: refs.length }, () => Promise.all(refs.map(ref => resolveOne(ctx, ref))), {
    successAttributes: result => ({ resolved_count: result.length }),
  })
}
```

app/packages/fnf/src/media/types.ts

```
import type { MediaBackend } from '../backend'
import type { FnfObservabilityContext, FnfObservabilityOptions } from '../observability'
import type { MediaRef } from '../types'

export type UploadType = 'image' | 'video' | 'audio'

/**
 * fnf-web's input-media discriminator per plane (input-media-model.ts) — the
 * `MediaRef.type` that goes on the job wire verbatim (e.g. seedance
 * medias[].data.type). Backends may return a more specific type (a job type
 * like `nano_banana_job` on fetched media) — preserve those; this is the
 * fallback vocabulary.
 */
export const REF_TYPE_BY_UPLOAD: Record<UploadType, string> = {
  image: 'media_input',
  video: 'video_input',
  audio: 'audio_input',
}

/** Convert backend upload-plane labels into submit-ready ref discriminators. */
export function normalizeMediaRefType(type: string | undefined, fallback: UploadType): string {
  return type === 'image' || type === 'video' || type === 'audio'
    ? REF_TYPE_BY_UPLOAD[type]
    : (type ?? REF_TYPE_BY_UPLOAD[fallback])
}

/** A caller-facing media reference: a structured ref, or a string (URL or id). */
export type MediaReference = string | MediaRef

export interface MediaListOptions {
  type: UploadType
  cursor?: string | number
  size?: number
}

export interface MediaListResult {
  items: MediaRef[]
  cursor?: string | number
}

export interface MediaGetOptions {
  id: string
  type: UploadType
}

// ■■ upload ■■
/** The transport-agnostic byte container — all three are valid fetch BodyInit. */
export type MediaBytes = Blob | ArrayBuffer | Uint8Array

/**
 * Ready bytes, or a lazy thunk that produces them in the fetch-capable context.
 * Do not JSON-serialize browser File/Blob/ArrayBuffer/Uint8Array into this
 * shape. For server bridges, send files as multipart FormData, read bytes
 * server-side, then pass a real Blob/ArrayBuffer/Uint8Array here.
 */
export type MediaSource = MediaBytes | { read: () => Promise<MediaBytes> }

/**
 * The injected byte/data-plane capability. `transfer` does the raw presigned
 * PUT (no API host, no auth, Content-Type only) — it is deliberately NOT on the
 * `MediaBackend` JSON port nor on `Transport` (which is GET|POST + JSON only).
 * `fetchBytes` (optional) backs `uploadMediaFromUrl`.
 */
export interface BinaryUploader {
  transfer: (args: { uploadUrl: string, bytes: MediaBytes, contentType: string, signal?: AbortSignal }) => Promise<void>
  fetchBytes?: (url: string, opts?: { maxBytes?: number, signal?: AbortSignal }) => Promise<{ bytes: MediaBytes, contentType: string }>
}
```

```ts
export interface UploadInput {
  source: MediaSource
  type?: UploadType
  contentType?: string
  filename?: string
  /** Tagged onto the returned MediaRef so it drops straight into a job's media role. */
  role?: string
  jobId?: string
  forceIpCheck?: boolean
  forceNsfwCheck?: boolean
  startSeconds?: number
  endSeconds?: number
  /**
   * Throw on a moderation-blocked confirm. Default true. Covers BOTH wire
   * shapes: a 2xx confirm body with a blocked status (`MediaModerationError`)
   * and the HTTP 422 `error_type` twin (`IpDetectedError`/
   * `IpCheckRateLimitError`) — with `false`, both come back as a moderation
   * `UploadResult` instead.
   */
  throwOnModeration?: boolean
  /** Cancels the upload (checked between steps; aborts the binary PUT in flight). */
  signal?: AbortSignal
  /** Extra wire fields for presign AND confirm (the product sends `force_ip_check`/`surface` there). */
  extra?: Record<string, unknown>
}

export interface UploadSlot {
  mediaId: string
  uploadUrl: string
  url?: string
  type: UploadType
  contentType: string
  method: 'PUT'
}

export interface UploadModeration {
  status: string
  ipCheckFinished?: boolean | null
  ipDetected?: boolean | null
}

export interface UploadResult {
  ref: MediaRef
  mediaId: string
  status: string
  type: UploadType
  contentType: string
  filename: string
  url?: string
  moderation?: UploadModeration
}

export type SafeUploadResult =
  | { ok: true, result: UploadResult }
  | { ok: false, error: { code: string, message: string, status?: number, data?: unknown } }

/**
 * Optional resolver for turning a bare job id into a media ref (a completed
 * generation used as an input). Injected so the media half stays independent
 * of the jobs half — wire it from a job client when you need it.
 */
export type ResolveJobRef = (id: string) => Promise<MediaRef | undefined>

/** Config for `createMediaClient`. */
export interface MediaClientConfig {
  /** The transport-agnostic media adapter. Use one from `@higgsfield/fnf-adapters`, or your own. */
  mediaAdapter: MediaBackend
  /** The byte plane for the presigned PUT. Defaults to `createFetchUploader()`. */
  blobUploader?: BinaryUploader
  /** Optional: resolve a job id to a media ref (e.g. from a job client) for `resolve`. */
```

```
  resolveJob?: ResolveJobRef
  observability?: FnfObservabilityOptions
}

/** The shared context every media operation consumes. Build with `createMediaContext`. */
export interface MediaContext {
  mediaAdapter: MediaBackend
  blobUploader: BinaryUploader
  resolveJob?: ResolveJobRef
  observability: FnfObservabilityContext
}

export interface MediaClient {
  get: (id: string, type: UploadType) => Promise<MediaRef>
  list: (opts: MediaListOptions) => Promise<MediaListResult>
  resolve: (refs: MediaReference[]) => Promise<MediaRef[]>
  /**
   * Upload bytes end-to-end (presign → PUT → confirm) → submit-ready MediaRef.
   * `source` must be real binary data, not a JSON-shaped object. Invalid sources
   * fail early with `invalid_media_source`.
   */
  upload: (input: UploadInput) => Promise<UploadResult>
  /** Download a remote URL then upload it — same options as `upload` (needs `BinaryUploader.fetchBytes`). */
  uploadFromUrl: (input: Omit<UploadInput, 'source' | 'contentType'> & { url: string, maxBytes?: number }) => Promise<UploadResult>
  /** Presign only — returns the upload slot (the client does the PUT itself). */
  getUploadUrl: (req: { type: UploadType, filename?: string, contentType?: string, extra?: Record<string, unknown> }) => Promise<UploadSlot>
  /** Confirm a previously-uploaded media id. */
  confirm: (req: { mediaId: string, type: UploadType, filename?: string, jobId?: string, forceIpCheck?: boolean, forceNsfwCheck?: boolean, startSeconds?: number, endSeconds?: number, role?: string, th
}
```

app/packages/fnf/src/media/upload.ts

```
import type { MediaRef } from '../types'
import type {
  BinaryUploader,
  MediaBytes,
  MediaContext,
  MediaSource,
  SafeUploadResult,
  UploadInput,
  UploadModeration,
  UploadResult,
  UploadSlot,
  UploadType,
} from './types'
import { ApiJobError, throwIfAborted } from '../errors'
import { observeAsync } from '../observability'
import { ConfirmError, InvalidMediaSourceError, MediaModerationError, PresignError, UploadNotSupportedError } from './errors'
import { defaultFilenameForContentType, inferContentType, inferUploadType } from './mime'
import { REF_TYPE_BY_UPLOAD } from './types'

/**
 * Full flow: presign → read bytes → PUT → confirm → submit-ready `UploadResult`.
 * `input.signal` cancels between steps (typed `JobAbortedError`) and aborts the PUT in flight.
 */
export async function uploadMedia(ctx: MediaContext, input: UploadInput): Promise<UploadResult> {
  return observeAsync(ctx.observability, 'fnf.media.upload', {
    media_type: input.type ?? 'auto',
    has_role: Boolean(input.role),
    force_ip_check: input.forceIpCheck === true,
    force_nsfw_check: input.forceNsfwCheck === true,
  }, async () => {
    throwIfAborted(input.signal)
    const bytes = await readSource(input.source)
    const contentType = contentTypeOf(input, bytes)
    const type = input.type ?? inferUploadType(contentType)
    const filename = input.filename ?? defaultFilenameForContentType(contentType, type)

    const slot = await getUploadUrl(ctx, { type, filename, contentType, extra: input.extra })
    throwIfAborted(input.signal)
    // PUT with the slot's content type (the backend's signed value when the
    // presign returned one, else the same local inference fed into it).
    await observeAsync(ctx.observability, 'fnf.media.transfer', {
      media_id: slot.mediaId,
      media_type: type,
      content_type: slot.contentType,
    }, () => transferBytes(ctx.blobUploader, slot, bytes, undefined, input.signal))
    throwIfAborted(input.signal)
    const result = await confirmMedia(ctx, {
      mediaId: slot.mediaId,
      type,
      filename,
      role: input.role,
      jobId: input.jobId,
      forceIpCheck: input.forceIpCheck,
      forceNsfwCheck: input.forceNsfwCheck,
      startSeconds: input.startSeconds,
      endSeconds: input.endSeconds,
      throwOnModeration: input.throwOnModeration,
      // The product sends its extra wire fields (surface) at confirm too.
      extra: input.extra,
    })
    // confirm doesn't know the slot's url/contentType; prefer the richer values.
    return {
      ...result,
      contentType,
      url: result.url ?? slot.url,
      ref: result.ref.url ? result.ref : { ...result.ref, ...(slot.url ? { url: slot.url } : {}) },
    }
  }, {
```

```
    successAttributes: result => ({
      media_id: result.mediaId,
      media_type: result.type,
      status: result.status,
      content_type: result.contentType,
      ...(result.moderation?.status ? { moderation_status: result.moderation.status } : {}),
    }),
  })
}

/** No-throw variant for Comlink/iframe boundaries (mirrors `safeSubmit`). */
export async function safeUploadMedia(ctx: MediaContext, input: UploadInput): Promise<SafeUploadResult> {
  try {
    return { ok: true, result: await uploadMedia(ctx, input) }
  }
  catch (err) {
    if (err instanceof ApiJobError)
      return { ok: false, error: err.toJSON() }
    return { ok: false, error: { code: 'unexpected', message: err instanceof Error ? err.message : String(err) } }
  }
}

/** The `uploadMedia` options, minus the byte source, plus the URL to ingest. */
export type UploadFromUrlInput = Omit<UploadInput, 'source' | 'contentType'> & { url: string, maxBytes?: number }

/**
 * Download a remote URL then upload it — same options as `uploadMedia`
 *  (role/jobId/moderation flags pass through). Needs `BinaryUploader.fetchBytes`.
 */
export async function uploadMediaFromUrl(ctx: MediaContext, input: UploadFromUrlInput): Promise<UploadResult> {
  return observeAsync(ctx.observability, 'fnf.media.upload_from_url', {
    ...(input.maxBytes !== undefined ? { max_bytes: input.maxBytes } : {}),
    media_type: input.type ?? 'auto',
  }, async () => {
    if (!ctx.blobUploader.fetchBytes)
      throw new UploadNotSupportedError()
    throwIfAborted(input.signal)
    const { url, maxBytes, ...rest } = input
    const { bytes, contentType } = await ctx.blobUploader.fetchBytes(url, { maxBytes, signal: input.signal })
    const filename = rest.filename ?? url.split('/').pop()?.split('?')[0]
    return uploadMedia(ctx, { ...rest, source: bytes, contentType, filename })
  }, {
    successAttributes: result => ({ media_id: result.mediaId, media_type: result.type, status: result.status }),
  })
}

// ■■ the individual steps (exported for callers that drive the flow themselves) ■■

/** Step 1 — presign. Returns the upload slot (the bytes are PUT to `slot.uploadUrl`). */
export async function getUploadUrl(
  ctx: MediaContext,
  req: { type: UploadType, filename?: string, contentType?: string, extra?: Record<string, unknown> },
): Promise<UploadSlot> {
  return observeAsync(ctx.observability, 'fnf.media.presign', {
    media_type: req.type,
    ...(req.contentType ? { content_type: req.contentType } : {}),
  }, async () => {
    if (!ctx.mediaAdapter.getUploadUrl)
      throw new UploadNotSupportedError()
    let raw: Record<string, unknown>
    try {
      raw = (await ctx.mediaAdapter.getUploadUrl(req) ?? {}) as Record<string, unknown>
    }
    catch (err) {
      // Adapters map their own HTTP failures to typed errors; a foreign throw
      // (custom adapter, socket error) still has to surface as the typed step
      // failure — same contract as confirmMedia below.
      if (err instanceof ApiJobError)
        throw err
      throw new PresignError(err instanceof Error ? err.message : String(err))
```

```
    }
    const mediaId = str(raw.id) ?? str(raw.media_id)
    const uploadUrl = str(raw.upload_url) ?? str(raw.uploadUrl)
    if (!mediaId || !uploadUrl)
      throw new PresignError()
    return {
      mediaId,
      uploadUrl,
      url: str(raw.url),
      type: req.type,
      // The backend's signed Content-Type wins — the product PUTs audio/video
      // with the presign response's content_type; a locally-inferred variant
      // (m4a/x-m4a…) could fail the presigned-URL signature.
      contentType: str(raw.content_type) ?? inferContentType(req.filename, req.contentType),
      method: 'PUT',
    }
  }, {
    successAttributes: slot => ({ media_id: slot.mediaId, media_type: slot.type, content_type: slot.contentType }),
  })
}

/** Step 2 — the raw binary PUT, via the injected uploader. */
export async function transferBytes(uploader: BinaryUploader, slot: UploadSlot, bytes: MediaBytes, contentType?: string, signal?: AbortSignal): Promise<void> {
  await uploader.transfer({ uploadUrl: slot.uploadUrl, bytes, contentType: contentType ?? slot.contentType, signal })
}

/** Step 3 — confirm. Normalizes to an `UploadResult`; throws on moderation block. */
export async function confirmMedia(
  ctx: MediaContext,
  req: { mediaId: string, type: UploadType, filename?: string, jobId?: string, forceIpCheck?: boolean, forceNsfwCheck?: boolean, startSeconds?: number, endSeconds?: number, role?: string, throwOnModer
): Promise<UploadResult> {
  return observeAsync(ctx.observability, 'fnf.media.confirm', {
    media_id: req.mediaId,
    media_type: req.type,
    force_ip_check: req.forceIpCheck === true,
    force_nsfw_check: req.forceNsfwCheck === true,
  }, async () => {
    if (!ctx.mediaAdapter.confirmMedia)
      throw new UploadNotSupportedError()
    let raw: Record<string, unknown> = {}
    let blockedCode: string | undefined
    try {
      raw = (await ctx.mediaAdapter.confirmMedia(req) ?? {}) as Record<string, unknown>
    }
    catch (err) {
      // The backend signals a blocked upload in TWO shapes: a 2xx body with a
      // blocked status (handled below) and an HTTP 422 with the same verdict as
      // detail.error_type. throwOnModeration must not depend on which shape the
      // backend picked, so the 422 twin honors the opt-out too.
      if (err instanceof ApiJobError) {
        if (req.throwOnModeration === false && (err.code === 'ip_detected' || err.code === 'ip_check_rate_limit_reached'))
          blockedCode = err.code
        else throw err
      }
      else {
        throw new ConfirmError(err instanceof Error ? err.message : String(err))
      }
    }

    const moderation = moderationOf(raw) ?? (blockedCode ? { status: blockedCode } : undefined)
    const blocked = moderation && isBlockedModeration(moderation.status)
    if (blocked && req.throwOnModeration !== false)
      throw new MediaModerationError(moderation.status)

    const id = str(raw.id) ?? req.mediaId
    const url = str(raw.url) ?? str(raw.result_url)
    // The ref's `type` is the product's input-media discriminator (it goes on the
    // job wire verbatim, e.g. seedance medias[].data.type) — one per upload plane,
    // mirroring fnf-web's input-media model. Derived from the request, not the
    // response echo, so non-fnf adapters can't leak their own type strings.
```

```
    const ref: MediaRef = { id, type: REF_TYPE_BY_UPLOAD[req.type], ...(url ? { url } : {}), ...(req.role ? { role: req.role } : {}) }
    return {
      ref,
      mediaId: id,
      status: moderation?.status ?? str(raw.status) ?? 'uploaded',
      type: req.type,
      contentType: inferContentType(req.filename, undefined),
      filename: req.filename ?? defaultFilenameForContentType(inferContentType(req.filename, undefined), req.type),
      ...(url ? { url } : {}),
      ...(moderation ? { moderation } : {}),
    }
  }, {
    successAttributes: result => ({
      media_id: result.mediaId,
      media_type: result.type,
      status: result.status,
      ...(result.moderation?.status ? { moderation_status: result.moderation.status } : {}),
    }),
  })
}

/** The verdicts that block an upload (vs informational moderation statuses). */
export function isBlockedModeration(status: string): boolean {
  return status === 'ip_detected' || status === 'nsfw' || status === 'ip_check_rate_limit_reached'
}

// ■■ helpers ■■

const MODERATION_STATUSES = new Set(['ip_detected', 'nsfw', 'ip_check_rate_limit_reached', 'not_ready'])

function moderationOf(raw: Record<string, unknown>): UploadModeration | undefined {
  const status = str(raw.status)
  const ipCheckFinished = bool(raw.ip_check_finished)
  const ipDetected = bool(raw.ip_detected)
  // Only surface `moderation` when the confirm actually carries IP/NSFW signal —
  // a plain `uploaded` status is the upload state, not a moderation verdict.
  if ((status == null || !MODERATION_STATUSES.has(status)) && ipCheckFinished === undefined && ipDetected === undefined)
    return undefined
  return { status: status ?? 'not_ready', ...(ipCheckFinished !== undefined ? { ipCheckFinished } : {}), ...(ipDetected !== undefined ? { ipDetected } : {}) }
}

async function readSource(source: MediaSource): Promise<MediaBytes> {
  const bytes = typeof source === 'object' && source !== null && 'read' in source ? await source.read() : source
  if (!isMediaBytes(bytes))
    throw new InvalidMediaSourceError()
  return bytes
}

function contentTypeOf(input: UploadInput, bytes?: MediaBytes): string {
  const fromBlob = typeof Blob !== 'undefined' && bytes instanceof Blob && bytes.type ? bytes.type : undefined
  return inferContentType(input.filename, input.contentType ?? fromBlob)
}

function isMediaBytes(value: unknown): value is MediaBytes {
  return (
    value instanceof Uint8Array
    || value instanceof ArrayBuffer
    || (typeof Blob !== 'undefined' && value instanceof Blob)
  )
}

function str(v: unknown): string | undefined {
  return typeof v === 'string' ? v : undefined
}

function bool(v: unknown): boolean | null | undefined {
  return typeof v === 'boolean' ? v : v === null ? null : undefined
}
```

app/packages/fnf/src/normalize.ts

```
import type { Normalize } from './z'

/** A settings value the SDK snapped during normalization (e.g. 16:10 → 16:9). */
export interface Adjustment {
  field: string
  from: unknown
  to: unknown
}

/** Opt-in normalization kinds, requested per-submit via `adjust: [...]`. Off by default. */
export type AdjustKind = 'near-aspect-ratio' | 'near-duration'

const ADJUST_BY_KIND: Record<Normalize['kind'], AdjustKind> = {
  aspectRatio: 'near-aspect-ratio',
  duration: 'near-duration',
}

function ratio(value: string): number | null {
  const parts = value.split(':')
  if (parts.length !== 2)
    return null
  const w = Number(parts[0])
  const h = Number(parts[1])
  if (!Number.isFinite(w) || !Number.isFinite(h) || h === 0)
    return null
  return w / h
}

export function closestAspectRatio(value: string, options: readonly string[]): string {
  if (options.includes(value))
    return value
  const target = ratio(value)
  if (target == null)
    return options[0] ?? value
  let best = options[0] ?? value
  let bestDist = Infinity
  for (const option of options) {
    const r = ratio(option)
    if (r == null)
      continue
    const dist = Math.abs(Math.log(r) - Math.log(target))
    if (dist < bestDist) {
      bestDist = dist
      best = option
    }
  }
  return best
}

export function clampDuration(value: number, n: Extract<Normalize, { kind: 'duration' }>): number {
  if (n.values && n.values.length > 0) {
    let best = n.values[0]
    let bestDist = Infinity
    for (const candidate of n.values) {
      const dist = Math.abs(candidate - value)
      if (dist < bestDist) {
        bestDist = dist
        best = candidate
      }
    }
    return best
  }
  let out = value
  if (n.min !== undefined)
    out = Math.max(n.min, out)
  if (n.max !== undefined)
    out = Math.min(n.max, out)
  return out
```

```
}

export function normalizeSettings(
  settings: Record<string, unknown>,
  normalizers: Record<string, Normalize>,
  enabled: ReadonlySet<AdjustKind>,
): { settings: Record<string, unknown>, adjustments: Adjustment[] } {
  const out: Record<string, unknown> = { ...settings }
  const adjustments: Adjustment[] = []
  for (const [key, n] of Object.entries(normalizers)) {
    // Opt-in per kind: a normalizer runs only when its `adjust` name was requested.
    if (!enabled.has(ADJUST_BY_KIND[n.kind]))
      continue
    const value = out[key]
    if (value == null)
      continue
    const next = n.kind === 'aspectRatio'
      ? closestAspectRatio(String(value), n.options)
      : clampDuration(Number(value), n)
    if (next !== value)
      adjustments.push({ field: key, from: value, to: next })
    out[key] = next
  }
  return { settings: out, adjustments }
}
```

app/packages/fnf/src/observability/__tests__/observability.test.ts

```
import type { GenerationBackend, MediaBackend, ProfileBackend } from '../../backend'
import type { BinaryUploader } from '../../media'
import type { FnfObservationEvent } from '../index'
import { describe, expect, it, vi } from 'vitest'
import { createJobClient } from '../../client'
import { defineJob } from '../../define-job'
import { ApiJobError } from '../../errors'
import { createMediaClient } from '../../media'
import { createProfileClient } from '../../profile'
import { z } from '../../z'
import { composeObservers, createNoopObserver, withObservedGenerationBackend, withObservedTransport, withObservedUploader } from '../index'

const demo = defineJob({
  jobSetType: 'demo',
  outputType: 'image',
  params: { prompt: true, settings: { aspectRatio: z.aspectRatio(['1:1']) } },
})

// Inline port stubs — this suite tests the observability layer, which must not
// depend on concrete adapters (those live in @higgsfield/fnf-adapters).
function stubBackend(cost = 1): GenerationBackend {
  return {
    createJobs: async ({ jobSetType }) => [{ id: 'job-1', job_set_type: jobSetType, status: 'completed', result_url: 'memory://out' }],
    getJob: async id => ({ id, status: 'completed' }),
    listJobs: async () => ({ items: [] }),
    estimateCost: async () => ({ credits: cost }),
  }
}

function recorder() {
  const events: FnfObservationEvent[] = []
  return {
    events,
    observer: (event: FnfObservationEvent) => {
      events.push(event)
    },
  }
}

describe('fnf observability', () => {
  it('emits job lifecycle spans without changing client behavior', async () => {
    const rec = recorder()
    const client = createJobClient({
      adapter: stubBackend(7),
      jobs: [demo],
      observability: { observer: rec.observer, traceId: 'trace-1' },
    })

    const submitted = await client.submit({ model: 'demo', prompt: { instruction: 'secret prompt' }, settings: { aspectRatio: '1:1' } })
    await client.cost({ model: 'demo', prompt: { instruction: 'secret prompt' }, settings: { aspectRatio: '1:1' } })

    expect(submitted.generations).toHaveLength(1)
    expect(rec.events.map(e => `${e.name}:${e.phase}`)).toContain('fnf.job.submit:start')
    expect(rec.events.map(e => `${e.name}:${e.phase}`)).toContain('fnf.job.submit:success')
    expect(rec.events.find(e => e.name === 'fnf.job.submit' && e.phase === 'success')?.attributes).toMatchObject({ generation_count: 1 })
    expect(JSON.stringify(rec.events)).not.toContain('secret prompt')
  })

  it('emits typed error metadata for SDK failures', async () => {
    const rec = recorder()
    const client = createJobClient({
      adapter: stubBackend(),
      jobs: [demo],
      observability: { observer: rec.observer },
    })

    await expect(client.submit({ model: 'missing_model' } as never)).rejects.toBeInstanceOf(ApiJobError)
```

```
  const error = rec.events.find(e => e.name === 'fnf.job.submit' && e.phase === 'error')
  expect(error?.error).toMatchObject({ code: 'unknown_model' })
})

it('observes transport requests with sanitized paths and safe status metadata', async () => {
  const rec = recorder()
  const transport = withObservedTransport(
    async () => ({ status: 201, body: { ok: true } }),
    { observer: rec.observer },
  )

  await transport({ method: 'POST', path: '/jobs/topaz-image?dry_run=true&token=secret', body: { prompt: 'secret' } })

  const success = rec.events.find(e => e.name === 'fnf.transport.request' && e.phase === 'success')
  expect(success?.attributes).toMatchObject({ method: 'POST', path: '/jobs/topaz-image?dry_run&token', status: 201 })
  expect(JSON.stringify(rec.events)).not.toContain('secret')
})

it('media upload spans do not expose upload URLs, filenames, bytes, or result URLs', async () => {
  const rec = recorder()
  const mediaBackend: MediaBackend = {
    getMedia: async () => ({}),
    listMedia: async () => ({ items: [] }),
    getUploadUrl: async () => ({ id: 'm1', url: 'https://cdn/private.png', upload_url: 'https://s3/private-put' }),
    confirmMedia: async () => ({ id: 'm1', status: 'uploaded', url: 'https://cdn/private.png' }),
  }
  const media = createMediaClient({
    mediaAdapter: mediaBackend,
    blobUploader: { transfer: async () => {} },
    observability: { observer: rec.observer },
  })

  await media.upload({ source: new Uint8Array([1, 2, 3]), filename: 'private-cat.png', role: 'image' })

  const text = JSON.stringify(rec.events)
  expect(rec.events.map(e => e.name)).toEqual(expect.arrayContaining(['fnf.media.upload', 'fnf.media.presign', 'fnf.media.transfer', 'fnf.media.confirm']))
  expect(text).not.toContain('private-cat.png')
  expect(text).not.toContain('https://s3/private-put')
  expect(text).not.toContain('https://cdn/private.png')
  expect(text).not.toContain('[1,2,3]')
})

it('profile spans expose only coarse metadata and ids', async () => {
  const rec = recorder()
  const workspaces = [{ id: 'w1', name: 'Secret Team', type: 'private', user_role: 'owner' }]
  const profileBackend: ProfileBackend = {
    getUser: async () => ({ id: 'u1', email: 'private@example.com', workspace_id: 'w1' }),
    listWorkspaces: async () => workspaces,
    getCurrentWorkspace: async () => workspaces[0],
    getWorkspaceWallet: async () => ({ subscription_balance: 0 }),
    switchWorkspace: async () => ({}),
  }
  const profile = createProfileClient({
    profileAdapter: profileBackend,
    observability: { observer: rec.observer },
  })

  await profile.getSnapshot()

  const snapshot = rec.events.find(e => e.name === 'fnf.profile.get_snapshot' && e.phase === 'success')
  expect(snapshot?.attributes).toMatchObject({ has_user: true, workspace_count: 1, has_current_workspace: true })
  expect(JSON.stringify(rec.events)).not.toContain('private@example.com')
  expect(JSON.stringify(rec.events)).not.toContain('Secret Team')
})

it('wrappers preserve behavior and observer failures do not affect operations', async () => {
  const observer = vi.fn(() => {
    throw new Error('observer exploded')
  })
  const onObserverError = vi.fn()
```

```
    const backend = withObservedGenerationBackend(stubBackend(), { observer, onObserverError })
    const client = createJobClient({ adapter: backend, jobs: [demo] })

    await expect(client.submit({ model: 'demo', prompt: { instruction: 'x' }, settings: { aspectRatio: '1:1' } })).resolves.toHaveProperty('generations')
    expect(onObserverError).toHaveBeenCalled()
  })

  it('compose/noop observers and uploader wrappers are safe defaults', async () => {
    const rec = recorder()
    const observer = composeObservers(createNoopObserver(), rec.observer)
    const uploader: BinaryUploader = withObservedUploader({
      transfer: async () => {},
      fetchBytes: async () => ({ bytes: new Uint8Array([1]), contentType: 'image/png' }),
    }, { observer })

    await uploader.transfer({ uploadUrl: 'https://s3/private', bytes: new Uint8Array([1]), contentType: 'image/png' })
    await uploader.fetchBytes?.('https://cdn/private.png')

    expect(rec.events.map(e => e.name)).toEqual(['fnf.media.transfer', 'fnf.media.transfer', 'fnf.media.fetch_bytes', 'fnf.media.fetch_bytes'])
    expect(JSON.stringify(rec.events)).not.toContain('https://s3/private')
    expect(JSON.stringify(rec.events)).not.toContain('https://cdn/private.png')
  })

  it('withObservedTransport preserves error behavior', async () => {
    const rec = recorder()
    const transport = withObservedTransport(async () => {
      throw new ApiJobError('network', 'offline')
    }, { observer: rec.observer })

    await expect(transport({ method: 'GET', path: '/health' })).rejects.toMatchObject({ code: 'network' })
    expect(rec.events.find(e => e.phase === 'error')?.error).toMatchObject({ code: 'network' })
  })
})
```

app/packages/fnf/src/observability/index.ts

```
import type { GenerationBackend, MediaBackend, ProfileBackend } from '../backend'
import type { BinaryUploader } from '../media/types'
import type { Transport } from '../transport'
import { ApiJobError } from '../errors'

export type FnfObservationPhase = 'start' | 'success' | 'error' | 'event'
export type FnfObservationAttributeValue = string | number | boolean | null
export type FnfObservationAttributes = Record<string, FnfObservationAttributeValue>

export interface FnfObservationError {
  code: string
  status?: number
  name?: string
}

export interface FnfObservationEvent {
  id: string
  traceId: string
  parentId?: string
  name: string
  phase: FnfObservationPhase
  timestamp: number
  durationMs?: number
  attributes: FnfObservationAttributes
  error?: FnfObservationError
}

export type FnfObserver = (event: FnfObservationEvent) => void | Promise<void>

export interface FnfObservabilityOptions {
  observer?: FnfObserver
  traceId?: string
  parentId?: string
  attributes?: FnfObservationAttributes | (() => FnfObservationAttributes)
  onObserverError?: (error: unknown, event: FnfObservationEvent) => void
  now?: () => number
  idFactory?: () => string
}

export interface FnfObservabilityContext {
  observer?: FnfObserver
  traceId: string
  parentId?: string
  attributes: () => FnfObservationAttributes
  onObserverError?: (error: unknown, event: FnfObservationEvent) => void
  now: () => number
  idFactory: () => string
}

export interface ObserveAsyncOptions<T> {
  parentId?: string
  attributes?: FnfObservationAttributes
  successAttributes?: (result: T) => FnfObservationAttributes
  errorAttributes?: (error: unknown) => FnfObservationAttributes
}

let fallbackSeq = 0

export function createNoopObserver(): FnfObserver {
  return () => {}
}

export function composeObservers(...observers: Array<FnfObserver | undefined | null | false>): FnfObserver {
  const live = observers.filter(Boolean) as FnfObserver[]
  return (event) => {
    for (const observer of live) {
      const result = observer(event)
      if (isPromiseLike(result))
```

```
        void result.catch(() => {})
    }
  }
}

export function createConsoleObserver(consoleLike: Pick<Console, 'debug' | 'error'> = console): FnfObserver {
  return (event) => {
    const log = event.phase === 'error' ? consoleLike.error : consoleLike.debug
    log.call(consoleLike, `[${event.name}] ${event.phase}`, event)
  }
}

export function createObservabilityContext(options?: FnfObservabilityOptions): FnfObservabilityContext {
  const idFactory = options?.idFactory ?? defaultId
  return {
    ...(options?.observer ? { observer: options.observer } : {}),
    traceId: options?.traceId ?? idFactory(),
    ...(options?.parentId ? { parentId: options.parentId } : {}),
    attributes: () => normalizeAttributes(typeof options?.attributes === 'function'
      ? safeReadAttributes(options.attributes)
      : options?.attributes),
    ...(options?.onObserverError ? { onObserverError: options.onObserverError } : {}),
    now: options?.now ?? Date.now,
    idFactory,
  }
}

export function observeEvent(
  ctx: FnfObservabilityContext | FnfObservabilityOptions | undefined,
  name: string,
  attributes: FnfObservationAttributes = {},
  options: { parentId?: string, phase?: FnfObservationPhase, error?: unknown, durationMs?: number } = {},
): void {
  const obs = asContext(ctx)
  if (!obs.observer)
    return
  const event = makeEvent(obs, {
    name,
    phase: options.phase ?? 'event',
    attributes,
    ...(options.parentId ? { parentId: options.parentId } : {}),
    ...(options.durationMs !== undefined ? { durationMs: options.durationMs } : {}),
    ...(options.error !== undefined ? { error: errorInfo(options.error) } : {}),
  })
  emit(obs, event)
}

export async function observeAsync<T>(
  ctx: FnfObservabilityContext | FnfObservabilityOptions | undefined,
  name: string,
  attributes: FnfObservationAttributes,
  fn: () => Promise<T>,
  options: ObserveAsyncOptions<T> = {},
): Promise<T> {
  const obs = asContext(ctx)
  if (!obs.observer)
    return fn()

  const id = obs.idFactory()
  const startedAt = obs.now()
  const base = { ...attributes, ...options.attributes }
  emit(obs, makeEvent(obs, {
    id,
    name,
    phase: 'start',
    attributes: base,
    ...(options.parentId ? { parentId: options.parentId } : {}),
  }))
  try {
    const result = await fn()
```

```
    emit(obs, makeEvent(obs, {
      id,
      name,
      phase: 'success',
      durationMs: Math.max(0, obs.now() - startedAt),
      attributes: { ...base, ...safeReadAttributes(() => options.successAttributes?.(result) ?? {}) },
      ...(options.parentId ? { parentId: options.parentId } : {}),
    }))
    return result
  }
  catch (error) {
    emit(obs, makeEvent(obs, {
      id,
      name,
      phase: 'error',
      durationMs: Math.max(0, obs.now() - startedAt),
      attributes: { ...base, ...safeReadAttributes(() => options.errorAttributes?.(error) ?? {}) },
      error: errorInfo(error),
      ...(options.parentId ? { parentId: options.parentId } : {}),
    }))
    throw error
  }
}

export function withObservedTransport(transport: Transport, observability?: FnfObservabilityOptions | FnfObservabilityContext): Transport {
  const ctx = asContext(observability)
  return req => observeAsync(
    ctx,
    'fnf.transport.request',
    {
      method: req.method,
      path: sanitizePath(req.path),
    },
    () => transport(req),
    {
      successAttributes: res => ({ status: res.status }),
      errorAttributes: (error) => {
        const info = errorInfo(error)
        return {
          ...(info.status !== undefined ? { status: info.status } : {}),
          error_code: info.code,
        }
      },
    },
  )
}

export function withObservedGenerationBackend(backend: GenerationBackend, observability?: FnfObservabilityOptions | FnfObservabilityContext): GenerationBackend {
  const ctx = asContext(observability)
  return {
    createJobs: req => observeAsync(ctx, 'fnf.backend.generation.create_jobs', { job_set_type: req.jobSetType }, () => backend.createJobs(req)),
    getJob: id => observeAsync(ctx, 'fnf.backend.generation.get_job', { generation_id: id }, () => backend.getJob(id)),
    ...(backend.getJobSet ? { getJobSet: id => observeAsync(ctx, 'fnf.backend.generation.get_job_set', { job_set_id: id }, () => backend.getJobSet!(id)) } : {}),
    listJobs: query => observeAsync(ctx, 'fnf.backend.generation.list_jobs', listQueryAttributes(query), () => backend.listJobs(query)),
    estimateCost: req => observeAsync(ctx, 'fnf.backend.generation.estimate_cost', { job_set_type: req.jobSetType }, () => backend.estimateCost(req)),
    ...(backend.cancelJob ? { cancelJob: id => observeAsync(ctx, 'fnf.backend.generation.cancel_job', { generation_id: id }, () => backend.cancelJob!(id)) } : {}),
    // Attributes carry the job type only — never the params or the resolved
    // confirmation token (privacy rule: tokens don't go into observability).
    ...(backend.confirm ? { confirm: req => observeAsync(ctx, 'fnf.backend.generation.confirm', { job_set_type: req.jobSetType }, () => backend.confirm!(req)) } : {}),
  }
}

export function withObservedMediaBackend(backend: MediaBackend, observability?: FnfObservabilityOptions | FnfObservabilityContext): MediaBackend {
  const ctx = asContext(observability)
  return {
    getMedia: query => observeAsync(ctx, 'fnf.backend.media.get_media', { media_id: query.id, media_type: query.type }, () => backend.getMedia(query)),
    listMedia: query => observeAsync(ctx, 'fnf.backend.media.list_media', { media_type: query.type, ...(query.size !== undefined ? { size: query.size } : {}) }, () => backend.listMedia(query)),
    ...(backend.getUploadUrl
      ? {
          getUploadUrl: req => observeAsync(ctx, 'fnf.backend.media.get_upload_url', {
```

```
            media_type: req.type,
            ...(req.contentType ? { content_type: req.contentType } : {}),
          }, () => backend.getUploadUrl!(req)),
        }
      : {}),
    ...(backend.confirmMedia
      ? {
          confirmMedia: req => observeAsync(ctx, 'fnf.backend.media.confirm_media', {
            media_id: req.mediaId,
            media_type: req.type,
          }, () => backend.confirmMedia!(req)),
        }
      : {}),
  }
}

export function withObservedProfileBackend(backend: ProfileBackend, observability?: FnfObservabilityOptions | FnfObservabilityContext): ProfileBackend {
  const ctx = asContext(observability)
  return {
    getUser: () => observeAsync(ctx, 'fnf.backend.profile.get_user', {}, () => backend.getUser()),
    listWorkspaces: () => observeAsync(ctx, 'fnf.backend.profile.list_workspaces', {}, () => backend.listWorkspaces()),
    getCurrentWorkspace: () => observeAsync(ctx, 'fnf.backend.profile.get_current_workspace', {}, () => backend.getCurrentWorkspace()),
    getWorkspaceWallet: () => observeAsync(ctx, 'fnf.backend.profile.get_workspace_wallet', {}, () => backend.getWorkspaceWallet()),
    switchWorkspace: req => observeAsync(ctx, 'fnf.backend.profile.switch_workspace', { workspace_id: req.workspaceId }, () => backend.switchWorkspace(req)),
  }
}

export function withObservedUploader(uploader: BinaryUploader, observability?: FnfObservabilityOptions | FnfObservabilityContext): BinaryUploader {
  const ctx = asContext(observability)
  return {
    transfer: args => observeAsync(ctx, 'fnf.media.transfer', {
      content_type: args.contentType,
    }, () => uploader.transfer(args)),
    ...(uploader.fetchBytes
      ? {
          fetchBytes: (url, opts) => observeAsync(ctx, 'fnf.media.fetch_bytes', {
            ...(opts?.maxBytes !== undefined ? { max_bytes: opts.maxBytes } : {}),
          }, () => uploader.fetchBytes!(url, opts), {
            successAttributes: result => ({ content_type: result.contentType }),
          }),
        }
      : {}),
  }
}

function asContext(ctx: FnfObservabilityContext | FnfObservabilityOptions | undefined): FnfObservabilityContext {
  if (isObservabilityContext(ctx))
    return ctx
  return createObservabilityContext(ctx)
}

function isObservabilityContext(value: unknown): value is FnfObservabilityContext {
  return typeof value === 'object'
    && value !== null
    && 'traceId' in value
    && 'idFactory' in value
    && 'now' in value
    && 'attributes' in value
}

function makeEvent(
  ctx: FnfObservabilityContext,
  event: {
    id?: string
    name: string
    phase: FnfObservationPhase
    attributes?: FnfObservationAttributes
    parentId?: string
    durationMs?: number
    error?: FnfObservationError
```

```
  },
): FnfObservationEvent {
  return {
    id: event.id ?? ctx.idFactory(),
    traceId: ctx.traceId,
    ...(event.parentId ?? ctx.parentId ? { parentId: event.parentId ?? ctx.parentId } : {}),
    name: event.name,
    phase: event.phase,
    timestamp: ctx.now(),
    ...(event.durationMs !== undefined ? { durationMs: event.durationMs } : {}),
    attributes: normalizeAttributes({ ...ctx.attributes(), ...event.attributes }),
    ...(event.error ? { error: event.error } : {}),
  }
}

function emit(ctx: FnfObservabilityContext, event: FnfObservationEvent): void {
  if (!ctx.observer)
    return
  try {
    const result = ctx.observer(event)
    if (isPromiseLike(result)) {
      void result.catch((error: unknown) => {
        ctx.onObserverError?.(error, event)
      })
    }
  }
  catch (error) {
    ctx.onObserverError?.(error, event)
  }
}

function errorInfo(error: unknown): FnfObservationError {
  if (error instanceof ApiJobError) {
    return {
      code: error.code,
      ...(error.status !== undefined ? { status: error.status } : {}),
      name: error.name,
    }
  }
  if (error instanceof Error)
    return { code: 'unexpected', name: error.name }
  return { code: 'unexpected' }
}

function normalizeAttributes(attrs?: Record<string, unknown>): FnfObservationAttributes {
  const out: FnfObservationAttributes = {}
  for (const [key, value] of Object.entries(attrs ?? {})) {
    if (typeof value === 'string' || typeof value === 'number' || typeof value === 'boolean' || value === null)
      out[key] = value
  }
  return out
}

function safeReadAttributes(read: () => FnfObservationAttributes): FnfObservationAttributes {
  try {
    return read()
  }
  catch {
    return {}
  }
}

function sanitizePath(path: string): string {
  const [pathname, query] = path.split('?', 2)
  if (!query)
    return pathname
  const keys = [...new URLSearchParams(query).keys()]
  return keys.length > 0 ? `${pathname}?${keys.join('&')}` : pathname
}
```

```
function listQueryAttributes(query: { type?: string, size?: number, cursor?: string | number, parentId?: string, status?: string | string[], model?: string | string[] }): FnfObservationAttributes {
  return {
    ...(query.type ? { type: query.type } : {}),
    ...(query.size !== undefined ? { size: query.size } : {}),
    ...(query.cursor !== undefined ? { has_cursor: true } : {}),
    ...(query.parentId ? { parent_id: query.parentId } : {}),
    ...(query.status !== undefined ? { status_count: Array.isArray(query.status) ? query.status.length : 1 } : {}),
    ...(query.model !== undefined ? { model_count: Array.isArray(query.model) ? query.model.length : 1 } : {}),
  }
}

function defaultId(): string {
  const crypto = globalThis.crypto as { randomUUID?: () => string } | undefined
  if (crypto?.randomUUID)
    return crypto.randomUUID()
  fallbackSeq += 1
  return `fnf-obs-${Date.now().toString(36)}-${fallbackSeq.toString(36)}`
}

function isPromiseLike(value: unknown): value is Promise<void> {
  return typeof value === 'object' && value !== null && 'then' in value && typeof (value as { then?: unknown }).then === 'function'
}
```

app/packages/fnf/src/profile/client.ts

```
import type { ProfileClient, ProfileClientConfig } from './types'
import { createProfileContext } from './context'
import { getCurrentProfileWorkspace, getProfileCredits, getProfileSnapshot, getProfileUser, getProfileWallet, listProfileWorkspaces } from './get'
import { switchProfileWorkspace } from './switch'

/** Compose profile operations into a client. */
export function createProfileClient(config: ProfileClientConfig): ProfileClient {
  const ctx = createProfileContext(config)
  return {
    getUser: () => getProfileUser(ctx),
    listWorkspaces: () => listProfileWorkspaces(ctx),
    getCurrentWorkspace: () => getCurrentProfileWorkspace(ctx),
    getWallet: () => getProfileWallet(ctx),
    getCredits: options => getProfileCredits(ctx, options),
    getSnapshot: options => getProfileSnapshot(ctx, options),
    switchWorkspace: input => switchProfileWorkspace(ctx, input),
  }
}
```

app/packages/fnf/src/profile/context.ts

```
import type { ProfileClientConfig, ProfileContext } from './types'
import { createObservabilityContext } from '../observability'

/** Resolve profile config into the shared context every profile operation consumes. */
export function createProfileContext(config: ProfileClientConfig): ProfileContext {
  return {
    profileAdapter: config.profileAdapter,
    observability: createObservabilityContext(config.observability),
  }
}
```

app/packages/fnf/src/profile/credits.ts

```
import type { ProfileCredits, ProfileCreditsOptions, ProfileWorkspaceWallet } from './types'

export function calculateProfileCredits(
  wallet: ProfileWorkspaceWallet,
  options: ProfileCreditsOptions = {},
): ProfileCredits {
  const includeOnDemand = options.includeOnDemand ?? true

  const rawMonthlyRemaining = Math.max(0, wallet.subscriptionBalance)
  const rawPurchasedCredits = Math.max(0, wallet.creditsBalance)
  const rawOnDemandCredits = includeOnDemand ? Math.max(0, wallet.onDemandCredits) : 0
  const rawPlanCapacity = Math.max(0, wallet.subscriptionTotal)
  const rawAvailableCredits = rawMonthlyRemaining + rawPurchasedCredits
  const rawTotalAvailableCredits = rawAvailableCredits + rawOnDemandCredits
  const rawTotalProgressCapacity = rawPlanCapacity + rawOnDemandCredits
  const rawUsedCredits = Math.max(0, rawPlanCapacity - rawMonthlyRemaining)

  let availablePercent = 0
  if (rawTotalProgressCapacity > 0) {
    availablePercent = (rawTotalAvailableCredits / rawTotalProgressCapacity) * 100
  }
  else if (rawTotalAvailableCredits > 0) {
    availablePercent = 100
  }

  if (Number.isNaN(availablePercent))
    availablePercent = 0
  availablePercent = Math.round(Math.min(100, Math.max(0, availablePercent)))

  const raw = {
    availableCredits: rawAvailableCredits,
    usedCredits: rawUsedCredits,
    monthlyRemaining: rawMonthlyRemaining,
    purchasedCredits: rawPurchasedCredits,
    onDemandCredits: rawOnDemandCredits,
    totalAvailableCredits: rawTotalAvailableCredits,
    totalProgressCapacity: rawTotalProgressCapacity,
    planCapacity: rawPlanCapacity,
  }

  return {
    availablePercent,
    usagePercent: 100 - availablePercent,
    availableCredits: rawAvailableCredits / 100,
    totalAvailableCredits: rawTotalAvailableCredits / 100,
    totalProgressCapacity: rawTotalProgressCapacity / 100,
    usedCredits: rawUsedCredits / 100,
    monthlyRemaining: rawMonthlyRemaining / 100,
    purchasedCredits: rawPurchasedCredits / 100,
    onDemandCredits: rawOnDemandCredits / 100,
    planCapacity: rawPlanCapacity / 100,
    raw,
  }
}
```

app/packages/fnf/src/profile/get.ts

```
import type { ProfileContext, ProfileCredits, ProfileCreditsOptions, ProfileSnapshot, ProfileUser, ProfileWorkspace, ProfileWorkspaceWallet } from './types'
import { observeAsync } from '../observability'
import { calculateProfileCredits } from './credits'
import { mapProfileUser, mapProfileWallet, mapProfileWorkspace, mapProfileWorkspaces } from './mappers'

export async function getProfileUser(ctx: ProfileContext): Promise<ProfileUser | null> {
  return observeAsync(ctx.observability, 'fnf.profile.get_user', {}, async () => mapProfileUser(await ctx.profileAdapter.getUser()), {
    successAttributes: user => ({ has_user: user !== null, ...(user?.id ? { user_id: user.id } : {}) }),
  })
}

export async function listProfileWorkspaces(ctx: ProfileContext): Promise<ProfileWorkspace[]> {
  return observeAsync(ctx.observability, 'fnf.profile.list_workspaces', {}, async () => mapProfileWorkspaces(await ctx.profileAdapter.listWorkspaces()), {
    successAttributes: workspaces => ({ workspace_count: workspaces.length }),
  })
}

export async function getCurrentProfileWorkspace(ctx: ProfileContext): Promise<ProfileWorkspace | null> {
  return observeAsync(ctx.observability, 'fnf.profile.get_current_workspace', {}, async () => mapProfileWorkspace(await ctx.profileAdapter.getCurrentWorkspace()), {
    successAttributes: workspace => ({ has_workspace: workspace !== null, ...(workspace?.id ? { workspace_id: workspace.id } : {}) }),
  })
}

export async function getProfileWallet(ctx: ProfileContext): Promise<ProfileWorkspaceWallet | null> {
  return observeAsync(ctx.observability, 'fnf.profile.get_wallet', {}, async () => mapProfileWallet(await ctx.profileAdapter.getWorkspaceWallet()), {
    successAttributes: wallet => ({ has_wallet: wallet !== null }),
  })
}

export async function getProfileCredits(ctx: ProfileContext, options?: ProfileCreditsOptions): Promise<ProfileCredits | null> {
  return observeAsync(ctx.observability, 'fnf.profile.get_credits', { include_on_demand: options?.includeOnDemand !== false }, async () => {
    const wallet = await getProfileWallet(ctx)
    return wallet ? calculateProfileCredits(wallet, options) : null
  }, {
    successAttributes: credits => ({ has_credits: credits !== null }),
  })
}

export async function getProfileSnapshot(ctx: ProfileContext, options?: ProfileCreditsOptions): Promise<ProfileSnapshot> {
  return observeAsync(ctx.observability, 'fnf.profile.get_snapshot', { include_on_demand: options?.includeOnDemand !== false }, async () => {
    const [user, workspaces, currentWorkspace, wallet] = await Promise.all([
      getProfileUser(ctx),
      listProfileWorkspaces(ctx),
      getCurrentProfileWorkspace(ctx),
      getProfileWallet(ctx),
    ])

    return {
      user,
      workspaces,
      currentWorkspace,
      wallet,
      credits: wallet ? calculateProfileCredits(wallet, options) : null,
    }
  }, {
    successAttributes: snapshot => ({
      has_user: snapshot.user !== null,
      workspace_count: snapshot.workspaces.length,
      has_current_workspace: snapshot.currentWorkspace !== null,
      has_wallet: snapshot.wallet !== null,
      has_credits: snapshot.credits !== null,
    }),
  })
}
```

app/packages/fnf/src/profile/index.ts

```
export { createProfileClient } from './client'
export { createProfileContext } from './context'
export { calculateProfileCredits } from './credits'
export { getCurrentProfileWorkspace, getProfileCredits, getProfileSnapshot, getProfileUser, getProfileWallet, listProfileWorkspaces } from './get'
export { mapProfileUser, mapProfileWallet, mapProfileWorkspace, mapProfileWorkspaces } from './mappers'
export { switchProfileWorkspace } from './switch'
export type {
  ProfileClient,
  ProfileClientConfig,
  ProfileContext,
  ProfileCredits,
  ProfileCreditsOptions,
  ProfileCreditsRaw,
  ProfileSnapshot,
  ProfileUser,
  ProfileWorkspace,
  ProfileWorkspaceBlock,
  ProfileWorkspaceRole,
  ProfileWorkspaceType,
  ProfileWorkspaceWallet,
  SwitchWorkspaceInput,
} from './types'
```

app/packages/fnf/src/profile/mappers.ts

```
import type { ProfileUser, ProfileWorkspace, ProfileWorkspaceRole, ProfileWorkspaceType, ProfileWorkspaceWallet } from './types'

type UnknownRecord = Record<string, unknown>

export function mapProfileUser(raw: unknown): ProfileUser | null {
  if (!isRecord(raw))
    return null

  return {
    id: str(raw.id) ?? '',
    email: nullableStr(raw.email),
    businessEmail: nullableStr(raw.business_email ?? raw.businessEmail),
    verifiedBusinessEmail: nullableStr(raw.verified_business_email ?? raw.verifiedBusinessEmail),
    workspaceId: nullableStr(raw.workspace_id ?? raw.workspaceId),
    workspaceType: workspaceType(raw.workspace_type ?? raw.workspaceType),
    workspaceRole: workspaceRole(raw.workspace_role ?? raw.workspaceRole),
    workspaceMembershipExists: bool(raw.workspace_membership_exists ?? raw.workspaceMembershipExists, false),
    planType: nullableStr(raw.plan_type ?? raw.planType),
    billingPeriod: nullableStr(raw.billing_period ?? raw.billingPeriod),
    planEndsAt: raw.plan_ends_at ?? raw.planEndsAt ?? null,
    planVersion: nullableNum(raw.plan_version ?? raw.planVersion),
    credits: num(raw.credits, 0),
    totalPlanCredits: num(raw.total_plan_credits ?? raw.totalPlanCredits, 0),
    packageCredits: num(raw.package_credits ?? raw.packageCredits, 0),
    subscriptionCredits: num(raw.subscription_credits ?? raw.subscriptionCredits, 0),
    nextCreditAllocationAt: nullableStr(raw.next_credit_allocation_at ?? raw.nextCreditAllocationAt),
    dailyCredits: nullableNum(raw.daily_credits ?? raw.dailyCredits),
    text2keyframesCredits: nullableNum(raw.text2keyframes_credits ?? raw.text2keyframesCredits),
    faceSwapCredits: num(raw.face_swap_credits ?? raw.faceSwapCredits, 0),
    characterSwapCredits: num(raw.character_swap_credits ?? raw.characterSwapCredits, 0),
    soulCredits: num(raw.soul_credits ?? raw.soulCredits, 0),
    wan25VideoCredits: nullableNum(raw.wan2_5_video_credits ?? raw.wan25VideoCredits),
    qwenCameraControlCredits: nullableNum(raw.qwen_camera_control_credits ?? raw.qwenCameraControlCredits),
    cohort: nullableStr(raw.cohort),
    promoState: nullableStr(raw.promo_state ?? raw.promoState),
    purchasedPaidBoards: bool(raw.purchased_paid_boards ?? raw.purchasedPaidBoards, false),
    hasUnlim: bool(raw.has_unlim ?? raw.hasUnlim, false),
    hasFlexUnlim: bool(raw.has_flex_unlim ?? raw.hasFlexUnlim, false),
    autoPublish: bool(raw.auto_publish ?? raw.autoPublish, false),
    veo3FastGenerationsCount: num(raw.veo3_fast_generations_count ?? raw.veo3FastGenerationsCount, 0),
    isProPlanVeo3Available: nullableBool(raw.is_pro_plan_veo3_available ?? raw.isProPlanVeo3Available),
    hideReducedNanoBanana2Concurrent: bool(raw.hide_reduced_nano_banana_2_concurrent ?? raw.hideReducedNanoBanana2Concurrent, false),
    isCancelInited: bool(raw.is_cancel_inited ?? raw.isCancelInited, false),
    isPauseScheduled: bool(raw.is_pause_scheduled ?? raw.isPauseScheduled, false),
    isPaused: bool(raw.is_paused ?? raw.isPaused, false),
    pauseResumesAt: nullableStr(raw.pause_resumes_at ?? raw.pauseResumesAt),
    pauseStartsAt: nullableStr(raw.pause_starts_at ?? raw.pauseStartsAt),
    subscriptionDowngradesAt: nullableStr(raw.subscription_downgrades_at ?? raw.subscriptionDowngradesAt),
    lastDailyCreditsAwardedAt: nullableStr(raw.last_daily_credits_awarded_at ?? raw.lastDailyCreditsAwardedAt),
    appealAppliedAt: nullableStr(raw.appeal_applied_at ?? raw.appealAppliedAt),
    suspendedAt: nullableStr(raw.suspended_at ?? raw.suspendedAt),
    blockedAt: nullableStr(raw.blocked_at ?? raw.blockedAt),
    blockReason: nullableStr(raw.block_reason ?? raw.blockReason),
    isGiftSubscription: bool(raw.is_gift_subscription ?? raw.isGiftSubscription, false),
    showGenerationActivityNotice: nullableBool(raw.show_generation_activity_notice ?? raw.showGenerationActivityNotice),
    isTestUser: bool(raw.is_test_user ?? raw.isTestUser, false),
  }
}

export function mapProfileWorkspace(raw: unknown): ProfileWorkspace | null {
  if (!isRecord(raw))
    return null

  const role = workspaceRole(raw.user_role ?? raw.role ?? raw.workspace_role ?? raw.workspaceRole)
  const type = workspaceType(raw.type) ?? 'private'
  const rawBlock = raw.sub_workspace_block ?? raw.subWorkspaceBlock
  const block = isRecord(rawBlock) ? rawBlock : null
```

```
  return {
    id: str(raw.id) ?? '',
    name: str(raw.name) ?? '',
    clerkOrganizationId: nullableStr(raw.clerk_organization_id ?? raw.clerkOrganizationId),
    type,
    role,
    isOwner: bool(raw.isOwner ?? raw.is_owner, role === 'owner'),
    isAdmin: bool(raw.isAdmin ?? raw.is_admin, role === 'admin'),
    isMember: bool(raw.isMember ?? raw.is_member, role === 'member'),
    isTeamWorkspace: bool(raw.isTeamWorkspace ?? raw.is_team_workspace, type === 'shared'),
    avatarUrl: nullableStr(raw.avatar_url ?? raw.avatarUrl),
    description: nullableStr(raw.bio ?? raw.description),
    gracePeriodType: gracePeriodType(raw.grace_period_type ?? raw.gracePeriodType),
    ssoStatus: nullableStr(raw.sso_status ?? raw.ssoStatus),
    isEnterpriseSubWorkspace: bool(raw.is_enterprise_sub_workspace ?? raw.isEnterpriseSubWorkspace, false),
    subWorkspaceBlock: block
      ? {
          reason: str(block.reason) ?? 'unknown',
          ownerEmail: nullableStr(block.owner_email ?? block.ownerEmail),
        }
      : null,
  }
}

export function mapProfileWorkspaces(raw: unknown): ProfileWorkspace[] {
  const items = Array.isArray(raw)
    ? raw
    : isRecord(raw) && Array.isArray(raw.workspaces)
      ? raw.workspaces
      : isRecord(raw) && Array.isArray(raw.items)
        ? raw.items
        : []

  return items
    .map(item => mapProfileWorkspace(item))
    .filter((item): item is ProfileWorkspace => item !== null)
}

export function mapProfileWallet(raw: unknown): ProfileWorkspaceWallet | null {
  if (!isRecord(raw))
    return null

  return {
    id: str(raw.workspace_id ?? raw.workspaceId ?? raw.id) ?? '',
    subscriptionBalance: num(raw.subscription_balance ?? raw.subscriptionBalance, 0),
    subscriptionTotal: num(raw.total_credits ?? raw.subscription_total ?? raw.subscriptionTotal, 0),
    creditsBalance: num(raw.credits_balance ?? raw.creditsBalance, 0),
    onDemandCredits: Math.max(0, num(raw.on_demand_credits ?? raw.onDemandCredits, 0)),
    walletCreatedAt: nullableStr(raw.wallet_created_at ?? raw.walletCreatedAt),
    nextPaymentDate: nullableStr(raw.next_credit_allocation_date ?? raw.nextPaymentDate),
  }
}

function isRecord(value: unknown): value is UnknownRecord {
  return value !== null && typeof value === 'object' && !Array.isArray(value)
}

function str(value: unknown): string | undefined {
  return typeof value === 'string' ? value : undefined
}

function nullableStr(value: unknown): string | null {
  return typeof value === 'string' ? value : null
}

function num(value: unknown, fallback: number): number {
  return typeof value === 'number' && Number.isFinite(value) ? value : fallback
}

function nullableNum(value: unknown): number | null {
```

```
  return typeof value === 'number' && Number.isFinite(value) ? value : null
}

function bool(value: unknown, fallback: boolean): boolean {
  return typeof value === 'boolean' ? value : fallback
}

function nullableBool(value: unknown): boolean | null {
  return typeof value === 'boolean' ? value : null
}

function workspaceType(value: unknown): ProfileWorkspaceType | null {
  return value === 'private' || value === 'shared' ? value : null
}

function workspaceRole(value: unknown): ProfileWorkspaceRole | null {
  return value === 'owner' || value === 'admin' || value === 'member' ? value : null
}

function gracePeriodType(value: unknown): 'soft' | 'strict' | null {
  return value === 'soft' || value === 'strict' ? value : null
}
```

app/packages/fnf/src/profile/switch.ts

```
import type { ProfileContext, ProfileSnapshot, SwitchWorkspaceInput } from './types'
import { ValidationError } from '../errors'
import { observeAsync } from '../observability'
import { getProfileSnapshot } from './get'

export async function switchProfileWorkspace(ctx: ProfileContext, input: SwitchWorkspaceInput): Promise<ProfileSnapshot> {
  return observeAsync(ctx.observability, 'fnf.profile.switch_workspace', { workspace_id: input.workspaceId }, async () => {
    const workspaceId = input.workspaceId.trim()
    if (!workspaceId)
      throw new ValidationError('switchWorkspace requires a non-empty workspaceId')

    await ctx.profileAdapter.switchWorkspace({ workspaceId })
    return getProfileSnapshot(ctx)
  }, {
    successAttributes: snapshot => ({
      has_current_workspace: snapshot.currentWorkspace !== null,
      ...(snapshot.currentWorkspace?.id ? { current_workspace_id: snapshot.currentWorkspace.id } : {}),
    }),
  })
}
```

app/packages/fnf/src/profile/types.ts

```
import type { ProfileBackend } from '../backend'
import type { FnfObservabilityContext, FnfObservabilityOptions } from '../observability'

export type ProfileWorkspaceType = 'private' | 'shared'
export type ProfileWorkspaceRole = 'owner' | 'admin' | 'member'

export interface ProfileUser {
  id: string
  email: string | null
  businessEmail: string | null
  verifiedBusinessEmail: string | null
  workspaceId: string | null
  workspaceType: ProfileWorkspaceType | null
  workspaceRole: ProfileWorkspaceRole | null
  workspaceMembershipExists: boolean
  planType: string | null
  billingPeriod: string | null
  planEndsAt: unknown
  planVersion: number | null
  credits: number
  totalPlanCredits: number
  packageCredits: number
  subscriptionCredits: number
  nextCreditAllocationAt: string | null
  dailyCredits: number | null
  text2keyframesCredits: number | null
  faceSwapCredits: number
  characterSwapCredits: number
  soulCredits: number
  wan25VideoCredits: number | null
  qwenCameraControlCredits: number | null
  cohort: string | null
  promoState: string | null
  purchasedPaidBoards: boolean
  hasUnlim: boolean
  hasFlexUnlim: boolean
  autoPublish: boolean
  veo3FastGenerationsCount: number
  isProPlanVeo3Available: boolean | null
  hideReducedNanoBanana2Concurrent: boolean
  isCancelInited: boolean
  isPauseScheduled: boolean
  isPaused: boolean
  pauseResumesAt: string | null
  pauseStartsAt: string | null
  subscriptionDowngradesAt: string | null
  lastDailyCreditsAwardedAt: string | null
  appealAppliedAt: string | null
  suspendedAt: string | null
  blockedAt: string | null
  blockReason: string | null
  isGiftSubscription: boolean
  showGenerationActivityNotice: boolean | null
  isTestUser: boolean
}

export interface ProfileWorkspaceBlock {
  reason: 'out_of_credits' | string
  ownerEmail: string | null
}

export interface ProfileWorkspace {
  id: string
  name: string
  clerkOrganizationId: string | null
  type: ProfileWorkspaceType
  role: ProfileWorkspaceRole | null
  isOwner: boolean
```

```
  isAdmin: boolean
  isMember: boolean
  isTeamWorkspace: boolean
  avatarUrl: string | null
  description: string | null
  gracePeriodType: 'soft' | 'strict' | null
  ssoStatus: string | null
  isEnterpriseSubWorkspace: boolean
  subWorkspaceBlock: ProfileWorkspaceBlock | null
}

export interface ProfileWorkspaceWallet {
  id: string
  subscriptionBalance: number
  subscriptionTotal: number
  creditsBalance: number
  onDemandCredits: number
  walletCreatedAt: string | null
  nextPaymentDate: string | null
}

export interface ProfileCreditsRaw {
  availableCredits: number
  usedCredits: number
  monthlyRemaining: number
  purchasedCredits: number
  onDemandCredits: number
  totalAvailableCredits: number
  totalProgressCapacity: number
  planCapacity: number
}

export interface ProfileCredits {
  /** Monthly allocation plus purchased/top-up credits, normalized from backend credit-cents. */
  availableCredits: number
  /** Available credits plus optional on-demand/auto-refill credits, normalized. */
  totalAvailableCredits: number
  /** Monthly plan capacity plus optional on-demand capacity, normalized. */
  totalProgressCapacity: number
  /** Consumed monthly plan credits, normalized. */
  usedCredits: number
  monthlyRemaining: number
  purchasedCredits: number
  onDemandCredits: number
  planCapacity: number
  availablePercent: number
  usagePercent: number
  raw: ProfileCreditsRaw
}

export interface ProfileCreditsOptions {
  /**
   * Include wallet.onDemandCredits in totalAvailableCredits/progress capacity.
   * Defaults to true because it is spendable balance; app UIs with extra
   * auto-refill policy can pass false and show it separately.
   */
  includeOnDemand?: boolean
}

export interface ProfileSnapshot {
  user: ProfileUser | null
  workspaces: ProfileWorkspace[]
  currentWorkspace: ProfileWorkspace | null
  wallet: ProfileWorkspaceWallet | null
  credits: ProfileCredits | null
}

export interface SwitchWorkspaceInput {
  workspaceId: string
}
```

```
export interface ProfileClientConfig {
  /** The transport-agnostic profile adapter. Use one from `@higgsfield/fnf-adapters`, or your own. */
  profileAdapter: ProfileBackend
  observability?: FnfObservabilityOptions
}

export interface ProfileContext {
  profileAdapter: ProfileBackend
  observability: FnfObservabilityContext
}

export interface ProfileClient {
  getUser: () => Promise<ProfileUser | null>
  listWorkspaces: () => Promise<ProfileWorkspace[]>
  getCurrentWorkspace: () => Promise<ProfileWorkspace | null>
  getWallet: () => Promise<ProfileWorkspaceWallet | null>
  getCredits: (options?: ProfileCreditsOptions) => Promise<ProfileCredits | null>
  getSnapshot: (options?: ProfileCreditsOptions) => Promise<ProfileSnapshot>
  switchWorkspace: (input: SwitchWorkspaceInput) => Promise<ProfileSnapshot>
}
```

app/packages/fnf/src/realtime/__tests__/realtime.test.ts

```
import type { ConfirmSubmit, RealtimeBackend } from '../../backend'
import type { RealtimeChainEditWireRequest, RealtimeImageRef } from '../types'
import { describe, expect, it, vi } from 'vitest'
import { createRealtimeClient } from '../client'
import {
  buildRealtimeChainEditRequest,
  parseRealtimeChainEditResult,
  parseRealtimeCustomStyle,
} from '../codec'

const MEDIA_ID = '11111111-1111-4111-8111-111111111111'
const CHAIN_ID = '22222222-2222-4222-8222-222222222222'
const JOB_SET_ID = '33333333-3333-4333-8333-333333333333'
const JOB_ID = '44444444-4444-4444-8444-444444444444'
const STYLE_ID = '55555555-5555-4555-8555-555555555555'

const image: RealtimeImageRef = {
  id: MEDIA_ID,
  type: 'media_input',
  url: 'https://private.example/source.png',
  meta: { width: 1024, height: 1024 },
}

const editResponse = {
  chain_id: CHAIN_ID,
  job_set_id: JOB_SET_ID,
  job_id: JOB_ID,
  status: 'in_progress',
  created_at: 123.5,
}

function backend(overrides: Partial<RealtimeBackend> = {}): RealtimeBackend {
  return {
    editRealtimeChain: vi.fn(async () => editResponse),
    estimateRealtimeChainCost: vi.fn(async () => ({ credits: 1, credits_exact: 0.75 })),
    finalizeRealtimeChain: vi.fn(async req => ({ chain_id: req.chain_id, status: 'finalized' })),
    listRealtimeCustomStyles: vi.fn(async () => ({ items: [], next_cursor: null })),
    createRealtimeCustomStyle: vi.fn(async req => ({ id: STYLE_ID, prompt: req.prompt, reference: req.reference ?? null })),
    updateRealtimeCustomStyle: vi.fn(async (id, req) => ({ id, prompt: req.prompt ?? 'old', reference: req.reference ?? null })),
    deleteRealtimeCustomStyle: vi.fn(async () => undefined),
    ...overrides,
  }
}

describe('realtime Flux Klein edit chains', () => {
  it('maps camelCase input to the exact edit wire and strips MediaRef display metadata', () => {
    expect(buildRealtimeChainEditRequest({
      chainId: CHAIN_ID,
      params: {
        prompt: 'Make this cinematic',
        mode: 'basic',
        resolution: '2k',
        aspectRatio: '16:9',
        images: [image],
      },
    })).toEqual({
      chain_id: CHAIN_ID,
      params: {
        prompt: 'Make this cinematic',
        mode: 'basic',
        resolution: '2k',
        aspect_ratio: '16:9',
        medias: [{
          role: 'image',
          data: { id: image.id, type: image.type },
        }],
      },
    })
```

```
})

it('runs the shared confirmation contract against the exact wire body and forwards its token', async () => {
  const adapter = backend()
  const confirm: ConfirmSubmit = vi.fn(async () => 'approved-token')
  const client = createRealtimeClient({ adapter, confirm })
  const input = {
    params: {
      prompt: 'Turn it into a sketch',
      resolution: '1k' as const,
      aspectRatio: '1:1' as const,
      images: [image],
    },
  }
  const expected = buildRealtimeChainEditRequest(input)

  await expect(client.editChain(input)).resolves.toEqual({
    chainId: editResponse.chain_id,
    jobSetId: editResponse.job_set_id,
    jobId: editResponse.job_id,
    status: 'in_progress',
    createdAt: 123.5,
  })
  expect(confirm).toHaveBeenCalledWith({
    jobSetType: 'flux_klein_realtime',
    params: expected,
  })
  expect(adapter.editRealtimeChain).toHaveBeenCalledWith({
    ...expected,
    confirmation_token: 'approved-token',
  })
})

it('uses the adapter confirmation compatibility bridge when one is installed', async () => {
  const confirm: ConfirmSubmit = vi.fn(async () => 'adapter-token')
  const adapter = Object.assign(backend(), { confirm })
  const client = createRealtimeClient({ adapter })

  await client.editChain({
    params: {
      prompt: 'Edit once',
      resolution: '1k',
      aspectRatio: '1:1',
      images: [image],
    },
  })

  expect(confirm).toHaveBeenCalledOnce()
  expect(adapter.editRealtimeChain).toHaveBeenCalledWith(expect.objectContaining({ confirmation_token: 'adapter-token' }))
})

it('accepts an already-issued token without opening a second confirmation', async () => {
  const adapter = backend()
  const confirm: ConfirmSubmit = vi.fn(async () => 'unexpected-token')
  const client = createRealtimeClient({ adapter, confirm })

  await client.editChain({
    params: {
      prompt: 'Edit once',
      resolution: '1k',
      aspectRatio: '1:1',
      images: [image],
    },
  }, { confirmationToken: 'external-token' })

  expect(confirm).not.toHaveBeenCalled()
  expect(adapter.editRealtimeChain).toHaveBeenCalledWith(expect.objectContaining({ confirmation_token: 'external-token' }))
})

it('serializes structured controls and allows four explicit images with a saved style', () => {
```

```
  const request = buildRealtimeChainEditRequest({
    params: {
      settings: {
        style: 'custom',
        customStyle: 'editorial flash',
        mood: 'dramatic',
        palette: 'custom',
        customColors: ['#FF00AA'],
        sketchInfluence: 65,
        referenceMode: 'character',
        target: 'selection',
        customStyleId: STYLE_ID,
      },
      resolution: '2k',
      aspectRatio: '3:4',
      images: [image, image, image, image],
    },
  })

  expect(request.params).toMatchObject({
    mode: 'advanced',
    aspect_ratio: '3:4',
    settings: {
      custom_style: 'editorial flash',
      custom_colors: ['#FF00AA'],
      sketch_influence: 65,
      reference_mode: 'character',
      custom_style_id: STYLE_ID,
    },
  })
  expect(request.params.medias).toHaveLength(4)

  expect(() => buildRealtimeChainEditRequest({
    params: {
      settings: { style: 'custom', customStyleId: STYLE_ID },
      resolution: '1k',
      aspectRatio: '1:1',
      images: [image, image, image, image, image],
    },
  })).toThrow(expect.objectContaining({ code: 'validation' }))
})

it('polls the returned edit job through the internal read-only job entry', async () => {
  const getJob = vi.fn()
    .mockResolvedValueOnce({
      id: JOB_ID,
      job_set_id: JOB_SET_ID,
      job_set_type: 'flux_klein_edit',
      status: 'in_progress',
      params: { prompt: 'One edit' },
    })
    .mockResolvedValueOnce({
      id: JOB_ID,
      job_set_id: JOB_SET_ID,
      job_set_type: 'flux_klein_edit',
      status: 'completed',
      result_url: 'https://cdn.example/edit.png',
      params: { prompt: 'One edit' },
    })
  const adapter = Object.assign(backend(), { getJob })
  const client = createRealtimeClient({
    adapter,
    poll: { intervalMs: 0, timeoutMs: 1_000 },
    scheduler: { sleep: async () => {} },
  })

  const submitted = await client.editChain({
    params: {
      prompt: 'One edit',
      resolution: '1k',
```

```
      aspectRatio: '1:1',
      images: [image],
    },
  })
  await expect(client.pollEditJob(submitted.jobId)).resolves.toMatchObject({
    id: JOB_ID,
    jobSetId: JOB_SET_ID,
    model: 'flux_klein_edit',
    type: 'image',
    status: 'completed',
    results: { rawUrl: 'https://cdn.example/edit.png' },
  })
  expect(getJob).toHaveBeenCalledTimes(2)
})

it.each([
  ['both prompt and settings', { prompt: 'x', settings: { style: 'custom' } }],
  ['neither prompt nor settings', {}],
  ['an oversized prompt', { prompt: 'x'.repeat(10_001) }],
])('rejects %s before sending', async (_label, instruction) => {
  const adapter = backend()
  const client = createRealtimeClient({ adapter })

  await expect(client.editChain({
    params: {
      ...instruction,
      resolution: '1k',
      aspectRatio: '1:1',
      images: [image],
    } as never,
  })).rejects.toMatchObject({ code: 'validation' })
  expect(adapter.editRealtimeChain).not.toHaveBeenCalled()
})

it('rejects non-canonical ids, non-image media types, and invalid response ids', async () => {
  const adapter = backend()
  const client = createRealtimeClient({ adapter })

  expect(() => buildRealtimeChainEditRequest({
    chainId: 'not-a-uuid',
    params: {
      prompt: 'Edit once',
      resolution: '1k',
      aspectRatio: '1:1',
      images: [{ ...image, id: 'https://cdn.example/source.png', type: 'video_input' as never }],
    },
  })).toThrow(expect.objectContaining({ code: 'validation' }))
  expect(() => parseRealtimeChainEditResult({ ...editResponse, job_id: 'not-a-uuid' }))
    .toThrow(expect.objectContaining({ code: 'validation' }))
  await expect(client.updateCustomStyle('not-a-uuid', {})).rejects.toMatchObject({ code: 'validation' })
  await expect(client.createCustomStyle({ reference: { mediaId: 'not-a-uuid' } }))
    .rejects.toMatchObject({ code: 'validation' })
  expect(adapter.updateRealtimeCustomStyle).not.toHaveBeenCalled()
  expect(adapter.createRealtimeCustomStyle).not.toHaveBeenCalled()
})

it('does not retry a failed edit because each attempt can reserve credits', async () => {
  const editRealtimeChain = vi.fn(async (_req: RealtimeChainEditWireRequest) => {
    throw new Error('connection dropped')
  })
  const client = createRealtimeClient({ adapter: backend({ editRealtimeChain }) })

  await expect(client.editChain({
    params: {
      prompt: 'One attempt only',
      resolution: '1k',
      aspectRatio: '1:1',
      images: [image],
    },
  })).rejects.toThrow('connection dropped')
```

```
    expect(editRealtimeChain).toHaveBeenCalledOnce()
  })
})

describe('realtime cost, finalize, and custom styles', () => {
  it('maps cost/finalize responses and uses the public chain id', async () => {
    const adapter = backend()
    const client = createRealtimeClient({ adapter })

    await expect(client.estimateChainCost({
      resolution: '2k',
      aspectRatio: '16:9',
    })).resolves.toEqual({ credits: 1, creditsExact: 0.75 })
    expect(adapter.estimateRealtimeChainCost).toHaveBeenCalledWith({
      mode: 'advanced',
      resolution: '2k',
      aspect_ratio: '16:9',
    })

    await expect(client.finalizeChain({ chainId: CHAIN_ID })).resolves.toEqual({
      chainId: CHAIN_ID,
      status: 'finalized',
    })
    expect(adapter.finalizeRealtimeChain).toHaveBeenCalledWith({ chain_id: CHAIN_ID })
  })

  it('maps saved custom styles and every CRUD request without leaking snake_case', async () => {
    const styleWire = {
      id: STYLE_ID,
      prompt: 'soft flash',
      reference: {
        media_id: image.id,
        url: 'https://cdn.example/style.png',
        source_name: 'style.png',
        content_type: 'image/png',
      },
    }
    expect(parseRealtimeCustomStyle(styleWire)).toEqual({
      id: STYLE_ID,
      prompt: 'soft flash',
      reference: {
        mediaId: image.id,
        url: 'https://cdn.example/style.png',
        sourceName: 'style.png',
        contentType: 'image/png',
      },
    })

    const adapter = backend({
      listRealtimeCustomStyles: vi.fn(async () => ({ items: [styleWire], next_cursor: 123.5 })),
      createRealtimeCustomStyle: vi.fn(async () => styleWire),
      updateRealtimeCustomStyle: vi.fn(async () => ({ ...styleWire, prompt: 'new prompt', reference: null })),
    })
    const client = createRealtimeClient({ adapter })

    await expect(client.listCustomStyles({ cursor: 123, size: 20 })).resolves.toEqual({
      items: [parseRealtimeCustomStyle(styleWire)],
      nextCursor: 123.5,
    })
    expect(adapter.listRealtimeCustomStyles).toHaveBeenCalledWith({ cursor: 123, size: 20 })

    await client.createCustomStyle({
      prompt: 'soft flash',
      reference: {
        mediaId: image.id,
        url: 'https://cdn.example/style.png',
        sourceName: 'style.png',
        contentType: 'image/png',
      },
    })
```

```
    expect(adapter.createRealtimeCustomStyle).toHaveBeenCalledWith({
      prompt: 'soft flash',
      reference: {
        media_id: image.id,
        url: 'https://cdn.example/style.png',
        source_name: 'style.png',
        content_type: 'image/png',
      },
    })

    await expect(client.updateCustomStyle(STYLE_ID, {
      prompt: 'new prompt',
      clearReference: true,
    })).resolves.toMatchObject({ prompt: 'new prompt', reference: null })
    expect(adapter.updateRealtimeCustomStyle).toHaveBeenCalledWith(STYLE_ID, {
      prompt: 'new prompt',
      clear_reference: true,
    })

    await expect(client.deleteCustomStyle(STYLE_ID)).resolves.toBeUndefined()
    expect(adapter.deleteRealtimeCustomStyle).toHaveBeenCalledWith(STYLE_ID)
  })

  it('validates custom-style pagination and reference metadata before requests', async () => {
    const adapter = backend()
    const client = createRealtimeClient({ adapter })

    await expect(client.listCustomStyles({ cursor: Number.NaN })).rejects.toMatchObject({ code: 'validation' })
    await expect(client.createCustomStyle({
      reference: { mediaId: image.id, url: 'file:///private/image.png' },
    })).rejects.toMatchObject({ code: 'validation' })
    expect(adapter.listRealtimeCustomStyles).not.toHaveBeenCalled()
    expect(adapter.createRealtimeCustomStyle).not.toHaveBeenCalled()
  })
})
```

app/packages/fnf/src/realtime/client.ts

```
import type { ConfirmSubmit, GenerationBackend, RealtimeBackend } from '../backend'
import type { FnfObservabilityOptions } from '../observability'
import type { Generation } from '../types'
import type {
  CreateRealtimeCustomStyleInput,
  RealtimeChainCostEstimate,
  RealtimeChainCostInput,
  RealtimeChainEditInput,
  RealtimeChainEditOptions,
  RealtimeChainEditResult,
  RealtimeChainFinalizeInput,
  RealtimeChainFinalizeResult,
  RealtimeCustomStyle,
  RealtimeCustomStyleListQuery,
  RealtimeCustomStyleListResult,
  RealtimeEditPollOptions,
  UpdateRealtimeCustomStyleInput,
} from './types'
import { createContext } from '../client/context'
import { getGeneration } from '../client/get'
import { pollGeneration } from '../client/poll'
import { defineJob } from '../define-job'
import { ApiJobError, ConfirmationRejectedError } from '../errors'
import { createObservabilityContext, observeAsync } from '../observability'
import {
  buildCreateRealtimeCustomStyleRequest,
  buildRealtimeChainCostRequest,
  buildRealtimeChainEditRequest,
  buildRealtimeChainFinalizeRequest,
  buildUpdateRealtimeCustomStyleRequest,
  parseRealtimeChainCostEstimate,
  parseRealtimeChainEditResult,
  parseRealtimeChainFinalizeResult,
  parseRealtimeCustomStyle,
  parseRealtimeCustomStyleList,
  requireRealtimeUuid,
  validateRealtimeCustomStyleListQuery,
} from './codec'

const REALTIME_EDIT_READ_ENTRY = defineJob({
  jobSetType: 'flux_klein_edit',
  outputType: 'image',
  params: { settings: {} },
})

async function requestSubmitConfirmation(
  confirm: ConfirmSubmit,
  request: Parameters<ConfirmSubmit>[0],
): Promise<string | undefined> {
  let token: string | void
  try {
    token = await confirm(request)
  }
  catch (error) {
    throw error instanceof ApiJobError
      ? error
      : new ConfirmationRejectedError(error instanceof Error ? error.message : undefined)
  }
  return typeof token === 'string' ? token : undefined
}

export interface RealtimeClientConfig {
  adapter: RealtimeBackend
  /** The same host confirmation contract used by normal generation submits. */
  confirm?: ConfirmSubmit
  /** Optional job read port for getEditJob/pollEditJob. */
  jobAdapter?: Pick<GenerationBackend, 'getJob'>
  /** Direct job reader; takes precedence over jobAdapter. */
```

```
  getJob?: GenerationBackend['getJob']
  poll?: { intervalMs?: number, timeoutMs?: number }
  scheduler?: { sleep?: (ms: number) => Promise<void>, isActive?: () => boolean }
  observability?: FnfObservabilityOptions
}

export interface RealtimeClient {
  /** Submit exactly one attempt. This operation is intentionally never retried. */
  editChain: (input: RealtimeChainEditInput, options?: RealtimeChainEditOptions) => Promise<RealtimeChainEditResult>
  /** Read one realtime edit result through the shared generation parser. */
  getEditJob: (jobId: string) => Promise<Generation>
  /** Poll one realtime edit to a terminal generation state. */
  pollEditJob: (jobId: string, options?: RealtimeEditPollOptions) => Promise<Generation>
  estimateChainCost: (input: RealtimeChainCostInput) => Promise<RealtimeChainCostEstimate>
  finalizeChain: (input: RealtimeChainFinalizeInput) => Promise<RealtimeChainFinalizeResult>
  listCustomStyles: (query?: RealtimeCustomStyleListQuery) => Promise<RealtimeCustomStyleListResult>
  createCustomStyle: (input: CreateRealtimeCustomStyleInput) => Promise<RealtimeCustomStyle>
  updateCustomStyle: (styleId: string, input: UpdateRealtimeCustomStyleInput) => Promise<RealtimeCustomStyle>
  deleteCustomStyle: (styleId: string) => Promise<void>
}

export function createRealtimeClient(config: RealtimeClientConfig): RealtimeClient {
  const observability = createObservabilityContext(config.observability)
  // Match createJobClient's compatibility bridge: generated-app adapters may
  // still carry the host callback while callers migrate to explicit clients.
  const confirm = config.confirm ?? (config.adapter as RealtimeBackend & { confirm?: ConfirmSubmit }).confirm
  const editJobContext = createEditJobContext(config)

  return {
    editChain: (input, options = {}) => observeAsync(observability, 'fnf.realtime.edit_chain', {
      continuing_chain: input.chainId !== undefined && input.chainId !== null,
    }, async () => {
      const wire = buildRealtimeChainEditRequest(input)
      const confirmationToken = options.confirmationToken ?? (confirm
        ? await requestSubmitConfirmation(confirm, {
            jobSetType: 'flux_klein_realtime',
            // Approval hashes the exact edit body that will be submitted,
            // before the opaque token itself is attached.
            params: wire,
          })
        : undefined)
      const request = confirmationToken === undefined
        ? wire
        : { ...wire, confirmation_token: confirmationToken }
      return parseRealtimeChainEditResult(await config.adapter.editRealtimeChain(request))
    }, {
      successAttributes: result => ({
        chain_id: result.chainId,
        job_set_id: result.jobSetId,
        job_id: result.jobId,
        status: result.status,
      }),
    }),

    getEditJob: (jobId) => {
      const id = requireRealtimeUuid(jobId, 'jobId')
      return getGeneration(requireEditJobContext(editJobContext), id, REALTIME_EDIT_READ_ENTRY)
    },

    pollEditJob: (jobId, options = {}) => {
      const id = requireRealtimeUuid(jobId, 'jobId')
      return pollGeneration(requireEditJobContext(editJobContext), id, {
        ...options,
        entry: REALTIME_EDIT_READ_ENTRY,
      })
    },

    estimateChainCost: input => observeAsync(observability, 'fnf.realtime.estimate_chain_cost', {
      mode: input.mode ?? 'advanced',
      resolution: input.resolution,
```

```
    }, async () => parseRealtimeChainCostEstimate(
      await config.adapter.estimateRealtimeChainCost(buildRealtimeChainCostRequest(input)),
    )),

    finalizeChain: input => observeAsync(observability, 'fnf.realtime.finalize_chain', {
      chain_id: input.chainId,
    }, async () => parseRealtimeChainFinalizeResult(
      await config.adapter.finalizeRealtimeChain(buildRealtimeChainFinalizeRequest(input)),
    ), { successAttributes: result => ({ chain_id: result.chainId, status: result.status }) }),

    listCustomStyles: (query = {}) => observeAsync(observability, 'fnf.realtime.list_custom_styles', {}, async () => {
      validateRealtimeCustomStyleListQuery(query)
      return parseRealtimeCustomStyleList(await config.adapter.listRealtimeCustomStyles(query))
    }, { successAttributes: result => ({ style_count: result.items.length }) }),

    createCustomStyle: input => observeAsync(observability, 'fnf.realtime.create_custom_style', {}, async () => (
      parseRealtimeCustomStyle(await config.adapter.createRealtimeCustomStyle(buildCreateRealtimeCustomStyleRequest(input)))
    ), { successAttributes: result => ({ style_id: result.id }) }),

    updateCustomStyle: (styleId, input) => observeAsync(observability, 'fnf.realtime.update_custom_style', {
      style_id: styleId,
    }, async () => {
      requireRealtimeUuid(styleId, 'styleId')
      return parseRealtimeCustomStyle(await config.adapter.updateRealtimeCustomStyle(
        styleId,
        buildUpdateRealtimeCustomStyleRequest(input),
      ))
    }, { successAttributes: result => ({ style_id: result.id }) }),

    deleteCustomStyle: (styleId) => observeAsync(observability, 'fnf.realtime.delete_custom_style', {
      style_id: styleId,
    }, async () => {
      requireRealtimeUuid(styleId, 'styleId')
      await config.adapter.deleteRealtimeCustomStyle(styleId)
    }),
  }
}

function createEditJobContext(config: RealtimeClientConfig) {
  const adapterWithJobRead = config.adapter as RealtimeBackend & Partial<Pick<GenerationBackend, 'getJob'>>
  const adapterGetJob = adapterWithJobRead.getJob
  const jobAdapter = config.jobAdapter
  const getJob = config.getJob
    ?? (jobAdapter ? (id: string) => jobAdapter.getJob(id) : undefined)
    ?? (typeof adapterGetJob === 'function' ? (id: string) => adapterGetJob.call(adapterWithJobRead, id) : undefined)
  if (!getJob)
    return undefined

  return createContext({
    adapter: readOnlyGenerationAdapter(getJob),
    jobs: [REALTIME_EDIT_READ_ENTRY],
    ...(config.poll ? { poll: config.poll } : {}),
    ...(config.scheduler ? { scheduler: config.scheduler } : {}),
    ...(config.observability ? { observability: config.observability } : {}),
  })
}

function requireEditJobContext(context: ReturnType<typeof createEditJobContext>) {
  if (!context) {
    throw new ApiJobError(
      'not_supported',
      'Realtime edit job reads require `jobAdapter`, `getJob`, or a realtime adapter with `getJob`',
    )
  }
  return context
}

function readOnlyGenerationAdapter(getJob: GenerationBackend['getJob']): GenerationBackend {
  const unsupported = async (): Promise<never> => {
    throw new ApiJobError('not_supported', 'The realtime edit reader supports job reads only')
```

```
  }
  return {
    createJobs: unsupported,
    getJob,
    listJobs: unsupported,
    estimateCost: unsupported,
  }
}
```

app/packages/fnf/src/realtime/codec.ts

```
import type {
  CreateRealtimeCustomStyleInput,
  CreateRealtimeCustomStyleWireRequest,
  RealtimeAspectRatio,
  RealtimeChainCostEstimate,
  RealtimeChainCostInput,
  RealtimeChainCostWireRequest,
  RealtimeChainEditInput,
  RealtimeChainEditResult,
  RealtimeChainEditWireRequest,
  RealtimeChainFinalizeInput,
  RealtimeChainFinalizeResult,
  RealtimeChainFinalizeWireRequest,
  RealtimeCustomStyle,
  RealtimeCustomStyleListQuery,
  RealtimeCustomStyleListResult,
  RealtimeCustomStyleReference,
  RealtimeCustomStyleReferenceWire,
  RealtimeMode,
  RealtimeSettings,
  RealtimeSettingsWire,
  UpdateRealtimeCustomStyleInput,
  UpdateRealtimeCustomStyleWireRequest,
} from './types'
import { ValidationError } from '../errors'

const MODES = new Set<RealtimeMode>(['basic', 'advanced'])
const RESOLUTIONS = new Set(['1k', '2k'])
const ASPECT_RATIOS = new Set<RealtimeAspectRatio>([
  '16:9',
  '4:3',
  '1:1',
  '3:4',
  '9:16',
  '2:3',
  '1:2',
  '2:1',
  '4:5',
  '3:2',
  '5:4',
  '21:9',
])
const HEX_COLOR = /^#[0-9a-fA-F]{6}$/
const CANONICAL_UUID = /^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[1-5][0-9a-fA-F]{3}-[89abAB][0-9a-fA-F]{3}-[0-9a-fA-F]{12}$/
const IMAGE_JOB_TYPE = /^[a-z0-9_]+_job$/

interface Issue {
  loc: string[]
  msg: string
}

export function buildRealtimeChainEditRequest(input: RealtimeChainEditInput): RealtimeChainEditWireRequest {
  const issues: Issue[] = []
  const params = input?.params
  if (!isRecord(params))
    throw new ValidationError('Invalid realtime edit input', [{ loc: ['params'], msg: 'params are required' }])

  const mode = validateMode(params.mode, ['params', 'mode'], issues)
  validateResolution(params.resolution, ['params', 'resolution'], issues)
  validateAspectRatio(params.aspectRatio, ['params', 'aspectRatio'], issues)

  const hasPrompt = typeof params.prompt === 'string'
  const hasSettings = isRecord(params.settings)
  if (hasPrompt === hasSettings)
    issues.push({ loc: ['params'], msg: 'exactly one of prompt or settings is required' })
  if (hasPrompt && (params.prompt.length < 1 || params.prompt.length > 10_000))
    issues.push({ loc: ['params', 'prompt'], msg: 'prompt must contain between 1 and 10000 characters' })
```

```
  let settings: RealtimeSettingsWire | undefined
  if (hasSettings)
    settings = buildSettings(params.settings as unknown as RealtimeSettings, issues)

  if (!Array.isArray(params.images) || params.images.length < 1 || params.images.length > 4) {
    issues.push({ loc: ['params', 'images'], msg: 'images must contain between 1 and 4 items' })
  }
  else {
    params.images.forEach((image, index) => {
      if (!isRecord(image) || !isCanonicalUuid(image.id))
        issues.push({ loc: ['params', 'images', String(index), 'id'], msg: 'image id must be a canonical UUID' })
      if (!isRecord(image) || !isRealtimeImageType(image.type))
        issues.push({ loc: ['params', 'images', String(index), 'type'], msg: 'image type must be media_input or an *_job type' })
    })
  }

  if (input.chainId !== undefined && input.chainId !== null && !isCanonicalUuid(input.chainId))
    issues.push({ loc: ['chainId'], msg: 'chainId must be a canonical UUID' })

  if (issues.length > 0)
    throw new ValidationError('Invalid realtime edit input', issues)

  const wireParams = {
    ...(hasPrompt ? { prompt: params.prompt as string } : {}),
    ...(settings ? { settings } : {}),
    mode,
    resolution: params.resolution,
    aspect_ratio: params.aspectRatio,
    medias: params.images.map(image => ({
      role: 'image' as const,
      data: { id: image.id, type: image.type },
    })),
  }

  return {
    ...(input.chainId !== undefined ? { chain_id: input.chainId } : {}),
    params: wireParams,
  }
}

export function buildRealtimeChainCostRequest(input: RealtimeChainCostInput): RealtimeChainCostWireRequest {
  const issues: Issue[] = []
  const mode = validateMode(input?.mode, ['mode'], issues)
  validateResolution(input?.resolution, ['resolution'], issues)
  validateAspectRatio(input?.aspectRatio, ['aspectRatio'], issues)
  if (issues.length > 0)
    throw new ValidationError('Invalid realtime cost input', issues)
  return {
    mode,
    resolution: input.resolution,
    aspect_ratio: input.aspectRatio,
  }
}

export function buildRealtimeChainFinalizeRequest(input: RealtimeChainFinalizeInput): RealtimeChainFinalizeWireRequest {
  if (!input || !isCanonicalUuid(input.chainId))
    throw new ValidationError('Invalid realtime finalize input', [{ loc: ['chainId'], msg: 'chainId must be a canonical UUID' }])
  return { chain_id: input.chainId }
}

export function buildCreateRealtimeCustomStyleRequest(input: CreateRealtimeCustomStyleInput): CreateRealtimeCustomStyleWireRequest {
  const issues: Issue[] = []
  validateOptionalPrompt(input?.prompt, ['prompt'], issues, false)
  if (input?.reference !== undefined && input.reference !== null)
    validateStyleReference(input.reference, ['reference'], issues)
  if (issues.length > 0)
    throw new ValidationError('Invalid realtime custom style input', issues)
  return {
    prompt: input?.prompt ?? '',
    ...(input?.reference !== undefined ? { reference: toStyleReferenceWire(input.reference) } : {}),
```

```
  }
}

export function buildUpdateRealtimeCustomStyleRequest(input: UpdateRealtimeCustomStyleInput): UpdateRealtimeCustomStyleWireRequest {
  const issues: Issue[] = []
  validateOptionalPrompt(input?.prompt, ['prompt'], issues, true)
  if (input?.reference !== undefined && input.reference !== null)
    validateStyleReference(input.reference, ['reference'], issues)
  if (input?.clearReference !== undefined && typeof input.clearReference !== 'boolean')
    issues.push({ loc: ['clearReference'], msg: 'clearReference must be a boolean' })
  if (input?.clearReference === true && input.reference !== undefined && input.reference !== null)
    issues.push({ loc: ['reference'], msg: 'reference cannot be set when clearReference is true' })
  if (issues.length > 0)
    throw new ValidationError('Invalid realtime custom style input', issues)
  return {
    ...(input?.prompt !== undefined ? { prompt: input.prompt } : {}),
    ...(input?.reference !== undefined ? { reference: toStyleReferenceWire(input.reference) } : {}),
    ...(input?.clearReference !== undefined ? { clear_reference: input.clearReference } : {}),
  }
}

export function validateRealtimeCustomStyleListQuery(query: RealtimeCustomStyleListQuery): void {
  const issues: Issue[] = []
  if (query.cursor !== undefined && (!Number.isFinite(query.cursor) || query.cursor <= 0))
    issues.push({ loc: ['cursor'], msg: 'cursor must be a positive finite number' })
  if (query.size !== undefined && (!Number.isInteger(query.size) || query.size < 1 || query.size > 100))
    issues.push({ loc: ['size'], msg: 'size must be an integer between 1 and 100' })
  if (issues.length > 0)
    throw new ValidationError('Invalid realtime custom style query', issues)
}

export function parseRealtimeChainEditResult(body: unknown): RealtimeChainEditResult {
  const value = unwrapData(body)
  if (!isRecord(value))
    throw invalidResponse('edit')
  return {
    chainId: requiredUuid(value.chain_id, 'chain_id'),
    jobSetId: requiredUuid(value.job_set_id, 'job_set_id'),
    jobId: requiredUuid(value.job_id, 'job_id'),
    status: requiredString(value.status, 'status'),
    createdAt: requiredNumber(value.created_at, 'created_at'),
  }
}

export function parseRealtimeChainCostEstimate(body: unknown): RealtimeChainCostEstimate {
  const value = unwrapData(body)
  if (!isRecord(value))
    throw invalidResponse('cost')
  return {
    credits: requiredNumber(value.credits, 'credits'),
    creditsExact: requiredNumber(value.credits_exact, 'credits_exact'),
  }
}

export function parseRealtimeChainFinalizeResult(body: unknown): RealtimeChainFinalizeResult {
  const value = unwrapData(body)
  if (!isRecord(value))
    throw invalidResponse('finalize')
  const status = value.status
  if (status !== 'finalized' && status !== 'released')
    throw invalidResponse('finalize', 'status')
  return {
    chainId: requiredUuid(value.chain_id, 'chain_id'),
    status,
  }
}

export function parseRealtimeCustomStyle(body: unknown): RealtimeCustomStyle {
  const value = unwrapData(body)
  if (!isRecord(value))
```

```
    throw invalidResponse('custom style')
  if (typeof value.prompt !== 'string')
    throw invalidResponse('custom style', 'prompt')
  return {
    id: requiredUuid(value.id, 'id'),
    prompt: value.prompt,
    reference: value.reference === null || value.reference === undefined
      ? null
      : parseStyleReference(value.reference),
  }
}

export function parseRealtimeCustomStyleList(body: unknown): RealtimeCustomStyleListResult {
  const value = unwrapData(body)
  if (!isRecord(value) || !Array.isArray(value.items))
    throw invalidResponse('custom style list')
  const nextCursor = value.next_cursor
  if (nextCursor !== undefined && nextCursor !== null && (typeof nextCursor !== 'number' || !Number.isFinite(nextCursor)))
    throw invalidResponse('custom style list', 'next_cursor')
  return {
    items: value.items.map(parseRealtimeCustomStyle),
    ...(typeof nextCursor === 'number' ? { nextCursor } : {}),
  }
}

function buildSettings(settings: RealtimeSettings, issues: Issue[]): RealtimeSettingsWire {
  if (typeof settings.style !== 'string' || settings.style.length < 1 || settings.style.length > 200)
    issues.push({ loc: ['params', 'settings', 'style'], msg: 'style must contain between 1 and 200 characters' })
  if (settings.customStyle !== undefined && (typeof settings.customStyle !== 'string' || settings.customStyle.length > 500))
    issues.push({ loc: ['params', 'settings', 'customStyle'], msg: 'customStyle must be at most 500 characters' })
  if (settings.mood !== undefined && (typeof settings.mood !== 'string' || settings.mood.length < 1 || settings.mood.length > 200))
    issues.push({ loc: ['params', 'settings', 'mood'], msg: 'mood must contain between 1 and 200 characters' })
  if (settings.palette !== undefined && (typeof settings.palette !== 'string' || settings.palette.length < 1 || settings.palette.length > 200))
    issues.push({ loc: ['params', 'settings', 'palette'], msg: 'palette must contain between 1 and 200 characters' })
  if (settings.customColors !== undefined) {
    if (!Array.isArray(settings.customColors) || settings.customColors.length > 8) {
      issues.push({ loc: ['params', 'settings', 'customColors'], msg: 'customColors must contain at most 8 items' })
    }
    else {
      settings.customColors.forEach((color, index) => {
        if (typeof color !== 'string' || !HEX_COLOR.test(color))
          issues.push({ loc: ['params', 'settings', 'customColors', String(index)], msg: 'color must use #RRGGBB format' })
      })
    }
  }
  if (settings.sketchInfluence !== undefined && (!Number.isInteger(settings.sketchInfluence) || settings.sketchInfluence < 0 || settings.sketchInfluence > 100))
    issues.push({ loc: ['params', 'settings', 'sketchInfluence'], msg: 'sketchInfluence must be an integer between 0 and 100' })
  if (settings.referenceMode !== undefined && settings.referenceMode !== 'style' && settings.referenceMode !== 'character')
    issues.push({ loc: ['params', 'settings', 'referenceMode'], msg: 'referenceMode must be style or character' })
  if (settings.target !== undefined && settings.target !== 'canvas' && settings.target !== 'selection')
    issues.push({ loc: ['params', 'settings', 'target'], msg: 'target must be canvas or selection' })
  if (settings.customStyleId !== undefined && settings.customStyleId !== null && !isCanonicalUuid(settings.customStyleId))
    issues.push({ loc: ['params', 'settings', 'customStyleId'], msg: 'customStyleId must be a canonical UUID' })

  return {
    style: settings.style,
    ...(settings.customStyle !== undefined ? { custom_style: settings.customStyle } : {}),
    ...(settings.mood !== undefined ? { mood: settings.mood } : {}),
    ...(settings.palette !== undefined ? { palette: settings.palette } : {}),
    ...(settings.customColors !== undefined ? { custom_colors: settings.customColors } : {}),
    ...(settings.sketchInfluence !== undefined ? { sketch_influence: settings.sketchInfluence } : {}),
    ...(settings.referenceMode !== undefined ? { reference_mode: settings.referenceMode } : {}),
    ...(settings.target !== undefined ? { target: settings.target } : {}),
    ...(settings.customStyleId !== undefined ? { custom_style_id: settings.customStyleId } : {}),
  }
}

function validateMode(value: unknown, loc: string[], issues: Issue[]): RealtimeMode {
  const mode = value ?? 'advanced'
  if (!MODES.has(mode as RealtimeMode))
```

```
    issues.push({ loc, msg: 'mode must be basic or advanced' })
  return mode as RealtimeMode
}

function validateResolution(value: unknown, loc: string[], issues: Issue[]): void {
  if (!RESOLUTIONS.has(value as string))
    issues.push({ loc, msg: 'resolution must be 1k or 2k' })
}

function validateAspectRatio(value: unknown, loc: string[], issues: Issue[]): void {
  if (!ASPECT_RATIOS.has(value as RealtimeAspectRatio))
    issues.push({ loc, msg: 'unsupported aspect ratio' })
}

function validateOptionalPrompt(value: unknown, loc: string[], issues: Issue[], nullable: boolean): void {
  if (value === undefined || (nullable && value === null))
    return
  if (typeof value !== 'string' || value.length > 500)
    issues.push({ loc, msg: 'prompt must be at most 500 characters' })
}

function validateStyleReference(reference: RealtimeCustomStyleReference, loc: string[], issues: Issue[]): void {
  if (!isRecord(reference) || !isCanonicalUuid(reference.mediaId))
    issues.push({ loc: [...loc, 'mediaId'], msg: 'mediaId must be a canonical UUID' })
  for (const key of ['url', 'sourceName', 'contentType'] as const) {
    const value = reference[key]
    if (value !== undefined && value !== null && typeof value !== 'string')
      issues.push({ loc: [...loc, key], msg: `${key} must be a string or null` })
  }
  if (typeof reference.url === 'string' && !isHttpUrl(reference.url))
    issues.push({ loc: [...loc, 'url'], msg: 'url must use http or https' })
  if (typeof reference.sourceName === 'string' && reference.sourceName.length > 500)
    issues.push({ loc: [...loc, 'sourceName'], msg: 'sourceName must be at most 500 characters' })
  if (typeof reference.contentType === 'string' && reference.contentType.length > 255)
    issues.push({ loc: [...loc, 'contentType'], msg: 'contentType must be at most 255 characters' })
}

function isHttpUrl(value: string): boolean {
  try {
    const url = new URL(value)
    return url.protocol === 'http:' || url.protocol === 'https:'
  }
  catch {
    return false
  }
}

function toStyleReferenceWire(reference: RealtimeCustomStyleReference | null): RealtimeCustomStyleReferenceWire | null {
  if (reference === null)
    return null
  return {
    media_id: reference.mediaId,
    ...(reference.url !== undefined ? { url: reference.url } : {}),
    ...(reference.sourceName !== undefined ? { source_name: reference.sourceName } : {}),
    ...(reference.contentType !== undefined ? { content_type: reference.contentType } : {}),
  }
}

function parseStyleReference(body: unknown): RealtimeCustomStyleReference {
  if (!isRecord(body))
    throw invalidResponse('custom style reference')
  return {
    mediaId: requiredUuid(body.media_id, 'media_id'),
    ...(optionalNullableString(body.url, 'url')),
    ...(optionalNullableString(body.source_name, 'source_name', 'sourceName')),
    ...(optionalNullableString(body.content_type, 'content_type', 'contentType')),
  }
}

function optionalNullableString(value: unknown, wireKey: string, publicKey = wireKey): Record<string, string | null> {
```

```
  if (value === undefined)
    return {}
  if (value !== null && typeof value !== 'string')
    throw invalidResponse('custom style reference', wireKey)
  return { [publicKey]: value }
}

function requiredString(value: unknown, key: string): string {
  if (typeof value !== 'string' || value.length < 1)
    throw invalidResponse('realtime', key)
  return value
}

function requiredUuid(value: unknown, key: string): string {
  if (!isCanonicalUuid(value))
    throw invalidResponse('realtime', key)
  return value
}

function requiredNumber(value: unknown, key: string): number {
  if (typeof value !== 'number' || !Number.isFinite(value))
    throw invalidResponse('realtime', key)
  return value
}

function invalidResponse(kind: string, field?: string): ValidationError {
  return new ValidationError(`Invalid realtime ${kind} response`, field ? [{ loc: [field], msg: `${field} is invalid` }] : undefined)
}

function unwrapData(body: unknown): unknown {
  return isRecord(body) && Object.prototype.hasOwnProperty.call(body, 'data') ? body.data : body
}

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value)
}

function isCanonicalUuid(value: unknown): value is string {
  return typeof value === 'string' && CANONICAL_UUID.test(value)
}

function isRealtimeImageType(value: unknown): value is 'media_input' | `${string}_job` {
  return value === 'media_input' || (typeof value === 'string' && IMAGE_JOB_TYPE.test(value))
}

/** @internal Validate UUIDs used in realtime path parameters and job reads. */
export function requireRealtimeUuid(value: unknown, name: string): string {
  if (!isCanonicalUuid(value))
    throw new ValidationError(`Invalid realtime ${name}`, [{ loc: [name], msg: `${name} must be a canonical UUID` }])
  return value
}
```

app/packages/fnf/src/realtime/index.ts

```
export { createRealtimeClient } from './client'
export type { RealtimeClient, RealtimeClientConfig } from './client'
export {
  buildCreateRealtimeCustomStyleRequest,
  buildRealtimeChainCostRequest,
  buildRealtimeChainEditRequest,
  buildRealtimeChainFinalizeRequest,
  buildUpdateRealtimeCustomStyleRequest,
  parseRealtimeChainCostEstimate,
  parseRealtimeChainEditResult,
  parseRealtimeChainFinalizeResult,
  parseRealtimeCustomStyle,
  parseRealtimeCustomStyleList,
  validateRealtimeCustomStyleListQuery,
} from './codec'
export type {
  CreateRealtimeCustomStyleInput,
  RealtimeAspectRatio,
  RealtimeChainCostEstimate,
  RealtimeChainCostInput,
  RealtimeChainEditInput,
  RealtimeChainEditOptions,
  RealtimeChainEditResult,
  RealtimeChainFinalizeInput,
  RealtimeChainFinalizeResult,
  RealtimeCustomStyle,
  RealtimeCustomStyleListQuery,
  RealtimeCustomStyleListResult,
  RealtimeCustomStyleReference,
  RealtimeEditParams,
  RealtimeEditPollOptions,
  RealtimeImageRef,
  RealtimeImageType,
  RealtimeMode,
  RealtimeReferenceMode,
  RealtimeResolution,
  RealtimeSettings,
  RealtimeTarget,
  UpdateRealtimeCustomStyleInput,
} from './types'
```

app/packages/fnf/src/realtime/types.ts

```
import type { Generation, MediaRef } from '../types'

export type RealtimeMode = 'basic' | 'advanced'
export type RealtimeResolution = '1k' | '2k'
export type RealtimeAspectRatio =
  | '16:9'
  | '4:3'
  | '1:1'
  | '3:4'
  | '9:16'
  | '2:3'
  | '1:2'
  | '2:1'
  | '4:5'
  | '3:2'
  | '5:4'
  | '21:9'

export type RealtimeReferenceMode = 'style' | 'character'
export type RealtimeTarget = 'canvas' | 'selection'

export type RealtimeImageType = 'media_input' | `${string}_job`

/** A persisted image upload or image-job result accepted by realtime editing. */
export type RealtimeImageRef = Omit<MediaRef, 'type'> & {
  type: RealtimeImageType
}

/** Structured controls compiled by the backend into a Flux Klein prompt. */
export interface RealtimeSettings {
  /** Built-in style key, or `custom` with customStyle/customStyleId. */
  style: string
  customStyle?: string
  mood?: string
  palette?: string
  customColors?: string[]
  sketchInfluence?: number
  referenceMode?: RealtimeReferenceMode
  target?: RealtimeTarget
  /** A saved custom style. Its reference image is appended server-side. */
  customStyleId?: string | null
}

type RealtimePromptEdit = {
  prompt: string
  settings?: never
}

type RealtimeSettingsEdit = {
  prompt?: never
  settings: RealtimeSettings
}

/** Exactly one of prompt or settings is required. */
export type RealtimeEditParams = (RealtimePromptEdit | RealtimeSettingsEdit) & {
  mode?: RealtimeMode
  resolution: RealtimeResolution
  aspectRatio: RealtimeAspectRatio
  /**
   * Explicit source images for this attempt. Up to four are accepted. Saved
   * style references are expanded server-side by the existing realtime API.
   */
  images: RealtimeImageRef[]
}

export interface RealtimeChainEditInput {
  /** Omit/null to begin a chain; pass the previous result's chainId to continue it. */
  chainId?: string | null
```

```
  params: RealtimeEditParams
}

export interface RealtimeChainEditResult {
  chainId: string
  jobSetId: string
  /** Pass this to RealtimeClient.getEditJob/pollEditJob. */
  jobId: string
  status: string
  createdAt: number
}

export interface RealtimeChainCostInput {
  mode?: RealtimeMode
  resolution: RealtimeResolution
  aspectRatio: RealtimeAspectRatio
}

export interface RealtimeChainCostEstimate {
  /** Rounded display value returned by fnf. */
  credits: number
  /** Exact fractional credit value. */
  creditsExact: number
}

export interface RealtimeChainFinalizeInput {
  chainId: string
}

export interface RealtimeChainFinalizeResult {
  chainId: string
  status: 'finalized' | 'released'
}

export interface RealtimeCustomStyleReference {
  mediaId: string
  url?: string | null
  sourceName?: string | null
  contentType?: string | null
}

export interface RealtimeCustomStyle {
  id: string
  prompt: string
  reference: RealtimeCustomStyleReference | null
}

export interface RealtimeCustomStyleListQuery {
  /** Float epoch-seconds cursor returned by the previous page. */
  cursor?: number
  size?: number
}

export interface RealtimeCustomStyleListResult {
  items: RealtimeCustomStyle[]
  nextCursor?: number
}

export interface CreateRealtimeCustomStyleInput {
  prompt?: string
  reference?: RealtimeCustomStyleReference | null
}

export interface UpdateRealtimeCustomStyleInput {
  /** Omit/null to keep the current prompt. */
  prompt?: string | null
  /** Omit/null to keep the current reference; use clearReference to remove it. */
  reference?: RealtimeCustomStyleReference | null
  clearReference?: boolean
}
```

```ts
/** Per-call transport metadata; confirmation tokens are opaque to the SDK. */
export interface RealtimeChainEditOptions {
  confirmationToken?: string
}

export interface RealtimeEditPollOptions {
  signal?: AbortSignal
  /** Fires after every successful job read, including intermediate statuses. */
  onProgress?: (generation: Generation) => void
}

// Raw adapter wire contracts. They stay out of the public realtime barrel;
// application-facing inputs/results above are camelCase.
export interface RealtimeSettingsWire {
  style: string
  custom_style?: string
  mood?: string
  palette?: string
  custom_colors?: string[]
  sketch_influence?: number
  reference_mode?: RealtimeReferenceMode
  target?: RealtimeTarget
  custom_style_id?: string | null
}

export interface RealtimeEditParamsWire extends Record<string, unknown> {
  prompt?: string
  settings?: RealtimeSettingsWire
  mode: RealtimeMode
  resolution: RealtimeResolution
  aspect_ratio: RealtimeAspectRatio
  medias: Array<{ role: 'image', data: { id: string, type: RealtimeImageType } }>
}

export interface RealtimeChainEditWireRequest extends Record<string, unknown> {
  chain_id?: string | null
  params: RealtimeEditParamsWire
  confirmation_token?: string
}

export interface RealtimeChainCostWireRequest extends Record<string, unknown> {
  mode: RealtimeMode
  resolution: RealtimeResolution
  aspect_ratio: RealtimeAspectRatio
}

export interface RealtimeChainFinalizeWireRequest extends Record<string, unknown> {
  chain_id: string
}

export interface RealtimeCustomStyleReferenceWire {
  media_id: string
  url?: string | null
  source_name?: string | null
  content_type?: string | null
}

export interface CreateRealtimeCustomStyleWireRequest extends Record<string, unknown> {
  prompt: string
  reference?: RealtimeCustomStyleReferenceWire | null
}

export interface UpdateRealtimeCustomStyleWireRequest extends Record<string, unknown> {
  prompt?: string | null
  reference?: RealtimeCustomStyleReferenceWire | null
  clear_reference?: boolean
}
```

app/packages/fnf/src/references/__tests__/references.test.ts

```
import type { ReferenceBackend } from '../../backend'
import { describe, expect, it, vi } from 'vitest'
import { createReferenceClient } from '../client'
import { parseReference } from '../codec'

function adapter(overrides: Partial<ReferenceBackend> = {}): ReferenceBackend {
  return {
    getReferenceElement: vi.fn(),
    listReferenceElements: vi.fn(async () => ({ items: [], next_cursor: null })),
    ...overrides,
  }
}

const characterElementBody = {
  id: '11111111-1111-4111-8111-111111111111',
  name: 'Ada',
  category: 'character',
  status: 'completed',
  description: 'Consistent character',
  medias: [{
    id: '22222222-2222-4222-8222-222222222222',
    type: 'media_input',
    url: 'https://cdn.example/ada.webp',
  }],
  video_medias: [],
  character_id: '33333333-3333-4333-8333-333333333333',
  created_at: 123,
}

describe('Elements reference workflow', () => {
  it('maps a Character Element and exposes the Soul character id', () => {
    expect(parseReference(characterElementBody)).toEqual({
      id: '11111111-1111-4111-8111-111111111111',
      name: 'Ada',
      category: 'character',
      status: 'completed',
      description: 'Consistent character',
      medias: [{
        id: '22222222-2222-4222-8222-222222222222',
        type: 'media_input',
        url: 'https://cdn.example/ada.webp',
      }],
      videoMedias: [],
      characterId: '33333333-3333-4333-8333-333333333333',
      createdAt: 123,
    })
  })

  it('reads one Element instead of a custom-reference record', async () => {
    const backend = adapter({ getReferenceElement: vi.fn(async () => characterElementBody) })
    const client = createReferenceClient({ adapter: backend })

    await expect(client.get(characterElementBody.id)).resolves.toMatchObject({
      id: characterElementBody.id,
      characterId: characterElementBody.character_id,
    })
    expect(backend.getReferenceElement).toHaveBeenCalledWith(characterElementBody.id)
  })

  it('polls one processing Element until it completes', async () => {
    const statuses = ['processing', 'completed']
    const backend = adapter({
      getReferenceElement: vi.fn(async () => ({
        ...characterElementBody,
        status: statuses.shift(),
      })),
    })
    const client = createReferenceClient({ adapter: backend, scheduler: { sleep: async () => {} } })
```

```
    const completed = await client.wait({
      id: characterElementBody.id,
      name: 'Ada',
      category: 'character',
      status: 'processing',
      description: null,
      medias: [],
      videoMedias: [],
    })

    expect(completed.status).toBe('completed')
    expect(backend.getReferenceElement).toHaveBeenCalledTimes(2)
  })

  it('lists only Character Elements when requested', async () => {
    const backend = adapter({
      listReferenceElements: vi.fn(async () => ({
        items: [characterElementBody],
        next_cursor: 123.5,
      })),
    })
    const client = createReferenceClient({ adapter: backend })

    await expect(client.list({ category: 'character', size: 20 })).resolves.toMatchObject({
      items: [{ id: characterElementBody.id, category: 'character' }],
      nextCursor: 123.5,
    })
    expect(backend.listReferenceElements).toHaveBeenCalledWith({ category: 'character', size: 20 })
  })
})
```

app/packages/fnf/src/references/client.ts

```
import type { ReferenceBackend } from '../backend'
import type {
  Reference,
  ReferenceListQuery,
  ReferenceListResult,
  ReferenceWaitOptions,
} from './types'
import type { FnfObservabilityOptions } from '../observability'
import { ApiJobError, JobAbortedError, JobTimeoutError } from '../errors'
import { createObservabilityContext, observeAsync } from '../observability'
import { parseReference } from './codec'

export interface ReferenceClientConfig {
  adapter: ReferenceBackend
  poll?: { intervalMs?: number, timeoutMs?: number }
  scheduler?: { sleep?: (milliseconds: number) => Promise<void> }
  observability?: FnfObservabilityOptions
}

export interface ReferenceClient {
  get: (id: string) => Promise<Reference>
  list: (query?: ReferenceListQuery) => Promise<ReferenceListResult>
  wait: (reference: Reference, options?: ReferenceWaitOptions) => Promise<Reference>
}

const DEFAULT_INTERVAL_MS = 2000
const DEFAULT_TIMEOUT_MS = 600_000

function isTerminal(reference: Reference): boolean {
  return reference.status !== 'processing' && reference.status !== 'ip_checking'
}

export function createReferenceClient(config: ReferenceClientConfig): ReferenceClient {
  const observability = createObservabilityContext(config.observability)
  const intervalMs = config.poll?.intervalMs ?? DEFAULT_INTERVAL_MS
  const timeoutMs = config.poll?.timeoutMs ?? DEFAULT_TIMEOUT_MS
  const sleep = config.scheduler?.sleep ?? (milliseconds => new Promise(resolve => setTimeout(resolve, milliseconds)))

  const get = (id: string): Promise<Reference> => observeAsync(
    observability,
    'fnf.reference.get',
    { reference_id: id },
    async () => {
      if (!config.adapter.getReferenceElement)
        throw new ApiJobError('not_supported', 'Reference reads are not supported by this adapter')
      return parseReference(await config.adapter.getReferenceElement(id))
    },
    { successAttributes: reference => ({ reference_id: reference.id, status: reference.status }) },
  )

  const client: ReferenceClient = {
    get,
    list: (query = {}) => observeAsync(observability, 'fnf.reference.list', {}, async () => {
      if (!config.adapter.listReferenceElements)
        throw new ApiJobError('not_supported', 'Reference listing is not supported by this adapter')
      const body = await config.adapter.listReferenceElements(query)
      if (typeof body !== 'object' || body === null || Array.isArray(body))
        throw new ApiJobError('invalid_response', 'Invalid reference list response')
      const page = body as { items?: unknown, next_cursor?: unknown, cursor?: unknown, has_more?: unknown }
      if (!Array.isArray(page.items))
        throw new ApiJobError('invalid_response', 'Invalid reference list response')
      const rawCursor = page.next_cursor ?? page.cursor
      const nextCursor = page.has_more === false
        ? undefined
        : typeof rawCursor === 'number' ? rawCursor : undefined
      return {
        items: page.items.map(parseReference),
        ...(nextCursor !== undefined ? { nextCursor } : {}),
```

```
      }
    }, { successAttributes: page => ({ reference_count: page.items.length }) }),
    wait: async (reference, options = {}) => {
      if (isTerminal(reference))
        return reference
      const startedAt = Date.now()
      const waitIntervalMs = options.intervalMs ?? intervalMs
      const waitTimeoutMs = options.timeoutMs ?? timeoutMs
      let current = reference
      while (Date.now() - startedAt < waitTimeoutMs) {
        if (options.signal?.aborted)
          throw new JobAbortedError()
        await sleep(waitIntervalMs)
        if (options.signal?.aborted)
          throw new JobAbortedError()
        current = await get(reference.id)
        options.onProgress?.(current)
        if (isTerminal(current))
          return current
      }
      throw new JobTimeoutError(reference.id, waitTimeoutMs)
    },
  }

  return client
}
```

app/packages/fnf/src/references/codec.ts

```
import type { Reference, ReferenceCategory, ReferenceMedia, ReferenceStatus } from './types'
import { ValidationError } from '../errors'

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value)
}

function optionalString(value: unknown): string | undefined {
  return typeof value === 'string' && value.length > 0 ? value : undefined
}

function normalizeStatus(value: unknown): ReferenceStatus {
  if (
    value === 'processing'
    || value === 'completed'
    || value === 'kling_failed'
    || value === 'voice_clone_failed'
    || value === 'failed'
    || value === 'ip_checking'
    || value === 'ip_detected'
    || value === 'nsfw'
  ) {
    return value
  }
  return null
}

function normalizeCategory(value: unknown): ReferenceCategory {
  if (
    value === 'auto'
    || value === 'character'
    || value === 'environment'
    || value === 'prop'
    || value === 'auto:character'
    || value === 'auto:environment'
    || value === 'auto:prop'
    || value === 'voice'
    || value === 'character_ip_verified'
    || value === 'environment_ip_verified'
  ) {
    return value
  }
  return null
}

function parseMediaList(value: unknown): ReferenceMedia[] {
  if (!Array.isArray(value))
    return []

  return value.flatMap((item) => {
    if (!isRecord(item))
      return []
    const id = optionalString(item.id)
    const url = optionalString(item.url)
    const type = optionalString(item.type)
    if (!id || !url || !type)
      return []
    return [{
      id,
      url,
      type,
    }]
  })
}

export function parseReference(body: unknown): Reference {
  const value = isRecord(body) && isRecord(body.data) ? body.data : body
  if (!isRecord(value))
```

```
    throw new ValidationError('Invalid reference response')

  const id = optionalString(value.id) ?? optionalString(value.id_)
  if (!id)
    throw new ValidationError('Invalid reference response', [{ loc: ['id'], msg: 'reference id is required' }])

  const name = optionalString(value.name)
  if (!name)
    throw new ValidationError('Invalid reference response', [{ loc: ['name'], msg: 'reference name is required' }])

  const description = optionalString(value.description) ?? null
  const characterId = optionalString(value.character_id) ?? optionalString(value.soul_character_id)
  return {
    id,
    name,
    category: normalizeCategory(value.category),
    status: normalizeStatus(value.status),
    description,
    medias: parseMediaList(value.medias),
    videoMedias: parseMediaList(value.video_medias),
    ...(characterId ? { characterId } : {}),
    ...(typeof value.created_at === 'number' ? { createdAt: value.created_at } : {}),
  }
}
```

app/packages/fnf/src/references/index.ts

```
export { createReferenceClient } from './client'
export type { ReferenceClient, ReferenceClientConfig } from './client'
export { parseReference } from './codec'
export type {
  Reference,
  ReferenceCategory,
  ReferenceMedia,
  ReferenceListQuery,
  ReferenceListResult,
  ReferenceStatus,
  ReferenceWaitOptions,
} from './types'
```

app/packages/fnf/src/references/types.ts

```
export type ReferenceCategory =
  | 'auto'
  | 'character'
  | 'environment'
  | 'prop'
  | 'auto:character'
  | 'auto:environment'
  | 'auto:prop'
  | 'voice'
  | 'character_ip_verified'
  | 'environment_ip_verified'
  | null

export type ReferenceStatus =
  | 'processing'
  | 'completed'
  | 'kling_failed'
  | 'voice_clone_failed'
  | 'failed'
  | 'ip_checking'
  | 'ip_detected'
  | 'nsfw'
  | null

export interface ReferenceMedia {
  id: string
  url: string
  type: string
}

export interface Reference {
  id: string
  name: string
  category: ReferenceCategory
  status: ReferenceStatus
  description: string | null
  medias: ReferenceMedia[]
  videoMedias: ReferenceMedia[]
  /** Custom-reference id for Character Elements; pass it to a compatible Soul generation model. */
  characterId?: string
  createdAt?: number
}

export interface ReferenceListQuery {
  cursor?: number
  size?: number
  category?: Exclude<ReferenceCategory, null>
}

export interface ReferenceListResult {
  items: Reference[]
  nextCursor?: number
}

export interface ReferenceWaitOptions {
  signal?: AbortSignal
  intervalMs?: number
  timeoutMs?: number
  onProgress?: (reference: Reference) => void
}
```

app/packages/fnf/src/registry.ts

```
import type { JobEntry } from './define-job'

export type Registry = Map<string, JobEntry>

export function buildRegistry(jobs: readonly JobEntry[]): Registry {
  const registry: Registry = new Map()
  for (const job of jobs) {
    // Last-write-wins here would silently serialize through the wrong codecs
    // while the type level still advertises both entries — fail loudly instead.
    if (registry.has(job.jobSetType))
      throw new Error(`buildRegistry: duplicate jobSetType '${job.jobSetType}' — register one entry per backend job type`)
    registry.set(job.jobSetType, job)
  }
  return registry
}
```

app/packages/fnf/src/selectors.ts

```
import type { JobEntry } from './define-job'
import type { Generation, GenerationResults, GenerationStatus, MediaRef, OutputType } from './types'
import { isTerminal, TERMINAL_STATUSES } from './types'

/**
 * Selectors — pure, tree-shakeable derivations over the read model, so the
 * production layer doesn't re-derive them ad-hoc in every component
 * (`results?.raw?.url ?? min`, status bucketing, kind-by-extension, …).
 */

// ■■ urls ■■

/** The full-quality result url, when the generation has produced one. */
export function getRawUrl(generation: Generation): string | undefined {
  return generation.results?.rawUrl
}

/** The best url for a grid/preview: min → thumbnail → raw. */
export function getPreviewUrl(generation: Generation): string | undefined {
  const results = generation.results
  return results?.minUrl ?? results?.thumbnailUrl ?? results?.rawUrl
}

// ■■ status ■■

/** The three buckets UI actually branches on. */
export type JobPhase = 'progress' | 'failed' | 'completed'

const FAILED_STATUSES: ReadonlySet<GenerationStatus> = new Set(
  [...TERMINAL_STATUSES].filter(status => status !== 'completed'),
)

/**
 * Bucket a status (or generation) into the UI phases: anything non-terminal —
 * including statuses this SDK build doesn't know yet — is `progress`.
 */
export function getJobPhase(source: Generation | GenerationStatus): JobPhase {
  const status = typeof source === 'string' ? source : source.status
  if (status === 'completed')
    return 'completed'
  return FAILED_STATUSES.has(status) ? 'failed' : 'progress'
}

/** Terminal = the backend will not change this job again (completed OR failed). */
export function isTerminalJobStatus(status: GenerationStatus): boolean {
  return isTerminal(status)
}

/**
 * Failed-family terminal statuses (failed / nsfw / canceled / ip_detected).
 * NOTE: broader than `wait({ throwOnFail })`, which deliberately exempts
 * `canceled` (a user action, not a failure) — and than the product's fail
 * bucket (failed/nsfw/ip_detected). Here `canceled` still buckets as 'failed'
 * because the UI phases have nowhere else terminal-but-not-completed to go.
 */
export function isFailedJobStatus(status: GenerationStatus): boolean {
  return FAILED_STATUSES.has(status)
}

// ■■ predicates over the read model (UI branches + TS narrowing) ■■

/** The generation finished successfully. */
export function isCompleted(generation: Generation): boolean {
  return generation.status === 'completed'
}

/** The generation ended in a failed-family status (failed/nsfw/canceled/ip_detected). */
export function isFailed(generation: Generation): boolean {
```

```
  return FAILED_STATUSES.has(generation.status)
}

/** Still in flight — the backend will keep changing it. */
export function isGenerating(generation: Generation): boolean {
  return !isTerminal(generation.status)
}

/** NARROWING guard: after it, `generation.results` is non-optional — no `?.` in render code. */
export function hasResult(generation: Generation): generation is Generation & { results: GenerationResults } {
  return generation.results != null
}

/**
 * NARROWING guard by model: after `isFromJob(gen, seedance2_0)`, `gen.input` is
 * typed as that model's submit input — `gen.input.settings.duration`
 * autocompletes. The runtime check is just the model discriminator; the typed
 * view is sound because parse round-trips the declared shape.
 */
export function isFromJob<Type extends string, Settings, Env>(
  generation: Generation,
  entry: JobEntry<Type, Settings, Env>,
): generation is Generation & { model: Type, input: Generation['input'] & Env & { model: Type, settings: Settings } } {
  return generation.model === entry.jobSetType
}

// ■■ media kind ■■

const VIDEO_EXTENSIONS = new Set(['mp4', 'mov', 'webm', 'm4v', 'avi', 'mkv'])
const IMAGE_EXTENSIONS = new Set(['png', 'jpg', 'jpeg', 'webp', 'gif', 'avif', 'heic', 'heif', 'bmp', 'svg'])

/**
 * The visual kind of a url / media ref / generation: a Generation answers from
 * its declared output type; urls answer by extension (query/hash stripped);
 * anything unrecognizable is undefined.
 */
export function getMediaType(source: string | MediaRef | Generation | undefined): OutputType | undefined {
  if (source == null)
    return undefined
  if (typeof source === 'object' && 'status' in source)
    return source.type
  const url = typeof source === 'string' ? source : source.url
  if (!url)
    return undefined
  const path = url.split(/[?#]/)[0]
  const extension = path.slice(path.lastIndexOf('.') + 1).toLowerCase()
  if (VIDEO_EXTENSIONS.has(extension))
    return 'video'
  if (IMAGE_EXTENSIONS.has(extension))
    return 'image'
  return undefined
}
```

app/packages/fnf/src/spec.ts

```
import type { JobEntry } from './define-job'
import type { Generation, GenerationInput, GenerationResults, GenerationStatus, MediaInput, PromptInput } from './types'
import { ApiJobError, ValidationError } from './errors'
import { checkMedia, mediaCodec } from './groups/media'
import { promptCodec } from './groups/prompt'

export interface JobResponse {
  id: string
  job_set_id?: string
  job_set_parent_id?: string | null
  status: string
  result_url?: string | null
  min_result_url?: string | null
  thumbnail_url?: string | null
  params?: Record<string, unknown>
  created_at?: number
  fail_reason?: string | null
}

/**
 * Parse a job's settings (applies schema defaults), wrapping a raw zod failure
 * into the typed `ValidationError` — so callers see a `validation` error with
 * structured issues, not a bare ZodError surfaced as `unknown`. Does NOT
 * snap/normalize values; that is the separate, opt-in `adjust()` step.
 */
export function parseSettings(input: GenerationInput, entry: JobEntry): Record<string, unknown> {
  try {
    return entry.settingsSchema.parse(input.settings ?? {}) as Record<string, unknown>
  }
  catch (err) {
    if (err instanceof ApiJobError)
      throw err
    throw new ValidationError('Invalid settings', (err as { issues?: unknown }).issues ?? String(err))
  }
}

/**
 * Build the wire `params` body posted to the backend (prompt/media envelopes +
 * settings, flattened). No normalization here — settings go as provided (the
 * backend clamps); call `adjust()` first for client-side snapping.
 */
export function buildWireParams(input: GenerationInput, entry: JobEntry): Record<string, unknown> {
  const settings = parseSettings(input, entry)

  // Cardinality + cross-role rules + the job's cross-field validate hook —
  // aggregated, so the caller sees ALL problems at once (unlike fnf-web's
  // first-throw), before any wire building or I/O.
  const issues = [
    ...(entry.media ? checkMedia(entry.media, input.media) : []),
    // validate sees parsed settings (defaults applied) — same contract as credits.
    ...(entry.validate?.({ ...input, settings }) ?? []),
  ]
  if (issues.length > 0)
    throw new ValidationError(issues.map(i => i.msg).join('; '), issues)

  let wire: Record<string, unknown> = {}
  if (entry.prompt)
    Object.assign(wire, promptCodec.serialize(input.prompt ?? {}))
  if (entry.media)
    Object.assign(wire, mediaCodec(entry.media).serialize(input.media ?? {}))
  // Settings: the local key is the wire key unless the job tagged an explicit
  // wire name via `z.wire` (e.g. aspectRatio → aspect_ratio). Explicitly-passed
  // undefined stays off the wire (structured transports would carry the key).
  for (const [key, value] of Object.entries(settings)) {
    if (value === undefined)
      continue
    wire[entry.wireNames[key] ?? key] = value
  }
```

```
  // Universal targeting fields (every model accepts them on the wire).
  if (input.folderId)
    wire.folder_id = input.folderId
  if (input.parentId)
    wire.parent_id = input.parentId
  // `count` is NOT a wire param — it's the client-side fan-out handled in submit().
  // `batch_size`, where a model declares it, is an ordinary setting (already above).
  if (input.extra)
    Object.assign(wire, input.extra)
  if (entry.finalize)
    wire = entry.finalize(wire, input)
  return wire
}

export function parseGeneration(job: JobResponse, entry: JobEntry): Generation {
  const stored = job.params ?? {}
  // A lossy finalize (keys deleted/renamed on serialize) gets its inverse
  // applied here, so get-then-resubmit round-trips the original settings
  // instead of falling back to schema defaults.
  const wire = entry.restore ? { ...stored, ...entry.restore(stored) } : stored
  const media = entry.media ? mediaCodec(entry.media) : undefined

  const prompt = entry.prompt ? (promptCodec.parse(wire) as PromptInput) : undefined
  const mediaInput = media ? (media.parse(wire) as MediaInput) : undefined

  const claimed = new Set<string>([
    ...(entry.prompt ? promptCodec.wireKeys : []),
    ...(media ? media.wireKeys : []),
    'folder_id',
    'parent_id',
  ])
  const folderId = typeof wire.folder_id === 'string' ? wire.folder_id : undefined
  const parentId = typeof wire.parent_id === 'string' ? wire.parent_id : undefined
  // Invert the explicit wire names so wire keys map back to local settings keys
  // (with the identity fallback for unmapped/legacy keys).
  const wireToLocal: Record<string, string> = {}
  for (const [local, wireKey] of Object.entries(entry.wireNames))
    wireToLocal[wireKey] = local
  const settings: Record<string, unknown> = {}
  const extra: Record<string, unknown> = {}
  for (const [key, value] of Object.entries(wire)) {
    if (claimed.has(key))
      continue
    const local = wireToLocal[key] ?? key
    if (local in entry.settingsMap)
      settings[local] = value
    else extra[key] = value
  }

  const input: GenerationInput = {
    model: entry.jobSetType,
    ...(folderId ? { folderId } : {}),
    ...(parentId ? { parentId } : {}),
    ...(prompt ? { prompt } : {}),
    ...(mediaInput ? { media: mediaInput } : {}),
    settings,
    ...(Object.keys(extra).length > 0 ? { extra } : {}),
  }

  const results = buildResults(entry, job)

  return {
    id: job.id,
    ...(job.job_set_id ? { jobSetId: job.job_set_id } : {}),
    ...(job.job_set_parent_id ? { parentJobSetId: job.job_set_parent_id } : {}),
    model: entry.jobSetType,
    type: entry.outputType,
    status: job.status as GenerationStatus,
    input,
    ...(results ? { results } : {}),
```

```
    ...(job.fail_reason ? { failReason: job.fail_reason } : {}),
    ...(job.created_at !== undefined ? { createdAt: job.created_at } : {}),
  }
}

function buildResults(entry: JobEntry, job: JobResponse): GenerationResults | undefined {
  if (job.status !== 'completed' || !job.result_url)
    return undefined
  const results: GenerationResults = { rawUrl: job.result_url }
  if (entry.outputType === 'image' && job.min_result_url)
    results.minUrl = job.min_result_url
  if (entry.outputType === 'image' && !job.min_result_url && job.thumbnail_url)
    results.thumbnailUrl = job.thumbnail_url
  if (entry.outputType === 'video' && (job.thumbnail_url || job.min_result_url))
    results.thumbnailUrl = job.thumbnail_url ?? job.min_result_url ?? undefined
  return results
}
```

app/packages/fnf/src/transport.ts

```
/**
 * Low-level HTTP request/response shape used by HTTP-based adapters. Kept
 * deliberately small and JSON-serializable so it survives Comlink/iframe
 * boundaries. Non-HTTP adapters (websocket, in-process) need not use it.
 */
export interface TransportRequest {
  method: 'GET' | 'POST' | 'PUT' | 'PATCH' | 'DELETE'
  path: string
  body?: unknown
  headers?: Record<string, string>
  /** Cancels the in-flight request. Not serializable — strip before a Comlink/iframe hop. */
  signal?: AbortSignal
}

export interface TransportResponse {
  status: number
  body: unknown
}

export type Transport = (req: TransportRequest) => Promise<TransportResponse>
```

app/packages/fnf/src/typed-api.test-d.ts

```
import type { FnfObservabilityOptions, FnfObservationEvent, FnfObservationPhase, FnfObserver, GenerationBackend, ProfileBackend } from './index'
/* Type-level checks for the model/settings autocomplete on submit.
 * Checked by `tsc` (included in src/**), not run by vitest (not a *.test.ts).
 * Backends are declared, not constructed — the concrete adapters live in
 * @higgsfield/fnf-adapters (typed there); only the ports matter here. */
import {
  bytedanceVideoUpscale,
  createJobClient,
  createNoopObserver,
  createObservabilityContext,
  createProfileClient,
  defineJob,
  gptImage2,
  grokImagine,
  grokImagineV15,
  happyHorse,
  higgsfieldVideoUpscale,
  kling3_0,
  kling3MotionControl,
  nanoBanana2,
  nanoBanana2Upscale,
  nanoBananaFlash,
  recraftV41Image,
  RecraftV41Model,
  seedance2_0,
  Seedance2AspectRatio,
  seedreamV4_5,
  soraEnhanceVideo,
  soulCinemaImage,
  soulV2Image,
  topazImageGenerativeUpscale,
  topazImageUpscale,
  topazVideoUpscale,
  veo3_1Lite,
  wan27,
  z,
} from './index'

declare const backend: GenerationBackend
declare const profileBackend: ProfileBackend

const client = createJobClient({
  adapter: backend,
  jobs: [nanoBanana2, seedance2_0], // ← the source of autocomplete
})

const imageClient = createJobClient({
  adapter: backend,
  jobs: [soulV2Image, soulCinemaImage, gptImage2, seedreamV4_5, nanoBanana2, nanoBananaFlash, nanoBanana2Upscale, recraftV41Image, topazImageUpscale, topazImageGenerativeUpscale],
})

const videoClient = createJobClient({
  adapter: backend,
  jobs: [seedance2_0, kling3_0, kling3MotionControl, happyHorse, grokImagine, grokImagineV15, veo3_1Lite, wan27, topazVideoUpscale, higgsfieldVideoUpscale, soraEnhanceVideo, bytedanceVideoUpscale],
})

const profileClient = createProfileClient({
  profileAdapter: profileBackend,
})

const observer: FnfObserver = (event) => {
  expectType<FnfObservationEvent>(event)
  expectType<FnfObservationPhase>(event.phase)
  expectType<string>(event.id)
  expectType<string>(event.traceId)
  expectType<string | number | boolean | null | undefined>(event.attributes.model)
}
```

```
const observabilityOptions: FnfObservabilityOptions = {
  observer,
  traceId: 'trace-1',
  attributes: { app: 'typed-api', debug: true, count: 1, empty: null },
}

void createObservabilityContext(observabilityOptions)
void createNoopObserver()

// @ts-expect-error observability attributes accept only primitive metadata values
const badObservabilityOptions: FnfObservabilityOptions = { attributes: { unsafe: { nested: true } } }
void badObservabilityOptions

declare function expectType<T>(value: T): void

// model autocompletes to the registered job set types; settings typed per model
void client.submit({ model: 'nano_banana_2', prompt: { instruction: 'x' }, settings: { aspectRatio: '1:1', resolution: '2k' } })
void client.submit({ model: 'seedance_2_0', settings: { duration: 10, aspectRatio: '16:9' } })

// optional/defaulted settings may be omitted (nano: resolution defaults; seedance: seed optional)
void client.submit({ model: 'nano_banana_2', settings: { aspectRatio: '1:1' } })
void client.submit({ model: 'seedance_2_0', settings: { duration: 8, aspectRatio: '16:9' } })

// @ts-expect-error duration is required for seedance_2_0
void client.submit({ model: 'seedance_2_0', settings: { aspectRatio: '16:9' } })

// @ts-expect-error unknown model
void client.submit({ model: 'not_a_model', settings: {} })

// @ts-expect-error wrong resolution enum value
void client.submit({ model: 'nano_banana_2', settings: { aspectRatio: '1:1', resolution: '8k' } })

// @ts-expect-error settings from a different model
void client.submit({ model: 'nano_banana_2', settings: { duration: 5 } })

// aspectRatio is the literal union of the model's allowed ratios
// @ts-expect-error '5:5' is not a nano_banana_2 aspect ratio
void client.submit({ model: 'nano_banana_2', settings: { aspectRatio: '5:5' } })

// seedance duration is a continuous 4–15 range (plain number type; range enforced at runtime)
void client.submit({ model: 'seedance_2_0', settings: { duration: 7, aspectRatio: '16:9' } })

// object-enum members are accepted interchangeably with raw values (structural, not nominal)
void client.submit({ model: 'seedance_2_0', settings: { duration: 8, aspectRatio: Seedance2AspectRatio.r16x9 } })

void imageClient.submit({ model: 'text2image_soul_v2', settings: { aspectRatio: '3:4', quality: '1080p' } })
void imageClient.submit({ model: 'soul_cinematic', settings: { cinematicVariant: 'sultan', batchSize: 2 } })
void imageClient.submit({ model: 'gpt_image_2', prompt: { instruction: 'x' }, settings: { aspectRatio: 'auto', quality: 'high', resolution: '2k', subModel: 'videotape-alpha' } })
void imageClient.submit({ model: 'seedream_v4_5', prompt: { instruction: 'x' }, settings: { aspectRatio: '3:4', quality: 'basic' } })
void imageClient.submit({ model: 'nano_banana_flash', prompt: { instruction: 'x' }, media: { image: { id: 'u', type: 'media_input' } }, settings: { resolution: '2k' } })
void imageClient.submit({ model: 'nano_banana_2_upscale', media: { image: { id: 'u', type: 'media_input' } }, settings: { resolution: '4k', aspectRatio: '16:9' } })
void imageClient.submit({ model: 'recraft_v4_1', prompt: { instruction: 'x' }, settings: { model: RecraftV41Model.vector, resolution: '2k', colors: ['#ffffff'] } })
void imageClient.submit({ model: 'topaz_image', media: { image: { id: 'u', type: 'media_input' } }, settings: { sourceWidth: 512, sourceHeight: 512, factor: 'x2', model: 'Standard V2' } })
void imageClient.submit({ model: 'topaz_image_generative', prompt: { instruction: 'restore' }, media: { image: { id: 'u', type: 'media_input' } }, settings: { sourceWidth: 512, sourceHeight: 512, crea

void videoClient.submit({ model: 'seedance_2_0', prompt: { instruction: 'x' }, settings: { duration: 5, aspectRatio: '16:9', mode: 'fast', bitrateMode: 'high' } })
void videoClient.submit({ model: 'kling3_0', prompt: { instruction: 'x', enhance: true }, media: { start_image: { id: 's', type: 'media_input' } }, settings: { duration: 5, aspectRatio: '16:9', mode:
void videoClient.submit({ model: 'kling3_0_motion_control', media: { image: { id: 'i', type: 'media_input' }, video: { id: 'v', type: 'video_input' } }, settings: { mode: 'pro', characterOrientation:
void videoClient.submit({ model: 'happy_horse_video', prompt: { instruction: 'x' }, settings: { resolution: '1080p', aspectRatio: '3:4', duration: 5, batchSize: 2 } })
void videoClient.submit({ model: 'grok_video', prompt: { instruction: 'x' }, settings: { resolution: '720p', aspectRatio: 'auto', duration: 6 } })
void videoClient.submit({ model: 'grok_video_v15', prompt: { instruction: 'x' }, media: { start_image: { id: 's', type: 'media_input' } }, settings: { resolution: '480p', duration: 2 } })
void videoClient.submit({ model: 'veo3_1_lite', prompt: { instruction: 'x' }, media: { start_image: { id: 's', type: 'media_input' }, end_image: { id: 'e', type: 'media_input' } }, settings: { resolut
void videoClient.submit({ model: 'wan2_7', prompt: { instruction: 'x', negative: 'blur' }, settings: { quality: '1080p', aspectRatio: '4:3', duration: 5 } })
void videoClient.submit({ model: 'topaz_video', media: { video: { id: 'v', type: 'video_input' } }, settings: { sourceWidth: 1280, sourceHeight: 720, model: 'slp-2.5', enhancementModel: 'iris-3', fram
void videoClient.submit({ model: 'video_upscale', media: { video: { id: 'v', type: 'video_input' } }, settings: { sourceWidth: 1280, sourceHeight: 720, scaleFactor: '4k' } })
void videoClient.submit({ model: 'video_deflicker', media: { video: { id: 'v', type: 'video_input' } }, settings: { sourceWidth: 1280, sourceHeight: 720 } })
void videoClient.submit({ model: 'bytedance_video_upscale', media: { video: { id: 'v', type: 'video_input' } }, settings: { sourceWidth: 1280, sourceHeight: 720, resolution: '2k', preset: 'aigc', fps:

void profileClient.getUser().then((user) => {
  if (user) {
```

```
    expectType<string | null>(user.workspaceId)
    expectType<'private' | 'shared' | null>(user.workspaceType)
    expectType<'owner' | 'admin' | 'member' | null>(user.workspaceRole)
    expectType<string | null>(user.planType)
    expectType<string | null>(user.billingPeriod)
    expectType<number>(user.totalPlanCredits)
    // @ts-expect-error profile user fields are camelCase in the SDK
    expectType<unknown>(user.workspace_id)
  }
})

void profileClient.listWorkspaces().then((workspaces) => {
  expectType<string | null | undefined>(workspaces[0]?.clerkOrganizationId)
  expectType<string | null | undefined>(workspaces[0]?.avatarUrl)
  // @ts-expect-error workspace fields are camelCase in the SDK
  expectType<unknown>(workspaces[0]?.clerk_organization_id)
})

void profileClient.getCredits({ includeOnDemand: true }).then((credits) => {
  expectType<number | undefined>(credits?.totalAvailableCredits)
  expectType<number | undefined>(credits?.raw.totalAvailableCredits)
})

void profileClient.switchWorkspace({ workspaceId: 'workspace-1' }).then((snapshot) => {
  expectType<string | undefined>(snapshot.currentWorkspace?.id)
  expectType<number | undefined>(snapshot.credits?.availablePercent)
})

// @ts-expect-error switchWorkspace accepts camelCase workspaceId only
void profileClient.switchWorkspace({ workspace_id: 'workspace-1' })

// @ts-expect-error GPT Image 2 has a closed resolution enum
void imageClient.submit({ model: 'gpt_image_2', prompt: { instruction: 'x' }, settings: { resolution: '8k' } })

// @ts-expect-error Nano Banana 2 declares only the image media role
void imageClient.submit({ model: 'nano_banana_flash', prompt: { instruction: 'x' }, media: { bogus: { id: 'u', type: 'media_input' } }, settings: {} })

// @ts-expect-error Nano Banana 2 Upscale has a closed resolution enum
void imageClient.submit({ model: 'nano_banana_2_upscale', media: { image: { id: 'u', type: 'media_input' } }, settings: { resolution: '1k' } })

// @ts-expect-error Topaz image has a closed factor enum
void imageClient.submit({ model: 'topaz_image', media: { image: { id: 'u', type: 'media_input' } }, settings: { factor: 'x3' } })

// @ts-expect-error Recraft model is the frontend discriminator, not any arbitrary string
void imageClient.submit({ model: 'recraft_v4_1', prompt: { instruction: 'x' }, settings: { model: 'recraft-v5' } })

// @ts-expect-error Seedance Fast does not use a separate registered model id
void videoClient.submit({ model: 'seedance2_0_fast', prompt: { instruction: 'x' }, settings: { duration: 5, aspectRatio: '16:9' } })

// @ts-expect-error Kling 3.0 has a closed mode enum
void videoClient.submit({ model: 'kling3_0', prompt: { instruction: 'x' }, settings: { duration: 5, aspectRatio: '16:9', mode: 'turbo' } })

// @ts-expect-error Motion Control declares only image/video media roles
void videoClient.submit({ model: 'kling3_0_motion_control', media: { start_image: { id: 's', type: 'media_input' } }, settings: { mode: 'std' } })

// @ts-expect-error Grok Imagine has a closed resolution enum
void videoClient.submit({ model: 'grok_video', prompt: { instruction: 'x' }, settings: { resolution: '1080p' } })

// @ts-expect-error Veo 3.1 Lite duration is 4 | 6 | 8
void videoClient.submit({ model: 'veo3_1_lite', prompt: { instruction: 'x' }, settings: { duration: 5 } })

// @ts-expect-error Wan 2.7 exposes quality, not arbitrary resolutions
void videoClient.submit({ model: 'wan2_7', prompt: { instruction: 'x' }, settings: { quality: '4k' } })

// @ts-expect-error Topaz video has a closed enhancement model enum
void videoClient.submit({ model: 'topaz_video', media: { video: { id: 'v', type: 'video_input' } }, settings: { enhancementModel: 'unknown' } })

// @ts-expect-error Bytedance video has a closed preset enum
void videoClient.submit({ model: 'bytedance_video_upscale', media: { video: { id: 'v', type: 'video_input' } }, settings: { preset: 'cinematic' } })
```

```
// @ts-expect-error Video upscale declares only the video media role
void videoClient.submit({ model: 'video_upscale', media: { image: { id: 'i', type: 'media_input' } }, settings: {} })

// media keys are narrowed to the job's declared roles
void client.submit({ model: 'seedance_2_0', settings: { duration: 8, aspectRatio: '16:9' }, media: { start_image: { id: 'u', type: 'media_input' } } })
// @ts-expect-error 'bogus' is not a declared media role of seedance_2_0
void client.submit({ model: 'seedance_2_0', settings: { duration: 8, aspectRatio: '16:9' }, media: { bogus: { id: 'u', type: 'media_input' } } })

// A job that declares no media and no prompt exposes neither in its input type.
const textOnly = defineJob({
  jobSetType: 'text_only',
  outputType: 'image',
  params: {
    settings: { steps: z.number() },
  },
  // credits receives the TYPED settings of this model
  credits: ({ settings }) => settings.steps * 2,
})

void defineJob({
  jobSetType: 'priced_t',
  outputType: 'image',
  params: { settings: { steps: z.number() } },
  // @ts-expect-error 'nope' is not a settings key of this model
  credits: ({ settings }) => settings.nope,
})
const textClient = createJobClient({ adapter: backend, jobs: [textOnly] })
void textClient.submit({ model: 'text_only', settings: { steps: 4 } })
// @ts-expect-error this job declares no prompt
void textClient.submit({ model: 'text_only', settings: { steps: 4 }, prompt: { instruction: 'x' } })
// @ts-expect-error this job declares no media
void textClient.submit({ model: 'text_only', settings: { steps: 4 }, media: { image: { id: 'u', type: 'media_input' } } })
```

app/packages/fnf/src/types.ts

```
export type OutputType = 'image' | 'video'

export type MediaFormat = 'wrapped' | 'unwrapped' | 'single'

export type GenerationStatus =
  | 'pending'
  | 'waiting'
  | 'queued'
  | 'in_progress'
  | 'ip_detect'
  | 'completed'
  | 'failed'
  | 'nsfw'
  | 'canceled'
  | 'ip_detected'

export const TERMINAL_STATUSES: ReadonlySet<GenerationStatus> = new Set([
  'completed',
  'failed',
  'nsfw',
  'canceled',
  'ip_detected',
])

export function isTerminal(status: GenerationStatus): boolean {
  return TERMINAL_STATUSES.has(status)
}

/**
 * Locally-known facts about a media input (intrinsic size, duration) used by
 * pre-submit validation AND by jobs that derive wire params from intrinsic
 * size (kling's width/height come from the start image; seedance/nano resolve
 * an 'auto' ratio from the first image). The meta OBJECT is never serialized —
 * the codec sends only id/type/url — but attaching it can change what the
 * submitted wire looks like, not just validation outcomes. Populate it from
 * data the app already has, or opt in to `resolveMediaMeta` to measure.
 */
export interface MediaMeta {
  width?: number
  height?: number
  /** Intrinsic duration in seconds (video/audio). */
  durationSec?: number
}

export interface MediaRef {
  id: string
  type: string
  url?: string
  role?: string
  meta?: MediaMeta
}

export interface PromptInput {
  instruction?: string
  enhance?: boolean
  negative?: string
  system?: string
}

export type MediaInput = Record<string, MediaRef | MediaRef[] | undefined>

export interface GenerationInput<S = Record<string, unknown>> {
  model: string
  count?: number
  folderId?: string
  /** Parent job set id — derived jobs (upscale/outpaint/…) link to their source via wire `parent_id`. */
  parentId?: string
  prompt?: PromptInput
```

```
  media?: MediaInput
  settings: S
  extra?: Record<string, unknown>
}

export interface GenerationResults {
  rawUrl: string
  minUrl?: string
  thumbnailUrl?: string
}

export interface Generation {
  id: string
  jobSetId?: string
  /** Set when this generation is a derived job (upscale/outpaint/…) of another job set. */
  parentJobSetId?: string
  model: string
  type: OutputType
  status: GenerationStatus
  input: GenerationInput
  results?: GenerationResults
  failReason?: string
  createdAt?: number
}
```

app/packages/fnf/src/workflow-platform/__tests__/fetch-transport.test.ts

```
import { describe, expect, it, vi } from 'vitest'
import { createFetchTransport } from '../fetch-transport'

interface FakeCall {
  url: string
  init: RequestInit
}

function fakeFetch(status: number, body: unknown) {
  const calls: FakeCall[] = []
  const fn = vi.fn(async (url: string, init: RequestInit) => {
    calls.push({ url, init })
    return {
      status,
      text: async () => (body === undefined ? '' : JSON.stringify(body)),
    } as Response
  })
  return { fn: fn as unknown as typeof globalThis.fetch, calls }
}

describe('createFetchTransport', () => {
  it('trims the base url, JSON-encodes the body, and parses the JSON response', async () => {
    const { fn, calls } = fakeFetch(200, { id: 'job-1', status: 'queued' })
    const transport = createFetchTransport({ baseUrl: 'https://dev-fnf.higgsfield.ai/', fetch: fn })
    const res = await transport({ method: 'POST', path: '/mcp/jobs', body: { a: 1 } })

    expect(calls[0].url).toBe('https://dev-fnf.higgsfield.ai/mcp/jobs')
    expect(calls[0].init.method).toBe('POST')
    expect(calls[0].init.body).toBe('{"a":1}')
    expect((calls[0].init.headers as Headers).get('content-type')).toBe('application/json')
    expect(res).toEqual({ status: 200, body: { id: 'job-1', status: 'queued' } })
  })

  it('applies injected auth headers (function form) to every request', async () => {
    const { fn, calls } = fakeFetch(200, {})
    const transport = createFetchTransport({
      baseUrl: 'https://dev-fnf.higgsfield.ai',
      headers: () => ({ 'fnf-mcp-secret': 's3cr3t' }),
      fetch: fn,
    })
    await transport({ method: 'GET', path: '/mcp/jobs/x' })
    expect((calls[0].init.headers as Headers).get('fnf-mcp-secret')).toBe('s3cr3t')
  })

  it('merges headers case-insensitively: a lowercase content-type override replaces the default', async () => {
    const { fn, calls } = fakeFetch(200, {})
    const transport = createFetchTransport({
      baseUrl: 'https://x',
      headers: { 'content-type': 'application/vnd.custom+json' },
      fetch: fn,
    })
    await transport({ method: 'POST', path: '/y', body: {} })
    expect((calls[0].init.headers as Headers).get('content-type')).toBe('application/vnd.custom+json')
  })

  it('passes the PUT method through verbatim (job cancel rides it)', async () => {
    const { fn, calls } = fakeFetch(200, { success: true })
    const transport = createFetchTransport({ baseUrl: 'https://x', fetch: fn })
    const res = await transport({ method: 'PUT', path: '/jobs/j1/cancel' })
    expect(calls[0].init.method).toBe('PUT')
    expect(res).toEqual({ status: 200, body: { success: true } })
  })

  it('returns the raw text body when the response is not JSON', async () => {
    const { fn } = fakeFetch(502, undefined)
    const transport = createFetchTransport({ baseUrl: 'https://x', fetch: fn })
    const res = await transport({ method: 'GET', path: '/health' })
    expect(res.status).toBe(502)
```

```
    expect(res.body).toBeUndefined()
  })
})
```

app/packages/fnf/src/workflow-platform/__tests__/workflow-platform-adapter.test.ts

```
import type { FnfAdapter, RealtimeBackend } from '../../backend'
import type { Transport, TransportRequest } from '../../transport'
import { describe, expect, it, vi } from 'vitest'
import { createJobClient, getPreviewUrl, getRawUrl } from '../../client'
import { nanoBanana2, seedance2_0 } from '../../jobs'
import { createWorkflowPlatformAdapter } from '../workflow-platform-adapter'

const okTransport: Transport = async () => ({ status: 200, body: {} })
const characterImages = Array.from({ length: 5 }, (_, index) => ({
  id: `media-${index + 1}`,
  type: 'media_input' as const,
}))

function createRecordingAdapter(bodyFor?: (req: TransportRequest) => unknown): {
  adapter: FnfAdapter & RealtimeBackend
  calls: TransportRequest[]
} {
  const calls: TransportRequest[] = []
  const adapter = createWorkflowPlatformAdapter({
    transport: async (req) => {
      calls.push(req)
      return { status: 200, body: bodyFor?.(req) ?? {} }
    },
  })
  return { adapter, calls }
}

describe('createWorkflowPlatformAdapter realtime operations', () => {
  it('uses static realtime routes and preserves wire bodies', async () => {
    const { adapter, calls } = createRecordingAdapter()
    const params = {
      prompt: 'Make this cinematic',
      mode: 'advanced' as const,
      resolution: '2k' as const,
      aspect_ratio: '16:9' as const,
      medias: [{ role: 'image' as const, data: { id: 'media-1', type: 'media_input' as const } }],
    }

    await adapter.editRealtimeChain({ chain_id: 'chain-1', params, confirmation_token: 'approved' })
    await adapter.estimateRealtimeChainCost({ mode: 'advanced', resolution: '2k', aspect_ratio: '16:9' })
    await adapter.finalizeRealtimeChain({ chain_id: 'chain-1' })
    await adapter.listRealtimeCustomStyles({ cursor: 123.5, size: 20 })
    await adapter.createRealtimeCustomStyle({ prompt: 'soft flash' })
    await adapter.updateRealtimeCustomStyle('style/1', { prompt: 'hard flash', clear_reference: true })
    await adapter.deleteRealtimeCustomStyle('style/1')

    expect(calls.map(call => ({ method: call.method, path: call.path, body: call.body }))).toEqual([
      { method: 'POST', path: '/realtime/chain/edit', body: { chain_id: 'chain-1', params, confirmation_token: 'approved' } },
      { method: 'POST', path: '/realtime/chain/cost', body: { mode: 'advanced', resolution: '2k', aspect_ratio: '16:9' } },
      { method: 'POST', path: '/realtime/chain/finalize', body: { chain_id: 'chain-1' } },
      { method: 'GET', path: '/realtime/custom-styles?cursor=123.5&size=20', body: undefined },
      { method: 'POST', path: '/realtime/custom-styles', body: { prompt: 'soft flash' } },
      { method: 'PATCH', path: '/realtime/custom-styles/style%2F1', body: { prompt: 'hard flash', clear_reference: true } },
      { method: 'DELETE', path: '/realtime/custom-styles/style%2F1', body: undefined },
    ])
  })
})

describe('createWorkflowPlatformAdapter construction', () => {
  it('requires a baseUrl or injected transport', () => {
    expect(() => createWorkflowPlatformAdapter()).toThrow(/baseUrl.*transport/i)
    expect(() => createWorkflowPlatformAdapter({ transport: okTransport })).not.toThrow()
    expect(() => createWorkflowPlatformAdapter({ baseUrl: 'https://wfp.example' })).not.toThrow()
  })

  it('sends configured auth and context headers over fetch', async () => {
    const fetchMock = vi.fn(async () => new Response(JSON.stringify({ id: 'u1' }), { status: 200 }))
    const adapter = createWorkflowPlatformAdapter({
```

```
      baseUrl: 'https://wfp.example/',
      getToken: async () => 'token-1',
      userId: async () => 'user-1',
      workspaceId: async () => 'workspace-1',
      appId: async () => 'app-1',
      fetch: fetchMock as unknown as typeof globalThis.fetch,
    })

    await adapter.getUser()

    const [url, init] = fetchMock.mock.calls[0] as unknown as [string, RequestInit]
    const headers = init.headers as Headers
    expect(url).toBe('https://wfp.example/user')
    expect(headers.get('authorization')).toBe('Bearer token-1')
    expect(headers.get('hf-user-id')).toBe('user-1')
    expect(headers.get('hf-workspace-id')).toBe('workspace-1')
    expect(headers.get('hf-app-id')).toBe('app-1')
  })
})

describe('createWorkflowPlatformAdapter profile operations', () => {
  it('uses static WFP profile endpoints', async () => {
    const { adapter, calls } = createRecordingAdapter()

    await adapter.getUser()
    await adapter.listWorkspaces()
    await adapter.getCurrentWorkspace()
    await adapter.getWorkspaceWallet()
    await adapter.switchWorkspace({ workspaceId: 'workspace-1' })

    expect(calls.map(call => ({ method: call.method, path: call.path, body: call.body }))).toEqual([
      { method: 'GET', path: '/user', body: undefined },
      { method: 'GET', path: '/workspaces', body: undefined },
      { method: 'GET', path: '/workspaces/current', body: undefined },
      { method: 'GET', path: '/workspaces/wallet', body: undefined },
      {
        method: 'POST',
        path: '/workspaces/switch',
        body: { workspace_id: 'workspace-1' },
      },
    ])
  })
})

describe('createWorkflowPlatformAdapter job operations', () => {
  it('serializes submit, cost, reads, list, and cancel through static /jobs routes', async () => {
    const { adapter, calls } = createRecordingAdapter()

    await adapter.createJobs({
      jobSetType: 'seedance_2_0',
      params: { prompt: 'x', duration: 5 },
      confirmationToken: 'intent-1',
    })
    await adapter.estimateCost({ jobSetType: 'seedance_2_0', params: { duration: 5 } })
    await adapter.getJob('job-1')
    await adapter.getJobSet?.('set-1')
    await adapter.listJobs({
      type: 'video',
      cursor: 'cursor-1',
      size: 20,
      status: ['queued', 'completed'],
      model: ['seedance_2_0', 'kling3_0'],
    })
    await adapter.cancelJob?.('job-1')
    await adapter.createCharacter?.({
      name: 'Ada',
      type: 'soul_cinematic',
      input_images: characterImages,
    })
    await adapter.getCharacter?.('character-1')
```

```
  await adapter.getReferenceElement?.('element-1')
  await adapter.listReferenceElements?.({ cursor: 123.5, size: 20, category: 'character' })

  expect(calls.map(call => ({ method: call.method, path: call.path, body: call.body }))).toEqual([
    {
      method: 'POST',
      path: '/jobs/submit',
      body: {
        job_set_type: 'seedance_2_0',
        params: { prompt: 'x', duration: 5 },
        confirmation_token: 'intent-1',
      },
    },
    {
      method: 'POST',
      path: '/jobs/cost',
      body: {
        job_set_type: 'seedance_2_0',
        params: { duration: 5 },
      },
    },
    { method: 'GET', path: '/jobs/job-1', body: undefined },
    { method: 'GET', path: '/jobs/sets/set-1', body: undefined },
    {
      method: 'GET',
      path: '/jobs?gen_type=video&cursor=cursor-1&size=20&status=queued&status=completed&job_set_type=seedance_2_0&job_set_type=kling3_0',
      body: undefined,
    },
    {
      method: 'POST',
      path: '/jobs/job-1/cancel',
      body: undefined,
    },
    {
      method: 'POST',
      path: '/custom-references',
      body: {
        name: 'Ada',
        type: 'soul_cinematic',
        input_images: characterImages,
      },
    },
    { method: 'GET', path: '/custom-references/character-1', body: undefined },
    { method: 'GET', path: '/reference-elements/element-1', body: undefined },
    { method: 'GET', path: '/reference-elements?cursor=123.5&size=20&category=character', body: undefined },
  ])
})

it('rejects parent-id list filters because the WFP feed route cannot support them', async () => {
  const { adapter } = createRecordingAdapter()

  await expect(adapter.listJobs({ type: 'image', parentId: 'parent-1' })).rejects.toMatchObject({
    code: 'not_supported',
  })
})

it('unwraps WFP { data } responses', async () => {
  const adapter = createWorkflowPlatformAdapter({
    transport: async () => ({ status: 200, body: { data: { credits: 23 } } }),
  })

  await expect(adapter.estimateCost({ jobSetType: 'seedance_2_0', params: {} })).resolves.toEqual({ credits: 23 })
})

it('normalizes flat completed image reads into SDK result urls', async () => {
  const adapter = createWorkflowPlatformAdapter({
    transport: async () => ({
      status: 200,
      body: {
        id: 'job-1',
```

```
        job_set_type: 'nano_banana_2',
        status: 'completed',
        result_url: 'https://cdn.example/raw.png',
        min_result_url: 'https://cdn.example/min.webp',
        params: { prompt: 'portrait' },
      },
    }),
  })
  const client = createJobClient({ adapter, jobs: [nanoBanana2] })

  const generation = await client.get('job-1')

  expect(generation.results).toEqual({
    rawUrl: 'https://cdn.example/raw.png',
    minUrl: 'https://cdn.example/min.webp',
  })
  expect(getRawUrl(generation)).toBe('https://cdn.example/raw.png')
  expect(getPreviewUrl(generation)).toBe('https://cdn.example/min.webp')
  expect(generation.input.prompt?.instruction).toBe('portrait')
})

it('normalizes nested product feed results for listJobs', async () => {
  const adapter = createWorkflowPlatformAdapter({
    transport: async () => ({
      status: 200,
      body: {
        jobs: [
          {
            id: 'job-1',
            job_set_type: 'nano_banana_2',
            job_set_id: 'set-1',
            status: 'completed',
            results: {
              raw: { url: 'https://cdn.example/raw.png' },
              min: { url: 'https://cdn.example/min.webp' },
            },
            params: { prompt: 'history prompt' },
            created_at: 123,
          },
        ],
        has_more: false,
      },
    }),
  })
  const client = createJobClient({ adapter, jobs: [nanoBanana2] })

  const list = await client.list({ type: 'image', size: 50 })

  expect(list.items).toHaveLength(1)
  expect(list.items[0]).toMatchObject({
    id: 'job-1',
    jobSetId: 'set-1',
    status: 'completed',
    results: {
      rawUrl: 'https://cdn.example/raw.png',
      minUrl: 'https://cdn.example/min.webp',
    },
    createdAt: 123,
  })
  expect(getPreviewUrl(list.items[0]!)).toBe('https://cdn.example/min.webp')
  expect(list.items[0]?.input.prompt?.instruction).toBe('history prompt')
})

it('normalizes job-set product payloads and video thumbnail urls', async () => {
  const adapter = createWorkflowPlatformAdapter({
    transport: async () => ({
      status: 200,
      body: {
        id: 'set-1',
        type: 'seedance_2_0',
```

```
          params: { prompt: 'video prompt', duration: 5, aspect_ratio: '16:9', resolution: '720p' },
          jobs: [
            {
              id: 'job-1',
              status: 'completed',
              result_url: 'https://cdn.example/video.mp4',
              thumbnail_url: 'https://cdn.example/thumb.webp',
            },
          ],
        },
      }),
    })
    const client = createJobClient({ adapter, jobs: [seedance2_0] })

    const [generation] = await client.getSet('set-1')

    expect(generation).toMatchObject({
      id: 'job-1',
      jobSetId: 'set-1',
      model: 'seedance_2_0',
      type: 'video',
      status: 'completed',
      results: {
        rawUrl: 'https://cdn.example/video.mp4',
        thumbnailUrl: 'https://cdn.example/thumb.webp',
      },
    })
    expect(getRawUrl(generation!)).toBe('https://cdn.example/video.mp4')
    expect(getPreviewUrl(generation!)).toBe('https://cdn.example/thumb.webp')
    expect(generation?.input.prompt?.instruction).toBe('video prompt')
  })
})

describe('createWorkflowPlatformAdapter media operations', () => {
  it('serializes presign and confirm through static /jobs media routes', async () => {
    const { adapter, calls } = createRecordingAdapter()

    await adapter.getUploadUrl?.({ type: 'image', filename: 'frame.png', contentType: 'image/png', extra: { surface: 'test' } })
    await adapter.getUploadUrl?.({ type: 'video', filename: 'clip.mp4', contentType: 'video/mp4' })
    await adapter.getUploadUrl?.({ type: 'audio', filename: 'audio.mp3', contentType: 'audio/mpeg' })
    await adapter.confirmMedia?.({
      mediaId: 'media-1',
      type: 'image',
      filename: 'frame.png',
      jobId: 'job-1',
      forceIpCheck: true,
      forceNsfwCheck: false,
      startSeconds: 1,
      endSeconds: 3,
      extra: { role: 'start_image' },
    })

    expect(calls.map(call => ({ method: call.method, path: call.path, body: call.body }))).toEqual([
      {
        method: 'POST',
        path: '/jobs/media/presign',
        body: {
          type: 'image',
          filename: 'frame.png',
          content_type: 'image/png',
          extra: { surface: 'test' },
        },
      },
      {
        method: 'POST',
        path: '/jobs/media/presign',
        body: {
          type: 'video',
          filename: 'clip.mp4',
          content_type: 'video/mp4',
```

```
        },
      },
      {
        method: 'POST',
        path: '/jobs/media/presign',
        body: {
          type: 'audio',
          filename: 'audio.mp3',
          content_type: 'audio/mpeg',
        },
      },
      {
        method: 'POST',
        path: '/jobs/media/media-1/confirm',
        body: {
          type: 'image',
          filename: 'frame.png',
          job_id: 'job-1',
          force_ip_check: true,
          force_nsfw_check: false,
          start_seconds: 1,
          end_seconds: 3,
          extra: { role: 'start_image' },
        },
      },
    ])
  })

  it('serializes media reads through static /jobs media routes', async () => {
    const { adapter, calls } = createRecordingAdapter()

    await adapter.getMedia({ id: 'media-1', type: 'image' })
    await adapter.listMedia({ type: 'video', cursor: 'cursor-1', size: 15 })

    expect(calls.map(call => ({ method: call.method, path: call.path, body: call.body }))).toEqual([
      { method: 'GET', path: '/jobs/media/media-1?type=image', body: undefined },
      { method: 'GET', path: '/jobs/media?type=video&cursor=cursor-1&size=15', body: undefined },
    ])
  })
})

describe('createWorkflowPlatformAdapter safety and errors', () => {
  it('never calls direct fnf-web, media, marketplace, or workspace-detail routes', async () => {
    const { adapter, calls } = createRecordingAdapter()

    await adapter.createJobs({ jobSetType: 'gpt_image_2', params: { use_unlim: true } })
    await adapter.estimateCost({ jobSetType: 'gpt_image_2', params: { use_unlim: true } })
    await adapter.getJob('job-1')
    await adapter.getJobSet?.('set-1')
    await adapter.listJobs({ type: 'image' })
    await adapter.cancelJob?.('job-1')
    await adapter.getUploadUrl?.({ type: 'image' })
    await adapter.confirmMedia?.({ mediaId: 'media-1', type: 'image' })
    await adapter.getMedia({ id: 'media-1', type: 'image' })
    await adapter.listMedia({ type: 'image' })
    await adapter.getUser()
    await adapter.listWorkspaces()
    await adapter.getCurrentWorkspace()
    await adapter.getWorkspaceWallet()
    await adapter.switchWorkspace({ workspaceId: 'workspace-1' })

    const paths = calls.map(call => call.path).join('\n')
    for (const forbidden of ['/jobs/v2', '/job-sets', '/apps-marketplace', '/workspaces/details', '/workspaces/context'])
      expect(paths).not.toContain(forbidden)

    for (const forbiddenPrefix of ['/media', '/video', '/audio'])
      expect(calls.some(call => call.path.startsWith(forbiddenPrefix))).toBe(false)
  })

  it('maps typed HTTP errors through the existing error catalog', async () => {
```

```
    const adapter = createWorkflowPlatformAdapter({
      transport: async () => ({
        status: 403,
        body: { data: { detail: { error_type: 'workspace_selection_required' } } },
      }),
    })

    await expect(adapter.getUser()).rejects.toMatchObject({ code: 'workspace_selection_required' })
  })

  it('wraps thrown transport errors as ApiJobError network failures', async () => {
    const adapter = createWorkflowPlatformAdapter({
      transport: async () => {
        throw new Error('socket closed')
      },
    })

    await expect(adapter.getUser()).rejects.toMatchObject({
      code: 'network',
      message: 'Network error: socket closed',
    })
  })

  it('does not expose token, user id, workspace id, or app id in observability events', async () => {
    const events: unknown[] = []
    const fetchMock = vi.fn(async () => new Response(JSON.stringify({ id: 'u1' }), { status: 200 }))
    const adapter = createWorkflowPlatformAdapter({
      baseUrl: 'https://wfp.example',
      getToken: async () => 'secret-token',
      userId: 'user-sensitive',
      workspaceId: 'workspace-sensitive',
      appId: 'app-sensitive',
      fetch: fetchMock as unknown as typeof globalThis.fetch,
      observability: {
        observer: (event) => {
          events.push(event)
        },
        idFactory: () => `id-${events.length + 1}`,
        now: () => 10,
      },
    })

    await adapter.getUser()

    const serialized = JSON.stringify(events)
    expect(serialized).toContain('fnf.transport.request')
    expect(serialized).toContain('/user')
    expect(serialized).not.toContain('secret-token')
    expect(serialized).not.toContain('user-sensitive')
    expect(serialized).not.toContain('workspace-sensitive')
    expect(serialized).not.toContain('app-sensitive')
  })
})
```

app/packages/fnf/src/workflow-platform/fetch-transport.ts

```
import type { FnfObservabilityOptions } from '../observability'
import type { Transport } from '../transport'
import { withObservedTransport } from '../observability'

type MaybePromise<T> = T | Promise<T>

export interface FetchTransportOptions {
  /** Backend origin, e.g. "https://dev-fnf.higgsfield.ai". Trailing slash is trimmed. */
  baseUrl: string
  /**
   * Static or lazily-resolved headers applied to every request (e.g. auth).
   * A function form supports async sources (rotating secrets, async storage).
   */
  headers?: Record<string, string> | (() => MaybePromise<Record<string, string>>)
  /** Override the fetch implementation (defaults to global fetch). */
  fetch?: typeof globalThis.fetch
  observability?: FnfObservabilityOptions
}

/**
 * Isomorphic Transport over the Fetch API. JSON in, JSON out. Works in Node,
 * the browser, and plugin webviews (Figma) that expose a global fetch. Adobe
 * UXP, which forbids direct fetch from the webview, needs a Comlink adapter
 * instead — this transport is the reference for the fetch-capable environments.
 */
export function createFetchTransport(options: FetchTransportOptions): Transport {
  const base = options.baseUrl.replace(/\/$/, '')
  const doFetch = options.fetch ?? globalThis.fetch.bind(globalThis)

  const transport: Transport = async (req) => {
    const injected = typeof options.headers === 'function' ? await options.headers() : options.headers ?? {}
    // Headers.set replaces case-insensitively — a consumer passing lowercase
    // 'content-type' overrides the default instead of producing a combined,
    // invalid header (which a plain object spread would).
    const headers = new Headers({ 'Content-Type': 'application/json' })
    for (const [key, value] of Object.entries(injected))
      headers.set(key, value)
    for (const [key, value] of Object.entries(req.headers ?? {}))
      headers.set(key, value)

    const res = await doFetch(base + req.path, {
      method: req.method,
      headers,
      body: req.body === undefined ? undefined : JSON.stringify(req.body),
      signal: req.signal,
    })

    const text = await res.text()
    let body: unknown
    try {
      body = text.length > 0 ? JSON.parse(text) : undefined
    }
    catch {
      body = text
    }
    return { status: res.status, body }
  }
  return options.observability ? withObservedTransport(transport, options.observability) : transport
}
```

app/packages/fnf/src/workflow-platform/index.ts

```
/**
 * The ONE adapter implementation bundled with the SDK: the Workflow Platform
 * adapter for `https://fnf.internal` (generated apps / Supercomputer flows),
 * plus the fetch transport and wire normalization it is built on. Bundled so
 * a host that vendors only `@higgsfield/fnf` is self-sufficient for generated
 * apps. Every other adapter (fnf-web, dev, apps-marketplace, memory) lives in
 * `@higgsfield/fnf-adapters`.
 */
export { createFetchTransport } from './fetch-transport'
export type { FetchTransportOptions } from './fetch-transport'
// Shared product-wire normalization — also consumed by the HTTP adapters in
// @higgsfield/fnf-adapters, so the two packages normalize identically.
export { normalizeJobLike, normalizeJobListBody, normalizeJobSetBody, normalizeProductJob } from './job-response-normalize'
export type { NormalizedJobResponse } from './job-response-normalize'
export { createWorkflowPlatformAdapter } from './workflow-platform-adapter'
export type { WorkflowPlatformAdapterOptions } from './workflow-platform-adapter'
```

app/packages/fnf/src/workflow-platform/job-response-normalize.ts

```
import type { JobResponse } from '../spec'

export type NormalizedJobResponse = JobResponse & {
  job_set_type?: string
  cost?: number | null
}

type UnknownRecord = Record<string, unknown>

/** Normalize product/feed job payloads into the SDK's flat job read shape. */
export function normalizeProductJob(job: UnknownRecord, set?: UnknownRecord): NormalizedJobResponse {
  const status = normalizeStatus(stringValue(job.status) ?? 'pending', job.ip_check_finished === false)
  const urls = extractResultUrls(job)
  const meta = recordValue(job.meta)
  const params = recordValue(job.params) ?? recordValue(set?.params)

  return cleanUndefined({
    id: stringValue(job.id) ?? '',
    job_set_type: stringValue(job.job_set_type) ?? stringValue(set?.type) ?? stringValue(set?.job_set_type),
    job_set_id: stringValue(job.job_set_id) ?? stringValue(set?.id),
    job_set_parent_id: nullableStringValue(job.job_set_parent_id) ?? nullableStringValue(set?.parent_id),
    status,
    result_url: urls.rawUrl,
    min_result_url: urls.minUrl,
    thumbnail_url: urls.thumbnailUrl,
    params,
    created_at: numberValue(job.created_at) ?? numberValue(set?.created_at),
    fail_reason: nullableStringValue(job.fail_reason) ?? nullableStringValue(meta?.fail_reason),
    cost: numberOrNullValue(job.cost),
  })
}

export function normalizeJobListBody(body: unknown): unknown {
  const record = recordValue(body)
  if (!record)
    return body

  if (Array.isArray(record.jobs))
    return { ...record, jobs: record.jobs.map(item => normalizeJobLike(item)) }
  if (Array.isArray(record.items))
    return { ...record, items: record.items.map(item => normalizeJobLike(item)) }
  return record
}

export function normalizeJobSetBody(body: unknown): unknown {
  if (Array.isArray(body))
    return body.map(item => normalizeJobLike(item))

  const record = recordValue(body)
  if (!record)
    return body

  if (Array.isArray(record.jobs))
    return record.jobs.map(item => normalizeJobLike(item, record))
  if (Array.isArray(record.items))
    return record.items.map(item => normalizeJobLike(item, record))
  return []
}

export function normalizeJobLike(job: unknown, set?: unknown): NormalizedJobResponse {
  const record = recordValue(job)
  return normalizeProductJob(record ?? {}, recordValue(set) ?? undefined)
}

function normalizeStatus(status: string, isWaitingOnIpCheck: boolean): string {
  const normalized = status === 'waiting' ? 'queued' : status
  return normalized === 'completed' && isWaitingOnIpCheck ? 'ip_detect' : normalized
}
```

```
function extractResultUrls(job: UnknownRecord): {
  rawUrl: string | null
  minUrl: string | null
  thumbnailUrl: string | null
} {
  const results = recordValue(job.results)
  return {
    rawUrl: nestedResultUrl(results, 'raw') ?? nullableStringValue(job.result_url),
    minUrl: nestedResultUrl(results, 'min') ?? nullableStringValue(job.min_result_url),
    thumbnailUrl:
      nullableStringValue(job.thumbnail_url)
      ?? nestedResultUrl(results, 'thumbnail')
      ?? nullableStringValue(results?.thumbnail_url)
      ?? nullableStringValue(recordValue(results?.raw)?.thumbnail_url)
      ?? nullableStringValue(recordValue(results?.min)?.thumbnail_url),
  }
}

function nestedResultUrl(results: UnknownRecord | undefined, key: string): string | null {
  const value = results?.[key]
  if (typeof value === 'string')
    return value
  return nullableStringValue(recordValue(value)?.url)
}

function cleanUndefined<T extends UnknownRecord>(value: T): T {
  return Object.fromEntries(Object.entries(value).filter(([, item]) => item !== undefined)) as T
}

function recordValue(value: unknown): UnknownRecord | undefined {
  return typeof value === 'object' && value !== null && !Array.isArray(value)
    ? value as UnknownRecord
    : undefined
}

function stringValue(value: unknown): string | undefined {
  return typeof value === 'string' ? value : undefined
}

function nullableStringValue(value: unknown): string | null {
  return typeof value === 'string' ? value : value === null ? null : null
}

function numberValue(value: unknown): number | undefined {
  return typeof value === 'number' ? value : undefined
}

function numberOrNullValue(value: unknown): number | null | undefined {
  return typeof value === 'number' ? value : value === null ? null : undefined
}
```

app/packages/fnf/src/workflow-platform/workflow-platform-adapter.ts

```
import type { CharacterBackend, ConfirmMediaRequest, ConfirmSubmit, FnfAdapter, JobListQuery, MediaGetQuery, MediaListQuery, RealtimeBackend, ReferenceBackend, SwitchWorkspaceRequest, UploadUrlRequest
import type { OptionalHeaderSource, RequiredHeaderSource } from '../headers'
import type { FnfObservabilityOptions } from '../observability'
import type { Transport, TransportRequest, TransportResponse } from '../transport'
import { ApiJobError, errorFromResponse } from '../errors'
import { authorizationHeader, cleanHeaders, optionalNamedHeader } from '../headers'
import { withObservedTransport } from '../observability'
import { createFetchTransport } from './fetch-transport'
import { normalizeJobLike, normalizeJobListBody, normalizeJobSetBody } from './job-response-normalize'

export type { FnfAdapter } from '../backend'

export interface WorkflowPlatformAdapterOptions {
  /** Workflow Platform origin. Required unless `transport` is injected. */
  baseUrl?: string
  /** Optional user-scoped token source; sent as `Authorization: Bearer <token>`. */
  getToken?: () => Promise<string | null>
  /** Optional acting user id; sent as `hf-user-id`. */
  userId?: RequiredHeaderSource
  /** Optional active workspace id; sent as `hf-workspace-id`. */
  workspaceId?: OptionalHeaderSource
  /** Optional generated app id; sent as `hf-app-id`. */
  appId?: OptionalHeaderSource
  fetch?: typeof globalThis.fetch
  /** Inject a transport directly (tests / custom). Overrides baseUrl/fetch/header options. */
  transport?: Transport
  observability?: FnfObservabilityOptions
  /**
   * Host confirmation gate run by `submit` before any create request. Its
   * resolved token is sent as `confirmation_token` on the submit body. See
   * `ConfirmSubmit`.
   */
  confirm?: ConfirmSubmit
}

/**
 * Workflow Platform adapter.
 *
 * The SDK keeps doing typed validation and param serialization, then sends
 * static WFP routes. WFP owns final fnf.internal route selection and
 * product-specific request-body quirks. This adapter intentionally stays under:
 * `/user`, `/workspaces/*`, `/reference-elements/*`, `/custom-references/*`,
 * `/realtime/*`, and `/jobs/*`.
 */
export function createWorkflowPlatformAdapter(options: WorkflowPlatformAdapterOptions = {}): FnfAdapter & ReferenceBackend & CharacterBackend & RealtimeBackend {
  if (!options.transport && !options.baseUrl)
    throw new Error('createWorkflowPlatformAdapter requires `baseUrl` or an explicit `transport`')

  const baseTransport = options.transport ?? createFetchTransport({
    baseUrl: (options.baseUrl ?? '').replace(/\/$/, ''),
    headers: async () => cleanHeaders({
      ...(await authorizationHeader(options.getToken)),
      ...(await optionalNamedHeader('hf-user-id', options.userId)),
      ...(await optionalNamedHeader('hf-workspace-id', options.workspaceId)),
      ...(await optionalNamedHeader('hf-app-id', options.appId)),
    }),
    fetch: options.fetch,
  })
  const transport = options.observability ? withObservedTransport(baseTransport, options.observability) : baseTransport

  async function send(method: TransportRequest['method'], path: string, body?: unknown): Promise<unknown> {
    let res: TransportResponse
    try {
      res = await transport({ method, path, body })
    }
    catch (err) {
      if (err instanceof ApiJobError)
        throw err
```

```
    throw new ApiJobError('network', `Network error: ${err instanceof Error ? err.message : String(err)}`)
  }

  const normalized = unwrapData(res.body)
  const error = errorFromResponse(res.status, normalized)
  if (error)
    throw error
  return normalized
}

return {
  // ■■ jobs ■■
  ...(options.confirm ? { confirm: options.confirm } : {}),
  createJobs: ({ jobSetType, params, confirmationToken }) => send('POST', '/jobs/submit', {
    job_set_type: jobSetType,
    params,
    ...(confirmationToken !== undefined ? { confirmation_token: confirmationToken } : {}),
  }),

  getJob: async id => normalizeJobReadBody(await send('GET', `/jobs/${encodeURIComponent(id)}`)),

  getJobSet: async id => normalizeJobSetBody(await send('GET', `/jobs/sets/${encodeURIComponent(id)}`)),

  async listJobs(query) {
    if (query.parentId !== undefined)
      throw new ApiJobError('not_supported', 'GET /jobs has no parent filter - the Workflow Platform feed cannot list a job set\'s children (parentId)')
    return normalizeJobListBody(await send('GET', jobsFeedPath(query)))
  },

  estimateCost: ({ jobSetType, params }) => send('POST', '/jobs/cost', {
    job_set_type: jobSetType,
    params,
  }),

  createCharacter: req => send('POST', '/custom-references', req),

  getCharacter: id => send('GET', `/custom-references/${encodeURIComponent(id)}`),

  getReferenceElement: id => send('GET', `/reference-elements/${encodeURIComponent(id)}`),

  listReferenceElements: query => send('GET', referenceListPath(query)),

  editRealtimeChain: req => send('POST', '/realtime/chain/edit', req),

  estimateRealtimeChainCost: req => send('POST', '/realtime/chain/cost', req),

  finalizeRealtimeChain: req => send('POST', '/realtime/chain/finalize', req),

  listRealtimeCustomStyles: query => send('GET', realtimeCustomStylesPath(query)),

  createRealtimeCustomStyle: req => send('POST', '/realtime/custom-styles', req),

  updateRealtimeCustomStyle: (styleId, req) => send(
    'PATCH',
    `/realtime/custom-styles/${encodeURIComponent(styleId)}`,
    req,
  ),

  deleteRealtimeCustomStyle: styleId => send(
    'DELETE',
    `/realtime/custom-styles/${encodeURIComponent(styleId)}`,
  ),

  cancelJob: id => send('POST', `/jobs/${encodeURIComponent(id)}/cancel`),

  // ■■ media through WFP /jobs static routes ■■
  getMedia: query => send('GET', mediaGetPath(query)),

  listMedia: query => send('GET', mediaListPath(query)),
```

```
    getUploadUrl: req => send('POST', '/jobs/media/presign', mediaPresignBody(req)),

    confirmMedia: req => send(
      'POST',
      `/jobs/media/${encodeURIComponent(req.mediaId)}/confirm`,
      mediaConfirmBody(req),
    ),

    // ■■ profile ■■
    getUser: () => send('GET', '/user'),

    listWorkspaces: () => send('GET', '/workspaces'),

    getCurrentWorkspace: () => send('GET', '/workspaces/current'),

    getWorkspaceWallet: () => send('GET', '/workspaces/wallet'),

    switchWorkspace: (req: SwitchWorkspaceRequest) => send('POST', '/workspaces/switch', {
      workspace_id: req.workspaceId,
    }),
  }
}

function realtimeCustomStylesPath(query: { cursor?: number, size?: number }): string {
  const search = new URLSearchParams()
  if (query.cursor !== undefined)
    search.set('cursor', String(query.cursor))
  if (query.size !== undefined)
    search.set('size', String(query.size))
  const suffix = search.toString()
  return suffix ? `/realtime/custom-styles?${suffix}` : '/realtime/custom-styles'
}

function referenceListPath(query: { cursor?: number, size?: number, category?: string }): string {
  const search = new URLSearchParams()
  if (query.cursor !== undefined)
    search.set('cursor', String(query.cursor))
  if (query.size !== undefined)
    search.set('size', String(query.size))
  if (query.category !== undefined)
    search.set('category', query.category)
  const suffix = search.toString()
  return suffix ? `/reference-elements?${suffix}` : '/reference-elements'
}

function unwrapData(body: unknown): unknown {
  if (isRecord(body) && Object.prototype.hasOwnProperty.call(body, 'data'))
    return body.data
  return body
}

function normalizeJobReadBody(body: unknown): unknown {
  if (isRecord(body) && isRecord(body.job))
    return normalizeJobLike(body.job, body.job_set)
  if (isRecord(body))
    return normalizeJobLike(body)
  return body
}

function jobsFeedPath(query: JobListQuery): string {
  const search = new URLSearchParams()
  append(search, 'gen_type', query.type)
  append(search, 'cursor', query.cursor)
  append(search, 'size', query.size)
  appendMany(search, 'status', query.status)
  appendMany(search, 'job_set_type', query.model)
  const qs = search.toString()
  return qs ? `/jobs?${qs}` : '/jobs'
}
```

```
function mediaGetPath(query: MediaGetQuery): string {
  return pathWithQuery(`/jobs/media/${encodeURIComponent(query.id)}`, {
    type: query.type,
  })
}

function mediaListPath(query: MediaListQuery): string {
  return pathWithQuery('/jobs/media', {
    type: query.type,
    cursor: query.cursor,
    size: query.size,
  })
}

function mediaPresignBody(req: UploadUrlRequest): Record<string, unknown> {
  return cleanBody({
    type: req.type,
    filename: req.filename,
    content_type: req.contentType,
    extra: req.extra,
  })
}

function mediaConfirmBody(req: ConfirmMediaRequest): Record<string, unknown> {
  return cleanBody({
    type: req.type,
    filename: req.filename,
    job_id: req.jobId,
    force_ip_check: req.forceIpCheck,
    force_nsfw_check: req.forceNsfwCheck,
    start_seconds: req.startSeconds,
    end_seconds: req.endSeconds,
    extra: req.extra,
  })
}

function pathWithQuery(path: string, query: Record<string, string | number | undefined>): string {
  const search = new URLSearchParams()
  for (const [key, value] of Object.entries(query))
    append(search, key, value)
  const qs = search.toString()
  return qs ? `${path}?${qs}` : path
}

function append(search: URLSearchParams, key: string, value: string | number | undefined): void {
  if (value !== undefined)
    search.append(key, String(value))
}

function appendMany(search: URLSearchParams, key: string, value: string | string[] | undefined): void {
  if (Array.isArray(value)) {
    for (const item of value)
      search.append(key, item)
    return
  }
  if (value !== undefined)
    search.append(key, value)
}

function cleanBody<T extends Record<string, unknown>>(body: T): Record<string, unknown> {
  const out: Record<string, unknown> = {}
  for (const [key, value] of Object.entries(body)) {
    if (value !== undefined)
      out[key] = value
  }
  return out
}

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === 'object' && value !== null && !Array.isArray(value)
```

```
}
```

app/packages/fnf/src/z.ts

```
import * as base from 'zod/mini'

export type Normalize =
  | { kind: 'aspectRatio', options: readonly string[] }
  | { kind: 'duration', values?: readonly number[], min?: number, max?: number }

/**
 * Side table from a normalized schema object to its descriptor. Keyed on the
 * schema instance via a WeakMap so we never mutate the zod object (no extra
 * own-property, no `as unknown as`) and don't depend on zod tolerating stray
 * properties across versions.
 */
const NORMALIZERS = new WeakMap<object, Normalize>()

/**
 * Look a schema up in a side table, walking zod wrapper chains
 * (`z.optional(...)` / `z._default(...)` expose the tagged inner schema via
 * `_zod.def.innerType`) so wrapping a tagged schema doesn't silently drop
 * its normalizer / wire name.
 */
function lookup<T>(schema: unknown, table: WeakMap<object, T>): T | undefined {
  let current = typeof schema === 'object' && schema !== null ? (schema as object) : undefined
  for (let depth = 0; current && depth < 10; depth++) {
    const hit = table.get(current)
    if (hit !== undefined)
      return hit
    const inner = (current as { _zod?: { def?: { innerType?: unknown } } })._zod?.def?.innerType
    current = typeof inner === 'object' && inner !== null ? (inner as object) : undefined
  }
  return undefined
}

export function getNormalize(schema: unknown): Normalize | undefined {
  return lookup(schema, NORMALIZERS)
}

/**
 * Side table mapping a settings schema to its wire field name. By default a
 * job's settings key IS the wire key; `z.wire('aspect_ratio', schema)` decouples
 * them so the typed API stays camelCase while the wire emits the backend's
 * canonical name. Same WeakMap pattern as the normalizers — no schema mutation.
 */
const WIRE_NAMES = new WeakMap<object, string>()

export function getWireName(schema: unknown): string | undefined {
  return lookup(schema, WIRE_NAMES)
}

/** Tag a settings schema with an explicit wire field name (typed identity). */
export function wire<T extends object>(name: string, schema: T): T {
  WIRE_NAMES.set(schema, name)
  return schema
}

/**
 * Phantom marker that carries the *static* input type of a normalized field —
 * the literal union of allowed values — separate from the runtime schema, which
 * stays permissive so normalization can run. Never present at runtime; read by
 * `SettingsInput` to type the field. See `FieldInput` in define-job.ts.
 */
export declare const NORMALIZE_TYPE: unique symbol
export interface NormalizeType<T> { readonly [NORMALIZE_TYPE]: T }

export function aspectRatio<const O extends readonly string[]>(options: O) {
  // Runtime accepts any ratio string; normalization maps it to the closest
  // allowed option. (An enum would reject inputs like "1920:1081" before we can
  // normalize.) The static type, though, is the literal union `O[number]` so
  // callers get autocomplete on the canonical ratios.
```

```
  const schema = base.string()
  NORMALIZERS.set(schema, { kind: 'aspectRatio', options })
  return schema as typeof schema & NormalizeType<O[number]>
}

export function duration<const V extends readonly number[] = never>(
  opts: { values?: V, min?: number, max?: number },
) {
  // Same shape as aspectRatio: permissive `number` at runtime, literal union of
  // `values` at the type level (or plain `number` for a min/max range).
  const schema = base.number()
  NORMALIZERS.set(schema, { kind: 'duration', ...opts })
  return schema as typeof schema & NormalizeType<[V] extends [never] ? number : V[number]>
}

function _default<T extends base.ZodMiniType>(
  innerType: T,
  defaultValue: unknown,
): base.ZodMiniDefault<T> & (T extends NormalizeType<infer U> ? NormalizeType<U | undefined> : object) {
  return base._default(innerType, defaultValue as never) as base.ZodMiniDefault<T> & (T extends NormalizeType<infer U> ? NormalizeType<U | undefined> : object)
}

export const z = { ...base, _default, aspectRatio, duration, wire }
```

app/packages/fnf/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2021",
    "lib": ["ES2021"],
    "types": ["node"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "verbatimModuleSyntax": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules"]
}
```

app/packages/fnf/vite.config.ts

```
import { readFileSync } from 'node:fs'
import { defineConfig } from 'vite'
import dts from 'vite-plugin-dts'

// Build entries are derived from the public `exports` map so a new subpath
// added to package.json is picked up here (and by scripts/make-publish-package.mjs)
// without touching this config.
const pkg = JSON.parse(readFileSync(new URL('./package.json', import.meta.url), 'utf8')) as {
  exports: Record<string, string>
}

const entry = Object.fromEntries(
  Object.entries(pkg.exports)
    .filter(([, target]) => target.endsWith('.ts'))
    .map(([subpath, target]) => [subpath === '.' ? 'index' : subpath.slice(2), target]),
)

export default defineConfig({
  plugins: [
    dts({
      entryRoot: 'src',
      exclude: ['src/**/__tests__/**', 'src/**/*.test.ts', 'src/**/*.test-d.ts'],
    }),
  ],
  build: {
    target: 'es2021',
    outDir: 'dist',
    emptyOutDir: true,
    minify: false,
    // No source maps: they would embed the full original TypeScript source
    // (sourcesContent) in the published tarball.
    sourcemap: false,
    lib: {
      entry,
      formats: ['es'],
    },
    rollupOptions: {
      // zod is intentionally bundled — the published package has no runtime
      // dependencies, so external consumers install nothing else.
      output: {
        entryFileNames: '[name].js',
        // Shared modules (error classes, error-code registry, media side-effect
        // registrations) must land in common chunks so every entrypoint sees a
        // single copy — duplicating them would break instanceof and rehydration.
        chunkFileNames: 'chunks/[name]-[hash].js',
      },
    },
  },
})
```

app/packages/fnf/vitest.config.ts

```
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    globals: true,
    environment: 'node',
    include: ['src/**/*.test.ts'],
  },
})
```

app/packages/quanta/.gitignore

```
storybook-static/
```

app/packages/quanta/ai/AGENTS.md

````
# Instructions for AI agents using `@higgsfield/quanta`

Canonical AI guide for generating code that consumes Quanta. Quanta is the
Higgsfield design system package: theme runtime, Tailwind token utilities, and
React components skinned with those tokens. Package/tool pointers should route
here first.

## TL;DR rules

1. **Import the CSS exactly once at the app root** (root CSS or root client
   entry):
   ```ts
   import '@higgsfield/quanta/theme.css'
   import '@higgsfield/quanta/primitives.css'
   import '@higgsfield/quanta/tailwind.css'
   ```
2. **Use public subpath imports only.** Example:
   `import { Button } from '@higgsfield/quanta/button'`. Never import from
   `src/components/*`.
3. **Use Quanta components before custom markup.** Button, Input, Modal,
   Dropdown, Command, Vault, Tabs, form controls, status, feedback, and display
   primitives are already available.
4. **Prefix design-semantic utilities with `q-`.** Use
   `bg-q-background-primary`, `text-q-body-md-regular`,
   `border-q-border-subtle`, `z-q-modal`.
5. **Spacing is native Tailwind scale, not old token numbers.** Use `p-4`,
   `gap-2`, `h-10`, `md:grid`. Do not use stale classes like `p-400`,
   `gap-200`, or `mt-300`.
6. **Use composite typography.** Use `text-q-headline-md-semi-bold`, not
   `text-2xl font-semibold`.
7. **Do not invent utilities or hardcode colors/sizes.** Check
   `src/css/tailwind/*.css` or ask for a token gap.
8. **Generated apps should look finished, not merely functional.** Build real
   layouts with clear hierarchy, breathing room, responsive constraints,
   empty/loading/error states, and polished control choices.

## The `q-` namespace

Quanta shares a Tailwind build with legacy Higgsfield UI packages; to avoid
collisions, design-semantic utilities are prefixed with `q-`. Rule of thumb:
color, type, z-index, border-width, and component classes use `q-`; spacing,
layout, sizing, and breakpoints use native Tailwind. Border widths:
`border-q-hairline` (0.5px), `border-q-thin`, `border-q-medium`,
`border-q-thick`.

Do not expose Quanta internals as app API. `q-button-*`, `q-menu-*`,
`q-modal-*`, and similar classes exist so Quanta components can render. App
code should use components and recipe helpers such as `button()` only when
styling a non-button trigger.

## Utility quick reference

### Color

```txt
Surface:  bg-q-background-{primary,secondary,secondary-strong,tertiary,inverse,glass,elevated-start,elevated-end}
Text:     text-q-text-{primary,secondary,tertiary,inverse,disabled,link,brand,danger,success,warning,info,on-overlay-secondary,on-overlay-tertiary}
Border:   border-q-border-{default,subtle,strong,focus,error,warning,success,inverse}
State:    bg/text/border-q-state-{error,success,warning,info}-*
Brand:    bg/text/border-q-brand-{primary,blue,cyan,green,orange,pink,red,yellow,yellow-light}
```

Prefer semantic roles. Raw palette utilities are for Quanta internals only.

### Typography

Composite typography utilities include font size, line-height, font weight,
letter spacing, and font family where needed.
````

```txt
Roles:    display, headline, title, body, label, caption, mono, accent
Sizes:    lg, md, sm
Weights:  black, bold, semi-bold, medium, regular
```

Families per role: `display`/`headline`/`title` render Space Grotesk (the
brand headline face), `body`/`label`/`caption` render Inter, `accent` is
Space Grotesk bold (tight tracking; pair with `uppercase` for the hero/CTA
look), `mono` is IBM Plex Mono.

Z-index

```txt
z-q-{base,dropdown,sticky,overlay,modal,popover,toast,tooltip}   ← 8 semantic keys (0..70 in steps of 10)
```

Semantic names are the canonical choice over Tailwind's `z-N`.

Spacing and breakpoints

Use native Tailwind spacing from the configured scale:
`0`, `0.5`, `1`, `1.5`, `2`, `2.5`, `3`, `4`, `5`, `6`, `7`, `8`, `10`,
`12`, `14`, `16`, `20`, `24`.

Examples: `p-6`, `px-4`, `gap-3`, `space-y-4`, `h-10`, `w-80`.

Breakpoints are mobile-first. Quanta ships three named variants (see
`src/css/tailwind/breakpoint.css`): `q-tablet:` (48rem), `q-desktop:` (80rem),
`q-wide:` (120rem). Tailwind's default `sm:`/`md:`/`lg:`/`xl:`/`2xl:` variants
also work. There are no `tablet:`/`desktop:`/`wide:` variants.

Component catalog

Import every component by subpath, e.g.
`import { Modal } from '@higgsfield/quanta/modal'`.

Actions and navigation

Component	Import	Use
`Button`, `button`	`@higgsfield/quanta/button`	Primary/secondary actions, links with `as="a"`, trigger composition with `asChild`
`NavigationMenu`	`@higgsfield/quanta/navigation-menu`	Product nav, app top nav, grouped menus
`Sidebar`	`@higgsfield/quanta/sidebar`	Product navigation rail — header switcher, sections, composite `Item` rows (`ItemIcon`/`ItemLabel`/`ItemMeta`/`ItemEnd`), built-in collapse (`Toggle`)
`Tabs`	`@higgsfield/quanta/tabs`	View switching, segmented controls, modal header tabs
`Dropdown`	`@higgsfield/quanta/dropdown`	Menus, model pickers, searchable selection, submenus
`Command`	`@higgsfield/quanta/cmdk`	Command palette, quick switchers, keyboard-first search
`ButtonGroup`, `buttonGroup`	`@higgsfield/quanta/button-group`	Segmented action clusters and icon toolbars — propagates `size`/`variant`, `attached` joins, `orientation` stacks

Button variants:

- Product actions: `primary`, `secondary`, `tertiary`, `outline`, `ghost`, `brandSoft`
  — variant colors do NOT follow the token names: `primary` = flat lime,
  `secondary` = solid white, `tertiary` = dark white/10 glass. In this
  template's apps, ordinary/navigation actions use the dark
  `tertiary`/`ghost`; generation CTAs use `marketingPrimary`
- Destructive: `danger`, `dangerSoft`
- Marketing: `marketingPrimary`, `marketingSecondary`,
  `marketingTertiary`, `marketingGhost`
- Special accents: `specialBrand`, `specialPink`

Use `iconOnly` only for true icon buttons with an accessible label.

Forms and controls

Component	Import	Use
`Input`	`@higgsfield/quanta/input`	Labelled single-line fields, helper/error text, prefix/suffix icons
`Textarea`	`@higgsfield/quanta/textarea`	Labelled multi-line fields, prompts, comments, notes
`Select`	`@higgsfield/quanta/select`	Single/multiple choice from a known option set — field-styled trigger, grouped glass popup, leading icons/badges; selected row highlights; `connected` on `Ro

```
| `Autocomplete` | `@higgsfield/quanta/autocomplete` | Type-to-filter selection — search input + filtered glass list, rich item rows, clear affix, empty state; `connected` on `Root` nests the input ir
| `Checkbox`, `CheckboxLabel` | `@higgsfield/quanta/checkbox` | Multi-select booleans and labelled checkbox rows |
| `RadioGroup`, `Radio`, `RadioLabel` | `@higgsfield/quanta/radio` | Exclusive choices |
| `Switch` | `@higgsfield/quanta/switch` | Binary settings that apply immediately |
| `Slider` | `@higgsfield/quanta/slider` | Stepped or continuous numeric settings |
| `Toggle` | `@higgsfield/quanta/toggle` | Pressed/unpressed toolbar controls |
| `Chip` | `@higgsfield/quanta/chip` | Compact selectable/filter-like chips |

Use the dedicated control for the setting shape. Do not model binary settings
as a `Button`; use `Switch`, `Checkbox`, or `Toggle`.

### Overlays and feedback

| Component | Import | Use |
|---|---|---|
| `Modal` | `@higgsfield/quanta/modal` | Centered dialogs, confirmation, editors, complex panels |
| `Vault` | `@higgsfield/quanta/vault` | Edge-docked drawer/sheet, especially mobile or tool panels |
| `Toaster`, `toast` | `@higgsfield/quanta/sonner` | App-root toast viewport and imperative notifications |
| `Progress` | `@higgsfield/quanta/progress` | Bar, line, dots, determinate/indeterminate progress |
| `Loader` | `@higgsfield/quanta/loader` | Indeterminate spinner — dots/circle/stars/shine, slot-tinted |
| `Tooltip` | `@higgsfield/quanta/tooltip` | Hover/focus hint popup — small inverted surface, optional arrow, `Provider` shares delay |
| `CloseButton`, `closeButton` | `@higgsfield/quanta/close-button` | Round dismiss for overlays; recipe styles a framework close part |

Mount one `Toaster` near the app root before calling `toast.*`.

### Display primitives

| Component | Import | Use |
|---|---|---|
| `Avatar` | `@higgsfield/quanta/avatar` | User/workspace identity, presence |
| `Badge`, `badge` | `@higgsfield/quanta/badge` | New/pro/status marker |
| `Tag` | `@higgsfield/quanta/tag` | Removable categories or model metadata |
| `Dot` | `@higgsfield/quanta/dot` | Presence/status point |
| `Kbd`, `KbdSequence` | `@higgsfield/quanta/kbd` | Keyboard shortcuts |
| `Divider` | `@higgsfield/quanta/divider` | Structural separators |
| `NotFound` | `@higgsfield/quanta/not-found` | Empty/not-found states where available |
| `Accordion` | `@higgsfield/quanta/accordion` | Expandable content sections — single/multiple open, `list`/`separated` surface, `sm`/`md`/`lg` size (scales padding, radius, type) |
| `Icon`, `icon` | `@higgsfield/quanta/icon` | Render an icon glyph at a token `size`/`color` (`as={SomeIcon}`); `icon()` recipe styles a non-Icon element |
| `Typography`, `typography` | `@higgsfield/quanta/typography` | Semantic text as a component — `variant` (display/headline/title/body/label/caption/mono/accent + size/weight) + `color`; the component

### Layout and surfaces

| Component | Import | Use |
|---|---|---|
| `Grid` | `@higgsfield/quanta/grid` | Responsive grids — fixed/`auto-fit`/`auto-fill` columns, `minColWidth`, `gap`, `Grid.Item` `colSpan`/`rowSpan`; `animate` FLIP-animates reflow/reorder/filter (pe
| `VirtualGrid` | `@higgsfield/quanta/grid` | Windowed, data-driven grid for big feeds & galleries — `items`+`renderItem`, `overscan`, fixed `cols` or responsive `minColWidth`, uniform `rowHeight`. Pa
| `Media` | `@higgsfield/quanta/media` | Fixed-ratio image/video tiles — `ratio`/`fit`/`rounded`, `Overlay`/`Caption`/`Fallback` parts, `useMediaFallback` onError flow; `Media.Video autoPlayInView` pl
| `Glass`, `glass` | `@higgsfield/quanta/glass` | Real frosted-glass surface — backdrop blur+saturate, specular edge + top sheen; pin over colorful media |
| `Card`, `card` | `@higgsfield/quanta/card` | Glass/solid content surface — `Header`/`Body`/`Footer` parts, `surface` glass\|solid, `elevation` flat\|raised; `card()` recipe |

## Component API reference

Per-component props. **These are the props you need — do NOT open the component
`.tsx` source to re-derive them.** Only the component's OWN props are listed;
every component also forwards native element / Base UI props (`className`,
`ref`, `onClick`, `render`, controlled `value`/`open` state, etc.). Lowercase
names (`button()`, `card()`) are class-string recipe helpers for styling a
non-component element. `SlotColor` = `'brand' | 'neutral' | 'success' | 'error'
| 'warning' | 'info'`.

### Actions and navigation

**`Button`** — `@higgsfield/quanta/button`
`variant` (`'primary' | 'secondary' | 'tertiary' | 'outline' | 'ghost' | 'danger' | 'dangerSoft' | 'brandSoft' | 'marketingPrimary' | 'marketingSecondary' | 'marketingTertiary' | 'marketingGhost' | 's

**`ButtonGroup`** — `@higgsfield/quanta/button-group`
`orientation` (`'horizontal' | 'vertical'`; default `'horizontal'`); `attached` (bool; default `true` — segmented vs spaced); `size`/`variant` (propagate to child Buttons; child's own wins).

**`Tabs`** — `@higgsfield/quanta/tabs` · Parts: `Root`/`List`/`Tab`/`Panel`
`Root`: `variant` (`'underline' | 'pill' | 'segmented' | 'soft'`; default `'underline'`), `shape` (`'rounded' | 'pill' | 'icon'`; default `'rounded'`), `surface` (`'glass' | 'flat'`; default `'glass'`
```

`Dropdown` — `@higgsfield/quanta/dropdown` · Parts: `Root`/`Trigger`/`Content`/`Group`/`Label`/`Separator`/`Item`/`Sub`/`SubTrigger`/`SubContent` (+ Item sub-parts)
`Root`: `selectionMode` (`'single' | 'multiple'`; default `'multiple'`), `selected`/`defaultSelected` (`string[]`), `onSelected`, `openOnHover`, + `open`/`onOpenChange`. `Content`: `size` (`'compact'

`Command` — `@higgsfield/quanta/cmdk` · Parts: `Root`/`Dialog`/`Input`/`List`/`Empty`/`Loading`/`Group`/`Item`/`Separator`/`Shortcut`/`Body`/`Detail`/`Footer`/`Action`
`Root`: `value`/`defaultValue`/`onValueChange`, `shouldFilter` (default `true`), `filter`, `loading`, `loop` (default `true`), `label`. `Dialog`: + `open`/`defaultOpen`/`onOpenChange`, `shortcut` (e.g

`NavigationMenu` — `@higgsfield/quanta/navigation-menu` · Parts: `Root`/`List`/`Item`/`Trigger`/`Content`/`Link`/`Actions`/`Action`/`Menu`/`MenuItem`… (rich mega-menu set)
`Root`: `side` (default `'bottom'`), `align` (default `'center'`), `sideOffset` (default `8`). `Trigger`/`Link`: `accent` (bool, lime), `active` (bool, sets `aria-current`). `Action`: `iconOnly`, `hre

`Sidebar` — `@higgsfield/quanta/sidebar` · Parts: `Root`/`Header`/`Switcher`/`Toggle`/`Body`/`Search`/`Section`/`SectionHeader`/`SectionItems`/`Item`/`ItemIcon`/`ItemLabel`/`ItemMeta`/`ItemEnd`/`F
`Root`: `collapsed`/`defaultCollapsed`/`onCollapsedChange`, `product` (`'cinema-studio' | 'marketing-studio' | 'supercomputer'`), `flush`. `Item`: slots `start`/`title`/`meta`/`end`, `size` (`'md' | '

Forms and controls

`Input` — `@higgsfield/quanta/input`
`label` (ReactNode); `description` (ReactNode — helper text, hidden while error shows); `error` (ReactNode — presence triggers invalid state); `invalid` (bool — force invalid); `required` (bool; apper

`Textarea` — `@higgsfield/quanta/textarea`
Same field shape as `Input`: `label`, `description`, `error`, `invalid`, `required`, `start` (top-pinned), `end` (bottom-pinned); `rows`; + native `textarea` props. Built on Base UI `Field`.

`Select` — `@higgsfield/quanta/select` · Parts: `Root`/`Trigger`/`Value`/`Icon`/`Content`/`Group`/`GroupLabel`/`Separator`/`Item`/`ItemText`/`ItemIndicator`
`Root`: `connected` (bool — popup as seamless extension), + `value`/`defaultValue`/`onValueChange`/`multiple`. `Trigger`: `size` (`'sm' | 'md' | 'lg'`; default `'md'`), `invalid`. `Value`: `placeholde

`Autocomplete` — `@higgsfield/quanta/autocomplete` · Parts: `Root`/`Input`/`Content`/`List`/`Item`/`Group`/`Empty`/`Clear`… (type-to-filter)
`Root`: `connected`, + `value`/`defaultValue`/`onValueChange`/`items`/`filter`. `Input`: `start` (default search icon; `null` removes), `clear` (bool; default `true`), `clearLabel`. `Content`: `solid`

`Checkbox`, `CheckboxLabel` — `@higgsfield/quanta/checkbox`
`Checkbox`: `color` (`'brand' | 'white'`; default `'brand'`), `size` (`'sm' | 'md' | 'lg'`; default `'md'`), `indeterminate`, + `checked`/`defaultChecked`/`onCheckedChange`/`disabled`. `CheckboxLabel`

`Radio`, `RadioGroup`, `RadioLabel` — `@higgsfield/quanta/radio`
`Radio`: `color` (`SlotColor`; default `'brand'`), `size` (`'sm' | 'md' | 'lg'`), + `value`/`disabled`. `RadioGroup`: `value`/`defaultValue`/`onValueChange`. `RadioLabel`: `value` (required), `label`,

`Switch`, `SwitchLabel` — `@higgsfield/quanta/switch`
`Switch`: `color` (`SlotColor`; default `'brand'`), `size` (`'small' | 'medium' | 'default'`; default `'small'`), + `checked`/`defaultChecked`/`onCheckedChange`/`disabled`. `SwitchLabel`: `label`, `de

`Slider` — `@higgsfield/quanta/slider`
Common: `disabled`, `showTicks`, `onChange` (`(n) => void` continuous), `onChangeEnd` (on release), `value`/`defaultValue`, `aria-label`. Stepped mode (default): `mode?: 'stepped'`, `steps` (number; m

`Toggle` — `@higgsfield/quanta/toggle`
`color` (`SlotColor`; default `'brand'`), `size` (`'sm' | 'md' | 'lg'`), `start`/`end` slots, `children` (label), + `pressed`/`defaultPressed`/`onPressedChange`.

`Chip` — `@higgsfield/quanta/chip`
`color` (`SlotColor`; default `'brand'`), `size` (`'xxs' | 'xs' | 'sm' | 'md'`; default `'sm'`), `selected` (bool), `start`/`end` slots, `children` (label). Renders `<button type="button" aria-pressed

Overlays and feedback

`Modal` — `@higgsfield/quanta/modal` · Parts: `Root`/`Trigger`/`Close`/`Content`/`Header`/`Title`/`Description`/`CloseButton`/`BackButton`/`Search`/`Body`/`Workspace`/`Footer`/`FooterActions`
`Content`: `size` (`'xs' | 'sm' | 'md' | 'lg' | 'xl' | '2xl'`; default `'md'`), `initialFocus`, `container`. `Header`: `flush`. `Body`/`Workspace`: `padded` (default `true`). `Footer`/`FooterActions`:

`Vault` — `@higgsfield/quanta/vault` · Parts: `Root`/`Trigger`/`Content`/`Header`/`Body`/`Footer`/`Title`/`Close`
`Root`: `side` (`'bottom' | 'top' | 'left' | 'right'`; default `'bottom'`). `Content`: `side` (override), `handle` (bool; default `true` for bottom). `Header`: `title`, `start`/`end`, `closeButton`. `

`Toaster`, `toast` — `@higgsfield/quanta/sonner`
`Toaster`: `position` (`'top-left' | 'top-center' | 'top-right' | 'bottom-left' | 'bottom-center' | 'bottom-right'`; default `'bottom-right'`), `limit` (default `3`), `duration` (ms; default `5000`),

`Progress` — `@higgsfield/quanta/progress`
`value` (omit for indeterminate), `max` (default `100`), `variant` (`'bar' | 'dots' | 'line'`; default `'bar'`), `shape` (`'linear' | 'circular'`; default `'linear'`), `steps` (default `4`), `size` (`

`Loader` — `@higgsfield/quanta/loader`
`variant` (`'dots' | 'circle' | 'stars' | 'shine'`; default `'circle'`), `size` (`'xxs' | 'xs' | 'sm' | 'md' | 'lg'`), `color` (`SlotColor`), `animated`, `aria-label` (default `'Loading'`).

`Tooltip` — `@higgsfield/quanta/tooltip` · Parts: `Provider`/`Root`/`Trigger`/`Content`
`Root`: `delay` (ms; default `600`), `closeDelay`, `hoverable`. `Content`: `side` (default `'top'`), `align` (default `'center'`), `sideOffset` (default `6`), `arrow` (bool). Share a `Provider` for co

`CloseButton` — `@higgsfield/quanta/close-button`
`size` (`'sm' | 'md' | 'lg' | 'xl'`; default `'md'`), `children` (override cross glyph), `aria-label` (default `'Close'`).

Display primitives

`Avatar` — `@higgsfield/quanta/avatar`

`size` (`'xxs' | 'xs' | 'sm' | 'md' | 'lg' | 'xl'`; default `'md'`), `src`, `alt` (drives initials + auto color), `fallback`, `color` (`'neutral' | 'orange' | 'mint' | 'blue' | 'pink' | 'purple' | 'br

`Badge` — `@higgsfield/quanta/badge`
`variant` (`'blue' | 'lime' | 'pink' | 'purple' | 'limeSubtle' | 'nBrand' | 'nBlue'`; default `'blue'`), `size` (`'xs' | 'sm'`; default `'xs'`), `text` (falls back to `children`).

`Tag` — `@higgsfield/quanta/tag`
`color` (`SlotColor`; default `'neutral'`), `start`/`end` slots, `onRemove` (`() => void` — renders trailing ✕), `removeLabel`. Label is `children`.

`Dot` — `@higgsfield/quanta/dot`
`color` (`'green' | 'yellow' | 'red' | 'grey'`; default `'green'`), `size` (`'md' | 'sm' | 'xs'`), `animation` (`'pulse' | 'glow'`), `label` (sets `role="img"`).

`Kbd`, `KbdSequence` — `@higgsfield/quanta/kbd`
`Kbd`: native `kbd` props (single size). `KbdSequence`: `keys` (`ReactNode[]` — strings wrap in `<Kbd>`), `separator` (default `'+'`; `null` omits).

`Divider` — `@higgsfield/quanta/divider`
`orientation` (`'horizontal' | 'vertical'`; default `'horizontal'`), `children` (optional inline label, horizontal only).

`NotFound` — `@higgsfield/quanta/not-found`
`icon`, `title`, `subtitle`, `actions`, `size` (`'sm' | 'md' | 'lg'`), `variant` (`'plain' | 'card' | 'outline'`; default `'plain'`). Use for empty states.

`Accordion` — `@higgsfield/quanta/accordion` · Parts: `Root`/`Item`/`Trigger`/`Panel`
`Root`: `variant` (`'list' | 'separated'`; default `'list'`), `size` (`'sm' | 'md' | 'lg'`), + `value`/`openMultiple`. `Trigger`: `start`/`end` slots (default rotating chevron), label is `children`. `

`Icon` — `@higgsfield/quanta/icon`
`size` (`'xs' | 'sm' | 'md' | 'lg' | 'xl'` → 12/16/20/24/28px; default `'md'`), `color` (`'primary' | 'secondary' | 'tertiary' | 'brand' | 'accent' | 'inverse' | 'disabled' | 'error' | 'success' | 'w

`Typography` — `@higgsfield/quanta/typography`
`as` (ElementType; default `'p'`), `variant` (`TypographyVariant`; default `'body-md-regular'` — `{display|headline|title|body|label|caption|mono|accent}-{lg|md|sm}-{weight}`), `color` (same set as Ic

Layout and surfaces

`Grid`, `VirtualGrid` — `@higgsfield/quanta/grid` · `Grid.Item` for cells
`Grid`: `cols` (`number | 'auto-fit' | 'auto-fill'`; default `1`), `minColWidth` (CSS length STRING for auto-fit/fill, e.g. `"13rem"` — never a number), `gap`/`gapX`/`gapY` (`1..24` scale), `flow` (`'

`Media` — `@higgsfield/quanta/media` · Parts: `Root`/`Image`/`Video`/`Fallback`/`Overlay`/`Caption`
`Root`: `ratio` (`'square' | 'video' | 'portrait' | 'wide' | 'auto'` or a number; default `'video'`), `rounded` (`'none' | 'sm' | 'md' | 'lg' | 'full'`; default `'md'`). `Image`: `fit` (`'cover' | 'co

`Glass` — `@higgsfield/quanta/glass`
`blur` (`'sm' | 'md' | 'lg'`; default `'md'`), `elevation` (`'flat' | 'raised'`), `rounded` (`'200'…'600' | 'full'`; default `'600'`), `interactive` (bool), `tint` (`SlotColor`). Pin over colorful med

`Card` — `@higgsfield/quanta/card` · Parts: `Root`/`Header`/`Title`/`Description`/`Body`/`Footer`
`Root`: `surface` (`'glass' | 'solid'`; default `'glass'`), `elevation` (`'flat' | 'raised'`). `Header`: `title`, `description`, `actions`. `Footer`: `actions`. Do not nest cards inside cards.

Component recipes

Modal

```tsx
<Modal.Root>
  <Modal.Trigger render={<Button variant="secondary">Open</Button>} />
  <Modal.Content size="lg">
    <Modal.Header>
      <Modal.Title>Edit settings</Modal.Title>
      <Modal.CloseButton />
    </Modal.Header>
    <Modal.Body>
      <Modal.Workspace>
        <Input label="Name" />
      </Modal.Workspace>
    </Modal.Body>
    <Modal.Footer>
      <Modal.FooterActions>
        <Button>Save</Button>
      </Modal.FooterActions>
    </Modal.Footer>
  </Modal.Content>
</Modal.Root>
```

`Modal.Header` and `Modal.Footer` are plain rows you COMPOSE — they have no

`title`/`actions` prop. Put a `Modal.Title` (+ optional `Modal.CloseButton`) in the header and a `Modal.FooterActions` (optionally a `Modal.FooterCaption`) in the footer. `<Modal.Header title="…" />` is NOT a heading — `title` is just the native HTML tooltip attribute, so it type-checks but renders nothing visible. (Unlike `Vault.Footer`/`Card.Footer`, which do take an `actions` prop — the parts are not uniform.) Use `Modal.Workspace` for the inset frosted pane inside modal bodies. For split layouts, place multiple workspaces inside your own `flex` or `grid`.

Command palette

```tsx
<Command.Dialog shortcut="mod+k" label="Command menu">
  <Command.Input placeholder="Search actions..." />
  <Command.List>
    <Command.Empty>No results.</Command.Empty>
    <Command.Group heading="Actions">
      <Command.Item
        title="New project"
        subtitle="Create a fresh workspace"
        onSelect={createProject}
      />
    </Command.Group>
  </Command.List>
</Command.Dialog>
```

Use `Command.Body`, `Command.Detail`, `Command.Footer`, and `Command.Action` for two-pane palettes.

Vault drawer

```tsx
<Vault.Root side="bottom">
  <Vault.Trigger render={<Button variant="secondary">Filters</Button>} />
  <Vault.Content>
    <Vault.Header title="Filters" />
    <Vault.Body>...</Vault.Body>
    <Vault.Footer actions={<Button>Apply</Button>} />
  </Vault.Content>
</Vault.Root>
```

Use `Vault` for drawers/sheets. Use `Modal` for centered dialogs.

Premium layout rules for generated apps

1. **Start with the real app surface, not a marketing page** — if the prompt asks for a tool, editor, dashboard, feed, generator, or workspace, that tool is the first viewport.
2. **Use a stable shell:** `min-h-dvh`, optional sidebar, main work area, constrained scroll regions. No app header/top bar — Higgsfield chrome provides the global header; a page title is a heading inside the work area.
3. **Give layouts breathing room:** `p-4`–`p-8`, `gap-3`–`gap-6`, visible hierarchy — text hugging borders looks unfinished.
4. **Use professional type scale:** page titles `text-q-title-lg-semi-bold` or `text-q-headline-md-semi-bold`, body `text-q-body-md-regular`, metadata `text-q-caption-sm-medium`. Do not make every label the same size.
5. **Choose the correct control:** icons for icon-only actions, `Button` for commands, `Tabs` for modes, `Dropdown` for option sets, `Switch`/`Checkbox` for booleans, `Slider` for ranges, `Input` for typed values.
6. **Do not nest cards inside cards** — use full-width bands, shells, sidebars, workspaces, or repeated item cards.
7. **Make empty/loading/error states first-class:** `Progress`, `Tag`, `Badge`, `NotFound`, and clear local state surfaces rather than blank panels.
8. **Keep text inside containers:** `min-w-0`, `truncate`, responsive grids, fixed control dimensions where needed.
9. **Dark surfaces need contrast:** pair `bg-q-background-primary` with nested `bg-q-background-secondary` or component surfaces, plus subtle borders.
10. **No arbitrary brand decoration:** no gradient orbs, blur blobs, raw

````
    purple/blue gradients, or stock illustration unless the prompt calls
    for it.

## Component override tokens

A generated tier of GLOBAL component-dimension knobs is published on `:root` in
`src/css/components/component.css`. These are the single-value sizing defaults
a consumer would set to resize a component — NOT per-variant color/state logic,
which stays inline in each component's CSS.

| Token | Default | Resizes |
|---|---|---|
| `--q-modal-width` | `40rem` | Modal popup width |
| `--q-modal-width-{xs,sm,md,lg,xl,2xl}` | `21.8125rem`…`87rem` | Modal `size` variant widths |
| `--q-menu-min-width` | `160px` | Dropdown/menu min width |
| `--q-menu-max-width` | `400px` | Dropdown/menu max width |
| `--q-sidebar-width` | `14.9375rem` | Sidebar rail width |
| `--q-sidebar-width-collapsed` | `3.25rem` | Collapsed sidebar rail width |
| `--q-nav-col-width` | `192px` | NavigationMenu legacy column width |
| `--q-slider-width` | `11.5rem` | Slider track width |
| `--q-sonner-width` | `22.5rem` | Toast viewport width |
| `--q-vault-width` | `22rem` | Vault drawer width |
| `--q-textarea-min-height` | `10.25rem` | Textarea min height |

To resize component X, set its `--q-<comp>-<dim>` on an ancestor (or the
component instance) — the closer scope wins the cascade, and that override IS
the mechanism. Example:

```tsx
// Wider modal for one screen — no prop, no custom CSS, just override the knob.
<div style={{ '--q-modal-width': '52rem' }}>
  <Modal.Root>…</Modal.Root>
</div>
```

## When in doubt

Check the component `index.ts` exports and `src/css/tailwind/*.css` utility
names; ask for a token gap instead of inventing values.
````

app/packages/quanta/CLAUDE.md

```
@./ai/AGENTS.md
```

app/packages/quanta/package.json

```
{
  "name": "@higgsfield/quanta",
  "version": "0.0.0",
  "private": true,
  "description": "Higgsfield design tokens and theming runtime",
  "author": "support@higgsfield.ai",
  "type": "module",
  "sideEffects": [
    "**/*.css"
  ],
  "files": [
    "ai",
    "src"
  ],
  "scripts": {
    "tokens:read": "tsx scripts/tokens-read/read.ts",
    "tokens:emit": "tsx scripts/tokens-emit/emit.ts",
    "test": "vitest run",
    "test:watch": "vitest",
    "storybook": "storybook dev -p 6007",
    "storybook:build": "storybook build",
    "ci:deploy": "wrangler deploy",
    "typecheck": "tsc -p tsconfig.json && tsc -p scripts/tsconfig.json",
    "typecheck:native": "tsgo -p tsconfig.json && tsgo -p scripts/tsconfig.json"
  },
  "dependencies": {
    "@base-ui/react": "1.5.0"
  },
  "optionalDependencies": {
    "@higgsfield-ai/icons": "^3.2.0"
  },
  "devDependencies": {
    "@storybook/addon-a11y": "^10.1.11",
    "@storybook/addon-docs": "^10.1.11",
    "@storybook/addon-themes": "^10.1.11",
    "@storybook/react": "^10.1.11",
    "@storybook/react-vite": "^10.1.11",
    "@tailwindcss/vite": "^4.1.4",
    "@testing-library/jest-dom": "^6.6.3",
    "@testing-library/react": "^16.1.0",
    "@testing-library/user-event": "^14.5.2",
    "@types/node": "^20.11.0",
    "@types/react": "19.2.1",
    "@types/react-dom": "^19.0.0",
    "@typescript/native-preview": "7.0.0-dev.20260619.1",
    "@vitejs/plugin-react": "^4.3.0",
    "dotenv": "^16.4.5",
    "happy-dom": "^20.9.0",
    "storybook": "^10.2.10",
    "tailwindcss": "^4.1.4",
    "tsx": "^4.19.0",
    "typescript": "^5.8.3",
    "vite": "^6.0.0",
    "vitest": "^4.0.17"
  },
  "peerDependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "exports": {
    "./tailwind.css": "./src/css/tailwind/index.css",
    "./theme.css": "./src/css/theme/index.css",
    "./primitives.css": "./src/css/primitives/index.css",
    "./runtime": "./src/runtime/index.ts",
    "./*": "./src/components/*/index.ts",
    "./package.json": "./package.json"
  }
}
```

app/packages/quanta/README.md

````
# @higgsfield/quanta

Higgsfield design tokens + React components. Source of truth for color,
typography, spacing, breakpoints, z-index, and border-width.
Framework-agnostic — works on top of Tailwind v4 or plain CSS.

## Quick start

```tsx
<div className="bg-q-background-primary text-q-text-primary p-4 rounded-lg">
  <h2 className="text-q-headline-md-semi-bold">Hello quanta</h2>
  <p className="text-q-body-md-regular text-q-text-secondary">World</p>
</div>
```

> **The `q-` namespace:** design-semantic utilities (color, typography, z-index,
> border-width, components) are prefixed with `q-` so they coexist with the
> legacy `@higgsfield/ui` system in one Tailwind build. Structural primitives
> (spacing, breakpoints) stay native, no prefix. Details: [`ai/AGENTS.md`](./ai/AGENTS.md).

## Package exports

| Path | Contents |
|---|---|
| `@higgsfield/quanta/tailwind.css` | `@theme` + `@utility` for Tailwind v4 |
| `@higgsfield/quanta/theme.css` | semantic aliases (color/typography by `data-theme`) |
| `@higgsfield/quanta/primitives.css` | raw literal values (`--hf-color-grey-900` etc.) |
| `@higgsfield/quanta/runtime` | `ThemeController`, `bootstrapScript`, `defineTheme`, `readInitialThemeState` |
| `@higgsfield/quanta/<component>` | React components — e.g. `/button`, `/dropdown` |

For large uniform asset feeds, use `VirtualGrid` from
`@higgsfield/quanta/grid`. Its optional `onEndReached` callback is deduplicated
per item count, so cursor pagination can stay infinite without mounting every
image or video.

Full token taxonomy, component APIs, and generation rules for AI agents live in
[`ai/AGENTS.md`](./ai/AGENTS.md). Token source: `src/css/tailwind/*.css`.

## License

Internal use only.
````

app/packages/quanta/src/components/accordion/accordion.css

```
/**
 * Accordion — quanta skin over Base UI `accordion`. Two looks: a hairline-
 * separated `list` (default) and standalone glass `separated` cards. The panel
 * collapse rides the Base UI `--accordion-panel-height` var with a `height`
 * transition; an inner content wrapper carries the padding so text doesn't
 * reflow mid-animation. Motion degrades under `prefers-reduced-motion`.
 *
 * Tokens only. The glass `separated` surface reuses the same recipe as `q-card`
 * (background-glass + 40px blur + radius-600).
 */
@source "./accordion.tsx";

/* ■■ Root ■■■
```

```
  padding-inline: var(--hf-space-200);
  background-color: transparent;
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--hf-radius-200);
  color: var(--hf-color-text-primary);
  text-align: start;
  cursor: pointer;
  outline: none;
  transition-property: background-color, color, box-shadow, opacity;
  transition-duration: var(--hf-duration-150);
  transition-timing-function: var(--hf-ease-out);
  @apply text-q-body-md-medium;
}

/* Separated cards already provide the card inset; give the trigger the card
 * horizontal padding instead of the bare list padding. */
.q-accordion-separated .q-accordion-trigger {
  padding-inline: var(--hf-space-500);
}

.q-accordion-trigger:hover {
  background-color: var(--hf-color-overlay-hover);
}

.q-accordion-trigger:focus-visible {
  box-shadow: inset 0 0 0 var(--hf-border-width-medium) var(--hf-color-border-focus);
}

.q-accordion-trigger[data-disabled] {
  opacity: var(--hf-opacity-disabled);
  pointer-events: none;
}

/* ■■ Trigger parts ■■■
```

```
  height: var(--accordion-panel-height);
  transition-property: height;
  transition-duration: var(--hf-duration-220);
  transition-timing-function: var(--hf-ease-out-expo);
}

/* Base UI sets the start/end style heights to 0 for the open/close animation. */
.q-accordion-panel[data-starting-style],
.q-accordion-panel[data-ending-style] {
  height: var(--hf-space-0);
}

@utility q-accordion-panel-content {
  padding-block-end: var(--hf-space-400);
  padding-inline: var(--hf-space-200);
  color: var(--hf-color-text-secondary);
  @apply text-q-body-sm-regular;
}

.q-accordion-separated .q-accordion-panel-content {
  padding-inline: var(--hf-space-500);
}

/* ■■ Sizes ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * `md` is the baseline above. `sm`/`lg` scale the row padding, the trigger + card
 * radius, and the typography. Compound selectors (`.q-accordion-sm.q-accordion-
 * separated …`) beat the variant rules so the separated insets scale too. */

/* sm — denser rows, tighter radius, smaller text. */
.q-accordion-sm .q-accordion-trigger {
  gap: var(--hf-space-200);
  padding-block: var(--hf-space-300);
  padding-inline: var(--hf-space-150);
  border-radius: var(--hf-radius-150);
  @apply text-q-body-sm-medium;
}
.q-accordion-sm.q-accordion-separated .q-accordion-trigger {
  padding-inline: var(--hf-space-400);
}
.q-accordion-sm.q-accordion-separated .q-accordion-item {
  border-radius: var(--hf-radius-400);
}
.q-accordion-sm .q-accordion-panel-content {
  padding-block-end: var(--hf-space-300);
  padding-inline: var(--hf-space-150);
  @apply text-q-caption-sm-regular;
}
.q-accordion-sm.q-accordion-separated .q-accordion-panel-content {
  padding-inline: var(--hf-space-400);
}

/* lg — roomier rows, larger radius, larger text. */
.q-accordion-lg .q-accordion-trigger {
  gap: var(--hf-space-400);
  padding-block: var(--hf-space-500);
  padding-inline: var(--hf-space-300);
  border-radius: var(--hf-radius-300);
  @apply text-q-body-lg-medium;
}
.q-accordion-lg.q-accordion-separated .q-accordion-trigger {
  padding-inline: var(--hf-space-600);
}
.q-accordion-lg .q-accordion-panel-content {
  padding-block-end: var(--hf-space-500);
  padding-inline: var(--hf-space-300);
  @apply text-q-body-md-regular;
}
.q-accordion-lg.q-accordion-separated .q-accordion-panel-content {
  padding-inline: var(--hf-space-600);
}
```

```
/* ■■ Reduced motion ■■■
```

app/packages/quanta/src/components/accordion/accordion.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { createRef } from 'react'
import { describe, expect, it, vi } from 'vitest'
import { Accordion } from './index.ts'

/** A minimal three-item accordion built from the public parts. */
function Basic(props: Parameters<typeof Accordion.Root>[0] = {}) {
  return (
    <Accordion.Root {...props}>
      <Accordion.Item value="a">
        <Accordion.Trigger>First</Accordion.Trigger>
        <Accordion.Panel>Panel A</Accordion.Panel>
      </Accordion.Item>
      <Accordion.Item value="b">
        <Accordion.Trigger>Second</Accordion.Trigger>
        <Accordion.Panel>Panel B</Accordion.Panel>
      </Accordion.Item>
      <Accordion.Item value="c" disabled>
        <Accordion.Trigger>Third</Accordion.Trigger>
        <Accordion.Panel>Panel C</Accordion.Panel>
      </Accordion.Item>
    </Accordion.Root>
  )
}

describe('<Accordion>', () => {
  it('renders triggers as buttons inside accessible headings', () => {
    render(<Basic />)
    const trigger = screen.getByRole('button', { name: 'First' })
    expect(trigger).toHaveClass('q-accordion-trigger')
    // Base UI wraps the trigger in an <h3> Header.
    expect(trigger.closest('.q-accordion-header')?.tagName).toBe('H3')
  })

  it('expands a panel when its trigger is pressed and toggles aria-expanded', async () => {
    const user = userEvent.setup()
    render(<Basic />)
    const trigger = screen.getByRole('button', { name: 'First' })
    expect(trigger).toHaveAttribute('aria-expanded', 'false')

    await user.click(trigger)
    expect(trigger).toHaveAttribute('aria-expanded', 'true')
    expect(trigger).toHaveAttribute('data-panel-open')
    expect(screen.getByText('Panel A')).toBeInTheDocument()
  })

  it('collapses the open item when a second is opened (single mode default)', async () => {
    const user = userEvent.setup()
    render(<Basic />)
    const first = screen.getByRole('button', { name: 'First' })
    const second = screen.getByRole('button', { name: 'Second' })

    await user.click(first)
    expect(first).toHaveAttribute('aria-expanded', 'true')
    await user.click(second)
    expect(first).toHaveAttribute('aria-expanded', 'false')
    expect(second).toHaveAttribute('aria-expanded', 'true')
  })

  it('keeps multiple panels open when multiple is set', async () => {
    const user = userEvent.setup()
    render(<Basic multiple />)
    const first = screen.getByRole('button', { name: 'First' })
    const second = screen.getByRole('button', { name: 'Second' })

    await user.click(first)
    await user.click(second)
```

```
    expect(first).toHaveAttribute('aria-expanded', 'true')
    expect(second).toHaveAttribute('aria-expanded', 'true')
  })

  it('honours defaultValue to open an item initially', () => {
    render(<Basic defaultValue={['b']} />)
    expect(screen.getByRole('button', { name: 'Second' })).toHaveAttribute('aria-expanded', 'true')
    expect(screen.getByRole('button', { name: 'First' })).toHaveAttribute('aria-expanded', 'false')
  })

  it('fires onValueChange with the new open set', async () => {
    const user = userEvent.setup()
    const onValueChange = vi.fn()
    render(<Basic onValueChange={onValueChange} />)
    await user.click(screen.getByRole('button', { name: 'First' }))
    expect(onValueChange).toHaveBeenCalled()
    expect(onValueChange.mock.lastCall?.[0]).toEqual(['a'])
  })

  it('disables an item via the disabled prop', async () => {
    const user = userEvent.setup()
    render(<Basic />)
    const disabled = screen.getByRole('button', { name: 'Third' })
    expect(disabled).toHaveAttribute('data-disabled')
    await user.click(disabled)
    expect(disabled).toHaveAttribute('aria-expanded', 'false')
  })

  it('applies the variant class to the root', () => {
    const { rerender } = render(<Basic variant="separated" />)
    const root = screen.getByRole('button', { name: 'First' }).closest('.q-accordion') as HTMLElement
    expect(root).toHaveClass('q-accordion-separated')

    rerender(<Basic variant="list" />)
    expect(screen.getByRole('button', { name: 'First' }).closest('.q-accordion')).toHaveClass('q-accordion-list')
  })

  it('renders start and end slots in the trigger', () => {
    render(
      <Accordion.Root>
        <Accordion.Item value="a">
          <Accordion.Trigger start={<span data-testid="lead" />} end={<span data-testid="trail" />}>
            Labelled
          </Accordion.Trigger>
          <Accordion.Panel>Body</Accordion.Panel>
        </Accordion.Item>
      </Accordion.Root>,
    )
    expect(screen.getByTestId('lead').closest('.q-accordion-trigger-start')).toBeInTheDocument()
    expect(screen.getByTestId('trail').closest('.q-accordion-trigger-end')).toBeInTheDocument()
    expect(screen.getByText('Labelled').closest('.q-accordion-trigger-label')).toBeInTheDocument()
  })

  it('forwards a ref to the root element', () => {
    const ref = createRef<HTMLDivElement>()
    render(
      <Accordion.Root ref={ref}>
        <Accordion.Item value="a">
          <Accordion.Trigger>First</Accordion.Trigger>
          <Accordion.Panel>Panel A</Accordion.Panel>
        </Accordion.Item>
      </Accordion.Root>,
    )
    expect(ref.current).toBeInstanceOf(HTMLElement)
    expect(ref.current).toHaveClass('q-accordion')
  })
})
```

app/packages/quanta/src/components/accordion/accordion.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { Accordion as Primitive } from '@base-ui/react/accordion'
import { IconChevronDownMediumOutlined } from '@higgsfield-ai/icons/IconChevronDownMediumOutlined'
import { Icon } from '../icon/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * Accordion — Base UI `accordion` (roving focus, keyboard, ARIA, the
 * `--accordion-panel-height` collapse var) skinned with quanta. Base UI owns all
 * behaviour/state/`data-*`; quanta only paints + forwards `className`/`ref`.
 *
 * Two looks via `Root`'s `variant`:
 *   • `list` (default) — a flat column of rows separated by hairline rules.
 *   • `separated` — each item is its own bordered glass card with a gap between.
 *
 * The `Trigger` renders inside the Base UI `Header` and shows the caller's label
 * plus a chevron `Icon` that rotates 180° on `[data-panel-open]`. The `Panel`
 * animates its height open/closed off the Base UI `--accordion-panel-height` var
 * (degrades under `prefers-reduced-motion`).
 *
 * Multi-part: `export const Accordion = { Root, Item, Trigger, Panel }`.
 */

export type AccordionVariant = 'list' | 'separated'
export type AccordionSize = 'sm' | 'md' | 'lg'

const VARIANT_CLASS = {
  list: 'q-accordion-list',
  separated: 'q-accordion-separated',
} satisfies Record<AccordionVariant, string>

// md is the default metrics baked into the base utilities; sm/lg scale the row
// padding, the trigger + card radius, and the typography down/up.
const SIZE_CLASS = {
  sm: 'q-accordion-sm',
  md: '',
  lg: 'q-accordion-lg',
} satisfies Record<AccordionSize, string>

export type AccordionRootProps = Omit<ComponentProps<typeof Primitive.Root>, 'className'> & {
  className?: string
  /** Visual look: a hairline-separated list (default) or standalone glass cards. */
  variant?: AccordionVariant
  /** Density: scales row padding, the trigger + card radius, and typography. */
  size?: AccordionSize
}

function Root({ className, variant = 'list', size = 'md', ...props }: AccordionRootProps) {
  return (
    <Primitive.Root
      className={cx('q-accordion', VARIANT_CLASS[variant], SIZE_CLASS[size], className)}
      {...props}
    />
  )
}

export type AccordionItemProps = Omit<ComponentProps<typeof Primitive.Item>, 'className'> & {
  className?: string
}

function Item({ className, ...props }: AccordionItemProps) {
  return (
    <Primitive.Item
      className={cx('q-accordion-item', className)}
      {...props}
    />
```

```
  )
}

export type AccordionTriggerProps = Omit<ComponentProps<typeof Primitive.Trigger>, 'className'> & {
  className?: string
  /** Class for the Base UI `Header` wrapper (the `<h3>`). */
  headerClassName?: string
  /** Leading slot (icon / any node), before the label. */
  start?: ReactNode
  /** Replace the default rotating chevron in the trailing slot. */
  end?: ReactNode
}

/**
 * Trigger renders inside the Base UI `Header` (an `<h3>`). The caller's label is
 * `children`; a chevron in the trailing slot rotates 180° on `[data-panel-open]`.
 */
function Trigger({ children, className, headerClassName, start, end, ...props }: AccordionTriggerProps) {
  return (
    <Primitive.Header className={cx('q-accordion-header', headerClassName)}>
      <Primitive.Trigger className={cx('q-accordion-trigger', className)} {...props}>
        {start != null ? <span className="q-accordion-trigger-start">{start}</span> : null}
        <span className="q-accordion-trigger-label">{children}</span>
        <span className="q-accordion-trigger-end">
          {end ?? (
            <Icon size="sm" className="q-accordion-chevron">
              <IconChevronDownMediumOutlined />
            </Icon>
          )}
        </span>
      </Primitive.Trigger>
    </Primitive.Header>
  )
}

export type AccordionPanelProps = Omit<ComponentProps<typeof Primitive.Panel>, 'className'> & {
  className?: string
  /** Class for the inner content wrapper (padding/typography live here). */
  contentClassName?: string
}

/**
 * Panel animates height via the Base UI `--accordion-panel-height` var. The panel
 * itself is `overflow: hidden` and transitions `height`; an inner wrapper holds
 * the real padding so content doesn't reflow during the collapse.
 */
function Panel({ children, className, contentClassName, ...props }: AccordionPanelProps) {
  return (
    <Primitive.Panel className={cx('q-accordion-panel', className)} {...props}>
      <div className={cx('q-accordion-panel-content', contentClassName)}>{children}</div>
    </Primitive.Panel>
  )
}

export const Accordion = { Root, Item, Trigger, Panel }
```

app/packages/quanta/src/components/accordion/index.ts

```
export { Accordion } from './accordion.tsx'
export type {
  AccordionItemProps,
  AccordionPanelProps,
  AccordionRootProps,
  AccordionSize,
  AccordionTriggerProps,
  AccordionVariant,
} from './accordion.tsx'
```

app/packages/quanta/src/components/autocomplete/autocomplete.css

```
/**
 * Autocomplete presentation utilities (q-autocomplete-* namespace).
 *
 * The Autocomplete reuses the quanta primitives wholesale and only adds the
 * combobox-specific chrome here:
 *   • INPUT  — the canonical field surface (`q-field-control` / `q-field-input` /
 *              `q-field-affix` from input.css); this file only adds the clear
 *              button + the cursor affordance so the whole control reads as a
 *              type-to-search field.
 *   • POPUP  — the dropdown glass card (`q-dropdown-content` from dropdown.css);
 *              this file scopes the filtered list to a scrollable region.
 *   • ROWS   — the shared `q-menu-item*` rows (menu.css), unchanged.
 *
 * Token-only. Animation degrades under prefers-reduced-motion (inherited from
 * q-dropdown-content).
 */

@source "./autocomplete.tsx";

/* The field control behaves as a clickable text-search surface. */
@utility q-autocomplete-control {
  cursor: text;
}

/* Auto-hiding clear button (Base UI toggles its [hidden]/visible state). Sits in
 * the trailing affix slot at the same 20px footprint as q-field-affix. */
@utility q-autocomplete-clear {
  align-items: center;
  appearance: none;
  background: transparent;
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--hf-radius-100);
  color: var(--hf-color-icon-secondary);
  cursor: pointer;
  display: inline-flex;
  flex-shrink: 0;
  height: var(--hf-space-500); /* 20px */
  justify-content: center;
  outline: none;
  padding: var(--hf-space-0);
  transition-property: color, background-color;
  transition-duration: var(--hf-duration-130);
  width: var(--hf-space-500);

  &:hover {
    background-color: var(--hf-color-overlay-hover);
    color: var(--hf-color-icon-primary);
  }

  &:focus-visible {
    box-shadow: inset 0 0 0 var(--hf-border-width-medium) var(--hf-color-border-focus);
  }

  & svg {
    height: 100%;
    width: 100%;
  }
}

/* Positioner spans the field width so the popup matches the input, with a
 * min-width floor so a narrow search field still shows full rows. */
@utility q-autocomplete-positioner {
  width: var(--anchor-width);
  min-width: var(--q-menu-min-width);
  outline: none;
}

/* ── Connected (nested field) ──────────────────────────────────────────── */
```

```
 * Mirrors Select: the popup is a WIDER rounded card that overlaps + sits BEHIND
 * the input, which keeps the higher z-index and stays typable on top. The popup
 * is anchored to the field WRAPPER (see autocomplete.tsx), centred and wider so
 * the input tucks inside, with top padding so the rows clear the input. Rendered
 * without the portal so the input wins the z-index in the shared stacking context. */
.q-autocomplete-positioner.q-autocomplete-positioner-connected {
  width: calc(var(--anchor-width) + 2 * var(--hf-space-200)); /* wider than the field */
  z-index: var(--hf-z-index-dropdown);
}

.q-autocomplete-content.q-autocomplete-content-connected {
  width: 100%;
  min-width: var(--hf-space-0);
  max-width: none;
  /* Clear the input that overlaps the top (default 40px field + breathing room). */
  padding-top: calc(var(--hf-space-1000) + var(--hf-space-400));
}

/* The input sits ON TOP of the popup it nests in (no-portal shared stacking ctx). */
.q-autocomplete-control.q-autocomplete-control-connected {
  position: relative;
  z-index: calc(var(--hf-z-index-dropdown) + 1);
}

/* The popup is the dropdown glass card; cap its height and let the list scroll. */
@utility q-autocomplete-content {
  max-height: var(--available-height, none);
  width: 100%;
}

/* Scrollable list region inside the popup. */
@utility q-autocomplete-list {
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-050); /* 2px — dropdown row rhythm */
  max-height: calc(var(--hf-space-2400) * 4); /* 320px — keeps long lists scrollable */
  min-width: var(--hf-space-0);
  overflow-y: auto;
  overscroll-behavior: contain;
}

/* Grouped rows share the popup's row rhythm (mirrors q-dropdown-group). */
@utility q-autocomplete-group {
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-050);
  min-width: var(--hf-space-0);
}

/* Empty state — centered, with comfortable padding around the NotFound block.
 * Base UI keeps the (role=status, aria-live) wrapper mounted but EMPTY while there
 * are matches; collapse it then with :empty so it doesn't show as a stray band
 * (an apparent divider) above the list. */
@utility q-autocomplete-empty {
  align-items: center;
  display: flex;
  justify-content: center;
  padding: var(--hf-space-400) var(--hf-space-200); /* 16px / 8px */

  &:empty {
    display: none;
  }
}
```

app/packages/quanta/src/components/autocomplete/autocomplete.test.tsx

```
import { render, screen, waitFor } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { createRef } from 'react'
import { describe, expect, it, vi } from 'vitest'
import { Autocomplete } from './index.ts'

const FRUITS = ['Apple', 'Apricot', 'Banana', 'Blueberry', 'Cherry', 'Grape', 'Mango', 'Orange']

function Basic(props: Partial<Parameters<typeof Autocomplete.Root>[0]> = {}) {
  return (
    <Autocomplete.Root items={FRUITS} openOnInputClick {...props}>
      <Autocomplete.Input placeholder="Search fruits" />
      <Autocomplete.Content>
        <Autocomplete.Empty>No fruits found.</Autocomplete.Empty>
        <Autocomplete.List>
          {(item: string) => (
            <Autocomplete.Item key={item} value={item}>{item}</Autocomplete.Item>
          )}
        </Autocomplete.List>
      </Autocomplete.Content>
    </Autocomplete.Root>
  )
}

describe('Autocomplete', () => {
  it('renders the input on the canonical field surface with a search affix', () => {
    render(<Basic />)
    const input = screen.getByPlaceholderText('Search fruits')
    expect(input).toHaveClass('q-field-input')
    expect(input.closest('.q-field-control')).toHaveClass('q-autocomplete-control')
  })

  it('exposes the input as a combobox', () => {
    render(<Basic />)
    expect(screen.getByRole('combobox')).toBe(screen.getByPlaceholderText('Search fruits'))
  })

  it('filters the list as the user types (Base UI matching)', async () => {
    const user = userEvent.setup()
    render(<Basic />)
    const input = screen.getByRole('combobox')
    await user.click(input)
    await user.type(input, 'ap')

    // Substring match against the items prop: Apple, Apricot, Grape.
    await waitFor(() => expect(screen.getByRole('option', { name: 'Apple' })).toBeInTheDocument())
    expect(screen.getByRole('option', { name: 'Apricot' })).toBeInTheDocument()
    expect(screen.getByRole('option', { name: 'Grape' })).toBeInTheDocument()
    expect(screen.queryByRole('option', { name: 'Banana' })).not.toBeInTheDocument()
  })

  it('renders the empty state when nothing matches', async () => {
    const user = userEvent.setup()
    render(<Basic />)
    const input = screen.getByRole('combobox')
    await user.click(input)
    await user.type(input, 'zzz')
    expect(await screen.findByText('No fruits found.')).toBeInTheDocument()
  })

  it('fires onValueChange as the input changes', async () => {
    const user = userEvent.setup()
    const onValueChange = vi.fn()
    render(<Basic onValueChange={onValueChange} />)
    await user.type(screen.getByRole('combobox'), 'man')
    expect(onValueChange).toHaveBeenCalled()
    expect(onValueChange.mock.calls.at(-1)?.[0]).toBe('man')
  })
```

```
it('clears the value via the Clear button', async () => {
  const user = userEvent.setup()
  render(<Basic />)
  const input = screen.getByRole('combobox') as HTMLInputElement
  await user.type(input, 'Banana')
  expect(input.value).toBe('Banana')

  const clearBtn = document.querySelector('.q-autocomplete-clear')
  expect(clearBtn).not.toBeNull()
  await user.click(clearBtn!)
  expect(input.value).toBe('')
})

it('paints popup + rows with the shared glass / menu utilities', async () => {
  const user = userEvent.setup()
  render(<Basic />)
  await user.click(screen.getByRole('combobox'))
  const option = await screen.findByRole('option', { name: 'Apple' })
  expect(option).toHaveClass('q-menu-item')
  expect(option.closest('.q-dropdown-content')).toHaveClass('q-autocomplete-content')
})

it('composes rich item rows from the shared parts', async () => {
  const user = userEvent.setup()
  render(
    <Autocomplete.Root items={FRUITS} openOnInputClick>
      <Autocomplete.Input placeholder="Search" />
      <Autocomplete.Content>
        <Autocomplete.List>
          {(item: string) => (
            <Autocomplete.Item key={item} value={item}>
              <Autocomplete.ItemContent>
                <Autocomplete.ItemTitleRow>
                  <Autocomplete.ItemTitle>{item}</Autocomplete.ItemTitle>
                </Autocomplete.ItemTitleRow>
                <Autocomplete.ItemDescription>A fruit</Autocomplete.ItemDescription>
              </Autocomplete.ItemContent>
            </Autocomplete.Item>
          )}
        </Autocomplete.List>
      </Autocomplete.Content>
    </Autocomplete.Root>,
  )
  await user.click(screen.getByRole('combobox'))
  const title = await screen.findByText('Apple')
  expect(title).toHaveClass('q-menu-item-title')
  expect(screen.getAllByText('A fruit')[0]).toHaveClass('q-menu-item-description')
})

it('selects an item on click and fills the input', async () => {
  const user = userEvent.setup()
  render(<Basic />)
  const input = screen.getByRole('combobox') as HTMLInputElement
  await user.click(input)
  await user.click(await screen.findByRole('option', { name: 'Cherry' }))
  expect(input.value).toBe('Cherry')
})

it('forwards a ref to the underlying input element', () => {
  const ref = createRef<HTMLInputElement>()
  render(
    <Autocomplete.Root items={FRUITS}>
      <Autocomplete.Input ref={ref} placeholder="Search" />
      <Autocomplete.Content>
        <Autocomplete.List>
          {(item: string) => <Autocomplete.Item key={item} value={item}>{item}</Autocomplete.Item>}
        </Autocomplete.List>
      </Autocomplete.Content>
    </Autocomplete.Root>,
  )
```

```
    expect(ref.current).toBeInstanceOf(HTMLInputElement)
    expect(ref.current).toBe(screen.getByPlaceholderText('Search'))
  })

  it('omits the clear button when clear is false', () => {
    render(
      <Autocomplete.Root items={FRUITS}>
        <Autocomplete.Input placeholder="Search" clear={false} />
        <Autocomplete.Content>
          <Autocomplete.List>
            {(item: string) => <Autocomplete.Item key={item} value={item}>{item}</Autocomplete.Item>}
          </Autocomplete.List>
        </Autocomplete.Content>
      </Autocomplete.Root>,
    )
    expect(document.querySelector('.q-autocomplete-clear')).toBeNull()
  })
})
```

app/packages/quanta/src/components/autocomplete/autocomplete.tsx

```
'use client'

import type { ComponentProps, ReactNode, RefObject } from 'react'
import { createContext, useContext, useRef } from 'react'
import { Autocomplete as Primitive } from '@base-ui/react/autocomplete'
import { IconCrossMediumOutlined as ClearIcon } from '@higgsfield-ai/icons/IconCrossMediumOutlined'
import { IconMagnifyingGlass2Outlined as SearchIcon } from '@higgsfield-ai/icons/IconMagnifyingGlass2Outlined'
import { Icon } from '../icon/index.ts'
import { NotFound } from '../not-found/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * Autocomplete — a filter-as-you-type combobox on the Base UI `Autocomplete`
 * primitive (which owns filtering, roving highlight, keyboard nav, ARIA, the
 * portal + positioning, and all `data-*` state). Quanta only paints: the input
 * is the canonical quanta field surface (white-5%, radius 12, lime focus ring),
 * and the popup reuses the dropdown glass card + shared `q-menu-item*` rows (see
 * `autocomplete.css`, `dropdown.css`, `menu.css`).
 *
 * COMPOSITION-FIRST (same contract as Dropdown / Command). `Autocomplete.Item`
 * is a styled row that renders whatever children you give it — typically a bare
 * label, or the shared `q-menu-item*` parts (icon / label / description /
 * trailing) for richer rows. Base UI does the matching from the `items` prop on
 * `Root`; the `List` renders the filtered subset:
 *
 *   <Autocomplete.Root items={fruits}>
 *     <Autocomplete.Input placeholder="Search fruits…" />
 *     <Autocomplete.Content>
 *       <Autocomplete.Empty>No fruits found.</Autocomplete.Empty>
 *       <Autocomplete.List>
 *         {(item: string) => (
 *           <Autocomplete.Item key={item} value={item}>{item}</Autocomplete.Item>
 *         )}
 *       </Autocomplete.List>
 *     </Autocomplete.Content>
 *   </Autocomplete.Root>
 *
 * Pass `value` / `defaultValue` / `onValueChange` / `items` / `filter` /
 * `openOnInputClick` etc. straight through to `Autocomplete.Root` (Base UI).
 * `Autocomplete.Content` bundles the Portal + Positioner + Popup; drop the
 * `List`, `Group`, `GroupLabel`, `Item` and `Empty` inside it.
 */

/* ■■ Root ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
 * Passes everything through to Base UI (it owns the value/filter model).
 * `connected` flows Root → Input + Content so the popup attaches flush to the
 * field as one seamless surface (mirrors Select). */
/**
 * `connected` flows to Input + Content; `controlRef` points the Positioner at
 * the FIELD WRAPPER (not the inner Base UI input, which is narrower and inset by
 * the search/clear affixes) so the popup matches the full field width + edge.
 */
const AutocompleteUiContext = createContext<{
  connected: boolean
  controlRef: RefObject<HTMLDivElement | null>
}>({ connected: false, controlRef: { current: null } })

export type AutocompleteRootProps = ComponentProps<typeof Primitive.Root> & {
  /**
   * Render the popup as a seamless extension of the input: same width, no gap,
   * merged corners so the field + list read as one continuous surface.
   */
  connected?: boolean
}

export function AutocompleteRoot({ connected = false, ...props }: AutocompleteRootProps) {
  const controlRef = useRef<HTMLDivElement>(null)
  return (
```

```
    <AutocompleteUiContext.Provider value={{ connected, controlRef }}>
      <Primitive.Root {...props} />
    </AutocompleteUiContext.Provider>
  )
}

/* ■■ Input ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
 * The quanta field surface (q-field-control) wrapping the Base UI Input, with a
 * leading search icon and an auto-hiding Clear button. Both icon slots are
 * overridable; `clear={false}` removes the clear affordance. */
export type AutocompleteInputProps =
  Omit<ComponentProps<typeof Primitive.Input>, 'className'> & {
    className?: string
    /** Class for the field-surface wrapper that holds the icon + input + clear. */
    controlClassName?: string
    /** Leading slot (20px). Defaults to a search icon; pass `null` to remove. */
    start?: ReactNode
    /** Show the auto-hiding Base UI Clear button (default true). */
    clear?: boolean
    /** Accessible label for the clear button. */
    clearLabel?: string
  }

const DEFAULT_START = <Icon size="md"><SearchIcon /></Icon>

export function AutocompleteInput({
  className,
  controlClassName,
  start = DEFAULT_START,
  clear = true,
  clearLabel = 'Clear',
  ...props
}: AutocompleteInputProps) {
  const { connected, controlRef } = useContext(AutocompleteUiContext)
  return (
    <div ref={controlRef} className={cx('q-field-control', 'q-autocomplete-control', connected && 'q-autocomplete-control-connected', controlClassName)}>
      {start != null ? <span className="q-field-affix">{start}</span> : null}
      <Primitive.Input
        className={cx('q-field-input', className)}
        autoComplete="off"
        autoCorrect="off"
        spellCheck={false}
        {...props}
      />
      {clear
        ? (
            <Primitive.Clear
              className="q-autocomplete-clear"
              aria-label={clearLabel}
              render={<button type="button" />}
            >
              <Icon size="sm"><ClearIcon /></Icon>
            </Primitive.Clear>
          )
        : null}
    </div>
  )
}

/* ■■ Content ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
 * Portal + Positioner + glass Popup, mirroring Dropdown.Content. The popup reuses
 * the dropdown glass surface (q-dropdown-content) so menus stay visually
 * identical. `solid` swaps the frosted glass for an opaque surface. */
export type AutocompleteContentProps =
  Omit<ComponentProps<typeof Primitive.Popup>, 'className'> & {
    className?: string
    /** Opaque surface instead of frosted glass. */
    solid?: boolean
    /** Side offset of the popup from the input (px). */
    sideOffset?: number
```

```
    /** Class for the Base UI Positioner. */
    positionerClassName?: string
    /** Portal container. */
    container?: ComponentProps<typeof Primitive.Portal>['container']
  }

export function AutocompleteContent({
  className,
  solid = false,
  sideOffset,
  positionerClassName,
  container,
  children,
  ...props
}: AutocompleteContentProps) {
  const { connected, controlRef } = useContext(AutocompleteUiContext)
  // Connected = the popup NESTS the field (mirrors Select): a wider rounded card
  // that overlaps + sits BEHIND the input (the input keeps the higher z-index and
  // stays typable on top), centred, with top padding so the rows clear the input.
  // Pull it up over the 40px field. Base UI Combobox REQUIRES the Portal, so the
  // popup stays portaled and the input wins via z-index in the root stacking
  // context. The floating glass keeps its 6px gap.
  const resolvedSideOffset = sideOffset ?? (connected ? -48 : 6)
  return (
    <Primitive.Portal container={container}>
      <Primitive.Positioner
        anchor={controlRef}
        className={cx('q-autocomplete-positioner', connected && 'q-autocomplete-positioner-connected', positionerClassName)}
        align={connected ? 'center' : undefined}
        sideOffset={resolvedSideOffset}
      >
        <Primitive.Popup
          className={cx(
            'q-dropdown-content',
            'q-autocomplete-content',
            solid && 'q-dropdown-content-solid',
            connected && 'q-autocomplete-content-connected',
            className,
          )}
          {...props}
        >
          {children}
        </Primitive.Popup>
      </Primitive.Positioner>
    </Primitive.Portal>
  )
}

/* ■■ List ■■■
```

```tsx
/* ■■ Group / GroupLabel ■■■
```

```
}
export function AutocompleteItemTitleRow({ className, ...props }: PartProps) {
  return <span className={cx('q-menu-item-title-row', className)} {...props} />
}
export function AutocompleteItemTitle({ className, ...props }: PartProps) {
  return <span className={cx('q-menu-item-title', className)} {...props} />
}
export function AutocompleteItemDescription({ className, ...props }: PartProps) {
  return <span className={cx('q-menu-item-description', className)} {...props} />
}
export function AutocompleteItemTrailing({ className, ...props }: PartProps) {
  return <span className={cx('q-menu-item-trailing', className)} {...props} />
}

/** `Autocomplete` namespace — the parts API. */
export const Autocomplete = Object.assign(AutocompleteRoot, {
  Root: AutocompleteRoot,
  Input: AutocompleteInput,
  Content: AutocompleteContent,
  List: AutocompleteList,
  Item: AutocompleteItem,
  Group: AutocompleteGroup,
  GroupLabel: AutocompleteGroupLabel,
  Collection: AutocompleteCollection,
  Empty: AutocompleteEmpty,
  Clear: AutocompleteClear,
  ItemIcon: AutocompleteItemIcon,
  ItemContent: AutocompleteItemContent,
  ItemTitleRow: AutocompleteItemTitleRow,
  ItemTitle: AutocompleteItemTitle,
  ItemDescription: AutocompleteItemDescription,
  ItemTrailing: AutocompleteItemTrailing,
})
```

app/packages/quanta/src/components/autocomplete/index.ts

```
export {
  Autocomplete,
  AutocompleteClear,
  AutocompleteCollection,
  AutocompleteContent,
  AutocompleteEmpty,
  AutocompleteGroup,
  AutocompleteGroupLabel,
  AutocompleteInput,
  AutocompleteItem,
  AutocompleteItemContent,
  AutocompleteItemDescription,
  AutocompleteItemIcon,
  AutocompleteItemTitle,
  AutocompleteItemTitleRow,
  AutocompleteItemTrailing,
  AutocompleteList,
  AutocompleteRoot,
} from './autocomplete.tsx'
export type {
  AutocompleteClearProps,
  AutocompleteCollectionProps,
  AutocompleteContentProps,
  AutocompleteEmptyProps,
  AutocompleteGroupLabelProps,
  AutocompleteGroupProps,
  AutocompleteInputProps,
  AutocompleteItemProps,
  AutocompleteListProps,
  AutocompleteRootProps,
} from './autocomplete.tsx'
```

app/packages/quanta/src/components/avatar/avatar.css

```
/**
 * Avatar presentation — pinned to the Figma "Avatar" component (node 1405:5456)
 * and its presence dot (node 1418:94, shipped standalone as Dot).
 *
 * The colour / typography / border-width values are emitted q-* token utilities
 * applied in avatar.tsx; the TS unions there (COLOR_BG / COLOR_FG / SIZE_*) are
 * the single source of truth. This file points Tailwind's scanner at the
 * colocated source (Tailwind skips node_modules and quanta is symlinked, so the
 * literal class strings must be registered explicitly — same as button.css) and
 * defines the one piece of geometry that has no token: where the dot sits.
 */
@source "./avatar.tsx";

/**
 * Anchors the presence dot on the avatar circle's bottom-right (45°) perimeter
 * point — matching Figma, where the dot's centre lands on the rim rather than
 * in the bounding-box corner. The offset is `50% + (√2 / 2) · 50% = 85.355%`,
 * which is proportional to the diameter, so a single rule fits every size. The
 * dot is then pulled back by half its own box to centre it on that point.
 */
@utility q-avatar-status {
  position: absolute;
  /* no token: proportional rim anchor (50% + √2/2·50%), scales with diameter — no px/spacing token applies */
  left: 85.355%;
  top: 85.355%;
  /* no token: self-centering offset is a proportion of the dot's own box, not a fixed length */
  transform: translate(-50%, -50%);
}
```

app/packages/quanta/src/components/avatar/avatar.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Avatar } from './index.ts'

describe('<Avatar>', () => {
  it('shows initials derived from alt (up to two words)', () => {
    render(<Avatar alt="Aria Zhang" />)
    expect(screen.getByText('AZ')).toBeInTheDocument()
  })

  it('renders the initials through Typography with the matching mono composite', () => {
    render(<Avatar alt="Aria Zhang" size="md" />)
    const initials = screen.getByText('AZ')
    expect(initials.tagName).toBe('SPAN')
    expect(initials).toHaveClass('text-q-mono-lg-semi-bold')
  })

  it('uses the dashed pending type ramp for the placeholder initials', () => {
    render(<Avatar alt="Add" variant="pending" size="md" />)
    // pending md = mono-sm (12px), distinct from the filled md ramp (mono-lg).
    expect(screen.getByText('A')).toHaveClass('text-q-mono-sm-semi-bold')
  })

  it('renders a custom fallback node', () => {
    render(<Avatar fallback={<strong>HF</strong>} />)
    expect(screen.getByText('HF')).toBeInTheDocument()
  })

  it('applies className to the avatar root', () => {
    const { container } = render(<Avatar alt="Aria Zhang" className="custom-avatar" />)
    const root = container.querySelector('.custom-avatar')
    expect(root).not.toBeNull()
    expect(root).toHaveClass('size-q-1000')
  })

  it('renders a labelled status dot', () => {
    render(<Avatar alt="Aria Zhang" status="online" />)
    expect(screen.getByRole('img', { name: 'online' })).toBeInTheDocument()
  })

  it('anchors the status dot on the avatar rim', () => {
    render(<Avatar alt="Aria Zhang" status="online" />)
    expect(screen.getByRole('img', { name: 'online' })).toHaveClass('q-avatar-status')
  })

  it('uses the Figma size ramp for xl avatars', () => {
    const { container } = render(<Avatar alt="Aria Zhang" size="xl" />)
    expect(container.querySelector('.size-q-1400')).not.toBeNull()
  })

  it('uses the Figma mono ramp: 16px for xl, 10px for xxs', () => {
    const xl = render(<Avatar alt="Aria Zhang" size="xl" />)
    expect(xl.container.querySelector('.text-q-mono-lg-semi-bold')).not.toBeNull()
    xl.unmount()
    const xxs = render(<Avatar alt="Aria Zhang" size="xxs" />)
    expect(xxs.container.querySelector('.text-q-mono-xs-semi-bold')).not.toBeNull()
  })

  it('keeps the legacy 2xs alias mapped to the Figma xxs size', () => {
    const { container } = render(<Avatar alt="Aria Zhang" size="2xs" />)
    expect(container.querySelector('.size-q-500')).not.toBeNull()
  })

  it('uses one initial for compact xs and xxs avatars', () => {
    render(<Avatar alt="Aria Zhang" size="xs" />)
    expect(screen.getByText('A')).toBeInTheDocument()
    expect(screen.queryByText('AZ')).not.toBeInTheDocument()
  })
```

```
  it('scales the online dot to the Figma status size for xl avatars', () => {
    render(<Avatar alt="Aria Zhang" size="xl" status="online" />)
    expect(screen.getByRole('img', { name: 'online' })).toHaveClass('size-q-200')
  })

  it('paints the status dot with the Figma presence color', () => {
    render(<Avatar alt="Aria Zhang" status="away" />)
    expect(screen.getByRole('img', { name: 'away' })).toHaveClass('bg-q-brand-yellow')
  })

  it('paints a colored disk via the matched palette bg + fg tokens', () => {
    const { container } = render(<Avatar alt="Aria Zhang" color="blue" />)
    const disk = container.querySelector('.bg-q-palette-blue-bg')
    expect(disk).not.toBeNull()
    expect(disk).toHaveClass('text-q-palette-blue-text')
  })

  it('gives light disks a dark fg so initials stay legible', () => {
    const { container } = render(<Avatar alt="Dana Kane" color="yellow" />)
    const disk = container.querySelector('.bg-q-brand-yellow')
    expect(disk).toHaveClass('text-q-text-inverse')
  })

  it('auto-derives a stable color from alt when none is given', () => {
    const a = render(<Avatar alt="Sam Park" />)
    const first = a.container.querySelector('[class*="bg-q-"]')?.className
    a.unmount()
    expect(first).toBeTruthy()
    const b = render(<Avatar alt="Sam Park" />)
    expect(b.container.querySelector('[class*="bg-q-"]')?.className).toBe(first)
  })

  it('renders a custom badge in the rim slot, replacing the default status Dot', () => {
    render(<Avatar alt="Aria Zhang" status="online" badge={<span data-testid="count">9+</span>} />)
    const badge = screen.getByTestId('count')
    expect(badge.closest('.q-avatar-status')).not.toBeNull()
    // the custom badge wins — no default presence Dot is rendered
    expect(screen.queryByRole('img', { name: 'online' })).not.toBeInTheDocument()
  })

  it('renders a dashed placeholder with no colored fill', () => {
    const { container } = render(<Avatar variant="pending" alt="Add" />)
    expect(container.querySelector('.border-dashed')).not.toBeNull()
    expect(container.querySelector('.border-q-medium')).not.toBeNull()
    expect(container.querySelector('.border-q-transparent-light-30')).not.toBeNull()
    expect(container.querySelector('svg.q-avatar-dash')).toBeNull()
    expect(container.querySelector('[class*="bg-q-palette-"]')).toBeNull()
  })

  it('treats an image avatar with no explicit color as a neutral surface', () => {
    const { container } = render(<Avatar src="/x.png" alt="Member" />)
    expect(container.querySelector('[class*="bg-q-palette-"]')).toBeNull()
    expect(container.querySelector('.bg-q-background-elevated-start')).not.toBeNull()
  })
})
```

app/packages/quanta/src/components/avatar/avatar.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { Avatar as Primitive } from '@base-ui/react/avatar'
import { Dot, type DotColor, type DotSize } from '../dot/index.ts'
import { Typography, type TypographyVariant, typography } from '../typography/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * Avatar — circular presence pinned to the Figma "Avatar" component
 * (node 1405:5456). Renders, in priority order, a photo, a custom fallback, or
 * mono-cased initials on a palette-coloured disk; an optional dashed variant is
 * the empty / "add" placeholder.
 *
 * Composable parts (every part is a replaceable node with a default):
 *   • `fallback` — the disk CONTENT slot (defaults to initials from `alt`); pass
 *     any node to override.
 *   • `badge` — the rim slot (defaults to the presence `<Dot>` derived from
 *     `status`); pass any node — a count `<Badge>`, a verified check, a custom
 *     `<Dot>` — to replace it. `status` stays as the convenience default.
 *   • `render` (via Base UI Avatar.Root) swaps the host element.
 */

export type AvatarSize = 'xxs' | '2xs' | 'xs' | 'sm' | 'md' | 'lg' | 'xl'
export type AvatarStatus = 'online' | 'offline' | 'away' | 'busy'
export type AvatarColor =
  | 'neutral'
  | 'orange'
  | 'mint'
  | 'blue'
  | 'pink'
  | 'purple'
  | 'brown'
  | 'yellow'
export type AvatarVariant = 'filled' | 'pending' | 'dashed'

type AvatarSizeKey = Exclude<AvatarSize, '2xs'>

const SIZE_BOX = {
  xxs: 'size-q-500', // 20px
  xs: 'size-q-600', // 24px
  sm: 'size-q-800', // 32px
  md: 'size-q-1000', // 40px
  lg: 'size-q-1200', // 48px
  xl: 'size-q-1400', // 56px
} satisfies Record<AvatarSizeKey, string>

// Initials type ramp — the Typography `variant` is the single source of truth.
// The Root applies it as a class via `typography({ variant })` so a
// consumer-passed `fallback` slot inherits the exact type, and the default
// initials render through <Typography> with the same variant (render-identical).

// Filled-disk ramp — pinned to the Figma "type=text" variants.
const SIZE_TEXT_VARIANT = {
  xxs: 'mono-xs-semi-bold', // 10px
  xs: 'mono-xs-semi-bold', // 10px
  sm: 'mono-sm-semi-bold', // 12px
  md: 'mono-lg-semi-bold', // 16px
  lg: 'mono-lg-semi-bold', // 16px
  xl: 'mono-lg-semi-bold', // 16px
} satisfies Record<AvatarSizeKey, TypographyVariant>

// Dashed-placeholder ramp — Figma's "type=pending" variants use a distinct,
// smaller type scale than the filled disks (md 12 vs 16, lg 14 vs 16, etc.).
const SIZE_TEXT_DASHED_VARIANT = {
  xxs: 'mono-xs-semi-bold', // 10px
  xs: 'mono-sm-semi-bold', // 12px
  sm: 'mono-sm-semi-bold', // 12px
```

```
  md: 'mono-sm-semi-bold', // 12px
  lg: 'mono-md-semi-bold', // 14px
  xl: 'mono-lg-semi-bold', // 16px
} satisfies Record<AvatarSizeKey, TypographyVariant>

// Pending ramp stroke weight — Figma uses thin (1px) below md, medium (1.5px) at md+.
const PENDING_BORDER = {
  xxs: 'border-q-thin',
  xs: 'border-q-thin',
  sm: 'border-q-thin',
  md: 'border-q-medium',
  lg: 'border-q-medium',
  xl: 'border-q-medium',
} satisfies Record<AvatarSizeKey, string>

const COLOR_BG = {
  orange: 'bg-q-palette-orange-bg',
  mint: 'bg-q-palette-mint-bg',
  blue: 'bg-q-palette-blue-bg',
  pink: 'bg-q-palette-pink-bg',
  purple: 'bg-q-palette-purple-bg',
  brown: 'bg-q-palette-brown-bg',
  yellow: 'bg-q-brand-yellow',
  neutral: 'bg-q-background-elevated-start',
} satisfies Record<AvatarColor, string>

const COLOR_FG = {
  orange: 'text-q-text-primary',
  mint: 'text-q-palette-mint-text',
  blue: 'text-q-palette-blue-text',
  pink: 'text-q-text-primary',
  purple: 'text-q-text-primary',
  brown: 'text-q-palette-brown-text',
  yellow: 'text-q-text-inverse',
  neutral: 'text-q-text-primary',
} satisfies Record<AvatarColor, string>

/** Presence colour the Figma dot uses for each avatar status. */
const STATUS_DOT_COLOR = {
  online: 'green',
  away: 'yellow',
  busy: 'red',
  offline: 'grey',
} satisfies Record<AvatarStatus, DotColor>

/** Dot scale the design pairs with each avatar size. */
const STATUS_DOT_SIZE = {
  xxs: 'xs',
  xs: 'xs',
  sm: 'sm',
  md: 'md',
  lg: 'md',
  xl: 'md',
} satisfies Record<AvatarSizeKey, DotSize>

const AUTO: AvatarColor[] = ['orange', 'mint', 'blue', 'pink', 'purple', 'brown', 'yellow']

type AvatarImageProps =
  Omit<ComponentProps<typeof Primitive.Image>, 'alt' | 'children' | 'className' | 'src'> & {
    className?: string
  }

export type AvatarProps = Omit<ComponentProps<typeof Primitive.Root>, 'children'> & {
  size?: AvatarSize
  src?: string
  alt?: string
  fallback?: ReactNode
  color?: AvatarColor
  status?: AvatarStatus
  /**
```

```
   * Rim slot. Defaults to the presence `<Dot>` for `status`; pass any node
   * (a `<Badge>`, a verified icon, a custom `<Dot>`) to replace it.
   */
  badge?: ReactNode
  variant?: AvatarVariant
  imageProps?: AvatarImageProps
}

export function Avatar({
  size = 'md',
  src,
  alt,
  fallback,
  color,
  status,
  badge,
  variant = 'filled',
  imageProps,
  className,
  ...props
}: AvatarProps) {
  const sizeKey = normalizeSize(size)
  const pending = variant === 'pending' || variant === 'dashed'
  // Only filled disks read a palette colour — skip the hash for placeholders.
  const avatarColor: AvatarColor = pending ? 'neutral' : (color ?? (src ? 'neutral' : autoColor(alt)))
  // One source for the initials type; the Root paints it (so a custom `fallback`
  // inherits) and the default initials below render with the same variant.
  const textVariant = (pending ? SIZE_TEXT_DASHED_VARIANT : SIZE_TEXT_VARIANT)[sizeKey]
  const textClass = typography({ variant: textVariant })
  const {
    className: imageClassName,
    decoding = 'async',
    loading = 'lazy',
    ...restImageProps
  } = imageProps ?? {}

  return (
    <Primitive.Root
      className={state => cx(
        'relative inline-flex shrink-0 select-none items-center justify-center overflow-visible rounded-q-full',
        SIZE_BOX[sizeKey],
        textClass,
        pending
          ? cx('border-dashed border-q-transparent-light-30 text-q-text-primary', PENDING_BORDER[sizeKey])
          : cx(COLOR_BG[avatarColor], COLOR_FG[avatarColor]),
        typeof className === 'function' ? className(state) : className,
      )}
      {...props}
    >
      {src && !pending
        ? (
            <Primitive.Image
              src={src}
              alt={alt ?? ''}
              loading={loading}
              decoding={decoding}
              className={cx('pointer-events-none absolute inset-0 size-full rounded-q-full object-cover', imageClassName)}
              {...restImageProps}
            />
          )
        : null}

      <Primitive.Fallback className="flex size-full items-center justify-center overflow-hidden rounded-q-full">
        {fallback ?? (
          <Typography as="span" variant={textVariant}>
            {initials(alt, sizeKey)}
          </Typography>
        )}
      </Primitive.Fallback>
```

```
      {/* Rim slot: a custom `badge` wins; otherwise the default presence Dot. */}
      {badge != null
        ? <span className="q-avatar-status">{badge}</span>
        : status != null
          ? (
              <Dot
                label={status}
                color={STATUS_DOT_COLOR[status]}
                size={STATUS_DOT_SIZE[sizeKey]}
                className="q-avatar-status"
              />
            )
          : null}
    </Primitive.Root>
  )
}

function normalizeSize(size: AvatarSize): AvatarSizeKey {
  return size === '2xs' ? 'xxs' : size
}

function initials(name: string | undefined, size: AvatarSizeKey): string {
  if (!name)
    return ''
  if (size === 'xs' || size === 'xxs')
    return name.trim()[0]?.toUpperCase() ?? ''
  return name
    .trim()
    .split(/\s+/)
    .slice(0, 2)
    .map(w => w[0]?.toUpperCase() ?? '')
    .join('')
}

function autoColor(name?: string): AvatarColor {
  if (!name)
    return 'neutral'
  let h = 0
  for (let i = 0; i < name.length; i++)
    h = (Math.imul(h, 31) + name.charCodeAt(i)) >>> 0
  return AUTO[h % AUTO.length]
}
```

app/packages/quanta/src/components/avatar/index.ts

```
export { Avatar } from './avatar.tsx'
export type {
  AvatarColor,
  AvatarProps,
  AvatarSize,
  AvatarStatus,
  AvatarVariant,
} from './avatar.tsx'
```

app/packages/quanta/src/components/badge/badge.css

```
/**
 * Badge presentation utilities (q-namespaced), matched to the Figma Badge
 * component at node 526:456.
 */

@source "./badge.tsx";

@utility q-badge {
  --q-badge-bg: var(--hf-color-brand-blue);
  --q-badge-fg: var(--hf-color-text-primary);
  --q-badge-height: var(--hf-type-line-height-200);
  --q-badge-min-width: var(--hf-space-800);
  --q-badge-padding-left: var(--hf-space-100);
  --q-badge-padding-right: var(--hf-space-150);
  --q-badge-radius: var(--hf-radius-050);
  --q-badge-font-family: "Space_Grotesk:Bold", var(--hf-type-family-grotesk); /* no token: Figma-exported font face literal; fallback uses the family token */
  --q-badge-font-size: var(--hf-type-size-100);
  --q-badge-letter-spacing: var(--hf-type-letter-spacing-none);
  --q-badge-skew: -10.89deg; /* no token: Figma badge skew angle */
  --q-badge-scale-y: 1; /* no token: unitless transform scale factor */
  --q-badge-frame-transform: none;

  align-items: center;
  color: var(--q-badge-fg);
  display: inline-flex;
  height: var(--q-badge-height);
  justify-content: center;
  line-height: var(--q-badge-height);
  min-width: var(--q-badge-min-width);
  position: relative;
  vertical-align: middle;
}

@utility q-badge-frame {
  align-items: center;
  box-sizing: border-box;
  display: inline-flex;
  height: var(--q-badge-height);
  justify-content: center;
  min-width: var(--q-badge-min-width);
  position: relative;
  transform: var(--q-badge-frame-transform);
  transform-origin: center;
}

@utility q-badge-surface {
  align-items: center;
  background: var(--q-badge-bg);
  border-radius: var(--q-badge-radius);
  box-sizing: border-box;
  display: inline-flex;
  gap: var(--hf-space-050);
  height: var(--q-badge-height);
  justify-content: center;
  min-width: var(--q-badge-min-width);
  overflow: hidden;
  padding: var(--hf-space-0) var(--q-badge-padding-right) var(--hf-space-0) var(--q-badge-padding-left);
  position: relative;
}

@utility q-badge-text {
  color: currentColor;
  /* Figma faux-bold: a 0.5px stroke in the text's own colour thickens the caps
   * (matches `-webkit-text-stroke: 0.5px var(--icon-inverse)` in node 526:456;
   * currentColor adapts it to each variant's fg). 0.5px = the hairline token. */
  -webkit-text-stroke: var(--hf-border-width-hairline) currentColor;
  display: flex;
  flex-direction: column;
```

```
  font-family: var(--q-badge-font-family);
  font-size: var(--q-badge-font-size);
  font-style: normal;
  font-weight: var(--hf-type-weight-bold);
  font-synthesis: none;
  flex-shrink: 0;
  justify-content: center;
  line-height: 0;
  position: relative;
  text-align: center;
  text-transform: uppercase;
  white-space: nowrap;
  word-break: break-word;
}

@utility q-badge-label {
  line-height: normal;
}

.q-badge-skew {
  --q-badge-height: 15px; /* no token: 0.9375rem not on the space scale */
  --q-badge-min-width: var(--hf-space-800);
  --q-badge-padding-left: var(--hf-space-100);
  --q-badge-padding-right: var(--hf-space-150);
  --q-badge-radius: var(--hf-radius-050);
  /* Figma label letter-spacing scales with size: -0.6px at xs (12px), -0.8px at
   * sm (16px) — same proportional pattern the compact text uses. */
  --q-badge-scale-y: 0.98; /* no token: unitless transform scale factor */
  --q-badge-frame-transform: skewX(var(--q-badge-skew)) scaleY(var(--q-badge-scale-y));
  --q-badge-skew-overhang: 1.4169px; /* no token: skew bounding-box overhang compensation */

  padding-inline: var(--q-badge-skew-overhang);
}

.q-badge-compact {
  --q-badge-height: var(--hf-type-line-height-100);
  --q-badge-min-width: var(--hf-space-700);
  --q-badge-padding-left: var(--hf-space-100);
  --q-badge-padding-right: var(--hf-space-150);
  --q-badge-radius: var(--hf-radius-050);
  --q-badge-font-family: var(--hf-type-family-grotesk);
  --q-badge-letter-spacing: var(--hf-type-letter-spacing-slight);
  --q-badge-scale-y: 1;
  --q-badge-frame-transform: none;
  --q-badge-slant: 2px; /* no token: the brand tag's right-edge lean (Figma 526:456 ticket); constant px so the lean reads the same at any width */

  /* The brand "ticket" shape, kept but made FLEXIBLE so it GROWS with its text.
   * The Figma `clip-path: path()` was fixed-coordinate (28×12 / 40×17) with the
   * label position:absolute on top, so any text past "new" spilled off the fill.
   * `shape()` recreates the SAME geometry responsively: the right edge leans in by
   * --q-badge-slant (the ticket signature) and every corner is an `arc` of
   * --q-badge-radius so they read ROUNDED, not cut (polygon vertices are sharp and
   * border-radius can't round a clip-path). The label flows INSIDE the surface, so
   * the fill always covers the text. border-radius (base) is the fallback for
   * engines without shape() → a plain rounded pill that still grows. */
  .q-badge-surface {
    clip-path: shape(
      from var(--q-badge-radius) 0,
      line to calc(100% - var(--q-badge-radius)) 0,
      arc to 100% var(--q-badge-radius) of var(--q-badge-radius) cw,
      line to calc(100% - var(--q-badge-slant)) calc(100% - var(--q-badge-radius)),
      arc to calc(100% - var(--q-badge-slant) - var(--q-badge-radius)) 100% of var(--q-badge-radius) cw,
      line to var(--q-badge-radius) 100%,
      arc to 0 calc(100% - var(--q-badge-radius)) of var(--q-badge-radius) cw,
      line to 0 var(--q-badge-radius),
      arc to var(--q-badge-radius) 0 of var(--q-badge-radius) cw,
      close
    );
  }
```

```
  .q-badge-text {
    display: block;
    letter-spacing: calc(var(--hf-type-letter-spacing-slight) * 1.5); /* no token: 1.5 is a unitless multiplier */
    line-height: 13px;
    text-align: start;
  }

  .q-badge-label {
    line-height: inherit;
  }
}

@utility q-badge-blue {
  --q-badge-bg: var(--hf-gradient-badge-blue);
  --q-badge-fg: var(--hf-color-text-primary);
}

@utility q-badge-lime {
  --q-badge-bg: var(--hf-color-brand-primary);
  --q-badge-fg: var(--hf-color-text-inverse);
}

@utility q-badge-pink {
  --q-badge-bg: var(--hf-gradient-badge-pink);
  --q-badge-fg: var(--hf-color-text-primary);
}

@utility q-badge-purple {
  --q-badge-bg: var(--hf-gradient-badge-purple);
  --q-badge-fg: var(--hf-color-text-primary);
}

@utility q-badge-lime-subtle {
  --q-badge-bg: var(--hf-color-transparent-lime-15);
  --q-badge-fg: var(--hf-color-text-link);
}

@utility q-badge-n-brand {
  --q-badge-bg: var(--hf-color-brand-primary);
  --q-badge-fg: var(--hf-color-text-inverse);
}

@utility q-badge-n-blue {
  --q-badge-bg: var(--hf-color-brand-cyan);
  --q-badge-fg: var(--hf-color-text-inverse);
}

/*
 * Size sm — Figma "Size=sm" variants (node 526:456).
 * Default (no size class) is xs, baked into the shape utilities above. The sm
 * deltas differ by shape, so they're applied as shape×size compound rules.
 */
.q-badge-skew.q-badge-sm {
  --q-badge-height: var(--hf-space-500);
  --q-badge-padding-left: var(--hf-space-100);
  --q-badge-padding-right: var(--hf-space-150);
  --q-badge-font-size: var(--hf-type-size-300);
  --q-badge-letter-spacing: calc(var(--hf-type-letter-spacing-slight) * 2); /* no token: 2 unitless multiplier → -0.8px (Figma node 526:456 inspect) */
  --q-badge-skew-overhang: 1.8892px; /* no token: skew bounding-box overhang compensation */
}

.q-badge-compact.q-badge-sm {
  --q-badge-min-width: var(--hf-space-1000);
  --q-badge-font-size: var(--hf-type-size-300);
  --q-badge-height: 17px; /* no token: 1.0625rem not on the space scale */
  --q-badge-slant: 2.5px; /* no token: taller tag keeps the same lean angle */
}
.q-badge-compact.q-badge-sm .q-badge-text {
  letter-spacing: calc(var(--hf-type-letter-spacing-slight) * 2); /* no token: 2 is a unitless multiplier */
}
```

app/packages/quanta/src/components/badge/badge.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Badge, badge } from './index.ts'

describe('badge() class-builder', () => {
  it('defaults to the blue skewed variant', () => {
    expect(badge()).toBe('q-badge q-badge-skew q-badge-blue')
  })

  it('applies compact variants and extra classes', () => {
    expect(badge({ variant: 'nBrand' }, 'is-custom', false)).toBe(
      'q-badge q-badge-compact q-badge-n-brand is-custom',
    )
  })

  it('applies the subtle lime skewed variant', () => {
    expect(badge({ variant: 'limeSubtle' })).toBe('q-badge q-badge-skew q-badge-lime-subtle')
  })
})

describe('<Badge>', () => {
  it('renders default text and classes', () => {
    render(<Badge />)

    const badgeControl = screen.getByText('Tag').closest('.q-badge')
    expect(badgeControl).toHaveClass('q-badge', 'q-badge-skew', 'q-badge-blue')
    expect(screen.getByText('Tag').closest('.q-badge-frame')).toBeTruthy()
  })

  it('renders compact default text', () => {
    render(<Badge variant="nBlue" />)

    const badgeControl = screen.getByText('new').closest('.q-badge')
    expect(badgeControl).toHaveClass('q-badge-compact', 'q-badge-n-blue')
  })

  it('lets children override text and forwards native props', () => {
    render(<Badge variant="purple" className="is-custom" title="Status">Exclusive</Badge>)

    const badgeControl = screen.getByText('Exclusive').closest('.q-badge')
    expect(badgeControl).toHaveClass('q-badge-purple', 'is-custom')
    expect(badgeControl).toHaveAttribute('title', 'Status')
  })
})
```

app/packages/quanta/src/components/badge/badge.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'

/**
 * Badge — a small presentational status marker. Figma variants Blue/Lime/Pink/
 * Purple/LimeSubtle are skewed uppercase caps; NBrand/NBlue are compact "new" markers.
 * Two sizes (Figma node 526:456): xs (default) and sm.
 */

export type BadgeVariant = 'blue' | 'lime' | 'pink' | 'purple' | 'limeSubtle' | 'nBrand' | 'nBlue'
export type BadgeSize = 'xs' | 'sm'

export interface BadgeOptions {
  variant?: BadgeVariant
  size?: BadgeSize
}

const VARIANT_CLASS = {
  blue: 'q-badge-blue',
  lime: 'q-badge-lime',
  pink: 'q-badge-pink',
  purple: 'q-badge-purple',
  limeSubtle: 'q-badge-lime-subtle',
  nBrand: 'q-badge-n-brand',
  nBlue: 'q-badge-n-blue',
} satisfies Record<BadgeVariant, string>

const SHAPE_CLASS = {
  blue: 'q-badge-skew',
  lime: 'q-badge-skew',
  pink: 'q-badge-skew',
  purple: 'q-badge-skew',
  limeSubtle: 'q-badge-skew',
  nBrand: 'q-badge-compact',
  nBlue: 'q-badge-compact',
} satisfies Record<BadgeVariant, string>

// xs is the default baked into the shape utilities; only sm adds a marker class.
const SIZE_CLASS = {
  xs: '',
  sm: 'q-badge-sm',
} satisfies Record<BadgeSize, string>

export type BadgeProps = ComponentProps<'span'> & {
  text?: ReactNode
  variant?: BadgeVariant
  size?: BadgeSize
}

export function badge(options: BadgeOptions = {}, ...extra: ClassValue[]): string {
  const { variant = 'blue', size = 'xs' } = options
  return cx('q-badge', SHAPE_CLASS[variant], VARIANT_CLASS[variant], SIZE_CLASS[size], ...extra)
}

export function Badge({ variant = 'blue', size = 'xs', text, className, children, ...props }: BadgeProps) {
  const isCompact = variant === 'nBrand' || variant === 'nBlue'
  const content = children ?? text ?? (isCompact ? 'new' : 'Tag')
  const label = (
    <span className="q-badge-text">
      <span className="q-badge-label">{content}</span>
    </span>
  )

  return (
    <span
```

```
      className={badge({ variant, size }, className)}
      {...props}
    >
      <span className="q-badge-frame">
        <span className="q-badge-surface">
          {label}
        </span>
      </span>
    </span>
  )
}
```

app/packages/quanta/src/components/badge/index.ts

```
export { Badge, badge } from './badge.tsx'
export type { BadgeOptions, BadgeProps, BadgeSize, BadgeVariant } from './badge.tsx'
```

app/packages/quanta/src/components/button-group/button-group.css

```
/**
 * ButtonGroup presentation utilities (q-namespaced). A pure-quanta LAYOUT around
 * quanta <Button>s — it owns no fills/typography; it only arranges the buttons
 * and, when attached, re-shapes their corners + collapses their adjacent edges
 * into a single shared hairline so the row reads as one segmented control.
 *
 * Token-only: the inner corners are zeroed to radius/none and the edge overlap
 * pulls each non-first button back by one --hf-border-width-thin (the 1px edge
 * worn by the outline/secondary buttons) so two abutting 1px borders render as
 * one hairline. The outer corners keep whatever radius the child Button's size
 * already set (radius/150..300) — the group never sets a radius, it only zeroes
 * the inner ones, so each size's outer rounding is preserved.
 *
 * Crisp for edged variants (outline / secondary / ghost-on-hover); acceptable
 * for solid fills (primary/danger) where there is no visible border to collapse.
 */

@source "./button-group.tsx";

@utility q-button-group {
  /* Edged buttons that overlap must paint above their neighbour on hover/focus
   * so the active outline isn't clipped by the next button's collapsed edge. */
  display: inline-flex;
  max-width: 100%;
  position: relative;

  & > .q-button {
    position: relative;
  }

  /* Lift the hovered/focused button so its full border + focus ring win over
   * the collapsed shared edge of its neighbours. */
  & > .q-button:hover,
  & > .q-button:focus-visible {
    z-index: 1;
  }
}

@utility q-button-group-horizontal {
  flex-direction: row;
}

@utility q-button-group-vertical {
  flex-direction: column;
  align-items: stretch;
}

/* ■■ Spaced: independent buttons, a small token gap, no edge surgery ■■■■■■■■■ */
@utility q-button-group-spaced {
  gap: var(--hf-space-200);
}

/* ■■ Attached: segmented control ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
 * Zero the inner corners and overlap neighbouring edges so the shared border
 * collapses to one hairline. Logical properties (start/end) keep this correct
 * under RTL and for both axes via the orientation-scoped rules below. */

/* Horizontal attached: collapse along the inline axis. */
.q-button-group-attached.q-button-group-horizontal > .q-button:not(:first-child) {
  border-start-start-radius: var(--hf-radius-0);
  border-end-start-radius: var(--hf-radius-0);
  margin-inline-start: calc(-1 * var(--hf-border-width-thin));
}

.q-button-group-attached.q-button-group-horizontal > .q-button:not(:last-child) {
  border-start-end-radius: var(--hf-radius-0);
  border-end-end-radius: var(--hf-radius-0);
}
```

```
/* Vertical attached: collapse along the block axis. */
.q-button-group-attached.q-button-group-vertical > .q-button:not(:first-child) {
  border-start-start-radius: var(--hf-radius-0);
  border-start-end-radius: var(--hf-radius-0);
  margin-block-start: calc(-1 * var(--hf-border-width-thin));
}

.q-button-group-attached.q-button-group-vertical > .q-button:not(:last-child) {
  border-end-start-radius: var(--hf-radius-0);
  border-end-end-radius: var(--hf-radius-0);
}
```

app/packages/quanta/src/components/button-group/button-group.test.tsx

```
import { render, screen } from '@testing-library/react'
import { createRef } from 'react'
import { describe, expect, it } from 'vitest'
import { Button } from '../button/index.ts'
import { ButtonGroup, buttonGroup } from './index.ts'

describe('ButtonGroup', () => {
  it('renders a role="group" with the composite class and its buttons', () => {
    render(
      <ButtonGroup aria-label="Text style">
        <Button>Bold</Button>
        <Button>Italic</Button>
      </ButtonGroup>,
    )
    const group = screen.getByRole('group', { name: 'Text style' })
    expect(group).toHaveClass('q-button-group')
    expect(group.querySelectorAll('.q-button')).toHaveLength(2)
  })

  it('defaults to attached + horizontal', () => {
    render(<ButtonGroup><Button>One</Button></ButtonGroup>)
    const group = screen.getByRole('group')
    expect(group).toHaveClass('q-button-group-attached', 'q-button-group-horizontal')
    expect(group).not.toHaveClass('q-button-group-spaced')
  })

  it('renders the spaced shape when attached={false}', () => {
    render(<ButtonGroup attached={false}><Button>One</Button></ButtonGroup>)
    const group = screen.getByRole('group')
    expect(group).toHaveClass('q-button-group-spaced')
    expect(group).not.toHaveClass('q-button-group-attached')
  })

  it('applies the vertical orientation class', () => {
    render(<ButtonGroup orientation="vertical"><Button>One</Button></ButtonGroup>)
    expect(screen.getByRole('group')).toHaveClass('q-button-group-vertical')
  })

  it('propagates size + variant to child Buttons', () => {
    render(
      <ButtonGroup size="sm" variant="outline">
        <Button>A</Button>
        <Button>B</Button>
      </ButtonGroup>,
    )
    for (const btn of screen.getAllByRole('button')) {
      expect(btn).toHaveClass('q-button-sm', 'q-button-outline')
    }
  })

  it("does not override a child's own size/variant", () => {
    render(
      <ButtonGroup size="sm" variant="outline">
        <Button>Shared</Button>
        <Button size="lg" variant="primary">Own</Button>
      </ButtonGroup>,
    )
    const own = screen.getByRole('button', { name: 'Own' })
    expect(own).toHaveClass('q-button-lg', 'q-button-primary')
    expect(own).not.toHaveClass('q-button-sm', 'q-button-outline')
  })

  it('leaves children untouched when no shared size/variant is set', () => {
    render(
      <ButtonGroup>
        <Button variant="ghost">Ghost</Button>
      </ButtonGroup>,
    )
```

```
    expect(screen.getByRole('button', { name: 'Ghost' })).toHaveClass('q-button-ghost')
  })

  it('forwards ref + extra props to the group div', () => {
    const ref = createRef<HTMLDivElement>()
    render(<ButtonGroup ref={ref} data-testid="grp"><Button>One</Button></ButtonGroup>)
    expect(ref.current).toBe(screen.getByTestId('grp'))
    expect(ref.current).toHaveAttribute('role', 'group')
  })

  it('lets the caller className win (appended last)', () => {
    render(<ButtonGroup className="custom-x"><Button>One</Button></ButtonGroup>)
    expect(screen.getByRole('group')).toHaveClass('q-button-group', 'custom-x')
  })

  it('buttonGroup() recipe emits the same class string', () => {
    expect(buttonGroup()).toBe('q-button-group q-button-group-horizontal q-button-group-attached')
    expect(buttonGroup({ orientation: 'vertical', attached: false }, 'extra'))
      .toBe('q-button-group q-button-group-vertical q-button-group-spaced extra')
  })
})
```

app/packages/quanta/src/components/button-group/button-group.tsx

```
'use client'

import type { ComponentProps, ReactElement, ReactNode } from 'react'
import { Children, cloneElement, isValidElement } from 'react'
import type { ButtonSize, ButtonVariant } from '../button/index.ts'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'

export type ButtonGroupOrientation = 'horizontal' | 'vertical'

export interface ButtonGroupOptions {
  /** Layout axis. `horizontal` (default) lays buttons in a row; `vertical` in a column. */
  orientation?: ButtonGroupOrientation
  /**
   * `true` (default) joins the buttons into a segmented control: inner corner
   * radii are removed and adjacent borders collapse onto one shared hairline so
   * only the outer corners stay rounded. `false` renders a spaced row/column
   * with a small gap between independent buttons.
   */
  attached?: boolean
}

/**
 * Orientation → axis class. The literal strings (not a template) are what the
 * Tailwind scanner extracts from this file — see `@source "./button-group.tsx"`
 * in button-group.css. `satisfies Record<…>` keeps the union the single source
 * of truth.
 */
const ORIENTATION_CLASS = {
  horizontal: 'q-button-group-horizontal',
  vertical: 'q-button-group-vertical',
} satisfies Record<ButtonGroupOrientation, string>

/** Build the button-group class string. Also usable to style a non-div host. */
export function buttonGroup(options: ButtonGroupOptions = {}, ...extra: ClassValue[]): string {
  const { orientation = 'horizontal', attached = true } = options
  return cx(
    'q-button-group',
    ORIENTATION_CLASS[orientation],
    attached ? 'q-button-group-attached' : 'q-button-group-spaced',
    ...extra,
  )
}

export type ButtonGroupProps = ComponentProps<'div'> & ButtonGroupOptions & {
  /**
   * Propagate one `size` to every child `<Button>` via `cloneElement`, so the
   * caller sets it once for the whole group. A child's own `size` wins (it is
   * only injected where the child didn't set one).
   */
  size?: ButtonSize
  /** Propagate one `variant` to every child `<Button>` — same precedence as `size`. */
  variant?: ButtonVariant
  /** The grouped buttons. Any node; quanta `<Button>`s get `size`/`variant` injected. */
  children?: ReactNode
}

/** Children that already declare the prop keep their own value; otherwise inject the group default. */
function injectSharedProps(
  children: ReactNode,
  shared: { size?: ButtonSize, variant?: ButtonVariant },
): ReactNode {
  if (shared.size === undefined && shared.variant === undefined) return children
  return Children.map(children, (child) => {
    if (!isValidElement(child)) return child
    const childProps = child.props as { size?: ButtonSize, variant?: ButtonVariant }
    const next: { size?: ButtonSize, variant?: ButtonVariant } = {}
    if (shared.size !== undefined && childProps.size === undefined) next.size = shared.size
```

```
    if (shared.variant !== undefined && childProps.variant === undefined) next.variant = shared.variant
    return Object.keys(next).length
      ? cloneElement(child as ReactElement<{ size?: ButtonSize, variant?: ButtonVariant }>, next)
      : child
  })
}

/**
 * ButtonGroup — a pure-quanta layout that groups quanta `<Button>`s into a row
 * or column. Two shapes:
 *
 *   attached (default) → a segmented control: inner radii removed, adjacent
 *     borders collapse to one shared hairline, only the outer corners round.
 *   spaced             → independent buttons with a small gap.
 *
 * It renders a `role="group"` div (pass `aria-label` to name it) and forwards
 * `ref` + `...props` to that div. Set `size` / `variant` once and they propagate
 * to every child `<Button>` (a child's own value wins).
 *
 *   <ButtonGroup aria-label="Text style" variant="outline">
 *     <Button>Bold</Button><Button>Italic</Button><Button>Underline</Button>
 *   </ButtonGroup>
 */
export function ButtonGroup({
  orientation = 'horizontal',
  attached = true,
  size,
  variant,
  className,
  children,
  ...props
}: ButtonGroupProps) {
  return (
    <div
      role="group"
      className={buttonGroup({ orientation, attached }, className)}
      {...props}
    >
      {injectSharedProps(children, { size, variant })}
    </div>
  )
}
```

app/packages/quanta/src/components/button-group/index.ts

```
export { ButtonGroup, buttonGroup } from './button-group.tsx'
export type { ButtonGroupOptions, ButtonGroupOrientation, ButtonGroupProps } from './button-group.tsx'
```

app/packages/quanta/src/components/button/button.css

```
/**
 * Button presentation utilities (q-namespaced). Composed by @apply-ing quanta
 * q- tokens; every color / spacing / radius / border-width / typography / shadow
 * value resolves to an emitted quanta token (or a token-derived value). Icons
 * are children, sized per size via `& svg`.
 *
 * Pinned PIXEL-PERFECT to Figma (dark theme). Default and Marketing share ONE
 * 5-step size ramp (Figma: "geometry identical to Button; only the visual layer
 * differs"); the size names were rebased to xxs..lg (previously xs..xl):
 *   Default   set 143:2356 / 107:2976   primary·secondary·tertiary·outline·
 *             ghost·danger·dangerSoft·BrandSoft       (sizes xxs/xs/sm/md)
 *   Marketing set 1533:1490              marketing primary/secondary/tertiary/
 *             ghost · special brand/pink   (primary/secondary xs..lg; rest xxs..lg)
 *   states    default/hover/focus/disabled
 *
 * Per size — height / h-padding / v-padding / radius / icon / type style. Type
 * is the COMPOSITE text-q-* utility (= Figma's named style) — NEVER hand-rolled,
 * so family(Inter)/size/weight/line-height/letter-spacing track the tokens:
 *   xxs  h 24  px 6   py 0   r 150(6)   icon 12  caption-sm-medium
 *   xs   h 32  px 8   py 6   r 200(8)   icon 16  label-sm-medium
 *   sm   h 40  px 12  py 8   r 250(10)  icon 20  label-sm-medium  (danger r 300)
 *   md   h 48  px 16  py 12  r 300(12)  icon 24  label-lg-semi-bold
 *   lg   h 56  px 20  py 16  r 300(12)  icon 24  label-lg-semi-bold
 * Weight is medium up to sm (xxs/xs/sm); semi-bold at md/lg — uniform across
 * variants (no per-variant weight bump).
 *
 * Notes:
 *   - radius uses the emitted --hf-radius-* tokens (150/200/250/300).
 *   - hover/focus = a flat overlay composited over the fill:
 *     lime / glass / red lighten with white 20% (transparent-light-20), white
 *     darkens with black 50% (transparent-dark-50). focus adds the lime ring.
 */

/* button() emits literal q-button-* strings (VARIANT_CLASS / SIZE_CLASS in
 * button.tsx). Tailwind skips node_modules for automatic content detection and
 * quanta is consumed as a symlinked package, so point the scanner explicitly at
 * the colocated source; the literals there match the @utility rules below. The
 * TS unions are the single source of truth — no hand-maintained safelist. */
@source "./button.tsx";

/* Reusable keyboard-focus rings — a tokenized background-colored offset gap
 * then a tokenized ring (Figma double-shadow: 0 0 0 2px background/primary,
 * 0 0 0 4px border/focus). Apply to any interactive element; offset assumes the
 * element sits on background-primary. General-purpose: hoist to a shared
 * focus.css if a second component needs them.
 *   q-focus-ring       — default (lime border-focus)
 *   q-focus-ring-error — destructive (red state/error-glow) */
@utility q-focus-ring {
  &:focus-visible {
    box-shadow:
      0 0 0 var(--hf-border-width-thick) var(--hf-color-background-primary),
      0 0 0 var(--hf-space-100) var(--hf-color-border-focus);
  }
}
@utility q-focus-ring-error {
  &:focus-visible {
    box-shadow:
      0 0 0 var(--hf-border-width-thick) var(--hf-color-background-primary),
      0 0 0 var(--hf-space-100) var(--hf-color-state-error-glow);
  }
}

@utility q-button {
  /* layout — [icon] label [icon] as centered grid columns */
  @apply inline-grid grid-flow-col max-w-full content-center items-center justify-center overflow-hidden;
  /* label */
  @apply text-center text-ellipsis whitespace-nowrap select-none;
  /* interaction */
```

```
  @apply cursor-pointer outline-none;
  @apply transition-[color,background-color,background-image,border-color,box-shadow];
  @apply q-focus-ring;
  @apply disabled:pointer-events-none disabled:opacity-50;
}

@utility q-button-label-frame {
  align-items: center;
  display: flex;
  gap: var(--hf-space-0);
  justify-content: center;
  min-inline-size: 0;
  overflow: hidden;
  padding-block: var(--hf-space-0);
  padding-inline: var(--hf-space-150);
}

.q-button:has(.q-button-label-frame) {
  gap: var(--hf-space-0);
}

/* Sizes — height/min-width on the quanta spacing scale; min-width == height
 * makes icon-only buttons square. Typography is the composite text-q-* utility
 * (= Figma's named type style) — never hand-rolled. Legacy direct-child icons
 * keep gap 6px; start/end slots use q-button-label-frame and collapse it to 0. */
@utility q-button-xxs {
  height: var(--hf-space-600);
  min-width: var(--hf-space-600);
  gap: var(--hf-space-150);
  padding: var(--hf-space-0) var(--hf-space-300);
  border-radius: var(--hf-radius-150);
  @apply text-q-caption-sm-medium;
  & svg {
    width: var(--hf-space-300);
    height: var(--hf-space-300);
  }
}
@utility q-button-xs {
  height: var(--hf-space-800);
  min-width: var(--hf-space-800);
  gap: var(--hf-space-150);
  padding: var(--hf-space-150) var(--hf-space-300);
  border-radius: var(--hf-radius-200);
  @apply text-q-label-sm-medium;
  & svg {
    width: var(--hf-space-400);
    height: var(--hf-space-400);
  }
}
@utility q-button-sm {
  height: var(--hf-space-1000);
  min-width: var(--hf-space-1000);
  gap: var(--hf-space-150);
  padding: var(--hf-space-200) var(--hf-space-400);
  border-radius: var(--hf-radius-250);
  @apply text-q-label-sm-medium;
  & svg {
    width: var(--hf-space-500);
    height: var(--hf-space-500);
  }
}
@utility q-button-md {
  height: var(--hf-space-1200);
  min-width: var(--hf-space-1200);
  gap: var(--hf-space-150);
  padding: var(--hf-space-400) var(--hf-space-400);
  border-radius: var(--hf-radius-300);
  @apply text-q-label-lg-semi-bold;
  & svg {
    width: var(--hf-space-600);
```

```
    height: var(--hf-space-600);
  }
}
@utility q-button-lg {
  height: var(--hf-space-1400);
  min-width: var(--hf-space-1400);
  gap: var(--hf-space-150);
  padding: var(--hf-space-400) var(--hf-space-500);
  border-radius: var(--hf-radius-300);
  @apply text-q-label-lg-semi-bold;
  & svg {
    width: var(--hf-space-600);
    height: var(--hf-space-600);
  }
}

/* ■■■■■■■■■■■■■■■■■■■■■■■■■■ Default variants (143:2356) ■■■■■■■■■■■■■■■■■■■■■■■ */
 * fill / text / border / shadow / hover, each pinned to Figma. */

/* primary: lime brand, dark label, 1.5px light edge; hover +20% white overlay.
 * Figma gives the border as raw rgba(255,255,255,0.45), with no variable. */
@utility q-button-primary {
  @apply bg-q-button-brand text-q-text-inverse;
  &:hover,
  &:focus-visible {
    background-image: linear-gradient(var(--hf-color-transparent-light-20), var(--hf-color-transparent-light-20));
  }
}

/* secondary: white, dark label + drop shadow (0 2 4 black/10); hover +50% black overlay */
@utility q-button-secondary {
  @apply bg-q-button-primary text-q-text-inverse;
  box-shadow: var(--hf-shadow-raised-sm);
  &:hover,
  &:focus-visible {
    background-image: linear-gradient(var(--hf-color-transparent-dark-50), var(--hf-color-transparent-dark-50));
  }
  &:focus-visible {
    box-shadow:
      0 0 0 var(--hf-border-width-thick) var(--hf-color-background-primary),
      0 0 0 var(--hf-space-100) var(--hf-color-border-focus),
      var(--hf-shadow-raised-sm);
  }
}

/* tertiary: translucent glass surface (white 10%) + background blur 40px
 * (Figma background-blur radius 80 → CSS /2); hover +20% white overlay */
@utility q-button-tertiary {
  background-color: var(--hf-color-transparent-light-10);
  @apply text-q-text-primary;
  backdrop-filter: blur(var(--hf-space-1000));
  &:hover,
  &:focus-visible {
    background-image: linear-gradient(var(--hf-color-transparent-light-20), var(--hf-color-transparent-light-20));
  }
}

/* outline: transparent, thin default-border (white 10%); hover/focus reveal a dim fill */
@utility q-button-outline {
  @apply bg-transparent text-q-text-primary;
  @apply border-q-thin border-q-border-default;
  @apply hover:bg-q-overlay-dim-strong focus-visible:bg-q-overlay-dim-strong;
}

/* ghost: transparent, no border; hover/focus reveal a dim fill */
@utility q-button-ghost {
  @apply bg-transparent text-q-text-primary;
  @apply hover:bg-q-overlay-dim-strong focus-visible:bg-q-overlay-dim-strong;
}
```

```
/* danger: destructive red, white label; hover +20% white overlay.
 * Focus ring is the red error-glow (not the lime default) — Figma 106:3513. */
@utility q-button-danger {
  @apply bg-q-button-destructive text-q-text-primary;
  &:hover,
  &:focus-visible {
    background-image: linear-gradient(var(--hf-color-transparent-light-20), var(--hf-color-transparent-light-20));
  }
  @apply q-focus-ring-error;
}

/* dangerSoft: error-bg surface, error-fg label; hover +20% white overlay.
 * Focus ring is the red error-glow (not the lime default) — Figma 106:3753. */
@utility q-button-danger-soft {
  @apply bg-q-state-error-bg text-q-state-error-fg;
  &:hover,
  &:focus-visible {
    background-image: linear-gradient(var(--hf-color-transparent-light-20), var(--hf-color-transparent-light-20));
  }
  @apply q-focus-ring-error;
}

/* brandSoft: transparent-lime-10 surface, lime link label + icon; hover +20% white
 * overlay; default lime focus ring (inherited from q-button). Figma 2376:1486
 * (fill transparent/lime/10, text text/link, icon icon/accent). */
@utility q-button-brand-soft {
  @apply bg-q-transparent-lime-10 text-q-text-link;
  &:hover,
  &:focus-visible {
    background-image: linear-gradient(var(--hf-color-transparent-light-20), var(--hf-color-transparent-light-20));
  }
}

/* ■■ Variant × size exceptions (Figma is not purely size-driven) ■■■■■■■■■■■■■■ */
/* danger + dangerSoft round one step more than their size at sm (radius/300 =
 * 12 vs the size's radius/250 = 10). Compound selectors (specificity 0,2,0)
 * beat the size @utility (0,1,0). */
.q-button-danger.q-button-sm,
.q-button-danger-soft.q-button-sm { border-radius: var(--hf-radius-300); }

.q-button-marketing-primary.q-button-xs,
.q-button-marketing-secondary.q-button-xs,
.q-button-marketing-tertiary.q-button-xs,
.q-button-marketing-ghost.q-button-xs {
  padding-block-end: var(--hf-space-200);
  padding-block-start: var(--hf-space-150);
}

.q-button-marketing-primary.q-button-sm,
.q-button-marketing-secondary.q-button-sm,
.q-button-marketing-tertiary.q-button-sm,
.q-button-marketing-ghost.q-button-sm {
  padding-block-end: var(--hf-space-250);
  padding-block-start: var(--hf-space-200);
}

/* md keeps Marketing's asymmetric vertical padding (extra at the bottom to
 * optically centre the label above the −3px bevel); typography inherits the
 * size's label-lg-semi-bold composite (no font override — Figma 18px, not 16). */
.q-button-marketing-primary.q-button-md,
.q-button-marketing-secondary.q-button-md,
.q-button-marketing-tertiary.q-button-md,
.q-button-marketing-ghost.q-button-md {
  padding-block-end: var(--hf-space-400);
  padding-block-start: var(--hf-space-400);
}

.q-button-marketing-primary.q-button-lg,
.q-button-marketing-secondary.q-button-lg {
  padding-block-end: var(--hf-space-500);
```

```
  padding-block-start: var(--hf-space-400);
}

.q-button-marketing-tertiary.q-button-lg,
.q-button-marketing-ghost.q-button-lg {
  padding: var(--hf-space-400);
}

.q-button-marketing-tertiary.q-button-xxs,
.q-button-marketing-ghost.q-button-xxs {
  padding-inline: var(--hf-space-100);
}

/* ■■■■■■■■■■■■■■■■■■■■■■ Marketing variants (1322:4366) ■■■■■■■■■■■■■■■■■■■■■■■■
 * Same geometry as the Default Button; only the visual layer differs. The 3D
 * key is a single 3px INSET BOTTOM line (box-shadow inset 0 -3px) — NOT an
 * extruded face — over soft drop shadows. Three Figma effect stacks:
 *
 *   GlossyLime (primary)   bevel inset 0 -3px #829b19 (olive)
 *                          + drops 0 6 4 #00000040 · 0 32 24 #00000026
 *   Glossy (secondary)     bevel inset 0 -3px #00000052 + same drops
 *   Glass (tertiary/ghost) backdrop-blur 12 · top sheen inset 0 2 3 white/5
 *                          · drop 0 2 4 -0.5 #0000001f
 *
 * The -3px line is token-derived: -(border-width-thick + thin) = -3px.
 * Raw values below are exactly the custom Figma effects/fills with no token. */

/* marketing primary: glossy lime CTA. Two stacked fill layers — a gloss sheen
 * (transparent → opaque highlight, brightness pooling at the bottom) over the
 * solid brand base — plus the olive bottom bevel. */
@utility q-button-marketing-primary {
  background-color: var(--hf-color-button-brand);
  background-image:
    var(--hf-gradient-special-gloss-lime),
    linear-gradient(90deg, var(--hf-color-button-brand) 0%, var(--hf-color-button-brand) 100%);
  border: var(--hf-border-width-none) solid transparent;
  color: var(--hf-color-text-inverse);
  box-shadow:
    inset 0 calc(-1 * (var(--hf-border-width-thick) + var(--hf-border-width-thin))) 0 0 #829b19, /* no token: Figma olive bevel color, no --hf-color-* equivalent */
    0 var(--hf-space-150) var(--hf-space-100) var(--hf-space-0) var(--hf-color-transparent-dark-25),
    0 var(--hf-space-800) var(--hf-space-600) var(--hf-space-0) var(--hf-color-transparent-dark-15);

  &:hover,
  &:focus-visible {
    background-image:
      linear-gradient(var(--hf-color-transparent-light-20), var(--hf-color-transparent-light-20)),
      var(--hf-gradient-special-gloss-lime),
      linear-gradient(90deg, var(--hf-color-button-brand) 0%, var(--hf-color-button-brand) 100%);
  }

  &:focus-visible {
    box-shadow:
      0 0 0 var(--hf-border-width-thick) var(--hf-color-background-primary),
      0 0 0 var(--hf-space-100) var(--hf-color-border-focus),
      inset 0 calc(-1 * (var(--hf-border-width-thick) + var(--hf-border-width-thin))) 0 0 #829b19, /* no token: Figma olive bevel color, no --hf-color-* equivalent */
      0 var(--hf-space-150) var(--hf-space-100) var(--hf-space-0) var(--hf-color-transparent-dark-25),
      0 var(--hf-space-800) var(--hf-space-600) var(--hf-space-0) var(--hf-color-transparent-dark-15);
  }
}

/* marketing secondary: glossy white with the dark bottom bevel. hover darkens
 * with black 10%. */
@utility q-button-marketing-secondary {
  background-color: var(--hf-color-button-primary);
  border: var(--hf-border-width-none) solid transparent;
  color: var(--hf-color-text-inverse);
  box-shadow:
    inset 0 calc(-1 * (var(--hf-border-width-thick) + var(--hf-border-width-thin))) 0 0 var(--hf-color-transparent-dark-32),
    0 var(--hf-space-150) var(--hf-space-100) var(--hf-space-0) var(--hf-color-transparent-dark-25),
    0 var(--hf-space-800) var(--hf-space-600) var(--hf-space-0) var(--hf-color-transparent-dark-15);
```

```
  &:hover,
  &:focus-visible {
    background-image: linear-gradient(var(--hf-color-transparent-dark-10), var(--hf-color-transparent-dark-10));
  }

  &:focus-visible {
    box-shadow:
      0 0 0 var(--hf-border-width-thick) var(--hf-color-background-primary),
      0 0 0 var(--hf-space-100) var(--hf-color-border-focus),
      inset 0 calc(-1 * (var(--hf-border-width-thick) + var(--hf-border-width-thin))) 0 0 var(--hf-color-transparent-dark-32),
      0 var(--hf-space-150) var(--hf-space-100) var(--hf-space-0) var(--hf-color-transparent-dark-25),
      0 var(--hf-space-800) var(--hf-space-600) var(--hf-space-0) var(--hf-color-transparent-dark-15);
  }
}

/* marketing tertiary: glass with a hairline edge + background blur 12px. The
 * top sheen is an inner white/5 highlight; the drop has a -0.5px spread. */
@utility q-button-marketing-tertiary {
  background-color: var(--hf-color-transparent-light-05);
  /* Edge bumped from border/subtle (white-5%, identical to the fill + inset
   * sheen → invisible) to a readable white-10% hairline so the glass edge
   * actually reads now the elevating drop shadow is gone. */
  border: var(--hf-border-width-thin) solid var(--hf-color-border-default);
  color: var(--hf-color-text-primary);
  box-shadow:
    inset 0 var(--hf-border-width-thick) calc(var(--hf-border-width-thick) + var(--hf-border-width-thin)) var(--hf-space-0) var(--hf-color-transparent-light-05);
  backdrop-filter: blur(var(--hf-space-300));

  &:hover,
  &:focus-visible {
    background-image: linear-gradient(var(--hf-color-transparent-light-20), var(--hf-color-transparent-light-20));
  }

  &:focus-visible {
    box-shadow:
      0 0 0 var(--hf-border-width-thick) var(--hf-color-background-primary),
      0 0 0 var(--hf-space-100) var(--hf-color-border-focus),
      inset 0 var(--hf-border-width-thick) calc(var(--hf-border-width-thick) + var(--hf-border-width-thin)) var(--hf-space-0) var(--hf-color-transparent-light-05);
  }
}

/* marketing ghost: same Figma Glass effect as tertiary, only without the border. */
@utility q-button-marketing-ghost {
  background-color: var(--hf-color-transparent-light-05);
  border: var(--hf-border-width-none) solid transparent;
  color: var(--hf-color-text-primary);
  box-shadow:
    inset 0 var(--hf-border-width-thick) calc(var(--hf-border-width-thick) + var(--hf-border-width-thin)) var(--hf-space-0) var(--hf-color-transparent-light-05);
  backdrop-filter: blur(var(--hf-space-300));

  &:hover,
  &:focus-visible {
    background-image: linear-gradient(var(--hf-color-transparent-light-20), var(--hf-color-transparent-light-20));
  }

  &:focus-visible {
    box-shadow:
      0 0 0 var(--hf-border-width-thick) var(--hf-color-background-primary),
      0 0 0 var(--hf-space-100) var(--hf-color-border-focus),
      inset 0 var(--hf-border-width-thick) calc(var(--hf-border-width-thick) + var(--hf-border-width-thin)) var(--hf-space-0) var(--hf-color-transparent-light-05);
  }
}

/* ■■■■■■■■■■■■■■ Special Button (2100:1476 default · 2100:1477 pink) ■■■■■■■■■■■
 * Tall credit CTA, fixed 112px wide. A solid token fill (button/brand or
 * brand/pink) lit by a gloss sheen, carrying the Figma "Glossy" effect: a -3px
 * inset bevel over two drop shadows. Geometry: radius/300, py space/600,
 * px space/500, gap space/150, column-centered; height is content-driven
 * (24+24 padding + 12 label + 6 gap + 16 price = 82, the Figma frame height).
 *
```

```
 * Figma binds the base fills to tokens, but the gloss and Glossy effect colors
 * are raw values; keep those exact values here. */
@utility q-button-special-brand {
  --q-button-special-bg: var(--hf-color-button-brand);
  --q-button-special-fg: var(--hf-color-text-inverse);
  --q-button-special-gloss: var(--hf-gradient-special-gloss-lime-angled);
}

@utility q-button-special-pink {
  --q-button-special-bg: var(--hf-color-brand-pink);
  --q-button-special-fg: var(--hf-color-text-primary);
  --q-button-special-gloss: var(--hf-gradient-special-gloss-pink);
}

.q-button-special-brand,
.q-button-special-pink {
  align-items: center;
  background-color: var(--q-button-special-bg);
  background-image:
    var(--q-button-special-gloss),
    linear-gradient(90deg, var(--q-button-special-bg) 0%, var(--q-button-special-bg) 100%);
  block-size: auto;
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--hf-radius-300);
  color: var(--q-button-special-fg);
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-150);
  inline-size: calc(var(--hf-space-2000) + var(--hf-space-800)); /* 112px = 80 + 32 */
  justify-content: center;
  min-inline-size: calc(var(--hf-space-2000) + var(--hf-space-800));
  overflow: clip;
  padding: var(--hf-space-600) var(--hf-space-500);
  text-transform: none;
  /* Figma "Glossy": -3px inset bevel over two drops (-3px = -(thick + thin)) */
  box-shadow:
    inset 0 calc(-1 * (var(--hf-border-width-thick) + var(--hf-border-width-thin))) 0 0 var(--hf-color-transparent-dark-32),
    0 var(--hf-space-150) var(--hf-space-100) var(--hf-space-0) var(--hf-color-transparent-dark-25),
    0 var(--hf-space-800) var(--hf-space-600) var(--hf-space-0) var(--hf-color-transparent-dark-15);

  /* sparkle lead-icon: 16×16, inherits the content color. */
  & svg {
    color: var(--q-button-special-fg);
    height: var(--hf-space-400);
    width: var(--hf-space-400);
  }

  /* hover/focus lighten with white 20%, composited over the gloss sheen. */
  &:hover,
  &:focus-visible {
    background-image:
      linear-gradient(var(--hf-color-transparent-light-20), var(--hf-color-transparent-light-20)),
      var(--q-button-special-gloss),
      linear-gradient(90deg, var(--q-button-special-bg) 0%, var(--q-button-special-bg) 100%);
  }

  &:focus-visible {
    box-shadow:
      0 0 0 var(--hf-border-width-thick) var(--hf-color-background-primary),
      0 0 0 var(--hf-space-100) var(--hf-color-border-focus),
      inset 0 calc(-1 * (var(--hf-border-width-thick) + var(--hf-border-width-thin))) 0 0 var(--hf-color-transparent-dark-32),
      0 var(--hf-space-150) var(--hf-space-100) var(--hf-space-0) var(--hf-color-transparent-dark-25),
      0 var(--hf-space-800) var(--hf-space-600) var(--hf-space-0) var(--hf-color-transparent-dark-15);
  }
}

@utility q-button-special-label {
  color: var(--q-button-special-fg);
  /* no token: Figma-specific weighted font face name; token grotesk family kept as fallback */
  font-family: "Space_Grotesk:Bold", var(--hf-type-family-grotesk);
```

```
  font-size: var(--hf-type-size-100);
  font-weight: var(--hf-type-weight-bold);
  letter-spacing: var(--hf-type-letter-spacing-none);
  line-height: var(--hf-type-line-height-100);
  text-transform: uppercase;
}

@utility q-button-special-price {
  align-items: flex-start;
  color: var(--q-button-special-fg);
  display: flex;
  font-family: var(--hf-type-family-secondary);
  font-size: var(--hf-type-size-100);
  font-weight: var(--hf-type-weight-medium);
  gap: var(--hf-space-100);
  letter-spacing: var(--hf-type-letter-spacing-none); /* Figma caption/sm/medium = 0 */
  line-height: var(--hf-type-line-height-300);
}

@utility q-button-special-price-old {
  text-decoration-line: line-through;
  text-decoration-skip-ink: none;
  text-underline-position: from-font;
}

/* Icon-only: collapse horizontal padding so the button is square (min-w == h). */
@utility q-button-icon-only {
  padding-inline: var(--hf-space-0);
}

.q-button-special-brand.q-button-icon-only,
.q-button-special-pink.q-button-icon-only {
  padding-inline: var(--hf-space-500);
}
```

app/packages/quanta/src/components/button/button.test.tsx

```
import { render, screen } from '@testing-library/react'
import { createRef } from 'react'
import { IconPlusMediumOutlined as PlusIcon } from '@higgsfield-ai/icons/IconPlusMediumOutlined'
import { describe, expect, it } from 'vitest'
import { Button, button } from './index.ts'

describe('button() class-builder', () => {
  it('defaults to primary + sm', () => {
    expect(button()).toBe('q-button q-button-primary q-button-sm')
  })
  it('applies variant + size', () => {
    expect(button({ variant: 'danger', size: 'lg' })).toBe('q-button q-button-danger q-button-lg')
  })
  it('adds icon-only and extra classes, dropping falsy', () => {
    expect(button({ iconOnly: true }, 'w-full', false)).toBe(
      'q-button q-button-primary q-button-sm q-button-icon-only w-full',
    )
  })
  it('kebab-cases camelCase variants (dangerSoft → danger-soft)', () => {
    expect(button({ variant: 'dangerSoft' })).toBe('q-button q-button-danger-soft q-button-sm')
  })
  it('includes marketing variants and the lg size', () => {
    expect(button({ variant: 'marketingPrimary', size: 'lg' })).toBe(
      'q-button q-button-marketing-primary q-button-lg',
    )
  })
  it('clamps marketing primary/secondary xxs up to xs', () => {
    expect(button({ variant: 'marketingPrimary', size: 'xxs' })).toBe(
      'q-button q-button-marketing-primary q-button-xs',
    )
    expect(button({ variant: 'marketingSecondary', size: 'xxs' })).toBe(
      'q-button q-button-marketing-secondary q-button-xs',
    )
  })
  it('keeps xxs for default and marketing glass variants', () => {
    expect(button({ variant: 'primary', size: 'xxs' })).toBe(
      'q-button q-button-primary q-button-xxs',
    )
    expect(button({ variant: 'marketingTertiary', size: 'xxs' })).toBe(
      'q-button q-button-marketing-tertiary q-button-xxs',
    )
    expect(button({ variant: 'marketingGhost', size: 'xxs' })).toBe(
      'q-button q-button-marketing-ghost q-button-xxs',
    )
  })
  it('includes special variants', () => {
    expect(button({ variant: 'specialPink' })).toBe(
      'q-button q-button-special-pink q-button-sm',
    )
  })
})

describe('<Button>', () => {
  it('renders a <button type="button"> with the classes', () => {
    render(<Button variant="secondary" size="sm">Go</Button>)
    const btn = screen.getByRole('button', { name: 'Go' })
    expect(btn.tagName).toBe('BUTTON')
    expect(btn).toHaveAttribute('type', 'button')
    expect(btn).toHaveClass('q-button', 'q-button-secondary', 'q-button-sm')
  })
  it('merges caller className and forwards native props', () => {
    render(<Button className="w-full" disabled>Go</Button>)
    const btn = screen.getByRole('button', { name: 'Go' })
    expect(btn).toHaveClass('q-button', 'w-full')
    expect(btn).toBeDisabled()
  })
  it('renders as a link via `as` (no type attribute)', () => {
    render(<Button as="a" href="/go" variant="ghost">Go</Button>)
```

```
    const link = screen.getByRole('link', { name: 'Go' })
    expect(link.tagName).toBe('A')
    expect(link).toHaveAttribute('href', '/go')
    expect(link).not.toHaveAttribute('type')
    expect(link).toHaveClass('q-button', 'q-button-ghost')
  })
  it('applies icon-only', () => {
    render(<Button iconOnly aria-label="Add"><PlusIcon /></Button>)
    expect(screen.getByRole('button', { name: 'Add' })).toHaveClass('q-button-icon-only')
  })
  it('renders marketing variants', () => {
    render(<Button variant="marketingTertiary" size="lg">Go</Button>)
    expect(screen.getByRole('button', { name: 'Go' })).toHaveClass(
      'q-button',
      'q-button-marketing-tertiary',
      'q-button-lg',
    )
  })
  it('renders special variants', () => {
    render(<Button variant="specialBrand">Generate</Button>)
    expect(screen.getByRole('button', { name: 'Generate' })).toHaveClass(
      'q-button',
      'q-button-special-brand',
      'q-button-sm',
    )
  })
  it('composes start / end slots around the label, in order', () => {
    render(
      <Button start={<PlusIcon data-testid="lead" />} end={<span data-testid="trail">⌘K</span>}>
        Search
      </Button>,
    )
    const btn = screen.getByRole('button', { name: /search/i })
    expect(btn).toHaveTextContent('Search')
    expect(screen.getByTestId('lead')).toBeInTheDocument()
    expect(screen.getByTestId('trail')).toBeInTheDocument()
    expect(btn.querySelector('.q-button-label-frame')).toHaveTextContent('Search')
    // start precedes the label which precedes end
    const order = btn.textContent ?? ''
    expect(btn.querySelector('[data-testid="trail"]')).toBeInTheDocument()
    expect(order.indexOf('Search')).toBeGreaterThanOrEqual(0)
  })
  it('renders only the label when no slots are passed (back-compat)', () => {
    render(<Button>Save</Button>)
    expect(screen.getByRole('button', { name: 'Save' })).toHaveTextContent('Save')
  })
  it('lets the caller override the default type', () => {
    render(<Button type="submit">Go</Button>)
    expect(screen.getByRole('button', { name: 'Go' })).toHaveAttribute('type', 'submit')
  })
  it('forwards a ref to the underlying button', () => {
    const ref = createRef<HTMLButtonElement>()
    render(<Button ref={ref}>Go</Button>)
    expect(ref.current).toBeInstanceOf(HTMLButtonElement)
  })
  it('forwards a ref to the polymorphic element', () => {
    const ref = createRef<HTMLAnchorElement>()
    render(<Button as="a" href="/go" ref={ref}>Go</Button>)
    expect(ref.current).toBeInstanceOf(HTMLAnchorElement)
  })
})

describe('<Button asChild>', () => {
  it('merges styling onto the child without adding a wrapper', () => {
    render(
      <Button asChild variant="ghost" size="sm">
        <a href="/go">Go</a>
      </Button>,
    )
    const link = screen.getByRole('link', { name: 'Go' })
```

```
    expect(link.tagName).toBe('A')
    expect(link).toHaveAttribute('href', '/go')
    expect(link).toHaveClass('q-button', 'q-button-ghost', 'q-button-sm')
    // no implicit type="button" leaks onto a non-button child
    expect(link).not.toHaveAttribute('type')
    // Slot renders the child in place — no surrounding <button>
    expect(screen.queryByRole('button')).toBeNull()
  })
  it('forwards a ref through the slot to the child', () => {
    const ref = createRef<HTMLButtonElement>()
    render(
      <Button asChild ref={ref}>
        <button>Go</button>
      </Button>,
    )
    expect(ref.current).toBeInstanceOf(HTMLButtonElement)
  })
})
```

app/packages/quanta/src/components/button/button.tsx

```
'use client'

import type { ComponentPropsWithRef, ElementType, ReactElement, ReactNode, Ref } from 'react'
import { createElement } from 'react'
import { useRender } from '@base-ui/react/use-render'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'

export type ButtonVariant =
  | 'primary' | 'secondary' | 'tertiary' | 'outline' | 'ghost' | 'danger' | 'dangerSoft' | 'brandSoft'
  | 'marketingPrimary' | 'marketingSecondary' | 'marketingTertiary' | 'marketingGhost'
  | 'specialBrand' | 'specialPink'
export type ButtonSize = 'xxs' | 'xs' | 'sm' | 'md' | 'lg'

export interface ButtonOptions {
  variant?: ButtonVariant
  size?: ButtonSize
  iconOnly?: boolean
}

/**
 * Variant/size → literal class strings. These literals (not a `q-button-${x}`
 * template) are what Tailwind's scanner extracts from this file — see the
 * `@source "./button.tsx"` in button.css — so every utility is generated.
 * `satisfies Record<…>` makes the unions the single source of truth: adding a
 * variant or size fails to compile until its class is registered here.
 */
const VARIANT_CLASS = {
  primary: 'q-button-primary',
  secondary: 'q-button-secondary',
  tertiary: 'q-button-tertiary',
  outline: 'q-button-outline',
  ghost: 'q-button-ghost',
  danger: 'q-button-danger',
  dangerSoft: 'q-button-danger-soft',
  brandSoft: 'q-button-brand-soft',
  marketingPrimary: 'q-button-marketing-primary',
  marketingSecondary: 'q-button-marketing-secondary',
  marketingTertiary: 'q-button-marketing-tertiary',
  marketingGhost: 'q-button-marketing-ghost',
  specialBrand: 'q-button-special-brand',
  specialPink: 'q-button-special-pink',
} satisfies Record<ButtonVariant, string>

const SIZE_CLASS = {
  xxs: 'q-button-xxs',
  xs: 'q-button-xs',
  sm: 'q-button-sm',
  md: 'q-button-md',
  lg: 'q-button-lg',
} satisfies Record<ButtonSize, string>

/** Marketing primary/secondary have no xxs (start at xs); glass variants keep xxs. */
const MARKETING_NO_XXS_VARIANTS = new Set<ButtonVariant>([
  'marketingPrimary', 'marketingSecondary',
])

/** Build the button class string. Also usable to style non-button elements. */
export function button(options: ButtonOptions = {}, ...extra: ClassValue[]): string {
  const { variant = 'primary', size = 'sm', iconOnly = false } = options
  // Marketing primary/secondary have no xxs — clamp to xs (Figma set 1533:1490:
  // marketing primary/secondary cover xs/sm/md/lg; tertiary/ghost also add xxs).
  const effectiveSize: ButtonSize = size === 'xxs' && MARKETING_NO_XXS_VARIANTS.has(variant) ? 'xs' : size
  return cx(
    'q-button',
    VARIANT_CLASS[variant],
    SIZE_CLASS[effectiveSize],
    iconOnly && 'q-button-icon-only',
```

```
    ...extra,
  )
}

type ButtonProps<E extends ElementType> = ButtonOptions & {
  /** Render as a different element (e.g. `as="a"`) with button styling. */
  as?: E
  /**
   * Merge button styling onto the single child instead of rendering an element.
   * Use to compose with trigger primitives —
   * `<Tooltip.Trigger render={<Button>…</Button>} />` or
   * `<Button asChild><a href>…</a></Button>` — so there's no extra DOM node and
   * one ref reaches the child. Composes via Base UI's `useRender` (the public
   * successor to radix `Slot`). Takes precedence over `as`.
   */
  asChild?: boolean
  className?: string
  /** The label. Any node. */
  children?: ReactNode
  /**
   * Leading slot before the label — defaults to nothing; pass any node (a quanta
   * icon, `<Avatar>`, `<Dot>`, a spinner…). Gap + icon sizing come from the size.
   */
  start?: ReactNode
  /** Trailing slot after the label — any node (a `<Badge>`, chevron, kbd…). */
  end?: ReactNode
} & Omit<ComponentPropsWithRef<E>, 'as' | 'asChild' | 'className' | 'children' | 'start' | 'end' | keyof ButtonOptions>

/**
 * Button — composite + AI-friendly: every part is a replaceable node with a
 * sensible default (nothing). The label is `children`; `start` / `end` are slots
 * that take any node, so you compose OTHER quanta components into the button
 * rather than hand-rolling markup:
 *
 *   <Button>Save</Button>                                  // label only
 *   <Button start={<PlusIcon/>}>New project</Button>       // leading icon
 *   <Button end={<Badge variant="nBrand">new</Badge>}>Upgrade</Button>
 *   <Button iconOnly aria-label="Search" start={<SearchIcon/>} />
 *   <Button as="a" href="/x">Link</Button>  ·  <Button asChild><a/></Button>
 */
export function Button<E extends ElementType = 'button'>(props: ButtonProps<E>) {
  const { as, asChild, variant, size, iconOnly, className, children, start, end, ref, ...rest } = props
  const cls = button({ variant, size, iconOnly }, className)

  // Composite content: `start` / `end` slots flank the label. They're optional
  // grid columns — the size's gap + `& svg` rule space and size them — so the
  // legacy icon-as-children pattern (`<Button><Icon/>Label</Button>`) is
  // unchanged when no slot is passed. Not applicable under `asChild` (the child
  // owns its own content).
  const hasSlots = start != null || end != null
  const hasLabelFrame = hasSlots && !iconOnly && children != null
  const content = hasSlots
    ? (
        <>
          {start}
          {hasLabelFrame ? <span className="q-button-label-frame">{children}</span> : children}
          {end}
        </>
      )
    : children

  // asChild → merge styling onto the caller's single child element.
  // as="x"  → render that element/tag.
  // default → a real <button> (gets implicit type="button" via defaultTagName).
  const render = asChild
    ? (children as ReactElement)
    : as
      ? createElement(as)
      : undefined
```

```
  // Only a real <button> gets the implicit type; `as="a"` / asChild must not.
  const isNativeButton = !asChild && (as === undefined || as === 'button')

  // useRender is called unconditionally (rules of hooks). It merges our props
  // with the render element's: className strings join, other props overwrite.
  return useRender({
    render,
    defaultTagName: 'button',
    ref: ref as Ref<Element> | undefined,
    props: {
      className: cls,
      ...(isNativeButton ? { type: 'button' as const } : {}),
      // When asChild, the child supplies its own children; don't override them.
      ...(asChild ? {} : { children: content }),
      ...rest,
    },
  })
}
```

app/packages/quanta/src/components/button/index.ts

```
export type { ButtonOptions, ButtonSize, ButtonVariant } from './button.tsx'
export { Button, button } from './button.tsx'
```

app/packages/quanta/src/components/card/card.css

```
/**
 * Card — the shared glass/solid SURFACE that Modal, Vault, Sonner, Dropdown,
 * cmdk and NavigationMenu each hand-roll today. It consolidates that exact
 * recipe so new surfaces don't reinvent it. The glass surface is PIXEL-IDENTICAL
 * to the Modal popup (Figma _Modal 1976:1460 = `q-modal`):
 *
 *   background-glass + 40px backdrop blur + radius-600 (24px) — and NOTHING else.
 *   No border, no box-shadow: the translucent fill + blur ARE the surface (the
 *   old inner-white "sheen" box-shadow was redundant given the blur/bg, and is
 *   removed). `q-card-raised` opts into the floating drop shadow used by
 *   popovers/toasts/sheets; `q-card-solid` swaps to the opaque secondary surface
 *   (no blur) for in-page panels.
 *
 * Composition parts mirror those components: Header / Title / Description / Body
 * / Footer. Tokens only. This component intentionally does NOT modify
 * Modal/Vault/etc.; it's the reusable primitive going forward.
 */
@source "./card.tsx";

@utility q-card {
  --q-card-radius: var(--hf-radius-600); /* 24px — the Modal surface radius; overridable per instance */

  box-sizing: border-box;
  display: flex;
  flex-direction: column;
  min-width: var(--hf-space-0);
  border-radius: var(--q-card-radius);
  background-color: var(--hf-color-background-glass);
  backdrop-filter: blur(var(--hf-space-1000)); /* 40px — Figma _Modal background blur */
  color: var(--hf-color-text-primary);
  /* No border, no box-shadow — the glass fill + 40px blur ARE the surface,
   * pixel-identical to the Modal popup. `q-card-raised` opts into elevation. */
}

/* Opaque secondary surface, no blur — in-page panels / cards on a solid bg. */
@utility q-card-solid {
  background-color: var(--hf-color-background-secondary);
  backdrop-filter: none;
}

/* Floating elevation — adds the soft drop shadow used by overlays/toasts/sheets. */
@utility q-card-raised {
  box-shadow: var(--hf-shadow-raised);
}

/* ■■ Composition parts ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
/* Header / Footer carry full padding; Body too, but its TOP padding collapses
 * when it directly follows a Header (the header already provides the top inset)
 * — so both "Header + Body" and bare "Body" compositions read correctly. */
@utility q-card-header {
  flex: none;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--hf-space-300);
  padding: var(--hf-space-500);
}

@utility q-card-heading {
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-050);
  min-width: var(--hf-space-0);
}

@utility q-card-title {
  @apply text-q-body-md-semi-bold;
  color: var(--hf-color-text-primary);
```

```
}

@utility q-card-description {
  @apply text-q-body-sm-regular;
  color: var(--hf-color-text-secondary);
}

@utility q-card-actions {
  flex: none;
  display: flex;
  align-items: center;
  gap: var(--hf-space-200);
}

@utility q-card-body {
  flex: 1 1 auto;
  min-height: var(--hf-space-0);
  padding: var(--hf-space-500);
}

.q-card-header + .q-card-body {
  padding-top: var(--hf-space-0);
}

@utility q-card-footer {
  flex: none;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--hf-space-300);
  padding: var(--hf-space-500);
  border-top: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
}
```

app/packages/quanta/src/components/card/card.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Card, card } from './index.ts'

describe('<Card>', () => {
  it('renders the glass surface by default', () => {
    const { container } = render(<Card>content</Card>)
    const root = container.firstElementChild!
    expect(root).toHaveClass('q-card')
    expect(root).not.toHaveClass('q-card-solid')
    expect(root).not.toHaveClass('q-card-raised')
  })

  it('applies surface + elevation variants', () => {
    const { container } = render(<Card surface="solid" elevation="raised">x</Card>)
    expect(container.firstElementChild).toHaveClass('q-card', 'q-card-solid', 'q-card-raised')
  })

  it('composes Header (title/description/actions), Body and Footer', () => {
    render(
      <Card>
        <Card.Header title="Share" description="Anyone with the link" actions={<button>Done</button>} />
        <Card.Body>Body text</Card.Body>
        <Card.Footer>Footer text</Card.Footer>
      </Card>,
    )
    // Title / Description render through Typography (composite + color) while
    // keeping their q-card-* class, on the original <div> tag.
    const title = screen.getByText('Share')
    expect(title.tagName).toBe('DIV')
    expect(title).toHaveClass('q-card-title', 'text-q-body-md-semi-bold', 'text-q-text-primary')
    const description = screen.getByText('Anyone with the link')
    expect(description).toHaveClass('q-card-description', 'text-q-body-sm-regular', 'text-q-text-secondary')
    expect(screen.getByRole('button', { name: 'Done' }).closest('.q-card-actions')).toBeInTheDocument()
    expect(screen.getByText('Body text')).toHaveClass('q-card-body')
    expect(screen.getByText('Footer text')).toHaveClass('q-card-footer')
  })

  it('forwards className + native div props', () => {
    const { container } = render(<Card className="extra" data-testid="c" aria-label="panel">x</Card>)
    const root = container.firstElementChild!
    expect(root).toHaveClass('q-card', 'extra')
    expect(root).toHaveAttribute('aria-label', 'panel')
  })

  it('defaults the root to a <div> (back-compat)', () => {
    const { container } = render(<Card>x</Card>)
    expect(container.firstElementChild?.tagName).toBe('DIV')
  })

  it('forwards ref to the root surface element', () => {
    let node: HTMLDivElement | null = null
    const { container } = render(<Card ref={(el) => { node = el }}>x</Card>)
    expect(node).toBe(container.firstElementChild)
    expect(node).toHaveClass('q-card')
  })

  it('forwards ref to the swapped `render` element', () => {
    let node: HTMLAnchorElement | null = null
    render(<Card render={<a href="/p" ref={(el) => { node = el }} />}>Open</Card>)
    expect(node).not.toBeNull()
    expect(node!.tagName).toBe('A')
    expect(node).toHaveClass('q-card')
  })

  it('swaps the host element via `render`, keeping the surface class', () => {
    render(<Card render={<a href="/p" />}>Open</Card>)
    const link = screen.getByRole('link', { name: 'Open' })
```

```
    expect(link.tagName).toBe('A')
    expect(link).toHaveClass('q-card')
    expect(link).toHaveAttribute('href', '/p')
  })

  it('`card()` builds the surface class string for any element', () => {
    expect(card()).toBe('q-card')
    expect(card({ surface: 'solid', elevation: 'raised' })).toBe('q-card q-card-solid q-card-raised')
    expect(card({}, 'mt-4')).toBe('q-card mt-4')
  })
})
```

app/packages/quanta/src/components/card/card.tsx

```
'use client'

import type { ComponentProps, ReactElement, ReactNode, Ref } from 'react'
import { useRender } from '@base-ui/react/use-render'
import { Typography } from '../typography/index.ts'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'

/**
 * Card — the reusable glass/solid surface that the overlay components (Modal,
 * Vault, Sonner, Dropdown, cmdk, NavigationMenu) each build inline today. Use
 * it for any panel, popover body, sheet, tile, or section that needs the
 * quanta glass look, instead of re-deriving the recipe.
 *
 *   <Card elevation="raised">
 *     <Card.Header title="Share" description="Anyone with the link"
 *       actions={<Button>Done</Button>} />
 *     <Card.Body>…</Card.Body>
 *     <Card.Footer>…</Card.Footer>
 *   </Card>
 *
 * `surface` = 'glass' (background-glass + 40px blur, no border/shadow — pixel-
 * identical to the Modal popup, default) | 'solid' (opaque secondary, no blur).
 * `elevation` = 'flat' (bare surface, default) | 'raised' (adds the floating
 * drop shadow for popovers/toasts/sheets). All slots are optional — a bare
 * `<Card>` with arbitrary children works too. `card()` is a class-builder for
 * styling a non-div element as a card surface.
 *
 * Host element is swappable via `render` (Base UI useRender) — keep the surface
 * but render a semantic or interactive root:
 *   <Card render={<article/>}>…</Card>
 *   <Card render={<a href="/p"/>}>…</Card>   ·   <Card render={<Link/>}>…</Card>
 */

export type CardSurface = 'glass' | 'solid'
export type CardElevation = 'flat' | 'raised'

export interface CardOptions {
  surface?: CardSurface
  elevation?: CardElevation
}

const SURFACE_CLASS = {
  glass: '',
  solid: 'q-card-solid',
} satisfies Record<CardSurface, string>

const ELEVATION_CLASS = {
  flat: '',
  raised: 'q-card-raised',
} satisfies Record<CardElevation, string>

/** Build the card surface class string — usable to skin any element as a card. */
export function card(options: CardOptions = {}, ...extra: ClassValue[]): string {
  const { surface = 'glass', elevation = 'flat' } = options
  return cx('q-card', SURFACE_CLASS[surface], ELEVATION_CLASS[elevation], ...extra)
}

export type CardProps = Omit<ComponentProps<'div'>, keyof CardOptions> & CardOptions & {
  /**
   * Swap the root element/component while keeping the surface styling — a
   * semantic `<article>` / `<section>`, or a clickable `<a>` / `<button>` /
   * framework `<Link>`. Defaults to a `<div>`.
   */
  render?: ReactElement
}

function Root({ surface, elevation, className, render, ref, ...props }: CardProps) {
```

```
  // `ref` must go to useRender's dedicated `ref` option — it is NOT picked up
  // from `props`, so spreading it there would drop it (it would never reach the
  // rendered root / `render` element).
  return useRender({
    render,
    defaultTagName: 'div',
    ref: ref as Ref<Element> | undefined,
    props: { className: card({ surface, elevation }, className), ...props },
  })
}

export type CardHeaderProps = Omit<ComponentProps<'div'>, 'title'> & {
  /** Primary heading. Any node. */
  title?: ReactNode
  /** Secondary line under the title. Any node. */
  description?: ReactNode
  /** Trailing controls (buttons, close, etc.). Any node. */
  actions?: ReactNode
}

function Header({ title, description, actions, children, className, ...props }: CardHeaderProps) {
  return (
    <div className={cx('q-card-header', className)} {...props}>
      {children ?? (
        <>
          {title != null || description != null
            ? (
                <div className="q-card-heading">
                  {title != null ? <Title>{title}</Title> : null}
                  {description != null ? <Description>{description}</Description> : null}
                </div>
              )
            : null}
          {actions != null ? <div className="q-card-actions">{actions}</div> : null}
        </>
      )}
    </div>
  )
}

type DivProps = ComponentProps<'div'>
// Title / Description render through Typography (exact-equivalent of the
// composite + color the q-card-title / q-card-description CSS already applied):
// `as="div"` preserves the original tag, the q-card-* class is kept (recipe /
// external styling + sibling selectors), and className stays last so callers win.
function Title({ className, color: _color, ...props }: DivProps) {
  return <Typography as="div" variant="body-md-semi-bold" color="primary" className={cx('q-card-title', className)} {...props} />
}
function Description({ className, color: _color, ...props }: DivProps) {
  return <Typography as="div" variant="body-sm-regular" color="secondary" className={cx('q-card-description', className)} {...props} />
}
function Body({ className, ...props }: DivProps) {
  return <div className={cx('q-card-body', className)} {...props} />
}

export type CardFooterProps = ComponentProps<'div'> & {
  actions?: ReactNode
}
function Footer({ actions, children, className, ...props }: CardFooterProps) {
  return (
    <div className={cx('q-card-footer', className)} {...props}>
      {children ?? (actions != null ? <div className="q-card-actions">{actions}</div> : null)}
    </div>
  )
}

export const Card = Object.assign(Root, {
  Header,
  Title,
  Description,
```

```
  Body,
  Footer,
})
```

app/packages/quanta/src/components/card/index.ts

```
export { Card, card } from './card.tsx'
export type {
  CardElevation,
  CardFooterProps,
  CardHeaderProps,
  CardOptions,
  CardProps,
  CardSurface,
} from './card.tsx'
```

app/packages/quanta/src/components/checkbox/checkbox.css

```
/**
 * Checkbox presentation utilities (q-namespaced), matched to the Figma
 * component at node 481:795 (parent 478:1397).
 *
 * Figma ground truth (Dev Mode variable defs / metadata / exported vectors):
 *   Two-layer structure:
 *     frame  (tap + focus target)  sm=20 md=24 lg=28  → --hf-space-500/600/700
 *     box    (visible square)      sm=18 md=20 lg=24  → frame minus inset
 *   frame radius 8px (custom in Figma), box radius 6px (radius/150)
 *   border 1.5px (border-width/medium)
 *
 *   Per-state (brand), verified against exported SVGs:
 *     inactive   1.5px border, border/default #ffffff1a
 *     hover      1.5px border, custom #4f6109
 *     active     fill icon/accent #d1fe17, glyph icon/inverse #1a1a1a
 *     focused    active + frame bg/primary + transparent-lime-20 ring @ r8
 *     disabled   fill background/glass #23262abf, NO visible border, glyph icon/disabled #484e56
 *     indeterminate           fill #d1fe17, minus #1a1a1a
 *     indeterminate_disabled  fill #23262a (secondary-strong), NO border, minus #484e56
 */

@source "./checkbox.tsx";

@utility q-checkbox {
  --q-checkbox-fill: var(--hf-color-icon-accent);
  --q-checkbox-fg: var(--hf-color-icon-inverse);
  --q-checkbox-hover-border: var(--hf-color-lime-900);
  --q-checkbox-focus-ring: var(--hf-color-transparent-lime-20);
  --q-checkbox-target-size: var(--hf-space-600);
  --q-checkbox-box-size: var(--hf-space-500);

  align-items: center;
  background-color: transparent;
  border: var(--hf-border-width-none) solid transparent;
  box-sizing: border-box;
  color: var(--q-checkbox-fg);
  cursor: pointer;
  display: inline-flex;
  flex-shrink: 0;
  height: var(--q-checkbox-target-size);
  justify-content: center;
  outline: none;
  padding: var(--hf-space-0);
  position: relative;
  border-radius: var(--hf-radius-200);
  transition-property: background-color, border-color, box-shadow, color;
  vertical-align: top;
  width: var(--q-checkbox-target-size);
}

@utility q-checkbox-box {
  align-items: center;
  background-color: transparent;
  border: var(--hf-border-width-medium) solid var(--hf-color-border-default);
  border-radius: var(--hf-radius-150);
  box-sizing: border-box;
  color: transparent;
  display: flex;
  height: var(--q-checkbox-box-size);
  justify-content: center;
  transition-property: background-color, border-color, color;
  width: var(--q-checkbox-box-size);
}

@utility q-checkbox-indicator {
  align-items: center;
  color: currentColor;
  display: flex;
```

```
  height: var(--q-checkbox-box-size);
  justify-content: center;
  width: var(--q-checkbox-box-size);

  /* Glyph occupies ~70% of the box (the Figma check/minus sit inset, not
   * edge-to-edge); the 10×8 viewBox scales with `meet`, so this also tames the
   * stroke proportionally. % keeps it correct across sm/md/lg. */
  & svg {
    height: 70%;
    width: 70%;
  }
}

@utility q-checkbox-sm {
  --q-checkbox-target-size: var(--hf-space-500);
  --q-checkbox-box-size: calc(var(--hf-space-500) - var(--hf-space-050));
}

@utility q-checkbox-md {
  --q-checkbox-target-size: var(--hf-space-600);
  --q-checkbox-box-size: var(--hf-space-500);
}

@utility q-checkbox-lg {
  --q-checkbox-target-size: var(--hf-space-700);
  --q-checkbox-box-size: var(--hf-space-600);
}

@utility q-checkbox-brand {
  --q-checkbox-fill: var(--hf-color-icon-accent);
  --q-checkbox-fg: var(--hf-color-icon-inverse);
  --q-checkbox-hover-border: var(--hf-color-lime-900);
  --q-checkbox-focus-ring: var(--hf-color-transparent-lime-20);
}

@utility q-checkbox-white {
  --q-checkbox-fill: var(--hf-color-palette-white);
  --q-checkbox-fg: var(--hf-color-icon-inverse);
  --q-checkbox-hover-border: var(--hf-color-transparent-light-30);
  --q-checkbox-focus-ring: var(--hf-color-transparent-light-20);
}

.q-checkbox:not([data-checked]):not([data-indeterminate]):not([data-disabled]):hover .q-checkbox-box {
  border-color: var(--q-checkbox-hover-border);
}

.q-checkbox[data-checked] .q-checkbox-box,
.q-checkbox[data-indeterminate] .q-checkbox-box {
  background-color: var(--q-checkbox-fill);
  border-color: var(--q-checkbox-fill);
  color: var(--q-checkbox-fg);
}

.q-checkbox:focus-visible {
  background-color: var(--hf-color-background-primary);
  box-shadow: 0 0 0 var(--hf-border-width-thick) var(--q-checkbox-focus-ring);
}

.q-checkbox[data-disabled] {
  cursor: not-allowed;
}

/* Disabled base: glass fill + icon/disabled glyph. Only the UNCHECKED disabled
 * box shows the icon/disabled outline (so the empty target stays visible). */
.q-checkbox[data-disabled] .q-checkbox-box {
  background-color: var(--hf-color-background-glass);
  border-color: var(--hf-color-icon-disabled);
  color: var(--hf-color-icon-disabled);
}
```

```
/* Disabled + checked: NO visible border — a filled box needs no outline (Figma
 * 481:814 shows the check on the glass fill with no border line; the box's 1.5px
 * border is the fill colour, not a grey edge). */
.q-checkbox[data-disabled][data-checked]:not([data-indeterminate]) .q-checkbox-box {
  border-color: transparent;
}

/* indeterminate_disabled: opaque secondary-strong fill, NO border, icon/disabled minus. */
.q-checkbox[data-disabled][data-indeterminate] .q-checkbox-box {
  background-color: var(--hf-color-background-secondary-strong);
  border-color: var(--hf-color-background-secondary-strong);
  color: var(--hf-color-icon-disabled);
}

.q-checkbox[data-disabled]:not([data-checked]):not([data-indeterminate]) .q-checkbox-box {
  color: transparent;
}

@utility q-checkbox-label {
  align-items: flex-start;
  color: var(--hf-color-text-primary);
  cursor: pointer;
  display: flex;
  gap: var(--hf-space-300);
  min-width: var(--hf-space-0);
}

@utility q-checkbox-label-left {
  justify-content: flex-start;
}

@utility q-checkbox-label-right {
  justify-content: space-between;
}

@utility q-checkbox-label-text {
  display: flex;
  flex: 1 1 var(--hf-space-0);
  flex-direction: column;
  gap: var(--hf-space-100);
  min-width: var(--hf-space-0);
  overflow-wrap: anywhere;
}

/* Title/description colour + composite type are now applied by <Typography>
 * (color="primary"/"tertiary", variant label-sm-medium/label-md-medium/label-sm-regular)
 * in checkbox.tsx. Only the md title's line-height override has no Typography
 * equivalent, so it stays here. */
@utility q-checkbox-label-md {
  .q-checkbox-label-title {
    line-height: var(--hf-space-500);
  }
}
```

app/packages/quanta/src/components/checkbox/checkbox.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Checkbox, CheckboxLabel, checkbox } from './index.ts'

describe('checkbox() class-builder', () => {
  it('defaults to brand + md', () => {
    expect(checkbox()).toBe('q-checkbox q-checkbox-brand q-checkbox-md')
  })

  it('applies color + size', () => {
    expect(checkbox({ color: 'white', size: 'lg' })).toBe('q-checkbox q-checkbox-white q-checkbox-lg')
  })

  it('merges extra classes and ignores falsy values', () => {
    expect(checkbox({ color: 'white', size: 'sm' }, 'is-custom', false)).toBe(
      'q-checkbox q-checkbox-white q-checkbox-sm is-custom',
    )
  })
})

describe('<Checkbox>', () => {
  it('renders a Base UI checkbox with Figma classes', () => {
    render(<Checkbox aria-label="Accept" defaultChecked color="white" size="sm" />)

    const control = screen.getByRole('checkbox', { name: 'Accept' })
    expect(control).toHaveAttribute('aria-checked', 'true')
    expect(control).toHaveClass('q-checkbox', 'q-checkbox-white', 'q-checkbox-sm')
  })

  it('renders an indeterminate state', () => {
    render(<Checkbox aria-label="Partially selected" indeterminate />)

    const control = screen.getByRole('checkbox', { name: 'Partially selected' })
    expect(control).toHaveAttribute('aria-checked', 'mixed')
    expect(control).toHaveAttribute('data-indeterminate')
  })

  it('merges caller classes', () => {
    render(<Checkbox aria-label="Custom" className="is-custom" />)
    expect(screen.getByRole('checkbox', { name: 'Custom' })).toHaveClass('q-checkbox', 'is-custom')
  })
})

describe('<CheckboxLabel>', () => {
  it('renders the label, description, and checkbox via Typography', () => {
    render(<CheckboxLabel label="Use feature" description="Description" />)

    // Title: Typography(label-sm-medium / primary) keeps the q-checkbox-label-title hook.
    const title = screen.getByText('Use feature')
    expect(title).toHaveClass('q-checkbox-label-title', 'text-q-label-sm-medium', 'text-q-text-primary')
    // Description: Typography(label-sm-regular / tertiary) keeps its class hook.
    const description = screen.getByText('Description')
    expect(description).toHaveClass(
      'q-checkbox-label-description',
      'text-q-label-sm-regular',
      'text-q-text-tertiary',
    )
    expect(screen.getByRole('checkbox', { name: /Use feature/ })).toHaveClass('q-checkbox')
  })

  it('supports right-aligned checkbox and medium label typography', () => {
    render(<CheckboxLabel direction="right" size="md" label="Use feature" />)
    const title = screen.getByText('Use feature')
    // md title switches the Typography variant.
    expect(title).toHaveClass('q-checkbox-label-title', 'text-q-label-md-medium')
    expect(title.closest('.q-checkbox-label')).toHaveClass(
      'q-checkbox-label-right',
      'q-checkbox-label-md',
```

```
    )
  })
})
```

app/packages/quanta/src/components/checkbox/checkbox.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { Checkbox as Primitive } from '@base-ui/react/checkbox'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'
import { Typography, type TypographyVariant } from '../typography/index.ts'

/**
 * Indicator glyphs inlined to match the Figma checkbox EXACTLY (node 481:795):
 * a chunky 2px-stroke check on a 10×8 grid with round caps/joins — distinct from
 * the package `IconCheckmark2MediumOutlined`, which is a different shape on a
 * 24×24 grid at stroke 1.5 (≈half the weight). `currentColor` inherits the
 * indicator fg (icon-inverse on the filled disk). The minus mirrors the check's
 * weight so checked/indeterminate read as one family.
 */
function CheckGlyph() {
  return (
    <svg viewBox="0 0 10 8" fill="none" aria-hidden="true" xmlns="http://www.w3.org/2000/svg">
      <path d="M1 4.06L3.34 6.4L8.56 1" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round" strokeLinejoin="round" />
    </svg>
  )
}

function MinusGlyph() {
  return (
    <svg viewBox="0 0 10 8" fill="none" aria-hidden="true" xmlns="http://www.w3.org/2000/svg">
      <path d="M2 4H8" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round" />
    </svg>
  )
}

export type CheckboxSize = 'sm' | 'md' | 'lg'
// Figma node 481:795 ships exactly two colors: brand (lime) and white.
export type CheckboxColor = 'brand' | 'white'
export type CheckboxLabelDirection = 'left' | 'right'
export type CheckboxLabelSize = 'sm' | 'md'

export interface CheckboxOptions {
  color?: CheckboxColor
  size?: CheckboxSize
}

const COLOR_CLASS = {
  brand: 'q-checkbox-brand',
  white: 'q-checkbox-white',
} satisfies Record<CheckboxColor, string>

const SIZE_CLASS = {
  sm: 'q-checkbox-sm',
  md: 'q-checkbox-md',
  lg: 'q-checkbox-lg',
} satisfies Record<CheckboxSize, string>

const LABEL_DIRECTION_CLASS = {
  left: 'q-checkbox-label-left',
  right: 'q-checkbox-label-right',
} satisfies Record<CheckboxLabelDirection, string>

const LABEL_SIZE_CLASS = {
  sm: 'q-checkbox-label-sm',
  md: 'q-checkbox-label-md',
} satisfies Record<CheckboxLabelSize, string>

// Title composite per label size (mirrors q-checkbox-label-{sm,md} in checkbox.css).
// The md case still carries a line-height override via the q-checkbox-label-title class.
const LABEL_TITLE_VARIANT = {
  sm: 'label-sm-medium',
```

```
  md: 'label-md-medium',
} satisfies Record<CheckboxLabelSize, TypographyVariant>

export function checkbox(options: CheckboxOptions = {}, ...extra: ClassValue[]): string {
  const { color = 'brand', size = 'md' } = options
  return cx('q-checkbox', COLOR_CLASS[color], SIZE_CLASS[size], ...extra)
}

export type CheckboxProps = ComponentProps<typeof Primitive.Root> & CheckboxOptions

export function Checkbox({ color, size, className, indeterminate, ...props }: CheckboxProps) {
  return (
    <Primitive.Root
      indeterminate={indeterminate}
      className={state =>
        checkbox(
          { color, size },
          typeof className === 'function' ? className(state) : className,
        )}
      {...props}
    >
      <span className="q-checkbox-box">
        <Primitive.Indicator className="q-checkbox-indicator">
          {indeterminate ? <MinusGlyph /> : <CheckGlyph />}
        </Primitive.Indicator>
      </span>
    </Primitive.Root>
  )
}

export interface CheckboxLabelProps extends Omit<ComponentProps<'label'>, 'color'> {
  label?: ReactNode
  description?: ReactNode
  direction?: CheckboxLabelDirection
  size?: CheckboxLabelSize
  color?: CheckboxColor
  checkboxSize?: CheckboxSize
  checkboxProps?: Omit<CheckboxProps, 'color' | 'size'>
}

export function CheckboxLabel({
  label = 'Label',
  description,
  direction = 'left',
  size = 'sm',
  color,
  checkboxSize = size === 'md' ? 'md' : 'sm',
  checkboxProps,
  className,
  children,
  ...props
}: CheckboxLabelProps) {
  const checkboxNode = (
    <Checkbox
      color={color}
      size={checkboxSize}
      {...checkboxProps}
    />
  )
  const textNode = (
    <span className="q-checkbox-label-text">
      <Typography
        as="span"
        variant={LABEL_TITLE_VARIANT[size]}
        color="primary"
        className="q-checkbox-label-title"
      >
        {children ?? label}
      </Typography>
      {description
```

```
        ? (
            <Typography
              as="span"
              variant="label-sm-regular"
              color="tertiary"
              className="q-checkbox-label-description"
            >
              {description}
            </Typography>
          )
        : null}
    </span>
  )

  return (
    <label
      className={cx(
        'q-checkbox-label',
        LABEL_DIRECTION_CLASS[direction],
        LABEL_SIZE_CLASS[size],
        className,
      )}
      {...props}
    >
      {direction === 'left' ? checkboxNode : textNode}
      {direction === 'left' ? textNode : checkboxNode}
    </label>
  )
}
```

app/packages/quanta/src/components/checkbox/index.ts

```
export { Checkbox, CheckboxLabel, checkbox } from './checkbox.tsx'
export type {
  CheckboxColor,
  CheckboxLabelDirection,
  CheckboxLabelProps,
  CheckboxLabelSize,
  CheckboxOptions,
  CheckboxProps,
  CheckboxSize,
} from './checkbox.tsx'
```

app/packages/quanta/src/components/chip/chip.css

```
/**
 * Chip presentation utilities (q-namespaced).
 *
 * Visual language is taken from the Checkbox (node 481:795):
 *   - SELECTED chip = solid fill of the matching checkbox checked state
 *     (brand → solid icon/accent #d1fe17 + icon/inverse #1a1a1a text/icon).
 *   - hover border, focus ring and disabled colors reuse the same tokens
 *     as the checkbox (lime-900 hover, transparent-lime-20 ring,
 *     background/secondary-strong #23262a + icon/disabled #484e56 disabled).
 *   - 1.5px border (border-width/medium), pill radius.
 *
 * Sizes: xxs=20 xs=24 sm=28 md=32 (heights → --hf-space-500/600/700/800).
 */

@source "./chip.tsx";

@utility q-chip {
  --q-chip-bg: transparent;
  --q-chip-fg: var(--hf-color-text-secondary);
  --q-chip-border: var(--hf-color-border-default);
  --q-chip-hover-bg: var(--hf-color-overlay-hover);
  --q-chip-hover-fg: var(--hf-color-text-primary);
  --q-chip-hover-border: var(--hf-color-lime-900);
  --q-chip-selected-bg: var(--hf-color-icon-accent);
  --q-chip-selected-fg: var(--hf-color-icon-inverse);
  --q-chip-ring: var(--hf-color-transparent-lime-20);
  --q-chip-height: var(--hf-space-700);
  --q-chip-gap: var(--hf-space-100);
  --q-chip-padding-x: var(--hf-space-300);
  --q-chip-icon-size: var(--hf-space-400);

  align-items: center;
  background-color: var(--q-chip-bg);
  border: var(--hf-border-width-medium) solid var(--q-chip-border);
  border-radius: var(--hf-space-2400);
  box-sizing: border-box;
  color: var(--q-chip-fg);
  cursor: pointer;
  display: inline-flex;
  flex-shrink: 0;
  gap: var(--q-chip-gap);
  height: var(--q-chip-height);
  justify-content: center;
  max-width: 100%;
  min-width: var(--q-chip-height);
  outline: none;
  overflow: hidden;
  padding: var(--hf-space-0) var(--q-chip-padding-x);
  text-align: center;
  text-overflow: ellipsis;
  transition-property: background-color, border-color, box-shadow, color;
  user-select: none;
  white-space: nowrap;

  & svg {
    flex-shrink: 0;
    height: var(--q-chip-icon-size);
    width: var(--q-chip-icon-size);
  }
}

@utility q-chip-xxs {
  --q-chip-height: var(--hf-space-500);
  --q-chip-gap: var(--hf-space-050);
  --q-chip-padding-x: var(--hf-space-200);
  --q-chip-icon-size: var(--hf-space-300);
  @apply text-q-label-xs-medium;
}
```

```
@utility q-chip-xs {
  --q-chip-height: var(--hf-space-600);
  --q-chip-gap: var(--hf-space-100);
  --q-chip-padding-x: var(--hf-space-250);
  --q-chip-icon-size: var(--hf-space-300);
  @apply text-q-label-xs-medium;
}

@utility q-chip-sm {
  --q-chip-height: var(--hf-space-700);
  --q-chip-gap: var(--hf-space-100);
  --q-chip-padding-x: var(--hf-space-300);
  --q-chip-icon-size: var(--hf-space-400);
  @apply text-q-label-sm-medium;
}

@utility q-chip-md {
  --q-chip-height: var(--hf-space-800);
  --q-chip-gap: var(--hf-space-150);
  --q-chip-padding-x: var(--hf-space-400);
  --q-chip-icon-size: var(--hf-space-500);
  @apply text-q-label-md-medium;
}

@utility q-chip-brand {
  --q-chip-hover-border: var(--hf-color-lime-900);
  --q-chip-selected-bg: var(--hf-color-icon-accent);
  --q-chip-selected-fg: var(--hf-color-icon-inverse);
  --q-chip-ring: var(--hf-color-transparent-lime-20);
}

@utility q-chip-neutral {
  --q-chip-hover-border: var(--hf-color-border-strong);
  --q-chip-selected-bg: var(--hf-color-background-secondary-strong);
  --q-chip-selected-fg: var(--hf-color-text-primary);
  --q-chip-ring: var(--hf-color-transparent-dark-20);
}

@utility q-chip-success {
  --q-chip-hover-border: var(--hf-color-state-success-fg);
  --q-chip-selected-bg: var(--hf-color-state-success-fg);
  --q-chip-selected-fg: var(--hf-color-text-inverse);
  --q-chip-ring: var(--hf-color-state-success-glow);
}

@utility q-chip-error {
  --q-chip-hover-border: var(--hf-color-state-error-fg);
  --q-chip-selected-bg: var(--hf-color-state-error-fg);
  --q-chip-selected-fg: var(--hf-color-text-inverse);
  --q-chip-ring: var(--hf-color-state-error-glow);
}

@utility q-chip-warning {
  --q-chip-hover-border: var(--hf-color-state-warning-fg);
  --q-chip-selected-bg: var(--hf-color-state-warning-fg);
  --q-chip-selected-fg: var(--hf-color-text-inverse);
  --q-chip-ring: var(--hf-color-state-warning-glow);
}

@utility q-chip-info {
  --q-chip-hover-border: var(--hf-color-state-info-fg);
  --q-chip-selected-bg: var(--hf-color-state-info-fg);
  --q-chip-selected-fg: var(--hf-color-text-inverse);
  --q-chip-ring: var(--hf-color-state-info-bg);
}

.q-chip:not([data-selected]):not(:disabled):hover {
  background-color: var(--q-chip-hover-bg);
  border-color: var(--q-chip-hover-border);
  color: var(--q-chip-hover-fg);
```

```
}

.q-chip[data-selected],
.q-chip-selected {
  background-color: var(--q-chip-selected-bg);
  border-color: var(--q-chip-selected-bg);
  color: var(--q-chip-selected-fg);
}

.q-chip:focus-visible {
  box-shadow:
    0 0 0 var(--hf-border-width-thick) var(--hf-color-background-primary),
    0 0 0 var(--hf-space-100) var(--q-chip-ring);
}

.q-chip:disabled {
  background-color: transparent;
  border-color: var(--hf-color-border-subtle);
  color: var(--hf-color-text-disabled);
  cursor: not-allowed;
}

.q-chip:disabled[data-selected],
.q-chip:disabled.q-chip-selected {
  background-color: var(--hf-color-background-secondary-strong);
  border-color: var(--hf-color-background-secondary-strong);
  color: var(--hf-color-icon-disabled);
}
```

app/packages/quanta/src/components/chip/chip.test.tsx

```
import { render, screen } from '@testing-library/react'
import { IconCircleOutlined as CircleIcon } from '@higgsfield-ai/icons/IconCircleOutlined'
import { describe, expect, it } from 'vitest'
import { Chip, chip } from './index.ts'

describe('chip() class-builder', () => {
  it('defaults to brand + sm', () => {
    expect(chip()).toBe('q-chip q-chip-brand q-chip-sm')
  })

  it('applies color + size + selected', () => {
    expect(chip({ color: 'success', size: 'md', selected: true })).toBe(
      'q-chip q-chip-success q-chip-md q-chip-selected',
    )
  })

  it('merges extra classes and drops falsy values', () => {
    expect(chip({ color: 'error' }, 'is-custom', false)).toBe('q-chip q-chip-error q-chip-sm is-custom')
  })
})

describe('<Chip>', () => {
  it('renders a button with default type and classes', () => {
    render(<Chip>Filter</Chip>)

    const chipControl = screen.getByRole('button', { name: 'Filter' })
    expect(chipControl).toHaveAttribute('type', 'button')
    expect(chipControl).toHaveAttribute('aria-pressed', 'false')
    expect(chipControl).toHaveClass('q-chip', 'q-chip-brand', 'q-chip-sm')
  })

  it('renders selected state', () => {
    render(<Chip color="neutral" size="md" selected>Filter</Chip>)

    const chipControl = screen.getByRole('button', { name: 'Filter' })
    expect(chipControl).toHaveAttribute('aria-pressed', 'true')
    expect(chipControl).toHaveAttribute('data-selected')
    expect(chipControl).toHaveClass('q-chip-neutral', 'q-chip-md', 'q-chip-selected')
  })

  it('forwards native props and caller classes', () => {
    render(<Chip className="is-custom" disabled type="submit">Filter</Chip>)

    const chipControl = screen.getByRole('button', { name: 'Filter' })
    expect(chipControl).toHaveClass('q-chip', 'is-custom')
    expect(chipControl).toBeDisabled()
    expect(chipControl).toHaveAttribute('type', 'submit')
  })

  it('composes start / end slots around the label', () => {
    render(<Chip start={<CircleIcon data-testid="lead" />} end={<span data-testid="count">3</span>}>Tags</Chip>)
    const chipControl = screen.getByRole('button', { name: /tags/i })
    expect(chipControl).toHaveTextContent('Tags')
    expect(screen.getByTestId('lead')).toBeInTheDocument()
    expect(screen.getByTestId('count')).toBeInTheDocument()
  })

  it('renders bare children when no slot is passed (back-compat)', () => {
    render(<Chip><CircleIcon data-testid="icon" />Label</Chip>)
    expect(screen.getByTestId('icon')).toBeInTheDocument()
    expect(screen.getByRole('button')).toHaveTextContent('Label')
  })
})
```

app/packages/quanta/src/components/chip/chip.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'
import type { SlotColor } from '../utils/slot.ts'

/**
 * Chip — a selectable filter pill (a real toggle <button> with `aria-pressed`).
 * When `selected`, the chip takes the SOLID fill of the matching checkbox
 * (e.g. brand → solid #d1fe17 surface with #1a1a1a text). For a removable
 * labeled category use Tag; for a standalone two-state control use Toggle.
 *
 * Composable, like Button: the label is `children`; `start` / `end` are optional
 * slots (any node — a leading icon, a trailing count `<Badge>`) that default to
 * nothing. The gap + `& svg` sizing space them; the legacy icon-as-children
 * pattern (`<Chip><Icon/>Label</Chip>`) is byte-for-byte unchanged when no slot
 * is passed.
 */

export type ChipProps = Omit<ComponentProps<'button'>, 'color'> & {
  /** Semantic color when selected. Default 'brand'. */
  color?: ChipColor
  size?: ChipSize
  selected?: boolean
  /** Leading slot (icon, any node) before the label. */
  start?: ReactNode
  /** Trailing slot (count, badge, any node) after the label. */
  end?: ReactNode
}

export type ChipColor = SlotColor
export type ChipSize = 'xxs' | 'xs' | 'sm' | 'md'

export interface ChipOptions {
  color?: ChipColor
  size?: ChipSize
  selected?: boolean
}

const COLOR_CLASS = {
  brand: 'q-chip-brand',
  neutral: 'q-chip-neutral',
  success: 'q-chip-success',
  error: 'q-chip-error',
  warning: 'q-chip-warning',
  info: 'q-chip-info',
} satisfies Record<ChipColor, string>

const SIZE_CLASS = {
  xxs: 'q-chip-xxs',
  xs: 'q-chip-xs',
  sm: 'q-chip-sm',
  md: 'q-chip-md',
} satisfies Record<ChipSize, string>

export function chip(options: ChipOptions = {}, ...extra: ClassValue[]): string {
  const { color = 'brand', size = 'sm', selected = false } = options
  return cx('q-chip', COLOR_CLASS[color], SIZE_CLASS[size], selected && 'q-chip-selected', ...extra)
}

export function Chip({ color, size, selected = false, className, type, start, end, children, ...props }: ChipProps) {
  // Slots flank the label only when set; otherwise children render bare so the
  // legacy icon-as-children pattern is unchanged. Gap + `& svg` do the spacing.
  const content = start != null || end != null ? <>{start}{children}{end}</> : children
  return (
    <button
      type={type ?? 'button'}
```

```
      aria-pressed={selected}
      data-selected={selected ? '' : undefined}
      className={chip({ color, size, selected }, className)}
      {...props}
    >
      {content}
    </button>
  )
}
```

app/packages/quanta/src/components/chip/index.ts

```
export { Chip, chip } from './chip.tsx'
export type { ChipColor, ChipOptions, ChipProps, ChipSize } from './chip.tsx'
```

app/packages/quanta/src/components/close-button/close-button.css

```
/**
 * CloseButton — round dismiss control, pixel-matched to the Figma "CloseButton"
 * (node 2052:109). Sizes 24/32/40/48 = padding space/100·150·200·300 with the
 * cross glyph sized by <Icon> (16/20/24/24), radius/full. States:
 *   • default — transparent/dark/05 (white 5%) fill, icon/secondary
 *   • hover   — + overlay/hover (composites to ≈ white 10%), icon/primary
 *   • focus   — hover fill + 2px border/focus (lime) ring, icon/primary
 *
 * Geometry is driven by CSS vars set per size; the surface/states are tokens-only.
 */

@source "./close-button.tsx";

@utility q-close {
  /* md (32) defaults — overridden by the size utilities below. */
  --q-close-pad: var(--hf-space-150); /* 6px */

  appearance: none;
  background-color: var(--hf-color-overlay-dim-soft); /* transparent/dark/05 = white 5% in dark theme */
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--hf-radius-full);
  box-sizing: border-box;
  align-items: center;
  color: var(--hf-color-icon-secondary); /* Figma default state: icon/secondary (brightens to icon/primary on hover/focus) */
  cursor: pointer;
  display: inline-flex;
  flex: none;
  justify-content: center;
  padding: var(--q-close-pad);
  outline: none;
  line-height: 0;
  transition-property: background-color, background-image, color, box-shadow;
  transition-duration: var(--hf-duration-150);
  transition-timing-function: var(--hf-ease-out); /* cubic-bezier(0, 0, 0.2, 1) */
  vertical-align: top;

  /* hover = base fill + overlay/hover layer (composites to ≈ white 10%), theme-safe. */
  &:hover {
    background-image: linear-gradient(var(--hf-color-overlay-hover), var(--hf-color-overlay-hover));
    color: var(--hf-color-icon-primary);
  }

  /* focus = "hover + FocusStrong ring": keep the hover overlay fill and
   * add the border/focus ring drawn inside the fixed 24/32/40/48 frame. */
  &:focus-visible {
    background-image: linear-gradient(var(--hf-color-overlay-hover), var(--hf-color-overlay-hover));
    color: var(--hf-color-icon-primary);
    box-shadow: inset 0 0 0 var(--hf-border-width-thick) var(--hf-color-border-focus);
  }

  &:disabled,
  &[data-disabled] {
    cursor: not-allowed;
    opacity: var(--hf-opacity-disabled);
  }
}

/* Sizes — Figma 24 / 32 / 40 / 48. Icon glyph size is owned by <Icon> (tsx). */
@utility q-close-sm {
  --q-close-pad: var(--hf-space-100); /* 4px  → 24 (icon 16) */
}
@utility q-close-md {
  --q-close-pad: var(--hf-space-150); /* 6px  → 32 (icon 20) */
}
@utility q-close-lg {
  --q-close-pad: var(--hf-space-200); /* 8px  → 40 (icon 24) */
}
@utility q-close-xl {
```

```
  --q-close-pad: var(--hf-space-300); /* 12px → 48 (icon 24) */
}

@media (prefers-reduced-motion: reduce) {
  .q-close { transition-duration: 1ms; /* no token: reduced-motion near-zero; --hf-duration-instant is 0ms */ }
}
```

app/packages/quanta/src/components/close-button/close-button.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { CloseButton, closeButton } from './index.ts'

describe('<CloseButton>', () => {
  it('renders a button with the default cross glyph (painted by <Icon>) and accessible name', () => {
    render(<CloseButton />)
    const btn = screen.getByRole('button', { name: 'Close' })
    expect(btn).toHaveClass('q-close', 'q-close-md')
    // Icon is node-only: q-icon lands directly on the glyph <svg>, sized to the md disc.
    const glyph = btn.querySelector('svg.q-icon')
    expect(glyph).toHaveClass('q-icon-md')
    expect(btn).toHaveAttribute('type', 'button')
  })

  it('applies the requested size class and matching icon size', () => {
    render(<CloseButton size="xl" />)
    const btn = screen.getByRole('button')
    expect(btn).toHaveClass('q-close-xl')
    // xl disc uses the 24px (lg) glyph.
    expect(btn.querySelector('svg.q-icon')).toHaveClass('q-icon-lg')
  })

  it('allows overriding the accessible label and children', () => {
    render(<CloseButton aria-label="Dismiss"><span data-testid="custom">x</span></CloseButton>)
    const btn = screen.getByRole('button', { name: 'Dismiss' })
    expect(screen.getByTestId('custom')).toBeInTheDocument()
    expect(btn.querySelector('svg')).not.toBeInTheDocument()
  })

  it('forwards className and native button props', () => {
    render(<CloseButton className="is-custom" disabled />)
    const btn = screen.getByRole('button')
    expect(btn).toHaveClass('is-custom')
    expect(btn).toBeDisabled()
  })

  it('closeButton() recipe builds the q-close class string for non-button elements', () => {
    expect(closeButton()).toBe('q-close q-close-md')
    expect(closeButton({ size: 'sm' }, 'extra')).toBe('q-close q-close-sm extra')
  })
})
```

app/packages/quanta/src/components/close-button/close-button.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { IconCrossMediumOutlined as CloseIcon } from '@higgsfield-ai/icons/IconCrossMediumOutlined'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'
import { Icon, type IconSize } from '../icon/index.ts'

/**
 * CloseButton — the round dismiss control for modals, dialogs, sheets and any
 * overlay, pixel-matched to the Figma "CloseButton" (node 2052:109). A faint
 * disc with a cross glyph: white-5% fill + primary icon at rest, a stronger
 * fill on hover, and a lime focus ring on keyboard focus.
 *
 *   <CloseButton onClick={close} />          // standalone <button>
 *   <Dialog.Close className={closeButton()}/> // style a framework close part
 *
 * Sizes map to the Figma 24 / 32 / 40 / 48 discs (`sm` / `md` / `lg` / `xl`).
 */

export type CloseButtonSize = 'sm' | 'md' | 'lg' | 'xl'

const SIZE_CLASS = {
  sm: 'q-close-sm',
  md: 'q-close-md',
  lg: 'q-close-lg',
  xl: 'q-close-xl',
} satisfies Record<CloseButtonSize, string>

// Disc size → cross-glyph Icon size, matching the Figma 16/20/24/24 glyphs that
// the `--q-close-icon` CSS var previously drove (now owned by <Icon>).
const ICON_SIZE = {
  sm: 'sm',
  md: 'md',
  lg: 'lg',
  xl: 'lg',
} satisfies Record<CloseButtonSize, IconSize>

export interface CloseButtonOptions {
  /** Disc size — Figma 24/32/40/48. Defaults to `md` (32). */
  size?: CloseButtonSize
}

/**
 * Build the close-button class string. Use this to apply the styling to a
 * non-`<button>` close element, e.g. a Base UI `Dialog.Close` / `Toast.Close`.
 */
export function closeButton({ size = 'md' }: CloseButtonOptions = {}, ...extra: ClassValue[]): string {
  return cx('q-close', SIZE_CLASS[size], ...extra)
}

export { CloseIcon }

export type CloseButtonProps = Omit<ComponentProps<'button'>, 'children'> & {
  size?: CloseButtonSize
  /** Override the default cross glyph. */
  children?: ReactNode
}

export function CloseButton({ size = 'md', className, type, children, 'aria-label': ariaLabel, ...props }: CloseButtonProps) {
  return (
    <button
      type={type ?? 'button'}
      aria-label={ariaLabel ?? 'Close'}
      className={closeButton({ size }, className)}
      {...props}
    >
      {children ?? <Icon as={CloseIcon} size={ICON_SIZE[size]} />}
```

```
      </button>
    )
  }
```

app/packages/quanta/src/components/close-button/index.ts

```
export { CloseButton, CloseIcon, closeButton } from './close-button.tsx'
export type { CloseButtonOptions, CloseButtonProps, CloseButtonSize } from './close-button.tsx'
```

app/packages/quanta/src/components/cmdk/cmdk.css

```
/**
 * Command (cmdk) presentation — a hand-rolled command palette skinned with
 * quanta tokens. Dialog usage composes the shared Modal component, so command
 * only owns the searchable content layout and item states here.
 */
@source "./cmdk.tsx";

/* Inline root (fills its parent; dialog shell/size comes from Modal.Content). */
@utility q-command {
  display: flex;
  flex: 1 1 auto;
  width: 100%;
  height: 100%;
  min-height: var(--hf-space-0);
  min-width: var(--hf-space-0);
  flex-direction: column;
}

@utility q-command-dialog-context {
  display: contents;
}

/* Search row. */
@utility q-command-input-row {
  flex: none;
  display: flex;
  align-items: center;
  width: 100%;
  height: var(--hf-space-1400);
  padding: var(--hf-space-0) var(--hf-space-400);
  border-bottom: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
}

.q-modal-header .q-command-input-row {
  flex: 1 1 var(--hf-space-0);
  height: var(--hf-space-1000);
  min-width: var(--hf-space-0);
  padding: var(--hf-space-0);
  border-bottom: var(--hf-border-width-none) solid transparent;
}

/* Scroll region. */
@utility q-command-list {
  flex: 1 1 auto;
  min-height: var(--hf-space-0);
  overflow-y: auto;
  overscroll-behavior: contain;
  padding: var(--hf-space-200);
  scroll-padding-block: var(--hf-space-200);
}

@utility q-command-empty {
  padding: var(--hf-space-800) var(--hf-space-400);
  text-align: center;
  color: var(--hf-color-text-secondary);
  @apply text-q-body-sm-regular;
}

@utility q-command-loading {
  padding: var(--hf-space-600) var(--hf-space-400);
  text-align: center;
  color: var(--hf-color-text-secondary);
  @apply text-q-body-sm-regular;
}

@utility q-command-group {
  padding-bottom: var(--hf-space-100);
}
```

```
/* Auto divider between consecutive VISIBLE groups: a visible group that follows
 * another visible group gets a top divider. Hidden (empty) groups are skipped by
 * the `:not([hidden])` match, so filtering never leaves orphan/double dividers
 * and no manual <Command.Separator> is needed between groups. */
.q-command-group:not([hidden]) ~ .q-command-group:not([hidden]) {
  /* Etched 2-line groove, matching the Divider component: a dark line
   * (transparent-dark-15) over a light line (transparent-light-10). */
  border-top: var(--hf-border-width-thin) solid var(--hf-color-transparent-dark-15);
  box-shadow: inset 0 var(--hf-border-width-thin) 0 var(--hf-color-transparent-light-10);
  margin-top: var(--hf-space-200);
  padding-top: var(--hf-space-200);
}
@utility q-command-group-heading {
  /* Equal top/bottom padding so the label sits centered in its band. */
  padding: var(--hf-space-200) var(--hf-space-300);
  color: var(--hf-color-text-on-overlay-secondary); /* Dropdown q-menu-group-label */
  @apply text-q-label-xs-medium;
}

/* A command row. */
@utility q-command-item {
  display: flex;
  align-items: center;
  gap: var(--hf-space-200); /* 8px — Dropdown q-menu-item */
  min-height: calc(var(--hf-space-800) + var(--hf-space-100)); /* 36px — Dropdown q-menu-item */
  padding: var(--hf-space-200); /* 8px uniform — Dropdown q-menu-item */
  border-radius: var(--hf-radius-200);
  color: var(--hf-color-text-primary);
  cursor: pointer;
  user-select: none;
  @apply text-q-body-sm-medium;

  &[data-active] {
    background-color: var(--hf-color-overlay-hover);
  }
  &[aria-disabled="true"] {
    color: var(--hf-color-text-on-overlay-tertiary);
    cursor: not-allowed;
  }
}
@utility q-command-item-icon {
  flex: none;
  display: flex;
  align-items: center;
  justify-content: center;
  color: var(--hf-color-icon-primary); /* Dropdown q-menu-item-icon */
}
@utility q-command-item-content {
  display: flex;
  flex: 1 1 auto;
  min-width: var(--hf-space-0);
  flex-direction: column;
}
@utility q-command-item-title {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
/* A bare ItemTitle (no ItemContent wrapper) grows + truncates within the row;
 * inside ItemContent it stays a plain block that fills the column. */
.q-command-item > .q-command-item-title {
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
}
@utility q-command-item-description {
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
  color: var(--hf-color-text-secondary);
  @apply text-q-caption-sm-regular;
```

```
}
@utility q-command-item-trailing {
  flex: none;
  display: flex;
  align-items: center;
  margin-inline-start: auto; /* push to the trailing edge (Dropdown q-menu-item-trailing) */
  color: var(--hf-color-text-tertiary);
  @apply text-q-caption-sm-regular;
}

/* Spacing wrapper around the Divider component (Command.Separator). */
@utility q-command-separator {
  margin-block: var(--hf-space-200);
}

/* `Command.Shortcut` renders the canonical `Kbd` component (no q-command-shortcut
 * utility — the reinvented keycap was removed in favour of reusing Kbd). */

/* ■■ Optional two-pane layout: list sidebar + detail panel + footer ■■■■■■■■■■■■■■ */

/* Row region between Input and Footer. */
@utility q-command-body {
  display: flex;
  flex: 1 1 auto;
  gap: var(--hf-space-200);
  min-height: var(--hf-space-0);
}
/* Inside the two-pane body the list becomes a sized sidebar pane. */
.q-command-body > .q-command-list {
  flex: 1 1 0;
  min-width: var(--hf-space-0);
}

/* Right detail pane. Command.Detail renders this on a Modal.Workspace (frosted
 * surface + flex sizing + stretch come from `q-modal-workspace`); this only adds
 * the positioning context for the scroll layer. The content lives in an absolute
 * layer so the pane contributes NO height to the row — the LIST pane sizes the
 * palette and this fills that height + scrolls. Showing/hiding it therefore never
 * resizes the modal (no hover taper/loop): the list rules the height, never this. */
@utility q-command-detail {
  position: relative;
}
@utility q-command-detail-scroll {
  position: absolute;
  inset: var(--hf-space-0);
  overflow-y: auto;
  overscroll-behavior: contain;
  padding: var(--hf-space-300);
}

/* Bottom bar. */
@utility q-command-footer {
  flex: none;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--hf-space-300);
  padding: var(--hf-space-300) var(--hf-space-400);
  border-top: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  color: var(--hf-color-text-secondary);
}

@utility q-command-footer-caption {
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
}

@utility q-command-footer-actions {
  display: inline-flex;
  flex: none;
```

```
  align-items: center;
  gap: var(--hf-space-150);
}

/* Footer confirm action (Enter equivalent). */
@utility q-command-action {
  display: inline-flex;
  align-items: center;
  gap: var(--hf-space-200);
  padding: var(--hf-space-100) var(--hf-space-200);
  border-radius: var(--hf-radius-200);
  color: var(--hf-color-text-primary);
  cursor: pointer;
  /* `ease` is a standard CSS keyword timing-function, not a tokenized cubic-bezier. */
  transition: background-color var(--hf-duration-150) ease, opacity var(--hf-duration-150) ease;
  @apply text-q-body-sm-medium;

  &:hover { background-color: var(--hf-color-overlay-hover); }
  &:disabled { opacity: var(--hf-opacity-muted); cursor: not-allowed; }
}

@media (prefers-reduced-motion: reduce) {
  .q-command-action {
    /* no token: 1ms reduced-motion near-disable has no duration token (smallest is 0ms instant) */
    transition-duration: 1ms;
  }
}
```

app/packages/quanta/src/components/cmdk/cmdk.test.tsx

```
import { createRef } from 'react'
import { render, screen, waitFor } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { describe, expect, it, vi } from 'vitest'
import { Modal } from '../modal/index.ts'
import type { CommandFilter } from './index.ts'
import { Command } from './index.ts'

function setup(onSelect = vi.fn()) {
  render(
    <Command label="Test menu">
      <Command.Input placeholder="Search" />
      <Command.List>
        <Command.Empty>Nothing found.</Command.Empty>
        <Command.Group heading="Files">
          <Command.Item onSelect={() => onSelect('new')}>
            <Command.ItemContent>
              <Command.ItemTitle>New File</Command.ItemTitle>
              <Command.ItemDescription>Create a blank file</Command.ItemDescription>
            </Command.ItemContent>
          </Command.Item>
          <Command.Item onSelect={() => onSelect('open')}><Command.ItemTitle>Open File</Command.ItemTitle></Command.Item>
        </Command.Group>
        <Command.Group heading="Settings">
          <Command.Item onSelect={() => onSelect('theme')}>
            <Command.ItemTitle>Toggle Theme</Command.ItemTitle>
            <Command.ItemTrailing>■T</Command.ItemTrailing>
          </Command.Item>
        </Command.Group>
      </Command.List>
    </Command>,
  )
  return { onSelect }
}

const item = (label: string) => screen.getByText(label).closest('[data-command-item]')!

describe('<Command>', () => {
  it('renders a combobox and every item', () => {
    setup()
    expect(screen.getByRole('combobox')).toBeInTheDocument()
    expect(screen.getByText('New File')).toBeVisible()
    expect(screen.getByText('Toggle Theme')).toBeVisible()
  })

  it('Command.Input composes the shared Input component', () => {
    setup()
    const input = screen.getByRole('combobox')
    expect(input).toHaveClass('q-field-input')
    expect(input.closest('.q-field')).toBeInTheDocument()
    expect(input.closest('.q-field-control')).toBeInTheDocument()
    expect(document.querySelector('.q-command-input')).toBeNull()
  })

  it('Command.Input renders its default search glyph via the Icon component', () => {
    setup()
    const input = screen.getByRole('combobox')
    // The default leading glyph is wrapped in <Icon size="md"> (q-icon-md = 20px),
    // and sits in the Input affix slot.
    const affix = input.closest('.q-field-control')!.querySelector('.q-field-affix')!
    const glyph = affix.querySelector('.q-icon')!
    expect(glyph).toHaveClass('q-icon', 'q-icon-md')
    // NODE-ONLY: the .q-icon class is painted directly on the glyph <svg> — no wrapper.
    expect(glyph.tagName.toLowerCase()).toBe('svg')
  })

  it('forwards a caller ref to the listbox node without dropping the internal ref', () => {
    const ref = createRef<HTMLDivElement>()
```

```
  render(
    <Command label="Ref">
      <Command.Input />
      <Command.List ref={ref}>
        <Command.Item><Command.ItemTitle>One</Command.ItemTitle></Command.Item>
      </Command.List>
    </Command>,
  )
  // Caller ref lands on the listbox root, and internal nav still works
  // (the internal listRef is merged, not overwritten).
  expect(ref.current).toBe(screen.getByRole('listbox'))
  expect(ref.current).toHaveClass('q-command-list')
})

it('fuzzy-filters items and hides groups with no matches', async () => {
  const user = userEvent.setup()
  setup()
  await user.type(screen.getByRole('combobox'), 'theme')
  expect(screen.getByText('Toggle Theme')).toBeVisible()
  expect(item('New File')).not.toBeVisible()
  expect(screen.getByText('Files').closest('[role="group"]')).not.toBeVisible()
})

it('shows the empty state when nothing matches', async () => {
  const user = userEvent.setup()
  setup()
  await user.type(screen.getByRole('combobox'), 'zzzzz')
  expect(screen.getByText('Nothing found.')).toBeInTheDocument()
})

it('suppresses the empty state while loading', () => {
  const Palette = ({ loading }: { loading: boolean }) => (
    <Command label="L" loading={loading} defaultValue="zzz">
      <Command.Input />
      <Command.List>
        <Command.Empty>Nothing found.</Command.Empty>
        <Command.Item><Command.ItemTitle>Alpha</Command.ItemTitle></Command.Item>
      </Command.List>
    </Command>
  )
  const { rerender } = render(<Palette loading />)
  expect(screen.queryByText('Nothing found.')).not.toBeInTheDocument()
  rerender(<Palette loading={false} />)
  expect(screen.getByText('Nothing found.')).toBeInTheDocument()
})

it('uses a custom filter when provided', async () => {
  const user = userEvent.setup()
  // Prefix match — stricter than the default fuzzy/substring scorer.
  const filter: CommandFilter = (value, search) => (value.toLowerCase().startsWith(search.toLowerCase()) ? 1 : 0)
  render(
    <Command label="F" filter={filter}>
      <Command.Input />
      <Command.List>
        <Command.Item><Command.ItemTitle>Apple</Command.ItemTitle></Command.Item>
        <Command.Item><Command.ItemTitle>Grape</Command.ItemTitle></Command.Item>
      </Command.List>
    </Command>,
  )
  await user.type(screen.getByRole('combobox'), 'ap')
  expect(screen.getByText('Apple')).toBeVisible() // starts with "ap"
  expect(item('Grape')).not.toBeVisible() // default fuzzy matches "ap" in "grApe"; prefix filter does not
})

it('highlights the first item and moves with ArrowDown', async () => {
  const user = userEvent.setup()
  setup()
  expect(item('New File')).toHaveAttribute('data-active')
  screen.getByRole('combobox').focus()
  await user.keyboard('{ArrowDown}')
```

```
  expect(item('Open File')).toHaveAttribute('data-active')
  expect(item('New File')).not.toHaveAttribute('data-active')
})

it('navigates with Ctrl+n / Ctrl+p (vim bindings)', async () => {
  const user = userEvent.setup()
  setup()
  screen.getByRole('combobox').focus()
  await user.keyboard('{Control>}n{/Control}')
  expect(item('Open File')).toHaveAttribute('data-active')
  await user.keyboard('{Control>}p{/Control}')
  expect(item('New File')).toHaveAttribute('data-active')
})

it('wraps with loop (default) and stops at the ends when loop is false', async () => {
  const user = userEvent.setup()
  const Palette = ({ loop }: { loop?: boolean }) => (
    <Command label="Loop" loop={loop}>
      <Command.Input />
      <Command.List>
        <Command.Item><Command.ItemTitle>One</Command.ItemTitle></Command.Item>
        <Command.Item><Command.ItemTitle>Two</Command.ItemTitle></Command.Item>
      </Command.List>
    </Command>
  )
  const { rerender } = render(<Palette />)
  screen.getByRole('combobox').focus()
  await user.keyboard('{ArrowUp}') // from first → wraps to last
  expect(item('Two')).toHaveAttribute('data-active')

  rerender(<Palette loop={false} />)
  await user.keyboard('{ArrowDown}') // at last, no loop → stays
  expect(item('Two')).toHaveAttribute('data-active')
})

it('composes items from icon / content / title / description / trailing parts', () => {
  render(
    <Command label="Slots">
      <Command.Input />
      <Command.List>
        <Command.Item>
          <Command.ItemIcon><span data-testid="ico" /></Command.ItemIcon>
          <Command.ItemContent>
            <Command.ItemTitle>Deploy</Command.ItemTitle>
            <Command.ItemDescription>ship it</Command.ItemDescription>
          </Command.ItemContent>
          <Command.ItemTrailing>■D</Command.ItemTrailing>
        </Command.Item>
      </Command.List>
    </Command>,
  )
  const row = screen.getByText('Deploy').closest('[data-command-item]')!
  expect(screen.getByText('Deploy')).toHaveClass('q-command-item-title')
  expect(screen.getByText('ship it')).toHaveClass('q-command-item-description')
  expect(screen.getByText('■D')).toHaveClass('q-command-item-trailing')
  expect(row.querySelector('.q-command-item-icon')).toContainElement(screen.getByTestId('ico'))
  expect(row.querySelector('.q-command-item-content')).toContainElement(screen.getByText('Deploy'))
})

it('detail pane reflects the active item and updates on navigation', async () => {
  const user = userEvent.setup()
  render(
    <Command label="Detail">
      <Command.Input />
      <Command.Body>
        <Command.List>
          <Command.Item detail={<p>Alpha details</p>}><Command.ItemTitle>Alpha</Command.ItemTitle></Command.Item>
          <Command.Item detail={<p>Beta details</p>}><Command.ItemTitle>Beta</Command.ItemTitle></Command.Item>
        </Command.List>
        <Command.Detail />
```

```
      </Command.Body>
    </Command>,
  )
  expect(await screen.findByText('Alpha details')).toBeInTheDocument()
  screen.getByRole('combobox').focus()
  await user.keyboard('{ArrowDown}')
  expect(await screen.findByText('Beta details')).toBeInTheDocument()
  expect(screen.queryByText('Alpha details')).not.toBeInTheDocument()
})

it('renders the two-pane layout: a Workspace list + the self-wrapping Detail pane', async () => {
  render(
    <Command label="Workspace detail">
      <Command.Body>
        <Modal.Workspace padded={false}>
          <Command.Input />
          <Command.List>
            <Command.Item detail={<p>Alpha details</p>}><Command.ItemTitle>Alpha</Command.ItemTitle></Command.Item>
            <Command.Item detail={<p>Beta details</p>}><Command.ItemTitle>Beta</Command.ItemTitle></Command.Item>
          </Command.List>
        </Modal.Workspace>
        {/* Command.Detail self-wraps in a Modal.Workspace — no manual wrapper. */}
        <Command.Detail />
      </Command.Body>
    </Command>,
  )

  // The list Workspace + the Detail's own Workspace = two frosted panes.
  expect(document.querySelectorAll('.q-modal-workspace')).toHaveLength(2)
  expect(screen.getByText('Alpha details').closest('.q-command-detail')).toHaveClass('q-modal-workspace')
  expect(await screen.findByText('Alpha details')).toBeInTheDocument()
})

it('hides the detail pane when the active item has no detail', async () => {
  const user = userEvent.setup()
  const { container } = render(
    <Command label="Mixed">
      <Command.Input />
      <Command.Body>
        <Command.List>
          <Command.Item detail={<p>Has detail</p>}><Command.ItemTitle>WithDetail</Command.ItemTitle></Command.Item>
          <Command.Item><Command.ItemTitle>Plain</Command.ItemTitle></Command.Item>
        </Command.List>
        <Command.Detail />
      </Command.Body>
    </Command>,
  )
  expect(await screen.findByText('Has detail')).toBeInTheDocument()
  expect(container.querySelector('.q-command-detail')).toBeInTheDocument()
  screen.getByRole('combobox').focus()
  await user.keyboard('{ArrowDown}') // → "Plain" (no detail)
  await waitFor(() => expect(container.querySelector('.q-command-detail')).not.toBeInTheDocument())
})

it('footer action label tracks the active item', async () => {
  const user = userEvent.setup()
  render(
    <Command label="Footer label">
      <Command.Input />
      <Command.List>
        <Command.Item action="Run one"><Command.ItemTitle>One</Command.ItemTitle></Command.Item>
        <Command.Item action="Run two"><Command.ItemTitle>Two</Command.ItemTitle></Command.Item>
      </Command.List>
      <Command.Footer><Command.Action fallback="Pick"><span data-testid="kbd">↵</span></Command.Action></Command.Footer>
    </Command>,
  )
  expect(await screen.findByText('Run one')).toBeInTheDocument()
  screen.getByRole('combobox').focus()
  await user.keyboard('{ArrowDown}')
  expect(await screen.findByText('Run two')).toBeInTheDocument()
```

```
    expect(screen.queryByText('Run one')).not.toBeInTheDocument()
  })

  it('footer action runs the active item', async () => {
    const onSelect = vi.fn()
    const user = userEvent.setup()
    render(
      <Command label="Footer">
        <Command.Input />
        <Command.List><Command.Item onSelect={onSelect}><Command.ItemTitle>Run</Command.ItemTitle></Command.Item></Command.List>
        <Command.Footer><Command.Action>Go</Command.Action></Command.Footer>
      </Command>,
    )
    await user.click(screen.getByRole('button', { name: 'Go' }))
    expect(onSelect).toHaveBeenCalledTimes(1)
  })

  it('selects via Enter (active item) and via click', async () => {
    const user = userEvent.setup()
    const { onSelect } = setup()
    screen.getByRole('combobox').focus()
    await user.keyboard('{Enter}')
    expect(onSelect).toHaveBeenCalledWith('new')
    await user.click(screen.getByText('Toggle Theme'))
    expect(onSelect).toHaveBeenCalledWith('theme')
  })

  it('Command.Shortcut composes the canonical Kbd (renders a <kbd> pill)', () => {
    render(<Command.Shortcut>■K</Command.Shortcut>)
    const k = screen.getByText('■K')
    expect(k.tagName).toBe('KBD')
    // the Kbd pill styling (Figma _Shortcut), not the old q-command-shortcut
    expect(k).toHaveClass('rounded-q-100', 'bg-q-overlay-hover', 'text-q-caption-sm-medium')
  })

  it('Command.Dialog composes the shared Modal shell and size presets', () => {
    render(
      <Command.Dialog defaultOpen size="lg" label="Command menu">
        <Command.Input />
        <Command.List>
          <Command.Item><Command.ItemTitle>Open dashboard</Command.ItemTitle></Command.Item>
        </Command.List>
        <Command.Footer
          caption="Footer label"
          actions={<Command.Action fallback="Select"><span>↵</span></Command.Action>}
        />
      </Command.Dialog>,
    )

    const dialog = screen.getByRole('dialog', { name: 'Command menu' })
    const input = screen.getByRole('combobox')
    const footerAction = screen.getByRole('button', { name: /select/i })
    expect(dialog).toHaveClass('q-modal', 'q-modal-size-lg')
    expect(document.querySelector('.q-modal-body')).toBeInTheDocument()
    expect(document.querySelector('.q-modal-workspace')).toBeInTheDocument()
    expect(input.closest('.q-modal-header')).toBeInTheDocument()
    expect(input.closest('.q-modal-workspace')).toBeNull()
    expect(screen.getByRole('listbox').closest('.q-modal-workspace')).toBeInTheDocument()
    expect(screen.getByText('Footer label')).toHaveClass('q-modal-caption')
    expect(footerAction.closest('.q-modal-footer')).toBeInTheDocument()
    expect(footerAction.closest('.q-modal-workspace')).toBeNull()
  })
})
```

app/packages/quanta/src/components/cmdk/cmdk.tsx

```
'use client'

import type { ChangeEvent, ComponentProps, KeyboardEvent, ReactNode, Ref } from 'react'
import { IconMagnifyingGlass2Outlined as SearchIcon } from '@higgsfield-ai/icons/IconMagnifyingGlass2Outlined'
import {
  Children,
  createContext,
  isValidElement,
  useCallback,
  useContext,
  useEffect,
  useId,
  useLayoutEffect,
  useMemo,
  useRef,
  useState,
} from 'react'
import type { ModalSize } from '../modal/index.ts'
import { Divider } from '../divider/index.ts'
import { Icon } from '../icon/index.ts'
import type { InputProps } from '../input/index.ts'
import { Input } from '../input/index.ts'
import { Kbd } from '../kbd/index.ts'
import { Modal } from '../modal/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * Command (cmdk) — a fast, filterable command palette skinned with quanta
 * tokens. Hand-rolled (like the `cmdk` library) for an always-open, inline,
 * grouped, keyboard-driven list: fuzzy filtering, roving arrow-key navigation,
 * groups with headings, empty + loading states, per-item detail / action, and an
 * optional `■K` Dialog shell composed from Modal (Base UI Dialog owns the
 * overlay, focus trap and a11y through the shared modal).
 *
 * COMPOSITION-FIRST (same rules as Dropdown). `Command.Item` is a styled,
 * stateful row that renders whatever children you give it — it owns the
 * registry / filtering / highlight / selection behaviour, but makes NO
 * assumption about the shape of your data. Build any row out of the exported
 * parts (`ItemIcon` / `ItemContent` / `ItemTitle` / `ItemDescription` /
 * `ItemTrailing`), exactly like `Dropdown.Item`:
 *
 *   <Command.Dialog shortcut="mod+k" label="Command menu">
 *     <Command.Input placeholder="Type a command…" />
 *     <Command.List>
 *       <Command.Empty>No results.</Command.Empty>
 *       <Command.Group heading="Actions">
 *         <Command.Item onSelect={…}>
 *           <Command.ItemIcon><Icon><Plus/></Icon></Command.ItemIcon>
 *           <Command.ItemTitle>New file</Command.ItemTitle>
 *           <Command.ItemTrailing><Command.Shortcut>■N</Command.Shortcut></Command.ItemTrailing>
 *         </Command.Item>
 *       </Command.Group>
 *     </Command.List>
 *   </Command.Dialog>
 *
 * An Item's searchable text is its `value` prop, or — when omitted — the plain
 * text of its children; `keywords` adds extra search terms. `Command.Root`
 * accepts a custom `filter`, `loading` (suppresses Empty), and `loop` (default).
 */

/* ■■ Fuzzy match: substring (strong) or subsequence (weak); empty query = all. */
function score(query: string, text: string): number {
  if (!query) return 1
  const q = query.toLowerCase()
  const t = text.toLowerCase()
  if (t.includes(q)) return 2
  let qi = 0
  for (let ti = 0; ti < t.length && qi < q.length; ti++) {
```

```
    if (t[ti] === q[qi]) qi++
  }
  return qi === q.length ? 1 : 0
}

/** Best-effort plain text from an arbitrary node (for matching). */
function nodeText(node: ReactNode): string {
  if (node == null || node === false || node === true) return ''
  if (typeof node === 'string' || typeof node === 'number') return String(node)
  if (Array.isArray(node)) return node.map(nodeText).join(' ')
  if (isValidElement(node)) return nodeText((node.props as { children?: ReactNode }).children)
  return ''
}

/** Scores an item against the query. >0 keeps the item (and orders by strength). */
export type CommandFilter = (value: string, search: string, keywords: string) => number
const defaultFilter: CommandFilter = (value, search, keywords) => score(search, `${value} ${keywords}`.trim())

/** Populate every ref in `refs` with the same node (object or callback refs). */
function mergeRefs<T>(...refs: (Ref<T> | undefined)[]) {
  return (node: T | null) => {
    for (const ref of refs) {
      if (typeof ref === 'function')
        ref(node)
      else if (ref)
        (ref as { current: T | null }).current = node
    }
  }
}

interface ItemMeta { id: string, value: string, keywords: string, groupId: string | null, disabled: boolean }

interface CommandContextValue {
  query: string
  matched: Set<string>
  loading: boolean
  activeId: string | null
  setActiveId: (id: string | null) => void
  register: (meta: ItemMeta, onSelect?: () => void) => () => void
  select: (id: string) => void
  groupHasMatch: (groupId: string) => boolean
  listId: string
  /** Detail node of the currently-active item (for the optional detail pane). */
  activeDetail: ReactNode
  setActiveDetail: (node: ReactNode) => void
  /** Footer action label of the currently-active (hovered/highlighted) item. */
  activeAction: ReactNode
  setActiveAction: (node: ReactNode) => void
}

const CommandContext = createContext<CommandContextValue | null>(null)
function useCommand() {
  const ctx = useContext(CommandContext)
  if (!ctx) throw new Error('Command parts must be used within <Command>')
  return ctx
}

/* The group a nested Item belongs to (null at top level). */
const GroupContext = createContext<string | null>(null)

export interface CommandProps extends Omit<ComponentProps<'div'>, 'onSelect'> {
  /** Controlled search value. */
  value?: string
  defaultValue?: string
  onValueChange?: (value: string) => void
  /** Disable built-in filtering (caller filters items themselves). */
  shouldFilter?: boolean
  /** Custom scorer: `(value, search, keywords) => number`; >0 keeps the item. */
  filter?: CommandFilter
  /** Async state — keeps `Command.Empty` hidden while results are loading. */
```

```
  loading?: boolean
  /** Arrow-key navigation wraps past the ends (default true). */
  loop?: boolean
  /** Accessible label for the listbox. */
  label?: string
}

/** Root: owns search + the item registry + keyboard navigation. */
function CommandRoot({
  value,
  defaultValue = '',
  onValueChange,
  shouldFilter = true,
  filter,
  loading = false,
  loop = true,
  label = 'Command menu',
  className,
  children,
  ...props
}: CommandProps) {
  const [uncontrolled, setUncontrolled] = useState(defaultValue)
  const query = value ?? uncontrolled
  const setQuery = useCallback((q: string) => {
    if (value === undefined) setUncontrolled(q)
    onValueChange?.(q)
  }, [value, onValueChange])

  const listId = useId()
  const registry = useRef(new Map<string, ItemMeta>())
  const selects = useRef(new Map<string, () => void>())
  const [version, setVersion] = useState(0)
  const [activeId, setActiveId] = useState<string | null>(null)
  const [activeDetail, setActiveDetail] = useState<ReactNode>(null)
  const [activeAction, setActiveAction] = useState<ReactNode>(null)
  const listRef = useRef<HTMLDivElement>(null)

  const register = useCallback((meta: ItemMeta, onSelect?: () => void) => {
    registry.current.set(meta.id, meta)
    if (onSelect) selects.current.set(meta.id, onSelect)
    setVersion(v => v + 1)
    return () => {
      registry.current.delete(meta.id)
      selects.current.delete(meta.id)
      setVersion(v => v + 1)
    }
  }, [])

  const matched = useMemo(() => {
    const set = new Set<string>()
    const scorer = filter ?? defaultFilter
    for (const [id, meta] of registry.current) {
      if (!shouldFilter || scorer(meta.value, query, meta.keywords) > 0) set.add(id)
    }
    return set
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [query, version, shouldFilter, filter])

  const groupHasMatch = useCallback((groupId: string) => {
    for (const [id, meta] of registry.current) {
      if (meta.groupId === groupId && matched.has(id)) return true
    }
    return false
  }, [matched])

  const select = useCallback((id: string) => {
    if (registry.current.get(id)?.disabled) return
    selects.current.get(id)?.()
  }, [])
```

```
/** Visible item ids in DOM order (source of truth for nav). */
const visibleIds = useCallback(() => {
  const root = listRef.current
  if (!root) return [] as string[]
  return Array.from(root.querySelectorAll<HTMLElement>('[data-command-item]:not([hidden]):not([aria-disabled="true"])'))
    .map(el => el.id)
}, [])

// Reset / clamp the active item whenever the visible set changes.
useEffect(() => {
  const ids = visibleIds()
  if (ids.length === 0) { setActiveId(null); return }
  setActiveId(prev => (prev && ids.includes(prev) ? prev : ids[0]))
}, [matched, visibleIds])

// Keep the active item scrolled into view.
useEffect(() => {
  if (!activeId) return
  const el = listRef.current?.querySelector<HTMLElement>(`#${CSS.escape(activeId)}`)
  el?.scrollIntoView?.({ block: 'nearest' })
}, [activeId])

const onKeyDown = useCallback((e: KeyboardEvent<HTMLDivElement>) => {
  const ids = visibleIds()
  if (ids.length === 0) return
  const i = activeId ? ids.indexOf(activeId) : -1
  const goNext = () => {
    const next = i + 1
    setActiveId(ids[next >= ids.length ? (loop ? 0 : ids.length - 1) : next])
  }
  const goPrev = () => {
    const prev = i - 1
    setActiveId(ids[prev < 0 ? (loop ? ids.length - 1 : 0) : prev])
  }
  // Vim-style navigation: Ctrl+n / Ctrl+p (down / up).
  if (e.ctrlKey && (e.key === 'n' || e.key === 'p')) {
    e.preventDefault()
    e.key === 'n' ? goNext() : goPrev()
  }
  else if (e.key === 'ArrowDown') {
    e.preventDefault()
    goNext()
  }
  else if (e.key === 'ArrowUp') {
    e.preventDefault()
    goPrev()
  }
  else if (e.key === 'Home') {
    e.preventDefault()
    setActiveId(ids[0])
  }
  else if (e.key === 'End') {
    e.preventDefault()
    setActiveId(ids[ids.length - 1])
  }
  else if (e.key === 'Enter' && activeId) {
    e.preventDefault()
    select(activeId)
  }
}, [activeId, loop, select, visibleIds])

const ctx = useMemo<CommandContextValue>(() => ({
  query, matched, loading, activeId, setActiveId, register, select, groupHasMatch, listId,
  activeDetail, setActiveDetail, activeAction, setActiveAction,
}), [query, matched, loading, activeId, register, select, groupHasMatch, listId, activeDetail, activeAction])

return (
  <CommandContext.Provider value={ctx}>
    <div
      className={cx('q-command', className)}
```

```
        onKeyDown={onKeyDown}
        // eslint-disable-next-line jsx-a11y/no-noninteractive-element-interactions
        {...props}
      >
        <SearchSync query={query} setQuery={setQuery} listId={listId} label={label} listRef={listRef}>
          {children}
        </SearchSync>
      </div>
    </CommandContext.Provider>
  )
}

/* Carries search wiring down to Input/List via a tiny secondary context. */
interface SearchCtx {
  query: string
  setQuery: (q: string) => void
  listId: string
  label: string
  listRef: React.RefObject<HTMLDivElement | null>
}
const SearchContext = createContext<SearchCtx | null>(null)
function SearchSync({ children, ...ctx }: SearchCtx & { children: ReactNode }) {
  return <SearchContext.Provider value={ctx}>{children}</SearchContext.Provider>
}

export type CommandInputProps = Omit<InputProps, 'value' | 'onChange'>

/** The search box — drives the filter; arrow keys navigate the list. */
export function CommandInput({
  start = <Icon size="md"><SearchIcon /></Icon>,
  className,
  placeholder = 'Type a command or search…',
  ...props
}: CommandInputProps) {
  const search = useContext(SearchContext)
  const { activeId, listId } = useCommand()
  const setSearchQuery = search?.setQuery
  const handleChange = useCallback((event: ChangeEvent<HTMLInputElement>) => {
    setSearchQuery?.(event.target.value)
  }, [setSearchQuery])

  if (!search) throw new Error('Command.Input must be used within <Command>')

  return (
    <div className="q-command-input-row">
      <Input
        {...props}
        className={className}
        value={search.query}
        onChange={handleChange}
        placeholder={placeholder}
        start={start}
        role="combobox"
        aria-expanded="true"
        aria-controls={listId}
        aria-activedescendant={activeId ?? undefined}
        autoComplete="off"
        autoCorrect="off"
        spellCheck={false}
      />
    </div>
  )
}

export type CommandListProps = ComponentProps<'div'>

/** Scrollable listbox region. */
export function CommandList({ ref, className, children, ...props }: CommandListProps) {
  const search = useContext(SearchContext)
  if (!search) throw new Error('Command.List must be used within <Command>')
```

```
  return (
    <div
      ref={mergeRefs(search.listRef, ref)}
      id={search.listId}
      role="listbox"
      aria-label={search.label}
      className={cx('q-command-list', className)}
      {...props}
    >
      {children}
    </div>
  )
}

export type CommandEmptyProps = ComponentProps<'div'>

/** Shown only when the query matches nothing — and never while loading. */
export function CommandEmpty({ className, children = 'No results found.', ...props }: CommandEmptyProps) {
  const { matched, loading } = useCommand()
  if (loading || matched.size > 0) return null
  return <div className={cx('q-command-empty', className)} role="presentation" {...props}>{children}</div>
}

export type CommandLoadingProps = ComponentProps<'div'>

/** Render inside the list while results load (pair with `loading` on Command). */
export function CommandLoading({ className, children = 'Loading…', ...props }: CommandLoadingProps) {
  return <div className={cx('q-command-loading', className)} role="presentation" {...props}>{children}</div>
}

export type CommandGroupProps = ComponentProps<'div'> & { heading?: ReactNode }

/** A labelled group; hides itself (and its heading) when nothing inside matches. */
export function CommandGroup({ heading, className, children, ...props }: CommandGroupProps) {
  const groupId = useId()
  const { groupHasMatch } = useCommand()
  const visible = groupHasMatch(groupId)
  return (
    <GroupContext.Provider value={groupId}>
      <div className={cx('q-command-group', className)} role="group" hidden={!visible} {...props}>
        {heading != null ? <div className="q-command-group-heading" aria-hidden>{heading}</div> : null}
        {children}
      </div>
    </GroupContext.Provider>
  )
}

export interface CommandItemProps extends Omit<ComponentProps<'div'>, 'onSelect' | 'title'> {
  /** Rich content shown in `<Command.Detail>` while this item is active. */
  detail?: ReactNode
  /** Footer action label shown in `<Command.Action>` while this item is hovered/active (e.g. "Open dashboard"). */
  action?: ReactNode
  /** Explicit search text (overrides the text extracted from children). */
  value?: string
  /** Extra search terms beyond the children's text. */
  keywords?: string
  disabled?: boolean
  onSelect?: () => void
}

/**
 * A selectable command row. Compose it from the exported parts (`ItemIcon` /
 * `ItemContent` / `ItemTitle` / `ItemDescription` / `ItemTrailing`), mirroring
 * `Dropdown.Item`. It registers itself for keyboard navigation and filters
 * itself out when it doesn't match the query (matched on its children's text +
 * `keywords`, or an explicit `value`).
 */
export function CommandItem({
  detail,
  action,
```

```
  value,
  keywords,
  disabled = false,
  onSelect,
  className,
  children,
  ...props
}: CommandItemProps) {
  const id = useId()
  const groupId = useContext(GroupContext)
  const { matched, activeId, setActiveId, register, select, setActiveDetail, setActiveAction } = useCommand()
  const resolvedValue = value ?? nodeText(children)
  const resolvedKeywords = keywords ?? ''

  useLayoutEffect(() => {
    return register({ id, value: resolvedValue, keywords: resolvedKeywords, groupId, disabled }, onSelect)
    // re-register when the searchable text / disabled / group changes
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [id, resolvedValue, resolvedKeywords, groupId, disabled])

  const visible = matched.has(id)
  const active = activeId === id

  // When this item becomes active (hover or keyboard), publish its detail to
  // <Command.Detail> and its action label to <Command.Action>.
  useEffect(() => {
    if (active) {
      setActiveDetail(detail ?? null)
      setActiveAction(action ?? null)
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [active])

  return (
    <div
      id={id}
      data-command-item=""
      role="option"
      aria-selected={active}
      aria-disabled={disabled || undefined}
      data-active={active || undefined}
      hidden={!visible}
      className={cx('q-command-item', className)}
      onPointerMove={() => { if (!disabled) setActiveId(id) }}
      onClick={() => select(id)}
      {...props}
    >
      {children}
    </div>
  )
}

/* ■■ Item parts (compose any row layout; all forward className/children) ■■■■■■■ */

type PartProps = ComponentProps<'span'>

/** Leading slot — icon, avatar, dot, etc. */
export function CommandItemIcon({ className, ...props }: PartProps) {
  return <span className={cx('q-command-item-icon', className)} {...props} />
}

/** Content column — stacks the title and description. Optional: a bare
 * `Command.ItemTitle` works too (it grows + truncates on its own). */
export function CommandItemContent({ className, ...props }: PartProps) {
  return <span className={cx('q-command-item-content', className)} {...props} />
}

/** Primary label. */
export function CommandItemTitle({ className, ...props }: PartProps) {
  return <span className={cx('q-command-item-title', className)} {...props} />
```

```
}

/** Secondary line under the title. */
export function CommandItemDescription({ className, ...props }: PartProps) {
  return <span className={cx('q-command-item-description', className)} {...props} />
}

/** Trailing slot — shortcut, badge, chevron, count… (pushed to the right). */
export function CommandItemTrailing({ className, ...props }: PartProps) {
  return <span className={cx('q-command-item-trailing', className)} {...props} />
}

export function CommandSeparator({ className, ...props }: ComponentProps<'div'>) {
  // The Divider component draws the etched line; this wrapper owns the spacing.
  return (
    <div className={cx('q-command-separator', className)} {...props}>
      <Divider />
    </div>
  )
}

/**
 * A keyboard-shortcut pill in an item's trailing slot. Composes the canonical
 * `Kbd` (the cmdk shortcut has no bespoke design — reuse Kbd rather than
 * reinvent the pill). For multi-key combos joined by a separator use
 * `KbdSequence` directly in the item's `ItemTrailing`.
 */
export function CommandShortcut(props: ComponentProps<typeof Kbd>) {
  return <Kbd {...props} />
}

/* ■■ Two-pane layout (optional: list sidebar + detail panel) ■■■■■■■■■■■■■■■■■■■■■ */

export type CommandBodyProps = ComponentProps<'div'>
/** Row region between Input and Footer — holds the List (left) + Detail (right). */
export function CommandBody({ className, ...props }: CommandBodyProps) {
  return <div className={cx('q-command-body', className)} {...props} />
}

export type CommandDetailProps = ComponentProps<'div'>
/**
 * Right pane — renders the active item's `detail`. Renders nothing (so the list
 * goes full-width) when the active item carries no `detail`.
 *
 * Its content lives in an absolutely-positioned scroll layer, so the pane
 * contributes ZERO height to the row: the LIST pane sizes the palette and the
 * detail fills that height + scrolls. That's what keeps showing/hiding it from
 * resizing the modal — without which a tall detail grows the centered modal,
 * shifts the row under the cursor, and ping-pongs the hover (the detail
 * appearing/disappearing loop). Self-contained: drop it straight into
 * `Command.Body`, no second `Modal.Workspace` needed.
 */
export function CommandDetail({ className, ...props }: CommandDetailProps) {
  const { activeId, activeDetail } = useCommand()
  if (!activeId || activeDetail == null) return null
  // Rendered as the shared Modal.Workspace surface (a frosted pane, matching the
  // list) — so the two-pane layout stays on-design — with the content in an
  // absolute scroll layer so the pane contributes ZERO height: the LIST sizes the
  // palette, this never does (no hover taper/loop). Drop it straight into
  // `Command.Body`; it self-wraps and removes itself for items without a detail.
  return (
    <Modal.Workspace padded={false} className={cx('q-command-detail', className)} {...props}>
      <div className="q-command-detail-scroll">{activeDetail}</div>
    </Modal.Workspace>
  )
}

export type CommandFooterProps = ComponentProps<'div'> & {
  /** Leading footer label/caption when Command.Footer is rendered in Command.Dialog. */
  caption?: ReactNode
```

```
  /** Trailing footer actions when Command.Footer is rendered in Command.Dialog. */
  actions?: ReactNode
  /** Stretch footer actions when rendered by Modal.Footer. */
  full?: boolean
}
/** Bottom bar — e.g. brand on the left, an Enter-to-confirm action on the right. */
export function CommandFooter({ caption, actions, full: _full, children, className, ...props }: CommandFooterProps) {
  return (
    <div className={cx('q-command-footer', className)} {...props}>
      {children ?? (
        <>
          {caption != null ? <span className="q-command-footer-caption">{caption}</span> : <span aria-hidden />}
          {actions != null ? <span className="q-command-footer-actions">{actions}</span> : null}
        </>
      )}
    </div>
  )
}

export interface CommandActionProps extends ComponentProps<'button'> {
  /** Label shown when no active item provides an `action` (e.g. "Select"). */
  fallback?: ReactNode
}
/**
 * Footer confirm button. Shows the active item's `action` label (falling back to
 * `fallback`) followed by `children` (typically a `<Kbd>`), and runs the active
 * item on click — the click equivalent of pressing Enter.
 */
export function CommandAction({ fallback, className, children, ...props }: CommandActionProps) {
  const { activeId, select, activeAction } = useCommand()
  const label = activeAction ?? fallback
  return (
    <button
      type="button"
      className={cx('q-command-action', className)}
      onClick={() => { if (activeId) select(activeId) }}
      disabled={!activeId}
      {...props}
    >
      {label != null ? <span className="q-command-action-label">{label}</span> : null}
      {children}
    </button>
  )
}

/* ■■ Dialog shell (■K palette) ■■■
```

```
        full: props.full,
      }
    }
    else {
      body.push(child)
    }
  }

  return { body, footer, header }
}

export interface CommandDialogProps extends CommandProps {
  open?: boolean
  defaultOpen?: boolean
  onOpenChange?: (open: boolean) => void
  /** Global hotkey to toggle the palette, e.g. `"mod+k"`. */
  shortcut?: string
  /** Modal size preset. */
  size?: ModalSize
  /** Class for the shared Modal.Content shell. */
  className?: string
  backdropClassName?: string
  container?: ComponentProps<typeof Modal.Content>['container']
  initialFocus?: ComponentProps<typeof Modal.Content>['initialFocus']
  finalFocus?: ComponentProps<typeof Modal.Content>['finalFocus']
}

/** The command palette in the shared Modal shell with an optional hotkey. */
export function CommandDialog({
  open,
  defaultOpen = false,
  onOpenChange,
  shortcut,
  size = 'md',
  label = 'Command menu',
  className,
  backdropClassName,
  container,
  initialFocus = true,
  finalFocus,
  children,
  ...commandProps
}: CommandDialogProps) {
  const [uncontrolled, setUncontrolled] = useState(defaultOpen)
  const isOpen = open ?? uncontrolled
  const setOpen = useCallback((next: boolean) => {
    if (open === undefined) setUncontrolled(next)
    onOpenChange?.(next)
  }, [open, onOpenChange])

  useEffect(() => {
    if (!shortcut) return
    const onKey = (e: globalThis.KeyboardEvent) => {
      if (matchShortcut(e, shortcut)) {
        e.preventDefault()
        setOpen(!isOpen)
      }
    }
    window.addEventListener('keydown', onKey)
    return () => window.removeEventListener('keydown', onKey)
  }, [shortcut, isOpen, setOpen])

  const { body, footer, header } = splitDialogChildren(children)

  return (
    <CommandRoot label={label} className="q-command-dialog-context" {...commandProps}>
      <Modal.Root open={isOpen} onOpenChange={setOpen}>
        <Modal.Content
          aria-label={label}
          size={size}
```

```
          className={className}
          backdropClassName={backdropClassName}
          container={container}
          initialFocus={initialFocus}
          finalFocus={finalFocus}
        >
          <Modal.Header>
            {header.length > 0 ? header : <CommandInput />}
            <Modal.CloseButton />
          </Modal.Header>
          <Modal.Body padded={false}>
            {body}
          </Modal.Body>
          {footer != null
            ? (
                <Modal.Footer full={footer.full}>
                  {footer.children ?? (
                    <>
                      {footer.caption != null ? <Modal.FooterCaption>{footer.caption}</Modal.FooterCaption> : <Modal.Spacer />}
                      {footer.actions != null ? <Modal.FooterActions full={footer.full}>{footer.actions}</Modal.FooterActions> : null}
                    </>
                  )}
                </Modal.Footer>
              )
            : null}
        </Modal.Content>
      </Modal.Root>
    </CommandRoot>
  )
}

/* `Command` is both the inline root and the namespace holding the parts. */
export const Command = Object.assign(CommandRoot, {
  Root: CommandRoot,
  Dialog: CommandDialog,
  Input: CommandInput,
  List: CommandList,
  Empty: CommandEmpty,
  Loading: CommandLoading,
  Group: CommandGroup,
  Item: CommandItem,
  ItemIcon: CommandItemIcon,
  ItemContent: CommandItemContent,
  ItemTitle: CommandItemTitle,
  ItemDescription: CommandItemDescription,
  ItemTrailing: CommandItemTrailing,
  Separator: CommandSeparator,
  Shortcut: CommandShortcut,
  Body: CommandBody,
  Detail: CommandDetail,
  Footer: CommandFooter,
  Action: CommandAction,
})
```

app/packages/quanta/src/components/cmdk/index.ts

```
export {
  Command,
  CommandAction,
  CommandBody,
  CommandDetail,
  CommandDialog,
  CommandEmpty,
  CommandFooter,
  CommandGroup,
  CommandInput,
  CommandItem,
  CommandItemContent,
  CommandItemDescription,
  CommandItemIcon,
  CommandItemTitle,
  CommandItemTrailing,
  CommandList,
  CommandLoading,
  CommandSeparator,
  CommandShortcut,
} from './cmdk.tsx'
export type {
  CommandActionProps,
  CommandBodyProps,
  CommandDetailProps,
  CommandDialogProps,
  CommandEmptyProps,
  CommandFilter,
  CommandFooterProps,
  CommandGroupProps,
  CommandInputProps,
  CommandItemProps,
  CommandListProps,
  CommandLoadingProps,
  CommandProps,
} from './cmdk.tsx'
```

app/packages/quanta/src/components/divider/divider.css

```
/**
 * Divider presentation — the bare etched separator from Figma node 834:1556
 * (`height: 1px; align-self: stretch`, edge-to-edge). The visible mark is a 2px
 * etched line: a 1px darker rule overflowing 1px ABOVE the 1px layout box, and a
 * 1px lighter rule inside it.
 *
 *   top    = black 15%  (Figma drop-shadow transparent/light/15 by value)
 *   bottom = white 10%  (Figma fill        transparent/dark/10  by value)
 *
 * Matched to quanta tokens BY VALUE: #00000026 = transparent-dark-15 and
 * #ffffff1a = transparent-light-10. The rules span the full width (no inset) —
 * this single primitive is used everywhere, including dropdown menus (which used
 * to special-case an inset `q-menu-separator`; that was dropped).
 *
 * @source points Tailwind's scanner at the colocated source (quanta is a
 * symlinked dep and Tailwind skips node_modules) — same convention as dot.css.
 */
@source "./divider.tsx";

@utility q-divider {
  border: 0;
  border-width: 0;
  box-sizing: border-box;
  height: var(--hf-border-width-thin);
  overflow: visible;
  position: relative;
}

.q-divider::before,
.q-divider::after {
  content: "";
  display: block;
  height: var(--hf-border-width-thin);
  left: var(--hf-space-0);
  position: absolute;
  right: var(--hf-space-0);
}

.q-divider::before {
  background-color: var(--hf-color-transparent-dark-15);
  top: calc(var(--hf-border-width-thin) * -1);
}

.q-divider::after {
  background-color: var(--hf-color-transparent-light-10);
  top: var(--hf-space-0);
}

@utility q-divider-vertical {
  border: 0;
  border-width: 0;
  box-sizing: border-box;
  overflow: visible;
  position: relative;
  width: var(--hf-border-width-thin);
}

.q-divider-vertical::before,
.q-divider-vertical::after {
  bottom: var(--hf-space-0);
  content: "";
  display: block;
  position: absolute;
  top: var(--hf-space-0);
  width: var(--hf-border-width-thin);
}

.q-divider-vertical::before {
```

```
  background-color: var(--hf-color-transparent-dark-15);
  left: calc(var(--hf-border-width-thin) * -1);
}

.q-divider-vertical::after {
  background-color: var(--hf-color-transparent-light-10);
  left: var(--hf-space-0);
}
```

app/packages/quanta/src/components/divider/divider.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Divider } from './index.ts'

describe('<Divider>', () => {
  it('defaults to a semantic <hr> with the etched horizontal styling', () => {
    const { container } = render(<Divider data-testid="d" />)
    const hr = container.querySelector('hr')!
    expect(hr).toBeInTheDocument()
    expect(hr).toHaveClass('q-divider', 'block', 'w-full')
  })

  it('renders vertical orientation as an <hr> that stretches in a flex row', () => {
    const { container } = render(<Divider orientation="vertical" />)
    const hr = container.querySelector('hr')!
    expect(hr).toHaveAttribute('aria-orientation', 'vertical')
    expect(hr).toHaveClass('q-divider-vertical', 'self-stretch')
  })

  it('switches to role=separator with flanking etched rules when labelled', () => {
    render(<Divider>or</Divider>)
    const sep = screen.getByRole('separator')
    expect(sep.tagName).toBe('DIV')
    expect(sep).toHaveAttribute('aria-orientation', 'horizontal')
    const label = screen.getByText('or')
    expect(label.tagName).toBe('SPAN')
    expect(label).toHaveClass('text-q-text-tertiary', 'text-q-caption-sm-medium')
    const rules = sep.querySelectorAll('[aria-hidden="true"]')
    expect(rules).toHaveLength(2)
    rules.forEach(rule => expect(rule).toHaveClass('q-divider'))
  })

  it('forwards className and native hr props on the unlabelled variant', () => {
    const { container } = render(<Divider className="is-custom" id="sep" />)
    const hr = container.querySelector('hr')!
    expect(hr).toHaveClass('is-custom')
    expect(hr).toHaveAttribute('id', 'sep')
  })
})
```

app/packages/quanta/src/components/divider/divider.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { Typography } from '../typography/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * Divider — a presentational separator matching the Figma divider (node
 * 834:1556): a bare, edge-to-edge etched 2px line (a 1px darker rule overflowing
 * 1px above a 1px layout box). Renders a semantic `<hr>`; when a label is
 * supplied it switches to `<div role="separator">` with two rules flanking the
 * text. This single primitive is used everywhere — including dropdown menus (the
 * old inset `q-menu-separator` was dropped).
 *
 * Usage:
 *   <Divider />                            // horizontal etched rule
 *   <Divider orientation="vertical" />     // self-stretches inside a flex row
 *   <Divider>or</Divider>                  // labelled — text-q-text-tertiary caption
 */

export type DividerOrientation = 'horizontal' | 'vertical'

export type DividerProps = Omit<ComponentProps<'hr'>, 'children'> & {
  orientation?: DividerOrientation
  /** Optional inline label rendered between two rules (horizontal only). */
  children?: ReactNode
}

export function Divider({
  orientation = 'horizontal',
  className,
  children,
  ...props
}: DividerProps) {
  // Labelled — needs to contain text, so we can't use <hr>.
  if (children != null && orientation === 'horizontal') {
    return (
      <div
        role="separator"
        aria-orientation="horizontal"
        className={cx('flex w-full items-center gap-2 align-middle', className)}
        {...(props as ComponentProps<'div'>)}
      >
        <span className="q-divider flex-1" aria-hidden />
        <Typography as="span" variant="caption-sm-medium" color="tertiary">
          {children}
        </Typography>
        <span className="q-divider flex-1" aria-hidden />
      </div>
    )
  }

  if (orientation === 'vertical') {
    return (
      <hr
        aria-orientation="vertical"
        className={cx('q-divider-vertical m-0 inline-block self-stretch', className)}
        {...props}
      />
    )
  }

  return (
    <hr
      className={cx('q-divider m-0 block w-full', className)}
      {...props}
    />
  )
```

```
}
```

app/packages/quanta/src/components/divider/index.ts

```
export { Divider } from './divider.tsx'
export type { DividerOrientation, DividerProps } from './divider.tsx'
```

app/packages/quanta/src/components/dot/dot.css

```
/**
 * Dot presentation — pinned to the Figma "Status Dot" component (node 1418:94).
 *
 * Dot composes emitted q-* token utilities (palette fills, the glass ring
 * colour, border-widths, and spacing sizes) plus native backdrop-blur. The
 * stroke is outside the filled Figma dot (`box-content` in dot.tsx), so the
 * visual boxes are 12 / 9 / 7px around 8 / 6 / 4px fills. This file points
 * Tailwind's scanner at the colocated source (Tailwind skips node_modules and
 * quanta is symlinked, so the literal class strings in dot.tsx — e.g.
 * `bg-q-palette-mint-bg`, `border-q-background-glass` — must be registered
 * explicitly, same convention as avatar.css / button.css). The TS unions
 * (SIZE / RING / RING_COLOR / FILL) are the single source of truth.
 *
 * Decorative motion (opt-in via the `animation` prop) lives here. Both presence
 * animations derive their colour from `currentColor` (the component sets `color`
 * to the fill while animating), animate only compositor-friendly properties
 * (opacity + transform) so they stay smooth, and are switched off under
 * `prefers-reduced-motion`.
 */
@source "./dot.tsx";

/* The glass ring's backdrop blur is Figma's BackgroundGlassBlur (radius 80 →
 * 40px), the same 40px the modal/vault/sonner glass use — tokenized via
 * --hf-space-1000 (was the non-token `backdrop-blur-2xl` Tailwind preset). */
@utility q-dot {
  background-clip: padding-box;
  backdrop-filter: blur(var(--hf-space-1000));
}

/* ■■ pulse — radar-style rings ripple outward from the dot (live / online). ■■ *
 * Two staggered ::before/::after rings (one half a cycle behind) read as a
 * continuous ripple. They sit behind the dot (z-index -1) so the fill stays
 * crisp; `inset: 0` anchors them to the colored fill, which then scales out. */
@keyframes q-dot-pulse {
  from {
    opacity: 0.5;
    transform: scale(1);
  }

  to {
    opacity: 0;
    transform: scale(2.75);
  }
}

@utility q-dot-pulse {
  position: relative;

  &::before,
  &::after {
    content: "";
    position: absolute;
    inset: var(--hf-space-0);
    z-index: -1; /* no token: stacking layer, not a spacing scale value */
    border-radius: var(--hf-radius-full);
    background-color: currentColor;
    pointer-events: none;
    will-change: transform, opacity;
    /* no token: 2s — no matching --hf-duration-* (longest is 1400ms) */
    animation: q-dot-pulse 2s var(--hf-ease-out) infinite;
  }

  &::after {
    animation-delay: var(--hf-duration-1000); /* 1000ms = 1s */
  }
}

/* ■■ glow — a soft coloured halo breathes around the dot. A constantly-blurred
```

```
 * currentColor disc behind the dot animates ONLY opacity + scale (both
 * compositor-friendly), so the breathing stays perfectly smooth — unlike an
 * animated box-shadow, which repaints every frame and janks. ■■ */
@keyframes q-dot-glow {
  0%,
  100% {
    opacity: 0.18;
    transform: scale(1.4);
  }

  50% {
    opacity: 0.38;
    transform: scale(1.9);
  }
}

@utility q-dot-glow {
  position: relative;

  &::before {
    content: "";
    position: absolute;
    inset: var(--hf-space-0);
    z-index: -1; /* no token: stacking layer, not a spacing scale value */
    border-radius: var(--hf-radius-full);
    background-color: currentColor;
    filter: blur(var(--hf-space-050)); /* 2px — tight soft halo */
    pointer-events: none;
    will-change: transform, opacity;
    /* no token: 2.4s — no matching --hf-duration-*; ease-in-out keyword is not
     * byte-identical to --hf-ease-in-out (cubic-bezier(0.4, 0, 0.2, 1)) */
    animation: q-dot-glow 2.4s ease-in-out infinite;
  }
}

@media (prefers-reduced-motion: reduce) {
  .q-dot-pulse::before,
  .q-dot-pulse::after,
  .q-dot-glow::before {
    display: none;
  }
}
```

app/packages/quanta/src/components/dot/dot.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Dot } from './index.ts'

describe('<Dot>', () => {
  it('defaults to a green medium dot with a thick glass ring', () => {
    const { container } = render(<Dot />)
    const dot = container.querySelector('span')
    expect(dot).toHaveClass(
      'rounded-q-full',
      'box-content',
      'size-q-200',
      'border-q-thick',
      'bg-q-palette-mint-bg',
      'border-q-background-glass',
      'q-dot',
    )
  })

  it('maps the Figma size ramp to fill box + outer stroke width', () => {
    const xs = render(<Dot size="xs" />)
    expect(xs.container.querySelector('span')).toHaveClass('size-q-100', 'border-q-medium', 'border-q-transparent-light-05')
    xs.unmount()

    const sm = render(<Dot size="sm" />)
    expect(sm.container.querySelector('span')).toHaveClass('size-q-150', 'border-q-medium', 'border-q-background-glass')
    sm.unmount()

    const md = render(<Dot size="md" />)
    expect(md.container.querySelector('span')).toHaveClass('size-q-200', 'border-q-thick', 'border-q-background-glass')
  })

  it('uses the exact Figma border color for every color and size variant', () => {
    const colors = ['green', 'yellow', 'red', 'grey'] as const
    const sizes = ['md', 'sm', 'xs'] as const

    for (const color of colors) {
      for (const size of sizes) {
        const { container, unmount } = render(<Dot color={color} size={size} />)
        const expected = color === 'green' && size === 'xs'
          ? 'border-q-transparent-light-05'
          : 'border-q-background-glass'

        expect(container.querySelector('span')).toHaveClass(expected)
        unmount()
      }
    }
  })

  it('paints each presence colour with its Figma palette token', () => {
    const cases = [
      ['green', 'bg-q-palette-mint-bg'],
      ['yellow', 'bg-q-brand-yellow'],
      ['red', 'bg-q-palette-pink-bg'],
      ['grey', 'bg-q-icon-secondary'],
    ] as const

    for (const [color, fill] of cases) {
      const { container, unmount } = render(<Dot color={color} />)
      expect(container.querySelector('span')).toHaveClass(fill)
      unmount()
    }
  })

  it('exposes an accessible image when a label is given', () => {
    render(<Dot color="red" size="sm" label="busy" />)
    const dot = screen.getByRole('img', { name: 'busy' })
    expect(dot).toHaveClass('size-q-150', 'border-q-medium', 'bg-q-palette-pink-bg')
```

```
  })

  it('is hidden from assistive tech when unlabelled', () => {
    const { container } = render(<Dot />)
    const dot = container.querySelector('span')
    expect(dot).toHaveAttribute('aria-hidden', 'true')
    expect(screen.queryByRole('img')).not.toBeInTheDocument()
  })

  it('forwards className and native span props', () => {
    const { container } = render(<Dot className="is-custom" title="Presence" />)
    const dot = container.querySelector('span')
    expect(dot).toHaveClass('is-custom', 'rounded-q-full')
    expect(dot).toHaveAttribute('title', 'Presence')
  })

  it('is static (no animation classes) by default', () => {
    const { container } = render(<Dot />)
    const dot = container.querySelector('span')
    expect(dot).not.toHaveClass('q-dot-pulse', 'q-dot-glow')
    // no ink (currentColor) class when not animating
    expect(dot).not.toHaveClass('text-q-palette-mint-bg')
  })

  it('opts into pulse / glow and inherits the fill colour for the effect', () => {
    const pulse = render(<Dot animation="pulse" />)
    // green default → fill + matching ink so the rings/halo use currentColor
    expect(pulse.container.querySelector('span')).toHaveClass('q-dot-pulse', 'text-q-palette-mint-bg')
    pulse.unmount()

    const glow = render(<Dot color="red" animation="glow" />)
    expect(glow.container.querySelector('span')).toHaveClass('q-dot-glow', 'text-q-palette-pink-bg')
  })
})
```

app/packages/quanta/src/components/dot/dot.tsx

```
'use client'

import type { ComponentProps } from 'react'
import { cx } from '../utils/cx.ts'

/**
 * Dot — a presence / status indicator pinned to the Figma `_AvatarStatus`
 * component (node 1418:94): a palette-coloured circle with an outer translucent
 * glass stroke and BackgroundGlassBlur. Standalone and generic — Avatar composes
 * it for its presence badge, but it is not avatar-specific.
 *
 * Figma ramp (fill box / outer stroke / visual box):
 *   md → 8px / 2px / 12px   sm → 6px / 1.5px / 9px   xs → 4px / 1.5px / 7px
 *
 * Optional decorative motion via `animation` (see DotAnimation) — colour-derived
 * and reduced-motion-safe.
 */

export type DotColor = 'green' | 'yellow' | 'red' | 'grey'
export type DotSize = 'md' | 'sm' | 'xs'
/**
 * Opt-in decorative motion (disabled under `prefers-reduced-motion`):
 *   pulse — radar-style rings ripple out of the dot (a live / online beacon)
 *   glow  — a soft coloured halo breathes around the dot
 */
export type DotAnimation = 'pulse' | 'glow'

const SIZE_CLASS = {
  md: 'size-q-200', // 8px
  sm: 'size-q-150', // 6px
  xs: 'size-q-100', // 4px
} satisfies Record<DotSize, string>

const RING = {
  md: 'border-q-thick', // 2px
  sm: 'border-q-medium', // 1.5px
  xs: 'border-q-medium', // 1.5px
} satisfies Record<DotSize, string>

const RING_COLOR = {
  green: {
    md: 'border-q-background-glass',
    sm: 'border-q-background-glass',
    // Figma variable `transparent/dark/05` is white 5% in this dark design.
    xs: 'border-q-transparent-light-05',
  },
  yellow: {
    md: 'border-q-background-glass',
    sm: 'border-q-background-glass',
    xs: 'border-q-background-glass',
  },
  red: {
    md: 'border-q-background-glass',
    sm: 'border-q-background-glass',
    xs: 'border-q-background-glass',
  },
  grey: {
    md: 'border-q-background-glass',
    sm: 'border-q-background-glass',
    xs: 'border-q-background-glass',
  },
} satisfies Record<DotColor, Record<DotSize, string>>

const FILL = {
  green: 'bg-q-palette-mint-bg',
  yellow: 'bg-q-brand-yellow',
  red: 'bg-q-palette-pink-bg',
  grey: 'bg-q-icon-secondary',
```

```
} satisfies Record<DotColor, string>

/**
 * Sets `color` to the fill so the animations (which use `currentColor` for their
 * rings / halo) inherit the dot's colour. Applied only when animating.
 */
const INK = {
  green: 'text-q-palette-mint-bg',
  yellow: 'text-q-brand-yellow',
  red: 'text-q-palette-pink-bg',
  grey: 'text-q-icon-secondary',
} satisfies Record<DotColor, string>

const ANIMATION = {
  pulse: 'q-dot-pulse',
  glow: 'q-dot-glow',
} satisfies Record<DotAnimation, string>

export type DotProps = ComponentProps<'span'> & {
  color?: DotColor
  size?: DotSize
  /** Decorative motion — `pulse` (ripple rings) or `glow` (breathing halo). */
  animation?: DotAnimation
  /** Accessible name; when set the dot is exposed as `role="img"`, otherwise it is hidden. */
  label?: string
}

export function Dot({
  color = 'green',
  size = 'md',
  animation,
  label,
  className,
  role,
  'aria-label': ariaLabel,
  'aria-hidden': ariaHidden,
  ...props
}: DotProps) {
  const accessibleLabel = ariaLabel ?? label

  return (
    <span
      role={role ?? (accessibleLabel ? 'img' : undefined)}
      aria-label={accessibleLabel}
      aria-hidden={ariaHidden ?? (accessibleLabel ? undefined : true)}
      className={cx(
        'q-dot box-content block shrink-0 rounded-q-full',
        SIZE_CLASS[size],
        RING[size],
        RING_COLOR[color][size],
        FILL[color],
        animation && ANIMATION[animation],
        animation && INK[color],
        className,
      )}
      {...props}
    />
  )
}
```

app/packages/quanta/src/components/dot/index.ts

```
export { Dot } from './dot.tsx'
export type { DotAnimation, DotColor, DotProps, DotSize } from './dot.tsx'
```

app/packages/quanta/src/components/dropdown/dropdown.css

```
/**
 * Dropdown presentation utilities (q-dropdown-* namespace), matched to the
 * Figma Dropdown/Menu system (Menu / Select board 1302:82, plus _Menu* parts
 * 236:3238 / 537:346 / 585:1145 / 676:2123 / 650:769 / 995:2266).
 *
 * The ROW SHELL is the shared `q-menu-item*` primitive (see menu.css) — the
 * dropdown and navigation-menu render identical rows. Everything here is
 * dropdown-specific: the glass popup, search bar, group/label rhythm, leading
 * media tile, trailing meta, the selection check, and the submenu trigger.
 * `Dropdown.Item` is composition-only — it renders whatever parts the consumer
 * nests, so these utilities never assume a fixed row data shape.
 */

@source "./dropdown.tsx";

/* ■■ Popup ■■■
```

```
  &[data-starting-style][data-side="inline-start"],
  &[data-ending-style][data-side="inline-start"] {
    transform: scale(0.94) translateX(var(--hf-space-100));
  }

  &[data-ending-style] {
    transition-duration: var(--hf-duration-130);
    transition-timing-function: var(--hf-ease-in);
  }

  &[data-instant] {
    transition-duration: var(--hf-duration-instant);
  }
}

/* First/last rows follow the wrapper radius so their hover/selected fill nests
 * into the rounded popup. The search layout adds a wrapper, so its trailing rows
 * are matched through it (the search bar owns the top edge). */
.q-dropdown-content > .q-menu-item:first-child {
  border-top-left-radius: var(--q-dropdown-item-outer-radius);
  border-top-right-radius: var(--q-dropdown-item-outer-radius);
}

.q-dropdown-content > .q-menu-item:last-child,
.q-dropdown-content > .q-dropdown-group:last-child > .q-menu-item:last-child,
.q-dropdown-content > .q-dropdown-search-wrap:last-child > .q-menu-item:last-child,
.q-dropdown-content > .q-dropdown-search-wrap:last-child > .q-dropdown-group:last-child > .q-menu-item:last-child {
  border-bottom-left-radius: var(--q-dropdown-item-outer-radius);
  border-bottom-right-radius: var(--q-dropdown-item-outer-radius);
}

/* Distinct width presets (default = :root 160–400). compact is narrower than
 * default and large is wider, on the MIN so they differ even for short content. */
@utility q-dropdown-content-compact {
  --q-dropdown-radius: var(--hf-radius-300);
  --q-menu-min-width: calc(var(--hf-space-2400) + var(--hf-space-1200)); /* 144px */
  --q-menu-max-width: calc(var(--hf-space-2400) * 2 + var(--hf-space-200)); /* 200px */
}

@utility q-dropdown-content-large {
  --q-dropdown-radius: var(--hf-radius-300);
  --q-menu-min-width: calc(var(--hf-space-2400) * 3 + var(--hf-space-800)); /* 320px */
  --q-menu-max-width: calc(var(--hf-space-2400) * 5); /* 480px */
}

@utility q-dropdown-content-panel {
  --q-dropdown-radius: var(--hf-radius-500);
  --q-dropdown-item-outer-radius: var(--hf-radius-300);
}

@utility q-dropdown-content-solid {
  backdrop-filter: none;
  background-color: var(--hf-color-background-secondary);
}

/* Grouped rows share the popup's row rhythm. */
.q-dropdown-content .q-dropdown-group,
.q-dropdown-content .q-dropdown-search-wrap {
  display: flex;
  flex-direction: column;
  gap: var(--q-dropdown-gap, var(--hf-space-050));
}

/* A row carrying a trailing button gets the taller Figma button-row height. */
.q-dropdown-content .q-menu-item:has(.q-button) {
  min-height: var(--hf-space-1000); /* 40px */
}

/* ■■ Trigger (keeps the hover/open look while its popup is open) ■■■■■■■■■■■■■ */
.q-dropdown-trigger[aria-expanded="true"].q-menu-item,
```

```
.q-dropdown-trigger[data-open].q-menu-item,
.q-dropdown-trigger[data-popup-open].q-menu-item,
.q-dropdown-submenu-trigger[aria-expanded="true"].q-menu-item,
.q-dropdown-submenu-trigger[data-open].q-menu-item,
.q-dropdown-submenu-trigger[data-popup-open].q-menu-item {
  background-color: var(--hf-color-overlay-hover);
}

.q-dropdown-trigger[aria-expanded="true"].q-button-primary,
.q-dropdown-trigger[data-open].q-button-primary,
.q-dropdown-trigger[data-popup-open].q-button-primary,
.q-dropdown-trigger[aria-expanded="true"].q-button-tertiary,
.q-dropdown-trigger[data-open].q-button-tertiary,
.q-dropdown-trigger[data-popup-open].q-button-tertiary,
.q-dropdown-trigger[aria-expanded="true"].q-button-danger,
.q-dropdown-trigger[data-open].q-button-danger,
.q-dropdown-trigger[data-popup-open].q-button-danger,
.q-dropdown-trigger[aria-expanded="true"].q-button-danger-soft,
.q-dropdown-trigger[data-open].q-button-danger-soft,
.q-dropdown-trigger[data-popup-open].q-button-danger-soft,
.q-dropdown-trigger[aria-expanded="true"].q-button-brand-soft,
.q-dropdown-trigger[data-open].q-button-brand-soft,
.q-dropdown-trigger[data-popup-open].q-button-brand-soft,
.q-dropdown-trigger[aria-expanded="true"].q-button-marketing-tertiary,
.q-dropdown-trigger[data-open].q-button-marketing-tertiary,
.q-dropdown-trigger[data-popup-open].q-button-marketing-tertiary,
.q-dropdown-trigger[aria-expanded="true"].q-button-marketing-ghost,
.q-dropdown-trigger[data-open].q-button-marketing-ghost,
.q-dropdown-trigger[data-popup-open].q-button-marketing-ghost {
  background-image: linear-gradient(var(--hf-color-transparent-light-20), var(--hf-color-transparent-light-20));
}

.q-dropdown-trigger[aria-expanded="true"].q-button-secondary,
.q-dropdown-trigger[data-open].q-button-secondary,
.q-dropdown-trigger[data-popup-open].q-button-secondary {
  background-image: linear-gradient(var(--hf-color-transparent-dark-50), var(--hf-color-transparent-dark-50));
}

.q-dropdown-trigger[aria-expanded="true"].q-button-outline,
.q-dropdown-trigger[data-open].q-button-outline,
.q-dropdown-trigger[data-popup-open].q-button-outline,
.q-dropdown-trigger[aria-expanded="true"].q-button-ghost,
.q-dropdown-trigger[data-open].q-button-ghost,
.q-dropdown-trigger[data-popup-open].q-button-ghost {
  background-color: var(--hf-color-overlay-dim-strong);
}

/* ■■ Search bar (Figma _MenuSearch 676:2123) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-dropdown-search-wrap {
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-050);
}

@utility q-dropdown-search {
  align-items: center;
  background-color: transparent;
  border-radius: var(--hf-radius-300);
  box-sizing: border-box;
  color: var(--hf-color-icon-on-overlay-tertiary);
  display: flex;
  flex-shrink: 0;
  gap: var(--hf-space-200);
  min-height: var(--hf-space-800);
  padding: var(--hf-space-150); /* 6px uniform */
}

@utility q-dropdown-search-icon {
  align-items: center;
  display: inline-flex;
```

```
  flex-shrink: 0;
  height: calc(var(--hf-space-500) - var(--hf-space-050)); /* 18px */
  justify-content: center;
  width: calc(var(--hf-space-500) - var(--hf-space-050));

  & svg {
    height: 100%;
    width: 100%;
  }
}

@utility q-dropdown-search-input {
  appearance: none;
  background: transparent;
  border: var(--hf-border-width-none) solid transparent;
  color: var(--hf-color-text-primary);
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  outline: none;
  padding: var(--hf-space-0);
  @apply text-q-body-sm-regular;

  &::placeholder {
    color: var(--hf-color-text-on-overlay-secondary);
  }
}

/* ■■ Leading media tile (Figma _MenuItemLarge mediaSlot, 36px) ■■■■■■■■■■■■■■■ *
 * A row that carries a media tile becomes the rich "large" row: 56px tall, with
 * a 12px caption title and a grey secondary description — all driven by the
 * composed parts, no presentation prop. */
@utility q-dropdown-item-media {
  align-items: center;
  background-color: var(--hf-color-background-secondary);
  border: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  border-radius: var(--hf-radius-200);
  box-shadow: var(--hf-shadow-inset-tile-hairline);
  color: var(--hf-color-icon-secondary);
  display: inline-flex;
  flex-shrink: 0;
  height: calc(var(--hf-space-800) + var(--hf-space-100)); /* 36px */
  justify-content: center;
  overflow: clip;
  width: calc(var(--hf-space-800) + var(--hf-space-100));

  & img {
    height: 100%;
    object-fit: cover;
    width: 100%;
  }

  & svg {
    height: var(--hf-space-500);
    width: var(--hf-space-500);
  }
}

.q-menu-item:has(.q-dropdown-item-media) {
  min-height: var(--hf-space-1400); /* 56px — Figma _MenuItemLarge */
}

.q-menu-item:has(.q-dropdown-item-media) .q-menu-item-title {
  @apply text-q-caption-sm-medium; /* 12px */
}

.q-menu-item:has(.q-dropdown-item-media) .q-menu-item-description {
  color: var(--hf-color-text-secondary); /* #828282 — large-row description */
}

/* ■■ Trailing meta (inline count) + inset chip ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
```

```
@utility q-dropdown-item-meta {
  color: var(--hf-color-text-secondary);
  flex-shrink: 0;
  white-space: nowrap;
  @apply text-q-caption-sm-regular;
}

@utility q-dropdown-meta-chip {
  align-items: center;
  background-color: var(--hf-color-transparent-light-05);
  border-radius: var(--hf-radius-100);
  color: var(--hf-color-text-on-overlay-secondary);
  display: inline-flex;
  flex-shrink: 0;
  gap: var(--hf-space-100);
  padding: var(--hf-space-050) var(--hf-space-100);
  white-space: nowrap;
  @apply text-q-caption-xs-regular;

  & svg {
    height: var(--hf-space-300);
    width: var(--hf-space-300);
  }
}

/* ■■ Selection / submenu glyph ■■■
```

app/packages/quanta/src/components/dropdown/dropdown.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { createRef, useState } from 'react'
import { IconFolder1Outlined as FolderIcon } from '@higgsfield-ai/icons/IconFolder1Outlined'
import { describe, expect, it, vi } from 'vitest'
import { NotFound } from '../not-found/index.ts'
import { Dropdown } from './index.ts'

/** Compose a simple title-only row (the pure-composition primitives). */
function Row({
  title,
  indicator,
  ...props
}: { title: string, indicator?: 'check' | 'checkbox' | 'switch' } & Parameters<typeof Dropdown.Item>[0]) {
  return (
    <Dropdown.Item {...props}>
      <Dropdown.ItemContent>
        <Dropdown.ItemTitleRow>
          <Dropdown.ItemTitle>{title}</Dropdown.ItemTitle>
        </Dropdown.ItemTitleRow>
      </Dropdown.ItemContent>
      {props.selectable
        ? <Dropdown.ItemTrailing><Dropdown.ItemIndicator indicator={indicator} /></Dropdown.ItemTrailing>
        : null}
    </Dropdown.Item>
  )
}

function Basic() {
  return (
    <Dropdown.Root defaultOpen>
      <Dropdown.Trigger>Open</Dropdown.Trigger>
      <Dropdown.Content>
        <Dropdown.Group>
          <Dropdown.Label>Section</Dropdown.Label>
          <Row title="Plain" />
        </Dropdown.Group>
        <Dropdown.Separator />
      </Dropdown.Content>
    </Dropdown.Root>
  )
}

describe('Dropdown containers', () => {
  it('marks the root trigger so open menus can keep the trigger hover treatment', () => {
    render(<Basic />)
    const trigger = screen.getByRole('button', { name: 'Open' })
    expect(trigger).toHaveClass('q-dropdown-trigger')
    expect(trigger).toHaveAttribute('data-open')
    expect(trigger).toHaveAttribute('data-popup-open')
  })

  it('keeps hover-open opt-in so default dropdowns still require click', async () => {
    const user = userEvent.setup()
    render(
      <Dropdown.Root>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Row title="Action" />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    const trigger = screen.getByRole('button', { name: 'Open' })
    await user.hover(trigger)
    expect(screen.queryByRole('menuitem', { name: 'Action' })).not.toBeInTheDocument()

    await user.click(trigger)
    expect(screen.getByRole('menuitem', { name: 'Action' })).toBeInTheDocument()
```

```
  })

  it('opens from trigger hover when openOnHover is enabled', async () => {
    const user = userEvent.setup()
    render(
      <Dropdown.Root openOnHover>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Row title="Action" />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    const trigger = screen.getByRole('button', { name: 'Open' })
    await user.hover(trigger)
    await screen.findByRole('menuitem', { name: 'Action' })
    expect(trigger).toHaveAttribute('data-open')
  })

  it('renders content (defaultOpen) with the dropdown-content class', () => {
    render(<Basic />)
    expect(screen.getByRole('menu')).toHaveClass('q-dropdown-content')
  })

  it('renders a standalone section label', () => {
    render(<Basic />)
    expect(screen.getByText('Section')).toHaveClass('q-menu-group-label')
  })

  it('applies the size preset to the popup', () => {
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content size="large"><Row title="X" /></Dropdown.Content>
      </Dropdown.Root>,
    )
    expect(screen.getByRole('menu')).toHaveClass('q-dropdown-content-large')
  })

  it('applies surface and shape presets to the popup', () => {
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content surface="solid" shape="panel"><Row title="X" /></Dropdown.Content>
      </Dropdown.Root>,
    )
    expect(screen.getByRole('menu')).toHaveClass('q-dropdown-content-solid', 'q-dropdown-content-panel')
  })
})

describe('Dropdown.Item composition', () => {
  it('is a styled row that renders arbitrary children verbatim', () => {
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Dropdown.Item value="folder">
            <Dropdown.ItemIcon><FolderIcon /></Dropdown.ItemIcon>
            <Dropdown.ItemContent>
              <Dropdown.ItemTitleRow>
                <Dropdown.ItemTitle>Folder</Dropdown.ItemTitle>
                <Dropdown.ItemMeta>18</Dropdown.ItemMeta>
              </Dropdown.ItemTitleRow>
              <Dropdown.ItemDescription>12 items</Dropdown.ItemDescription>
            </Dropdown.ItemContent>
            <Dropdown.ItemTrailing>meta</Dropdown.ItemTrailing>
          </Dropdown.Item>
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    const item = screen.getByRole('menuitem', { name: /Folder/ })
```

```
    expect(item).toHaveClass('q-menu-item')
    expect(item.querySelector('.q-menu-item-icon')).toBeInTheDocument()
    expect(item.querySelector('.q-menu-item-label')).toBeInTheDocument()
    expect(item.querySelector('.q-dropdown-item-meta')).toHaveTextContent('18')
    expect(item.querySelector('.q-menu-item-description')).toHaveTextContent('12 items')
    expect(item.querySelector('.q-menu-item-trailing')).toHaveTextContent('meta')
  })

  it('renders a rich media row (large variant via composed media tile)', () => {
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Dropdown.Item value="model">
            <Dropdown.ItemMedia><FolderIcon /></Dropdown.ItemMedia>
            <Dropdown.ItemContent>
              <Dropdown.ItemTitleRow><Dropdown.ItemTitle>Model row</Dropdown.ItemTitle></Dropdown.ItemTitleRow>
              <Dropdown.ItemDescription>Rich subtitle</Dropdown.ItemDescription>
            </Dropdown.ItemContent>
          </Dropdown.Item>
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    const item = screen.getByRole('menuitem', { name: /Model row/ })
    expect(item.querySelector('.q-dropdown-item-media')).toBeInTheDocument()
  })

  it('closes the menu when a non-selectable item is clicked', async () => {
    const user = userEvent.setup()
    function Controlled() {
      const [open, setOpen] = useState(true)
      return (
        <Dropdown.Root open={open} onOpenChange={setOpen}>
          <Dropdown.Trigger>Open</Dropdown.Trigger>
          <Dropdown.Content><Row title="Action" /></Dropdown.Content>
        </Dropdown.Root>
      )
    }
    render(<Controlled />)
    await user.click(screen.getByRole('menuitem', { name: 'Action' }))
    expect(screen.queryByRole('menuitem', { name: 'Action' })).not.toBeInTheDocument()
  })

  it('fires onSelect for a non-selectable action item', async () => {
    const user = userEvent.setup()
    const onSelect = vi.fn()
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content><Row title="Action" onSelect={onSelect} /></Dropdown.Content>
      </Dropdown.Root>,
    )
    await user.click(screen.getByRole('menuitem', { name: 'Action' }))
    expect(onSelect).toHaveBeenCalled()
  })

  it('marks a disabled item', () => {
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content><Row title="Nope" disabled /></Dropdown.Content>
      </Dropdown.Root>,
    )
    expect(screen.getByRole('menuitem', { name: 'Nope' })).toHaveAttribute('data-disabled')
  })
})

describe('Dropdown.Item slot props (ergonomic API)', () => {
  it('builds the row anatomy from start / title / subtitle / end slots', () => {
    render(
```

```
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Dropdown.Item start={<FolderIcon />} title="Soul 2.0" subtitle="Ultra-real" end="2,482" />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    const item = screen.getByRole('menuitem', { name: /Soul 2\.0/ })
    expect(item.querySelector('.q-menu-item-icon')).toBeInTheDocument()
    expect(item.querySelector('.q-menu-item-title')).toHaveTextContent('Soul 2.0')
    expect(item.querySelector('.q-menu-item-description')).toHaveTextContent('Ultra-real')
    expect(item.querySelector('.q-menu-item-trailing')).toHaveTextContent('2,482')
  })

  it('uses the media tile slot for rich rows', () => {
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content><Dropdown.Item media={<FolderIcon />} title="Model" /></Dropdown.Content>
      </Dropdown.Root>,
    )
    expect(screen.getByRole('menuitem', { name: /Model/ }).querySelector('.q-dropdown-item-media')).toBeInTheDocument()
  })

  it('auto-renders the indicator for a selectable slot row', () => {
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Dropdown.Item title="On" selectable checked />
          <Dropdown.Item title="Off" selectable checked={false} />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    expect(screen.getByRole('menuitemcheckbox', { name: /On/ }).querySelector('.q-menu-item-trailing svg')).toBeInTheDocument()
    expect(screen.getByRole('menuitemcheckbox', { name: /Off/ }).querySelector('.q-menu-item-trailing svg')).not.toBeInTheDocument()
  })

  it('renders children verbatim (no slot wrapping) when no slot prop is passed', () => {
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Dropdown.Item><span data-testid="raw">raw child</span></Dropdown.Item>
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    const item = screen.getByRole('menuitem', { name: /raw child/ })
    expect(item.querySelector('[data-testid="raw"]')).toBeInTheDocument()
    expect(item.querySelector('.q-menu-item-label')).not.toBeInTheDocument()
  })
})

describe('Dropdown.Item selectable handles its own state', () => {
  it('toggles internally with NO value and NO handlers (keyed by title)', async () => {
    const user = userEvent.setup()
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content><Dropdown.Item title="Toggle me" selectable /></Dropdown.Content>
      </Dropdown.Root>,
    )
    const item = screen.getByRole('menuitemcheckbox', { name: /Toggle me/ })
    expect(item).toHaveAttribute('aria-checked', 'false')
    await user.click(item)
    expect(screen.getByRole('menuitemcheckbox', { name: /Toggle me/ })).toHaveAttribute('aria-checked', 'true')
  })

  it('notifies via onCheckedChange even without a value', async () => {
    const user = userEvent.setup()
```

```
    const onCheckedChange = vi.fn()
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content><Dropdown.Item title="Notify" selectable onCheckedChange={onCheckedChange} /></Dropdown.Content>
      </Dropdown.Root>,
    )
    await user.click(screen.getByRole('menuitemcheckbox', { name: /Notify/ }))
    expect(onCheckedChange).toHaveBeenLastCalledWith(true, expect.anything())
  })

  it('keys Root selection off the title when no value is given', async () => {
    const user = userEvent.setup()
    const onSelected = vi.fn()
    render(
      <Dropdown.Root defaultOpen selectionMode="single" onSelected={onSelected}>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Dropdown.Item title="Alpha" selectable />
          <Dropdown.Item title="Beta" selectable />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    await user.click(screen.getByRole('menuitemcheckbox', { name: /Beta/ }))
    expect(onSelected).toHaveBeenLastCalledWith(['Beta'])
  })
})

describe('Dropdown.Item selection', () => {
  it('selectable + ItemIndicator="checkbox" uses the real Checkbox and stays open on toggle', async () => {
    const user = userEvent.setup()
    function Controlled() {
      const [checked, setChecked] = useState(false)
      return (
        <Dropdown.Root defaultOpen>
          <Dropdown.Trigger>Open</Dropdown.Trigger>
          <Dropdown.Content>
            <Row title="Toggle" selectable indicator="checkbox" checked={checked} onCheckedChange={setChecked} />
          </Dropdown.Content>
        </Dropdown.Root>
      )
    }
    render(<Controlled />)
    const item = screen.getByRole('menuitemcheckbox', { name: /Toggle/ })
    expect(item.querySelector('.q-checkbox')).toBeInTheDocument()
    await user.click(item)
    expect(screen.getByRole('menuitemcheckbox', { name: /Toggle/ })).toHaveAttribute('aria-checked', 'true')
  })

  it('selectable + ItemIndicator="switch" uses the real Switch', () => {
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Row title="Switch" selectable indicator="switch" checked />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    expect(
      screen.getByRole('menuitemcheckbox', { name: /Switch/ }).querySelector('.q-switch'),
    ).toBeInTheDocument()
  })

  it('default ItemIndicator (check) shows a trailing check only when checked', () => {
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Row title="Sel" selectable checked />
          <Row title="Unsel" selectable checked={false} />
```

```
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    expect(screen.getByRole('menuitemcheckbox', { name: /Sel/ }).querySelector('.q-menu-item-trailing svg')).toBeInTheDocument()
    expect(screen.getByRole('menuitemcheckbox', { name: /Unsel/ }).querySelector('.q-menu-item-trailing svg')).not.toBeInTheDocument()
  })
})

describe('Dropdown Root selection state', () => {
  it('manages selection internally via item value (no per-item state needed)', async () => {
    const user = userEvent.setup()
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Row value="a" title="Alpha" selectable indicator="checkbox" />
          <Row value="b" title="Beta" selectable indicator="checkbox" />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    const alpha = screen.getByRole('menuitemcheckbox', { name: /Alpha/ })
    expect(alpha).toHaveAttribute('aria-checked', 'false')
    await user.click(alpha)
    expect(screen.getByRole('menuitemcheckbox', { name: /Alpha/ })).toHaveAttribute('aria-checked', 'true')
    expect(screen.getByRole('menuitemcheckbox', { name: /Beta/ })).toHaveAttribute('aria-checked', 'false')
  })

  it('seeds from defaultSelected', () => {
    render(
      <Dropdown.Root defaultOpen defaultSelected={['b']}>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Row value="a" title="Alpha" selectable />
          <Row value="b" title="Beta" selectable />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    expect(screen.getByRole('menuitemcheckbox', { name: /Alpha/ })).toHaveAttribute('aria-checked', 'false')
    expect(screen.getByRole('menuitemcheckbox', { name: /Beta/ })).toHaveAttribute('aria-checked', 'true')
  })

  it('fires onSelected with the next array on change (subscription)', async () => {
    const user = userEvent.setup()
    const onSelected = vi.fn()
    render(
      <Dropdown.Root defaultOpen onSelected={onSelected}>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Row value="a" title="Alpha" selectable />
          <Row value="b" title="Beta" selectable />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    await user.click(screen.getByRole('menuitemcheckbox', { name: /Alpha/ }))
    expect(onSelected).toHaveBeenLastCalledWith(['a'])
    await user.click(screen.getByRole('menuitemcheckbox', { name: /Beta/ }))
    expect(onSelected).toHaveBeenLastCalledWith(['a', 'b'])
  })

  it('single selectionMode keeps only one selected', async () => {
    const user = userEvent.setup()
    render(
      <Dropdown.Root defaultOpen selectionMode="single">
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Row value="a" title="Alpha" selectable />
          <Row value="b" title="Beta" selectable />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
```

```
    await user.click(screen.getByRole('menuitemcheckbox', { name: /Alpha/ }))
    await user.click(screen.getByRole('menuitemcheckbox', { name: /Beta/ }))
    expect(screen.getByRole('menuitemcheckbox', { name: /Alpha/ })).toHaveAttribute('aria-checked', 'false')
    expect(screen.getByRole('menuitemcheckbox', { name: /Beta/ })).toHaveAttribute('aria-checked', 'true')
  })

  it('per-item checked overrides Root state', async () => {
    const user = userEvent.setup()
    const onSelected = vi.fn()
    render(
      <Dropdown.Root defaultOpen onSelected={onSelected}>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Row value="a" title="Manual" selectable checked onCheckedChange={() => {}} />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    const item = screen.getByRole('menuitemcheckbox', { name: /Manual/ })
    expect(item).toHaveAttribute('aria-checked', 'true')
    await user.click(item)
    expect(onSelected).not.toHaveBeenCalled()
  })
})

describe('Dropdown submenu (Sub / SubTrigger / SubContent)', () => {
  function WithSub({ triggerRef }: { triggerRef?: ReturnType<typeof createRef<HTMLDivElement>> }) {
    return (
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Dropdown.Sub>
            <Dropdown.SubTrigger ref={triggerRef}>
              <Dropdown.ItemContent>
                <Dropdown.ItemTitleRow><Dropdown.ItemTitle>More</Dropdown.ItemTitle></Dropdown.ItemTitleRow>
              </Dropdown.ItemContent>
              <Dropdown.ItemTrailing><Dropdown.ItemSubChevron /></Dropdown.ItemTrailing>
            </Dropdown.SubTrigger>
            <Dropdown.SubContent>
              <Row title="Nested" />
            </Dropdown.SubContent>
          </Dropdown.Sub>
        </Dropdown.Content>
      </Dropdown.Root>
    )
  }

  it('renders a submenu trigger with a chevron', () => {
    render(<WithSub />)
    const trigger = screen.getByRole('menuitem', { name: /More/ })
    expect(trigger).toHaveClass('q-menu-item', 'q-dropdown-submenu-trigger')
    expect(trigger.querySelector('.q-menu-item-trailing svg')).toBeInTheDocument()
  })

  it('forwards a ref to the submenu trigger', () => {
    const ref = createRef<HTMLDivElement>()
    render(<WithSub triggerRef={ref} />)
    expect(ref.current).toBe(screen.getByRole('menuitem', { name: /More/ }))
  })

  it('opens the submenu from a click', async () => {
    const user = userEvent.setup()
    render(<WithSub />)
    const trigger = screen.getByRole('menuitem', { name: /More/ })
    await user.click(trigger)
    expect(await screen.findByRole('menuitem', { name: /Nested/ })).toBeInTheDocument()
  })

  it('builds a submenu trigger from slot props (start / title + auto chevron)', async () => {
    const user = userEvent.setup()
    render(
```

```
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content>
          <Dropdown.Sub>
            <Dropdown.SubTrigger start={<FolderIcon />} title="Move to" />
            <Dropdown.SubContent><Row title="Inbox" /></Dropdown.SubContent>
          </Dropdown.Sub>
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    const trigger = screen.getByRole('menuitem', { name: /Move to/ })
    expect(trigger.querySelector('.q-menu-item-title')).toHaveTextContent('Move to')
    expect(trigger.querySelector('.q-menu-item-trailing svg')).toBeInTheDocument()
    await user.click(trigger)
    expect(await screen.findByRole('menuitem', { name: /Inbox/ })).toBeInTheDocument()
  })
})

describe('Dropdown search (withSearch)', () => {
  it('renders a search box and filters items live', async () => {
    const user = userEvent.setup()
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content withSearch>
          <Row title="Apple" />
          <Row title="Banana" />
          <Row title="Cherry" />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    const input = screen.getByPlaceholderText('Search')
    await user.type(input, 'ban')
    expect(screen.getByRole('menuitem', { name: 'Banana' })).toBeInTheDocument()
    expect(screen.queryByRole('menuitem', { name: 'Apple' })).not.toBeInTheDocument()
    expect(screen.queryByRole('menuitem', { name: 'Cherry' })).not.toBeInTheDocument()
  })

  it('hides a group whose items all filter out', async () => {
    const user = userEvent.setup()
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content withSearch>
          <Dropdown.Group>
            <Dropdown.Label>Fruit</Dropdown.Label>
            <Row title="Apple" />
          </Dropdown.Group>
          <Dropdown.Group>
            <Dropdown.Label>Veg</Dropdown.Label>
            <Row title="Carrot" />
          </Dropdown.Group>
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    await user.type(screen.getByPlaceholderText('Search'), 'carrot')
    expect(screen.getByText('Veg')).toBeInTheDocument()
    expect(screen.queryByText('Fruit')).not.toBeInTheDocument()
  })

  it('filters rich items via the explicit value prop', async () => {
    const user = userEvent.setup()
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content withSearch>
          <Dropdown.Item value="seedance">
            <Dropdown.ItemContent><Dropdown.ItemTitleRow><Dropdown.ItemTitle>Seedance 2.0</Dropdown.ItemTitle></Dropdown.ItemTitleRow></Dropdown.ItemContent>
          </Dropdown.Item>
          <Dropdown.Item value="kling">
```

```
            <Dropdown.ItemContent><Dropdown.ItemTitleRow><Dropdown.ItemTitle>Kling 3.0</Dropdown.ItemTitle></Dropdown.ItemTitleRow></Dropdown.ItemContent>
          </Dropdown.Item>
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    await user.type(screen.getByPlaceholderText('Search'), 'kling')
    expect(screen.getByText('Kling 3.0')).toBeInTheDocument()
    expect(screen.queryByText('Seedance 2.0')).not.toBeInTheDocument()
  })

  it('shows the default NotFound when a search matches nothing', async () => {
    const user = userEvent.setup()
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content withSearch>
          <Row title="Apple" />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    await user.type(screen.getByPlaceholderText('Search'), 'zzz')
    expect(screen.queryByRole('menuitem', { name: 'Apple' })).not.toBeInTheDocument()
    expect(screen.getByText('No results found')).toBeInTheDocument()
  })

  it('renders a custom notFound node when provided', async () => {
    const user = userEvent.setup()
    render(
      <Dropdown.Root defaultOpen>
        <Dropdown.Trigger>Open</Dropdown.Trigger>
        <Dropdown.Content withSearch notFound={<NotFound title="Nothing here" subtitle="Add a model first" />}>
          <Row title="Apple" />
        </Dropdown.Content>
      </Dropdown.Root>,
    )
    await user.type(screen.getByPlaceholderText('Search'), 'zzz')
    expect(screen.getByText('Nothing here')).toBeInTheDocument()
  })
})
```

app/packages/quanta/src/components/dropdown/dropdown.tsx

```
'use client'

import type { ComponentProps, ReactNode, Ref } from 'react'
import { IconCheckmark2MediumOutlined as CheckIcon } from '@higgsfield-ai/icons/IconCheckmark2MediumOutlined'
import { IconChevronRightMediumOutlined as ChevronRightIcon } from '@higgsfield-ai/icons/IconChevronRightMediumOutlined'
import { IconMagnifyingGlass2Outlined as SearchIcon } from '@higgsfield-ai/icons/IconMagnifyingGlass2Outlined'
import {
  createContext,
  isValidElement,
  useCallback,
  useContext,
  useEffect,
  useLayoutEffect,
  useMemo,
  useRef,
  useState,
} from 'react'
import { Menu as Primitive } from '@base-ui/react/menu'
import { Checkbox } from '../checkbox/index.ts'
import { Divider } from '../divider/index.ts'
import { NotFound } from '../not-found/index.ts'
import { Switch } from '../switch/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * Dropdown — a click-triggered menu on the Base UI `Menu` primitive (roving
 * focus, typeahead, keyboard, ARIA, portal + positioning, submenu timing),
 * skinned with quanta's `q-dropdown-*` presentation utilities (see
 * `dropdown.css`). The row shell reuses the shared `q-menu-item*` primitives
 * (see `menu.css`) so the dropdown and navigation-menu stay visually identical.
 *
 * SLOTS BY DEFAULT, composition when you need it. The common row is one tag —
 * pass `start` / `media` / `title` / `subtitle` / `badge` / `end` and `Item`
 * builds the standard anatomy (and, for `selectable` rows, the indicator) for
 * you. Drop down to the exported parts only for a bespoke layout:
 *
 *   <Dropdown.Root selectionMode="single" onSelected={([id]) => setModel(id)}>
 *     <Dropdown.Trigger render={<Button>Open</Button>} />
 *     <Dropdown.Content withSearch>
 *       <Dropdown.Group>
 *         <Dropdown.Label>Models</Dropdown.Label>
 *
 *         // easy — slot props build the row + indicator
 *         <Dropdown.Item
 *           selectable
 *           media={<img src={cover} />}
 *           title="Soul 2.0"
 *           subtitle="Ultra-real visuals"
 *           badge={<Badge variant="lime" text="new" />}
 *         />
 *         <Dropdown.Item title="Credits" end="2,482 left" />
 *
 *         // full control — compose the parts yourself
 *         <Dropdown.Item value="custom">
 *           <Dropdown.ItemContent>
 *             <Dropdown.ItemTitleRow><Dropdown.ItemTitle>Custom</Dropdown.ItemTitle></Dropdown.ItemTitleRow>
 *           </Dropdown.ItemContent>
 *         </Dropdown.Item>
 *       </Dropdown.Group>
 *     </Dropdown.Content>
 *   </Dropdown.Root>
 *
 * SELECTION STATE just works. A `selectable` Item is stateful on its own — with
 * no `value` and no handlers it toggles internally (keyed by its `title`/text
 * through `Root`, or its own state as a last resort). To be notified, pass ONE
 * handler: `onSelected` on Root (the full `string[]`) or `onCheckedChange` on
 * the Item (that row's boolean). Control it by passing `selected` on Root or
 * `checked` on the Item; either still fires the handlers.
```

```
 *
 * `withSearch` on Content adds a filter bar that hides non-matching Items (and
 * empty Groups / Subs) live. An Item's searchable text is its `value` prop, or
 * — when omitted — the plain text extracted from its children.
 */

export type DropdownIndicator = 'check' | 'checkbox' | 'switch'
export type DropdownSelectionMode = 'single' | 'multiple'
export type DropdownContentSurface = 'glass' | 'solid'
export type DropdownContentShape = 'default' | 'panel'
export type DropdownContentSize = 'compact' | 'default' | 'large'

/* ■■ Search filtering ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

const QueryContext = createContext<string>('')

/**
 * Render-time match tally. The old static walk (subtreeMatches) couldn't see
 * Items wrapped in custom components, so search broke for every composed row.
 * Instead each Item bumps this counter when it actually renders (matched), and a
 * gate rendered AFTER the list reads the total — works for any composition.
 */
const MatchCountContext = createContext<{ n: number } | null>(null)

/** Per-Group tally (same idea, scoped) so a Group can hide when all ITS rows filter out. */
const GroupCountContext = createContext<{ n: number } | null>(null)

/** Pull the plain text out of an arbitrary ReactNode (best-effort, for search). */
function extractText(node: ReactNode): string {
  if (node == null || node === false || node === true)
    return ''
  if (typeof node === 'string' || typeof node === 'number')
    return String(node)
  if (Array.isArray(node))
    return node.map(extractText).join(' ')
  if (isValidElement(node))
    return extractText((node.props as { children?: ReactNode }).children)
  return ''
}

/** All whitespace-split terms must appear (case-insensitive substring). */
function matchQuery(text: string, query: string): boolean {
  if (!query)
    return true
  const haystack = text.toLowerCase()
  return query
    .toLowerCase()
    .split(/\s+/)
    .filter(Boolean)
    .every(term => haystack.includes(term))
}

/** Searchable / selection-key text for an Item: explicit `value`, else its `title`, else its children. */
function itemText(props: ItemProps): string {
  if (props.value != null)
    return props.value
  const fromTitle = extractText(props.title)
  return fromTitle || extractText(props.children)
}

/* ■■ Selection state (owned by Root, exposed as `selected: string[]`) ■■■■■■■■■ */

interface SelectionContextValue {
  selected: string[]
  isSelected: (value: string) => boolean
  toggle: (value: string, next: boolean) => void
}

const SelectionContext = createContext<SelectionContextValue | null>(null)
```

```
interface OpenContextValue {
  open: boolean
  openOnHover: boolean
  scheduleHoverOpen: () => void
  scheduleHoverClose: () => void
  cancelHoverClose: () => void
}

const noop = () => {}

const OpenContext = createContext<OpenContextValue>({
  open: false,
  openOnHover: false,
  scheduleHoverOpen: noop,
  scheduleHoverClose: noop,
  cancelHoverClose: noop,
})

const HOVER_OPEN_DELAY = 80
const HOVER_CLOSE_DELAY = 120

/* ■■ Per-item state (lets composed parts read the row's selection/disabled) ■■ */

interface ItemContextValue {
  checked: boolean
  disabled: boolean
  selectable: boolean
  value?: string
}

const ItemContext = createContext<ItemContextValue>({
  checked: false,
  disabled: false,
  selectable: false,
})

/* ■■ Trigger ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

type TriggerProps = Omit<ComponentProps<typeof Primitive.Trigger>, 'className'> & {
  className?: string
}

function Trigger({ className, onPointerEnter, onPointerLeave, ...props }: TriggerProps) {
  const { open, openOnHover, scheduleHoverOpen, scheduleHoverClose, cancelHoverClose } = useContext(OpenContext)
  const handlePointerEnter = useCallback<NonNullable<TriggerProps['onPointerEnter']>>((event) => {
    onPointerEnter?.(event)
    if (!event.defaultPrevented && openOnHover) {
      cancelHoverClose()
      scheduleHoverOpen()
    }
  }, [cancelHoverClose, onPointerEnter, openOnHover, scheduleHoverOpen])
  const handlePointerLeave = useCallback<NonNullable<TriggerProps['onPointerLeave']>>((event) => {
    onPointerLeave?.(event)
    if (!event.defaultPrevented && openOnHover)
      scheduleHoverClose()
  }, [onPointerLeave, openOnHover, scheduleHoverClose])

  return (
    <Primitive.Trigger
      {...props}
      className={cx('q-dropdown-trigger', className)}
      data-open={open ? '' : undefined}
      data-popup-open={open ? '' : undefined}
      onPointerEnter={handlePointerEnter}
      onPointerLeave={handlePointerLeave}
    />
  )
}

/* ■■ Root ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
```

```
type RootProps = ComponentProps<typeof Primitive.Root> & {
  /** Controlled set of selected item `value`s. Pair with `onSelected`. */
  selected?: string[]
  /** Initial selection for the uncontrolled (internal-state) case. */
  defaultSelected?: string[]
  /** Subscribe to selection changes — fires with the next `string[]`. */
  onSelected?: (selected: string[]) => void
  /** 'multiple' (default) toggles items independently; 'single' keeps one. */
  selectionMode?: DropdownSelectionMode
  /** Open from pointer hover as well as click. Pointer leave closes after a short menu-safe delay. */
  openOnHover?: boolean
}

type RootOpenChangeHandler = NonNullable<RootProps['onOpenChange']>
type RootOpenChangeEvent = Parameters<RootOpenChangeHandler>[1]

/**
 * Root owns selection state. Uncontrolled by default (internal `useState`
 * seeded by `defaultSelected`); pass `selected` to control it. Either way,
 * `onSelected` fires with the next array on every change.
 */
function Root({
  open,
  defaultOpen,
  onOpenChange,
  selected,
  defaultSelected,
  onSelected,
  selectionMode = 'multiple',
  openOnHover = false,
  ...props
}: RootProps) {
  const [internal, setInternal] = useState<string[]>(defaultSelected ?? [])
  const [internalOpen, setInternalOpen] = useState(defaultOpen ?? false)
  const hoverOpenTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null)
  const hoverCloseTimeoutRef = useRef<ReturnType<typeof setTimeout> | null>(null)
  const isControlled = selected != null
  const current = isControlled ? selected : internal
  const isOpenControlled = open != null
  const currentOpen = isOpenControlled ? open : internalOpen

  const isSelected = useCallback((value: string) => current.includes(value), [current])

  const toggle = useCallback((value: string, next: boolean) => {
    const compute = (prev: string[]): string[] => {
      if (selectionMode === 'single')
        return next ? [value] : prev.filter(v => v !== value)
      if (next)
        return prev.includes(value) ? prev : [...prev, value]
      return prev.filter(v => v !== value)
    }
    if (isControlled) {
      onSelected?.(compute(current))
    }
    else {
      setInternal((prev) => {
        const nextState = compute(prev)
        onSelected?.(nextState)
        return nextState
      })
    }
  }, [current, isControlled, onSelected, selectionMode])

  const ctx = useMemo<SelectionContextValue>(
    () => ({ selected: current, isSelected, toggle }),
    [current, isSelected, toggle],
  )

  const cancelHoverOpen = useCallback(() => {
    if (hoverOpenTimeoutRef.current == null)
```

```
      return
    clearTimeout(hoverOpenTimeoutRef.current)
    hoverOpenTimeoutRef.current = null
  }, [])

  const cancelHoverClose = useCallback(() => {
    if (hoverCloseTimeoutRef.current == null)
      return
    clearTimeout(hoverCloseTimeoutRef.current)
    hoverCloseTimeoutRef.current = null
  }, [])

  const updateOpen = useCallback((next: boolean, event?: RootOpenChangeEvent) => {
    cancelHoverOpen()
    cancelHoverClose()
    if (currentOpen === next)
      return
    if (!isOpenControlled)
      setInternalOpen(next)
    onOpenChange?.(next, event as RootOpenChangeEvent)
  }, [cancelHoverClose, cancelHoverOpen, currentOpen, isOpenControlled, onOpenChange])

  const scheduleHoverOpen = useCallback(() => {
    if (!openOnHover || currentOpen)
      return
    cancelHoverClose()
    cancelHoverOpen()
    hoverOpenTimeoutRef.current = setTimeout(() => {
      hoverOpenTimeoutRef.current = null
      updateOpen(true)
    }, HOVER_OPEN_DELAY)
  }, [cancelHoverClose, cancelHoverOpen, currentOpen, openOnHover, updateOpen])
  const scheduleHoverClose = useCallback(() => {
    if (!openOnHover)
      return
    cancelHoverOpen()
    cancelHoverClose()
    hoverCloseTimeoutRef.current = setTimeout(() => updateOpen(false), HOVER_CLOSE_DELAY)
  }, [cancelHoverClose, cancelHoverOpen, openOnHover, updateOpen])

  const openCtx = useMemo<OpenContextValue>(
    () => ({ open: currentOpen, openOnHover, scheduleHoverOpen, scheduleHoverClose, cancelHoverClose }),
    [cancelHoverClose, currentOpen, openOnHover, scheduleHoverClose, scheduleHoverOpen],
  )

  const handleOpenChange = useCallback<RootOpenChangeHandler>((next, event) => {
    updateOpen(next, event as RootOpenChangeEvent)
  }, [updateOpen])

  useEffect(() => () => {
    cancelHoverOpen()
    cancelHoverClose()
  }, [cancelHoverClose, cancelHoverOpen])

  return (
    <OpenContext.Provider value={openCtx}>
      <SelectionContext.Provider value={ctx}>
        <Primitive.Root open={currentOpen} onOpenChange={handleOpenChange} {...props} />
      </SelectionContext.Provider>
    </OpenContext.Provider>
  )
}

/* ■■ Content (Portal + Positioner + Popup, optional search) ■■■■■■■■■■■■■■■■■■■■ */

type ContentProps = Omit<ComponentProps<typeof Primitive.Popup>, 'className'> & {
  className?: string
  positionerClassName?: string
  size?: DropdownContentSize
  surface?: DropdownContentSurface
```

```
  shape?: DropdownContentShape
  side?: ComponentProps<typeof Primitive.Positioner>['side']
  align?: ComponentProps<typeof Primitive.Positioner>['align']
  sideOffset?: ComponentProps<typeof Primitive.Positioner>['sideOffset']
  alignOffset?: ComponentProps<typeof Primitive.Positioner>['alignOffset']
  collisionPadding?: ComponentProps<typeof Primitive.Positioner>['collisionPadding']
  container?: ComponentProps<typeof Primitive.Portal>['container']
  /** Show a search bar that filters items live. */
  withSearch?: boolean
  searchPlaceholder?: string
  /** Rendered in place of the list when a search filters out every item. */
  notFound?: ReactNode
}

const SIZE_CLASS = {
  compact: 'q-dropdown-content-compact',
  default: '',
  large: 'q-dropdown-content-large',
} satisfies Record<DropdownContentSize, string>

const SURFACE_CLASS = {
  glass: '',
  solid: 'q-dropdown-content-solid',
} satisfies Record<DropdownContentSurface, string>

const SHAPE_CLASS = {
  default: '',
  panel: 'q-dropdown-content-panel',
} satisfies Record<DropdownContentShape, string>

function contentClass(size: DropdownContentSize, surface: DropdownContentSurface, shape: DropdownContentShape, className?: string) {
  return cx('q-dropdown-content', SIZE_CLASS[size], SURFACE_CLASS[surface], SHAPE_CLASS[shape], className)
}

function Content({
  className,
  positionerClassName,
  size = 'default',
  surface = 'glass',
  shape = 'default',
  side = 'bottom',
  align = 'start',
  sideOffset = 4,
  alignOffset,
  collisionPadding,
  container,
  withSearch = false,
  searchPlaceholder = 'Search',
  notFound,
  children,
  onPointerEnter,
  onPointerLeave,
  ...props
}: ContentProps) {
  const { openOnHover, scheduleHoverClose, cancelHoverClose } = useContext(OpenContext)
  const handlePointerEnter = useCallback<NonNullable<ContentProps['onPointerEnter']>>((event) => {
    onPointerEnter?.(event)
    if (!event.defaultPrevented && openOnHover)
      cancelHoverClose()
  }, [cancelHoverClose, onPointerEnter, openOnHover])
  const handlePointerLeave = useCallback<NonNullable<ContentProps['onPointerLeave']>>((event) => {
    onPointerLeave?.(event)
    if (!event.defaultPrevented && openOnHover)
      scheduleHoverClose()
  }, [onPointerLeave, openOnHover, scheduleHoverClose])

  return (
    <Primitive.Portal container={container}>
      <Primitive.Positioner
        className={positionerClassName}
```

```
        side={side}
        align={align}
        sideOffset={sideOffset}
        alignOffset={alignOffset}
        collisionPadding={collisionPadding}
      >
        <Primitive.Popup
          className={contentClass(size, surface, shape, className)}
          onPointerEnter={handlePointerEnter}
          onPointerLeave={handlePointerLeave}
          {...props}
        >
          <SearchableContent withSearch={withSearch} placeholder={searchPlaceholder} notFound={notFound}>
            {children}
          </SearchableContent>
        </Primitive.Popup>
      </Primitive.Positioner>
    </Primitive.Portal>
  )
}

/** Holds query state INSIDE the popup so it resets every time the menu reopens. */
function SearchableContent({
  withSearch,
  placeholder,
  notFound,
  children,
}: {
  withSearch: boolean
  placeholder: string
  notFound?: ReactNode
  children: ReactNode
}) {
  const [query, setQuery] = useState('')
  const inputRef = useRef<HTMLInputElement>(null)
  const rootRef = useRef<HTMLDivElement>(null)

  // Keep focus in the search field after Base UI's open-focus settles.
  useEffect(() => {
    if (!withSearch)
      return
    const id = requestAnimationFrame(() => inputRef.current?.focus())
    return () => cancelAnimationFrame(id)
  }, [withSearch])

  if (!withSearch)
    return <>{children}</>

  const focusFirstItem = () => {
    const first = rootRef.current?.querySelector<HTMLElement>('[role^="menuitem"]:not([data-disabled])')
    first?.focus()
  }

  // Fresh per render: Items bump it as they render; SearchEmpty (last) reads it.
  const matchCount = { n: 0 }

  return (
    <div ref={rootRef} className="q-dropdown-search-wrap">
      <div className="q-dropdown-search">
        <span className="q-dropdown-search-icon"><SearchIcon /></span>
        <input
          ref={inputRef}
          className="q-dropdown-search-input"
          placeholder={placeholder}
          value={query}
          onChange={e => setQuery(e.target.value)}
          onKeyDown={(e) => {
            // Base UI's menu treats the popup as a composite widget and
            // preventDefaults typing for typeahead. Keep the input editable by
            // stopping keystrokes from reaching the menu — except navigation
```

```
            // keys we explicitly want the menu (or us) to handle.
            if (e.key === 'ArrowDown') {
              e.preventDefault()
              focusFirstItem()
              return
            }
            if (e.key === 'Escape')
              return // let it bubble so the menu closes
            e.stopPropagation()
          }}
        />
      </div>
      <Divider className="shrink-0" />
      <QueryContext.Provider value={query}>
        <MatchCountContext.Provider value={matchCount}>
          {children}
          <SearchEmpty fallback={notFound} />
        </MatchCountContext.Provider>
      </QueryContext.Provider>
    </div>
  )
}

/** Rendered LAST in the search scope, so the match tally is complete: shows the
 * NotFound fallback iff the query matched nothing. Composition-safe — no static
 * introspection of children (which broke for wrapped Items). */
function SearchEmpty({ fallback }: { fallback?: ReactNode }) {
  const query = useContext(QueryContext)
  const matchCount = useContext(MatchCountContext)
  if (query !== '' && matchCount != null && matchCount.n === 0)
    return <>{fallback ?? <NotFound title="No results found" subtitle="Try a different search" />}</>
  return null
}

/* ■■ Separator ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

type SeparatorProps = Omit<ComponentProps<typeof Primitive.Separator>, 'className'> & {
  className?: string
}
function Separator({ className, ...props }: SeparatorProps) {
  return <Primitive.Separator render={<Divider />} className={cx('shrink-0', className)} {...props} />
}

/* ■■ Group (group; hides itself when filtered empty) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

type GroupProps = Omit<ComponentProps<typeof Primitive.Group>, 'className'> & {
  className?: string
}

function Group({ className, children, ...props }: GroupProps) {
  const query = useContext(QueryContext)
  const [empty, setEmpty] = useState(false)
  // A wrapped Item only reveals its match by rendering, so tally the Group's own
  // rendered rows (composition-safe) and hide the group when none survived. The
  // counter is fresh each render; children bump it before this layout effect reads it.
  const groupCount = { n: 0 }
  useLayoutEffect(() => {
    setEmpty(query !== '' && groupCount.n === 0)
  })
  if (empty)
    return null
  return (
    <Primitive.Group className={cx('q-dropdown-group', className)} {...props}>
      <GroupCountContext.Provider value={groupCount}>{children}</GroupCountContext.Provider>
    </Primitive.Group>
  )
}

/** Section label (Figma _MenuLabel). Standalone, or as a Group's heading.
 * Reuses the shared `q-menu-group-label` primitive (also used by cmdk). */
```

```
function Label({ className, ...props }: ComponentProps<'div'>) {
  return <div className={cx('q-menu-group-label', className)} {...props} />
}

/* ■■ Item parts (compose any row layout; all forward className/children) ■■■■■ */

type PartProps = ComponentProps<'span'>

function ItemIcon({ className, ...props }: PartProps) {
  return <span className={cx('q-menu-item-icon', className)} {...props} />
}

/** Leading media tile (image/icon, 36px) for rich rows. */
function ItemMedia({ className, ...props }: PartProps) {
  return <span className={cx('q-dropdown-item-media', className)} {...props} />
}

/** Content column — stacks the title row and description (Figma 2px gap). */
function ItemContent({ className, ...props }: PartProps) {
  return <span className={cx('q-menu-item-label', className)} {...props} />
}

function ItemTitleRow({ className, ...props }: PartProps) {
  return <span className={cx('q-menu-item-title-row', className)} {...props} />
}

function ItemTitle({ className, ...props }: PartProps) {
  return <span className={cx('q-menu-item-title', className)} {...props} />
}

function ItemDescription({ className, ...props }: PartProps) {
  return <span className={cx('q-menu-item-description', className)} {...props} />
}

/** Inline meta inside the title row (e.g. a count). */
function ItemMeta({ className, ...props }: PartProps) {
  return <span className={cx('q-dropdown-item-meta', className)} {...props} />
}

/** Trailing slot — count, indicator, chevron, button… (pushed to the right). */
function ItemTrailing({ className, ...props }: PartProps) {
  return <span className={cx('q-menu-item-trailing', className)} {...props} />
}

/** Small inset metadata chip. */
function ItemMetaChip({ className, ...props }: PartProps) {
  return <span className={cx('q-dropdown-meta-chip', className)} {...props} />
}

/** Submenu affordance chevron for use inside a SubTrigger's ItemTrailing. */
function ItemSubChevron({ className }: { className?: string }) {
  return <ChevronRightIcon className={cx('q-dropdown-check', className)} />
}

/**
 * Selection indicator for a `selectable` Item. Reads the row's checked state
 * from context, so place it anywhere (typically inside ItemTrailing). The
 * `indicator` style mirrors a real Checkbox / Switch, or a trailing check.
 */
function ItemIndicator({ indicator = 'check' }: { indicator?: DropdownIndicator }) {
  const { checked } = useContext(ItemContext)
  if (indicator === 'checkbox')
    return <Checkbox checked={checked} size="sm" tabIndex={-1} aria-hidden className="pointer-events-none" />
  if (indicator === 'switch')
    return <Switch checked={checked} size="small" tabIndex={-1} aria-hidden className="pointer-events-none" />
  return checked ? <CheckIcon className="q-dropdown-check" /> : null
}

/* ■■ Item (one styled, stateful, composition-only row) ■■■■■■■■■■■■■■■■■■■■■■■■ */
```

```
type ItemProps = Omit<ComponentProps<typeof Primitive.Item>, 'className' | 'title'> & {
  className?: string
  ref?: Ref<HTMLDivElement>
  /** Stable identity for search + Root selection. Defaults to the row's `title` / text. */
  value?: string
  /** Stateful selectable row (stays open on toggle) instead of a plain action. */
  selectable?: boolean
  disabled?: boolean
  /** Destructive row — red title + icon (Figma _MenuActions "Delete"). */
  danger?: boolean
  /** Controlled checked state (wins over Root / internal state). */
  checked?: boolean
  /** Notified on every toggle of a selectable row — controlled or not. */
  onCheckedChange?: ComponentProps<typeof Primitive.CheckboxItem>['onCheckedChange']
  /** Action handler for non-selectable items (closes on click). */
  onSelect?: ComponentProps<typeof Primitive.Item>['onClick']
  // ■■ Slot props: the ergonomic default. Pass any of these and the row builds
  //    the standard anatomy for you; pass `children` instead for full control. ■■
  /** Leading icon — small `ItemIcon` slot. */
  start?: ReactNode
  /** Leading 36px media tile (`ItemMedia` slot). Wins over `start`. */
  media?: ReactNode
  /** Primary line. Providing any slot prop switches the row to slot rendering. */
  title?: ReactNode
  /** Secondary line under the title. */
  subtitle?: ReactNode
  /** Inline node beside the title (badge, count…). */
  badge?: ReactNode
  /** Trailing content (shortcut, meta…). A selectable row's indicator is appended automatically. */
  end?: ReactNode
  /** Selection indicator style for a selectable slot row. */
  indicator?: DropdownIndicator
  children?: ReactNode
}

function Item(props: ItemProps) {
  const {
    className,
    value,
    selectable = false,
    disabled,
    danger = false,
    checked,
    onCheckedChange,
    onSelect,
    start,
    media,
    title,
    subtitle,
    badge,
    end,
    indicator = 'check',
    children,
    ...rest
  } = props

  const query = useContext(QueryContext)
  const selection = useContext(SelectionContext)
  const matchCount = useContext(MatchCountContext)
  const groupCount = useContext(GroupCountContext)
  // Internal fallback state so a `selectable` row works with NO value/handlers.
  const [selfChecked, setSelfChecked] = useState(false)
  const disabledState = disabled === true

  // Filter when searching.
  if (query && !matchQuery(itemText(props), query))
    return null

  // Survived the filter (or not searching) → tally for the NotFound gate
  // (top-level) and for its Group (so an all-filtered group hides itself).
```

```
if (matchCount)
  matchCount.n++
if (groupCount)
  groupCount.n++

// Slot rendering activates only when a slot is passed; otherwise `children`
// render verbatim (back-compat with the fully-composed API).
const hasSlots = title != null || subtitle != null || start != null || media != null || badge != null || end != null
const body = hasSlots
  ? (
      <>
        {media != null ? <ItemMedia>{media}</ItemMedia> : start != null ? <ItemIcon>{start}</ItemIcon> : null}
        {title != null || subtitle != null || badge != null
          ? (
              <ItemContent>
                <ItemTitleRow>
                  {title != null ? <ItemTitle>{title}</ItemTitle> : null}
                  {badge}
                </ItemTitleRow>
                {subtitle != null ? <ItemDescription>{subtitle}</ItemDescription> : null}
              </ItemContent>
            )
          : null}
        {end != null || selectable
          ? <ItemTrailing>{end}{selectable ? <ItemIndicator indicator={indicator} /> : null}</ItemTrailing>
          : null}
      </>
    )
  : children

// Non-selectable → plain action item (closes on click, no selection state).
if (!selectable) {
  return (
    <ItemContext.Provider value={{ checked: false, disabled: disabledState, selectable: false, value }}>
      <Primitive.Item className={cx('q-menu-item', danger && 'q-menu-item-danger', className)} disabled={disabled} onClick={onSelect} {...rest}>
        {body}
      </Primitive.Item>
    </ItemContext.Provider>
  )
}

// Selectable → stateful checkbox row, stays open on toggle. The checked state
// lives in (priority): the controlled `checked` prop → Root, keyed by the
// resolved value → the row's own internal state. `onCheckedChange` always
// fires, so a consumer only ever needs to pass a handler (never wire state).
const resolvedValue = itemText(props)
const controlled = checked !== undefined
const rootKeyed = !controlled && selection != null && resolvedValue !== ''
const resolvedChecked = controlled ? checked : rootKeyed ? selection!.isSelected(resolvedValue) : selfChecked
const handleCheckedChange: NonNullable<ItemProps['onCheckedChange']> = (next, event) => {
  if (rootKeyed)
    selection!.toggle(resolvedValue, next)
  else if (!controlled)
    setSelfChecked(next)
  onCheckedChange?.(next, event)
}

return (
  <ItemContext.Provider value={{ checked: resolvedChecked, disabled: disabledState, selectable: true, value: value ?? (resolvedValue || undefined) }}>
    <Primitive.CheckboxItem
      className={cx('q-menu-item', danger && 'q-menu-item-danger', className)}
      checked={resolvedChecked}
      onCheckedChange={handleCheckedChange}
      disabled={disabled}
      closeOnClick={false}
      {...rest}
    >
      {body}
    </Primitive.CheckboxItem>
  </ItemContext.Provider>
```

```
  )
}

/* ■■ Submenu (explicit, composable) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

function Sub({ children, ...props }: ComponentProps<typeof Primitive.SubmenuRoot>) {
  // Submenus stay visible during search (their trigger label isn't filtered);
  // nested items self-filter via their own QueryContext when the submenu opens.
  return <Primitive.SubmenuRoot {...props}>{children}</Primitive.SubmenuRoot>
}

type SubTriggerProps = Omit<ComponentProps<typeof Primitive.SubmenuTrigger>, 'className' | 'title'> & {
  className?: string
  disabled?: boolean
  // ■■ Slot props (same ergonomic default as Item; pass `children` for full control). ■■
  /** Leading icon (small `ItemIcon` slot). */
  start?: ReactNode
  /** Primary line. Providing any slot prop switches to slot rendering. */
  title?: ReactNode
  /** Secondary line under the title. */
  subtitle?: ReactNode
  /** Trailing content before the submenu chevron (which is always appended). */
  end?: ReactNode
}

function SubTrigger({ className, disabled, start, title, subtitle, end, children, ...props }: SubTriggerProps) {
  const hasSlots = title != null || subtitle != null || start != null || end != null
  return (
    <Primitive.SubmenuTrigger
      className={cx('q-menu-item', 'q-dropdown-submenu-trigger', className)}
      disabled={disabled}
      {...props}
    >
      {hasSlots
        ? (
            <>
              {start != null ? <ItemIcon>{start}</ItemIcon> : null}
              {title != null || subtitle != null
                ? (
                    <ItemContent>
                      {title != null ? <ItemTitleRow><ItemTitle>{title}</ItemTitle></ItemTitleRow> : null}
                      {subtitle != null ? <ItemDescription>{subtitle}</ItemDescription> : null}
                    </ItemContent>
                  )
                : null}
              <ItemTrailing>{end}<ItemSubChevron /></ItemTrailing>
            </>
          )
        : children}
    </Primitive.SubmenuTrigger>
  )
}

type SubContentProps = Omit<ComponentProps<typeof Primitive.Popup>, 'className'> & {
  className?: string
  /**
   * Gap (px) between the parent menu's outer edge and the nested submenu.
   * Pre-compensated for the parent's 8px content padding. Default 4.
   */
  sideOffset?: number
  /**
   * Cross-axis nudge (px). Defaults to -8 so the submenu's first row lines up
   * with the trigger (cancels the 8px content padding).
   */
  alignOffset?: number
  container?: ComponentProps<typeof Primitive.Portal>['container']
}

function SubContent({ className, sideOffset = 4, alignOffset = -8, container, children, ...props }: SubContentProps) {
  return (
```

```
    <Primitive.Portal container={container}>
      <Primitive.Positioner side="right" align="start" sideOffset={sideOffset + 8} alignOffset={alignOffset}>
        <Primitive.Popup className={cx('q-dropdown-content', className)} {...props}>
          {/* Reset the query so sub-items aren't filtered by the parent search. */}
          <QueryContext.Provider value="">{children}</QueryContext.Provider>
        </Primitive.Popup>
      </Primitive.Positioner>
    </Primitive.Portal>
  )
}

export const Dropdown = {
  Root,
  Trigger,
  Content,
  Group,
  Label,
  Separator,
  Item,
  ItemIcon,
  ItemMedia,
  ItemContent,
  ItemTitleRow,
  ItemTitle,
  ItemDescription,
  ItemMeta,
  ItemTrailing,
  ItemMetaChip,
  ItemIndicator,
  ItemSubChevron,
  Sub,
  SubTrigger,
  SubContent,
}
```

app/packages/quanta/src/components/dropdown/index.ts

```
export { Dropdown } from './dropdown.tsx'
export type {
  DropdownContentShape,
  DropdownContentSize,
  DropdownContentSurface,
  DropdownIndicator,
  DropdownSelectionMode,
} from './dropdown.tsx'
```

app/packages/quanta/src/components/glass/glass.css

```
/**
 * Glass — a convincing frosted-glass surface. Pure quanta (no Base UI primitive).
 *
 * The realism comes from LAYERING five effects (see glass.tsx JSDoc):
 *   1. backdrop-filter: blur(token) saturate(~1.6)   — saturate pops the backdrop
 *   2. translucent fill (background-glass)
 *   3. specular EDGE highlight — inset box-shadows (light top/left + dark bottom/right)
 *   4. TOP SHEEN — ::before linear-gradient (light → transparent), low opacity
 *   5. elevation drop shadow (raised)
 *
 * Token note: the medium blur (40px) maps to --hf-space-1000 exactly (the Modal /
 * Card glass radius). The sm (20px) and lg (64px) blur radii, the saturate factor,
 * and the sheen geometry/opacity have NO design token, so each is encoded as a
 * documented, caller-overridable `--q-glass-*` custom-property default inside the
 * @utility (precedent: --q-menu-min-width / --q-slider-width). FLAGGED for the
 * design-tokens owner. Colors are tokens only (background-glass, transparent-*).
 */
@source "./glass.tsx";

@utility q-glass {
  /* ■■ Tunable defaults (no token; caller-overridable knobs, FLAGGED). ■■■■■■■■ */
  --q-glass-blur: var(--hf-space-1000); /* 40px — medium; matches Modal/Card glass */
  --q-glass-saturate: 1.6; /* no token: glass saturation boost */
  --q-glass-radius: var(--hf-radius-600); /* 24px — quanta surface radius */
  --q-glass-sheen-height: 50%; /* no token: top reflection covers the upper half */
  --q-glass-sheen-color: var(--hf-color-transparent-light-15); /* glossy top reflection */

  position: relative;
  box-sizing: border-box;
  display: flex;
  flex-direction: column;
  min-width: var(--hf-space-0);
  isolation: isolate; /* keep the ::before sheen scoped above the backdrop, below content */
  overflow: clip; /* clip the sheen + tint wash to the rounded corners */

  border-radius: var(--q-glass-radius);
  background-color: var(--hf-color-background-glass);
  color: var(--hf-color-text-primary);

  /* (1) blur + (2) the saturate that makes the backdrop read like real glass. */
  backdrop-filter: blur(var(--q-glass-blur)) saturate(var(--q-glass-saturate));

  /* (3) Specular EDGE highlight — a bright 1px inset light line catching the top/
   * left edge + a faint dark inset on the bottom/right, so the rim reads as glass.
   * (1px = --hf-border-width-thin; both offsets are hairline.) */
  box-shadow:
    inset var(--hf-border-width-thin) var(--hf-border-width-thin) var(--hf-space-0) var(--hf-color-transparent-light-20),
    inset calc(-1 * var(--hf-border-width-thin)) calc(-1 * var(--hf-border-width-thin)) var(--hf-space-0) var(--hf-color-transparent-dark-15);

  transition-property: box-shadow, transform, background-color;
  transition-duration: var(--hf-duration-normal); /* 200ms */
  transition-timing-function: var(--hf-ease-out);

  /* (4) TOP SHEEN — the glossy reflection over the upper portion of the pane. */
  &::before {
    content: "";
    position: absolute;
    inset-block-start: var(--hf-space-0);
    inset-inline: var(--hf-space-0);
    height: var(--q-glass-sheen-height);
    pointer-events: none;
    z-index: -1; /* behind content, above the backdrop fill */
    background-image: linear-gradient(
      to bottom,
      var(--q-glass-sheen-color),
      transparent
    );
  }
```

```
}

/* ■■ (1) Blur radii — sm/lg have no spacing token; documented FLAGGED knobs. ■■ */
@utility q-glass-blur-sm {
  --q-glass-blur: 20px; /* no token: 20px backdrop blur (half of the 40px md) */
}
@utility q-glass-blur-md {
  --q-glass-blur: var(--hf-space-1000); /* 40px */
}
@utility q-glass-blur-lg {
  --q-glass-blur: 64px; /* no token: 64px backdrop blur (heavy frost) */
}

/* ■■ Corner radius — token-mapped steps. ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-glass-rounded-200 { --q-glass-radius: var(--hf-radius-200); } /* 8px */
@utility q-glass-rounded-300 { --q-glass-radius: var(--hf-radius-300); } /* 12px */
@utility q-glass-rounded-400 { --q-glass-radius: var(--hf-radius-400); } /* 16px */
@utility q-glass-rounded-500 { --q-glass-radius: var(--hf-radius-500); } /* 20px */
@utility q-glass-rounded-600 { --q-glass-radius: var(--hf-radius-600); } /* 24px */
@utility q-glass-rounded-full { --q-glass-radius: var(--hf-radius-full); }

/* ■■ (5) Elevation — float the pane with the shared raised drop shadow while
 * keeping the specular inset edge highlight. ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-glass-raised {
  box-shadow:
    inset var(--hf-border-width-thin) var(--hf-border-width-thin) var(--hf-space-0) var(--hf-color-transparent-light-20),
    inset calc(-1 * var(--hf-border-width-thin)) calc(-1 * var(--hf-border-width-thin)) var(--hf-space-0) var(--hf-color-transparent-dark-15),
    var(--hf-shadow-raised);
}

/* ■■ Optional faint tint — a wash of the slot color over the glass + a tinted
 * top-left edge, so the pane reads as e.g. brand glass. Driven by the --q-tint*
 * vars set inline via slotStyle(tint). ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-glass-tinted {
  background-color: color-mix(in oklch, var(--q-tint) 12%, var(--hf-color-background-glass));

  &::before {
    /* layer a faint colored sheen on top of the white gloss for a tinted edge. */
    background-image:
      linear-gradient(to bottom, color-mix(in oklch, var(--q-tint) 22%, transparent), transparent),
      linear-gradient(to bottom, var(--q-glass-sheen-color), transparent);
  }
}

/* ■■ Interactive — subtle brighten + lift on hover (and keyboard focus). ■■■■■■■■ */
@utility q-glass-interactive {
  cursor: pointer;

  &:hover {
    background-color: color-mix(in oklch, var(--hf-color-transparent-light-05), var(--hf-color-background-glass));
    transform: translateY(calc(-1 * var(--hf-space-050))); /* lift 2px */
  }

  &:focus-visible {
    outline: var(--hf-border-width-medium) solid var(--hf-color-border-focus);
    outline-offset: var(--hf-border-width-medium);
  }
}

/* Tinted + interactive hover keeps the tint, just brightens it a touch. */
.q-glass-tinted.q-glass-interactive:hover {
  background-color: color-mix(in oklch, var(--q-tint) 18%, var(--hf-color-background-glass));
}

/* Reduced motion: keep the brighten, drop the lift + transition. */
@media (prefers-reduced-motion: reduce) {
  .q-glass {
    transition-property: box-shadow, background-color;
  }
  .q-glass-interactive:hover {
```

```
    transform: none;
  }
}
```

app/packages/quanta/src/components/glass/glass.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { createRef } from 'react'
import { describe, expect, it, vi } from 'vitest'
import { Glass, glass } from './index.ts'

describe('Glass', () => {
  it('renders children inside the glass surface', () => {
    render(<Glass>Frosted</Glass>)
    const surface = screen.getByText('Frosted')
    expect(surface).toHaveClass('q-glass')
  })

  it('applies the default blur / elevation / rounded classes', () => {
    render(<Glass data-testid="g">x</Glass>)
    const surface = screen.getByTestId('g')
    expect(surface).toHaveClass('q-glass', 'q-glass-blur-md', 'q-glass-rounded-600')
    expect(surface).not.toHaveClass('q-glass-raised')
  })

  it('maps blur / elevation / rounded props to their utility classes', () => {
    render(<Glass data-testid="g" blur="lg" elevation="raised" rounded="300">x</Glass>)
    const surface = screen.getByTestId('g')
    expect(surface).toHaveClass('q-glass-blur-lg', 'q-glass-raised', 'q-glass-rounded-300')
  })

  it('opts into the interactive treatment', () => {
    render(<Glass data-testid="g" interactive>x</Glass>)
    expect(screen.getByTestId('g')).toHaveClass('q-glass-interactive')
  })

  it('applies a tint via the slot system (class + --q-tint var)', () => {
    render(<Glass data-testid="g" tint="brand">x</Glass>)
    const surface = screen.getByTestId('g')
    expect(surface).toHaveClass('q-glass-tinted')
    expect(surface.style.getPropertyValue('--q-tint')).not.toBe('')
  })

  it('does not set the tint class or var when no tint is given', () => {
    render(<Glass data-testid="g">x</Glass>)
    const surface = screen.getByTestId('g')
    expect(surface).not.toHaveClass('q-glass-tinted')
    expect(surface.style.getPropertyValue('--q-tint')).toBe('')
  })

  it('keeps the caller className last and forwards arbitrary props', () => {
    render(<Glass data-testid="g" className="custom" aria-label="panel">x</Glass>)
    const surface = screen.getByTestId('g')
    expect(surface).toHaveClass('q-glass', 'custom')
    expect(surface).toHaveAttribute('aria-label', 'panel')
  })

  it('swaps the host element via render while keeping the surface', () => {
    render(<Glass render={<article />} data-testid="g">x</Glass>)
    const surface = screen.getByTestId('g')
    expect(surface.tagName).toBe('ARTICLE')
    expect(surface).toHaveClass('q-glass')
  })

  it('forwards a click on an interactive render host', async () => {
    const user = userEvent.setup()
    const onClick = vi.fn()
    render(<Glass render={<button type="button" />} interactive onClick={onClick}>Press</Glass>)
    await user.click(screen.getByRole('button', { name: 'Press' }))
    expect(onClick).toHaveBeenCalledOnce()
  })

  it('forwards a ref to the root element', () => {
```

```
    const ref = createRef<HTMLDivElement>()
    render(<Glass ref={ref} data-testid="g">x</Glass>)
    expect(ref.current).toBe(screen.getByTestId('g'))
  })
})

describe('glass() recipe', () => {
  it('builds the default class string', () => {
    expect(glass()).toBe('q-glass q-glass-blur-md q-glass-rounded-600')
  })

  it('reflects options and appends extra classes', () => {
    expect(glass({ blur: 'sm', elevation: 'raised', rounded: 'full', interactive: true }, 'extra')).toBe(
      'q-glass q-glass-blur-sm q-glass-raised q-glass-rounded-full q-glass-interactive extra',
    )
  })
})
```

app/packages/quanta/src/components/glass/glass.tsx

```
'use client'

import type { ComponentProps, CSSProperties, ReactElement, Ref } from 'react'
import { useRender } from '@base-ui/react/use-render'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'
import { type SlotColor, slotStyle } from '../utils/slot.ts'

/**
 * Glass — a convincing "real glass" surface (frosted glassmorphism done well).
 * Pure quanta (no Base UI primitive); the host element is swappable via `render`
 * (Base UI `useRender`) so the surface can be an `<article>`, an `<a>`, a nav
 * `<header>`, etc. while keeping the glass look.
 *
 * The recipe layers MULTIPLE effects for realism (this is the whole point — see
 * glass.css for the token wiring):
 *   1. `backdrop-filter: blur(token) saturate(~1.6)` — the saturate makes the
 *      blurred backdrop pop like real glass.
 *   2. a translucent fill (`background-glass`).
 *   3. a SPECULAR EDGE HIGHLIGHT — inset box-shadows: a bright top/left light
 *      line (`transparent-light-*`) + a faint dark bottom/right, so light catches
 *      the glass edge.
 *   4. a TOP SHEEN — a `::before` linear-gradient (light → transparent) over the
 *      top portion, low opacity, for the glossy reflection.
 *   5. an elevation drop shadow for `raised`.
 *
 * `backdrop-filter` needs CONTENT BEHIND it to read — place Glass over a colorful
 * image/gradient (see stories).
 *
 *   <Glass blur="md" elevation="raised" rounded="600" interactive>
 *     …content…
 *   </Glass>
 */

export type GlassBlur = 'sm' | 'md' | 'lg'
export type GlassElevation = 'flat' | 'raised'
/** Radius token step (matches `--hf-radius-*`). */
export type GlassRounded = '200' | '300' | '400' | '500' | '600' | 'full'

export interface GlassOptions {
  /** Backdrop-blur radius (token-mapped). Default 'md'. */
  blur?: GlassBlur
  /** Drop shadow. Default 'flat'. */
  elevation?: GlassElevation
  /** Corner radius token step. Default '600' (24px — the quanta surface radius). */
  rounded?: GlassRounded
  /** Subtle hover brighten + lift. Default false. */
  interactive?: boolean
}

const BLUR_CLASS = {
  sm: 'q-glass-blur-sm',
  md: 'q-glass-blur-md',
  lg: 'q-glass-blur-lg',
} satisfies Record<GlassBlur, string>

const ELEVATION_CLASS = {
  flat: '',
  raised: 'q-glass-raised',
} satisfies Record<GlassElevation, string>

const ROUNDED_CLASS = {
  200: 'q-glass-rounded-200',
  300: 'q-glass-rounded-300',
  400: 'q-glass-rounded-400',
  500: 'q-glass-rounded-500',
  600: 'q-glass-rounded-600',
  full: 'q-glass-rounded-full',
```

```
} satisfies Record<GlassRounded, string>

/** Build the glass surface class string — usable to skin any element as glass. */
export function glass(options: GlassOptions = {}, ...extra: ClassValue[]): string {
  const { blur = 'md', elevation = 'flat', rounded = '600', interactive = false } = options
  return cx(
    'q-glass',
    BLUR_CLASS[blur],
    ELEVATION_CLASS[elevation],
    ROUNDED_CLASS[rounded],
    interactive && 'q-glass-interactive',
    ...extra,
  )
}

export type GlassProps = Omit<ComponentProps<'div'>, keyof GlassOptions> & GlassOptions & {
  /**
   * Optional faint tint applied to the glass (slot system). When set, the fill
   * and edge highlight pick up the color so the pane reads as e.g. brand glass.
   */
  tint?: SlotColor
  /**
   * Swap the root element/component while keeping the glass surface — a semantic
   * `<article>` / `<section>` / `<header>`, or a clickable `<a>` / `<button>`.
   * Defaults to a `<div>`.
   */
  render?: ReactElement
}

export function Glass({
  blur,
  elevation,
  rounded,
  interactive,
  tint,
  className,
  style,
  render,
  ref,
  ...props
}: GlassProps) {
  // `ref` must go to useRender's dedicated `ref` option (it is NOT read from
  // props), so spreading it via props would drop it from the rendered root.
  return useRender({
    render,
    defaultTagName: 'div',
    ref: ref as Ref<Element> | undefined,
    props: {
      className: glass(
        { blur, elevation, rounded, interactive },
        tint != null && 'q-glass-tinted',
        className,
      ),
      // Tint wires the private --q-tint* vars; only spread when a tint is set so
      // the neutral glass keeps its exact transparent-light highlight.
      style: tint != null ? ({ ...slotStyle(tint), ...style } as CSSProperties) : style,
      ...props,
    },
  })
}
```

app/packages/quanta/src/components/glass/index.ts

```
export { Glass, glass } from './glass.tsx'
export type {
  GlassBlur,
  GlassElevation,
  GlassOptions,
  GlassProps,
  GlassRounded,
} from './glass.tsx'
```

app/packages/quanta/src/components/grid/grid-gap.ts

```
/**
 * Shared grid gap scale — the native Tailwind spacing steps quanta exposes for
 * grids, as literal class maps (so Tailwind's scanner sees every class that can
 * render) plus a px map for the virtualizer's row math. Used by both `Grid` and
 * `VirtualGrid`.
 */

/** 1..24 — the native Tailwind gap scale steps quanta exposes. */
export type GridGap = 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 10 | 12 | 14 | 16 | 20 | 24

export const GAP_CLASS = {
  1: 'gap-1',
  2: 'gap-2',
  3: 'gap-3',
  4: 'gap-4',
  5: 'gap-5',
  6: 'gap-6',
  7: 'gap-7',
  8: 'gap-8',
  10: 'gap-10',
  12: 'gap-12',
  14: 'gap-14',
  16: 'gap-16',
  20: 'gap-20',
  24: 'gap-24',
} satisfies Record<GridGap, string>

export const GAP_X_CLASS = {
  1: 'gap-x-1',
  2: 'gap-x-2',
  3: 'gap-x-3',
  4: 'gap-x-4',
  5: 'gap-x-5',
  6: 'gap-x-6',
  7: 'gap-x-7',
  8: 'gap-x-8',
  10: 'gap-x-10',
  12: 'gap-x-12',
  14: 'gap-x-14',
  16: 'gap-x-16',
  20: 'gap-x-20',
  24: 'gap-x-24',
} satisfies Record<GridGap, string>

export const GAP_Y_CLASS = {
  1: 'gap-y-1',
  2: 'gap-y-2',
  3: 'gap-y-3',
  4: 'gap-y-4',
  5: 'gap-y-5',
  6: 'gap-y-6',
  7: 'gap-y-7',
  8: 'gap-y-8',
  10: 'gap-y-10',
  12: 'gap-y-12',
  14: 'gap-y-14',
  16: 'gap-y-16',
  20: 'gap-y-20',
  24: 'gap-y-24',
} satisfies Record<GridGap, string>

// Native Tailwind spacing: step × 0.25rem = step × 4px at the 16px root quanta
// assumes. The virtualizer needs the row gap in px to size rows correctly.
export const GAP_PX = {
  1: 4,
  2: 8,
  3: 12,
  4: 16,
```

```
  5: 20,
  6: 24,
  7: 28,
  8: 32,
  10: 40,
  12: 48,
  14: 56,
  16: 64,
  20: 80,
  24: 96,
} satisfies Record<GridGap, number>
```

app/packages/quanta/src/components/grid/grid.css

```
/**
 * Grid presentation — a pure-quanta CSS-grid layout primitive.
 *
 * `q-grid` is the base track. By default it lays out `--q-grid-cols` (default 1)
 * equal columns via `repeat(n, minmax(0, 1fr))` — the `minmax(0, 1fr)` floor lets
 * cells shrink below their content (so `min-w-0` / `truncate` work inside). The
 * column count rides on the private `--q-grid-cols` var that `Grid` sets inline.
 *
 * `q-grid-autofit` / `q-grid-autofill` swap the template to a responsive
 * `repeat(auto-fit|auto-fill, minmax(--q-grid-min, 1fr))` track — the grid fits as
 * many `--q-grid-min`-wide (default 16rem) columns as the container allows. The
 * `--q-grid-min` default is a documented component knob (overridable inline /
 * by an ancestor), the same pattern as `--q-menu-min-width` in menu.css.
 *
 * `q-grid-item` is a layout-neutral cell hook — spans/starts ride on inline
 * `grid-column` / `grid-row` placement; this class exists only as a stable
 * styling/selector anchor and carries no own visual style.
 *
 * Token note: gaps are applied as native Tailwind `gap-N` classes from the .tsx
 * (the shared spacing scale), not here — so this file owns only the track shape.
 *
 * @source points Tailwind's scanner at the colocated source (quanta is a
 * symlinked dep and Tailwind skips node_modules) — same convention as divider.css.
 */
@source "./grid.tsx";
@source "./grid-gap.ts";
@source "./virtual-grid.tsx";

@utility q-grid {
  display: grid;
  grid-template-columns: repeat(var(--q-grid-cols, 1), minmax(0, 1fr));
}

@utility q-grid-autofit {
  grid-template-columns: repeat(auto-fit, minmax(var(--q-grid-min, 16rem), 1fr));
}

@utility q-grid-autofill {
  grid-template-columns: repeat(auto-fill, minmax(var(--q-grid-min, 16rem), 1fr));
}

@utility q-grid-item {
  min-width: 0;
}

/* ■■ Virtual grid (windowed) ■■■
```

```
  will-change: transform;
}
```

app/packages/quanta/src/components/grid/grid.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Grid } from './index.ts'

describe('<Grid>', () => {
  it('renders the grid track with the base utility class', () => {
    const { container } = render(<Grid>cells</Grid>)
    const root = container.firstElementChild!
    expect(root).toHaveClass('q-grid')
    expect(root.tagName).toBe('DIV')
  })

  it('wires a fixed column count through the --q-grid-cols var (no autofit)', () => {
    const { container } = render(<Grid cols={3}>x</Grid>)
    const root = container.firstElementChild as HTMLElement
    expect(root.style.getPropertyValue('--q-grid-cols')).toBe('3')
    expect(root).not.toHaveClass('q-grid-autofit')
    expect(root).not.toHaveClass('q-grid-autofill')
  })

  it('switches to an auto-fit track and sets --q-grid-min', () => {
    const { container } = render(<Grid cols="auto-fit" minColWidth="16rem">x</Grid>)
    const root = container.firstElementChild as HTMLElement
    expect(root).toHaveClass('q-grid-autofit')
    expect(root.style.getPropertyValue('--q-grid-min')).toBe('16rem')
    // auto tracks do not pin a fixed column count
    expect(root.style.getPropertyValue('--q-grid-cols')).toBe('')
  })

  it('uses auto-fill when requested', () => {
    const { container } = render(<Grid cols="auto-fill">x</Grid>)
    expect(container.firstElementChild).toHaveClass('q-grid-autofill')
  })

  it('maps gap / flow / align / justify to native tailwind classes', () => {
    const { container } = render(
      <Grid gap={4} flow="dense" align="center" justify="stretch">x</Grid>,
    )
    expect(container.firstElementChild).toHaveClass(
      'gap-4',
      'grid-flow-row-dense',
      'items-center',
      'justify-items-stretch',
    )
  })

  it('lets per-axis gap override the shared gap', () => {
    const { container } = render(<Grid gap={4} gapX={2}>x</Grid>)
    const root = container.firstElementChild!
    expect(root).toHaveClass('gap-x-2', 'gap-y-4')
    expect(root).not.toHaveClass('gap-4')
  })

  it('forwards className (caller wins / last) and native props', () => {
    const { container } = render(<Grid className="extra" data-testid="g" aria-label="gallery">x</Grid>)
    const root = container.firstElementChild!
    expect(root).toHaveClass('q-grid', 'extra')
    expect(root).toHaveAttribute('aria-label', 'gallery')
    expect(root).toHaveAttribute('data-testid', 'g')
  })

  it('forwards ref to the root grid element', () => {
    let node: HTMLDivElement | null = null
    const { container } = render(<Grid ref={(el) => { node = el }}>x</Grid>)
    expect(node).toBe(container.firstElementChild)
    expect(node).toHaveClass('q-grid')
  })
```

```
describe('<Grid.Item>', () => {
  it('renders the cell hook class', () => {
    render(<Grid.Item>cell</Grid.Item>)
    expect(screen.getByText('cell')).toHaveClass('q-grid-item')
  })

  it('applies colSpan / rowSpan / colStart as inline grid placement', () => {
    render(<Grid.Item colSpan={2} rowSpan={3} colStart={1}>cell</Grid.Item>)
    const cell = screen.getByText('cell')
    expect(cell.style.gridColumn).toBe('span 2 / span 2')
    expect(cell.style.gridRow).toBe('span 3 / span 3')
    expect(cell.style.gridColumnStart).toBe('1')
  })

  it('omits placement styles when spans are unset', () => {
    render(<Grid.Item>cell</Grid.Item>)
    const cell = screen.getByText('cell')
    expect(cell.style.gridColumn).toBe('')
    expect(cell.style.gridRow).toBe('')
  })

  it('forwards ref + className on the item', () => {
    let node: HTMLDivElement | null = null
    render(<Grid.Item className="tile" ref={(el) => { node = el }}>cell</Grid.Item>)
    expect(node).not.toBeNull()
    expect(node).toHaveClass('q-grid-item', 'tile')
  })

  it('exposes flipKey as data-flip-key (and omits it when unset)', () => {
    const { rerender } = render(<Grid.Item flipKey="abc">cell</Grid.Item>)
    expect(screen.getByText('cell')).toHaveAttribute('data-flip-key', 'abc')
    rerender(<Grid.Item>cell</Grid.Item>)
    expect(screen.getByText('cell')).not.toHaveAttribute('data-flip-key')
  })
})

describe('animate (FLIP)', () => {
  it('renders children and survives a reorder (FLIP no-ops without WAAPI in the test DOM)', () => {
    const { rerender } = render(
      <Grid cols={3} animate data-testid="g">
        <Grid.Item flipKey="a">A</Grid.Item>
        <Grid.Item flipKey="b">B</Grid.Item>
      </Grid>,
    )
    expect(screen.getByText('A')).toHaveAttribute('data-flip-key', 'a')
    rerender(
      <Grid cols={3} animate data-testid="g">
        <Grid.Item flipKey="b">B</Grid.Item>
        <Grid.Item flipKey="a">A</Grid.Item>
      </Grid>,
    )
    expect(screen.getByTestId('g')).toHaveClass('q-grid')
    expect(screen.getByText('A')).toBeInTheDocument()
    expect(screen.getByText('B')).toBeInTheDocument()
  })
})

it('composes into a real grid (track + spanning item)', () => {
  render(
    <Grid cols={4} gap={3} data-testid="track">
      <Grid.Item colSpan={2} data-testid="featured">featured</Grid.Item>
      <Grid.Item>a</Grid.Item>
      <Grid.Item>b</Grid.Item>
    </Grid>,
  )
  const track = screen.getByTestId('track')
  expect(track).toHaveClass('q-grid', 'gap-3')
  expect((track as HTMLElement).style.getPropertyValue('--q-grid-cols')).toBe('4')
  expect(screen.getByTestId('featured').style.gridColumn).toBe('span 2 / span 2')
})
```

```
})
```

app/packages/quanta/src/components/grid/grid.tsx

```
'use client'

import type { ComponentProps, CSSProperties } from 'react'
import { Children, isValidElement, useMemo } from 'react'
import { GAP_CLASS, GAP_X_CLASS, GAP_Y_CLASS, type GridGap } from './grid-gap.ts'
import { useFlip } from '../utils/use-flip.ts'
import { cx } from '../utils/cx.ts'

export type { GridGap }

/**
 * Grid — a pure-quanta CSS-grid layout primitive covering "all kinds of grids".
 * No Base UI: it is a thin, token-clean wrapper over `display: grid` that lets
 * you express column counts, auto-fit/auto-fill responsive tracks, gaps, flow,
 * and item alignment without ever inlining an arbitrary value.
 *
 * Parts: `Grid` (the track) + `Grid.Item` (an optional cell that can span).
 *
 *   <Grid cols={3} gap={4}>…</Grid>                  // 3 equal columns
 *   <Grid cols="auto-fit" minColWidth="16rem" gap={4}>…</Grid>  // responsive
 *   <Grid cols={4} gap={3}>
 *     <Grid.Item colSpan={2} rowSpan={2}>featured</Grid.Item>
 *     …
 *   </Grid>
 *
 * Column count is wired through the private `--q-grid-cols` / `--q-grid-min`
 * CSS vars (the sanctioned dynamic-style escape hatch — like `--q-slider-width`),
 * read by the `q-grid` utility. The gap uses the native Tailwind `gap-N` scale
 * (mapped exhaustively below), so spacing stays on the shared spacing scale.
 *
 * RESPONSIVE NOTE: per-breakpoint `cols` objects ({ base, tablet, desktop }) are
 * intentionally OUT OF SCOPE here — use `cols="auto-fit"` + `minColWidth` for
 * fluid layouts (the columns reflow automatically), or set `--q-grid-cols` per
 * breakpoint on a wrapper. FLAGGED in the manifest.
 */

export type GridCols = number | 'auto-fit' | 'auto-fill'
export type GridFlow = 'row' | 'col' | 'dense'
export type GridAlign = 'start' | 'center' | 'end' | 'stretch'
export type GridJustify = 'start' | 'center' | 'end' | 'stretch'
/** 1..12 — the span steps for `Grid.Item`. */
export type GridSpan = 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12

export type GridProps = ComponentProps<'div'> & {
  /**
   * A fixed number of equal columns, or `'auto-fit'`/`'auto-fill'` to lay out as
   * many `minColWidth`-wide columns as fit. Defaults to a single column.
   */
  cols?: GridCols
  /** Min column width for `auto-fit`/`auto-fill` tracks. Token-based length. */
  minColWidth?: string
  /** Native Tailwind gap step applied to both axes. Ignored if gapX/gapY set. */
  gap?: GridGap
  /** Native Tailwind column-gap step (overrides `gap` horizontally). */
  gapX?: GridGap
  /** Native Tailwind row-gap step (overrides `gap` vertically). */
  gapY?: GridGap
  /** grid-auto-flow. */
  flow?: GridFlow
  /** align-items. */
  align?: GridAlign
  /** justify-items. */
  justify?: GridJustify
  /**
   * FLIP-animate the cells when the layout changes (reorder / filter / add).
   * Give each animating child a stable `flipKey` (e.g. `Grid.Item flipKey={id}`)
   * so the hook can match cells across renders. Honors `prefers-reduced-motion`.
   */
```

```
  animate?: boolean
}

const FLOW_CLASS = {
  row: 'grid-flow-row',
  col: 'grid-flow-col',
  dense: 'grid-flow-row-dense',
} satisfies Record<GridFlow, string>

const ALIGN_CLASS = {
  start: 'items-start',
  center: 'items-center',
  end: 'items-end',
  stretch: 'items-stretch',
} satisfies Record<GridAlign, string>

const JUSTIFY_CLASS = {
  start: 'justify-items-start',
  center: 'justify-items-center',
  end: 'justify-items-end',
  stretch: 'justify-items-stretch',
} satisfies Record<GridJustify, string>

function Root({
  cols = 1,
  minColWidth,
  gap,
  gapX,
  gapY,
  flow,
  align,
  justify,
  animate = false,
  className,
  style,
  ref,
  children,
  ...props
}: GridProps) {
  const autoTrack = cols === 'auto-fit' || cols === 'auto-fill'

  // FLIP key: the children's flipKey/key sequence. When it changes (reorder /
  // filter / add), useFlip animates the cells from their old boxes; null when
  // not animating, so the effect is a cheap no-op.
  const flipDependency = useMemo(() => {
    if (!animate) return null
    return Children.toArray(children)
      .map(child => (isValidElement(child) ? String((child.props as { flipKey?: unknown }).flipKey ?? child.key ?? '') : ''))
      .join('|')
  }, [animate, children])
  const flipRef = useFlip<HTMLDivElement>(flipDependency)
  // Dynamic track config rides on private CSS vars — the sanctioned escape hatch
  // for values that can't be a static class (precedent: --q-slider-width).
  const gridVars = {
    ...(autoTrack ? null : { '--q-grid-cols': cols }),
    ...(minColWidth != null ? { '--q-grid-min': minColWidth } : null),
  } as CSSProperties

  // Per-axis gap wins over the single `gap` on that axis; fall back to `gap`.
  const gapClasses
    = gapX != null || gapY != null
      ? cx(
          gapX != null ? GAP_X_CLASS[gapX] : gap != null ? GAP_X_CLASS[gap] : undefined,
          gapY != null ? GAP_Y_CLASS[gapY] : gap != null ? GAP_Y_CLASS[gap] : undefined,
        )
      : gap != null
        ? GAP_CLASS[gap]
        : undefined

  return (
```

```
    <div
      className={cx(
        'q-grid',
        cols === 'auto-fill' ? 'q-grid-autofill' : autoTrack ? 'q-grid-autofit' : undefined,
        gapClasses,
        flow != null ? FLOW_CLASS[flow] : undefined,
        align != null ? ALIGN_CLASS[align] : undefined,
        justify != null ? JUSTIFY_CLASS[justify] : undefined,
        className,
      )}
      style={{ ...gridVars, ...style }}
      ref={animate ? flipRef : ref}
      {...props}
    >
      {children}
    </div>
  )
}

export type GridItemProps = ComponentProps<'div'> & {
  /** Number of columns this cell spans (1..12). */
  colSpan?: GridSpan
  /** Number of rows this cell spans (1..12). */
  rowSpan?: GridSpan
  /** 1-based column line this cell starts on. */
  colStart?: GridSpan
  /** Stable id for FLIP layout animation (see Grid `animate`). Sets `data-flip-key`. */
  flipKey?: string | number
}

function Item({ colSpan, rowSpan, colStart, flipKey, className, style, ...props }: GridItemProps) {
  // Span/start are positional integers, not on any token scale — they ride on
  // inline grid placement (the sanctioned dynamic-style escape hatch).
  const placement = {
    ...(colSpan != null ? { gridColumn: `span ${colSpan} / span ${colSpan}` } : null),
    ...(rowSpan != null ? { gridRow: `span ${rowSpan} / span ${rowSpan}` } : null),
    ...(colStart != null ? { gridColumnStart: colStart } : null),
  } as CSSProperties

  return (
    <div
      className={cx('q-grid-item', className)}
      style={{ ...placement, ...style }}
      data-flip-key={flipKey != null ? String(flipKey) : undefined}
      {...props}
    />
  )
}

export const Grid = Object.assign(Root, { Item })
```

app/packages/quanta/src/components/grid/index.ts

```
export { Grid } from './grid.tsx'
export type {
  GridAlign,
  GridCols,
  GridFlow,
  GridGap,
  GridItemProps,
  GridJustify,
  GridProps,
  GridSpan,
} from './grid.tsx'

export { VirtualGrid } from './virtual-grid.tsx'
export type { VirtualGridItemMeta, VirtualGridProps } from './virtual-grid.tsx'

export { useGridVirtualizer } from './use-grid-virtualizer.ts'
export type { UseGridVirtualizerOptions } from './use-grid-virtualizer.ts'

// Re-exported viewport/animation primitives (also used by Media.Video).
export { useInView } from '../utils/use-in-view.ts'
export type { UseInViewOptions } from '../utils/use-in-view.ts'
export { useFlip } from '../utils/use-flip.ts'
export type { UseFlipOptions } from '../utils/use-flip.ts'
```

app/packages/quanta/src/components/grid/use-grid-virtualizer.ts

```
'use client'

import { useEffect, useRef, useState } from 'react'

/**
 * useGridVirtualizer — headless windowing for a UNIFORM grid (equal cells). It
 * chunks `count` items into rows of `columns`, then — off the scroll container's
 * `scrollTop`/`clientHeight` — renders only the rows in view plus `overscan`
 * rows above and below. Zero dependencies; the uniform-grid case needs no
 * per-item measurement, just row arithmetic.
 *
 * Attach the returned `scrollRef` to the scroll container (fixed height,
 * `overflow:auto`). Give an inner sizer the returned `totalHeight` so the
 * scrollbar reflects the full set, and translate the rendered block by
 * `offsetY`. Render `items.slice(start, end)`.
 *
 * Scroll handling is rAF-throttled and passive; SSR/test safe (no rAF →
 * synchronous measure).
 */
export type UseGridVirtualizerOptions = {
  /** Total number of items. */
  count: number
  /** Columns per row (>= 1). */
  columns: number
  /** Estimated cell (row) height in px — the row gap is added on top. */
  rowHeight: number
  /** Row gap in px (match the visual gap so the math lines up). Default 0. */
  rowGap?: number
  /** Extra rows rendered above/below the viewport. Default 3. */
  overscan?: number
  /**
   * SUSTAINED scroll speed (px/ms, smoothed) above which `isScrolling` flips true
   * so cells can defer expensive work (image/video/API loads) during a fast
   * fling. Smooth/slow scrolling stays below it (loads keep happening, buffered
   * by `overscan`); as soon as a fast scroll *slows down* the smoothed speed
   * drops back under it and loads resume. Set `0` to never defer. Default 3.
   */
  velocityThreshold?: number
}

export function useGridVirtualizer<T extends HTMLElement = HTMLDivElement>(options: UseGridVirtualizerOptions) {
  const { count, columns, rowHeight, rowGap = 0, overscan = 3, velocityThreshold = 3 } = options
  const scrollRef = useRef<T>(null)

  const cols = Math.max(1, columns)
  const rowStride = rowHeight + rowGap
  const rowCount = Math.ceil(count / cols)
  const totalHeight = rowCount > 0 ? rowCount * rowStride - rowGap : 0

  // Seed with a first window so SSR / first paint shows content immediately.
  const [view, setView] = useState({
    start: 0,
    end: Math.min(count, cols * (overscan + 2)),
    offsetY: 0,
    isScrolling: false,
  })

  useEffect(() => {
    const el = scrollRef.current
    if (el == null) return
    const clock = () => (typeof performance !== 'undefined' ? performance.now() : 0)
    let frame = 0
    let settle: ReturnType<typeof setTimeout> | undefined
    let lastTop = el.scrollTop
    let lastTime = clock()
    let smoothVelocity = 0

    const measure = () => {
```

```
      frame = 0
      const scrollTop = el.scrollTop
      const viewport = el.clientHeight || rowStride
      const firstVisible = Math.floor(scrollTop / rowStride)
      const lastVisible = Math.ceil((scrollTop + viewport) / rowStride)
      // Clamp to real rows so a shrunk dataset / over-large scrollTop can't push
      // the window past the end and blank the grid.
      const maxRow = Math.max(0, rowCount - 1)
      const firstRow = Math.min(maxRow, Math.max(0, firstVisible - overscan))
      const lastRow = Math.min(maxRow, lastVisible + overscan)

      // Smoothed scroll speed (px/ms): an EMA so a single-frame spike or the
      // first frame after idle can't trip the gate — only SUSTAINED fast
      // scrolling does, and slowing back down decays it under the threshold so
      // loads resume before the user even stops.
      const time = clock()
      // Floor dt at ~one 120Hz frame so a tiny interval (unthrottled rAF / high-
      // refresh display) can't inflate the speed into a false fast-scroll trip.
      const dt = Math.max(time - lastTime, 8)
      const instant = Math.abs(scrollTop - lastTop) / dt
      lastTop = scrollTop
      lastTime = time
      smoothVelocity = smoothVelocity * 0.7 + instant * 0.3
      const fast = velocityThreshold > 0 && smoothVelocity > velocityThreshold

      const start = firstRow * cols
      const end = Math.min(count, (lastRow + 1) * cols)
      const offsetY = firstRow * rowStride
      // Commit ONLY on a real change — returning the previous reference makes
      // React bail out, so steady/slow scrolling within a row doesn't re-render
      // (and re-run renderItem for) the whole window every frame. This identity
      // churn was what made the cells flash on every scroll move.
      setView(prev =>
        prev.start === start && prev.end === end && prev.offsetY === offsetY && prev.isScrolling === fast
          ? prev
          : { start, end, offsetY, isScrolling: fast },
      )

      // When the fling decays or the user lifts off, clear the flag shortly after
      // so loads resume even if the last sampled frame was still fast.
      if (settle !== undefined) clearTimeout(settle)
      if (fast) settle = setTimeout(() => setView(v => (v.isScrolling ? { ...v, isScrolling: false } : v)), 140)
    }

    const onScroll = () => {
      if (typeof requestAnimationFrame === 'undefined') return measure()
      if (frame === 0) frame = requestAnimationFrame(measure)
    }

    measure()
    el.addEventListener('scroll', onScroll, { passive: true })
    return () => {
      el.removeEventListener('scroll', onScroll)
      if (frame !== 0 && typeof cancelAnimationFrame !== 'undefined') cancelAnimationFrame(frame)
      if (settle !== undefined) clearTimeout(settle)
    }
  }, [count, cols, rowStride, rowCount, overscan, velocityThreshold])

  return {
    scrollRef,
    totalHeight,
    start: view.start,
    end: view.end,
    offsetY: view.offsetY,
    isScrolling: view.isScrolling,
    rowCount,
    columns: cols,
  }
}
```

app/packages/quanta/src/components/grid/virtual-grid.test.tsx

```
import { fireEvent, render, screen } from '@testing-library/react'
import { describe, expect, it, vi } from 'vitest'
import { VirtualGrid } from './index.ts'

interface Photo { id: number, label: string }
const ITEMS: Photo[] = Array.from({ length: 100 }, (_, i) => ({ id: i, label: `item-${i}` }))

function renderCell(p: Photo) {
  return <span data-testid="cell">{p.label}</span>
}

describe('<VirtualGrid>', () => {
  it('renders the viewport, the full-height sizer, and the grid track', () => {
    const { container } = render(
      <VirtualGrid items={ITEMS} cols={2} rowHeight={100} getKey={p => p.id} renderItem={renderCell} />,
    )
    expect(container.querySelector('.q-virtual-grid')).not.toBeNull()
    const sizer = container.querySelector('.q-virtual-grid-sizer') as HTMLElement
    expect(sizer).not.toBeNull()
    // 100 items / 2 cols = 50 rows → the sizer reserves the full scroll height.
    expect(Number.parseInt(sizer.style.height, 10)).toBeGreaterThan(1000)
    expect(container.querySelector('.q-grid.q-virtual-grid-track')).not.toBeNull()
  })

  it('mounts only a windowed subset, not the whole list', () => {
    render(<VirtualGrid items={ITEMS} cols={2} rowHeight={100} getKey={p => p.id} renderItem={renderCell} />)
    const cells = screen.getAllByTestId('cell')
    expect(cells.length).toBeGreaterThan(0)
    expect(cells.length).toBeLessThan(ITEMS.length)
    expect(screen.getByText('item-0')).toBeInTheDocument()
    expect(screen.queryByText('item-99')).not.toBeInTheDocument()
  })

  it('pins the fixed column count and the uniform row height on the track', () => {
    const { container } = render(
      <VirtualGrid items={ITEMS} cols={3} rowHeight={120} renderItem={renderCell} />,
    )
    const track = container.querySelector('.q-virtual-grid-track') as HTMLElement
    expect(track.style.getPropertyValue('--q-grid-cols')).toBe('3')
    expect(track.style.gridAutoRows).toBe('120px')
  })

  it('wraps each visible item in a q-grid-item and applies the gap class', () => {
    const { container } = render(
      <VirtualGrid items={ITEMS.slice(0, 6)} cols={2} rowHeight={100} gap={6} renderItem={renderCell} />,
    )
    expect(container.querySelectorAll('.q-grid-item').length).toBeGreaterThan(0)
    expect(container.querySelector('.q-virtual-grid-track')).toHaveClass('gap-6')
  })

  it('falls back to the index key without getKey', () => {
    render(<VirtualGrid items={ITEMS.slice(0, 3)} cols={1} rowHeight={80} renderItem={renderCell} />)
    expect(screen.getByText('item-0')).toBeInTheDocument()
  })

  it('passes an isScrolling meta to renderItem (false at rest, so content loads)', () => {
    const seen: boolean[] = []
    render(
      <VirtualGrid
        items={ITEMS.slice(0, 4)}
        cols={2}
        rowHeight={100}
        renderItem={(p, _i, meta) => {
          seen.push(meta.isScrolling)
          return <span data-testid="cell">{p.label}</span>
        }}
      />,
    )
```

```
    expect(seen.length).toBeGreaterThan(0)
    expect(seen.every(v => v === false)).toBe(true)
  })

  it('calls onEndReached from the scroll viewport once per item count', () => {
    const onEndReached = vi.fn()
    const renderGrid = (items: Photo[]) => (
      <VirtualGrid
        items={items}
        cols={2}
        rowHeight={100}
        endReachedThresholdPx={100}
        onEndReached={onEndReached}
        renderItem={renderCell}
      />
    )
    const { container, rerender } = render(renderGrid(ITEMS.slice(0, 4)))
    const viewport = container.querySelector('.q-virtual-grid') as HTMLElement

    // A short initial list fills itself once; repeated scroll events do not
    // duplicate the same page request.
    expect(onEndReached).toHaveBeenCalledOnce()
    fireEvent.scroll(viewport)
    fireEvent.scroll(viewport)
    expect(onEndReached).toHaveBeenCalledOnce()

    Object.defineProperties(viewport, {
      clientHeight: { configurable: true, value: 200 },
      scrollHeight: { configurable: true, value: 1000 },
    })
    rerender(renderGrid(ITEMS.slice(0, 6)))
    expect(onEndReached).toHaveBeenCalledOnce()

    viewport.scrollTop = 699
    fireEvent.scroll(viewport)
    expect(onEndReached).toHaveBeenCalledOnce()
    viewport.scrollTop = 700
    fireEvent.scroll(viewport)
    fireEvent.scroll(viewport)
    expect(onEndReached).toHaveBeenCalledTimes(2)
  })
})
```

app/packages/quanta/src/components/grid/virtual-grid.tsx

```
'use client'

import type { CSSProperties, ReactNode } from 'react'
import { useEffect, useRef, useState } from 'react'
import { GAP_CLASS, GAP_PX, type GridGap } from './grid-gap.ts'
import { useGridVirtualizer } from './use-grid-virtualizer.ts'
import { cx } from '../utils/cx.ts'

/**
 * VirtualGrid — a windowed, data-driven uniform grid for big feeds & galleries.
 * Where `Grid` lays out arbitrary children, VirtualGrid takes an `items` array +
 * `renderItem` and renders ONLY the rows in view (plus `overscan` rows) via
 * `useGridVirtualizer`, so 10k cells cost the same as a screenful.
 *
 *   <VirtualGrid
 *     items={photos} rowHeight={220} minColWidth={180} gap={3} overscan={4}
 *     height="40rem" getKey={p => p.id}
 *     renderItem={p => <Media ratio="square"><Media.Image src={p.src} /></Media>}
 *   />
 *
 * Uniform rows: each cell is laid out at exactly `rowHeight` (`grid-auto-rows`),
 * which is what lets the scroll math stay exact without measuring every cell.
 * Columns are `cols` (fixed) or derived from the measured width + `minColWidth`
 * (ResizeObserver). Pair with `Media.Video autoPlayInView` for a feed that only
 * plays the clips actually on screen.
 */
/** Per-cell render context — lets a cell defer expensive work during fast scroll. */
export type VirtualGridItemMeta = {
  /** True while the user is flinging fast enough to defer image/video/API loads. */
  isScrolling: boolean
}

export type VirtualGridProps<Item> = {
  /** The full dataset. Only the visible window is rendered. */
  items: readonly Item[]
  /**
   * Render one cell (wrapped in a `q-grid-item`). `meta.isScrolling` is true
   * during a fast fling — render a cheap placeholder then and load the real
   * image/video/fetch only when it is false (slow scroll or settled), so a fast
   * scroll fires no requests.
   */
  renderItem: (item: Item, index: number, meta: VirtualGridItemMeta) => ReactNode
  /** Stable React key per item. Defaults to the index. */
  getKey?: (item: Item, index: number) => string | number
  /** Fixed column count. Omit and set `minColWidth` for responsive columns. */
  cols?: number
  /** Min column width in px — columns are derived from the measured width. */
  minColWidth?: number
  /** Cell (row) height in px — drives `grid-auto-rows` and the scroll math. */
  rowHeight: number
  /** Gap on both axes (shared GridGap scale). Default 4. */
  gap?: GridGap
  /** Extra rows rendered above/below the viewport. Default 3. */
  overscan?: number
  /** Scroll speed (px/ms) above which `meta.isScrolling` defers loads. Default 1.5; `0` disables. */
  velocityThreshold?: number
  /** Height of the scroll viewport (CSS length). Default `32rem`. */
  height?: string
  /** Called once per item count when the viewport reaches the end threshold. */
  onEndReached?: () => void
  /** Distance from the viewport end that triggers `onEndReached`. Default 400px. */
  endReachedThresholdPx?: number
  /** Class for the inner grid track. */
  className?: string
  /** Class for the scroll viewport. */
  viewportClassName?: string
  style?: CSSProperties
}
```

```
export function VirtualGrid<Item>({
  items,
  renderItem,
  getKey,
  cols,
  minColWidth,
  rowHeight,
  gap = 4,
  overscan = 3,
  velocityThreshold,
  height = '32rem',
  onEndReached,
  endReachedThresholdPx = 400,
  className,
  viewportClassName,
  style,
}: VirtualGridProps<Item>) {
  const gapPx = GAP_PX[gap]

  // Responsive column count derived from the measured viewport width when `cols`
  // isn't fixed — `floor((width + gap) / (minColWidth + gap))`.
  const [width, setWidth] = useState(0)
  const columns
    = cols
      ?? (minColWidth != null && width > 0 ? Math.max(1, Math.floor((width + gapPx) / (minColWidth + gapPx))) : 1)

  const { scrollRef, totalHeight, start, end, offsetY, isScrolling } = useGridVirtualizer<HTMLDivElement>({
    count: items.length,
    columns,
    rowHeight,
    rowGap: gapPx,
    overscan,
    velocityThreshold,
  })
  const endReachedForCount = useRef<number | undefined>(undefined)

  useEffect(() => {
    if (cols != null || minColWidth == null) return
    const el = scrollRef.current
    if (el == null || typeof ResizeObserver === 'undefined') return
    const ro = new ResizeObserver(() => setWidth(el.clientWidth))
    ro.observe(el)
    setWidth(el.clientWidth)
    return () => ro.disconnect()
  }, [cols, minColWidth, scrollRef])

  useEffect(() => {
    if (!onEndReached) {
      endReachedForCount.current = undefined
      return
    }
    const el = scrollRef.current
    if (el == null) return
    const threshold = Math.max(0, endReachedThresholdPx)
    const maybeReachEnd = () => {
      const remaining = el.scrollHeight - el.scrollTop - el.clientHeight
      if (remaining > threshold || endReachedForCount.current === items.length) return
      endReachedForCount.current = items.length
      onEndReached()
    }
    maybeReachEnd()
    el.addEventListener('scroll', maybeReachEnd, { passive: true })
    return () => el.removeEventListener('scroll', maybeReachEnd)
  }, [endReachedThresholdPx, items.length, onEndReached, scrollRef])

  const cells: ReactNode[] = []
  for (let i = start; i < end; i++) {
    const item = items[i]
    if (item === undefined) continue
    cells.push(
```

```
      <div key={getKey ? getKey(item, i) : i} className="q-grid-item">
        {renderItem(item, i, { isScrolling })}
      </div>,
    )
  }

  return (
    <div ref={scrollRef} className={cx('q-virtual-grid', viewportClassName)} style={{ height, ...style }}>
      <div className="q-virtual-grid-sizer" style={{ height: totalHeight }}>
        <div
          className={cx('q-grid', 'q-virtual-grid-track', GAP_CLASS[gap], className)}
          style={{
            transform: `translateY(${offsetY}px)`,
            gridAutoRows: `${rowHeight}px`,
            '--q-grid-cols': columns,
          } as CSSProperties}
        >
          {cells}
        </div>
      </div>
    </div>
  )
}
```

app/packages/quanta/src/components/icon/icon.css

```
/**
 * Icon presentation — the standardized glyph sizing.
 *
 * NODE-ONLY: `q-icon` is applied DIRECTLY to the glyph `<svg>` (no wrapper). It
 * sizes the svg to a fixed `--hf-icon-<size>` square (the size scale lives in the
 * codegen-emitted primitives/icon.css → tailwind/icon.css). Setting `width`/
 * `height` as CSS properties overrides a glyph's intrinsic `width="24"`/
 * `height="24"` attributes, so every icon renders at the requested token size.
 * This replaces the per-component `& svg { width/height }` rules.
 *
 * `display: inline-flex` keeps the svg an inline-level box (so it flows beside
 * text exactly like the previous wrapper did) and zeroes the inline-baseline
 * descender whitespace that a default `display: inline` svg would add — matching
 * the old wrapper's `line-height: 0`.
 *
 * Color is left to `currentColor` (the glyphs use `stroke`/`fill="currentColor"`)
 * — Icon either inherits the surrounding text color or sets it via a
 * `text-q-icon-*` utility from the `color` prop (see icon.tsx).
 *
 * Tailwind skips node_modules and quanta is symlinked, so the literal class
 * strings in icon.tsx (`q-icon-md`, `text-q-icon-primary`, …) must be registered
 * via @source here, same convention as the other components.
 */
@source "./icon.tsx";

@utility q-icon {
  --q-icon-size: var(--hf-icon-md);

  display: inline-flex;
  flex-shrink: 0;
  width: var(--q-icon-size);
  height: var(--q-icon-size);
}

@utility q-icon-xs {
  --q-icon-size: var(--hf-icon-xs);
}

@utility q-icon-sm {
  --q-icon-size: var(--hf-icon-sm);
}

@utility q-icon-md {
  --q-icon-size: var(--hf-icon-md);
}

@utility q-icon-lg {
  --q-icon-size: var(--hf-icon-lg);
}

@utility q-icon-xl {
  --q-icon-size: var(--hf-icon-xl);
}
```

app/packages/quanta/src/components/icon/icon.test.tsx

```
import type { Ref, SVGProps } from 'react'
import { createRef } from 'react'
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Icon, icon } from './index.ts'

/**
 * A minimal stand-in glyph that mirrors how @higgsfield-ai/icons render: it
 * spreads incoming props onto the <svg> last. Unlike the real package glyphs it
 * also forwards `ref` to the svg, so the ref-forwarding test exercises a glyph
 * that is capable of receiving it.
 */
function Glyph({ ref, ...props }: SVGProps<SVGSVGElement> & { ref?: Ref<SVGSVGElement> }) {
  return (
    <svg ref={ref} viewBox="0 0 24 24" width={24} height={24} {...props}>
      <path stroke="currentColor" d="M5 13.875 9.2 18 19 7" />
    </svg>
  )
}

describe('<Icon>', () => {
  it('renders the glyph itself as the single element (no wrapper) — default md, decorative', () => {
    const { container } = render(<Icon as={Glyph} />)
    const svg = container.querySelector('svg')
    // the svg IS the root: no wrapping span around it
    expect(container.firstElementChild).toBe(svg)
    expect(svg).toHaveClass('q-icon', 'q-icon-md')
    expect(svg).toHaveAttribute('aria-hidden', 'true')
  })

  it('renders the glyph passed as children, painting q-icon onto its svg', () => {
    const { container } = render(<Icon><Glyph /></Icon>)
    const svg = container.querySelector('svg')
    expect(container.firstElementChild).toBe(svg)
    expect(svg).toHaveClass('q-icon', 'q-icon-md')
  })

  it('maps each size to its token utility class on the svg', () => {
    const cases = [
      ['xs', 'q-icon-xs'],
      ['sm', 'q-icon-sm'],
      ['md', 'q-icon-md'],
      ['lg', 'q-icon-lg'],
      ['xl', 'q-icon-xl'],
    ] as const

    for (const [size, cls] of cases) {
      const { container, unmount } = render(<Icon as={Glyph} size={size} />)
      expect(container.querySelector('svg')).toHaveClass('q-icon', cls)
      unmount()
    }
  })

  it('inherits currentColor by default (no color utility)', () => {
    const { container } = render(<Icon as={Glyph} />)
    expect(container.querySelector('svg')?.getAttribute('class')).not.toMatch(/text-q-icon-/)
  })

  it('applies a quanta icon color token when color is set', () => {
    const { container } = render(<Icon as={Glyph} color="brand" />)
    expect(container.querySelector('svg')).toHaveClass('text-q-icon-brand')
  })

  it('exposes an accessible image when a label is given', () => {
    render(<Icon as={Glyph} label="Search" />)
    const img = screen.getByRole('img', { name: 'Search' })
    expect(img.tagName.toLowerCase()).toBe('svg')
    expect(img).toHaveClass('q-icon')
```

```
    expect(img).not.toHaveAttribute('aria-hidden')
  })

  it('is hidden from assistive tech when unlabelled', () => {
    render(<Icon as={Glyph} />)
    expect(screen.queryByRole('img')).not.toBeInTheDocument()
  })

  it('forwards ref to the glyph svg', () => {
    const ref = createRef<SVGSVGElement>()
    render(<Icon as={Glyph} ref={ref} />)
    expect(ref.current).toBeInstanceOf(SVGSVGElement)
    expect(ref.current).toHaveClass('q-icon')
  })

  it('forwards className last (caller wins ordering) onto the svg', () => {
    const { container } = render(<Icon as={Glyph} className="is-custom" />)
    const svg = container.querySelector('svg')
    expect(svg).toHaveClass('q-icon', 'q-icon-md', 'is-custom')
    expect(svg?.getAttribute('class')?.trim().endsWith('is-custom')).toBe(true)
  })

  it('`as` wins when both `as` and `children` are supplied', () => {
    const { container } = render(
      <Icon as={Glyph}><svg data-testid="child-svg" /></Icon>,
    )
    expect(container.querySelectorAll('svg')).toHaveLength(1)
    expect(container.querySelector('[data-testid="child-svg"]')).not.toBeInTheDocument()
  })

  it('recipe `icon()` returns the composite class string', () => {
    expect(icon()).toBe('q-icon q-icon-md')
    expect(icon({ size: 'lg', color: 'success' })).toBe('q-icon q-icon-lg text-q-icon-success')
    expect(icon({ size: 'xs' }, 'extra')).toBe('q-icon q-icon-xs extra')
  })
})
```

app/packages/quanta/src/components/icon/icon.tsx

```
'use client'

import type { ComponentType, ReactElement, Ref, SVGProps } from 'react'
import { cloneElement, isValidElement } from 'react'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'

/**
 * Icon — paints a glyph from `@higgsfield-ai/icons` (or any SVG component) with
 * quanta's standardized size, color, and a11y. It replaces the scattered
 * `& svg { width/height }` rules across components: every consumer sizes via the
 * `--hf-icon-*` token scale (xs..xl -> 12/16/20/24/28px) and inherits color from
 * `currentColor` by default.
 *
 * NODE-ONLY: Icon renders the glyph itself as the single, primary element — no
 * wrapper. The size/color/a11y/ref props are applied DIRECTLY to the glyph
 * `<svg>`, so the output is exactly one element. Supply the glyph one of two
 * tree-shake-friendly ways (static per-icon imports keep bundlers pulling only
 * the one glyph you reference):
 *
 *   import { IconMagnifyingGlassOutlined } from '@higgsfield-ai/icons/IconMagnifyingGlassOutlined'
 *   <Icon as={IconMagnifyingGlassOutlined} size="md" label="Search" />        // `as`
 *   <Icon size="md" label="Search"><IconMagnifyingGlassOutlined /></Icon>     // children
 *
 * Sizing strategy (icon.css): `q-icon` sets `width`/`height: var(--hf-icon-<size>)`
 * on the `<svg>` itself, overriding whatever intrinsic `width`/`height` the glyph
 * carries, so every icon renders at the requested token size.
 *
 * a11y: decorative by default (`aria-hidden`). Pass `label` to expose it as
 * `role="img"` with an accessible name.
 *
 * ref: forwarded to the glyph as the React 19 ref-as-prop. It reaches the `<svg>`
 * DOM node only for glyphs that forward their ref; the `@higgsfield-ai/icons`
 * glyphs are plain function components that spread props onto the svg but do not
 * forward ref, so for those the ref lands on the component, not the svg element.
 */

export type IconSize = 'xs' | 'sm' | 'md' | 'lg' | 'xl'

/**
 * Quanta icon color tokens -> `text-q-icon-*` utilities. Default is `currentColor`
 * (the icon inherits the surrounding text color) — omit `color` for that.
 */
export type IconColor =
  | 'primary'
  | 'secondary'
  | 'tertiary'
  | 'brand'
  | 'accent'
  | 'inverse'
  | 'disabled'
  | 'error'
  | 'success'
  | 'warning'
  | 'info'

const SIZE_CLASS = {
  xs: 'q-icon-xs',
  sm: 'q-icon-sm',
  md: 'q-icon-md',
  lg: 'q-icon-lg',
  xl: 'q-icon-xl',
} satisfies Record<IconSize, string>

const COLOR_CLASS = {
  primary: 'text-q-icon-primary',
  secondary: 'text-q-icon-secondary',
  tertiary: 'text-q-icon-tertiary',
```

```
  brand: 'text-q-icon-brand',
  accent: 'text-q-icon-accent',
  inverse: 'text-q-icon-inverse',
  disabled: 'text-q-icon-disabled',
  error: 'text-q-icon-error',
  success: 'text-q-icon-success',
  warning: 'text-q-icon-warning',
  info: 'text-q-icon-info',
} satisfies Record<IconColor, string>

export interface IconOptions {
  size?: IconSize
  color?: IconColor
}

/** A glyph component shape (matches every `@higgsfield-ai/icons/<Icon>` export). */
export type IconGlyph = ComponentType<SVGProps<SVGSVGElement>>

export type IconProps = {
  /** Token size (default `md` = 20px). Maps to `var(--hf-icon-<size>)`. */
  size?: IconSize
  /** Quanta icon color token. Omit to inherit `currentColor`. */
  color?: IconColor
  /**
   * Accessible name. When set, the icon is exposed as `role="img"` with this
   * label; otherwise it is decorative (`aria-hidden`).
   */
  label?: string
  /** Extra classes appended after the recipe classes (caller wins ordering). */
  className?: string
  /** Forwarded to the glyph (ref-as-prop). See the component JSDoc on ref. */
  ref?: Ref<SVGSVGElement>
  /**
   * Glyph component to render (e.g. `IconMagnifyingGlassOutlined`). Alternative
   * to passing the glyph element as `children`. Takes precedence when both are
   * given.
   */
  as?: IconGlyph
  /** The glyph element when not using `as` (e.g. `<IconMagnifyingGlassOutlined />`). */
  children?: ReactElement<SVGProps<SVGSVGElement>>
}

/** Recipe — the composite icon class string, for styling a glyph element directly. */
export function icon(options: IconOptions = {}, ...extra: ClassValue[]): string {
  const { size = 'md', color } = options
  return cx('q-icon', SIZE_CLASS[size], color && COLOR_CLASS[color], ...extra)
}

export function Icon({
  size = 'md',
  color,
  as: As,
  children,
  label,
  className,
  ref,
  role,
  'aria-label': ariaLabel,
  'aria-hidden': ariaHidden,
}: IconProps & Pick<SVGProps<SVGSVGElement>, 'role' | 'aria-label' | 'aria-hidden'>) {
  const accessibleLabel = ariaLabel ?? label
  // Props painted DIRECTLY onto the glyph svg (no wrapper element). `q-icon`
  // sizes the svg via --hf-icon-*; color flows through `currentColor`.
  const glyphProps: SVGProps<SVGSVGElement> = {
    ref,
    role: role ?? (accessibleLabel ? 'img' : undefined),
    'aria-label': accessibleLabel,
    'aria-hidden': ariaHidden ?? (accessibleLabel ? undefined : true),
    className: icon({ size, color }, className),
  }
```

```
  // `as` wins over `children`: render the glyph component with our props.
  if (As) return <As {...glyphProps} />
  // `children` is a single glyph element; clone it so the props (incl. our
  // className) merge onto its svg — the glyphs spread incoming props last.
  if (isValidElement(children)) return cloneElement(children, glyphProps)
  return null
}
```

app/packages/quanta/src/components/icon/index.ts

```
export { Icon, icon } from './icon.tsx'
export type { IconColor, IconGlyph, IconOptions, IconProps, IconSize } from './icon.tsx'
```

app/packages/quanta/src/components/input/index.ts

```
export { Input } from './input.tsx'
export type { InputProps } from './input.tsx'
```

app/packages/quanta/src/components/input/input.css

```
/**
 * Field scaffolding shared by Input and Textarea, pixel-matched to the Figma
 * TextField (node 342:1354) / TextArea (node 2134:76).
 *
 * Base UI Field owns label/description/error association and validity; these
 * `q-field-*` utilities only paint. Structure: label (caption/sm/medium) ·
 * control surface (white-5%, radius/300, 1.5px border that turns lime on focus /
 * red on error) · helper (caption/sm/regular, tertiary) or error (red).
 *
 * Figma ships a single 40px filled style — no size or "plain" variants. The
 * multiline control modifier lives in textarea.css (its only consumer).
 */

@source "./input.tsx";

/* Field root — label / control / helper stack. */
@utility q-field {
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-100); /* 4px */
  width: 100%;

  &:has(:disabled),
  &:has([data-disabled]) {
    opacity: var(--hf-opacity-disabled);
  }
}

/* Label row. */
@utility q-field-label {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: var(--hf-space-100); /* 4px */
  padding-inline-end: var(--hf-space-200); /* 8px */
  color: var(--hf-color-text-primary);
  @apply text-q-caption-sm-medium; /* Inter Medium 12px */
}
@utility q-field-label-invalid {
  color: var(--hf-color-state-error-fg);
}
@utility q-field-required {
  color: var(--hf-color-state-error-fg);
  @apply text-q-label-sm-medium;
}

/* Control surface — the bordered white-5% box that holds prefix/input/suffix.
 * Single Figma size: 40px tall, 12px inline padding, radius/300 (12px). */
@utility q-field-control {
  box-sizing: border-box;
  display: flex;
  align-items: center;
  gap: var(--hf-space-150); /* 6px */
  width: 100%;
  height: var(--hf-space-1000); /* 40px */
  padding-inline: var(--hf-space-300); /* 12px */
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--hf-radius-300); /* 12px */
  background-color: var(--hf-color-overlay-dim-soft); /* white 5% */
  color: var(--hf-color-icon-secondary); /* prefix/suffix glyph color */

  &:hover {
    background-image: linear-gradient(90deg, var(--hf-color-overlay-hover), var(--hf-color-overlay-hover));
  }

  &:focus-within {
    overflow: hidden;
    box-shadow: inset 0 0 0 var(--hf-border-width-medium) var(--hf-color-border-focus);
```

```
  }

  &:has(:disabled),
  &:has([data-disabled]) {
    cursor: not-allowed;
  }
}

/* Error — red border, beats focus. */
@utility q-field-control-invalid {
  box-shadow: inset 0 0 0 var(--hf-border-width-medium) var(--hf-color-border-error);

  &:focus-within {
    box-shadow: inset 0 0 0 var(--hf-border-width-medium) var(--hf-color-border-error);
  }
}

/* Prefix / suffix icon slot (20px). */
@utility q-field-affix {
  display: inline-flex;
  flex-shrink: 0;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-500); /* 20px */
  height: var(--hf-space-500);

  & svg {
    width: 100%;
    height: 100%;
  }
}

/* The actual input / textarea element (transparent; the surface is the wrapper). */
@utility q-field-input {
  flex: 1 1 var(--hf-space-0);
  min-width: 0;
  width: 100%;
  height: var(--hf-space-400); /* label/sm line-height: 16px */
  appearance: none;
  background: transparent;
  border: var(--hf-border-width-none) solid transparent;
  padding: var(--hf-space-0);
  outline: none;
  resize: none;
  color: var(--hf-color-text-primary);
  @apply text-q-label-sm-regular; /* Inter Regular 14px */

  &::placeholder {
    color: var(--hf-color-text-on-overlay-secondary); /* white 50% */
    opacity: 1;
  }

  &:disabled {
    cursor: not-allowed;
  }
}

/* Helper / error text. */
@utility q-field-description {
  color: var(--hf-color-text-tertiary);
  @apply text-q-caption-sm-regular;
}
@utility q-field-error {
  color: var(--hf-color-state-error-fg);
  @apply text-q-caption-sm-regular;
}
```

app/packages/quanta/src/components/input/input.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { describe, expect, it } from 'vitest'
import { Input } from './index.ts'

describe('<Input>', () => {
  it('associates the label with the control', () => {
    render(<Input label="Email" placeholder="you@example.com" />)
    const input = screen.getByLabelText('Email')
    expect(input).toHaveClass('q-field-input')
    expect(input.tagName).toBe('INPUT')
  })

  it('renders the helper description', () => {
    render(<Input label="Name" description="We'll never share this" />)
    expect(screen.getByText('We\'ll never share this')).toHaveClass('q-field-description')
  })

  it('shows the error (red) state and message instead of the description', () => {
    const { container } = render(
      <Input label="Name" description="helper" error="Please enter only letters" defaultValue="Mary387" />,
    )
    expect(screen.getByText('Please enter only letters')).toHaveClass('q-field-error')
    expect(screen.queryByText('helper')).not.toBeInTheDocument()
    expect(container.querySelector('.q-field-control')).toHaveClass('q-field-control-invalid')
    expect(container.querySelector('.q-field-label')).toHaveClass('q-field-label-invalid')
    expect(screen.getByDisplayValue('Mary387')).toHaveAttribute('aria-invalid', 'true')
  })

  it('renders start and end slots', () => {
    const { container } = render(
      <Input label="Search" start={<span data-testid="pre" />} end={<span data-testid="suf" />} />,
    )
    expect(container.querySelectorAll('.q-field-affix')).toHaveLength(2)
    expect(screen.getByTestId('pre')).toBeInTheDocument()
    expect(screen.getByTestId('suf')).toBeInTheDocument()
  })

  it('accepts the deprecated prefix / suffix aliases (back-compat)', () => {
    render(
      <Input label="Search" prefix={<span data-testid="pre" />} suffix={<span data-testid="suf" />} />,
    )
    expect(screen.getByTestId('pre')).toBeInTheDocument()
    expect(screen.getByTestId('suf')).toBeInTheDocument()
  })

  it('accepts typed input', async () => {
    const user = userEvent.setup()
    render(<Input label="Name" />)
    const input = screen.getByLabelText('Name')
    await user.type(input, 'Ada')
    expect(input).toHaveValue('Ada')
  })
})
```

app/packages/quanta/src/components/input/input.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { Field } from '@base-ui/react/field'
import { cx } from '../utils/cx.ts'

/**
 * Input — a single-line labelled text field on Base UI `Field` (label /
 * description / error association + validity), pixel-matched to the Figma
 * TextField (node 342:1354). One 40px filled surface (white-5%, radius 12) whose
 * 1.5px border turns lime on focus and red on error. Provide `error` for the
 * invalid state (red label + border + message). Pass `start`/`end` for
 * leading/trailing 20px icon slots. For multi-line input use `Textarea`.
 *
 *   <Input label="Email" placeholder="you@example.com" description="We'll never share it" />
 *   <Input label="Name" error="Please enter only letters" defaultValue="Mary387" />
 */

type FieldProps = {
  /** Label above the control. */
  label?: ReactNode
  /** Helper text below the control (hidden while an `error` shows). */
  description?: ReactNode
  /** Error message — its presence puts the field in the invalid (red) state. */
  error?: ReactNode
  /** Force the invalid state without a message. */
  invalid?: boolean
  /** Append a red `*` to the label. */
  required?: boolean
  /** Leading slot (20px icon, any node) inside the control. */
  start?: ReactNode
  /** Trailing slot (20px icon, any node) inside the control. */
  end?: ReactNode
  /** @deprecated Use `start` — kept as an alias for back-compat. */
  prefix?: ReactNode
  /** @deprecated Use `end` — kept as an alias for back-compat. */
  suffix?: ReactNode
  /** Swap the underlying control element (e.g. a masked / 3rd-party input). */
  render?: ComponentProps<typeof Field.Control>['render']
  /** Class for the root Field wrapper. */
  className?: string
  /** Class for the control surface. */
  controlClassName?: string
  /** Class for the `<input>` element. */
  inputClassName?: string
  /** Props forwarded to the Base UI `Field.Root`. */
  fieldProps?: ComponentProps<typeof Field.Root>
}

export type InputProps = FieldProps & Omit<ComponentProps<'input'>, 'size' | 'prefix' | 'color' | 'children'>

export function Input({
  label,
  description,
  error,
  invalid: invalidProp,
  required = false,
  start,
  end,
  prefix,
  suffix,
  render,
  className,
  controlClassName,
  inputClassName,
  fieldProps,
  ...controlProps
}: InputProps) {
```

```
  const invalid = invalidProp ?? error != null
  // `start`/`end` are canonical; `prefix`/`suffix` are the back-compat aliases.
  const lead = start ?? prefix
  const trail = end ?? suffix

  return (
    <Field.Root className={cx('q-field', className)} {...fieldProps}>
      {label != null
        ? (
            <Field.Label className={cx('q-field-label', invalid && 'q-field-label-invalid')}>
              {label}
              {required ? <span aria-hidden className="q-field-required">*</span> : null}
            </Field.Label>
          )
        : null}

      <div
        className={cx(
          'q-field-control',
          invalid && 'q-field-control-invalid',
          controlClassName,
        )}
      >
        {lead != null ? <span className="q-field-affix">{lead}</span> : null}
        {/* `ref` rides in `...controlProps` and Base UI forwards it to the
            rendered <input> — the primary node — so it is never dropped. */}
        <Field.Control
          className={cx('q-field-input', inputClassName)}
          aria-invalid={invalid || undefined}
          render={render}
          {...controlProps}
        />
        {trail != null ? <span className="q-field-affix">{trail}</span> : null}
      </div>

      {invalid && error != null
        ? <Field.Error match className="q-field-error">{error}</Field.Error>
        : description != null
          ? <Field.Description className="q-field-description">{description}</Field.Description>
          : null}
    </Field.Root>
  )
}
```

app/packages/quanta/src/components/kbd/index.ts

```
export { Kbd, KbdSequence } from './kbd.tsx'
export type { KbdProps, KbdSequenceProps } from './kbd.tsx'
```

app/packages/quanta/src/components/kbd/kbd.css

```
/**
 * Kbd presentation — composes only emitted q-* utilities (color, border-width,
 * typography) plus native spacing/sizing, so there are no @utility rules here.
 * This file exists solely to point Tailwind's scanner at the colocated source:
 * quanta is consumed as a symlinked package and Tailwind skips node_modules
 * during automatic content detection, so the literal class strings in kbd.tsx
 * must be registered explicitly — same convention as dot.css / avatar.css.
 */
@source "./kbd.tsx";
```

app/packages/quanta/src/components/kbd/kbd.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Kbd, KbdSequence } from './index.ts'

describe('<Kbd>', () => {
  it('renders a semantic <kbd> as the Figma _Shortcut pill (single size, token styles)', () => {
    render(<Kbd>K</Kbd>)
    const key = screen.getByText('K')
    expect(key.tagName).toBe('KBD')
    expect(key).toHaveClass(
      'inline-flex',
      'h-5',
      'gap-0.5',
      'rounded-q-100',
      'px-1',
      'border-q-hairline',
      'border-q-border-subtle',
      'bg-q-overlay-hover',
      'text-q-text-primary',
      'text-q-caption-sm-medium',
    )
  })

  it('forwards className and native kbd props', () => {
    render(<Kbd className="is-custom" title="Command">⌘</Kbd>)
    const key = screen.getByText('⌘')
    expect(key).toHaveClass('is-custom')
    expect(key).toHaveAttribute('title', 'Command')
  })
})

describe('<KbdSequence>', () => {
  it('wraps string keys in <Kbd> and renders separators between them', () => {
    const { container } = render(<KbdSequence keys={['⌘', 'K']} />)
    const kbds = container.querySelectorAll('kbd')
    expect(kbds).toHaveLength(2)
    expect(kbds[0]).toHaveTextContent('⌘')
    expect(kbds[1]).toHaveTextContent('K')

    const separators = container.querySelectorAll('[aria-hidden="true"]')
    // one separator between the two keys, styled via Typography
    expect(separators).toHaveLength(1)
    expect(separators[0]).toHaveTextContent('+')
    expect(separators[0]).toHaveClass('text-q-caption-sm-regular', 'text-q-text-tertiary')
  })

  it('omits the separator when set to null', () => {
    const { container } = render(<KbdSequence keys={['⌘', 'A']} separator={null} />)
    expect(container.querySelector('[aria-hidden="true"]')).toBeNull()
  })

  it('accepts a custom separator and Kbd children', () => {
    const { container } = render(
      <KbdSequence separator="then">
        <Kbd>G</Kbd>
        <Kbd>I</Kbd>
      </KbdSequence>,
    )
    expect(container.querySelectorAll('kbd')).toHaveLength(2)
    expect(container.querySelector('[aria-hidden="true"]')).toHaveTextContent('then')
  })
})
```

app/packages/quanta/src/components/kbd/kbd.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { Children } from 'react'
import { Typography } from '../typography/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * Kbd — a keyboard-shortcut pill, pinned to the Figma `_Shortcut` (node
 * 1157:4028): a single 20px-tall chip holding optional ⌘ / ⇧ glyphs and the
 * key, on a translucent white-5% surface with a subtle hairline.
 *
 *   h 20 · px 4 · gap 2 · radius 4 · caption-sm-medium 12 · text-primary
 *   surface transparent/dark/05 (white-5%) · border border/subtle
 *
 * Figma ships ONE size — there are no size variants. Compose a combo inside one
 * pill (`<Kbd>⌘ K</Kbd>`), or use `KbdSequence` for separate keys joined by a
 * separator. Tokens only: surface `bg-q-overlay-hover` (the theme-adaptive
 * white-5% / black-5% mirror of Figma's `transparent/dark/05`), border
 * `border-q-hairline border-q-border-subtle` (the 0.5px hairline matching Figma
 * exactly), radius `rounded-q-100`, type `text-q-caption-sm-medium`.
 */

export type KbdProps = ComponentProps<'kbd'>

export function Kbd({ className, children, color: _color, ...props }: KbdProps) {
  // The `<kbd>` is the text-styled surface the component owns: its composite type
  // (caption-sm-medium) + color (text-primary) come from Typography; the chip's
  // own layout/border/surface classes ride in className (applied last).
  return (
    <Typography
      as="kbd"
      variant="caption-sm-medium"
      color="primary"
      className={cx(
        'inline-flex h-5 shrink-0 items-center justify-center gap-0.5 rounded-q-100 px-1 align-middle',
        'border-q-hairline border-q-border-subtle bg-q-overlay-hover',
        className,
      )}
      {...props}
    >
      {children}
    </Typography>
  )
}

/**
 * KbdSequence — lays out several keys, joined by a separator (default "+").
 * Pass a `keys` array (strings wrap in `<Kbd>`, nodes pass through) or `<Kbd>`
 * children. The separator is aria-hidden so the `<kbd>` semantics stay clean.
 */

export type KbdSequenceProps = Omit<ComponentProps<'span'>, 'children'> & {
  /** Separator rendered between keys. Default "+". Set to null to omit. */
  separator?: ReactNode
  /** Keys to render. Strings are wrapped in `<Kbd>`; ReactNodes pass through. */
  keys?: ReactNode[]
  children?: ReactNode
}

export function KbdSequence({
  separator = '+',
  keys,
  className,
  children,
  ...props
}: KbdSequenceProps) {
  const items: ReactNode[] = keys
```

```
    ? keys.map((k, i) =>
        typeof k === 'string' || typeof k === 'number'
          ? <Kbd key={i}>{k}</Kbd>
          : k,
      )
    : Children.toArray(children)

  return (
    <span
      className={cx('inline-flex items-center gap-1 align-middle', className)}
      {...props}
    >
      {items.map((item, i) => (
        <span key={i} className="inline-flex items-center gap-1">
          {item}
          {separator != null && i < items.length - 1
            ? (
                <Typography
                  as="span"
                  variant="caption-sm-regular"
                  color="tertiary"
                  aria-hidden
                >
                  {separator}
                </Typography>
              )
            : null}
        </span>
      ))}
    </span>
  )
}
```

app/packages/quanta/src/components/loader/index.ts

```
export { Loader } from './loader.tsx'
export type { LoaderColor, LoaderProps, LoaderSize, LoaderVariant } from './loader.tsx'
```

app/packages/quanta/src/components/loader/loader.css

```
/**
 * Loader presentation — slot-tinted, Base-less indeterminate indicator. The
 * accent comes from the slot system (`--q-tint`, set by `slotStyle(color)` on the
 * root); neutral surfaces use `background-tertiary` (same track as Progress).
 * Size is driven by per-size CSS custom properties. Motion degrades under
 * `prefers-reduced-motion` and can be disabled per-instance via `data-static`.
 */

@source "./loader.tsx";

@utility q-loader {
  /* md defaults — overridden by the size utilities below. */
  --q-loader-size: var(--hf-space-800); /* 32px box */
  --q-loader-dot: var(--hf-space-250); /* 10px dot */
  --q-loader-dot-gap: var(--hf-space-100); /* 4px gap between dots */

  position: relative;
  display: inline-flex;
  flex: none;
  align-items: center;
  justify-content: center;
  width: var(--q-loader-size);
  height: var(--q-loader-size);
  color: var(--q-tint);
}

@utility q-loader-xxs {
  --q-loader-size: var(--hf-space-300); /* 12px */
  --q-loader-dot: var(--hf-space-100); /* 4px */
  --q-loader-dot-gap: var(--hf-space-050); /* 2px */
}
@utility q-loader-xs {
  --q-loader-size: var(--hf-space-400); /* 16px */
  --q-loader-dot: var(--hf-space-150); /* 6px */
  --q-loader-dot-gap: var(--hf-space-050); /* 2px */
}
@utility q-loader-sm {
  --q-loader-size: var(--hf-space-600); /* 24px */
  --q-loader-dot: var(--hf-space-200); /* 8px */
  --q-loader-dot-gap: var(--hf-space-100); /* 4px */
}
@utility q-loader-md {
  --q-loader-size: var(--hf-space-800); /* 32px */
  --q-loader-dot: var(--hf-space-250); /* 10px */
  --q-loader-dot-gap: var(--hf-space-100); /* 4px */
}
@utility q-loader-lg {
  --q-loader-size: var(--hf-space-1200); /* 48px */
  --q-loader-dot: var(--hf-space-400); /* 16px */
  --q-loader-dot-gap: var(--hf-space-150); /* 6px */
}

/* ■■ dots — a tight 2×2 square of dots, blinking clockwise around it ■■■■■■■■■ */
.q-loader-dots {
  display: grid;
  grid-template-columns: repeat(2, var(--q-loader-dot));
  grid-auto-rows: var(--q-loader-dot);
  gap: var(--q-loader-dot-gap);
  place-content: center;
}
@utility q-loader-dot {
  width: var(--q-loader-dot);
  height: var(--q-loader-dot);
  border-radius: var(--hf-radius-full);
  background-color: var(--q-tint);
  animation: q-loader-blink var(--hf-duration-1200) var(--hf-ease-linear) infinite;
}
/* Grid order is TL, TR, BL, BR; delay them clockwise (TL→TR→BR→BL). */
```

```
/* no token: stagger delays 0.3s/0.9s/0.6s — 0.9s/0.6s have no matching --hf-duration-*; kept the family raw for consistency */
.q-loader-dots .q-loader-dot:nth-child(1) { animation-delay: 0s; }
.q-loader-dots .q-loader-dot:nth-child(2) { animation-delay: 0.3s; } /* no token: stagger delay, no matching --hf-duration-* */
.q-loader-dots .q-loader-dot:nth-child(3) { animation-delay: 0.9s; } /* no token: stagger delay, no matching --hf-duration-* */
.q-loader-dots .q-loader-dot:nth-child(4) { animation-delay: 0.6s; } /* no token: stagger delay, no matching --hf-duration-* */
@keyframes q-loader-blink {
  0%, 100% { opacity: 1; }
  50% { opacity: 0.15; }
}

/* ■■ circle — spinning ring (neutral track + accent arc) ■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-loader-spinner {
  width: var(--q-loader-size);
  height: var(--q-loader-size);
  animation: q-loader-spin var(--hf-duration-800) var(--hf-ease-linear) infinite;
}
.q-loader-spinner-track { stroke: var(--hf-color-transparent-light-10); }
.q-loader-spinner-arc { stroke: var(--q-tint); }
@keyframes q-loader-spin {
  to { transform: rotate(360deg); }
}

/* ■■ stars — twinkling sparkles (the marketing / AI motif) ■■■■■■■■■■■■■■■■■■■■ */
@utility q-loader-star {
  position: absolute;
  color: var(--q-tint);
  /* no token: ease-in-out keyword (cubic-bezier(.42,0,.58,1)) != --hf-ease-in-out (cubic-bezier(.4,0,.2,1)), left as keyword */
  animation: q-loader-twinkle var(--hf-duration-1400) ease-in-out infinite;
}
.q-loader-stars .q-loader-star-main {
  inset: var(--hf-space-0);
  width: 100%;
  height: 100%;
}
.q-loader-stars .q-loader-star-sub {
  top: var(--hf-space-0);
  right: var(--hf-space-0);
  width: 45%;
  height: 45%;
  animation-delay: var(--hf-duration-700);
}
@keyframes q-loader-twinkle {
  0%, 100% { opacity: 0.35; transform: scale(0.6); }
  50% { opacity: 1; transform: scale(1); }
}

/* ■■ shine — an accent gloss sweeping across a tile (media-grid shimmer) ■■■■■■ */
@utility q-loader-shine {
  border-radius: var(--hf-radius-300);
  background-color: var(--hf-color-background-tertiary);
  overflow: hidden;
}
.q-loader-shine::after {
  content: "";
  position: absolute;
  inset: var(--hf-space-0);
  /* Sweep is inlined (was --hf-gradient-loader-sweep): that upstream gradient
   * token hardcodes the slot var, so the loader owns its sweep here to keep the
   * slot var name (--q-tint) fully local. Same value, tinted by the slot accent. */
  background-image: linear-gradient(105deg, transparent 35%, var(--q-tint) 50%, transparent 65%);
  transform: translateX(-100%);
  /* no token: ease-in-out keyword (cubic-bezier(.42,0,.58,1)) != --hf-ease-in-out (cubic-bezier(.4,0,.2,1)), left as keyword */
  animation: q-loader-shine var(--hf-duration-1300) ease-in-out infinite;
}
@keyframes q-loader-shine {
  0% { transform: translateX(-100%); }
  100% { transform: translateX(100%); }
}

/* ■■ Motion off (animated={false}) + reduced-motion ■■■■■■■■■■■■■■■■■■■■■■■■■■ */
```

```
.q-loader[data-static] .q-loader-dot,
.q-loader[data-static] .q-loader-spinner,
.q-loader[data-static] .q-loader-star { animation: none; }
.q-loader-shine[data-static]::after { animation: none; }

@media (prefers-reduced-motion: reduce) {
  .q-loader-dot,
  .q-loader-spinner,
  .q-loader-star { animation: none; }
  .q-loader-shine::after { animation: none; }
}
```

app/packages/quanta/src/components/loader/loader.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Loader } from './index.ts'

describe('<Loader>', () => {
  it('renders a status role with the default label, variant and size', () => {
    render(<Loader />)
    const el = screen.getByRole('status', { name: 'Loading' })
    expect(el).toHaveClass('q-loader', 'q-loader-circle', 'q-loader-md')
  })

  it('renders four dots for the dots variant', () => {
    const { container } = render(<Loader variant="dots" />)
    expect(container.querySelectorAll('.q-loader-dot')).toHaveLength(4)
  })

  it('renders the spinner svg for the circle variant', () => {
    const { container } = render(<Loader variant="circle" />)
    expect(container.querySelector('.q-loader-spinner')).toBeInTheDocument()
  })

  it('renders two sparkles for the stars variant', () => {
    const { container } = render(<Loader variant="stars" />)
    expect(container.querySelectorAll('.q-loader-star')).toHaveLength(2)
  })

  it('applies the requested size and a custom label', () => {
    render(<Loader variant="shine" size="lg" aria-label="Generating" />)
    expect(screen.getByRole('status', { name: 'Generating' })).toHaveClass('q-loader-shine', 'q-loader-lg')
  })

  it('wires the slot color via inline custom properties', () => {
    render(<Loader color="success" />)
    const el = screen.getByRole('status')
    expect(el.style.getPropertyValue('--q-tint')).not.toBe('')
  })

  it('disables motion with animated={false}', () => {
    render(<Loader />)
    const animated = screen.getByRole('status')
    expect(animated).not.toHaveAttribute('data-static')

    render(<Loader animated={false} aria-label="Static" />)
    expect(screen.getByRole('status', { name: 'Static' })).toHaveAttribute('data-static')
  })

  it('forwards className and native div props', () => {
    render(<Loader className="is-custom" id="load" />)
    const el = screen.getByRole('status')
    expect(el).toHaveClass('is-custom')
    expect(el).toHaveAttribute('id', 'load')
  })
})
```

app/packages/quanta/src/components/loader/loader.tsx

```
'use client'

import type { ComponentProps, CSSProperties } from 'react'
import { cx } from '../utils/cx.ts'
import type { SlotColor } from '../utils/slot.ts'
import { slotStyle } from '../utils/slot.ts'

/**
 * Loader — an indeterminate loading indicator, slot-tinted with the quanta
 * colour system (`color` → `slotStyle`, so light/dark and any `defineTheme()`
 * brand are automatic). One component, four motifs that echo the rest of the
 * system:
 *
 *   - `dots`   — four dots blinking in sequence around a circle (the accent).
 *   - `circle` (default) — a spinning ring with an accent arc over a neutral track.
 *   - `stars`  — twinkling sparkles, the marketing/“AI” motif (cf. specialBrand).
 *   - `shine`  — an accent gloss sweeping across a tile (the media-grid shimmer).
 *
 * `role="status"` + `aria-label` announce it to assistive tech; the glyphs are
 * decorative. Motion degrades under `prefers-reduced-motion` and can be turned
 * off per-instance with `animated={false}`.
 */

export type LoaderVariant = 'dots' | 'circle' | 'stars' | 'shine'
export type LoaderSize = 'xxs' | 'xs' | 'sm' | 'md' | 'lg'
export type LoaderColor = SlotColor

const SIZE_CLASS = {
  xxs: 'q-loader-xxs',
  xs: 'q-loader-xs',
  sm: 'q-loader-sm',
  md: 'q-loader-md',
  lg: 'q-loader-lg',
} satisfies Record<LoaderSize, string>

const VARIANT_CLASS = {
  dots: 'q-loader-dots',
  circle: 'q-loader-circle',
  stars: 'q-loader-stars',
  shine: 'q-loader-shine',
} satisfies Record<LoaderVariant, string>

export interface LoaderProps extends Omit<ComponentProps<'div'>, 'color'> {
  variant?: LoaderVariant
  size?: LoaderSize
  color?: LoaderColor
  /** Run the animation. Default true. */
  animated?: boolean
  'aria-label'?: string
}

/** Four-point sparkle (Figma "AI" star). Scales with the loader box. */
function Sparkle({ className }: { className?: string }) {
  return (
    <svg className={className} viewBox="0 0 24 24" fill="currentColor" aria-hidden>
      <path d="M12 0c.6 6.7 5.3 11.4 12 12-6.7.6-11.4 5.3-12 12-.6-6.7-5.3-11.4-12-12C6.7 11.4 11.4 6.7 12 0Z" />
    </svg>
  )
}

export function Loader({
  variant = 'circle',
  size = 'md',
  color = 'brand',
  animated = true,
  className,
  style,
  'aria-label': ariaLabel = 'Loading',
```

```
  ...props
}: LoaderProps) {
  const common = {
    role: 'status' as const,
    'aria-label': ariaLabel,
    'aria-live': 'polite' as const,
    'data-static': animated ? undefined : '',
    style: { ...slotStyle(color), ...style } as CSSProperties,
    ...props,
  }
  const rootClass = cx('q-loader', VARIANT_CLASS[variant], SIZE_CLASS[size], className)

  if (variant === 'dots') {
    return (
      <div {...common} className={rootClass}>
        {Array.from({ length: 4 }, (_, i) => <span key={i} className="q-loader-dot" />)}
      </div>
    )
  }

  if (variant === 'stars') {
    return (
      <div {...common} className={rootClass}>
        <Sparkle className="q-loader-star q-loader-star-main" />
        <Sparkle className="q-loader-star q-loader-star-sub" />
      </div>
    )
  }

  if (variant === 'shine') {
    // The gloss sweep is a ::after; nothing else to render.
    return <div {...common} className={rootClass} />
  }

  // circle — spinning ring (neutral track + accent arc).
  return (
    <div {...common} className={rootClass}>
      <svg className="q-loader-spinner" viewBox="0 0 24 24" fill="none" aria-hidden>
        <circle className="q-loader-spinner-track" cx="12" cy="12" r="9" strokeWidth="2.5" />
        <path className="q-loader-spinner-arc" d="M12 3a9 9 0 0 0 1 9 9" strokeWidth="2.5" strokeLinecap="round" />
      </svg>
    </div>
  )
}
```

app/packages/quanta/src/components/media/index.ts

```
export { Media, useMediaFallback } from './media.tsx'
export type {
  MediaCaptionProps,
  MediaFallbackProps,
  MediaFit,
  MediaImageProps,
  MediaOverlayPlacement,
  MediaOverlayProps,
  MediaRatio,
  MediaRootProps,
  MediaRounded,
  MediaVideoProps,
} from './media.tsx'
```

app/packages/quanta/src/components/media/media.css

```
/**
 * Media — pure-quanta presentational surface for images & videos at a fixed
 * aspect ratio (no Figma node; a composition primitive for media grids and video
 * cards). The TS unions in media.tsx (RATIO_CLASS / FIT_CLASS / ROUNDED_CLASS /
 * OVERLAY_PLACEMENT_CLASS) are the single source of truth; this file registers
 * the colocated source for Tailwind's scanner (quanta is symlinked, so literal
 * class strings must be registered explicitly) and defines the box geometry,
 * the two ratio presets that have no `aspect-*` utility, the fill rule, the
 * fallback surface, and the overlay placements / gradient scrim — tokens only.
 */
@source "./media.tsx";

/* The aspect-ratio box. Clips overflow so the corner radius (set via the
 * rounded-q-* utility on the same element) curves the media inside it. */
@utility q-media {
  position: relative;
  display: block;
  width: 100%;
  overflow: hidden;
  /* Custom numeric ratio is wired through this var from the `ratio` prop; the
   * preset ratios use the `aspect-*` utility instead and leave this unset. */
  aspect-ratio: var(--q-media-ratio, auto);
  background-color: var(--hf-color-background-secondary);
}

/* Ratio presets with no native `aspect-*` utility — portrait 3/4, wide 21/9. */
@utility q-media-portrait {
  aspect-ratio: 3 / 4;
}
@utility q-media-wide {
  aspect-ratio: 21 / 9;
}

/* Image / Video fill the box edge-to-edge; object-fit comes from FIT_CLASS. */
@utility q-media-fill {
  display: block;
  width: 100%;
  height: 100%;
  /* `auto` ratio boxes have no fixed height, so let the media set intrinsic
   * size; fixed-ratio boxes stretch it to the box via the 100% above. */
  object-position: center;
}

/* The empty / broken-source slot — a tinted box with centered content. */
@utility q-media-fallback {
  position: absolute;
  inset: 0;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: var(--hf-space-200);
  padding: var(--hf-space-400);
  background-color: var(--hf-color-background-elevated-start);
  color: var(--hf-color-text-tertiary);
  text-align: center;
}

/* An absolutely-positioned layer the caller fills. Base layout is centered
 * content; placement utilities re-anchor / add the scrim. */
@utility q-media-overlay {
  position: absolute;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: var(--hf-space-200);
  pointer-events: none;
```

```
  /* Interactive children (a play button) re-enable pointer events themselves. */
  & > * {
    pointer-events: auto;
  }
}

@utility q-media-overlay-fill {
  inset: 0;
}

@utility q-media-overlay-center {
  inset: 0;
}

/* Bottom band with a dark→transparent gradient scrim for legible caption text. */
@utility q-media-overlay-bottom {
  right: 0;
  bottom: 0;
  left: 0;
  align-items: flex-end;
  justify-content: flex-start;
  padding: var(--hf-space-400);
  background-image: linear-gradient(
    to top,
    var(--hf-color-transparent-dark-50),
    transparent
  );
}

/* Top band scrim — mirror of bottom, for top-anchored badges / titles. */
@utility q-media-overlay-top {
  top: 0;
  right: 0;
  left: 0;
  align-items: flex-start;
  justify-content: flex-start;
  padding: var(--hf-space-400);
  background-image: linear-gradient(
    to bottom,
    var(--hf-color-transparent-dark-50),
    transparent
  );
}

/* A small label region under / over the media. */
@utility q-media-caption {
  @apply text-q-caption-sm-medium;
  display: flex;
  align-items: center;
  gap: var(--hf-space-200);
  padding: var(--hf-space-200) var(--hf-space-300);
  color: var(--hf-color-text-secondary);
}
```

app/packages/quanta/src/components/media/media.test.tsx

```
import { fireEvent, render, screen } from '@testing-library/react'
import { createRef } from 'react'
import { describe, expect, it } from 'vitest'
import { Media, useMediaFallback } from './index.ts'

describe('Media.Root', () => {
  it('renders a box with the composite class and a default ratio/rounded', () => {
    render(
      <Media data-testid="root">
        <Media.Image src="/x.jpg" alt="x" />
      </Media>,
    )
    const root = screen.getByTestId('root')
    expect(root).toHaveClass('q-media')
    // defaults: ratio="video" → aspect-video, rounded="md" → rounded-q-300
    expect(root).toHaveClass('aspect-video')
    expect(root).toHaveClass('rounded-q-300')
  })

  it('maps preset ratios + rounded tokens to their utilities', () => {
    render(<Media data-testid="root" ratio="portrait" rounded="full" />)
    const root = screen.getByTestId('root')
    expect(root).toHaveClass('q-media-portrait')
    expect(root).toHaveClass('rounded-q-full')
    expect(root).not.toHaveClass('aspect-video')
  })

  it('wires a numeric ratio through the --q-media-ratio var, not a class', () => {
    render(<Media data-testid="root" ratio={4 / 3} />)
    const root = screen.getByTestId('root')
    expect(root).not.toHaveClass('aspect-video')
    expect(root.style.getPropertyValue('--q-media-ratio')).toBe(String(4 / 3))
  })

  it('forwards ref to the root DOM node', () => {
    const ref = createRef<HTMLDivElement>()
    render(<Media ref={ref} />)
    expect(ref.current).toBeInstanceOf(HTMLDivElement)
    expect(ref.current).toHaveClass('q-media')
  })

  it('keeps caller className last so callers win ordering', () => {
    render(<Media data-testid="root" className="custom-x" />)
    const cls = screen.getByTestId('root').className
    expect(cls.trim().endsWith('custom-x')).toBe(true)
  })
})

describe('Media.Image / Media.Video', () => {
  it('renders an img filling the box with object-fit + lazy loading defaults', () => {
    render(<Media.Image src="/photo.jpg" alt="A photo" />)
    const img = screen.getByRole('img', { name: 'A photo' })
    expect(img).toHaveClass('q-media-fill')
    expect(img).toHaveClass('object-cover')
    expect(img).toHaveAttribute('loading', 'lazy')
  })

  it('honors the contain fit', () => {
    render(<Media.Image src="/photo.jpg" alt="A photo" fit="contain" />)
    expect(screen.getByRole('img', { name: 'A photo' })).toHaveClass('object-contain')
  })

  it('renders a video and passes poster/controls through', () => {
    render(<Media.Video data-testid="vid" poster="/poster.jpg" controls muted />)
    const vid = screen.getByTestId('vid')
    expect(vid.tagName).toBe('VIDEO')
    expect(vid).toHaveClass('q-media-fill')
    expect(vid).toHaveAttribute('poster', '/poster.jpg')
```

```
  })

  it('autoPlayInView forces muted + lazies preload (autoplay requirements)', () => {
    render(<Media.Video data-testid="vid" src="/clip.mp4" autoPlayInView loop />)
    const vid = screen.getByTestId('vid') as HTMLVideoElement
    expect(vid.muted).toBe(true)
    expect(vid).toHaveAttribute('preload', 'metadata')
    expect(vid).toHaveAttribute('loop')
  })

  it('leaves preload unset for a normal (non-autoplay) video', () => {
    render(<Media.Video data-testid="vid" src="/clip.mp4" />)
    expect(screen.getByTestId('vid')).not.toHaveAttribute('preload')
  })
})

describe('Media.Overlay / Media.Fallback / Media.Caption', () => {
  it('places the overlay via the placement utility', () => {
    render(<Media.Overlay data-testid="ov" placement="bottom">scrim</Media.Overlay>)
    const ov = screen.getByTestId('ov')
    expect(ov).toHaveClass('q-media-overlay')
    expect(ov).toHaveClass('q-media-overlay-bottom')
  })

  it('renders fallback content', () => {
    render(<Media.Fallback>No image</Media.Fallback>)
    expect(screen.getByText('No image')).toHaveClass('q-media-fallback')
  })

  it('renders a caption', () => {
    render(<Media.Caption>Sunset</Media.Caption>)
    expect(screen.getByText('Sunset')).toHaveClass('q-media-caption')
  })
})

describe('useMediaFallback (broken-source flow)', () => {
  function BrokenImage() {
    const { failed, onError } = useMediaFallback()
    return (
      <Media>
        {failed
          ? <Media.Fallback>Broken</Media.Fallback>
          : <Media.Image src="/missing.jpg" alt="thing" onError={onError} />}
      </Media>
    )
  }

  it('flips to the fallback when the image errors (mirrors Avatar onError)', () => {
    render(<BrokenImage />)
    const img = screen.getByRole('img', { name: 'thing' })
    expect(screen.queryByText('Broken')).toBeNull()
    fireEvent.error(img)
    expect(screen.getByText('Broken')).toBeInTheDocument()
    expect(screen.queryByRole('img', { name: 'thing' })).toBeNull()
  })
})
```

app/packages/quanta/src/components/media/media.tsx

```
'use client'

import type { ComponentProps, CSSProperties, MutableRefObject, ReactNode } from 'react'
import { useCallback, useEffect, useRef, useState } from 'react'
import { useInView } from '../utils/use-in-view.ts'
import { cx } from '../utils/cx.ts'

/**
 * Media — a pure-quanta presentational surface for images & videos held at a
 * fixed aspect ratio. It is the building block for media grids, video cards, and
 * cover tiles: a clipped `aspect-ratio` box you fill with an `Image`/`Video`, an
 * automatic `Fallback` (mirrors Avatar's onError→fallback flow), an absolutely-
 * positioned `Overlay` (scrims, play buttons, badges), and a `Caption` strip.
 *
 * Composition parts (each replaceable / optional):
 *   • `Media.Root`    — the aspect-ratio box; clips overflow and owns the corner
 *     radius. `ratio` picks a preset (square/video/portrait/wide/auto) or pass a
 *     numeric `ratio` (e.g. `4 / 3`) for a custom value. `rounded` selects a
 *     radius token; children inherit it via `overflow: hidden`.
 *   • `Media.Image`   — `<img>` filling the box; `fit` = object-fit cover|contain,
 *     lazy-loaded by default, calls `onError` → caller can flip to `Fallback`.
 *   • `Media.Video`   — `<video>` filling the box; controls/autoPlay/loop/muted/
 *     poster pass straight through, `fit` = object-fit.
 *   • `Media.Fallback`— the empty / broken-source slot: a tinted box with centered
 *     content (an `<Icon>`, initials, or any node).
 *   • `Media.Overlay` — an absolutely-positioned layer (gradient scrim, centered
 *     play button, top-right badge) the caller fills; `placement` positions it.
 *   • `Media.Caption` — a small label region rendered under / over the media.
 *
 * Tokens only. No Base UI — this is a layout/presentation primitive.
 */

export type MediaRatio = 'square' | 'video' | 'portrait' | 'wide' | 'auto'
export type MediaFit = 'cover' | 'contain'
export type MediaRounded = 'none' | 'sm' | 'md' | 'lg' | 'full'
export type MediaOverlayPlacement = 'fill' | 'top' | 'bottom' | 'center'

// Aspect-ratio presets → the `aspect-*` utility. `auto` opts out (intrinsic).
const RATIO_CLASS = {
  square: 'aspect-square',
  video: 'aspect-video',
  portrait: 'q-media-portrait',
  wide: 'q-media-wide',
  auto: '',
} satisfies Record<MediaRatio, string>

const FIT_CLASS = {
  cover: 'object-cover',
  contain: 'object-contain',
} satisfies Record<MediaFit, string>

// Corner radius → emitted radius utilities (the box clips, so children follow).
const ROUNDED_CLASS = {
  none: 'rounded-q-0',
  sm: 'rounded-q-150',
  md: 'rounded-q-300',
  lg: 'rounded-q-500',
  full: 'rounded-q-full',
} satisfies Record<MediaRounded, string>

const OVERLAY_PLACEMENT_CLASS = {
  fill: 'q-media-overlay-fill',
  top: 'q-media-overlay-top',
  bottom: 'q-media-overlay-bottom',
  center: 'q-media-overlay-center',
} satisfies Record<MediaOverlayPlacement, string>

export type MediaRootProps = Omit<ComponentProps<'div'>, 'children'> & {
```

```
  /**
   * Aspect ratio. A preset name, or a number (e.g. `16 / 9`, `4 / 3`) for a
   * custom ratio applied via the `aspect-ratio` CSS property.
   */
  ratio?: MediaRatio | number
  /** Corner radius token. The box clips, so the media follows the curve. */
  rounded?: MediaRounded
  children?: ReactNode
}

function Root({ ratio = 'video', rounded = 'md', className, style, children, ...props }: MediaRootProps) {
  const numeric = typeof ratio === 'number'
  return (
    <div
      className={cx(
        'q-media',
        numeric ? undefined : RATIO_CLASS[ratio],
        ROUNDED_CLASS[rounded],
        className,
      )}
      // The custom-ratio var is the one dynamic value that cannot be a class.
      style={numeric ? ({ '--q-media-ratio': ratio, ...style } as CSSProperties) : style}
      {...props}
    >
      {children}
    </div>
  )
}

export type MediaImageProps = Omit<ComponentProps<'img'>, 'children'> & {
  /** object-fit. `cover` (default) crops to fill; `contain` letterboxes. */
  fit?: MediaFit
}

function Image({ fit = 'cover', loading = 'lazy', decoding = 'async', className, alt = '', ...props }: MediaImageProps) {
  return (
    <img
      alt={alt}
      loading={loading}
      decoding={decoding}
      className={cx('q-media-fill', FIT_CLASS[fit], className)}
      {...props}
    />
  )
}

export type MediaVideoProps = Omit<ComponentProps<'video'>, 'children'> & {
  /** object-fit. `cover` (default) crops to fill; `contain` letterboxes. */
  fit?: MediaFit
  /**
   * Autoplay (muted) ONLY while the video is on screen, and pause when it scrolls
   * away — the optimized feed/gallery pattern (IntersectionObserver, no scroll
   * listeners). Forces `muted` + `playsInline` (required for programmatic
   * autoplay) and lazies `preload` to `metadata`. Pass `loop` for short clips.
   */
  autoPlayInView?: boolean
  /** Visibility ratio (0..1) at which an `autoPlayInView` clip starts. Default 0.5. */
  inViewThreshold?: number
}

function Video({
  fit = 'cover',
  autoPlayInView = false,
  inViewThreshold = 0.5,
  muted,
  playsInline,
  preload,
  className,
  ref: forwardedRef,
  ...props
```

```
}: MediaVideoProps) {
  const { ref: inViewRef, inView } = useInView<HTMLVideoElement>({ threshold: inViewThreshold })
  const videoRef = useRef<HTMLVideoElement | null>(null)

  // Fan the node out to our play/pause ref, the in-view observer, and any
  // forwarded ref — only needed in the autoplay path.
  const setRef = useCallback(
    (node: HTMLVideoElement | null) => {
      videoRef.current = node
      inViewRef(node)
      if (typeof forwardedRef === 'function') forwardedRef(node)
      else if (forwardedRef != null) (forwardedRef as MutableRefObject<HTMLVideoElement | null>).current = node
    },
    [inViewRef, forwardedRef],
  )

  useEffect(() => {
    if (!autoPlayInView) return
    const video = videoRef.current
    if (video == null) return
    if (inView) {
      // Autoplay can reject (no gesture / not muted) — swallow it, the poster shows.
      void video.play()?.catch(() => {})
    } else {
      video.pause()
    }
  }, [autoPlayInView, inView])

  return (
    <video
      ref={autoPlayInView ? setRef : forwardedRef}
      className={cx('q-media-fill', FIT_CLASS[fit], className)}
      muted={autoPlayInView ? true : muted}
      playsInline={autoPlayInView ? true : playsInline}
      preload={preload ?? (autoPlayInView ? 'metadata' : undefined)}
      {...props}
    />
  )
}

export type MediaFallbackProps = ComponentProps<'div'>

/** The empty / broken-source slot — a tinted box with centered content. */
function Fallback({ className, ...props }: MediaFallbackProps) {
  return <div className={cx('q-media-fallback', className)} {...props} />
}

export type MediaOverlayProps = ComponentProps<'div'> & {
  /** Where the overlay sits: full bleed, a top/bottom band, or centered. */
  placement?: MediaOverlayPlacement
}

/** An absolutely-positioned layer — gradient scrim, play button, corner badge. */
function Overlay({ placement = 'fill', className, ...props }: MediaOverlayProps) {
  return <div className={cx('q-media-overlay', OVERLAY_PLACEMENT_CLASS[placement], className)} {...props} />
}

export type MediaCaptionProps = ComponentProps<'div'>

/** A small label region. Compose Typography / Badge / text inside it. */
function Caption({ className, ...props }: MediaCaptionProps) {
  return <div className={cx('q-media-caption', className)} {...props} />
}

export const Media = Object.assign(Root, {
  Root,
  Image,
  Video,
  Fallback,
  Overlay,
```

```
  Caption,
})

/**
 * Convenience hook for the broken-source pattern: wire `onError` to flip a flag,
 * then render `Media.Fallback` instead of `Media.Image`. Mirrors Avatar's
 * onError→fallback flow without forcing it on every consumer.
 *
 *   const { failed, onError } = useMediaFallback()
 *   {failed ? <Media.Fallback>…</Media.Fallback>
 *           : <Media.Image src={src} onError={onError} />}
 */
export function useMediaFallback() {
  const [failed, setFailed] = useState(false)
  return { failed, onError: () => setFailed(true), reset: () => setFailed(false) }
}
```

app/packages/quanta/src/components/menu/menu.css

```
/**
 * Shared menu ROW primitives (q-menu-* namespace).
 *
 * These are the cross-component building blocks for menu-style rows, matched to
 * the Figma _MenuItem (236:3238) / _MenuLabel (537:346). They are intentionally
 * presentation-only and data-agnostic: a row is `q-menu-item`, its parts are
 * `q-menu-item-icon` / `-label` / `-title-row` / `-title` / `-description` /
 * `-trailing`, and a section heading is `q-menu-group-label`.
 *
 * Consumers:
 *   • dropdown   — composes these inside `Dropdown.Item` (see dropdown.css for
 *                  the popup / search / media / submenu chrome).
 *   • navigation-menu — reuses the row + parts.
 *   • cmdk       — reuses `q-menu-item`, `q-menu-item-icon`, `q-menu-group-label`.
 *
 * Keep this file to the SHARED primitives only. Component-specific chrome lives
 * in that component's own stylesheet.
 */

/* @source the consumers so Tailwind keeps these utilities (they're referenced
 * from TSX in sibling packages consumed as a symlink). */
@source "../dropdown/dropdown.tsx";
@source "../navigation-menu/navigation-menu.tsx";
@source "../cmdk/cmdk.tsx";

/* ■■ Row ■■■
```

```
}

/* Destructive row (Figma _MenuActions "Delete", 1343:2626): red title + icon.
 * Hover still layers overlay-hover underneath — the danger colour persists. */
.q-menu-item-danger {
  color: var(--hf-color-text-danger);
}
.q-menu-item-danger .q-menu-item-icon,
.q-menu-item-danger .q-menu-item-trailing {
  color: var(--hf-color-icon-error);
}

/* ■■ Parts ■■■
```

```
  margin-inline-start: auto;
}

/* ■■ Section heading (Figma _MenuLabel) ■■■
```

app/packages/quanta/src/components/modal/index.ts

```
export { Modal, modal } from './modal.tsx'
export type { ModalOptions, ModalSize } from './modal.tsx'
```

app/packages/quanta/src/components/modal/modal.css

```
/**
 * Modal presentation utilities (q-namespaced), pixel-matched to the Figma Modal
 * system (Components file, node 1947:1403 — container + Header/Body/Footer types).
 *
 * Base UI Dialog owns focus trap, scroll lock, escape, ARIA and exit-mount
 * timing (it holds the popup mounted until these CSS transitions finish via the
 * Web Animations API), so these utilities only paint and animate.
 *
 * Glass model (glass is on the MODAL, not the overlay):
 *   • Backdrop is a plain dim scrim — NO blur.
 *   • Popup is the glass: background-glass + backdrop-filter, frosting what's
 *     behind the panel. Header, Footer and the side-padding strips are this glass.
 *   • Body is a lighter inset "window" (overlay-dim-soft = white 5%, radius 16).
 *
 * radius / motion are NOT emitted as quanta tokens, so — like button.css —
 * they are authored from spacing tokens (`var(--hf-space-*)`) + native curves.
 * The glass box-shadow now uses the emitted `var(--hf-shadow-modal)` composite.
 * Fixed Figma rem widths and sub-pixel geometry are the only literals
 * (documented exception).
 */

@source "./modal.tsx";

/* ■■ Backdrop — dim scrim only (no blur; glass lives on the panel). ■■■■■■■■■■■■ */
@utility q-modal-backdrop {
  position: fixed;
  inset: var(--hf-space-0);
  z-index: var(--hf-z-index-overlay);
  background-color: var(--hf-color-overlay-scrim);
  transition-property: opacity;
  transition-duration: var(--hf-duration-normal); /* 200ms */
  transition-timing-function: var(--hf-ease-out); /* cubic-bezier(0, 0, 0.2, 1) */

  &[data-starting-style],
  &[data-ending-style] {
    opacity: 0;
  }
}

/* ■■ Popup — centered frosted glass card (container = the glass). ■■■■■■■■■■■■■■■ */
@utility q-modal {
  position: fixed;
  left: 50%;
  top: 50%;
  transform: translate(-50%, -50%);
  z-index: var(--hf-z-index-modal);

  box-sizing: border-box;
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-200); /* 8px */
  width: min(var(--q-modal-width), calc(100vw - var(--hf-space-800)));
  max-height: calc(100dvh - var(--hf-space-1600));
  padding: var(--hf-space-200); /* 8px */

  background-color: var(--hf-color-background-glass);
  backdrop-filter: blur(var(--hf-space-1000)); /* 40px — Figma _Modal background blur */
  /* Figma _Modal (1976:1460) carries no border — only the Glass effect. */
  border-radius: var(--hf-radius-600); /* 24px */

  outline: none;
  overflow: clip;

  transition-property: opacity, transform;
  transition-duration: var(--hf-duration-normal); /* 200ms */
  transition-timing-function: var(--hf-ease-out); /* ease-out (enter) — cubic-bezier(0, 0, 0.2, 1) */

  &[data-starting-style],
```

```
  &[data-ending-style] {
    opacity: 0;
    transform: translate(-50%, -50%) scale(0.96);
  }

  &[data-ending-style] {
    transition-duration: var(--hf-duration-150); /* 150ms */
    transition-timing-function: var(--hf-ease-in); /* ease-in (exit) — cubic-bezier(0.4, 0, 1, 1) */
  }
}

/* Figma size variants — each maps the active width to its generated per-size
 * override knob (defaults live on :root in css/components/component.css). */
@utility q-modal-size-xs {
  --q-modal-width: var(--q-modal-width-xs); /* 349px — New folder / Sub window */
}
@utility q-modal-size-sm {
  --q-modal-width: var(--q-modal-width-sm); /* 469px — Small / Medium semantic sizes */
}
@utility q-modal-size-md {
  --q-modal-width: var(--q-modal-width-md); /* 640px — Default / master */
}
@utility q-modal-size-lg {
  --q-modal-width: var(--q-modal-width-lg); /* 708px — Grid / Selector / Search / Tabs */
}
@utility q-modal-size-xl {
  --q-modal-width: var(--q-modal-width-xl); /* 946px — Left sidebar */
}
@utility q-modal-size-2xl {
  --q-modal-width: var(--q-modal-width-2xl); /* 1392px — Large */
}

/* ■■ Header — transparent (sits on the container glass). Figma _Modal/Header
 * (2034:1773): top-level gap 4 / padding 0. Title and controls own their 40px
 * row geometry. ■■ */
@utility q-modal-header {
  box-sizing: border-box;
  display: flex;
  align-items: center;
  flex-shrink: 0;
  gap: var(--hf-space-100);
  min-height: var(--hf-space-1000);
  overflow: clip;
  padding: var(--hf-space-0);
}

/* Search / tabs headers run flush to the container padding (Figma px 0). */
@utility q-modal-header-flush {
  padding: var(--hf-space-0);
}

/* leading group (back button + title) for the "back" header. */
@utility q-modal-header-lead {
  display: flex;
  align-items: center;
  flex: 1 1 var(--hf-space-0);
  gap: var(--hf-space-100);
  min-width: var(--hf-space-0);
  overflow: clip;
}

.q-modal-header > .q-close,
.q-modal-header-lead > .q-close {
  margin: var(--hf-space-100);
}

@utility q-modal-title {
  align-items: center;
  flex: 1 1 var(--hf-space-0);
  display: flex;
```

```
  min-height: var(--hf-space-1000);
  min-width: var(--hf-space-0);
  margin: var(--hf-space-0);
  overflow: hidden;
  padding: var(--hf-space-200);
  text-overflow: ellipsis;
  white-space: nowrap;
  color: var(--hf-color-text-primary);
  @apply text-q-body-md-semi-bold; /* Inter Semi Bold 16px */
}

@utility q-modal-description {
  margin: var(--hf-space-0);
  padding: var(--hf-space-0) var(--hf-space-200); /* align under the header (px 8) */
  color: var(--hf-color-text-secondary);
  @apply text-q-body-sm-regular;
}

/* flex spacer that pushes a trailing control to the right. */
@utility q-modal-spacer {
  flex: 1 1 var(--hf-space-0);
}

/* ■■ Body — neutral scroll region; the caller arranges the Workspaces. ■■■■■■■ *
 * The body carries NO window surface and imposes NO layout direction: drop a
 * single Workspace to fill it, or your own div (flex row / column / grid) that
 * holds several Workspaces, in any order. It's a vertical-scroll column with
 * `min-height: 0` so a flex chain shrinks-to-scroll instead of cropping. */
@utility q-modal-body {
  box-sizing: border-box;
  display: flex;
  flex-direction: column;
  /* basis auto (NOT 0): size to content so the body doesn't collapse in a
   * content-sized popup; min-height 0 lets it shrink-to-scroll when capped. */
  flex: 1 1 auto;
  min-height: 0;
  width: 100%;
  gap: var(--hf-space-200); /* 8px between stacked children */
  overflow: auto;
}

/* ■■ Workspace — the lighter inset "window" (Figma transparent/dark/05 = white 5%). ■■ */
@utility q-modal-workspace {
  box-sizing: border-box;
  flex: 1 1 auto; /* fills a lone body, shares a row/column; caller can override */
  min-width: 0;
  min-height: 0; /* allow shrink so the pane scrolls instead of cropping */
  overflow: auto;
  border-radius: var(--hf-radius-400); /* 16px */
  background-color: var(--hf-color-transparent-light-05);
  color: var(--hf-color-text-primary);
}

@utility q-modal-workspace-padded {
  padding: var(--hf-space-300); /* 12px */
}

/* ■■ Footer — transparent; Figma _Modal/Footer = 48px track. Caption/actions
 * own their inner padding; full footer adds px8/py4 around the filled button. */
@utility q-modal-footer {
  box-sizing: border-box;
  display: flex;
  align-items: center;
  flex-shrink: 0;
  gap: var(--hf-space-100);
  min-height: var(--hf-space-1200);
  overflow: clip;
}

@utility q-modal-footer-full {
```

```
  gap: var(--hf-space-0);
  padding: var(--hf-space-100) var(--hf-space-200);
}

@utility q-modal-caption {
  align-items: center;
  flex: 1 1 var(--hf-space-0);
  display: flex;
  min-height: var(--hf-space-1200);
  min-width: var(--hf-space-0);
  overflow: hidden;
  padding: var(--hf-space-200);
  text-overflow: ellipsis;
  white-space: nowrap;
  color: var(--hf-color-text-on-overlay-secondary);
  @apply text-q-caption-sm-regular;
}

@utility q-modal-actions {
  display: flex;
  align-items: center;
  flex-shrink: 0;
  gap: var(--hf-space-150);
  margin-inline-start: auto;
  overflow: clip;
  padding: var(--hf-space-100);
}

/* actions that fill the footer width (Figma footer type=full). */
@utility q-modal-actions-full {
  flex: 1 1 var(--hf-space-0);
  padding: var(--hf-space-0);

  & > * {
    flex: 1 1 var(--hf-space-0);
  }
}

/* The close / back disc is the shared CloseButton (q-close, node 2052:109). */

/* ■■ Search row (header type=search) — Figma _MenuSearch (2034:1718): magnifier
 * + input, px12, h40, fully-rounded pill, white-5% fill. */
@utility q-modal-search {
  box-sizing: border-box;
  display: flex;
  flex: 1 1 var(--hf-space-0);
  align-items: center;
  gap: var(--hf-space-200); /* 8px */
  min-height: var(--hf-space-1000);
  min-width: var(--hf-space-0);
  padding: var(--hf-space-0) var(--hf-space-300);
  border-radius: var(--hf-radius-full);
  background-color: var(--hf-color-overlay-dim-soft); /* white 5% */
  color: var(--hf-color-icon-on-overlay-secondary);
}

@utility q-modal-search-icon {
  display: inline-flex;
  flex-shrink: 0;
  align-items: center;
  justify-content: center;
  height: var(--hf-icon-md); /* 20px — Figma _MenuSearch keeps the icon at 20px */
  width: var(--hf-icon-md);
  /* The glyph is wrapped in <Icon size="md"> (icon.css owns svg sizing). */
}

@utility q-modal-search-input {
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  appearance: none;
```

```
  border: var(--hf-border-width-none) solid transparent;
  background: transparent;
  padding: var(--hf-space-0);
  outline: none;
  color: var(--hf-color-text-primary);
  @apply text-q-body-sm-regular; /* Inter Regular 14px */

  &::placeholder {
    color: var(--hf-color-text-on-overlay-secondary); /* white 50% */
  }
}

/* Respect reduced-motion: snap open/close instead of fading + scaling. */
@media (prefers-reduced-motion: reduce) {
  .q-modal-backdrop,
  .q-modal {
    transition-duration: 1ms; /* no token: near-instant snap; --hf-duration-instant is 0ms */
  }

  .q-modal[data-starting-style],
  .q-modal[data-ending-style] {
    transform: translate(-50%, -50%);
  }
}
```

app/packages/quanta/src/components/modal/modal.test.tsx

```
import { render, screen, waitFor } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { describe, expect, it } from 'vitest'
import { Modal, modal } from './index.ts'

describe('modal() class-builder', () => {
  it('defaults to the md size', () => {
    expect(modal()).toBe('q-modal q-modal-size-md')
  })

  it('applies size and extra classes', () => {
    expect(modal({ size: 'lg' }, 'custom')).toBe('q-modal q-modal-size-lg custom')
  })

  it('includes the compact Figma xs size', () => {
    expect(modal({ size: 'xs' })).toBe('q-modal q-modal-size-xs')
  })
})

describe('<Modal> composition', () => {
  it('renders the Base UI dialog with composed quanta parts', () => {
    render(
      <Modal.Root defaultOpen>
        <Modal.Content>
          <Modal.Header>
            <Modal.Title>Modal title</Modal.Title>
            <Modal.CloseButton />
          </Modal.Header>
          <Modal.Body>Body content</Modal.Body>
          <Modal.Footer>
            <Modal.FooterCaption>Footer caption</Modal.FooterCaption>
            <Modal.FooterActions><button type="button">Confirm</button></Modal.FooterActions>
          </Modal.Footer>
        </Modal.Content>
      </Modal.Root>,
    )

    const dialog = screen.getByRole('dialog', { name: 'Modal title' })
    expect(dialog).toHaveClass('q-modal', 'q-modal-size-md')
    expect(screen.getByText('Body content')).toHaveClass('q-modal-workspace', 'q-modal-workspace-padded')
    expect(screen.getByText('Footer caption')).toHaveClass('q-modal-caption')
    expect(screen.getByRole('button', { name: 'Confirm' }).closest('.q-modal-actions')).not.toBeNull()
    const closeButton = screen.getByRole('button', { name: 'Close' })
    expect(closeButton).toHaveClass('q-close')
    expect(closeButton.querySelector('.q-icon')).not.toBeNull()
  })

  it('composes the back / search / tabs header layouts', () => {
    const back = render(
      <Modal.Root defaultOpen>
        <Modal.Content aria-label="Back">
          <Modal.Header>
            <Modal.HeaderLead>
              <Modal.BackButton />
              <Modal.Title>Step 2</Modal.Title>
            </Modal.HeaderLead>
            <Modal.Spacer />
            <Modal.CloseButton />
          </Modal.Header>
        </Modal.Content>
      </Modal.Root>,
    )
    expect(back.container.ownerDocument.querySelector('.q-modal-header-lead')).not.toBeNull()
    const backButton = back.getByRole('button', { name: 'Back' })
    expect(backButton).toHaveClass('q-close')
    expect(backButton.querySelector('.q-icon')).not.toBeNull()
    back.unmount()
```

```
  const search = render(
    <Modal.Root defaultOpen>
      <Modal.Content aria-label="Search">
        <Modal.Header flush>
          <Modal.Search placeholder="Find…" />
          <Modal.CloseButton />
        </Modal.Header>
      </Modal.Content>
    </Modal.Root>,
  )
  expect(search.getByPlaceholderText('Find…')).toHaveClass('q-modal-search-input')
  expect(search.container.ownerDocument.querySelector('.q-modal-header-flush')).not.toBeNull()
  expect(search.container.ownerDocument.querySelector('.q-modal-search-icon .q-icon')).not.toBeNull()
  search.unmount()

  const tabs = render(
    <Modal.Root defaultOpen>
      <Modal.Content aria-label="Tabs">
        <Modal.Header flush>
          <div data-testid="pill">tabs</div>
          <Modal.Spacer />
          <Modal.CloseButton />
        </Modal.Header>
      </Modal.Content>
    </Modal.Root>,
  )
  expect(tabs.getByTestId('pill')).toBeInTheDocument()
})

it('wraps plain body content in a single window, but keeps explicit Workspaces', () => {
  const single = render(
    <Modal.Root defaultOpen>
      <Modal.Content aria-label="Single"><Modal.Body>Plain</Modal.Body></Modal.Content>
    </Modal.Root>,
  )
  expect(single.container.ownerDocument.querySelectorAll('.q-modal-workspace')).toHaveLength(1)
  expect(single.getByText('Plain')).toHaveClass('q-modal-workspace')
  single.unmount()

  const split = render(
    <Modal.Root defaultOpen>
      <Modal.Content size="xl" aria-label="Split">
        <Modal.Body>
          <Modal.Workspace className="w-40 flex-none">Nav</Modal.Workspace>
          <Modal.Workspace>Content</Modal.Workspace>
        </Modal.Body>
      </Modal.Content>
    </Modal.Root>,
  )
  expect(split.container.ownerDocument.querySelectorAll('.q-modal-workspace')).toHaveLength(2)
  expect(split.getByText('Nav')).toHaveClass('q-modal-workspace')
  expect(split.getByText('Content')).toHaveClass('q-modal-workspace')
})

it('applies the size preset to the popup', () => {
  render(
    <Modal.Root defaultOpen>
      <Modal.Content size="lg" aria-label="Large modal"><Modal.Body /></Modal.Content>
    </Modal.Root>,
  )
  expect(screen.getByRole('dialog', { name: 'Large modal' })).toHaveClass('q-modal-size-lg')
})

it('stretches footer actions when full', () => {
  render(
    <Modal.Root defaultOpen>
      <Modal.Content aria-label="Full footer">
        <Modal.Footer full>
          <Modal.FooterActions full><button type="button">Done</button></Modal.FooterActions>
        </Modal.Footer>
```

```
        </Modal.Content>
      </Modal.Root>,
    )
    expect(document.querySelector('.q-modal-footer-full')).not.toBeNull()
    expect(screen.getByRole('button', { name: 'Done' }).closest('.q-modal-actions-full')).not.toBeNull()
  })

  it('orders a trailing header control before the close button', () => {
    render(
      <Modal.Root defaultOpen>
        <Modal.Content>
          <Modal.Header>
            <Modal.Title>With actions</Modal.Title>
            <button type="button">Settings</button>
            <Modal.CloseButton />
          </Modal.Header>
        </Modal.Content>
      </Modal.Root>,
    )
    const settings = screen.getByRole('button', { name: 'Settings' })
    const close = screen.getByRole('button', { name: 'Close' })
    expect(settings.compareDocumentPosition(close) & Node.DOCUMENT_POSITION_FOLLOWING).toBeTruthy()
  })

  it('renders a header with no close affordance (body-only sheet)', () => {
    render(
      <Modal.Root defaultOpen>
        <Modal.Content aria-label="Sheet">
          <Modal.Header><Modal.Title>No close</Modal.Title></Modal.Header>
        </Modal.Content>
      </Modal.Root>,
    )
    expect(screen.getByText('No close')).toHaveClass('q-modal-title')
    expect(screen.queryByRole('button', { name: 'Close' })).not.toBeInTheDocument()
  })

  it('closes when the close button is pressed', async () => {
    const user = userEvent.setup()
    render(
      <Modal.Root defaultOpen>
        <Modal.Content>
          <Modal.Header>
            <Modal.Title>Dismissable</Modal.Title>
            <Modal.CloseButton />
          </Modal.Header>
          <Modal.Body>Body</Modal.Body>
        </Modal.Content>
      </Modal.Root>,
    )
    expect(screen.getByRole('dialog', { name: 'Dismissable' })).toBeInTheDocument()
    await user.click(screen.getByRole('button', { name: 'Close' }))
    await waitFor(() =>
      expect(screen.queryByRole('dialog', { name: 'Dismissable' })).not.toBeInTheDocument(),
    )
  })
})
```

app/packages/quanta/src/components/modal/modal.tsx

```
'use client'

import type { ComponentProps, ReactNode, Ref } from 'react'
import { IconChevronLeftMediumOutlined as ChevronLeftIcon } from '@higgsfield-ai/icons/IconChevronLeftMediumOutlined'
import { IconMagnifyingGlass2Outlined as SearchIcon } from '@higgsfield-ai/icons/IconMagnifyingGlass2Outlined'
import { Children, isValidElement, useRef } from 'react'
import { Dialog as Primitive } from '@base-ui/react/dialog'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'
import { CloseIcon, closeButton } from '../close-button/index.ts'
import { Icon } from '../icon/index.ts'

/**
 * Modal — Base UI Dialog (focus trap, scroll lock, escape, a11y, portal, and
 * exit-mount timing) skinned with quanta tokens, pixel-matched to the Figma
 * modal system (node 1947:1403). Dialog is centered (no Positioner), so the
 * glass card lives on `Popup` (z-q-modal) and the dim scrim on `Backdrop`.
 *
 * COMPOSITION-FIRST (same rules as Dropdown / NavigationMenu / Sidebar). The
 * component owns the DESIGN — the glass card, the 40px header row, the inset
 * body window, the 48px footer — and every title / control / caption / action is
 * CONTENT you compose. Header and Footer hold ANY nodes:
 *
 *   <Modal.Root>
 *     <Modal.Trigger render={<Button>Open</Button>} />
 *     <Modal.Content size="md">
 *       <Modal.Header>
 *         <Modal.Title>New element</Modal.Title>
 *         <Modal.CloseButton />
 *       </Modal.Header>
 *       <Modal.Body><Modal.Workspace>…</Modal.Workspace></Modal.Body>
 *       <Modal.Footer>
 *         <Modal.FooterCaption>{caption}</Modal.FooterCaption>
 *         <Modal.FooterActions>
 *           <Button variant="secondary">Cancel</Button>
 *           <Button>Confirm</Button>
 *         </Modal.FooterActions>
 *       </Modal.Footer>
 *     </Modal.Content>
 *   </Modal.Root>
 *
 * Header layouts (Figma default / back / search / tabs) are just different
 * compositions: drop a `Title` + `CloseButton`, a `BackButton` inside a
 * `HeaderLead`, a `Search`, or a Tabs pill + `Spacer` + `CloseButton`. Add
 * `flush` to a header whose row/pill spans the full width (search / tabs).
 */

export type ModalSize = 'xs' | 'sm' | 'md' | 'lg' | 'xl' | '2xl'

const SIZE_CLASS = {
  xs: 'q-modal-size-xs',
  sm: 'q-modal-size-sm',
  md: 'q-modal-size-md',
  lg: 'q-modal-size-lg',
  xl: 'q-modal-size-xl',
  '2xl': 'q-modal-size-2xl',
} satisfies Record<ModalSize, string>

export interface ModalOptions {
  size?: ModalSize
}

/** Build the modal popup class string. Also usable to style a non-popup element. */
export function modal(options: ModalOptions = {}, ...extra: ClassValue[]): string {
  const { size = 'md' } = options
  return cx('q-modal', SIZE_CLASS[size], ...extra)
}
```

```
/** Populate every ref in `refs` with the same node (object or callback refs). */
function mergeRefs<T>(...refs: (Ref<T> | undefined)[]) {
  return (node: T | null) => {
    for (const ref of refs) {
      if (typeof ref === 'function')
        ref(node)
      else if (ref)
        (ref as { current: T | null }).current = node
    }
  }
}

/* ■■ Passthrough parts (Base UI owns behavior; quanta only names them). ■■■■■■■■ */
const Root = Primitive.Root
const Trigger = Primitive.Trigger
/** Raw dismiss trigger — wrap a Button via `render`, or pass your own children. */
const Close = Primitive.Close

type TitleProps = Omit<ComponentProps<typeof Primitive.Title>, 'className'> & { className?: string }
function Title({ className, ...props }: TitleProps) {
  return <Primitive.Title className={cx('q-modal-title', className)} {...props} />
}

type DescriptionProps = Omit<ComponentProps<typeof Primitive.Description>, 'className'> & { className?: string }
function Description({ className, ...props }: DescriptionProps) {
  return <Primitive.Description className={cx('q-modal-description', className)} {...props} />
}

type CloseButtonProps = Omit<ComponentProps<typeof Primitive.Close>, 'className'> & { className?: string }
/** Styled round dismiss button (Figma disc) — sits at the trailing end of a header. */
function CloseButton({ className, children, ...props }: CloseButtonProps) {
  return (
    <Primitive.Close aria-label="Close" className={closeButton({}, className)} {...props}>
      {children ?? <Icon as={CloseIcon} size="md" />}
    </Primitive.Close>
  )
}

type BackButtonProps = Omit<ComponentProps<'button'>, 'className'> & { className?: string }
/** Styled round back button (Figma disc) for the "back" header. */
function BackButton({ className, children, type, ...props }: BackButtonProps) {
  return (
    <button type={type ?? 'button'} aria-label="Back" className={closeButton({}, className)} {...props}>
      {children ?? <Icon as={ChevronLeftIcon} size="md" />}
    </button>
  )
}

type SearchProps = Omit<ComponentProps<'input'>, 'className' | 'size'> & {
  className?: string
  inputClassName?: string
  icon?: ReactNode
}
/** Search row for the "search" header (magnifier + input). */
function Search({ className, inputClassName, icon, placeholder = 'Search', type, ...props }: SearchProps) {
  return (
    <div className={cx('q-modal-search', className)}>
      <span className="q-modal-search-icon">{icon ?? <Icon as={SearchIcon} size="md" />}</span>
      <input className={cx('q-modal-search-input', inputClassName)} placeholder={placeholder} type={type ?? 'search'} {...props} />
    </div>
  )
}

/* ■■ Content: Portal + Backdrop + the centered glass Popup ■■■■■■■■■■■■■■■■■■■■ */
type ContentProps = Omit<ComponentProps<typeof Primitive.Popup>, 'className'> & {
  className?: string
  /** Width preset (Figma sizes). Use className/style for one-off dimensions. */
  size?: ModalSize
  backdropClassName?: string
  /** Portal mount node. Defaults to document.body. */
```

```
  container?: ComponentProps<typeof Primitive.Portal>['container']
}

function Content({ size = 'md', backdropClassName, container, className, children, initialFocus, ref, ...props }: ContentProps) {
  // Focus the popup itself (not the first tabbable) so opening doesn't ring the ✕.
  const popupRef = useRef<HTMLDivElement>(null)
  return (
    <Primitive.Portal container={container}>
      <Primitive.Backdrop className={cx('q-modal-backdrop', backdropClassName)} />
      <Primitive.Popup
        ref={mergeRefs(popupRef, ref)}
        initialFocus={initialFocus ?? popupRef}
        className={modal({ size }, className)}
        {...props}
      >
        {children}
      </Primitive.Popup>
    </Primitive.Portal>
  )
}

/* ■■ Header: a 40px row holding composed controls ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
type HeaderProps = ComponentProps<'div'> & {
  /** Run the header content flush to the card padding (Figma search / tabs headers). */
  flush?: boolean
}
function Header({ flush = false, className, ...props }: HeaderProps) {
  return <div className={cx('q-modal-header', flush && 'q-modal-header-flush', className)} {...props} />
}

/** Leading group inside a header (e.g. a BackButton + Title) for the "back" layout. */
function HeaderLead({ className, ...props }: ComponentProps<'div'>) {
  return <div className={cx('q-modal-header-lead', className)} {...props} />
}

/** Flex spacer — pushes following controls to the trailing end (header or footer). */
function Spacer({ className, ...props }: ComponentProps<'span'>) {
  return <span aria-hidden className={cx('q-modal-spacer', className)} {...props} />
}

/* ■■ Workspace + Body ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
type WorkspaceProps = ComponentProps<'div'> & {
  /** Apply the default content padding. Set false for edge-to-edge content. */
  padded?: boolean
}

/**
 * The inset "window" — a frosted, lighter pane inside the body. Place a single
 * one to fill the body, or several inside your own layout div (flex row /
 * column / grid) for split layouts like the Figma "Left sidebar" / "Selector".
 */
function Workspace({ className, padded = true, ...props }: WorkspaceProps) {
  return <div className={cx('q-modal-workspace', padded && 'q-modal-workspace-padded', className)} {...props} />
}

type BodyProps = ComponentProps<'div'> & {
  /** Padding for the auto-wrapped single Workspace (ignored when you nest your own). */
  padded?: boolean
}

/**
 * Body — the scrollable region between header and footer. It imposes NO layout:
 * arrange Workspaces however you like, or pass a single `Modal.Workspace`. Plain
 * content with no Workspace anywhere is auto-wrapped in one full Workspace so the
 * window effect still applies. Scrolls (never crops) when content overflows.
 */
function Body({ className, padded = true, children, ...props }: BodyProps) {
  const hasWorkspace = (nodes: ReactNode): boolean =>
    Children.toArray(nodes).some(c =>
      isValidElement(c) && (c.type === Workspace || hasWorkspace((c.props as { children?: ReactNode }).children)),
```

```
    )
  return (
    <div className={cx('q-modal-body', className)} {...props}>
      {hasWorkspace(children) ? children : <Workspace padded={padded}>{children}</Workspace>}
    </div>
  )
}

/* ■■ Footer: a 48px row holding a caption + actions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
type FooterProps = ComponentProps<'div'> & {
  /** Stretch the footer for full-width actions (Figma footer type=full). */
  full?: boolean
}
function Footer({ full = false, className, ...props }: FooterProps) {
  return <div className={cx('q-modal-footer', full && 'q-modal-footer-full', className)} {...props} />
}

/** Leading footer caption (muted helper text). */
function FooterCaption({ className, ...props }: ComponentProps<'div'>) {
  return <div className={cx('q-modal-caption', className)} {...props} />
}

type FooterActionsProps = ComponentProps<'div'> & {
  /** Stretch the actions to fill the footer width (each child grows equally). */
  full?: boolean
}
/** Trailing footer actions (buttons), pushed to the right by default. */
function FooterActions({ full = false, className, ...props }: FooterActionsProps) {
  return <div className={cx('q-modal-actions', full && 'q-modal-actions-full', className)} {...props} />
}

export const Modal = {
  Root,
  Trigger,
  Close,
  Content,
  Header,
  HeaderLead,
  Spacer,
  Title,
  Description,
  CloseButton,
  BackButton,
  Search,
  Body,
  Workspace,
  Footer,
  FooterCaption,
  FooterActions,
}
```

app/packages/quanta/src/components/navigation-menu/index.ts

```
export { NavigationMenu } from './navigation-menu.tsx'
export type {
  NavigationMenuActionProps,
  NavigationMenuItemProps,
  NavigationMenuLinkProps,
  NavigationMenuMenuItemProps,
  NavigationMenuMenuProps,
  NavigationMenuRootProps,
  NavigationMenuTriggerProps,
  NavMenuLayout,
  NavMenuSize,
  NavRows,
} from './navigation-menu.tsx'
```

app/packages/quanta/src/components/navigation-menu/navigation-menu.css

```
/**
 * NavigationMenu presentation — Base UI NavigationMenu skinned with quanta
 * tokens. Registers the colocated source (quanta is symlinked) and authors the
 * bar + morphing popup as `@utility q-nav-*` rules.
 *
 * The legacy grid keeps `--q-nav-col-width` overridable; Figma mega-menu
 * variants use exact panel/column sizes from the component board.
 *
 * Default `MenuItem` deliberately REUSES the shared `q-menu-item*` utilities.
 * Large NavigationMenu rows add their own 44px media slot and label treatment.
 * Base UI drives the size-morph via `--popup-width` / `--popup-height` and the
 * cross-fade via `data-activation-direction`.
 */
@source "./navigation-menu.tsx";

/* ■■ The bar ■■■
```

```
  @apply text-q-body-sm-medium;
  text-decoration: none;
  white-space: nowrap;
  cursor: pointer;
  user-select: none;
  transition: background-color var(--hf-duration-150) ease, color var(--hf-duration-150) ease; /* ease keyword not in --hf-ease-* */

  &:hover { background-color: var(--hf-color-overlay-hover); color: var(--hf-color-text-primary); }
  &[data-popup-open] { background-color: var(--hf-color-overlay-hover); color: var(--hf-color-text-primary); }
  /* Current section — Base UI may set data-active; the `active` prop sets aria-current. */
  &[data-active],
  &[aria-current="page"] { color: var(--hf-color-text-primary); }
  &[data-disabled] { color: var(--hf-color-text-disabled); pointer-events: none; }
}

/* Leading icon slot inside a bar item (24px in Figma _HeaderNavItem). */
@utility q-nav-item-icon {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  flex-shrink: 0;
  width: var(--hf-space-600);
  height: var(--hf-space-600);

  & svg { width: 100%; height: 100%; }
}

/* Accent treatment — the lime "Supercomputer" item. Pair with any
 * label/icon/badge; Figma uses brand text without extra glow. */
@utility q-nav-item-accent {
  color: var(--hf-color-brand-primary);

  &:hover,
  &[data-popup-open] { color: var(--hf-color-brand-primary); }
}

/* ■■ Right-side actions cluster ■■■
```

```
  white-space: nowrap;
  cursor: pointer;
  user-select: none;
  transition: background-color var(--hf-duration-150) ease; /* ease keyword not in --hf-ease-* */

  &:hover { background-color: var(--hf-color-overlay-hover); }
  & svg { width: var(--hf-icon-sm); height: var(--hf-icon-sm); } /* 16px glyph */
}

@utility q-nav-action-label {
  padding-inline: var(--hf-space-150);
}

/* Icon-only square action (e.g. search). */
@utility q-nav-action-icon {
  width: var(--hf-space-800);
  min-width: var(--hf-space-800);
  height: var(--hf-space-800);
  padding: var(--hf-space-200);
}

/* Vertical bar divider (12px) — wraps the Divider component so it self-stretches
 * to a fixed height instead of the full bar height. Figma "Line" (node 2206:1044). */
@utility q-nav-separator {
  display: inline-flex;
  align-items: stretch;
  align-self: center;
  flex-shrink: 0;
  height: var(--hf-space-300); /* 12px */
}

/* ■■ The shared morphing popup shell ■■■
```

```
  height: 100%;
  overflow: hidden;
}

/* The panel itself — moved into the viewport when its item is active; sizes to
 * content (which Base UI reads into --popup-*), cross-fades by direction. */
@utility q-nav-content {
  width: max-content;
  height: max-content;
  transition: opacity var(--hf-duration-normal) ease, transform var(--hf-duration-normal) var(--hf-ease-emphasized); /* no token: ease keyword not in --hf-ease-* */

  &[data-starting-style],
  &[data-ending-style] { opacity: 0; }
  &[data-starting-style][data-activation-direction="left"],
  &[data-ending-style][data-activation-direction="right"] { transform: translateX(var(--hf-space-1000)); }
  &[data-starting-style][data-activation-direction="right"],
  &[data-ending-style][data-activation-direction="left"] { transform: translateX(calc(-1 * var(--hf-space-1000))); }
}

/* ■■ The panel layout ■■■
```

```
  align-items: flex-start;
  gap: var(--hf-space-100);
  width: 100%;
}

/* Rows per column before wrapping to the next column. */
@utility q-nav-rows-1 { grid-template-rows: repeat(1, auto); }
@utility q-nav-rows-2 { grid-template-rows: repeat(2, auto); }
@utility q-nav-rows-3 { grid-template-rows: repeat(3, auto); }
@utility q-nav-rows-4 { grid-template-rows: repeat(4, auto); }

/* A group is its own full-height column (heading + stacked items). */
@utility q-nav-group {
  grid-row: 1 / -1;
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-050);
  width: var(--q-nav-col-width);
  min-width: var(--q-nav-col-width);
  flex-shrink: 0;
}

@utility q-nav-group-label {
  padding: var(--hf-space-200);
  color: var(--hf-color-transparent-light-50);
  @apply text-q-label-xs-medium;
}

.q-nav-menu-size-image .q-nav-group:first-child { --q-nav-col-width: 21.375rem; } /* 342px */
.q-nav-menu-size-image .q-nav-group:nth-child(2) { --q-nav-col-width: 21.125rem; } /* 338px */
.q-nav-menu-size-video .q-nav-group:first-child { --q-nav-col-width: 21.375rem; } /* 342px */
.q-nav-menu-size-video .q-nav-group:nth-child(2) { --q-nav-col-width: 23.4375rem; } /* 375px */
.q-nav-menu-size-audio .q-nav-group:first-child { --q-nav-col-width: 15.4375rem; } /* 247px */
.q-nav-menu-size-audio .q-nav-group:nth-child(2) { --q-nav-col-width: 19.3125rem; } /* 309px */
.q-nav-menu-size-plugins .q-nav-group:first-child { --q-nav-col-width: 17.5rem; } /* 280px */

/* Panel row over the shared q-menu-item base — fills its grid/column cell. */
@utility q-nav-menu-item {
  min-width: var(--q-nav-col-width);
  text-decoration: none;
  width: 100%;
}

/* Rich "large" row — applied automatically when the row composes a MenuMedia
 * tile (44px). 60px tall, 12px gap, label/md title. The muted description is the
 * shared q-menu-item-description (on-overlay-secondary = white 50%). */
.q-nav-menu-item:has(.q-nav-menu-media) {
  gap: var(--hf-space-300); /* 12px */
  min-height: calc(var(--hf-space-1200) + var(--hf-space-300)); /* 60px */
}

.q-nav-menu-item:has(.q-nav-menu-media) .q-menu-item-title-row {
  min-height: var(--hf-space-500); /* 20px */
}

.q-nav-menu-item:has(.q-nav-menu-media) .q-menu-item-title {
  @apply text-q-label-md-medium;
}

@utility q-nav-menu-media {
  align-items: center;
  background-color: var(--hf-color-transparent-light-05);
  border: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  border-radius: var(--hf-radius-200);
  box-shadow: var(--hf-shadow-inset-tile-hairline);
  color: var(--hf-color-icon-primary);
  display: inline-flex;
  flex-shrink: 0;
  height: calc(var(--hf-space-1000) + var(--hf-space-100)); /* 44px */
  justify-content: center;
```

```
  overflow: clip;
  width: calc(var(--hf-space-1000) + var(--hf-space-100)); /* 44px */

  & svg,
  & img {
    height: var(--hf-space-600);
    width: var(--hf-space-600);
  }
}

@utility q-nav-menu-separator {
  display: flex;
  flex-shrink: 0;
  padding-inline: var(--hf-space-200);
  width: 100%;
}

@utility q-nav-avatar-item {
  border-radius: var(--hf-radius-full);
  gap: var(--hf-space-0);
  padding: var(--hf-space-0);

  &:hover,
  &[data-popup-open] {
    background-color: transparent;
  }
}

@utility q-nav-header-avatar {
  height: calc(var(--hf-space-800) + var(--hf-space-100)); /* 36px */
  width: calc(var(--hf-space-800) + var(--hf-space-100)); /* 36px */
}

/* Optional side rail (promo / media / CTA). */
@utility q-nav-featured {
  flex: none;
  display: flex;
  flex-direction: column;
  justify-content: flex-end;
  gap: var(--hf-space-200);
  width: calc(var(--hf-space-2400) * 3); /* 18rem */
  padding: var(--hf-space-400);
  border-radius: var(--hf-space-400);
  background-color: var(--hf-color-background-secondary);
  color: var(--hf-color-text-primary);
}

@media (prefers-reduced-motion: reduce) {
  .q-nav-popup,
  .q-nav-content { transition-duration: var(--hf-duration-80); }
}
```

app/packages/quanta/src/components/navigation-menu/navigation-menu.test.tsx

```
import type { ReactNode } from 'react'
import { createRef } from 'react'
import { render, screen, waitFor } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { describe, expect, it } from 'vitest'
import { NavigationMenu } from './index.ts'

/** Compose a simple panel row (title-only). */
function Row({ title, ...props }: { title: string } & Parameters<typeof NavigationMenu.MenuItem>[0]) {
  return (
    <NavigationMenu.MenuItem {...props}>
      <NavigationMenu.MenuItemContent>
        <NavigationMenu.MenuItemTitleRow>
          <NavigationMenu.MenuItemTitle>{title}</NavigationMenu.MenuItemTitle>
        </NavigationMenu.MenuItemTitleRow>
      </NavigationMenu.MenuItemContent>
    </NavigationMenu.MenuItem>
  )
}

/** Compose a rich (large) panel row with a media tile. */
function MediaRow({ title, subtitle, media, ...props }: { title: string, subtitle?: string, media?: ReactNode } & Parameters<typeof NavigationMenu.MenuItem>[0]) {
  return (
    <NavigationMenu.MenuItem {...props}>
      <NavigationMenu.MenuMedia data-testid="media">{media ?? <span />}</NavigationMenu.MenuMedia>
      <NavigationMenu.MenuItemContent>
        <NavigationMenu.MenuItemTitleRow>
          <NavigationMenu.MenuItemTitle>{title}</NavigationMenu.MenuItemTitle>
        </NavigationMenu.MenuItemTitleRow>
        {subtitle ? <NavigationMenu.MenuItemDescription>{subtitle}</NavigationMenu.MenuItemDescription> : null}
      </NavigationMenu.MenuItemContent>
    </NavigationMenu.MenuItem>
  )
}

function setup() {
  render(
    <NavigationMenu.Root>
      <NavigationMenu.List>
        <NavigationMenu.Item>
          <NavigationMenu.Trigger>Products</NavigationMenu.Trigger>
          <NavigationMenu.Content>
            <NavigationMenu.Menu rows={3}>
              <NavigationMenu.Group>
                <NavigationMenu.GroupLabel>Create</NavigationMenu.GroupLabel>
                <Row title="Image" href="/image" />
                <Row title="Video" href="/video" />
              </NavigationMenu.Group>
              <Row title="Pricing" href="/pricing" />
            </NavigationMenu.Menu>
          </NavigationMenu.Content>
        </NavigationMenu.Item>
        <NavigationMenu.Item>
          <NavigationMenu.Link href="/docs">Docs</NavigationMenu.Link>
        </NavigationMenu.Item>
      </NavigationMenu.List>
    </NavigationMenu.Root>,
  )
}

describe('<NavigationMenu>', () => {
  it('renders bar triggers and plain links', () => {
    setup()
    expect(screen.getByRole('button', { name: /products/i })).toBeInTheDocument()
    expect(screen.getByRole('link', { name: 'Docs' })).toHaveAttribute('href', '/docs')
  })

  it('opens the panel and reveals its (grouped + ungrouped) rows', async () => {
```

```
  const user = userEvent.setup()
  setup()
  await user.click(screen.getByRole('button', { name: /products/i }))
  await waitFor(() => expect(screen.getByText('Image')).toBeInTheDocument())
  expect(screen.getByText('Video')).toBeInTheDocument()
  expect(screen.getByText('Pricing')).toBeInTheDocument()
  expect(screen.getByText('Create')).toHaveClass('q-nav-group-label')
  expect(screen.getByRole('link', { name: /image/i })).toHaveAttribute('href', '/image')
})

it('applies the requested row count to the grid', async () => {
  const user = userEvent.setup()
  setup()
  await user.click(screen.getByRole('button', { name: /products/i }))
  await waitFor(() => expect(document.querySelector('.q-nav-menu-grid')).toBeInTheDocument())
  expect(document.querySelector('.q-nav-menu-grid')).toHaveClass('q-nav-rows-3')
})

it('renders Figma-sized column menus with large media rows', async () => {
  const user = userEvent.setup()
  render(
    <NavigationMenu.Root>
      <NavigationMenu.List>
        <NavigationMenu.Item>
          <NavigationMenu.Trigger>Image</NavigationMenu.Trigger>
          <NavigationMenu.Content>
            <NavigationMenu.Menu size="image" layout="columns">
              <NavigationMenu.Group>
                <NavigationMenu.GroupLabel>Features</NavigationMenu.GroupLabel>
                <MediaRow title="Create Image" subtitle="AI image generation" href="/image" />
              </NavigationMenu.Group>
            </NavigationMenu.Menu>
          </NavigationMenu.Content>
        </NavigationMenu.Item>
      </NavigationMenu.List>
    </NavigationMenu.Root>,
  )
  await user.click(screen.getByRole('button', { name: /image/i }))
  await waitFor(() => expect(document.querySelector('.q-nav-menu-size-image')).toBeInTheDocument())
  expect(document.querySelector('.q-nav-menu-columns')).toHaveClass('q-nav-rows-2')
  expect(screen.getByRole('link', { name: /create image/i })).toHaveClass('q-nav-menu-item')
  expect(screen.getByTestId('media')).toHaveClass('q-nav-menu-media')
})

it('renders custom menu content without the grid wrapper', async () => {
  const user = userEvent.setup()
  render(
    <NavigationMenu.Root>
      <NavigationMenu.List>
        <NavigationMenu.Item>
          <NavigationMenu.Trigger>Custom</NavigationMenu.Trigger>
          <NavigationMenu.Content>
            <NavigationMenu.Menu layout="custom">
              <div data-testid="custom-content">Custom content</div>
              <NavigationMenu.MenuSeparator data-testid="custom-separator" />
            </NavigationMenu.Menu>
          </NavigationMenu.Content>
        </NavigationMenu.Item>
      </NavigationMenu.List>
    </NavigationMenu.Root>,
  )
  await user.click(screen.getByRole('button', { name: /custom/i }))
  await waitFor(() => expect(screen.getByTestId('custom-content')).toBeInTheDocument())
  expect(document.querySelector('.q-nav-menu-layout-custom')).toBeInTheDocument()
  expect(document.querySelector('.q-nav-menu-layout-custom .q-nav-menu-grid')).not.toBeInTheDocument()
  expect(screen.getByTestId('custom-separator')).toHaveClass('q-nav-menu-separator')
})

it('can render menu content statically outside a root', () => {
  render(
```

```
    <NavigationMenu.Menu standalone size="plugins" layout="columns">
      <NavigationMenu.Group>
        <NavigationMenu.GroupLabel>Adobe Plugins</NavigationMenu.GroupLabel>
        <MediaRow title="Premiere Pro" subtitle="Higgsfield inside Premiere" href="/premiere" interactive={false} />
      </NavigationMenu.Group>
    </NavigationMenu.Menu>,
  )
  expect(screen.getByText('Adobe Plugins')).toBeInTheDocument()
  expect(screen.getByRole('link', { name: /premiere pro/i })).toHaveAttribute('href', '/premiere')
  expect(screen.getByTestId('media')).toHaveClass('q-nav-menu-media')
  expect(document.querySelector('.q-nav-menu')).toHaveClass('q-nav-menu-static')
})

it('renders logo, an accent link with composed icon/badge, and the actions cluster', () => {
  const { container } = render(
    <NavigationMenu.Root>
      <NavigationMenu.Logo><span data-testid="logo" /></NavigationMenu.Logo>
      <NavigationMenu.List>
        <NavigationMenu.Item>
          <NavigationMenu.Link href="/sc" accent>
            <NavigationMenu.ItemIcon><span data-testid="lead" /></NavigationMenu.ItemIcon>
            Supercomputer
            <span data-testid="badge" />
          </NavigationMenu.Link>
        </NavigationMenu.Item>
      </NavigationMenu.List>
      <NavigationMenu.Actions>
        <NavigationMenu.Action iconOnly aria-label="Search"><span /></NavigationMenu.Action>
        <NavigationMenu.Action href="/pricing">Pricing</NavigationMenu.Action>
      </NavigationMenu.Actions>
    </NavigationMenu.Root>,
  )
  expect(screen.getByTestId('logo').closest('.q-nav-logo')).toBeInTheDocument()
  const sc = screen.getByRole('link', { name: /supercomputer/i })
  expect(sc).toHaveClass('q-nav-item', 'q-nav-item-accent')
  expect(screen.getByTestId('lead').closest('.q-nav-item-icon')).toBeInTheDocument()
  expect(screen.getByTestId('badge')).toBeInTheDocument()
  expect(container.querySelector('.q-nav-actions')).toBeInTheDocument()
  expect(screen.getByRole('button', { name: 'Search' })).toHaveClass('q-nav-action', 'q-nav-action-icon')
  expect(screen.getByRole('link', { name: 'Pricing' })).toHaveAttribute('href', '/pricing')
})

it('Action defaults to a <button> and renders an <a> from href', () => {
  render(
    <NavigationMenu.Root>
      <NavigationMenu.Actions>
        <NavigationMenu.Action>Plain</NavigationMenu.Action>
        <NavigationMenu.Action href="/go">Linked</NavigationMenu.Action>
      </NavigationMenu.Actions>
    </NavigationMenu.Root>,
  )
  const plain = screen.getByRole('button', { name: 'Plain' })
  expect(plain.tagName).toBe('BUTTON')
  expect(plain).toHaveAttribute('type', 'button')
  expect(plain).toHaveClass('q-nav-action')
  const linked = screen.getByRole('link', { name: 'Linked' })
  expect(linked.tagName).toBe('A')
  expect(linked).toHaveAttribute('href', '/go')
  expect(linked).not.toHaveAttribute('type')
})

it('forwards refs to the trigger button and the plain link', () => {
  const triggerRef = createRef<HTMLButtonElement>()
  const linkRef = createRef<HTMLAnchorElement>()
  render(
    <NavigationMenu.Root>
      <NavigationMenu.List>
        <NavigationMenu.Item>
          <NavigationMenu.Trigger ref={triggerRef}>Products</NavigationMenu.Trigger>
          <NavigationMenu.Content>
```

```
            <NavigationMenu.Menu><Row title="Image" href="/image" /></NavigationMenu.Menu>
          </NavigationMenu.Content>
        </NavigationMenu.Item>
        <NavigationMenu.Item>
          <NavigationMenu.Link href="/docs" ref={linkRef}>Docs</NavigationMenu.Link>
        </NavigationMenu.Item>
      </NavigationMenu.List>
    </NavigationMenu.Root>,
  )
  expect(triggerRef.current?.tagName).toBe('BUTTON')
  expect(linkRef.current?.tagName).toBe('A')
  expect(linkRef.current).toHaveAttribute('href', '/docs')
})

it('marks the current section with active (aria-current=page) on a link and a trigger', () => {
  render(
    <NavigationMenu.Root>
      <NavigationMenu.List>
        <NavigationMenu.Item>
          <NavigationMenu.Link href="/image" active>Image</NavigationMenu.Link>
        </NavigationMenu.Item>
        <NavigationMenu.Item>
          <NavigationMenu.Trigger active>Video</NavigationMenu.Trigger>
          <NavigationMenu.Content><NavigationMenu.Menu><Row title="Create" href="/v" /></NavigationMenu.Menu></NavigationMenu.Content>
        </NavigationMenu.Item>
        <NavigationMenu.Item>
          <NavigationMenu.Link href="/docs">Docs</NavigationMenu.Link>
        </NavigationMenu.Item>
      </NavigationMenu.List>
    </NavigationMenu.Root>,
  )
  expect(screen.getByRole('link', { name: 'Image' })).toHaveAttribute('aria-current', 'page')
  expect(screen.getByRole('button', { name: /video/i })).toHaveAttribute('aria-current', 'page')
  expect(screen.getByRole('link', { name: 'Docs' })).not.toHaveAttribute('aria-current')
})

it('Action swaps its element via render, keeping the pill styling', () => {
  render(
    <NavigationMenu.Root>
      <NavigationMenu.Actions>
        <NavigationMenu.Action render={<a href="/custom" data-testid="custom" />}>Custom</NavigationMenu.Action>
      </NavigationMenu.Actions>
    </NavigationMenu.Root>,
  )
  const custom = screen.getByTestId('custom')
  expect(custom.tagName).toBe('A')
  expect(custom).toHaveAttribute('href', '/custom')
  expect(custom).toHaveClass('q-nav-action')
  expect(custom).not.toHaveAttribute('type')
})
})
```

app/packages/quanta/src/components/navigation-menu/navigation-menu.tsx

```
'use client'

import type { ComponentProps, ReactElement, ReactNode, Ref } from 'react'
import { useId } from 'react'
import { NavigationMenu as Primitive } from '@base-ui/react/navigation-menu'
import { useRender } from '@base-ui/react/use-render'
import { Divider } from '../divider/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * NavigationMenu — the product header: a logo, a bar of nav items (each either a
 * plain link or a trigger for a morphing mega-menu panel), and a right-side
 * actions cluster. Built on the Base UI `NavigationMenu` primitive (keyboard +
 * roving focus, hover/click open, the shared size-morphing popup, ARIA, portal +
 * positioning), skinned with quanta's `q-nav-*` utilities.
 *
 * COMPOSITION-FIRST (same rules as Dropdown). No `label` / `title` / `subtitle`
 * / `start` / `end` props — every part is a node you compose, so a bar item or
 * a panel row can hold ANY content. The component owns the design (the bar pill,
 * the glass action pills, the accent treatment, the morphing panel, the row
 * shell + 44px media tile); you own the content.
 *
 *   <NavigationMenu.Root>
 *     <NavigationMenu.Logo><Wordmark/></NavigationMenu.Logo>
 *     <NavigationMenu.List>
 *       <NavigationMenu.Item>                        // trigger + panel
 *         <NavigationMenu.Trigger>
 *           <NavigationMenu.ItemIcon><ImageIcon/></NavigationMenu.ItemIcon>
 *           Image
 *         </NavigationMenu.Trigger>
 *         <NavigationMenu.Content>
 *           <NavigationMenu.Menu size="image" layout="columns">…</NavigationMenu.Menu>
 *         </NavigationMenu.Content>
 *       </NavigationMenu.Item>
 *
 *       <NavigationMenu.Item>                         // plain link
 *         <NavigationMenu.Link href="/sc" accent>
 *           <NavigationMenu.ItemIcon><Spark/></NavigationMenu.ItemIcon>
 *           Supercomputer
 *           <Badge variant="nBrand">new</Badge>
 *         </NavigationMenu.Link>
 *       </NavigationMenu.Item>
 *     </NavigationMenu.List>
 *
 *     <NavigationMenu.Actions>
 *       <NavigationMenu.ActionsGroup>
 *         <NavigationMenu.Action iconOnly aria-label="Search"><Search/></NavigationMenu.Action>
 *         <NavigationMenu.Action href="/pricing"><Diamond/>Pricing<Badge>30% OFF</Badge></NavigationMenu.Action>
 *       </NavigationMenu.ActionsGroup>
 *       <NavigationMenu.Separator />
 *       <Avatar … />
 *     </NavigationMenu.Actions>
 *   </NavigationMenu.Root>
 *
 * PANEL ROWS are pure composition too — a `MenuItem` is a styled row that holds
 * whatever parts you nest. A row that contains a `MenuMedia` (the 44px tile)
 * automatically becomes the rich 60px "large" row (`label/md` title, muted
 * description) — driven by the composed content, not a prop.
 */

/** Rows of items a column holds before wrapping to the next column. */
export type NavRows = 1 | 2 | 3 | 4

const ROWS_CLASS = {
  1: 'q-nav-rows-1',
  2: 'q-nav-rows-2',
  3: 'q-nav-rows-3',
  4: 'q-nav-rows-4',
```

```
} satisfies Record<NavRows, string>

export type NavMenuSize = 'auto' | 'image' | 'video' | 'audio' | 'plugins'
export type NavMenuLayout = 'grid' | 'columns' | 'custom'

const MENU_SIZE_CLASS = {
  auto: '',
  image: 'q-nav-menu-size-image',
  video: 'q-nav-menu-size-video',
  audio: 'q-nav-menu-size-audio',
  plugins: 'q-nav-menu-size-plugins',
} satisfies Record<NavMenuSize, string>

const MENU_LAYOUT_CLASS = {
  grid: 'q-nav-menu-layout-grid',
  columns: 'q-nav-menu-layout-columns',
  custom: 'q-nav-menu-layout-custom',
} satisfies Record<NavMenuLayout, string>

/* ■■ Root: the bar primitive + the shared morphing popup shell ■■■■■■■■■■■■■■■■ */

export type NavigationMenuRootProps = Omit<ComponentProps<typeof Primitive.Root>, 'className'> & {
  className?: string
  side?: ComponentProps<typeof Primitive.Positioner>['side']
  align?: ComponentProps<typeof Primitive.Positioner>['align']
  sideOffset?: ComponentProps<typeof Primitive.Positioner>['sideOffset']
  alignOffset?: ComponentProps<typeof Primitive.Positioner>['alignOffset']
  collisionPadding?: ComponentProps<typeof Primitive.Positioner>['collisionPadding']
  container?: ComponentProps<typeof Primitive.Portal>['container']
}

/**
 * Renders the `<nav>` bar (logo / list / actions are its children) PLUS the
 * single shared Portal › Positioner › Popup › Viewport that every Item's
 * `Content` morphs into when it opens.
 */
function Root({
  side = 'bottom',
  align = 'center',
  sideOffset = 8,
  alignOffset,
  collisionPadding = 16,
  container,
  className,
  children,
  ...props
}: NavigationMenuRootProps) {
  return (
    <Primitive.Root className={cx('q-nav-root', className)} {...props}>
      {children}
      <Primitive.Portal container={container}>
        <Primitive.Positioner
          className="q-nav-positioner"
          side={side}
          align={align}
          sideOffset={sideOffset}
          alignOffset={alignOffset}
          collisionPadding={collisionPadding}
        >
          <Primitive.Popup className="q-nav-popup">
            <Primitive.Viewport className="q-nav-viewport" />
          </Primitive.Popup>
        </Primitive.Positioner>
      </Primitive.Portal>
    </Primitive.Root>
  )
}

/** Logo slot — content is the dev's. */
function Logo({ className, ...props }: ComponentProps<'div'>) {
```

```
  return <div className={cx('q-nav-logo', className)} {...props} />
}

type ListProps = Omit<ComponentProps<typeof Primitive.List>, 'className'> & { className?: string }
/** The horizontal bar of items. */
function List({ className, ...props }: ListProps) {
  return <Primitive.List className={cx('q-nav-list', className)} {...props} />
}

/* ■■ Bar item: a wrapper that holds a Link, or a Trigger + Content ■■■■■■■■■■■■ */

export type NavigationMenuItemProps = Omit<ComponentProps<typeof Primitive.Item>, 'className'> & {
  className?: string
  /** Stable identity for controlled open state; auto-generated otherwise. */
  value?: string
}

/** A single bar slot. Put a `Link` inside for a plain item, or a `Trigger` +
 * `Content` pair for a panel trigger. */
function Item({ value, children, ...props }: NavigationMenuItemProps) {
  const autoId = useId()
  return <Primitive.Item value={value ?? autoId} {...props}>{children}</Primitive.Item>
}

export type NavigationMenuTriggerProps = Omit<ComponentProps<typeof Primitive.Trigger>, 'className'> & {
  className?: string
  /** Lime accent treatment. */
  accent?: boolean
  /** Current-section indication — sets `aria-current="page"` + the active style. */
  active?: boolean
  ref?: Ref<HTMLButtonElement>
}

/** The bar pill that opens a panel. Compose its label/icon/badge as children. */
function Trigger({ className, accent, active, children, ...props }: NavigationMenuTriggerProps) {
  return (
    <Primitive.Trigger
      className={cx('q-nav-item', accent && 'q-nav-item-accent', className)}
      aria-current={active ? 'page' : undefined}
      {...props}
    >
      {children}
    </Primitive.Trigger>
  )
}

type ContentProps = Omit<ComponentProps<typeof Primitive.Content>, 'className'> & { className?: string }
/** The panel mounted when its Item opens — wrap a `Menu` (or any content). */
function Content({ className, ...props }: ContentProps) {
  return <Primitive.Content className={cx('q-nav-content', className)} {...props} />
}

export type NavigationMenuLinkProps = Omit<ComponentProps<typeof Primitive.Link>, 'className'> & {
  className?: string
  /** Lime accent treatment. */
  accent?: boolean
  /** Current-section indication — sets `aria-current="page"` + the active style. */
  active?: boolean
}

/** A plain bar link pill. Compose its label/icon/badge as children. */
function Link({ className, accent, active, children, ...props }: NavigationMenuLinkProps) {
  return (
    <Primitive.Link
      className={cx('q-nav-item', accent && 'q-nav-item-accent', className)}
      aria-current={active ? 'page' : undefined}
      {...props}
    >
      {children}
    </Primitive.Link>
```

```
  )
}

/** Leading icon slot for a bar Trigger / Link (24px). */
function ItemIcon({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-nav-item-icon', className)} {...props} />
}

/* ■■ Right-side actions ■■■
```

```
function Menu({ rows = 2, size = 'auto', layout = 'grid', standalone = false, featured, className, children, ...props }: NavigationMenuMenuProps) {
  return (
    <div className={cx('q-nav-menu', standalone && 'q-nav-menu-static', MENU_SIZE_CLASS[size], MENU_LAYOUT_CLASS[layout], className)} {...props}>
      {layout === 'custom'
        ? children
        : (
            <div className={cx(layout === 'columns' ? 'q-nav-menu-columns' : 'q-nav-menu-grid', ROWS_CLASS[rows])}>
              {children}
            </div>
          )}
      {featured != null ? <div className="q-nav-featured">{featured}</div> : null}
    </div>
  )
}

/* ■■ Group: a labeled cluster of rows (its own full-height column) ■■■■■■■■■■■ */

function Group({ className, ...props }: ComponentProps<'div'>) {
  return <div className={cx('q-nav-group', className)} {...props} />
}

/** Column heading inside a Group. */
function GroupLabel({ className, ...props }: ComponentProps<'div'>) {
  return <div className={cx('q-nav-group-label', className)} {...props} />
}

/* ■■ MenuItem: a composition-only panel row ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

export type NavigationMenuMenuItemProps = {
  href?: string
  render?: ComponentProps<typeof Primitive.Link>['render']
  /** Set false to render a static row (no `NavigationMenu.Link`) outside a Root. */
  interactive?: boolean
  className?: string
  children?: ReactNode
} & Omit<ComponentProps<typeof Primitive.Link>, 'href' | 'render' | 'className'>

/**
 * A panel row. Renders a real `NavigationMenu.Link` (keyboard + active state)
 * and reuses the shared `q-menu-item*` row primitives (see menu.css). Compose
 * any layout from `MenuItemIcon` / `MenuMedia` / `MenuItemContent` /
 * `MenuItemTitle` / `MenuItemDescription` / `MenuItemTrailing`. Include a
 * `MenuMedia` to get the rich 60px row.
 */
function MenuItem({ href, render, interactive = true, className, children, ...props }: NavigationMenuMenuItemProps) {
  const itemClass = cx('q-menu-item', 'q-nav-menu-item', className)

  if (!interactive) {
    const staticProps = props as ComponentProps<'a'>
    return href != null
      ? <a className={itemClass} href={href} {...staticProps}>{children}</a>
      : <div className={itemClass} {...(props as ComponentProps<'div'>)}>{children}</div>
  }

  return (
    <Primitive.Link className={itemClass} href={href} render={render} {...props}>
      {children}
    </Primitive.Link>
  )
}

/** Leading icon (20px) for a default panel row. */
function MenuItemIcon({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-menu-item-icon', className)} {...props} />
}

/** Leading media tile (44px glass) — its presence makes the row the large row. */
function MenuMedia({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-nav-menu-media', className)} {...props} />
}
```

```
/** Content column — stacks the title row and description. */
function MenuItemContent({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-menu-item-label', className)} {...props} />
}

function MenuItemTitleRow({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-menu-item-title-row', className)} {...props} />
}

function MenuItemTitle({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-menu-item-title', className)} {...props} />
}

function MenuItemDescription({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-menu-item-description', className)} {...props} />
}

function MenuItemTrailing({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-menu-item-trailing', className)} {...props} />
}

/** Horizontal separator inside custom NavigationMenu panels. */
function MenuSeparator({ className, ...props }: ComponentProps<'div'>) {
  return (
    <div className={cx('q-nav-menu-separator', className)} {...props}>
      <Divider orientation="horizontal" />
    </div>
  )
}

export const NavigationMenu = {
  Root,
  Logo,
  List,
  Item,
  Trigger,
  Content,
  Link,
  ItemIcon,
  Actions,
  ActionsGroup,
  Action,
  Separator,
  Menu,
  Group,
  GroupLabel,
  MenuItem,
  MenuItemIcon,
  MenuMedia,
  MenuItemContent,
  MenuItemTitleRow,
  MenuItemTitle,
  MenuItemDescription,
  MenuItemTrailing,
  MenuSeparator,
}
```

app/packages/quanta/src/components/not-found/index.ts

```
export { NotFound } from './not-found.tsx'
export type { NotFoundProps, NotFoundSize, NotFoundVariant } from './not-found.tsx'
```

app/packages/quanta/src/components/not-found/not-found.css

```
/**
 * NotFound — empty / zero-results state primitive (icon tile + title +
 * subtitle). Matched to the Figma "Menu / Empty" state but framework-agnostic:
 * usable inside menus, panels, lists, or anywhere an empty state is needed.
 *
 * Self-contained `q-not-found-*` namespace — no dependency on the menu layer.
 *
 * Sizing is driven by CSS vars set on the root (md defaults) and overridden by
 * the `q-not-found-{sm,md,lg}` size utilities. Variants (`plain`/`card`/
 * `outline`) only change the OUTER surface — the glassy icon tile is untouched.
 */

@source "./not-found.tsx";

@utility q-not-found {
  /* md defaults — overridden by the size utilities below. */
  --q-nf-gap: var(--hf-space-200);
  --q-nf-pad: var(--hf-space-300);
  --q-nf-min-h: calc(var(--hf-space-2400) + var(--hf-space-300));
  --q-nf-tile: var(--hf-space-1000); /* 40px */
  --q-nf-icon: var(--hf-space-600); /* 24px */
  --q-nf-radius: var(--hf-radius-500); /* 20px — card / outline corner */

  align-items: center;
  color: var(--hf-color-text-tertiary);
  display: flex;
  flex-direction: column;
  gap: var(--q-nf-gap);
  justify-content: center;
  min-height: var(--q-nf-min-h);
  padding: var(--q-nf-pad);
  text-align: center;
}

/* ■■ Glassy icon tile — IDENTICAL across every size/variant (scaled only). ■■■■ */
@utility q-not-found-icon {
  align-items: center;
  background-color: var(--hf-color-overlay-hover);
  border-radius: var(--hf-radius-full);
  box-shadow:
    inset var(--hf-space-0) var(--hf-space-050) var(--hf-space-150) var(--hf-space-0) var(--hf-color-transparent-light-15),
    var(--hf-space-0) var(--hf-space-050) var(--hf-space-050) calc(-1 * var(--hf-space-050)) var(--hf-color-shadow-sm);
  color: var(--hf-color-icon-tertiary);
  display: inline-flex;
  height: var(--q-nf-tile);
  justify-content: center;
  width: var(--q-nf-tile);

  & svg {
    height: var(--q-nf-icon);
    width: var(--q-nf-icon);
  }
}

@utility q-not-found-text {
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-050);
}

/* ■■ Actions row — optional CTA cluster (e.g. a quanta Button). ■■■■■■■■■■■■■■■■ */
@utility q-not-found-actions {
  align-items: center;
  display: flex;
  gap: var(--hf-space-200);
  justify-content: center;
}
```

```
/* Title / subtitle type + color now live on <Typography> (per-size variant +
   secondary/tertiary color) — see not-found.tsx. */

/* ■■ Sizes — override the geometry vars; type lives on <Typography>. ■■■■■■■■■■ */
@utility q-not-found-sm {
  --q-nf-gap: var(--hf-space-150);
  --q-nf-pad: var(--hf-space-200);
  --q-nf-min-h: calc(var(--hf-space-1600) + var(--hf-space-200));
  --q-nf-tile: var(--hf-space-800); /* 32px */
  --q-nf-icon: var(--hf-space-500); /* 20px */
  --q-nf-radius: var(--hf-radius-400); /* 16px */
}

@utility q-not-found-md {
  --q-nf-gap: var(--hf-space-200);
  --q-nf-pad: var(--hf-space-300);
  --q-nf-min-h: calc(var(--hf-space-2400) + var(--hf-space-300));
  --q-nf-tile: var(--hf-space-1000); /* 40px */
  --q-nf-icon: var(--hf-space-600); /* 24px */
  --q-nf-radius: var(--hf-radius-500); /* 20px */
}

@utility q-not-found-lg {
  --q-nf-gap: var(--hf-space-300);
  --q-nf-pad: var(--hf-space-500);
  --q-nf-min-h: calc(var(--hf-space-2400) + var(--hf-space-1000));
  --q-nf-tile: var(--hf-space-1400); /* 56px */
  --q-nf-icon: var(--hf-space-700); /* 28px */
  --q-nf-radius: var(--hf-radius-600); /* 24px */
}

/* ■■ Variants — outer surface only; the glassy tile stays the same. ■■■■■■■■■■■ */
@utility q-not-found-plain {
  /* Transparent — drops into an existing (often glass) surface. */
  background-color: transparent;
}

/* Self-contained frosted-glass panel (same glass idiom as Modal / Dropdown). */
@utility q-not-found-card {
  background-color: var(--hf-color-background-glass);
  border: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  border-radius: var(--q-nf-radius);
  backdrop-filter: blur(var(--hf-space-1000)); /* 40px */
  box-shadow: var(--hf-space-0) var(--hf-space-050) var(--hf-space-100) calc(-1 * var(--hf-space-050)) var(--hf-color-shadow-sm);
}

/* Dashed drop-zone outline — subtle border, transparent fill. */
@utility q-not-found-outline {
  background-color: transparent;
  border: var(--hf-border-width-thin) dashed var(--hf-color-border-subtle);
  border-radius: var(--q-nf-radius);
}
```

app/packages/quanta/src/components/not-found/not-found.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { NotFound } from './index.ts'

describe('<NotFound>', () => {
  it('renders icon, title and subtitle slots through Typography (md composite + color)', () => {
    render(
      <NotFound
        icon={<span data-testid="icon">x</span>}
        title="No matches"
        subtitle="Refine your query"
      />,
    )
    expect(screen.getByTestId('icon')).toBeInTheDocument()
    // title → caption-sm-medium / secondary, subtitle → caption-sm-regular / tertiary (md)
    expect(screen.getByText('No matches')).toHaveClass('text-q-caption-sm-medium', 'text-q-text-secondary')
    expect(screen.getByText('Refine your query')).toHaveClass('text-q-caption-sm-regular', 'text-q-text-tertiary')
  })

  it('steps the title / subtitle composite by size (lg → body-sm)', () => {
    render(<NotFound size="lg" title="Big title" subtitle="Big subtitle" />)
    expect(screen.getByText('Big title')).toHaveClass('text-q-body-sm-medium', 'text-q-text-secondary')
    expect(screen.getByText('Big subtitle')).toHaveClass('text-q-body-sm-regular', 'text-q-text-tertiary')
  })

  it('wraps the icon in the glass tile', () => {
    render(<NotFound icon={<span data-testid="i">i</span>} title="T" />)
    const tile = screen.getByTestId('i').closest('.q-not-found-icon')
    expect(tile).toBeInTheDocument()
  })

  it('omits the text column entirely when neither title nor subtitle is given', () => {
    const { container } = render(<NotFound icon={<span>i</span>} />)
    expect(container.querySelector('.q-not-found-text')).not.toBeInTheDocument()
  })

  it('omits the icon tile when no icon is given', () => {
    const { container } = render(<NotFound title="Only title" />)
    expect(container.querySelector('.q-not-found-icon')).not.toBeInTheDocument()
    expect(screen.getByText('Only title')).toBeInTheDocument()
  })

  it('accepts ReactNode title / subtitle (not just strings)', () => {
    render(
      <NotFound
        title={<strong data-testid="rich-title">Rich</strong>}
        subtitle={<em data-testid="rich-sub">sub</em>}
      />,
    )
    expect(screen.getByTestId('rich-title')).toBeInTheDocument()
    expect(screen.getByTestId('rich-sub')).toBeInTheDocument()
  })

  it('forwards className and native div props onto the root', () => {
    const { container } = render(<NotFound className="is-custom" id="empty" title="T" />)
    const root = container.querySelector('.q-not-found')!
    expect(root).toHaveClass('is-custom')
    expect(root).toHaveAttribute('id', 'empty')
  })

  it('defaults to the md size and plain variant', () => {
    const { container } = render(<NotFound title="T" />)
    const root = container.querySelector('.q-not-found')!
    expect(root).toHaveClass('q-not-found-md', 'q-not-found-plain')
  })

  it('applies the requested size and variant classes', () => {
    const { container } = render(<NotFound title="T" size="lg" variant="card" />)
```

```
    const root = container.querySelector('.q-not-found')!
    expect(root).toHaveClass('q-not-found-lg', 'q-not-found-card')
    expect(root).not.toHaveClass('q-not-found-md', 'q-not-found-plain')
  })

  it('renders the actions slot in its own row', () => {
    render(<NotFound title="T" actions={<button type="button">Clear filters</button>} />)
    const cta = screen.getByRole('button', { name: 'Clear filters' })
    expect(cta.closest('.q-not-found-actions')).toBeInTheDocument()
  })

  it('omits the actions row when no actions are given (back-compat)', () => {
    const { container } = render(<NotFound icon={<span>i</span>} title="T" subtitle="S" />)
    expect(container.querySelector('.q-not-found-actions')).not.toBeInTheDocument()
    // default host element is still a <div>, byte-for-byte with the old markup
    expect(container.querySelector('.q-not-found')!.tagName).toBe('DIV')
  })

  it('forwards ref to the root DOM node', () => {
    let node: HTMLElement | null = null
    render(<NotFound ref={el => { node = el }} title="T" />)
    expect(node).not.toBeNull()
    expect(node).toHaveClass('q-not-found')
  })

  it('forwards ref to the swapped host element via render', () => {
    let node: HTMLElement | null = null
    render(<NotFound ref={el => { node = el }} render={<button type="button" />} title="T" />)
    expect(node).not.toBeNull()
    expect(node!.tagName).toBe('BUTTON')
    expect(node).toHaveClass('q-not-found')
  })

  it('swaps the host element via render, keeping the surface classes', () => {
    render(<NotFound render={<button type="button" data-testid="dz" />} variant="outline" title="Drop files" />)
    const dz = screen.getByTestId('dz')
    expect(dz.tagName).toBe('BUTTON')
    expect(dz).toHaveClass('q-not-found', 'q-not-found-outline')
    expect(screen.getByText('Drop files')).toHaveClass('text-q-caption-sm-medium')
  })
})
```

app/packages/quanta/src/components/not-found/not-found.tsx

```
'use client'

import type { ComponentProps, ReactElement, ReactNode, Ref } from 'react'
import { useRender } from '@base-ui/react/use-render'
import type { TypographyVariant } from '../typography/index.ts'
import { Typography } from '../typography/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * NotFound — a standalone empty / zero-results state primitive. Every slot is
 * a free ReactNode for full control:
 *
 *   <NotFound
 *     icon={<SearchIcon />}
 *     title="No results found"
 *     subtitle="Try a different search term"
 *     actions={<Button size="sm">Clear filters</Button>}
 *   />
 *
 * Matched to the Figma "Menu / Empty" state but framework-agnostic — drop it
 * into a menu, panel, list, or any container that needs an empty state.
 *
 * `size` scales the glass icon tile, icon and typography (sm / md / lg).
 * `variant` controls the surface: `plain` (transparent, drops into an existing
 * surface), `card` (its own frosted-glass panel) or `outline` (a subtle dashed
 * drop-zone). The glassy icon tile is identical across every size and variant.
 *
 * The host element is swappable via `render` (Base UI useRender) — e.g. make
 * the `outline` drop-zone a `<button>` / `<label>` upload trigger.
 */

export type NotFoundSize = 'sm' | 'md' | 'lg'
export type NotFoundVariant = 'plain' | 'card' | 'outline'

const SIZE_CLASS = {
  sm: 'q-not-found-sm',
  md: 'q-not-found-md',
  lg: 'q-not-found-lg',
} satisfies Record<NotFoundSize, string>

const VARIANT_CLASS = {
  plain: 'q-not-found-plain',
  card: 'q-not-found-card',
  outline: 'q-not-found-outline',
} satisfies Record<NotFoundVariant, string>

// Per-size composite typography for the title / subtitle lines — exact
// equivalents of the former `q-not-found-{title,subtitle}` @apply rules (which
// stepped the composite by size via descendant selectors). Now funneled through
// <Typography> so the type lives in one place.
const TITLE_VARIANT = {
  sm: 'caption-xs-medium',
  md: 'caption-sm-medium',
  lg: 'body-sm-medium',
} satisfies Record<NotFoundSize, TypographyVariant>

const SUBTITLE_VARIANT = {
  sm: 'caption-xs-regular',
  md: 'caption-sm-regular',
  lg: 'body-sm-regular',
} satisfies Record<NotFoundSize, TypographyVariant>

export type NotFoundProps = Omit<ComponentProps<'div'>, 'title'> & {
  /** Leading icon node — wrapped in the glass tile. Any ReactNode. */
  icon?: ReactNode
  /** Primary line. Any ReactNode. */
  title?: ReactNode
  /** Secondary line. Any ReactNode. */
```

```
  subtitle?: ReactNode
  /** Trailing CTA cluster (e.g. a quanta Button). Any ReactNode. */
  actions?: ReactNode
  /** Scale of the tile, icon and text. Defaults to `md`. */
  size?: NotFoundSize
  /** Surface treatment — keeps the glassy tile in every case. Defaults to `plain`. */
  variant?: NotFoundVariant
  /** Swap the root element/component (e.g. a `<button>` / `<label>` drop-zone). Defaults to `<div>`. */
  render?: ReactElement
}

export function NotFound({ icon, title, subtitle, actions, size = 'md', variant = 'plain', className, children, render, ref, ...props }: NotFoundProps) {
  // `ref` must go to useRender's dedicated `ref` option — it is NOT picked up
  // from `props`, so spreading it there would drop it (it would never reach the
  // rendered root / `render` element).
  return useRender({
    render,
    defaultTagName: 'div',
    ref: ref as Ref<Element> | undefined,
    props: {
      className: cx('q-not-found', SIZE_CLASS[size], VARIANT_CLASS[variant], className),
      children: (
        <>
          {icon != null ? <span className="q-not-found-icon">{icon}</span> : null}
          {(title != null || subtitle != null) ? (
            <span className="q-not-found-text">
              {title != null ? <Typography as="span" variant={TITLE_VARIANT[size]} color="secondary">{title}</Typography> : null}
              {subtitle != null ? <Typography as="span" variant={SUBTITLE_VARIANT[size]} color="tertiary">{subtitle}</Typography> : null}
            </span>
          ) : null}
          {actions != null ? <span className="q-not-found-actions">{actions}</span> : null}
          {children}
        </>
      ),
      ...props,
    },
  })
}
```

app/packages/quanta/src/components/progress/index.ts

```
export { Progress } from './progress.tsx'
export type {
  ProgressColor,
  ProgressProps,
  ProgressShape,
  ProgressSize,
  ProgressVariant,
} from './progress.tsx'
```

app/packages/quanta/src/components/progress/progress.css

```
/**
 * Progress presentation — Base-less, slot-tinted indicator (mirrors Slider's
 * track + accent-fill model). Registers the colocated source for Tailwind's
 * scanner (quanta is symlinked) and authors every surface as `@utility
 * q-progress-*`. The accent comes from the slot system (`--q-tint`, set by
 * `slotStyle(color)` on the root); the track is the neutral
 * `background-tertiary` (same as the slider track). Size is driven by per-size
 * CSS custom properties. Motion degrades under `prefers-reduced-motion` and can
 * be turned off per-instance via `data-static` (the `animated={false}` prop).
 */
@source "./progress.tsx";

/* Per-size dimensions. */
/* --q-progress-ring-stroke is in viewBox-100 units (NOT px): renders as
 * `value * ring/100` px. Kept in viewBox units so stroke-dasharray (also viewBox
 * units) stays consistent — see the accuracy note on .q-progress-ring-arc. */
@utility q-progress-xxs {
  --q-progress-h: var(--hf-space-050);   /* 2px bar/segment */
  --q-progress-dot: var(--hf-space-100); /* 4px dot */
  --q-progress-conn: var(--hf-space-050);
  --q-progress-gap: var(--hf-space-050);
  --q-progress-ring: var(--hf-space-800); /* 32px circular diameter */
  --q-progress-ring-stroke: 10;
}
@utility q-progress-xs {
  --q-progress-h: var(--hf-space-100);   /* 4px */
  --q-progress-dot: var(--hf-space-150); /* 6px */
  --q-progress-conn: var(--hf-space-050);
  --q-progress-gap: var(--hf-space-100);
  --q-progress-ring: var(--hf-space-1000); /* 40px */
  --q-progress-ring-stroke: 9;
}
@utility q-progress-sm {
  --q-progress-h: var(--hf-space-150);   /* 6px */
  --q-progress-dot: var(--hf-space-200); /* 8px */
  --q-progress-conn: var(--hf-space-050);
  --q-progress-gap: var(--hf-space-150);
  --q-progress-ring: var(--hf-space-1200); /* 48px */
  --q-progress-ring-stroke: 8.5;
}
@utility q-progress-md {
  --q-progress-h: var(--hf-space-200);   /* 8px */
  --q-progress-dot: var(--hf-space-250); /* 10px */
  --q-progress-conn: var(--hf-space-050);
  --q-progress-gap: var(--hf-space-200);
  --q-progress-ring: var(--hf-space-1600); /* 64px */
  --q-progress-ring-stroke: 8;
}
@utility q-progress-lg {
  --q-progress-h: var(--hf-space-250);   /* 10px */
  --q-progress-dot: var(--hf-space-300); /* 12px */
  --q-progress-conn: var(--hf-space-050);
  --q-progress-gap: var(--hf-space-250);
  --q-progress-ring: var(--hf-space-2400); /* 96px */
  --q-progress-ring-stroke: 7;
}

/* ── Bar ──────────────────────────────────────────────────────────────────── */
@utility q-progress {
  position: relative;
  width: 100%;
  height: var(--q-progress-h);
  border-radius: var(--hf-radius-full);
  overflow: hidden;
  background-color: var(--hf-color-background-tertiary);
}
@utility q-progress-fill {
  position: absolute;
```

```
  inset-block: var(--hf-space-0);
  left: var(--hf-space-0);
  border-radius: var(--hf-radius-full);
  background-color: var(--q-tint);
  transition: width var(--hf-duration-350) var(--hf-ease-in-out-circ);
  will-change: width;
}
@utility q-progress-indeterminate {
  width: 40%; /* no token: percentage track width, not a spacing px */
  /* `ease-in-out` left as a standard CSS keyword (no cubic-bezier token equivalent). */
  animation: q-progress-slide var(--hf-duration-1300) ease-in-out infinite;
}
@keyframes q-progress-slide {
  0% { left: -45%; }
  100% { left: 100%; }
}

/* ■■ Stepped container (dots + line) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-progress-steps {
  display: flex;
  align-items: center;
  gap: var(--q-progress-gap);
  width: 100%;
}

/* ■■ Line segments ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-progress-segment {
  position: relative;
  flex: 1 1 auto;
  height: var(--q-progress-h);
  border-radius: var(--hf-radius-full);
  overflow: hidden;
  background-color: var(--hf-color-background-tertiary);
}
@utility q-progress-segment-fill {
  position: absolute;
  inset-block: var(--hf-space-0);
  left: var(--hf-space-0);
  border-radius: var(--hf-radius-full);
  background-color: var(--q-tint);
  transition: width var(--hf-duration-350) var(--hf-ease-in-out-circ);
}

/* ■■ Dotted bar (a row of equal dots; the first N filled) ■■■■■■■■■■■■■■■■■■■■■■■■
 * AUTO-FIT: each dot is absolutely placed at a fraction of the wrapper width
 * (its `left` is computed inline), so any number of dots span 0→100% and OVERLAP
 * when crowded (e.g. 100 dots in a narrow track) instead of overflowing — the row
 * never exceeds the wrapper. Few dots → spread out; many → condensed/overlapping. */
@utility q-progress-dots {
  position: relative;
  width: 100%;
  height: var(--q-progress-dot);
}
@utility q-progress-dot {
  position: absolute;
  top: var(--hf-space-0);
  width: var(--q-progress-dot);
  height: var(--q-progress-dot);
  transform: translateX(-50%); /* centre each dot on its computed point */
  border-radius: var(--hf-radius-full);
  background-color: var(--hf-color-background-tertiary);
  transition: background-color var(--hf-duration-slow) ease; /* no token: `ease` keyword has no --hf-ease-* equivalent */
}
.q-progress-dot[data-state="complete"] {
  background-color: var(--q-tint);
  z-index: 1; /* filled dots paint above pending so the progress edge stays crisp when dots overlap */
}

/* ■■ Circular forms (ring / segmented ring / dotted ring) ■■■■■■■■■■■■■■■■■■■■■■■■
 * One square SVG box (--q-progress-ring). Strokes use `non-scaling-stroke` so the
```

```
 * thickness stays --q-progress-h px regardless of the viewBox scale; arc geometry
 * (dash lengths, dot positions) is in viewBox units and scales with the box. */
@utility q-progress-circular {
  position: relative;
  display: inline-flex;
  width: var(--q-progress-ring);
  height: var(--q-progress-ring);
}
.q-progress-ring-svg {
  display: block;
  width: 100%;
  height: 100%;
}
.q-progress-ring-track {
  fill: none;
  stroke: var(--hf-color-background-tertiary);
  stroke-width: var(--q-progress-ring-stroke);
}
.q-progress-ring-arc {
  fill: none;
  stroke: var(--q-tint);
  stroke-width: var(--q-progress-ring-stroke);
  /* Butt cap: the arc ends exactly at the value angle (a round cap overshoots by
   * half the stroke and collides with the start cap near 100% — the "mustache"). */
  stroke-linecap: butt;
  /* stroke-width + dasharray are BOTH in viewBox units (no non-scaling-stroke):
   * non-scaling-stroke measures the dash in rendered px, which mismatches the
   * viewBox-unit circumference C → the ring fills wrong (≈full at 60%). */
  transition: stroke-dashoffset var(--hf-duration-350) var(--hf-ease-in-out-circ);
}
.q-progress-ring-indeterminate {
  /* No value to be accurate about — round caps read nicer on the spinner. */
  stroke-linecap: round;
  transform-box: fill-box;
  transform-origin: center;
  /* no token: 0.9s has no --hf-duration-* equivalent */
  animation: q-progress-spin 0.9s var(--hf-ease-linear) infinite;
}
@keyframes q-progress-spin { to { transform: rotate(360deg); } }

/* segmented ring — one stroked arc chunk per segment (butt caps so adjacent
 * segments never overlap and each chunk is exactly its share of the circle). */
.q-progress-ring-seg {
  fill: none;
  stroke: var(--hf-color-background-tertiary);
  stroke-width: var(--q-progress-ring-stroke);
  stroke-linecap: butt;
  transition: stroke var(--hf-duration-slow) ease; /* no token: `ease` keyword has no --hf-ease-* equivalent */
}
.q-progress-ring-seg[data-state="complete"] { stroke: var(--q-tint); }

/* dotted ring — dots arranged around the circle */
.q-progress-ring-dot {
  fill: var(--hf-color-background-tertiary);
  transition: fill var(--hf-duration-slow) ease; /* no token: `ease` keyword has no --hf-ease-* equivalent */
}
.q-progress-ring-dot[data-state="complete"] { fill: var(--q-tint); }

/* Center label (circular). */
@utility q-progress-center {
  position: absolute;
  inset: var(--hf-space-0);
  display: grid;
  place-items: center;
  color: var(--hf-color-text-primary);
  @apply text-q-caption-sm-medium;
}

/* ■■ Motion off (animated={false}) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
.q-progress[data-static] .q-progress-fill,
```

```
.q-progress-steps[data-static] .q-progress-segment-fill { transition: none; }
.q-progress[data-static] .q-progress-indeterminate { animation: none; left: var(--hf-space-0); width: 100%; }
.q-progress-dots[data-static] .q-progress-dot { transition: none; }
.q-progress-circular[data-static] .q-progress-ring-arc,
.q-progress-circular[data-static] .q-progress-ring-seg,
.q-progress-circular[data-static] .q-progress-ring-dot { transition: none; }
.q-progress-circular[data-static] .q-progress-ring-indeterminate { animation: none; }

@media (prefers-reduced-motion: reduce) {
  .q-progress-fill,
  .q-progress-segment-fill { transition: none; }
  .q-progress-dot { transition: none; }
  .q-progress-indeterminate { animation: none; left: var(--hf-space-0); width: 100%; }
  .q-progress-ring-arc,
  .q-progress-ring-seg,
  .q-progress-ring-dot { transition: none; }
  .q-progress-ring-indeterminate { animation: none; }
}
```

app/packages/quanta/src/components/progress/progress.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Progress } from './index.ts'

describe('<Progress> bar', () => {
  it('exposes progressbar a11y and reflects value as fill width', () => {
    const { container } = render(<Progress value={40} aria-label="Upload" />)
    const bar = screen.getByRole('progressbar', { name: 'Upload' })
    expect(bar).toHaveAttribute('aria-valuenow', '40')
    expect(bar).toHaveAttribute('aria-valuemin', '0')
    expect(bar).toHaveAttribute('aria-valuemax', '100')
    expect(container.querySelector('.q-progress-fill')).toHaveStyle({ width: '40%' })
  })

  it('is indeterminate (no valuenow, sliding fill) when value is omitted', () => {
    const { container } = render(<Progress aria-label="Loading" />)
    expect(screen.getByRole('progressbar')).not.toHaveAttribute('aria-valuenow')
    expect(container.querySelector('.q-progress-indeterminate')).toBeInTheDocument()
  })

  it('honors max', () => {
    render(<Progress value={3} max={5} aria-label="Steps" />)
    const bar = screen.getByRole('progressbar')
    expect(bar).toHaveAttribute('aria-valuenow', '3')
    expect(bar).toHaveAttribute('aria-valuemax', '5')
  })

  it('applies the slot color via --q-tint', () => {
    const { container } = render(<Progress value={50} color="success" aria-label="x" />)
    expect((container.firstChild as HTMLElement).style.getPropertyValue('--q-tint')).not.toBe('')
  })

  it('disables motion with animated={false}', () => {
    render(<Progress value={50} animated={false} aria-label="x" />)
    expect(screen.getByRole('progressbar')).toHaveAttribute('data-static', '')
  })
})

describe('<Progress> line', () => {
  it('renders N segments and fills them sequentially', () => {
    const { container } = render(<Progress variant="line" steps={4} value={60} aria-label="Steps" />)
    const segs = container.querySelectorAll('.q-progress-segment')
    expect(segs).toHaveLength(4)
    // p=0.6, n=4 → fills: 100, 100, 40, 0
    expect(segs[0]).toHaveAttribute('data-state', 'complete')
    expect(segs[1]).toHaveAttribute('data-state', 'complete')
    expect(segs[2]).toHaveAttribute('data-state', 'active')
    expect(segs[3]).toHaveAttribute('data-state', 'pending')
  })
})

describe('<Progress> dots', () => {
  it('renders a row of N dots (no connectors) with the first round(p·n) filled', () => {
    const { container } = render(<Progress variant="dots" steps={10} value={50} aria-label="Steps" />)
    const dots = container.querySelectorAll('.q-progress-dot')
    expect(dots).toHaveLength(10)
    expect(container.querySelectorAll('.q-progress-connector')).toHaveLength(0)
    const complete = [...dots].filter(d => d.getAttribute('data-state') === 'complete')
    expect(complete).toHaveLength(5) // round(0.5 * 10)
    expect(dots[4]).toHaveAttribute('data-state', 'complete')
    expect(dots[5]).toHaveAttribute('data-state', 'pending')
  })

  it('marks all dots complete at 100%', () => {
    const { container } = render(<Progress variant="dots" steps={3} value={100} aria-label="Done" />)
    container.querySelectorAll('.q-progress-dot').forEach(d => expect(d).toHaveAttribute('data-state', 'complete'))
  })
})
```

```
describe('<Progress> circular', () => {
  it('renders a ring (track + accent arc) for circular bar with a11y', () => {
    const { container } = render(<Progress shape="circular" value={50} aria-label="Ring" />)
    const root = screen.getByRole('progressbar', { name: 'Ring' })
    expect(root).toHaveClass('q-progress-circular')
    expect(root).toHaveAttribute('aria-valuenow', '50')
    expect(container.querySelector('.q-progress-ring-track')).toBeInTheDocument()
    expect(container.querySelector('.q-progress-ring-arc')).toBeInTheDocument()
  })

  it('spins an indeterminate ring when value is omitted', () => {
    const { container } = render(<Progress shape="circular" aria-label="Loading" />)
    expect(screen.getByRole('progressbar')).not.toHaveAttribute('aria-valuenow')
    expect(container.querySelector('.q-progress-ring-indeterminate')).toBeInTheDocument()
  })

  it('renders N arc segments, the first round(p·n) complete', () => {
    const { container } = render(<Progress shape="circular" variant="line" steps={4} value={50} aria-label="Segs" />)
    const segs = container.querySelectorAll('.q-progress-ring-seg')
    expect(segs).toHaveLength(4)
    expect([...segs].filter(s => s.getAttribute('data-state') === 'complete')).toHaveLength(2)
  })

  it('renders N dots around the ring, the first round(p·n) complete', () => {
    const { container } = render(<Progress shape="circular" variant="dots" steps={8} value={50} aria-label="Dots" />)
    const dots = container.querySelectorAll('.q-progress-ring-dot')
    expect(dots).toHaveLength(8)
    expect([...dots].filter(d => d.getAttribute('data-state') === 'complete')).toHaveLength(4)
  })

  it('renders a center label from children', () => {
    render(<Progress shape="circular" value={62} aria-label="x">62%</Progress>)
    expect(screen.getByText('62%')).toHaveClass('q-progress-center')
  })

  it('applies the requested size class (xxs … lg)', () => {
    render(<Progress shape="circular" size="xxs" value={10} aria-label="tiny" />)
    expect(screen.getByRole('progressbar')).toHaveClass('q-progress-xxs')
  })
})
```

app/packages/quanta/src/components/progress/progress.tsx

```
'use client'

import type { ComponentProps, CSSProperties, ReactNode } from 'react'
import { cx } from '../utils/cx.ts'
import type { SlotColor } from '../utils/slot.ts'
import { slotStyle } from '../utils/slot.ts'

/**
 * Progress — an animated progress indicator skinned with quanta tokens. The
 * accent is the slot system (`color` → `slotStyle`); the track is the neutral
 * `background-tertiary`. Two orthogonal axes plus a creative meter:
 *
 *   `variant`:
 *     - `bar`  (default) — a continuous fill. Omit `value` for indeterminate.
 *     - `line` — `steps` equal segments that fill in order.
 *     - `dots` — `steps` dots; the first `round(value/max · steps)` are filled.
 *
 *   `shape` (bar/line/dots): `linear` (default) or `circular` — the circular
 *     forms are a ring, a segmented ring, and a dotted ring. Pass `children` to
 *     label the center.
 *
 * Animations degrade under `prefers-reduced-motion` and `animated={false}`.
 */

export type ProgressVariant = 'bar' | 'dots' | 'line'
export type ProgressShape = 'linear' | 'circular'
export type ProgressSize = 'xxs' | 'xs' | 'sm' | 'md' | 'lg'
export type ProgressColor = SlotColor

const SIZE_CLASS = {
  xxs: 'q-progress-xxs',
  xs: 'q-progress-xs',
  sm: 'q-progress-sm',
  md: 'q-progress-md',
  lg: 'q-progress-lg',
} satisfies Record<ProgressSize, string>

export interface ProgressProps extends Omit<ComponentProps<'div'>, 'color'> {
  /** Current value in `[0, max]`. Omit for an indeterminate `bar` / ring. */
  value?: number
  /** Upper bound. Default 100. */
  max?: number
  variant?: ProgressVariant
  /** Linear (default) or circular — applies to bar / line / dots. */
  shape?: ProgressShape
  /** Number of steps for line / dots. Default 4. */
  steps?: number
  size?: ProgressSize
  color?: ProgressColor
  /** Transitions + indeterminate / active-step motion. Default true. */
  animated?: boolean
  /** Center label for circular shapes (e.g. `62%`). */
  children?: ReactNode
  'aria-label'?: string
}

const clamp01 = (n: number) => Math.max(0, Math.min(1, n))
const stepState = (fill: number) => (fill >= 100 ? 'complete' : fill > 0 ? 'active' : 'pending')

/* ■■ SVG geometry (viewBox 0 0 100 100, centred). ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
const R = 42 // ring radius
const C = 2 * Math.PI * R // circumference
const ORBIT = 40 // dot orbit radius
const DOT_R = 7 // dot radius

/** Point on a circle centred at (50,50). Angle in deg, 0° = 3 o'clock, clockwise. */
function polar(r: number, deg: number): [number, number] {
  const a = (deg * Math.PI) / 180
```

```
  return [50 + r * Math.cos(a), 50 + r * Math.sin(a)]
}

export function Progress({
  value,
  max = 100,
  variant = 'bar',
  shape = 'linear',
  steps = 4,
  size = 'md',
  color = 'brand',
  animated = true,
  className,
  style,
  children,
  'aria-label': ariaLabel,
  ...props
}: ProgressProps) {
  const indeterminate = value === undefined
  const p = indeterminate ? 0 : clamp01(value / (max || 1))
  const n = Math.max(1, Math.round(steps))
  const filled = Math.round(p * n)

  const aria = {
    role: 'progressbar' as const,
    'aria-label': ariaLabel,
    'aria-valuemin': 0,
    'aria-valuemax': max,
    ...(indeterminate ? {} : { 'aria-valuenow': Math.round(p * max) }),
  }
  const rootStyle: CSSProperties = { ...slotStyle(color), ...style }
  const dataStatic = animated ? undefined : ''
  const center = children != null ? <span className="q-progress-center">{children}</span> : null

  // ■■ Circular bar / line / dots. ■■■
```

```
            cx="50"
            cy="50"
            r={R}
            style={indeterminate ? { strokeDasharray: `${C * 0.25} ${C}` } : { strokeDasharray: C, strokeDashoffset: C * (1 - p) }}
          />
        </g>
      )
    }
    return (
      <div {...aria} data-static={dataStatic} className={cx('q-progress-circular', SIZE_CLASS[size], className)} style={rootStyle} {...props}>
        <svg className="q-progress-ring-svg" viewBox="0 0 100 100" aria-hidden>{svg}</svg>
        {center}
      </div>
    )
  }

  // ■■ Linear stepped variants (line / dots). ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  if (variant === 'line') {
    return (
      <div {...aria} data-static={dataStatic} className={cx('q-progress-steps', SIZE_CLASS[size], className)} style={rootStyle} {...props}>
        {Array.from({ length: n }, (_, i) => {
          const fill = clamp01(p * n - i) * 100
          return (
            <span key={i} className="q-progress-segment" data-state={stepState(fill)}>
              <span className="q-progress-segment-fill" style={{ width: `${fill}%` }} />
            </span>
          )
        })}
      </div>
    )
  }
  if (variant === 'dots') {
    return (
      <div {...aria} data-static={dataStatic} className={cx('q-progress-dots', SIZE_CLASS[size], className)} style={rootStyle} {...props}>
        {Array.from({ length: n }, (_, i) => {
          // Place each dot at a fraction of the track so they span the full width
          // and overlap when there are too many to fit (never overflowing it).
          const frac = n > 1 ? i / (n - 1) : 0.5
          return (
            <span
              key={i}
              className="q-progress-dot"
              data-state={i < filled ? 'complete' : 'pending'}
              style={{ left: `calc(var(--q-progress-dot) / 2 + ${frac} * (100% - var(--q-progress-dot)))` }}
            />
          )
        })}
      </div>
    )
  }

  // ■■ Linear bar. ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  return (
    <div {...aria} data-static={dataStatic} className={cx('q-progress', SIZE_CLASS[size], className)} style={rootStyle} {...props}>
      <span
        className={cx('q-progress-fill', indeterminate && 'q-progress-indeterminate')}
        style={indeterminate ? undefined : { width: `${p * 100}%` }}
      />
    </div>
  )
}
```

app/packages/quanta/src/components/radio/index.ts

```
export { Radio, RadioGroup, RadioLabel, radio } from './radio.tsx'
export type {
  RadioColor,
  RadioGroupProps,
  RadioLabelDirection,
  RadioLabelProps,
  RadioLabelSize,
  RadioOptions,
  RadioProps,
  RadioSize,
} from './radio.tsx'
```

app/packages/quanta/src/components/radio/radio.css

```
/**
 * Radio presentation utilities (q-namespaced). Derived 1:1 from the Checkbox
 * (checkbox.css, Figma node 481:795) — same target/box ramp, border-width,
 * colour tokens, hover/focus/disabled treatment and label sub-component — with
 * two differences that make it a radio:
 *   • the box (and focus target) are circular (border-radius: full), and
 *   • the indicator is a centred filled dot instead of a check glyph.
 * No indeterminate state. A Radio must live inside a RadioGroup.
 *
 * Selected mirrors the checkbox: the box fills with the colour's `fill` and the
 * dot is drawn in the inverse `fg`.
 */

@source "./radio.tsx";

/* Vertical group stack. */
@utility q-radio-group {
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-300);
}

@utility q-radio {
  --q-radio-fill: var(--hf-color-icon-accent);
  --q-radio-fg: var(--hf-color-icon-inverse);
  --q-radio-hover-border: var(--hf-color-lime-900);
  --q-radio-focus-ring: var(--hf-color-transparent-lime-20);
  --q-radio-target-size: var(--hf-space-600);
  --q-radio-box-size: var(--hf-space-500);

  align-items: center;
  background-color: transparent;
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--hf-radius-full);
  box-sizing: border-box;
  color: var(--q-radio-fg);
  cursor: pointer;
  display: inline-flex;
  flex-shrink: 0;
  height: var(--q-radio-target-size);
  justify-content: center;
  outline: none;
  padding: var(--hf-space-0);
  position: relative;
  transition-property: background-color, border-color, box-shadow, color;
  width: var(--q-radio-target-size);
}

@utility q-radio-box {
  align-items: center;
  background-color: transparent;
  border: var(--hf-border-width-medium) solid var(--hf-color-border-default);
  border-radius: var(--hf-radius-full);
  box-sizing: border-box;
  color: transparent;
  display: flex;
  height: var(--q-radio-box-size);
  justify-content: center;
  transition-property: background-color, border-color, color;
  width: var(--q-radio-box-size);
}

@utility q-radio-indicator {
  align-items: center;
  color: currentColor;
  display: flex;
  height: var(--q-radio-box-size);
  justify-content: center;
```

```
  width: var(--q-radio-box-size);
}

/* The selected dot — currentColor (= the ring colour when checked). Hidden
 * (scaled to 0) by default; it pops in with a slight overshoot when selected and
 * scales back out when deselected (the Indicator is keepMounted, so both the
 * enter and exit transition). */
@utility q-radio-dot {
  background-color: currentColor;
  border-radius: var(--hf-radius-full);
  height: calc(var(--q-radio-box-size) * 0.6);
  width: calc(var(--q-radio-box-size) * 0.6);
  opacity: 0;
  transform: scale(0);
  transition-property: transform, opacity;
  transition-duration: var(--hf-duration-normal);
  transition-timing-function: var(--hf-ease-spring);
}

@utility q-radio-sm {
  --q-radio-target-size: var(--hf-space-500);
  --q-radio-box-size: calc(var(--hf-space-500) - var(--hf-space-050));
}

@utility q-radio-md {
  --q-radio-target-size: var(--hf-space-600);
  --q-radio-box-size: var(--hf-space-500);
}

@utility q-radio-lg {
  --q-radio-target-size: var(--hf-space-700);
  --q-radio-box-size: var(--hf-space-600);
}

@utility q-radio-brand {
  --q-radio-fill: var(--hf-color-icon-accent);
  --q-radio-fg: var(--hf-color-icon-inverse);
  --q-radio-hover-border: var(--hf-color-lime-900);
  --q-radio-focus-ring: var(--hf-color-transparent-lime-20);
}

@utility q-radio-white {
  --q-radio-fill: var(--hf-color-palette-white);
  --q-radio-fg: var(--hf-color-icon-inverse);
  --q-radio-hover-border: var(--hf-color-palette-white);
  --q-radio-focus-ring: var(--hf-color-transparent-light-20);
}

@utility q-radio-neutral {
  --q-radio-fill: var(--hf-color-background-secondary-strong);
  --q-radio-fg: var(--hf-color-text-primary);
  --q-radio-hover-border: var(--hf-color-border-strong);
  --q-radio-focus-ring: var(--hf-color-transparent-dark-20);
}

@utility q-radio-success {
  --q-radio-fill: var(--hf-color-state-success-fg);
  --q-radio-fg: var(--hf-color-text-inverse);
  --q-radio-hover-border: var(--hf-color-state-success-fg);
  --q-radio-focus-ring: var(--hf-color-state-success-glow);
}

@utility q-radio-error {
  --q-radio-fill: var(--hf-color-state-error-fg);
  --q-radio-fg: var(--hf-color-text-inverse);
  --q-radio-hover-border: var(--hf-color-state-error-fg);
  --q-radio-focus-ring: var(--hf-color-state-error-glow);
}

@utility q-radio-warning {
```

```
  --q-radio-fill: var(--hf-color-state-warning-fg);
  --q-radio-fg: var(--hf-color-text-inverse);
  --q-radio-hover-border: var(--hf-color-state-warning-fg);
  --q-radio-focus-ring: var(--hf-color-state-warning-glow);
}

@utility q-radio-info {
  --q-radio-fill: var(--hf-color-state-info-fg);
  --q-radio-fg: var(--hf-color-text-inverse);
  --q-radio-hover-border: var(--hf-color-state-info-fg);
  --q-radio-focus-ring: var(--hf-color-state-info-bg);
}

.q-radio:not([data-checked]):not([data-disabled]):hover .q-radio-box {
  border-color: var(--q-radio-hover-border);
}

/* Selected: coloured ring + coloured dot, transparent middle (classic radio). */
.q-radio[data-checked] .q-radio-box {
  background-color: transparent;
  border-color: var(--q-radio-fill);
  color: var(--q-radio-fill);
}

.q-radio[data-checked] .q-radio-dot {
  opacity: 1;
  transform: scale(1);
}

.q-radio:focus-visible {
  background-color: var(--hf-color-background-primary);
  box-shadow: 0 0 0 var(--hf-border-width-thick) var(--q-radio-focus-ring);
}

.q-radio[data-disabled] {
  cursor: not-allowed;
}

/* Disabled keeps the ring + dot shape, muted (no fill). */
.q-radio[data-disabled] .q-radio-box {
  background-color: transparent;
  border-color: var(--hf-color-icon-disabled);
  color: var(--hf-color-icon-disabled);
}

@media (prefers-reduced-motion: reduce) {
  .q-radio-dot {
    transition: none;
  }
}

@utility q-radio-label {
  align-items: flex-start;
  color: var(--hf-color-text-primary);
  cursor: pointer;
  display: flex;
  gap: var(--hf-space-300);
  min-width: var(--hf-space-0);
}

@utility q-radio-label-left {
  justify-content: flex-start;
}

@utility q-radio-label-right {
  justify-content: space-between;
}

@utility q-radio-label-text {
  display: flex;
```

```
  flex: 1 1 var(--hf-space-0);
  flex-direction: column;
  gap: var(--hf-space-100);
  min-width: var(--hf-space-0);
  overflow-wrap: anywhere;
}

/* Title/description colour + composite type are now applied by <Typography>
 * (color="primary"/"tertiary", variant label-sm-medium/label-md-medium for the
 * title per size, label-sm-regular for the description) in radio.tsx. Unlike the
 * checkbox there is no md line-height override, so the q-radio-label-{sm,md}
 * size classes carry no rules of their own and are no longer emitted. */
```

app/packages/quanta/src/components/radio/radio.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { describe, expect, it } from 'vitest'
import { Radio, RadioGroup, RadioLabel, radio } from './index.ts'

describe('radio() class-builder', () => {
  it('defaults to brand + md', () => {
    expect(radio()).toBe('q-radio q-radio-brand q-radio-md')
  })

  it('applies color + size and merges extras', () => {
    expect(radio({ color: 'white', size: 'lg' }, 'is-custom', false)).toBe(
      'q-radio q-radio-white q-radio-lg is-custom',
    )
  })
})

describe('<Radio> in a <RadioGroup>', () => {
  it('renders Base UI radios with Figma classes and reflects the selected value', () => {
    render(
      <RadioGroup defaultValue="a">
        <Radio value="a" aria-label="A" color="white" size="sm" />
        <Radio value="b" aria-label="B" />
      </RadioGroup>,
    )
    const a = screen.getByRole('radio', { name: 'A' })
    expect(a).toHaveClass('q-radio', 'q-radio-white', 'q-radio-sm')
    expect(a).toHaveAttribute('aria-checked', 'true')
    expect(screen.getByRole('radio', { name: 'B' })).toHaveAttribute('aria-checked', 'false')
  })

  it('moves selection on click (single-select)', async () => {
    const user = userEvent.setup()
    render(
      <RadioGroup defaultValue="a">
        <Radio value="a" aria-label="A" />
        <Radio value="b" aria-label="B" />
      </RadioGroup>,
    )
    await user.click(screen.getByRole('radio', { name: 'B' }))
    expect(screen.getByRole('radio', { name: 'B' })).toHaveAttribute('aria-checked', 'true')
    expect(screen.getByRole('radio', { name: 'A' })).toHaveAttribute('aria-checked', 'false')
  })

  it('applies the group layout class', () => {
    const { container } = render(
      <RadioGroup aria-label="group"><Radio value="a" aria-label="A" /></RadioGroup>,
    )
    expect(container.querySelector('[role="radiogroup"]')).toHaveClass('q-radio-group')
  })
})

describe('<RadioLabel>', () => {
  it('renders the label, description and a selected radio', () => {
    render(
      <RadioGroup defaultValue="x">
        <RadioLabel value="x" label="Option X" description="Description" />
      </RadioGroup>,
    )
    // Title/description are now rendered by <Typography>: the composite type +
    // semantic colour classes sit alongside the kept q-radio-label-* hooks.
    const title = screen.getByText('Option X')
    expect(title).toHaveClass('q-radio-label-title', 'text-q-label-sm-medium', 'text-q-text-primary')
    const description = screen.getByText('Description')
    expect(description).toHaveClass('q-radio-label-description', 'text-q-label-sm-regular', 'text-q-text-tertiary')
    const r = screen.getByRole('radio')
    expect(r).toHaveClass('q-radio')
    expect(r).toHaveAttribute('aria-checked', 'true')
```

```
  })

  it('supports right-aligned radio and medium label typography', () => {
    render(
      <RadioGroup>
        <RadioLabel value="x" direction="right" size="md" label="Option" />
      </RadioGroup>,
    )
    expect(screen.getByText('Option').closest('.q-radio-label')).toHaveClass(
      'q-radio-label-right',
      'q-radio-label-md',
    )
    // md label size drives the md title composite via <Typography>.
    expect(screen.getByText('Option')).toHaveClass('text-q-label-md-medium')
  })
})
```

app/packages/quanta/src/components/radio/radio.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { Radio as Primitive } from '@base-ui/react/radio'
import { RadioGroup as PrimitiveGroup } from '@base-ui/react/radio-group'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'
import type { SlotColor } from '../utils/slot.ts'
import { Typography, type TypographyVariant } from '../typography/index.ts'

/**
 * Radio — Base UI `Radio` skinned with quanta tokens, derived 1:1 from Checkbox
 * (same colour / size scale, states, focus ring and label sub-component). It is
 * circular with a centred dot indicator instead of a square with a check, and
 * has no indeterminate state. Always render Radios inside a `RadioGroup`, which
 * owns the selected `value`.
 *
 *   <RadioGroup defaultValue="a">
 *     <RadioLabel value="a" label="Option A" />
 *     <RadioLabel value="b" label="Option B" />
 *   </RadioGroup>
 */

export type RadioSize = 'sm' | 'md' | 'lg'
export type RadioColor = SlotColor | 'white'
export type RadioLabelDirection = 'left' | 'right'
export type RadioLabelSize = 'sm' | 'md'

export interface RadioOptions {
  color?: RadioColor
  size?: RadioSize
}

const COLOR_CLASS = {
  brand: 'q-radio-brand',
  white: 'q-radio-white',
  neutral: 'q-radio-neutral',
  success: 'q-radio-success',
  error: 'q-radio-error',
  warning: 'q-radio-warning',
  info: 'q-radio-info',
} satisfies Record<RadioColor, string>

const SIZE_CLASS = {
  sm: 'q-radio-sm',
  md: 'q-radio-md',
  lg: 'q-radio-lg',
} satisfies Record<RadioSize, string>

const LABEL_DIRECTION_CLASS = {
  left: 'q-radio-label-left',
  right: 'q-radio-label-right',
} satisfies Record<RadioLabelDirection, string>

const LABEL_SIZE_CLASS = {
  sm: 'q-radio-label-sm',
  md: 'q-radio-label-md',
} satisfies Record<RadioLabelSize, string>

// Title composite per label size (mirrors the old q-radio-label-{sm,md} @apply
// rules in radio.css, now applied by <Typography>).
const LABEL_TITLE_VARIANT = {
  sm: 'label-sm-medium',
  md: 'label-md-medium',
} satisfies Record<RadioLabelSize, TypographyVariant>

export function radio(options: RadioOptions = {}, ...extra: ClassValue[]): string {
  const { color = 'brand', size = 'md' } = options
```

```
  return cx('q-radio', COLOR_CLASS[color], SIZE_CLASS[size], ...extra)
}

/** Groups Radios and owns the selected `value` (controlled or `defaultValue`). */
export type RadioGroupProps = ComponentProps<typeof PrimitiveGroup>

export function RadioGroup({ className, ...props }: RadioGroupProps) {
  return (
    <PrimitiveGroup
      className={state => cx('q-radio-group', typeof className === 'function' ? className(state) : className)}
      {...props}
    />
  )
}

export type RadioProps = ComponentProps<typeof Primitive.Root> & RadioOptions

export function Radio({ color, size, className, ...props }: RadioProps) {
  return (
    <Primitive.Root
      className={state =>
        radio(
          { color, size },
          typeof className === 'function' ? className(state) : className,
        )}
      {...props}
    >
      <span className="q-radio-box">
        {/* keepMounted so the dot can transition in AND out (scale + fade). */}
        <Primitive.Indicator keepMounted className="q-radio-indicator">
          <span className="q-radio-dot" />
        </Primitive.Indicator>
      </span>
    </Primitive.Root>
  )
}

export interface RadioLabelProps extends Omit<ComponentProps<'label'>, 'color'> {
  /** The radio's value within the group (required). */
  value: RadioProps['value']
  label?: ReactNode
  description?: ReactNode
  direction?: RadioLabelDirection
  size?: RadioLabelSize
  color?: RadioColor
  radioSize?: RadioSize
  radioProps?: Omit<RadioProps, 'color' | 'size' | 'value'>
}

export function RadioLabel({
  value,
  label = 'Label',
  description,
  direction = 'left',
  size = 'sm',
  color,
  radioSize = size === 'md' ? 'md' : 'sm',
  radioProps,
  className,
  children,
  ...props
}: RadioLabelProps) {
  const radioNode = (
    <Radio
      value={value}
      color={color}
      size={radioSize}
      {...radioProps}
    />
  )
```

```
  const textNode = (
    <span className="q-radio-label-text">
      <Typography
        as="span"
        variant={LABEL_TITLE_VARIANT[size]}
        color="primary"
        className="q-radio-label-title"
      >
        {children ?? label}
      </Typography>
      {description
        ? (
            <Typography
              as="span"
              variant="label-sm-regular"
              color="tertiary"
              className="q-radio-label-description"
            >
              {description}
            </Typography>
          )
        : null}
    </span>
  )

  return (
    <label
      className={cx(
        'q-radio-label',
        LABEL_DIRECTION_CLASS[direction],
        LABEL_SIZE_CLASS[size],
        className,
      )}
      {...props}
    >
      {direction === 'left' ? radioNode : textNode}
      {direction === 'left' ? textNode : radioNode}
    </label>
  )
}
```

app/packages/quanta/src/components/select/index.ts

```
export { Select } from './select.tsx'
export type {
  SelectContentSurface,
  SelectSize,
} from './select.tsx'
```

app/packages/quanta/src/components/select/select.css

```
/**
 * Select presentation utilities (q-select-* namespace).
 *
 * The TRIGGER reuses the Input field surface (`q-field-control`, see input.css):
 * the bordered white-5% box, lime focus ring, and red `q-field-control-invalid`
 * ring. These rules only add the value/placeholder typography, the flipping
 * chevron, and the sm/lg height presets.
 *
 * The POPUP reuses the Dropdown glass card (`q-dropdown-content`, see
 * dropdown.css) and the shared `q-menu-item*` row primitives (see menu.css), so
 * Select options look identical to Dropdown rows. Only Select-specific chrome —
 * the selected-row glyph alignment, scroll arrows, and group rhythm — lives here.
 */

@source "./select.tsx";

/* ■■ Trigger ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
 * Built on q-field-control; a button (not an input wrapper), so it owns the
 * value text directly and stays a single flex row. */
@utility q-select-trigger {
  cursor: pointer;
  text-align: start;
  appearance: none;
  color: var(--hf-color-text-primary);
  @apply text-q-label-sm-regular; /* Inter Regular 14px — matches the input element */

  &:focus-visible {
    outline: none;
    box-shadow: inset 0 0 0 var(--hf-border-width-medium) var(--hf-color-border-focus);
  }

  /* Keep the focus-style ring while the popup is open. */
  &[data-popup-open] {
    box-shadow: inset 0 0 0 var(--hf-border-width-medium) var(--hf-color-border-focus);
  }

  /* Invalid ring beats the open/focus ring (mirrors q-field-control-invalid). */
  &.q-field-control-invalid[data-popup-open],
  &.q-field-control-invalid:focus-visible {
    box-shadow: inset 0 0 0 var(--hf-border-width-medium) var(--hf-color-border-error);
  }

  &:disabled {
    cursor: not-allowed;
  }
}

/* Height presets — md (40px) is the default q-field-control height. */
@utility q-select-trigger-sm {
  height: var(--hf-space-800); /* 32px */
  padding-inline: var(--hf-space-200); /* 8px */
  @apply text-q-caption-sm-regular; /* 12px */
}

@utility q-select-trigger-lg {
  height: calc(var(--hf-space-1000) + var(--hf-space-200)); /* 48px */
  padding-inline: var(--hf-space-400); /* 16px */
  @apply text-q-body-md-regular; /* 16px */
}

/* ■■ Value (selected label / placeholder) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-select-value {
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
```

```
/* Placeholder tint when no value is selected (Base UI marks the trigger). */
.q-select-trigger[data-placeholder] .q-select-value {
  color: var(--hf-color-text-on-overlay-secondary); /* white 50% */
}

/* ■■ Trigger chevron ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-select-icon {
  display: inline-flex;
  flex-shrink: 0;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-500); /* 20px */
  height: var(--hf-space-500);
  color: var(--hf-color-icon-secondary);
  transition-property: transform;
  transition-duration: var(--hf-duration-180);
  transition-timing-function: var(--hf-ease-out-expo);

  & svg {
    width: 100%;
    height: 100%;
  }
}

.q-select-trigger[data-popup-open] .q-select-icon {
  transform: rotate(180deg);
}

/* ■■ Popup ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-select-content {
  /* The List owns scrolling; cap the popup so long option sets stay usable. */
  max-height: var(--available-height, none);
}

/* ■■ Connected (nested field) ■■■■■■■■■■■■■■■■■■■
```

```
  gap: var(--hf-space-050);
  min-height: var(--hf-space-0);
  overflow-y: auto;
  overscroll-behavior: contain;
}

@utility q-select-group {
  display: flex;
  flex-direction: column;
  gap: var(--hf-space-050);
}

@utility q-select-separator {
  height: var(--hf-border-width-thin);
  margin: var(--hf-space-100) var(--hf-space-0); /* 4px breathing room */
  background-color: var(--hf-color-border-subtle);
  border: var(--hf-border-width-none) solid transparent;
}

/* ■■ Option row ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ *
 * Built on q-menu-item: hover/highlight/focus/disabled come for free. Base UI
 * marks the chosen row with data-selected — keep it readable without a heavy
 * fill (the trailing check is the primary cue). */
@utility q-select-item {
  scroll-margin: var(--hf-space-200);
}

/* Selected row carries the same highlight fill as hover/keyboard-highlight, so
 * the current choice reads as an active, rounded pill inside the popup. */
.q-select-item[data-selected] {
  background-color: var(--hf-color-overlay-hover);
  color: var(--hf-color-text-primary);
}

@utility q-select-item-text {
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

/* Inside an ItemContent column the title sizes naturally — the `flex: 1 1 0`
 * above is for the ROW layout and would collapse the title's height to 0 in
 * the column stack. */
.q-menu-item-label .q-select-item-text {
  flex: none;
  width: 100%;
}

@utility q-select-item-indicator {
  display: inline-flex;
  flex-shrink: 0;
  align-items: center;
  justify-content: center;
  margin-inline-start: auto;
  color: var(--hf-color-icon-primary);
}

/* ■■ Compact builder-picker preset (Figma SC App Builder pickers: Duration /
 * Aspect Ratio / Voice). Rounded popup (radius/400), 44px radius/250 rows with
 * body-sm titles, caption secondary description, a subtle selected fill, and
 * the accent (lime) check. Popup min-width nudged to 200px for readable two-line
 * rows; max stays the default 400. */
@utility q-select-content-picker {
  --q-dropdown-radius: var(--hf-radius-400);
  --q-dropdown-item-outer-radius: var(--hf-radius-250);
  --q-menu-min-width: calc(var(--hf-space-2400) * 2 + var(--hf-space-200)); /* 200px */
}
```

```
.q-select-content-picker .q-menu-item {
  border-radius: var(--hf-radius-250);
  min-height: calc(var(--hf-space-1000) + var(--hf-space-100)); /* 44px */
  padding: var(--hf-space-200) var(--hf-space-300); /* 8 × 12 */
  @apply text-q-body-sm-medium;
}

.q-select-content-picker .q-menu-item-description {
  color: var(--hf-color-text-secondary);
  @apply text-q-caption-sm-regular;
}

.q-select-content-picker .q-select-item[data-selected] {
  background-color: var(--hf-color-transparent-light-10);
}

.q-select-content-picker .q-dropdown-check {
  color: var(--hf-color-icon-accent); /* lime check — builder pickers */
}

/* ■■ Scroll arrows (Base UI ScrollUp/DownArrow) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-select-scroll-arrow {
  display: flex;
  flex-shrink: 0;
  align-items: center;
  justify-content: center;
  height: var(--hf-space-500); /* 20px */
  color: var(--hf-color-icon-secondary);

  &[data-side] {
    /* Sit flush with the popup edges. */
    margin: calc(-1 * var(--hf-space-100)) var(--hf-space-0);
  }
}

@utility q-select-scroll-arrow-up {
  &::before {
    content: '';
    width: var(--hf-space-300);
    height: var(--hf-space-300);
    border-top: var(--hf-border-width-medium) solid currentColor;
    border-right: var(--hf-border-width-medium) solid currentColor;
    transform: rotate(-45deg);
    margin-top: var(--hf-space-100);
  }
}

@utility q-select-scroll-arrow-down {
  &::before {
    content: '';
    width: var(--hf-space-300);
    height: var(--hf-space-300);
    border-bottom: var(--hf-border-width-medium) solid currentColor;
    border-right: var(--hf-border-width-medium) solid currentColor;
    transform: rotate(45deg);
    margin-bottom: var(--hf-space-100);
  }
}

/* ■■ Reduced motion ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@media (prefers-reduced-motion: reduce) {
  .q-select-icon {
    transition-duration: 1ms; /* no token: 1ms reduced-motion near-disable; smallest duration token is 0ms instant */
  }
}
```

app/packages/quanta/src/components/select/select.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { createRef } from 'react'
import { IconFolder1Outlined as FolderIcon } from '@higgsfield-ai/icons/IconFolder1Outlined'
import { describe, expect, it, vi } from 'vitest'
import { Select } from './index.ts'

/** A title-only option (the common composition). */
function Option({ value, label }: { value: string, label: string }) {
  return (
    <Select.Item value={value}>
      <Select.ItemText>{label}</Select.ItemText>
      <Select.ItemIndicator />
    </Select.Item>
  )
}

// Base UI resolves the trigger Value label from `items` when the popup is closed.
const MODEL_ITEMS = { soul: 'Soul 2.0', gpt: 'GPT Image 2', kling: 'Kling 3.0' }

function Basic({ defaultOpen = false, ...rootProps }: Partial<Parameters<typeof Select.Root>[0]> & { defaultOpen?: boolean }) {
  return (
    <Select.Root defaultOpen={defaultOpen} items={MODEL_ITEMS} {...rootProps}>
      <Select.Trigger>
        <Select.Value placeholder="Choose a model" />
        <Select.Icon />
      </Select.Trigger>
      <Select.Content>
        <Select.Group>
          <Select.GroupLabel>Models</Select.GroupLabel>
          <Option value="soul" label="Soul 2.0" />
          <Option value="gpt" label="GPT Image 2" />
          <Option value="kling" label="Kling 3.0" />
        </Select.Group>
      </Select.Content>
    </Select.Root>
  )
}

describe('Select trigger', () => {
  it('renders a combobox trigger that looks like a field', () => {
    render(<Basic />)
    const trigger = screen.getByRole('combobox')
    expect(trigger).toHaveClass('q-field-control', 'q-select-trigger')
  })

  it('shows the placeholder when no value is selected', () => {
    render(<Basic />)
    expect(screen.getByRole('combobox')).toHaveTextContent('Choose a model')
    expect(screen.getByRole('combobox')).toHaveAttribute('data-placeholder')
  })

  it('applies the size preset to the trigger', () => {
    render(
      <Select.Root>
        <Select.Trigger size="lg"><Select.Value placeholder="x" /><Select.Icon /></Select.Trigger>
        <Select.Content><Option value="a" label="A" /></Select.Content>
      </Select.Root>,
    )
    expect(screen.getByRole('combobox')).toHaveClass('q-select-trigger-lg')
  })

  it('paints the invalid ring when invalid', () => {
    render(
      <Select.Root>
        <Select.Trigger invalid><Select.Value placeholder="x" /><Select.Icon /></Select.Trigger>
        <Select.Content><Option value="a" label="A" /></Select.Content>
      </Select.Root>,
```

```
    )
    const trigger = screen.getByRole('combobox')
    expect(trigger).toHaveClass('q-field-control-invalid')
    expect(trigger).toHaveAttribute('data-invalid')
  })

  it('disables the trigger when the root is disabled', () => {
    render(
      <Select.Root disabled>
        <Select.Trigger><Select.Value placeholder="x" /><Select.Icon /></Select.Trigger>
        <Select.Content><Option value="a" label="A" /></Select.Content>
      </Select.Root>,
    )
    expect(screen.getByRole('combobox')).toBeDisabled()
  })

  it('forwards a ref to the trigger button', () => {
    const ref = createRef<HTMLButtonElement>()
    render(
      <Select.Root>
        <Select.Trigger ref={ref}><Select.Value placeholder="x" /><Select.Icon /></Select.Trigger>
        <Select.Content><Option value="a" label="A" /></Select.Content>
      </Select.Root>,
    )
    expect(ref.current).toBe(screen.getByRole('combobox'))
  })
})

describe('Select popup', () => {
  it('renders the dropdown glass surface (defaultOpen)', () => {
    render(<Basic defaultOpen />)
    // The glass surface is the popup; role=listbox is on the inner scroll list.
    expect(screen.getByRole('listbox').closest('.q-select-content')).toHaveClass('q-dropdown-content', 'q-select-content')
  })

  it('applies the solid surface preset', () => {
    render(
      <Select.Root defaultOpen>
        <Select.Trigger><Select.Value placeholder="x" /><Select.Icon /></Select.Trigger>
        <Select.Content surface="solid"><Option value="a" label="A" /></Select.Content>
      </Select.Root>,
    )
    expect(screen.getByRole('listbox').closest('.q-select-content')).toHaveClass('q-dropdown-content-solid')
  })

  it('renders a group label and option rows', () => {
    render(<Basic defaultOpen />)
    expect(screen.getByText('Models')).toHaveClass('q-menu-group-label')
    expect(screen.getByRole('option', { name: 'Soul 2.0' })).toHaveClass('q-menu-item', 'q-select-item')
  })

  it('renders an option with a leading icon (composition)', () => {
    render(
      <Select.Root defaultOpen>
        <Select.Trigger><Select.Value placeholder="x" /><Select.Icon /></Select.Trigger>
        <Select.Content>
          <Select.Item value="folder">
            <Select.ItemIcon><FolderIcon /></Select.ItemIcon>
            <Select.ItemText>Folder</Select.ItemText>
            <Select.ItemIndicator />
          </Select.Item>
        </Select.Content>
      </Select.Root>,
    )
    const option = screen.getByRole('option', { name: 'Folder' })
    expect(option.querySelector('.q-menu-item-icon')).toBeInTheDocument()
  })
})

describe('Select selection', () => {
```

```
  it('seeds the trigger value from defaultValue', () => {
    render(<Basic defaultValue="gpt" />)
    expect(screen.getByRole('combobox')).toHaveTextContent('GPT Image 2')
    expect(screen.getByRole('combobox')).not.toHaveAttribute('data-placeholder')
  })

  it('selects an option on click and echoes it into the trigger', async () => {
    const user = userEvent.setup()
    render(<Basic />)
    await user.click(screen.getByRole('combobox'))
    await user.click(await screen.findByRole('option', { name: 'Kling 3.0' }))
    expect(screen.getByRole('combobox')).toHaveTextContent('Kling 3.0')
  })

  it('fires onValueChange with the chosen value', async () => {
    const user = userEvent.setup()
    const onValueChange = vi.fn()
    render(<Basic onValueChange={onValueChange} />)
    await user.click(screen.getByRole('combobox'))
    await user.click(await screen.findByRole('option', { name: 'Soul 2.0' }))
    expect(onValueChange).toHaveBeenCalledWith('soul', expect.anything())
  })

  it('marks the selected option with data-selected', () => {
    render(<Basic defaultOpen defaultValue="soul" />)
    expect(screen.getByRole('option', { name: 'Soul 2.0' })).toHaveAttribute('data-selected')
    expect(screen.getByRole('option', { name: 'GPT Image 2' })).not.toHaveAttribute('data-selected')
  })

  it('supports multiple selection (value is an array)', async () => {
    const user = userEvent.setup()
    const onValueChange = vi.fn()
    render(
      <Select.Root multiple onValueChange={onValueChange}>
        <Select.Trigger><Select.Value placeholder="Pick" /><Select.Icon /></Select.Trigger>
        <Select.Content>
          <Option value="a" label="Alpha" />
          <Option value="b" label="Beta" />
        </Select.Content>
      </Select.Root>,
    )
    await user.click(screen.getByRole('combobox'))
    await user.click(await screen.findByRole('option', { name: 'Alpha' }))
    expect(onValueChange).toHaveBeenLastCalledWith(['a'], expect.anything())
    await user.click(screen.getByRole('option', { name: 'Beta' }))
    expect(onValueChange).toHaveBeenLastCalledWith(['a', 'b'], expect.anything())
  })
})
```

app/packages/quanta/src/components/select/select.tsx

```
'use client'

import type { ComponentProps, ReactNode, Ref } from 'react'
import { createContext, useContext } from 'react'
import { IconCheckmark2MediumOutlined as CheckIcon } from '@higgsfield-ai/icons/IconCheckmark2MediumOutlined'
import { IconChevronDownMediumOutlined as ChevronIcon } from '@higgsfield-ai/icons/IconChevronDownMediumOutlined'
import { Select as Primitive } from '@base-ui/react/select'
import { cx } from '../utils/cx.ts'

/**
 * Select — a form-field dropdown on the Base UI `Select` primitive (controlled /
 * uncontrolled value, single or `multiple`, keyboard typeahead, ARIA listbox,
 * portal + positioning, hidden form input), skinned with quanta tokens.
 *
 * The TRIGGER looks like an `Input` field: the shared `q-field-control` surface
 * (bordered, ~40px tall, white-5% fill, lime focus ring, red `invalid` ring) with
 * a chevron `Select.Icon` on the right that flips on open and a placeholder when
 * empty. The POPUP is the dropdown glass/solid surface (`q-dropdown-content`),
 * and each option reuses the shared `q-menu-item*` row primitives (see menu.css),
 * so Select and Dropdown stay visually identical.
 *
 * COMPOSITION-FIRST. `Select.Item` renders whatever children you give it — wrap
 * `Select.ItemText` (the label echoed back into the trigger) with any leading
 * icon, badge, or meta, and a trailing check `Select.ItemIndicator` paints
 * automatically when the row is selected. Group options with `Select.Group` +
 * `Select.GroupLabel`, divide with `Select.Separator`:
 *
 *   <Select.Root defaultValue="soul" onValueChange={setModel}>
 *     <Select.Trigger>
 *       <Select.Value placeholder="Choose a model" />
 *       <Select.Icon />
 *     </Select.Trigger>
 *     <Select.Content>
 *       <Select.Group>
 *         <Select.GroupLabel>Models</Select.GroupLabel>
 *         <Select.Item value="soul">
 *           <Select.ItemIcon><HiggsfieldIcon /></Select.ItemIcon>
 *           <Select.ItemText>Soul 2.0</Select.ItemText>
 *           <Select.ItemIndicator />
 *         </Select.Item>
 *       </Select.Group>
 *     </Select.Content>
 *   </Select.Root>
 */

export type SelectSize = 'sm' | 'md' | 'lg'
export type SelectContentSurface = 'glass' | 'solid'
export type SelectContentSize = 'default' | 'picker'

/* ■■ Root (owns value/open state via Base UI) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

/**
 * `connected` flows Root → Trigger + Content so both parts coordinate the
 * seamless seam (the popup attaches flush to the field as one surface) without
 * the caller wiring two props.
 */
const SelectUiContext = createContext<{ connected: boolean }>({ connected: false })

type RootProps<Value, Multiple extends boolean | undefined> = Primitive.Root.Props<Value, Multiple> & {
  /**
   * Render the popup as a seamless extension of the trigger: same width, no gap,
   * and merged corners so the field + list read as one continuous surface.
   */
  connected?: boolean
}

/** Groups all parts; passes `value`/`defaultValue`/`onValueChange`/`multiple` straight through. */
function Root<Value, Multiple extends boolean | undefined = false>({ connected = false, ...props }: RootProps<Value, Multiple>) {
```

```
  return (
    <SelectUiContext.Provider value={{ connected }}>
      <Primitive.Root {...props} />
    </SelectUiContext.Provider>
  )
}

/* ■■ Trigger (the form-field surface) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

const TRIGGER_SIZE_CLASS = {
  sm: 'q-select-trigger-sm',
  md: '',
  lg: 'q-select-trigger-lg',
} satisfies Record<SelectSize, string>

type TriggerProps = Omit<ComponentProps<typeof Primitive.Trigger>, 'className'> & {
  className?: string
  /** Trigger height/type scale. `md` (40px) is the default Figma field size. */
  size?: SelectSize
  /** Invalid (error) state — paints the red field ring. */
  invalid?: boolean
  /**
   * Skip the field-surface classes entirely — the `render` host owns ALL
   * styling. Use when the trigger is another quanta control, e.g. the builder
   * setting row: `<Select.Trigger bare render={<SettingTrigger label="Voice" />}>`.
   * Base UI still drives `data-popup-open` / `data-placeholder` on the host.
   */
  bare?: boolean
}

function Trigger({ className, size = 'md', invalid = false, bare = false, ...props }: TriggerProps) {
  const { connected } = useContext(SelectUiContext)
  return (
    <Primitive.Trigger
      className={cx(
        !bare && 'q-field-control',
        !bare && 'q-select-trigger',
        !bare && TRIGGER_SIZE_CLASS[size],
        !bare && connected && 'q-select-trigger-connected',
        invalid && 'q-field-control-invalid',
        className,
      )}
      data-invalid={invalid ? '' : undefined}
      {...props}
    />
  )
}

/* ■■ Value (selected label echoed into the trigger) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

type ValueProps = Omit<ComponentProps<typeof Primitive.Value>, 'className'> & {
  className?: string
}

function Value({ className, ...props }: ValueProps) {
  return <Primitive.Value className={cx('q-select-value', className)} {...props} />
}

/* ■■ Icon (the trigger chevron — flips on open) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

type IconProps = Omit<ComponentProps<typeof Primitive.Icon>, 'className'> & {
  className?: string
}

function Icon({ className, children, ...props }: IconProps) {
  return (
    <Primitive.Icon className={cx('q-select-icon', className)} {...props}>
      {children ?? <ChevronIcon />}
    </Primitive.Icon>
  )
```

```
}

/* ■■ Content (Portal + Positioner + Popup + scrollable List) ■■■■■■■■■■■■■■■■■■ */

const SURFACE_CLASS = {
  glass: '',
  solid: 'q-dropdown-content-solid',
} satisfies Record<SelectContentSurface, string>

/** `picker` — the builder-picker preset (Duration / Aspect Ratio / Voice rows):
 * compact two-line rows, body-sm titles, accent check. */
const CONTENT_SIZE_CLASS = {
  default: '',
  picker: 'q-select-content-picker',
} satisfies Record<SelectContentSize, string>

type ContentProps = Omit<ComponentProps<typeof Primitive.Popup>, 'className'> & {
  className?: string
  positionerClassName?: string
  surface?: SelectContentSurface
  /** Popup scale: `default` menu rows or the compact `picker` builder rows. */
  size?: SelectContentSize
  side?: ComponentProps<typeof Primitive.Positioner>['side']
  align?: ComponentProps<typeof Primitive.Positioner>['align']
  sideOffset?: ComponentProps<typeof Primitive.Positioner>['sideOffset']
  alignOffset?: ComponentProps<typeof Primitive.Positioner>['alignOffset']
  collisionPadding?: ComponentProps<typeof Primitive.Positioner>['collisionPadding']
  /**
   * Whether the popup overlaps the trigger so the selected row aligns with the
   * trigger value. Off by default so the popup sits below the field like a menu.
   */
  alignItemWithTrigger?: boolean
  container?: ComponentProps<typeof Primitive.Portal>['container']
}

function Content({
  className,
  positionerClassName,
  surface = 'glass',
  size = 'default',
  side = 'bottom',
  align = 'start',
  sideOffset = 4,
  alignOffset,
  collisionPadding,
  alignItemWithTrigger = false,
  container,
  children,
  ...props
}: ContentProps) {
  const { connected } = useContext(SelectUiContext)
  // Connected = the popup NESTS the field: a wider rounded card that overlaps and
  // sits BEHIND the still-rounded trigger (the field gets a higher z-index), with
  // top padding so the rows clear the field. We pull the popup up over the trigger
  // (negative sideOffset, tuned to the default 40px field) and centre it so the
  // narrower field tucks inside the wider popup. It is rendered WITHOUT the portal
  // so the trigger can paint on top of the popup in the same stacking context.
  const tree = (
    <Primitive.Positioner
      className={cx('q-select-positioner', connected && 'q-select-positioner-connected', positionerClassName)}
      side={side}
      align={connected ? 'center' : align}
      sideOffset={connected ? -48 : sideOffset}
      alignOffset={alignOffset}
      collisionPadding={collisionPadding}
      alignItemWithTrigger={alignItemWithTrigger}
    >
      <Primitive.Popup
        className={cx('q-dropdown-content', 'q-select-content', SURFACE_CLASS[surface], CONTENT_SIZE_CLASS[size], connected && 'q-select-content-connected', className)}
        {...props}
```

```
      >
        <Primitive.ScrollUpArrow className="q-select-scroll-arrow q-select-scroll-arrow-up" />
        <Primitive.List className="q-select-list">{children}</Primitive.List>
        <Primitive.ScrollDownArrow className="q-select-scroll-arrow q-select-scroll-arrow-down" />
      </Primitive.Popup>
    </Primitive.Positioner>
  )
  return connected ? tree : <Primitive.Portal container={container}>{tree}</Primitive.Portal>
}

/* ■■ Group + GroupLabel + Separator ■■■
```

```
/** The option label echoed back into the trigger `Value` when selected. */
function ItemText({ className, ...props }: ItemTextProps) {
  return <Primitive.ItemText className={cx('q-select-item-text', className)} {...props} />
}

type ItemIndicatorProps = Omit<ComponentProps<typeof Primitive.ItemIndicator>, 'className'> & {
  className?: string
  children?: ReactNode
}

/** Trailing check — Base UI mounts it only while the row is selected. */
function ItemIndicator({ className, children, ...props }: ItemIndicatorProps) {
  return (
    <Primitive.ItemIndicator className={cx('q-select-item-indicator', className)} {...props}>
      {children ?? <CheckIcon className="q-dropdown-check" />}
    </Primitive.ItemIndicator>
  )
}

export const Select = {
  Root,
  Trigger,
  Value,
  Icon,
  Content,
  Group,
  GroupLabel,
  Separator,
  Item,
  ItemIcon,
  ItemContent,
  ItemText,
  ItemDescription,
  ItemIndicator,
}
```

app/packages/quanta/src/components/sidebar/index.ts

```
export { Sidebar } from './sidebar.tsx'
export type {
  SidebarActionVisibility,
  SidebarCollaboratorsProps,
  SidebarFooterItemProps,
  SidebarFooterVariant,
  SidebarItemProps,
  SidebarItemSize,
  SidebarItemVariant,
  SidebarProduct,
  SidebarProjectThumbnailProps,
  SidebarPromoBadgeProps,
  SidebarRootProps,
  SidebarSearchProps,
} from './sidebar.tsx'
```

app/packages/quanta/src/components/sidebar/sidebar.css

```
/**
 * Sidebar presentation, matched to the Figma Sidebar component set:
 * primitives 2438:253, variants 2441:1407, cinema instance 2798:8274.
 *
 * Most values are emitted design tokens. Literal dimensions appear only where
 * Figma itself exposes product/frame dimensions or custom assets: product rail
 * widths, the 810px Storybook frame height, shared-badge custom stroke colors,
 * and the promo badge gradient/transform geometry.
 */
@source "./sidebar.tsx";

@utility q-sidebar {
  --q-sidebar-radius: var(--hf-radius-400);

  box-sizing: border-box;
  display: flex;
  flex-direction: column;
  width: var(--q-sidebar-width);
  height: 100%;
  min-height: var(--hf-space-0);
  overflow: hidden;
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--q-sidebar-radius);
  background-color: var(--hf-color-background-elevated-end);
  box-shadow: inset 0 0 0 var(--hf-border-width-thin) var(--hf-color-border-subtle);
  color: var(--hf-color-text-primary);
  transition: width var(--hf-duration-220) var(--hf-ease-in-out);
}

.q-sidebar[data-product="cinema-studio"]:not(.q-sidebar-collapsed) {
  --q-sidebar-width: 14.4375rem; /* 231px, Figma product=cinema-studio */
}

.q-sidebar[data-product="marketing-studio"]:not(.q-sidebar-collapsed) {
  --q-sidebar-width: 14.9375rem; /* 239px, Figma product=marketing-studio */
}

.q-sidebar[data-product="supercomputer"]:not(.q-sidebar-collapsed) {
  --q-sidebar-width: 12.5625rem; /* 201px, Figma product=supercomputer */
}

@utility q-sidebar-flush {
  --q-sidebar-radius: var(--hf-radius-0);
}

/* Header: 52px high, 8px padding, then the 36px workspace badge row. */
@utility q-sidebar-header {
  box-sizing: border-box;
  display: flex;
  flex: none;
  align-items: center;
  width: 100%;
  height: calc(var(--hf-space-1200) + var(--hf-space-100));
  padding: var(--hf-space-200);
  gap: var(--hf-space-200);
}

@utility q-sidebar-switcher {
  box-sizing: border-box;
  display: flex;
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  align-items: center;
  height: calc(var(--hf-space-600) + var(--hf-space-300));
  padding: var(--hf-space-150);
  gap: var(--hf-space-200);
  overflow: hidden;
  border: var(--hf-border-width-none) solid transparent;
```

```
  border-radius: var(--hf-radius-300);
  background-color: transparent;
  color: var(--hf-color-text-primary);
  font: inherit;
  text-align: start;
  cursor: pointer;
  transition: background-color var(--hf-duration-150) ease;

  &:hover { background-color: var(--hf-color-overlay-hover); }
  &:focus-visible {
    outline: none;
    box-shadow: inset 0 0 0 var(--hf-border-width-thick) var(--hf-color-border-focus);
  }
}

@utility q-sidebar-logo {
  display: inline-flex;
  flex: none;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-600);
  height: var(--hf-space-600);
  overflow: visible;
  border-radius: var(--hf-radius-200);

  & > svg,
  & > img {
    display: block;
    width: 100%;
    height: 100%;
  }

  & > svg {
    overflow: visible;
  }
}

@utility q-sidebar-switcher-name {
  display: flex;
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  align-items: center;
  gap: var(--hf-space-100);
  overflow: hidden;
  color: var(--hf-color-text-primary);
  white-space: nowrap;
  text-overflow: ellipsis;

  /* The chevron is wrapped in <Icon size="sm" color="secondary"> (icon.css owns
     svg sizing + the color token); only the flex reset stays component-owned. */
  & > .q-sidebar-switcher-chevron {
    flex: none;
  }
}

@utility q-sidebar-toggle {
  display: inline-flex;
  flex: none;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-600);
  height: var(--hf-space-600);
  padding: var(--hf-space-0);
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--hf-radius-200);
  background-color: transparent;
  color: var(--hf-color-icon-secondary);
  cursor: pointer;
  transition: color var(--hf-duration-150) ease, background-color var(--hf-duration-150) ease;
```

```
  &:hover {
    color: var(--hf-color-icon-primary);
    background-color: var(--hf-color-overlay-hover);
  }
  &:focus-visible {
    outline: none;
    box-shadow: inset 0 0 0 var(--hf-border-width-thick) var(--hf-color-border-focus);
  }
  & > svg {
    width: var(--hf-space-400);
    height: var(--hf-space-400);
  }
}

@utility q-sidebar-body {
  display: flex;
  flex: 1 1 auto;
  flex-direction: column;
  width: 100%;
  min-height: var(--hf-space-0);
  /* Scroll the sections when they exceed the rail (header + footer stay pinned as
   * flex:none siblings). X stays clipped so collapsing labels still clip cleanly;
   * `flex:1 1 auto` + `min-height:0` above are what let this shrink-to-scroll. */
  overflow-x: hidden;
  overflow-y: auto;
  overscroll-behavior: contain;
}

@utility q-sidebar-section {
  box-sizing: border-box;
  display: flex;
  flex: none;
  flex-direction: column;
  width: 100%;
  padding: var(--hf-space-200);
  gap: var(--hf-space-050);
}

.q-sidebar-section:has(.q-sidebar-section-header) {
  gap: var(--hf-space-200);
}

@utility q-sidebar-section-items {
  display: flex;
  flex-direction: column;
  width: 100%;
  gap: var(--hf-space-050);
}

@utility q-sidebar-section-header {
  box-sizing: border-box;
  display: flex;
  flex: none;
  align-items: center;
  width: 100%;
  height: var(--hf-space-700);
  padding: var(--hf-space-150);
  gap: var(--hf-space-200);
  overflow: hidden;
}

@utility q-sidebar-section-title {
  display: flex;
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  flex-direction: column;
  justify-content: center;
  overflow: hidden;
  color: var(--hf-color-text-secondary);
  white-space: nowrap;
```

```
  text-overflow: ellipsis;
}

@utility q-sidebar-section-actions {
  display: inline-flex;
  flex: none;
  align-items: center;
  gap: var(--hf-space-100);
  color: var(--hf-color-icon-secondary);

  & > * {
    box-sizing: border-box;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    width: var(--hf-space-600);
    height: var(--hf-space-600);
    padding: var(--hf-space-0);
    border: var(--hf-border-width-none) solid transparent;
    border-radius: var(--hf-radius-200);
    background-color: transparent;
    color: inherit;
  }

  & > button {
    cursor: pointer;
  }

  & > button:hover,
  & > [aria-pressed="true"] {
    color: var(--hf-color-icon-primary);
    background-color: var(--hf-color-overlay-hover);
  }
}

@utility q-sidebar-row {
  box-sizing: border-box;
  display: flex;
  align-items: center;
  width: 100%;
  overflow: hidden;
  border: var(--hf-border-width-none) solid transparent;
  background-color: transparent;
  color: var(--hf-color-text-primary);
  font: inherit;
  text-align: start;
  text-decoration: none;
  cursor: pointer;
  gap: var(--hf-space-200);
  transition: background-color var(--hf-duration-150) ease;

  &:hover { background-color: var(--hf-color-overlay-hover); }
  &:focus-visible {
    outline: none;
    box-shadow: inset 0 0 0 var(--hf-border-width-thick) var(--hf-color-border-focus);
  }
}

@utility q-sidebar-selected {
  background-color: var(--hf-color-transparent-light-10);

  &:hover { background-color: var(--hf-color-transparent-light-10); }
}

.q-sidebar-item.q-sidebar-selected:not(.q-sidebar-projectitem) .q-sidebar-label {
  color: var(--hf-color-text-primary);
}

/* Disabled row — the whole row dims, no hover/pointer (matches the menu row). */
.q-sidebar-row:disabled,
```

```
.q-sidebar-row[aria-disabled="true"] {
  opacity: var(--hf-opacity-disabled);
  pointer-events: none;
  cursor: default;
}

@utility q-sidebar-icon {
  display: inline-flex;
  flex: none;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-650);
  height: var(--hf-space-650);
  color: var(--hf-color-icon-primary);

  & > svg,
  & > img {
    width: var(--hf-space-650);
    height: var(--hf-space-650);
  }
}

@utility q-sidebar-label {
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  overflow: hidden;
  color: var(--hf-color-text-primary);
  white-space: nowrap;
  text-overflow: ellipsis;
}

@utility q-sidebar-meta {
  flex: none;
  overflow: hidden;
  color: var(--hf-color-transparent-light-30);
  white-space: nowrap;
  text-overflow: ellipsis;
}

/* The count keeps the Figma project-row's wider tracking; a descendant selector
   outranks the element-level text-q-caption-sm-regular utility deterministically
   (so the composite supplies family/size/weight/line-height, this the tracking). */
.q-sidebar-row .q-sidebar-meta {
  letter-spacing: var(--hf-type-letter-spacing-wide);
}

@utility q-sidebar-end {
  display: inline-flex;
  flex: none;
  align-items: center;
  gap: var(--hf-space-100);
  color: var(--hf-color-icon-secondary);

  & > svg {
    width: var(--hf-space-600);
    height: var(--hf-space-600);
  }
}

@utility q-sidebar-item {
  min-height: calc(var(--hf-space-600) + var(--hf-space-300));
  padding: var(--hf-space-150);
  border-radius: var(--hf-radius-300);
}

@utility q-sidebar-item-sm {
  min-height: var(--hf-space-800);
  padding: var(--hf-space-100) var(--hf-space-150);
  border-radius: var(--hf-radius-250);
}
```

```
@utility q-sidebar-projectitem {
  padding: var(--hf-space-150) var(--hf-space-300) var(--hf-space-150) var(--hf-space-150);
}

.q-sidebar-projectitem.q-sidebar-item-sm {
  min-height: var(--hf-space-700);
  padding: var(--hf-space-100) var(--hf-space-300) var(--hf-space-100) var(--hf-space-150);
  border-radius: var(--hf-radius-200);
}

@utility q-sidebar-actionrow {
  position: relative;
  display: block;
}

@utility q-sidebar-pinrow {
  position: relative;
  display: block;
}

@utility q-sidebar-action {
  position: absolute;
  inset-inline-end: var(--hf-space-150);
  top: 50%;
  z-index: 1;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  opacity: 1;
  transform: translateY(-50%);
  transition: opacity var(--hf-duration-150) ease, transform var(--hf-duration-150) ease;
  will-change: opacity, transform;
}

.q-sidebar-actionrow-project .q-sidebar-action {
  inset-inline-end: var(--hf-space-300);
}

.q-sidebar-actionrow-hover .q-sidebar-action {
  opacity: 0;
  pointer-events: none;
  transform: translateY(-50%) translateX(var(--hf-space-100));
}

.q-sidebar-actionrow-hover:hover .q-sidebar-action,
.q-sidebar-actionrow-hover:focus-within .q-sidebar-action {
  opacity: 1;
  pointer-events: auto;
  transform: translateY(-50%) translateX(var(--hf-space-0));
}

.q-sidebar-actionrow-hover:hover .q-sidebar-meta,
.q-sidebar-actionrow-hover:hover .q-sidebar-end,
.q-sidebar-actionrow-hover:focus-within .q-sidebar-meta,
.q-sidebar-actionrow-hover:focus-within .q-sidebar-end {
  opacity: 0;
}

@utility q-sidebar-action-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-600);
  height: var(--hf-space-600);
  padding: var(--hf-space-0);
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--hf-radius-150);
  background-color: transparent;
  color: var(--hf-color-icon-secondary);
  cursor: pointer;
```

```
  transition: background-color var(--hf-duration-150) ease, color var(--hf-duration-150) ease, transform var(--hf-duration-150) ease;

  &:hover,
  &[data-popup-open] {
    background-color: var(--hf-color-overlay-hover);
    color: var(--hf-color-icon-primary);
  }
  &:active {
    transform: scale(0.96);
  }
  &:focus-visible {
    outline: none;
    box-shadow: inset 0 0 0 var(--hf-border-width-thick) var(--hf-color-border-focus);
  }
  & > svg {
    width: var(--hf-space-400);
    height: var(--hf-space-400);
  }
}

@utility q-sidebar-pin {
  position: absolute;
  inset-inline-end: var(--hf-space-150);
  top: 50%;
  z-index: 1;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-500);
  height: var(--hf-space-500);
  padding: var(--hf-space-0);
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--hf-radius-150);
  background-color: transparent;
  color: var(--hf-color-icon-tertiary);
  cursor: pointer;
  opacity: 0;
  transform: translateY(-50%) translateX(var(--hf-space-100)) scale(0.96);
  transition: opacity var(--hf-duration-150) ease, color var(--hf-duration-150) ease, background-color var(--hf-duration-150) ease, transform var(--hf-duration-150) ease;
  will-change: opacity, transform;

  &:hover {
    color: var(--hf-color-icon-primary);
    background-color: var(--hf-color-overlay-hover);
  }
  &:focus-visible {
    opacity: 1;
    outline: none;
    box-shadow: inset 0 0 0 var(--hf-border-width-thick) var(--hf-color-border-focus);
  }
  &[data-pinned] {
    opacity: 1;
    color: var(--hf-color-icon-primary);
    transform: translateY(-50%) translateX(var(--hf-space-0)) scale(1);
  }
  & > svg {
    width: calc(var(--hf-space-300) + var(--hf-space-050));
    height: calc(var(--hf-space-300) + var(--hf-space-050));
  }
}

.q-sidebar-actionrow-project .q-sidebar-pin {
  inset-inline-end: var(--hf-space-300);
}

.q-sidebar-projectitem .q-sidebar-end > svg {
  width: var(--hf-space-400);
  height: var(--hf-space-400);
}
```

```
.q-sidebar-actionrow-selected .q-sidebar-pin > svg,
.q-sidebar-pinned .q-sidebar-pin > svg {
  width: var(--hf-space-400);
  height: var(--hf-space-400);
}

.q-sidebar-pinrow:hover .q-sidebar-pin,
.q-sidebar-pinrow:focus-within .q-sidebar-pin {
  opacity: 1;
  transform: translateY(-50%) translateX(var(--hf-space-0)) scale(1);
}
.q-sidebar-pinrow:hover .q-sidebar-meta,
.q-sidebar-pinrow:hover .q-sidebar-end,
.q-sidebar-pinrow:focus-within .q-sidebar-meta,
.q-sidebar-pinrow:focus-within .q-sidebar-end,
.q-sidebar-pinned .q-sidebar-meta,
.q-sidebar-pinned .q-sidebar-end { opacity: 0; }

@utility q-sidebar-search {
  box-sizing: border-box;
  display: flex;
  align-items: center;
  width: 100%;
  min-height: calc(var(--hf-space-600) + var(--hf-space-300));
  padding: var(--hf-space-150);
  gap: var(--hf-space-200);
  border-radius: var(--hf-radius-300);
  background-color: var(--hf-color-overlay-hover);
  color: var(--hf-color-icon-secondary);
  animation: q-sidebar-search-enter var(--hf-duration-160) var(--hf-ease-out-expo) both;
  will-change: opacity, transform;

  /* The default magnifier is wrapped in <Icon size="lg"> (icon.css owns svg
     sizing); it inherits this container's icon-secondary color via currentColor.
     A custom `icon` slot (any node) is still sized by this fallback. */
  & > .q-sidebar-search-icon {
    flex: none;
  }

  & > svg {
    flex: none;
    width: var(--hf-space-600);
    height: var(--hf-space-600);
  }
}

@keyframes q-sidebar-search-enter {
  from {
    opacity: 0;
    transform: translateY(calc(-1 * var(--hf-space-100))) scale(0.98);
  }
  to {
    opacity: 1;
    transform: translateY(var(--hf-space-0)) scale(1);
  }
}

@utility q-sidebar-search-input {
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  padding: var(--hf-space-0);
  border: var(--hf-border-width-none) solid transparent;
  outline: none;
  appearance: none;
  background: transparent;
  color: var(--hf-color-text-primary);

  &::placeholder { color: var(--hf-color-text-secondary); }
}
```

```
@utility q-sidebar-footer {
  box-sizing: border-box;
  display: flex;
  flex: none;
  flex-direction: column;
  width: 100%;
  padding: var(--hf-space-200);
  gap: var(--hf-space-200);
}

@utility q-sidebar-footeritem {
  min-height: calc(var(--hf-space-600) + var(--hf-space-300));
  padding: var(--hf-space-150);
  border-radius: var(--hf-radius-300);

  & > .q-sidebar-icon,
  & > .q-sidebar-icon > svg,
  & > .q-sidebar-icon > img {
    width: var(--hf-icon-md);
    height: var(--hf-icon-md);
  }
}

@utility q-sidebar-footeritem-promo {
  padding: var(--hf-space-100) var(--hf-space-300) var(--hf-space-100) var(--hf-space-150);
  gap: var(--hf-space-200);
  background-color: var(--hf-color-transparent-pink-10);

  & .q-sidebar-label {
    flex: 1 1 var(--hf-space-0);
    padding-inline: var(--hf-space-100);
    color: var(--hf-color-brand-pink);
    letter-spacing: var(--hf-type-letter-spacing-none);
  }
  & .q-sidebar-icon { color: var(--hf-color-brand-pink); }
  &:hover { background-color: var(--hf-color-transparent-pink-10); }
}

@utility q-sidebar-footeritem-login {
  padding: var(--hf-space-100) var(--hf-space-150);
  background-color: var(--hf-color-button-secondary);

  &:hover { background-color: var(--hf-color-button-secondary); }
}

/* Composite row pieces. */
@utility q-sidebar-project-thumb {
  position: relative;
  display: inline-flex;
  flex: none;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-600);
  height: var(--hf-space-600);
  overflow: visible;
  border-radius: var(--hf-radius-200);
  color: var(--hf-color-icon-secondary);

  & > img,
  & > .q-sidebar-project-thumb-fallback {
    position: absolute;
    inset: var(--hf-space-0);
    display: inline-flex;
    align-items: center;
    justify-content: center;
    width: 100%;
    height: 100%;
    overflow: hidden;
    border-radius: inherit;
    background-color: var(--hf-color-transparent-light-05);
```

```
    object-fit: cover;
    /* no token: drop layer is blur border-width-medium 1.5px and spread space-050 -2px, which matches neither inset-tile (drop blur 2px) nor inset-tile-hairline (drop spread -0.5px); not byte-identic
    box-shadow:
      0 var(--hf-space-050) var(--hf-border-width-medium) calc(-1 * var(--hf-space-050)) var(--hf-color-shadow-sm),
      inset 0 var(--hf-space-050) var(--hf-space-150) var(--hf-space-0) var(--hf-color-transparent-light-15);
    backdrop-filter: blur(var(--hf-space-300));
  }

  & svg {
    width: var(--hf-space-600);
    height: var(--hf-space-600);
  }
}

@utility q-sidebar-collaborators {
  display: inline-flex;
  isolation: isolate;
  align-items: flex-start;
}

@utility q-sidebar-collaborator-avatar {
  position: relative;
  z-index: 2;
  display: inline-flex;
  flex: none;
  width: 1.125rem; /* no token: 18px avatar — between --hf-space-400 (16) and 500 (20) */
  height: 1.125rem; /* no token: 18px avatar */
  margin-inline-end: calc(-1 * var(--hf-space-050));
  /* Clip to a circle so ANY source image (not just pre-cropped Figma PNGs) is round. */
  overflow: hidden;
  border-radius: var(--hf-radius-full);

  & > img {
    position: absolute;
    inset: -11.11%;
    width: 122.22%;
    height: 122.22%;
    object-fit: cover;
  }
}

@utility q-sidebar-collaborator-count {
  z-index: 1;
  display: inline-flex;
  flex: none;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-500);
  height: 1.125rem; /* no token: 18px count pill — between --hf-space-400 (16) and 500 (20) */
  padding-inline: var(--hf-space-100);
  overflow: hidden;
  border: var(--hf-border-width-thin) solid var(--hf-color-border-default);
  border-radius: var(--hf-radius-full);
  background-color: var(--hf-color-transparent-light-10);
  color: var(--hf-color-transparent-light-50);
  text-overflow: ellipsis;
  white-space: nowrap;
}

@utility q-sidebar-promo-badge {
  display: inline-flex;
  flex: none;
  align-items: center;
  justify-content: center;
  width: 3.441125rem; /* renders to 57.834px after the Figma custom skew */
  height: 0.939375rem; /* renders to 14.73px after the Figma custom scale */
  overflow: hidden;
  padding-inline: var(--hf-space-100);
  border-radius: var(--hf-radius-050);
  background: var(--hf-gradient-badge-pink);
```

```
  color: var(--hf-color-text-primary);
  font-family: var(--hf-type-family-grotesk);
  font-size: var(--hf-type-size-100);
  font-weight: var(--hf-type-weight-bold);
  letter-spacing: -0.015rem; /* -0.24px, Figma custom */
  line-height: normal;
  text-transform: uppercase;
  white-space: nowrap;
  transform: skewX(-10.89deg) scaleY(0.98); /* Figma custom */
}

/* Collapsed icon rail. */
@utility q-sidebar-collapsed {
  --q-sidebar-width: var(--q-sidebar-width-collapsed); /* collapsed rail; default lives in generated component tier */
}

.q-sidebar-switcher-name,
.q-sidebar-label,
.q-sidebar-meta,
.q-sidebar-end {
  transition: opacity var(--hf-duration-160) var(--hf-ease-in-out);
}

.q-sidebar-collapsed .q-sidebar-switcher-name,
.q-sidebar-collapsed .q-sidebar-label,
.q-sidebar-collapsed .q-sidebar-meta,
.q-sidebar-collapsed .q-sidebar-end {
  opacity: 0;
  pointer-events: none;
}

.q-sidebar-collapsed .q-sidebar-toggle,
.q-sidebar-collapsed .q-sidebar-pin,
.q-sidebar-collapsed .q-sidebar-search {
  display: none;
}

.q-sidebar-collapsed .q-sidebar-switcher,
.q-sidebar-collapsed .q-sidebar-row {
  flex: none;
  width: calc(var(--hf-space-600) + var(--hf-space-300));
}

.q-sidebar-collapsed .q-sidebar-row {
  padding-inline: var(--hf-space-150);
}

.q-sidebar-collapsed .q-sidebar-section:not(:has(.q-sidebar-section-actions)) .q-sidebar-section-header {
  height: var(--hf-space-0);
  padding: var(--hf-space-0);
}

.q-sidebar-collapsed .q-sidebar-section-title {
  flex-basis: var(--hf-space-0);
  display: none;
}

.q-sidebar-collapsed .q-sidebar-section-actions {
  width: var(--hf-space-600);
  overflow: hidden;
}

.q-sidebar-collapsed .q-sidebar-section-actions > * {
  flex: none;
}

@media (prefers-reduced-motion: reduce) {
  .q-sidebar,
  .q-sidebar-action,
  .q-sidebar-action-button,
```

```
  .q-sidebar-pin,
  .q-sidebar-switcher-name,
  .q-sidebar-label,
  .q-sidebar-meta,
  .q-sidebar-end {
    transition: none;
  }

  .q-sidebar-search {
    animation: none;
  }
}
```

app/packages/quanta/src/components/sidebar/sidebar.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { describe, expect, it, vi } from 'vitest'
import { Sidebar } from './index.ts'

/** Compose a label-only row. */
function Row({ label, ...props }: { label: string } & Parameters<typeof Sidebar.Item>[0]) {
  return (
    <Sidebar.Item {...props}>
      <Sidebar.ItemLabel>{label}</Sidebar.ItemLabel>
    </Sidebar.Item>
  )
}

describe('<Sidebar>', () => {
  it('renders the rail with header, body sections and footer', () => {
    const { container } = render(
      <Sidebar.Root aria-label="Main">
        <Sidebar.Header>
          <Sidebar.Switcher>
            <Sidebar.Title>Cinema Studio <Sidebar.SwitcherChevron /></Sidebar.Title>
          </Sidebar.Switcher>
        </Sidebar.Header>
        <Sidebar.Body>
          <Sidebar.Section>
            <Sidebar.SectionItems>
              <Row label="Home" selected />
            </Sidebar.SectionItems>
          </Sidebar.Section>
        </Sidebar.Body>
        <Sidebar.Footer>
          <Sidebar.FooterItem variant="login"><Sidebar.ItemLabel>Login</Sidebar.ItemLabel></Sidebar.FooterItem>
        </Sidebar.Footer>
      </Sidebar.Root>,
    )
    expect(container.querySelector('.q-sidebar')).toBeInTheDocument()
    expect(screen.getByText('Cinema Studio')).toHaveClass('q-sidebar-switcher-name')
    expect(container.querySelector('.q-sidebar-body')).toBeInTheDocument()
    expect(container.querySelector('.q-sidebar-footer')).toBeInTheDocument()
  })

  it('carries the composite type utilities on the row parts (token adoption)', () => {
    render(
      <Sidebar.Section>
        <Sidebar.SectionHeader><Sidebar.SectionTitle>Projects</Sidebar.SectionTitle></Sidebar.SectionHeader>
        <Sidebar.SectionItems>
          <Sidebar.Item>
            <Sidebar.ItemLabel>Blue Horizon</Sidebar.ItemLabel>
            <Sidebar.ItemMeta>484</Sidebar.ItemMeta>
          </Sidebar.Item>
        </Sidebar.SectionItems>
      </Sidebar.Section>,
    )
    expect(screen.getByText('Projects')).toHaveClass('q-sidebar-section-title', 'text-q-label-xs-medium')
    expect(screen.getByText('Blue Horizon')).toHaveClass('q-sidebar-label', 'text-q-body-sm-medium')
    expect(screen.getByText('484')).toHaveClass('q-sidebar-meta', 'text-q-caption-sm-regular')
  })

  it('renders the switcher chevron through Icon (token-sized, decorative)', () => {
    const { container } = render(<Sidebar.SwitcherChevron />)
    const chevron = container.querySelector('.q-sidebar-switcher-chevron')
    expect(chevron).toHaveClass('q-icon', 'q-icon-sm')
    expect(chevron).toHaveAttribute('aria-hidden', 'true')
  })

  it('renders the default search glyph through Icon (token-sized)', () => {
    const { container } = render(<Sidebar.Search />)
    expect(container.querySelector('.q-sidebar-search-icon')).toHaveClass('q-icon', 'q-icon-lg')
```

```
})

it('marks the selected item with the selected class + aria-current', () => {
  render(<Row label="Home" selected />)
  const item = screen.getByRole('button', { name: 'Home' })
  expect(item).toHaveClass('q-sidebar-row', 'q-sidebar-item', 'q-sidebar-selected')
  expect(item).toHaveAttribute('aria-current', 'page')
})

it('renders an Item as a link when href is set', () => {
  render(<Row label="Home" href="/home" />)
  const link = screen.getByRole('link', { name: 'Home' })
  expect(link).toHaveAttribute('href', '/home')
  expect(link).toHaveClass('q-sidebar-item')
})

it('applies the sm size class', () => {
  render(<Row label="Chat" size="sm" />)
  expect(screen.getByRole('button')).toHaveClass('q-sidebar-item-sm')
})

it('composes icon, label, meta and end in order, host swappable via render', () => {
  render(
    <Sidebar.Item render={<a href="/x" data-testid="link" />}>
      <Sidebar.ItemIcon><span data-testid="s" /></Sidebar.ItemIcon>
      <Sidebar.ItemLabel>Item</Sidebar.ItemLabel>
      <Sidebar.ItemMeta>9</Sidebar.ItemMeta>
      <Sidebar.ItemEnd><span data-testid="e" /></Sidebar.ItemEnd>
    </Sidebar.Item>,
  )
  const root = screen.getByTestId('link')
  expect(root.tagName).toBe('A')
  expect(root).toHaveClass('q-sidebar-item')
  expect(screen.getByTestId('s').closest('.q-sidebar-icon')).toBeInTheDocument()
  const s = screen.getByTestId('s'); const e = screen.getByTestId('e')
  expect(s.compareDocumentPosition(e) & Node.DOCUMENT_POSITION_FOLLOWING).toBeTruthy()
})

it('renders no pin button without onPinChange', () => {
  render(<Row label="Plain" />)
  expect(document.querySelector('.q-sidebar-pinrow')).toBeNull()
  expect(screen.queryByRole('button', { name: /pin/i })).not.toBeInTheDocument()
  expect(screen.getByRole('button', { name: 'Plain' })).toHaveClass('q-sidebar-item')
})

it('renders a pin toggle as a sibling (not nested) and fires onPinChange', async () => {
  const user = userEvent.setup()
  const onPinChange = vi.fn()
  render(<Row label="Project" onPinChange={onPinChange} />)
  const row = screen.getByRole('button', { name: 'Project' })
  const pin = screen.getByRole('button', { name: 'Pin' })
  expect(row).not.toContainElement(pin)
  expect(pin).toHaveAttribute('aria-pressed', 'false')
  await user.click(pin)
  expect(onPinChange).toHaveBeenCalledWith(true)
})

it('renders a row action as a sibling overlay', () => {
  render(<Row label="Project" action={<button type="button">Actions</button>} />)
  const row = screen.getByRole('button', { name: 'Project' })
  const action = screen.getByRole('button', { name: 'Actions' })
  expect(row).not.toContainElement(action)
  expect(action.closest('.q-sidebar-action')).toBeInTheDocument()
  expect(action.closest('.q-sidebar-actionrow')).toBeInTheDocument()
})

it('reflects the pinned state (filled, unpin label, aria-pressed)', () => {
  render(<Row label="Project" pinned onPinChange={vi.fn()} />)
  const pin = screen.getByRole('button', { name: 'Unpin' })
  expect(pin).toHaveAttribute('aria-pressed', 'true')
```

```
  expect(pin).toHaveAttribute('data-pinned')
  expect(pin.closest('.q-sidebar-pinrow')).toHaveClass('q-sidebar-pinned')
})

it('applies footer variant classes (promo / login)', () => {
  const { rerender } = render(<Sidebar.FooterItem variant="promo"><Sidebar.ItemLabel>Pricing</Sidebar.ItemLabel></Sidebar.FooterItem>)
  expect(screen.getByRole('button')).toHaveClass('q-sidebar-footeritem-promo')
  rerender(<Sidebar.FooterItem variant="login"><Sidebar.ItemLabel>Login</Sidebar.ItemLabel></Sidebar.FooterItem>)
  expect(screen.getByRole('button')).toHaveClass('q-sidebar-footeritem-login')
})

it('renders a section header from SectionTitle + SectionActions', () => {
  const { container } = render(
    <Sidebar.Section>
      <Sidebar.SectionHeader>
        <Sidebar.SectionTitle>Projects</Sidebar.SectionTitle>
        <Sidebar.SectionActions><span data-testid="add" /></Sidebar.SectionActions>
      </Sidebar.SectionHeader>
      <Sidebar.SectionItems><Row label="Alpha" /></Sidebar.SectionItems>
    </Sidebar.Section>,
  )
  expect(screen.getByText('Projects')).toHaveClass('q-sidebar-section-title')
  expect(container.querySelector('.q-sidebar-section-actions')).toContainElement(screen.getByTestId('add'))
})

it('renders the header switcher + a trailing toggle', () => {
  render(
    <Sidebar.Header>
      <Sidebar.Switcher><Sidebar.Title>WS</Sidebar.Title></Sidebar.Switcher>
      <Sidebar.Toggle>Collapse</Sidebar.Toggle>
    </Sidebar.Header>,
  )
  expect(screen.getByText('WS')).toHaveClass('q-sidebar-switcher-name')
  expect(screen.getByRole('button', { name: 'Collapse' })).toHaveClass('q-sidebar-toggle')
})

it('collapses to an icon strip', () => {
  const { container } = render(
    <Sidebar.Root collapsed>
      <Sidebar.Body><Row label="Home" /></Sidebar.Body>
    </Sidebar.Root>,
  )
  const root = container.querySelector('.q-sidebar')!
  expect(root).toHaveClass('q-sidebar-collapsed')
  expect(root).toHaveAttribute('data-collapsed', '')
})

it('uses defaultCollapsed for the initial uncontrolled state', () => {
  const { container } = render(
    <Sidebar.Root defaultCollapsed aria-label="Main"><Sidebar.Body /></Sidebar.Root>,
  )
  expect(container.querySelector('.q-sidebar')).toHaveClass('q-sidebar-collapsed')
})

it('Toggle flips the uncontrolled rail and reflects aria-expanded + default label', async () => {
  const user = userEvent.setup()
  const { container } = render(
    <Sidebar.Root aria-label="Main">
      <Sidebar.Header><Sidebar.Toggle><span data-testid="glyph" /></Sidebar.Toggle></Sidebar.Header>
    </Sidebar.Root>,
  )
  const root = container.querySelector('.q-sidebar')!
  const toggle = screen.getByRole('button', { name: 'Collapse sidebar' })
  expect(root).not.toHaveClass('q-sidebar-collapsed')
  expect(toggle).toHaveAttribute('aria-expanded', 'true')
  await user.click(toggle)
  expect(root).toHaveClass('q-sidebar-collapsed')
  const expand = screen.getByRole('button', { name: 'Expand sidebar' })
  expect(expand).toHaveAttribute('aria-expanded', 'false')
})
```

```
it('fires onCollapsedChange and stays put while collapse is controlled', async () => {
  const user = userEvent.setup()
  const onCollapsedChange = vi.fn()
  const { container } = render(
    <Sidebar.Root collapsed onCollapsedChange={onCollapsedChange} aria-label="Main">
      <Sidebar.Header><Sidebar.Toggle><span /></Sidebar.Toggle></Sidebar.Header>
    </Sidebar.Root>,
  )
  const root = container.querySelector('.q-sidebar')!
  await user.click(screen.getByRole('button', { name: 'Expand sidebar' }))
  expect(onCollapsedChange).toHaveBeenCalledWith(false)
  expect(root).toHaveClass('q-sidebar-collapsed') // unchanged until the parent flips `collapsed`
})

it('lets a custom Toggle onClick suppress the collapse via preventDefault', async () => {
  const user = userEvent.setup()
  const onClick = vi.fn((event: { preventDefault: () => void }) => event.preventDefault())
  const { container } = render(
    <Sidebar.Root aria-label="Main">
      <Sidebar.Toggle onClick={onClick}><span /></Sidebar.Toggle>
    </Sidebar.Root>,
  )
  await user.click(screen.getByRole('button', { name: 'Collapse sidebar' }))
  expect(onClick).toHaveBeenCalled()
  expect(container.querySelector('.q-sidebar')).not.toHaveClass('q-sidebar-collapsed')
})

it('keeps a text-labelled Toggle name and no aria-expanded outside a Root', () => {
  render(<Sidebar.Toggle>Collapse</Sidebar.Toggle>)
  const toggle = screen.getByRole('button', { name: 'Collapse' })
  expect(toggle).not.toHaveAttribute('aria-expanded')
})

it('expands a collapsed rail when the switcher is clicked, but not when expanded', async () => {
  const user = userEvent.setup()
  const onClick = vi.fn()
  const { container } = render(
    <Sidebar.Root defaultCollapsed aria-label="Main">
      <Sidebar.Header>
        <Sidebar.Switcher onClick={onClick}><Sidebar.Logo><span /></Sidebar.Logo></Sidebar.Switcher>
      </Sidebar.Header>
    </Sidebar.Root>,
  )
  const root = container.querySelector('.q-sidebar')!
  const switcher = container.querySelector('.q-sidebar-switcher')!
  expect(root).toHaveClass('q-sidebar-collapsed')
  await user.click(switcher)
  expect(onClick).toHaveBeenCalledTimes(1)
  expect(root).not.toHaveClass('q-sidebar-collapsed') // expanded
  // Expanded: clicking the switcher does not collapse (workspace-switch role intact).
  await user.click(switcher)
  expect(root).not.toHaveClass('q-sidebar-collapsed')
})

it('builds an Item row from start / title / meta / end slots', () => {
  render(<Sidebar.Item start={<span data-testid="i" />} title="Home" meta="9" end={<span data-testid="e" />} />)
  const row = screen.getByRole('button', { name: /Home/ })
  expect(row.querySelector('.q-sidebar-icon')).toContainElement(screen.getByTestId('i'))
  expect(row.querySelector('.q-sidebar-label')).toHaveTextContent('Home')
  expect(row.querySelector('.q-sidebar-meta')).toHaveTextContent('9')
  expect(row.querySelector('.q-sidebar-end')).toContainElement(screen.getByTestId('e'))
})

it('builds a FooterItem from slots', () => {
  render(<Sidebar.FooterItem variant="promo" start={<span data-testid="d" />} title="Pricing" end={<Sidebar.PromoBadge />} />)
  const row = screen.getByRole('button', { name: /Pricing/ })
  expect(row).toHaveClass('q-sidebar-footeritem-promo')
  expect(row.querySelector('.q-sidebar-label')).toHaveTextContent('Pricing')
  expect(row.querySelector('.q-sidebar-promo-badge')).toBeInTheDocument()
})
```

```
  it('renders children verbatim when no slot prop is passed (back-compat)', () => {
    render(<Sidebar.Item><span data-testid="raw">raw</span></Sidebar.Item>)
    const row = screen.getByRole('button', { name: /raw/ })
    expect(row.querySelector('[data-testid="raw"]')).toBeInTheDocument()
    expect(row.querySelector('.q-sidebar-label')).not.toBeInTheDocument()
  })

  it('self-manages the pin when uncontrolled (onPinChange only)', async () => {
    const user = userEvent.setup()
    const onPinChange = vi.fn()
    render(<Sidebar.Item title="Project" onPinChange={onPinChange} />)
    expect(screen.getByRole('button', { name: 'Pin' })).toHaveAttribute('aria-pressed', 'false')
    await user.click(screen.getByRole('button', { name: 'Pin' }))
    expect(onPinChange).toHaveBeenLastCalledWith(true)
    expect(screen.getByRole('button', { name: 'Unpin' })).toHaveAttribute('aria-pressed', 'true')
  })
})
```

app/packages/quanta/src/components/sidebar/sidebar.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { createContext, useCallback, useContext, useMemo, useState } from 'react'
import { useRender } from '@base-ui/react/use-render'
import { IconChevronGrabberVerticalOutlined as ChevronGrabberGlyph } from '@higgsfield-ai/icons/IconChevronGrabberVerticalOutlined'
import { IconMagnifyingGlassOutlined as SearchGlyph } from '@higgsfield-ai/icons/IconMagnifyingGlassOutlined'
import { IconPinFilledThin as PinGlyph } from '@higgsfield-ai/icons/IconPinFilledThin'
import { Icon } from '../icon/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * Sidebar — the product navigation rail, pixel-matched to the Figma "Sidebar"
 * system (primitives 2438:253, variants 2441:1407).
 *
 * SLOTS BY DEFAULT, composition when you need it (same rules as Dropdown). The
 * component owns the DESIGN — the rail, the 36px switcher, the row shells, the
 * pin / action overlays, the collapsed icon-strip. A row is one tag: pass
 * `start` / `title` / `meta` / `end` and it builds the anatomy; compose the
 * exported parts (`ItemIcon` / `ItemLabel` / `ItemMeta` / `ItemEnd`) via
 * `children` only for a bespoke layout.
 *
 *   <Sidebar.Root product="cinema-studio">
 *     <Sidebar.Header>
 *       <Sidebar.Switcher>
 *         <Sidebar.Logo><Logo/></Sidebar.Logo>
 *         <Sidebar.Title>Cinema Studio <Sidebar.SwitcherChevron/></Sidebar.Title>
 *       </Sidebar.Switcher>
 *       <Sidebar.Toggle><CollapseIcon/></Sidebar.Toggle>
 *     </Sidebar.Header>
 *
 *     <Sidebar.Body>
 *       <Sidebar.Section>
 *         <Sidebar.SectionItems>
 *           <Sidebar.Item selected start={<HomeIcon/>} title="Home" />
 *         </Sidebar.SectionItems>
 *       </Sidebar.Section>
 *
 *       <Sidebar.Section>
 *         <Sidebar.SectionHeader>
 *           <Sidebar.SectionTitle>Projects</Sidebar.SectionTitle>
 *         </Sidebar.SectionHeader>
 *         <Sidebar.SectionItems>
 *           <Sidebar.Item
 *             variant="project"
 *             start={<Sidebar.ProjectThumbnail src={cover}/>}
 *             title="Blue Horizon"
 *             meta="484"
 *             onPinChange={togglePin}
 *           />
 *         </Sidebar.SectionItems>
 *       </Sidebar.Section>
 *     </Sidebar.Body>
 *
 *     <Sidebar.Footer>
 *       <Sidebar.FooterItem variant="promo" start={<DiamondIcon/>} title="Pricing" end={<Sidebar.PromoBadge/>} />
 *     </Sidebar.Footer>
 *   </Sidebar.Root>
 *
 * Collapsing to the icon strip is BUILT IN: `Sidebar.Toggle` flips the rail with
 * no wiring, the Root owns the state (uncontrolled via `defaultCollapsed`, or
 * controlled via `collapsed` + `onCollapsedChange`), and collapsed labels / meta
 * / actions hide while icons centre. Rows render a `<button>`, an `<a>` when
 * `href` is set, or any element via `render` (Base UI useRender, e.g. a Link).
 */

export type SidebarItemSize = 'md' | 'sm'
export type SidebarItemVariant = 'nav' | 'project'
```

```
export type SidebarFooterVariant = 'default' | 'promo' | 'login'
export type SidebarProduct = 'cinema-studio' | 'marketing-studio' | 'supercomputer'
export type SidebarActionVisibility = 'always' | 'hover'

/* ■■ Collapse state (owned by Root, consumed by Toggle) ■■■■■■■■■■■■■■■■■■■■■■■■ */
type SidebarCollapseContextValue = { collapsed: boolean, toggle: () => void }
const SidebarCollapseContext = createContext<SidebarCollapseContextValue | null>(null)

/* ■■ Root ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
export type SidebarRootProps = ComponentProps<'aside'> & {
  /** Icon-strip mode (controlled): labels/meta/actions hide and icons center. */
  collapsed?: boolean
  /** Initial collapsed state when uncontrolled (Toggle drives it from here). */
  defaultCollapsed?: boolean
  /** Fires whenever the collapsed state changes (Toggle, or a controlled set). */
  onCollapsedChange?: (collapsed: boolean) => void
  /** Figma product variant; controls the variant rail width only. */
  product?: SidebarProduct
  /** Square corners (docked flush to a screen edge). */
  flush?: boolean
}
function Root({
  collapsed: collapsedProp,
  defaultCollapsed = false,
  onCollapsedChange,
  product,
  flush = false,
  className,
  children,
  ...props
}: SidebarRootProps) {
  const [internalCollapsed, setInternalCollapsed] = useState(defaultCollapsed)
  const isControlled = collapsedProp != null
  const collapsed = isControlled ? collapsedProp : internalCollapsed

  const setCollapsed = useCallback((next: boolean) => {
    if (!isControlled) setInternalCollapsed(next)
    onCollapsedChange?.(next)
  }, [isControlled, onCollapsedChange])

  const collapseContext = useMemo<SidebarCollapseContextValue>(
    () => ({ collapsed, toggle: () => setCollapsed(!collapsed) }),
    [collapsed, setCollapsed],
  )

  return (
    <SidebarCollapseContext.Provider value={collapseContext}>
      <aside
        data-collapsed={collapsed ? '' : undefined}
        data-product={product}
        className={cx('q-sidebar', collapsed && 'q-sidebar-collapsed', flush && 'q-sidebar-flush', className)}
        {...props}
      >
        {children}
      </aside>
    </SidebarCollapseContext.Provider>
  )
}

/* ■■ Header: layout slot for the switcher + toggle ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
function Header({ className, ...props }: ComponentProps<'div'>) {
  return <div className={cx('q-sidebar-header', className)} {...props} />
}

/**
 * The workspace switcher button (compose a Logo + Title inside). When the rail is
 * collapsed the Toggle is hidden, so clicking the switcher (just the logo then)
 * expands the rail — the collapse round-trips with no wiring. A custom `onClick`
 * runs first and can `preventDefault()` to keep the rail collapsed; when expanded
 * the switcher never auto-toggles, so its workspace-switching role is untouched.
```

```
 */
function Switcher({ className, type, onClick, ...props }: ComponentProps<'button'>) {
  const collapse = useContext(SidebarCollapseContext)
  return (
    <button
      type={type ?? 'button'}
      className={cx('q-sidebar-switcher', className)}
      onClick={(event) => {
        onClick?.(event)
        if (!event.defaultPrevented && collapse?.collapsed) collapse.toggle()
      }}
      {...props}
    />
  )
}

/** Brand mark slot. */
function Logo({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-sidebar-logo', className)} {...props} />
}

/** Workspace name (compose a SwitcherChevron after the text if wanted). */
function Title({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-sidebar-switcher-name', 'text-q-body-sm-medium', className)} {...props} />
}

/** The up/down workspace-switcher chevron. */
function SwitcherChevron({ className }: { className?: string }) {
  return <Icon as={ChevronGrabberGlyph} size="sm" color="secondary" className={cx('q-sidebar-switcher-chevron', className)} />
}

/**
 * Square icon button that collapses / expands the rail. Inside a `Sidebar.Root`
 * it is self-wiring: clicking flips the Root's collapsed state, it carries
 * `aria-expanded`, and an icon-only Toggle gets a default Collapse/Expand label.
 * A custom `onClick` runs first and can `preventDefault()` to suppress the flip.
 */
function Toggle({ className, type, children, onClick, 'aria-label': ariaLabel, ...props }: ComponentProps<'button'>) {
  const collapse = useContext(SidebarCollapseContext)
  // A string child names the button itself; an icon child needs the default label.
  const labelFromText = typeof children === 'string'
  const resolvedLabel = ariaLabel
    ?? (labelFromText || collapse == null ? undefined : collapse.collapsed ? 'Expand sidebar' : 'Collapse sidebar')
  return (
    <button
      type={type ?? 'button'}
      aria-expanded={collapse ? !collapse.collapsed : undefined}
      aria-label={resolvedLabel}
      className={cx('q-sidebar-toggle', className)}
      onClick={(event) => {
        onClick?.(event)
        if (!event.defaultPrevented) collapse?.toggle()
      }}
      {...props}
    >
      {children}
    </button>
  )
}

/* ■■ Body ■■■
```

```
  inputClassName?: string
}
function Search({ icon, className, inputClassName, placeholder = 'Search', ...props }: SidebarSearchProps) {
  return (
    <div className={cx('q-sidebar-search', className)}>
      {icon ?? <Icon as={SearchGlyph} size="lg" className="q-sidebar-search-icon" />}
      <input className={cx('q-sidebar-search-input', 'text-q-body-sm-medium', inputClassName)} placeholder={placeholder} {...props} />
    </div>
  )
}

/* ■■ Section: an optional header + a stack of items ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
function Section({ className, ...props }: ComponentProps<'div'>) {
  return <div className={cx('q-sidebar-section', className)} {...props} />
}

/** The section heading row (compose a SectionTitle + optional SectionActions). */
function SectionHeader({ className, ...props }: ComponentProps<'div'>) {
  return <div className={cx('q-sidebar-section-header', className)} {...props} />
}

function SectionTitle({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-sidebar-section-title', 'text-q-label-xs-medium', className)} {...props} />
}

/** Trailing section-header actions (search / sort / add icon buttons). */
function SectionActions({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-sidebar-section-actions', className)} {...props} />
}

/** The items stack inside a Section. */
function SectionItems({ className, ...props }: ComponentProps<'div'>) {
  return <div className={cx('q-sidebar-section-items', className)} {...props} />
}

/* ■■ Row parts (compose any row out of these) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
function ItemIcon({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-sidebar-icon', className)} {...props} />
}

function ItemLabel({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-sidebar-label', 'text-q-body-sm-medium', className)} {...props} />
}

function ItemMeta({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-sidebar-meta', 'text-q-caption-sm-regular', className)} {...props} />
}

function ItemEnd({ className, ...props }: ComponentProps<'span'>) {
  return <span className={cx('q-sidebar-end', className)} {...props} />
}

/** A bare icon button for section-header / row actions (e.g. a menu trigger). */
function ActionButton({ className, type, ...props }: ComponentProps<'button'>) {
  return <button type={type ?? 'button'} className={cx('q-sidebar-action-button', className)} {...props} />
}

/* ■■ Row slots (the ergonomic default; compose the parts directly for full control) ■■ */
type RowSlotProps = {
  /** Leading icon — `ItemIcon` slot. */
  start?: ReactNode
  /** Row label — `ItemLabel` slot. */
  title?: ReactNode
  /** Trailing count — `ItemMeta` slot. */
  meta?: ReactNode
  /** Trailing content (pin, collaborators, badge) — `ItemEnd` slot. */
  end?: ReactNode
}

/** Build a row body from slots; falls back to `children` when no slot is set (back-compat). */
```

```
function rowBody({ start, title, meta, end }: RowSlotProps, children: ReactNode): ReactNode {
  if (start == null && title == null && meta == null && end == null)
    return children
  return (
    <>
      {start != null ? <ItemIcon>{start}</ItemIcon> : null}
      {title != null ? <ItemLabel>{title}</ItemLabel> : null}
      {meta != null ? <ItemMeta>{meta}</ItemMeta> : null}
      {end != null ? <ItemEnd>{end}</ItemEnd> : null}
    </>
  )
}

/* ■■ Item: a row from slot props, or composed parts ■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
type RowOwnProps = {
  /** Interactive row action rendered as a sibling overlay, not inside the row button. */
  action?: ReactNode
  /** Whether the sibling row action is always visible or revealed on row hover/focus. */
  actionVisibility?: SidebarActionVisibility
  /** Link target: renders an `<a>` instead of a `<button>`. */
  href?: string
  /** Swap the host element (Base UI useRender, e.g. a router Link). */
  render?: useRender.RenderProp
}
type RowProps = Omit<ComponentProps<'button'>, 'title'> & RowOwnProps

export type SidebarItemProps = RowProps & RowSlotProps & {
  size?: SidebarItemSize
  /** Figma row primitive: NavItem or ProjectItem. */
  variant?: SidebarItemVariant
  selected?: boolean
  /** Pinned visual state — controlled. Omit (with `onPinChange`) to let the row manage its own pin. */
  pinned?: boolean
  /** Enable a pin toggle (a pin button that reveals on hover). `(next) => void`. */
  onPinChange?: (pinned: boolean) => void
}

/**
 * A sidebar row. The common row is one tag — pass `start` / `title` / `meta` /
 * `end` slot props and the row builds the standard anatomy; compose `ItemIcon` /
 * `ItemLabel` / `ItemMeta` / `ItemEnd` (via `children`) for a bespoke layout.
 * Renders a `<button>`, an `<a>` when `href` is set, or any element via `render`.
 * Pass `onPinChange` for a hover-revealed pin toggle (it self-manages its pinned
 * state when `pinned` is omitted), or `action` for a sibling overlay control.
 */
function Item({
  action,
  actionVisibility = 'always',
  size = 'md',
  variant = 'nav',
  selected = false,
  pinned,
  onPinChange,
  href,
  render,
  className,
  start,
  title,
  meta,
  end,
  children,
  ...rest
}: SidebarItemProps) {
  const [selfPinned, setSelfPinned] = useState(false)
  const pinnedResolved = pinned ?? selfPinned
  const isLink = href != null
  const main = useRender({
    render,
    defaultTagName: isLink ? 'a' : 'button',
    props: {
```

```
      className: cx(
        'q-sidebar-row',
        size === 'sm' ? 'q-sidebar-item-sm' : 'q-sidebar-item',
        variant === 'project' && 'q-sidebar-projectitem',
        selected && 'q-sidebar-selected',
        className,
      ),
      ...(isLink ? { href } : { type: 'button' as const }),
      ...(selected ? { 'aria-current': 'page' as const } : {}),
      ...rest,
      children: rowBody({ start, title, meta, end }, children),
    },
  })
  const hasPin = onPinChange != null
  if (action == null && !hasPin)
    return main

  // Action/pin controls are siblings of the row, never nested inside the row
  // button/link — keeps dropdown triggers and pin buttons valid HTML.
  return (
    <div
      className={cx(
        'q-sidebar-actionrow',
        hasPin && 'q-sidebar-pinrow',
        actionVisibility === 'hover' && 'q-sidebar-actionrow-hover',
        variant === 'project' && 'q-sidebar-actionrow-project',
        selected && 'q-sidebar-actionrow-selected',
        pinnedResolved && 'q-sidebar-pinned',
      )}
    >
      {main}
      {action != null ? <span className="q-sidebar-action">{action}</span> : null}
      {hasPin ? (
        <button
          type="button"
          className="q-sidebar-pin"
          data-pinned={pinnedResolved ? '' : undefined}
          aria-pressed={pinnedResolved}
          aria-label={pinnedResolved ? 'Unpin' : 'Pin'}
          onClick={() => {
            if (pinned == null)
              setSelfPinned(!pinnedResolved)
            onPinChange(!pinnedResolved)
          }}
        >
          <PinGlyph />
        </button>
      ) : null}
    </div>
  )
}

/* ■■ Footer ■■■
```

```
      children: rowBody({ start, title, meta, end }, children),
    },
  })
}

/* ■■ Figma composite pieces used inside Sidebar rows ■■■■■■■■■■■■■■■■■■■■■■■■■■ */
export type SidebarProjectThumbnailProps = ComponentProps<'span'> & {
  src?: string
  alt?: string
  fallback?: ReactNode
}
/**
 * Thumbnail tile. Any corner overlay — e.g. a composed `Sidebar.SharedBadge` —
 * is passed as `children`; the badge is NOT a built-in feature of the thumbnail
 * (the thumb's CSS positions a `.q-sidebar-shared-badge` child at the corner).
 */
function ProjectThumbnail({ src, alt = '', fallback, className, children, ...props }: SidebarProjectThumbnailProps) {
  return (
    <span className={cx('q-sidebar-project-thumb', className)} {...props}>
      {src != null
        ? <img src={src} alt={alt} />
        : <span className="q-sidebar-project-thumb-fallback">{fallback}</span>}
      {children}
    </span>
  )
}

export type SidebarCollaboratorsProps = ComponentProps<'span'> & {
  avatars?: Array<{ src: string, alt?: string }>
  count?: ReactNode
}
function Collaborators({ avatars = [], count, className, ...props }: SidebarCollaboratorsProps) {
  return (
    <span className={cx('q-sidebar-collaborators', className)} {...props}>
      {avatars.map((avatar, index) => (
        <span className="q-sidebar-collaborator-avatar" key={`${avatar.src}-${index}`}>
          <img src={avatar.src} alt={avatar.alt ?? ''} />
        </span>
      ))}
      {count != null ? <span className="q-sidebar-collaborator-count text-q-caption-xs-medium">{count}</span> : null}
    </span>
  )
}

export type SidebarPromoBadgeProps = ComponentProps<'span'>
function PromoBadge({ className, children = '50% OFF', ...props }: SidebarPromoBadgeProps) {
  return (
    <span className={cx('q-sidebar-promo-badge', className)} {...props}>
      {children}
    </span>
  )
}

export const Sidebar = {
  Root,
  Header,
  Switcher,
  Logo,
  Title,
  SwitcherChevron,
  Toggle,
  Body,
  Search,
  Section,
  SectionHeader,
  SectionTitle,
  SectionActions,
  SectionItems,
  Item,
  ItemIcon,
```

```
  ItemLabel,
  ItemMeta,
  ItemEnd,
  ActionButton,
  Footer,
  FooterItem,
  ProjectThumbnail,
  Collaborators,
  PromoBadge,
}
```

app/packages/quanta/src/components/slider/index.ts

```
export { Slider } from './slider.tsx'
export type { ContinuousSliderProps, SliderMode, SliderProps, SteppedSliderProps } from './slider.tsx'
```

app/packages/quanta/src/components/slider/slider.css

```
/**
 * Slider presentation.
 *
 * Most surfaces compose emitted q-* utilities (background, border and
 * border-width tokens) plus native spacing/sizing inline in slider.tsx — this
 * file registers those literal strings for Tailwind's scanner (quanta is a
 * symlinked package, which automatic content detection skips; same convention
 * as dot.css / kbd.css / divider.css).
 *
 * It also defines the default track WIDTH and the glass surface. The Figma
 * CardSizeControl track is 184px (11.5rem) — a fixed component dimension with no
 * matching spacing token (the scale jumps 64px → 96px), so it lives here as a
 * documented `--q-slider-width` default rather than an inline arbitrary
 * `w-[184px]`. Callers override by passing any width utility/class
 * (e.g. `className="w-[280px]"` or a token width), or by setting `--q-slider-width`.
 *
 * The track is a frosted-glass surface (same recipe as Modal / Sonner / Vault):
 * `background-glass` + 40px backdrop-blur + a `border-subtle` edge. The fill
 * (white-5% `overlay-hover`) and the ticks / current-divider (white-20%
 * `border-strong`) are authored as q-* utilities inline in slider.tsx.
 */
@source "./slider.tsx";

@utility q-slider {
  width: var(--q-slider-width);
  background-color: var(--hf-color-background-glass);
  backdrop-filter: blur(var(--hf-space-1000)); /* 40px */
  border: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
}
```

app/packages/quanta/src/components/slider/slider.test.tsx

```
import { act, fireEvent, render, screen } from '@testing-library/react'
import { createRef } from 'react'
import { describe, expect, it, vi } from 'vitest'
import { Slider } from './index.ts'

function trackRect(width = 200, left = 0) {
  return {
    x: left,
    y: 0,
    left,
    top: 0,
    right: left + width,
    bottom: 28,
    width,
    height: 28,
    toJSON() {},
  }
}

function mockTrackWidth(el: HTMLElement, width = 200) {
  const rect = trackRect(width)
  vi.spyOn(el, 'getBoundingClientRect').mockReturnValue(rect as DOMRect)
}

describe('<Slider> stepped (default)', () => {
  it('renders the glass track + interior notch ticks + 1 fill bar', () => {
    const { container } = render(<Slider aria-label="x" />)
    const slider = screen.getByRole('slider', { name: 'x' })
    // Glass surface (background-glass + blur + border) lives on the q-slider utility.
    expect(slider).toHaveClass('q-slider', 'h-7', 'rounded-lg', 'overflow-hidden', 'touch-none')
    expect(container.querySelectorAll('[aria-hidden="true"][style*="width"]')).toHaveLength(1)
    // steps=3 default → 1 interior notch tick (endpoints are the track edges).
    expect(container.querySelectorAll('.h-2.w-px')).toHaveLength(1)
  })

  it('notch fill: 0% at start (reachable) → 100% + rounded right at the last step', () => {
    const { container, rerender } = render(<Slider aria-label="x" steps={3} value={0} />)
    let fill = container.querySelector('[style*="width"]') as HTMLElement
    // step 0 is the empty start — not a cumulative 1/N fill.
    expect(fill.style.width).toBe('0%')
    expect(fill).not.toHaveClass('rounded-r-lg')

    rerender(<Slider aria-label="x" steps={3} value={1} />)
    fill = container.querySelector('[style*="width"]') as HTMLElement
    expect(fill.style.width).toBe('50%') // notch 1 of [0, .5, 1]

    rerender(<Slider aria-label="x" steps={3} value={2} />)
    fill = container.querySelector('[style*="width"]') as HTMLElement
    expect(fill.style.width).toBe('100%')
    expect(fill).toHaveClass('rounded-r-lg', 'border-r-0')
  })

  it('interior notch ticks scale with steps (steps-2)', () => {
    const { container } = render(<Slider aria-label="x" steps={5} value={0} />)
    // 5 notches → 3 interior ticks at 25% / 50% / 75%.
    const ticks = container.querySelectorAll('.h-2.w-px')
    expect(ticks).toHaveLength(3)
  })

  it('pointer can reach step 0 (start) by dragging to the far left', () => {
    const onChange = vi.fn()
    render(<Slider aria-label="x" steps={4} defaultValue={3} onChange={onChange} />)
    const track = screen.getByRole('slider')
    mockTrackWidth(track, 300)
    fireEvent.pointerDown(track, { pointerId: 1, clientX: 0, button: 0, pointerType: 'mouse' })
    expect(onChange).toHaveBeenLastCalledWith(0)
  })
```

```
  it('pointer drag snaps to the segment under the pointer', () => {
    const onChange = vi.fn()
    const onChangeEnd = vi.fn()
    render(<Slider aria-label="x" steps={3} defaultValue={2} onChange={onChange} onChangeEnd={onChangeEnd} />)
    const track = screen.getByRole('slider')
    mockTrackWidth(track, 300) // each segment 100 px wide

    fireEvent.pointerDown(track, { pointerId: 1, clientX: 50, button: 0, pointerType: 'mouse' })
    expect(onChange).toHaveBeenLastCalledWith(0)

    fireEvent.pointerMove(track, { pointerId: 1, clientX: 150 })
    expect(onChange).toHaveBeenLastCalledWith(1)

    fireEvent.pointerMove(track, { pointerId: 1, clientX: 290 })
    expect(onChange).toHaveBeenLastCalledWith(2)

    fireEvent.pointerUp(track, { pointerId: 1, clientX: 290 })
    expect(onChangeEnd).toHaveBeenCalledWith(2)
  })

  it('keyboard ←/→/Home/End', () => {
    const onChange = vi.fn()
    render(<Slider aria-label="x" steps={4} defaultValue={1} onChange={onChange} />)
    const slider = screen.getByRole('slider')
    act(() => slider.focus())
    fireEvent.keyDown(slider, { key: 'ArrowRight' })
    expect(onChange).toHaveBeenLastCalledWith(2)
    fireEvent.keyDown(slider, { key: 'End' })
    expect(onChange).toHaveBeenLastCalledWith(3)
    fireEvent.keyDown(slider, { key: 'Home' })
    expect(onChange).toHaveBeenLastCalledWith(0)
  })

  it('forwards ref to the root slider node without breaking internal measurement', () => {
    const ref = createRef<HTMLDivElement>()
    const onChange = vi.fn()
    render(<Slider aria-label="x" steps={4} defaultValue={3} onChange={onChange} ref={ref} />)
    const track = screen.getByRole('slider', { name: 'x' })
    expect(ref.current).toBe(track)
    // Internal trackRef still drives pointer math (a caller ref must not clobber it).
    mockTrackWidth(track, 300)
    fireEvent.pointerDown(track, { pointerId: 1, clientX: 0, button: 0, pointerType: 'mouse' })
    expect(onChange).toHaveBeenLastCalledWith(0)
  })

  it('aria attributes reflect stepped semantics', () => {
    render(<Slider aria-label="x" steps={5} value={2} />)
    const s = screen.getByRole('slider')
    expect(s).toHaveAttribute('aria-valuemin', '0')
    expect(s).toHaveAttribute('aria-valuemax', '4')
    expect(s).toHaveAttribute('aria-valuenow', '2')
  })
})

describe('<Slider> continuous', () => {
  it('renders without ticks by default', () => {
    const { container } = render(<Slider mode="continuous" aria-label="vol" />)
    expect(container.querySelectorAll('.h-2.w-px')).toHaveLength(0)
  })

  it('fill = (value - min) / (max - min)', () => {
    const { container } = render(
      <Slider mode="continuous" aria-label="vol" min={0} max={100} value={25} />,
    )
    const fill = container.querySelector('[style*="width"]') as HTMLElement
    expect(fill.style.width).toBe('25%')
  })

  it('drag produces fractional values (free, step=0)', () => {
    const onChange = vi.fn()
```

```
    render(<Slider mode="continuous" aria-label="vol" min={0} max={100} defaultValue={0} onChange={onChange} />)
    const track = screen.getByRole('slider')
    mockTrackWidth(track, 200)

    fireEvent.pointerDown(track, { pointerId: 1, clientX: 50, button: 0, pointerType: 'mouse' })
    // 50 / 200 = 25%
    expect(onChange).toHaveBeenLastCalledWith(25)

    fireEvent.pointerMove(track, { pointerId: 1, clientX: 137 })
    // 137 / 200 = 68.5%
    expect(onChange).toHaveBeenLastCalledWith(68.5)
  })

  it('step snaps to the nearest increment', () => {
    const onChange = vi.fn()
    render(
      <Slider mode="continuous" aria-label="vol" min={0} max={100} step={10} onChange={onChange} />,
    )
    const track = screen.getByRole('slider')
    mockTrackWidth(track, 200)
    fireEvent.pointerDown(track, { pointerId: 1, clientX: 47, button: 0, pointerType: 'mouse' })
    // 47/200 = 23.5 → snap to 20
    expect(onChange).toHaveBeenLastCalledWith(20)
  })

  it('keyboard moves by step (or 1% if free)', () => {
    const onChange = vi.fn()
    render(
      <Slider mode="continuous" aria-label="vol" min={0} max={100} step={5} defaultValue={50} onChange={onChange} />,
    )
    const slider = screen.getByRole('slider')
    act(() => slider.focus())
    fireEvent.keyDown(slider, { key: 'ArrowRight' })
    expect(onChange).toHaveBeenLastCalledWith(55)
    fireEvent.keyDown(slider, { key: 'PageUp' })
    expect(onChange).toHaveBeenLastCalledWith(100) // capped
    fireEvent.keyDown(slider, { key: 'Home' })
    expect(onChange).toHaveBeenLastCalledWith(0)
  })

  it('honours showTicks + steps for visual marks', () => {
    const { container } = render(
      <Slider mode="continuous" aria-label="vol" min={0} max={100} steps={5} showTicks />,
    )
    expect(container.querySelectorAll('.h-2.w-px')).toHaveLength(5)
  })
})

describe('<Slider> motion + disabled', () => {
  it('drag suspends the width transition; releasing restores it', () => {
    const { container } = render(<Slider aria-label="x" />)
    const track = screen.getByRole('slider')
    mockTrackWidth(track, 300)
    let fill = container.querySelector('[style*="width"]') as HTMLElement
    expect(fill).toHaveClass('transition-[width]', 'duration-200', 'ease-out')

    fireEvent.pointerDown(track, { pointerId: 1, clientX: 50, button: 0, pointerType: 'mouse' })
    fill = container.querySelector('[style*="width"]') as HTMLElement
    expect(fill).toHaveClass('transition-none')

    fireEvent.pointerUp(track, { pointerId: 1, clientX: 50 })
    fill = container.querySelector('[style*="width"]') as HTMLElement
    expect(fill).toHaveClass('transition-[width]')
  })

  it('disabled blocks pointer + keyboard + sets tabIndex -1', () => {
    const onChange = vi.fn()
    render(<Slider aria-label="x" disabled onChange={onChange} />)
    const slider = screen.getByRole('slider')
    expect(slider).toHaveAttribute('tabIndex', '-1')
```

```
    expect(slider).toHaveAttribute('aria-disabled', 'true')
    expect(slider).toHaveClass('pointer-events-none', 'opacity-50')
    fireEvent.keyDown(slider, { key: 'ArrowRight' })
    expect(onChange).not.toHaveBeenCalled()
  })
})
```

app/packages/quanta/src/components/slider/slider.tsx

```
'use client'

import type { ComponentProps, KeyboardEvent, PointerEvent as ReactPointerEvent, Ref } from 'react'
import { useCallback, useRef, useState } from 'react'
import { cx } from '../utils/cx.ts'

/**
 * Slider — segmented step control (a.k.a. "control bar"). Visual pinned to the
 * Figma CardSizeControl component (node 1332:3549): equal segments, cumulative
 * left-anchored fill bar, centered tick inside every segment.
 *
 * Supports two modes:
 *   - `mode="stepped"` (default) — N discrete notches. Value `k` (0..N-1) sits at
 *     `k/(N-1)` of the track, so step 0 is the empty start (reachable) and the
 *     last step fills the bar. Drag/click snap to the nearest notch; the fill
 *     edge lands exactly on the current notch's tick.
 *   - `mode="continuous"` — a free 0..1 (or min..max) bar; drag-anywhere, with
 *     an optional `step` to quantize. Ticks are hidden by default.
 *
 * Visuals (default-dark theme — light theme inherits via the same semantic
 * tokens):
 *   - track: `bg-q-background-tertiary`, `rounded-lg` (8px), `h-7` (28px),
 *            `overflow-hidden` so the fill bar is clipped by the rounded shape.
 *   - fill:  absolute, anchored to the track edges (`inset-y-0 left-0`),
 *            `bg-q-overlay-hover` (white-5%) with a
 *            `border-r-q-thin border-q-border-strong` (white-20%) divider on
 *            the right edge. `rounded-l-lg` matches the outer track radius;
 *            when full it also rounds the right corner and drops the divider.
 *            width transitions 200 ms ease-out except while actively dragging.
 *   - tick:  decorative overlay inset by `050` (2 px) on all sides; a `w-px
 *            h-2` mark per segment, `bg-q-border-strong`, rounded-sm.
 *
 * Pointer: a single Pointer Events handler on the track captures the pointer
 * and tracks mouse / touch / pen identically; `touch-action: none` keeps the
 * page from scrolling during a drag.
 */

export type SliderMode = 'stepped' | 'continuous'

type CommonProps = Omit<
  ComponentProps<'div'>,
  'onChange' | 'defaultValue' | 'onPointerDown' | 'onPointerMove' | 'onPointerUp'
> & {
  /** Disable interaction. Visual remains rendered, opacity dimmed. */
  disabled?: boolean
  /** Render the per-segment ticks. Default: true in stepped, false in continuous. */
  showTicks?: boolean
  /** Called continuously as the value changes (drag / key / click). */
  onChange?: (value: number) => void
  /** Called once when the user releases the pointer (drag end). */
  onChangeEnd?: (value: number) => void
  'aria-label'?: string
}

export type SteppedSliderProps = CommonProps & {
  mode?: 'stepped'
  /** Number of discrete steps. Min 2. Default 3. */
  steps?: number
  /** 0-indexed selected step. */
  value?: number
  defaultValue?: number
}

export type ContinuousSliderProps = CommonProps & {
  mode: 'continuous'
  /** Range min. Default 0. */
  min?: number
  /** Range max. Default 1. */
```

```
  max?: number
  /** Snap increment. Default 0 (= free, no snap). */
  step?: number
  /** Number of tick marks to render when showTicks is true. Default 0. */
  steps?: number
  /** Current value in [min..max]. */
  value?: number
  defaultValue?: number
}

export type SliderProps = SteppedSliderProps | ContinuousSliderProps

const clamp = (n: number, min: number, max: number) => Math.max(min, Math.min(max, n))

// Fan a single node out to several refs (internal measuring ref + caller ref).
function mergeRefs<T>(...refs: (Ref<T> | undefined)[]) {
  return (node: T | null) => {
    for (const ref of refs) {
      if (typeof ref === 'function')
        ref(node)
      else if (ref)
        (ref as { current: T | null }).current = node
    }
  }
}

export function Slider(rawProps: SliderProps) {
  const {
    mode = 'stepped',
    value,
    defaultValue,
    onChange,
    onChangeEnd,
    disabled = false,
    showTicks,
    className,
    onKeyDown,
    ref,
    'aria-label': ariaLabel,
    ...rest
  } = rawProps as SliderProps & { mode?: SliderMode, ref?: Ref<HTMLDivElement> }

  // Mode-specific config.
  const steps = Math.max(2, (rawProps as SteppedSliderProps).steps ?? (mode === 'stepped' ? 3 : 0))
  const min = (rawProps as ContinuousSliderProps).min ?? 0
  const max = (rawProps as ContinuousSliderProps).max ?? 1
  const snap = (rawProps as ContinuousSliderProps).step ?? 0
  const ticksVisible = showTicks ?? (mode === 'stepped')

  // Normalise default.
  const initial = (() => {
    if (defaultValue !== undefined)
      return defaultValue
    if (mode === 'stepped')
      return 0
    return min
  })()

  const [internal, setInternal] = useState<number>(() => sanitize(initial, mode, steps, min, max, snap))
  const isControlled = value !== undefined
  const current = isControlled ? sanitize(value as number, mode, steps, min, max, snap) : internal

  const trackRef = useRef<HTMLDivElement | null>(null)
  const [isDragging, setIsDragging] = useState(false)
  const latestRef = useRef(current)
  latestRef.current = current

  const commitValue = useCallback(
    (next: number) => {
      const clean = sanitize(next, mode, steps, min, max, snap)
```

```
    if (clean === latestRef.current)
      return
    latestRef.current = clean
    if (!isControlled)
      setInternal(clean)
    onChange?.(clean)
  },
  [isControlled, max, min, mode, onChange, snap, steps],
)

const valueFromPointer = useCallback(
  (clientX: number): number => {
    const track = trackRef.current
    if (!track)
      return latestRef.current
    const rect = track.getBoundingClientRect()
    if (rect.width <= 0)
      return latestRef.current
    const f = clamp((clientX - rect.left) / rect.width, 0, 1)
    if (mode === 'stepped') {
      // Notches at i/(steps-1); snap to the nearest so f=0 → 0 (start) and
      // f=1 → steps-1 (end) are both reachable.
      return clamp(Math.round(f * (steps - 1)), 0, steps - 1)
    }
    return min + f * (max - min)
  },
  [max, min, mode, steps],
)

const handlePointerDown = (e: ReactPointerEvent<HTMLDivElement>) => {
  if (disabled || e.button !== 0 && e.pointerType === 'mouse')
    return
  e.preventDefault()
  const track = trackRef.current
  if (!track)
    return
  try {
    track.setPointerCapture(e.pointerId)
  }
  catch {
    // ignore — some browsers throw on synthetic events in tests
  }
  setIsDragging(true)
  commitValue(valueFromPointer(e.clientX))
}

const handlePointerMove = (e: ReactPointerEvent<HTMLDivElement>) => {
  if (!isDragging || disabled)
    return
  commitValue(valueFromPointer(e.clientX))
}

const handlePointerEnd = (e: ReactPointerEvent<HTMLDivElement>) => {
  if (!isDragging)
    return
  setIsDragging(false)
  const track = trackRef.current
  if (track && track.hasPointerCapture?.(e.pointerId)) {
    try {
      track.releasePointerCapture(e.pointerId)
    }
    catch {}
  }
  onChangeEnd?.(latestRef.current)
}

const keyboardStep = (() => {
  if (mode === 'stepped')
    return 1
  if (snap > 0)
```

```
    return snap
  return (max - min) / 100
})()

const handleKeyDown = (e: KeyboardEvent<HTMLDivElement>) => {
  onKeyDown?.(e)
  if (e.defaultPrevented || disabled)
    return
  switch (e.key) {
    case 'ArrowLeft':
    case 'ArrowDown':
      e.preventDefault()
      commitValue(current - keyboardStep)
      break
    case 'ArrowRight':
    case 'ArrowUp':
      e.preventDefault()
      commitValue(current + keyboardStep)
      break
    case 'PageDown':
      e.preventDefault()
      commitValue(current - keyboardStep * 10)
      break
    case 'PageUp':
      e.preventDefault()
      commitValue(current + keyboardStep * 10)
      break
    case 'Home':
      e.preventDefault()
      commitValue(mode === 'stepped' ? 0 : min)
      break
    case 'End':
      e.preventDefault()
      commitValue(mode === 'stepped' ? steps - 1 : max)
      break
  }
}

// Fill fraction in [0..1]. Stepped: notch position k/(N-1) — step 0 is empty.
const fillPct = (() => {
  if (mode === 'stepped')
    return (current / (steps - 1)) * 100
  if (max === min)
    return 0
  return ((current - min) / (max - min)) * 100
})()
const isFull = fillPct >= 100 - 1e-6

// Continuous tick marks (decorative scale, evenly spread in an inset grid).
const contTickCount = ticksVisible && mode === 'continuous'
  ? ((rawProps as ContinuousSliderProps).steps ?? 0)
  : 0

return (
  <div
    ref={mergeRefs(trackRef, ref)}
    role="slider"
    tabIndex={disabled ? -1 : 0}
    aria-label={ariaLabel}
    aria-valuemin={mode === 'stepped' ? 0 : min}
    aria-valuemax={mode === 'stepped' ? steps - 1 : max}
    aria-valuenow={current}
    aria-orientation="horizontal"
    aria-disabled={disabled || undefined}
    data-disabled={disabled ? '' : undefined}
    data-dragging={isDragging ? '' : undefined}
    data-mode={mode}
    onKeyDown={handleKeyDown}
    onPointerDown={handlePointerDown}
    onPointerMove={handlePointerMove}
```

```
      onPointerUp={handlePointerEnd}
      onPointerCancel={handlePointerEnd}
      className={cx(
        'q-slider q-focus-ring relative inline-flex h-7 items-center overflow-hidden rounded-lg',
        'cursor-pointer touch-none select-none outline-none',
        disabled && 'pointer-events-none cursor-not-allowed opacity-50',
        className,
      )}
      {...rest}
    >
      <span
        aria-hidden
        style={{ width: `${fillPct}%` }}
        className={cx(
          'pointer-events-none absolute inset-y-0 left-0 bg-q-overlay-hover',
          'border-r-q-thin border-q-border-strong',
          'rounded-l-lg',
          'ease-out',
          isDragging ? 'transition-none' : 'transition-[width] duration-200',
          isFull && 'rounded-r-lg border-r-0',
        )}
      />
      {/* Stepped ticks: interior notches at the SAME full-track fraction as the
       * fill, so the fill edge lands exactly on the current notch's tick. The
       * endpoints (0 / full) are the track edges, so no edge-clipped ticks. */}
      {ticksVisible && mode === 'stepped' && steps > 2 && (
        <span aria-hidden className="pointer-events-none absolute inset-0">
          {Array.from({ length: steps - 2 }, (_, i) => (
            <span
              key={i}
              className="absolute top-1/2 h-2 w-px -translate-x-1/2 -translate-y-1/2 rounded-sm bg-q-border-strong"
              style={{ left: `${((i + 1) / (steps - 1)) * 100}%` }}
            />
          ))}
        </span>
      )}
      {contTickCount > 0 && (
        <span
          aria-hidden
          className="pointer-events-none absolute inset-0.5 grid items-center"
          style={{ gridTemplateColumns: `repeat(${contTickCount}, minmax(0, 1fr))`, gap: 'var(--hf-space-050)' }}
        >
          {Array.from({ length: contTickCount }, (_, i) => (
            <span key={i} className="flex h-full items-center justify-center">
              <span className="h-2 w-px rounded-sm bg-q-border-strong" />
            </span>
          ))}
        </span>
      )}
    </div>
  )
}

function sanitize(
  value: number,
  mode: SliderMode,
  steps: number,
  min: number,
  max: number,
  snap: number,
): number {
  if (mode === 'stepped')
    return clamp(Math.round(value), 0, Math.max(0, steps - 1))
  const lo = Math.min(min, max)
  const hi = Math.max(min, max)
  let v = clamp(value, lo, hi)
  if (snap > 0) {
    const offset = Math.round((v - lo) / snap) * snap
    v = clamp(lo + offset, lo, hi)
  }
```

```
  return v
}
```

app/packages/quanta/src/components/sonner/index.ts

```
export { sonnerManager, toast, Toaster } from './sonner.tsx'
export type {
  SonnerActionConfig,
  SonnerPosition,
  SonnerVariant,
  ToasterProps,
} from './sonner.tsx'
```

app/packages/quanta/src/components/sonner/sonner.css

```
/**
 * Sonner (toast) presentation — Base UI Toast skinned with quanta tokens.
 * Registers the colocated source for Tailwind's scanner (quanta is symlinked,
 * so literal class strings in sonner.tsx must be declared) and authors the
 * surfaces as `@utility q-sonner-*` rules. The one non-token value is the toast
 * width, exposed as the overridable `--q-sonner-width` component dimension
 * whose default lives in the generated component-token tier
 * (src/css/components/component.css :root).
 */
@source "./sonner.tsx";

/* Fixed, corner-anchored region — the positioning context + hover target for
 * the stack (toasts are absolutely stacked inside). pointer-events pass through
 * the empty area; only the toasts are interactive. */
@utility q-sonner-viewport {
  position: fixed;
  z-index: var(--hf-z-index-toast);
  width: min(var(--q-sonner-width), calc(100vw - var(--hf-space-800)));
  pointer-events: none;

  & > * { pointer-events: auto; }

  &[data-position$="-right"] { right: var(--hf-space-400); }
  &[data-position$="-left"] { left: var(--hf-space-400); }
  &[data-position$="-center"] { left: 50%; transform: translateX(-50%); }
  &[data-position^="top"] { top: var(--hf-space-400); }
  &[data-position^="bottom"] { bottom: var(--hf-space-400); }
}

/* The toast card — glassy frosted surface (matches the Modal glass), absolutely
 * positioned so toasts pile into a stack. Collapsed: behind toasts peek up,
 * scale down and clip to the front height; expanded (hover/focus, or `expand`)
 * they spread to natural offsets + heights. Base UI owns the stack vars
 * (--toast-index / -offset-y / -frontmost-height / -height) + swipe movement. */
@utility q-sonner {
  --q-sonner-peek: var(--hf-space-250); /* 10px — collapsed peek per layer */
  --q-sonner-gap: 0.875rem;  /* 14px — gap between expanded toasts; no token: 0.875rem not in space scale (300=0.75rem→400=1rem) */

  position: absolute;
  left: var(--hf-space-0);
  right: var(--hf-space-0);
  width: 100%;
  /* front (index 0) paints on top; behind toasts only peek a sliver. */
  z-index: calc(1000 - var(--toast-index));
  box-sizing: border-box;
  display: flex;
  align-items: flex-start;
  gap: var(--hf-space-300);
  height: var(--toast-frontmost-height); /* collapsed: uniform with the front toast */
  padding: var(--hf-space-300) var(--hf-space-400);
  border-radius: var(--hf-radius-400);
  border: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  background-color: var(--hf-color-background-glass);
  backdrop-filter: blur(var(--hf-space-1000)); /* 40px — glass */
  color: var(--hf-color-text-primary);
  overflow: hidden;
  box-shadow: var(--hf-shadow-overlay);

  transition-property: transform, opacity, height;
  transition-duration: var(--hf-duration-400);
  transition-timing-function: var(--hf-ease-out-quart);
}

/* ■■ Bottom positions: anchor to the bottom, stack upward ■■■■■■■■■■■■■■■■■■■■■■ */
[data-position^="bottom"] .q-sonner {
  bottom: var(--hf-space-0);
  transform-origin: bottom center;
  /* Same size for every stacked toast — only offset (no scale-down). */
```

```
  transform: translateY(calc(var(--toast-index) * var(--q-sonner-peek) * -1));
}
[data-position^="bottom"] .q-sonner[data-expanded],
[data-position^="bottom"][data-expand] .q-sonner {
  height: var(--toast-height);
  transform: translateY(calc((var(--toast-offset-y) + var(--toast-index) * var(--q-sonner-gap)) * -1));
}
[data-position^="bottom"] .q-sonner[data-starting-style],
[data-position^="bottom"] .q-sonner[data-ending-style] {
  opacity: 0;
  transform: translateY(110%);
}

/* ■■ Top positions: anchor to the top, stack downward ■■■■■■■■■■■■■■■■■■■■■■■■■ */
[data-position^="top"] .q-sonner {
  top: var(--hf-space-0);
  transform-origin: top center;
  /* Same size for every stacked toast — only offset (no scale-down). */
  transform: translateY(calc(var(--toast-index) * var(--q-sonner-peek)));
}
[data-position^="top"] .q-sonner[data-expanded],
[data-position^="top"][data-expand] .q-sonner {
  height: var(--toast-height);
  transform: translateY(calc(var(--toast-offset-y) + var(--toast-index) * var(--q-sonner-gap)));
}
[data-position^="top"] .q-sonner[data-starting-style],
[data-position^="top"] .q-sonner[data-ending-style] {
  opacity: 0;
  transform: translateY(-110%);
}

/* Over the configured `limit` → kept mounted for the stack count, but hidden. */
.q-sonner[data-limited] { opacity: 0; pointer-events: none; }

/* Hover bridge — when expanded the stack spreads apart with a real gap between
 * cards. The viewport passes pointer events through its empty area (only the
 * cards are interactive), so the gaps are dead zones: dragging the cursor across
 * one drops the hover, collapsing the stack, which slides a card back under the
 * cursor and re-expands → flicker. A transparent pseudo fills each gap so the
 * hover region stays continuous. `overflow` is relaxed to visible while expanded
 * (clipping is only needed for the collapsed peek) so the bridge isn't clipped. */
[data-position] .q-sonner[data-expanded],
[data-position][data-expand] .q-sonner {
  overflow: visible;
}
[data-position^="bottom"] .q-sonner[data-expanded]::after,
[data-position^="bottom"][data-expand] .q-sonner::after {
  content: "";
  position: absolute;
  left: var(--hf-space-0);
  right: var(--hf-space-0);
  bottom: 100%;
  height: var(--q-sonner-gap);
}
[data-position^="top"] .q-sonner[data-expanded]::after,
[data-position^="top"][data-expand] .q-sonner::after {
  content: "";
  position: absolute;
  left: var(--hf-space-0);
  right: var(--hf-space-0);
  top: 100%;
  height: var(--q-sonner-gap);
}

/* Swipe-to-dismiss — the dragged (front) toast follows the pointer. Scoped to
 * `[data-position]` + placed last so it wins over the stack/expanded transform. */
[data-position] .q-sonner[data-swiping] {
  transition-duration: var(--hf-duration-instant);
  transform: translate(var(--toast-swipe-movement-x, 0), var(--toast-swipe-movement-y, 0));
}
```

```
@utility q-sonner-icon {
  display: flex;
  flex: none;
  align-items: center;
  justify-content: center;
  color: var(--hf-color-text-secondary);
}

@utility q-sonner-text {
  display: flex;
  flex: 1 1 auto;
  min-width: var(--hf-space-0);
  flex-direction: column;
  gap: var(--hf-space-050);
  padding-top: var(--hf-space-050);
}

@utility q-sonner-title {
  @apply text-q-body-sm-medium;
  color: var(--hf-color-text-primary);
}

@utility q-sonner-description {
  @apply text-q-caption-sm-regular;
  color: var(--hf-color-text-secondary);
}

@utility q-sonner-action {
  flex: none;
  align-self: center;
  padding: var(--hf-space-100) var(--hf-space-250);
  border-radius: var(--hf-radius-250);
  background-color: var(--hf-color-background-inverse);
  color: var(--hf-color-text-inverse);
  @apply text-q-label-sm-medium;
  cursor: pointer;
  transition: opacity var(--hf-duration-150) ease; /* `ease` keyword not in ease tokens */

  &:hover { opacity: 0.85; }
}

/* Holds a custom ReactNode action (e.g. a quanta Button) in the same slot as
 * the built-in action button. Layout only — the node brings its own styling. */
@utility q-sonner-action-slot {
  flex: none;
  align-self: center;
  display: flex;
  align-items: center;
  gap: var(--hf-space-200);
}

@utility q-sonner-close {
  flex: none;
  display: flex;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-500);
  height: var(--hf-space-500);
  margin: calc(-1 * var(--hf-space-050)) calc(-1 * var(--hf-space-100)) 0 0;
  border-radius: var(--hf-radius-full);
  color: var(--hf-color-text-tertiary);
  cursor: pointer;
  opacity: 0;
  transition: opacity var(--hf-duration-150) ease, background-color var(--hf-duration-150) ease, color var(--hf-duration-150) ease; /* `ease` keyword not in ease tokens */

  &:hover {
    background-color: var(--hf-color-overlay-dim-soft);
    color: var(--hf-color-text-primary);
  }
}
```

```
.q-sonner:hover .q-sonner-close,
.q-sonner-close:focus-visible {
  opacity: 1;
}

/* Variant icon accents (state colors). */
.q-sonner-success .q-sonner-icon { color: var(--hf-color-state-success-fg); }
.q-sonner-error .q-sonner-icon { color: var(--hf-color-state-error-fg); }
.q-sonner-warning .q-sonner-icon { color: var(--hf-color-state-warning-fg); }
.q-sonner-info .q-sonner-icon { color: var(--hf-color-state-info-fg); }

/* Loading spinner. */
@utility q-sonner-spinner {
  animation: q-sonner-spin var(--hf-duration-700) linear infinite; /* `linear` keyword not in ease tokens */
}
@keyframes q-sonner-spin {
  to { transform: rotate(360deg); }
}

@media (prefers-reduced-motion: reduce) {
  /* Keep the stack transforms (they ARE the layout) — just snap instead of animate. */
  .q-sonner { transition-duration: 1ms; } /* no token: 1ms snap sentinel; instant token is 0ms */
  .q-sonner-spinner { animation: none; }
}
```

app/packages/quanta/src/components/sonner/sonner.test.tsx

```
import { act, render, screen, waitFor } from '@testing-library/react'
import { afterEach, describe, expect, it } from 'vitest'
import { Toaster, toast } from './index.ts'

afterEach(() => {
  act(() => toast.dismiss())
})

describe('toast imperative API', () => {
  it('exposes the sonner-shaped surface', () => {
    expect(typeof toast).toBe('function')
    for (const m of ['success', 'error', 'warning', 'info', 'loading', 'message', 'dismiss', 'promise'])
      expect(typeof (toast as unknown as Record<string, unknown>)[m]).toBe('function')
  })
})

describe('<Toaster>', () => {
  it('renders a fired toast with its title + description', async () => {
    render(<Toaster />)
    act(() => { toast.success('Saved', { description: 'All good' }) })

    expect(await screen.findByText('Saved')).toBeInTheDocument()
    expect(screen.getByText('All good')).toBeInTheDocument()
  })

  it('applies the variant class', async () => {
    render(<Toaster />)
    act(() => { toast.error('Boom') })

    const title = await screen.findByText('Boom')
    expect(title.closest('.q-sonner')).toHaveClass('q-sonner-error')
  })

  it('renders the variant glyph through <Icon>', async () => {
    render(<Toaster />)
    act(() => { toast.success('Saved') })

    const card = (await screen.findByText('Saved')).closest('.q-sonner')
    // the status glyph is an <Icon>: q-icon is painted on the glyph svg inside the icon slot
    expect(card?.querySelector('.q-sonner-icon .q-icon')).toBeInTheDocument()
  })

  it('dismiss() removes toasts', async () => {
    render(<Toaster />)
    act(() => { toast('Temporary') })
    expect(await screen.findByText('Temporary')).toBeInTheDocument()

    act(() => { toast.dismiss() })
    await waitFor(() => expect(screen.queryByText('Temporary')).not.toBeInTheDocument())
  })

  it('renders a simple { label, onClick } action via the built-in button (back-compat)', async () => {
    render(<Toaster />)
    act(() => { toast('Message sent', { action: { label: 'Undo' } }) })
    const action = await screen.findByText('Undo')
    // built-in action keeps the q-sonner-action button styling
    expect(action).toHaveClass('q-sonner-action')
  })

  it('renders a custom ReactNode action in the action slot', async () => {
    render(<Toaster />)
    act(() => {
      toast('Saved', { action: <button type="button" data-testid="custom-action">Open</button> })
    })
    const custom = await screen.findByTestId('custom-action')
    expect(custom.closest('.q-sonner-action-slot')).toBeInTheDocument()
    // the custom node is NOT wrapped in the built-in action button
    expect(custom).not.toHaveClass('q-sonner-action')
```

```
  })
})
```

app/packages/quanta/src/components/sonner/sonner.tsx

```
'use client'

import type { ComponentProps, CSSProperties, ReactNode } from 'react'
import { isValidElement } from 'react'
import { Toast as Primitive } from '@base-ui/react/toast'
import { IconCircleCheckOutlined as SuccessGlyph } from '@higgsfield-ai/icons/IconCircleCheckOutlined'
import { IconCircleInfoOutlined as InfoGlyph } from '@higgsfield-ai/icons/IconCircleInfoOutlined'
import { IconCircleXOutlined as ErrorGlyph } from '@higgsfield-ai/icons/IconCircleXOutlined'
import { IconExclamationTriangleOutlined as WarningGlyph } from '@higgsfield-ai/icons/IconExclamationTriangleOutlined'
import { CloseIcon } from '../close-button/index.ts'
import { Icon } from '../icon/index.ts'
import { cx } from '../utils/cx.ts'

/**
 * Sonner — an opinionated toast system on Base UI Toast, skinned with quanta
 * tokens. The imperative `toast()` API (`toast.success(...)`, `toast.promise(...)`)
 * mirrors the `sonner` library: it drives a module-level toast manager, so toasts
 * can be fired from anywhere (event handlers, effects, non-React code). Mount one
 * `<Toaster />` near the app root; Base UI owns timing, stacking, swipe-to-dismiss,
 * focus and a11y — quanta only paints.
 *
 *   import { Toaster, toast } from '@higgsfield/quanta/sonner'
 *   <Toaster position="bottom-right" />
 *   toast.success('Saved', { description: 'Your changes are live.' })
 */

export type SonnerVariant = 'default' | 'success' | 'error' | 'warning' | 'info' | 'loading'
export type SonnerPosition =
  | 'top-left' | 'top-center' | 'top-right'
  | 'bottom-left' | 'bottom-center' | 'bottom-right'

/** Per-toast payload carried on the Base UI toast `data` bag. */
interface SonnerData {
  variant?: SonnerVariant
  /** A custom leading node (overrides the variant icon). */
  icon?: ReactNode
  /** A custom action node (e.g. a quanta Button) — overrides the simple action button. */
  action?: ReactNode
}

/** The simple built-in action button config (sonner-shaped). */
export interface SonnerActionConfig {
  label: string
  onClick?: () => void
}

/** Distinguish the `{ label, onClick }` config from an arbitrary ReactNode action. */
function isActionConfig(a: ReactNode | SonnerActionConfig): a is SonnerActionConfig {
  return typeof a === 'object' && a !== null && !isValidElement(a) && 'label' in a
}

const VARIANT_CLASS = {
  default: 'q-sonner-default',
  success: 'q-sonner-success',
  error: 'q-sonner-error',
  warning: 'q-sonner-warning',
  info: 'q-sonner-info',
  loading: 'q-sonner-loading',
} satisfies Record<SonnerVariant, string>

type SwipeDirection = 'up' | 'down' | 'left' | 'right'

/** bottom-* swipes down, top-* swipes up; all dismiss horizontally too. */
const SWIPE = {
  'top-left': ['up', 'left'],
  'top-center': ['up'],
  'top-right': ['up', 'right'],
  'bottom-left': ['down', 'left'],
```

```
  'bottom-center': ['down'],
  'bottom-right': ['down', 'right'],
} satisfies Record<SonnerPosition, SwipeDirection[]>

/* ■■ The module-level manager — the imperative API talks to this. ■■■■■■■■■■■■■■ */
export const sonnerManager = Primitive.createToastManager<SonnerData>()

interface ToastOptions {
  description?: ReactNode
  /** Auto-dismiss delay in ms (0 keeps it until dismissed). */
  duration?: number
  icon?: ReactNode
  /**
   * A trailing action. Either the simple `{ label, onClick }` button, or any
   * `ReactNode` (e.g. a quanta `Button`, a link, or several buttons).
   */
  action?: ReactNode | SonnerActionConfig
  id?: string
}

function emit(variant: SonnerVariant, title: ReactNode, options: ToastOptions = {}): string {
  const { description, duration, icon, action, id } = options
  // The config object → Base UI's built-in Action button (back-compat); any
  // other node → rendered verbatim in the action slot.
  const cfg = action != null && isActionConfig(action) ? action : undefined
  const customAction = action != null && !isActionConfig(action) ? action : undefined
  return sonnerManager.add({
    title,
    description,
    type: variant,
    timeout: duration,
    id,
    data: { variant, icon, action: customAction },
    actionProps: cfg
      ? { children: cfg.label, onClick: cfg.onClick }
      : undefined,
  })
}

/** Imperative toast API (sonner-shaped). */
export const toast = Object.assign(
  (title: ReactNode, options?: ToastOptions) => emit('default', title, options),
  {
    success: (title: ReactNode, options?: ToastOptions) => emit('success', title, options),
    error: (title: ReactNode, options?: ToastOptions) => emit('error', title, options),
    warning: (title: ReactNode, options?: ToastOptions) => emit('warning', title, options),
    info: (title: ReactNode, options?: ToastOptions) => emit('info', title, options),
    loading: (title: ReactNode, options?: ToastOptions) => emit('loading', title, { duration: 0, ...options }),
    message: (title: ReactNode, options?: ToastOptions) => emit('default', title, options),
    dismiss: (id?: string) => sonnerManager.close(id),
    /** Pending → resolved/rejected, sonner-style. */
    promise: <T,>(
      promise: Promise<T>,
      msgs: { loading: ReactNode, success: ReactNode | ((v: T) => ReactNode), error: ReactNode | ((e: unknown) => ReactNode) },
    ) => sonnerManager.promise(promise, {
      loading: msgs.loading as string,
      success: msgs.success as never,
      error: msgs.error as never,
    }),
  },
)

function Glyph({ variant }: { variant: SonnerVariant }) {
  // The status glyphs come from @higgsfield-ai/icons; <Icon size="md"> = 20px,
  // matching the previous `size-5`. Decorative — the toast title carries meaning.
  switch (variant) {
    case 'success':
      return <Icon as={SuccessGlyph} size="md" />
    case 'error':
      return <Icon as={ErrorGlyph} size="md" />
```

```
    case 'warning':
      return <Icon as={WarningGlyph} size="md" />
    case 'info':
      return <Icon as={InfoGlyph} size="md" />
    case 'loading':
      // Hand-rolled spinner (not a glyph from the icon package): keep the inline
      // SVG so its custom q-sonner-spinner animation + currentColor strokes stay.
      return <svg viewBox="0 0 20 20" fill="none" aria-hidden className="size-5 q-sonner-spinner"><circle cx="10" cy="10" r="8" stroke="currentColor" strokeOpacity="0.25" strokeWidth="1.5" /><path d="
    default:
      return null
  }
}

/* ■■ The rendered toast + the viewport ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
function SonnerToast({ toast: t, swipe }: { toast: Primitive.Root.ToastObject<SonnerData>, swipe: SwipeDirection[] }) {
  const variant = (t.data?.variant ?? t.type ?? 'default') as SonnerVariant
  const icon = t.data?.icon ?? <Glyph variant={variant} />
  const actionNode = t.data?.action
  return (
    <Primitive.Root toast={t} swipeDirection={swipe} className={cx('q-sonner', VARIANT_CLASS[variant])}>
      {icon ? <span className="q-sonner-icon">{icon}</span> : null}
      <div className="q-sonner-text">
        <Primitive.Title className="q-sonner-title" />
        <Primitive.Description className="q-sonner-description" />
      </div>
      {actionNode != null
        ? <div className="q-sonner-action-slot">{actionNode}</div>
        : t.actionProps ? <Primitive.Action className="q-sonner-action" /> : null}
      <Primitive.Close className="q-sonner-close" aria-label="Dismiss"><Icon as={CloseIcon} size="sm" /></Primitive.Close>
    </Primitive.Root>
  )
}

function ToastList({ swipe }: { swipe: SwipeDirection[] }) {
  const { toasts } = Primitive.useToastManager()
  return toasts.map(t => <SonnerToast key={t.id} toast={t as Primitive.Root.ToastObject<SonnerData>} swipe={swipe} />)
}

export interface ToasterProps extends Omit<ComponentProps<typeof Primitive.Viewport>, 'children' | 'className'> {
  position?: SonnerPosition
  /** Max simultaneously-visible toasts; older ones collapse behind / queue. Default 3. */
  limit?: number
  /** Default auto-dismiss in ms. Default 5000. */
  duration?: number
  /**
   * Expand the stack by default. When false (default) toasts collapse into a
   * peeking pile and expand on hover / focus (Sonner behaviour).
   */
  expand?: boolean
  /** Gap between toasts when expanded, in px. Default 14. */
  gap?: number
  className?: string
}

/**
 * Mount once near the app root. Toasts collapse into a glassy stack and expand
 * on hover/focus (or always, with `expand`). `limit` caps the visible pile;
 * Base UI owns the stack offsets, swipe-to-dismiss, timing, focus and a11y.
 */
export function Toaster({ position = 'bottom-right', limit = 3, duration = 5000, expand = false, gap, className, style, ...props }: ToasterProps) {
  const viewportStyle = gap == null
    ? style
    : { ...style, ...({ '--q-sonner-gap': `${gap}px` } as Record<string, string>) } as CSSProperties
  return (
    <Primitive.Provider toastManager={sonnerManager} limit={limit} timeout={duration}>
      <Primitive.Portal>
        <Primitive.Viewport
          data-position={position}
          data-expand={expand ? '' : undefined}
          style={viewportStyle}
```

```
          className={cx('q-sonner-viewport', className)}
          {...props}
        >
          <ToastList swipe={SWIPE[position]} />
        </Primitive.Viewport>
      </Primitive.Portal>
    </Primitive.Provider>
  )
}
```

app/packages/quanta/src/components/switch/index.ts

```
export { Switch, SwitchLabel } from './switch.tsx'
export type {
  SwitchLabelDirection,
  SwitchLabelProps,
  SwitchLabelSize,
  SwitchProps,
  SwitchSize,
} from './switch.tsx'
```

app/packages/quanta/src/components/switch/switch.css

```
/**
 * Switch presentation utilities (q-namespaced), matched to the Figma Switch
 * component at nodes 285:267-281.
 */

@source "./switch.tsx";

@utility q-switch {
  --q-switch-track-bg: var(--hf-color-button-tertiary);
  --q-switch-track-bg-checked: var(--q-tint, var(--hf-color-button-brand));
  --q-switch-thumb-bg: var(--hf-color-button-primary);
  --q-switch-thumb-border: var(--hf-color-transparent-dark-10);
  --q-switch-thumb-shadow-soft: var(--hf-shadow-thumb-soft);
  --q-switch-thumb-shadow-strong: var(--hf-shadow-thumb-strong);
  --q-switch-focus-ring: var(--hf-color-transparent-lime-20);
  --q-switch-padding: var(--hf-space-050);
  --q-switch-track-width: var(--hf-space-700);
  --q-switch-track-height: var(--hf-space-400);
  --q-switch-thumb-size: var(--hf-space-300);
  --q-switch-thumb-radius: var(--hf-space-200);
  --q-switch-thumb-translate: calc(var(--q-switch-track-width) - var(--q-switch-thumb-size) - var(--q-switch-padding) - var(--q-switch-padding));
  /* Motion: `ease` (swift) decelerates without overshoot so the thumb never
   * clips at the track edge; `press-scale` stretches the thumb inward on
   * pointer-down. */
  --q-switch-duration-track: var(--hf-duration-normal);
  --q-switch-duration-thumb: var(--hf-duration-260);
  --q-switch-ease: var(--hf-ease-swift);
  --q-switch-press-scale: 1.14; /* no token: unitless press-stretch scale factor */

  align-items: center;
  background-color: var(--q-switch-track-bg);
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--hf-space-300);
  box-sizing: border-box;
  cursor: pointer;
  display: inline-flex;
  flex-shrink: 0;
  height: var(--q-switch-track-height);
  outline: none;
  overflow: clip;
  padding: var(--hf-space-0);
  position: relative;
  transition-property: background-color, box-shadow, opacity;
  transition-duration: var(--q-switch-duration-track);
  transition-timing-function: var(--q-switch-ease);
  width: var(--q-switch-track-width);
}

@utility q-switch-thumb {
  background-color: var(--q-switch-thumb-bg);
  border: var(--hf-border-width-none) solid transparent;
  border-radius: var(--q-switch-thumb-radius);
  box-shadow: var(--q-switch-thumb-shadow-soft);
  box-sizing: border-box;
  display: block;
  height: var(--q-switch-thumb-size);
  left: var(--q-switch-padding);
  position: absolute;
  top: var(--q-switch-padding);
  /* Unchecked thumb anchors at its left edge so a press-stretch grows
   * rightward, toward the track centre (never past the left edge). */
  transform-origin: left center;
  transition-property: border-color, box-shadow, transform;
  transition-duration: var(--q-switch-duration-thumb);
  transition-timing-function: var(--q-switch-ease);
  width: var(--q-switch-thumb-size);
  will-change: transform;
}
```

```
@utility q-switch-small {
  --q-switch-track-width: var(--hf-space-700);
  --q-switch-track-height: var(--hf-space-400);
  --q-switch-thumb-size: var(--hf-space-300);
  --q-switch-thumb-radius: var(--hf-space-200);
}

@utility q-switch-medium {
  --q-switch-track-width: var(--hf-space-1000);
  --q-switch-track-height: var(--hf-space-500);
  --q-switch-thumb-size: var(--hf-space-400);
  --q-switch-thumb-radius: var(--hf-space-200);
}

@utility q-switch-default {
  --q-switch-track-width: calc(var(--hf-space-1000) + var(--hf-space-100));
  --q-switch-track-height: var(--hf-space-600);
  --q-switch-thumb-size: var(--hf-space-500);
  --q-switch-thumb-radius: var(--hf-space-300);
}

/* Track colour eases between unchecked grey and checked brand (the track's
 * own `transition: background-color` handles the fade). */
.q-switch[data-checked] {
  background-color: var(--q-switch-track-bg-checked);
}

/* Checked thumb anchors at its right edge so a press-stretch grows leftward,
 * toward the track centre (never past the right edge). */
.q-switch[data-checked] .q-switch-thumb {
  transform: translateX(var(--q-switch-thumb-translate));
  transform-origin: right center;
}

.q-switch[data-checked]:not([data-disabled]) .q-switch-thumb {
  border-color: var(--q-switch-thumb-border);
  border-width: var(--hf-border-width-thin);
  box-shadow:
    var(--q-switch-thumb-shadow-strong),
    var(--q-switch-thumb-shadow-soft);
}

/* Tactile press: the thumb stretches toward the track centre on pointer-down,
 * then settles back when released or as it slides to the other end. */
.q-switch:not([data-disabled]):active .q-switch-thumb {
  transform: scaleX(var(--q-switch-press-scale));
}

.q-switch[data-checked]:not([data-disabled]):active .q-switch-thumb {
  transform: translateX(var(--q-switch-thumb-translate)) scaleX(var(--q-switch-press-scale));
}

.q-switch:focus-visible {
  box-shadow:
    0 0 0 var(--hf-border-width-thick) var(--hf-color-background-primary),
    0 0 0 var(--hf-space-100) var(--q-switch-focus-ring);
}

.q-switch[data-disabled] {
  cursor: not-allowed;
  opacity: var(--hf-opacity-disabled);
}

.q-switch[data-disabled] .q-switch-thumb {
  box-shadow: var(--q-switch-thumb-shadow-soft);
}

/* ■■ Labelled switch (SwitchLabel) — mirrors q-checkbox-label* ■■■■■■■■■■■■■■■■■ */
@utility q-switch-label {
  align-items: flex-start;
```

```
  color: var(--hf-color-text-primary);
  cursor: pointer;
  display: flex;
  gap: var(--hf-space-300);
  min-width: var(--hf-space-0);
}

@utility q-switch-label-left {
  justify-content: flex-start;
}

@utility q-switch-label-right {
  justify-content: space-between;
}

@utility q-switch-label-text {
  display: flex;
  flex: 1 1 var(--hf-space-0);
  flex-direction: column;
  gap: var(--hf-space-100);
  min-width: var(--hf-space-0);
  overflow-wrap: anywhere;
}

/* Title, description, and label-size classes (q-switch-label-title /
 * q-switch-label-description / q-switch-label-sm / q-switch-label-md) are now
 * plain structural/test marker classes carried on the elements — their
 * composite typography (label-*-medium / label-sm-regular) and text colour are
 * owned by the <Typography> the SwitchLabel renders, which picks the variant per
 * size via LABEL_TITLE_VARIANT. No CSS rules are needed for them here. */

@media (prefers-reduced-motion: reduce) {
  .q-switch,
  .q-switch-thumb {
    transition-duration: 1ms; /* no token: reduced-motion near-instant; --hf-duration-instant is 0ms */
  }

  /* Skip the press-stretch entirely for motion-sensitive users. */
  .q-switch:not([data-disabled]):active .q-switch-thumb {
    transform: none;
  }

  .q-switch[data-checked]:not([data-disabled]):active .q-switch-thumb {
    transform: translateX(var(--q-switch-thumb-translate));
  }
}
```

app/packages/quanta/src/components/switch/switch.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { describe, expect, it } from 'vitest'
import { Switch, SwitchLabel } from './index.ts'

describe('<Switch>', () => {
  it('renders the small switch by default', () => {
    render(<Switch aria-label="Notifications" />)

    expect(screen.getByRole('switch', { name: 'Notifications' })).toHaveClass(
      'q-switch',
      'q-switch-small',
    )
  })

  it('supports the default Figma size', () => {
    render(<Switch aria-label="Notifications" size="default" />)

    expect(screen.getByRole('switch', { name: 'Notifications' })).toHaveClass('q-switch-default')
  })

  it('supports the medium Figma size', () => {
    render(<Switch aria-label="Notifications" size="medium" />)

    expect(screen.getByRole('switch', { name: 'Notifications' })).toHaveClass('q-switch-medium')
  })

  it('reflects checked and disabled state attributes', () => {
    render(<Switch aria-label="Notifications" checked disabled />)

    const control = screen.getByRole('switch', { name: 'Notifications' })
    expect(control).toBeChecked()
    expect(control).toHaveAttribute('aria-disabled', 'true')
    expect(control).toHaveAttribute('data-checked')
    expect(control).toHaveAttribute('data-disabled')
  })

  it('toggles when clicked', async () => {
    const user = userEvent.setup()
    render(<Switch aria-label="Notifications" />)

    const control = screen.getByRole('switch', { name: 'Notifications' })
    expect(control).not.toBeChecked()

    await user.click(control)

    expect(control).toBeChecked()
  })

  it('renders a bare control with no label wrapper (back-compat)', () => {
    const { container } = render(<Switch aria-label="Bare" />)
    expect(container.querySelector('.q-switch-label')).not.toBeInTheDocument()
    expect(screen.getByRole('switch', { name: 'Bare' })).toHaveClass('q-switch', 'q-switch-small')
  })
})

describe('<SwitchLabel>', () => {
  it('renders the label, description and a Switch with Typography composites', () => {
    render(<SwitchLabel label="Wi-Fi" description="Connect automatically" switchProps={{ checked: true }} />)
    // Title is rendered via <Typography> (label-sm-medium for the default sm
    // label size) but still carries the structural marker class.
    const title = screen.getByText('Wi-Fi')
    expect(title).toHaveClass('q-switch-label-title', 'text-q-label-sm-medium', 'text-q-text-primary')
    const desc = screen.getByText('Connect automatically')
    expect(desc).toHaveClass('q-switch-label-description', 'text-q-label-sm-regular', 'text-q-text-tertiary')
    const control = screen.getByRole('switch')
    expect(control).toHaveClass('q-switch')
    expect(control).toBeChecked()
```

```
  })

  it('supports right-aligned switch and medium label typography', () => {
    render(<SwitchLabel direction="right" size="md" label="Email digest" />)
    const title = screen.getByText('Email digest')
    // md label size resolves to the label-md-medium Typography composite.
    expect(title).toHaveClass('q-switch-label-title', 'text-q-label-md-medium')
    expect(title.closest('.q-switch-label')).toHaveClass(
      'q-switch-label-right',
      'q-switch-label-md',
    )
  })

  it('lets children override the title and forwards color/size to the Switch', () => {
    render(
      <SwitchLabel color="success" switchSize="default" switchProps={{ checked: true }}>
        <strong data-testid="rich">Custom</strong>
      </SwitchLabel>,
    )
    expect(screen.getByTestId('rich')).toBeInTheDocument()
    expect(screen.getByRole('switch')).toHaveClass('q-switch-default')
  })
})
```

app/packages/quanta/src/components/switch/switch.tsx

```
'use client'

import type { ComponentProps, CSSProperties, ReactNode } from 'react'
import { Switch as Primitive } from '@base-ui/react/switch'
import { Typography, type TypographyVariant } from '../typography/index.ts'
import { cx } from '../utils/cx.ts'
import { type SlotColor, slotStyle } from '../utils/slot.ts'

/**
 * Switch — a Base UI primitive skinned with quanta tokens.
 *
 * Base UI owns behavior + a11y (role="switch", hidden <input>, keyboard, focus,
 * form integration); quanta paints its state data-attributes (data-checked /
 * data-unchecked / data-disabled).
 */

export type SwitchSize = 'small' | 'medium' | 'default'

export type SwitchProps = Omit<ComponentProps<typeof Primitive.Root>, 'size'> & {
  /** Slot color — sets `--q-tint`; every surface derives from it. Default 'brand'. */
  color?: SlotColor
  size?: SwitchSize
}

const SIZE_CLASS = {
  small: 'q-switch-small',
  medium: 'q-switch-medium',
  default: 'q-switch-default',
} satisfies Record<SwitchSize, string>

export function Switch({ className, color = 'brand', size = 'small', style, ...props }: SwitchProps) {
  return (
    <Primitive.Root
      style={{ ...slotStyle(color), ...style } as CSSProperties}
      // className can be a string or a `(state) => string` — resolve the
      // caller's value against state before merging with our base classes.
      className={state => cx(
        'q-switch',
        SIZE_CLASS[size],
        typeof className === 'function' ? className(state) : className,
      )}
      {...props}
    >
      <Primitive.Thumb className="q-switch-thumb" />
    </Primitive.Root>
  )
}

/* ■■ Labelled switch — the trio composite (mirrors CheckboxLabel / RadioLabel) ■■ */

export type SwitchLabelDirection = 'left' | 'right'
export type SwitchLabelSize = 'sm' | 'md'

const LABEL_DIRECTION_CLASS = {
  left: 'q-switch-label-left',
  right: 'q-switch-label-right',
} satisfies Record<SwitchLabelDirection, string>

const LABEL_SIZE_CLASS = {
  sm: 'q-switch-label-sm',
  md: 'q-switch-label-md',
} satisfies Record<SwitchLabelSize, string>

// Title composite per label size — the exact variants the size utilities used to
// apply via descendant selectors (`.q-switch-label-sm .q-switch-label-title`).
const LABEL_TITLE_VARIANT = {
  sm: 'label-sm-medium',
  md: 'label-md-medium',
```

```
} satisfies Record<SwitchLabelSize, TypographyVariant>

export interface SwitchLabelProps extends Omit<ComponentProps<'label'>, 'color'> {
  /** Primary line. Any ReactNode (also overridable via `children`). */
  label?: ReactNode
  /** Secondary line under the title. Any ReactNode. */
  description?: ReactNode
  /** `left` = switch then text (default); `right` = text then switch (settings rows). */
  direction?: SwitchLabelDirection
  /** Label typography scale. Default `sm`. */
  size?: SwitchLabelSize
  /** Slot color forwarded to the Switch. */
  color?: SlotColor
  /** Switch size. Defaults from `size` (`md` → `default`, else `medium`). */
  switchSize?: SwitchSize
  /** Extra props for the underlying Switch (e.g. `checked`, `onCheckedChange`). */
  switchProps?: Omit<SwitchProps, 'color' | 'size'>
}

/**
 * SwitchLabel — a Switch paired with a title + optional description, the same
 * labelled-control composite as `CheckboxLabel` / `RadioLabel`. `label` /
 * `description` take any node; `children` overrides the title. Compose richer
 * titles (e.g. a Badge) by passing nodes to `label`.
 */
export function SwitchLabel({
  label = 'Label',
  description,
  direction = 'left',
  size = 'sm',
  color,
  switchSize = size === 'md' ? 'default' : 'medium',
  switchProps,
  className,
  children,
  ...props
}: SwitchLabelProps) {
  const switchNode = <Switch color={color} size={switchSize} {...switchProps} />
  const textNode = (
    <span className="q-switch-label-text">
      <Typography
        as="span"
        variant={LABEL_TITLE_VARIANT[size]}
        color="primary"
        className="q-switch-label-title"
      >
        {children ?? label}
      </Typography>
      {description
        ? (
            <Typography
              as="span"
              variant="label-sm-regular"
              color="tertiary"
              className="q-switch-label-description"
            >
              {description}
            </Typography>
          )
        : null}
    </span>
  )

  return (
    <label
      className={cx(
        'q-switch-label',
        LABEL_DIRECTION_CLASS[direction],
        LABEL_SIZE_CLASS[size],
        className,
```

```
      )}
      {...props}
    >
      {direction === 'left' ? switchNode : textNode}
      {direction === 'left' ? textNode : switchNode}
    </label>
  )
}
```

app/packages/quanta/src/components/tabs/index.ts

```
export { Tabs } from './tabs.tsx'
export type {
  ListProps,
  PanelProps,
  RootProps,
  TabItem,
  TabProps,
  TabsOptions,
  TabsShape,
  TabsSurface,
  TabsTone,
  TabsVariant,
} from './tabs.tsx'
```

app/packages/quanta/src/components/tabs/tabs.css

```
/**
 * Tabs presentation utilities, matched to the Figma Tabs / Segment /
 * Segmented Control nodes:
 * 1639:4673, 1646:275, 1825:362, 2700:199, 2701:248, 2701:310.
 */

@source "./tabs.tsx";

@utility q-tabs {
  --q-tabs-accent: var(--q-tint, var(--hf-color-brand-primary));
  --q-tabs-focus-ring: var(--hf-color-transparent-lime-20);
  --q-tabs-indicator-bg: var(--hf-color-border-inverse);
  --q-tabs-indicator-fg: var(--hf-color-text-primary);
  --q-tabs-indicator-icon: var(--hf-color-icon-primary);
  --q-tabs-segment-radius: var(--hf-radius-250);
  --q-tabs-selected-radius: var(--hf-radius-200);
  --q-tabs-tab-fg: var(--hf-color-text-tertiary);
  --q-tabs-tab-hover-bg: transparent;
  --q-tabs-tab-hover-fg: var(--hf-color-brand-primary);
  --q-tabs-tab-icon: currentColor;
  --q-tabs-hover-release-delay: var(--q-tabs-transition-duration);
  --q-tabs-hover-release-duration: var(--hf-duration-fast);
  --q-tabs-transition-duration: var(--hf-duration-normal);
  --q-tabs-transition-ease: var(--hf-ease-out);

  display: block;
  width: 100%;
}

@utility q-tabs-list {
  align-items: center;
  box-sizing: border-box;
  display: inline-flex;
  isolation: isolate;
  max-width: 100%;
  position: relative;
}

@utility q-tabs-tab {
  align-items: center;
  appearance: none;
  background-color: transparent;
  border: var(--hf-border-width-none) solid transparent;
  box-sizing: border-box;
  color: var(--q-tabs-tab-fg);
  cursor: pointer;
  display: inline-flex;
  flex-shrink: 0;
  justify-content: center;
  outline: none;
  padding: var(--hf-space-250) var(--hf-space-400);
  position: relative;
  transition-duration: var(--q-tabs-transition-duration);
  transition-property: background-color, border-color, box-shadow, color, opacity;
  transition-timing-function: var(--q-tabs-transition-ease);
  user-select: none;
  white-space: nowrap;
  z-index: 1;

  & svg {
    color: var(--q-tabs-tab-icon);
    flex-shrink: 0;
    height: var(--q-tabs-icon-size, var(--hf-icon-sm));
    width: var(--q-tabs-icon-size, var(--hf-icon-sm));
  }
}

@utility q-tabs-tab-content {
```

```
  align-items: center;
  display: inline-flex;
  gap: var(--q-tabs-content-gap, var(--hf-space-150));
  justify-content: center;
  min-width: var(--hf-space-0);
  overflow: hidden;
  padding: var(--hf-space-0) var(--q-tabs-label-padding-x, var(--hf-space-150));
  position: relative;
  text-overflow: ellipsis;
  z-index: 1;
}

@utility q-tabs-tab-icon {
  align-items: center;
  display: inline-flex;
  flex-shrink: 0;
  justify-content: center;
}

@utility q-tabs-tab-label {
  min-width: var(--hf-space-0);
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

/* Width-locking label: a zero-height ::after carries a semibold copy of the
 * text, forcing the box to the widest (bold) footprint. The visible text flows
 * above it and can switch weight on select without changing width — so the
 * indicator and neighbouring tabs never wobble. */
@utility q-tabs-tab-text {
  display: inline-flex;
  flex-direction: column;
  white-space: nowrap;
}

.q-tabs-tab-text::after {
  content: attr(data-text);
  font-weight: var(--hf-type-weight-medium);
  height: 0;
  overflow: hidden;
  visibility: hidden;
  letter-spacing: var(--label-xs-medium-letter-spacing);
}

@utility q-tabs-tab-secondary {
  color: var(--q-tabs-tab-secondary-fg, var(--hf-color-text-tertiary));
  min-width: var(--hf-space-0);
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

@utility q-tabs-tab-icon-only {
  --q-tabs-content-gap: var(--hf-space-0);
  --q-tabs-label-padding-x: var(--hf-space-0);
  --q-tabs-icon-size: var(--hf-icon-md); /* icon-only glyph is 20px (labels stay 16px) */
}

@utility q-tabs-indicator {
  bottom: auto;
  box-sizing: border-box;
  left: var(--active-tab-left);
  pointer-events: none;
  position: absolute;
  top: var(--active-tab-top);
  transition-duration: var(--q-tabs-transition-duration);
  transition-property: background, background-color, border-color, border-radius, box-shadow, height, left, top, width;
  transition-timing-function: var(--q-tabs-transition-ease);
  will-change: height, left, top, width;
```

```
  z-index: 0;
}

@utility q-tabs-panel {
  animation: q-tabs-panel-in var(--q-tabs-transition-duration) var(--q-tabs-transition-ease);
  color: var(--hf-color-text-primary);
  outline: none;
  padding-top: var(--hf-space-400);
}

@utility q-tabs-list-fill {
  width: 100%;

  & .q-tabs-tab {
    flex: 1 1 var(--hf-space-0);
    /* Allow shrinking below intrinsic label width so every tab gets an equal
     * share (nowrap labels otherwise floor each tab at its text width). */
    min-width: var(--hf-space-0);
  }
}

/* When the list is full-width, expand the root too — segmented/soft default to
 * inline-block/width:auto, so without this the 100% list has nothing to fill. */
.q-tabs:has(.q-tabs-list-fill) {
  display: block;
  width: 100%;
}

.q-tabs-underline {
  --q-tabs-indicator-bg: var(--hf-color-border-inverse);
  --q-tabs-tab-fg: var(--hf-color-text-tertiary);
  --q-tabs-tab-hover-bg: transparent;
  --q-tabs-tab-hover-fg: var(--hf-color-brand-primary);
}

.q-tabs-underline.q-tabs-tone-accent {
  --q-tabs-indicator-bg: var(--q-tabs-accent);
  --q-tabs-indicator-fg: var(--q-tabs-accent);
}

.q-tabs-underline .q-tabs-list {
  border-bottom: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  gap: var(--hf-space-200);
  height: calc(var(--hf-space-1000) + var(--hf-space-300));
  overflow: clip;
  padding-inline: var(--hf-space-0);
}

.q-tabs-underline .q-tabs-tab {
  min-height: calc(var(--hf-space-1000) + var(--hf-space-300));
  padding: 0;
  @apply text-q-body-sm-medium;
}

/* Figma underline tab keeps body/sm/medium in every state (inactive/active/
 * active-accent); only colour + the 2px bottom border change on select. */
.q-tabs-underline .q-tabs-tab[data-active] {
  color: var(--q-tabs-indicator-fg);
}

/* Width-lock ghost matches the medium weight (no weight change on select). */
.q-tabs-underline .q-tabs-tab-text::after {
  font-weight: var(--hf-type-weight-medium);
}

.q-tabs-underline .q-tabs-tab:not([data-active]):not([data-disabled]):hover {
  background-color: var(--q-tabs-tab-hover-bg);
  color: var(--q-tabs-tab-hover-fg);
}
```

```
.q-tabs-underline .q-tabs-indicator {
  background-color: var(--q-tabs-indicator-bg);
  bottom: var(--hf-space-0);
  height: var(--hf-border-width-thick);
  left: var(--active-tab-left);
  top: auto;
  width: var(--active-tab-width);
  z-index: 2;
}

.q-tabs-pill {
  --q-tabs-indicator-bg: var(--hf-color-button-primary);
  --q-tabs-indicator-fg: var(--hf-color-text-inverse);
  --q-tabs-tab-fg: var(--hf-color-text-secondary);
  --q-tabs-tab-hover-bg: transparent;
  --q-tabs-tab-hover-fg: var(--hf-color-text-primary);
}

.q-tabs-pill .q-tabs-list {
  border-radius: var(--hf-space-2400);
  gap: var(--hf-space-050);
  overflow: clip;
  padding: var(--hf-space-100);
}

.q-tabs-pill .q-tabs-tab {
  border-radius: var(--hf-space-2400);
  min-height: var(--hf-space-800);
  padding: var(--hf-space-100) var(--hf-space-200);
  @apply text-q-label-xs-medium;
}

.q-tabs-pill .q-tabs-tab-content {
  --q-tabs-label-padding-x: var(--hf-space-200);
}

.q-tabs-pill .q-tabs-tab[data-active] {
  color: var(--q-tabs-indicator-fg);
  @apply text-q-label-xs-medium;
}

.q-tabs-pill .q-tabs-tab-text::after {
  font-weight: var(--hf-type-weight-medium);
}

.q-tabs-pill .q-tabs-tab:not([data-active]):not([data-disabled]):hover {
  color: var(--q-tabs-tab-hover-fg);
}

.q-tabs-pill .q-tabs-indicator {
  background-color: var(--q-tabs-indicator-bg);
  border-radius: var(--hf-space-2400);
  height: var(--active-tab-height);
  width: var(--active-tab-width);
}

.q-tabs-segmented {
  --q-tabs-content-gap: var(--hf-space-0);
  --q-tabs-glass-blur: var(--hf-space-300);
  --q-tabs-glass-selected-bg: var(--hf-gradient-glass-sheen);
  --q-tabs-icon-size: var(--hf-icon-sm);
  --q-tabs-indicator-bg: var(--hf-color-background-glass);
  --q-tabs-indicator-border: var(--hf-color-border-subtle);
  --q-tabs-indicator-fg: var(--hf-color-text-primary);
  --q-tabs-indicator-icon: var(--hf-color-icon-primary);
  --q-tabs-label-padding-x: var(--hf-space-100);
  --q-tabs-segment-radius: var(--hf-radius-200);
  --q-tabs-selected-radius: var(--hf-radius-200);
  --q-tabs-tab-fg: var(--hf-color-text-on-overlay-secondary);
  --q-tabs-tab-hover-bg: var(--hf-color-overlay-hover);
```

```
  --q-tabs-tab-hover-fg: var(--hf-color-text-primary);
  --q-tabs-tab-icon: var(--hf-color-icon-on-overlay-secondary);
  --q-tabs-track-bg: var(--hf-color-overlay-hover);
  --q-tabs-track-radius: var(--hf-radius-300);

  display: inline-block;
  width: auto;
}

.q-tabs-segmented.q-tabs-shape-pill {
  --q-tabs-segment-radius: var(--hf-radius-full);
  --q-tabs-selected-radius: var(--hf-radius-full);
  --q-tabs-track-radius: var(--hf-radius-full);
}

.q-tabs-segmented.q-tabs-shape-icon {
  --q-tabs-icon-size: var(--hf-icon-sm);
  --q-tabs-label-padding-x: var(--hf-space-0);
}

.q-tabs-segmented.q-tabs-tone-default {
  --q-tabs-indicator-bg: var(--hf-color-overlay-hover);
  --q-tabs-indicator-border: var(--hf-color-border-subtle);
  --q-tabs-indicator-fg: var(--hf-color-text-primary);
  --q-tabs-indicator-icon: var(--hf-color-icon-primary);
}

.q-tabs-segmented.q-tabs-tone-solid {
  --q-tabs-indicator-bg: var(--hf-color-button-primary);
  --q-tabs-indicator-fg: var(--hf-color-text-inverse);
  --q-tabs-indicator-icon: var(--hf-color-icon-inverse);
}

.q-tabs-segmented.q-tabs-tone-brand-soft {
  --q-tabs-indicator-bg: var(--hf-color-transparent-lime-15);
  --q-tabs-indicator-fg: var(--hf-color-brand-primary);
  --q-tabs-indicator-icon: var(--hf-color-icon-accent);
}

.q-tabs-segmented.q-tabs-tone-brand {
  --q-tabs-indicator-bg: var(--hf-color-brand-primary);
  --q-tabs-indicator-fg: var(--hf-color-text-inverse);
  --q-tabs-indicator-icon: var(--hf-color-icon-inverse);
}

.q-tabs-segmented .q-tabs-list {
  align-items: center;
  background-color: var(--q-tabs-track-bg);
  border: var(--hf-border-width-thin) solid var(--hf-color-border-default);
  border-radius: var(--q-tabs-track-radius);
  height: var(--hf-space-1000);
  overflow: clip;
  padding: var(--hf-space-100);
}

.q-tabs-segmented.q-tabs-surface-glass .q-tabs-list {
  backdrop-filter: blur(var(--q-tabs-glass-blur));
  /* no token: -0.5px spread and 3px blur have no exact space token; this drop+dark-12 color combo matches no shadow composite role (modal uses dark-15) so kept as a raw multi-layer glass shadow with
  box-shadow:
    0 var(--hf-space-050) var(--hf-space-100) -0.5px var(--hf-color-transparent-dark-12),
    inset 0 var(--hf-space-050) 3px var(--hf-color-transparent-light-05);
}

.q-tabs-segmented.q-tabs-surface-flat .q-tabs-list {
  box-shadow: none;
}

.q-tabs-segmented .q-tabs-tab {
  border-radius: var(--q-tabs-segment-radius);
  height: var(--hf-space-800);
```

```
  min-height: var(--hf-space-800);
  padding: var(--hf-space-100) var(--hf-space-200);
  @apply text-q-label-xs-medium;
}

.q-tabs-segmented .q-tabs-tab-content {
  --q-tabs-content-gap: var(--hf-space-0);
  --q-tabs-label-padding-x: var(--hf-space-0);
  overflow: visible;
}

.q-tabs-segmented .q-tabs-tab-text {
  padding-inline: var(--hf-space-100);
}

.q-tabs-segmented .q-tabs-tab-text::after {
  font-weight: var(--hf-type-weight-medium);
}

.q-tabs-segmented .q-tabs-tab[data-active] {
  color: var(--q-tabs-indicator-fg);
  @apply text-q-label-xs-medium;
}

.q-tabs-segmented .q-tabs-tab[data-active] svg {
  color: var(--q-tabs-indicator-icon);
}

.q-tabs-segmented .q-tabs-tab:not([data-active]):not([data-disabled]):hover {
  background-color: var(--q-tabs-tab-hover-bg);
  border-radius: var(--q-tabs-selected-radius);
  color: var(--q-tabs-tab-hover-fg);
}

.q-tabs-segmented .q-tabs-tab:not([data-active]):not([data-disabled]):hover svg {
  color: var(--hf-color-icon-primary);
}

.q-tabs-segmented .q-tabs-indicator {
  background-color: var(--q-tabs-indicator-bg);
  border: var(--hf-border-width-thin) solid transparent;
  border-radius: var(--q-tabs-selected-radius);
  height: var(--active-tab-height);
  width: var(--active-tab-width);
}

.q-tabs-segmented.q-tabs-tone-default .q-tabs-indicator {
  border-color: var(--q-tabs-indicator-border);
}

/* Keep the clicked hover fill alive while the selected pill travels to it,
 * then release it once the pill has arrived. */
.q-tabs-segmented .q-tabs-tab[data-active] {
  background-color: transparent;
  transition-delay:
    var(--q-tabs-hover-release-delay),
    var(--hf-duration-instant),
    var(--hf-duration-instant),
    var(--hf-duration-instant),
    var(--hf-duration-instant);
  transition-duration:
    var(--q-tabs-hover-release-duration),
    var(--q-tabs-transition-duration),
    var(--q-tabs-transition-duration),
    var(--q-tabs-transition-duration),
    var(--q-tabs-transition-duration);
  transition-property: background-color, border-color, box-shadow, color, opacity;
}

.q-tabs-segmented.q-tabs-tone-glass .q-tabs-indicator {
```

```
  backdrop-filter: blur(var(--q-tabs-glass-blur));
  background: var(--q-tabs-glass-selected-bg);
  border-color: var(--q-tabs-indicator-border);
  /* no token: 3px blur has no exact space token; this drop+dark-12 color combo matches no shadow composite role so kept as a raw multi-layer glass shadow with tokenized color */
  box-shadow:
    0 var(--hf-space-050) var(--hf-space-050) var(--hf-space-0) var(--hf-color-transparent-dark-12),
    inset 0 var(--hf-space-050) 3px var(--hf-color-transparent-light-05);
}

.q-tabs-segmented.q-tabs-shape-icon .q-tabs-list {
  height: var(--hf-space-1000);
}

.q-tabs-segmented.q-tabs-shape-icon .q-tabs-tab {
  height: var(--hf-space-800);
  min-height: var(--hf-space-800);
  min-width: var(--hf-space-800);
  padding: var(--hf-space-0);
}

.q-tabs-segmented.q-tabs-shape-icon .q-tabs-tab-content {
  --q-tabs-content-gap: var(--hf-space-0);
  --q-tabs-label-padding-x: var(--hf-space-0);
}

.q-tabs-segmented.q-tabs-shape-icon.q-tabs-tone-glass {
  --q-tabs-glass-selected-bg: var(--hf-gradient-glass-sheen-alt);
}

/* ■■ Media soft tabs (Figma node 1639:4673) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * Track: transparent/dark/05 or overlay/scrim, radius/300, p4, gap4.
 * Tab: px12/py8, gap6, radius/200, caption/sm/medium, 24px passed icons.
 * Selected: transparent/dark/05 + border/subtle, text/icon primary. */
.q-tabs-soft {
  --q-tabs-selected-bg: var(--hf-color-overlay-hover);
  --q-tabs-selected-radius: var(--hf-radius-200);
  --q-tabs-tab-fg: var(--hf-color-text-secondary);
  --q-tabs-tab-icon: var(--hf-color-icon-secondary);
  --q-tabs-tab-hover-bg: var(--hf-color-overlay-hover);
  --q-tabs-tab-hover-fg: var(--hf-color-text-primary);
  --q-tabs-tab-selected-fg: var(--hf-color-text-primary);
  --q-tabs-tab-selected-icon: var(--hf-color-icon-primary);
  --q-tabs-icon-size: var(--hf-icon-sm);

  display: inline-block;
  width: auto;
}

.q-tabs-soft .q-tabs-list {
  background-color: var(--hf-color-overlay-hover);
  border-radius: var(--hf-radius-300);
  gap: var(--hf-space-100);
  padding: var(--hf-space-100);
}

.q-tabs-soft.q-tabs-soft-scrim .q-tabs-list {
  background-color: var(--hf-color-overlay-scrim);
}

/* The sliding selection pill (white-5% surface that animates between tabs). */
.q-tabs-soft .q-tabs-indicator {
  background-color: var(--q-tabs-selected-bg);
  border: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  border-radius: var(--q-tabs-selected-radius);
  height: var(--active-tab-height);
  width: var(--active-tab-width);
}

.q-tabs-soft .q-tabs-tab {
  border-radius: var(--hf-radius-200);
```

```
  gap: var(--hf-space-150);
  padding: var(--hf-space-200) var(--hf-space-300);
  @apply text-q-caption-sm-medium;
}

.q-tabs-soft .q-tabs-tab-content {
  --q-tabs-content-gap: var(--hf-space-150);
  --q-tabs-label-padding-x: var(--hf-space-0);
  overflow: visible;
}

.q-tabs-soft .q-tabs-tab[data-active] {
  background-color: transparent;
  color: var(--q-tabs-tab-selected-fg);
  transition-delay:
    var(--q-tabs-hover-release-delay),
    var(--hf-duration-instant),
    var(--hf-duration-instant),
    var(--hf-duration-instant),
    var(--hf-duration-instant);
  transition-duration:
    var(--q-tabs-hover-release-duration),
    var(--q-tabs-transition-duration),
    var(--q-tabs-transition-duration),
    var(--q-tabs-transition-duration),
    var(--q-tabs-transition-duration);
  transition-property: background-color, border-color, box-shadow, color, opacity;
  @apply text-q-caption-sm-medium;
}

.q-tabs-soft .q-tabs-tab-text::after {
  font-weight: var(--hf-type-weight-medium);
}

.q-tabs-soft .q-tabs-tab[data-active] svg {
  color: var(--q-tabs-tab-selected-icon);
}

.q-tabs-soft .q-tabs-tab:not([data-active]):not([data-disabled]):hover {
  background-color: var(--q-tabs-tab-hover-bg);
  color: var(--q-tabs-tab-hover-fg);
}

.q-tabs-soft .q-tabs-tab:not([data-active]):not([data-disabled]):hover svg {
  color: var(--hf-color-icon-primary);
}

.q-tabs-tab:focus-visible {
  box-shadow: 0 0 0 var(--hf-border-width-thick) var(--q-tabs-focus-ring);
}

.q-tabs-tab[data-disabled] {
  cursor: not-allowed;
  opacity: var(--hf-opacity-muted);
}

/* Icon-only tabs are a 1:1 square; the corner radius comes from the shape, so
 * only the pill shape (full radius) reads as a complete circle — the rounded
 * shape stays a rounded square. The icon itself bumps to 20px (in the utility
 * above). The selected pill follows the same per-shape radius automatically. */
.q-tabs-segmented .q-tabs-tab-icon-only {
  padding: var(--hf-space-150);
}

.q-tabs-tab-icon-only .q-tabs-tab-content {
  padding: 0;
}

.q-tabs-soft .q-tabs-tab-icon-only {
  padding: var(--hf-space-150);
```

```
}

@keyframes q-tabs-panel-in {
  from {
    opacity: 0;
    transform: translateY(var(--hf-space-050));
  }

  to {
    opacity: 1;
    transform: translateY(var(--hf-space-0));
  }
}

@media (prefers-reduced-motion: reduce) {
  .q-tabs {
    --q-tabs-hover-release-delay: var(--hf-duration-instant);
    --q-tabs-transition-duration: var(--hf-duration-instant);
  }

  .q-tabs-panel {
    animation: none;
  }
}
```

app/packages/quanta/src/components/tabs/tabs.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Tabs } from './index.ts'

describe('<Tabs>', () => {
  it('marks the active tab and renders the animated indicator', () => {
    render(
      <Tabs.Root defaultValue="a">
        <Tabs.List>
          <Tabs.Tab value="a">A</Tabs.Tab>
          <Tabs.Tab value="b">B</Tabs.Tab>
        </Tabs.List>
        <Tabs.Panel value="a">Panel A</Tabs.Panel>
        <Tabs.Panel value="b">Panel B</Tabs.Panel>
      </Tabs.Root>,
    )
    const active = screen.getByRole('tab', { name: 'A' })
    const inactive = screen.getByRole('tab', { name: 'B' })

    expect(active).toHaveAttribute('data-active')
    expect(inactive).not.toHaveAttribute('data-active')
    expect(active.className).toContain('q-tabs-tab')
    expect(screen.getByRole('tablist').querySelector('.q-tabs-indicator')).toBeInTheDocument()
  })

  it('applies segmented, surface, and pill variant classes from the root options', () => {
    const { rerender } = render(
      <Tabs.Root variant="segmented" shape="pill" surface="flat" tone="brandSoft" defaultValue="a">
        <Tabs.List>
          <Tabs.Tab value="a">A</Tabs.Tab>
          <Tabs.Tab value="b">B</Tabs.Tab>
        </Tabs.List>
      </Tabs.Root>,
    )

    const root = screen.getByRole('tablist').parentElement as HTMLElement
    expect(root).toHaveClass('q-tabs-segmented')
    expect(root).toHaveClass('q-tabs-shape-pill')
    expect(root).toHaveClass('q-tabs-surface-flat')
    expect(root).toHaveClass('q-tabs-tone-brand-soft')

    rerender(
      <Tabs.Root variant="pill" defaultValue="a">
        <Tabs.List>
          <Tabs.Tab value="a">A</Tabs.Tab>
          <Tabs.Tab value="b">B</Tabs.Tab>
        </Tabs.List>
      </Tabs.Root>,
    )

    expect(screen.getByRole('tablist').parentElement as HTMLElement).toHaveClass('q-tabs-pill')
  })

  it('applies the full-width fill class when fullWidth is set', () => {
    render(
      <Tabs.Root defaultValue="a">
        <Tabs.List fullWidth aria-label="Full width tabs">
          <Tabs.Tab value="a">A</Tabs.Tab>
          <Tabs.Tab value="b">B</Tabs.Tab>
        </Tabs.List>
      </Tabs.Root>,
    )

    expect(screen.getByRole('tablist', { name: 'Full width tabs' })).toHaveClass('q-tabs-list-fill')
  })

  it('renders canonical start / subtitle / end slots into the tab', () => {
    render(
```

```
    <Tabs.Root defaultValue="a">
      <Tabs.List>
        <Tabs.Tab
          value="a"
          start={<span data-testid="lead" />}
          subtitle={<span data-testid="sub" />}
          end={<span data-testid="trail" />}
        >
          Label
        </Tabs.Tab>
      </Tabs.List>
    </Tabs.Root>,
  )
  expect(screen.getByTestId('lead').closest('.q-tabs-tab-icon')).toBeInTheDocument()
  expect(screen.getByTestId('sub').closest('.q-tabs-tab-secondary')).toBeInTheDocument()
  expect(screen.getByTestId('trail').closest('.q-tabs-tab-icon')).toBeInTheDocument()
  expect(screen.getByText('Label').closest('.q-tabs-tab-label')).toBeInTheDocument()
})

it('keeps the legacy icon / iconEnd / secondaryText aliases (back-compat)', () => {
  render(
    <Tabs.Root defaultValue="a">
      <Tabs.List>
        <Tabs.Tab
          value="a"
          icon={<span data-testid="legacy-lead" />}
          secondaryText={<span data-testid="legacy-sub" />}
          iconEnd={<span data-testid="legacy-trail" />}
        >
          Label
        </Tabs.Tab>
      </Tabs.List>
    </Tabs.Root>,
  )
  expect(screen.getByTestId('legacy-lead').closest('.q-tabs-tab-icon')).toBeInTheDocument()
  expect(screen.getByTestId('legacy-sub').closest('.q-tabs-tab-secondary')).toBeInTheDocument()
  expect(screen.getByTestId('legacy-trail').closest('.q-tabs-tab-icon')).toBeInTheDocument()
})

it('renders children bare (no slot wrappers) when no slot is passed (back-compat)', () => {
  render(
    <Tabs.Root defaultValue="a">
      <Tabs.List>
        <Tabs.Tab value="a">Plain</Tabs.Tab>
      </Tabs.List>
    </Tabs.Root>,
  )
  const tab = screen.getByRole('tab', { name: 'Plain' })
  expect(tab.querySelector('.q-tabs-tab-label')).toBeNull()
  expect(tab.querySelector('.q-tabs-tab-content')).toBeInTheDocument()
})

it('renders data-driven tabs from the items prop (value wiring, label, slots)', () => {
  render(
    <Tabs.Root defaultValue="b">
      <Tabs.List
        aria-label="Data tabs"
        items={[
          { value: 'a', label: 'Alpha', start: <span data-testid="ia" /> },
          { value: 'b', label: 'Beta' },
        ]}
      />
      <Tabs.Panel value="a">Panel A</Tabs.Panel>
      <Tabs.Panel value="b">Panel B</Tabs.Panel>
    </Tabs.Root>,
  )
  expect(screen.getByRole('tab', { name: /Alpha/ })).toBeInTheDocument()
  expect(screen.getByTestId('ia').closest('.q-tabs-tab-icon')).toBeInTheDocument()
  expect(screen.getByRole('tab', { name: 'Beta' })).toHaveAttribute('data-active')
  expect(screen.getByText('Panel B')).toBeInTheDocument()
```

```
    expect(screen.queryByText('Panel A')).toBeNull()
  })

  it('shows only the active panel', () => {
    render(
      <Tabs.Root defaultValue="a">
        <Tabs.List>
          <Tabs.Tab value="a">A</Tabs.Tab>
          <Tabs.Tab value="b">B</Tabs.Tab>
        </Tabs.List>
        <Tabs.Panel value="a">Panel A</Tabs.Panel>
        <Tabs.Panel value="b">Panel B</Tabs.Panel>
      </Tabs.Root>,
    )
    expect(screen.getByText('Panel A')).toBeInTheDocument()
    expect(screen.queryByText('Panel B')).toBeNull()
  })
})
```

app/packages/quanta/src/components/tabs/tabs.tsx

```
'use client'

import type { ComponentProps, CSSProperties, ReactNode } from 'react'
import { Children } from 'react'
import { Tabs as Primitive } from '@base-ui/react/tabs'
import { cx } from '../utils/cx.ts'
import { type SlotColor, slotStyle } from '../utils/slot.ts'

/**
 * Tabs — Base UI primitive (roving focus, keyboard, ARIA) skinned with quanta.
 * Variants cover the Figma underline tabs, text-pill tabs, and segmented
 * controls. `Tabs.List` includes a Base UI indicator so active states animate
 * between tab positions.
 *
 * Multi-part: `export const Tabs = { Root, List, Tab, Panel }`.
 */

export type TabsVariant = 'underline' | 'pill' | 'segmented' | 'soft'
export type TabsShape = 'rounded' | 'pill' | 'icon'
export type TabsSurface = 'glass' | 'flat'
export type TabsTone = 'default' | 'accent' | 'glass' | 'solid' | 'brandSoft' | 'brand'

const VARIANT_CLASS = {
  underline: 'q-tabs-underline',
  pill: 'q-tabs-pill',
  segmented: 'q-tabs-segmented',
  soft: 'q-tabs-soft',
} satisfies Record<TabsVariant, string>

const SHAPE_CLASS = {
  rounded: 'q-tabs-shape-rounded',
  pill: 'q-tabs-shape-pill',
  icon: 'q-tabs-shape-icon',
} satisfies Record<TabsShape, string>

const SURFACE_CLASS = {
  glass: 'q-tabs-surface-glass',
  flat: 'q-tabs-surface-flat',
} satisfies Record<TabsSurface, string>

const TONE_CLASS = {
  default: 'q-tabs-tone-default',
  accent: 'q-tabs-tone-accent',
  glass: 'q-tabs-tone-glass',
  solid: 'q-tabs-tone-solid',
  brandSoft: 'q-tabs-tone-brand-soft',
  brand: 'q-tabs-tone-brand',
} satisfies Record<TabsTone, string>

export interface TabsOptions {
  color?: SlotColor
  shape?: TabsShape
  surface?: TabsSurface
  tone?: TabsTone
  variant?: TabsVariant
}

export type RootProps = Omit<ComponentProps<typeof Primitive.Root>, 'className'> & {
  className?: string
} & TabsOptions

function Root({
  color = 'brand',
  className,
  shape = 'rounded',
  style,
  surface = 'glass',
  tone,
```

```
  variant = 'underline',
  ...props
}: RootProps) {
  const resolvedTone = tone ?? (variant === 'segmented' ? 'glass' : 'default')

  return (
    <Primitive.Root
      style={{ ...slotStyle(color), ...style } as CSSProperties}
      className={cx(
        'q-tabs',
        VARIANT_CLASS[variant],
        SHAPE_CLASS[shape],
        SURFACE_CLASS[surface],
        TONE_CLASS[resolvedTone],
        className,
      )}
      {...props}
    />
  )
}

export type ListProps = Omit<ComponentProps<typeof Primitive.List>, 'className'> & {
  className?: string
  indicator?: boolean
  /** Stretch the list to its container width and size tabs equally. */
  fullWidth?: boolean
  /**
   * Data-driven tabs: renders a `Tabs.Tab` per item instead of composing
   * children. Each item takes the same props as `Tabs.Tab` (`value`, `start`,
   * `end`, `subtitle`, `iconOnly`, `disabled`…) plus `label` for the text.
   * Falls back to `children` when omitted (back-compat).
   */
  items?: TabItem[]
}

function List({ children, items, className, indicator = true, fullWidth = false, ...props }: ListProps) {
  return (
    <Primitive.List
      className={cx('q-tabs-list', fullWidth && 'q-tabs-list-fill', className)}
      {...props}
    >
      {items != null
        ? items.map(({ label, ...tab }, index) => (
            <Tab key={tab.value != null ? String(tab.value) : index} {...tab}>{label}</Tab>
          ))
        : children}
      {indicator ? (
        <Primitive.Indicator
          className="q-tabs-indicator"
          renderBeforeHydration
        />
      ) : null}
    </Primitive.List>
  )
}

export type TabProps = Omit<ComponentProps<typeof Primitive.Tab>, 'className'> & {
  children?: ReactNode
  className?: string
  iconOnly?: boolean
  /** Leading slot (icon / any node). Canonical — matches Button/Chip/Input/Item. */
  start?: ReactNode
  /** Trailing slot (icon / badge / any node), after the label. */
  end?: ReactNode
  /** Muted secondary label rendered after the primary text. */
  subtitle?: ReactNode
  /** @deprecated Use `start`. */
  icon?: ReactNode
  /** @deprecated Use `end`. */
  iconEnd?: ReactNode
```

```
  /** @deprecated Use `subtitle`. */
  secondaryText?: ReactNode
  /** Class for the inner content wrapper (gap/padding live here). */
  contentClassName?: string
}

/** A data-driven tab for `Tabs.List items` — every `Tabs.Tab` prop plus `label` (the text). */
export type TabItem = Omit<TabProps, 'children'> & { label?: ReactNode }

/**
 * Wrap each string/number child in a width-locking "ghost" span. The visible
 * text can switch weight (medium ↔ semibold) on selection without changing the
 * box width, because an invisible bold copy reserves the widest footprint via
 * a 1×1 grid stack. This kills the sub-pixel reflow that made the sliding
 * indicator and neighbouring tabs wobble on select. Non-text nodes (icons)
 * pass through untouched.
 */
function lockTextWidth(children: ReactNode): ReactNode {
  return Children.map(children, (child) => {
    if (typeof child === 'string' || typeof child === 'number') {
      return (
        <span className="q-tabs-tab-text" data-text={String(child)}>
          {child}
        </span>
      )
    }
    return child
  })
}

function Tab({
  children,
  className,
  iconOnly = false,
  start,
  end,
  subtitle,
  icon,
  iconEnd,
  secondaryText,
  contentClassName,
  ...props
}: TabProps) {
  // Canonical start/end/subtitle, with the legacy icon/iconEnd/secondaryText
  // names kept as aliases (byte-identical rendering) for back-compat.
  const lead = start ?? icon
  const trail = end ?? iconEnd
  const sub = subtitle ?? secondaryText
  // Slot mode only when a slot prop is supplied. Otherwise render children
  // bare inside the flex content (preserving the icon-as-children pattern used
  // by the underline/pill/segmented variants, where the content gap spaces an
  // inline <Icon/> + label).
  const useSlots = lead != null || trail != null || sub != null
  return (
    <Primitive.Tab
      className={cx(
        'q-tabs-tab',
        iconOnly && 'q-tabs-tab-icon-only',
        className,
      )}
      {...props}
    >
      <span className={cx('q-tabs-tab-content', contentClassName)}>
        {useSlots ? (
          <>
            {lead != null ? <span className="q-tabs-tab-icon">{lead}</span> : null}
            {children != null ? <span className="q-tabs-tab-label">{lockTextWidth(children)}</span> : null}
            {sub != null ? <span className="q-tabs-tab-secondary">{sub}</span> : null}
            {trail != null ? <span className="q-tabs-tab-icon">{trail}</span> : null}
          </>
```

```
        ) : (
          lockTextWidth(children)
        )}
      </span>
    </Primitive.Tab>
  )
}

export type PanelProps = Omit<ComponentProps<typeof Primitive.Panel>, 'className'> & { className?: string }

function Panel({ className, ...props }: PanelProps) {
  return (
    <Primitive.Panel
      className={cx('q-tabs-panel', className)}
      {...props}
    />
  )
}

export const Tabs = { Root, List, Tab, Panel }
```

app/packages/quanta/src/components/tag/index.ts

```
export { Tag } from './tag.tsx'
export type { TagProps } from './tag.tsx'
```

app/packages/quanta/src/components/tag/tag.css

```
/**
 * Tag presentation — composes only emitted utilities (q-slot-* surfaces, q-*
 * colors/typography, native spacing/sizing/radius), so there are no @utility
 * rules here. This file exists solely to point Tailwind's scanner at the
 * colocated source: quanta is consumed as a symlinked package and Tailwind
 * skips node_modules during automatic content detection, so the literal class
 * strings in tag.tsx must be registered explicitly — same convention as
 * divider.css / dot.css / kbd.css / avatar.css.
 */
@source "./tag.tsx";
```

app/packages/quanta/src/components/tag/tag.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { describe, expect, it, vi } from 'vitest'
import { Tag } from './index.ts'

describe('<Tag>', () => {
  it('renders its children inside a non-interactive span with the slot tint', () => {
    render(<Tag>Beta</Tag>)
    const label = screen.getByText('Beta')
    // The label sits in a truncating inner span; the container carries the slot bg.
    const container = label.closest('span.q-slot-bg-10')
    expect(container).toBeInTheDocument()
    expect(container).toHaveClass('q-slot-text', 'text-q-caption-sm-medium')
  })

  it('defaults to the neutral slot color (sets the --q-tint custom properties)', () => {
    render(<Tag>X</Tag>)
    const container = screen.getByText('X').closest('span.q-slot-bg-10') as HTMLElement
    // slotStyle('neutral') wires the private slot vars inline.
    expect(container.style.getPropertyValue('--q-tint-fg')).not.toBe('')
  })

  it('renders a trailing remove button only when onRemove is provided', () => {
    const { rerender } = render(<Tag>Closable</Tag>)
    expect(screen.queryByRole('button')).not.toBeInTheDocument()

    rerender(<Tag onRemove={() => {}}>Closable</Tag>)
    expect(screen.getByRole('button', { name: 'Remove' })).toBeInTheDocument()
  })

  it('renders the remove glyph as a node-only Icon (decorative, xs-sized svg)', () => {
    render(<Tag onRemove={() => {}}>Closable</Tag>)
    const button = screen.getByRole('button', { name: 'Remove' })
    // Icon is node-only: q-icon/q-icon-xs and aria-hidden land directly on the svg.
    const glyph = button.querySelector('svg.q-icon.q-icon-xs')
    expect(glyph).toBeInTheDocument()
    expect(glyph).toHaveAttribute('aria-hidden', 'true')
  })

  it('invokes onRemove when the remove button is clicked', async () => {
    const onRemove = vi.fn()
    const user = userEvent.setup()
    render(<Tag onRemove={onRemove}>Closable</Tag>)
    await user.click(screen.getByRole('button', { name: 'Remove' }))
    expect(onRemove).toHaveBeenCalledOnce()
  })

  it('supports a custom remove label', () => {
    render(<Tag onRemove={() => {}} removeLabel="Dismiss tag">Closable</Tag>)
    expect(screen.getByRole('button', { name: 'Dismiss tag' })).toBeInTheDocument()
  })

  it('forwards className and native span props', () => {
    render(<Tag className="is-custom" id="t1">Y</Tag>)
    const container = screen.getByText('Y').closest('span.q-slot-bg-10')!
    expect(container).toHaveClass('is-custom')
    expect(container).toHaveAttribute('id', 't1')
  })

  it('renders start before the label and end after it (before remove)', async () => {
    const onRemove = vi.fn()
    render(
      <Tag
        start={<span data-testid="lead" />}
        end={<span data-testid="trail" />}
        onRemove={onRemove}
      >
        Label
```

```
      </Tag>,
    )
    const lead = screen.getByTestId('lead')
    const label = screen.getByText('Label')
    const trail = screen.getByTestId('trail')
    const remove = screen.getByRole('button', { name: 'Remove' })
    // DOM order: start → label → end → remove
    expect(lead.compareDocumentPosition(label) & Node.DOCUMENT_POSITION_FOLLOWING).toBeTruthy()
    expect(label.compareDocumentPosition(trail) & Node.DOCUMENT_POSITION_FOLLOWING).toBeTruthy()
    expect(trail.compareDocumentPosition(remove) & Node.DOCUMENT_POSITION_FOLLOWING).toBeTruthy()
  })

  it('renders no slot wrappers when start/end are omitted (back-compat)', () => {
    render(<Tag>Plain</Tag>)
    const container = screen.getByText('Plain').closest('span.q-slot-bg-10')!
    // only the truncating label span (no flanking slot spans, no remove button)
    expect(container.querySelectorAll(':scope > span')).toHaveLength(1)
    expect(container.querySelector('button')).toBeNull()
  })
})
```

app/packages/quanta/src/components/tag/tag.tsx

```
'use client'

import type { ComponentProps, CSSProperties, ReactNode } from 'react'
import { CloseIcon } from '../close-button/index.ts'
import { Icon } from '../icon/index.ts'
import { cx } from '../utils/cx.ts'
import { type SlotColor, slotStyle } from '../utils/slot.ts'

/**
 * Tag — a presentational labeled category, optionally removable. Soft slot tint
 * by default. When `onRemove` is provided, a trailing "×" button is rendered
 * (a real <button> nested in the <span> container — valid, since the container
 * is non-interactive).
 *
 * Composable like Chip/Button: the label is `children`; `start` / `end` are
 * optional slots (any node — a leading Dot/Avatar/icon, a trailing count Badge)
 * that default to nothing and sit before/after the label. `start`/`end` render
 * only when set, so the legacy `<Tag>Label</Tag>` markup is byte-for-byte
 * unchanged. `end` precedes the remove button when both are present.
 */

export type TagProps = ComponentProps<'span'> & {
  /** Slot color. Default 'neutral'. */
  color?: SlotColor
  /** Leading slot (Dot / Avatar / icon, any node) before the label. */
  start?: ReactNode
  /** Trailing slot (count Badge / Kbd, any node) after the label, before remove. */
  end?: ReactNode
  /** When set, renders a trailing remove button. */
  onRemove?: () => void
  removeLabel?: string
}

export function Tag({ color = 'neutral', start, end, onRemove, removeLabel = 'Remove', className, style, children, ...props }: TagProps) {
  return (
    <span
      style={{ ...slotStyle(color), ...style } as CSSProperties}
      className={cx(
        'inline-flex max-w-full items-center gap-1 rounded-q-150 px-2 py-0.5 align-middle text-q-caption-sm-medium',
        'q-slot-bg-10 q-slot-text',
        className,
      )}
      {...props}
    >
      {start != null ? <span className="inline-flex shrink-0 items-center [&_svg]:size-q-icon-xs">{start}</span> : null}
      <span className="truncate">{children}</span>
      {end != null ? <span className="inline-flex shrink-0 items-center [&_svg]:size-q-icon-xs">{end}</span> : null}
      {onRemove
        ? (
            <button
              type="button"
              onClick={onRemove}
              aria-label={removeLabel}
              className={cx(
                '-mr-0.5 inline-flex size-4 shrink-0 cursor-pointer items-center justify-center rounded-q-100 outline-none transition-colors',
                'hover:q-slot-bg-20 focus-visible:ring-2 focus-visible:q-slot-ring-40',
              )}
            >
              <Icon as={CloseIcon} size="xs" />
            </button>
          )
        : null}
    </span>
  )
}
```

app/packages/quanta/src/components/textarea/index.ts

```
export { Textarea } from './textarea.tsx'
export type { TextareaProps } from './textarea.tsx'
```

app/packages/quanta/src/components/textarea/textarea.css

```
/**
 * Textarea presentation — the multi-line control modifier for the shared field
 * scaffolding, pixel-matched to the Figma TextArea (node 2134:76). The `q-field-*`
 * base utilities (label, surface, input element, helper, error) live in
 * input.css; this file adds only the multiline override and points Tailwind's
 * scanner at the colocated source.
 *
 * Figma TextArea: same white-5% surface as TextField but the control grows
 * vertically — top-aligned (items-start), min-height 80px, 8px block padding.
 */

@source "./textarea.tsx";

@utility q-field-multiline {
  min-height: var(--q-textarea-min-height); /* default 10.25rem lives in generated component tier :root; Figma TextArea state frame 164px has no --hf-space-* equivalent (max is --hf-space-2400 = 96px) */
}

@utility q-field-control-multiline {
  flex: 1 0 var(--hf-space-0);
  height: auto;
  min-height: var(--hf-space-2000); /* 80px */
  flex-direction: column;
  align-items: flex-start;
  padding-block: var(--hf-space-200); /* 8px */
}

.q-field-input.q-field-input-multiline {
  align-self: stretch;
  flex: 1 1 var(--hf-space-0);
  height: auto;
  min-height: var(--hf-space-0);
  padding-block: var(--hf-space-050); /* 2px placeholder wrapper padding */
}
```

app/packages/quanta/src/components/textarea/textarea.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { describe, expect, it } from 'vitest'
import { Textarea } from './index.ts'

describe('<Textarea>', () => {
  it('renders a multiline control on the column surface', () => {
    const { container } = render(<Textarea label="Bio" rows={4} />)
    const ta = screen.getByLabelText('Bio')
    expect(ta.tagName).toBe('TEXTAREA')
    expect(ta).toHaveClass('q-field-input', 'q-field-input-multiline')
    expect(container.querySelector('.q-field')).toHaveClass('q-field-multiline')
    expect(container.querySelector('.q-field-control')).toHaveClass('q-field-control-multiline')
  })

  it('renders the helper description', () => {
    render(<Textarea label="Bio" description="We'll never share this" />)
    expect(screen.getByText('We\'ll never share this')).toHaveClass('q-field-description')
  })

  it('shows the error (red) state and message instead of the description', () => {
    const { container } = render(
      <Textarea label="Notes" description="helper" error="Please enter only letters" defaultValue="Mary387" />,
    )
    expect(screen.getByText('Please enter only letters')).toHaveClass('q-field-error')
    expect(screen.queryByText('helper')).not.toBeInTheDocument()
    expect(container.querySelector('.q-field-control')).toHaveClass('q-field-control-invalid')
    expect(container.querySelector('.q-field-label')).toHaveClass('q-field-label-invalid')
  })

  it('accepts typed input', async () => {
    const user = userEvent.setup()
    render(<Textarea label="Bio" />)
    const ta = screen.getByLabelText('Bio')
    await user.type(ta, 'Hello')
    expect(ta).toHaveValue('Hello')
  })

  it('renders canonical start / end affix slots', () => {
    render(
      <Textarea
        label="Bio"
        start={<span data-testid="lead" />}
        end={<span data-testid="trail" />}
      />,
    )
    expect(screen.getByTestId('lead').closest('.q-field-affix')).toBeInTheDocument()
    expect(screen.getByTestId('trail').closest('.q-field-affix')).toBeInTheDocument()
  })

  it('keeps the legacy prefix / suffix aliases (back-compat)', () => {
    render(
      <Textarea
        label="Bio"
        prefix={<span data-testid="legacy-lead" />}
        suffix={<span data-testid="legacy-trail" />}
      />,
    )
    expect(screen.getByTestId('legacy-lead').closest('.q-field-affix')).toBeInTheDocument()
    expect(screen.getByTestId('legacy-trail').closest('.q-field-affix')).toBeInTheDocument()
  })

  it('renders a bare textarea control with no affixes by default (back-compat)', () => {
    const { container } = render(<Textarea label="Bio" />)
    expect(container.querySelector('.q-field-affix')).toBeNull()
    expect(screen.getByLabelText('Bio').tagName).toBe('TEXTAREA')
  })
```

```
  it('swaps the control element via render', () => {
    render(<Textarea label="Bio" render={<textarea data-testid="custom" />} />)
    const ta = screen.getByTestId('custom')
    expect(ta.tagName).toBe('TEXTAREA')
    expect(ta).toHaveClass('q-field-input')
  })
})
```

app/packages/quanta/src/components/textarea/textarea.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { Field } from '@base-ui/react/field'
import { cx } from '../utils/cx.ts'

/**
 * Textarea — a multi-line labelled field on Base UI `Field`, pixel-matched to
 * the Figma TextArea (node 2134:76). Shares the Input field scaffolding
 * (`q-field-*` in input.css): the white-5% surface (radius/300, 1.5px border →
 * lime on focus / red on error) flexes inside the 164px Figma state frame with
 * an 80px minimum, top-aligned. Provide `error` for the invalid state; `rows`
 * sets the textarea's intrinsic row count when consumers grow the field.
 *
 *   <Textarea label="Bio" placeholder="Add description" description="We'll never share it" />
 *   <Textarea label="Notes" rows={6} error="Please enter only letters" defaultValue="Mary387" />
 */

type FieldProps = {
  /** Label above the control. */
  label?: ReactNode
  /** Helper text below the control (hidden while an `error` shows). */
  description?: ReactNode
  /** Error message — its presence puts the field in the invalid (red) state. */
  error?: ReactNode
  /** Force the invalid state without a message. */
  invalid?: boolean
  /** Append a red `*` to the label. */
  required?: boolean
  /** Leading slot (20px icon, any node), pinned to the top. */
  start?: ReactNode
  /** Trailing slot (20px icon, any node), pinned to the bottom. */
  end?: ReactNode
  /** @deprecated Use `start` — kept as an alias for back-compat. */
  prefix?: ReactNode
  /** @deprecated Use `end` — kept as an alias for back-compat. */
  suffix?: ReactNode
  /** Swap the underlying control element (e.g. an auto-grow / 3rd-party textarea). */
  render?: ComponentProps<typeof Field.Control>['render']
  /** Class for the root Field wrapper. */
  className?: string
  /** Class for the control surface. */
  controlClassName?: string
  /** Class for the `<textarea>` element. */
  inputClassName?: string
  /** Props forwarded to the Base UI `Field.Root`. */
  fieldProps?: ComponentProps<typeof Field.Root>
}

export type TextareaProps = FieldProps & Omit<ComponentProps<'textarea'>, 'prefix' | 'color' | 'children'>

export function Textarea({
  label,
  description,
  error,
  invalid: invalidProp,
  required = false,
  start,
  end,
  prefix,
  suffix,
  render,
  rows,
  className,
  controlClassName,
  inputClassName,
  fieldProps,
  ...controlProps
```

```
}: TextareaProps) {
  const invalid = invalidProp ?? error != null
  // `start`/`end` are canonical; `prefix`/`suffix` are the back-compat aliases.
  const lead = start ?? prefix
  const trail = end ?? suffix

  return (
    <Field.Root className={cx('q-field q-field-multiline', className)} {...fieldProps}>
      {label != null
        ? (
            <Field.Label className={cx('q-field-label', invalid && 'q-field-label-invalid')}>
              {label}
              {required ? <span aria-hidden className="q-field-required">*</span> : null}
            </Field.Label>
          )
        : null}

      <div
        className={cx(
          'q-field-control q-field-control-multiline',
          invalid && 'q-field-control-invalid',
          controlClassName,
        )}
      >
        {lead != null ? <span className="q-field-affix">{lead}</span> : null}
        {/* `ref` rides in `...controlProps` and Base UI forwards it to the
            rendered <textarea> — the primary node — so it is never dropped.
            Field.Control is input-typed; the rendered <textarea> takes textarea
            attrs/handlers at runtime — the cast bridges the element-type variance. */}
        <Field.Control
          render={render ?? <textarea rows={rows} />}
          className={cx('q-field-input q-field-input-multiline', inputClassName)}
          aria-invalid={invalid || undefined}
          {...(controlProps as unknown as ComponentProps<typeof Field.Control>)}
        />
        {trail != null ? <span className="q-field-affix">{trail}</span> : null}
      </div>

      {invalid && error != null
        ? <Field.Error match className="q-field-error">{error}</Field.Error>
        : description != null
          ? <Field.Description className="q-field-description">{description}</Field.Description>
          : null}
    </Field.Root>
  )
}
```

app/packages/quanta/src/components/toggle/index.ts

```
export { Toggle } from './toggle.tsx'
export type { ToggleProps, ToggleSize } from './toggle.tsx'
```

app/packages/quanta/src/components/toggle/toggle.css

```
/**
 * Toggle presentation utilities (q-namespaced). Geometry follows the same
 * 32/40/48px control ramp as the ready-for-dev Toggle page, with pressed state
 * painted by the shared slot-color system.
 */
@source "./toggle.tsx";

@utility q-toggle {
  @apply inline-flex shrink-0 cursor-pointer items-center justify-center rounded-q-300;
  @apply bg-transparent text-q-text-secondary outline-none select-none;
  @apply transition-colors;

  &:hover {
    @apply bg-q-overlay-hover;
  }

  &[data-pressed] {
    @apply q-slot-bg-10 q-slot-text;
  }

  &:focus-visible {
    @apply ring-2 q-slot-ring-40;
  }

  &[data-disabled] {
    @apply pointer-events-none opacity-q-disabled;
  }
}

@utility q-toggle-sm {
  height: var(--hf-space-800);
  min-width: var(--hf-space-800);
  gap: var(--hf-space-100);
  padding-inline: var(--hf-space-200);
  @apply text-q-label-sm-medium;

  & svg {
    height: var(--hf-space-400);
    width: var(--hf-space-400);
  }
}

@utility q-toggle-md {
  height: var(--hf-space-1000);
  min-width: var(--hf-space-1000);
  gap: var(--hf-space-200);
  padding-inline: var(--hf-space-300);
  @apply text-q-label-sm-medium;

  & svg {
    height: var(--hf-space-500);
    width: var(--hf-space-500);
  }
}

@utility q-toggle-lg {
  height: var(--hf-space-1200);
  min-width: var(--hf-space-1200);
  gap: var(--hf-space-200);
  padding-inline: var(--hf-space-400);
  @apply text-q-label-md-medium;

  & svg {
    height: var(--hf-space-600);
    width: var(--hf-space-600);
  }
}
```

app/packages/quanta/src/components/toggle/toggle.test.tsx

```
import { render, screen } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { describe, expect, it, vi } from 'vitest'
import { Toggle } from './index.ts'

describe('<Toggle>', () => {
  it('renders a pressable button, unpressed by default', () => {
    render(<Toggle>Bold</Toggle>)
    const btn = screen.getByRole('button', { name: 'Bold' })
    expect(btn).toHaveAttribute('aria-pressed', 'false')
  })

  it('reflects defaultPressed (uncontrolled) via data-pressed', () => {
    render(<Toggle defaultPressed>On</Toggle>)
    const btn = screen.getByRole('button', { name: 'On' })
    expect(btn).toHaveAttribute('aria-pressed', 'true')
    expect(btn).toHaveAttribute('data-pressed')
  })

  it('toggles on click and fires onPressedChange', async () => {
    const onPressedChange = vi.fn()
    const user = userEvent.setup()
    render(<Toggle onPressedChange={onPressedChange}>Tap</Toggle>)
    const btn = screen.getByRole('button', { name: 'Tap' })
    await user.click(btn)
    expect(onPressedChange).toHaveBeenCalledWith(true, expect.anything())
    expect(btn).toHaveAttribute('aria-pressed', 'true')
  })

  it('applies the md size classes by default', () => {
    render(<Toggle>M</Toggle>)
    expect(screen.getByRole('button', { name: 'M' })).toHaveClass('q-toggle', 'q-toggle-md')
  })

  it('maps size to its token sizing classes', () => {
    const { rerender } = render(<Toggle size="sm">S</Toggle>)
    expect(screen.getByRole('button', { name: 'S' })).toHaveClass('q-toggle-sm')

    rerender(<Toggle size="lg">L</Toggle>)
    expect(screen.getByRole('button', { name: 'L' })).toHaveClass('q-toggle-lg')
  })

  it('wires the slot color custom properties from the color prop', () => {
    render(<Toggle color="success">C</Toggle>)
    const btn = screen.getByRole('button', { name: 'C' }) as HTMLElement
    expect(btn.style.getPropertyValue('--q-tint')).not.toBe('')
  })

  it('forwards a string className alongside the base classes', () => {
    render(<Toggle className="is-custom">X</Toggle>)
    expect(screen.getByRole('button', { name: 'X' })).toHaveClass('is-custom')
  })

  it('renders start before and end after the label', () => {
    render(
      <Toggle start={<span data-testid="lead" />} end={<span data-testid="trail" />}>
        Label
      </Toggle>,
    )
    const btn = screen.getByRole('button', { name: /Label/ })
    const lead = screen.getByTestId('lead')
    const trail = screen.getByTestId('trail')
    expect(btn).toContainElement(lead)
    expect(btn).toContainElement(trail)
    // DOM order: start → label → end
    expect(lead.compareDocumentPosition(trail) & Node.DOCUMENT_POSITION_FOLLOWING).toBeTruthy()
    expect(btn.textContent).toBe('Label')
  })
```

```
  it('renders children bare when no slot is passed (back-compat)', () => {
    render(<Toggle>Plain</Toggle>)
    const btn = screen.getByRole('button', { name: 'Plain' })
    // no slot wrappers added — the button contains only the text
    expect(btn.childElementCount).toBe(0)
    expect(btn).toHaveTextContent('Plain')
  })
})
```

app/packages/quanta/src/components/toggle/toggle.tsx

```
'use client'

import type { ComponentProps, CSSProperties, ReactNode } from 'react'
import { Toggle as Primitive } from '@base-ui/react/toggle'
import { cx } from '../utils/cx.ts'
import { type SlotColor, slotStyle } from '../utils/slot.ts'

/**
 * Toggle — a two-state pressable button (Base UI `Toggle`). When pressed, a soft
 * slot tint (`q-slot-bg-10` + `q-slot-text`) derived from the `color` prop fills
 * it. State comes from Base UI's `data-pressed` attribute.
 *
 * Composable like Button/Chip: the label is `children`; `start` / `end` are
 * optional slots (any node — a leading icon, a trailing count `<Badge>`) that
 * default to nothing. The gap + `& svg` sizing space them; the legacy
 * icon-as-children pattern is byte-for-byte unchanged when no slot is passed.
 * Host element is swappable via Base UI `render` (passes straight through).
 */

export type ToggleSize = 'sm' | 'md' | 'lg'

const SIZE_CLASS = {
  sm: 'q-toggle-sm',
  md: 'q-toggle-md',
  lg: 'q-toggle-lg',
} satisfies Record<ToggleSize, string>

export type ToggleProps = ComponentProps<typeof Primitive> & {
  /** Slot color for the pressed tint. Default 'brand'. */
  color?: SlotColor
  size?: ToggleSize
  /** Leading slot (icon, any node) before the label. */
  start?: ReactNode
  /** Trailing slot (count, badge, any node) after the label. */
  end?: ReactNode
}

export function Toggle({ color = 'brand', size = 'md', start, end, className, style, children, ...props }: ToggleProps) {
  // Slots flank the label only when set; otherwise children render bare so the
  // legacy icon-as-children pattern is unchanged. Gap + `& svg` do the spacing.
  const content = start != null || end != null ? <>{start}{children}{end}</> : children
  return (
    <Primitive
      style={{ ...slotStyle(color), ...style } as CSSProperties}
      className={state => cx(
        'q-toggle',
        SIZE_CLASS[size],
        typeof className === 'function' ? className(state) : className,
      )}
      {...props}
    >
      {content}
    </Primitive>
  )
}
```

app/packages/quanta/src/components/tooltip/index.ts

```
export { Tooltip } from './tooltip.tsx'
export type {
  TooltipAlign,
  TooltipContentProps,
  TooltipProviderProps,
  TooltipRootProps,
  TooltipSide,
  TooltipTriggerProps,
} from './tooltip.tsx'
```

app/packages/quanta/src/components/tooltip/tooltip.css

```
/**
 * Tooltip presentation utilities (q-tooltip-* namespace) for the Base UI
 * `Tooltip` primitive. A small, legible inverted surface — `background-inverse`
 * + `text-inverse` (the standard tooltip look, theme-reactive). The Popup fades
 * and scales from its anchor-derived origin; the optional Arrow inherits the
 * same surface. Positioning, side/align, and collision are owned by Base UI.
 */

@source "./tooltip.tsx";

/* ■■ Positioner (transform origin + z-stacking; no own surface) ■■■■■■■■■■■■■■■ */
@utility q-tooltip-positioner {
  z-index: var(--hf-z-index-tooltip);
}

/* ■■ Popup (the inverted surface) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-tooltip {
  --q-tooltip-surface: var(--hf-color-background-inverse);

  background-color: var(--q-tooltip-surface);
  border-radius: var(--hf-radius-200);
  box-sizing: border-box;
  color: var(--hf-color-text-inverse);
  max-width: var(--q-tooltip-max-width, calc(var(--hf-space-2400) * 3)); /* 288px — keep tooltips tight */
  outline: none;
  padding: var(--hf-space-100) var(--hf-space-150); /* 4px 6px */
  width: max-content;
  @apply text-q-caption-sm-medium;

  /* Open / close: scale + fade from the anchor-derived origin. */
  transform-origin: var(--transform-origin);
  transition-property: opacity, transform;
  transition-duration: var(--hf-duration-180);
  transition-timing-function: var(--hf-ease-out-expo);
  will-change: transform, opacity;

  &[data-starting-style],
  &[data-ending-style] {
    opacity: 0;
    transform: scale(0.92);
  }

  &[data-starting-style][data-side="top"],
  &[data-ending-style][data-side="top"] {
    transform: scale(0.92) translateY(var(--hf-space-050));
  }
  &[data-starting-style][data-side="bottom"],
  &[data-ending-style][data-side="bottom"] {
    transform: scale(0.92) translateY(calc(-1 * var(--hf-space-050)));
  }
  &[data-starting-style][data-side="left"],
  &[data-ending-style][data-side="left"],
  &[data-starting-style][data-side="inline-start"],
  &[data-ending-style][data-side="inline-start"] {
    transform: scale(0.92) translateX(var(--hf-space-050));
  }
  &[data-starting-style][data-side="right"],
  &[data-ending-style][data-side="right"],
  &[data-starting-style][data-side="inline-end"],
  &[data-ending-style][data-side="inline-end"] {
    transform: scale(0.92) translateX(calc(-1 * var(--hf-space-050)));
  }

  &[data-ending-style] {
    transition-duration: var(--hf-duration-130);
    transition-timing-function: var(--hf-ease-in);
  }
```

```
  &[data-instant] {
    transition-duration: var(--hf-duration-instant);
  }
}

/* ▀▀ Arrow (same surface; rotated square nub on the anchored side) ▀▀▀▀▀▀▀▀▀▀▀▀ */
@utility q-tooltip-arrow {
  background-color: var(--q-tooltip-surface);
  height: var(--hf-space-200); /* 8px */
  width: var(--hf-space-200);

  &[data-side="top"] {
    bottom: calc(-1 * var(--hf-space-050));
    transform: rotate(45deg);
  }
  &[data-side="bottom"] {
    top: calc(-1 * var(--hf-space-050));
    transform: rotate(45deg);
  }
  &[data-side="left"],
  &[data-side="inline-start"] {
    right: calc(-1 * var(--hf-space-050));
    transform: rotate(45deg);
  }
  &[data-side="right"],
  &[data-side="inline-end"] {
    left: calc(-1 * var(--hf-space-050));
    transform: rotate(45deg);
  }
}

/* ▀▀ Reduced motion ▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀▀ */
@media (prefers-reduced-motion: reduce) {
  .q-tooltip {
    transition-duration: 1ms; /* no token: reduced-motion snap sentinel, instant token is 0ms */
  }

  .q-tooltip[data-starting-style],
  .q-tooltip[data-ending-style] {
    transform: none;
  }
}
```

app/packages/quanta/src/components/tooltip/tooltip.test.tsx

```
import { render, screen, waitFor } from '@testing-library/react'
import userEvent from '@testing-library/user-event'
import { createRef } from 'react'
import { describe, expect, it } from 'vitest'
import { Tooltip, type TooltipContentProps } from './index.ts'

function Basic({
  side,
  arrow,
  triggerRef,
  triggerClassName,
}: {
  side?: TooltipContentProps['side']
  arrow?: boolean
  triggerRef?: React.Ref<HTMLButtonElement>
  triggerClassName?: string
} = {}) {
  return (
    <Tooltip.Root>
      <Tooltip.Trigger ref={triggerRef} className={triggerClassName}>Trigger</Tooltip.Trigger>
      <Tooltip.Content side={side} arrow={arrow}>
        Helpful hint
      </Tooltip.Content>
    </Tooltip.Root>
  )
}

describe('Tooltip', () => {
  it('renders the trigger (a pure anchor, forwards className) and keeps the popup closed by default', () => {
    render(<Basic triggerClassName="custom-trigger" />)
    const trigger = screen.getByRole('button', { name: 'Trigger' })
    // The trigger has no own skin; a forwarded className lands on the element.
    expect(trigger).toHaveClass('custom-trigger')
    expect(screen.queryByText('Helpful hint')).not.toBeInTheDocument()
  })

  it('shows the tooltip content on focus and hides it on blur', async () => {
    const user = userEvent.setup()
    render(<Basic />)

    await user.tab()
    expect(screen.getByRole('button', { name: 'Trigger' })).toHaveFocus()
    await waitFor(() => expect(screen.getByText('Helpful hint')).toBeInTheDocument())

    await user.tab()
    await waitFor(() => expect(screen.queryByText('Helpful hint')).not.toBeInTheDocument())
  })

  it('opens with defaultOpen and exposes a tooltip role', async () => {
    render(
      <Tooltip.Root defaultOpen>
        <Tooltip.Trigger>Trigger</Tooltip.Trigger>
        <Tooltip.Content>Open now</Tooltip.Content>
      </Tooltip.Root>,
    )
    await waitFor(() => expect(screen.getByRole('tooltip')).toHaveTextContent('Open now'))
    expect(screen.getByRole('tooltip')).toHaveClass('q-tooltip')
  })

  it('paints the requested side onto the popup and renders an arrow when enabled', async () => {
    render(
      <Tooltip.Root defaultOpen>
        <Tooltip.Trigger>Trigger</Tooltip.Trigger>
        <Tooltip.Content side="right" arrow>Sided</Tooltip.Content>
      </Tooltip.Root>,
    )
    const popup = await screen.findByRole('tooltip')
    expect(popup).toHaveAttribute('data-side', 'right')
```

```
    expect(popup.querySelector('.q-tooltip-arrow')).not.toBeNull()
  })

  it('forwards a ref to the trigger element', () => {
    const ref = createRef<HTMLButtonElement>()
    render(<Basic triggerRef={ref} />)
    expect(ref.current).toBeInstanceOf(HTMLButtonElement)
    expect(ref.current).toHaveTextContent('Trigger')
  })

  it('forwards the Root delay down to the trigger', () => {
    render(
      <Tooltip.Root delay={250}>
        <Tooltip.Trigger>Trigger</Tooltip.Trigger>
        <Tooltip.Content>Hint</Tooltip.Content>
      </Tooltip.Root>,
    )
    // Sanity render — the delay is consumed by Base UI internally; ensure the
    // composed tree mounts without throwing and the trigger is present.
    expect(screen.getByRole('button', { name: 'Trigger' })).toBeInTheDocument()
  })

  it('shares a delay via Provider across multiple roots', () => {
    render(
      <Tooltip.Provider delay={100}>
        <Tooltip.Root>
          <Tooltip.Trigger>One</Tooltip.Trigger>
          <Tooltip.Content>First</Tooltip.Content>
        </Tooltip.Root>
        <Tooltip.Root>
          <Tooltip.Trigger>Two</Tooltip.Trigger>
          <Tooltip.Content>Second</Tooltip.Content>
        </Tooltip.Root>
      </Tooltip.Provider>,
    )
    expect(screen.getByRole('button', { name: 'One' })).toBeInTheDocument()
    expect(screen.getByRole('button', { name: 'Two' })).toBeInTheDocument()
  })
})
```

app/packages/quanta/src/components/tooltip/tooltip.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { createContext, useContext } from 'react'
import { Tooltip as Primitive } from '@base-ui/react/tooltip'
import { cx } from '../utils/cx.ts'

/**
 * Tooltip — a hover/focus popup on the Base UI `Tooltip` primitive (open timing,
 * focus/hover triggers, ARIA, portal + collision-aware positioning), skinned
 * with quanta's `q-tooltip-*` presentation utilities (see `tooltip.css`).
 *
 * COMPOSITION-FIRST. The parts mirror Base UI: `Tooltip.Provider` (optional,
 * groups delay so adjacent tooltips open instantly), `Tooltip.Root` (open state
 * + delay), `Tooltip.Trigger` (anchor — `render` any element/quanta component),
 * `Tooltip.Content` (the popup; bundles Portal → Positioner → Popup and an
 * optional Arrow). The skin is the standard small inverted surface
 * (`background-inverse` + `text-inverse`).
 *
 *   <Tooltip.Root>
 *     <Tooltip.Trigger render={<Button iconOnly aria-label="Settings"><Icon><GearIcon /></Icon></Button>} />
 *     <Tooltip.Content>Settings</Tooltip.Content>
 *   </Tooltip.Root>
 *
 * Wrap a cluster of triggers in `Tooltip.Provider` to share the open delay:
 *
 *   <Tooltip.Provider delay={300}>
 *     <Tooltip.Root>…</Tooltip.Root>
 *     <Tooltip.Root>…</Tooltip.Root>
 *   </Tooltip.Provider>
 */

export type TooltipSide = NonNullable<ComponentProps<typeof Primitive.Positioner>['side']>
export type TooltipAlign = NonNullable<ComponentProps<typeof Primitive.Positioner>['align']>

/* ■■ Provider (optional shared-delay grouping) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

export type TooltipProviderProps = ComponentProps<typeof Primitive.Provider>

function Provider(props: TooltipProviderProps) {
  return <Primitive.Provider {...props} />
}

/* Delay flows Root → Trigger via context so callers set it once on Root. */
const DelayContext = createContext<{ delay?: number, closeDelay?: number }>({})

/* ■■ Root (open state + per-trigger delay convenience) ■■■■■■■■■■■■■■■■■■■■■■■■■ */

export type TooltipRootProps = Omit<ComponentProps<typeof Primitive.Root>, 'children'> & {
  /** Trigger + Content. */
  children?: ReactNode
  /** Delay (ms) before opening on hover. Default 600 (Base UI). */
  delay?: number
  /** Delay (ms) before closing after the pointer leaves. Default 0. */
  closeDelay?: number
  /** Allow hovering into the popup itself without closing. */
  hoverable?: boolean
}

/**
 * Root groups the parts and owns open state. `delay` / `closeDelay` are forwarded
 * to the Trigger (where Base UI reads them); `hoverable` maps to Base UI's
 * `disableHoverablePopup` (inverted), defaulting to a non-hoverable tooltip.
 */
function Root({ delay, closeDelay, hoverable = false, children, ...props }: TooltipRootProps) {
  return (
    <Primitive.Root disableHoverablePopup={!hoverable} {...props}>
      <DelayContext.Provider value={{ delay, closeDelay }}>{children}</DelayContext.Provider>
```

```
    </Primitive.Root>
  )
}

/* ■■ Trigger ■■■
```

```
          {arrow ? <Primitive.Arrow className="q-tooltip-arrow" /> : null}
        </Primitive.Popup>
      </Primitive.Positioner>
    </Primitive.Portal>
  )
}

export const Tooltip = {
  Provider,
  Root,
  Trigger,
  Content,
}
```

app/packages/quanta/src/components/typography/index.ts

```
export { Typography, typography } from './typography.tsx'
export type { TypographyColor, TypographyOptions, TypographyProps, TypographyVariant } from './typography.tsx'
```

app/packages/quanta/src/components/typography/typography.css

```
/**
 * Typography registration (Mode B — registration-only; see ai/COMPONENT_STANDARD.md §3).
 *
 * Typography emits NO new rules: it is the single funnel that composes the
 * already-emitted composite type utilities (`text-q-<role>-<size>-<weight>` from
 * src/css/tailwind/typography.css) plus the semantic `text-q-text-*` colour
 * utilities (src/css/tailwind/color.css) and the native `truncate` utility.
 *
 * This file exists only to point Tailwind's scanner at the colocated source:
 * Tailwind skips node_modules and quanta is consumed as a symlinked workspace
 * package, so the literal class strings in typography.tsx (e.g.
 * `text-q-body-md-regular`, `text-q-text-secondary`) are NOT discovered unless
 * registered here. The TS unions + `satisfies Record<…>` maps in typography.tsx
 * are the single source of truth.
 */
@source "./typography.tsx";
```

app/packages/quanta/src/components/typography/typography.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Typography, typography } from './index.ts'

describe('<Typography>', () => {
  it('defaults to a <p> with the body-md-regular composite utility', () => {
    render(<Typography>Hello</Typography>)
    const el = screen.getByText('Hello')
    expect(el.tagName).toBe('P')
    expect(el).toHaveClass('text-q-body-md-regular')
  })

  it('applies the matching text-q-* utility for a given variant', () => {
    render(<Typography variant="headline-lg-semi-bold">Title</Typography>)
    expect(screen.getByText('Title')).toHaveClass('text-q-headline-lg-semi-bold')
  })

  it('renders the element named by `as`', () => {
    render(<Typography as="h1" variant="display-lg-bold">Big</Typography>)
    const el = screen.getByText('Big')
    expect(el.tagName).toBe('H1')
    expect(el).toHaveClass('text-q-display-lg-bold')
  })

  it('maps color to the semantic text-q-text-* utility', () => {
    render(<Typography color="secondary">Muted</Typography>)
    expect(screen.getByText('Muted')).toHaveClass('text-q-text-secondary')
  })

  it('omits a colour class when color is not set (inherits)', () => {
    render(<Typography>Inherit</Typography>)
    const cls = screen.getByText('Inherit').className
    expect(cls).not.toMatch(/text-q-text-/)
  })

  it('adds the truncate utility when truncate is set', () => {
    render(<Typography truncate>Long text</Typography>)
    expect(screen.getByText('Long text')).toHaveClass('truncate')
  })

  it('does not truncate by default', () => {
    render(<Typography>Plain</Typography>)
    expect(screen.getByText('Plain')).not.toHaveClass('truncate')
  })

  it('applies the caller className last so it wins ordering', () => {
    render(<Typography className="is-custom">Custom</Typography>)
    const el = screen.getByText('Custom')
    expect(el).toHaveClass('text-q-body-md-regular', 'is-custom')
    expect(el.className.trim().endsWith('is-custom')).toBe(true)
  })

  it('forwards native props to the rendered element', () => {
    render(<Typography as="span" title="tip" data-testid="t">x</Typography>)
    const el = screen.getByTestId('t')
    expect(el.tagName).toBe('SPAN')
    expect(el).toHaveAttribute('title', 'tip')
  })

  describe('typography() recipe', () => {
    it('returns the default body-md-regular class', () => {
      expect(typography()).toBe('text-q-body-md-regular')
    })

    it('composes variant + color + truncate + extra in order', () => {
      expect(typography({ variant: 'label-md-semi-bold', color: 'brand', truncate: true }, 'extra')).toBe(
        'text-q-label-md-semi-bold text-q-text-brand truncate extra',
      )
```

```
    })
  })
})
```

app/packages/quanta/src/components/typography/typography.tsx

```
'use client'

import type { ComponentPropsWithRef, ElementType } from 'react'
import { createElement } from 'react'
import type { ClassValue } from '../utils/cx.ts'
import { cx } from '../utils/cx.ts'

/**
 * Typography — the single funnel for ALL text in quanta apps. It is the only
 * place a consumer should pick a type role: instead of writing raw
 * `text-2xl font-bold tracking-tight`, you pick a `variant` and Typography
 * applies the matching emitted composite utility (`text-q-<variant>` from
 * `src/css/tailwind/typography.css`, which carries font-family + size +
 * line-height + weight + letter-spacing together).
 *
 * Polymorphic via `as` (default `p`). Optional `color` maps to the
 * `text-q-text-*` semantic roles; omit it to inherit the surrounding colour.
 * `truncate` clamps to a single line with an ellipsis. `className` is applied
 * last so callers always win.
 *
 * The `TypographyVariant` union is the single source of truth for the variant
 * set — `VARIANT_CLASS` is `satisfies Record<TypographyVariant, string>`, so a
 * missing entry fails to compile, and the literal `text-q-*` strings are what
 * Tailwind's scanner extracts (via `@source "./typography.tsx"`).
 */

export type TypographyVariant =
  | 'accent-2xs-bold'
  | 'accent-xs-bold'
  | 'accent-sm-bold'
  | 'accent-md-bold'
  | 'accent-lg-bold'
  | 'accent-xl-bold'
  | 'accent-2xl-bold'
  | 'accent-3xl-bold'
  | 'accent-4xl-bold'
  | 'accent-5xl-bold'
  | 'accent-6xl-bold'
  | 'body-sm-regular'
  | 'body-sm-medium'
  | 'body-sm-semi-bold'
  | 'body-md-regular'
  | 'body-md-medium'
  | 'body-md-semi-bold'
  | 'body-lg-regular'
  | 'body-lg-medium'
  | 'body-lg-semi-bold'
  | 'caption-xs-regular'
  | 'caption-xs-medium'
  | 'caption-xs-semi-bold'
  | 'caption-sm-regular'
  | 'caption-sm-medium'
  | 'caption-sm-semi-bold'
  | 'display-sm-medium'
  | 'display-sm-semi-bold'
  | 'display-sm-bold'
  | 'display-md-medium'
  | 'display-md-semi-bold'
  | 'display-md-bold'
  | 'display-lg-medium'
  | 'display-lg-semi-bold'
  | 'display-lg-bold'
  | 'headline-sm-medium'
  | 'headline-sm-semi-bold'
  | 'headline-sm-bold'
  | 'headline-md-medium'
  | 'headline-md-semi-bold'
  | 'headline-md-bold'
```

```
  | 'headline-lg-medium'
  | 'headline-lg-semi-bold'
  | 'headline-lg-bold'
  | 'label-xs-regular'
  | 'label-xs-medium'
  | 'label-xs-semi-bold'
  | 'label-xs-bold'
  | 'label-sm-regular'
  | 'label-sm-medium'
  | 'label-sm-semi-bold'
  | 'label-sm-bold'
  | 'label-md-regular'
  | 'label-md-medium'
  | 'label-md-semi-bold'
  | 'label-md-bold'
  | 'label-lg-regular'
  | 'label-lg-medium'
  | 'label-lg-semi-bold'
  | 'label-lg-bold'
  | 'mono-xs-regular'
  | 'mono-xs-medium'
  | 'mono-xs-semi-bold'
  | 'mono-sm-regular'
  | 'mono-sm-medium'
  | 'mono-sm-semi-bold'
  | 'mono-md-regular'
  | 'mono-md-medium'
  | 'mono-md-semi-bold'
  | 'mono-lg-regular'
  | 'mono-lg-medium'
  | 'mono-lg-semi-bold'
  | 'title-sm-medium'
  | 'title-sm-semi-bold'
  | 'title-sm-bold'
  | 'title-md-medium'
  | 'title-md-semi-bold'
  | 'title-md-bold'
  | 'title-lg-medium'
  | 'title-lg-semi-bold'
  | 'title-lg-bold'

export type TypographyColor =
  | 'primary'
  | 'secondary'
  | 'tertiary'
  | 'inverse'
  | 'disabled'
  | 'link'
  | 'brand'
  | 'danger'
  | 'success'
  | 'warning'
  | 'info'
  | 'on-overlay-secondary'
  | 'on-overlay-tertiary'

// Variant → its emitted composite utility. Literal `text-q-<variant>` strings
// so Tailwind's scanner extracts them; `satisfies Record<…>` makes the union
// the single source of truth (a missing entry fails to compile).
const VARIANT_CLASS = {
  'accent-2xs-bold': 'text-q-accent-2xs-bold',
  'accent-xs-bold': 'text-q-accent-xs-bold',
  'accent-sm-bold': 'text-q-accent-sm-bold',
  'accent-md-bold': 'text-q-accent-md-bold',
  'accent-lg-bold': 'text-q-accent-lg-bold',
  'accent-xl-bold': 'text-q-accent-xl-bold',
  'accent-2xl-bold': 'text-q-accent-2xl-bold',
  'accent-3xl-bold': 'text-q-accent-3xl-bold',
  'accent-4xl-bold': 'text-q-accent-4xl-bold',
  'accent-5xl-bold': 'text-q-accent-5xl-bold',
```

```
'accent-6xl-bold': 'text-q-accent-6xl-bold',
'body-sm-regular': 'text-q-body-sm-regular',
'body-sm-medium': 'text-q-body-sm-medium',
'body-sm-semi-bold': 'text-q-body-sm-semi-bold',
'body-md-regular': 'text-q-body-md-regular',
'body-md-medium': 'text-q-body-md-medium',
'body-md-semi-bold': 'text-q-body-md-semi-bold',
'body-lg-regular': 'text-q-body-lg-regular',
'body-lg-medium': 'text-q-body-lg-medium',
'body-lg-semi-bold': 'text-q-body-lg-semi-bold',
'caption-xs-regular': 'text-q-caption-xs-regular',
'caption-xs-medium': 'text-q-caption-xs-medium',
'caption-xs-semi-bold': 'text-q-caption-xs-semi-bold',
'caption-sm-regular': 'text-q-caption-sm-regular',
'caption-sm-medium': 'text-q-caption-sm-medium',
'caption-sm-semi-bold': 'text-q-caption-sm-semi-bold',
'display-sm-medium': 'text-q-display-sm-medium',
'display-sm-semi-bold': 'text-q-display-sm-semi-bold',
'display-sm-bold': 'text-q-display-sm-bold',
'display-md-medium': 'text-q-display-md-medium',
'display-md-semi-bold': 'text-q-display-md-semi-bold',
'display-md-bold': 'text-q-display-md-bold',
'display-lg-medium': 'text-q-display-lg-medium',
'display-lg-semi-bold': 'text-q-display-lg-semi-bold',
'display-lg-bold': 'text-q-display-lg-bold',
'headline-sm-medium': 'text-q-headline-sm-medium',
'headline-sm-semi-bold': 'text-q-headline-sm-semi-bold',
'headline-sm-bold': 'text-q-headline-sm-bold',
'headline-md-medium': 'text-q-headline-md-medium',
'headline-md-semi-bold': 'text-q-headline-md-semi-bold',
'headline-md-bold': 'text-q-headline-md-bold',
'headline-lg-medium': 'text-q-headline-lg-medium',
'headline-lg-semi-bold': 'text-q-headline-lg-semi-bold',
'headline-lg-bold': 'text-q-headline-lg-bold',
'label-xs-regular': 'text-q-label-xs-regular',
'label-xs-medium': 'text-q-label-xs-medium',
'label-xs-semi-bold': 'text-q-label-xs-semi-bold',
'label-xs-bold': 'text-q-label-xs-bold',
'label-sm-regular': 'text-q-label-sm-regular',
'label-sm-medium': 'text-q-label-sm-medium',
'label-sm-semi-bold': 'text-q-label-sm-semi-bold',
'label-sm-bold': 'text-q-label-sm-bold',
'label-md-regular': 'text-q-label-md-regular',
'label-md-medium': 'text-q-label-md-medium',
'label-md-semi-bold': 'text-q-label-md-semi-bold',
'label-md-bold': 'text-q-label-md-bold',
'label-lg-regular': 'text-q-label-lg-regular',
'label-lg-medium': 'text-q-label-lg-medium',
'label-lg-semi-bold': 'text-q-label-lg-semi-bold',
'label-lg-bold': 'text-q-label-lg-bold',
'mono-xs-regular': 'text-q-mono-xs-regular',
'mono-xs-medium': 'text-q-mono-xs-medium',
'mono-xs-semi-bold': 'text-q-mono-xs-semi-bold',
'mono-sm-regular': 'text-q-mono-sm-regular',
'mono-sm-medium': 'text-q-mono-sm-medium',
'mono-sm-semi-bold': 'text-q-mono-sm-semi-bold',
'mono-md-regular': 'text-q-mono-md-regular',
'mono-md-medium': 'text-q-mono-md-medium',
'mono-md-semi-bold': 'text-q-mono-md-semi-bold',
'mono-lg-regular': 'text-q-mono-lg-regular',
'mono-lg-medium': 'text-q-mono-lg-medium',
'mono-lg-semi-bold': 'text-q-mono-lg-semi-bold',
'title-sm-medium': 'text-q-title-sm-medium',
'title-sm-semi-bold': 'text-q-title-sm-semi-bold',
'title-sm-bold': 'text-q-title-sm-bold',
'title-md-medium': 'text-q-title-md-medium',
'title-md-semi-bold': 'text-q-title-md-semi-bold',
'title-md-bold': 'text-q-title-md-bold',
'title-lg-medium': 'text-q-title-lg-medium',
'title-lg-semi-bold': 'text-q-title-lg-semi-bold',
```

```
  'title-lg-bold': 'text-q-title-lg-bold',
} satisfies Record<TypographyVariant, string>

// Color → semantic text-colour utility. Literal strings for the scanner.
const COLOR_CLASS = {
  primary: 'text-q-text-primary',
  secondary: 'text-q-text-secondary',
  tertiary: 'text-q-text-tertiary',
  inverse: 'text-q-text-inverse',
  disabled: 'text-q-text-disabled',
  link: 'text-q-text-link',
  brand: 'text-q-text-brand',
  danger: 'text-q-text-danger',
  success: 'text-q-text-success',
  warning: 'text-q-text-warning',
  info: 'text-q-text-info',
  'on-overlay-secondary': 'text-q-text-on-overlay-secondary',
  'on-overlay-tertiary': 'text-q-text-on-overlay-tertiary',
} satisfies Record<TypographyColor, string>

export interface TypographyOptions {
  variant?: TypographyVariant
  color?: TypographyColor
  truncate?: boolean
}

export type TypographyProps<E extends ElementType = 'p'> = TypographyOptions & {
  /** Render as a different element (e.g. `as="h1"` / `as="span"`). Default `p`. */
  as?: E
} & Omit<ComponentPropsWithRef<E>, 'as' | 'color' | keyof TypographyOptions>

/**
 * Recipe — the class string for a given variant/color/truncate, so non-Typography
 * elements (a Base UI part, an existing tag) can be styled identically:
 * `<Trigger className={typography({ variant: 'label-md-semi-bold' })} />`.
 */
export function typography(options: TypographyOptions = {}, ...extra: ClassValue[]): string {
  const { variant = 'body-md-regular', color, truncate } = options
  return cx(VARIANT_CLASS[variant], color && COLOR_CLASS[color], truncate && 'truncate', ...extra)
}

export function Typography<E extends ElementType = 'p'>(props: TypographyProps<E>) {
  const { as, variant = 'body-md-regular', color, truncate, className, ...rest } = props
  return createElement(as ?? 'p', {
    // className LAST so callers win.
    className: typography({ variant, color, truncate }, className as ClassValue),
    ...rest,
  })
}
```

app/packages/quanta/src/components/utils/cx.test.tsx

```
import { describe, expect, it } from 'vitest'
import { cx } from './cx.ts'

describe('cx', () => {
  it('joins truthy class names with a space', () => {
    expect(cx('a', 'b')).toBe('a b')
  })

  it('drops falsy values', () => {
    expect(cx('a', false, undefined, null, '', 'b')).toBe('a b')
  })
})
```

app/packages/quanta/src/components/utils/cx.ts

```
export type ClassValue = string | false | null | undefined

/**
 * Minimal class-name joiner. No tailwind-merge: components apply a single
 *  composite `menu-*` class plus an optional caller `className`.
 */
export function cx(...values: ClassValue[]): string {
  return values.filter(Boolean).join(' ')
}
```

app/packages/quanta/src/components/utils/slot.ts

```
import type { CSSProperties } from 'react'

/**
 * The slot color system's TS half (the CSS half is src/css/tailwind/slot.css).
 *
 * A component takes a single semantic `color` prop and spreads `slotStyle(color)`
 * inline, which sets the private `--q-tint*` custom properties. The `q-slot-*`
 * utilities then derive every surface from them.
 *
 * This is the TINT pattern — one of quanta's two coloring models. It fits soft
 * single-accent surfaces (toggle, tag, switch, tabs, progress, loader). Selection
 * controls (checkbox, radio, chip) deliberately use the OTHER model: a per-color
 * `q-<comp>-<color>` utility table setting richer `--q-<comp>-fill/fg/hover/ring`
 * vars (a fill identity + a bespoke glow ring the tint can't express).
 *
 * CONTRACT: every value references an `--hf-color-*` runtime primitive (emitted
 * per data-theme in src/css/theme/color.css), so light/dark is automatic. NEVER
 * use a `--color-q-*` name here — those are @theme inline-only and have no
 * runtime value, so the slot would render unstyled.
 *
 * Resilience: each saturated `--q-tint` carries a nested var() fallback to
 * `brand-primary` (always set by managed themes, the documented default) so a
 * partial `defineTheme()` override degrades gracefully instead of rendering
 * transparent. See quanta-core-plan.md §5.4 / §5.4.1.
 */

export type SlotColor = 'brand' | 'neutral' | 'success' | 'error' | 'warning' | 'info'

/** Guaranteed-present token used as the universal degrade target. */
const FALLBACK = 'var(--hf-color-brand-primary)'

type SlotVars = {
  '--q-tint': string
  '--q-tint-fg': string
  /** Only set when it must differ from `--q-tint` (the utility defaults it to `--q-tint`). */
  '--q-tint-bg'?: string
  /** Only set when it must differ from `--q-tint` (the utility defaults it to `--q-tint`). */
  '--q-tint-border'?: string
}

/** color → the four `--q-tint*` properties (all sourced from `--hf-color-*`). */
export const SLOT: Record<SlotColor, SlotVars> = {
  brand: {
    '--q-tint': 'var(--hf-color-brand-primary)',
    '--q-tint-fg': 'var(--hf-color-text-inverse)',
  },
  neutral: {
    '--q-tint': `var(--hf-color-text-primary, ${FALLBACK})`,
    '--q-tint-bg': `var(--hf-color-background-secondary-strong, ${FALLBACK})`,
    '--q-tint-fg': 'var(--hf-color-text-primary)',
    '--q-tint-border': `var(--hf-color-border-strong, ${FALLBACK})`,
  },
  success: {
    '--q-tint': `var(--hf-color-state-success-fg, ${FALLBACK})`,
    '--q-tint-fg': 'var(--hf-color-text-inverse)',
  },
  error: {
    '--q-tint': `var(--hf-color-state-error-fg, ${FALLBACK})`,
    '--q-tint-fg': 'var(--hf-color-text-inverse)',
  },
  warning: {
    '--q-tint': `var(--hf-color-state-warning-fg, ${FALLBACK})`,
    '--q-tint-fg': 'var(--hf-color-text-inverse)',
  },
  info: {
    '--q-tint': `var(--hf-color-state-info-fg, ${FALLBACK})`,
    '--q-tint-fg': 'var(--hf-color-text-inverse)',
  },
```

```
}

/**
 * Inline style object that wires a `color` prop into the slot custom properties.
 * Spread it into a component's `style`, then style surfaces with `q-slot-*`.
 *
 *   style={{ ...slotStyle(color), ...style }}
 */
export function slotStyle(color: SlotColor): CSSProperties {
  return SLOT[color] as CSSProperties
}
```

app/packages/quanta/src/components/utils/use-flip.ts

```
'use client'

import { useLayoutEffect, useRef } from 'react'

/**
 * useFlip — layout (reflow/reorder/filter) animation via the FLIP technique
 * (First, Last, Invert, Play) driven by the Web Animations API.
 *
 * Attach the returned ref to a container; mark each animating child with
 * `data-flip-key="<stable id>"`. Whenever `dependency` changes, the hook reads
 * every child's *last* box (batched, before paint, in `useLayoutEffect`),
 * inverts it to where it *was* with a `transform`, then plays back to `none` —
 * a GPU-cheap transform tween, no layout thrash. New items fade + scale in.
 *
 * Degrades cleanly: no-ops under `prefers-reduced-motion` and where
 * `Element.animate` is unavailable (SSR / older test DOMs), so layout is always
 * correct even when the motion doesn't run.
 */
export type UseFlipOptions = {
  /** Move/enter duration (ms). Default 260. */
  duration?: number
  /** CSS easing for the tween. Default a soft ease-out-back-ish curve. */
  easing?: string
  /** Fade + scale newly-added items in. Default true. */
  animateEnter?: boolean
}

const REDUCED_MOTION = '(prefers-reduced-motion: reduce)'

export function useFlip<T extends HTMLElement = HTMLDivElement>(dependency: unknown, options: UseFlipOptions = {}) {
  const { duration = 260, easing = 'cubic-bezier(0.22, 1, 0.36, 1)', animateEnter = true } = options
  const ref = useRef<T>(null)
  const prevRects = useRef<Map<string, DOMRect>>(new Map())
  const isFirstRun = useRef(true)

  useLayoutEffect(() => {
    const container = ref.current
    if (container == null) return

    const items = Array.from(container.children).filter(
      (child): child is HTMLElement => child instanceof HTMLElement && child.dataset.flipKey != null,
    )

    // Batch READS (measure every "last" box) before any WRITE, to avoid thrash.
    const nextRects = new Map<string, DOMRect>()
    for (const el of items) nextRects.set(el.dataset.flipKey as string, el.getBoundingClientRect())

    const reduced = typeof matchMedia !== 'undefined' && matchMedia(REDUCED_MOTION).matches
    const canAnimate = typeof items[0]?.animate === 'function'

    if (!isFirstRun.current && !reduced && canAnimate) {
      for (const el of items) {
        const key = el.dataset.flipKey as string
        const last = nextRects.get(key) as DOMRect
        const first = prevRects.current.get(key)

        if (first == null) {
          // New item — fade + scale in (Invert/Play has nothing to invert).
          if (animateEnter) {
            el.animate([{ opacity: 0, transform: 'scale(0.96)' }, { opacity: 1, transform: 'none' }], { duration, easing })
          }
          continue
        }

        const dx = first.left - last.left
        const dy = first.top - last.top
        // Invert to the old spot, then Play back to the natural position.
        if (dx !== 0 || dy !== 0) {
```

```
          el.animate([{ transform: `translate(${dx}px, ${dy}px)` }, { transform: 'none' }], { duration, easing })
        }
      }
    }

    prevRects.current = nextRects
    isFirstRun.current = false
  }, [dependency, duration, easing, animateEnter])

  return ref
}
```

app/packages/quanta/src/components/utils/use-in-view.ts

```
'use client'

import type { RefCallback } from 'react'
import { useCallback, useEffect, useState } from 'react'

/**
 * useInView — reports whether an element is intersecting the viewport (or a
 * scroll `root`) via `IntersectionObserver`, the main-thread-cheap way to drive
 * viewport-only behaviour: lazy reveals, and — the headline use — autoplaying a
 * video only while it is on screen (see `Media.Video autoPlayInView`).
 *
 * SSR / test safe: when `IntersectionObserver` is unavailable it no-ops and
 * reports `inView: false`. Attach the returned `ref` (a ref callback, so it
 * re-observes if the node changes) to the element you want to track.
 */
export type UseInViewOptions = {
  /** Visibility ratio (0..1) at/above which the element counts as in view. Default 0.5. */
  threshold?: number
  /** Grow/shrink the detection box, e.g. `'200px'` to pre-trigger before entry. */
  rootMargin?: string
  /** Scroll container to observe against; defaults to the browser viewport. */
  root?: Element | null
  /** Latch to `true` on first entry and stop observing (one-shot reveals). */
  once?: boolean
}

export function useInView<T extends Element = HTMLElement>(options: UseInViewOptions = {}) {
  const { threshold = 0.5, rootMargin, root = null, once = false } = options
  const [node, setNode] = useState<T | null>(null)
  const [inView, setInView] = useState(false)

  // Ref callback → state so the effect re-observes whenever React swaps the node.
  const ref = useCallback<RefCallback<T>>((next) => setNode(next), [])

  useEffect(() => {
    if (node == null || typeof IntersectionObserver === 'undefined') return

    const observer = new IntersectionObserver(
      (entries) => {
        const entry = entries[0]
        if (entry == null) return
        const visible = entry.isIntersecting && entry.intersectionRatio >= threshold
        setInView(visible)
        if (visible && once) observer.disconnect()
      },
      { threshold, rootMargin: rootMargin ?? undefined, root },
    )
    observer.observe(node)
    return () => observer.disconnect()
  }, [node, threshold, rootMargin, root, once])

  return { ref, inView } as const
}
```

app/packages/quanta/src/components/vault/index.ts

```
export { Vault } from './vault.tsx'
export type {
  VaultContentProps,
  VaultFooterProps,
  VaultHeaderProps,
  VaultRootProps,
  VaultSide,
} from './vault.tsx'
```

app/packages/quanta/src/components/vault/vault.css

```
/**
 * Vault (drawer / sheet) presentation — Base UI Drawer skinned with quanta
 * tokens. Registers the colocated source (quanta is symlinked) and authors the
 * surface as `@utility q-vault-*`. The drawer width is the one non-token value,
 * exposed as the overridable `--q-vault-width` component dimension whose default
 * lives in the generated component tier (src/css/components/component.css :root).
 * Base UI drives the drag via `--drawer-swipe-movement-*`.
 */
@source "./vault.tsx";

/* Required wrapper around the Popup — Base UI binds swipe/drag + snap handling
 * and touch scroll-locking here. A non-blocking full-screen layer; the Popup
 * keeps its own fixed edge-docking (the viewport sets no transform, so the
 * Popup's `position: fixed` resolves against the browser viewport). */
@utility q-vault-viewport {
  position: fixed;
  inset: var(--hf-space-0);
  z-index: var(--hf-z-index-modal);
  pointer-events: none;

  & > * { pointer-events: auto; }
}

@utility q-vault-backdrop {
  position: fixed;
  inset: var(--hf-space-0);
  z-index: var(--hf-z-index-overlay);
  background-color: var(--hf-color-overlay-scrim);
  /* Fade with the dismiss drag — Base UI feeds `--drawer-swipe-progress`
   * (0 at rest → 1 when dismissed) and leaves the opacity to us. */
  opacity: calc(1 - var(--drawer-swipe-progress, 0));
  transition: opacity var(--hf-duration-400) var(--hf-ease-emphasized);

  &[data-starting-style],
  &[data-ending-style] { opacity: 0; }
}

/* The docked sheet — glassy frosted surface (matches the Modal glass).
 *
 * Transform model (the per-side rules below set it): Base UI leaves `transform`
 * to us and feeds two CSS vars — `--drawer-snap-point-offset` (the resting
 * offset for the active snap point, already sign-corrected per direction) and
 * `--drawer-swipe-movement-{x,y}` (the live drag delta). The open/dragging
 * position is their SUM; the closed state overrides with a full off-edge shift.
 * `touch-action` is intentionally NOT set here — Base UI's Popup applies a
 * direction-aware value inline so touch drag + inner scroll both work. */
@utility q-vault {
  position: fixed;
  z-index: var(--hf-z-index-modal);
  display: flex;
  flex-direction: column;
  background-color: var(--hf-color-background-glass);
  backdrop-filter: blur(var(--hf-space-1000)); /* 40px — glass */
  border: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  color: var(--hf-color-text-primary);
  box-shadow:
    inset 0 var(--hf-space-050) 3px var(--hf-color-transparent-light-05), /* no token: 3px blur radius has no spacing token (4px=--hf-space-100 differs) */
    0 var(--hf-space-100) var(--hf-space-600) calc(-1 * var(--hf-space-200)) var(--hf-color-transparent-dark-15);

  /* swipe-strength (0.1–1, set by Base UI on release) scales the snap duration. */
  transition: transform calc(var(--hf-duration-400) * var(--drawer-swipe-strength, 1)) var(--hf-ease-emphasized);

  &[data-swiping] { transition: none; } /* follow the finger, no easing */

  /* Edge-fill — a glass slab that continues past the docked (square) edge so a
   * rubber-band overshoot never reveals the page. It meets the sheet's flat
   * edge (the rounded corners are on the opposite side), so corners stay clean.
   * No `overflow: hidden` on the sheet, or this would be clipped. Off-screen at
```

```
   * rest; the per-side rules give it direction. */
  &::after {
    content: "";
    position: absolute;
    background-color: var(--hf-color-background-glass);
    backdrop-filter: blur(var(--hf-space-1000));
    pointer-events: none;
  }
}

/* ■■ Edge docking: open/drag position = snap offset + live drag; closed = off-edge ■■ */
@utility q-vault-bottom {
  inset-inline: var(--hf-space-0);
  bottom: var(--hf-space-0);
  width: 100%;
  max-height: 90vh;
  border-bottom: none;
  transform: translateY(calc(var(--drawer-snap-point-offset, 0px) + var(--drawer-swipe-movement-y, 0px)));
  &[data-starting-style], &[data-ending-style] { transform: translateY(100%); }
  &::after { top: 100%; inset-inline: var(--hf-space-0); height: 100vh; } /* fill below */
}
@utility q-vault-top {
  inset-inline: var(--hf-space-0);
  top: var(--hf-space-0);
  width: 100%;
  max-height: 90vh;
  border-top: none;
  transform: translateY(calc(var(--drawer-snap-point-offset, 0px) + var(--drawer-swipe-movement-y, 0px)));
  &[data-starting-style], &[data-ending-style] { transform: translateY(-100%); }
  &::after { bottom: 100%; inset-inline: var(--hf-space-0); height: 100vh; } /* fill above */
}
@utility q-vault-left {
  inset-block: var(--hf-space-0);
  left: var(--hf-space-0);
  height: 100%;
  width: min(var(--q-vault-width), 100vw);
  border-left: none;
  transform: translateX(calc(var(--drawer-snap-point-offset, 0px) + var(--drawer-swipe-movement-x, 0px)));
  &[data-starting-style], &[data-ending-style] { transform: translateX(-100%); }
  &::after { right: 100%; inset-block: var(--hf-space-0); width: 100vw; } /* fill left */
}
@utility q-vault-right {
  inset-block: var(--hf-space-0);
  right: var(--hf-space-0);
  height: 100%;
  width: min(var(--q-vault-width), 100vw);
  border-right: none;
  transform: translateX(calc(var(--drawer-snap-point-offset, 0px) + var(--drawer-swipe-movement-x, 0px)));
  &[data-starting-style], &[data-ending-style] { transform: translateX(100%); }
  &::after { left: 100%; inset-block: var(--hf-space-0); width: 100vw; } /* fill right */
}

/* Grab handle (decorative). */
@utility q-vault-handle {
  flex: none;
  width: var(--hf-space-1000);
  height: var(--hf-space-100);
  margin: var(--hf-space-300) auto var(--hf-space-100);
  border-radius: var(--hf-radius-full);
  background-color: var(--hf-color-border-strong);
}

@utility q-vault-header {
  flex: none;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--hf-space-300);
  padding: var(--hf-space-400) var(--hf-space-500);
}
```

```
/* Leading group — start + title + end, kept together at the leading edge so the
 * close disc stays trailing (space-between). Only rendered when a flank slot set. */
@utility q-vault-header-lead {
  display: flex;
  align-items: center;
  gap: var(--hf-space-300);
  min-width: var(--hf-space-0);
}

@utility q-vault-title {
  @apply text-q-body-md-semi-bold;
  color: var(--hf-color-text-primary);
}

@utility q-vault-description {
  @apply text-q-body-sm-regular;
  color: var(--hf-color-text-secondary);
}

/* The dismiss disc is the shared CloseButton (q-close, node 2052:109). */

/* Scrollable content region. */
@utility q-vault-body {
  flex: 1 1 auto;
  min-height: var(--hf-space-0);
  overflow: auto;
  padding: var(--hf-space-0) var(--hf-space-500) var(--hf-space-500);
}

@utility q-vault-footer {
  flex: none;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--hf-space-300);
  padding: var(--hf-space-400) var(--hf-space-500);
  border-top: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
}

@utility q-vault-caption {
  @apply text-q-caption-sm-regular;
  color: var(--hf-color-text-secondary);
}

@utility q-vault-actions {
  display: flex;
  align-items: center;
  gap: var(--hf-space-200);
}

/* Full-width actions (e.g. a single primary sheet button). */
@utility q-vault-actions-full {
  flex: 1 1 var(--hf-space-0);
}

@media (prefers-reduced-motion: reduce) {
  .q-vault,
  .q-vault-backdrop { transition-duration: var(--hf-duration-120); }
  .q-vault[data-swiping] { transition: none; }
}
```

app/packages/quanta/src/components/vault/vault.test.tsx

```
import { render, screen } from '@testing-library/react'
import { describe, expect, it } from 'vitest'
import { Vault } from './index.ts'

function open(side?: Parameters<typeof Vault.Root>[0]['side']) {
  return render(
    <Vault.Root defaultOpen side={side}>
      <Vault.Content>
        <Vault.Header title="Filters" />
        <Vault.Body>Body content</Vault.Body>
        <Vault.Footer caption="2 selected" />
      </Vault.Content>
    </Vault.Root>,
  )
}

describe('<Vault>', () => {
  it('renders the docked sheet with title, body and footer', () => {
    open()
    expect(screen.getByText('Filters')).toHaveClass('q-vault-title')
    expect(screen.getByText('Body content')).toHaveClass('q-vault-body')
    expect(screen.getByText('2 selected')).toHaveClass('q-vault-caption')
  })

  it('defaults to the bottom side and shows a handle', () => {
    open()
    const popup = screen.getByText('Filters').closest('.q-vault')
    expect(popup).toHaveClass('q-vault-bottom')
    expect(popup?.querySelector('.q-vault-handle')).toBeInTheDocument()
  })

  it('docks to the requested side (no handle off-bottom)', () => {
    open('right')
    const popup = screen.getByText('Filters').closest('.q-vault')
    expect(popup).toHaveClass('q-vault-right')
    expect(popup?.querySelector('.q-vault-handle')).not.toBeInTheDocument()
  })

  it('renders a default close button in the header with an Icon glyph', () => {
    open()
    const close = screen.getByRole('button', { name: 'Close' })
    expect(close).toHaveClass('q-close')
    // Icon is node-only: the glyph svg itself carries the q-icon sizing class.
    const glyph = close.querySelector('svg.q-icon.q-icon-md')
    expect(glyph).toBeInTheDocument()
  })

  it('renders header start / end flank slots before the close button', () => {
    render(
      <Vault.Root defaultOpen>
        <Vault.Content>
          <Vault.Header title="Filters" start={<button type="button">Back</button>} end={<span data-testid="end" />} />
        </Vault.Content>
      </Vault.Root>,
    )
    const lead = document.querySelector('.q-vault-header-lead') as HTMLElement
    expect(lead).toBeInTheDocument()
    const back = screen.getByRole('button', { name: 'Back' })
    const close = screen.getByRole('button', { name: 'Close' })
    expect(lead).toContainElement(back)
    expect(lead).toContainElement(screen.getByText('Filters'))
    expect(lead).toContainElement(screen.getByTestId('end'))
    // close stays after the lead group
    expect(back.compareDocumentPosition(close) & Node.DOCUMENT_POSITION_FOLLOWING).toBeTruthy()
  })

  it('keeps the title-only header bare (no lead wrapper) — back-compat', () => {
    open()
```

```
    expect(document.querySelector('.q-vault-header-lead')).toBeNull()
    expect(screen.getByText('Filters')).toHaveClass('q-vault-title')
  })

  it('stretches actions with the full footer prop', () => {
    render(
      <Vault.Root defaultOpen>
        <Vault.Content>
          <Vault.Header title="x" />
          <Vault.Footer full actions={<button type="button">Apply</button>} />
        </Vault.Content>
      </Vault.Root>,
    )
    const actions = screen.getByRole('button', { name: 'Apply' }).closest('.q-vault-actions')
    expect(actions).toHaveClass('q-vault-actions-full')
  })

  it('does not add the full class by default — back-compat', () => {
    render(
      <Vault.Root defaultOpen>
        <Vault.Content>
          <Vault.Header title="x" />
          <Vault.Footer actions={<button type="button">Done</button>} />
        </Vault.Content>
      </Vault.Root>,
    )
    const actions = screen.getByRole('button', { name: 'Done' }).closest('.q-vault-actions')
    expect(actions).not.toHaveClass('q-vault-actions-full')
  })
})
```

app/packages/quanta/src/components/vault/vault.tsx

```
'use client'

import type { ComponentProps, ReactNode } from 'react'
import { createContext, useContext } from 'react'
import { Drawer as Primitive } from '@base-ui/react/drawer'
import { cx } from '../utils/cx.ts'
import { CloseIcon, closeButton } from '../close-button/index.ts'
import { Icon } from '../icon/index.ts'

/**
 * Vault — an edge-docked, swipeable drawer / sheet on Base UI Drawer (the Vaul
 * model), skinned with quanta tokens. Base UI owns the drag physics, snap points,
 * focus trap, scroll lock and a11y; quanta paints the surface and docks it to a
 * chosen edge. Composition mirrors Modal: Root / Trigger / Content / Header /
 * Body / Footer.
 *
 *   <Vault.Root side="bottom">
 *     <Vault.Trigger render={<Button>Open</Button>} />
 *     <Vault.Content>
 *       <Vault.Header title="Filters" />
 *       <Vault.Body>…</Vault.Body>
 *       <Vault.Footer actions={<Button>Apply</Button>} />
 *     </Vault.Content>
 *   </Vault.Root>
 */

export type VaultSide = 'bottom' | 'top' | 'left' | 'right'

/** side → which edge the popup docks to + its slide/dismiss gesture. */
const SIDE_CLASS = {
  bottom: 'q-vault-bottom',
  top: 'q-vault-top',
  left: 'q-vault-left',
  right: 'q-vault-right',
} satisfies Record<VaultSide, string>

const SWIPE_DIRECTION = {
  bottom: 'down',
  top: 'up',
  left: 'left',
  right: 'right',
} satisfies Record<VaultSide, 'up' | 'down' | 'left' | 'right'>

const VaultContext = createContext<VaultSide>('bottom')

/* ■■ Passthrough parts (Base UI owns behavior). ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
const Trigger = Primitive.Trigger
const Close = Primitive.Close

export type VaultRootProps = ComponentProps<typeof Primitive.Root> & { side?: VaultSide }

/** Owns open state + the swipe gesture for the chosen edge. */
function Root({ side = 'bottom', children, ...props }: VaultRootProps) {
  return (
    <VaultContext.Provider value={side}>
      <Primitive.Root swipeDirection={SWIPE_DIRECTION[side]} {...props}>
        {children}
      </Primitive.Root>
    </VaultContext.Provider>
  )
}

type TitleProps = Omit<ComponentProps<typeof Primitive.Title>, 'className'> & { className?: string }
function Title({ className, ...props }: TitleProps) {
  return <Primitive.Title className={cx('q-vault-title', className)} {...props} />
}

type DescriptionProps = Omit<ComponentProps<typeof Primitive.Description>, 'className'> & { className?: string }
```

```
function Description({ className, ...props }: DescriptionProps) {
  return <Primitive.Description className={cx('q-vault-description', className)} {...props} />
}

type CloseButtonProps = Omit<ComponentProps<typeof Primitive.Close>, 'className'> & { className?: string }
function CloseButton({ className, children, ...props }: CloseButtonProps) {
  return (
    <Primitive.Close aria-label="Close" className={closeButton({}, className)} {...props}>
      {children ?? <Icon as={CloseIcon} size="md" />}
    </Primitive.Close>
  )
}

export type VaultContentProps = Omit<ComponentProps<typeof Primitive.Popup>, 'className'> & {
  className?: string
  /** Override the side from Root (rarely needed). */
  side?: VaultSide
  /** Show the grab handle. Defaults to true for the `bottom` side. */
  handle?: boolean
  backdropClassName?: string
  container?: ComponentProps<typeof Primitive.Portal>['container']
}

/**
 * Portal + Backdrop + Viewport + the edge-docked, swipeable Popup. The
 * `Drawer.Viewport` is REQUIRED — it's what enables Base UI's swipe/drag,
 * snap-point handling and touch scroll-locking (without it the Popup renders
 * but is undraggable).
 */
function Content({ side: sideProp, handle, backdropClassName, container, className, children, ...props }: VaultContentProps) {
  const ctxSide = useContext(VaultContext)
  const side = sideProp ?? ctxSide
  const showHandle = handle ?? side === 'bottom'
  return (
    <Primitive.Portal container={container}>
      <Primitive.Backdrop className={cx('q-vault-backdrop', backdropClassName)} />
      <Primitive.Viewport className="q-vault-viewport">
        <Primitive.Popup className={cx('q-vault', SIDE_CLASS[side], className)} {...props}>
          {showHandle ? <span className="q-vault-handle" aria-hidden /> : null}
          {children}
        </Primitive.Popup>
      </Primitive.Viewport>
    </Primitive.Portal>
  )
}

export type VaultHeaderProps = Omit<ComponentProps<'div'>, 'title'> & {
  title?: ReactNode
  /** Leading slot — a back button / icon / avatar before the title. Any node. */
  start?: ReactNode
  /** Trailing slot — sits just before the close affordance. Any node. */
  end?: ReactNode
  /** Close affordance: `true` (default button), `false` (none), or a custom node. */
  closeButton?: ReactNode | boolean
}

function Header({ title, start, end, closeButton = true, children, className, ...props }: VaultHeaderProps) {
  const close = closeButton === true ? <CloseButton /> : closeButton === false ? null : closeButton
  const titleNode = title != null ? <Title>{title}</Title> : null
  // start/end flank the title in a leading group (space-between keeps close at
  // the trailing edge). Without a flank slot, the bare title/spacer is unchanged.
  let lead: ReactNode
  if (children != null)
    lead = children
  else if (start != null || end != null)
    lead = <div className="q-vault-header-lead">{start}{titleNode}{end}</div>
  else
    lead = titleNode ?? <span />
  return (
    <div className={cx('q-vault-header', className)} {...props}>
```

```
      {lead}
      {close}
    </div>
  )
}

type BodyProps = ComponentProps<'div'>
/** Scrollable content region. */
function Body({ className, ...props }: BodyProps) {
  return <div className={cx('q-vault-body', className)} {...props} />
}

export type VaultFooterProps = ComponentProps<'div'> & {
  caption?: ReactNode
  actions?: ReactNode
  /** Stretch the actions to fill the footer width (full-width sheet buttons). */
  full?: boolean
}
function Footer({ caption, actions, full = false, children, className, ...props }: VaultFooterProps) {
  return (
    <div className={cx('q-vault-footer', className)} {...props}>
      {children ?? (
        <>
          {caption != null ? <span className="q-vault-caption">{caption}</span> : null}
          {actions != null ? <div className={cx('q-vault-actions', full && 'q-vault-actions-full')}>{actions}</div> : null}
        </>
      )}
    </div>
  )
}

export const Vault = {
  Root,
  Trigger,
  Content,
  Header,
  Body,
  Footer,
  Title,
  Description,
  Close,
  CloseButton,
}
```

app/packages/quanta/src/css/components/component.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --q-menu-max-width: calc(var(--hf-space-2400) * 4 + var(--hf-space-400));
  --q-menu-min-width: calc(var(--hf-space-2400) + var(--hf-space-1600));
  --q-modal-width: 40rem;
  --q-modal-width-2xl: 87rem;
  --q-modal-width-lg: 44.25rem;
  --q-modal-width-md: 40rem;
  --q-modal-width-sm: 29.3125rem;
  --q-modal-width-xl: 59.125rem;
  --q-modal-width-xs: 21.8125rem;
  --q-nav-col-width: calc(var(--hf-space-2400) * 2);
  --q-sidebar-width: 14.9375rem;
  --q-sidebar-width-collapsed: 3.25rem;
  --q-slider-width: 11.5rem;
  --q-sonner-width: 22.5rem;
  --q-textarea-min-height: 10.25rem;
  --q-vault-width: 22rem;
}
```

app/packages/quanta/src/css/primitives/border-width.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-border-width-none: 0px;
  --hf-border-width-hairline: 0.5px;
  --hf-border-width-thin: 1px;
  --hf-border-width-medium: 1.5px;
  --hf-border-width-thick: 2px;
}
```

app/packages/quanta/src/css/primitives/breakpoint.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-breakpoint-mobile: 20rem;
  --hf-breakpoint-tablet: 48rem;
  --hf-breakpoint-desktop: 80rem;
  --hf-breakpoint-wide: 120rem;
}
```

app/packages/quanta/src/css/primitives/color.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-color-blue-100: #ebf1ff;
  --hf-color-blue-1000: #001640;
  --hf-color-blue-1100: #000d26;
  --hf-color-blue-1200: #000714;
  --hf-color-blue-200: #ccddff;
  --hf-color-blue-300: #9fbfff;
  --hf-color-blue-400: #5b91fe;
  --hf-color-blue-500: #0256fe;
  --hf-color-blue-600: #0249d8;
  --hf-color-blue-700: #013aad;
  --hf-color-blue-800: #012d84;
  --hf-color-blue-900: #012161;

  --hf-color-cool-050: #5c626a;
  --hf-color-cool-100: #484e56;
  --hf-color-cool-150: #383e46;
  --hf-color-cool-200: #2a2d32;
  --hf-color-cool-250: #23262a;
  --hf-color-cool-300: #1c1e21;
  --hf-color-cool-350: #18191c;
  --hf-color-cool-400: #131416;

  --hf-color-cyan-100: #f7fdfe;
  --hf-color-cyan-1000: #273a3d;
  --hf-color-cyan-1100: #172324;
  --hf-color-cyan-1200: #0c1213;
  --hf-color-cyan-200: #ebfafd;
  --hf-color-cyan-300: #d9f6fa;
  --hf-color-cyan-400: #bfeff7;
  --hf-color-cyan-500: #9ce6f3;
  --hf-color-cyan-600: #85c4cf;
  --hf-color-cyan-700: #6a9ca5;
  --hf-color-cyan-800: #51787e;
  --hf-color-cyan-900: #3b575c;

  --hf-color-glass-dark: #23262abf;
  --hf-color-glass-light: #f4f4f4bf;

  --hf-color-green-100: #e6ffea;
  --hf-color-green-1000: #082b0d;
  --hf-color-green-1100: #061f09;
  --hf-color-green-1200: #041506;
  --hf-color-green-200: #bbf7c4;
  --hf-color-green-300: #86f998;
  --hf-color-green-400: #4ee466;
  --hf-color-green-500: #2eb844;
  --hf-color-green-600: #009919;
  --hf-color-green-700: #016f13;
  --hf-color-green-800: #0d4a17;
  --hf-color-green-900: #003d0a;
  --hf-color-green-glow: #00e62e;

  --hf-color-grey-050: #ffffff;
  --hf-color-grey-100: #f4f4f4;
  --hf-color-grey-150: #cecece;
  --hf-color-grey-200: #a8a8a8;
  --hf-color-grey-250: #7f7f7f;
  --hf-color-grey-300: #828282;
  --hf-color-grey-325: #626262;
  --hf-color-grey-350: #565656;
```

```
--hf-color-grey-400: #454545;
--hf-color-grey-450: #323232;
--hf-color-grey-500: #2a2a2a;
--hf-color-grey-550: #1a1a1a;
--hf-color-grey-600: #000000;

--hf-color-lime-100: #fbffec;
--hf-color-lime-1000: #344006;
--hf-color-lime-1100: #1f2603;
--hf-color-lime-1200: #111402;
--hf-color-lime-200: #f6ffd1;
--hf-color-lime-300: #eeffa7;
--hf-color-lime-400: #e1fe68;
--hf-color-lime-500: #d1fe17;
--hf-color-lime-600: #b2d814;
--hf-color-lime-700: #8ead10;
--hf-color-lime-800: #6d840c;
--hf-color-lime-900: #4f6109;
--hf-color-lime-alpha-05: #d1fe170d;
--hf-color-lime-alpha-10: #d1fe171a;
--hf-color-lime-alpha-100: #d1fe17;
--hf-color-lime-alpha-15: #d1fe1726;
--hf-color-lime-alpha-20: #d1fe1733;
--hf-color-lime-alpha-30: #d1fe174d;
--hf-color-lime-alpha-40: #d1fe1766;
--hf-color-lime-alpha-50: #d1fe1780;
--hf-color-lime-alpha-60: #d1fe1799;
--hf-color-lime-alpha-70: #d1fe17b2;
--hf-color-lime-alpha-80: #d1fe17cc;
--hf-color-lime-alpha-90: #d1fe17e5;

--hf-color-neutral-050: #ffffff;
--hf-color-neutral-100: #ededed;
--hf-color-neutral-150: #b4b4b4;
--hf-color-neutral-200: #4a4a4a;
--hf-color-neutral-250: #3e3e3e;
--hf-color-neutral-300: #272727;
--hf-color-neutral-350: #232323;
--hf-color-neutral-400: #1e1e1e;
--hf-color-neutral-450: #1d1d1d;
--hf-color-neutral-500: #171717;
--hf-color-neutral-550: #0e0e0e;
--hf-color-neutral-600: #000000;

--hf-color-orange-100: #fff8f5;
--hf-color-orange-1000: #8f2b00;
--hf-color-orange-1100: #732200;
--hf-color-orange-1200: #571a00;
--hf-color-orange-200: #ffdbcc;
--hf-color-orange-300: #ffd1bd;
--hf-color-orange-400: #ffb494;
--hf-color-orange-500: #ff773d;
--hf-color-orange-600: #ff4f03;
--hf-color-orange-700: #db4200;
--hf-color-orange-800: #c73c00;
--hf-color-orange-900: #ad3400;

--hf-color-pink-100: #ffebf2;
--hf-color-pink-1000: #400017;
--hf-color-pink-1100: #26000e;
--hf-color-pink-1200: #140007;
--hf-color-pink-200: #ffccde;
--hf-color-pink-300: #ff9ec1;
--hf-color-pink-400: #ff5994;
--hf-color-pink-500: #ff005b;
--hf-color-pink-600: #d9004d;
--hf-color-pink-700: #ad003e;
--hf-color-pink-800: #85002f;
--hf-color-pink-900: #610023;
--hf-color-pink-alpha-05: #ff005b0d;
```

```
--hf-color-pink-alpha-10: #ff005b1a;
--hf-color-pink-alpha-100: #ff005b;
--hf-color-pink-alpha-15: #ff005b26;
--hf-color-pink-alpha-20: #ff005b33;
--hf-color-pink-alpha-30: #ff005b4d;
--hf-color-pink-alpha-40: #ff005b66;
--hf-color-pink-alpha-50: #ff005b80;
--hf-color-pink-alpha-60: #ff005b99;
--hf-color-pink-alpha-70: #ff005bb2;
--hf-color-pink-alpha-80: #ff005bcc;
--hf-color-pink-alpha-90: #ff005be5;

--hf-color-purple-100: #ebecfe;
--hf-color-purple-1000: #361f83;
--hf-color-purple-1100: #2b1b6a;
--hf-color-purple-1200: #221358;
--hf-color-purple-200: #d5d5fc;
--hf-color-purple-300: #b9b8f5;
--hf-color-purple-400: #9e99f7;
--hf-color-purple-500: #8271f8;
--hf-color-purple-600: #7152f4;
--hf-color-purple-700: #623be2;
--hf-color-purple-800: #5230c2;
--hf-color-purple-900: #442aa1;

--hf-color-red-100: #fff0f1;
--hf-color-red-1000: #5c000f;
--hf-color-red-1100: #45000b;
--hf-color-red-1200: #320008;
--hf-color-red-200: #ffdfe1;
--hf-color-red-300: #ffc2c6;
--hf-color-red-400: #ff969d;
--hf-color-red-500: #ff5462;
--hf-color-red-600: #fa0019;
--hf-color-red-700: #d60016;
--hf-color-red-800: #9b0018;
--hf-color-red-900: #7a0013;
--hf-color-red-glow: #ff1f2e;

--hf-color-transparent-dark-05: #0000000d;
--hf-color-transparent-dark-10: #0000001a;
--hf-color-transparent-dark-100: #000000;
--hf-color-transparent-dark-12: #0000001f;
--hf-color-transparent-dark-15: #00000026;
--hf-color-transparent-dark-20: #00000033;
--hf-color-transparent-dark-25: #00000040;
--hf-color-transparent-dark-30: #0000004d;
--hf-color-transparent-dark-32: #00000052;
--hf-color-transparent-dark-40: #00000066;
--hf-color-transparent-dark-50: #00000080;
--hf-color-transparent-dark-60: #00000099;
--hf-color-transparent-dark-70: #000000b2;
--hf-color-transparent-dark-80: #000000cc;
--hf-color-transparent-dark-90: #000000e5;
--hf-color-transparent-light-05: #ffffff0d;
--hf-color-transparent-light-10: #ffffff1a;
--hf-color-transparent-light-100: #ffffff;
--hf-color-transparent-light-15: #ffffff26;
--hf-color-transparent-light-20: #ffffff33;
--hf-color-transparent-light-30: #ffffff4d;
--hf-color-transparent-light-40: #ffffff66;
--hf-color-transparent-light-50: #ffffff80;
--hf-color-transparent-light-60: #ffffff99;
--hf-color-transparent-light-70: #ffffffb2;
--hf-color-transparent-light-80: #ffffffcc;
--hf-color-transparent-light-90: #ffffffe5;

--hf-color-yellow-100: #fffde5;
--hf-color-yellow-1000: #6e5500;
--hf-color-yellow-1100: #523f00;
```

```
  --hf-color-yellow-1200: #3a2d00;
  --hf-color-yellow-200: #fffac2;
  --hf-color-yellow-300: #fff599;
  --hf-color-yellow-400: #fff266;
  --hf-color-yellow-500: #ffef33;
  --hf-color-yellow-600: #ffeb00;
  --hf-color-yellow-700: #dfab01;
  --hf-color-yellow-800: #b58b00;
  --hf-color-yellow-900: #8f6e00;
  --hf-color-yellow-glow: #fff05a;
}
```

app/packages/quanta/src/css/primitives/gradient.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-gradient-badge-blue: radial-gradient(ellipse 39.43% 136.54% at 51.64% 117.31%, rgb(79 201 220) 0%, rgb(50 162 238) 50%, rgb(22 123 255) 100%);
  --hf-gradient-badge-pink: radial-gradient(ellipse 39.43% 136.54% at 51.64% 117.31%, rgb(238 69 150) 0%, rgb(237 45 132) 50%, rgb(237 21 114) 100%);
  --hf-gradient-badge-purple: radial-gradient(ellipse 39.43% 136.54% at 51.64% 117.31%, rgb(238 69 150) 0%, rgb(187 45 194) 50%, rgb(136 21 237) 100%);
  --hf-gradient-glass-sheen: linear-gradient(148.916deg, rgba(255, 255, 255, 0.04) 22.606%, rgba(255, 255, 255, 0.06) 51.256%, rgba(255, 255, 255, 0.02) 79.906%), linear-gradient(90deg, rgba(35, 38, 4
  --hf-gradient-glass-sheen-alt: linear-gradient(123.898deg, rgba(255, 255, 255, 0.04) 22.606%, rgba(255, 255, 255, 0.06) 51.256%, rgba(255, 255, 255, 0.02) 79.906%), linear-gradient(90deg, rgba(35, 3
  --hf-gradient-loader-sweep: linear-gradient(105deg, transparent 35%, var(--_c) 50%, transparent 65%);
  --hf-gradient-special-gloss-lime: linear-gradient(rgba(255, 255, 20, 0) 0%, rgb(255, 255, 20) 100%);
  --hf-gradient-special-gloss-lime-angled: linear-gradient(191.434deg, rgba(255, 255, 20, 0) 12.723%, rgb(255, 255, 20) 75.8%);
  --hf-gradient-special-gloss-pink: linear-gradient(191.434deg, rgba(243, 100, 228, 0) 12.723%, rgb(250, 74, 231) 75.8%);
}
```

app/packages/quanta/src/css/primitives/icon.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-icon-xs: 0.75rem;
  --hf-icon-sm: 1rem;
  --hf-icon-md: 1.25rem;
  --hf-icon-lg: 1.5rem;
  --hf-icon-xl: 1.75rem;
}
```

app/packages/quanta/src/css/primitives/index.css

```
/**
 * Layer 1 — Primitives (manifest)
 *
 * The raw hf-* palette and other primitive categories. Final values that
 * reference nothing. Single source of truth for the whole system.
 *
 * One file per category, but only this manifest is exported
 * (`@higgsfield/quanta/primitives.css`). Granular per-category paths are
 * not exposed — need a subset? Import the manifest and tree-shake on the
 * consumer side, or add an export when a use case appears.
 */

@import "./color.css";
@import "./border-width.css";
@import "./breakpoint.css";
@import "./radius.css";
@import "./spacing.css";
@import "./shadow.css";
@import "./typography.css";
@import "./z-index.css";
@import "./motion.css";
@import "./icon.css";
@import "./opacity.css";
@import "./gradient.css";

/* Generated component-override tier — global `--q-<comp>-<dim>` default knobs
 * on :root. Non-Tailwind consumers get the override variables here too. */
@import "../components/component.css";
```

app/packages/quanta/src/css/primitives/motion.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-duration-instant: 0ms;
  --hf-duration-fast: 100ms;
  --hf-duration-normal: 200ms;
  --hf-duration-slow: 300ms;
  --hf-duration-slower: 500ms;
  --hf-duration-80: 80ms;
  --hf-duration-120: 120ms;
  --hf-duration-130: 130ms;
  --hf-duration-150: 150ms;
  --hf-duration-160: 160ms;
  --hf-duration-180: 180ms;
  --hf-duration-220: 220ms;
  --hf-duration-260: 260ms;
  --hf-duration-350: 350ms;
  --hf-duration-400: 400ms;
  --hf-duration-700: 700ms;
  --hf-duration-800: 800ms;
  --hf-duration-1000: 1000ms;
  --hf-duration-1200: 1200ms;
  --hf-duration-1300: 1300ms;
  --hf-duration-1400: 1400ms;
  --hf-ease-in: cubic-bezier(0.4, 0, 1, 1);
  --hf-ease-out: cubic-bezier(0, 0, 0.2, 1);
  --hf-ease-in-out: cubic-bezier(0.4, 0, 0.2, 1);
  --hf-ease-linear: linear;
  --hf-ease-spring: cubic-bezier(0.34, 1.56, 0.64, 1);
  --hf-ease-out-expo: cubic-bezier(0.16, 1, 0.3, 1);
  --hf-ease-emphasized: cubic-bezier(0.32, 0.72, 0, 1);
  --hf-ease-swift: cubic-bezier(0.2, 0, 0, 1);
  --hf-ease-in-out-circ: cubic-bezier(0.65, 0, 0.35, 1);
  --hf-ease-out-quart: cubic-bezier(0.22, 1, 0.36, 1);
}
```

app/packages/quanta/src/css/primitives/opacity.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-opacity-disabled: 0.5;
  --hf-opacity-muted: 0.4;
}
```

app/packages/quanta/src/css/primitives/radius.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-radius-0: 0;
  --hf-radius-050: 0.125rem;
  --hf-radius-100: 0.25rem;
  --hf-radius-150: 0.375rem;
  --hf-radius-200: 0.5rem;
  --hf-radius-250: 0.625rem;
  --hf-radius-300: 0.75rem;
  --hf-radius-400: 1rem;
  --hf-radius-500: 1.25rem;
  --hf-radius-600: 1.5rem;
  --hf-radius-full: 9999px;
}
```

app/packages/quanta/src/css/primitives/shadow.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-shadow-inset-tile: inset 0 var(--hf-space-050) var(--hf-space-150) var(--hf-space-0) var(--hf-color-transparent-light-15), 0 var(--hf-space-050) var(--hf-space-050) calc(-1 * var(--hf-space-050
  --hf-shadow-inset-tile-hairline: inset 0 var(--hf-space-050) var(--hf-space-150) var(--hf-space-0) var(--hf-color-transparent-light-15), 0 var(--hf-space-050) var(--hf-border-width-medium) -0.5px va
  --hf-shadow-modal: inset 0 var(--hf-space-050) calc(var(--hf-space-050) + var(--hf-border-width-thin)) var(--hf-space-0) var(--hf-color-transparent-light-05), 0 var(--hf-space-050) var(--hf-space-10
  --hf-shadow-overlay: inset 0 var(--hf-space-050) calc(var(--hf-space-050) + var(--hf-border-width-thin)) var(--hf-space-0) var(--hf-color-transparent-light-05), 0 var(--hf-space-100) var(--hf-space-
  --hf-shadow-raised: inset 0 var(--hf-space-050) calc(var(--hf-space-050) + var(--hf-border-width-thin)) var(--hf-space-0) var(--hf-color-transparent-light-05), var(--hf-space-0) var(--hf-space-100)
  --hf-shadow-raised-sm: 0 var(--hf-border-width-thick) var(--hf-space-100) var(--hf-space-0) var(--hf-color-transparent-dark-10);
  --hf-shadow-sheen: inset 0 var(--hf-space-050) calc(var(--hf-space-050) + var(--hf-border-width-thin)) var(--hf-space-0) var(--hf-color-transparent-light-05);
  --hf-shadow-thumb-soft: 0 var(--hf-border-width-thin) var(--hf-space-050) var(--hf-space-0) var(--hf-color-transparent-dark-15);
  --hf-shadow-thumb-strong: 0 var(--hf-border-width-thin) var(--hf-space-300) var(--hf-space-0) var(--hf-color-transparent-dark-30);
}
```

app/packages/quanta/src/css/primitives/spacing.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-space-0: 0;
  --hf-space-050: 0.125rem;
  --hf-space-100: 0.25rem;
  --hf-space-150: 0.375rem;
  --hf-space-200: 0.5rem;
  --hf-space-250: 0.625rem;
  --hf-space-300: 0.75rem;
  --hf-space-400: 1rem;
  --hf-space-500: 1.25rem;
  --hf-space-600: 1.5rem;
  --hf-space-650: 1.5rem;
  --hf-space-700: 1.75rem;
  --hf-space-800: 2rem;
  --hf-space-1000: 2.5rem;
  --hf-space-1200: 3rem;
  --hf-space-1400: 3.5rem;
  --hf-space-1600: 4rem;
  --hf-space-2000: 5rem;
  --hf-space-2400: 6rem;
}
```

app/packages/quanta/src/css/primitives/typography.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 *
 * TEMPLATE DIVERGENCE (deliberate, hand-edited): upstream codegen emits
 * --hf-type-family-primary-base: "Inter Display", but the product never
 * renders Inter Display — higgsfield.ai's rendered titles are Space Grotesk.
 * This template does not run `tokens:emit`, so the primary family is
 * re-pointed to "Space Grotesk" here so display/headline/title utilities
 * match the product's rendered look out of the box. If codegen is ever run
 * against this file, re-apply this override.
 */

:root {
  --hf-type-family-grotesk-base: "Space Grotesk";
  --hf-type-family-mono-base: "IBM Plex Mono";
  --hf-type-family-primary-base: "Space Grotesk";
  --hf-type-family-secondary-base: "Inter";

  --hf-type-weight-black: 900;
  --hf-type-weight-bold: 700;
  --hf-type-weight-medium: 500;
  --hf-type-weight-regular: 400;
  --hf-type-weight-semi-bold: 600;

  --hf-type-letter-spacing-loose: 0.00625rem;
  --hf-type-letter-spacing-none: 0;
  --hf-type-letter-spacing-slight: -0.025rem;
  --hf-type-letter-spacing-tight: -0.075rem;
  --hf-type-letter-spacing-wide: 0.0125rem;
}
```

app/packages/quanta/src/css/primitives/z-index.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:root {
  --hf-z-index-base: 0;
  --hf-z-index-dropdown: 10;
  --hf-z-index-sticky: 20;
  --hf-z-index-overlay: 30;
  --hf-z-index-modal: 40;
  --hf-z-index-popover: 50;
  --hf-z-index-toast: 60;
  --hf-z-index-tooltip: 70;
}
```

app/packages/quanta/src/css/tailwind/border-width.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@theme {
  --border-width-q-hairline: var(--hf-border-width-hairline);
  --border-width-q-thin: var(--hf-border-width-thin);
  --border-width-q-medium: var(--hf-border-width-medium);
  --border-width-q-thick: var(--hf-border-width-thick);
}
```

app/packages/quanta/src/css/tailwind/breakpoint.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@theme {
  --breakpoint-q-tablet: 48rem;
  --breakpoint-q-desktop: 80rem;
  --breakpoint-q-wide: 120rem;
}
```

app/packages/quanta/src/css/tailwind/color.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

/* semantic → Tailwind */
@theme inline {
  --color-q-background-elevated-end: var(--hf-color-background-elevated-end);
  --color-q-background-elevated-start: var(--hf-color-background-elevated-start);
  --color-q-background-glass: var(--hf-color-background-glass);
  --color-q-background-inverse: var(--hf-color-background-inverse);
  --color-q-background-primary: var(--hf-color-background-primary);
  --color-q-background-secondary: var(--hf-color-background-secondary);
  --color-q-background-secondary-strong: var(--hf-color-background-secondary-strong);
  --color-q-background-tertiary: var(--hf-color-background-tertiary);

  --color-q-border-default: var(--hf-color-border-default);
  --color-q-border-error: var(--hf-color-border-error);
  --color-q-border-focus: var(--hf-color-border-focus);
  --color-q-border-inverse: var(--hf-color-border-inverse);
  --color-q-border-strong: var(--hf-color-border-strong);
  --color-q-border-subtle: var(--hf-color-border-subtle);
  --color-q-border-success: var(--hf-color-border-success);
  --color-q-border-warning: var(--hf-color-border-warning);

  --color-q-brand-blue: var(--hf-color-brand-blue);
  --color-q-brand-cyan: var(--hf-color-brand-cyan);
  --color-q-brand-green: var(--hf-color-brand-green);
  --color-q-brand-orange: var(--hf-color-brand-orange);
  --color-q-brand-pink: var(--hf-color-brand-pink);
  --color-q-brand-primary: var(--hf-color-brand-primary);
  --color-q-brand-red: var(--hf-color-brand-red);
  --color-q-brand-yellow: var(--hf-color-brand-yellow);
  --color-q-brand-yellow-light: var(--hf-color-brand-yellow-light);

  --color-q-button-brand: var(--hf-color-button-brand);
  --color-q-button-destructive: var(--hf-color-button-destructive);
  --color-q-button-label-brand: var(--hf-color-button-label-brand);
  --color-q-button-label-destructive: var(--hf-color-button-label-destructive);
  --color-q-button-primary: var(--hf-color-button-primary);
  --color-q-button-secondary: var(--hf-color-button-secondary);
  --color-q-button-tertiary: var(--hf-color-button-tertiary);

  --color-q-icon-accent: var(--hf-color-icon-accent);
  --color-q-icon-brand: var(--hf-color-icon-brand);
  --color-q-icon-disabled: var(--hf-color-icon-disabled);
  --color-q-icon-error: var(--hf-color-icon-error);
  --color-q-icon-info: var(--hf-color-icon-info);
  --color-q-icon-inverse: var(--hf-color-icon-inverse);
  --color-q-icon-on-overlay-secondary: var(--hf-color-icon-on-overlay-secondary);
  --color-q-icon-on-overlay-tertiary: var(--hf-color-icon-on-overlay-tertiary);
  --color-q-icon-primary: var(--hf-color-icon-primary);
  --color-q-icon-secondary: var(--hf-color-icon-secondary);
  --color-q-icon-success: var(--hf-color-icon-success);
  --color-q-icon-tertiary: var(--hf-color-icon-tertiary);
  --color-q-icon-warning: var(--hf-color-icon-warning);

  --color-q-overlay-dim-soft: var(--hf-color-overlay-dim-soft);
  --color-q-overlay-dim-strong: var(--hf-color-overlay-dim-strong);
  --color-q-overlay-hover: var(--hf-color-overlay-hover);
  --color-q-overlay-scrim: var(--hf-color-overlay-scrim);
  --color-q-overlay-wash: var(--hf-color-overlay-wash);

  --color-q-palette-blue-bg: var(--hf-color-palette-blue-bg);
  --color-q-palette-blue-text: var(--hf-color-palette-blue-text);
  --color-q-palette-brown-bg: var(--hf-color-palette-brown-bg);
```

```
--color-q-palette-brown-text: var(--hf-color-palette-brown-text);
--color-q-palette-mint-bg: var(--hf-color-palette-mint-bg);
--color-q-palette-mint-text: var(--hf-color-palette-mint-text);
--color-q-palette-orange-bg: var(--hf-color-palette-orange-bg);
--color-q-palette-orange-text: var(--hf-color-palette-orange-text);
--color-q-palette-pink-bg: var(--hf-color-palette-pink-bg);
--color-q-palette-pink-text: var(--hf-color-palette-pink-text);
--color-q-palette-purple-bg: var(--hf-color-palette-purple-bg);
--color-q-palette-purple-text: var(--hf-color-palette-purple-text);
--color-q-palette-white: var(--hf-color-palette-white);

--color-q-shadow-2xl: var(--hf-color-shadow-2xl);
--color-q-shadow-lg: var(--hf-color-shadow-lg);
--color-q-shadow-md: var(--hf-color-shadow-md);
--color-q-shadow-sm: var(--hf-color-shadow-sm);
--color-q-shadow-xl: var(--hf-color-shadow-xl);

--color-q-state-error-bg: var(--hf-color-state-error-bg);
--color-q-state-error-fg: var(--hf-color-state-error-fg);
--color-q-state-error-fg-soft: var(--hf-color-state-error-fg-soft);
--color-q-state-error-glow: var(--hf-color-state-error-glow);
--color-q-state-info-bg: var(--hf-color-state-info-bg);
--color-q-state-info-fg: var(--hf-color-state-info-fg);
--color-q-state-info-fg-soft: var(--hf-color-state-info-fg-soft);
--color-q-state-success-bg: var(--hf-color-state-success-bg);
--color-q-state-success-fg: var(--hf-color-state-success-fg);
--color-q-state-success-fg-soft: var(--hf-color-state-success-fg-soft);
--color-q-state-success-glow: var(--hf-color-state-success-glow);
--color-q-state-warning-bg: var(--hf-color-state-warning-bg);
--color-q-state-warning-fg: var(--hf-color-state-warning-fg);
--color-q-state-warning-fg-soft: var(--hf-color-state-warning-fg-soft);
--color-q-state-warning-glow: var(--hf-color-state-warning-glow);

--color-q-text-brand: var(--hf-color-text-brand);
--color-q-text-danger: var(--hf-color-text-danger);
--color-q-text-disabled: var(--hf-color-text-disabled);
--color-q-text-info: var(--hf-color-text-info);
--color-q-text-inverse: var(--hf-color-text-inverse);
--color-q-text-link: var(--hf-color-text-link);
--color-q-text-on-overlay-secondary: var(--hf-color-text-on-overlay-secondary);
--color-q-text-on-overlay-tertiary: var(--hf-color-text-on-overlay-tertiary);
--color-q-text-primary: var(--hf-color-text-primary);
--color-q-text-secondary: var(--hf-color-text-secondary);
--color-q-text-success: var(--hf-color-text-success);
--color-q-text-tertiary: var(--hf-color-text-tertiary);
--color-q-text-warning: var(--hf-color-text-warning);

--color-q-transparent-lime-05: var(--hf-color-transparent-lime-05);
--color-q-transparent-lime-10: var(--hf-color-transparent-lime-10);
--color-q-transparent-lime-100: var(--hf-color-transparent-lime-100);
--color-q-transparent-lime-15: var(--hf-color-transparent-lime-15);
--color-q-transparent-lime-20: var(--hf-color-transparent-lime-20);
--color-q-transparent-lime-30: var(--hf-color-transparent-lime-30);
--color-q-transparent-lime-40: var(--hf-color-transparent-lime-40);
--color-q-transparent-lime-50: var(--hf-color-transparent-lime-50);
--color-q-transparent-lime-60: var(--hf-color-transparent-lime-60);
--color-q-transparent-lime-70: var(--hf-color-transparent-lime-70);
--color-q-transparent-lime-80: var(--hf-color-transparent-lime-80);
--color-q-transparent-lime-90: var(--hf-color-transparent-lime-90);
--color-q-transparent-pink-05: var(--hf-color-transparent-pink-05);
--color-q-transparent-pink-10: var(--hf-color-transparent-pink-10);
--color-q-transparent-pink-100: var(--hf-color-transparent-pink-100);
--color-q-transparent-pink-15: var(--hf-color-transparent-pink-15);
--color-q-transparent-pink-20: var(--hf-color-transparent-pink-20);
--color-q-transparent-pink-30: var(--hf-color-transparent-pink-30);
--color-q-transparent-pink-40: var(--hf-color-transparent-pink-40);
--color-q-transparent-pink-50: var(--hf-color-transparent-pink-50);
--color-q-transparent-pink-60: var(--hf-color-transparent-pink-60);
--color-q-transparent-pink-70: var(--hf-color-transparent-pink-70);
--color-q-transparent-pink-80: var(--hf-color-transparent-pink-80);
```

```
  --color-q-transparent-pink-90: var(--hf-color-transparent-pink-90);
}

/* primitives → Tailwind */
@theme inline {
  --color-q-blue-100: var(--hf-color-blue-100);
  --color-q-blue-1000: var(--hf-color-blue-1000);
  --color-q-blue-1100: var(--hf-color-blue-1100);
  --color-q-blue-1200: var(--hf-color-blue-1200);
  --color-q-blue-200: var(--hf-color-blue-200);
  --color-q-blue-300: var(--hf-color-blue-300);
  --color-q-blue-400: var(--hf-color-blue-400);
  --color-q-blue-500: var(--hf-color-blue-500);
  --color-q-blue-600: var(--hf-color-blue-600);
  --color-q-blue-700: var(--hf-color-blue-700);
  --color-q-blue-800: var(--hf-color-blue-800);
  --color-q-blue-900: var(--hf-color-blue-900);

  --color-q-cool-050: var(--hf-color-cool-050);
  --color-q-cool-100: var(--hf-color-cool-100);
  --color-q-cool-150: var(--hf-color-cool-150);
  --color-q-cool-200: var(--hf-color-cool-200);
  --color-q-cool-250: var(--hf-color-cool-250);
  --color-q-cool-300: var(--hf-color-cool-300);
  --color-q-cool-350: var(--hf-color-cool-350);
  --color-q-cool-400: var(--hf-color-cool-400);

  --color-q-cyan-100: var(--hf-color-cyan-100);
  --color-q-cyan-1000: var(--hf-color-cyan-1000);
  --color-q-cyan-1100: var(--hf-color-cyan-1100);
  --color-q-cyan-1200: var(--hf-color-cyan-1200);
  --color-q-cyan-200: var(--hf-color-cyan-200);
  --color-q-cyan-300: var(--hf-color-cyan-300);
  --color-q-cyan-400: var(--hf-color-cyan-400);
  --color-q-cyan-500: var(--hf-color-cyan-500);
  --color-q-cyan-600: var(--hf-color-cyan-600);
  --color-q-cyan-700: var(--hf-color-cyan-700);
  --color-q-cyan-800: var(--hf-color-cyan-800);
  --color-q-cyan-900: var(--hf-color-cyan-900);

  --color-q-glass-dark: var(--hf-color-glass-dark);
  --color-q-glass-light: var(--hf-color-glass-light);

  --color-q-green-100: var(--hf-color-green-100);
  --color-q-green-1000: var(--hf-color-green-1000);
  --color-q-green-1100: var(--hf-color-green-1100);
  --color-q-green-1200: var(--hf-color-green-1200);
  --color-q-green-200: var(--hf-color-green-200);
  --color-q-green-300: var(--hf-color-green-300);
  --color-q-green-400: var(--hf-color-green-400);
  --color-q-green-500: var(--hf-color-green-500);
  --color-q-green-600: var(--hf-color-green-600);
  --color-q-green-700: var(--hf-color-green-700);
  --color-q-green-800: var(--hf-color-green-800);
  --color-q-green-900: var(--hf-color-green-900);
  --color-q-green-glow: var(--hf-color-green-glow);

  --color-q-grey-050: var(--hf-color-grey-050);
  --color-q-grey-100: var(--hf-color-grey-100);
  --color-q-grey-150: var(--hf-color-grey-150);
  --color-q-grey-200: var(--hf-color-grey-200);
  --color-q-grey-250: var(--hf-color-grey-250);
  --color-q-grey-300: var(--hf-color-grey-300);
  --color-q-grey-325: var(--hf-color-grey-325);
  --color-q-grey-350: var(--hf-color-grey-350);
  --color-q-grey-400: var(--hf-color-grey-400);
  --color-q-grey-450: var(--hf-color-grey-450);
  --color-q-grey-500: var(--hf-color-grey-500);
  --color-q-grey-550: var(--hf-color-grey-550);
  --color-q-grey-600: var(--hf-color-grey-600);
```

```
--color-q-lime-100: var(--hf-color-lime-100);
--color-q-lime-1000: var(--hf-color-lime-1000);
--color-q-lime-1100: var(--hf-color-lime-1100);
--color-q-lime-1200: var(--hf-color-lime-1200);
--color-q-lime-200: var(--hf-color-lime-200);
--color-q-lime-300: var(--hf-color-lime-300);
--color-q-lime-400: var(--hf-color-lime-400);
--color-q-lime-500: var(--hf-color-lime-500);
--color-q-lime-600: var(--hf-color-lime-600);
--color-q-lime-700: var(--hf-color-lime-700);
--color-q-lime-800: var(--hf-color-lime-800);
--color-q-lime-900: var(--hf-color-lime-900);
--color-q-lime-alpha-05: var(--hf-color-lime-alpha-05);
--color-q-lime-alpha-10: var(--hf-color-lime-alpha-10);
--color-q-lime-alpha-100: var(--hf-color-lime-alpha-100);
--color-q-lime-alpha-15: var(--hf-color-lime-alpha-15);
--color-q-lime-alpha-20: var(--hf-color-lime-alpha-20);
--color-q-lime-alpha-30: var(--hf-color-lime-alpha-30);
--color-q-lime-alpha-40: var(--hf-color-lime-alpha-40);
--color-q-lime-alpha-50: var(--hf-color-lime-alpha-50);
--color-q-lime-alpha-60: var(--hf-color-lime-alpha-60);
--color-q-lime-alpha-70: var(--hf-color-lime-alpha-70);
--color-q-lime-alpha-80: var(--hf-color-lime-alpha-80);
--color-q-lime-alpha-90: var(--hf-color-lime-alpha-90);

--color-q-neutral-050: var(--hf-color-neutral-050);
--color-q-neutral-100: var(--hf-color-neutral-100);
--color-q-neutral-150: var(--hf-color-neutral-150);
--color-q-neutral-200: var(--hf-color-neutral-200);
--color-q-neutral-250: var(--hf-color-neutral-250);
--color-q-neutral-300: var(--hf-color-neutral-300);
--color-q-neutral-350: var(--hf-color-neutral-350);
--color-q-neutral-400: var(--hf-color-neutral-400);
--color-q-neutral-450: var(--hf-color-neutral-450);
--color-q-neutral-500: var(--hf-color-neutral-500);
--color-q-neutral-550: var(--hf-color-neutral-550);
--color-q-neutral-600: var(--hf-color-neutral-600);

--color-q-orange-100: var(--hf-color-orange-100);
--color-q-orange-1000: var(--hf-color-orange-1000);
--color-q-orange-1100: var(--hf-color-orange-1100);
--color-q-orange-1200: var(--hf-color-orange-1200);
--color-q-orange-200: var(--hf-color-orange-200);
--color-q-orange-300: var(--hf-color-orange-300);
--color-q-orange-400: var(--hf-color-orange-400);
--color-q-orange-500: var(--hf-color-orange-500);
--color-q-orange-600: var(--hf-color-orange-600);
--color-q-orange-700: var(--hf-color-orange-700);
--color-q-orange-800: var(--hf-color-orange-800);
--color-q-orange-900: var(--hf-color-orange-900);

--color-q-pink-100: var(--hf-color-pink-100);
--color-q-pink-1000: var(--hf-color-pink-1000);
--color-q-pink-1100: var(--hf-color-pink-1100);
--color-q-pink-1200: var(--hf-color-pink-1200);
--color-q-pink-200: var(--hf-color-pink-200);
--color-q-pink-300: var(--hf-color-pink-300);
--color-q-pink-400: var(--hf-color-pink-400);
--color-q-pink-500: var(--hf-color-pink-500);
--color-q-pink-600: var(--hf-color-pink-600);
--color-q-pink-700: var(--hf-color-pink-700);
--color-q-pink-800: var(--hf-color-pink-800);
--color-q-pink-900: var(--hf-color-pink-900);
--color-q-pink-alpha-05: var(--hf-color-pink-alpha-05);
--color-q-pink-alpha-10: var(--hf-color-pink-alpha-10);
--color-q-pink-alpha-100: var(--hf-color-pink-alpha-100);
--color-q-pink-alpha-15: var(--hf-color-pink-alpha-15);
--color-q-pink-alpha-20: var(--hf-color-pink-alpha-20);
--color-q-pink-alpha-30: var(--hf-color-pink-alpha-30);
--color-q-pink-alpha-40: var(--hf-color-pink-alpha-40);
```

```
--color-q-pink-alpha-50: var(--hf-color-pink-alpha-50);
--color-q-pink-alpha-60: var(--hf-color-pink-alpha-60);
--color-q-pink-alpha-70: var(--hf-color-pink-alpha-70);
--color-q-pink-alpha-80: var(--hf-color-pink-alpha-80);
--color-q-pink-alpha-90: var(--hf-color-pink-alpha-90);

--color-q-purple-100: var(--hf-color-purple-100);
--color-q-purple-1000: var(--hf-color-purple-1000);
--color-q-purple-1100: var(--hf-color-purple-1100);
--color-q-purple-1200: var(--hf-color-purple-1200);
--color-q-purple-200: var(--hf-color-purple-200);
--color-q-purple-300: var(--hf-color-purple-300);
--color-q-purple-400: var(--hf-color-purple-400);
--color-q-purple-500: var(--hf-color-purple-500);
--color-q-purple-600: var(--hf-color-purple-600);
--color-q-purple-700: var(--hf-color-purple-700);
--color-q-purple-800: var(--hf-color-purple-800);
--color-q-purple-900: var(--hf-color-purple-900);

--color-q-red-100: var(--hf-color-red-100);
--color-q-red-1000: var(--hf-color-red-1000);
--color-q-red-1100: var(--hf-color-red-1100);
--color-q-red-1200: var(--hf-color-red-1200);
--color-q-red-200: var(--hf-color-red-200);
--color-q-red-300: var(--hf-color-red-300);
--color-q-red-400: var(--hf-color-red-400);
--color-q-red-500: var(--hf-color-red-500);
--color-q-red-600: var(--hf-color-red-600);
--color-q-red-700: var(--hf-color-red-700);
--color-q-red-800: var(--hf-color-red-800);
--color-q-red-900: var(--hf-color-red-900);
--color-q-red-glow: var(--hf-color-red-glow);

--color-q-transparent-dark-05: var(--hf-color-transparent-dark-05);
--color-q-transparent-dark-10: var(--hf-color-transparent-dark-10);
--color-q-transparent-dark-100: var(--hf-color-transparent-dark-100);
--color-q-transparent-dark-12: var(--hf-color-transparent-dark-12);
--color-q-transparent-dark-15: var(--hf-color-transparent-dark-15);
--color-q-transparent-dark-20: var(--hf-color-transparent-dark-20);
--color-q-transparent-dark-25: var(--hf-color-transparent-dark-25);
--color-q-transparent-dark-30: var(--hf-color-transparent-dark-30);
--color-q-transparent-dark-32: var(--hf-color-transparent-dark-32);
--color-q-transparent-dark-40: var(--hf-color-transparent-dark-40);
--color-q-transparent-dark-50: var(--hf-color-transparent-dark-50);
--color-q-transparent-dark-60: var(--hf-color-transparent-dark-60);
--color-q-transparent-dark-70: var(--hf-color-transparent-dark-70);
--color-q-transparent-dark-80: var(--hf-color-transparent-dark-80);
--color-q-transparent-dark-90: var(--hf-color-transparent-dark-90);
--color-q-transparent-light-05: var(--hf-color-transparent-light-05);
--color-q-transparent-light-10: var(--hf-color-transparent-light-10);
--color-q-transparent-light-100: var(--hf-color-transparent-light-100);
--color-q-transparent-light-15: var(--hf-color-transparent-light-15);
--color-q-transparent-light-20: var(--hf-color-transparent-light-20);
--color-q-transparent-light-30: var(--hf-color-transparent-light-30);
--color-q-transparent-light-40: var(--hf-color-transparent-light-40);
--color-q-transparent-light-50: var(--hf-color-transparent-light-50);
--color-q-transparent-light-60: var(--hf-color-transparent-light-60);
--color-q-transparent-light-70: var(--hf-color-transparent-light-70);
--color-q-transparent-light-80: var(--hf-color-transparent-light-80);
--color-q-transparent-light-90: var(--hf-color-transparent-light-90);

--color-q-yellow-100: var(--hf-color-yellow-100);
--color-q-yellow-1000: var(--hf-color-yellow-1000);
--color-q-yellow-1100: var(--hf-color-yellow-1100);
--color-q-yellow-1200: var(--hf-color-yellow-1200);
--color-q-yellow-200: var(--hf-color-yellow-200);
--color-q-yellow-300: var(--hf-color-yellow-300);
--color-q-yellow-400: var(--hf-color-yellow-400);
--color-q-yellow-500: var(--hf-color-yellow-500);
--color-q-yellow-600: var(--hf-color-yellow-600);
```

```
  --color-q-yellow-700: var(--hf-color-yellow-700);
  --color-q-yellow-800: var(--hf-color-yellow-800);
  --color-q-yellow-900: var(--hf-color-yellow-900);
  --color-q-yellow-glow: var(--hf-color-yellow-glow);
}
```

app/packages/quanta/src/css/tailwind/components.css

```
/**
 * Component styles aggregator.
 *
 * Each component colocates its presentation CSS (q-* @utility rules) next to
 * its source. They're collected here — one @import per component — so the whole
 * set lands in the Tailwind bundle from a single entry point. Add a line here
 * when you add a component with its own .css.
 *
 * Pulled into the Tailwind bridge by ./index.css.
 */

@import "../../components/menu/menu.css";
@import "../../components/dropdown/dropdown.css";
@import "../../components/avatar/avatar.css";
@import "../../components/button/button.css";
@import "../../components/close-button/close-button.css";
@import "../../components/loader/loader.css";
@import "../../components/sidebar/sidebar.css";
@import "../../components/checkbox/checkbox.css";
@import "../../components/radio/radio.css";
@import "../../components/chip/chip.css";
@import "../../components/dot/dot.css";
@import "../../components/badge/badge.css";
@import "../../components/switch/switch.css";
@import "../../components/tabs/tabs.css";
@import "../../components/modal/modal.css";
@import "../../components/input/input.css";
@import "../../components/textarea/textarea.css";
@import "../../components/kbd/kbd.css";
@import "../../components/divider/divider.css";
@import "../../components/slider/slider.css";
@import "../../components/not-found/not-found.css";
@import "../../components/tag/tag.css";
@import "../../components/toggle/toggle.css";
@import "../../components/sonner/sonner.css";
@import "../../components/vault/vault.css";
@import "../../components/cmdk/cmdk.css";
@import "../../components/progress/progress.css";
@import "../../components/navigation-menu/navigation-menu.css";
@import "../../components/card/card.css";
@import "../../components/icon/icon.css";
@import "../../components/typography/typography.css";
@import "../../components/tooltip/tooltip.css";
@import "../../components/select/select.css";
@import "../../components/autocomplete/autocomplete.css";
@import "../../components/media/media.css";
@import "../../components/grid/grid.css";
@import "../../components/accordion/accordion.css";
@import "../../components/button-group/button-group.css";
@import "../../components/glass/glass.css";
@import "./slot.css";   /* slot system — component-internal styling primitives */
```

app/packages/quanta/src/css/tailwind/gradient.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@theme {
  --gradient-q-badge-blue: var(--hf-gradient-badge-blue);
  --gradient-q-badge-pink: var(--hf-gradient-badge-pink);
  --gradient-q-badge-purple: var(--hf-gradient-badge-purple);
  --gradient-q-glass-sheen: var(--hf-gradient-glass-sheen);
  --gradient-q-glass-sheen-alt: var(--hf-gradient-glass-sheen-alt);
  --gradient-q-loader-sweep: var(--hf-gradient-loader-sweep);
  --gradient-q-special-gloss-lime: var(--hf-gradient-special-gloss-lime);
  --gradient-q-special-gloss-lime-angled: var(--hf-gradient-special-gloss-lime-angled);
  --gradient-q-special-gloss-pink: var(--hf-gradient-special-gloss-pink);
}
```

app/packages/quanta/src/css/tailwind/icon.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@theme {
  --size-q-icon-xs: var(--hf-icon-xs);
  --size-q-icon-sm: var(--hf-icon-sm);
  --size-q-icon-md: var(--hf-icon-md);
  --size-q-icon-lg: var(--hf-icon-lg);
  --size-q-icon-xl: var(--hf-icon-xl);
}
```

app/packages/quanta/src/css/tailwind/index.css

```
/**
 * Layer 3 — Tailwind bridge (manifest)
 *
 * Final entry point for Tailwind v4 consumers. Imports the theme (Layer 1 + 2)
 * and aliases hf-* names to `--color-*` via `@theme inline` so Tailwind
 * generates utilities (`bg-surface-default`, `text-text-primary`, etc).
 *
 * Why `@theme inline` (and not plain `@theme`):
 * When Tailwind generates opacity modifiers (`bg-x/50`) it needs to see the
 * underlying var to synthesize `color-mix()`. With plain `@theme` a `var(...)`
 * value is treated as an opaque string and the modifier breaks.
 *
 * This file is for Tailwind ONLY. Non-Tailwind consumers import
 * `@higgsfield/quanta/theme.css` directly.
 *
 * One file per category. `@theme inline` blocks are additive.
 */

@import "../theme/index.css";

@import "./color.css";
@import "./border-width.css";
@import "./breakpoint.css";
@import "./radius.css";
@import "./spacing.css";
@import "./shadow.css";
@import "./typography.css";
@import "./z-index.css";
@import "./motion.css";
@import "./icon.css";
@import "./opacity.css";
@import "./gradient.css";

/* Generated component-override tier — global `--q-<comp>-<dim>` default knobs
 * on :root (modal/menu/sidebar/slider/sonner/vault widths, …). Imported BEFORE
 * components.css so each component reads its default here; a consumer override
 * on any closer scope wins the cascade. */
@import "../components/component.css";

/* Component presentation — q-* component utilities, aggregated in components.css
 * so they land in the Tailwind bundle. */
@import "./components.css";
```

app/packages/quanta/src/css/tailwind/motion.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@theme {
  --transition-duration-q-instant: var(--hf-duration-instant);
  --transition-duration-q-fast: var(--hf-duration-fast);
  --transition-duration-q-normal: var(--hf-duration-normal);
  --transition-duration-q-slow: var(--hf-duration-slow);
  --transition-duration-q-slower: var(--hf-duration-slower);
  --transition-duration-q-80: var(--hf-duration-80);
  --transition-duration-q-120: var(--hf-duration-120);
  --transition-duration-q-130: var(--hf-duration-130);
  --transition-duration-q-150: var(--hf-duration-150);
  --transition-duration-q-160: var(--hf-duration-160);
  --transition-duration-q-180: var(--hf-duration-180);
  --transition-duration-q-220: var(--hf-duration-220);
  --transition-duration-q-260: var(--hf-duration-260);
  --transition-duration-q-350: var(--hf-duration-350);
  --transition-duration-q-400: var(--hf-duration-400);
  --transition-duration-q-700: var(--hf-duration-700);
  --transition-duration-q-800: var(--hf-duration-800);
  --transition-duration-q-1000: var(--hf-duration-1000);
  --transition-duration-q-1200: var(--hf-duration-1200);
  --transition-duration-q-1300: var(--hf-duration-1300);
  --transition-duration-q-1400: var(--hf-duration-1400);
  --ease-q-in: var(--hf-ease-in);
  --ease-q-out: var(--hf-ease-out);
  --ease-q-in-out: var(--hf-ease-in-out);
  --ease-q-linear: var(--hf-ease-linear);
  --ease-q-spring: var(--hf-ease-spring);
  --ease-q-out-expo: var(--hf-ease-out-expo);
  --ease-q-emphasized: var(--hf-ease-emphasized);
  --ease-q-swift: var(--hf-ease-swift);
  --ease-q-in-out-circ: var(--hf-ease-in-out-circ);
  --ease-q-out-quart: var(--hf-ease-out-quart);
}
```

app/packages/quanta/src/css/tailwind/opacity.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@theme {
  --opacity-q-disabled: var(--hf-opacity-disabled);
  --opacity-q-muted: var(--hf-opacity-muted);
}
```

app/packages/quanta/src/css/tailwind/radius.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@theme {
  --radius-q-0: var(--hf-radius-0);
  --radius-q-050: var(--hf-radius-050);
  --radius-q-100: var(--hf-radius-100);
  --radius-q-150: var(--hf-radius-150);
  --radius-q-200: var(--hf-radius-200);
  --radius-q-250: var(--hf-radius-250);
  --radius-q-300: var(--hf-radius-300);
  --radius-q-400: var(--hf-radius-400);
  --radius-q-500: var(--hf-radius-500);
  --radius-q-600: var(--hf-radius-600);
  --radius-q-full: var(--hf-radius-full);
}
```

app/packages/quanta/src/css/tailwind/shadow.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@theme {
  --shadow-q-inset-tile: var(--hf-shadow-inset-tile);
  --shadow-q-inset-tile-hairline: var(--hf-shadow-inset-tile-hairline);
  --shadow-q-modal: var(--hf-shadow-modal);
  --shadow-q-overlay: var(--hf-shadow-overlay);
  --shadow-q-raised: var(--hf-shadow-raised);
  --shadow-q-raised-sm: var(--hf-shadow-raised-sm);
  --shadow-q-sheen: var(--hf-shadow-sheen);
  --shadow-q-thumb-soft: var(--hf-shadow-thumb-soft);
  --shadow-q-thumb-strong: var(--hf-shadow-thumb-strong);
}
```

app/packages/quanta/src/css/tailwind/slot.css

```
/**
 * Slot color system — component-INTERNAL styling primitives (concept ported
 * from mdigitalcn/uikit). Hand-authored, NOT codegen, NOT golden-gated.
 *
 * A component sets ONE private custom property `--q-tint` (plus optional siblings
 * `--q-tint-bg` / `--q-tint-fg` / `--q-tint-border`, and the shape/motion `--_radius` /
 * `--_duration` / `--_ease`) inline from a single semantic `color` prop — see
 * src/components/utils/slot.ts. Every interior surface then derives from it via
 * these utilities, so changing one prop re-skins the whole control.
 *
 * Authored ONLY inside component .tsx, never in app code — the `--q-tint*` props are
 * a private boundary, not consumer-facing q-* tokens.
 *
 * This is the TINT pattern (soft single-accent: toggle/tag/switch/tabs/progress/
 * loader). Selection controls (checkbox/radio/chip) use a separate per-color
 * utility table instead — see ai/COMPONENT_STANDARD.md §4 for when to use which.
 *
 * CONTRACT: `--q-tint` values MUST reference `--hf-color-*` runtime primitives
 * (emitted per data-theme), NEVER `--color-q-*` (those are @theme inline-only
 * and have no runtime value). Copying a `bg-q-*` token name into a SLOT map
 * silently breaks theming. See ai/AGENTS.md.
 */

/* Solid fills — bg reads --q-tint-bg, defaulting to --q-tint (uikit-aligned). */
@utility q-slot-bg    { background-color: var(--q-tint-bg, var(--q-tint)); }
@utility q-slot-bg-fg { background-color: var(--q-tint-fg, oklch(100% 0 0)); }

/* Soft tints — always derived from --q-tint (the saturated color), never --q-tint-bg. */
@utility q-slot-bg-10 { background-color: color-mix(in oklch, var(--q-tint) 10%, transparent); }
@utility q-slot-bg-20 { background-color: color-mix(in oklch, var(--q-tint) 20%, transparent); }

/* Foreground / text */
@utility q-slot-text    { color: var(--q-tint); }
@utility q-slot-text-fg { color: var(--q-tint-fg, oklch(100% 0 0)); }

/* Border — defaults to --q-tint when --q-tint-border unset. */
@utility q-slot-border { border-color: var(--q-tint-border, var(--q-tint)); }

/* Focus ring */
@utility q-slot-ring    { --tw-ring-color: var(--q-tint); }
@utility q-slot-ring-40 { --tw-ring-color: color-mix(in oklch, var(--q-tint) 40%, transparent); }

/* ■■ Shape / motion slots. --_radius/--_duration/--_ease are set inline per
 * component; until quanta radius/motion tokens are emitted (plan Phase 2)
 * components may use native utilities (rounded-lg, duration-200) instead. ■■ */
@utility q-slot-rounded  { border-radius: var(--_radius); }
@utility q-slot-duration { transition-duration: var(--_duration); }
@utility q-slot-ease     { transition-timing-function: var(--_ease); }
```

app/packages/quanta/src/css/tailwind/spacing.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@theme {
  --spacing-q-0: var(--hf-space-0);
  --spacing-q-050: var(--hf-space-050);
  --spacing-q-100: var(--hf-space-100);
  --spacing-q-150: var(--hf-space-150);
  --spacing-q-200: var(--hf-space-200);
  --spacing-q-250: var(--hf-space-250);
  --spacing-q-300: var(--hf-space-300);
  --spacing-q-400: var(--hf-space-400);
  --spacing-q-500: var(--hf-space-500);
  --spacing-q-600: var(--hf-space-600);
  --spacing-q-650: var(--hf-space-650);
  --spacing-q-700: var(--hf-space-700);
  --spacing-q-800: var(--hf-space-800);
  --spacing-q-1000: var(--hf-space-1000);
  --spacing-q-1200: var(--hf-space-1200);
  --spacing-q-1400: var(--hf-space-1400);
  --spacing-q-1600: var(--hf-space-1600);
  --spacing-q-2000: var(--hf-space-2000);
  --spacing-q-2400: var(--hf-space-2400);
}
```

app/packages/quanta/src/css/tailwind/typography.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@utility text-q-accent-2xl-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-1000);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1100);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-accent-2xs-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-200);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-accent-3xl-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-1100);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1200);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-accent-4xl-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-1200);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1300);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-accent-5xl-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-1300);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1400);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-accent-6xl-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-1400);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1400);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-accent-lg-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-800);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-900);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-accent-md-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-700);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-800);
```

```
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-accent-sm-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-600);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-700);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-accent-xl-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-900);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1000);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-accent-xs-bold {
  font-family:    var(--hf-type-family-grotesk);
  font-size:      var(--hf-type-size-500);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-body-lg-medium {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-400);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-body-lg-regular {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-400);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-body-lg-semi-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-400);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-body-md-medium {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-300);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-body-md-regular {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-300);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-body-md-semi-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-300);
```

```
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-body-sm-medium {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-200);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-500);
  letter-spacing: var(--hf-type-letter-spacing-loose);
}

@utility text-q-body-sm-regular {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-200);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-500);
  letter-spacing: var(--hf-type-letter-spacing-loose);
}

@utility text-q-body-sm-semi-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-200);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-500);
  letter-spacing: var(--hf-type-letter-spacing-loose);
}

@utility text-q-caption-sm-medium {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-caption-sm-regular {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-caption-sm-semi-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-caption-xs-medium {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-050);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-200);
  letter-spacing: var(--hf-type-letter-spacing-wide);
}

@utility text-q-caption-xs-regular {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-050);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-200);
  letter-spacing: var(--hf-type-letter-spacing-wide);
}

@utility text-q-caption-xs-semi-bold {
```

```
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-050);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-200);
  letter-spacing: var(--hf-type-letter-spacing-wide);
}

@utility text-q-display-lg-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1300);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1400);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-display-lg-medium {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1300);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-1400);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-display-lg-semi-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1300);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-1400);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-display-md-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1200);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1300);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-display-md-medium {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1200);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-1300);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-display-md-semi-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1200);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-1300);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-display-sm-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1100);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1200);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-display-sm-medium {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1100);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-1200);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}
```

```
@utility text-q-display-sm-semi-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1100);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-1200);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-headline-lg-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1000);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1100);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-headline-lg-medium {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1000);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-1100);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-headline-lg-semi-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-1000);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-1100);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-headline-md-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-900);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-1000);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-headline-md-medium {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-900);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-1000);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-headline-md-semi-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-900);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-1000);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-headline-sm-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-800);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-900);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-headline-sm-medium {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-800);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-900);
  letter-spacing: var(--hf-type-letter-spacing-tight);
```

```
}

@utility text-q-headline-sm-semi-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-800);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-900);
  letter-spacing: var(--hf-type-letter-spacing-tight);
}

@utility text-q-label-lg-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-400);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-label-lg-medium {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-400);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-label-lg-regular {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-400);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-label-lg-semi-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-400);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-label-md-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-300);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-500);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-md-medium {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-300);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-500);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-md-regular {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-300);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-500);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-md-semi-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-300);
  font-weight:    var(--hf-type-weight-semi-bold);
```

```
  line-height:    var(--hf-type-line-height-500);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-sm-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-200);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-sm-medium {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-200);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-sm-regular {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-200);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-sm-semi-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-200);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-xs-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-xs-medium {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-xs-regular {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-label-xs-semi-bold {
  font-family:    var(--hf-type-family-secondary);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-300);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-lg-medium {
  font-family:    var(--hf-type-family-mono);
```

```
  font-size:      var(--hf-type-size-300);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-lg-regular {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-300);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-lg-semi-bold {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-300);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-md-medium {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-200);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-500);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-md-regular {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-200);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-500);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-md-semi-bold {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-200);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-500);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-sm-medium {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-400);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-sm-regular {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-400);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-sm-semi-bold {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-100);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-400);
  letter-spacing: var(--hf-type-letter-spacing-none);
}
```

```
@utility text-q-mono-xs-medium {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-050);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-200);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-xs-regular {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-050);
  font-weight:    var(--hf-type-weight-regular);
  line-height:    var(--hf-type-line-height-200);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-mono-xs-semi-bold {
  font-family:    var(--hf-type-family-mono);
  font-size:      var(--hf-type-size-050);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-200);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility text-q-title-lg-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-700);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-800);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-title-lg-medium {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-700);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-800);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-title-lg-semi-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-700);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-800);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-title-md-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-600);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-700);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-title-md-medium {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-600);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-700);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-title-md-semi-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-600);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-700);
  letter-spacing: var(--hf-type-letter-spacing-slight);
```

```
}

@utility text-q-title-sm-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-500);
  font-weight:    var(--hf-type-weight-bold);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-title-sm-medium {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-500);
  font-weight:    var(--hf-type-weight-medium);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

@utility text-q-title-sm-semi-bold {
  font-family:    var(--hf-type-family-primary);
  font-size:      var(--hf-type-size-500);
  font-weight:    var(--hf-type-weight-semi-bold);
  line-height:    var(--hf-type-line-height-600);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}
```

app/packages/quanta/src/css/tailwind/z-index.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@theme {
  --z-index-q-base: var(--hf-z-index-base);
  --z-index-q-dropdown: var(--hf-z-index-dropdown);
  --z-index-q-sticky: var(--hf-z-index-sticky);
  --z-index-q-overlay: var(--hf-z-index-overlay);
  --z-index-q-modal: var(--hf-z-index-modal);
  --z-index-q-popover: var(--hf-z-index-popover);
  --z-index-q-toast: var(--hf-z-index-toast);
  --z-index-q-tooltip: var(--hf-z-index-tooltip);
}
```

app/packages/quanta/src/css/theme/color.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

@layer quanta-theme {

  :where(:root),
  :where([data-theme="default-light"]) {
    color-scheme: light;

    --hf-color-background-elevated-end: var(--hf-color-neutral-550);
    --hf-color-background-elevated-start: var(--hf-color-neutral-350);
    --hf-color-background-glass: var(--hf-color-glass-light);
    --hf-color-background-inverse: var(--hf-color-grey-550);
    --hf-color-background-primary: var(--hf-color-grey-050);
    --hf-color-background-secondary: var(--hf-color-grey-100);
    --hf-color-background-secondary-strong: var(--hf-color-neutral-100);
    --hf-color-background-tertiary: var(--hf-color-grey-250);

    --hf-color-border-default: var(--hf-color-transparent-dark-10);
    --hf-color-border-error: var(--hf-color-red-700);
    --hf-color-border-focus: var(--hf-color-lime-500);
    --hf-color-border-inverse: var(--hf-color-grey-550);
    --hf-color-border-strong: var(--hf-color-transparent-dark-20);
    --hf-color-border-subtle: var(--hf-color-transparent-dark-05);
    --hf-color-border-success: var(--hf-color-green-500);
    --hf-color-border-warning: var(--hf-color-yellow-700);

    --hf-color-brand-blue: var(--hf-color-blue-500);
    --hf-color-brand-cyan: var(--hf-color-cyan-600);
    --hf-color-brand-green: var(--hf-color-green-500);
    --hf-color-brand-orange: var(--hf-color-orange-400);
    --hf-color-brand-pink: var(--hf-color-pink-500);
    --hf-color-brand-primary: var(--hf-color-lime-500);
    --hf-color-brand-red: var(--hf-color-red-700);
    --hf-color-brand-yellow: var(--hf-color-yellow-500);
    --hf-color-brand-yellow-light: var(--hf-color-yellow-300);

    --hf-color-button-brand: var(--hf-color-lime-500);
    --hf-color-button-destructive: var(--hf-color-red-700);
    --hf-color-button-label-brand: var(--hf-color-grey-550);
    --hf-color-button-label-destructive: var(--hf-color-grey-050);
    --hf-color-button-primary: var(--hf-color-grey-600);
    --hf-color-button-secondary: var(--hf-color-neutral-100);
    --hf-color-button-tertiary: var(--hf-color-neutral-150);

    --hf-color-icon-accent: var(--hf-color-lime-500);
    --hf-color-icon-brand: var(--hf-color-lime-500);
    --hf-color-icon-disabled: var(--hf-color-grey-150);
    --hf-color-icon-error: var(--hf-color-red-700);
    --hf-color-icon-info: var(--hf-color-blue-500);
    --hf-color-icon-inverse: var(--hf-color-grey-050);
    --hf-color-icon-on-overlay-secondary: var(--hf-color-transparent-dark-50);
    --hf-color-icon-on-overlay-tertiary: var(--hf-color-transparent-dark-30);
    --hf-color-icon-primary: var(--hf-color-grey-550);
    --hf-color-icon-secondary: var(--hf-color-grey-300);
    --hf-color-icon-success: var(--hf-color-green-600);
    --hf-color-icon-tertiary: var(--hf-color-grey-200);
    --hf-color-icon-warning: var(--hf-color-yellow-700);

    --hf-color-overlay-dim-soft: var(--hf-color-transparent-dark-05);
    --hf-color-overlay-dim-strong: var(--hf-color-transparent-dark-20);
    --hf-color-overlay-hover: var(--hf-color-transparent-dark-05);
    --hf-color-overlay-scrim: var(--hf-color-transparent-dark-50);
    --hf-color-overlay-wash: var(--hf-color-transparent-light-50);
```

```
--hf-color-palette-blue-bg: var(--hf-color-blue-200);
--hf-color-palette-blue-text: var(--hf-color-blue-600);
--hf-color-palette-brown-bg: var(--hf-color-yellow-300);
--hf-color-palette-brown-text: var(--hf-color-yellow-700);
--hf-color-palette-mint-bg: var(--hf-color-green-200);
--hf-color-palette-mint-text: var(--hf-color-green-800);
--hf-color-palette-orange-bg: var(--hf-color-orange-300);
--hf-color-palette-orange-text: var(--hf-color-orange-800);
--hf-color-palette-pink-bg: var(--hf-color-pink-200);
--hf-color-palette-pink-text: var(--hf-color-pink-700);
--hf-color-palette-purple-bg: var(--hf-color-purple-200);
--hf-color-palette-purple-text: var(--hf-color-purple-800);
--hf-color-palette-white: var(--hf-color-grey-050);

--hf-color-shadow-2xl: var(--hf-color-transparent-dark-40);
--hf-color-shadow-lg: var(--hf-color-transparent-dark-20);
--hf-color-shadow-md: var(--hf-color-transparent-dark-15);
--hf-color-shadow-sm: var(--hf-color-transparent-dark-05);
--hf-color-shadow-xl: var(--hf-color-transparent-dark-30);

--hf-color-state-error-bg: var(--hf-color-red-100);
--hf-color-state-error-fg: var(--hf-color-red-700);
--hf-color-state-error-fg-soft: var(--hf-color-red-600);
--hf-color-state-error-glow: var(--hf-color-red-glow);
--hf-color-state-info-bg: var(--hf-color-blue-100);
--hf-color-state-info-fg: var(--hf-color-blue-500);
--hf-color-state-info-fg-soft: var(--hf-color-blue-400);
--hf-color-state-success-bg: var(--hf-color-green-100);
--hf-color-state-success-fg: var(--hf-color-green-600);
--hf-color-state-success-fg-soft: var(--hf-color-green-600);
--hf-color-state-success-glow: var(--hf-color-green-glow);
--hf-color-state-warning-bg: var(--hf-color-yellow-100);
--hf-color-state-warning-fg: var(--hf-color-yellow-700);
--hf-color-state-warning-fg-soft: var(--hf-color-yellow-600);
--hf-color-state-warning-glow: var(--hf-color-yellow-glow);

--hf-color-text-brand: var(--hf-color-lime-500);
--hf-color-text-danger: var(--hf-color-red-700);
--hf-color-text-disabled: var(--hf-color-grey-150);
--hf-color-text-info: var(--hf-color-blue-600);
--hf-color-text-inverse: var(--hf-color-grey-050);
--hf-color-text-link: var(--hf-color-lime-500);
--hf-color-text-on-overlay-secondary: var(--hf-color-transparent-dark-50);
--hf-color-text-on-overlay-tertiary: var(--hf-color-transparent-dark-30);
--hf-color-text-primary: var(--hf-color-grey-550);
--hf-color-text-secondary: var(--hf-color-grey-300);
--hf-color-text-success: var(--hf-color-green-600);
--hf-color-text-tertiary: var(--hf-color-grey-200);
--hf-color-text-warning: var(--hf-color-orange-600);

--hf-color-transparent-lime-05: var(--hf-color-lime-alpha-05);
--hf-color-transparent-lime-10: var(--hf-color-lime-alpha-10);
--hf-color-transparent-lime-100: var(--hf-color-lime-alpha-100);
--hf-color-transparent-lime-15: var(--hf-color-lime-alpha-15);
--hf-color-transparent-lime-20: var(--hf-color-lime-alpha-20);
--hf-color-transparent-lime-30: var(--hf-color-lime-alpha-30);
--hf-color-transparent-lime-40: var(--hf-color-lime-alpha-40);
--hf-color-transparent-lime-50: var(--hf-color-lime-alpha-50);
--hf-color-transparent-lime-60: var(--hf-color-lime-alpha-60);
--hf-color-transparent-lime-70: var(--hf-color-lime-alpha-70);
--hf-color-transparent-lime-80: var(--hf-color-lime-alpha-80);
--hf-color-transparent-lime-90: var(--hf-color-lime-alpha-90);
--hf-color-transparent-pink-05: var(--hf-color-pink-alpha-05);
--hf-color-transparent-pink-10: var(--hf-color-pink-alpha-10);
--hf-color-transparent-pink-100: var(--hf-color-pink-alpha-100);
--hf-color-transparent-pink-15: var(--hf-color-pink-alpha-15);
--hf-color-transparent-pink-20: var(--hf-color-pink-alpha-20);
--hf-color-transparent-pink-30: var(--hf-color-pink-alpha-30);
--hf-color-transparent-pink-40: var(--hf-color-pink-alpha-40);
--hf-color-transparent-pink-50: var(--hf-color-pink-alpha-50);
```

```
  --hf-color-transparent-pink-60: var(--hf-color-pink-alpha-60);
  --hf-color-transparent-pink-70: var(--hf-color-pink-alpha-70);
  --hf-color-transparent-pink-80: var(--hf-color-pink-alpha-80);
  --hf-color-transparent-pink-90: var(--hf-color-pink-alpha-90);
}

:where([data-theme="default-dark"]) {
  color-scheme: dark;

  --hf-color-background-elevated-end: var(--hf-color-cool-350);
  --hf-color-background-elevated-start: var(--hf-color-cool-150);
  --hf-color-background-glass: var(--hf-color-glass-dark);
  --hf-color-background-inverse: var(--hf-color-grey-050);
  --hf-color-background-primary: var(--hf-color-cool-400);
  --hf-color-background-secondary: var(--hf-color-cool-300);
  --hf-color-background-secondary-strong: var(--hf-color-cool-250);
  --hf-color-background-tertiary: var(--hf-color-cool-200);

  --hf-color-border-default: var(--hf-color-transparent-light-10);
  --hf-color-border-error: var(--hf-color-red-600);
  --hf-color-border-focus: var(--hf-color-lime-500);
  --hf-color-border-inverse: var(--hf-color-grey-050);
  --hf-color-border-strong: var(--hf-color-transparent-light-20);
  --hf-color-border-subtle: var(--hf-color-transparent-light-05);
  --hf-color-border-success: var(--hf-color-green-500);
  --hf-color-border-warning: var(--hf-color-yellow-500);

  --hf-color-brand-blue: var(--hf-color-blue-500);
  --hf-color-brand-cyan: var(--hf-color-cyan-500);
  --hf-color-brand-green: var(--hf-color-green-500);
  --hf-color-brand-orange: var(--hf-color-orange-600);
  --hf-color-brand-pink: var(--hf-color-pink-500);
  --hf-color-brand-primary: var(--hf-color-lime-500);
  --hf-color-brand-red: var(--hf-color-red-600);
  --hf-color-brand-yellow: var(--hf-color-yellow-500);
  --hf-color-brand-yellow-light: var(--hf-color-yellow-300);

  --hf-color-button-brand: var(--hf-color-lime-500);
  --hf-color-button-destructive: var(--hf-color-red-600);
  --hf-color-button-label-brand: var(--hf-color-grey-550);
  --hf-color-button-label-destructive: var(--hf-color-grey-050);
  --hf-color-button-primary: var(--hf-color-grey-050);
  --hf-color-button-secondary: var(--hf-color-cool-250);
  --hf-color-button-tertiary: var(--hf-color-cool-050);

  --hf-color-icon-accent: var(--hf-color-lime-500);
  --hf-color-icon-brand: var(--hf-color-lime-500);
  --hf-color-icon-disabled: var(--hf-color-cool-100);
  --hf-color-icon-error: var(--hf-color-red-600);
  --hf-color-icon-info: var(--hf-color-blue-500);
  --hf-color-icon-inverse: var(--hf-color-grey-550);
  --hf-color-icon-on-overlay-secondary: var(--hf-color-transparent-light-50);
  --hf-color-icon-on-overlay-tertiary: var(--hf-color-transparent-light-30);
  --hf-color-icon-primary: var(--hf-color-grey-050);
  --hf-color-icon-secondary: var(--hf-color-grey-300);
  --hf-color-icon-success: var(--hf-color-green-500);
  --hf-color-icon-tertiary: var(--hf-color-grey-325);
  --hf-color-icon-warning: var(--hf-color-yellow-700);

  --hf-color-overlay-dim-soft: var(--hf-color-transparent-light-05);
  --hf-color-overlay-dim-strong: var(--hf-color-transparent-light-20);
  --hf-color-overlay-hover: var(--hf-color-transparent-light-05);
  --hf-color-overlay-scrim: var(--hf-color-transparent-dark-50);
  --hf-color-overlay-wash: var(--hf-color-transparent-light-50);

  --hf-color-palette-blue-bg: var(--hf-color-blue-500);
  --hf-color-palette-blue-text: var(--hf-color-blue-100);
  --hf-color-palette-brown-bg: var(--hf-color-yellow-300);
  --hf-color-palette-brown-text: var(--hf-color-yellow-700);
  --hf-color-palette-mint-bg: var(--hf-color-green-400);
```

```
  --hf-color-palette-mint-text: var(--hf-color-green-900);
  --hf-color-palette-orange-bg: var(--hf-color-orange-600);
  --hf-color-palette-orange-text: var(--hf-color-orange-100);
  --hf-color-palette-pink-bg: var(--hf-color-pink-500);
  --hf-color-palette-pink-text: var(--hf-color-pink-100);
  --hf-color-palette-purple-bg: var(--hf-color-purple-600);
  --hf-color-palette-purple-text: var(--hf-color-purple-100);
  --hf-color-palette-white: var(--hf-color-grey-050);

  --hf-color-shadow-2xl: var(--hf-color-transparent-dark-40);
  --hf-color-shadow-lg: var(--hf-color-transparent-dark-20);
  --hf-color-shadow-md: var(--hf-color-transparent-dark-15);
  --hf-color-shadow-sm: var(--hf-color-transparent-dark-05);
  --hf-color-shadow-xl: var(--hf-color-transparent-dark-30);

  --hf-color-state-error-bg: var(--hf-color-red-1000);
  --hf-color-state-error-fg: var(--hf-color-red-600);
  --hf-color-state-error-fg-soft: var(--hf-color-red-500);
  --hf-color-state-error-glow: var(--hf-color-red-glow);
  --hf-color-state-info-bg: var(--hf-color-blue-1100);
  --hf-color-state-info-fg: var(--hf-color-blue-500);
  --hf-color-state-info-fg-soft: var(--hf-color-blue-400);
  --hf-color-state-success-bg: var(--hf-color-green-800);
  --hf-color-state-success-fg: var(--hf-color-green-500);
  --hf-color-state-success-fg-soft: var(--hf-color-green-400);
  --hf-color-state-success-glow: var(--hf-color-green-glow);
  --hf-color-state-warning-bg: var(--hf-color-yellow-1100);
  --hf-color-state-warning-fg: var(--hf-color-yellow-700);
  --hf-color-state-warning-fg-soft: var(--hf-color-yellow-500);
  --hf-color-state-warning-glow: var(--hf-color-yellow-glow);

  --hf-color-text-brand: var(--hf-color-lime-500);
  --hf-color-text-danger: var(--hf-color-red-600);
  --hf-color-text-disabled: var(--hf-color-cool-100);
  --hf-color-text-info: var(--hf-color-blue-400);
  --hf-color-text-inverse: var(--hf-color-grey-550);
  --hf-color-text-link: var(--hf-color-lime-500);
  --hf-color-text-on-overlay-secondary: var(--hf-color-transparent-light-50);
  --hf-color-text-on-overlay-tertiary: var(--hf-color-transparent-light-30);
  --hf-color-text-primary: var(--hf-color-grey-050);
  --hf-color-text-secondary: var(--hf-color-grey-300);
  --hf-color-text-success: var(--hf-color-green-500);
  --hf-color-text-tertiary: var(--hf-color-grey-325);
  --hf-color-text-warning: var(--hf-color-yellow-700);

  --hf-color-transparent-lime-05: var(--hf-color-lime-alpha-05);
  --hf-color-transparent-lime-10: var(--hf-color-lime-alpha-10);
  --hf-color-transparent-lime-100: var(--hf-color-lime-alpha-100);
  --hf-color-transparent-lime-15: var(--hf-color-lime-alpha-15);
  --hf-color-transparent-lime-20: var(--hf-color-lime-alpha-20);
  --hf-color-transparent-lime-30: var(--hf-color-lime-alpha-30);
  --hf-color-transparent-lime-40: var(--hf-color-lime-alpha-40);
  --hf-color-transparent-lime-50: var(--hf-color-lime-alpha-50);
  --hf-color-transparent-lime-60: var(--hf-color-lime-alpha-60);
  --hf-color-transparent-lime-70: var(--hf-color-lime-alpha-70);
  --hf-color-transparent-lime-80: var(--hf-color-lime-alpha-80);
  --hf-color-transparent-lime-90: var(--hf-color-lime-alpha-90);
  --hf-color-transparent-pink-05: var(--hf-color-pink-alpha-05);
  --hf-color-transparent-pink-10: var(--hf-color-pink-alpha-10);
  --hf-color-transparent-pink-100: var(--hf-color-pink-alpha-100);
  --hf-color-transparent-pink-15: var(--hf-color-pink-alpha-15);
  --hf-color-transparent-pink-20: var(--hf-color-pink-alpha-20);
  --hf-color-transparent-pink-30: var(--hf-color-pink-alpha-30);
  --hf-color-transparent-pink-40: var(--hf-color-pink-alpha-40);
  --hf-color-transparent-pink-50: var(--hf-color-pink-alpha-50);
  --hf-color-transparent-pink-60: var(--hf-color-pink-alpha-60);
  --hf-color-transparent-pink-70: var(--hf-color-pink-alpha-70);
  --hf-color-transparent-pink-80: var(--hf-color-pink-alpha-80);
  --hf-color-transparent-pink-90: var(--hf-color-pink-alpha-90);
}
```

```
}
```

app/packages/quanta/src/css/theme/fonts.css

```
/**
 * Web font @font-face declarations for the four design-token families.
 *
 * Ships with the PUBLISHED package (imported by ./theme.css and, transitively,
 * ./tailwind.css). Font binaries live alongside this file under ./fonts/ and
 * are referenced with file-relative url() so a consumer bundler (Vite/webpack)
 * resolves and emits them — Storybook (which imports the theme CSS) gets them too.
 *
 * Family names match the design tokens exactly (src/css/primitives/typography.css):
 *   "Space Grotesk" (primary + grotesk), "Inter" (secondary),
 *   "IBM Plex Mono" (mono).
 *
 * no token: web-font @font-face — Google subset woff2.
 * @font-face / unicode-range / local() / format() are not token-eligible values.
 *
 * Generated/assembled by hand; binaries: Inter (variable woff2, wght 100–900,
 * 7 subsets), Space Grotesk (variable woff2, 3 subsets), IBM Plex Mono
 * (woff2, 4 weights × 5 subsets); all fetched from Google Fonts css2.
 */

/* ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ Inter (secondary) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

/* cyrillic-ext */
@font-face {
  font-family: "Inter";
  font-style: normal;
  font-weight: 100 900;
  font-display: swap;
  src: url("./fonts/inter/inter-cyrillic-ext.woff2") format("woff2");
  unicode-range: U+0460-052F, U+1C80-1C8A, U+20B4, U+2DE0-2DFF, U+A640-A69F, U+FE2E-FE2F;
}

/* cyrillic */
@font-face {
  font-family: "Inter";
  font-style: normal;
  font-weight: 100 900;
  font-display: swap;
  src: url("./fonts/inter/inter-cyrillic.woff2") format("woff2");
  unicode-range: U+0301, U+0400-045F, U+0490-0491, U+04B0-04B1, U+2116;
}

/* greek-ext */
@font-face {
  font-family: "Inter";
  font-style: normal;
  font-weight: 100 900;
  font-display: swap;
  src: url("./fonts/inter/inter-greek-ext.woff2") format("woff2");
  unicode-range: U+1F00-1FFF;
}

/* greek */
@font-face {
  font-family: "Inter";
  font-style: normal;
  font-weight: 100 900;
  font-display: swap;
  src: url("./fonts/inter/inter-greek.woff2") format("woff2");
  unicode-range: U+0370-0377, U+037A-037F, U+0384-038A, U+038C, U+038E-03A1, U+03A3-03FF;
}

/* vietnamese */
@font-face {
  font-family: "Inter";
  font-style: normal;
  font-weight: 100 900;
  font-display: swap;
```

```
  src: url("./fonts/inter/inter-vietnamese.woff2") format("woff2");
  unicode-range: U+0102-0103, U+0110-0111, U+0128-0129, U+0168-0169, U+01A0-01A1, U+01AF-01B0, U+0300-0301, U+0303-0304, U+0308-0309, U+0323, U+0329, U+1EA0-1EF9, U+20AB;
}

/* latin-ext */
@font-face {
  font-family: "Inter";
  font-style: normal;
  font-weight: 100 900;
  font-display: swap;
  src: url("./fonts/inter/inter-latin-ext.woff2") format("woff2");
  unicode-range: U+0100-02BA, U+02BD-02C5, U+02C7-02CC, U+02CE-02D7, U+02DD-02FF, U+0304, U+0308, U+0329, U+1D00-1DBF, U+1E00-1E9F, U+1EF2-1EFF, U+2020, U+20A0-20AB, U+20AD-20C0, U+2113, U+2C60-2C7F,
}

/* latin */
@font-face {
  font-family: "Inter";
  font-style: normal;
  font-weight: 100 900;
  font-display: swap;
  src: url("./fonts/inter/inter-latin.woff2") format("woff2");
  unicode-range: U+0000-00FF, U+0131, U+0152-0153, U+02BB-02BC, U+02C6, U+02DA, U+02DC, U+0304, U+0308, U+0329, U+2000-206F, U+20AC, U+2122, U+2191, U+2193, U+2212, U+2215, U+FEFF, U+FFFD;
}

/* ■■■■■■■■■■■■■■■■■■■■ Space Grotesk (primary + grotesk) ■■■■■■■■■■■■■■■■■■■■■■ */

@font-face {
  font-family: "Space Grotesk";
  font-style: normal;
  font-weight: 300 700;
  font-display: swap;
  src: url("./fonts/space-grotesk/space-grotesk-vietnamese.woff2") format("woff2");
  unicode-range: U+0102-0103, U+0110-0111, U+0128-0129, U+0168-0169, U+01A0-01A1, U+01AF-01B0, U+0300-0301, U+0303-0304, U+0308-0309, U+0323, U+0329, U+1EA0-1EF9, U+20AB;
}

@font-face {
  font-family: "Space Grotesk";
  font-style: normal;
  font-weight: 300 700;
  font-display: swap;
  src: url("./fonts/space-grotesk/space-grotesk-latin-ext.woff2") format("woff2");
  unicode-range: U+0100-02BA, U+02BD-02C5, U+02C7-02CC, U+02CE-02D7, U+02DD-02FF, U+0304, U+0308, U+0329, U+1D00-1DBF, U+1E00-1E9F, U+1EF2-1EFF, U+2020, U+20A0-20AB, U+20AD-20C0, U+2113, U+2C60-2C7F,
}

@font-face {
  font-family: "Space Grotesk";
  font-style: normal;
  font-weight: 300 700;
  font-display: swap;
  src: url("./fonts/space-grotesk/space-grotesk-latin.woff2") format("woff2");
  unicode-range: U+0000-00FF, U+0131, U+0152-0153, U+02BB-02BC, U+02C6, U+02DA, U+02DC, U+0304, U+0308, U+0329, U+2000-206F, U+20AC, U+2122, U+2191, U+2193, U+2212, U+2215, U+FEFF, U+FFFD;
}

/* ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ IBM Plex Mono (mono) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

/* cyrillic-ext */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 400;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-cyrillic-ext-400.woff2") format("woff2");
  unicode-range: U+0460-052F, U+1C80-1C8A, U+20B4, U+2DE0-2DFF, U+A640-A69F, U+FE2E-FE2F;
}

/* cyrillic */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 400;
```

```
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-cyrillic-400.woff2") format("woff2");
  unicode-range: U+0301, U+0400-045F, U+0490-0491, U+04B0-04B1, U+2116;
}

/* vietnamese */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 400;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-vietnamese-400.woff2") format("woff2");
  unicode-range: U+0102-0103, U+0110-0111, U+0128-0129, U+0168-0169, U+01A0-01A1, U+01AF-01B0, U+0300-0301, U+0303-0304, U+0308-0309, U+0323, U+0329, U+1EA0-1EF9, U+20AB;
}

/* latin-ext */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 400;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-latin-ext-400.woff2") format("woff2");
  unicode-range: U+0100-02BA, U+02BD-02C5, U+02C7-02CC, U+02CE-02D7, U+02DD-02FF, U+0304, U+0308, U+0329, U+1D00-1DBF, U+1E00-1E9F, U+1EF2-1EFF, U+2020, U+20A0-20AB, U+20AD-20C0, U+2113, U+2C60-2C7F,
}

/* latin */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 400;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-latin-400.woff2") format("woff2");
  unicode-range: U+0000-00FF, U+0131, U+0152-0153, U+02BB-02BC, U+02C6, U+02DA, U+02DC, U+0304, U+0308, U+0329, U+2000-206F, U+20AC, U+2122, U+2191, U+2193, U+2212, U+2215, U+FEFF, U+FFFD;
}

/* cyrillic-ext */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 500;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-cyrillic-ext-500.woff2") format("woff2");
  unicode-range: U+0460-052F, U+1C80-1C8A, U+20B4, U+2DE0-2DFF, U+A640-A69F, U+FE2E-FE2F;
}

/* cyrillic */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 500;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-cyrillic-500.woff2") format("woff2");
  unicode-range: U+0301, U+0400-045F, U+0490-0491, U+04B0-04B1, U+2116;
}

/* vietnamese */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 500;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-vietnamese-500.woff2") format("woff2");
  unicode-range: U+0102-0103, U+0110-0111, U+0128-0129, U+0168-0169, U+01A0-01A1, U+01AF-01B0, U+0300-0301, U+0303-0304, U+0308-0309, U+0323, U+0329, U+1EA0-1EF9, U+20AB;
}

/* latin-ext */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 500;
```

```
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-latin-ext-500.woff2") format("woff2");
  unicode-range: U+0100-02BA, U+02BD-02C5, U+02C7-02CC, U+02CE-02D7, U+02DD-02FF, U+0304, U+0308, U+0329, U+1D00-1DBF, U+1E00-1E9F, U+1EF2-1EFF, U+2020, U+20A0-20AB, U+20AD-20C0, U+2113, U+2C60-2C7F,
}

/* latin */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 500;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-latin-500.woff2") format("woff2");
  unicode-range: U+0000-00FF, U+0131, U+0152-0153, U+02BB-02BC, U+02C6, U+02DA, U+02DC, U+0304, U+0308, U+0329, U+2000-206F, U+20AC, U+2122, U+2191, U+2193, U+2212, U+2215, U+FEFF, U+FFFD;
}

/* cyrillic-ext */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 600;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-cyrillic-ext-600.woff2") format("woff2");
  unicode-range: U+0460-052F, U+1C80-1C8A, U+20B4, U+2DE0-2DFF, U+A640-A69F, U+FE2E-FE2F;
}

/* cyrillic */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 600;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-cyrillic-600.woff2") format("woff2");
  unicode-range: U+0301, U+0400-045F, U+0490-0491, U+04B0-04B1, U+2116;
}

/* vietnamese */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 600;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-vietnamese-600.woff2") format("woff2");
  unicode-range: U+0102-0103, U+0110-0111, U+0128-0129, U+0168-0169, U+01A0-01A1, U+01AF-01B0, U+0300-0301, U+0303-0304, U+0308-0309, U+0323, U+0329, U+1EA0-1EF9, U+20AB;
}

/* latin-ext */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 600;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-latin-ext-600.woff2") format("woff2");
  unicode-range: U+0100-02BA, U+02BD-02C5, U+02C7-02CC, U+02CE-02D7, U+02DD-02FF, U+0304, U+0308, U+0329, U+1D00-1DBF, U+1E00-1E9F, U+1EF2-1EFF, U+2020, U+20A0-20AB, U+20AD-20C0, U+2113, U+2C60-2C7F,
}

/* latin */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 600;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-latin-600.woff2") format("woff2");
  unicode-range: U+0000-00FF, U+0131, U+0152-0153, U+02BB-02BC, U+02C6, U+02DA, U+02DC, U+0304, U+0308, U+0329, U+2000-206F, U+20AC, U+2122, U+2191, U+2193, U+2212, U+2215, U+FEFF, U+FFFD;
}

/* cyrillic-ext */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 700;
```

```
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-cyrillic-ext-700.woff2") format("woff2");
  unicode-range: U+0460-052F, U+1C80-1C8A, U+20B4, U+2DE0-2DFF, U+A640-A69F, U+FE2E-FE2F;
}

/* cyrillic */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 700;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-cyrillic-700.woff2") format("woff2");
  unicode-range: U+0301, U+0400-045F, U+0490-0491, U+04B0-04B1, U+2116;
}

/* vietnamese */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 700;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-vietnamese-700.woff2") format("woff2");
  unicode-range: U+0102-0103, U+0110-0111, U+0128-0129, U+0168-0169, U+01A0-01A1, U+01AF-01B0, U+0300-0301, U+0303-0304, U+0308-0309, U+0323, U+0329, U+1EA0-1EF9, U+20AB;
}

/* latin-ext */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 700;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-latin-ext-700.woff2") format("woff2");
  unicode-range: U+0100-02BA, U+02BD-02C5, U+02C7-02CC, U+02CE-02D7, U+02DD-02FF, U+0304, U+0308, U+0329, U+1D00-1DBF, U+1E00-1E9F, U+1EF2-1EFF, U+2020, U+20A0-20AB, U+20AD-20C0, U+2113, U+2C60-2C7F,
}

/* latin */
@font-face {
  font-family: "IBM Plex Mono";
  font-style: normal;
  font-weight: 700;
  font-display: swap;
  src: url("./fonts/ibm-plex-mono/ibm-plex-mono-latin-700.woff2") format("woff2");
  unicode-range: U+0000-00FF, U+0131, U+0152-0153, U+02BB-02BC, U+02C6, U+02DA, U+02DC, U+0304, U+0308, U+0329, U+2000-206F, U+20AC, U+2122, U+2191, U+2193, U+2212, U+2215, U+FEFF, U+FFFD;
}
```

app/packages/quanta/src/css/theme/fonts/ibm-plex-mono/OFL.txt

```
Copyright © 2017 IBM Corp. with Reserved Font Name "Plex".
https://github.com/IBM/plex

This Font Software is licensed under the SIL Open Font License, Version 1.1.
This license is copied below, and is also available with a FAQ at:
https://openfontlicense.org

-----------------------------------------------------------
SIL OPEN FONT LICENSE Version 1.1 - 26 February 2007
-----------------------------------------------------------

PREAMBLE
The goals of the Open Font License (OFL) are to stimulate worldwide
development of collaborative font projects, to support the font creation
efforts of academic and linguistic communities, and to provide a free and
open framework in which fonts may be shared and improved in partnership
with others.

The OFL allows the licensed fonts to be used, studied, modified and
redistributed freely as long as they are not sold by themselves. The
fonts, including any derivative works, can be bundled, embedded,
redistributed and/or sold with any software provided that any reserved
names are not used by derivative works. The fonts and derivatives,
however, cannot be released under any other type of license. The
requirement for fonts to remain under this license does not apply to any
document created using the fonts or their derivatives.

DEFINITIONS
"Font Software" refers to the set of files released by the Copyright
Holder(s) under this license and clearly marked as such. This may include
source files, build scripts and documentation.

"Reserved Font Name" refers to any names specified as such after the
copyright statement(s).

"Original Version" refers to the collection of Font Software components as
distributed by the Copyright Holder(s).

"Modified Version" refers to any derivative made by adding to, deleting,
or substituting -- in part or in whole -- any of the components of the
Original Version, by changing formats or by porting the Font Software to a
new environment.

"Author" refers to any designer, engineer, programmer, technical writer or
other person who contributed to the Font Software.

PERMISSION & CONDITIONS
Permission is hereby granted, free of charge, to any person obtaining a
copy of the Font Software, to use, study, copy, merge, embed, modify,
redistribute, and sell modified and unmodified copies of the Font
Software, subject to the following conditions:

1) Neither the Font Software nor any of its individual components, in
Original or Modified Versions, may be sold by itself.

2) Original or Modified Versions of the Font Software may be bundled,
redistributed and/or sold with any software, provided that each copy
contains the above copyright notice and this license. These can be
included either as stand-alone text files, human-readable headers or in
the appropriate machine-readable metadata fields within text or binary
files as long as those fields can be easily viewed by the user.

3) No Modified Version of the Font Software may use the Reserved Font
Name(s) unless explicit written permission is granted by the corresponding
Copyright Holder. This restriction only applies to the primary font name as
presented to the users.

4) The name(s) of the Copyright Holder(s) or the Author(s) of the Font
```

Software shall not be used to promote, endorse or advertise any Modified Version, except to acknowledge the contribution(s) of the Copyright Holder(s) and the Author(s) or with their explicit written permission.

5) The Font Software, modified or unmodified, in part or in whole, must be distributed entirely under this license, and must not be distributed under any other license. The requirement for fonts to remain under this license does not apply to any document created using the Font Software.

TERMINATION
This license becomes null and void if any of the above conditions are not met.

DISCLAIMER
THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

app/packages/quanta/src/css/theme/fonts/inter/OFL.txt

```
Copyright (c) 2016 The Inter Project Authors (https://github.com/rsms/inter)

This Font Software is licensed under the SIL Open Font License, Version 1.1.
This license is copied below, and is also available with a FAQ at:
http://scripts.sil.org/OFL

-----------------------------------------------------------
SIL OPEN FONT LICENSE Version 1.1 - 26 February 2007
-----------------------------------------------------------

PREAMBLE
The goals of the Open Font License (OFL) are to stimulate worldwide
development of collaborative font projects, to support the font creation
efforts of academic and linguistic communities, and to provide a free and
open framework in which fonts may be shared and improved in partnership
with others.

The OFL allows the licensed fonts to be used, studied, modified and
redistributed freely as long as they are not sold by themselves. The
fonts, including any derivative works, can be bundled, embedded,
redistributed and/or sold with any software provided that any reserved
names are not used by derivative works. The fonts and derivatives,
however, cannot be released under any other type of license. The
requirement for fonts to remain under this license does not apply
to any document created using the fonts or their derivatives.

DEFINITIONS
"Font Software" refers to the set of files released by the Copyright
Holder(s) under this license and clearly marked as such. This may
include source files, build scripts and documentation.

"Reserved Font Name" refers to any names specified as such after the
copyright statement(s).

"Original Version" refers to the collection of Font Software components as
distributed by the Copyright Holder(s).

"Modified Version" refers to any derivative made by adding to, deleting,
or substituting -- in part or in whole -- any of the components of the
Original Version, by changing formats or by porting the Font Software to a
new environment.

"Author" refers to any designer, engineer, programmer, technical
writer or other person who contributed to the Font Software.

PERMISSION AND CONDITIONS
Permission is hereby granted, free of charge, to any person obtaining
a copy of the Font Software, to use, study, copy, merge, embed, modify,
redistribute, and sell modified and unmodified copies of the Font
Software, subject to the following conditions:

1) Neither the Font Software nor any of its individual components,
in Original or Modified Versions, may be sold by itself.

2) Original or Modified Versions of the Font Software may be bundled,
redistributed and/or sold with any software, provided that each copy
contains the above copyright notice and this license. These can be
included either as stand-alone text files, human-readable headers or
in the appropriate machine-readable metadata fields within text or
binary files as long as those fields can be easily viewed by the user.

3) No Modified Version of the Font Software may use the Reserved Font
Name(s) unless explicit written permission is granted by the corresponding
Copyright Holder. This restriction only applies to the primary font name as
presented to the users.

4) The name(s) of the Copyright Holder(s) or the Author(s) of the Font
Software shall not be used to promote, endorse or advertise any
```

Modified Version, except to acknowledge the contribution(s) of the Copyright Holder(s) and the Author(s) or with their explicit written permission.

5) The Font Software, modified or unmodified, in part or in whole, must be distributed entirely under this license, and must not be distributed under any other license. The requirement for fonts to remain under this license does not apply to any document created using the Font Software.

TERMINATION
This license becomes null and void if any of the above conditions are not met.

DISCLAIMER
THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS IN THE FONT SOFTWARE.

app/packages/quanta/src/css/theme/fonts/space-grotesk/OFL.txt

```
Copyright 2020 The Space Grotesk Project Authors
(https://github.com/floriankarsten/space-grotesk),
with Reserved Font Name "Space Grotesk".

This Font Software is licensed under the SIL Open Font License, Version 1.1.
This license is copied below, and is also available with a FAQ at:
https://openfontlicense.org

-----------------------------------------------------------
SIL OPEN FONT LICENSE Version 1.1 - 26 February 2007
-----------------------------------------------------------

PREAMBLE
The goals of the Open Font License (OFL) are to stimulate worldwide
development of collaborative font projects, to support the font creation
efforts of academic and linguistic communities, and to provide a free and
open framework in which fonts may be shared and improved in partnership
with others.

The OFL allows the licensed fonts to be used, studied, modified and
redistributed freely as long as they are not sold by themselves. The
fonts, including any derivative works, can be bundled, embedded,
redistributed and/or sold with any software provided that any reserved
names are not used by derivative works. The fonts and derivatives,
however, cannot be released under any other type of license. The
requirement for fonts to remain under this license does not apply to any
document created using the fonts or their derivatives.

DEFINITIONS
"Font Software" refers to the set of files released by the Copyright
Holder(s) under this license and clearly marked as such. This may include
source files, build scripts and documentation.

"Reserved Font Name" refers to any names specified as such after the
copyright statement(s).

"Original Version" refers to the collection of Font Software components as
distributed by the Copyright Holder(s).

"Modified Version" refers to any derivative made by adding to, deleting,
or substituting -- in part or in whole -- any of the components of the
Original Version, by changing formats or by porting the Font Software to a
new environment.

"Author" refers to any designer, engineer, programmer, technical writer or
other person who contributed to the Font Software.

PERMISSION & CONDITIONS
Permission is hereby granted, free of charge, to any person obtaining a
copy of the Font Software, to use, study, copy, merge, embed, modify,
redistribute, and sell modified and unmodified copies of the Font
Software, subject to the following conditions:

1) Neither the Font Software nor any of its individual components, in
Original or Modified Versions, may be sold by itself.

2) Original or Modified Versions of the Font Software may be bundled,
redistributed and/or sold with any software, provided that each copy
contains the above copyright notice and this license. These can be
included either as stand-alone text files, human-readable headers or in
the appropriate machine-readable metadata fields within text or binary
files as long as those fields can be easily viewed by the user.

3) No Modified Version of the Font Software may use the Reserved Font
Name(s) unless explicit written permission is granted by the corresponding
Copyright Holder. This restriction only applies to the primary font name as
presented to the users.
```

```
4) The name(s) of the Copyright Holder(s) or the Author(s) of the Font
Software shall not be used to promote, endorse or advertise any Modified
Version, except to acknowledge the contribution(s) of the Copyright
Holder(s) and the Author(s) or with their explicit written permission.

5) The Font Software, modified or unmodified, in part or in whole, must be
distributed entirely under this license, and must not be distributed under
any other license. The requirement for fonts to remain under this license
does not apply to any document created using the Font Software.

TERMINATION
This license becomes null and void if any of the above conditions are not
met.

DISCLAIMER
THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT OF
COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE
COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL
DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM OTHER DEALINGS
IN THE FONT SOFTWARE.
```

app/packages/quanta/src/css/theme/index.css

```
/**
 * Layer 2 — Semantic (themed) (manifest)
 *
 * Semantic hf-* tokens. References primitives only, via var(...).
 * No Tailwind dependency — works in any CSS project.
 *
 * Contract: `data-theme="{brand}-{mode}"`. Quanta ships exactly one brand —
 * `default` (light + dark). Multi-brand is the consumer's extension point:
 * write your own `:where([data-theme="acme-light"])` blocks in your CSS.
 *
 * "Auto" (follow system) is resolved at runtime via runtime/controller.ts;
 * it is not a separate CSS theme.
 *
 * Selector: `:where(...)` — zero specificity. Consumer classes and nested
 * `data-theme` overrides win without specificity wars.
 * See spec/decisions.md → R1.
 */

@import "../primitives/index.css";

@import "./fonts.css";
@import "./color.css";
/* radius — global, no theme layer (primitives + tailwind only, like spacing/z-index) */
/* @import "./spacing.css";    — TBD */
/* shadow — global, no theme layer (primitives + tailwind only); each layer's
 * color is a var(--hf-color-*) ref themed downstream, geometry is global */
@import "./typography.css";
/* motion — global, no theme layer (primitives + tailwind only) */
```

app/packages/quanta/src/css/theme/typography.css

```
/**
 * Generated by quanta codegen — DO NOT EDIT.
 * Regenerate via `yarn workspace @higgsfield/quanta tokens:emit`.
 *
 * Source: @higgsfield-ai/design-tokens (Figma Variables).
 */

:where(:root) {
  --hf-type-family-grotesk: var(--hf-type-family-grotesk-base), sans-serif;
  --hf-type-family-mono: var(--hf-type-family-mono-base), ui-monospace, monospace;
  --hf-type-family-primary: var(--hf-type-family-primary-base), system-ui, sans-serif;
  --hf-type-family-secondary: var(--hf-type-family-secondary-base), system-ui, sans-serif;

  --hf-type-size-050: 0.625rem;
  --hf-type-size-100: 0.75rem;
  --hf-type-size-200: 0.875rem;
  --hf-type-size-300: 1rem;
  --hf-type-size-400: 1rem;
  --hf-type-size-500: 1.125rem;
  --hf-type-size-600: 1.25rem;
  --hf-type-size-700: 1.5rem;
  --hf-type-size-800: 1.75rem;
  --hf-type-size-900: 1.875rem;
  --hf-type-size-1000: 2rem;
  --hf-type-size-1100: 2.25rem;
  --hf-type-size-1200: 2.5rem;
  --hf-type-size-1300: 3rem;
  --hf-type-size-1400: 3.5rem;

  --hf-type-line-height-100: 0.75rem;
  --hf-type-line-height-200: 0.875rem;
  --hf-type-line-height-300: 1rem;
  --hf-type-line-height-400: 1rem;
  --hf-type-line-height-500: 1.125rem;
  --hf-type-line-height-600: 1.25rem;
  --hf-type-line-height-700: 1.5rem;
  --hf-type-line-height-800: 1.75rem;
  --hf-type-line-height-900: 1.875rem;
  --hf-type-line-height-1000: 2rem;
  --hf-type-line-height-1100: 2.25rem;
  --hf-type-line-height-1200: 2.5rem;
  --hf-type-line-height-1300: 3rem;
  --hf-type-line-height-1400: 3.5rem;
}

@media (min-width: 768px) {
  :where(:root) {
    --hf-type-size-400: 1.125rem;
    --hf-type-size-500: 1.25rem;
    --hf-type-size-600: 1.375rem;
    --hf-type-size-700: 1.625rem;
    --hf-type-size-800: 1.875rem;
    --hf-type-size-900: 2.125rem;
    --hf-type-size-1000: 2.25rem;
    --hf-type-size-1100: 2.625rem;
    --hf-type-size-1200: 3rem;
    --hf-type-size-1300: 3.5rem;
    --hf-type-size-1400: 4rem;
    --hf-type-line-height-400: 1.125rem;
    --hf-type-line-height-500: 1.25rem;
    --hf-type-line-height-600: 1.375rem;
    --hf-type-line-height-700: 1.625rem;
    --hf-type-line-height-800: 1.875rem;
    --hf-type-line-height-900: 2.125rem;
    --hf-type-line-height-1000: 2.25rem;
    --hf-type-line-height-1100: 2.625rem;
    --hf-type-line-height-1200: 3rem;
    --hf-type-line-height-1300: 3.5rem;
```

```
    --hf-type-line-height-1400: 4rem;
  }
}

@media (min-width: 1280px) {
  :where(:root) {
    --hf-type-size-600: 1.5rem;
    --hf-type-size-700: 1.75rem;
    --hf-type-size-800: 2rem;
    --hf-type-size-900: 2.25rem;
    --hf-type-size-1000: 2.5rem;
    --hf-type-size-1100: 3rem;
    --hf-type-size-1200: 3.5rem;
    --hf-type-size-1300: 4rem;
    --hf-type-size-1400: 4.5rem;
    --hf-type-line-height-600: 1.5rem;
    --hf-type-line-height-700: 1.75rem;
    --hf-type-line-height-800: 2rem;
    --hf-type-line-height-900: 2.25rem;
    --hf-type-line-height-1000: 2.5rem;
    --hf-type-line-height-1100: 3rem;
    --hf-type-line-height-1200: 3.5rem;
    --hf-type-line-height-1300: 4rem;
    --hf-type-line-height-1400: 4.5rem;
  }
}
```

app/packages/quanta/src/runtime/bootstrap.ts

```
/**
 * Anti-FOUC snippet generator — opinionated recipe implementation.
 *
 * Generates an inline <head> script that sets data-theme before first
 * paint. Used in SSR frameworks (Next.js, Astro, SvelteKit) via
 * dangerouslySetInnerHTML or is:inline.
 *
 * What the script does, in order:
 *   1. Reads persisted runtime themes from localStorage and injects
 *      <style> tags before paint — otherwise an override pinned to
 *      "ai-ocean" would apply before :where([data-theme="ai-ocean"])
 *      exists in the DOM.
 *   2. Reads the pinned override (if any) and applies it.
 *   3. If no override is set, resolves the managed theme from pref +
 *      prefers-color-scheme.
 *
 * Usage (Next.js):
 *   import { bootstrapScript } from "@higgsfield/quanta/runtime";
 *
 *   <script nonce={nonce} dangerouslySetInnerHTML={{ __html: bootstrapScript() }} />
 *
 * Usage (Astro):
 *   <script is:inline set:html={bootstrapScript()} />
 */

import {
  DEFAULT_OVERRIDE_STORAGE_KEY,
  DEFAULT_STORAGE_KEY,
  DEFAULT_THEMES_STORAGE_KEY,
} from './storage-keys.ts'

export interface BootstrapOptions {
  brand?: string
  /** localStorage key for the user's mode pref. Default 'hf:quanta:theme-pref'. */
  storageKey?: string
  /** localStorage key for the pinned override. Default 'hf:quanta:theme-override'. */
  overrideStorageKey?: string
  /** localStorage key for the runtime themes map. Default 'hf:quanta:runtime-themes'. */
  themesStorageKey?: string
}

export function bootstrapScript({
  brand = 'default',
  storageKey = DEFAULT_STORAGE_KEY,
  overrideStorageKey = DEFAULT_OVERRIDE_STORAGE_KEY,
  themesStorageKey = DEFAULT_THEMES_STORAGE_KEY,
}: BootstrapOptions = {}): string {
  return `(function () {
  var currentScript = document.currentScript;
  var styleNonce = currentScript && currentScript.nonce ? currentScript.nonce : "";

  // 1. Re-inject persisted runtime themes (defineTheme'd in a prior session).
  var themes = {};
  try { themes = JSON.parse(localStorage.getItem(${JSON.stringify(themesStorageKey)}) || "{}"); } catch (e) {}
  for (var name in themes) {
    if (!Object.prototype.hasOwnProperty.call(themes, name)) continue;
    var decls = "";
    for (var key in themes[name]) {
      if (!Object.prototype.hasOwnProperty.call(themes[name], key)) continue;
      decls += "  --hf-color-" + key + ": " + themes[name][key] + ";\\n";
    }
    var style = document.createElement("style");
    style.id = "hf-runtime-theme-" + name;
    // :where(...) keeps specificity at (0,0,0) so consumer .my-class
    // overrides win. Runtime themes win the cascade tie not by specificity
    // but by being UNLAYERED while theme/color.css is wrapped in
    // @layer quanta-theme — unlayered always beats layered per CSS spec.
    // See define-theme.ts → injectStyleTag for the rationale.
```

```
    if (styleNonce) style.nonce = styleNonce;
    style.textContent = ":where([data-theme=\\"" + name + "\\"]) {\\n" + decls + "}";
    document.head.appendChild(style);
  }

  // 2. Apply override if pinned, otherwise resolve managed theme.
  var override = null;
  try { override = localStorage.getItem(${JSON.stringify(overrideStorageKey)}); } catch (e) {}
  var html = document.documentElement;
  if (override) {
    html.setAttribute("data-theme", override);

    // 3a. Anti-FOUC fallback for runtime themes. The Tailwind stylesheet
    // may load after first paint (esp. in dev where Vite injects CSS via
    // JS) — without this, body would flash with browser defaults before
    // the cascade resolves. Inline html bg+color from the override's
    // tokens covers that window; once the proper class on <body> arrives,
    // it paints over and the inline html style becomes invisible.
    var ot = themes[override];
    if (ot) {
      if (ot["background-primary"]) html.style.backgroundColor = ot["background-primary"];
      if (ot["text-primary"]) html.style.color = ot["text-primary"];
    }
  } else {
    var pref = "auto";
    try { pref = localStorage.getItem(${JSON.stringify(storageKey)}) || "auto"; } catch (e) {}
    // Package default is DARK. Render light ONLY when:
    //   - user explicitly chose pref="light", OR
    //   - pref="auto" and OS explicitly signals prefers-color-scheme: light
    // Any other signal (dark, none, matchMedia missing) resolves to dark.
    var light = pref === "light" || (pref === "auto" && matchMedia("(prefers-color-scheme: light)").matches);
    var dark = !light;
    html.setAttribute("data-theme", ${JSON.stringify(brand)} + "-" + (dark ? "dark" : "light"));

    // 3b. Anti-FOUC for managed themes. Setting color-scheme tells the
    // browser to use dark/light system defaults for the canvas (page
    // background, native form controls) — no white flash before the
    // proper theme/color.css block sets the same property inside its
    // :where(...) block.
    html.style.colorScheme = dark ? "dark" : "light";
  }
})();`
}
```

app/packages/quanta/src/runtime/controller.test.ts

```
import { beforeEach, describe, expect, it, vi } from 'vitest'
import { readInitialThemeState } from './controller.ts'
import {
  DEFAULT_OVERRIDE_STORAGE_KEY,
  DEFAULT_STORAGE_KEY,
} from './storage-keys.ts'
import { memoryStorage } from './storage.ts'

/**
 * `prefers-color-scheme` is a 3-state signal: dark, light, or none.
 * Mock both query forms so tests can express each state explicitly.
 */
function mockMatchMedia(systemPref: 'dark' | 'light' | 'none') {
  window.matchMedia = vi.fn().mockImplementation((q: string) => ({
    matches: q.includes('dark')
      ? systemPref === 'dark'
      : q.includes('light')
        ? systemPref === 'light'
        : false,
    media: q,
    addEventListener: () => {},
    removeEventListener: () => {},
    addListener: () => {},
    removeListener: () => {},
    dispatchEvent: () => true,
    onchange: null,
  }))
}

describe('State machine behavior', () => {
  describe('readInitialThemeState', () => {
    beforeEach(() => {
      localStorage.clear()
      mockMatchMedia('none')
    })

    it('defaults to pref=auto, dark when storage empty + no system pref', () => {
      const s = readInitialThemeState()
      expect(s).toEqual({
        pref: 'auto',
        brand: 'default',
        resolvedTheme: 'default-dark',
        override: null,
        activeTheme: 'default-dark',
      })
    })

    it('honours stored pref=dark', () => {
      localStorage.setItem(DEFAULT_STORAGE_KEY, 'dark')
      const s = readInitialThemeState()
      expect(s.pref).toBe('dark')
      expect(s.resolvedTheme).toBe('default-dark')
      expect(s.activeTheme).toBe('default-dark')
    })

    it('resolves auto + system dark MQ → dark', () => {
      mockMatchMedia('dark')
      const s = readInitialThemeState()
      expect(s.pref).toBe('auto')
      expect(s.resolvedTheme).toBe('default-dark')
    })

    it('resolves auto + system light MQ → light', () => {
      mockMatchMedia('light')
      const s = readInitialThemeState()
      expect(s.pref).toBe('auto')
      expect(s.resolvedTheme).toBe('default-light')
    })
```

```
it('treats garbage pref as auto', () => {
  localStorage.setItem(DEFAULT_STORAGE_KEY, 'midnight')
  const s = readInitialThemeState()
  expect(s.pref).toBe('auto')
})

it('respects custom brand', () => {
  const s = readInitialThemeState({ brand: 'acme' })
  expect(s.brand).toBe('acme')
  expect(s.resolvedTheme).toBe('acme-dark')
})

it('override wins over resolved theme for activeTheme; resolved stays', () => {
  localStorage.setItem(DEFAULT_STORAGE_KEY, 'dark')
  localStorage.setItem(DEFAULT_OVERRIDE_STORAGE_KEY, 'ai-ocean')
  const s = readInitialThemeState()
  expect(s.resolvedTheme).toBe('default-dark')
  expect(s.override).toBe('ai-ocean')
  expect(s.activeTheme).toBe('ai-ocean')
})

it('reads override from injected storage adapter, not localStorage', () => {
  const override = memoryStorage()
  override.set(DEFAULT_OVERRIDE_STORAGE_KEY, 'from-memory')
  localStorage.setItem(DEFAULT_OVERRIDE_STORAGE_KEY, 'from-localstorage')
  const s = readInitialThemeState({ overrideStorage: override })
  expect(s.override).toBe('from-memory')
  expect(s.activeTheme).toBe('from-memory')
})

it('honours custom storage keys', () => {
  localStorage.setItem('my-pref', 'dark')
  localStorage.setItem('my-override', 'pinned')
  const s = readInitialThemeState({
    storageKey: 'my-pref',
    overrideStorageKey: 'my-override',
  })
  expect(s.pref).toBe('dark')
  expect(s.override).toBe('pinned')
})

it('SSR fallback: returns defaults when window is undefined', () => {
  const w = globalThis.window
  // @ts-expect-error — simulating SSR
  delete globalThis.window
  try {
    const s = readInitialThemeState({ brand: 'acme' })
    expect(s).toEqual({
      pref: 'auto',
      brand: 'acme',
      resolvedTheme: 'acme-dark',
      override: null,
      activeTheme: 'acme-dark',
    })
  }
  finally {
    globalThis.window = w
  }
})

it('does not throw when localStorage.getItem throws (Safari Private Mode)', () => {
  const original = Storage.prototype.getItem
  Storage.prototype.getItem = () => {
    throw new Error('SecurityError')
  }
  try {
    const s = readInitialThemeState()
    expect(s.pref).toBe('auto')
    expect(s.resolvedTheme).toBe('default-dark')
  }
```

```
      finally {
        Storage.prototype.getItem = original
      }
    })

    it('does not throw when matchMedia is absent (partial DOM)', () => {
      const original = window.matchMedia
      // @ts-expect-error — simulating partial DOM where matchMedia is missing
      window.matchMedia = undefined
      try {
        const s = readInitialThemeState()
        expect(s.pref).toBe('auto')
        // matchMedia missing → fall through to package default (dark)
        expect(s.resolvedTheme).toBe('default-dark')
      }
      finally {
        window.matchMedia = original
      }
    })
  })
})
```

app/packages/quanta/src/runtime/controller.ts

```
/**
 * ThemeController — runtime that manages the `data-theme` attribute.
 *
 * Methods are split cleanly across two domains:
 *
 *   ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *   ■ Managed — quanta's built-in themes (`{brand}-{mode}` contract)   ■
 *   ■   setPref / getPref     — user preference: auto | light | dark   ■
 *   ■   setBrand / getBrand   — brand string (quanta ships "default")  ■
 *   ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *   ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *   ■ Escape hatch — for themes quanta doesn't know about              ■
 *   ■   setOverride(name)     — pin any `data-theme` value            ■
 *   ■   setOverride(null)     — unpin, return to managed mode         ■
 *   ■   getOverride()         — current pinned value (null if none)   ■
 *   ■                                                                 ■
 *   ■ Use cases:                                                      ■
 *   ■   - a theme from defineTheme() ("ai-ocean", AI-generated)       ■
 *   ■   - consumer-side brands not built into quanta CSS             ■
 *   ■   - preview / A/B testing of themes                             ■
 *   ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 *
 * Contract between the two domains:
 *   - `setOverride()` pins the DOM to any value. pref/brand state is
 *     **preserved** — after `setOverride(null)` the resolved theme returns.
 *   - `setPref()` / `setBrand()` update internal state while an override is
 *     active but do not touch the DOM (override wins).
 *   - System theme changes (`prefers-color-scheme`) apply only when
 *     pref="auto" AND no override is pinned.
 *
 * Read access to state is via `subscribe`. The callback receives the full
 * `ThemeState` (pref, brand, resolvedTheme, override, activeTheme) and is
 * invoked immediately with the current state on subscription.
 *
 * Usage:
 *   const controller = new ThemeController({ brand: "default" });
 *   controller.setPref("dark");
 *   controller.subscribe(state => render(state));
 *
 *   // dynamic theme escape hatch
 *   defineTheme("ai-ocean", {...});
 *   controller.setOverride("ai-ocean");
 *   // ... later
 *   controller.setOverride(null);
 *
 *   // SPA teardown
 *   controller.destroy();
 */

import type { ThemeStorage } from './storage.ts'
import { hydratePersistedThemes } from './define-theme.ts'
import {
  DEFAULT_OVERRIDE_STORAGE_KEY,
  DEFAULT_STORAGE_KEY,
} from './storage-keys.ts'
import { localStorageAdapter } from './storage.ts'

export type ThemePref = 'auto' | 'light' | 'dark'

export interface ThemeControllerOptions {
  brand?: string
  /** localStorage key for the user's mode pref. Default: 'hf:quanta:theme-pref'. */
  storageKey?: string
  /** Storage key for the pinned override. Default: 'hf:quanta:theme-override'. */
  overrideStorageKey?: string
  /**
   * Storage backend for the override pin. Default: `localStorageAdapter`.
   * Try `urlAdapter()` for shareable theme links, `sessionStorageAdapter`
```

```
   * for per-tab overrides, or any custom `ThemeStorage`.
   */
  overrideStorage?: ThemeStorage
  /**
   * Read/write override from storage so the pinned theme survives reload.
   * Default: true. Set false for purely ephemeral overrides (A/B testing,
   * preview sessions).
   */
  persistOverride?: boolean
  /**
   * Storage backend used when `hydrateThemes` calls `hydratePersistedThemes`
   * on construct. Should match the `storage` you pass to `defineTheme`.
   * Default: `localStorageAdapter`.
   */
  themesStorage?: ThemeStorage
  /**
   * On construct, call `hydratePersistedThemes()` so persisted runtime-theme
   * <style> tags are re-injected before the first apply. Default: true.
   * Set false if you call hydrate manually elsewhere.
   */
  hydrateThemes?: boolean
  /**
   * CSP nonce for runtime <style> tags injected while hydrating persisted
   * themes. Pass the same nonce your framework puts on the bootstrap script.
   */
  styleNonce?: string
}

export interface ThemeState {
  /** User's mode preference. Survives override. */
  pref: ThemePref
  /** Current brand string. Survives override. */
  brand: string
  /** Theme that would apply from brand+pref alone. May differ from activeTheme. */
  resolvedTheme: string
  /** Active override value, or null if no override is pinned. */
  override: string | null
  /** Value actually applied to `data-theme` on `<html>`. = override ?? resolvedTheme. */
  activeTheme: string
}

export type ThemeSubscriber = (state: ThemeState) => void

export interface ReadInitialThemeStateOptions {
  brand?: string
  /** localStorage key for the user's mode pref. Default: 'hf:quanta:theme-pref'. */
  storageKey?: string
  /** Storage key for the pinned override. Default: 'hf:quanta:theme-override'. */
  overrideStorageKey?: string
  /**
   * Storage backend for the override pin. Default: `localStorageAdapter`.
   * Must match the backend you pass to `ThemeController`.
   */
  overrideStorage?: ThemeStorage
}

const VALID_PREFS: ReadonlyArray<ThemePref> = ['auto', 'light', 'dark']

/**
 * Compute the same ThemeState that `new ThemeController()` would expose on
 * first `subscribe()`, but without any side effects — no listeners, no DOM
 * mutation. Designed for React `useState(() => readInitialThemeState(...))`
 * so the first render has the correct pref/override and we avoid a flash of
 * wrong UI state before the effect-bound controller fires its first update.
 *
 * Browser-only — reads localStorage and matchMedia. On the server, or when
 * either API throws (Safari Private Mode, locked-down iframes, partial DOM
 * environments), falls back to documented defaults instead of crashing the
 * first render. React `useState` lazy init bubbles exceptions to the
 * nearest error boundary, so we must never throw here.
```

```
 */
export function readInitialThemeState({
  brand = 'default',
  storageKey = DEFAULT_STORAGE_KEY,
  overrideStorageKey = DEFAULT_OVERRIDE_STORAGE_KEY,
  overrideStorage = localStorageAdapter,
}: ReadInitialThemeStateOptions = {}): ThemeState {
  if (typeof window === 'undefined') {
    const resolvedTheme = `${brand}-dark`
    return { pref: 'auto', brand, resolvedTheme, override: null, activeTheme: resolvedTheme }
  }
  let storedPref: string | null = null
  try {
    storedPref = localStorage.getItem(storageKey)
  }
  catch {
    // privacy mode / SecurityError — fall through to 'auto'
  }
  const pref: ThemePref = VALID_PREFS.includes(storedPref as ThemePref)
    ? (storedPref as ThemePref)
    : 'auto'
  const override = overrideStorage.get(overrideStorageKey)
  // Package default is DARK. For pref='auto' we resolve to light ONLY when the
  // OS explicitly signals a light preference; any other signal (dark, none,
  // matchMedia missing) resolves to dark.
  let lightSystem = false
  if (pref === 'auto') {
    try {
      lightSystem = matchMedia('(prefers-color-scheme: light)').matches
    }
    catch {
      // matchMedia absent (partial DOM, jsdom edge cases) — fall through to dark
    }
  }
  const mode = pref === 'auto' ? (lightSystem ? 'light' : 'dark') : pref
  const resolvedTheme = `${brand}-${mode}`
  return { pref, brand, resolvedTheme, override, activeTheme: override ?? resolvedTheme }
}

export class ThemeController {
  #brand: string
  #storageKey: string
  #overrideStorageKey: string
  #overrideStorage: ThemeStorage
  #persistOverride: boolean
  #override: string | null = null
  #mq: MediaQueryList
  #onSystemChange: () => void
  #subscribers = new Set<ThemeSubscriber>()

  constructor({
    brand = 'default',
    storageKey = DEFAULT_STORAGE_KEY,
    overrideStorageKey = DEFAULT_OVERRIDE_STORAGE_KEY,
    overrideStorage = localStorageAdapter,
    persistOverride = true,
    themesStorage = localStorageAdapter,
    hydrateThemes = true,
    styleNonce,
  }: ThemeControllerOptions = {}) {
    this.#brand = brand
    this.#storageKey = storageKey
    this.#overrideStorageKey = overrideStorageKey
    this.#overrideStorage = overrideStorage
    this.#persistOverride = persistOverride
    this.#mq = matchMedia('(prefers-color-scheme: dark)')

    // Re-inject persisted runtime-theme <style> tags BEFORE reading the
    // override — otherwise an override pointing at "ai-ocean" would apply
    // before its CSS block is in the DOM, causing an unstyled flash.
```

```
  if (hydrateThemes)
    hydratePersistedThemes({ storage: themesStorage, styleNonce })

  if (this.#persistOverride) {
    const stored = this.#overrideStorage.get(this.#overrideStorageKey)
    if (stored)
      this.#override = stored
  }

  this.#onSystemChange = () => {
    // System change matters only if managed-mode is driving the DOM
    // (pref="auto" and no override pinning).
    if (this.#override === null && this.#readPref() === 'auto')
      this.#apply()
  }
  this.#mq.addEventListener('change', this.#onSystemChange)

  this.#apply()
}

// ═══════════════════════════════════════════════════════════════════════════
// Internal
// ═══════════════════════════════════════════════════════════════════════════

#readPref(): ThemePref {
  const stored = localStorage.getItem(this.#storageKey)
  return VALID_PREFS.includes(stored as ThemePref) ? (stored as ThemePref) : 'auto'
}

#resolveTheme(pref: ThemePref): string {
  if (pref !== 'auto')
    return `${this.#brand}-${pref}`
  // Package default is DARK. We resolve auto → light only when the OS
  // explicitly prefers light; any other signal (dark, none) → dark.
  const lightSystem = matchMedia('(prefers-color-scheme: light)').matches
  return `${this.#brand}-${lightSystem ? 'light' : 'dark'}`
}

#computeState(): ThemeState {
  const pref = this.#readPref()
  const resolvedTheme = this.#resolveTheme(pref)
  const activeTheme = this.#override ?? resolvedTheme
  return {
    pref,
    brand: this.#brand,
    resolvedTheme,
    override: this.#override,
    activeTheme,
  }
}

#apply(): void {
  const state = this.#computeState()
  document.documentElement.dataset.theme = state.activeTheme
  this.#subscribers.forEach(cb => cb(state))
}

// ═══════════════════════════════════════════════════════════════════════════
// Managed — built-in themes ({brand}-{mode} contract)
// ═══════════════════════════════════════════════════════════════════════════

setPref(pref: ThemePref): void {
  if (!VALID_PREFS.includes(pref)) {
    throw new Error(
      `setPref: invalid pref "${pref}". Expected one of: ${VALID_PREFS.join(', ')}.`,
    )
  }
  localStorage.setItem(this.#storageKey, pref)
  this.#apply()
}
```

```
  getPref(): ThemePref {
    return this.#readPref()
  }

  setBrand(brand: string): void {
    if (typeof brand !== 'string' || !brand) {
      throw new Error('setBrand: brand must be a non-empty string.')
    }
    this.#brand = brand
    this.#apply()
  }

  getBrand(): string {
    return this.#brand
  }

  // ═══════════════════════════════════════════════════════════════════════
  // Escape hatch — opaque-to-quanta theme value
  // ═══════════════════════════════════════════════════════════════════════

  /**
   * Pin `data-theme` to any string. Bypasses brand+pref resolution.
   * Pass `null` to clear and resume managed resolution.
   *
   * Use for:
   *   - dynamic themes defined via `defineTheme()` (e.g. "ai-ocean")
   *   - consumer-side brand themes not built into quanta
   *   - preview / A/B-test themes
   *
   * pref and brand state are preserved while an override is active.
   */
  setOverride(theme: string | null): void {
    if (theme !== null && (typeof theme !== 'string' || !theme)) {
      throw new Error('setOverride: expected non-empty string or null.')
    }
    if (this.#override === theme)
      return
    this.#override = theme
    if (this.#persistOverride) {
      if (theme === null)
        this.#overrideStorage.remove(this.#overrideStorageKey)
      else
        this.#overrideStorage.set(this.#overrideStorageKey, theme)
    }
    this.#apply()
  }

  getOverride(): string | null {
    return this.#override
  }

  // ═══════════════════════════════════════════════════════════════════════
  // Subscription / lifecycle
  // ═══════════════════════════════════════════════════════════════════════

  subscribe(callback: ThemeSubscriber): () => void {
    this.#subscribers.add(callback)
    callback(this.#computeState())
    return () => this.#subscribers.delete(callback)
  }

  destroy(): void {
    this.#mq.removeEventListener('change', this.#onSystemChange)
    this.#subscribers.clear()
  }
}
```

app/packages/quanta/src/runtime/define-theme.test.ts

```
import { afterEach, describe, expect, it } from 'vitest'
import { defineTheme, hasTheme, hydratePersistedThemes, listThemes, removeTheme } from './define-theme.ts'
import { DEFAULT_THEMES_STORAGE_KEY } from './storage-keys.ts'
import { memoryStorage } from './storage.ts'

afterEach(() => {
  for (const name of listThemes()) removeTheme(name)
})

describe('defineTheme — injection guard', () => {
  it('injects a :where block for valid tokens (the happy path still works)', () => {
    defineTheme('ai-ocean', {
      'surface-default': '#0c2461',
      'text-primary': 'oklch(0.92 0.02 240)',
      'brand-primary': 'var(--hf-color-fallback, #fbbf24)',
      'glow': 'linear-gradient(90deg, #000 0%, #fff 100%)',
    }, { persist: false })

    const tag = document.getElementById('hf-runtime-theme-ai-ocean')
    expect(tag?.textContent).toContain(':where([data-theme="ai-ocean"])')
    expect(tag?.textContent).toContain('--hf-color-glow: linear-gradient(90deg, #000 0%, #fff 100%);')
  })

  it.each([
    ['a `}` in a value escapes the block', { 'surface-default': '#fff} body{background:url(//evil)' }],
    ['a `;` in a value smuggles a sibling declaration', { 'surface-default': 'red; background-image: url(//evil)' }],
    ['a control character in a value', { 'surface-default': 'red\u0000' }],
  ])('rejects %s', (_label, tokens) => {
    expect(() => defineTheme('attack', tokens, { persist: false })).toThrowError(/invalid value/)
    expect(document.getElementById('hf-runtime-theme-attack')).toBeNull()
    expect(hasTheme('attack')).toBe(false)
  })

  it('rejects a token key that is not slug-shaped', () => {
    expect(() => defineTheme('t', { 'a:red;b': '#fff' }, { persist: false })).toThrowError(/invalid token key/)
  })

  it('rejects a theme name that would escape the attribute selector', () => {
    expect(() => defineTheme('x"]{}', { a: '#fff' }, { persist: false })).toThrowError(/invalid theme name/)
  })

  it('a bad redefinition does NOT destroy the valid theme it tried to replace', () => {
    defineTheme('keeper', { a: '#fff' }, { persist: false })
    expect(() => defineTheme('keeper', { a: 'bad}' }, { persist: false })).toThrowError(/invalid value/)
    expect(hasTheme('keeper')).toBe(true)
    expect(document.getElementById('hf-runtime-theme-keeper')).not.toBeNull()
  })

  it('a bad value never reaches persistence', () => {
    const storage = memoryStorage()
    expect(() => defineTheme('t', { a: 'bad}' }, { storage })).toThrowError(/invalid value/)
    expect(storage.get(DEFAULT_THEMES_STORAGE_KEY)).toBeNull()
  })
})

describe('hydratePersistedThemes — poisoned storage', () => {
  it('skips an entry someone wrote past the guard, hydrates the rest', () => {
    const storage = memoryStorage()
    storage.set(DEFAULT_THEMES_STORAGE_KEY, JSON.stringify({
      'evil': { a: '#fff} body{display:none' }, // written to storage directly
      'fine': { a: '#fff' },
    }))

    hydratePersistedThemes({ storage })

    expect(document.getElementById('hf-runtime-theme-evil')).toBeNull()
    expect(document.getElementById('hf-runtime-theme-fine')).not.toBeNull()
  })
```

```
})
```

app/packages/quanta/src/runtime/define-theme.ts

```
/**
 * Runtime theme injection — opinionated recipe implementation.
 *
 * Injects a <style> tag with a :where([data-theme="<name>"]) block,
 * populating --hf-color-* storage. Tailwind utilities pick up the values
 * via the @theme inline alias in tailwind.css.
 *
 * Activate via the ThemeController escape hatch (`controller.setOverride(name)`).
 * The controller pins data-theme to this name and preserves pref/brand so
 * clearing the override restores the managed theme.
 *
 * Usage:
 *   defineTheme("ai-ocean", {
 *     "surface-default":  "#0c2461",
 *     "text-primary":     "#dbeafe",
 *     "brand-primary":    "#fbbf24",
 *   });
 *   controller.setOverride("ai-ocean");
 *
 *   // later
 *   controller.setOverride(null);
 *   removeTheme("ai-ocean");
 *
 * Persistence:
 *   defineTheme writes tokens to `storage` (localStorage by default).
 *   After reload, calling `hydratePersistedThemes()` (or a ThemeController
 *   with `hydrateThemes: true`, the default) re-injects the <style> tags
 *   before the controller applies data-theme. Disable persistence with
 *   `defineTheme(name, tokens, { persist: false })` — an ephemeral theme.
 *   Switch the backend with
 *   `defineTheme(name, tokens, { storage: sessionStorageAdapter })`.
 *   In strict CSP environments, pass `{ styleNonce }` for injected <style>
 *   tags created after hydration.
 *
 * Without ThemeController you can set it manually:
 * `document.documentElement.dataset.theme = "ai-ocean"` — but then the
 * controller doesn't know about the change and subscribers won't fire.
 */

import type { ThemeStorage } from './storage.ts'
import { DEFAULT_THEMES_STORAGE_KEY } from './storage-keys.ts'
import { localStorageAdapter } from './storage.ts'

export type ThemeTokens = Record<string, string>

export interface DefineThemeOptions {
  /**
   * Write tokens to storage so the theme survives reload.
   * Default: true. Pass `false` for ephemeral / A-B-test overlays.
   */
  persist?: boolean
  /**
   * Storage backend for persisted tokens. Default: `localStorageAdapter`.
   * Use `sessionStorageAdapter` for per-tab themes, `memoryStorage()` for
   * SSR/tests, or any custom `ThemeStorage`. (URL not recommended for
   * tokens — JSON maps are too large for URLs.)
   */
  storage?: ThemeStorage
  /** CSP nonce for the runtime <style> tag injected into document.head. */
  styleNonce?: string
}

export interface HydrateOptions {
  /** Storage to read persisted themes from. Default: `localStorageAdapter`. */
  storage?: ThemeStorage
  /** CSP nonce for the runtime <style> tags injected into document.head. */
  styleNonce?: string
}
```

```
export interface RemoveThemeOptions {
  /** Storage to also remove persisted tokens from. Default: `localStorageAdapter`. */
  storage?: ThemeStorage
}

const PREFIX = 'hf' // matches the namespace in primitives.css / theme.css
const STYLE_ID_PREFIX = `${PREFIX}-runtime-theme-`
const THEMES_STORAGE_KEY = DEFAULT_THEMES_STORAGE_KEY

const styleTags = new Map<string, HTMLStyleElement>()

// ■■ injection guard ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Theme names become selector text and token keys/values become declaration
// text VERBATIM — and the docs pitch AI-generated themes, so treat all three
// as untrusted. textContent on a <style> can't break into HTML, but a `}` in
// a value (or a `"]` in a name) escapes the `:where()` block and writes
// arbitrary page CSS. Names/keys are slug-shaped; values may be any CSS
// expression that cannot terminate the declaration or the block.
const NAME_OR_KEY = /^[a-z0-9][\w-]*$/i
// eslint-disable-next-line no-control-regex -- control chars are the point
const VALUE_FORBIDDEN = /[;{}\u0000-\u001F\u007F]/

function assertInjectable(name: string, tokens: ThemeTokens): void {
  if (typeof name !== 'string' || !NAME_OR_KEY.test(name))
    throw new Error(`defineTheme: invalid theme name '${name}' — use letters, digits, '-', '_'`)
  for (const [key, val] of Object.entries(tokens)) {
    if (!NAME_OR_KEY.test(key))
      throw new Error(`defineTheme: invalid token key '${key}' in theme '${name}' — use letters, digits, '-', '_'`)
    if (typeof val !== 'string' || VALUE_FORBIDDEN.test(val))
      throw new Error(`defineTheme: invalid value for token '${key}' in theme '${name}' — a CSS value can't contain ';', '{', '}' or control characters`)
  }
}

export function defineTheme(name: string, tokens: ThemeTokens, options: DefineThemeOptions = {}): void {
  if (typeof name !== 'string' || !name) {
    throw new Error('defineTheme: name must be non-empty string')
  }
  if (!tokens || typeof tokens !== 'object') {
    throw new Error('defineTheme: tokens must be object')
  }
  // Validate BEFORE removeTheme below — a bad input must not destroy the
  // previously valid theme it was trying to replace.
  assertInjectable(name, tokens)

  const storage = options.storage ?? localStorageAdapter

  // removeTheme will also wipe the persisted entry — but only from the
  // SAME storage we're about to write to (consistent backend usage).
  removeTheme(name, { storage })
  injectStyleTag(name, tokens, { styleNonce: options.styleNonce })

  if (options.persist !== false)
    writePersistedTheme(storage, name, tokens)
}

export function removeTheme(name: string, options: RemoveThemeOptions = {}): void {
  const existing = styleTags.get(name)
  if (existing) {
    existing.remove()
    styleTags.delete(name)
  }
  removePersistedTheme(options.storage ?? localStorageAdapter, name)
}

export function listThemes(): string[] {
  return Array.from(styleTags.keys())
}

export function hasTheme(name: string): boolean {
  return styleTags.has(name)
```

```
}

/**
 * Re-inject persisted runtime themes from storage. Idempotent — skips
 * names that already have a <style> tag in the DOM. Call before applying
 * an override that targets a persisted theme; otherwise the cascade has
 * no matching block and the page flashes unstyled.
 *
 * ThemeController calls this automatically on construct (opt out via
 * `hydrateThemes: false`).
 */
export function hydratePersistedThemes(options: HydrateOptions = {}): void {
  const storage = options.storage ?? localStorageAdapter
  const persisted = readPersistedThemes(storage)
  for (const [name, tokens] of Object.entries(persisted)) {
    const existing = styleTags.get(name)
    if (!existing || (options.styleNonce && existing.nonce !== options.styleNonce)) {
      // A poisoned entry (something wrote to storage directly, or data
      // persisted before the injection guard existed) must not break boot —
      // skip it; every other theme still hydrates.
      try {
        injectStyleTag(name, tokens, { styleNonce: options.styleNonce })
      }
      catch {}
    }
  }
}

// ═══════════════════════════════════════════════════════════════════════════
// Internal — DOM
// ═══════════════════════════════════════════════════════════════════════════

interface InjectStyleTagOptions {
  styleNonce?: string
}

function injectStyleTag(name: string, tokens: ThemeTokens, options: InjectStyleTagOptions = {}): void {
  // The single chokepoint that turns tokens into CSS text — defineTheme
  // validated already, but hydration feeds persisted (storage-writable) data
  // through here too, so the guard lives at the write itself.
  assertInjectable(name, tokens)
  const id = `${STYLE_ID_PREFIX}${name}`

  // The bootstrap script (anti-FOUC) injects the same <style> in <head> before
  // React boots. If we re-inject blindly, we'd end up with two tags sharing
  // the same id — the second would be valid but removeTheme would only know
  // about one. Reuse any pre-existing element to keep the DOM canonical.
  const existing = document.getElementById(id)
  const styleNonce = options.styleNonce ?? (existing instanceof HTMLStyleElement ? existing.nonce : undefined)
  if (existing instanceof HTMLStyleElement) {
    existing.remove()
  }

  const decls = Object.entries(tokens)
    .map(([key, val]) => `  --${PREFIX}-color-${key}: ${val};`)
    .join('\n')

  const style = document.createElement('style')
  style.id = id
  if (styleNonce)
    style.nonce = styleNonce
  // Selector wrapped in `:where()` for specificity (0,0,0) — consumer overrides
  // with any class win for free. The baked `theme/color.css` is wrapped in
  // `@layer quanta-theme`, while this runtime-injected style is unlayered. Per the
  // CSS Cascade Layers spec, unlayered rules always beat layered rules
  // regardless of source order or specificity — so this runtime rule wins the
  // cascade at first paint even though Vite injects `theme/color.css` later
  // in DOM order. No specificity-wars, no FOUC.
  style.textContent = `:where([data-theme="${name}"]) {\n${decls}\n}`
  document.head.appendChild(style)
```

```
  styleTags.set(name, style)
}

// ═══════════════════════════════════════════════════════════════════════════
// Internal — storage
// ═══════════════════════════════════════════════════════════════════════════

function readPersistedThemes(storage: ThemeStorage): Record<string, ThemeTokens> {
  const raw = storage.get(THEMES_STORAGE_KEY)
  if (!raw)
    return {}
  try {
    const parsed = JSON.parse(raw) as unknown
    if (!parsed || typeof parsed !== 'object' || Array.isArray(parsed))
      return {}
    return parsed as Record<string, ThemeTokens>
  }
  catch {
    return {}
  }
}

function writePersistedTheme(storage: ThemeStorage, name: string, tokens: ThemeTokens): void {
  const all = readPersistedThemes(storage)
  all[name] = tokens
  storage.set(THEMES_STORAGE_KEY, JSON.stringify(all))
}

function removePersistedTheme(storage: ThemeStorage, name: string): void {
  const all = readPersistedThemes(storage)
  if (!(name in all))
    return
  delete all[name]
  if (Object.keys(all).length === 0)
    storage.remove(THEMES_STORAGE_KEY)
  else
    storage.set(THEMES_STORAGE_KEY, JSON.stringify(all))
}
```

app/packages/quanta/src/runtime/index.ts

```
export { bootstrapScript } from './bootstrap.ts'
export type { BootstrapOptions } from './bootstrap.ts'

export { readInitialThemeState, ThemeController } from './controller.ts'
export type {
  ReadInitialThemeStateOptions,
  ThemeControllerOptions,
  ThemePref,
  ThemeState,
  ThemeSubscriber,
} from './controller.ts'

export {
  defineTheme,
  hasTheme,
  hydratePersistedThemes,
  listThemes,
  removeTheme,
} from './define-theme.ts'
export type {
  DefineThemeOptions,
  HydrateOptions,
  RemoveThemeOptions,
  ThemeTokens,
} from './define-theme.ts'

export {
  localStorageAdapter,
  memoryStorage,
  sessionStorageAdapter,
  urlAdapter,
} from './storage.ts'
export type { ThemeStorage, UrlAdapterOptions } from './storage.ts'
```

app/packages/quanta/src/runtime/storage-keys.ts

```
export const DEFAULT_STORAGE_KEY = 'hf:quanta:theme-pref'
export const DEFAULT_OVERRIDE_STORAGE_KEY = 'hf:quanta:theme-override'
export const DEFAULT_THEMES_STORAGE_KEY = 'hf:quanta:runtime-themes'
```

app/packages/quanta/src/runtime/storage.test.ts

```
import { beforeEach, describe, expect, it } from 'vitest'
import {
  localStorageAdapter,
  memoryStorage,
  sessionStorageAdapter,
  urlAdapter,
} from './storage.ts'

describe('Side-effect contract', () => {
  describe('localStorageAdapter', () => {
    beforeEach(() => {
      localStorage.clear()
    })

    it('round-trips set → get', () => {
      localStorageAdapter.set('foo', 'bar')
      expect(localStorageAdapter.get('foo')).toBe('bar')
    })

    it('returns null for missing keys', () => {
      expect(localStorageAdapter.get('missing')).toBeNull()
    })

    it('remove() makes subsequent get() return null', () => {
      localStorageAdapter.set('foo', 'bar')
      localStorageAdapter.remove('foo')
      expect(localStorageAdapter.get('foo')).toBeNull()
    })

    it('does not throw if backing store throws on set', () => {
      const original = Storage.prototype.setItem
      Storage.prototype.setItem = () => {
        throw new Error('QuotaExceeded')
      }
      expect(() => localStorageAdapter.set('k', 'v')).not.toThrow()
      Storage.prototype.setItem = original
    })
  })

  describe('sessionStorageAdapter', () => {
    beforeEach(() => {
      sessionStorage.clear()
    })

    it('round-trips set → get → remove', () => {
      sessionStorageAdapter.set('a', '1')
      expect(sessionStorageAdapter.get('a')).toBe('1')
      sessionStorageAdapter.remove('a')
      expect(sessionStorageAdapter.get('a')).toBeNull()
    })

    it('is isolated from localStorage', () => {
      sessionStorageAdapter.set('iso', 'session')
      expect(localStorageAdapter.get('iso')).toBeNull()
    })
  })

  describe('urlAdapter (search mode)', () => {
    beforeEach(() => {
      window.history.replaceState(null, '', '/')
    })

    it('writes to ?search and reads back', () => {
      const adapter = urlAdapter()
      adapter.set('theme', 'ai-ocean')
      expect(window.location.search).toContain('theme=ai-ocean')
      expect(adapter.get('theme')).toBe('ai-ocean')
    })
```

```
  it('remove() drops the key from the URL', () => {
    const adapter = urlAdapter()
    adapter.set('theme', 'ai-ocean')
    adapter.remove('theme')
    expect(adapter.get('theme')).toBeNull()
    expect(window.location.search).not.toContain('theme=')
  })

  it('preserves other search params on set', () => {
    window.history.replaceState(null, '', '/?utm=src')
    const adapter = urlAdapter()
    adapter.set('theme', 'ai-ocean')
    expect(window.location.search).toContain('utm=src')
    expect(window.location.search).toContain('theme=ai-ocean')
  })

  it('encodes special characters', () => {
    const adapter = urlAdapter()
    adapter.set('theme', 'a b/c')
    expect(adapter.get('theme')).toBe('a b/c')
  })
})

describe('urlAdapter (hash mode)', () => {
  beforeEach(() => {
    window.history.replaceState(null, '', '/')
  })

  it('writes to #hash, not ?search', () => {
    const adapter = urlAdapter({ mode: 'hash' })
    adapter.set('theme', 'ai-ocean')
    expect(window.location.hash).toContain('theme=ai-ocean')
    expect(window.location.search).toBe('')
  })

  it('reads back from hash', () => {
    const adapter = urlAdapter({ mode: 'hash' })
    adapter.set('theme', 'ai-ocean')
    expect(adapter.get('theme')).toBe('ai-ocean')
  })

  it('ignores ?search params even when populated', () => {
    window.history.replaceState(null, '', '/?theme=from-search')
    const adapter = urlAdapter({ mode: 'hash' })
    expect(adapter.get('theme')).toBeNull()
  })
})

describe('memoryStorage', () => {
  it('round-trips within one instance', () => {
    const m = memoryStorage()
    m.set('k', 'v')
    expect(m.get('k')).toBe('v')
    m.remove('k')
    expect(m.get('k')).toBeNull()
  })

  it('is isolated between instances', () => {
    const a = memoryStorage()
    const b = memoryStorage()
    a.set('k', 'a-val')
    expect(b.get('k')).toBeNull()
  })

  it('does not touch localStorage', () => {
    const m = memoryStorage()
    m.set('mem', 'only')
    expect(localStorage.getItem('mem')).toBeNull()
  })
})
```

```
})
```

app/packages/quanta/src/runtime/storage.ts

```
/**
 * Storage adapters for runtime theme persistence.
 *
 * `defineTheme` (tokens map) and `ThemeController.setOverride` (pinned
 * theme name) can each be pointed at any storage backend implementing
 * the `ThemeStorage` interface — localStorage, sessionStorage, URL
 * search params, or a custom adapter.
 *
 * Recommended pairings:
 *   - localStorageAdapter — default; cross-tab, cross-reload (typical)
 *   - sessionStorageAdapter — per-tab themes (different theme per tab)
 *   - urlAdapter() — for the override name only; produces shareable
 *     "look-at-this-theme" links. Don't use for tokens map (too big for URL).
 *   - memoryStorage() — for SSR / tests; lives only in this process.
 *
 * The `pref` and `brand` storage (managed-mode user preferences) remain
 * on `localStorage` and are not pluggable — those are conceptually
 * device-bound user settings, not theme content.
 */

export interface ThemeStorage {
  get: (key: string) => string | null
  set: (key: string, value: string) => void
  remove: (key: string) => void
}

// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Built-in adapters
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

export const localStorageAdapter: ThemeStorage = {
  get(key) {
    try {
      return localStorage.getItem(key)
    }
    catch {
      return null
    }
  },
  set(key, value) {
    try {
      localStorage.setItem(key, value)
    }
    catch {
      // QuotaExceeded / privacy mode — silently degrade.
    }
  },
  remove(key) {
    try {
      localStorage.removeItem(key)
    }
    catch {
      // silently degrade
    }
  },
}

export const sessionStorageAdapter: ThemeStorage = {
  get(key) {
    try {
      return sessionStorage.getItem(key)
    }
    catch {
      return null
    }
  },
  set(key, value) {
    try {
```

```
      sessionStorage.setItem(key, value)
    }
    catch {
      // QuotaExceeded / privacy mode — silently degrade.
    }
  },
  remove(key) {
    try {
      sessionStorage.removeItem(key)
    }
    catch {
      // silently degrade
    }
  },
}

export interface UrlAdapterOptions {
  /**
   * Which part of the URL to use for storage. `search` → ?key=value
   * (visible in URL bar, shareable). `hash` → #key=value (client-only,
   * not sent to server). Default: 'search'.
   */
  mode?: 'search' | 'hash'
}

/**
 * URL search-param or hash storage. Best for the override NAME only
 * — token JSON maps are too large for URLs in practice. Use case:
 * shareable theme links ("send your friend a link with `?theme=ai-ocean`").
 *
 * Writes use `history.replaceState` — no navigation, no scroll jump.
 */
export function urlAdapter({ mode = 'search' }: UrlAdapterOptions = {}): ThemeStorage {
  function readParams(): URLSearchParams {
    if (mode === 'hash') {
      // Hash form like "#theme=ai-ocean&foo=bar". Strip the leading "#".
      return new URLSearchParams(window.location.hash.replace(/^#/, ''))
    }
    return new URLSearchParams(window.location.search)
  }

  function writeParams(params: URLSearchParams): void {
    const serialized = params.toString()
    const path = window.location.pathname
    if (mode === 'hash') {
      const newUrl = serialized ? `${path}${window.location.search}#${serialized}` : path + window.location.search
      window.history.replaceState(null, '', newUrl)
    }
    else {
      const newUrl = serialized ? `${path}?${serialized}${window.location.hash}` : path + window.location.hash
      window.history.replaceState(null, '', newUrl)
    }
  }

  return {
    get(key) {
      try {
        return readParams().get(key)
      }
      catch {
        return null
      }
    },
    set(key, value) {
      try {
        const params = readParams()
        params.set(key, value)
        writeParams(params)
      }
      catch {
```

```
        // SSR / no window — silently degrade.
      }
    },
    remove(key) {
      try {
        const params = readParams()
        params.delete(key)
        writeParams(params)
      }
      catch {
        // silently degrade
      }
    },
  }
}

/**
 * In-memory storage. Lives only in the current JS process — never
 * survives reload. Useful for SSR (no window) and tests.
 */
export function memoryStorage(): ThemeStorage {
  const store = new Map<string, string>()
  return {
    get: key => store.get(key) ?? null,
    set: (key, value) => {
      store.set(key, value)
    },
    remove: (key) => {
      store.delete(key)
    },
  }
}
```

app/packages/quanta/src/test/setup-components.ts

```
import { cleanup } from '@testing-library/react'
import { afterEach } from 'vitest'
import '@testing-library/jest-dom/vitest'

// @testing-library/react v16 auto-registers cleanup() after each test when it
// detects a global afterEach. Vitest does not expose afterEach as a global
// (globals: false), so we register it explicitly here.
afterEach(cleanup)

// Radix relies on a handful of DOM APIs that happy-dom does not implement.
// Polyfill them so menu open/close and focus management work under test.
const proto = Element.prototype as unknown as Record<string, unknown>
proto.hasPointerCapture ??= () => false
proto.setPointerCapture ??= () => {}
proto.releasePointerCapture ??= () => {}
proto.scrollIntoView ??= () => {}

if (typeof globalThis.ResizeObserver === 'undefined') {
  globalThis.ResizeObserver = class {
    observe() {}
    unobserve() {}
    disconnect() {}
  } as unknown as typeof ResizeObserver
}
```

app/packages/quanta/tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "esModuleInterop": true,
    "isolatedModules": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "resolveJsonModule": true,
    "allowImportingTsExtensions": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules", "scripts"]
}
```

app/packages/quanta/vitest.config.ts

```
import { defineConfig } from 'vitest/config'

export default defineConfig({
  test: {
    projects: [
      {
        test: {
          name: 'codegen',
          environment: 'node',
          include: ['scripts/**/*.test.ts'],
        },
      },
      {
        test: {
          name: 'runtime',
          environment: 'happy-dom',
          include: ['src/runtime/**/*.test.ts'],
        },
      },
      {
        test: {
          name: 'components',
          environment: 'happy-dom',
          include: ['src/components/**/*.test.tsx'],
          setupFiles: ['./src/test/setup-components.ts'],
          alias: {
            'use-sync-external-store/shim/with-selector':
              'use-sync-external-store/shim/with-selector.js',
            'use-sync-external-store/shim': 'use-sync-external-store/shim/index.js',
            'use-sync-external-store/with-selector': 'use-sync-external-store/with-selector.js',
          },
          server: {
            deps: {
              inline: [/use-sync-external-store/, /@base-ui\/react/],
            },
          },
        },
      },
    ],
  },
})
```

app/packages/quanta/wrangler.jsonc

```
{
  "$schema": "../../node_modules/wrangler/config-schema.json",
  "name": "fnf-quanta-storybook",

  // Required by wrangler even for assets-only workers.
  "compatibility_date": "2025-03-01",

  // Assets-only Worker — no `main`, just serves the static Storybook build.
  // `directory` is relative to this file: packages/quanta/storybook-static.
  "assets": {
    "directory": "./storybook-static",
    "not_found_handling": "single-page-application"
  }
}
```

app/packages/README.md

```
# Canonical Higgsfield packages

This directory is the source of truth for the local workspaces composed into
every scaffold. Make shared SDK, React binding, and Quanta fixes here; never
patch a generated template's ignored `app/packages` snapshot directly.

- `@higgsfield/fnf` — SDK core: jobs, media, profile, observability, adapters.
- `@higgsfield/fnf-react` — React provider, TanStack Query options, cache door,
  controllers.
- `@higgsfield/quanta` — Higgsfield design tokens, CSS entries, and React
  components.
- `@higgsfield/app-landing` — typed generated-content schema and shared
  Quanta-based landing blocks for simple and custom apps.

After changing canonical packages, refresh each template's ignored workspace
snapshot from the repository root:

```bash
node cli/src/index.js sync
```

Then run the relevant package tests/typechecks through an installed template
workspace. The CLI performs the same composition when it scaffolds a new app.

Before using or editing a vendored package, read its guide:

- `packages/fnf/ai/AGENTS.md`
- `packages/fnf-react/ai/AGENTS.md`
- `packages/quanta/ai/AGENTS.md`

Template-owned infrastructure lives under `app/src/module/**`. The
Supercomputer Design mode child bridge is `app/src/module/design-inspector`.
```

app/public/assets/landing/template-preview.svg

```
<svg xmlns="http://www.w3.org/2000/svg" width="1600" height="900" viewBox="0 0 1600 900" role="img" aria-labelledby="title description">
  <title id="title">Template app preview</title>
  <desc id="description">A neutral dark workspace placeholder that must be replaced during app adaptation.</desc>
  <rect width="1600" height="900" rx="56" fill="#18181b"/>
  <rect x="40" y="40" width="300" height="820" rx="32" fill="#232326"/>
  <rect x="380" y="40" width="1180" height="110" rx="28" fill="#232326"/>
  <rect x="380" y="190" width="760" height="670" rx="32" fill="#202023"/>
  <rect x="1180" y="190" width="380" height="320" rx="32" fill="#232326"/>
  <rect x="1180" y="550" width="380" height="310" rx="32" fill="#232326"/>
  <circle cx="98" cy="100" r="24" fill="#d9ff3f"/>
  <rect x="140" y="82" width="142" height="36" rx="18" fill="#3a3a3f"/>
  <rect x="72" y="190" width="236" height="58" rx="20" fill="#343439"/>
  <rect x="72" y="270" width="190" height="28" rx="14" fill="#303035"/>
  <rect x="72" y="322" width="210" height="28" rx="14" fill="#303035"/>
  <rect x="420" y="75" width="330" height="38" rx="19" fill="#3a3a3f"/>
  <circle cx="1485" cy="95" r="26" fill="#343439"/>
  <rect x="430" y="250" width="660" height="48" rx="24" fill="#3a3a3f"/>
  <rect x="430" y="332" width="660" height="420" rx="28" fill="#29292d"/>
  <path d="M620 590l115-120 92 84 88-112 125 162H620z" fill="#46464d"/>
  <circle cx="924" cy="435" r="48" fill="#d9ff3f" opacity=".75"/>
  <rect x="1225" y="250" width="210" height="34" rx="17" fill="#3a3a3f"/>
  <rect x="1225" y="320" width="286" height="22" rx="11" fill="#343439"/>
  <rect x="1225" y="370" width="246" height="22" rx="11" fill="#343439"/>
  <rect x="1225" y="610" width="210" height="34" rx="17" fill="#3a3a3f"/>
  <rect x="1225" y="680" width="286" height="22" rx="11" fill="#343439"/>
  <rect x="1225" y="730" width="246" height="22" rx="11" fill="#343439"/>
</svg>
```

app/public/farmverse-farm.svg

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 680 460" role="img" aria-labelledby="title desc"><title id="title">FarmVerse miniature farm</title><desc id="desc">Isometric crop plots, irrigation
```

app/scripts/check-adaptation.mjs

```
import { createHash } from "node:crypto";
import { existsSync, readFileSync } from "node:fs";
import { join } from "node:path";
import { pathToFileURL } from "node:url";

const root = process.cwd();
const activeUiFiles = [
  "src/layouts/custom.tsx",
  "src/components/custom-ui/custom-ui.tsx",
  "src/routes/index.tsx",
  "src/routes/app.tsx",
  "src/landing-content.ts",
];

const mockMarkers = [
  "Custom App",
  "Workspace overview",
  "Keep your workspace moving",
  "First workflow completed",
  "A concise view of current performance",
];

const failures = [];
const landingPath = join(root, "src/landing-content.ts");
if (!existsSync(landingPath)) {
  failures.push("src/landing-content.ts is required");
} else {
  const landingSource = readFileSync(landingPath, "utf8");
  for (const section of ["hero", "preview", "steps", "features", "showcase", "finalCta"]) {
    if (!new RegExp(`\\b${section}\\s*:`).test(landingSource)) {
      failures.push(`src/landing-content.ts is missing required ${section} content`);
    }
  }
  if (/TEMPLATE DEMO|template-preview/i.test(landingSource)) {
    failures.push("src/landing-content.ts still contains template landing content");
  }

  let landingContent;
  try {
    ({ landingContent } = await import(pathToFileURL(landingPath).href));
  } catch (error) {
    failures.push(
      `src/landing-content.ts could not be loaded: ${error instanceof Error ? error.message : String(error)}`,
    );
  }

  if (landingContent != null) {
    const media = [
      landingContent.preview.kind === "media" ? landingContent.preview.media : undefined,
      landingContent.steps.items[2]?.preview.media,
      ...landingContent.showcase.items.map((item) => item.media),
      landingContent.finalCta.backgroundMedia,
    ].filter(Boolean);
    const landingAssets = [];

    for (const item of media) {
      const asset = item.src;
      if (!asset.startsWith("/assets/landing/")) {
        failures.push(
          `src/landing-content.ts must use dedicated /assets/landing/ media: ${asset}`,
        );
        continue;
      }
      const assetPath = join(root, "public", asset.slice(1));
      if (!existsSync(assetPath)) {
        failures.push(`src/landing-content.ts references a missing asset: ${asset}`);
        continue;
      }
```

```
      landingAssets.push({ asset, path: assetPath });
    }

    if (landingAssets.length < 2) {
      failures.push(
        "src/landing-content.ts must include separate owned result and showcase media",
      );
    }

    const sourcePaths = new Set();
    const contentHashes = new Map();
    for (const { asset, path } of landingAssets) {
      if (sourcePaths.has(asset)) {
        failures.push(`src/landing-content.ts reuses landing media: ${asset}`);
      }
      sourcePaths.add(asset);

      const hash = createHash("sha256").update(readFileSync(path)).digest("hex");
      const existing = contentHashes.get(hash);
      if (existing != null && existing !== asset) {
        failures.push(
          `src/landing-content.ts uses duplicate files for ${existing} and ${asset}`,
        );
      } else {
        contentHashes.set(hash, asset);
      }
    }
  }
}

const landingRoutePath = join(root, "src/routes/index.tsx");
if (
  !existsSync(landingRoutePath) ||
  !readFileSync(landingRoutePath, "utf8").includes("LandingPage")
) {
  failures.push("src/routes/index.tsx must render the public LandingPage");
}

const appRoutePath = join(root, "src/routes/app.tsx");
if (!existsSync(appRoutePath) || !readFileSync(appRoutePath, "utf8").includes("previewMode")) {
  failures.push("src/routes/app.tsx must expose the full app and previewMode");
}

for (const relativePath of activeUiFiles) {
  const path = join(root, relativePath);
  if (!existsSync(path)) continue;
  const source = readFileSync(path, "utf8");
  for (const marker of mockMarkers) {
    if (source.includes(marker)) failures.push(`${relativePath} still contains mock copy: ${marker}`);
  }
  if (/\/presets\/|unsplash\.com|picsum\.photos|placehold\.co/.test(source)) {
    failures.push(`${relativePath} still references placeholder media`);
  }
}

const meta = JSON.parse(readFileSync(join(root, "src/app-meta.json"), "utf8"));
for (const field of ["og_title", "og_description", "favicon_url", "marketplace_cover_url"]) {
  if (typeof meta[field] !== "string" || meta[field].trim() === "") {
    failures.push(`src/app-meta.json must define ${field}`);
  }
}

if (failures.length > 0) {
  console.error("Custom template is not fully adapted:\n");
  for (const failure of failures) console.error(`- ${failure}`);
  process.exit(1);
}

console.log("Custom template adaptation check passed.");
```

app/scripts/check-ui.mjs

```
import { existsSync, readdirSync, readFileSync, statSync } from "node:fs";
import { extname, join, relative } from "node:path";

const root = process.cwd();
const uiRoots = [
  "src/layouts",
  "src/routes",
  "src/components/custom-ui",
  "src/features",
  "src/widgets",
];

const rules = [
  {
    label: "raw color literal",
    pattern: /#[0-9a-f]{3,8}\b|(?:rgb|hsl)a?\(/i,
  },
  {
    label: "arbitrary Tailwind color",
    pattern: /\b(?:bg|text|border|outline|ring|from|via|to)-\[[^\]]+\]/,
  },
  {
    label: "theme-specific dark variant",
    pattern: /\bdark:/,
  },
  {
    label: "unapproved component library",
    pattern: /from\s+["'](?:@mui|antd|@chakra-ui|semantic-ui|react-bootstrap|@mantine)/,
  },
  {
    label: "agent-facing instructional copy rendered in UI",
    pattern:
      /replace (?:all )?mock|before shipping|this screen is mock|composed from the same .* recipes|use metric cards/i,
  },
];

function filesUnder(path) {
  if (!existsSync(path)) return [];
  if (!statSync(path).isDirectory()) return [path];
  return readdirSync(path).flatMap((entry) => filesUnder(join(path, entry)));
}

const files = uiRoots
  .flatMap((path) => filesUnder(join(root, path)))
  .filter((path) => [".ts", ".tsx", ".js", ".jsx"].includes(extname(path)));

const failures = [];
for (const file of files) {
  const source = readFileSync(file, "utf8");
  const inspected = source.replace(/\/\*[\s\S]*?\*\//g, "").replace(/\/\/.*$/gm, "");
  for (const rule of rules) {
    const match = inspected.match(rule.pattern);
    if (!match || match.index == null) continue;
    const line = inspected.slice(0, match.index).split("\n").length;
    failures.push(`${relative(root, file)}:${line} — ${rule.label}: ${match[0]}`);
  }
  if (
    relative(root, file) !== "src/components/custom-ui/custom-ui.tsx" &&
    /<span[^>]+className=["'][^"']*\bsize-(?:9|10|12)\b[^"']*items-center[^"']*rounded-q-/s.test(
      inspected,
    )
  ) {
    failures.push(
      `${relative(root, file)} — hand-built raised icon tile; use VolumetricIconTile`,
    );
  }
}
```

```
const layoutPath = join(root, "src/layouts/custom.tsx");
if (existsSync(layoutPath)) {
  const layout = readFileSync(layoutPath, "utf8");
  for (const recipe of ["CustomAppShell", "WorkspaceContent"]) {
    if (!layout.includes(recipe)) failures.push(`src/layouts/custom.tsx — missing ${recipe}`);
  }
}

if (failures.length > 0) {
  console.error("Custom UI contract failed:\n");
  for (const failure of failures) console.error(`- ${failure}`);
  process.exit(1);
}

console.log(`Custom UI contract passed (${files.length} files checked).`);
```

app/scripts/verify-install.mjs

```
#!/usr/bin/env node
// Postinstall guard: bun occasionally PARTIALLY extracts a large package —
// thousands of files land but a few are silently missing — and the failure
// shows up far from the cause: with zod, `vite build` crashes at config load
// (ERR_MODULE_NOT_FOUND on a locale file) and tsc reports phantom errors
// inside zod/mini typings. Verify sentinel files per known-flaky package,
// self-heal by re-extracting, and fail fast with the fix.
import { existsSync, realpathSync, rmSync } from "node:fs";
import { spawnSync } from "node:child_process";
import { fileURLToPath } from "node:url";
import { join } from "node:path";

const ROOT = fileURLToPath(new URL("..", import.meta.url));

const PACKAGES = [
  {
    name: "zod",
    files: [
      "index.js",
      "index.d.ts",
      "v4/classic/index.js",
      "v4/core/index.js",
      "v4/core/index.d.ts",
      "v4/mini/index.js",
      "v4/mini/index.d.ts",
      "v4/locales/uk.js",
    ],
  },
];

const corrupt = [];
for (const { name, files } of PACKAGES) {
  const dir = join(ROOT, "node_modules", ...name.split("/"));
  // Nothing to verify if the package isn't materialized (e.g. partial installs).
  if (!existsSync(dir)) continue;
  const missing = files.filter((rel) => !existsSync(join(dir, rel)));
  if (missing.length > 0) corrupt.push({ name, dir, missing });
}

if (corrupt.length > 0) {
  // Self-heal once: drop the corrupt extractions and let bun re-materialize
  // them. HF_INSTALL_HEAL guards recursion (the inner install re-runs this
  // postinstall).
  if (!process.env.HF_INSTALL_HEAL) {
    for (const { name, dir, missing } of corrupt) {
      console.warn(
        `${name} is PARTIALLY extracted — missing ${missing.length} file(s) ` +
          `(${missing.slice(0, 5).join(", ")}). Re-extracting…`,
      );
      // dir may be a symlink into bun's isolated store (node_modules/.bun/…) —
      // remove the REAL store dir too, or bun just re-links the corrupt copy.
      const real = realpathSync(dir);
      rmSync(real, { recursive: true, force: true });
      if (real !== dir) rmSync(dir, { recursive: true, force: true });
    }
    const r = spawnSync("bun", ["install", "--frozen-lockfile"], {
      cwd: ROOT,
      stdio: "inherit",
      env: { ...process.env, HF_INSTALL_HEAL: "1" },
    });
    process.exit(r.status ?? 1);
  }
  for (const { name, missing } of corrupt) {
    console.error(
      `${name} is STILL partially extracted after a re-extract — ` +
        `missing: ${missing.slice(0, 10).join(", ")}\n` +
        `Fix manually: rm -rf node_modules/${name} && bun install`,
    );
```

```
  }
  process.exit(1);
}
```

app/src/app-meta.json

```
{
  "og_title": "FarmVerse — Explore a better farming future",
  "og_description": "An interactive virtual farm to experiment with crop choices, water plans, weather scenarios and sustainable farming trade-offs.",
  "og_image_url": null,
  "favicon_url": null,
  "og_video_url": null,
  "marketplace_cover_url": null
}
```

app/src/assets/icon-sparkles-soft.svg

```
<svg width="20" height="20" viewBox="0 0 20 20" fill="none" xmlns="http://www.w3.org/2000/svg">
<path d="M9.78702 3.88864C9.75127 3.57294 9.48435 3.33431 9.16668 3.33398C8.84893 3.33366 8.58152 3.57175 8.5451 3.88738C8.30842 5.93974 7.69479 7.38291 6.70487 8.37282C5.71494 9.36273 4.27177 9.9764
<path d="M16.4908 1.04968C16.4769 0.926909 16.3732 0.83411 16.2496 0.833985C16.126 0.833859 16.022 0.926451 16.0079 1.04919C15.9158 1.84733 15.6772 2.40857 15.2923 2.79353C14.9073 3.17851 14.346 3.417
</svg>
```

app/src/cloudflare-workers.d.ts

```
// The workerd runtime provides the `cloudflare:workers` module at runtime (the
// bundler keeps it external — see vite.config.ts). Declare it so `tsc` resolves
// the import; typed binding access is centralized in src/lib/bindings.server.ts.
declare module "cloudflare:workers" {
  export const env: unknown;
}
```

app/src/components/AGENTS.md

```
# Custom UI recipes

`@/components/custom-ui` is the required composition layer for this template.
Extend these recipes backward-compatibly instead of creating route-local visual
systems. Only `CustomAppShell` + `WorkspaceContent` are universal; page recipes
are optional when the center is a custom canvas or generations feed.

| Recipe | Purpose |
| --- | --- |
| `CustomAppShell` | Responsive desktop sidebar + compact mobile navigation |
| `SettingsRail` | Preset-style scrollable input/settings rail for either shell side |
| `WorkspaceContent` | Central window mode: scrolling `page`, fixed `canvas`, or canonical `generations` feed |
| `Page` | Width, responsive padding, and vertical page rhythm |
| `PageHeader` | Eyebrow/title/description/action hierarchy |
| `Section` | Section heading, helper copy, and optional trailing action |
| `Panel` | Canonical bordered raised surface |
| `VolumetricIconTile` | Canonical raised `sm/md/lg` icon box based on Figma Left_lg |
| `MetricCard` | Exactly two text levels: one large value + one short label; optional third fact becomes a compact top-right badge |
| `EmptyState` | Canonical zero-data state with one optional action |

## Rules for additions

- First compose Quanta primitives inside an existing recipe.
- Add a new shared recipe only after the pattern appears at least twice.
- Keep state and business logic in features/routes; recipes own presentation.
- Every interactive control needs keyboard behavior, focus visibility, labels,
  disabled state, and error/busy feedback where relevant.
- Keep every panel/card/toolbar/settings group to 3–4 primary children. Use
  progressive disclosure or split the group when more functionality exists.
- Do not create generic `Card`, `Container`, or `Box` wrappers that duplicate
  `Panel`, `Page`, or `Section`.
- Every neutral raised icon box uses `VolumetricIconTile`. Never recreate its
  single-surface border, gradient, radius, or shadow inline.
- Metric cards never stack three text rows. Keep only value + label; represent
  a brief delta/status with `badge`, or move longer context outside the card.
- Sidebar navigation always uses the shared `IconTile` with a Phosphor glyph;
  gradient tiles render `weight="fill"` and must never receive Lucide outlines.
- Every shell view wraps its center in `WorkspaceContent`. `generations` mode
  renders canonical `UserGenerations` with the shared Studio `PromptBox` dock;
  cards keep their built-in click-to-open `GenerationDetailModal`. Never
  hand-build a feed grid or import private `@/components/gallery` internals.
  Live feeds pass real `items`, `hasMore`, `loadingMore`, and a stable
  `onLoadMore`; seeded history requires explicit `demo`. Never auto-page a
  client-filtered subset of a global cursor.
- Do not copy components from the other templates unless the product genuinely
  needs that complete interaction contract.

## shadcn UI kit (`@/components/ui`)

`src/components/ui` is a complete shadcn v4 kit skinned with Quanta's
`--hf-*` tokens and pixel-matched to the Higgsfield Components Figma
(button, marketing-button, close-button, checkbox, switch, tabs/segmented
control, badge, toast, dialog, text field/area). It coexists with Quanta;
both resolve to the same tokens, so mixing them in one screen is safe.

Division of labor:

- Shell and canonical surfaces stay on the recipes above (`CustomAppShell`,
  `WorkspaceContent`, `Panel`, `MetricCard`, …) and Quanta primitives.
- Reach for `@/components/ui` for interaction primitives the recipes do not
  cover: dialog/sheet/drawer, dropdown/context/menubar, command palette,
  combobox, calendar, charts, OTP input, resizable panes, sidebar, toasts.
- Toasts: mount `<Toaster />` from `@/components/ui/sonner` once at the root,
  then `import { toast } from "sonner"`.
- Buttons: `Button` is the app button; `MarketingButton` /
  `MarketingButtonSpecial` are the glossy landing CTAs. Both take `loading`.
- `check:ui` rules still apply to app code that composes the kit: no raw
  colors, no arbitrary color classes, no `dark:`. Kit internals are exempt;
```

```
  do not restyle kit components inline — pass semantic classes or extend the
  kit itself.
- Add more components with `bunx shadcn add <name>` (`components.json` is
  configured); new ones inherit the theme through the shadcn alias variables
  in `src/styles.css`.
```

app/src/components/asset-library.tsx

```
import type { ChangeEvent, ReactElement } from "react";
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import type { MediaRef } from "@higgsfield/fnf/media";
import { AtSign as IconAtOutlined } from "lucide-react";
import { Search as IconMagnifyingGlassOutlined } from "lucide-react";
import { Plus as IconPlusMediumOutlined } from "lucide-react";
import { Avatar } from "@higgsfield/quanta/avatar";
import type { AvatarColor } from "@higgsfield/quanta/avatar";
import { Button } from "@higgsfield/quanta/button";
import { VirtualGrid } from "@higgsfield/quanta/grid";
import { Icon } from "@higgsfield/quanta/icon";
import { Input } from "@higgsfield/quanta/input";
import { Loader } from "@higgsfield/quanta/loader";
import { Media } from "@higgsfield/quanta/media";
import { Modal } from "@higgsfield/quanta/modal";
import { Tabs } from "@higgsfield/quanta/tabs";
import { Typography } from "@higgsfield/quanta/typography";

/**
 * Asset Library modal — Figma SC App Builder "Share Modal" (node 2125:15262).
 * THE app-wide asset picker: EVERY "+" / upload / attach / add-media action in
 * every app opens this modal (never a custom picker). A glass modal with a tab
 * menu header (Uploads / Image Generations / …), a segmented "All | Personal"
 * toolbar with Search, and a 5-col element grid (Upload tile + media cards).
 * Quanta components + tokens only.
 *
 * ■■ WIRING REQUIRED — do not ship this component unwired ■■■■■■■■■■■■■■■■■■■■■■
 * The modal's INTERACTIONS all work out of the box (tab/scope filtering,
 * search, upload flow with busy/error states, select-and-close), but a REAL
 * app must wire its data props:
 *
 *   1. `items`   — the library content. Load it from the app's own data:
 *                  the user's uploads (your D1/R2 records) and their
 *                  generations (the fnf feed — `jobsFeedQueryOptions` +
 *                  `selectGenerationMedia`), mapped to `AssetLibraryItem`s
 *                  with the right `kind` ("upload" | "image" | "video").
 *                  Without it the grid shows the demo placeholder set below.
 *   2. `onUpload` — the real upload path: receive the picked `File`, POST it
 *                  as multipart `FormData` to an app-local route that calls
 *                  `media.upload(...)` server-side, and resolve the
 *                  submit-ready item (`{ name, type, src, ref }` — `ref` is
 *                  the fnf MediaRef id / durable URL). Without it the Upload
 *                  tile falls back to a browser-local object URL preview
 *                  whose `ref` is undefined and which CANNOT be sent to
 *                  generation (blob: URLs never leave the browser).
 *
 *   3. `pagination` — independent cursor/loading/error state for uploads,
 *                  image generations, and video generations.
 *
 * Generation code must always submit `item.ref` — never `item.src`. Treat a
 * selection without `ref` as preview-only.
 *
 *   <AssetLibraryModal
 *     items={libraryItems}
 *     onUpload={async file => uploadToMediaRoute(file)}
 *     onSelect={item => setImage(item)}
 *     trigger={<Dropzone render={<button type="button" />} … />}
 *   />
 *
 * IMPORTANT: `trigger` is rendered AS the modal trigger (Base UI `render`
 * prop), so the element MUST spread incoming props (`onClick`, `ref`, aria)
 * onto a real DOM node — Quanta components and `@/components/*` all do. A
 * custom component that drops unknown props will silently not open the modal;
 * wrap it in a plain `<button type="button">` if needed. See
 * `@/components/AGENTS.md` for the full wiring contract.
 */

/** A picked library element, reported by `AssetLibraryModal.onSelect`. */
```

```
export interface AssetSelection {
  name: string;
  type: string;
  /** Preview/display URL. May be a browser-local `blob:` object URL when the
   * unwired upload fallback produced it — never submit `src` to generation. */
  src: string;
  /** The submit-ready reference (fnf MediaRef id / durable URL). This — and
   * ONLY this — is what generation submits use. Absent on preview-only items
   * from the unwired upload fallback. */
  ref?: MediaRef;
  /** Media presentation used by previews after selection. */
  kind?: "upload" | "image" | "video";
}

/** A library entry: a selection plus the metadata the modal filters on. */
export interface AssetLibraryItem extends AssetSelection {
  /** Shown under the "Personal" scope (the user's own items). Default: true. */
  personal?: boolean;
  /** Shown under the "Liked" tab. */
  liked?: boolean;
  badge?: string;
  badgeColor?: AvatarColor;
}

export type AssetLibraryTab = "uploads" | "image" | "video" | "liked";

export interface AssetLibraryPaginationPage {
  hasMore: boolean;
  loading: boolean;
  error?: string;
  onLoadMore: () => Promise<unknown>;
}

export type AssetLibraryPagination = Partial<Record<AssetLibraryTab, AssetLibraryPaginationPage>>;

// PLACEHOLDER ASSETS — template demo art (see /presets/*.png), rendered ONLY
// when no `items` prop is passed so the shipped layouts have something to
// show. When adapting this template into a real app, pass `items` wired to
// the app's own data (see WIRING REQUIRED above) — do not ship the demo set.
// Grep "PLACEHOLDER ASSETS" to find every site.
const THUMBS = [
  "/presets/how-product-works.png",
  "/presets/explain.png",
  "/presets/hyper-motion.png",
  "/presets/cover.png",
];

// PLACEHOLDER ASSETS — demo data; replace when adapting (see note above).
const DEMO_ITEMS: AssetLibraryItem[] = [
  {
    name: "@Ultraviolet",
    type: "Location",
    src: THUMBS[0],
    kind: "upload",
    badge: "T",
    badgeColor: "pink",
  },
  {
    name: "@Ultraviolet",
    type: "Character",
    src: THUMBS[1],
    kind: "upload",
    liked: true,
    badge: "C",
    badgeColor: "mint",
  },
  { name: "@Ultraviolet", type: "Location", src: THUMBS[2], kind: "image" },
  { name: "@Ultraviolet", type: "Location", src: THUMBS[3], kind: "image", liked: true },
  {
    name: "@Ultraviolet",
```

```
    type: "Location",
    src: THUMBS[2],
    kind: "image",
    badge: "G",
    badgeColor: "mint",
  },
  {
    name: "@Ultraviolet",
    type: "Location",
    src: THUMBS[1],
    kind: "video",
    badge: "A",
    badgeColor: "blue",
  },
  { name: "@Ultraviolet", type: "Location", src: THUMBS[0], kind: "video" },
  { name: "@Ultraviolet", type: "Location", src: THUMBS[0], kind: "image", personal: false },
  { name: "@Ultraviolet", type: "Location", src: THUMBS[0], kind: "image", personal: false },
];

const HEADER_TABS = [
  { value: "uploads", label: "Uploads" },
  { value: "image", label: "Image Generations" },
  { value: "video", label: "Video Generations" },
  { value: "liked", label: "Liked" },
];

const TAB_KIND: Record<string, AssetLibraryItem["kind"]> = {
  uploads: "upload",
  image: "image",
  video: "video",
};

/* ■■ Toolbar ■■■
```

```
    </div>
  );
}

/* ■■ Grid ■■■
```

```
  accept?: string;
  uploading?: boolean;
  onFile?: (file: File, close: () => void) => void;
}) {
  const inputRef = useRef<HTMLInputElement>(null);
  const closeRef = useRef<HTMLButtonElement>(null);
  const canUpload = accept != null && onFile != null;

  const handleChange = (event: ChangeEvent<HTMLInputElement>) => {
    const file = event.target.files?.[0];
    if (file != null && onFile != null) {
      onFile(file, () => closeRef.current?.click());
    }
    event.target.value = "";
  };

  const content = (
    <div className="flex w-full flex-col items-center gap-6 text-center">
      <span className="relative flex size-12 items-center justify-center overflow-hidden rounded-q-300 border border-q-border-subtle bg-q-transparent-light-05 shadow-q-raised">
        <span
          aria-hidden
          className="absolute inset-0 bg-gradient-to-b from-q-transparent-light-20 via-q-transparent-light-05 to-transparent mix-blend-overlay"
        />
        {uploading ? (
          <Loader size="sm" color="neutral" aria-label="Uploading" />
        ) : (
          <Icon as={IconAtOutlined} size="md" color="secondary" className="relative" />
        )}
      </span>
      <span className="flex flex-col items-center gap-1.5">
        <Typography as="span" variant="label-md-semi-bold" color="primary">
          No elements yet
        </Typography>
        <Typography
          as="span"
          variant="caption-sm-medium"
          color="secondary"
          className="max-w-[264px]"
        >
          Reuse your characters, locations, and props across every generation
        </Typography>
      </span>
    </div>
  );

  return (
    <>
      {canUpload ? (
        <button
          type="button"
          disabled={uploading}
          aria-label="Upload media"
          className="flex size-full items-center justify-center disabled:opacity-60"
          onClick={() => inputRef.current?.click()}
        >
          {content}
        </button>
      ) : (
        <div className="flex size-full items-center justify-center">{content}</div>
      )}
      {canUpload ? (
        <>
          <input
            ref={inputRef}
            type="file"
            accept={accept}
            className="hidden"
            onChange={handleChange}
          />
          <Modal.Close ref={closeRef} aria-hidden tabIndex={-1} className="hidden" />
```

```
        </>
      ) : null}
    </>
  );
}

function ElementCard({
  item,
  onSelect,
}: {
  item: AssetLibraryItem;
  onSelect?: (item: AssetSelection) => void;
}) {
  const videoSrc = item.ref?.url ?? item.src;
  const className =
    "flex h-full w-full flex-col gap-1.5 rounded-q-400 p-1 text-left transition-colors hover:bg-q-transparent-light-05";
  const children = (
    <>
      <Media ratio="auto" rounded="md" className="h-24 w-full">
        {item.kind === "video" ? (
          <Media.Video
            src={videoSrc}
            poster={videoSrc === item.src ? undefined : item.src}
            autoPlayInView
            loop
          />
        ) : (
          <Media.Image src={item.src} alt={item.name} />
        )}
        {item.badge != null ? (
          <span className="absolute bottom-1.5 left-1.5 z-10">
            <Avatar size="xxs" color={item.badgeColor} alt={item.badge} />
          </span>
        ) : null}
      </Media>
      <div className="flex flex-col gap-0.5 px-1 py-0.5">
        <Typography as="span" variant="caption-sm-semi-bold" color="primary" truncate>
          {item.name}
        </Typography>
        <Typography as="span" variant="caption-sm-regular" color="secondary" truncate>
          {item.type}
        </Typography>
      </div>
    </>
  );

  // With an `onSelect`, picking a card closes the modal and reports the choice;
  // without one it stays a passive tile (the original gallery behaviour).
  return onSelect != null ? (
    <Modal.Close
      className={className}
      onClick={() =>
        onSelect({
          name: item.name,
          type: item.type,
          src: item.kind === "video" ? videoSrc : item.src,
          ref: item.ref,
          kind: item.kind,
        })
      }
    >
      {children}
    </Modal.Close>
  ) : (
    <button type="button" className={className}>
      {children}
    </button>
  );
}
```

```
/* ■■ Modal ■■■
```

```
  [activeTab, visible],
);

const requestPage = useCallback(
  (retry = false) => {
    if (
      !open ||
      page == null ||
      page.loading ||
      loadingMoreRef.current[activeTab] === true ||
      (!retry && (!page.hasMore || page.error != null))
    ) {
      return;
    }

    loadingMoreRef.current[activeTab] = true;
    void Promise.resolve(page.onLoadMore())
      .catch(() => undefined)
      .finally(() => {
        loadingMoreRef.current[activeTab] = false;
      });
  },
  [activeTab, open, page],
);

useEffect(() => {
  if (
    !canAutoPage ||
    gridItems.length > 0 ||
    page == null ||
    page.loading ||
    !page.hasMore ||
    page.error != null
  ) {
    return;
  }
  requestPage();
}, [canAutoPage, gridItems.length, page, requestPage]);

const handleFile = async (file: File, close: () => void) => {
  setUploadError(null);
  if (onUpload == null) {
    // ponytail: unwired fallback — browser-local preview only (no `ref`);
    // wire `onUpload` for a submit-ready MediaRef (see WIRING REQUIRED).
    onSelect?.({ name: file.name, type: file.type || "Upload", src: URL.createObjectURL(file) });
    close();
    return;
  }
  setUploading(true);
  try {
    const uploaded = await onUpload(file);
    onSelect?.(uploaded);
    close();
  } catch {
    setUploadError("Upload failed — check your connection and try again.");
  } finally {
    setUploading(false);
  }
};

return (
  <Modal.Root open={open} onOpenChange={setOpen}>
    <Modal.Trigger render={trigger} />
    <Modal.Content size="xl">
      <Modal.Header flush className="px-2 py-1">
        <Tabs.Root
          variant="pill"
          value={activeTab}
          onValueChange={(value) => setTab(String(value) as AssetLibraryTab)}
          className="flex-1"
```

```
  >
    <Tabs.List items={headerTabs} />
  </Tabs.Root>
  <Modal.CloseButton />
</Modal.Header>

<div className="flex h-[595px] flex-col gap-px overflow-clip rounded-q-400">
  <AssetToolbar
    scope={scope}
    onScopeChange={setScope}
    query={query}
    onQueryChange={setQuery}
  />
  {uploadError != null ? (
    <div className="shrink-0 bg-q-transparent-light-05 px-3 py-2">
      <Typography as="p" variant="caption-sm-regular" color="danger">
        {uploadError}
      </Typography>
    </div>
  ) : null}
  <div
    aria-busy={page?.loading === true}
    className="relative min-h-0 flex-1 bg-q-transparent-light-05"
  >
    <VirtualGrid
      key={`${activeTab}:${scope}:${query}`}
      items={gridItems}
      cols={5}
      rowHeight={148}
      gap={3}
      overscan={2}
      height="100%"
      viewportClassName="p-2"
      getKey={(entry, index) =>
        entry.type === "upload"
          ? "upload"
          : `${entry.item.ref?.id ?? entry.item.src}:${index}`
      }
      renderItem={(entry) =>
        entry.type === "upload" ? (
          <UploadCard accept={accept} uploading={uploading} onFile={handleFile} />
        ) : (
          <ElementCard item={entry.item} onSelect={onSelect} />
        )
      }
      onEndReached={canAutoPage && page?.loading !== true ? requestPage : undefined}
      endReachedThresholdPx={240}
    />
    {gridItems.length === 0 && page?.loading ? (
      <div className="absolute inset-0 flex items-center justify-center py-10">
        <Loader size="sm" color="neutral" aria-label="Loading assets" />
      </div>
    ) : gridItems.length === 0 && page?.error != null ? (
      <div className="absolute inset-0 flex flex-col items-center justify-center gap-3 py-10 text-center">
        <Typography as="p" variant="caption-sm-regular" color="danger">
          {page.error}
        </Typography>
        <Button variant="tertiary" size="xs" onClick={() => requestPage(true)}>
          Retry
        </Button>
      </div>
    ) : gridItems.length === 0 ? (
      <div className="absolute inset-0 flex flex-col items-center justify-center gap-3 p-2">
        <AssetLibraryEmptyState
          accept={activeTab === "uploads" ? accept : undefined}
          uploading={uploading}
          onFile={activeTab === "uploads" ? handleFile : undefined}
        />
        {!canAutoPage && page?.hasMore ? (
          <Button variant="tertiary" size="xs" onClick={() => requestPage()}>
```

```
                  {query.trim() === "" ? "Load more" : "Search more"}
                </Button>
              ) : null}
            </div>
          ) : page?.loading ? (
            <div className="absolute inset-x-0 bottom-0 flex items-center justify-center bg-q-background-secondary py-3">
              <Loader size="sm" color="neutral" aria-label="Loading more assets" />
            </div>
          ) : page?.error != null ? (
            <div
              className="absolute inset-x-0 bottom-0 flex items-center justify-center gap-3 bg-q-background-secondary py-3"
              role="alert"
            >
              <Typography as="p" variant="caption-sm-regular" color="danger">
                {page.error}
              </Typography>
              <Button variant="tertiary" size="xs" onClick={() => requestPage(true)}>
                Retry
              </Button>
            </div>
          ) : null}
          {gridItems.length > 0 &&
          !canAutoPage &&
          page?.hasMore &&
          !page.loading &&
          page.error == null ? (
            <div className="absolute inset-x-0 bottom-0 flex items-center justify-center bg-q-background-secondary py-3">
              <Button variant="tertiary" size="xs" onClick={() => requestPage()}>
                {query.trim() === "" ? "Load more" : "Search more"}
              </Button>
            </div>
          ) : null}
        </div>
      </div>
    </Modal.Content>
  </Modal.Root>
 );
}
```

app/src/components/before-after-compare/before-after-compare.tsx

```
"use client";

import type { ComponentProps, KeyboardEvent, PointerEvent, ReactNode } from "react";
import { useCallback, useRef, useState } from "react";
import { ChevronLeft as IconChevronLeftOutlined } from "lucide-react";
import { ChevronRight as IconChevronRightOutlined } from "lucide-react";
import { Icon } from "@higgsfield/quanta/icon";
import { Media } from "@higgsfield/quanta/media";
import { Typography } from "@higgsfield/quanta/typography";
import { cn } from "@/lib/utils";

/**
 * BeforeAfterCompare — a draggable before/after image comparison slider. Quanta
 * ships no compare surface, so this composes Quanta primitives (`Media`, `Icon`,
 * `Typography` + `q-` tokens) into the split-view reveal used by the Higgsfield
 * "Skin Enhancer" app hero: the enhanced (`after`) image fills the frame, the
 * original (`before`) image is clipped to the left of a vertical divider, and a
 * white pill handle (chevron-left / chevron-right) drags the divider across.
 *
 *   <BeforeAfterCompare
 *     beforeSrc={original}
 *     afterSrc={enhanced}
 *     beforeLabel="Original"
 *     afterLabel="Enhanced"
 *   />
 *
 * The handle is a real `role="slider"` control: click / drag anywhere on the
 * frame, or focus the handle and use ←/→ (Home/End for the extremes). Dragging
 * uses pointer capture (no window listeners) and reads geometry only from event
 * handlers, so nothing touches `window` during render / SSR.
 */

export interface BeforeAfterCompareProps {
  /** The "before" image (the original) — revealed on the LEFT of the divider. */
  beforeSrc: string;
  /** The "after" image (the enhanced result) — fills the frame behind the divider. */
  afterSrc: string;
  /** Alt text for the before image. */
  beforeAlt?: string;
  /** Alt text for the after image. */
  afterAlt?: string;
  /** Corner chip over the before (left) side. Set `null` to hide. Default "Before". */
  beforeLabel?: ReactNode;
  /** Corner chip over the after (right) side. Set `null` to hide. Default "After". */
  afterLabel?: ReactNode;
  /** Aspect ratio, forwarded to `Media` (default `square`). */
  ratio?: ComponentProps<typeof Media>["ratio"];
  /** Initial divider position, 0–100 (percent from the left). Default 50. */
  defaultPosition?: number;
  className?: string;
}

const clamp = (value: number) => Math.min(100, Math.max(0, value));

/** A small frosted corner label — "Before" / "After". */
function CompareLabel({ side, children }: { side: "left" | "right"; children: ReactNode }) {
  return (
    <span
      className={cn(
        "pointer-events-none absolute top-3 z-10 rounded-q-full bg-q-transparent-dark-60 px-2.5 py-1 backdrop-blur-sm",
        side === "left" ? "left-3" : "right-3",
      )}
    >
      <Typography as="span" variant="caption-xs-medium" color="primary" className="uppercase">
        {children}
      </Typography>
    </span>
  );
```

```
}

export function BeforeAfterCompare({
  beforeSrc,
  afterSrc,
  beforeAlt = "Before",
  afterAlt = "After",
  beforeLabel = "Before",
  afterLabel = "After",
  ratio = "square",
  defaultPosition = 50,
  className,
}: BeforeAfterCompareProps) {
  const frameRef = useRef<HTMLDivElement>(null);
  const draggingRef = useRef(false);
  const [position, setPosition] = useState(() => clamp(defaultPosition));

  const updateFromClientX = useCallback((clientX: number) => {
    const frame = frameRef.current;
    if (frame == null) return;
    const rect = frame.getBoundingClientRect();
    if (rect.width === 0) return;
    setPosition(clamp(((clientX - rect.left) / rect.width) * 100));
  }, []);

  const handlePointerDown = (event: PointerEvent<HTMLDivElement>) => {
    draggingRef.current = true;
    event.currentTarget.setPointerCapture(event.pointerId);
    updateFromClientX(event.clientX);
  };

  const handlePointerMove = (event: PointerEvent<HTMLDivElement>) => {
    if (!draggingRef.current) return;
    updateFromClientX(event.clientX);
  };

  const handlePointerUp = (event: PointerEvent<HTMLDivElement>) => {
    draggingRef.current = false;
    if (event.currentTarget.hasPointerCapture(event.pointerId))
      event.currentTarget.releasePointerCapture(event.pointerId);
  };

  const handleKeyDown = (event: KeyboardEvent<HTMLButtonElement>) => {
    switch (event.key) {
      case "ArrowLeft":
        setPosition((p) => clamp(p - 2));
        event.preventDefault();
        break;
      case "ArrowRight":
        setPosition((p) => clamp(p + 2));
        event.preventDefault();
        break;
      case "Home":
        setPosition(0);
        event.preventDefault();
        break;
      case "End":
        setPosition(100);
        event.preventDefault();
        break;
    }
  };

  const rounded = Math.round(position);

  return (
    <div
      ref={frameRef}
      className={cn("relative touch-none select-none overflow-hidden rounded-q-300", className)}
      onPointerDown={handlePointerDown}
```

```
      onPointerMove={handlePointerMove}
      onPointerUp={handlePointerUp}
      onPointerCancel={handlePointerUp}
    >
      {/* Base layer — the enhanced "after" image sets the frame aspect. */}
      <Media ratio={ratio} rounded="none" className="w-full">
        <Media.Image src={afterSrc} alt={afterAlt} draggable={false} />
      </Media>

      {/* Reveal layer — the original "before" image, clipped to the left of the divider. */}
      <div
        className="absolute inset-0 overflow-hidden"
        style={{ clipPath: `inset(0 ${100 - position}% 0 0)` }}
        aria-hidden="true"
      >
        <Media.Image src={beforeSrc} alt="" draggable={false} className="size-full object-cover" />
      </div>

      {beforeLabel != null ? <CompareLabel side="left">{beforeLabel}</CompareLabel> : null}
      {afterLabel != null ? <CompareLabel side="right">{afterLabel}</CompareLabel> : null}

      {/* Divider line + draggable handle. */}
      <div className="pointer-events-none absolute inset-y-0 z-10" style={{ left: `${position}%` }}>
        <span
          className="absolute inset-y-0 left-1/2 w-0.5 -translate-x-1/2 bg-white/90 shadow-q-overlay"
          aria-hidden="true"
        />
        <button
          type="button"
          role="slider"
          aria-label="Compare before and after"
          aria-orientation="vertical"
          aria-valuemin={0}
          aria-valuemax={100}
          aria-valuenow={rounded}
          aria-valuetext={`${rounded}% enhanced`}
          onKeyDown={handleKeyDown}
          className="pointer-events-auto absolute top-1/2 left-1/2 flex h-9 -translate-x-1/2 -translate-y-1/2 cursor-ew-resize items-center rounded-q-full bg-white px-1 text-q-icon-inverse shadow-q-ov
        >
          <Icon as={IconChevronLeftOutlined} size="sm" />
          <Icon as={IconChevronRightOutlined} size="sm" />
        </button>
      </div>
    </div>
  );
}
```

app/src/components/before-after-compare/index.ts

```
export { BeforeAfterCompare } from "./before-after-compare.tsx";
export type { BeforeAfterCompareProps } from "./before-after-compare.tsx";
```

app/src/components/composer/composer.css

```
/**
 * Composer presentation — the prompt pane from the Figma builder panel (node
 * 2950:66575): white-5% surface + subtle border, radius/300, 12px padding;
 * caption label over a borderless text area; footer action row behind a
 * subtle top rule. The pane is the field: hover composites the dim overlay,
 * focus-within draws the lime inset ring (shared field language), disabled
 * fades the whole surface.
 */
@source "./composer.tsx";

@utility q-composer {
  /* Figma prompt-content minimum (104px free-text region) has no --hf-space-*
   * equivalent (scale tops out at 96px) — a documented, overridable knob. */
  --q-composer-min-height: 6.5rem;

  display: flex;
  flex-direction: column;
  gap: var(--hf-space-300);
  width: 100%;
  padding: var(--hf-space-300);
  border: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  border-radius: var(--hf-radius-300);
  background-color: var(--hf-color-overlay-dim-soft);
  transition-property: background-color, box-shadow;

  &:hover {
    background-image: linear-gradient(var(--hf-color-overlay-hover), var(--hf-color-overlay-hover));
  }

  &:focus-within {
    box-shadow: inset 0 0 0 var(--hf-border-width-medium) var(--hf-color-border-focus);
  }

  &:has(:disabled),
  &:has([data-disabled]) {
    opacity: var(--hf-opacity-disabled);
  }
}

/* Label + text area stack (Figma "Prompt Content": 8px gap, 4px inline inset). */
@utility q-composer-content {
  display: flex;
  flex: 1 1 auto;
  flex-direction: column;
  gap: var(--hf-space-200);
  padding-inline: var(--hf-space-100);
}

@utility q-composer-label {
  color: var(--hf-color-text-secondary);
  @apply text-q-caption-sm-medium;
}

/* The borderless control — the pane paints the surface, this only types. */
@utility q-composer-input {
  flex: 1 1 var(--hf-space-0);
  width: 100%;
  min-height: var(--q-composer-min-height);
  appearance: none;
  background: transparent;
  border: var(--hf-border-width-none) solid transparent;
  padding: var(--hf-space-0);
  outline: none;
  resize: none;
  color: var(--hf-color-text-primary);
  @apply text-q-body-sm-medium;

  &::placeholder {
```

```
    color: var(--hf-color-text-tertiary);
    opacity: 1;
  }
}

/* Footer action row behind a subtle rule (Figma "Line 145" + 12px gap). */
@utility q-composer-actions {
  display: flex;
  flex: none;
  align-items: center;
  gap: var(--hf-space-100);
  padding-top: var(--hf-space-300);
  border-top: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
}

/* The footer pill — re-skins a Button (ghost/xs): white-5% pill, 8/12 padding,
 * 12px label at 50% white, primary-tone 16px icons. Behavior stays Button's.
 * Later-in-bundle than button.css (components.css order), so same-specificity
 * overrides here win. */
@utility q-composer-action {
  padding-inline: var(--hf-space-200) var(--hf-space-300);
  border-radius: var(--hf-radius-full);
  background-color: var(--hf-color-overlay-dim-soft);
  color: var(--hf-color-text-on-overlay-secondary);
  @apply text-q-label-xs-medium;

  & svg {
    color: var(--hf-color-icon-primary);
  }
}
```

app/src/components/composer/composer.tsx

```
"use client";

import type { ComponentProps, ReactNode } from "react";
import { Field } from "@base-ui/react/field";
import { Button } from "@higgsfield/quanta/button";
import { cn as cx } from "@/lib/utils";

/**
 * Composer — the prompt-input surface of generation apps (Figma "Container"
 * node 2950:66575): one white-5% pane (radius/300) holding a caption label, a
 * borderless free-text area, and a footer action row (attach / link / model
 * pills) behind a subtle rule. It is the multi-line "what should we make?"
 * field every builder screen leads with.
 *
 * Like Input/Textarea it rides Base UI `Field` — the label is associated with
 * the control and validity states flow — but here the SURFACE IS THE FIELD:
 * the textarea paints no box of its own, the pane carries hover/focus/disabled.
 *
 *   <Composer
 *     label="What should the video explain?"
 *     placeholder="Type a topic, or attach files below"
 *     value={prompt}
 *     onChange={e => setPrompt(e.target.value)}
 *     actions={
 *       <>
 *         <Composer.Action start={<PlusIcon />}>Attach files</Composer.Action>
 *         <Composer.Action start={<LinkIcon />}>Link</Composer.Action>
 *       </>
 *     }
 *   />
 *
 * `Composer.Action` is the footer pill — a real quanta `Button` re-skinned by
 * `q-composer-action` (white-5% pill, 12px label), so focus/hover/disabled
 * behavior stays Button's. The text-area minimum height is the documented
 * `--q-composer-min-height` knob — override it on any ancestor to resize.
 */

type ComposerFieldProps = {
  /** Caption label above the text area. */
  label?: ReactNode;
  /** Footer slot (Composer.Action pills, counters, any node). */
  actions?: ReactNode;
  /** Swap the underlying control element (e.g. an auto-grow textarea). */
  render?: ComponentProps<typeof Field.Control>["render"];
  /** Class for the root pane. */
  className?: string;
  /** Class for the `<textarea>` element. */
  inputClassName?: string;
  /** Props forwarded to the Base UI `Field.Root`. */
  fieldProps?: ComponentProps<typeof Field.Root>;
};

export type ComposerProps = ComposerFieldProps &
  Omit<ComponentProps<"textarea">, "prefix" | "color" | "children">;

function Root({
  label,
  actions,
  render,
  rows,
  className,
  inputClassName,
  fieldProps,
  ...controlProps
}: ComposerProps) {
  return (
    <Field.Root className={cx("q-composer", className)} {...fieldProps}>
      <div className="q-composer-content">
```

```
        {label != null ? <Field.Label className="q-composer-label">{label}</Field.Label> : null}
        {/* `ref` rides in `...controlProps`; Base UI forwards it to the rendered
            <textarea> — the primary control node. Field.Control is input-typed;
            the cast bridges the element-type variance (Textarea precedent). */}
        <Field.Control
          render={render ?? <textarea rows={rows} />}
          className={cx("q-composer-input", inputClassName)}
          {...(controlProps as unknown as ComponentProps<typeof Field.Control>)}
        />
      </div>
      {actions != null ? <div className="q-composer-actions">{actions}</div> : null}
    </Field.Root>
  );
}

export type ComposerActionProps = Omit<ComponentProps<"button">, "color"> & {
  /** Leading slot (16px icon, any node) before the label. */
  start?: ReactNode;
  /** Trailing slot after the label. */
  end?: ReactNode;
};

/** Footer pill — a quanta Button re-skinned to the composer's white-5% pill. */
function Action({ className, ...props }: ComposerActionProps) {
  return (
    <Button variant="ghost" size="xs" className={cx("q-composer-action", className)} {...props} />
  );
}

export const Composer = Object.assign(Root, { Action });
```

app/src/components/composer/index.ts

```
export { Composer } from "./composer.tsx";
export type { ComposerActionProps, ComposerProps } from "./composer.tsx";
```

app/src/components/custom-ui/custom-ui.tsx

```
"use client";

import type { ComponentProps, CSSProperties, ReactNode } from "react";
import type { Icon as PhosphorIcon } from "@phosphor-icons/react";
import { Blocks as DefaultBrandIcon, PanelLeftClose, PanelRightClose } from "lucide-react";
import { Button } from "@higgsfield/quanta/button";
import { card } from "@higgsfield/quanta/card";
import { Icon, icon as iconClass } from "@higgsfield/quanta/icon";
import type { IconGlyph } from "@higgsfield/quanta/icon";
import { Sidebar } from "@higgsfield/quanta/sidebar";
import { Typography } from "@higgsfield/quanta/typography";
import { IconTile } from "@/components/icon-tile";
import type { IconTileGradient } from "@/components/icon-tile";
import { cn } from "@/lib/utils";

export interface NavigationItem {
  id: string;
  label: string;
  icon: PhosphorIcon;
  gradient?: IconTileGradient;
}

function FilledNavigationIcon({ glyph: Glyph }: { glyph: PhosphorIcon }) {
  return <Glyph weight="fill" aria-hidden className={iconClass({ size: "sm" })} />;
}

export interface CustomAppShellProps {
  brand: string;
  navigation: NavigationItem[];
  activeId: string;
  onNavigate: (id: string) => void;
  children: ReactNode;
  footer?: ReactNode;
  /** Navigation sidebar placement. Use `none` for two settings/tool rails. */
  navigationSide?: "left" | "right" | "none";
  /** Optional non-navigation rail, typically `SettingsRail`. */
  leftRail?: ReactNode;
  /** Optional non-navigation rail, typically `SettingsRail`. */
  rightRail?: ReactNode;
}

const TOP_GLOW =
  "radial-gradient(60% 80% at 50% 0%, var(--hf-color-transparent-light-10) 0%, var(--hf-color-transparent-light-05) 42%, transparent 72%)";
const TOP_DOTS =
  "radial-gradient(var(--hf-color-transparent-light-20) 1px, transparent 1px)";
const TOP_DOTS_MASK =
  "radial-gradient(55% 70% at 50% 0%, var(--hf-color-transparent-dark-100) 0%, var(--hf-color-transparent-dark-40) 45%, transparent 75%)";

function NavigationSidebar({
  brand,
  navigation,
  activeId,
  onNavigate,
  footer,
  side,
}: Omit<CustomAppShellProps, "children" | "leftRail" | "rightRail" | "navigationSide"> & {
  side: "left" | "right";
}) {
  return (
    <Sidebar.Root
      className="m-2.5 hidden lg:flex"
      style={
        { height: "calc(100% - 20px)", ["--q-sidebar-radius" as string]: "12px" } as CSSProperties
      }
    >
      <Sidebar.Header>
        <Sidebar.Switcher>
          <Sidebar.Logo>
```

```
            <span className="flex size-6 items-center justify-center rounded-q-200 bg-q-brand-primary text-q-icon-inverse">
              <Icon as={DefaultBrandIcon} size="sm" />
            </span>
          </Sidebar.Logo>
          <Sidebar.Title>{brand}</Sidebar.Title>
        </Sidebar.Switcher>
        <Sidebar.Toggle>
          <Icon as={side === "left" ? PanelLeftClose : PanelRightClose} size="md" />
        </Sidebar.Toggle>
      </Sidebar.Header>

      <Sidebar.Body>
        <Sidebar.Section>
          <Sidebar.SectionItems>
            {navigation.map((item) => (
              <Sidebar.Item
                key={item.id}
                selected={activeId === item.id}
                onClick={() => onNavigate(item.id)}
                start={<IconTile as={item.icon} gradient={item.gradient ?? "blue"} />}
                title={item.label}
              />
            ))}
          </Sidebar.SectionItems>
        </Sidebar.Section>
      </Sidebar.Body>

      {footer != null ? (
        <Sidebar.Footer>
          <div className="px-2 py-1">{footer}</div>
        </Sidebar.Footer>
      ) : null}
    </Sidebar.Root>
  );
}

/**
 * The mandatory custom-template shell. It owns responsive navigation, page
 * background, content overflow, spacing, and the maximum readable width.
 */
export function CustomAppShell({
  brand,
  navigation,
  activeId,
  onNavigate,
  children,
  footer,
  navigationSide = "left",
  leftRail,
  rightRail,
}: CustomAppShellProps) {
  const navigationSidebar =
    navigationSide === "none" ? null : (
      <NavigationSidebar
        brand={brand}
        navigation={navigation}
        activeId={activeId}
        onNavigate={onNavigate}
        footer={footer}
        side={navigationSide}
      />
    );
  const left = navigationSide === "left" ? navigationSidebar : leftRail;
  const right = navigationSide === "right" ? navigationSidebar : rightRail;

  return (
    <div className="flex h-dvh min-h-0 bg-q-background-primary text-q-text-primary">
      {left}

      <div className="flex min-w-0 flex-1 flex-col">
```

```
        <header className="flex shrink-0 items-center gap-2 overflow-x-auto border-b border-q-border-subtle px-3 py-2 lg:hidden">
          <Typography as="span" variant="label-sm-semi-bold" color="primary" className="mr-2">
            {brand}
          </Typography>
          {navigation.map((item) => (
            <Button
              key={item.id}
              variant={activeId === item.id ? "secondary" : "ghost"}
              size="xs"
              aria-label={item.label}
              start={<FilledNavigationIcon glyph={item.icon} />}
              onClick={() => onNavigate(item.id)}
            >
              <span className="hidden sm:inline">{item.label}</span>
            </Button>
          ))}
        </header>
        <main className="relative min-h-0 flex-1 overflow-hidden">
          <div
            aria-hidden
            className="pointer-events-none absolute inset-x-0 top-0 h-[600px]"
            style={{
              backgroundImage: TOP_DOTS,
              backgroundSize: "14px 14px",
              maskImage: TOP_DOTS_MASK,
              WebkitMaskImage: TOP_DOTS_MASK,
            }}
          />
          <div
            aria-hidden
            className="pointer-events-none absolute inset-x-0 top-0 h-[600px]"
            style={{ backgroundImage: TOP_GLOW }}
          />
          <div className="relative h-full min-h-0">{children}</div>
        </main>
      </div>
      {right}
    </div>
  );
}

export type WorkspaceContentMode = "page" | "canvas" | "generations";

export interface WorkspaceContentProps {
  mode?: WorkspaceContentMode;
  children: ReactNode;
  className?: string;
}

/**
 * Central shell window: scrolling page, fixed custom canvas, or the canonical
 * generations feed. Use one mode per view; side rails remain shell siblings.
 */
export function WorkspaceContent({
  mode = "page",
  children,
  className,
}: WorkspaceContentProps) {
  return (
    <div
      className={cn(
        "relative h-full min-h-0",
        mode === "page" && "overflow-y-auto",
        mode === "canvas" && "overflow-hidden",
        mode === "generations" && "flex flex-col overflow-hidden p-4",
        className,
      )}
    >
      {children}
    </div>
```

```
  );
}

export interface SettingsRailProps {
  title: ReactNode;
  description?: ReactNode;
  children: ReactNode;
  footer?: ReactNode;
  className?: string;
}

/**
 * Preset-style settings/input rail. Pass it to `leftRail` or `rightRail`; the
 * shell decides placement while the rail owns width, scrolling, and footer.
 */
export function SettingsRail({
  title,
  description,
  children,
  footer,
  className,
}: SettingsRailProps) {
  return (
    <aside
      className={card(
        { surface: "solid", elevation: "raised" },
        "m-2.5 hidden h-[calc(100%_-_20px)] w-85.5 shrink-0 flex-col gap-4 overflow-y-auto border-q-thin border-q-border-subtle p-3 lg:flex [&>*]:shrink-0",
        className,
      )}
    >
      <div className="flex flex-col gap-1 px-2 py-1">
        <Typography as="h2" variant="title-sm-semi-bold" color="primary">
          {title}
        </Typography>
        {description != null ? (
          <Typography as="p" variant="caption-sm-regular" color="secondary">
            {description}
          </Typography>
        ) : null}
      </div>
      <div className="flex flex-col gap-3">{children}</div>
      {footer != null ? (
        <div className="sticky bottom-0 mt-auto bg-q-background-secondary pt-3">{footer}</div>
      ) : null}
    </aside>
  );
}

export function Page({ className, ...props }: ComponentProps<"div">) {
  return (
    <div
      className={cn(
        "mx-auto flex w-full max-w-7xl flex-col gap-10 p-5 md:gap-12 md:p-8 lg:p-10",
        className,
      )}
      {...props}
    />
  );
}

export interface PageHeaderProps {
  eyebrow?: ReactNode;
  title: ReactNode;
  description?: ReactNode;
  actions?: ReactNode;
}

export function PageHeader({ eyebrow, title, description, actions }: PageHeaderProps) {
  return (
    <header className="flex flex-col items-center gap-6 py-8 text-center md:py-12">
```

```
      <div className="flex max-w-3xl flex-col items-center gap-3">
        {eyebrow != null ? (
          <Typography as="span" variant="label-xs-semi-bold" color="brand">
            {eyebrow}
          </Typography>
        ) : null}
        <Typography as="h1" variant="title-lg-semi-bold" color="primary">
          {title}
        </Typography>
        {description != null ? (
          <Typography as="p" variant="body-md-regular" color="secondary">
            {description}
          </Typography>
        ) : null}
      </div>
      {actions != null ? (
        <div className="flex flex-wrap items-center justify-center gap-2">{actions}</div>
      ) : null}
    </header>
  );
}

export interface SectionProps {
  title?: ReactNode;
  description?: ReactNode;
  actions?: ReactNode;
  children: ReactNode;
  className?: string;
}

export function Section({ title, description, actions, children, className }: SectionProps) {
  return (
    <section className={cn("flex flex-col gap-5", className)}>
      {title != null || actions != null ? (
        <div className="flex items-end justify-between gap-4">
          <div className="flex flex-col gap-1">
            {title != null ? (
              <Typography as="h2" variant="title-sm-semi-bold" color="primary">
                {title}
              </Typography>
            ) : null}
            {description != null ? (
              <Typography as="p" variant="body-sm-regular" color="secondary">
                {description}
              </Typography>
            ) : null}
          </div>
          {actions}
        </div>
      ) : null}
      {children}
    </section>
  );
}

export function Panel({ className, ...props }: ComponentProps<"div">) {
  return (
    <div
      className={cn(
        "rounded-q-500 border border-q-border-subtle bg-q-transparent-light-05 p-5 shadow-q-raised-sm md:p-6",
        className,
      )}
      {...props}
    />
  );
}

export type VolumetricIconTileSize = "sm" | "md" | "lg";

const VOLUMETRIC_ICON_TILE_SIZE = {
```

```
  sm: { root: "size-9 rounded-q-300", icon: "sm" },
  md: { root: "size-10 rounded-q-300", icon: "sm" },
  lg: { root: "size-12 rounded-q-300", icon: "md" },
} as const;

export interface VolumetricIconTileProps {
  icon: IconGlyph;
  size?: VolumetricIconTileSize;
  className?: string;
}

/** Canonical raised icon surface — Figma Left_lg (3617:50814). */
export function VolumetricIconTile({
  icon,
  size = "md",
  className,
}: VolumetricIconTileProps) {
  const dimensions = VOLUMETRIC_ICON_TILE_SIZE[size];
  return (
    <span
      className={cn(
        "relative flex shrink-0 items-center justify-center overflow-hidden border border-q-border-subtle bg-gradient-to-b from-q-transparent-light-20 via-q-transparent-dark-10 to-q-transparent-light-
        dimensions.root,
        className,
      )}
    >
      <Icon as={icon} size={dimensions.icon} />
    </span>
  );
}

export interface MetricCardProps {
  label: ReactNode;
  value: ReactNode;
  badge?: ReactNode;
  icon: IconGlyph;
}

export function MetricCard({ label, value, badge, icon }: MetricCardProps) {
  return (
    <Panel className="relative flex min-w-0 items-center gap-4">
      <VolumetricIconTile icon={icon} size="md" />
      <div className={cn("flex min-w-0 flex-1 flex-col gap-1", badge != null && "pr-12")}>
        <Typography as="strong" variant="title-md-semi-bold" color="primary" truncate>
          {value}
        </Typography>
        <Typography as="span" variant="caption-sm-medium" color="secondary">
          {label}
        </Typography>
      </div>
      {badge != null ? (
        <Typography
          as="span"
          variant="caption-xs-medium"
          color="secondary"
          className="absolute right-4 top-4 rounded-q-200 bg-q-transparent-light-10 px-2 py-1"
        >
          {badge}
        </Typography>
      ) : null}
    </Panel>
  );
}

export interface EmptyStateProps {
  icon: IconGlyph;
  title: ReactNode;
  description: ReactNode;
  action?: ReactNode;
}
```

```
export function EmptyState({ icon, title, description, action }: EmptyStateProps) {
  return (
    <div className="flex min-h-64 flex-col items-center justify-center gap-4 text-center">
      <VolumetricIconTile icon={icon} size="lg" />
      <div className="flex max-w-sm flex-col gap-1.5">
        <Typography as="h3" variant="label-md-semi-bold" color="primary">
          {title}
        </Typography>
        <Typography as="p" variant="body-sm-regular" color="secondary">
          {description}
        </Typography>
      </div>
      {action}
    </div>
  );
}
```

app/src/components/custom-ui/index.ts

```
export {
  CustomAppShell,
  EmptyState,
  MetricCard,
  Page,
  PageHeader,
  Panel,
  Section,
  SettingsRail,
  VolumetricIconTile,
  WorkspaceContent,
} from "./custom-ui";
export type {
  CustomAppShellProps,
  EmptyStateProps,
  MetricCardProps,
  NavigationItem,
  PageHeaderProps,
  SectionProps,
  SettingsRailProps,
  VolumetricIconTileProps,
  VolumetricIconTileSize,
  WorkspaceContentMode,
  WorkspaceContentProps,
} from "./custom-ui";
```

app/src/components/dropzone/dropzone-preview.tsx

```
import type { ReactNode } from "react";
import { Icon, type IconGlyph } from "@higgsfield/quanta/icon";
import { Media } from "@higgsfield/quanta/media";
import { Typography } from "@higgsfield/quanta/typography";
import { cn } from "@/lib/utils";

/**
 * DropzonePreview — the small preview card shown inside a `Dropzone` once a
 * selection is made (Figma App-detail after-selection state, node 3309:83654).
 * Two flavours, chosen by whether `label` is set:
 *
 *   • plain (no `label`) — the uploaded image: a ~100px white-ringed thumbnail
 *     with a raised shadow (the "Upload Image" tile once a photo is picked).
 *   • captioned (`label` set) — the chosen option: a ~88px white-ringed, slightly
 *     tilted card with a blurred dark scrim carrying a glyph + name (the
 *     "Select Animal" tile once an animal is picked).
 *
 * Quanta `Media` / `Icon` / `Typography` + `q-` tokens only.
 */

export interface DropzonePreviewProps {
  /** Preview image source. */
  src: string;
  /** Alt text (defaults to the string `label`, else empty). */
  alt?: string;
  /** Overlay caption — set it for the tilted, scrimmed "selected option" card. */
  label?: ReactNode;
  /** Small glyph shown above the caption in the overlay. */
  icon?: IconGlyph;
  className?: string;
}

export function DropzonePreview({ src, alt, label, icon, className }: DropzonePreviewProps) {
  const captioned = label != null;
  const altText = alt ?? (typeof label === "string" ? label : "");

  return (
    <div
      className={cn(
        "overflow-hidden border-2 border-white",
        captioned ? "size-22 -rotate-4 rounded-q-400" : "size-25 rounded-q-500 shadow-q-raised",
        className,
      )}
    >
      <Media ratio="square" rounded="none" className="size-full">
        <Media.Image src={src} alt={altText} />
        {captioned ? (
          <Media.Overlay
            placement="center"
            className="flex-col gap-1.5 bg-q-transparent-dark-40 px-3 py-2 backdrop-blur-md"
          >
            {icon != null ? <Icon as={icon} size="sm" color="primary" /> : null}
            <Typography
              as="span"
              variant="caption-xs-medium"
              color="primary"
              className="text-center"
            >
              {label}
            </Typography>
          </Media.Overlay>
        ) : null}
      </Media>
    </div>
  );
}
```

app/src/components/dropzone/dropzone.tsx

```
"use client";

import type { ComponentPropsWithRef, ReactElement, ReactNode, Ref } from "react";
import { useRender } from "@base-ui/react/use-render";
import { Icon, type IconGlyph } from "@higgsfield/quanta/icon";
import { Typography } from "@higgsfield/quanta/typography";
import { cn } from "@/lib/utils";

/**
 * Dropzone — the canonical labelled file / picker target (Figma SC App Builder
 * image-field, node 3313:51149). Quanta ships no upload surface, so
 * this composes Quanta primitives (`Icon`, `Typography` + `q-` tokens) into the
 * bordered, centered "icon → title → subtitle" tile the app pages use for
 * "Upload Image" / "Select Animal" style inputs.
 *
 *   <Dropzone icon={IconUpload} title="Upload Image"
 *     subtitle="PNG, JPG or Paste from Clipboard" />
 *
 * `border` picks the outline: `dashed` (the primary upload target, default) or
 * `solid` (a secondary picker). The host element swaps via `render` — keep the
 * default `<div>` for a passive tile or render a `<button>` / `<label>` to make
 * the whole area interactive (it gets hover + focus affordances from the classes).
 *
 * Once something is chosen, pass a `preview` node (typically a `DropzonePreview`)
 * to show the AFTER-SELECTION state (Figma node 3309:83654): the same bordered
 * tile now centered on a small preview card. The tile stays interactive, so
 * clicking it re-opens the picker to change the selection.
 */

export type DropzoneBorder = "dashed" | "solid";

const BORDER_CLASS = {
  dashed: "border-dashed",
  solid: "border-solid",
} satisfies Record<DropzoneBorder, string>;

export type DropzoneProps = Omit<ComponentPropsWithRef<"div">, "title"> & {
  /** Leading glyph shown above the text (a lucide / quanta icon). */
  icon?: IconGlyph;
  /** Bold primary line. */
  title?: ReactNode;
  /** Muted helper line under the title. */
  subtitle?: ReactNode;
  /** Outline style — `dashed` upload target (default) or `solid` picker. */
  border?: DropzoneBorder;
  /**
   * After-selection content. When set, the icon / title / subtitle empty state
   * is replaced by this node (a `DropzonePreview` of the chosen image / option),
   * centered in the same bordered tile.
   */
  preview?: ReactNode;
  /** Swap the host element (e.g. an interactive `<button>` / `<label>`). */
  render?: ReactElement;
};

export function Dropzone({
  icon,
  title,
  subtitle,
  border = "dashed",
  preview,
  className,
  render,
  ref,
  ...props
}: DropzoneProps) {
  const interactive = render != null;
```

```
  return useRender({
    render,
    defaultTagName: "div",
    ref: ref as Ref<Element> | undefined,
    props: {
      className: cn(
        "relative isolate flex min-h-40 flex-1 flex-col items-center justify-center gap-3 overflow-hidden rounded-q-400 border-[1.5px] border-q-border-default px-4 pb-5 pt-6 text-center shadow-[0_2px_
        BORDER_CLASS[border],
        interactive &&
          "cursor-pointer hover:border-q-border-strong hover:brightness-110 focus-visible:outline-2 focus-visible:outline-q-border-focus",
        className,
      ),
      children: (
        <>
          <span
            aria-hidden
            className="pointer-events-none absolute inset-0 -z-[1] rounded-q-400 bg-q-transparent-light-10 backdrop-blur-xl"
          />
          {preview != null ? (
            <div className="relative flex items-center justify-center">{preview}</div>
          ) : (
            <>
              {icon != null ? (
                <span className="relative flex items-center justify-center rounded-q-full border border-[rgba(197,197,197,0.3)] p-2.5 text-q-icon-primary shadow-[0_20.5px_10.3px_rgba(0,0,0,0.09),0_5px
                  <span
                    aria-hidden
                    className="pointer-events-none absolute inset-0 rounded-q-full bg-q-transparent-light-05"
                  />
                  <Icon as={icon} size="md" className="relative" />
                </span>
              ) : null}
              <div className="relative flex w-full flex-col items-center gap-1">
                {title != null ? (
                  <Typography as="span" variant="body-md-medium" color="primary">
                    {title}
                  </Typography>
                ) : null}
                {subtitle != null ? (
                  <Typography as="span" variant="body-sm-medium" color="secondary">
                    {subtitle}
                  </Typography>
                ) : null}
              </div>
            </>
          )}
          <span
            aria-hidden
            className="pointer-events-none absolute inset-0 rounded-[inherit] shadow-[inset_0_2px_3px_rgba(255,255,255,0.05)]"
          />
        </>
      ),
      ...props,
    },
  });
}
```

app/src/components/dropzone/index.ts

```
export { Dropzone } from "./dropzone.tsx";
export type { DropzoneBorder, DropzoneProps } from "./dropzone.tsx";
export { DropzonePreview } from "./dropzone-preview.tsx";
export type { DropzonePreviewProps } from "./dropzone-preview.tsx";
```

app/src/components/gallery/demo-data.ts

```
import type { GalleryItem, MediaKind } from "./types.ts";

/**
 * Demo data for the History gallery.
 *
 * Everything is generated deterministically (seeded PRNG) so the justified
 * layout is stable across renders — which keeps scroll-anchoring exact and makes
 * the virtualization behaviour reproducible. The pool mixes real still assets
 * (`/presets/*.png`) with the ffmpeg-generated hover-to-play clips
 * (`/gallery/*.mp4`) and spans a wide spread of aspect ratios: tall portraits,
 * wide landscapes, squares and true panoramas.
 */

// A tiny deterministic PRNG (mulberry32) — stable ids and ratios per index.
function mulberry32(seed: number) {
  let a = seed >>> 0;
  return () => {
    a |= 0;
    a = (a + 0x6d2b79f5) | 0;
    let t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

interface StillBase {
  src: string;
  alt: string;
  prompt: string;
}

// PLACEHOLDER ASSETS — template demo art (see /presets/*.png). When adapting
// this template into a real app, REPLACE media that represents the product
// (hero/example outputs, covers, before/after samples, feed items) with
// bespoke on-brand assets generated via the Higgsfield generation tools.
// Pure style-picker label thumbnails may keep simple placeholder art when
// real output depends on the user's own upload. Grep "PLACEHOLDER ASSETS"
// to find every site.
const STILLS: StillBase[] = [
  {
    src: "/presets/cover.png",
    alt: "Renaissance portrait holding a jar of dill pickles",
    prompt:
      "A renaissance oil-painting portrait of a woman cradling a glowing jar of Picklehaus dill pickles, candlelit chiaroscuro, rich golden fabrics.",
  },
  {
    src: "/presets/how-product-works.png",
    alt: "Cinematic product hero shot",
    prompt:
      "Cinematic product hero shot explaining how the pickling process works, warm studio light, shallow depth of field.",
  },
  {
    src: "/presets/explain.png",
    alt: "Candlelit concept explainer scene",
    prompt:
      "Moody candlelit scene explaining a concept, editorial photography, deep shadows and warm highlights.",
  },
  {
    src: "/presets/hyper-motion.png",
    alt: "Caramel popcorn splash in motion",
    prompt:
      "Hyper-motion macro of caramel popcorn bursting mid-air with sugar crystals, high-speed capture, glossy amber tones.",
  },
];

interface VideoBase {
  videoSrc: string;
  poster: string;
```

```
  alt: string;
  prompt: string;
  width: number;
  height: number;
}

// Real clips (ffmpeg Ken-Burns renders) with their true dimensions.
// PLACEHOLDER ASSETS — demo data; replace when adapting (see note above).
const VIDEOS: VideoBase[] = [
  {
    videoSrc: "/gallery/motion-landscape.mp4",
    poster: "/presets/hyper-motion.png",
    alt: "Popcorn burst, animated",
    prompt:
      "Slow push-in on caramel popcorn bursting mid-air, high-speed macro loop, glossy amber tones.",
    width: 854,
    height: 480,
  },
  {
    videoSrc: "/gallery/product-portrait.mp4",
    poster: "/presets/how-product-works.png",
    alt: "Product hero, animated",
    prompt: "Vertical hero reveal of the product with a gentle parallax zoom, warm studio light.",
    width: 480,
    height: 854,
  },
  {
    videoSrc: "/gallery/explain-square.mp4",
    poster: "/presets/explain.png",
    alt: "Candlelit concept, animated",
    prompt: "Square looping candlelit scene with drifting highlights, editorial mood.",
    width: 600,
    height: 600,
  },
  {
    videoSrc: "/gallery/cover-tall.mp4",
    poster: "/presets/cover.png",
    alt: "Renaissance portrait, animated",
    prompt: "Tall portrait with a slow breathing zoom, candlelit chiaroscuro, golden fabrics.",
    width: 512,
    height: 768,
  },
];

// A generous spread of aspect ratios (width : height) for the still items.
const ASPECTS: [number, number][] = [
  [9, 16], // tall portrait
  [2, 3], // portrait
  [3, 4], // portrait
  [1, 1], // square
  [4, 3], // landscape
  [3, 2], // landscape
  [16, 9], // wide
  [2, 1], // panoramic
  [21, 9], // ultra-wide panorama
  [5, 4], // near-square
];

/**
 * A dated batch definition: the label plus how many items land in it. Later
 * batches are appended lazily by `makeBatch` to simulate infinite history.
 */
// PLACEHOLDER ASSETS — demo data; replace when adapting (see note above).
export const GROUPS = [
  { id: "today", label: "Today", count: 34 },
  { id: "yesterday", label: "Yesterday", count: 42 },
  { id: "this-week", label: "Earlier this week", count: 56 },
  { id: "last-week", label: "Last week", count: 60 },
] as const;
```

```
/** Long side (px) used to synthesize still dimensions from an aspect ratio. */
const LONG_SIDE = 1280;

function makeStill(
  index: number,
  groupId: string,
  groupLabel: string,
  rand: () => number,
): GalleryItem {
  const base = STILLS[index % STILLS.length]!;
  const [aw, ah] = ASPECTS[Math.floor(rand() * ASPECTS.length)]!;
  const landscape = aw >= ah;
  const width = landscape ? LONG_SIDE : Math.round((LONG_SIDE * aw) / ah);
  const height = landscape ? Math.round((LONG_SIDE * ah) / aw) : LONG_SIDE;
  return {
    id: `${groupId}-img-${index}`,
    kind: "image",
    status: "ready",
    src: base.src,
    width,
    height,
    prompt: base.prompt,
    alt: base.alt,
    groupId,
    groupLabel,
  };
}

function makeVideo(index: number, groupId: string, groupLabel: string): GalleryItem {
  const base = VIDEOS[index % VIDEOS.length]!;
  return {
    id: `${groupId}-vid-${index}`,
    kind: "video",
    status: "ready",
    src: base.poster,
    videoSrc: base.videoSrc,
    width: base.width,
    height: base.height,
    prompt: base.prompt,
    alt: base.alt,
    groupId,
    groupLabel,
  };
}

/**
 * Build one dated batch of items. Roughly every 5th item is a video so the
 * feed reliably shows hover-to-play tiles. When `withGenerating` is set the
 * batch leads with a single `generating` placeholder tile (the pulsing card).
 */
export function makeBatch(
  groupId: string,
  groupLabel: string,
  count: number,
  seed: number,
  withGenerating = false,
): GalleryItem[] {
  const rand = mulberry32(seed);
  const items: GalleryItem[] = [];

  if (withGenerating) {
    items.push({
      id: `${groupId}-generating`,
      kind: "image",
      status: "generating",
      src: "",
      width: 3,
      height: 4,
      prompt: "",
      alt: "",
```

```
      groupId,
      groupLabel,
    });
  }

  for (let i = 0; i < count; i++) {
    const kind: MediaKind = i % 5 === 2 ? "video" : "image";
    items.push(
      kind === "video"
        ? makeVideo(i, groupId, groupLabel)
        : makeStill(i, groupId, groupLabel, rand),
    );
  }
  return items;
}

/** The initial dataset — the four seeded batches, a generating tile up top. */
export function makeInitialItems(): GalleryItem[] {
  return GROUPS.flatMap((group, gi) =>
    makeBatch(group.id, group.label, group.count, 1000 + gi * 97, gi === 0),
  );
}

/**
 * Lazily-appended "older" batch, used by the infinite-scroll loader. Each call
 * yields a fresh dated group ("Earlier · N") so appending never reflows the
 * batches already on screen above.
 */
export function makeOlderBatch(page: number): GalleryItem[] {
  const id = `earlier-${page}`;
  const label = page === 0 ? "Earlier" : `Earlier · ${page + 1}`;
  const count = 48;
  return makeBatch(id, label, count, 5000 + page * 131);
}
```

app/src/components/gallery/density-control.tsx

```
import { Slider } from "@higgsfield/quanta/slider";
import { DENSITY_ROW_HEIGHTS } from "./use-justified-gallery.ts";

/**
 * Tile-size control. A Quanta stepped `Slider` maps its notches to the
 * gallery's target row heights: dragging right makes tiles LARGER (fewer per
 * row), dragging left makes them smaller / denser. Changing it triggers a
 * layout recompute in the engine while scroll anchoring keeps the view stable.
 */
export interface DensityControlProps {
  value: number;
  onChange: (level: number) => void;
}

export function DensityControl({ value, onChange }: DensityControlProps) {
  const steps = DENSITY_ROW_HEIGHTS.length;
  return (
    <div className="flex items-center gap-2">
      <Slider
        aria-label="Tile density"
        steps={steps}
        value={value}
        onChange={onChange}
        className="w-32"
      />
    </div>
  );
}
```

app/src/components/gallery/gallery-tile.tsx

```
import { memo, useEffect, useRef, useState } from "react";
import type { CSSProperties } from "react";
import { Heart as IconFavorite } from "lucide-react";
import { Download as IconDownload } from "lucide-react";
import { Copy as IconContentCopy } from "lucide-react";
import { Share2 as IconShare } from "lucide-react";
import { Trash2 as IconDelete } from "lucide-react";
import { GenerationTile } from "@/components/generation-card";
import type { CardAction } from "@/components/generation-card";
import type { TileRect } from "./justified-engine.ts";
import type { GalleryItem, LoadTier } from "./types.ts";

/**
 * One gallery tile. It is absolutely positioned at the engine-computed rect and
 * composed from Quanta primitives:
 *   • `generating` items → `GenerationCard state="generating"` (the pulsing card).
 *   • ready items → a `GenerationCard` (image or hover-to-play video) that is the
 *     trigger of a `GenerationDetailModal`, preserving the click-to-open behavior.
 *
 * Load tiers are driven by the engine. Media remains mounted while its row is
 * windowed so scrolling never swaps completed generations for placeholders.
 */

export interface GalleryTileProps {
  item: GalleryItem;
  rect: TileRect;
  /** Row top, in content coordinates (the rect's x is row-relative). */
  top: number;
  tier: LoadTier;
  reducedMotion: boolean;
}

function rectStyle(rect: TileRect, top: number): CSSProperties {
  return {
    left: rect.x,
    top,
    width: rect.width,
    height: rect.height,
  } as CSSProperties;
}

/** Neutral fallback behind media that has not painted yet. */
function Placeholder() {
  return <span className="qg-placeholder" aria-hidden="true" />;
}

/** Still image with native tier-driven loading and no remount fade. */
function StillMedia({ item, tier }: { item: GalleryItem; tier: LoadTier }) {
  return (
    <>
      <Placeholder />
      <img
        className="absolute inset-0 size-full object-cover"
        src={item.src}
        alt={item.alt}
        loading={tier === "full" ? "eager" : "lazy"}
        decoding="async"
      />
    </>
  );
}

/**
 * Hover-to-play video: poster still by default, plays (muted / looped /
 * playsInline) on hover & focus, pauses and resets on leave. Respects reduced
 * motion — when set, the poster stays put and the clip never autoplays on hover.
 */
function HoverVideo({
```

```
  item,
  tier,
  playing,
  reducedMotion,
}: {
  item: GalleryItem;
  tier: LoadTier;
  playing: boolean;
  reducedMotion: boolean;
}) {
  const ref = useRef<HTMLVideoElement | null>(null);
  const [videoLoaded, setVideoLoaded] = useState(false);
  const active = playing && !reducedMotion;
  const hasPoster = item.src.length > 0;
  const showVideo = active && videoLoaded;

  useEffect(() => {
    const v = ref.current;
    if (v == null) return;
    if (active) {
      void v.play()?.catch(() => {});
    } else {
      v.pause();
      try {
        v.currentTime = 0;
      } catch {}
    }
  }, [active]);

  return (
    <>
      <Placeholder />
      {hasPoster ? (
        <img
          className="absolute inset-0 size-full object-cover"
          src={item.src}
          alt={item.alt}
          loading={tier === "full" ? "eager" : "lazy"}
          decoding="async"
          style={{ opacity: showVideo ? 0 : 1 }}
        />
      ) : null}
      {/* Historical video nodes stay network-idle until explicit hover/focus. */}
      <video
        ref={ref}
        className="absolute inset-0 size-full object-cover"
        muted
        loop
        playsInline
        preload="none"
        poster={hasPoster ? item.src : undefined}
        onLoadedData={() => setVideoLoaded(true)}
        style={{ opacity: showVideo ? 1 : 0 }}
      >
        <source src={item.videoSrc} type="video/mp4" />
      </video>
    </>
  );
}

function GalleryTileComponent({ item, rect, top, tier, reducedMotion }: GalleryTileProps) {
  const [hovered, setHovered] = useState(false);
  const [liked, setLiked] = useState(false);

  if (item.status === "generating") {
    return (
      <GenerationTile
        state="generating"
        ratio="auto"
        className="qg-tile"
```

```
        style={rectStyle(rect, top)}
      />
    );
  }

  const isVideo = item.kind === "video";
  const media = isVideo ? (
    <HoverVideo item={item} tier={tier} playing={hovered} reducedMotion={reducedMotion} />
  ) : (
    <StillMedia item={item} tier={tier} />
  );

  // Per-tile affordances. `CardActions` caps the visible controls at 3 (its
  // default `max`): with more than that, the first two render as glass buttons
  // and the rest collapse into the three-dots overflow menu.
  const actions: CardAction[] = [
    {
      id: "like",
      label: liked ? "Unlike" : "Like",
      icon: IconFavorite,
      onSelect: () => setLiked((v) => !v),
    },
    { id: "download", label: "Download", icon: IconDownload },
    {
      id: "copy",
      label: "Copy prompt",
      icon: IconContentCopy,
      onSelect: () => void navigator.clipboard?.writeText(item.prompt),
    },
    { id: "share", label: "Share", icon: IconShare },
    { id: "delete", label: "Delete", icon: IconDelete, danger: true },
  ];

  // The canonical single-generation tile: it owns the full-bleed open trigger and
  // the hover `CardActions` rail (siblings, never nested), so this feed reads the
  // SAME as every other generation card in the app. The feed only supplies the
  // engine-positioned frame + its media (hover-to-play video / tiered still).
  return (
    <GenerationTile
      ratio="auto"
      className="qg-tile"
      style={rectStyle(rect, top)}
      media={media}
      actions={actions}
      generation={{
        src: isVideo ? (item.videoSrc ?? item.src) : item.src,
        poster: isVideo ? item.src : undefined,
        mediaType: isVideo ? "video" : "image",
        aspectRatio: item.width / item.height,
        prompt: item.prompt,
      }}
      openLabel={`Open generation: ${item.prompt}`}
      onMouseEnter={isVideo ? () => setHovered(true) : undefined}
      onMouseLeave={isVideo ? () => setHovered(false) : undefined}
      onFocus={isVideo ? () => setHovered(true) : undefined}
      onBlur={isVideo ? () => setHovered(false) : undefined}
    />
  );
}

export const GalleryTile = memo(GalleryTileComponent);
```

app/src/components/gallery/gallery.css

```
/**
 * Justified-masonry History gallery — presentation-only styling that isn't
 * expressible as plain Tailwind utilities (the content-visibility hint).
 * Colours reference the global Quanta `--hf-*` custom properties. Imported directly by the gallery
 * module (Vite injects it); it intentionally does NOT touch the shared
 * `styles.css` entry. Class names are namespaced `qg-` (quanta-gallery).
 */

/* Scroll viewport — the windowed content lives inside an absolutely-sized
 * sizer, so tiles never trigger layout of their neighbours. */
.qg-viewport {
  overscroll-behavior: contain;
  will-change: scroll-position;
}

/* One tile shell — absolutely positioned by the engine's computed rect.
 * `content-visibility` lets the browser skip rendering work for the (rare)
 * off-screen tile still mounted during a fast fling. */
.qg-tile {
  position: absolute;
  overflow: hidden;
  border-radius: var(--hf-radius-300);
  background-color: var(--hf-color-background-secondary);
  content-visibility: auto;
}

/* Neutral fallback behind media that has not painted yet. */
.qg-placeholder {
  position: absolute;
  inset: 0;
  background-color: var(--hf-color-background-secondary);
}

@media (prefers-reduced-motion: reduce) {
  .qg-viewport {
    scroll-behavior: auto;
  }
}
```

app/src/components/gallery/index.ts

```
export { JustifiedGallery } from "./justified-gallery.tsx";
export type { JustifiedGalleryProps } from "./justified-gallery.tsx";
export { JustifiedLayoutEngine } from "./justified-engine.ts";
export type { Layout, LayoutRow, TileRect, EngineConfig } from "./justified-engine.ts";
export type { GalleryItem, MediaKind, ItemStatus, LoadTier } from "./types.ts";
export { makeInitialItems, makeOlderBatch, makeBatch, GROUPS } from "./demo-data.ts";
```

app/src/components/gallery/justified-engine.ts

```
import type { GalleryItem } from "./types.ts";

/**
 * JustifiedLayoutEngine — a bespoke, out-of-React layout + windowing engine for
 * a Flickr / Google-Photos style justified-masonry gallery.
 *
 * It is a plain TypeScript class with NO React dependency. The React layer feeds
 * it the item list and the current geometry (container width, target row height,
 * gap) and, on every scroll frame, asks it which rows are visible. All of the
 * heavy math — packing items into equal-height rows, justifying each row to the
 * container width, stacking dated groups with headers, binary-searching the
 * visible window, and classifying load tiers by distance from the viewport —
 * lives here, so the React tree only ever renders a screenful of absolutely
 * positioned tiles.
 *
 * ■■ The justified algorithm ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * Pick a target row height `h0`. Each item, scaled to `h0`, has display width
 * `h0 * aspect`. Greedily pack items into a row until their combined natural
 * width (plus gaps) reaches the container width, then solve for the exact row
 * height that makes the row fill the width precisely:
 *
 *     h = (containerWidth - gap * (n - 1)) / Σ aspect■
 *
 * and set each tile width to `h * aspect■`. The final (incomplete) row of a
 * group is left at the target height (never over-stretched) and left-aligned.
 */

/** One tile's resolved rectangle, relative to its row's top-left. */
export interface TileRect {
  /** Index into the flat items array. */
  index: number;
  item: GalleryItem;
  /** Horizontal offset within the content column. */
  x: number;
  width: number;
  height: number;
}

export type LayoutRowType = "header" | "tiles";

/** A laid-out row — either a dated group header or a justified row of tiles. */
export interface LayoutRow {
  type: LayoutRowType;
  /** Absolute top offset within the scrollable content. */
  y: number;
  height: number;
  /** Sequential index among rows (stable key). */
  key: string;
  /** header only — the group label. */
  label?: string;
  /** tiles only — the justified tiles in this row. */
  tiles?: TileRect[];
}

export interface Layout {
  rows: LayoutRow[];
  totalHeight: number;
  /** Number of items covered by this layout (for infinite-scroll bookkeeping). */
  itemCount: number;
}

export interface EngineConfig {
  /** Inner content width in px (viewport clientWidth). */
  containerWidth: number;
  /** Target/base row height in px — the density knob. */
  targetRowHeight: number;
  /** Gap between tiles AND between rows, in px. */
  gap: number;
```

```
  /** Header band height in px. */
  headerHeight: number;
  /** Extra space below each group. */
  groupGap: number;
  /**
   * Clamp on the per-row aspect sum before a row is force-closed, so a run of
   * ultra-wide panoramas can't produce a single skyscraper-tall row.
   */
  maxRowHeight: number;
  /**
   * Whether to split items into dated batches with "Today" / "Yesterday" / …
   * headers. When false, every item is packed into ONE continuous justified
   * feed — no headers, no group gaps, no mid-feed left-aligned partial rows.
   */
  grouped: boolean;
}

const DEFAULT_CONFIG: EngineConfig = {
  containerWidth: 0,
  targetRowHeight: 200,
  gap: 6,
  headerHeight: 44,
  groupGap: 20,
  maxRowHeight: 460,
  grouped: true,
};

interface GroupRange {
  id: string;
  label: string;
  start: number;
  end: number; // exclusive
}

export class JustifiedLayoutEngine {
  private items: GalleryItem[] = [];
  private groups: GroupRange[] = [];
  private config: EngineConfig = { ...DEFAULT_CONFIG };
  private layout: Layout = { rows: [], totalHeight: 0, itemCount: 0 };
  /** Per-item resolved geometry, indexed by item index — powers scroll anchoring. */
  private itemTops: number[] = [];
  private itemHeights: number[] = [];
  private dirty = true;

  setItems(items: GalleryItem[]): void {
    this.items = items;
    this.recomputeGroups();
    this.dirty = true;
  }

  /** Returns true if any geometry field actually changed. */
  setConfig(patch: Partial<EngineConfig>): boolean {
    let changed = false;
    let groupingChanged = false;
    for (const key of Object.keys(patch) as (keyof EngineConfig)[]) {
      const next = patch[key];
      if (next != null && next !== this.config[key]) {
        this.config[key] = next as never;
        changed = true;
        if (key === "grouped") groupingChanged = true;
      }
    }
    // Toggling grouping changes the group ranges, not just geometry.
    if (groupingChanged) this.recomputeGroups();
    if (changed) this.dirty = true;
    return changed;
  }

  getConfig(): Readonly<EngineConfig> {
    return this.config;
```

```
}

private recomputeGroups(): void {
  // Grouping off → one continuous group over the whole dataset (no headers).
  if (!this.config.grouped) {
    this.groups =
      this.items.length > 0 ? [{ id: "all", label: "", start: 0, end: this.items.length }] : [];
    return;
  }

  const groups: GroupRange[] = [];
  let current: GroupRange | null = null;
  for (let i = 0; i < this.items.length; i++) {
    const item = this.items[i]!;
    if (current == null || current.id !== item.groupId) {
      if (current != null) current.end = i;
      current = { id: item.groupId, label: item.groupLabel, start: i, end: i + 1 };
      groups.push(current);
    }
  }
  if (current != null) current.end = this.items.length;
  this.groups = groups;
}

/** Compute (or return cached) layout for the current items + config. */
compute(): Layout {
  if (!this.dirty) return this.layout;

  const { containerWidth, targetRowHeight, gap, headerHeight, groupGap, maxRowHeight } =
    this.config;
  const rows: LayoutRow[] = [];
  const itemTops = new Array<number>(this.items.length).fill(0);
  const itemHeights = new Array<number>(this.items.length).fill(0);

  if (containerWidth <= 0) {
    this.layout = { rows: [], totalHeight: 0, itemCount: this.items.length };
    this.itemTops = itemTops;
    this.itemHeights = itemHeights;
    this.dirty = false;
    return this.layout;
  }

  let y = 0;
  let rowKey = 0;

  const { grouped } = this.config;

  for (const group of this.groups) {
    // Group header band — only when dated grouping is enabled.
    if (grouped) {
      rows.push({
        type: "header",
        y,
        height: headerHeight,
        key: `h-${group.id}`,
        label: group.label,
      });
      y += headerHeight;
    }

    // Greedily pack this group's items into justified rows.
    let rowStart = group.start;
    let aspectSum = 0;

    const flushRow = (endExclusive: number, isLastRowOfGroup: boolean) => {
      const n = endExclusive - rowStart;
      if (n <= 0) return;
      const totalGap = gap * (n - 1);
      // The height that makes the row exactly fill the width.
      let rowHeight = (containerWidth - totalGap) / aspectSum;
```

```
      // The final partial row of a group is never stretched past the target.
      if (isLastRowOfGroup && rowHeight > targetRowHeight) rowHeight = targetRowHeight;
      // Guard against a run of panoramas producing a razor-thin row, or a
      // single portrait producing a skyscraper.
      rowHeight = Math.min(rowHeight, maxRowHeight);

      const tiles: TileRect[] = [];
      let x = 0;
      for (let i = rowStart; i < endExclusive; i++) {
        const item = this.items[i]!;
        const aspect = item.width / item.height;
        const w = rowHeight * aspect;
        tiles.push({ index: i, item, x, width: w, height: rowHeight });
        itemTops[i] = y;
        itemHeights[i] = rowHeight;
        x += w + gap;
      }
      rows.push({ type: "tiles", y, height: rowHeight, key: `r-${rowKey++}`, tiles });
      y += rowHeight + gap;
    };

    for (let i = group.start; i < group.end; i++) {
      const item = this.items[i]!;
      const aspect = item.width / item.height;
      aspectSum += aspect;
      // Natural width of the row so far at the target height.
      const naturalWidth = targetRowHeight * aspectSum + gap * (i - rowStart);
      if (naturalWidth >= containerWidth) {
        flushRow(i + 1, false);
        rowStart = i + 1;
        aspectSum = 0;
      }
    }
    // Trailing partial row.
    if (rowStart < group.end) flushRow(group.end, true);

    // Remove the trailing inter-row gap, add the group gap instead.
    y = y - gap + groupGap;
  }

  const totalHeight = Math.max(0, y - groupGap);
  this.layout = { rows, totalHeight, itemCount: this.items.length };
  this.itemTops = itemTops;
  this.itemHeights = itemHeights;
  this.dirty = false;
  return this.layout;
}

getLayout(): Layout {
  return this.compute();
}

/**
 * The visible window: the [startRow, endRow) slice of rows intersecting the
 * viewport expanded by `overscanPx` on each edge. Binary-searches the rows
 * (which are sorted by `y`) so cost is O(log n), independent of dataset size.
 */
getWindow(
  scrollTop: number,
  viewportHeight: number,
  overscanPx: number,
): { startRow: number; endRow: number } {
  const { rows } = this.compute();
  if (rows.length === 0) return { startRow: 0, endRow: 0 };

  const top = scrollTop - overscanPx;
  const bottom = scrollTop + viewportHeight + overscanPx;

  // First row whose bottom edge is past `top`.
  let lo = 0;
```

```
    let hi = rows.length - 1;
    let startRow = rows.length;
    while (lo <= hi) {
      const mid = (lo + hi) >> 1;
      const row = rows[mid]!;
      if (row.y + row.height >= top) {
        startRow = mid;
        hi = mid - 1;
      } else {
        lo = mid + 1;
      }
    }

    // First row whose top edge is at/after `bottom` → the exclusive end.
    lo = 0;
    hi = rows.length - 1;
    let endRow = rows.length;
    while (lo <= hi) {
      const mid = (lo + hi) >> 1;
      const row = rows[mid]!;
      if (row.y > bottom) {
        endRow = mid;
        hi = mid - 1;
      } else {
        lo = mid + 1;
      }
    }

    return { startRow: Math.min(startRow, rows.length), endRow };
  }

  /** Top offset of an item — used to re-pin the scroll after a re-layout. */
  getItemTop(index: number): number {
    this.compute();
    return this.itemTops[index] ?? 0;
  }

  getItemHeight(index: number): number {
    this.compute();
    return this.itemHeights[index] ?? 0;
  }

  /**
   * The topmost item at least partially visible at `scrollTop`, plus the pixel
   * offset between the viewport top and that item's top. The React layer stores
   * this before a density/resize re-layout and restores it afterwards so the
   * content under the user's eyes stays put (scroll anchoring).
   */
  findAnchor(scrollTop: number): { index: number; offset: number } {
    this.compute();
    if (this.items.length === 0) return { index: 0, offset: 0 };
    // Binary search itemTops for the last item whose top is <= scrollTop.
    let lo = 0;
    let hi = this.itemTops.length - 1;
    let index = 0;
    while (lo <= hi) {
      const mid = (lo + hi) >> 1;
      if (this.itemTops[mid]! <= scrollTop) {
        index = mid;
        lo = mid + 1;
      } else {
        hi = mid - 1;
      }
    }
    return { index, offset: scrollTop - (this.itemTops[index] ?? 0) };
  }
}
```

app/src/components/gallery/justified-gallery.tsx

```
import { useMemo } from "react";
import { Loader } from "@higgsfield/quanta/loader";
import { Typography } from "@higgsfield/quanta/typography";
import { GalleryTile } from "./gallery-tile.tsx";
import { DensityControl } from "./density-control.tsx";
import { useJustifiedGallery } from "./use-justified-gallery.ts";
import { useReducedMotion } from "./use-reduced-motion.ts";
import { makeInitialItems } from "./demo-data.ts";
import type { LoadTier } from "./types.ts";
import type { GalleryItem } from "./types.ts";
import "./gallery.css";

/**
 * JustifiedGallery — the virtualized, Flickr-style justified-masonry feed that
 * backs the History tab.
 *
 * Architecture:
 *   • `JustifiedLayoutEngine` (plain TS) owns all layout + windowing math.
 *   • `useJustifiedGallery` wires the engine to scroll / resize / density and
 *     exposes only the visible window of rows.
 *   • This component renders that window as absolutely-positioned `GalleryTile`s
 *     over a fixed-height sizer, so the DOM only ever holds a screenful of tiles
 *     regardless of dataset size.
 */

interface JustifiedGalleryCommonProps {
  /**
   * Split the feed into dated batches with "Today" / "Yesterday" / … headers.
   * Defaults to true; pass `false` for one continuous justified feed with no
   * day separators.
   */
  grouped?: boolean;
  hasMore?: boolean;
  loadingMore?: boolean;
  onLoadMore?: () => void | Promise<unknown>;
}

export type JustifiedGalleryProps = JustifiedGalleryCommonProps &
  ({ demo: true; items?: never } | { demo?: false; items: GalleryItem[] });

const numberFormat = new Intl.NumberFormat("en-US");

export function JustifiedGallery(props: JustifiedGalleryProps) {
  const { grouped = true } = props;
  const initial = useMemo(
    () => (props.demo ? makeInitialItems() : props.items),
    [props.demo, props.items],
  );
  const reducedMotion = useReducedMotion();

  const {
    viewportRef,
    layout,
    visibleRows,
    scrollTop,
    viewportHeight,
    density,
    setDensity,
    itemCount,
    loadingMore,
  } = useJustifiedGallery(initial, grouped, {
    demo: props.demo === true,
    hasMore: props.hasMore,
    loadingMore: props.loadingMore,
    onLoadMore: props.onLoadMore,
  });

  const viewTop = scrollTop;
```

```
  const viewBottom = scrollTop + viewportHeight;

  return (
    <div className="flex min-h-0 flex-1 flex-col gap-3">
      <header className="flex shrink-0 items-center justify-between gap-4 px-0.5">
        <div className="flex items-center gap-2">
          <Typography as="h2" variant="body-sm-semi-bold" color="primary">
            Your generations
          </Typography>
          <Typography as="span" variant="caption-sm-regular" color="tertiary">
            {numberFormat.format(itemCount)}
            {" items"}
          </Typography>
          {loadingMore && (
            <span className="flex items-center gap-1.5 text-q-text-tertiary">
              <Loader variant="circle" size="xs" color="neutral" aria-label="Loading more" />
              <Typography as="span" variant="caption-sm-regular" color="tertiary">
                Loading
              </Typography>
            </span>
          )}
        </div>
        <DensityControl value={density} onChange={setDensity} />
      </header>

      <div ref={viewportRef} className="qg-viewport relative min-h-0 flex-1 overflow-y-auto">
        {itemCount === 0 ? (
          <div className="flex h-full flex-col items-center justify-center gap-1 px-6 text-center">
            <Typography as="p" variant="body-md-semi-bold" color="primary">
              No generations yet
            </Typography>
            <Typography as="p" variant="caption-sm-regular" color="secondary">
              Generated results will appear here.
            </Typography>
          </div>
        ) : (
          <div className="relative w-full" style={{ height: layout.totalHeight }}>
            {visibleRows.map((row) => {
              if (row.type === "header") {
                return (
                  <div
                    key={row.key}
                    className="absolute inset-x-0 flex items-end px-0.5 pb-2"
                    style={{ top: row.y, height: row.height }}
                  >
                    <Typography as="h3" variant="caption-sm-medium" color="tertiary">
                      {row.label}
                    </Typography>
                  </div>
                );
              }
              const rowVisible = row.y < viewBottom && row.y + row.height > viewTop;
              const tier: LoadTier = rowVisible ? "full" : "near";
              return row.tiles!.map((rect) => (
                <GalleryTile
                  key={rect.item.id}
                  item={rect.item}
                  rect={rect}
                  top={row.y}
                  tier={tier}
                  reducedMotion={reducedMotion}
                />
              ));
            })}
          </div>
        )}
      </div>
    </div>
  );
}
```

app/src/components/gallery/types.ts

```
/**
 * Data model for the justified-masonry History gallery.
 *
 * A `GalleryItem` is a single generation. Its REAL pixel dimensions
 * (`width` / `height`) drive the justified layout — the engine derives each
 * tile's aspect ratio from these, never from a hard-coded ratio — so the demo
 * set deliberately mixes portrait, landscape, square and panoramic media.
 */

export type MediaKind = "image" | "video";
export type ItemStatus = "ready" | "generating";

export interface GalleryItem {
  /** Stable id — used as the React key. */
  id: string;
  /** `image` renders an `<img>`, `video` renders a hover-to-play `<video>` + poster. */
  kind: MediaKind;
  /** Lifecycle — `ready` shows the asset, `generating` shows the pulsing card. */
  status: ItemStatus;
  /** The still source: the image for `image`, the poster frame for `video`. */
  src: string;
  /** Video source (only meaningful when `kind === 'video'`). */
  videoSrc?: string;
  /** REAL intrinsic width in px — the numerator of the tile aspect ratio. */
  width: number;
  /** REAL intrinsic height in px — the denominator of the tile aspect ratio. */
  height: number;
  /** The generation prompt (shown in the detail modal + used as the a11y label). */
  prompt: string;
  /** Accessible alt text for the still. */
  alt: string;
  /** Which dated batch this item belongs to (e.g. `today`). */
  groupId: string;
  /** Human batch label rendered as the section header (e.g. `Today`). */
  groupLabel: string;
}

/** How near a tile is to the viewport — drives the load tier (quality upgrade). */
export type LoadTier = "full" | "near" | "far";
```

app/src/components/gallery/use-justified-gallery.ts

```
import { useCallback, useEffect, useLayoutEffect, useMemo, useRef, useState } from "react";
import type { RefObject } from "react";
import { JustifiedLayoutEngine } from "./justified-engine.ts";
import type { Layout, LayoutRow } from "./justified-engine.ts";
import type { GalleryItem } from "./types.ts";
import { makeOlderBatch } from "./demo-data.ts";

/**
 * Tile-size presets, indexed by the slider step. The slider runs left → right =
 * smaller → LARGER tiles, so this array is ASCENDING: step 0 is the smallest /
 * densest, the last step the largest / least-dense. The value is the target row
 * height in px that the engine packs against. The scale is biased toward larger
 * tiles (base default sits above the old minimum) since the small end read too
 * cramped.
 */
export const DENSITY_ROW_HEIGHTS = [220, 280, 360, 460, 580] as const;
export const DEFAULT_DENSITY = 1;

const GAP = 6;
/** Rows within this many px of the viewport edge are rendered (windowing). */
const OVERSCAN_PX = 400;
/** Load the next batch when the viewport bottom is within this px of the end. */
const INFINITE_MARGIN = 1200;
/** Cap total items so the demo can't grow without bound. */
const MAX_ITEMS = 1400;

interface InfiniteScrollOptions {
  demo?: boolean;
  hasMore?: boolean;
  loadingMore?: boolean;
  onLoadMore?: () => void | Promise<unknown>;
}

export interface UseJustifiedGalleryResult {
  viewportRef: RefObject<HTMLDivElement | null>;
  layout: Layout;
  /** The [startRow, endRow) slice of rows currently rendered. */
  visibleRows: LayoutRow[];
  /** Current scroll offset (px) — used to classify each tile's load tier. */
  scrollTop: number;
  viewportHeight: number;
  density: number;
  setDensity: (level: number) => void;
  itemCount: number;
  loadingMore: boolean;
}

export function useJustifiedGallery(
  items: GalleryItem[],
  grouped = true,
  { demo = false, hasMore = false, loadingMore = false, onLoadMore }: InfiniteScrollOptions = {},
): UseJustifiedGalleryResult {
  const viewportRef = useRef<HTMLDivElement | null>(null);
  const engineRef = useRef<JustifiedLayoutEngine>(null as unknown as JustifiedLayoutEngine);
  if (engineRef.current == null) engineRef.current = new JustifiedLayoutEngine();
  const engine = engineRef.current;

  const [width, setWidth] = useState(0);
  const [density, setDensityState] = useState(DEFAULT_DENSITY);
  // Scroll position is measurement, not render state. Making this state causes
  // every mounted media/dialog tile to rerender on every animation frame.
  const scrollTopRef = useRef(0);
  const [viewportHeight, setViewportHeight] = useState(0);
  const [range, setRange] = useState({ startRow: 0, endRow: 0 });

  // Infinite-scroll state: appended "older" batches beyond the initial set.
  const [olderItems, setOlderItems] = useState<GalleryItem[]>([]);
  const [demoLoadingMore, setDemoLoadingMore] = useState(false);
```

```
const pageRef = useRef(0);

const allItems = useMemo(
  () => (demo && olderItems.length > 0 ? [...items, ...olderItems] : items),
  [demo, items, olderItems],
);

const targetRowHeight = DENSITY_ROW_HEIGHTS[density] ?? DENSITY_ROW_HEIGHTS[DEFAULT_DENSITY]!;

// Feed geometry to the engine and (re)compute the layout. The engine caches
// internally, so this is cheap when nothing changed.
const layout = useMemo(() => {
  engine.setItems(allItems);
  engine.setConfig({ containerWidth: width, targetRowHeight, gap: GAP, grouped });
  return engine.getLayout();
}, [engine, allItems, width, targetRowHeight, grouped]);

// Anchor captured on the last scroll frame (against the pre-change layout) so
// a density change or resize can re-pin the same item under the viewport.
const anchorRef = useRef({ index: 0, offset: 0 });

const readRange = useCallback(() => {
  const el = viewportRef.current;
  if (el == null) return;
  const st = el.scrollTop;
  const vh = el.clientHeight;
  const win = engine.getWindow(st, vh, OVERSCAN_PX);
  scrollTopRef.current = st;
  setViewportHeight(vh);
  setRange((prev) => (prev.startRow === win.startRow && prev.endRow === win.endRow ? prev : win));
}, [engine]);

// Measure width + height with a ResizeObserver.
useLayoutEffect(() => {
  const el = viewportRef.current;
  if (el == null || typeof ResizeObserver === "undefined") return;
  const ro = new ResizeObserver(() => {
    setWidth(el.clientWidth);
    setViewportHeight(el.clientHeight);
  });
  ro.observe(el);
  setWidth(el.clientWidth);
  setViewportHeight(el.clientHeight);
  return () => ro.disconnect();
}, []);

// After any re-layout (width / density / items), recompute the window and
// restore the scroll anchor so the visible content stays stable.
useLayoutEffect(() => {
  const el = viewportRef.current;
  if (el == null) return;
  const anchor = anchorRef.current;
  if (anchor.index > 0) {
    const nextTop = engine.getItemTop(anchor.index) + anchor.offset;
    if (Math.abs(nextTop - el.scrollTop) > 0.5) el.scrollTop = nextTop;
  }
  readRange();
  // Depend on the layout object identity, which changes on every recompute.
}, [engine, layout, readRange]);

// Keep a stable loader that reads the freshest counts via refs.
const loadingRef = useRef(false);
const lastAutoLoadKeyRef = useRef<string | null>(null);
const countRef = useRef(allItems.length);
countRef.current = allItems.length;
const feedEdgeKeyRef = useRef("");
feedEdgeKeyRef.current = `${allItems.length}:${allItems[0]?.id ?? ""}:${allItems.at(-1)?.id ?? ""}`;

const triggerLoadMore = useCallback(() => {
  if (loadingRef.current) return;
```

```
  if (!demo && (loadingMore || !hasMore || onLoadMore == null)) return;
  if (demo && countRef.current >= MAX_ITEMS) return;
  if (lastAutoLoadKeyRef.current === feedEdgeKeyRef.current) return;
  lastAutoLoadKeyRef.current = feedEdgeKeyRef.current;
  loadingRef.current = true;
  if (demo) {
    setDemoLoadingMore(true);
    window.setTimeout(() => {
      const page = pageRef.current++;
      setOlderItems((prev) => [...prev, ...makeOlderBatch(page)]);
      loadingRef.current = false;
      setDemoLoadingMore(false);
    }, 120);
    return;
  }
  void Promise.resolve(onLoadMore?.())
    .catch(() => undefined)
    .finally(() => {
      loadingRef.current = false;
    });
}, [demo, hasMore, loadingMore, onLoadMore]);

// Fill short first pages, but never start before the viewport has geometry.
useEffect(() => {
  const el = viewportRef.current;
  if (
    el != null &&
    width > 0 &&
    el.clientHeight > 0 &&
    el.scrollTop + el.clientHeight >= layout.totalHeight - INFINITE_MARGIN
  ) {
    triggerLoadMore();
  }
}, [layout.totalHeight, loadingMore, triggerLoadMore, width]);

// Scroll handling: rAF-throttled window recompute + infinite-scroll trigger.
// Scroll position lives in a ref so frames within the same row window do not
// rerender every mounted media tile.
useEffect(() => {
  const el = viewportRef.current;
  if (el == null) return;

  let raf = 0;
  const onScroll = () => {
    if (raf !== 0) return;
    raf = requestAnimationFrame(() => {
      raf = 0;
      const st = el.scrollTop;

      // Capture the anchor for the next density/resize re-layout.
      anchorRef.current = engine.findAnchor(st);

      readRange();

      // Infinite scroll: append an older batch as the bottom nears.
      const nearBottom = st + el.clientHeight >= engine.getLayout().totalHeight - INFINITE_MARGIN;
      if (nearBottom) triggerLoadMore();
    });
  };

  el.addEventListener("scroll", onScroll, { passive: true });
  return () => {
    el.removeEventListener("scroll", onScroll);
    if (raf !== 0) cancelAnimationFrame(raf);
  };
}, [engine, readRange, triggerLoadMore]);

const setDensity = useCallback(
  (level: number) => {
    // The anchor was captured on the last scroll frame; refresh it against the
```

```
      // live scroll position right now so the re-pin is exact even without a
      // recent scroll event.
      const el = viewportRef.current;
      if (el != null) anchorRef.current = engine.findAnchor(el.scrollTop);
      setDensityState(level);
    },
    [engine],
  );

  const visibleRows = useMemo(
    () => layout.rows.slice(range.startRow, range.endRow),
    [layout, range.startRow, range.endRow],
  );

  return {
    viewportRef,
    layout,
    visibleRows,
    scrollTop: scrollTopRef.current,
    viewportHeight,
    density,
    setDensity,
    itemCount: allItems.length,
    loadingMore: demo ? demoLoadingMore : loadingMore,
  };
}
```

app/src/components/gallery/use-reduced-motion.ts

```
import { useEffect, useState } from "react";

/**
 * Tracks the `prefers-reduced-motion` media query. When true, the gallery does
 * NOT autoplay videos on hover (posters stay put) and skips fade-in transitions.
 */
export function useReducedMotion(): boolean {
  const [reduced, setReduced] = useState(false);

  useEffect(() => {
    if (typeof window === "undefined" || window.matchMedia == null) return;
    const mq = window.matchMedia("(prefers-reduced-motion: reduce)");
    const update = () => setReduced(mq.matches);
    update();
    mq.addEventListener("change", update);
    return () => mq.removeEventListener("change", update);
  }, []);

  return reduced;
}
```

app/src/components/generation-card/card-actions.tsx

```
import type { MouseEvent, ReactNode } from "react";
import { EllipsisVertical as IconMoreVertical } from "lucide-react";
import { Dropdown } from "@higgsfield/quanta/dropdown";
import { Icon, type IconGlyph } from "@higgsfield/quanta/icon";

/**
 * CardActions — the hover-revealed action rail for a gallery tile (heart /
 * download / copy / share …). It enforces ONE rule: a card shows at most
 * `max` (default 3) affordances. When there are more actions than that, only
 * the first `max - 1` render as glass icon buttons and the remainder collapse
 * into a trailing three-dots (`EllipsisVertical`) overflow menu — so a tile never
 * shows more than `max` controls.
 *
 * It is presentation-only and composition-first: the host passes the action
 * list, this component decides what stays inline vs. what goes in the menu.
 * Render it as a child of a `.group` tile; the rail fades in on hover / focus.
 *
 * The rail is a passive layer (`pointer-events-none`) so its empty space never
 * blocks the tile's underlying open-detail trigger — only the buttons
 * themselves are interactive. Each handler stops propagation so a click acts on
 * the tile action, not the card.
 */

export interface CardAction {
  /** Stable id — React key + menu row value. */
  id: string;
  /** Accessible label (button `aria-label` + menu row title). */
  label: string;
  /** Leading glyph. */
  icon: IconGlyph;
  /** Invoked on click / menu select. */
  onSelect?: () => void;
  /** Destructive row — red styling inside the overflow menu. */
  danger?: boolean;
}

export interface CardActionsProps {
  actions: CardAction[];
  /** Hard cap on visible affordances (buttons + the more-menu). Default 3. */
  max?: number;
}

/** Glass circular icon button shared by the inline actions and the more-trigger. */
const GLASS_BUTTON =
  "pointer-events-auto flex size-8 shrink-0 items-center justify-center rounded-q-full bg-q-transparent-dark-40 text-q-icon-primary backdrop-blur-md transition-colors hover:bg-q-transparent-dark-60 fo

function stop(event: MouseEvent) {
  event.stopPropagation();
}

/** One inline glass icon button. */
function ActionButton({ action }: { action: CardAction }) {
  return (
    <button
      type="button"
      aria-label={action.label}
      className={GLASS_BUTTON}
      onPointerDown={stop}
      onClick={(event) => {
        event.stopPropagation();
        action.onSelect?.();
      }}
    >
      <Icon as={action.icon} size="sm" />
    </button>
  );
}
```

```
/** The overflow menu — a three-dots trigger over a Dropdown of the extra rows. */
function MoreMenu({ actions }: { actions: CardAction[] }): ReactNode {
  return (
    <Dropdown.Root>
      <Dropdown.Trigger
        aria-label="More actions"
        className={GLASS_BUTTON}
        onPointerDown={stop}
        onClick={stop}
      >
        <Icon as={IconMoreVertical} size="sm" />
      </Dropdown.Trigger>
      <Dropdown.Content surface="solid" side="bottom" align="end" sideOffset={6}>
        {actions.map((action) => (
          <Dropdown.Item
            key={action.id}
            value={action.id}
            danger={action.danger}
            start={<Icon as={action.icon} size="sm" />}
            title={action.label}
            onSelect={() => action.onSelect?.()}
          />
        ))}
      </Dropdown.Content>
    </Dropdown.Root>
  );
}

export function CardActions({ actions, max = 3 }: CardActionsProps) {
  if (actions.length === 0) return null;

  // At most `max` controls: everything fits, OR (max - 1) buttons + a more-menu.
  const overflow = actions.length > max;
  const inline = overflow ? actions.slice(0, max - 1) : actions;
  const extra = overflow ? actions.slice(max - 1) : [];

  return (
    <div className="pointer-events-none absolute right-2 top-2 z-[2] flex flex-col items-center gap-1.5 opacity-0 transition-opacity duration-150 ease-out group-hover:opacity-100 group-focus-within:op
      {inline.map((action) => (
        <ActionButton key={action.id} action={action} />
      ))}
      {extra.length > 0 ? <MoreMenu actions={extra} /> : null}
    </div>
  );
}
```

app/src/components/generation-card/generation-card.css

```
/**
 * GenerationCard presentation — a feed / history generation tile (Figma
 * Supercomputer-2 "Feed/Card" 2001:84301) with a "generating" state (Figma
 * Cinema-Studio-V4 20037:25838): a radius/300 clipped media box, an optional
 * bottom title over the scrim, and — while generating — a brand gradient glow
 * pulsing at the top of the card behind a "Generating" status pill.
 *
 * Imported AFTER media.css (components.css order) so the caption paddings and
 * the generating overlay win over the base `q-media-overlay-*` layers.
 */
@source "./generation-card.tsx";

@utility q-generation-card {
  position: relative;
  display: block;
  width: 100%;
  min-width: var(--hf-space-0);
  overflow: hidden;
  border-radius: var(--hf-radius-300);
  background-color: var(--hf-color-background-secondary);

  /* Interactive hosts (render={<button/>} / <a>) — cursor, hover, focus ring. */
  &:is(button, a) {
    cursor: pointer;
    outline: none;
    text-align: start;
    @apply q-focus-ring;
  }
}

/* The composed Media box fills the tile. */
@utility q-generation-card-media {
  width: 100%;
  height: 100%;
}

/* Generating canvas — the empty box the glow sits over (5% white surface). */
@utility q-generation-card-canvas {
  background-color: var(--hf-color-transparent-light-05);
}

/* Bottom title row — tighter than the gallery caption (Figma feed: px-12/py-10). */
@utility q-generation-card-caption {
  padding: var(--hf-space-250) var(--hf-space-300);
}

@utility q-generation-card-title {
  display: block;
  min-width: var(--hf-space-0);
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

/* ■■ generating state — glow + status pill ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */

/* Full-bleed overlay that clips the glow to the card's rounded corners. */
@utility q-generation-card-generating {
  position: absolute;
  inset: var(--hf-space-0);
  overflow: hidden;
  border-radius: inherit;
  pointer-events: none;
}

/* The brand gradient glow bleeding from the TOP edge; opacity pulses. */
@utility q-generation-card-glow {
  position: absolute;
```

```
  inset-block-start: 0;
  inset-inline: 0;
  height: 45%;
  background-image: radial-gradient(
    120% 100% at 50% 0%,
    color-mix(in oklab, var(--hf-color-brand-primary) 42%, transparent) 0%,
    color-mix(in oklab, var(--hf-color-brand-primary) 14%, transparent) 45%,
    transparent 82%
  );
  /* no token: pulse easing keyword (ease-in-out) + duration > 1400ms scale max —
   * a softer, weaker, slower twinkle. */
  animation: q-generation-pulse 2600ms ease-in-out infinite;
}

@keyframes q-generation-pulse {
  0%,
  100% {
    opacity: 0.22;
  }
  50% {
    opacity: 0.6;
  }
}

/* The "Generating" pill — glass overlay chip pinned top-left (Figma 21759:51482). */
@utility q-generation-card-status {
  position: absolute;
  inset-block-start: var(--hf-space-200);
  inset-inline-start: var(--hf-space-200);
  display: inline-flex;
  align-items: center;
  gap: var(--hf-space-200);
  padding: var(--hf-space-150) var(--hf-space-400) var(--hf-space-150) var(--hf-space-250);
  border-radius: var(--hf-radius-full);
  background-color: var(--hf-color-overlay-hover);
  backdrop-filter: blur(var(--hf-space-300));
}

@media (prefers-reduced-motion: reduce) {
  .q-generation-card-glow {
    animation: none;
    opacity: 0.7;
  }
}
```

app/src/components/generation-card/generation-card.tsx

```
"use client";

import type { ComponentProps, ReactElement, ReactNode, Ref } from "react";
import { useRender } from "@base-ui/react/use-render";
import { Loader } from "@higgsfield/quanta/loader";
import { Media } from "@higgsfield/quanta/media";
import { Typography } from "@higgsfield/quanta/typography";
import { cn as cx } from "@/lib/utils";

/**
 * GenerationCard — a single generation result tile for feed / history grids
 * (Figma Supercomputer-2 "Feed/Card" 2001:84301 + Cinema-Studio-V4 generating
 * state 20037:25838). It composes `Media` (a fixed-ratio, radius/300 clipped
 * box) into the two states a generation moves through:
 *
 *   • `ready` (default) — the finished asset: a cover `Media.Image` (or any
 *     custom `media` node — e.g. a `Media.Video`), with an optional bottom
 *     `title` over the media scrim.
 *   • `generating` — the in-progress placeholder: a dark canvas with a brand
 *     gradient glow pulsing at the TOP of the card and a "Generating" status
 *     pill (spinner + brand label). The pulse is a CSS animation that honors
 *     `prefers-reduced-motion`.
 *
 *   <GenerationCard src={cover} alt="Pool float" />
 *   <GenerationCard state="generating" ratio="portrait" />
 *
 * Tokens only, composition-first: `media` swaps the default image, `children`
 * compose extra overlays inside the frame, and the host element is swappable via
 * `render` (Base UI `useRender`) for clickable tiles.
 */

export type GenerationCardState = "ready" | "generating";

export type GenerationCardProps = Omit<ComponentProps<"div">, "title"> & {
  /** Lifecycle state — `ready` shows the asset, `generating` the pulsing placeholder. */
  state?: GenerationCardState;
  /** Image source for the default `Media.Image` (ready state). Ignored when `media` is set. */
  src?: string;
  /** Alt text for the default image. */
  alt?: string;
  /** Aspect ratio, forwarded to `Media` (default `video`). */
  ratio?: ComponentProps<typeof Media>["ratio"];
  /** Custom media node (a `Media.Video`, a fallback…) instead of the default image. */
  media?: ReactNode;
  /** Optional bottom title, over the media scrim. Ready state only. */
  title?: ReactNode;
  /** Status pill label shown while generating. Default `Generating`. */
  generatingLabel?: ReactNode;
  /** Extra overlay content composed inside the frame. */
  children?: ReactNode;
  /** Swap the host element — `<button>`/`<a>`/`<Link>` for clickable tiles. */
  render?: ReactElement;
  ref?: Ref<Element>;
};

/** The pulsing top glow + "Generating" status pill (Cinema-Studio-V4 20037:25838). */
function GeneratingOverlay({ label }: { label: ReactNode }) {
  return (
    <span className="q-generation-card-generating">
      <span className="q-generation-card-glow" aria-hidden="true" />
      <span className="q-generation-card-status">
        {/* Loader owns role="status" + aria-label; the label text is its visible echo. */}
        <Loader
          variant="circle"
          size="xs"
          color="brand"
          aria-label={typeof label === "string" ? label : "Generating"}
        />
```

```
        <Typography as="span" variant="body-sm-medium" color="brand" aria-hidden="true">
          {label}
        </Typography>
      </span>
    </span>
  );
}

function GenerationCard({
  state = "ready",
  src,
  alt = "",
  ratio = "video",
  media,
  title,
  generatingLabel = "Generating",
  className,
  children,
  render,
  ref,
  ...props
}: GenerationCardProps) {
  const generating = state === "generating";

  const content = (
    <>
      <Media ratio={ratio} rounded="md" className="q-generation-card-media">
        {generating
          ? (media ?? <Media.Fallback className="q-generation-card-canvas" />)
          : (media ?? (src != null ? <Media.Image src={src} alt={alt} /> : <Media.Fallback />))}
      </Media>
      {!generating && title != null ? (
        <Media.Overlay placement="bottom" className="q-generation-card-caption">
          <Typography
            as="span"
            variant="body-sm-semi-bold"
            color="primary"
            className="q-generation-card-title"
          >
            {title}
          </Typography>
        </Media.Overlay>
      ) : null}
      {generating ? <GeneratingOverlay label={generatingLabel} /> : null}
      {children}
    </>
  );

  return useRender({
    render,
    defaultTagName: "div",
    ref: ref as Ref<Element> | undefined,
    props: {
      className: cx("q-generation-card", className),
      "data-state": state,
      children: content,
      ...props,
    },
  });
}

export { GenerationCard };
```

app/src/components/generation-card/generation-tile.tsx

```
import type { CSSProperties, FocusEventHandler, MouseEventHandler, ReactNode } from "react";
import { GenerationDetailModal } from "@/components/generation-detail";
import type {
  GenerationDetail,
  GenerationDetailAction,
  GenerationDetailPrimaryAction,
  GenerationDetailRow,
} from "@/components/generation-detail";
import { downloadMedia } from "@/lib/download-media";
import { cn as cx } from "@/lib/utils";
import { GenerationCard } from "./generation-card.tsx";
import type { GenerationCardProps, GenerationCardState } from "./generation-card.tsx";
import { CardActions } from "./card-actions.tsx";
import type { CardAction } from "./card-actions.tsx";

/**
 * GenerationTile — THE single canonical way to render ONE generation anywhere in
 * an app (a feed tile, a wizard result, a hero result, a variation grid cell).
 *
 * It locks in the two lifecycle states AND the hover mechanism so they read the
 * SAME everywhere:
 *   • `generating` → the pulsing `GenerationCard` placeholder (identical always).
 *   • `ready`      → the finished asset with a full-bleed `GenerationDetailModal`
 *     open trigger + the hover `CardActions` rail (≤3 controls, the rest collapse
 *     into a ■ menu). Clicking the tile opens the shared lightbox; the rail sits
 *     above it as siblings, never nested buttons.
 *
 * The MECHANISM is fixed (states + open-on-click + the capped hover rail); the
 * SET of actions is chosen PER context (`actions`) and extra overlays — badges,
 * a selection checkbox — compose in via `children`. `GenerationCard` is the
 * low-level primitive this composes; screens use `GenerationTile`, not a
 * hand-rolled hover overlay.
 */
export interface GenerationTileProps {
  /** Lifecycle state — `generating` shows the placeholder, `ready` the asset. */
  state?: GenerationCardState;
  /**
   * Detail descriptor. When set (and `openable` isn't `false`) a full-bleed
   * trigger opens the shared `GenerationDetailModal` on click. `src` falls back
   * to this when omitted.
   */
  generation?: GenerationDetail;
  /** Default image (ready state). Ignored when `media` is set; falls back to `generation.src`. */
  src?: string;
  /** Alt text for the default image. */
  alt?: string;
  /** Custom media node (e.g. a hover-to-play video) instead of the default image. */
  media?: ReactNode;
  /** Aspect ratio, forwarded to the card. */
  ratio?: GenerationCardProps["ratio"];
  /** Optional bottom title over the media scrim (ready state). */
  title?: ReactNode;
  /** Status pill label while generating. */
  generatingLabel?: ReactNode;
  /**
   * Hover action rail (like / download / copy / share / delete …). `CardActions`
   * caps it at 3 visible; extras collapse into a ■ menu. The set is contextual,
   * the mechanism is always the same.
   */
  actions?: CardAction[];
  /** Extra overlays inside the frame (badges, a selection checkbox), above the open trigger. */
  children?: ReactNode;
  /** Open the detail lightbox on click. Defaults to `true` when `generation` is provided. */
  openable?: boolean;
  /**
   * Customises the shared lightbox's right panel — same fixed layout, different
   * content. Forwarded to `GenerationDetailModal`; omit for canonical defaults.
   */
```

```
  detail?: {
    rows?: GenerationDetailRow[];
    primaryAction?: GenerationDetailPrimaryAction | null;
    actions?: GenerationDetailAction[];
  };
  /** Accessible label for the open trigger. */
  openLabel?: string;
  className?: string;
  style?: CSSProperties;
  onMouseEnter?: MouseEventHandler<HTMLDivElement>;
  onMouseLeave?: MouseEventHandler<HTMLDivElement>;
  /** Forwarded to the open trigger (used e.g. to drive hover-to-play video on keyboard focus). */
  onFocus?: FocusEventHandler<HTMLButtonElement>;
  onBlur?: FocusEventHandler<HTMLButtonElement>;
}

export function GenerationTile({
  state = "ready",
  generation,
  src,
  alt = "",
  media,
  ratio = "auto",
  title,
  generatingLabel = "Generating",
  actions,
  children,
  openable,
  detail,
  openLabel,
  className,
  style,
  onMouseEnter,
  onMouseLeave,
  onFocus,
  onBlur,
}: GenerationTileProps) {
  if (state === "generating") {
    return (
      <GenerationCard
        state="generating"
        ratio={ratio}
        generatingLabel={generatingLabel}
        className={className}
        style={style}
      />
    );
  }

  const canOpen = openable !== false && generation != null;
  const downloadUrl = generation?.src ?? src;
  const resolvedActions = actions?.map((action) =>
    action.id === "download" && action.onSelect == null && downloadUrl
      ? { ...action, onSelect: () => void downloadMedia(downloadUrl) }
      : action,
  );

  return (
    <GenerationCard
      ratio={ratio}
      src={media == null ? (src ?? generation?.src) : undefined}
      alt={alt}
      title={title}
      media={media}
      className={cx("group", className)}
      style={style}
      onMouseEnter={onMouseEnter}
      onMouseLeave={onMouseLeave}
    >
      {canOpen ? (
```

```
        <GenerationDetailModal
          generation={generation}
          detailRows={detail?.rows}
          primaryAction={detail?.primaryAction}
          actions={detail?.actions}
          trigger={
            <button
              type="button"
              aria-label={
                openLabel ??
                (generation?.prompt != null
                  ? `Open generation: ${generation.prompt}`
                  : "Open generation")
              }
              className="absolute inset-0 z-[1] cursor-pointer rounded-q-300 focus-visible:outline-2 focus-visible:-outline-offset-2 focus-visible:outline-q-border-focus"
              onFocus={onFocus}
              onBlur={onBlur}
            />
          }
        />
      ) : null}
      {children}
      {resolvedActions != null && resolvedActions.length > 0 ? (
        <CardActions actions={resolvedActions} />
      ) : null}
    </GenerationCard>
  );
}
```

app/src/components/generation-card/index.ts

```
export { GenerationCard } from "./generation-card.tsx";
export type { GenerationCardProps, GenerationCardState } from "./generation-card.tsx";
export { GenerationTile } from "./generation-tile.tsx";
export type { GenerationTileProps } from "./generation-tile.tsx";
export { CardActions } from "./card-actions.tsx";
export type { CardAction, CardActionsProps } from "./card-actions.tsx";
```

app/src/components/generation-detail.tsx

```
import type { ReactElement, ReactNode } from "react";
import { useState } from "react";
import { Dialog } from "@base-ui/react/dialog";
import { Download as IconDownloadOutlined } from "lucide-react";
import { Share2 as IconArrowShareRightOutlined } from "lucide-react";
import { ChevronUp as IconChevronTopMediumOutlined } from "lucide-react";
import { Info as IconCircleInfoOutlined } from "lucide-react";
import { Cloud as IconCloudFilled } from "lucide-react";
import { X as IconCrossMediumOutlined } from "lucide-react";
import { Ellipsis as IconDotGrid1x3HorizontalOutlined } from "lucide-react";
import { Heart as IconHeartOutlined } from "lucide-react";
import { Video as IconVideoOutlined } from "lucide-react";
import { Avatar } from "@higgsfield/quanta/avatar";
import { Button } from "@higgsfield/quanta/button";
import { glass } from "@higgsfield/quanta/glass";
import { Icon, type IconGlyph } from "@higgsfield/quanta/icon";
import { Media } from "@higgsfield/quanta/media";
import { Typography } from "@higgsfield/quanta/typography";
import { downloadMedia } from "@/lib/download-media";

/**
 * GenerationDetailModal — the full-screen "Viewer / Image" lightbox that opens
 * when a generation card is clicked (Figma SC App Builder, node 3019:100235).
 *
 * ■■ Anatomy ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * A full-viewport overlay (NOT a centered card) with three stacked layers:
 *   1. Backdrop — the generation itself, cover-filled, darkened by a scrim and
 *      washed by a heavy backdrop blur so the media reads as frosted glass behind
 *      everything. This is the "lightbox" effect.
 *   2. Stage — the crisp, contained media (image or video) centred in the left
 *      region.
 *   3. Info panel — a frosted glass card pinned to the right holding: an author
 *      row (avatar + name + Share + Close), a collapsible "Details" block
 *      (status / type / size / uploaded / last used + prompt), and a sticky
 *      action footer ("Turn to video" CTA + Download / Like / Share / More).
 *
 * ■■ Why Base UI Dialog directly (not Quanta `Modal`) ■■■■■■■■■■■■■■■■■■■■■■■■■
 * Quanta's `Modal` paints a centred, width-capped glass CARD (`q-modal`: fixed
 * 50/50 translate, `width: min(...)`, own backdrop-blur, 24px radius). A
 * full-bleed lightbox with an image backdrop + a right-docked panel is a
 * different surface entirely — reusing `Modal.Content` would mean overriding
 * nearly every one of those utilities. So we compose Base UI's `Dialog`
 * primitive (the same one `Modal` wraps — focus trap, scroll lock, escape,
 * a11y, portal, exit-mount timing) directly and skin it with Quanta tokens +
 * Quanta content components (`Media`, `Avatar`, `Button`, `Typography`, `Icon`,
 * and the `glass()` recipe).
 *
 * ■■ API ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * Mirrors `AssetLibraryModal({ trigger })`: pass the generation card as
 * `trigger` and (optionally) the `generation` data. Another agent can wire this
 * to the History grid by rendering a card element as the trigger:
 *
 *   <GenerationDetailModal
 *     trigger={<GenerationCard … />}
 *     generation={{ src, mediaType: 'image', author: { name }, prompt, … }}
 *   />
 *
 * ■■ Fixed layout, configurable content ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * This is THE canonical viewer — every finished generation opens through it,
 * and the two-column layout (media stage + right info panel with author row →
 * Details block → Prompt → action footer) never changes. What each surface may
 * customise is the info panel's CONTENT, without touching the layout:
 *   • `detailRows` — replace / add / remove / reorder the Details rows.
 *   • `primaryAction` — swap or hide (`null`) the full-width lime CTA.
 *   • `actions` — swap / add / remove the footer action buttons (`[]` hides).
 * Omit any of these to keep the canonical defaults.
 */
```

```
export interface GenerationDetail {
  /** Media source (image or video). Reuse a local `/presets/*` asset. */
  src: string;
  /** Renders a `<video>` when `'video'`, otherwise an `<img>`. Default `'image'`. */
  mediaType?: "image" | "video";
  /** Poster shown before a video plays. */
  poster?: string;
  /** Aspect ratio (width / height) of the crisp preview. Default `2 / 3` (portrait). */
  aspectRatio?: number;
  /** Author shown in the panel header. */
  author?: { name: string; role?: string; avatarSrc?: string };
  /** Storage status label (paired with a cloud glyph). */
  status?: string;
  /** File type, e.g. `JPG` / `MP4`. */
  fileType?: string;
  /** Human file size, e.g. `2.4 MB`. */
  size?: string;
  /** When the asset was created / uploaded. */
  uploadedAt?: string;
  /** When the asset was last used. */
  lastUsedAt?: string;
  /** The generation prompt. */
  prompt?: string;
}

/**
 * A single "label ■ value" row in the panel's Details block.
 *
 * The LAYOUT of the block is fixed; only its rows are configurable. Pass
 * `detailRows` to add / remove / reorder rows — omit it to keep the canonical
 * Status / Type / Size / Uploaded / Last used set.
 */
export interface GenerationDetailRow {
  id: string;
  label: string;
  value: ReactNode;
}

/**
 * A footer action button. The FIRST slot renders full-width (the lime CTA is
 * `primaryAction`); each entry here sits in the action row below it. Labeled
 * entries stretch (`flex-1`); `iconOnly` entries stay square.
 *
 * The row's LAYOUT is fixed; only which buttons appear is configurable.
 */
export interface GenerationDetailAction {
  id: string;
  label: string;
  icon: IconGlyph;
  onSelect?: () => void;
  /** Render as a square icon-only button (no label). Default `false`. */
  iconOnly?: boolean;
}

/** The full-width lime CTA above the action row (e.g. "Turn to video"). */
export interface GenerationDetailPrimaryAction {
  label: ReactNode;
  icon?: IconGlyph;
  onSelect?: () => void;
}

export interface GenerationDetailModalProps {
  /** The trigger element (e.g. a generation card). Rendered as the dialog trigger. */
  trigger: ReactElement;
  /** Data shown in the viewer. Falls back to a demo generation when omitted. */
  generation?: GenerationDetail;
  /**
   * Replaces the canonical Details rows (Status / Type / Size / Uploaded /
   * Last used). Same fixed layout — only the rows change. Omit for defaults.
   */
```

```
  detailRows?: GenerationDetailRow[];
  /**
   * The full-width lime CTA. Omit for the default "Turn to video"; pass `null`
   * to remove it entirely.
   */
  primaryAction?: GenerationDetailPrimaryAction | null;
  /**
   * The footer action buttons below the CTA. Omit for the default
   * Download / Like / Share / More set; pass `[]` to remove the row.
   */
  actions?: GenerationDetailAction[];
  /** Controlled open state (optional — the dialog self-manages otherwise). */
  open?: boolean;
  /** Open-state change callback (optional). */
  onOpenChange?: (open: boolean) => void;
  /** Start opened (uncontrolled). Handy for previews. */
  defaultOpen?: boolean;
}

// PLACEHOLDER ASSETS — template demo art (see /presets/*.png). When adapting
// this template into a real app, REPLACE media that represents the product
// (hero/example outputs, covers, before/after samples, feed items) with
// bespoke on-brand assets generated via the Higgsfield generation tools.
// Pure style-picker label thumbnails may keep simple placeholder art when
// real output depends on the user's own upload. Grep "PLACEHOLDER ASSETS"
// to find every site.
const DEMO_GENERATION: Required<Omit<GenerationDetail, "poster">> = {
  src: "/presets/how-product-works.png",
  mediaType: "image",
  aspectRatio: 2 / 3,
  author: { name: "retro_strawberry", role: "Author" },
  status: "Uploaded",
  fileType: "JPG",
  size: "2.4 MB",
  uploadedAt: "12.05.2026, 01:22",
  lastUsedAt: "12.05.2026, 16:43",
  prompt:
    "A model in a translucent floral raincoat standing beside pale horses in a windswept meadow, editorial fashion photography, soft daylight.",
};

/** The canonical lime CTA — used when `primaryAction` is omitted. */
const DEFAULT_PRIMARY_ACTION: GenerationDetailPrimaryAction = {
  label: "Turn to video",
  icon: IconVideoOutlined,
};

/** The canonical footer action row — used when `actions` is omitted. */
const DEFAULT_ACTIONS: GenerationDetailAction[] = [
  { id: "download", label: "Download", icon: IconDownloadOutlined },
  { id: "like", label: "Like", icon: IconHeartOutlined, iconOnly: true },
  { id: "share", label: "Share", icon: IconArrowShareRightOutlined, iconOnly: true },
  { id: "more", label: "More", icon: IconDotGrid1x3HorizontalOutlined, iconOnly: true },
];

/** A single "label ■ value" detail row. `value` may embed an icon/node. */
function DetailRow({ label, value }: { label: string; value: ReactNode }) {
  return (
    <div className="flex items-center gap-2">
      <Typography
        as="span"
        variant="body-sm-regular"
        className="shrink-0 text-q-transparent-light-50"
      >
        {label}
      </Typography>
      <div className="flex min-w-0 flex-1 items-center justify-end gap-1.5">
        {typeof value === "string" ? (
          <Typography
            as="span"
            variant="body-sm-regular"
```

```
            color="primary"
            truncate
            className="text-right"
          >
            {value}
          </Typography>
        ) : (
          value
        )}
      </div>
    </div>
  );
}

function InfoPanel({
  generation,
  detailRows,
  primaryAction,
  actions,
}: {
  generation: GenerationDetail;
  detailRows?: GenerationDetailRow[];
  primaryAction?: GenerationDetailPrimaryAction | null;
  actions?: GenerationDetailAction[];
}) {
  const [detailsOpen, setDetailsOpen] = useState(true);
  const data = {
    ...DEMO_GENERATION,
    ...generation,
    author: { ...DEMO_GENERATION.author, ...generation.author },
  };

  // Fixed layout, configurable content: rows/CTA/actions default to the
  // canonical set but callers may add, remove, or reorder them.
  const rows: GenerationDetailRow[] = detailRows ?? [
    {
      id: "status",
      label: "Status",
      value: (
        <>
          <Icon as={IconCloudFilled} size="sm" color="secondary" />
          <Typography as="span" variant="body-sm-regular" color="primary" truncate>
            {data.status}
          </Typography>
        </>
      ),
    },
    { id: "type", label: "Type", value: data.fileType },
    { id: "size", label: "Size", value: data.size },
    { id: "uploaded", label: "Uploaded", value: data.uploadedAt },
    { id: "lastUsed", label: "Last used", value: data.lastUsedAt },
  ];
  const primary = primaryAction === undefined ? DEFAULT_PRIMARY_ACTION : primaryAction;
  const footerActions = (actions ?? DEFAULT_ACTIONS).map((action) =>
    action.id === "download" && action.onSelect == null
      ? { ...action, onSelect: () => void downloadMedia(data.src) }
      : action,
  );

  return (
    <aside className="flex h-full w-full flex-col gap-2 p-2">
      {/* Author row */}
      <div className="flex items-center gap-2 p-1">
        <Avatar size="sm" src={data.author?.avatarSrc} alt={data.author?.name} color="mint" />
        <div className="flex min-w-0 flex-1 flex-col">
          <Typography as="span" variant="body-sm-medium" color="primary" truncate>
            {data.author?.name}
          </Typography>
          <Typography as="span" variant="caption-xs-regular" color="secondary" truncate>
            {data.author?.role ?? "Author"}
```

```
    </Typography>
  </div>
  <Dialog.Close
    aria-label="Close"
    className="flex size-8 shrink-0 items-center justify-center rounded-q-full bg-q-transparent-light-05 text-q-icon-primary transition-colors hover:bg-q-transparent-light-10"
  >
    <Icon as={IconCrossMediumOutlined} size="md" />
  </Dialog.Close>
</div>

{/* Details */}
<div className="flex flex-col gap-2 rounded-q-300 bg-q-transparent-light-05 p-2">
  <button
    type="button"
    onClick={() => setDetailsOpen((o) => !o)}
    className="flex items-center gap-2 px-1 py-1.5"
    aria-expanded={detailsOpen}
  >
    <Icon as={IconCircleInfoOutlined} size="sm" color="secondary" />
    <Typography
      as="span"
      variant="label-xs-medium"
      color="secondary"
      className="flex-1 text-left uppercase"
    >
      Details
    </Typography>
    <Icon
      as={IconChevronTopMediumOutlined}
      size="sm"
      color="secondary"
      className={detailsOpen ? undefined : "rotate-180"}
    />
  </button>

  {detailsOpen ? (
    <>
      {rows.length > 0 ? (
        <div className="flex flex-col gap-1 rounded-q-200 bg-q-transparent-light-05 p-3">
          {rows.map((row) => (
            <DetailRow key={row.id} label={row.label} value={row.value} />
          ))}
        </div>
      ) : null}

      {data.prompt != null && data.prompt !== "" ? (
        <div className="flex flex-col gap-1 rounded-q-200 bg-q-transparent-light-05 p-3">
          <Typography
            as="span"
            variant="body-sm-regular"
            className="text-q-transparent-light-50"
          >
            Prompt
          </Typography>
          <Typography as="p" variant="body-sm-regular" color="primary">
            {data.prompt}
          </Typography>
        </div>
      ) : null}
    </>
  ) : null}
</div>

<span aria-hidden className="flex-1" />

{/* Actions — fixed layout (full-width CTA + a row of buttons), configurable content. */}
{primary != null || footerActions.length > 0 ? (
  <div className="flex flex-col gap-2 p-2">
    {primary != null ? (
      <Button
```

```
              variant="marketingPrimary"
              size="sm"
              className="w-full"
              onClick={primary.onSelect}
              start={primary.icon != null ? <Icon as={primary.icon} size="sm" /> : undefined}
            >
              {primary.label}
            </Button>
          ) : null}
          {footerActions.length > 0 ? (
            <div className="flex items-center gap-2">
              {footerActions.map((action) =>
                action.iconOnly ? (
                  <Button
                    key={action.id}
                    variant="marketingTertiary"
                    size="sm"
                    iconOnly
                    aria-label={action.label}
                    onClick={action.onSelect}
                    start={<Icon as={action.icon} size="sm" />}
                  />
                ) : (
                  <Button
                    key={action.id}
                    variant="marketingTertiary"
                    size="sm"
                    className="flex-1"
                    onClick={action.onSelect}
                    start={<Icon as={action.icon} size="sm" />}
                  >
                    {action.label}
                  </Button>
                ),
              )}
            </div>
          ) : null}
        </div>
      ) : null}
    </aside>
  );
}

export function GenerationDetailModal({
  trigger,
  generation,
  detailRows,
  primaryAction,
  actions,
  open,
  onOpenChange,
  defaultOpen,
}: GenerationDetailModalProps) {
  const data = { ...DEMO_GENERATION, ...generation };

  // The stage frame takes the item's OWN aspect ratio and the media fills it
  // with `cover` — so the frame equals the image ratio and there are no
  // letterbox bars. The ratio-correct box is capped to the stage column by the
  // limiting axis: landscape/square is width-driven (q-media's default 100%
  // width), portrait is height-driven — either way max-h/max-w keep it inside
  // the column so it never overflows or slides under the info panel.
  const stageRatio = data.aspectRatio ?? 2 / 3;
  const stageFrameClass =
    stageRatio >= 1
      ? "max-h-full max-w-full shadow-q-overlay"
      : "h-full max-h-full w-auto! max-w-full shadow-q-overlay";

  return (
    <Dialog.Root open={open} onOpenChange={onOpenChange} defaultOpen={defaultOpen}>
      <Dialog.Trigger render={trigger} />
```

```
      <Dialog.Portal>
        <Dialog.Backdrop className="q-modal-backdrop" />
        <Dialog.Popup
          aria-label="Generation preview"
          className="fixed inset-0 z-q-modal flex outline-none transition-opacity duration-200 ease-out data-[ending-style]:opacity-0 data-[starting-style]:opacity-0"
        >
          {/* Layer 1 — frosted media backdrop. A darker scrim (dark-80) + a heavy
           * backdrop blur so everything behind the crisp stage reads as a dark,
           * blurred frost. */}
          <div
            aria-hidden
            className="pointer-events-none absolute inset-0 overflow-hidden bg-q-background-primary"
          >
            <img src={data.src} alt="" className="absolute inset-0 size-full object-cover" />
            <div className="absolute inset-0 bg-q-transparent-dark-80" />
            <div className="absolute inset-0 backdrop-blur-3xl" />
          </div>

          {/* Layer 2 — crisp stage. min-h-0/min-w-0 + overflow-hidden keep the
           * media inside this flex column so it never bleeds under the panel;
           * the frame carries the item's aspect ratio and the media covers it
           * (no letterbox bars), capped to the column by max-h-full/max-w-full. */}
          <div className="relative flex min-h-0 min-w-0 flex-1 items-center justify-center overflow-hidden p-6">
            {data.mediaType === "video" ? (
              <Media ratio={stageRatio} rounded="md" className={stageFrameClass}>
                <Media.Video src={data.src} poster={data.poster} autoPlayInView loop fit="cover" />
              </Media>
            ) : (
              <Media ratio={stageRatio} rounded="md" className={stageFrameClass}>
                <Media.Image src={data.src} alt="" fit="cover" />
              </Media>
            )}
          </div>

          {/* Layer 3 — info panel */}
          <div className="relative flex w-[366px] shrink-0 p-2">
            <div
              className={glass(
                { blur: "md", rounded: "500" },
                "flex min-h-0 flex-1 flex-col overflow-y-auto",
              )}
            >
              <InfoPanel
                generation={generation ?? DEMO_GENERATION}
                detailRows={detailRows}
                primaryAction={primaryAction}
                actions={actions}
              />
            </div>
          </div>
        </Dialog.Popup>
      </Dialog.Portal>
    </Dialog.Root>
  );
}

/**
 * Standalone demo — renders its own trigger button so the viewer can be
 * previewed without touching shared templates. Import into `main.tsx`
 * temporarily, or drop anywhere for a visual check.
 */
export function GenerationDetailDemo() {
  return (
    <div className="flex min-h-screen items-center justify-center bg-q-background-primary p-8">
      <GenerationDetailModal
        trigger={
          <Button variant="primary" size="md">
            Open generation
          </Button>
        }
```

```
      />
    </div>
  );
}

export default GenerationDetailDemo;
```

app/src/components/history-grid.tsx

```
import { UserGenerations } from "@/components/user-generations";
import type { UserGenerationsProps } from "@/components/user-generations";

/**
 * @deprecated Use `UserGenerations` from `@/components/user-generations` — it is
 * THE single canonical component for a browsable feed of the current user's
 * generations. `HistoryGrid` is kept only as a thin back-compat alias so older
 * call sites keep working; new code (and every History tab/page) renders
 * `<UserGenerations />` directly.
 */
export function HistoryGrid(props: UserGenerationsProps) {
  return <UserGenerations {...props} />;
}
```

app/src/components/icon-tile/icon-tile.css

```
/**
 * IconTile presentation — the 24px leading tile (Figma "Share Modal" leading
 * tile 2125:15286 + neutral "New folder" tile 2125:15346). Shared footprint:
 * space/600 box, radius/200 corners. The `gradient` look layers two blended
 * white sheens over an inline-style brand gradient (the base fill is set via
 * `style` since the stops are bespoke); the `neutral` look is the white-5%
 * raised box the sidebar's project thumbnail uses.
 *
 * The gradient border / drop+inset sheen / blended overlay+hard-light sheens
 * are the one necessary literal — branded fills with no exact Quanta token.
 */
@source "./icon-tile.tsx";

@utility q-icon-tile {
  position: relative;
  display: inline-flex;
  flex: none;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-600);
  height: var(--hf-space-600);
  overflow: hidden;
  border-radius: var(--hf-radius-200);
}

/* Neutral raised box (matches the sidebar project-thumbnail surface). */
@utility q-icon-tile-neutral {
  background-color: var(--hf-color-transparent-light-05);
  box-shadow: var(--hf-shadow-raised-sm);
}

/* Colored brand tile: white glyph, hairline light border, drop + inset
 * highlight, and two blended white sheens (::before overlay, ::after
 * hard-light) over the inline-style gradient base. Values are bespoke branded
 * literals — no exact Quanta token for these fills. */
@utility q-icon-tile-gradient {
  border: var(--hf-border-width-thin) solid rgba(197, 197, 197, 0.24);
  color: #fff;
  box-shadow:
    0 4px 2px rgba(0, 0, 0, 0.08),
    inset 0 2px 4px rgba(255, 255, 255, 0.24);

  &::before,
  &::after {
    content: "";
    position: absolute;
    inset: 0;
    pointer-events: none;
  }

  &::before {
    background-image: linear-gradient(to top, transparent, rgba(255, 255, 255, 0.2));
    mix-blend-mode: overlay;
  }

  &::after {
    background-image: linear-gradient(to top, transparent, rgba(255, 255, 255, 0.32));
    mix-blend-mode: hard-light;
  }
}

/* Lift the glyph above the gradient sheens. */
@utility q-icon-tile-glyph {
  position: relative;
  z-index: 1;
}
```

app/src/components/icon-tile/icon-tile.tsx

```
"use client";

import type { ComponentPropsWithRef } from "react";
import type { Icon as PhosphorIcon } from "@phosphor-icons/react";
import { icon } from "@higgsfield/quanta/icon";
import { cn as cx } from "@/lib/utils";

export type IconTileGradient =
  | "blue"
  | "teal"
  | "purple"
  | "pink"
  | "orange"
  | "green"
  | "red"
  | "indigo";

export const ICON_TILE_GRADIENT: Record<IconTileGradient, string> = {
  blue: "linear-gradient(135deg, rgb(65, 136, 190) 0%, rgb(14, 39, 114) 100%)",
  teal: "linear-gradient(135deg, rgb(81, 226, 224) 3.8675%, rgb(18, 92, 141) 93.451%)",
  purple: "linear-gradient(135deg, rgb(158, 120, 226) 0%, rgb(63, 26, 130) 100%)",
  pink: "linear-gradient(135deg, rgb(226, 110, 178) 0%, rgb(130, 20, 74) 100%)",
  orange: "linear-gradient(135deg, rgb(245, 168, 88) 0%, rgb(168, 66, 18) 100%)",
  green: "linear-gradient(135deg, rgb(104, 205, 128) 0%, rgb(20, 96, 58) 100%)",
  red: "linear-gradient(135deg, rgb(235, 108, 104) 0%, rgb(140, 22, 34) 100%)",
  indigo: "linear-gradient(135deg, rgb(110, 128, 226) 0%, rgb(30, 34, 130) 100%)",
};

export type IconTileProps = ComponentPropsWithRef<"span"> & {
  /** Phosphor glyph; gradient tiles always use its real filled artwork. */
  as: PhosphorIcon;
  gradient?: IconTileGradient | (string & {});
};

export function IconTile({ as, gradient, className, style, ...props }: IconTileProps) {
  const Glyph = as;
  const isGradient = gradient != null;
  const backgroundImage = isGradient
    ? (ICON_TILE_GRADIENT[gradient as IconTileGradient] ?? gradient)
    : undefined;

  return (
    <span
      className={cx(
        "q-icon-tile",
        isGradient ? "q-icon-tile-gradient" : "q-icon-tile-neutral",
        className,
      )}
      style={isGradient ? { backgroundImage, ...style } : style}
      {...props}
    >
      <Glyph
        weight={isGradient ? "fill" : "regular"}
        aria-hidden
        className={cx(
          icon({ size: "sm", color: isGradient ? undefined : "secondary" }),
          "q-icon-tile-glyph",
        )}
      />
    </span>
  );
}
```

app/src/components/icon-tile/index.ts

```
export { ICON_TILE_GRADIENT, IconTile } from "./icon-tile.tsx";
export type { IconTileGradient, IconTileProps } from "./icon-tile.tsx";
```

app/src/components/media-card/index.ts

```
export { MediaCard } from "./media-card.tsx";
export type {
  MediaCardActionProps,
  MediaCardFrame,
  MediaCardProps,
  MediaCardTitleVariant,
} from "./media-card.tsx";
```

app/src/components/media-card/media-card.css

```
/**
 * MediaCard presentation — the framed gallery/cover media tile (Figma preset
 * tiles 2950:66661…66695 + panel cover 2950:66568): radius/400 clipped frame
 * with a thin (cover) or thick (gallery) default border, the bottom title row
 * (scrimmed for tiles, bare for covers), the top-right action slot, and the
 * on-media dark-glass Action chip (2950:66569).
 *
 * Interactivity is CSS-only: a `button`/`a` host (via `render`) gets cursor,
 * a strong hover border, and the shared lime focus ring — the passive div
 * host stays inert. Imported AFTER media.css (components.css order) so the
 * caption paddings here win over `q-media-overlay-bottom`.
 */
@source "./media-card.tsx";

@utility q-media-card {
  position: relative;
  display: flex;
  flex-direction: column;
  width: 100%;
  min-width: var(--hf-space-0);
  padding: var(--hf-space-0);
  overflow: hidden;
  border-radius: var(--hf-radius-400);
  background-color: transparent;
  text-align: start;
  transition-property: border-color, box-shadow;

  /* Interactive hosts only (render={<button/>} / <a>) — cursor, hover, ring. */
  &:is(button, a) {
    cursor: pointer;
    outline: none;
    @apply q-focus-ring;

    &:hover {
      border-color: var(--hf-color-border-strong);
    }
  }
}

@utility q-media-card-frame-thin {
  border: var(--hf-border-width-thin) solid var(--hf-color-border-default);
}

@utility q-media-card-frame-thick {
  border: var(--hf-border-width-thick) solid var(--hf-color-border-default);
}

/* The composed Media box fills the frame; an explicit card height (e.g. a
 * fixed cover) stretches it via the column flex. */
@utility q-media-card-media {
  flex: 1 1 auto;
  min-height: var(--hf-space-0);
}

/* Bottom title row — overrides the q-media-overlay-bottom paddings (Figma:
 * tiles px-16/py-12, covers p-16). */
@utility q-media-card-caption {
  padding: var(--hf-space-300) var(--hf-space-400);
}

@utility q-media-card-caption-accent {
  padding: var(--hf-space-400);
  text-transform: uppercase;
  /* Smooth darkening that hugs the left, right, and bottom edges of the cover
   * (fading toward the top-center) so the accent title stays legible without a
   * hard band. Overrides q-media-overlay-bottom's bottom-only scrim. */
  background-image:
    linear-gradient(to top, var(--hf-color-transparent-dark-50), transparent),
```

```
    linear-gradient(to right, var(--hf-color-transparent-dark-50), transparent 50%),
    linear-gradient(to left, var(--hf-color-transparent-dark-50), transparent 50%);
}

/* Covers drop the gradient scrim (the artwork carries the contrast). */
@utility q-media-card-caption-bare {
  background-image: none;
}

@utility q-media-card-title {
  display: block;
  min-width: var(--hf-space-0);
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

/* Top-right slot for the action chip (Figma cover inset: 16px). */
@utility q-media-card-action-slot {
  position: absolute;
  inset-block-start: var(--hf-space-400);
  inset-inline-end: var(--hf-space-400);
  display: inline-flex;
}

/* The on-media dark-glass chip (2950:66569): 28px pill — black-30 glass,
 * blur 12, hairline default border, top sheen + soft drop, caption/sm text. */
@utility q-media-card-action {
  display: inline-flex;
  align-items: center;
  gap: var(--hf-space-150);
  height: var(--hf-space-700);
  padding: var(--hf-space-150) var(--hf-space-250);
  border: var(--hf-border-width-thin) solid var(--hf-color-border-default);
  border-radius: var(--hf-radius-250);
  background-color: var(--hf-color-transparent-dark-30);
  backdrop-filter: blur(var(--hf-space-300));
  color: var(--hf-color-text-primary);
  box-shadow:
    var(--hf-shadow-sheen),
    0 var(--hf-space-050) var(--hf-space-100) 0 var(--hf-color-transparent-dark-12);
  cursor: pointer;
  outline: none;
  white-space: nowrap;
  transition-property: background-color, background-image, border-color, box-shadow;
  @apply text-q-caption-sm-semi-bold;
  @apply q-focus-ring;

  &:hover {
    background-image: linear-gradient(
      var(--hf-color-transparent-light-20),
      var(--hf-color-transparent-light-20)
    );
  }

  &:disabled {
    pointer-events: none;
    opacity: var(--hf-opacity-disabled);
  }

  & svg {
    flex-shrink: 0;
    width: var(--hf-icon-sm);
    height: var(--hf-icon-sm);
  }
}
```

app/src/components/media-card/media-card.tsx

```
"use client";

import type { ComponentPropsWithRef, ReactElement, ReactNode, Ref } from "react";
import { useRender } from "@base-ui/react/use-render";
import { Media } from "@higgsfield/quanta/media";
import { Typography } from "@higgsfield/quanta/typography";
import { cn as cx } from "@/lib/utils";

/**
 * MediaCard — a framed media tile for preset galleries and cover pickers
 * (Figma SC App Builder: preset tiles 2950:66661…66695, panel cover
 * 2950:66568). It composes `Media` (the fixed-ratio media box) into the
 * gallery card anatomy: a token frame (thin/thick border or `none` for a
 * frameless tile, radius/400, clipped corners), a bottom title — over the dark
 * scrim for gallery tiles, bare for covers — and an optional top-right `action`
 * slot. Pass `selected` to ring the active tile in the lime brand color (single
 * -select galleries).
 *
 *   <MediaCard render={<button type="button" />} src={preset.src} alt={preset.title}
 *     title={preset.title} onClick={pick} />
 *
 *   <MediaCard ratio="auto" frame="thin" scrim={false} titleVariant="accent"
 *     className="h-40" src={cover} title="How product works"
 *     action={<MediaCard.Action>Change<PencilIcon /></MediaCard.Action>} />
 *
 * The host element is swappable via `render` (Base UI `useRender`): keep the
 * default `<div>` for a passive cover, render a `<button>`/`<a>`/`<Link>` for
 * clickable tiles — interactive hosts get cursor/hover/focus styling from CSS
 * alone. Do NOT combine an interactive host with `action` (nested interactive
 * elements); the action slot is for passive (div) cards.
 *
 * `media` replaces the default `<Media.Image>` with any node (e.g.
 * `<Media.Video autoPlayInView>`); `children` compose extra overlays inside
 * the frame (badges, progress) — both keep the card fully composition-first.
 */

export type MediaCardFrame = "none" | "thin" | "thick";
export type MediaCardTitleVariant = "body" | "accent";

const FRAME_CLASS = {
  none: "",
  thin: "q-media-card-frame-thin",
  thick: "q-media-card-frame-thick",
} satisfies Record<MediaCardFrame, string>;

/** Caption modifier per title variant — `accent` covers get p-16 + uppercase. */
const TITLE_VARIANT_CLASS = {
  body: "",
  accent: "q-media-card-caption-accent",
} satisfies Record<MediaCardTitleVariant, string>;

export type MediaCardProps = Omit<ComponentPropsWithRef<"div">, "title"> & {
  /** Image source for the default `Media.Image`. Ignored when `media` is set. */
  src?: string;
  /** Alt text for the default image. */
  alt?: string;
  /** Aspect ratio, forwarded to `Media` (default `video` — the gallery tile). */
  ratio?: ComponentPropsWithRef<typeof Media>["ratio"];
  /** Custom media node (a `Media.Video`, a fallback…) instead of the image. */
  media?: ReactNode;
  /** Bottom title. Any node. */
  title?: ReactNode;
  /** Title style: gallery `body` (default) or uppercase `accent` cover. */
  titleVariant?: MediaCardTitleVariant;
  /** Dark gradient behind the title (default true; covers pass false). */
  scrim?: boolean;
  /** Border weight: `thick` gallery tile (default), `thin` cover, or `none` (frameless). */
  frame?: MediaCardFrame;
```

```
  /**
   * Mark the tile as the active choice — draws the lime brand ring
   * (`ring-q-brand-primary`). Pair with `aria-pressed` on a `<button>` host for
   * an accessible single-select gallery. Best with `frame="none"`.
   */
  selected?: boolean;
  /** Top-right slot — e.g. `MediaCard.Action`. Only on passive (div) cards. */
  action?: ReactNode;
  /** Extra overlay content composed inside the frame. */
  children?: ReactNode;
  /** Swap the host element — `<button>`/`<a>`/`<Link>` for clickable tiles. */
  render?: ReactElement;
};

function Root({
  src,
  alt = "",
  ratio = "video",
  media,
  title,
  titleVariant = "body",
  scrim = true,
  frame = "thick",
  selected = false,
  action,
  children,
  className,
  render,
  ref,
  ...props
}: MediaCardProps) {
  const content = (
    <>
      <Media ratio={ratio} rounded="none" className="q-media-card-media">
        {media ?? (src != null ? <Media.Image src={src} alt={alt} /> : <Media.Fallback />)}
      </Media>
      {title != null ? (
        <Media.Overlay
          placement="bottom"
          className={cx(
            "q-media-card-caption",
            !scrim && "q-media-card-caption-bare",
            TITLE_VARIANT_CLASS[titleVariant],
          )}
        >
          <Typography
            as="span"
            variant={titleVariant === "accent" ? "accent-xs-bold" : "body-lg-semi-bold"}
            color="primary"
            className="q-media-card-title"
          >
            {title}
          </Typography>
        </Media.Overlay>
      ) : null}
      {action != null ? <span className="q-media-card-action-slot">{action}</span> : null}
      {children}
    </>
  );

  return useRender({
    render,
    defaultTagName: "div",
    ref: ref as Ref<Element> | undefined,
    props: {
      className: cx(
        "q-media-card",
        FRAME_CLASS[frame],
        selected && "ring-2 ring-q-brand-primary",
        className,
```

```
      ),
      children: content,
      ...props,
    },
  });
}

export type MediaCardActionProps = ComponentPropsWithRef<"button">;

/**
 * The on-media glass chip (Figma "secondary - default - xs - r_sm",
 * 2950:66569): a 28px dark-glass pill pinned over imagery — "Change", "Edit".
 * A bespoke control: no Button variant covers the on-media dark glass look
 * (flagged in the variant registry). Compose the label with a trailing 16px
 * icon as children.
 */
function Action({ className, type, ...props }: MediaCardActionProps) {
  return (
    <button type={type ?? "button"} className={cx("q-media-card-action", className)} {...props} />
  );
}

export const MediaCard = Object.assign(Root, { Action });
```

app/src/components/prompt-box/index.ts

```
export { PromptBox } from "./prompt-box.tsx";
export type {
  PromptBoxBodyProps,
  PromptBoxFieldProps,
  PromptBoxGenerateProps,
  PromptBoxModeProps,
  PromptBoxModeRailProps,
  PromptBoxPillProps,
  PromptBoxRootProps,
  PromptBoxSurface,
  PromptBoxUploadProps,
  PromptBoxUploadsProps,
} from "./prompt-box.tsx";
```

app/src/components/prompt-box/prompt-box.css

```
/**
 * PromptBox presentation — the Studio prompt dock (Figma Marketing-Studio node
 * 7259:51362; generation-state dock 21768:60842). A horizontal bar composed of a
 * vertical mode rail plus one shared right-side shell. That shell contains the
 * prompt content, reference upload tiles, and lime special CTA in a single compact
 * dock. Tokens only, save the branded lime CTA gradient, Figma's composite
 * prompt/CTA shadows (no matching shadow tokens), and two documented literal sizes
 * (70px rail / 112px CTA), which have no spacing-scale equivalent and are exposed
 * as overridable knobs.
 */
@source "./prompt-box.tsx";

@utility q-prompt-box {
  /* Figma literals with no --hf-space-* equivalent — documented, overridable. */
  --q-prompt-box-rail-width: 4.375rem; /* 70px */
  --q-prompt-box-generate-width: 7rem; /* 112px */

  position: relative;
  display: flex;
  align-items: flex-end;
  gap: var(--hf-space-200);
  padding: var(--hf-space-300);
}

@utility q-prompt-box-dock {
  display: flex;
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  padding: var(--hf-space-100);
  border-radius: var(--hf-radius-600);
  background-color: var(--hf-color-background-secondary);
  box-shadow:
    0 4px 2px rgba(0, 0, 0, 0.16),
    0 4px 6px rgba(0, 0, 0, 0.08);
}

@utility q-prompt-box-dock-surface {
  display: flex;
  flex: 1 1 var(--hf-space-0);
  align-items: flex-start;
  gap: var(--hf-space-200);
  min-width: var(--hf-space-0);
  padding: var(--hf-space-300);
  overflow: clip;
  border-radius: var(--hf-radius-500);
  background-color: var(--hf-color-transparent-light-05);
}

/* Frosted floating-bar skin (the generation-state dock). */
@utility q-prompt-box-glass {
  border: var(--hf-border-width-thin) solid var(--hf-color-transparent-light-30);
  border-radius: var(--hf-radius-500);
  background-color: var(--hf-color-background-glass);
  backdrop-filter: blur(10px) saturate(1.6);
  box-shadow: var(--hf-shadow-lg, 0 12px 32px rgba(0, 0, 0, 0.32));
}

/* ── Mode rail + Mode ──────────────────────────────────────────────── */
@utility q-prompt-box-mode-rail {
  display: flex;
  flex-direction: column;
  flex-shrink: 0;
  gap: var(--hf-space-100);
  width: var(--q-prompt-box-rail-width);
  padding: var(--hf-space-100);
  border-radius: var(--hf-radius-600);
  background-color: var(--hf-color-background-secondary);
  box-shadow:
```

```
    0 4px 4px rgba(0, 0, 0, 0.16),
    0 4px 12px rgba(0, 0, 0, 0.08);
}

@utility q-prompt-box-mode {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: var(--hf-space-100);
  width: 100%;
  height: 3.125rem; /* 50px — the Figma mode-button height */
  padding: var(--hf-space-150) var(--hf-space-300);
  border-radius: var(--hf-radius-400);
  background-color: transparent;
  color: var(--hf-color-text-secondary);
  cursor: pointer;
  outline: none;
  transition-property: background-color, background-image, color, box-shadow;

  &:hover {
    background-image: linear-gradient(var(--hf-color-overlay-hover), var(--hf-color-overlay-hover));
  }

  @apply q-focus-ring;

  &:disabled,
  &[data-disabled] {
    pointer-events: none;
    opacity: var(--hf-opacity-disabled);
  }
}

@utility q-prompt-box-mode-active {
  border-radius: var(--hf-radius-500);
  background-color: var(--hf-color-transparent-light-05);
  color: var(--hf-color-text-primary);
}

@utility q-prompt-box-mode-icon {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  color: currentcolor;

  & svg {
    width: var(--hf-icon-md);
    height: var(--hf-icon-md);
  }
}

@utility q-prompt-box-mode-label {
  @apply text-q-caption-xs-semi-bold;
}

/* ■■ Body: prompt content inside the shared right-side shell ■■■■■■■■■■■■■■■■■■ */
@utility q-prompt-box-body {
  display: flex;
  flex: 1 1 var(--hf-space-0);
  min-width: var(--hf-space-0);
  border-radius: var(--hf-radius-300);
}

@utility q-prompt-box-surface {
  display: flex;
  flex: 1 1 var(--hf-space-0);
  flex-direction: column;
  gap: var(--hf-space-100);
  min-width: var(--hf-space-0);
  padding: var(--hf-space-0);
```

```
  border-radius: var(--hf-radius-300);
  background-color: transparent;
  transition-property: background-color, box-shadow;
}

@utility q-prompt-box-field {
  --q-prompt-box-field-min-height: calc(var(--hf-space-1200) + var(--hf-space-100));

  flex: 1 1 auto;
  width: 100%;
  min-height: var(--q-prompt-box-field-min-height);
  padding: var(--hf-space-100);
  appearance: none;
  border: var(--hf-border-width-none) solid transparent;
  background: transparent;
  outline: none;
  resize: none;
  overflow: hidden;
  color: var(--hf-color-text-primary);
  @apply text-q-body-sm-regular;

  &::placeholder {
    color: var(--hf-color-text-secondary);
    opacity: 1;
  }
}

/* ■■ Actions (the footer pill row) ■■■
```

```
  padding-inline: var(--hf-space-0);
}

@utility q-prompt-box-pill-start {
  display: inline-flex;
  flex-shrink: 0;
  align-items: center;
  justify-content: center;
  color: var(--hf-color-icon-primary);

  & svg,
  & img {
    width: var(--hf-icon-sm);
    height: var(--hf-icon-sm);
  }

  /* A composed Avatar keeps its own size. */
  & > * {
    flex-shrink: 0;
  }
}

@utility q-prompt-box-pill-label {
  white-space: nowrap;
}

@utility q-prompt-box-pill-end {
  display: inline-flex;
  flex-shrink: 0;
  align-items: center;
  justify-content: center;
  color: var(--hf-color-icon-secondary);

  & svg {
    width: var(--hf-icon-sm);
    height: var(--hf-icon-sm);
  }
}

/* ■■ Uploads + Upload (reference-image tiles) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility q-prompt-box-uploads {
  display: flex;
  flex-shrink: 0;
  align-items: flex-start;
  gap: var(--hf-space-150);
}

@utility q-prompt-box-upload {
  position: relative;
  display: flex;
  flex-direction: column;
  flex-shrink: 0;
  justify-content: space-between;
  width: var(--hf-space-2000); /* 80px */
  height: var(--hf-space-2000);
  padding: var(--hf-space-150);
  overflow: clip;
  border: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  border-radius: var(--hf-radius-300);
  background-color: var(--hf-color-transparent-light-05);
  cursor: pointer;
  outline: none;
  transition-property: background-color, background-image, box-shadow;

  &:hover {
    background-image: linear-gradient(var(--hf-color-overlay-hover), var(--hf-color-overlay-hover));
  }

  @apply q-focus-ring;
}
```

```
@utility q-prompt-box-upload-media {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  object-fit: cover;
}

/* Scrim so the label stays legible over a picked image. */
@utility q-prompt-box-upload-filled {
  &::after {
    content: "";
    position: absolute;
    inset: 0;
    background-image: linear-gradient(to top, var(--hf-color-transparent-dark-60), transparent 60%);
    pointer-events: none;
  }
}

@utility q-prompt-box-upload-add {
  position: relative;
  z-index: 1;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-500);
  height: var(--hf-space-500);
  border-radius: var(--hf-radius-full);
  background-color: var(--hf-color-transparent-light-05);
  backdrop-filter: blur(2.5px);
  color: var(--hf-color-icon-primary);
  box-shadow: var(--hf-shadow-raised-sm, 0 2px 1.5px -0.5px rgba(0, 0, 0, 0.16));

  & svg {
    width: var(--hf-icon-sm);
    height: var(--hf-icon-sm);
  }
}

@utility q-prompt-box-upload-label {
  position: relative;
  z-index: 1;
  align-self: flex-start;
  text-align: left;
  color: var(--hf-color-text-primary);
  text-transform: uppercase;
  font-family: var(--hf-type-family-grotesk);
  font-size: var(--hf-type-size-050);
  font-weight: var(--hf-type-weight-bold);
  line-height: var(--hf-type-line-height-200);
  letter-spacing: var(--hf-type-letter-spacing-slight);
}

/* ■■ Generate (the branded lime CTA) ■■■
```

```
    linear-gradient(90deg, rgb(209, 254, 23) 0%, rgb(209, 254, 23) 100%);
  color: var(--hf-color-text-inverse);
  cursor: pointer;
  outline: none;
  box-shadow:
    inset 0 -3px 0 0 rgba(0, 0, 0, 0.32),
    0 32px 24px rgba(0, 0, 0, 0.15),
    0 6px 4px rgba(0, 0, 0, 0.25);
  transition-property: filter, box-shadow;

  &:hover {
    filter: brightness(1.04);
  }

  @apply q-focus-ring;

  &:disabled,
  &[data-disabled] {
    pointer-events: none;
    opacity: var(--hf-opacity-disabled);
  }
}

@utility q-prompt-box-generate-label {
  text-transform: uppercase;
  text-align: center;
  font-family: var(--hf-type-family-grotesk);
  font-size: var(--hf-type-size-100);
  font-weight: var(--hf-type-weight-bold);
  line-height: var(--hf-type-line-height-100);
  letter-spacing: var(--hf-type-letter-spacing-none);
}

@utility q-prompt-box-generate-meta {
  display: inline-flex;
  align-items: center;
  gap: var(--hf-space-100);
  color: var(--hf-color-text-inverse);

  & svg {
    width: var(--hf-icon-sm);
    height: var(--hf-icon-sm);
  }
}

@utility q-prompt-box-generate-old {
  text-decoration: line-through;
  color: rgba(0, 0, 0, 0.2);
  @apply text-q-caption-sm-medium;
}

@utility q-prompt-box-generate-cost {
  @apply text-q-caption-sm-semi-bold;
}
```

app/src/components/prompt-box/prompt-box.tsx

```
"use client";

import { Children, isValidElement } from "react";
import type { ComponentProps, ComponentPropsWithRef, ReactElement, ReactNode, Ref } from "react";
import { useRender } from "@base-ui/react/use-render";
import { Sparkle as SparklesGlyph } from "lucide-react";
import { cn as cx } from "@/lib/utils";

/**
 * PromptBox — the Studio "prompt dock" (Figma Marketing-Studio node 7259:51362,
 * reused in the Cinema-Studio generation state 21768:60842). The horizontal
 * generation bar every Studio screen leads with: a vertical generation-MODE rail
 * (Product / App), then one shared right-side dock containing the prompt SURFACE
 * (a free-text area over a footer row of setting PILLS), a strip of reference
 * UPLOAD tiles (Product / Avatar), and the special GENERATE button.
 *
 * It is the counterpart to `Composer` (the tall side-rail prompt pane): where
 * Composer is a single vertical field, PromptBox is the wide multi-slot dock
 * that carries the mode rail, inline setting pickers, upload tiles, and the CTA.
 *
 * COMPOSITION-FIRST (same rules as Composer / Sidebar / Modal). The component
 * owns the DESIGN — the two-layer prompt surface, the pill/tile/mode shells, the
 * lime CTA — and every control is CONTENT you compose:
 *
 *   <PromptBox.Root>
 *     <PromptBox.ModeRail>
 *       <PromptBox.Mode active start={<CubeIcon />}>Product</PromptBox.Mode>
 *       <PromptBox.Mode start={<WorldIcon />}>App</PromptBox.Mode>
 *     </PromptBox.ModeRail>
 *
 *     <PromptBox.Body>
 *       <PromptBox.Field placeholder="Describe the scene you imagine…" />
 *       <PromptBox.Actions>
 *         <PromptBox.Pill iconOnly aria-label="Add" start={<PlusIcon />} />
 *         <PromptBox.Pill start={<Avatar/>} end={<ChevronIcon/>}>UGC</PromptBox.Pill>
 *         <PromptBox.Pill iconOnly aria-label="Settings" start={<SlidersIcon />} />
 *       </PromptBox.Actions>
 *     </PromptBox.Body>
 *
 *     <PromptBox.Uploads>
 *       <PromptBox.Upload label="Product" />
 *       <PromptBox.Upload label="Avatar" src={cover} />
 *     </PromptBox.Uploads>
 *
 *     <PromptBox.Generate cost={3} oldCost={12} />
 *   </PromptBox.Root>
 *
 * SETTINGS OPEN AS DROPDOWNS: a `PromptBox.Pill` is a bare `<button>` (or any
 * element via `render`), so it drops straight into a `Select`/`Dropdown` trigger
 * exactly like `SettingTrigger` — Base UI drives `data-popup-open` on the host:
 *
 *   <Select.Root defaultValue="ugc">
 *     <Select.Trigger bare render={<PromptBox.Pill start={<Avatar/>} end={<ChevronIcon/>} />}>
 *       <Select.Value />
 *     </Select.Trigger>
 *     <Select.Content size="picker">…</Select.Content>
 *   </Select.Root>
 *
 * `surface="glass"` skins the whole dock as a frosted floating bar (the
 * generation-state dock) instead of the plain, centered before-generation dock.
 *
 * EVERY SLOT IS OPTIONAL. Because the dock is composition-first, a caller shows a
 * subset simply by omitting parts — no part is required. For prop-driven UIs that
 * would rather keep the JSX and toggle a boolean, the optional parts (`ModeRail`,
 * `Mode`, `Pill`, `Uploads`, `Upload`) accept `hidden`: when `true` the part
 * unmounts (renders `null`) so the surrounding flex gaps collapse cleanly —
 * unlike the native `hidden` attribute, which would leave an empty box in flow.
 * This is how the Studio prompt box makes the Product/App toggle, the
```

```
 * Product/Avatar upload tiles, and each inline setting individually switchable.
 */

export type PromptBoxSurface = "plain" | "glass";

const SURFACE_CLASS = {
  plain: "",
  glass: "q-prompt-box-glass",
} satisfies Record<PromptBoxSurface, string>;

/* ■■ Root ■■■
```

```
  start,
  children,
  render,
  className,
  ref,
  ...props
}: PromptBoxModeProps) {
  const element = useRender({
    render,
    defaultTagName: "button",
    ref: ref as Ref<Element> | undefined,
    props: {
      className: cx("q-prompt-box-mode", active && "q-prompt-box-mode-active", className),
      ...(render == null ? { type: "button" as const } : {}),
      ...(active ? { "aria-pressed": true } : {}),
      children: (
        <>
          {start != null ? <span className="q-prompt-box-mode-icon">{start}</span> : null}
          {children != null ? <span className="q-prompt-box-mode-label">{children}</span> : null}
        </>
      ),
      ...props,
    },
  });
  return hidden ? null : element;
}

/* ■■ Body (the two-layer prompt surface) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
export type PromptBoxBodyProps = ComponentProps<"div"> & {
  /** Class for the inner white-5% surface that hosts the field + actions. */
  surfaceClassName?: string;
};
function Body({ className, surfaceClassName, children, ...props }: PromptBoxBodyProps) {
  return (
    <div className={cx("q-prompt-box-body", className)} {...props}>
      <div className={cx("q-prompt-box-surface", surfaceClassName)}>{children}</div>
    </div>
  );
}

/* ■■ Field (the borderless prompt text area) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
export type PromptBoxFieldProps = Omit<ComponentProps<"textarea">, "children">;
function Field({ className, rows = 1, ...props }: PromptBoxFieldProps) {
  return <textarea rows={rows} className={cx("q-prompt-box-field", className)} {...props} />;
}

/* ■■ Actions (the footer pill row) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
function Actions({ className, ...props }: ComponentProps<"div">) {
  return <div className={cx("q-prompt-box-actions", className)} {...props} />;
}

/* ■■ Pill (a setting control / dropdown trigger) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
export type PromptBoxPillProps = Omit<ComponentPropsWithRef<"button">, "children"> & {
  /** Leading slot — a 16px icon or `<Avatar>`. */
  start?: ReactNode;
  /** Trailing slot — typically a chevron for dropdown pills. */
  end?: ReactNode;
  /** Pill label. Omit with `iconOnly` for a square glyph button (+ / settings). */
  children?: ReactNode;
  /** Square icon-only pill (no label). */
  iconOnly?: boolean;
  /** Unmount this setting pill (renders `null`). */
  hidden?: boolean;
  /** Swap the host (e.g. `Select.Trigger`/`Dropdown.Trigger`). Defaults to `<button>`. */
  render?: ReactElement;
};
function Pill({
  start,
  end,
  children,
```

```
  iconOnly = false,
  hidden = false,
  render,
  className,
  ref,
  ...props
}: PromptBoxPillProps) {
  const element = useRender({
    render,
    defaultTagName: "button",
    ref: ref as Ref<Element> | undefined,
    props: {
      className: cx("q-prompt-box-pill", iconOnly && "q-prompt-box-pill-icon-only", className),
      ...(render == null ? { type: "button" as const } : {}),
      children: (
        <>
          {start != null ? <span className="q-prompt-box-pill-start">{start}</span> : null}
          {children != null ? <span className="q-prompt-box-pill-label">{children}</span> : null}
          {end != null ? <span className="q-prompt-box-pill-end">{end}</span> : null}
        </>
      ),
      ...props,
    },
  });
  return hidden ? null : element;
}

/* ■■ Uploads + Upload (reference-image tiles) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
export type PromptBoxUploadsProps = ComponentProps<"div"> & {
  /** Unmount the whole reference-tile strip (renders `null`). */
  hidden?: boolean;
};
function Uploads({ hidden = false, className, ...props }: PromptBoxUploadsProps) {
  if (hidden) return null;
  return <div className={cx("q-prompt-box-uploads", className)} {...props} />;
}

export type PromptBoxUploadProps = Omit<ComponentPropsWithRef<"button">, "children"> & {
  /** Bottom label (e.g. "Product", "Avatar", "Character"). */
  label?: ReactNode;
  /** Filled-state image — when set the tile shows the picked reference. */
  src?: string;
  /** Alt text for the filled image. */
  alt?: string;
  /** Unmount this reference tile (renders `null`). */
  hidden?: boolean;
  /** The corner glyph (default a plus). Compose a Button/Avatar for a filled slot. */
  add?: ReactNode;
  /** Extra overlay content composed inside the tile. */
  children?: ReactNode;
  /** Swap the host element. Defaults to a `<button>`. */
  render?: ReactElement;
};
function Upload({
  label,
  src,
  alt = "",
  hidden = false,
  add,
  children,
  render,
  className,
  ref,
  ...props
}: PromptBoxUploadProps) {
  const element = useRender({
    render,
    defaultTagName: "button",
    ref: ref as Ref<Element> | undefined,
    props: {
```

```
      className: cx("q-prompt-box-upload", src != null && "q-prompt-box-upload-filled", className),
      ...(render == null ? { type: "button" as const } : {}),
      children: (
        <>
          {src != null ? <img className="q-prompt-box-upload-media" src={src} alt={alt} /> : null}
          <span className="q-prompt-box-upload-add">{add ?? <PlusGlyph />}</span>
          {label != null ? <span className="q-prompt-box-upload-label">{label}</span> : null}
          {children}
        </>
      ),
      ...props,
    },
  });
  return hidden ? null : element;
}

/** The small "+" corner glyph rendered inside an empty Upload tile by default. */
function PlusGlyph() {
  return (
    <svg viewBox="0 0 16 16" fill="none" aria-hidden width="16" height="16">
      <path d="M8 3.5v9M3.5 8h9" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round" />
    </svg>
  );
}

/* ■■ Generate (the special lime CTA) ■■■
```

```
      ...props,
    },
  });
}

export const PromptBox = {
  Root,
  ModeRail,
  Mode,
  Body,
  Field,
  Actions,
  Pill,
  Uploads,
  Upload,
  Generate,
};
```

app/src/components/rail-footer/index.ts

```
export { RailFooter } from "./rail-footer.tsx";
export type { RailFooterProps } from "./rail-footer.tsx";
```

app/src/components/rail-footer/rail-footer.tsx

```tsx
"use client";

import type { ComponentPropsWithRef, ReactNode } from "react";
import { cn } from "@/lib/utils";

/**
 * RailFooter — the pinned Generate CTA footer for a creation rail / input panel
 * (the tall `aside.q-card` on the left of the Preset / AI Stylist workspaces).
 *
 * A creation rail scrolls its own content when the chosen input fields are
 * taller than the viewport; the Generate CTA must stay reachable, so it lives
 * in this footer, which is `sticky bottom-0` inside the scrolling rail and
 * backed by a `linear-gradient` scrim (from the rail surface, `q-background-
 * secondary`, up to transparent) so the last fields fade under it instead of
 * being hidden by a hard edge. `mt-auto` makes it settle at the very bottom
 * when everything fits (the scrim then fades over empty space — invisible), and
 * pin over the content when it overflows.
 *
 * Pass the Generate `Button` (with its `{label} {sparkles} {credits}` cost slot)
 * as `children`. The rail itself must be the scroll container — stretch it to
 * the viewport (no `self-start`) and add `overflow-y-auto`:
 *
 *   <aside className={card({ surface: 'solid' }, 'flex flex-col gap-3 overflow-y-auto p-3')}>
 *     …fields…
 *     <RailFooter>
 *       <Button variant="marketingPrimary" size="lg" className="w-full" …>Generate</Button>
 *     </RailFooter>
 *   </aside>
 */
export interface RailFooterProps extends ComponentPropsWithRef<"div"> {
  /** The Generate CTA (a Quanta `marketingPrimary` Button, full-width). */
  children: ReactNode;
}

export function RailFooter({ children, className, ...props }: RailFooterProps) {
  return (
    <div
      className={cn(
        // Pinned to the bottom of the scrolling rail; `mt-auto` drops it to the
        // bottom when content is short. `-mb-3` bleeds past the rail's `p-3`
        // bottom padding so the scrim + CTA sit flush against the card's bottom
        // border instead of floating above it (`pb-3` restores the CTA inset).
        // The gradient scrim (rail surface → transparent) fades the scrolling
        // fields under the CTA.
        "sticky bottom-0 z-10 -mb-3 mt-auto flex flex-col bg-gradient-to-t from-q-background-secondary from-70% to-transparent pt-8 pb-3",
        className,
      )}
      {...props}
    >
      {children}
    </div>
  );
}
```

app/src/components/screen-empty-state.tsx

```
import type { ReactNode } from "react";
import { Typography } from "@higgsfield/quanta/typography";
import { cn as cx } from "@/lib/utils";

export interface ScreenEmptyStateContent {
  images: readonly [string, string, string];
  title: ReactNode;
  description: ReactNode;
}

export interface ScreenEmptyStateProps extends ScreenEmptyStateContent {
  action?: ReactNode;
  className?: string;
}

/**
 * Shared full-screen empty state from SC App Builder (Figma 3543:3851).
 * The preview must use representative generations from the current app.
 */
export function ScreenEmptyState({
  images,
  title,
  description,
  action,
  className,
}: ScreenEmptyStateProps) {
  return (
    <div
      className={cx(
        "relative flex min-h-0 flex-1 items-center justify-center overflow-hidden bg-q-background-primary px-6 py-12 text-center",
        className,
      )}
    >
      <div
        aria-hidden
        className="pointer-events-none absolute inset-0"
        style={{
          backgroundImage: "radial-gradient(rgba(255,255,255,0.08) 1px, transparent 1px)",
          backgroundSize: "24px 24px",
          maskImage: "radial-gradient(ellipse 70% 65% at 50% 50%, #000 0%, transparent 78%)",
          WebkitMaskImage:
            "radial-gradient(ellipse 70% 65% at 50% 50%, #000 0%, transparent 78%)",
        }}
      />

      <div className="relative flex w-full max-w-[444px] flex-col items-center gap-6">
        <div className="relative h-[105px] w-full max-w-[444px]" aria-hidden>
          <div className="absolute left-0 top-2.5 h-[85px] w-[38%] overflow-hidden rounded-q-400 opacity-70">
            <img src={images[0]} alt="" className="size-full object-cover" />
            <span className="absolute inset-0 bg-q-transparent-dark-30" />
          </div>
          <div className="absolute right-0 top-2.5 h-[85px] w-[38%] overflow-hidden rounded-q-400 opacity-70">
            <img src={images[2]} alt="" className="size-full object-cover" />
            <span className="absolute inset-0 bg-q-transparent-dark-30" />
          </div>
          <div className="absolute inset-y-0 left-1/2 z-[1] w-[41.5%] -translate-x-1/2 overflow-hidden rounded-[20px] border border-q-border-subtle bg-q-transparent-light-05 shadow-[inset_0_0_20px_rgb
            <img src={images[1]} alt="" className="size-full object-cover" />
          </div>
        </div>

        <div className="flex flex-col items-center gap-1">
          <Typography as="h2" variant="body-md-semi-bold" color="primary">
            {title}
          </Typography>
          <Typography as="p" variant="body-sm-regular" color="tertiary">
            {description}
          </Typography>
        </div>
```

```
        {action != null ? <div className="flex items-center justify-center">{action}</div> : null}
      </div>
    </div>
  );
}
```

app/src/components/scroll-scrub/scroll-scrub.css

```
.scroll-scrub {
  --ss-progress: 0;
  position: relative;
  isolation: isolate;
  color: var(--ss-ink);
  background: var(--ss-bg);
}

.scroll-scrub__stage {
  position: sticky;
  top: 0;
  z-index: 0;
  width: 100%;
  height: 100dvh;
  overflow: hidden;
  background: var(--ss-bg);
}

.scroll-scrub__media,
.scroll-scrub__layer,
.scroll-scrub__picture,
.scroll-scrub__poster,
.scroll-scrub__video {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
}

.scroll-scrub__media {
  z-index: 0;
}

.scroll-scrub__layer {
  margin: 0;
  overflow: hidden;
  opacity: 0;
  will-change: opacity;
}

.scroll-scrub__layer:first-child {
  opacity: 1;
}

.scroll-scrub__poster,
.scroll-scrub__video {
  display: block;
  object-fit: cover;
  object-position: var(--ss-object-position);
}

.scroll-scrub__poster {
  z-index: 0;
}

.scroll-scrub__video {
  z-index: 1;
}

.scroll-scrub__layer[data-video-painted="true"] .scroll-scrub__poster {
  visibility: hidden;
}

.scroll-scrub__progress {
  position: absolute;
  z-index: 5;
  top: 0;
  right: 0;
```

```
  left: 0;
  height: 2px;
  background: color-mix(in srgb, var(--ss-ink) 14%, transparent);
  pointer-events: none;
}

.scroll-scrub__progress span {
  display: block;
  width: 100%;
  height: 100%;
  background: var(--ss-accent);
  transform: scaleX(var(--ss-progress));
  transform-origin: left center;
}

.scroll-scrub__route {
  position: absolute;
  z-index: 6;
  top: clamp(1rem, 3vw, 2rem);
  right: clamp(1rem, 4vw, 4rem);
  left: clamp(1rem, 4vw, 4rem);
  display: flex;
  justify-content: flex-end;
  gap: 0.25rem;
  pointer-events: auto;
}

.scroll-scrub__route-button {
  min-height: 2.75rem;
  padding: 0.65rem 0.9rem;
  border: 0;
  color: var(--ss-muted);
  background: transparent;
  font: inherit;
  cursor: pointer;
}

.scroll-scrub__route-button:hover,
.scroll-scrub__route-button[aria-current="step"] {
  color: var(--ss-ink);
}

.scroll-scrub__route-button[aria-current="step"] {
  text-decoration: underline;
  text-decoration-color: var(--ss-accent);
  text-decoration-thickness: 0.15em;
  text-underline-offset: 0.3em;
}

.scroll-scrub__route-button:focus-visible {
  outline: 2px solid var(--ss-accent);
  outline-offset: 3px;
}

.scroll-scrub__story {
  position: relative;
  z-index: 3;
  margin-top: -100dvh;
  pointer-events: none;
}

.scroll-scrub__chapter,
.scroll-scrub__connector-band {
  position: relative;
}

.scroll-scrub__chapter-pin {
  position: sticky;
  top: 0;
  display: flex;
```

```
  align-items: center;
  min-height: 100dvh;
  padding: clamp(5rem, 9vw, 8rem) clamp(1.25rem, 6vw, 6rem);
  pointer-events: none;
}

.scroll-scrub__chapter[data-align="right"] .scroll-scrub__chapter-pin {
  justify-content: flex-end;
}

.scroll-scrub__copy {
  position: relative;
  width: min(40rem, 48vw);
  pointer-events: auto;
}

.scroll-scrub__copy::before {
  position: absolute;
  z-index: -1;
  inset: -3rem -5rem;
  content: "";
  background: radial-gradient(
    ellipse at center,
    color-mix(in srgb, var(--ss-bg) 88%, transparent) 0%,
    color-mix(in srgb, var(--ss-bg) 58%, transparent) 52%,
    transparent 76%
  );
  pointer-events: none;
}

.scroll-scrub__kicker {
  max-width: 30ch;
  margin: 0 0 1rem;
  color: var(--ss-accent);
  font-size: 0.8rem;
  font-weight: 700;
  letter-spacing: 0.12em;
  text-transform: uppercase;
}

.scroll-scrub__title {
  max-width: 13ch;
  margin: 0;
  color: var(--ss-ink);
  font: inherit;
  font-size: clamp(2.75rem, 6vw, 6.75rem);
  font-weight: 700;
  line-height: 0.94;
  letter-spacing: -0.045em;
  text-wrap: balance;
}

.scroll-scrub__body {
  max-width: 42ch;
  margin: 1.5rem 0 0;
  color: var(--ss-muted);
  font-size: clamp(1rem, 1.35vw, 1.2rem);
  line-height: 1.55;
}

.scroll-scrub__tags {
  display: flex;
  flex-wrap: wrap;
  gap: 0.55rem;
  margin: 1.5rem 0 0;
  padding: 0;
  list-style: none;
}

.scroll-scrub__tags li {
```

```
  padding: 0.45rem 0.7rem;
  border: 1px solid color-mix(in srgb, var(--ss-ink) 20%, transparent);
  color: var(--ss-ink);
  font-size: 0.82rem;
}

.scroll-scrub__actions {
  display: flex;
  flex-wrap: wrap;
  gap: 0.75rem;
  margin-top: 2rem;
}

@media (max-width: 860px) {
  .scroll-scrub__poster,
  .scroll-scrub__video {
    object-position: var(--ss-mobile-position);
  }

  .scroll-scrub__route {
    right: max(0.75rem, env(safe-area-inset-right));
    left: max(0.75rem, env(safe-area-inset-left));
    justify-content: flex-start;
    overflow-x: auto;
    overscroll-behavior-x: contain;
    scrollbar-width: thin;
  }

  .scroll-scrub__route-button {
    flex: 0 0 auto;
  }

  .scroll-scrub__chapter-pin,
  .scroll-scrub__chapter[data-align="right"] .scroll-scrub__chapter-pin {
    align-items: flex-end;
    justify-content: flex-start;
    padding: 6.5rem max(1.25rem, env(safe-area-inset-right))
      calc(4rem + env(safe-area-inset-bottom))
      max(1.25rem, env(safe-area-inset-left));
  }

  .scroll-scrub__copy {
    width: min(100%, 36rem);
  }

  .scroll-scrub__copy::before {
    inset: -3rem -2rem;
    background: linear-gradient(
      to top,
      var(--ss-bg) 0%,
      color-mix(in srgb, var(--ss-bg) 82%, transparent) 58%,
      transparent 100%
    );
  }

  .scroll-scrub__title {
    max-width: 14ch;
    font-size: clamp(2.3rem, 11vw, 4.5rem);
  }
}

@media (prefers-reduced-motion: reduce) {
  .scroll-scrub__chapter {
    min-height: 100dvh !important;
  }

  .scroll-scrub__connector-band {
    display: none;
  }
```

```
  .scroll-scrub__progress span {
    transition: none;
  }
}
```

app/src/components/scroll-scrub/scroll-scrub.tsx

```
/* Scroll scrub React/TanStack reference implementation. */

import { useEffect, useMemo, useRef, useState } from "react";
import type { CSSProperties, ReactNode } from "react";

import "./scroll-scrub.css";

export interface ScrollScrubScene {
  id: string;
  label: string;
  /** Exact first frame of the deployed desktop clip. */
  poster: string;
  /** Exact first frame of mobileClip; provide whenever mobileClip is set. */
  mobilePoster?: string;
  clip: string;
  mobileClip?: string;
  title: string;
  body: string;
  kicker?: string;
  tags?: string[];
  actions?: ReactNode;
  align?: "left" | "right";
  /** Viewport-heights assigned to this scene. More distance means slower scrub. */
  scroll?: number;
  /** 0..0.6. Slow the middle of the clip without changing either seam frame. */
  linger?: number;
  objectPosition?: string;
  mobileObjectPosition?: string;
}

export interface ScrollScrubConnector {
  /** Exact first frame of this connector clip; never substitute a scene still. */
  poster: string;
  /** Exact first frame of mobileClip; provide whenever mobileClip is set. */
  mobilePoster?: string;
  clip: string;
  mobileClip?: string;
  scroll?: number;
}

export interface ScrollScrubTheme {
  background: string;
  ink: string;
  muted: string;
  accent: string;
}

export interface ScrollScrubProps {
  scenes: ScrollScrubScene[];
  /** Leave empty for continuous-forward architecture A. */
  connectors?: (ScrollScrubConnector | null)[];
  theme: ScrollScrubTheme;
  className?: string;
  onActiveSectionChange?: (index: number) => void;
}

interface Segment {
  key: string;
  kind: "scene" | "connector";
  sectionIndex: number;
  nextSectionIndex: number;
  poster: string;
  mobilePoster?: string;
  clip: string;
  mobileClip?: string;
  weight: number;
  linger: number;
  objectPosition: string;
```

```
  mobileObjectPosition: string;
  scene?: ScrollScrubScene;
}

interface RuntimeSegment extends Segment {
  band: HTMLElement;
  layer: HTMLElement;
  start: number;
  end: number;
  current: number;
  target: number;
  visible: boolean;
  loading: boolean;
  ready: boolean;
  failed: boolean;
  loadedSource?: string;
  video?: HTMLVideoElement;
  objectUrl?: string;
  abort?: AbortController;
}

interface Controller {
  jumpToSection: (index: number) => void;
}

type ThemeStyle = CSSProperties & Record<`--ss-${string}`, string | number>;

const clamp = (value: number, min = 0, max = 1) =>
  Math.min(max, Math.max(min, value));

const smoothstep = (value: number) => {
  const x = clamp(value);
  return x * x * (3 - 2 * x);
};

const lingerEase = (value: number, amount: number) => {
  const x = clamp(value);
  const linger = clamp(amount, 0, 0.6);
  const centered = x - 0.5;
  return (1 - linger) * x + linger * (4 * centered ** 3 + 0.5);
};

function buildSegments(
  scenes: ScrollScrubScene[],
  connectors: (ScrollScrubConnector | null)[]
): Segment[] {
  const result: Segment[] = [];

  for (const [index, scene] of scenes.entries()) {
    if (scene.mobileClip && !scene.mobilePoster) {
      throw new Error(`Scene ${scene.id} needs mobilePoster for mobileClip`);
    }
    result.push({
      clip: scene.clip,
      key: `scene:${scene.id}`,
      kind: "scene",
      linger: scene.linger ?? 0,
      mobileClip: scene.mobileClip,
      mobilePoster: scene.mobilePoster,
      mobileObjectPosition:
        scene.mobileObjectPosition ?? scene.objectPosition ?? "50% 50%",
      nextSectionIndex: index,
      objectPosition: scene.objectPosition ?? "50% 50%",
      poster: scene.poster,
      scene,
      sectionIndex: index,
      weight: scene.scroll ?? 1.4,
    });

    const connector = connectors[index];
```

```
    if (index < scenes.length - 1 && connector?.clip) {
      if (connector.mobileClip && !connector.mobilePoster) {
        throw new Error(
          `Connector after ${scene.id} needs mobilePoster for mobileClip`
        );
      }
      const nextScene = scenes[index + 1];
      result.push({
        clip: connector.clip,
        key: `connector:${scene.id}:${nextScene.id}`,
        kind: "connector",
        linger: 0,
        mobileClip: connector.mobileClip,
        mobilePoster: connector.mobilePoster,
        mobileObjectPosition:
          nextScene.mobileObjectPosition ??
          nextScene.objectPosition ??
          "50% 50%",
        nextSectionIndex: index + 1,
        objectPosition: nextScene.objectPosition ?? "50% 50%",
        poster: connector.poster,
        sectionIndex: index,
        weight: connector.scroll ?? 0.8,
      });
    }
  }

  return result;
}

export function ScrollScrub({
  scenes,
  connectors,
  theme,
  className,
  onActiveSectionChange,
}: ScrollScrubProps) {
  const rootRef = useRef<HTMLElement>(null);
  const controllerRef = useRef<Controller | null>(null);
  const onActiveRef = useRef(onActiveSectionChange);
  const [activeSection, setActiveSection] = useState(0);
  const segments = useMemo(
    () => buildSegments(scenes, connectors ?? []),
    [connectors, scenes]
  );

  // Keep the latest callback reachable from the scroll loop without making it a
  // dependency of the controller effect. Synced in an effect, never during
  // render — a render-phase ref write breaks under React Compiler.
  useEffect(() => {
    onActiveRef.current = onActiveSectionChange;
  }, [onActiveSectionChange]);

  useEffect(() => {
    const root = rootRef.current;
    if (!root || segments.length === 0) {
      return;
    }

    const layerNodes = [
      ...root.querySelectorAll<HTMLElement>("[data-scroll-scrub-layer]"),
    ];
    const bandNodes = [
      ...root.querySelectorAll<HTMLElement>("[data-scroll-scrub-band]"),
    ];
    if (
      layerNodes.length !== segments.length ||
      bandNodes.length !== segments.length
    ) {
      throw new Error("ScrollScrub segment markup is out of sync");
```

```
}

const reduceMotion = window.matchMedia(
  "(prefers-reduced-motion: reduce)"
).matches;
const coarsePointer = window.matchMedia(
  "(hover: none) and (pointer: coarse)"
).matches;
const smallViewport = window.matchMedia("(max-width: 860px)");
const isMobile = () => coarsePointer || smallViewport.matches;
const sourceFor = (segment: RuntimeSegment) =>
  isMobile() && segment.mobileClip ? segment.mobileClip : segment.clip;
const runtime: RuntimeSegment[] = segments.map((segment, index) => ({
  ...segment,
  band: bandNodes[index],
  current: 0,
  end: 0,
  failed: false,
  layer: layerNodes[index],
  loading: false,
  ready: false,
  start: 0,
  target: 0,
  visible: index === 0,
}));

let active = -1;
let destroyed = false;
let dirty = true;
let frame = 0;
let rootTop = 0;
let total = 1;
let viewportHeight = window.innerHeight;
let layoutWidth = window.innerWidth;
let userReady = false;

const unloadClip = (segment: RuntimeSegment) => {
  segment.abort?.abort();
  segment.video?.remove();
  if (segment.objectUrl) {
    URL.revokeObjectURL(segment.objectUrl);
  }
  delete segment.abort;
  delete segment.video;
  delete segment.objectUrl;
  delete segment.loadedSource;
  segment.loading = false;
  segment.ready = false;
  segment.failed = false;
  segment.current = segment.target;
  delete segment.layer.dataset.videoPainted;
  delete segment.layer.dataset.videoFailed;
};

const layout = () => {
  const pageY = window.scrollY || window.pageYOffset;
  rootTop = root.getBoundingClientRect().top + pageY;
  viewportHeight = window.innerHeight;
  layoutWidth = window.innerWidth;

  for (const segment of runtime) {
    if (
      segment.loadedSource &&
      segment.loadedSource !== sourceFor(segment)
    ) {
      unloadClip(segment);
    }
    const rect = segment.band.getBoundingClientRect();
    segment.start = rect.top + pageY - rootTop;
    segment.end = segment.start + rect.height;
```

```
  }
  total = Math.max(runtime.at(-1)?.end ?? viewportHeight, viewportHeight);
  dirty = true;
};

const primeVideo = async (video?: HTMLVideoElement) => {
  if (!video || !isMobile()) {
    return;
  }
  try {
    await video.play();
    video.pause();
  } catch {
    // Keep the poster; a later user gesture/seek can retry naturally.
  }
};

const loadClip = async (segment: RuntimeSegment) => {
  const source = sourceFor(segment);
  if (
    reduceMotion ||
    destroyed ||
    segment.loading ||
    segment.ready ||
    segment.failed ||
    !source
  ) {
    return;
  }

  segment.loading = true;
  segment.loadedSource = source;
  segment.abort = new AbortController();
  const request = segment.abort;

  try {
    const response = await fetch(source, {
      signal: request.signal,
    });
    if (!response.ok) {
      throw new Error(`Clip failed: ${response.status}`);
    }
    const blob = await response.blob();
    if (
      destroyed ||
      request.signal.aborted ||
      segment.loadedSource !== source
    ) {
      return;
    }

    const objectUrl = URL.createObjectURL(blob);
    const video = document.createElement("video");
    video.className = "scroll-scrub__video";
    video.muted = true;
    video.playsInline = true;
    video.preload = "auto";
    video.setAttribute("muted", "");
    video.setAttribute("playsinline", "");
    video.src = objectUrl;

    video.addEventListener(
      "loadedmetadata",
      () => {
        if (segment.video !== video || segment.loadedSource !== source) {
          return;
        }
        segment.ready = true;
        segment.loading = false;
        dirty = true;
```

```
      },
      { once: true }
    );
    video.addEventListener(
      "loadeddata",
      () => {
        if (
          userReady &&
          segment.video === video &&
          segment.loadedSource === source
        ) {
          void primeVideo(video);
        }
      },
      { once: true }
    );
    video.addEventListener(
      "error",
      () => {
        if (segment.video !== video) {
          return;
        }
        video.remove();
        URL.revokeObjectURL(objectUrl);
        delete segment.video;
        delete segment.objectUrl;
        segment.failed = true;
        segment.loading = false;
        segment.ready = false;
        delete segment.layer.dataset.videoPainted;
        segment.layer.dataset.videoFailed = "true";
      },
      { once: true }
    );
    video.addEventListener(
      "seeked",
      () => {
        if (segment.video === video && segment.loadedSource === source) {
          segment.layer.dataset.videoPainted = "true";
        }
      },
      { once: true }
    );

    segment.layer.append(video);
    segment.objectUrl = objectUrl;
    segment.video = video;
  } catch (error) {
    if (
      request.signal.aborted ||
      (error instanceof Error && error.name === "AbortError") ||
      segment.loadedSource !== source
    ) {
      return;
    }
    segment.layer.dataset.videoFailed = "true";
    segment.failed = true;
    segment.loading = false;
  }
};

const readScroll = () => {
  const pageY = window.scrollY || window.pageYOffset;
  const y = clamp(pageY - rootTop, 0, total);
  const crossfade = 0.1 * viewportHeight;
  let currentIndex = 0;

  for (const [index, segment] of runtime.entries()) {
    if (y >= segment.start) {
      currentIndex = index;
```

```
    }

    const length = Math.max(segment.end - segment.start, 1);
    const local = clamp((y - segment.start) / length);
    segment.target = segment.linger
      ? lingerEase(local, segment.linger)
      : local;

    let outside = 0;
    if (y < segment.start) {
      outside = segment.start - y;
    }
    if (y > segment.end) {
      outside = y - segment.end;
    }
    let opacity = smoothstep(1 - outside / Math.max(crossfade, 1));
    if (reduceMotion) {
      opacity = outside === 0 ? 1 : 0;
    }

    segment.visible = opacity > 0.001;
    segment.layer.style.opacity = String(opacity);
    segment.layer.style.zIndex = index === currentIndex ? "2" : "1";

    if (
      y > segment.start - 1.5 * viewportHeight &&
      y < segment.end + 1.5 * viewportHeight
    ) {
      void loadClip(segment);
    }
  }

  const current = runtime[currentIndex];
  const currentLength = Math.max(current.end - current.start, 1);
  const currentProgress = clamp((y - current.start) / currentLength);
  const nextActive =
    current.kind === "connector" && currentProgress >= 0.5
      ? current.nextSectionIndex
      : current.sectionIndex;

  if (nextActive !== active) {
    active = nextActive;
    root.dataset.activeSection = String(active);
    setActiveSection(active);
    onActiveRef.current?.(active);
  }

  root.style.setProperty("--ss-progress", String(clamp(y / total)));
};

const updateVideos = () => {
  for (const segment of runtime) {
    const { video } = segment;
    if (!video || !segment.ready || video.seeking) {
      continue;
    }
    if (
      !segment.visible &&
      Math.abs(segment.current - segment.target) < 0.002
    ) {
      continue;
    }

    segment.current += (segment.target - segment.current) * 0.2;
    const targetTime =
      clamp(segment.current, 0, 0.999) * (video.duration || 1);
    const epsilon = isMobile() ? 0.02 : 0.008;
    if (Math.abs(video.currentTime - targetTime) > epsilon) {
      try {
        video.currentTime = targetTime;
```

```
      } catch {
        // Keep the last painted frame while the browser catches up.
      }
    }
  }
};

const tick = () => {
  if (destroyed) {
    return;
  }
  if (dirty) {
    dirty = false;
    readScroll();
  }
  updateVideos();
  frame = window.requestAnimationFrame(tick);
};

const onScroll = () => {
  dirty = true;
};
const onResize = () => {
  if (coarsePointer && window.innerWidth === layoutWidth) {
    return;
  }
  layout();
};
const onFirstGesture = () => {
  if (userReady) {
    return;
  }
  userReady = true;
  for (const segment of runtime) {
    void primeVideo(segment.video);
  }
};

controllerRef.current = {
  jumpToSection(index) {
    const segment = runtime.find(
      (candidate) =>
        candidate.kind === "scene" && candidate.sectionIndex === index
    );
    if (!segment) {
      return;
    }
    const top =
      rootTop + segment.start + 0.15 * (segment.end - segment.start);
    window.scrollTo({
      behavior: reduceMotion ? "auto" : "smooth",
      top,
    });
  },
};

window.addEventListener("scroll", onScroll, { passive: true });
window.addEventListener("resize", onResize);
window.addEventListener("orientationchange", layout);
window.addEventListener("pointerdown", onFirstGesture, {
  once: true,
  passive: true,
});
window.addEventListener("touchstart", onFirstGesture, {
  once: true,
  passive: true,
});

layout();
frame = window.requestAnimationFrame(tick);
```

```
  return () => {
    destroyed = true;
    controllerRef.current = null;
    window.cancelAnimationFrame(frame);
    window.removeEventListener("scroll", onScroll);
    window.removeEventListener("resize", onResize);
    window.removeEventListener("orientationchange", layout);
    window.removeEventListener("pointerdown", onFirstGesture);
    window.removeEventListener("touchstart", onFirstGesture);
    root.style.removeProperty("--ss-progress");
    delete root.dataset.activeSection;

    for (const segment of runtime) {
      unloadClip(segment);
      segment.layer.style.removeProperty("opacity");
      segment.layer.style.removeProperty("z-index");
    }
  };
}, [segments]);

if (scenes.length === 0) {
  return null;
}

const themeStyle: ThemeStyle = {
  "--ss-accent": theme.accent,
  "--ss-bg": theme.background,
  "--ss-ink": theme.ink,
  "--ss-muted": theme.muted,
};

return (
  <section
    className={["scroll-scrub", className].filter(Boolean).join(" ")}
    ref={rootRef}
    style={themeStyle}
  >
    <div className="scroll-scrub__stage">
      <div aria-hidden="true" className="scroll-scrub__media">
        {segments.map((segment, index) => {
          const layerStyle: ThemeStyle = {
            "--ss-mobile-position": segment.mobileObjectPosition,
            "--ss-object-position": segment.objectPosition,
          };
          return (
            <figure
              className={`scroll-scrub__layer scroll-scrub__layer--${segment.kind}`}
              data-scroll-scrub-layer=""
              key={segment.key}
              style={layerStyle}
            >
              <picture className="scroll-scrub__picture">
                {segment.mobilePoster ? (
                  <source
                    media="(hover: none) and (pointer: coarse), (max-width: 860px)"
                    srcSet={segment.mobilePoster}
                  />
                ) : null}
                <img
                  alt=""
                  className="scroll-scrub__poster"
                  decoding="async"
                  fetchPriority={index === 0 ? "high" : "auto"}
                  loading={index === 0 ? "eager" : "lazy"}
                  src={segment.poster}
                />
              </picture>
            </figure>
          );
        })}
```

```
  </div>

  <div aria-hidden="true" className="scroll-scrub__progress">
    <span />
  </div>

  <nav aria-label="Scroll chapters" className="scroll-scrub__route">
    {scenes.map((scene, index) => (
      <button
        aria-current={activeSection === index ? "step" : undefined}
        className="scroll-scrub__route-button"
        key={scene.id}
        onClick={() => controllerRef.current?.jumpToSection(index)}
        type="button"
      >
        <span>{scene.label}</span>
      </button>
    ))}
  </nav>
</div>

<div className="scroll-scrub__story">
  {segments.map((segment) => {
    const bandStyle: CSSProperties = {
      minHeight: `${Math.max(segment.weight, 0.2) * 100}dvh`,
    };

    if (segment.kind === "connector") {
      return (
        <div
          aria-hidden="true"
          className="scroll-scrub__connector-band"
          data-scroll-scrub-band=""
          key={segment.key}
          style={bandStyle}
        />
      );
    }

    const { scene } = segment;
    if (!scene) {
      return null;
    }
    const Heading = segment.sectionIndex === 0 ? "h1" : "h2";

    return (
      <article
        className="scroll-scrub__chapter"
        data-align={scene.align ?? "left"}
        data-scroll-scrub-band=""
        id={scene.id}
        key={segment.key}
        style={bandStyle}
      >
        <div className="scroll-scrub__chapter-pin">
          <div className="scroll-scrub__copy">
            {scene.kicker ? (
              <p className="scroll-scrub__kicker">{scene.kicker}</p>
            ) : null}
            <Heading className="scroll-scrub__title">
              {scene.title}
            </Heading>
            <p className="scroll-scrub__body">{scene.body}</p>
            {scene.tags?.length ? (
              <ul className="scroll-scrub__tags">
                {scene.tags.map((tag) => (
                  <li key={tag}>{tag}</li>
                ))}
              </ul>
            ) : null}
```

```
                {scene.actions ? (
                  <div className="scroll-scrub__actions">{scene.actions}</div>
                ) : null}
              </div>
            </div>
          </article>
        );
      })}
    </div>
  </section>
 );
}
```

app/src/components/setting-trigger/index.ts

```
export { SettingTrigger } from "./setting-trigger.tsx";
export type { SettingTriggerProps } from "./setting-trigger.tsx";
```

app/src/components/setting-trigger/setting-trigger.css

```
/**
 * SettingTrigger presentation — the compact labelled picker row from the Figma
 * builder panels (nodes 2950:66590 / 2950:66597 / 2950:66603): white-5% surface
 * + same-tone subtle border, radius/300, 12×8 padding; a caption label over the
 * body/sm value (secondary tone while empty), trailing 16px chevron.
 *
 * States follow the shared field language: hover composites the dim overlay
 * (like `q-field-control`), keyboard focus uses the lime `q-focus-ring`,
 * disabled fades via the opacity token.
 */
@source "./setting-trigger.tsx";

@utility q-setting-trigger {
  display: flex;
  align-items: center;
  gap: var(--hf-space-200);
  width: 100%;
  min-width: var(--hf-space-0);
  padding: var(--hf-space-200) var(--hf-space-300);
  border: var(--hf-border-width-thin) solid var(--hf-color-border-subtle);
  border-radius: var(--hf-radius-300);
  background-color: var(--hf-color-overlay-dim-soft);
  text-align: start;
  cursor: pointer;
  outline: none;
  transition-property: background-color, background-image, border-color, box-shadow;

  &:hover {
    background-image: linear-gradient(var(--hf-color-overlay-hover), var(--hf-color-overlay-hover));
  }

  @apply q-focus-ring;

  &:disabled,
  &[data-disabled] {
    pointer-events: none;
    opacity: var(--hf-opacity-disabled);
  }
}

/* Leading slot — 20px glyph box, secondary tone (mirrors q-field-affix). The
 * extra inline-end margin adds breathing room after the icon before the
 * label/value (on top of the row's base gap). */
@utility q-setting-trigger-start {
  display: inline-flex;
  flex-shrink: 0;
  align-items: center;
  justify-content: center;
  width: var(--hf-space-500);
  height: var(--hf-space-500);
  margin-inline-end: var(--hf-space-200);
  color: var(--hf-color-icon-secondary);

  & svg {
    width: 100%;
    height: 100%;
  }
}

/* Label + value stack. Figma: 2px gap, fills the row, truncates. */
@utility q-setting-trigger-body {
  display: flex;
  flex: 1 1 var(--hf-space-0);
  flex-direction: column;
  gap: var(--hf-space-050);
  min-width: var(--hf-space-0);
}
```

```
@utility q-setting-trigger-label {
  color: var(--hf-color-text-secondary);
  @apply text-q-caption-sm-medium;
}

@utility q-setting-trigger-value {
  overflow: hidden;
  color: var(--hf-color-text-primary);
  text-overflow: ellipsis;
  white-space: nowrap;
  @apply text-q-body-sm-medium;
}

/* Empty state — the placeholder takes the secondary tone (Figma "Select voice"). */
@utility q-setting-trigger-placeholder {
  color: var(--hf-color-text-secondary);
}

/* Popup-trigger states (Base UI sets the data-attributes when the row hosts a
 * Select/Dropdown trigger): open keeps the hover fill; the placeholder tint
 * follows data-placeholder when the value is a `Select.Value`. */
.q-setting-trigger[data-popup-open] {
  background-image: linear-gradient(var(--hf-color-overlay-hover), var(--hf-color-overlay-hover));
}

.q-setting-trigger[data-placeholder] .q-setting-trigger-value {
  color: var(--hf-color-text-secondary);
}

/* Trailing slot — the 16px chevron (or any node), primary icon tone. */
@utility q-setting-trigger-end {
  display: inline-flex;
  flex-shrink: 0;
  align-items: center;
  justify-content: center;
  color: var(--hf-color-icon-primary);

  & svg {
    width: var(--hf-icon-sm);
    height: var(--hf-icon-sm);
  }
}
```

app/src/components/setting-trigger/setting-trigger.tsx

```
"use client";

import type { ComponentPropsWithRef, ReactElement, ReactNode, Ref } from "react";
import { useRender } from "@base-ui/react/use-render";
import { ChevronRight as ChevronIcon } from "lucide-react";
import { Icon } from "@higgsfield/quanta/icon";
import { cn as cx } from "@/lib/utils";

/**
 * SettingTrigger — a compact labelled value row that OPENS a picker (Figma
 * "Selectbox" rows: node 2950:66590 "Voice / Select voice", 2950:66597
 * "Aspect Ratio / 16:9", 2950:66603 "Duration / Auto"). The staple control of
 * builder side panels: a caption label over the current value, a trailing
 * chevron, on the white-5% field surface (radius/300, subtle border).
 *
 * It is a TRIGGER, not a value editor — compose it with the overlay that picks
 * the value (Modal, Vault, Dropdown, Select) via their `render`/trigger props,
 * or handle `onClick` directly:
 *
 *   <SettingTrigger label="Voice" placeholder="Select voice" onClick={openVoices} />
 *   <SettingTrigger label="Aspect Ratio">16:9</SettingTrigger>
 *   <Modal.Trigger render={<SettingTrigger label="Duration">Auto</SettingTrigger>} />
 *
 * The canonical builder picker hosts a `Select` (see select.tsx): the row is
 * the bare trigger, `Select.Value` is the value slot, and Base UI drives the
 * open ring + placeholder tint via `data-popup-open` / `data-placeholder`:
 *
 *   <Select.Root defaultValue="16:9">
 *     <Select.Trigger bare render={<SettingTrigger label="Aspect Ratio" />}>
 *       <Select.Value placeholder="Select ratio" />
 *     </Select.Trigger>
 *     <Select.Content size="large">…</Select.Content>
 *   </Select.Root>
 *
 * `children` is the current value; when empty, `placeholder` renders in the
 * muted (secondary) tone. `start`/`end` are free slots — `end` defaults to the
 * 16px chevron-right. The host element is swappable via `render` (Base UI
 * `useRender`); the default is a real `<button type="button">`.
 */

export type SettingTriggerProps = Omit<ComponentPropsWithRef<"button">, "children"> & {
  /** Caption above the value. Any node. */
  label?: ReactNode;
  /** The current value. Any node. When null/undefined, `placeholder` shows. */
  children?: ReactNode;
  /** Muted text shown while there is no value. */
  placeholder?: ReactNode;
  /** Leading slot (20px icon, any node) before the label/value stack. */
  start?: ReactNode;
  /** Trailing slot — defaults to the 16px chevron-right glyph. */
  end?: ReactNode;
  /**
   * Swap the root element/component while keeping the row styling — e.g. an
   * `<a>` or a framework `<Link>`. Defaults to a `<button type="button">`.
   */
  render?: ReactElement;
};

export function SettingTrigger({
  label,
  children,
  placeholder,
  start,
  end,
  render,
  className,
  ref,
  ...props
```

```
}: SettingTriggerProps) {
  const isEmpty = children == null;
  const content = (
    <>
      {start != null ? <span className="q-setting-trigger-start">{start}</span> : null}
      <span className="q-setting-trigger-body">
        {label != null ? <span className="q-setting-trigger-label">{label}</span> : null}
        <span className={cx("q-setting-trigger-value", isEmpty && "q-setting-trigger-placeholder")}>
          {isEmpty ? placeholder : children}
        </span>
      </span>
      <span className="q-setting-trigger-end">{end ?? <Icon size="sm" as={ChevronIcon} />}</span>
    </>
  );

  // Only the default host is a real <button> and gets the implicit type;
  // a `render` element owns its own semantics.
  return useRender({
    render,
    defaultTagName: "button",
    ref: ref as Ref<Element> | undefined,
    props: {
      className: cx("q-setting-trigger", className),
      ...(render == null ? { type: "button" as const } : {}),
      children: content,
      ...props,
    },
  });
}
```

app/src/components/step-rail/index.ts

```
export { StepRail } from "./step-rail.tsx";
export type { StepRailProps, StepRailStep } from "./step-rail.tsx";
```

app/src/components/step-rail/step-rail.tsx

```
"use client";

import type { ReactElement, ReactNode } from "react";
import { useRender } from "@base-ui/react/use-render";
import { Check as IconCheckOutlined } from "lucide-react";
import { Icon } from "@higgsfield/quanta/icon";
import { Typography } from "@higgsfield/quanta/typography";
import { cn } from "@/lib/utils";

/**
 * StepRail — a horizontal numbered step indicator for short in-app wizards
 * (modeled on the Higgsfield "Shots" app header: `1 Upload — 2 Grid — 3
 * Upscale`). Quanta ships no stepper, so this composes Quanta primitives
 * (`Icon`, `Typography`) + `q-` tokens into the numbered-badge / connector row.
 *
 * It is IN-APP navigation, not an app header (the Higgsfield host owns the top
 * chrome): a row of numbered badges joined by connector lines. Completed steps
 * show a brand check, the current step a brand-filled number, upcoming steps a
 * muted number. Pass `reachable` to make already-unlocked steps clickable via
 * `onStepChange`; steps outside it render as disabled markers.
 *
 *   <StepRail
 *     steps={[{ id: 'upload', label: 'Upload' }, …]}
 *     current={step}
 *     reachable={['upload', 'grid']}
 *     onStepChange={setStep}
 *   />
 *
 * The host element is swappable via `render` (Base UI `useRender`) for semantics
 * like `<nav>`.
 */

export interface StepRailStep {
  /** Stable id used for selection + React keys. */
  id: string;
  /** Label shown beside the number badge. */
  label: ReactNode;
}

export interface StepRailProps {
  /** Ordered steps, left → right. */
  steps: StepRailStep[];
  /** The active step id. */
  current: string;
  /**
   * Ids the user may jump to (already unlocked). When set, only these steps are
   * clickable; the rest render as passive markers. Omit to make every step
   * clickable.
   */
  reachable?: string[];
  /** Fired when a reachable step is clicked. */
  onStepChange?: (id: string) => void;
  /** Swap the host element (defaults to a `<nav>`). */
  render?: ReactElement;
  className?: string;
}

interface StepBadgeProps {
  index: number;
  state: "complete" | "current" | "upcoming";
}

/** The numbered circle — brand check (complete), brand number (current), muted (upcoming). */
function StepBadge({ index, state }: StepBadgeProps) {
  return (
    <span
      className={cn(
        "flex size-7 shrink-0 items-center justify-center rounded-q-full text-q-body-sm-semi-bold transition-colors",
```

```
        state === "upcoming"
          ? "bg-q-transparent-light-10 text-q-text-secondary"
          : "bg-q-brand-primary text-q-text-inverse",
      )}
    >
      {state === "complete" ? <Icon as={IconCheckOutlined} size="sm" /> : index + 1}
    </span>
  );
}

export function StepRail({
  steps,
  current,
  reachable,
  onStepChange,
  render,
  className,
}: StepRailProps) {
  const currentIndex = steps.findIndex((step) => step.id === current);

  const content = steps.map((step, index) => {
    const state =
      index < currentIndex ? "complete" : index === currentIndex ? "current" : "upcoming";
    const canClick = onStepChange != null && (reachable == null || reachable.includes(step.id));

    return (
      <div key={step.id} className="flex min-w-0 items-center gap-3">
        {index > 0 ? (
          <span
            aria-hidden
            className={cn(
              "h-px w-6 shrink-0 transition-colors sm:w-12",
              index <= currentIndex ? "bg-q-brand-primary" : "bg-q-border-subtle",
            )}
          />
        ) : null}
        <button
          type="button"
          disabled={!canClick}
          aria-current={state === "current" ? "step" : undefined}
          onClick={canClick ? () => onStepChange?.(step.id) : undefined}
          className={cn(
            "flex items-center gap-2 rounded-q-full px-1 py-0.5 transition-opacity",
            canClick
              ? "cursor-pointer focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-q-border-focus"
              : "cursor-default",
            state === "upcoming" && "opacity-70",
          )}
        >
          <StepBadge index={index} state={state} />
          <Typography
            as="span"
            variant="body-sm-semi-bold"
            color={state === "upcoming" ? "secondary" : "primary"}
            className="hidden truncate sm:inline"
          >
            {step.label}
          </Typography>
        </button>
      </div>
    );
  });

  return useRender({
    render,
    defaultTagName: "nav",
    props: {
      "aria-label": "Progress",
      className: cn("flex items-center justify-center", className),
      children: content,
```

```
    },
  });
}
```

app/src/components/StructuredData.tsx

```
/**
 * SSR-safe JSON-LD structured data component.
 * Place at the top of page JSX, before visible content.
 * The `json` prop must be a pre-stringified JSON string (module-level const).
 */
export function StructuredData({ json }: { json: string }) {
  return <script type="application/ld+json" dangerouslySetInnerHTML={{ __html: json }} />;
}
```

app/src/components/template-modal/index.ts

```
export { TemplateModal } from "./template-modal";
export type { TemplateModalColumns, TemplateModalProps, TemplateOption } from "./template-modal";
```

app/src/components/template-modal/template-modal.tsx

```
import type { ReactElement, ReactNode } from "react";
import { useState } from "react";
import { Media } from "@higgsfield/quanta/media";
import { Modal } from "@higgsfield/quanta/modal";
import { Typography } from "@higgsfield/quanta/typography";
import { cn } from "@/lib/utils";

/**
 * TemplateModal — the "All Presets" picker (Figma SC App Builder, node
 * 3117:143969). A glass `Modal` whose body is a plain grid of selectable option
 * tiles: each a preview image with a label underneath, the active one ringed by
 * the lime brand border. Distinct from the Studio `TemplatePickerModal` (which
 * is a tabbed/searchable gallery of triptych cards) — this is the generic
 * "choose a template / preset / animal / option set" chooser, so any trigger
 * that implies picking one option from a set can reuse it via `trigger`, exactly
 * like `AssetLibraryModal`.
 *
 *   <TemplateModal
 *     title="Select Animal"
 *     options={ANIMALS}
 *     defaultValue="deer"
 *     onSelect={handleSelect}
 *     trigger={<Dropzone render={<button type="button" />} … />}
 *   />
 *
 * Quanta components + `q-` tokens only.
 */

export interface TemplateOption {
  /** Stable id used for selection + React keys. */
  id: string;
  /** Label shown beneath the preview. */
  label: string;
  /** Preview image source. */
  image: string;
  /** Alt text for the preview (defaults to `label`). */
  alt?: string;
}

const COLUMN_CLASS = {
  2: "grid-cols-2",
  3: "grid-cols-3",
  4: "grid-cols-4",
  5: "grid-cols-5",
} as const;

export type TemplateModalColumns = keyof typeof COLUMN_CLASS;

export interface TemplateModalProps {
  /** The trigger element (e.g. a `Dropzone`/button). Rendered as the Modal trigger. */
  trigger: ReactElement;
  /** Selectable option tiles. */
  options: TemplateOption[];
  /** Header title. */
  title?: ReactNode;
  /** Controlled selected id. */
  value?: string;
  /** Uncontrolled initial selected id. */
  defaultValue?: string;
  /** Fired when a tile is chosen. */
  onSelect?: (option: TemplateOption) => void;
  /** Grid columns (default 4, matching Figma). */
  columns?: TemplateModalColumns;
  /** Close the modal once an option is chosen (default true). */
  closeOnSelect?: boolean;
  /** Start opened (uncontrolled) — handy for previews. */
  defaultOpen?: boolean;
}
```

```
interface OptionCardProps {
  option: TemplateOption;
  selected: boolean;
  closeOnSelect: boolean;
  onSelect: (option: TemplateOption) => void;
}

/** A single selectable preset tile — preview + label, lime-ringed when active. */
function OptionCard({ option, selected, closeOnSelect, onSelect }: OptionCardProps) {
  const className = cn(
    "group flex flex-col gap-2 rounded-q-600 text-left transition-transform",
    "focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-q-border-focus",
  );
  const children = (
    <>
      <Media
        ratio="auto"
        rounded="md"
        className={cn(
          "h-60 w-full border-2 bg-q-background-secondary transition-colors",
          selected
            ? "border-q-brand-primary"
            : "border-transparent group-hover:border-q-border-strong",
        )}
      >
        <Media.Image src={option.image} alt={option.alt ?? option.label} />
      </Media>
      <Typography as="span" variant="body-md-semi-bold" color="primary" truncate className="px-0.5">
        {option.label}
      </Typography>
    </>
  );

  const handleClick = () => onSelect(option);

  return closeOnSelect ? (
    <Modal.Close className={className} onClick={handleClick} aria-pressed={selected}>
      {children}
    </Modal.Close>
  ) : (
    <button type="button" className={className} onClick={handleClick} aria-pressed={selected}>
      {children}
    </button>
  );
}

export function TemplateModal({
  trigger,
  options,
  title = "All Presets",
  value,
  defaultValue,
  onSelect,
  columns = 4,
  closeOnSelect = true,
  defaultOpen,
}: TemplateModalProps) {
  const [internal, setInternal] = useState(defaultValue);
  const selectedId = value ?? internal;

  const handleSelect = (option: TemplateOption) => {
    if (value === undefined) setInternal(option.id);
    onSelect?.(option);
  };

  return (
    <Modal.Root defaultOpen={defaultOpen}>
      <Modal.Trigger render={trigger} />
      <Modal.Content size="xl">
        <Modal.Header>
```

```
          <Modal.Title>{title}</Modal.Title>
          <Modal.CloseButton />
        </Modal.Header>

        <div className="min-h-0 flex-1 overflow-y-auto">
          <div className={cn("grid gap-5 p-1", COLUMN_CLASS[columns])}>
            {options.map((option) => (
              <OptionCard
                key={option.id}
                option={option}
                selected={option.id === selectedId}
                closeOnSelect={closeOnSelect}
                onSelect={handleSelect}
              />
            ))}
          </div>
        </div>
      </Modal.Content>
    </Modal.Root>
  );
}
```

app/src/components/template-picker.tsx

```
import type { ReactElement, ReactNode } from "react";
import { useMemo, useState } from "react";
import { User as IconCryptopunkOutlined } from "lucide-react";
import { DollarSign as IconDollarOutlined } from "lucide-react";
import { WandSparkles as IconImagineAiFilled } from "lucide-react";
import { BatteryFull as IconBatteryFullFilled } from "lucide-react";
import { Calendar as IconDateYearlyFilled } from "lucide-react";
import { House as IconHomeRoundDoorFilled } from "lucide-react";
import { Search as IconMagnifyingGlassOutlined } from "lucide-react";
import { Play as IconPlayFilled } from "lucide-react";
import { Music as IconTiktokOutlined } from "lucide-react";
import { Video as IconVideoOutlined } from "lucide-react";
import { Button } from "@higgsfield/quanta/button";
import { Icon } from "@higgsfield/quanta/icon";
import { Media } from "@higgsfield/quanta/media";
import { Modal } from "@higgsfield/quanta/modal";
import { Tabs } from "@higgsfield/quanta/tabs";
import { Typography } from "@higgsfield/quanta/typography";

/**
 * TemplatePickerModal — the full-screen "template settings" picker that opens
 * from the Studio prompt box (Figma Marketing-Studio: the settings/Templates
 * control on node 7259:51362 opens this). A glass `Modal` (spirit of the
 * asset-library modal) whose body is a gallery of selectable IMAGE / VIDEO
 * template tiles, filtered by category (All / TikTok / UGC / Commercial) and by
 * media type (All / Image / Video).
 *
 * ■■ Figma note ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
 * The referenced node (7259:51362) resolves to the Studio *prompt box* itself;
 * Figma exposes no standalone "template modal" node. The tile design is taken
 * verbatim from the Studio gallery cards (node 7137:108927 — brand header,
 * rounded triptych, gradient badge + title/subtitle + lime "Try"), and lifted
 * into a Quanta glass Modal following the asset-library composition. The exported
 * `TemplateCard` + `TEMPLATES` are exported so an in-page
 * gallery and the modal render identical tiles.
 */

// PLACEHOLDER ASSETS — template demo art (see /presets/*.png). When adapting
// this template into a real app, REPLACE media that represents the product
// (hero/example outputs, covers, before/after samples, feed items) with
// bespoke on-brand assets generated via the Higgsfield generation tools.
// Pure style-picker label thumbnails may keep simple placeholder art when
// real output depends on the user's own upload. Grep "PLACEHOLDER ASSETS"
// to find every site.
const THUMBS = [
  "/presets/how-product-works.png",
  "/presets/explain.png",
  "/presets/hyper-motion.png",
  "/presets/cover.png",
] as const;

/** Branded lead-tile gradients (no Quanta gradient token — documented literals). */
const BADGE_GRADIENT = {
  tiktok: "linear-gradient(135deg, rgb(45, 204, 211) 3.87%, rgb(241, 32, 74) 93.45%)",
  blue: "linear-gradient(135deg, rgb(81, 180, 226) 3.87%, rgb(24, 64, 182) 93.45%)",
  pink: "linear-gradient(135deg, rgb(226, 81, 180) 3.87%, rgb(141, 18, 55) 93.45%)",
} as const;

type BadgeGradient = keyof typeof BADGE_GRADIENT;
type LeadGlyph = typeof IconBatteryFullFilled;
export type TemplateCategory = "tiktok" | "ugc" | "commercial";
export type TemplateKind = "image" | "video";

export interface TemplateItem {
  id: string;
  title: string;
  subtitle: string;
  category: TemplateCategory;
```

```
  kind: TemplateKind;
  images: [string, string, string];
  icon: LeadGlyph;
  gradient: BadgeGradient;
}

// PLACEHOLDER ASSETS — demo data; replace when adapting (see note above).
export const TEMPLATES: TemplateItem[] = [
  {
    id: "ugc-gadget",
    title: "UGC Gadget save me",
    subtitle: "Turn long videos into short clips",
    category: "ugc",
    kind: "video",
    images: [THUMBS[0], THUMBS[1], THUMBS[2]],
    icon: IconBatteryFullFilled,
    gradient: "tiktok",
  },
  {
    id: "giant-figure",
    title: "Giant figure",
    subtitle: "Product hero, larger than life",
    category: "tiktok",
    kind: "image",
    images: [THUMBS[1], THUMBS[3], THUMBS[0]],
    icon: IconImagineAiFilled,
    gradient: "tiktok",
  },
  {
    id: "classic-modern",
    title: "Classic meets modern",
    subtitle: "Editorial style transfer",
    category: "commercial",
    kind: "image",
    images: [THUMBS[2], THUMBS[0], THUMBS[1]],
    icon: IconDateYearlyFilled,
    gradient: "blue",
  },
  {
    id: "couple-home",
    title: "Couple sharing home",
    subtitle: "Lifestyle story in 3 shots",
    category: "ugc",
    kind: "video",
    images: [THUMBS[3], THUMBS[2], THUMBS[0]],
    icon: IconHomeRoundDoorFilled,
    gradient: "pink",
  },
  {
    id: "unbox-hype",
    title: "Unboxing hype",
    subtitle: "Fast-cut reveal for TikTok",
    category: "tiktok",
    kind: "video",
    images: [THUMBS[0], THUMBS[2], THUMBS[3]],
    icon: IconImagineAiFilled,
    gradient: "tiktok",
  },
  {
    id: "studio-lookbook",
    title: "Studio lookbook",
    subtitle: "Clean commercial catalogue",
    category: "commercial",
    kind: "image",
    images: [THUMBS[1], THUMBS[0], THUMBS[3]],
    icon: IconDateYearlyFilled,
    gradient: "blue",
  },
];
```

```
/** 24px branded gradient badge with a white glyph — the Studio card lead tile. */
function GradientBadge({ as, gradient }: { as: LeadGlyph; gradient: BadgeGradient }) {
  return (
    <span className="relative flex items-center justify-center overflow-hidden rounded-q-250 border border-[rgba(197,197,197,0.24)] p-q-200 text-white shadow-[0_4px_4px_rgba(0,0,0,0.08),inset_0_2px_4p
      <span
        aria-hidden
        className="absolute inset-0"
        style={{ backgroundImage: BADGE_GRADIENT[gradient] }}
      />
      <span
        aria-hidden
        className="absolute inset-0 bg-gradient-to-t from-transparent to-white/20 mix-blend-overlay"
      />
      <span
        aria-hidden
        className="absolute inset-0 bg-gradient-to-t from-transparent to-white/[0.32] mix-blend-hard-light"
      />
      <Icon as={as} size="md" className="relative" />
    </span>
  );
}

const TRIPTYCH_CORNERS = [
  "rounded-tl-q-500 rounded-bl-q-500 rounded-tr-q-150 rounded-br-q-150",
  "rounded-q-150",
  "rounded-tr-q-500 rounded-br-q-500 rounded-tl-q-150 rounded-bl-q-150",
] as const;

/** Centered play badge over the media — shown for `video` templates. */
function PlayBadge() {
  return (
    <Media.Overlay placement="center">
      <span className="flex size-10 items-center justify-center rounded-q-full bg-q-transparent-dark-40 text-white backdrop-blur-sm">
        <Icon as={IconPlayFilled} size="md" />
      </span>
    </Media.Overlay>
  );
}

/**
 * Preview layout of a template tile:
 *   • `single`   — ONE full-width shot (the base tile). Default.
 *   • `triptych` — the 3-shot rounded triptych variation.
 */
export type TemplateCardVariant = "single" | "triptych";

export interface TemplateCardProps {
  template: TemplateItem;
  /** Preview layout. `single` (one full-width image) is the base; `triptych` is the 3-shot variation. */
  variant?: TemplateCardVariant;
  /** Fired by the "Try" action (wire to seed the prompt box / start a generation). */
  onTry?: (template: TemplateItem) => void;
  /** Swap the "Try" label (e.g. "Use"). */
  tryLabel?: ReactNode;
}

/**
 * A single marketing template tile — Figma Marketing-Studio gallery card
 * (7137:108927): co-brand header, a rounded preview (base = one full-width shot;
 * the `triptych` variant shows 3 shots — video templates carry a play badge),
 * and a footer with a gradient category badge, the title/subtitle, and the lime
 * "Try" CTA.
 */
export function TemplateCard({
  template,
  variant = "single",
  onTry,
  tryLabel = "Try",
}: TemplateCardProps) {
  return (
```

```
    <div className="flex flex-col gap-q-200 rounded-q-600 bg-q-transparent-light-05 p-q-200 shadow-[0_2px_6px_rgba(0,0,0,0.15)]">
      <div className="flex h-60 items-stretch gap-1.5">
        {variant === "triptych" ? (
          template.images.map((src, index) => (
            <Media
              key={index}
              ratio="auto"
              rounded="none"
              className={`min-w-0 flex-1 border border-q-border-subtle ${TRIPTYCH_CORNERS[index]}`}
            >
              <Media.Image src={src} alt={`${template.title} — shot ${index + 1}`} />
              {template.kind === "video" && index === 1 ? <PlayBadge /> : null}
            </Media>
          ))
        ) : (
          <Media
            ratio="auto"
            rounded="none"
            className="min-w-0 flex-1 rounded-q-500 border border-q-border-subtle"
          >
            <Media.Image src={template.images[0]} alt={template.title} />
            {template.kind === "video" ? <PlayBadge /> : null}
          </Media>
        )}
      </div>
      <div className="flex items-center gap-3 px-2 py-1">
        <GradientBadge as={template.icon} gradient={template.gradient} />
        <div className="flex min-w-0 flex-1 flex-col gap-1">
          <Typography as="span" variant="label-md-medium" color="primary" truncate>
            {template.title}
          </Typography>
          <Typography as="span" variant="caption-sm-regular" color="secondary" truncate>
            {template.subtitle}
          </Typography>
        </div>
        <Button variant="marketingPrimary" size="sm" onClick={() => onTry?.(template)}>
          {tryLabel}
        </Button>
      </div>
    </div>
  );
}

/* ■■ Filter tabs ■■■
```

```
  const visible = useMemo(
    () =>
      TEMPLATES.filter(
        (t) =>
          (category === "all" || t.category === category) && (kind === "all" || t.kind === kind),
      ),
    [category, kind],
  );

  return (
    <Modal.Root defaultOpen={defaultOpen}>
      <Modal.Trigger render={trigger} />
      <Modal.Content size="2xl">
        <Modal.Header flush className="px-2 py-1">
          <Tabs.Root variant="pill" value={category} onValueChange={setCategory} className="flex-1">
            <Tabs.List items={CATEGORY_TABS} />
          </Tabs.Root>
          <Modal.CloseButton />
        </Modal.Header>

        <div className="flex items-center justify-between gap-4 px-1 pb-3">
          <Tabs.Root variant="segmented" value={kind} onValueChange={setKind}>
            <Tabs.List items={TYPE_TABS} />
          </Tabs.Root>
          <Button
            variant="tertiary"
            size="sm"
            start={<Icon as={IconMagnifyingGlassOutlined} size="sm" />}
          >
            Search
          </Button>
        </div>

        <div className="min-h-0 flex-1 overflow-y-auto">
          <div className="grid grid-cols-2 gap-5 p-1">
            {visible.map((template) => (
              <TemplateCard key={template.id} template={template} onTry={onSelect} tryLabel="Use" />
            ))}
          </div>
        </div>

        <Modal.Footer>
          <Modal.FooterCaption>
            {visible.length} template
            {visible.length === 1 ? "" : "s"}
          </Modal.FooterCaption>
        </Modal.Footer>
      </Modal.Content>
    </Modal.Root>
  );
}
```

app/src/components/three-farm-scene.tsx

```
import { useEffect, useRef } from "react";
import * as THREE from "three";
type Weather = "Typical" | "Dry spell" | "Heavy rain";
export default function ThreeFarmScene({water,weather}:{water:number;weather:Weather}) {
 const mountRef=useRef<HTMLDivElement>(null),waterRef=useRef(water),weatherRef=useRef(weather);
 useEffect(()=>{waterRef.current=water;weatherRef.current=weather;},[water,weather]);
 useEffect(()=>{
  const host=mountRef.current;if(!host)return;let renderer: import("three").WebGLRenderer;let frame=0;let disposed=false;
  try{
   const THREE=requireThree();renderer=new THREE.WebGLRenderer({antialias:true,alpha:true,powerPreference:"low-power"});
  }catch{return;}
  const scene=new THREE.Scene();scene.background=null;const camera=new THREE.OrthographicCamera(-5.4,5.4,3.7,-3.7,.1,100);camera.position.set(8,10,9);camera.lookAt(0,0,0);camera.zoom=1.02;camera.updat
  renderer.setPixelRatio(Math.min(window.devicePixelRatio||1,1.6));renderer.setSize(host.clientWidth,host.clientHeight);renderer.outputColorSpace=THREE.SRGBColorSpace;renderer.setClearColor(0x000000,0
  scene.add(new THREE.HemisphereLight(0xf4f4df,0x718a69,2.1));const sun=new THREE.DirectionalLight(0xffefcf,2.5);sun.position.set(4,9,5);scene.add(sun);const fill=new THREE.DirectionalLight(0xd5e9de,.
  const world=new THREE.Group();scene.add(world);const mat=(color:number,roughness=.85)=>new THREE.MeshStandardMaterial({color,roughness});const soil=mat(0x78946c),edge=mat(0x627e5e),plotMat=mat(0x6c8
  const addBox=(w:number,h:number,d:number,x:number,y:number,z:number,material:THREE.Material,group=world)=>{const m=new THREE.Mesh(new THREE.BoxGeometry(w,h,d),material);m.position.set(x,y,z);group.a
  addBox(7.4,.25,5.5,0,-.18,0,edge);addBox(7.28,.09,5.38,0,-.03,0,soil);
  for(let row=0;row<2;row++)for(let col=0;col<3;col++){const x=(col-1)*2.15,z=(row-.5)*2.12;addBox(1.82,.12,1.65,x,.05,z,plotMat);for(let r=0;r<5;r++){addBox(1.58,.025,.035,x,.125,z-.62+r*.31,rowMat);
  for(const z of [-1.06,1.06])addBox(6.75,.035,.105,0,.02,z,waterMat);for(const x of [-1.075,1.075])addBox(.09,.035,4.2,x,.02,0,waterTop);
  // Small water tank and a miniature farmhouse with a pitched-looking roof.
  addBox(.8,.38,.68,-2.75,.22,-2.02,waterMat);addBox(.82,.09,.7,-2.75,.45,-2.02,waterTop);addBox(.78,.35,.68,2.62,.2,-2.02,houseMat);const roof=new THREE.Mesh(new THREE.ConeGeometry(.64,.48,4),roofMat
  for(const [x,z] of [[-3.15,-1.8],[-3.15,1.9],[3.15,1.9],[3.15,-1.4],[-2.9,2.15]] as number[][]){const trunk=new THREE.Mesh(new THREE.CylinderGeometry(.06,.09,.45,6),trunkMat);trunk.position.set(x,.2
  const cloudMat=mat(0xf5f4e6);const cloud=new THREE.Group();scene.add(cloud);for(const [x,y,z,r] of [[0,0,0,.32],[.3,.05,0,.24],[-.28,.03,.02,.22]] as number[][]){const puff=new THREE.Mesh(new THREE.
  const rainMat=new THREE.MeshBasicMaterial({color:0x71a9c3});const rainDrops:THREE.Mesh[]=[];for(let i=0;i<32;i++){const drop=new THREE.Mesh(new THREE.BoxGeometry(.018,.17,.018),rainMat);drop.positio
  let dragging=false,lastX=0,lastY=0;const down=(e:PointerEvent)=>{dragging=true;lastX=e.clientX;lastY=e.clientY;renderer.domElement.setPointerCapture?.(e.pointerId);renderer.domElement.style.cursor='
  const resize=new ResizeObserver(()=>{if(!host.clientWidth||!host.clientHeight)return;renderer.setSize(host.clientWidth,host.clientHeight);});resize.observe(host);
  const clock=new THREE.Clock();const animate=()=>{if(disposed)return;frame=requestAnimationFrame(animate);const t=clock.getElapsedTime();cloud.position.x=-3.4+Math.sin(t*.13)*.15;cloud2.position.x=3.
  return()=>{disposed=true;cancelAnimationFrame(frame);resize.disconnect();renderer.domElement.removeEventListener("pointerdown",down);renderer.domElement.removeEventListener("pointermove",move);rende
 },[]);
 return <div className="three-farm-viewport"><img className="farm-svg-image" src="/farmverse-farm.svg" alt="Isometric miniature farm with crop plots, irrigation channels and a farmhouse"/><div classNa
}
function requireThree(){return THREE;}
```

app/src/components/ui/accordion.tsx

```
import * as React from "react"
import { Accordion as AccordionPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { ChevronDownIcon, ChevronUpIcon } from "lucide-react"

function Accordion({
  className,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Root>) {
  return (
    <AccordionPrimitive.Root
      data-slot="accordion"
      className={cn("flex w-full flex-col", className)}
      {...props}
    />
  )
}

function AccordionItem({
  className,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Item>) {
  return (
    <AccordionPrimitive.Item
      data-slot="accordion-item"
      className={cn("not-last:border-b", className)}
      {...props}
    />
  )
}

function AccordionTrigger({
  className,
  children,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Trigger>) {
  return (
    <AccordionPrimitive.Header className="flex">
      <AccordionPrimitive.Trigger
        data-slot="accordion-trigger"
        className={cn(
          "group/accordion-trigger relative flex flex-1 items-start justify-between rounded-lg border border-transparent py-2.5 text-left text-sm font-medium transition-all outline-none hover:underlir
          className
        )}
        {...props}
      >
        {children}
        <ChevronDownIcon data-slot="accordion-trigger-icon" className="pointer-events-none shrink-0 group-aria-expanded/accordion-trigger:hidden" />
        <ChevronUpIcon data-slot="accordion-trigger-icon" className="pointer-events-none hidden shrink-0 group-aria-expanded/accordion-trigger:inline" />
      </AccordionPrimitive.Trigger>
    </AccordionPrimitive.Header>
  )
}

function AccordionContent({
  className,
  children,
  ...props
}: React.ComponentProps<typeof AccordionPrimitive.Content>) {
  return (
    <AccordionPrimitive.Content
      data-slot="accordion-content"
      className="overflow-hidden text-sm data-open:animate-accordion-down data-closed:animate-accordion-up"
      {...props}
    >
      <div
        className={cn(
```

```
          "h-(--radix-accordion-content-height) pt-0 pb-2.5 [&_a]:underline [&_a]:underline-offset-3 [&_a]:hover:text-foreground [&_p:not(:last-child)]:mb-4",
          className
        )}
      >
        {children}
      </div>
    </AccordionPrimitive.Content>
  )
}

export { Accordion, AccordionItem, AccordionTrigger, AccordionContent }
```

app/src/components/ui/alert-dialog.tsx

```
import * as React from "react"
import { AlertDialog as AlertDialogPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"

function AlertDialog({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Root>) {
  return <AlertDialogPrimitive.Root data-slot="alert-dialog" {...props} />
}

function AlertDialogTrigger({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Trigger>) {
  return (
    <AlertDialogPrimitive.Trigger data-slot="alert-dialog-trigger" {...props} />
  )
}

function AlertDialogPortal({
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Portal>) {
  return (
    <AlertDialogPrimitive.Portal data-slot="alert-dialog-portal" {...props} />
  )
}

function AlertDialogOverlay({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Overlay>) {
  return (
    <AlertDialogPrimitive.Overlay
      data-slot="alert-dialog-overlay"
      className={cn(
        "fixed inset-0 z-50 bg-black/10 duration-100 supports-backdrop-filter:backdrop-blur-xs data-open:animate-in data-open:fade-in-0 data-closed:animate-out data-closed:fade-out-0",
        className
      )}
      {...props}
    />
  )
}

function AlertDialogContent({
  className,
  size = "default",
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Content> & {
  size?: "default" | "sm"
}) {
  return (
    <AlertDialogPortal>
      <AlertDialogOverlay />
      <AlertDialogPrimitive.Content
        data-slot="alert-dialog-content"
        data-size={size}
        className={cn(
          "group/alert-dialog-content fixed top-1/2 left-1/2 z-50 grid w-full -translate-x-1/2 -translate-y-1/2 gap-4 rounded-xl bg-popover p-4 text-popover-foreground ring-1 ring-foreground/10 durati
          className
        )}
        {...props}
      />
    </AlertDialogPortal>
  )
}

function AlertDialogHeader({
```

```
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="alert-dialog-header"
      className={cn(
        "grid grid-rows-[auto_1fr] place-items-center gap-1.5 text-center has-data-[slot=alert-dialog-media]:grid-rows-[auto_auto_1fr] has-data-[slot=alert-dialog-media]:gap-x-4 sm:group-data-[size=de
        className
      )}
      {...props}
    />
  )
}

function AlertDialogFooter({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="alert-dialog-footer"
      className={cn(
        "-mx-4 -mb-4 flex flex-col-reverse gap-2 rounded-b-xl border-t bg-muted/50 p-4 group-data-[size=sm]/alert-dialog-content:grid group-data-[size=sm]/alert-dialog-content:grid-cols-2 sm:flex-row
        className
      )}
      {...props}
    />
  )
}

function AlertDialogMedia({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="alert-dialog-media"
      className={cn(
        "mb-2 inline-flex size-10 items-center justify-center rounded-md bg-muted sm:group-data-[size=default]/alert-dialog-content:row-span-2 *:[svg:not([class*='size-'])]:size-6",
        className
      )}
      {...props}
    />
  )
}

function AlertDialogTitle({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Title>) {
  return (
    <AlertDialogPrimitive.Title
      data-slot="alert-dialog-title"
      className={cn(
        "font-heading text-base font-medium sm:group-data-[size=default]/alert-dialog-content:group-has-data-[slot=alert-dialog-media]/alert-dialog-content:col-start-2",
        className
      )}
      {...props}
    />
  )
}

function AlertDialogDescription({
  className,
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Description>) {
  return (
    <AlertDialogPrimitive.Description
      data-slot="alert-dialog-description"
```

```
      className={cn(
        "text-sm text-balance text-muted-foreground md:text-pretty *:[a]:underline *:[a]:underline-offset-3 *:[a]:hover:text-foreground",
        className
      )}
      {...props}
    />
  )
}

function AlertDialogAction({
  className,
  variant = "default",
  size = "default",
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Action> &
  Pick<React.ComponentProps<typeof Button>, "variant" | "size">) {
  return (
    <Button variant={variant} size={size} asChild>
      <AlertDialogPrimitive.Action
        data-slot="alert-dialog-action"
        className={cn(className)}
        {...props}
      />
    </Button>
  )
}

function AlertDialogCancel({
  className,
  variant = "outline",
  size = "default",
  ...props
}: React.ComponentProps<typeof AlertDialogPrimitive.Cancel> &
  Pick<React.ComponentProps<typeof Button>, "variant" | "size">) {
  return (
    <Button variant={variant} size={size} asChild>
      <AlertDialogPrimitive.Cancel
        data-slot="alert-dialog-cancel"
        className={cn(className)}
        {...props}
      />
    </Button>
  )
}

export {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
  AlertDialogHeader,
  AlertDialogMedia,
  AlertDialogOverlay,
  AlertDialogPortal,
  AlertDialogTitle,
  AlertDialogTrigger,
}
```

app/src/components/ui/alert.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@/lib/utils"

const alertVariants = cva(
  "group/alert relative grid w-full gap-0.5 rounded-lg border px-2.5 py-2 text-left text-sm has-data-[slot=alert-action]:relative has-data-[slot=alert-action]:pr-18 has-[>svg]:grid-cols-[auto_1fr] has
  {
    variants: {
      variant: {
        default: "bg-card text-card-foreground",
        destructive:
          "bg-card text-destructive *:data-[slot=alert-description]:text-destructive/90 *:[svg]:text-current",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)

function Alert({
  className,
  variant,
  ...props
}: React.ComponentProps<"div"> & VariantProps<typeof alertVariants>) {
  return (
    <div
      data-slot="alert"
      role="alert"
      className={cn(alertVariants({ variant }), className)}
      {...props}
    />
  )
}

function AlertTitle({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="alert-title"
      className={cn(
        "font-medium group-has-[>svg]/alert:col-start-2 [&_a]:underline [&_a]:underline-offset-3 [&_a]:hover:text-foreground",
        className
      )}
      {...props}
    />
  )
}

function AlertDescription({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="alert-description"
      className={cn(
        "text-sm text-balance text-muted-foreground md:text-pretty [&_a]:underline [&_a]:underline-offset-3 [&_a]:hover:text-foreground [&_p:not(:last-child)]:mb-4",
        className
      )}
      {...props}
    />
  )
}

function AlertAction({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
```

```
      data-slot="alert-action"
      className={cn("absolute top-2 right-2", className)}
      {...props}
    />
  )
}

export { Alert, AlertTitle, AlertDescription, AlertAction }
```

app/src/components/ui/aspect-ratio.tsx

```
"use client"

import { AspectRatio as AspectRatioPrimitive } from "radix-ui"

function AspectRatio({
  ...props
}: React.ComponentProps<typeof AspectRatioPrimitive.Root>) {
  return <AspectRatioPrimitive.Root data-slot="aspect-ratio" {...props} />
}

export { AspectRatio }
```

app/src/components/ui/attachment.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Slot } from "radix-ui"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"

const attachmentVariants = cva(
  "group/attachment relative flex w-fit max-w-full min-w-0 shrink-0 flex-wrap rounded-xl border bg-card text-card-foreground transition-colors focus-within:ring-1 focus-within:ring-ring/50 has-[>a,>bu
  {
    variants: {
      size: {
        default:
          "gap-2 text-sm has-data-[slot=attachment-content]:px-2.5 has-data-[slot=attachment-content]:py-2 has-data-[slot=attachment-media]:p-2",
        sm: "gap-2.5 text-xs has-data-[slot=attachment-content]:px-2 has-data-[slot=attachment-content]:py-1.5 has-data-[slot=attachment-media]:p-1.5",
        xs: "gap-1.5 rounded-lg text-xs has-data-[slot=attachment-content]:px-1.5 has-data-[slot=attachment-content]:py-1 has-data-[slot=attachment-media]:p-1",
      },
      orientation: {
        horizontal: "min-w-40 items-center",
        vertical: "w-24 flex-col has-data-[slot=attachment-content]:w-30",
      },
    },
  }
)

function Attachment({
  className,
  state = "done",
  size = "default",
  orientation = "horizontal",
  ...props
}: React.ComponentProps<"div"> &
  VariantProps<typeof attachmentVariants> & {
    state?: "idle" | "uploading" | "processing" | "error" | "done"
  }) {
  return (
    <div
      data-slot="attachment"
      data-state={state}
      data-size={size}
      data-orientation={orientation}
      className={cn(attachmentVariants({ size, orientation }), className)}
      {...props}
    />
  )
}

const attachmentMediaVariants = cva(
  "relative flex aspect-square w-10 shrink-0 items-center justify-center overflow-hidden rounded-lg bg-muted text-foreground group-data-[orientation=vertical]/attachment:w-full group-data-[size=sm]/at
  {
    variants: {
      variant: {
        icon: "",
        image:
          "opacity-60 group-data-[state=done]/attachment:opacity-100 group-data-[state=idle]/attachment:opacity-100 *:[img]:aspect-square *:[img]:w-full *:[img]:object-cover",
      },
    },
    defaultVariants: {
      variant: "icon",
    },
  }
)

function AttachmentMedia({
  className,
  variant = "icon",
  ...props
}: React.ComponentProps<"div"> & VariantProps<typeof attachmentMediaVariants>) {
```

```
  return (
    <div
      data-slot="attachment-media"
      data-variant={variant}
      className={cn(attachmentMediaVariants({ variant }), className)}
      {...props}
    />
  )
}

function AttachmentContent({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="attachment-content"
      className={cn(
        "max-w-full min-w-0 flex-1 leading-tight group-data-[orientation=vertical]/attachment:px-1",
        className
      )}
      {...props}
    />
  )
}

function AttachmentTitle({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="attachment-title"
      className={cn(
        "block max-w-full min-w-0 truncate font-medium group-data-[state=processing]/attachment:shimmer group-data-[state=uploading]/attachment:shimmer",
        className
      )}
      {...props}
    />
  )
}

function AttachmentDescription({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="attachment-description"
      className={cn(
        "mt-0.5 block min-w-0 truncate text-xs text-muted-foreground group-data-[state=error]/attachment:text-destructive/80",
        "max-w-full",
        className
      )}
      {...props}
    />
  )
}

function AttachmentActions({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="attachment-actions"
      className={cn(
        "relative z-20 flex shrink-0 items-center group-data-[orientation=vertical]/attachment:absolute group-data-[orientation=vertical]/attachment:top-3 group-data-[orientation=vertical]/attachment:
        className
      )}
```

```
      {...props}
    />
  )
}

function AttachmentAction({
  className,
  variant,
  size = "icon-xs",
  ...props
}: React.ComponentProps<typeof Button>) {
  return (
    <Button
      data-slot="attachment-action"
      variant={variant ?? "ghost"}
      size={size}
      className={cn(className)}
      {...props}
    />
  )
}

function AttachmentTrigger({
  className,
  asChild = false,
  type,
  ...props
}: React.ComponentProps<"button"> & {
  asChild?: boolean
}) {
  const Comp = asChild ? Slot.Root : "button"

  return (
    <Comp
      data-slot="attachment-trigger"
      type={asChild ? undefined : (type ?? "button")}
      className={cn("absolute inset-0 z-10 outline-none", className)}
      {...props}
    />
  )
}

function AttachmentGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="attachment-group"
      className={cn(
        "flex min-w-0 scroll-fade-x snap-x snap-mandatory scroll-px-1 scrollbar-none gap-3 overflow-x-auto overscroll-x-contain py-1 *:data-[slot=attachment]:flex-none *:data-[slot=attachment]:snap-st
        className
      )}
      {...props}
    />
  )
}

export {
  Attachment,
  AttachmentGroup,
  AttachmentMedia,
  AttachmentContent,
  AttachmentTitle,
  AttachmentDescription,
  AttachmentActions,
  AttachmentAction,
  AttachmentTrigger,
}
```

app/src/components/ui/avatar.tsx

```
import * as React from "react"
import { Avatar as AvatarPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function Avatar({
  className,
  size = "default",
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Root> & {
  size?: "default" | "sm" | "lg"
}) {
  return (
    <AvatarPrimitive.Root
      data-slot="avatar"
      data-size={size}
      className={cn(
        "group/avatar relative flex size-8 shrink-0 rounded-full select-none after:absolute after:inset-0 after:rounded-full after:border after:border-border after:mix-blend-darken data-[size=lg]:size
        className
      )}
      {...props}
    />
  )
}

function AvatarImage({
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Image>) {
  return (
    <AvatarPrimitive.Image
      data-slot="avatar-image"
      className={cn(
        "aspect-square size-full rounded-full object-cover",
        className
      )}
      {...props}
    />
  )
}

function AvatarFallback({
  className,
  ...props
}: React.ComponentProps<typeof AvatarPrimitive.Fallback>) {
  return (
    <AvatarPrimitive.Fallback
      data-slot="avatar-fallback"
      className={cn(
        "flex size-full items-center justify-center rounded-full bg-muted text-sm text-muted-foreground group-data-[size=sm]/avatar:text-xs",
        className
      )}
      {...props}
    />
  )
}

function AvatarBadge({ className, ...props }: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="avatar-badge"
      className={cn(
        "absolute right-0 bottom-0 z-10 inline-flex items-center justify-center rounded-full bg-primary text-primary-foreground bg-blend-color ring-2 ring-background select-none",
        "group-data-[size=sm]/avatar:size-2 group-data-[size=sm]/avatar:[&>svg]:hidden",
        "group-data-[size=default]/avatar:size-2.5 group-data-[size=default]/avatar:[&>svg]:size-2",
        "group-data-[size=lg]/avatar:size-3 group-data-[size=lg]/avatar:[&>svg]:size-2",
        className
      )}
```

```
      {...props}
    />
  )
}

function AvatarGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="avatar-group"
      className={cn(
        "group/avatar-group flex -space-x-2 *:data-[slot=avatar]:ring-2 *:data-[slot=avatar]:ring-background",
        className
      )}
      {...props}
    />
  )
}

function AvatarGroupCount({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="avatar-group-count"
      className={cn(
        "relative flex size-8 shrink-0 items-center justify-center rounded-full bg-muted text-sm text-muted-foreground ring-2 ring-background group-has-data-[size=lg]/avatar-group:size-10 group-has-da
        className
      )}
      {...props}
    />
  )
}

export {
  Avatar,
  AvatarImage,
  AvatarFallback,
  AvatarGroup,
  AvatarGroupCount,
  AvatarBadge,
}
```

app/src/components/ui/badge.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Slot } from "radix-ui"

import { cn } from "@/lib/utils"

const badgeVariants = cva(
  "group/badge inline-flex h-5 w-fit shrink-0 items-center justify-center gap-1 overflow-hidden rounded-4xl border border-transparent px-2 py-0.5 text-xs font-medium whitespace-nowrap transition-all f
  {
    variants: {
      variant: {
        default: "bg-primary text-primary-foreground [a]:hover:bg-primary/80",
        secondary:
          "bg-secondary text-secondary-foreground [a]:hover:bg-secondary/80",
        destructive:
          "bg-destructive/10 text-destructive focus-visible:ring-destructive/20 dark:bg-destructive/20 dark:focus-visible:ring-destructive/40 [a]:hover:bg-destructive/20",
        outline:
          "border-border text-foreground [a]:hover:bg-muted [a]:hover:text-muted-foreground",
        ghost:
          "hover:bg-muted hover:text-muted-foreground dark:hover:bg-muted/50",
        link: "text-primary underline-offset-4 hover:underline",
        /* Figma Badge (page 526:276): skewed Space Grotesk caps chips */
        blue: "badge-gradient-blue -skew-x-[11deg] gap-0.5 rounded-[2px] pl-1 pr-1.5 font-grotesk font-bold text-white uppercase",
        pink: "badge-gradient-pink -skew-x-[11deg] gap-0.5 rounded-[2px] pl-1 pr-1.5 font-grotesk font-bold text-white uppercase",
        purple:
          "badge-gradient-purple -skew-x-[11deg] gap-0.5 rounded-[2px] pl-1 pr-1.5 font-grotesk font-bold text-white uppercase",
        lime: "-skew-x-[11deg] gap-0.5 rounded-[2px] bg-primary pl-1 pr-1.5 font-grotesk font-bold text-[#1a1a1a] uppercase",
      },
      size: {
        default: "",
        xs: "h-[15px] text-xs",
        sm: "h-5 text-[16px]",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

function Badge({
  className,
  variant = "default",
  size = "default",
  asChild = false,
  ...props
}: React.ComponentProps<"span"> &
  VariantProps<typeof badgeVariants> & { asChild?: boolean }) {
  const Comp = asChild ? Slot.Root : "span"

  return (
    <Comp
      data-slot="badge"
      data-variant={variant}
      data-size={size}
      className={cn(badgeVariants({ variant, size }), className)}
      {...props}
    />
  )
}

export { Badge, badgeVariants }
```

app/src/components/ui/breadcrumb.tsx

```
import * as React from "react"
import { Slot } from "radix-ui"

import { cn } from "@/lib/utils"
import { ChevronRightIcon, MoreHorizontalIcon } from "lucide-react"

function Breadcrumb({ className, ...props }: React.ComponentProps<"nav">) {
  return (
    <nav
      aria-label="breadcrumb"
      data-slot="breadcrumb"
      className={cn(className)}
      {...props}
    />
  )
}

function BreadcrumbList({ className, ...props }: React.ComponentProps<"ol">) {
  return (
    <ol
      data-slot="breadcrumb-list"
      className={cn(
        "flex flex-wrap items-center gap-1.5 text-sm wrap-break-word text-muted-foreground",
        className
      )}
      {...props}
    />
  )
}

function BreadcrumbItem({ className, ...props }: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="breadcrumb-item"
      className={cn("inline-flex items-center gap-1", className)}
      {...props}
    />
  )
}

function BreadcrumbLink({
  asChild,
  className,
  ...props
}: React.ComponentProps<"a"> & {
  asChild?: boolean
}) {
  const Comp = asChild ? Slot.Root : "a"

  return (
    <Comp
      data-slot="breadcrumb-link"
      className={cn("transition-colors hover:text-foreground", className)}
      {...props}
    />
  )
}

function BreadcrumbPage({ className, ...props }: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="breadcrumb-page"
      role="link"
      aria-disabled="true"
      aria-current="page"
      className={cn("font-normal text-foreground", className)}
      {...props}
    />
```

```
  )
}

function BreadcrumbSeparator({
  children,
  className,
  ...props
}: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="breadcrumb-separator"
      role="presentation"
      aria-hidden="true"
      className={cn("[&>svg]:size-3.5", className)}
      {...props}
    >
      {children ?? (
        <ChevronRightIcon />
      )}
    </li>
  )
}

function BreadcrumbEllipsis({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="breadcrumb-ellipsis"
      role="presentation"
      aria-hidden="true"
      className={cn(
        "flex size-5 items-center justify-center [&>svg]:size-4",
        className
      )}
      {...props}
    >
      <MoreHorizontalIcon
      />
      <span className="sr-only">More</span>
    </span>
  )
}

export {
  Breadcrumb,
  BreadcrumbList,
  BreadcrumbItem,
  BreadcrumbLink,
  BreadcrumbPage,
  BreadcrumbSeparator,
  BreadcrumbEllipsis,
}
```

app/src/components/ui/bubble.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Slot } from "radix-ui"

import { cn } from "@/lib/utils"

function BubbleGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="bubble-group"
      className={cn("flex min-w-0 flex-col gap-2", className)}
      {...props}
    />
  )
}

const bubbleVariants = cva(
  "group/bubble relative flex w-fit max-w-[80%] min-w-0 flex-col gap-1 group-data-[align=end]/message:self-end data-[align=end]:self-end data-[variant=ghost]:max-w-full",
  {
    variants: {
      variant: {
        default:
          "*:data-[slot=bubble-content]:bg-primary *:data-[slot=bubble-content]:text-primary-foreground [&>[data-slot=bubble-content]:is(button,a):hover]:bg-primary/80",
        secondary:
          "*:data-[slot=bubble-content]:bg-secondary *:data-[slot=bubble-content]:text-secondary-foreground [&>[data-slot=bubble-content]:is(button,a):hover]:bg-[color-mix(in_oklch,var(--secondary),va
        muted:
          "*:data-[slot=bubble-content]:bg-muted [&>[data-slot=bubble-content]:is(button,a):hover]:bg-[color-mix(in_oklch,var(--muted),var(--foreground)_5%)]",
        tinted:
          "*:data-[slot=bubble-content]:bg-[oklch(from_var(--primary)_0.93_calc(c*0.4)_h)] *:data-[slot=bubble-content]:text-foreground dark:*:data-[slot=bubble-content]:bg-[oklch(from_var(--primary)_
        outline:
          "*:data-[slot=bubble-content]:border-border *:data-[slot=bubble-content]:bg-background [&>[data-slot=bubble-content]:is(button,a):hover]:bg-muted [&>[data-slot=bubble-content]:is(button,a):h
        ghost:
          "border-none *:data-[slot=bubble-content]:rounded-none *:data-[slot=bubble-content]:bg-transparent *:data-[slot=bubble-content]:p-0 [&>[data-slot=bubble-content]:is(button,a):hover]:bg-muted
        destructive:
          "*:data-[slot=bubble-content]:bg-destructive/10 *:data-[slot=bubble-content]:text-destructive dark:*:data-[slot=bubble-content]:bg-destructive/20 [&>[data-slot=bubble-content]:is(button,a):h
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)

function Bubble({
  variant = "default",
  align = "start",
  className,
  ...props
}: React.ComponentProps<"div"> &
  VariantProps<typeof bubbleVariants> & {
    align?: "start" | "end"
  }) {
  return (
    <div
      data-slot="bubble"
      data-variant={variant}
      data-align={align}
      className={cn(bubbleVariants({ variant }), className)}
      {...props}
    />
  )
}

function BubbleContent({
  asChild = false,
  className,
  ...props
}: React.ComponentProps<"div"> & {
```

```
  asChild?: boolean
}) {
  const Comp = asChild ? Slot.Root : "div"

  return (
    <Comp
      data-slot="bubble-content"
      className={cn(
        "w-fit max-w-full min-w-0 overflow-hidden rounded-xl border border-transparent px-3 py-2 text-sm leading-relaxed wrap-break-word group-data-[align=end]/bubble:self-end [button]:text-left [butt
        className
      )}
      {...props}
    />
  )
}

const bubbleReactionsVariants = cva(
  "absolute z-10 flex w-fit shrink-0 items-center justify-center gap-1 rounded-full bg-muted px-1.5 py-0.5 text-sm ring-3 ring-card has-[button]:p-0",
  {
    variants: {
      side: {
        top: "top-0 -translate-y-3/4",
        bottom: "bottom-0 translate-y-3/4",
      },
      align: {
        start: "left-3",
        end: "right-3",
      },
    },
    defaultVariants: {
      side: "bottom",
      align: "end",
    },
  }
)

function BubbleReactions({
  side = "bottom",
  align = "end",
  className,
  ...props
}: React.ComponentProps<"div"> & {
  align?: "start" | "end"
  side?: "top" | "bottom"
}) {
  return (
    <div
      data-slot="bubble-reactions"
      data-align={align}
      data-side={side}
      className={cn(bubbleReactionsVariants({ side, align }), className)}
      {...props}
    />
  )
}

export { BubbleGroup, Bubble, BubbleContent, BubbleReactions }
```

app/src/components/ui/button-group.tsx

```
import { cva, type VariantProps } from "class-variance-authority"
import { Slot } from "radix-ui"

import { cn } from "@/lib/utils"
import { Separator } from "@/components/ui/separator"

const buttonGroupVariants = cva(
  "group/button-group flex w-fit items-stretch *:focus-visible:relative *:focus-visible:z-10 has-[>[data-slot=button-group]]:gap-2 has-[select[aria-hidden=true]:last-child]:[&>[data-slot=select-trigge
  {
    variants: {
      orientation: {
        horizontal:
          "[&>*:not(:first-child)]:rounded-l-none [&>*:not(:first-child)]:border-l-0 [&>*:not(:last-child)]:rounded-r-none [&>[data-slot]:not(:has(~[data-slot]))]:rounded-r-lg!",
        vertical:
          "flex-col [&>*:not(:first-child)]:rounded-t-none [&>*:not(:first-child)]:border-t-0 [&>*:not(:last-child)]:rounded-b-none [&>[data-slot]:not(:has(~[data-slot]))]:rounded-b-lg!",
      },
    },
    defaultVariants: {
      orientation: "horizontal",
    },
  }
)

function ButtonGroup({
  className,
  orientation,
  ...props
}: React.ComponentProps<"div"> & VariantProps<typeof buttonGroupVariants>) {
  return (
    <div
      role="group"
      data-slot="button-group"
      data-orientation={orientation}
      className={cn(buttonGroupVariants({ orientation }), className)}
      {...props}
    />
  )
}

function ButtonGroupText({
  className,
  asChild = false,
  ...props
}: React.ComponentProps<"div"> & {
  asChild?: boolean
}) {
  const Comp = asChild ? Slot.Root : "div"

  return (
    <Comp
      className={cn(
        "flex items-center gap-2 rounded-lg border bg-muted px-2.5 text-sm font-medium [&_svg]:pointer-events-none [&_svg:not([class*='size-'])]:size-4",
        className
      )}
      {...props}
    />
  )
}

function ButtonGroupSeparator({
  className,
  orientation = "vertical",
  ...props
}: React.ComponentProps<typeof Separator>) {
  return (
    <Separator
      data-slot="button-group-separator"
      orientation={orientation}
```

```
      className={cn(
        "relative self-stretch bg-input data-horizontal:mx-px data-horizontal:w-auto data-vertical:my-px data-vertical:h-auto",
        className
      )}
      {...props}
    />
  )
}

export {
  ButtonGroup,
  ButtonGroupSeparator,
  ButtonGroupText,
  buttonGroupVariants,
}
```

app/src/components/ui/button.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Slot } from "radix-ui"

import { cn } from "@/lib/utils"
import { Spinner } from "@/components/ui/spinner"

const buttonVariants = cva(
  "group/button inline-flex shrink-0 items-center justify-center rounded-lg border border-transparent bg-clip-padding text-sm font-medium whitespace-nowrap transition-all outline-none select-none focu
  {
    variants: {
      variant: {
        default:
          "bg-primary text-primary-foreground hover:brightness-90 active:brightness-[.8]",
        outline:
          "border-border bg-background hover:bg-muted hover:text-foreground aria-expanded:bg-muted aria-expanded:text-foreground dark:border-input dark:bg-input/30 dark:hover:bg-input/50",
        secondary:
          "bg-secondary text-secondary-foreground hover:bg-accent aria-expanded:bg-secondary aria-expanded:text-secondary-foreground",
        ghost:
          "hover:bg-muted hover:text-foreground aria-expanded:bg-muted aria-expanded:text-foreground dark:hover:bg-muted/50",
        destructive:
          "bg-destructive/10 text-destructive hover:bg-destructive/20 focus-visible:border-destructive/40 focus-visible:ring-destructive/20 dark:bg-destructive/20 dark:hover:bg-destructive/30 dark:foc
        link: "text-primary underline-offset-4 hover:underline",
      },
      /* sizes match higgsfield.ai .button-sm/-md/-lg exactly */
      size: {
        default:
          "h-10 min-w-10 gap-1.5 rounded-[0.625rem] px-2.5 has-data-[icon=inline-end]:pr-2 has-data-[icon=inline-start]:pl-2",
        xs: "h-6 gap-1 rounded-md px-2 text-xs in-data-[slot=button-group]:rounded-lg has-data-[icon=inline-end]:pr-1.5 has-data-[icon=inline-start]:pl-1.5 [&_svg:not([class*='size-'])]:size-3",
        sm: "h-8 min-w-8 gap-1.5 rounded-lg px-2 text-xs in-data-[slot=button-group]:rounded-lg has-data-[icon=inline-end]:pr-1.5 has-data-[icon=inline-start]:pl-1.5 [&_svg:not([class*='size-'])]:size
        lg: "h-12 min-w-12 gap-2 rounded-xl px-3 text-base has-data-[icon=inline-end]:pr-2.5 has-data-[icon=inline-start]:pl-2.5",
        icon: "size-10 rounded-[0.625rem]",
        "icon-xs":
          "size-6 rounded-md in-data-[slot=button-group]:rounded-lg [&_svg:not([class*='size-'])]:size-3",
        "icon-sm": "size-8 rounded-lg in-data-[slot=button-group]:rounded-lg",
        "icon-lg": "size-12 rounded-xl",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

function Button({
  className,
  variant = "default",
  size = "default",
  asChild = false,
  loading = false,
  disabled,
  children,
  ...props
}: React.ComponentProps<"button"> &
  VariantProps<typeof buttonVariants> & {
    asChild?: boolean
    loading?: boolean
  }) {
  const Comp = asChild ? Slot.Root : "button"

  return (
    <Comp
      data-slot="button"
      data-variant={variant}
      data-size={size}
      aria-busy={loading || undefined}
      data-loading={loading || undefined}
```

```
      disabled={disabled || loading}
      className={cn(
        buttonVariants({ variant, size, className }),
        /* loading = "working", not "unavailable": keep full opacity, keep width */
        loading && "pointer-events-none relative opacity-100!"
      )}
      {...props}
    >
      {/* ponytail: asChild expects a single child, so it skips the spinner overlay */}
      {loading && !asChild ? (
        <>
          <span className="absolute inset-0 grid place-items-center">
            <Spinner />
          </span>
          <span aria-hidden className="invisible contents">
            {children}
          </span>
        </>
      ) : (
        children
      )}
    </Comp>
  )
}

export { Button, buttonVariants }
```

app/src/components/ui/calendar.tsx

```
"use client"

import * as React from "react"
import {
  DayPicker,
  getDefaultClassNames,
  type DayButton,
  type Locale,
} from "react-day-picker"

import { cn } from "@/lib/utils"
import { Button, buttonVariants } from "@/components/ui/button"
import { ChevronLeftIcon, ChevronRightIcon, ChevronDownIcon } from "lucide-react"

function Calendar({
  className,
  classNames,
  showOutsideDays = true,
  captionLayout = "label",
  buttonVariant = "ghost",
  locale,
  formatters,
  components,
  ...props
}: React.ComponentProps<typeof DayPicker> & {
  buttonVariant?: React.ComponentProps<typeof Button>["variant"]
}) {
  const defaultClassNames = getDefaultClassNames()

  return (
    <DayPicker
      showOutsideDays={showOutsideDays}
      className={cn(
        "group/calendar bg-background p-2 [--cell-radius:var(--radius-md)] [--cell-size:--spacing(7)] in-data-[slot=card-content]:bg-transparent in-data-[slot=popover-content]:bg-transparent",
        String.raw`rtl:**:[.rdp-button\_next>svg]:rotate-180`,
        String.raw`rtl:**:[.rdp-button\_previous>svg]:rotate-180`,
        className
      )}
      captionLayout={captionLayout}
      locale={locale}
      formatters={{
        formatMonthDropdown: (date) =>
          date.toLocaleString(locale?.code, { month: "short" }),
        ...formatters,
      }}
      classNames={{
        root: cn("w-fit", defaultClassNames.root),
        months: cn(
          "relative flex flex-col gap-4 md:flex-row",
          defaultClassNames.months
        ),
        month: cn("flex w-full flex-col gap-4", defaultClassNames.month),
        nav: cn(
          "absolute inset-x-0 top-0 flex w-full items-center justify-between gap-1",
          defaultClassNames.nav
        ),
        button_previous: cn(
          buttonVariants({ variant: buttonVariant }),
          "size-(--cell-size) p-0 select-none aria-disabled:opacity-50",
          defaultClassNames.button_previous
        ),
        button_next: cn(
          buttonVariants({ variant: buttonVariant }),
          "size-(--cell-size) p-0 select-none aria-disabled:opacity-50",
          defaultClassNames.button_next
        ),
        month_caption: cn(
          "flex h-(--cell-size) w-full items-center justify-center px-(--cell-size)",
```

```
      defaultClassNames.month_caption
    ),
    dropdowns: cn(
      "flex h-(--cell-size) w-full items-center justify-center gap-1.5 text-sm font-medium",
      defaultClassNames.dropdowns
    ),
    dropdown_root: cn(
      "relative rounded-(--cell-radius)",
      defaultClassNames.dropdown_root
    ),
    dropdown: cn(
      "absolute inset-0 bg-popover opacity-0",
      defaultClassNames.dropdown
    ),
    caption_label: cn(
      "font-medium select-none",
      captionLayout === "label"
        ? "text-sm"
        : "flex items-center gap-1 rounded-(--cell-radius) text-sm [&>svg]:size-3.5 [&>svg]:text-muted-foreground",
      defaultClassNames.caption_label
    ),
    month_grid: cn("w-full border-collapse", defaultClassNames.month_grid),
    weekdays: cn("flex", defaultClassNames.weekdays),
    weekday: cn(
      "flex-1 rounded-(--cell-radius) text-[0.8rem] font-normal text-muted-foreground select-none",
      defaultClassNames.weekday
    ),
    week: cn("mt-2 flex w-full", defaultClassNames.week),
    week_number_header: cn(
      "w-(--cell-size) select-none",
      defaultClassNames.week_number_header
    ),
    week_number: cn(
      "text-[0.8rem] text-muted-foreground select-none",
      defaultClassNames.week_number
    ),
    day: cn(
      "group/day relative aspect-square h-full w-full rounded-(--cell-radius) p-0 text-center select-none [&:last-child[data-selected=true]_button]:rounded-r-(--cell-radius)",
      props.showWeekNumber
        ? "[&:nth-child(2)[data-selected=true]_button]:rounded-l-(--cell-radius)"
        : "[&:first-child[data-selected=true]_button]:rounded-l-(--cell-radius)",
      defaultClassNames.day
    ),
    range_start: cn(
      "relative isolate z-0 rounded-l-(--cell-radius) bg-muted after:absolute after:inset-y-0 after:right-0 after:w-4 after:bg-muted",
      defaultClassNames.range_start
    ),
    range_middle: cn("rounded-none", defaultClassNames.range_middle),
    range_end: cn(
      "relative isolate z-0 rounded-r-(--cell-radius) bg-muted after:absolute after:inset-y-0 after:left-0 after:w-4 after:bg-muted",
      defaultClassNames.range_end
    ),
    today: cn(
      "rounded-(--cell-radius) bg-muted text-foreground data-[selected=true]:rounded-none",
      defaultClassNames.today
    ),
    outside: cn(
      "text-muted-foreground aria-selected:text-muted-foreground",
      defaultClassNames.outside
    ),
    disabled: cn(
      "text-muted-foreground opacity-50",
      defaultClassNames.disabled
    ),
    hidden: cn("invisible", defaultClassNames.hidden),
    ...classNames,
  }}
  components={{
    Root: ({ className, rootRef, ...props }) => {
      return (
```

```
            <div
              data-slot="calendar"
              ref={rootRef}
              className={cn(className)}
              {...props}
            />
          )
        },
        Chevron: ({ className, orientation, ...props }) => {
          if (orientation === "left") {
            return (
              <ChevronLeftIcon className={cn("size-4", className)} {...props} />
            )
          }

          if (orientation === "right") {
            return (
              <ChevronRightIcon className={cn("size-4", className)} {...props} />
            )
          }

          return (
            <ChevronDownIcon className={cn("size-4", className)} {...props} />
          )
        },
        DayButton: ({ ...props }) => (
          <CalendarDayButton locale={locale} {...props} />
        ),
        WeekNumber: ({ children, ...props }) => {
          return (
            <td {...props}>
              <div className="flex size-(--cell-size) items-center justify-center text-center">
                {children}
              </div>
            </td>
          )
        },
        ...components,
      }}
      {...props}
    />
  )
}

function CalendarDayButton({
  className,
  day,
  modifiers,
  locale,
  ...props
}: React.ComponentProps<typeof DayButton> & { locale?: Partial<Locale> }) {
  const defaultClassNames = getDefaultClassNames()

  const ref = React.useRef<HTMLButtonElement>(null)
  React.useEffect(() => {
    if (modifiers.focused) ref.current?.focus()
  }, [modifiers.focused])

  return (
    <Button
      ref={ref}
      variant="ghost"
      size="icon"
      data-day={day.date.toLocaleDateString(locale?.code)}
      data-selected-single={
        modifiers.selected &&
        !modifiers.range_start &&
        !modifiers.range_end &&
        !modifiers.range_middle
      }
```

```
      data-range-start={modifiers.range_start}
      data-range-end={modifiers.range_end}
      data-range-middle={modifiers.range_middle}
      className={cn(
        "relative isolate z-10 flex aspect-square size-auto w-full min-w-(--cell-size) flex-col gap-1 border-0 leading-none font-normal group-data-[focused=true]/day:relative group-data-[focused=true]
        defaultClassNames.day,
        className
      )}
      {...props}
    />
  )
}

export { Calendar, CalendarDayButton }
```

app/src/components/ui/card.tsx

```
import * as React from "react"

import { cn } from "@/lib/utils"

function Card({
  className,
  size = "default",
  ...props
}: React.ComponentProps<"div"> & { size?: "default" | "sm" }) {
  return (
    <div
      data-slot="card"
      data-size={size}
      className={cn(
        "group/card flex flex-col gap-(--card-spacing) overflow-hidden rounded-xl bg-card py-(--card-spacing) text-sm text-card-foreground ring-1 ring-foreground/10 [--card-spacing:--spacing(4)] has-d
        className
      )}
      {...props}
    />
  )
}

function CardHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-header"
      className={cn(
        "group/card-header @container/card-header grid auto-rows-min items-start gap-1 rounded-t-xl px-(--card-spacing) has-data-[slot=card-action]:grid-cols-[1fr_auto] has-data-[slot=card-descriptior
        className
      )}
      {...props}
    />
  )
}

function CardTitle({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-title"
      className={cn(
        "font-heading text-base leading-snug font-medium group-data-[size=sm]/card:text-sm",
        className
      )}
      {...props}
    />
  )
}

function CardDescription({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-description"
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  )
}

function CardAction({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-action"
      className={cn(
        "col-start-2 row-span-2 row-start-1 self-start justify-self-end",
        className
      )}
      {...props}
    />
```

```
  )
}

function CardContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-content"
      className={cn("px-(--card-spacing)", className)}
      {...props}
    />
  )
}

function CardFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="card-footer"
      className={cn(
        "flex items-center rounded-b-xl border-t bg-muted/50 p-(--card-spacing)",
        className
      )}
      {...props}
    />
  )
}

export {
  Card,
  CardHeader,
  CardFooter,
  CardTitle,
  CardAction,
  CardDescription,
  CardContent,
}
```

app/src/components/ui/carousel.tsx

```
import * as React from "react"
import useEmblaCarousel, {
  type UseEmblaCarouselType,
} from "embla-carousel-react"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"
import { ChevronLeftIcon, ChevronRightIcon } from "lucide-react"

type CarouselApi = UseEmblaCarouselType[1]
type UseCarouselParameters = Parameters<typeof useEmblaCarousel>
type CarouselOptions = UseCarouselParameters[0]
type CarouselPlugin = UseCarouselParameters[1]

type CarouselProps = {
  opts?: CarouselOptions
  plugins?: CarouselPlugin
  orientation?: "horizontal" | "vertical"
  setApi?: (api: CarouselApi) => void
}

type CarouselContextProps = {
  carouselRef: ReturnType<typeof useEmblaCarousel>[0]
  api: ReturnType<typeof useEmblaCarousel>[1]
  scrollPrev: () => void
  scrollNext: () => void
  canScrollPrev: boolean
  canScrollNext: boolean
} & CarouselProps

const CarouselContext = React.createContext<CarouselContextProps | null>(null)

function useCarousel() {
  const context = React.useContext(CarouselContext)

  if (!context) {
    throw new Error("useCarousel must be used within a <Carousel />")
  }

  return context
}

function Carousel({
  orientation = "horizontal",
  opts,
  setApi,
  plugins,
  className,
  children,
  ...props
}: React.ComponentProps<"div"> & CarouselProps) {
  const [carouselRef, api] = useEmblaCarousel(
    {
      ...opts,
      axis: orientation === "horizontal" ? "x" : "y",
    },
    plugins
  )
  const [canScrollPrev, setCanScrollPrev] = React.useState(false)
  const [canScrollNext, setCanScrollNext] = React.useState(false)

  const onSelect = React.useCallback((api: CarouselApi) => {
    if (!api) return
    setCanScrollPrev(api.canScrollPrev())
    setCanScrollNext(api.canScrollNext())
  }, [])

  const scrollPrev = React.useCallback(() => {
```

```
    api?.scrollPrev()
  }, [api])

  const scrollNext = React.useCallback(() => {
    api?.scrollNext()
  }, [api])

  const handleKeyDown = React.useCallback(
    (event: React.KeyboardEvent<HTMLDivElement>) => {
      if (event.key === "ArrowLeft") {
        event.preventDefault()
        scrollPrev()
      } else if (event.key === "ArrowRight") {
        event.preventDefault()
        scrollNext()
      }
    },
    [scrollPrev, scrollNext]
  )

  React.useEffect(() => {
    if (!api || !setApi) return
    setApi(api)
  }, [api, setApi])

  React.useEffect(() => {
    if (!api) return
    onSelect(api)
    api.on("reInit", onSelect)
    api.on("select", onSelect)

    return () => {
      api?.off("select", onSelect)
    }
  }, [api, onSelect])

  return (
    <CarouselContext.Provider
      value={{
        carouselRef,
        api: api,
        opts,
        orientation:
          orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),
        scrollPrev,
        scrollNext,
        canScrollPrev,
        canScrollNext,
      }}
    >
      <div
        onKeyDownCapture={handleKeyDown}
        className={cn("relative", className)}
        role="region"
        aria-roledescription="carousel"
        data-slot="carousel"
        {...props}
      >
        {children}
      </div>
    </CarouselContext.Provider>
  )
}

function CarouselContent({ className, ...props }: React.ComponentProps<"div">) {
  const { carouselRef, orientation } = useCarousel()

  return (
    <div
      ref={carouselRef}
```

```
      className="overflow-hidden"
      data-slot="carousel-content"
    >
      <div
        className={cn(
          "flex",
          orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
          className
        )}
        {...props}
      />
    </div>
  )
}

function CarouselItem({ className, ...props }: React.ComponentProps<"div">) {
  const { orientation } = useCarousel()

  return (
    <div
      role="group"
      aria-roledescription="slide"
      data-slot="carousel-item"
      className={cn(
        "min-w-0 shrink-0 grow-0 basis-full",
        orientation === "horizontal" ? "pl-4" : "pt-4",
        className
      )}
      {...props}
    />
  )
}

function CarouselPrevious({
  className,
  variant = "outline",
  size = "icon-sm",
  ...props
}: React.ComponentProps<typeof Button>) {
  const { orientation, scrollPrev, canScrollPrev } = useCarousel()

  return (
    <Button
      data-slot="carousel-previous"
      variant={variant}
      size={size}
      className={cn(
        "absolute touch-manipulation rounded-full",
        orientation === "horizontal"
          ? "inset-y-0 -left-12 my-auto"
          : "-top-12 left-1/2 -translate-x-1/2 rotate-90",
        className
      )}
      disabled={!canScrollPrev}
      onClick={scrollPrev}
      {...props}
    >
      <ChevronLeftIcon />
      <span className="sr-only">Previous slide</span>
    </Button>
  )
}

function CarouselNext({
  className,
  variant = "outline",
  size = "icon-sm",
  ...props
}: React.ComponentProps<typeof Button>) {
  const { orientation, scrollNext, canScrollNext } = useCarousel()
```

```
  return (
    <Button
      data-slot="carousel-next"
      variant={variant}
      size={size}
      className={cn(
        "absolute touch-manipulation rounded-full",
        orientation === "horizontal"
          ? "inset-y-0 -right-12 my-auto"
          : "-bottom-12 left-1/2 -translate-x-1/2 rotate-90",
        className
      )}
      disabled={!canScrollNext}
      onClick={scrollNext}
      {...props}
    >
      <ChevronRightIcon />
      <span className="sr-only">Next slide</span>
    </Button>
  )
}

export {
  type CarouselApi,
  Carousel,
  CarouselContent,
  CarouselItem,
  CarouselPrevious,
  CarouselNext,
  useCarousel,
}
```

app/src/components/ui/chart.tsx

```
"use client"

import * as React from "react"
import * as RechartsPrimitive from "recharts"
import type { TooltipValueType } from "recharts"

import { cn } from "@/lib/utils"

// Format: { THEME_NAME: CSS_SELECTOR }
const THEMES = { light: "", dark: ".dark" } as const

const INITIAL_DIMENSION = { width: 320, height: 200 } as const
type TooltipNameType = number | string

export type ChartConfig = Record<
  string,
  {
    label?: React.ReactNode
    icon?: React.ComponentType
  } & (
    | { color?: string; theme?: never }
    | { color?: never; theme: Record<keyof typeof THEMES, string> }
  )
>

type ChartContextProps = {
  config: ChartConfig
}

const ChartContext = React.createContext<ChartContextProps | null>(null)

function useChart() {
  const context = React.useContext(ChartContext)

  if (!context) {
    throw new Error("useChart must be used within a <ChartContainer />")
  }

  return context
}

function ChartContainer({
  id,
  className,
  children,
  config,
  initialDimension = INITIAL_DIMENSION,
  ...props
}: React.ComponentProps<"div"> & {
  config: ChartConfig
  children: React.ComponentProps<
    typeof RechartsPrimitive.ResponsiveContainer
  >["children"]
  initialDimension?: {
    width: number
    height: number
  }
}) {
  const uniqueId = React.useId()
  const chartId = `chart-${id ?? uniqueId.replace(/:/g, "")}`

  return (
    <ChartContext.Provider value={{ config }}>
      <div
        data-slot="chart"
        data-chart={chartId}
        className={cn(
          "flex aspect-video justify-center text-xs [&_.recharts-cartesian-axis-tick_text]:fill-muted-foreground [&_.recharts-cartesian-grid_line[stroke='#ccc']]:stroke-border/50 [&_.recharts-curve.re
```

```
          className
        )}
        {...props}
      >
        <ChartStyle id={chartId} config={config} />
        <RechartsPrimitive.ResponsiveContainer
          initialDimension={initialDimension}
        >
          {children}
        </RechartsPrimitive.ResponsiveContainer>
      </div>
    </ChartContext.Provider>
  )
}

const ChartStyle = ({ id, config }: { id: string; config: ChartConfig }) => {
  const colorConfig = Object.entries(config).filter(
    ([, config]) => config.theme ?? config.color
  )

  if (!colorConfig.length) {
    return null
  }

  return (
    <style
      dangerouslySetInnerHTML={{
        __html: Object.entries(THEMES)
          .map(
            ([theme, prefix]) => `
${prefix} [data-chart=${id}] {
${colorConfig
  .map(([key, itemConfig]) => {
    const color =
      itemConfig.theme?.[theme as keyof typeof itemConfig.theme] ??
      itemConfig.color
    return color ? `  --color-${key}: ${color};` : null
  })
  .join("\n")}
}
`
          )
          .join("\n"),
      }}
    />
  )
}

const ChartTooltip = RechartsPrimitive.Tooltip

function ChartTooltipContent({
  active,
  payload,
  className,
  indicator = "dot",
  hideLabel = false,
  hideIndicator = false,
  label,
  labelFormatter,
  labelClassName,
  formatter,
  color,
  nameKey,
  labelKey,
}: React.ComponentProps<typeof RechartsPrimitive.Tooltip> &
  React.ComponentProps<"div"> & {
    hideLabel?: boolean
    hideIndicator?: boolean
    indicator?: "line" | "dot" | "dashed"
    nameKey?: string
```

```
  labelKey?: string
} & Omit<
  RechartsPrimitive.DefaultTooltipContentProps<
    TooltipValueType,
    TooltipNameType
  >,
  "accessibilityLayer"
>) {
const { config } = useChart()

const tooltipLabel = React.useMemo(() => {
  if (hideLabel || !payload?.length) {
    return null
  }

  const [item] = payload
  const key = `${labelKey ?? item?.dataKey ?? item?.name ?? "value"}`
  const itemConfig = getPayloadConfigFromPayload(config, item, key)
  const value =
    !labelKey && typeof label === "string"
      ? (config[label]?.label ?? label)
      : itemConfig?.label

  if (labelFormatter) {
    return (
      <div className={cn("font-medium", labelClassName)}>
        {labelFormatter(value, payload)}
      </div>
    )
  }

  if (!value) {
    return null
  }

  return <div className={cn("font-medium", labelClassName)}>{value}</div>
}, [
  label,
  labelFormatter,
  payload,
  hideLabel,
  labelClassName,
  config,
  labelKey,
])

if (!active || !payload?.length) {
  return null
}

const nestLabel = payload.length === 1 && indicator !== "dot"

return (
  <div
    className={cn(
      "grid min-w-32 items-start gap-1.5 rounded-lg border border-border/50 bg-background px-2.5 py-1.5 text-xs shadow-xl",
      className
    )}
  >
    {!nestLabel ? tooltipLabel : null}
    <div className="grid gap-1.5">
      {payload
        .filter((item) => item.type !== "none")
        .map((item, index) => {
          const key = `${nameKey ?? item.name ?? item.dataKey ?? "value"}`
          const itemConfig = getPayloadConfigFromPayload(config, item, key)
          const indicatorColor = color ?? item.payload?.fill ?? item.color

          return (
            <div
```

```
              key={index}
              className={cn(
                "flex w-full flex-wrap items-stretch gap-2 [&>svg]:h-2.5 [&>svg]:w-2.5 [&>svg]:text-muted-foreground",
                indicator === "dot" && "items-center"
              )}
            >
              {formatter && item?.value !== undefined && item.name ? (
                formatter(item.value, item.name, item, index, item.payload)
              ) : (
                <>
                  {itemConfig?.icon ? (
                    <itemConfig.icon />
                  ) : (
                    !hideIndicator && (
                      <div
                        className={cn(
                          "shrink-0 rounded-[2px] border-(--color-border) bg-(--color-bg)",
                          {
                            "h-2.5 w-2.5": indicator === "dot",
                            "w-1": indicator === "line",
                            "w-0 border-[1.5px] border-dashed bg-transparent":
                              indicator === "dashed",
                            "my-0.5": nestLabel && indicator === "dashed",
                          }
                        )}
                        style={
                          {
                            "--color-bg": indicatorColor,
                            "--color-border": indicatorColor,
                          } as React.CSSProperties
                        }
                      />
                    )
                  )}
                  <div
                    className={cn(
                      "flex flex-1 justify-between leading-none",
                      nestLabel ? "items-end" : "items-center"
                    )}
                  >
                    <div className="grid gap-1.5">
                      {nestLabel ? tooltipLabel : null}
                      <span className="text-muted-foreground">
                        {itemConfig?.label ?? item.name}
                      </span>
                    </div>
                    {item.value != null && (
                      <span className="font-mono font-medium text-foreground tabular-nums">
                        {typeof item.value === "number"
                          ? item.value.toLocaleString()
                          : String(item.value)}
                      </span>
                    )}
                  </div>
                </>
              )}
            </div>
          )
        })}
      </div>
    </div>
  )
}

const ChartLegend = RechartsPrimitive.Legend

function ChartLegendContent({
  className,
  hideIcon = false,
  payload,
```

```
  verticalAlign = "bottom",
  nameKey,
}: React.ComponentProps<"div"> & {
  hideIcon?: boolean
  nameKey?: string
} & RechartsPrimitive.DefaultLegendContentProps) {
  const { config } = useChart()

  if (!payload?.length) {
    return null
  }

  return (
    <div
      className={cn(
        "flex items-center justify-center gap-4",
        verticalAlign === "top" ? "pb-3" : "pt-3",
        className
      )}
    >
      {payload
        .filter((item) => item.type !== "none")
        .map((item, index) => {
          const key = `${nameKey ?? item.dataKey ?? "value"}`
          const itemConfig = getPayloadConfigFromPayload(config, item, key)

          return (
            <div
              key={index}
              className={cn(
                "flex items-center gap-1.5 [&>svg]:h-3 [&>svg]:w-3 [&>svg]:text-muted-foreground"
              )}
            >
              {itemConfig?.icon && !hideIcon ? (
                <itemConfig.icon />
              ) : (
                <div
                  className="h-2 w-2 shrink-0 rounded-[2px]"
                  style={{
                    backgroundColor: item.color,
                  }}
                />
              )}
              {itemConfig?.label}
            </div>
          )
        })}
    </div>
  )
}

function getPayloadConfigFromPayload(
  config: ChartConfig,
  payload: unknown,
  key: string
) {
  if (typeof payload !== "object" || payload === null) {
    return undefined
  }

  const payloadPayload =
    "payload" in payload &&
    typeof payload.payload === "object" &&
    payload.payload !== null
      ? payload.payload
      : undefined

  let configLabelKey: string = key

  if (
```

```
    key in payload &&
    typeof payload[key as keyof typeof payload] === "string"
  ) {
    configLabelKey = payload[key as keyof typeof payload] as string
  } else if (
    payloadPayload &&
    key in payloadPayload &&
    typeof payloadPayload[key as keyof typeof payloadPayload] === "string"
  ) {
    configLabelKey = payloadPayload[
      key as keyof typeof payloadPayload
    ] as string
  }

  return configLabelKey in config ? config[configLabelKey] : config[key]
}

export {
  ChartContainer,
  ChartTooltip,
  ChartTooltipContent,
  ChartLegend,
  ChartLegendContent,
  ChartStyle,
}
```

app/src/components/ui/checkbox.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Checkbox as CheckboxPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { CheckIcon, MinusIcon } from "lucide-react"

/* Figma Checkbox (page 478:1123): 18/20/24px box, radius 6, brand (lime) or white fill */
const checkboxVariants = cva(
  "peer relative grid shrink-0 place-content-center rounded-[6px] border border-white/20 transition-colors outline-none group-has-disabled/field:opacity-40 after:absolute after:-inset-x-3 after:-inset
  {
    variants: {
      size: {
        sm: "size-[18px] [&_svg]:size-3.5",
        md: "size-5 [&_svg]:size-4",
        lg: "size-6 [&_svg]:size-[18px]",
      },
      tone: {
        brand:
          "hover:border-primary/50 focus-visible:ring-primary/20 data-checked:bg-primary data-checked:text-primary-foreground data-indeterminate:bg-primary data-indeterminate:text-primary-foreground",
        white:
          "hover:border-white/50 focus-visible:ring-white/20 data-checked:bg-white data-checked:text-[#131517] data-indeterminate:bg-white data-indeterminate:text-[#131517]",
      },
    },
    defaultVariants: {
      size: "sm",
      tone: "brand",
    },
  }
)

function Checkbox({
  className,
  size = "sm",
  tone = "brand",
  ...props
}: React.ComponentProps<typeof CheckboxPrimitive.Root> &
  VariantProps<typeof checkboxVariants>) {
  return (
    <CheckboxPrimitive.Root
      data-slot="checkbox"
      data-size={size}
      data-tone={tone}
      className={cn(checkboxVariants({ size, tone, className }))}
      {...props}
    >
      <CheckboxPrimitive.Indicator
        data-slot="checkbox-indicator"
        className="grid place-content-center text-current transition-none"
      >
        {props.checked === "indeterminate" ? (
          <MinusIcon strokeWidth={3} />
        ) : (
          <CheckIcon strokeWidth={3} />
        )}
      </CheckboxPrimitive.Indicator>
    </CheckboxPrimitive.Root>
  )
}

export { Checkbox, checkboxVariants }
```

app/src/components/ui/close-button.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { XIcon } from "lucide-react"

import { cn } from "@/lib/utils"

/* Figma CloseButton (node 2052:109): outer 24/32/40/48 = inner circle + 4px focus gap */
const closeButtonVariants = cva(
  "inline-flex shrink-0 items-center justify-center rounded-full bg-white/5 text-foreground transition-colors outline-none hover:bg-white/10 focus-visible:ring-2 focus-visible:ring-ring focus-visible:
  {
    variants: {
      size: {
        sm: "size-4 [&_svg]:size-3",
        md: "size-6 [&_svg]:size-4",
        lg: "size-8 [&_svg]:size-5",
        xl: "size-10 [&_svg]:size-6",
      },
    },
    defaultVariants: {
      size: "md",
    },
  }
)

function CloseButton({
  className,
  size = "md",
  ...props
}: React.ComponentProps<"button"> & VariantProps<typeof closeButtonVariants>) {
  return (
    <button
      type="button"
      data-slot="close-button"
      data-size={size}
      className={cn(closeButtonVariants({ size, className }))}
      {...props}
    >
      <XIcon />
      <span className="sr-only">Close</span>
    </button>
  )
}

export { CloseButton, closeButtonVariants }
```

app/src/components/ui/collapsible.tsx

```
"use client"

import { Collapsible as CollapsiblePrimitive } from "radix-ui"

function Collapsible({
  ...props
}: React.ComponentProps<typeof CollapsiblePrimitive.Root>) {
  return <CollapsiblePrimitive.Root data-slot="collapsible" {...props} />
}

function CollapsibleTrigger({
  ...props
}: React.ComponentProps<typeof CollapsiblePrimitive.CollapsibleTrigger>) {
  return (
    <CollapsiblePrimitive.CollapsibleTrigger
      data-slot="collapsible-trigger"
      {...props}
    />
  )
}

function CollapsibleContent({
  ...props
}: React.ComponentProps<typeof CollapsiblePrimitive.CollapsibleContent>) {
  return (
    <CollapsiblePrimitive.CollapsibleContent
      data-slot="collapsible-content"
      {...props}
    />
  )
}

export { Collapsible, CollapsibleTrigger, CollapsibleContent }
```

app/src/components/ui/combobox.tsx

```
import * as React from "react"
import { Combobox as ComboboxPrimitive } from "@base-ui/react"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"
import {
  InputGroup,
  InputGroupAddon,
  InputGroupButton,
  InputGroupInput,
} from "@/components/ui/input-group"
import { ChevronDownIcon, XIcon, CheckIcon } from "lucide-react"

const Combobox = ComboboxPrimitive.Root

function ComboboxValue({ ...props }: ComboboxPrimitive.Value.Props) {
  return <ComboboxPrimitive.Value data-slot="combobox-value" {...props} />
}

function ComboboxTrigger({
  className,
  children,
  ...props
}: ComboboxPrimitive.Trigger.Props) {
  return (
    <ComboboxPrimitive.Trigger
      data-slot="combobox-trigger"
      className={cn("[&_svg:not([class*='size-'])]:size-4", className)}
      {...props}
    >
      {children}
      <ChevronDownIcon className="pointer-events-none size-4 text-muted-foreground" />
    </ComboboxPrimitive.Trigger>
  )
}

function ComboboxClear({ className, ...props }: ComboboxPrimitive.Clear.Props) {
  return (
    <ComboboxPrimitive.Clear
      data-slot="combobox-clear"
      render={<InputGroupButton variant="ghost" size="icon-xs" />}
      className={cn(className)}
      {...props}
    >
      <XIcon className="pointer-events-none" />
    </ComboboxPrimitive.Clear>
  )
}

function ComboboxInput({
  className,
  children,
  disabled = false,
  showTrigger = true,
  showClear = false,
  ...props
}: ComboboxPrimitive.Input.Props & {
  showTrigger?: boolean
  showClear?: boolean
}) {
  return (
    <InputGroup className={cn("w-auto", className)}>
      <ComboboxPrimitive.Input
        render={<InputGroupInput disabled={disabled} />}
        {...props}
      />
      <InputGroupAddon align="inline-end">
        {showTrigger && (
```

```
          <InputGroupButton
            size="icon-xs"
            variant="ghost"
            asChild
            data-slot="input-group-button"
            className="group-has-data-[slot=combobox-clear]/input-group:hidden data-pressed:bg-transparent"
            disabled={disabled}
          >
            <ComboboxTrigger />
          </InputGroupButton>
        )}
        {showClear && <ComboboxClear disabled={disabled} />}
      </InputGroupAddon>
      {children}
    </InputGroup>
  )
}

function ComboboxContent({
  className,
  side = "bottom",
  sideOffset = 6,
  align = "start",
  alignOffset = 0,
  anchor,
  ...props
}: ComboboxPrimitive.Popup.Props &
  Pick<
    ComboboxPrimitive.Positioner.Props,
    "side" | "align" | "sideOffset" | "alignOffset" | "anchor"
  >) {
  return (
    <ComboboxPrimitive.Portal>
      <ComboboxPrimitive.Positioner
        side={side}
        sideOffset={sideOffset}
        align={align}
        alignOffset={alignOffset}
        anchor={anchor}
        className="isolate z-50"
      >
        <ComboboxPrimitive.Popup
          data-slot="combobox-content"
          data-chips={!!anchor}
          className={cn("group/combobox-content relative max-h-(--available-height) w-(--anchor-width) max-w-(--available-width) min-w-[calc(var(--anchor-width)+--spacing(7))] origin-(--transform-orig
          {...props}
        />
      </ComboboxPrimitive.Positioner>
    </ComboboxPrimitive.Portal>
  )
}

function ComboboxList({ className, ...props }: ComboboxPrimitive.List.Props) {
  return (
    <ComboboxPrimitive.List
      data-slot="combobox-list"
      className={cn(
        "no-scrollbar max-h-[min(calc(--spacing(72)---spacing(9)),calc(var(--available-height)---spacing(9)))] scroll-py-1 overflow-y-auto overscroll-contain p-1 data-empty:p-0",
        className
      )}
      {...props}
    />
  )
}

function ComboboxItem({
  className,
  children,
  ...props
}: ComboboxPrimitive.Item.Props) {
```

```
  return (
    <ComboboxPrimitive.Item
      data-slot="combobox-item"
      className={cn(
        "relative flex w-full cursor-default items-center gap-2 rounded-md py-1 pr-8 pl-1.5 text-sm outline-hidden select-none data-highlighted:bg-accent data-highlighted:text-accent-foreground not-da
        className
      )}
      {...props}
    >
      {children}
      <ComboboxPrimitive.ItemIndicator
        render={
          <span className="pointer-events-none absolute right-2 flex size-4 items-center justify-center" />
        }
      >
        <CheckIcon className="pointer-events-none" />
      </ComboboxPrimitive.ItemIndicator>
    </ComboboxPrimitive.Item>
  )
}

function ComboboxGroup({ className, ...props }: ComboboxPrimitive.Group.Props) {
  return (
    <ComboboxPrimitive.Group
      data-slot="combobox-group"
      className={cn(className)}
      {...props}
    />
  )
}

function ComboboxLabel({
  className,
  ...props
}: ComboboxPrimitive.GroupLabel.Props) {
  return (
    <ComboboxPrimitive.GroupLabel
      data-slot="combobox-label"
      className={cn("px-2 py-1.5 text-xs text-muted-foreground", className)}
      {...props}
    />
  )
}

function ComboboxCollection({ ...props }: ComboboxPrimitive.Collection.Props) {
  return (
    <ComboboxPrimitive.Collection data-slot="combobox-collection" {...props} />
  )
}

function ComboboxEmpty({ className, ...props }: ComboboxPrimitive.Empty.Props) {
  return (
    <ComboboxPrimitive.Empty
      data-slot="combobox-empty"
      className={cn(
        "hidden w-full justify-center py-2 text-center text-sm text-muted-foreground group-data-empty/combobox-content:flex",
        className
      )}
      {...props}
    />
  )
}

function ComboboxSeparator({
  className,
  ...props
}: ComboboxPrimitive.Separator.Props) {
  return (
    <ComboboxPrimitive.Separator
      data-slot="combobox-separator"
```

```
      className={cn("-mx-1 my-1 h-px bg-border", className)}
      {...props}
    />
  )
}

function ComboboxChips({
  className,
  ...props
}: React.ComponentPropsWithRef<typeof ComboboxPrimitive.Chips> &
  ComboboxPrimitive.Chips.Props) {
  return (
    <ComboboxPrimitive.Chips
      data-slot="combobox-chips"
      className={cn(
        "flex min-h-8 flex-wrap items-center gap-1 rounded-lg border border-input bg-transparent bg-clip-padding px-2.5 py-1 text-sm transition-colors focus-within:border-ring focus-within:ring-3 focu
        className
      )}
      {...props}
    />
  )
}

function ComboboxChip({
  className,
  children,
  showRemove = true,
  ...props
}: ComboboxPrimitive.Chip.Props & {
  showRemove?: boolean
}) {
  return (
    <ComboboxPrimitive.Chip
      data-slot="combobox-chip"
      className={cn(
        "flex h-[calc(--spacing(5.25))] w-fit items-center justify-center gap-1 rounded-sm bg-muted px-1.5 text-xs font-medium whitespace-nowrap text-foreground has-disabled:pointer-events-none has-di
        className
      )}
      {...props}
    >
      {children}
      {showRemove && (
        <ComboboxPrimitive.ChipRemove
          render={<Button variant="ghost" size="icon-xs" />}
          className="-ml-1 opacity-50 hover:opacity-100"
          data-slot="combobox-chip-remove"
        >
          <XIcon className="pointer-events-none" />
        </ComboboxPrimitive.ChipRemove>
      )}
    </ComboboxPrimitive.Chip>
  )
}

function ComboboxChipsInput({
  className,
  ...props
}: ComboboxPrimitive.Input.Props) {
  return (
    <ComboboxPrimitive.Input
      data-slot="combobox-chip-input"
      className={cn("min-w-16 flex-1 outline-none", className)}
      {...props}
    />
  )
}

function useComboboxAnchor() {
  return React.useRef<HTMLDivElement | null>(null)
}
```

```
export {
  Combobox,
  ComboboxInput,
  ComboboxContent,
  ComboboxList,
  ComboboxItem,
  ComboboxGroup,
  ComboboxLabel,
  ComboboxCollection,
  ComboboxEmpty,
  ComboboxSeparator,
  ComboboxChips,
  ComboboxChip,
  ComboboxChipsInput,
  ComboboxTrigger,
  ComboboxValue,
  useComboboxAnchor,
}
```

app/src/components/ui/command.tsx

```
"use client"

import * as React from "react"
import { Command as CommandPrimitive } from "cmdk"

import { cn } from "@/lib/utils"
import {
  Dialog,
  DialogContent,
  DialogDescription,
  DialogHeader,
  DialogTitle,
} from "@/components/ui/dialog"
import {
  InputGroup,
  InputGroupAddon,
} from "@/components/ui/input-group"
import { SearchIcon, CheckIcon } from "lucide-react"

function Command({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive>) {
  return (
    <CommandPrimitive
      data-slot="command"
      className={cn(
        "flex size-full flex-col overflow-hidden rounded-xl! bg-popover p-1 text-popover-foreground",
        className
      )}
      {...props}
    />
  )
}

function CommandDialog({
  title = "Command Palette",
  description = "Search for a command to run...",
  children,
  className,
  showCloseButton = false,
  ...props
}: React.ComponentProps<typeof Dialog> & {
  title?: string
  description?: string
  className?: string
  showCloseButton?: boolean
}) {
  return (
    <Dialog {...props}>
      <DialogHeader className="sr-only">
        <DialogTitle>{title}</DialogTitle>
        <DialogDescription>{description}</DialogDescription>
      </DialogHeader>
      <DialogContent
        className={cn(
          "top-1/3 translate-y-0 overflow-hidden rounded-xl! p-0",
          className
        )}
        showCloseButton={showCloseButton}
      >
        {children}
      </DialogContent>
    </Dialog>
  )
}

function CommandInput({
```

```
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Input>) {
  return (
    <div data-slot="command-input-wrapper" className="p-1 pb-0">
      <InputGroup className="h-8! rounded-lg! border-input/30 bg-input/30 shadow-none! *:data-[slot=input-group-addon]:pl-2!">
        <CommandPrimitive.Input
          data-slot="command-input"
          className={cn(
            "w-full text-sm outline-hidden disabled:cursor-not-allowed disabled:opacity-50",
            className
          )}
          {...props}
        />
        <InputGroupAddon>
          <SearchIcon className="size-4 shrink-0 opacity-50" />
        </InputGroupAddon>
      </InputGroup>
    </div>
  )
}

function CommandList({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.List>) {
  return (
    <CommandPrimitive.List
      data-slot="command-list"
      className={cn(
        "no-scrollbar max-h-72 scroll-py-1 overflow-x-hidden overflow-y-auto outline-none",
        className
      )}
      {...props}
    />
  )
}

function CommandEmpty({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Empty>) {
  return (
    <CommandPrimitive.Empty
      data-slot="command-empty"
      className={cn("py-6 text-center text-sm", className)}
      {...props}
    />
  )
}

function CommandGroup({
  className,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Group>) {
  return (
    <CommandPrimitive.Group
      data-slot="command-group"
      className={cn(
        "overflow-hidden p-1 text-foreground **:[[cmdk-group-heading]]:px-2 **:[[cmdk-group-heading]]:py-1.5 **:[[cmdk-group-heading]]:text-xs **:[[cmdk-group-heading]]:font-medium **:[[cmdk-group-hea
        className
      )}
      {...props}
    />
  )
}

function CommandSeparator({
  className,
  ...props
```

```
}: React.ComponentProps<typeof CommandPrimitive.Separator>) {
  return (
    <CommandPrimitive.Separator
      data-slot="command-separator"
      className={cn("-mx-1 h-px bg-border", className)}
      {...props}
    />
  )
}

function CommandItem({
  className,
  children,
  ...props
}: React.ComponentProps<typeof CommandPrimitive.Item>) {
  return (
    <CommandPrimitive.Item
      data-slot="command-item"
      className={cn(
        "group/command-item relative flex cursor-default items-center gap-2 rounded-sm px-2 py-1.5 text-sm outline-hidden select-none in-data-[slot=dialog-content]:rounded-lg! data-[disabled=true]:poi
        className
      )}
      {...props}
    >
      {children}
      <CheckIcon className="ml-auto opacity-0 group-has-data-[slot=command-shortcut]/command-item:hidden group-data-[checked=true]/command-item:opacity-100" />
    </CommandPrimitive.Item>
  )
}

function CommandShortcut({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="command-shortcut"
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground group-data-selected/command-item:text-foreground",
        className
      )}
      {...props}
    />
  )
}

export {
  Command,
  CommandDialog,
  CommandInput,
  CommandList,
  CommandEmpty,
  CommandGroup,
  CommandItem,
  CommandShortcut,
  CommandSeparator,
}
```

app/src/components/ui/context-menu.tsx

```
import * as React from "react"
import { ContextMenu as ContextMenuPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { ChevronRightIcon, CheckIcon } from "lucide-react"

function ContextMenu({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Root>) {
  return <ContextMenuPrimitive.Root data-slot="context-menu" {...props} />
}

function ContextMenuTrigger({
  className,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Trigger>) {
  return (
    <ContextMenuPrimitive.Trigger
      data-slot="context-menu-trigger"
      className={cn("select-none", className)}
      {...props}
    />
  )
}

function ContextMenuGroup({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Group>) {
  return (
    <ContextMenuPrimitive.Group data-slot="context-menu-group" {...props} />
  )
}

function ContextMenuPortal({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Portal>) {
  return (
    <ContextMenuPrimitive.Portal data-slot="context-menu-portal" {...props} />
  )
}

function ContextMenuSub({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Sub>) {
  return <ContextMenuPrimitive.Sub data-slot="context-menu-sub" {...props} />
}

function ContextMenuRadioGroup({
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.RadioGroup>) {
  return (
    <ContextMenuPrimitive.RadioGroup
      data-slot="context-menu-radio-group"
      {...props}
    />
  )
}

function ContextMenuContent({
  className,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Content> & {
  side?: "top" | "right" | "bottom" | "left"
}) {
  return (
    <ContextMenuPrimitive.Portal>
      <ContextMenuPrimitive.Content
        data-slot="context-menu-content"
```

```
        className={cn("z-50 max-h-(--radix-context-menu-content-available-height) min-w-36 origin-(--radix-context-menu-content-transform-origin) overflow-x-hidden overflow-y-auto rounded-lg bg-popove
        {...props}
      />
    </ContextMenuPrimitive.Portal>
  )
}

function ContextMenuItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Item> & {
  inset?: boolean
  variant?: "default" | "destructive"
}) {
  return (
    <ContextMenuPrimitive.Item
      data-slot="context-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "group/context-menu-item relative flex cursor-default items-center gap-1.5 rounded-md px-1.5 py-1 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground data-inset:pl-
        className
      )}
      {...props}
    />
  )
}

function ContextMenuSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.SubTrigger> & {
  inset?: boolean
}) {
  return (
    <ContextMenuPrimitive.SubTrigger
      data-slot="context-menu-sub-trigger"
      data-inset={inset}
      className={cn(
        "flex cursor-default items-center gap-1.5 rounded-md px-1.5 py-1 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground data-inset:pl-7 data-open:bg-accent data-open:t
        className
      )}
      {...props}
    >
      {children}
      <ChevronRightIcon className="ml-auto" />
    </ContextMenuPrimitive.SubTrigger>
  )
}

function ContextMenuSubContent({
  className,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.SubContent>) {
  return (
    <ContextMenuPrimitive.SubContent
      data-slot="context-menu-sub-content"
      className={cn("z-50 min-w-32 origin-(--radix-context-menu-content-transform-origin) overflow-hidden rounded-lg border bg-popover p-1 text-popover-foreground shadow-lg duration-100 data-[side=bot
      {...props}
    />
  )
}

function ContextMenuCheckboxItem({
  className,
  children,
```

```
  checked,
  inset,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.CheckboxItem> & {
  inset?: boolean
}) {
  return (
    <ContextMenuPrimitive.CheckboxItem
      data-slot="context-menu-checkbox-item"
      data-inset={inset}
      className={cn(
        "relative flex cursor-default items-center gap-1.5 rounded-md py-1 pr-8 pl-1.5 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground data-inset:pl-7 data-disabled:poi
        className
      )}
      checked={checked}
      {...props}
    >
      <span className="pointer-events-none absolute right-2">
        <ContextMenuPrimitive.ItemIndicator>
          <CheckIcon
          />
        </ContextMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </ContextMenuPrimitive.CheckboxItem>
  )
}

function ContextMenuRadioItem({
  className,
  children,
  inset,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.RadioItem> & {
  inset?: boolean
}) {
  return (
    <ContextMenuPrimitive.RadioItem
      data-slot="context-menu-radio-item"
      data-inset={inset}
      className={cn(
        "relative flex cursor-default items-center gap-1.5 rounded-md py-1 pr-8 pl-1.5 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground data-inset:pl-7 data-disabled:poi
        className
      )}
      {...props}
    >
      <span className="pointer-events-none absolute right-2">
        <ContextMenuPrimitive.ItemIndicator>
          <CheckIcon
          />
        </ContextMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </ContextMenuPrimitive.RadioItem>
  )
}

function ContextMenuLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Label> & {
  inset?: boolean
}) {
  return (
    <ContextMenuPrimitive.Label
      data-slot="context-menu-label"
      data-inset={inset}
      className={cn(
        "px-1.5 py-1 text-xs font-medium text-muted-foreground data-inset:pl-7",
```

```
        className
      )}
      {...props}
    />
  )
}

function ContextMenuSeparator({
  className,
  ...props
}: React.ComponentProps<typeof ContextMenuPrimitive.Separator>) {
  return (
    <ContextMenuPrimitive.Separator
      data-slot="context-menu-separator"
      className={cn("-mx-1 my-1 h-px bg-border", className)}
      {...props}
    />
  )
}

function ContextMenuShortcut({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="context-menu-shortcut"
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground group-focus/context-menu-item:text-accent-foreground",
        className
      )}
      {...props}
    />
  )
}

export {
  ContextMenu,
  ContextMenuTrigger,
  ContextMenuContent,
  ContextMenuItem,
  ContextMenuCheckboxItem,
  ContextMenuRadioItem,
  ContextMenuLabel,
  ContextMenuSeparator,
  ContextMenuShortcut,
  ContextMenuGroup,
  ContextMenuPortal,
  ContextMenuSub,
  ContextMenuSubContent,
  ContextMenuSubTrigger,
  ContextMenuRadioGroup,
}
```

app/src/components/ui/dialog.tsx

```
"use client"

import * as React from "react"
import { Dialog as DialogPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"
import { CloseButton } from "@/components/ui/close-button"

function Dialog({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Root>) {
  return <DialogPrimitive.Root data-slot="dialog" {...props} />
}

function DialogTrigger({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Trigger>) {
  return <DialogPrimitive.Trigger data-slot="dialog-trigger" {...props} />
}

function DialogPortal({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Portal>) {
  return <DialogPrimitive.Portal data-slot="dialog-portal" {...props} />
}

function DialogClose({
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Close>) {
  return <DialogPrimitive.Close data-slot="dialog-close" {...props} />
}

function DialogOverlay({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Overlay>) {
  return (
    <DialogPrimitive.Overlay
      data-slot="dialog-overlay"
      className={cn(
        "fixed inset-0 isolate z-50 bg-black/50 duration-100 data-open:animate-in data-open:fade-in-0 data-closed:animate-out data-closed:fade-out-0",
        className
      )}
      {...props}
    />
  )
}

function DialogContent({
  className,
  children,
  showCloseButton = true,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Content> & {
  showCloseButton?: boolean
}) {
  return (
    <DialogPortal>
      <DialogOverlay />
      <DialogPrimitive.Content
        data-slot="dialog-content"
        className={cn(
          /* Figma Menu (node 1472:7012): glass panel — rgba(35,38,42,.75) + 40px blur, radius 24, p-8px */
          "fixed top-1/2 left-1/2 z-50 grid w-full max-w-[calc(100%-2rem)] -translate-x-1/2 -translate-y-1/2 gap-2 rounded-3xl border border-white/5 bg-[#23262a]/75 p-2 text-sm text-popover-foreground
          className
        )}
        {...props}
```

```
      >
        {children}
        {showCloseButton && (
          <DialogPrimitive.Close data-slot="dialog-close" asChild>
            <CloseButton className="absolute top-3 right-3" size="lg" />
          </DialogPrimitive.Close>
        )}
      </DialogPrimitive.Content>
    </DialogPortal>
  )
}

function DialogHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="dialog-header"
      className={cn("flex flex-col gap-1 p-2 pr-12", className)}
      {...props}
    />
  )
}

function DialogFooter({
  className,
  showCloseButton = false,
  children,
  ...props
}: React.ComponentProps<"div"> & {
  showCloseButton?: boolean
}) {
  return (
    <div
      data-slot="dialog-footer"
      className={cn(
        "flex flex-col-reverse gap-1.5 p-1 sm:flex-row sm:items-center sm:justify-end",
        className
      )}
      {...props}
    >
      {children}
      {showCloseButton && (
        <DialogPrimitive.Close asChild>
          <Button variant="outline">Close</Button>
        </DialogPrimitive.Close>
      )}
    </div>
  )
}

function DialogTitle({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Title>) {
  return (
    <DialogPrimitive.Title
      data-slot="dialog-title"
      className={cn(
        "font-heading text-base leading-6 font-semibold",
        className
      )}
      {...props}
    />
  )
}

function DialogDescription({
  className,
  ...props
}: React.ComponentProps<typeof DialogPrimitive.Description>) {
  return (
```

```
    <DialogPrimitive.Description
      data-slot="dialog-description"
      className={cn(
        "text-sm text-muted-foreground *:[a]:underline *:[a]:underline-offset-3 *:[a]:hover:text-foreground",
        className
      )}
      {...props}
    />
  )
}

export {
  Dialog,
  DialogClose,
  DialogContent,
  DialogDescription,
  DialogFooter,
  DialogHeader,
  DialogOverlay,
  DialogPortal,
  DialogTitle,
  DialogTrigger,
}
```

app/src/components/ui/direction.tsx

```
"use client"

import * as React from "react"
import { Direction } from "radix-ui"

function DirectionProvider({
  dir,
  direction,
  children,
}: React.ComponentProps<typeof Direction.DirectionProvider> & {
  direction?: React.ComponentProps<typeof Direction.DirectionProvider>["dir"]
}) {
  return (
    <Direction.DirectionProvider dir={direction ?? dir}>
      {children}
    </Direction.DirectionProvider>
  )
}

const useDirection = Direction.useDirection

export { DirectionProvider, useDirection }
```

app/src/components/ui/drawer.tsx

```
import * as React from "react"
import { Drawer as DrawerPrimitive } from "vaul"

import { cn } from "@/lib/utils"

function Drawer({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Root>) {
  return <DrawerPrimitive.Root data-slot="drawer" {...props} />
}

function DrawerTrigger({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Trigger>) {
  return <DrawerPrimitive.Trigger data-slot="drawer-trigger" {...props} />
}

function DrawerPortal({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Portal>) {
  return <DrawerPrimitive.Portal data-slot="drawer-portal" {...props} />
}

function DrawerClose({
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Close>) {
  return <DrawerPrimitive.Close data-slot="drawer-close" {...props} />
}

function DrawerOverlay({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Overlay>) {
  return (
    <DrawerPrimitive.Overlay
      data-slot="drawer-overlay"
      className={cn(
        "fixed inset-0 z-50 bg-black/10 supports-backdrop-filter:backdrop-blur-xs data-open:animate-in data-open:fade-in-0 data-closed:animate-out data-closed:fade-out-0",
        className
      )}
      {...props}
    />
  )
}

function DrawerContent({
  className,
  children,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Content>) {
  return (
    <DrawerPortal data-slot="drawer-portal">
      <DrawerOverlay />
      <DrawerPrimitive.Content
        data-slot="drawer-content"
        className={cn(
          "group/drawer-content fixed z-50 flex h-auto flex-col bg-popover text-sm text-popover-foreground data-[vaul-drawer-direction=bottom]:inset-x-0 data-[vaul-drawer-direction=bottom]:bottom-0 da
          className
        )}
        {...props}
      >
        <div className="mx-auto mt-4 hidden h-1 w-[100px] shrink-0 rounded-full bg-muted group-data-[vaul-drawer-direction=bottom]/drawer-content:block" />
        {children}
      </DrawerPrimitive.Content>
    </DrawerPortal>
  )
}
```

```
function DrawerHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="drawer-header"
      className={cn(
        "flex flex-col gap-0.5 p-4 group-data-[vaul-drawer-direction=bottom]/drawer-content:text-center group-data-[vaul-drawer-direction=top]/drawer-content:text-center md:gap-0.5 md:text-left",
        className
      )}
      {...props}
    />
  )
}

function DrawerFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="drawer-footer"
      className={cn("mt-auto flex flex-col gap-2 p-4", className)}
      {...props}
    />
  )
}

function DrawerTitle({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Title>) {
  return (
    <DrawerPrimitive.Title
      data-slot="drawer-title"
      className={cn(
        "font-heading text-base font-medium text-foreground",
        className
      )}
      {...props}
    />
  )
}

function DrawerDescription({
  className,
  ...props
}: React.ComponentProps<typeof DrawerPrimitive.Description>) {
  return (
    <DrawerPrimitive.Description
      data-slot="drawer-description"
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  )
}

export {
  Drawer,
  DrawerPortal,
  DrawerOverlay,
  DrawerTrigger,
  DrawerClose,
  DrawerContent,
  DrawerHeader,
  DrawerFooter,
  DrawerTitle,
  DrawerDescription,
}
```

app/src/components/ui/dropdown-menu.tsx

```
"use client"

import * as React from "react"
import { DropdownMenu as DropdownMenuPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { CheckIcon, ChevronRightIcon } from "lucide-react"

function DropdownMenu({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Root>) {
  return <DropdownMenuPrimitive.Root data-slot="dropdown-menu" {...props} />
}

function DropdownMenuPortal({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Portal>) {
  return (
    <DropdownMenuPrimitive.Portal data-slot="dropdown-menu-portal" {...props} />
  )
}

function DropdownMenuTrigger({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Trigger>) {
  return (
    <DropdownMenuPrimitive.Trigger
      data-slot="dropdown-menu-trigger"
      {...props}
    />
  )
}

function DropdownMenuContent({
  className,
  align = "start",
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Content>) {
  return (
    <DropdownMenuPrimitive.Portal>
      <DropdownMenuPrimitive.Content
        data-slot="dropdown-menu-content"
        sideOffset={sideOffset}
        align={align}
        className={cn("z-50 max-h-(--radix-dropdown-menu-content-available-height) w-(--radix-dropdown-menu-trigger-width) min-w-32 origin-(--radix-dropdown-menu-content-transform-origin) overflow-x-h
        {...props}
      />
    </DropdownMenuPrimitive.Portal>
  )
}

function DropdownMenuGroup({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Group>) {
  return (
    <DropdownMenuPrimitive.Group data-slot="dropdown-menu-group" {...props} />
  )
}

function DropdownMenuItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Item> & {
  inset?: boolean
  variant?: "default" | "destructive"
```

```
}) {
  return (
    <DropdownMenuPrimitive.Item
      data-slot="dropdown-menu-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "group/dropdown-menu-item relative flex cursor-default items-center gap-1.5 rounded-md px-1.5 py-1 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground not-data-[var
        className
      )}
      {...props}
    />
  )
}

function DropdownMenuCheckboxItem({
  className,
  children,
  checked,
  inset,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.CheckboxItem> & {
  inset?: boolean
}) {
  return (
    <DropdownMenuPrimitive.CheckboxItem
      data-slot="dropdown-menu-checkbox-item"
      data-inset={inset}
      className={cn(
        "relative flex cursor-default items-center gap-1.5 rounded-md py-1 pr-8 pl-1.5 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground focus:**:text-accent-foreground d
        className
      )}
      checked={checked}
      {...props}
    >
      <span
        className="pointer-events-none absolute right-2 flex items-center justify-center"
        data-slot="dropdown-menu-checkbox-item-indicator"
      >
        <DropdownMenuPrimitive.ItemIndicator>
          <CheckIcon
          />
        </DropdownMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </DropdownMenuPrimitive.CheckboxItem>
  )
}

function DropdownMenuRadioGroup({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.RadioGroup>) {
  return (
    <DropdownMenuPrimitive.RadioGroup
      data-slot="dropdown-menu-radio-group"
      {...props}
    />
  )
}

function DropdownMenuRadioItem({
  className,
  children,
  inset,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.RadioItem> & {
  inset?: boolean
}) {
  return (
    <DropdownMenuPrimitive.RadioItem
```

```
      data-slot="dropdown-menu-radio-item"
      data-inset={inset}
      className={cn(
        "relative flex cursor-default items-center gap-1.5 rounded-md py-1 pr-8 pl-1.5 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground focus:**:text-accent-foreground d
        className
      )}
      {...props}
    >
      <span
        className="pointer-events-none absolute right-2 flex items-center justify-center"
        data-slot="dropdown-menu-radio-item-indicator"
      >
        <DropdownMenuPrimitive.ItemIndicator>
          <CheckIcon
          />
        </DropdownMenuPrimitive.ItemIndicator>
      </span>
      {children}
    </DropdownMenuPrimitive.RadioItem>
  )
}

function DropdownMenuLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Label> & {
  inset?: boolean
}) {
  return (
    <DropdownMenuPrimitive.Label
      data-slot="dropdown-menu-label"
      data-inset={inset}
      className={cn(
        "px-1.5 py-1 text-xs font-medium text-muted-foreground data-inset:pl-7",
        className
      )}
      {...props}
    />
  )
}

function DropdownMenuSeparator({
  className,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Separator>) {
  return (
    <DropdownMenuPrimitive.Separator
      data-slot="dropdown-menu-separator"
      className={cn("-mx-1 my-1 h-px bg-border", className)}
      {...props}
    />
  )
}

function DropdownMenuShortcut({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="dropdown-menu-shortcut"
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground group-focus/dropdown-menu-item:text-accent-foreground",
        className
      )}
      {...props}
    />
  )
}
```

```
function DropdownMenuSub({
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.Sub>) {
  return <DropdownMenuPrimitive.Sub data-slot="dropdown-menu-sub" {...props} />
}

function DropdownMenuSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.SubTrigger> & {
  inset?: boolean
}) {
  return (
    <DropdownMenuPrimitive.SubTrigger
      data-slot="dropdown-menu-sub-trigger"
      data-inset={inset}
      className={cn(
        "flex cursor-default items-center gap-1.5 rounded-md px-1.5 py-1 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground not-data-[variant=destructive]:focus:**:text-ac
        className
      )}
      {...props}
    >
      {children}
      <ChevronRightIcon className="ml-auto" />
    </DropdownMenuPrimitive.SubTrigger>
  )
}

function DropdownMenuSubContent({
  className,
  ...props
}: React.ComponentProps<typeof DropdownMenuPrimitive.SubContent>) {
  return (
    <DropdownMenuPrimitive.SubContent
      data-slot="dropdown-menu-sub-content"
      className={cn("z-50 min-w-[96px] origin-(--radix-dropdown-menu-content-transform-origin) overflow-hidden rounded-lg bg-popover p-1 text-popover-foreground shadow-lg ring-1 ring-foreground/10 dur
      {...props}
    />
  )
}

export {
  DropdownMenu,
  DropdownMenuPortal,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuGroup,
  DropdownMenuLabel,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
  DropdownMenuRadioGroup,
  DropdownMenuRadioItem,
  DropdownMenuSeparator,
  DropdownMenuShortcut,
  DropdownMenuSub,
  DropdownMenuSubTrigger,
  DropdownMenuSubContent,
}
```

app/src/components/ui/empty.tsx

```
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@/lib/utils"

function Empty({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="empty"
      className={cn(
        "flex w-full min-w-0 flex-1 flex-col items-center justify-center gap-4 rounded-xl border-dashed p-6 text-center text-balance",
        className
      )}
      {...props}
    />
  )
}

function EmptyHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="empty-header"
      className={cn("flex max-w-sm flex-col items-center gap-2", className)}
      {...props}
    />
  )
}

const emptyMediaVariants = cva(
  "mb-2 flex shrink-0 items-center justify-center [&_svg]:pointer-events-none [&_svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "bg-transparent",
        icon: "flex size-8 shrink-0 items-center justify-center rounded-lg bg-muted text-foreground [&_svg:not([class*='size-'])]:size-4",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)

function EmptyMedia({
  className,
  variant = "default",
  ...props
}: React.ComponentProps<"div"> & VariantProps<typeof emptyMediaVariants>) {
  return (
    <div
      data-slot="empty-icon"
      data-variant={variant}
      className={cn(emptyMediaVariants({ variant, className }))}
      {...props}
    />
  )
}

function EmptyTitle({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="empty-title"
      className={cn(
        "font-heading text-sm font-medium tracking-tight",
        className
      )}
      {...props}
    />
  )
```

```
}

function EmptyDescription({ className, ...props }: React.ComponentProps<"p">) {
  return (
    <div
      data-slot="empty-description"
      className={cn(
        "text-sm/relaxed text-muted-foreground [&>a]:underline [&>a]:underline-offset-4 [&>a:hover]:text-primary",
        className
      )}
      {...props}
    />
  )
}

function EmptyContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="empty-content"
      className={cn(
        "flex w-full max-w-sm min-w-0 flex-col items-center gap-2.5 text-sm text-balance",
        className
      )}
      {...props}
    />
  )
}

export {
  Empty,
  EmptyHeader,
  EmptyTitle,
  EmptyDescription,
  EmptyContent,
  EmptyMedia,
}
```

app/src/components/ui/field.tsx

```
import { useMemo } from "react"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@/lib/utils"
import { Label } from "@/components/ui/label"
import { Separator } from "@/components/ui/separator"

function FieldSet({ className, ...props }: React.ComponentProps<"fieldset">) {
  return (
    <fieldset
      data-slot="field-set"
      className={cn(
        "flex flex-col gap-4 has-[>[data-slot=checkbox-group]]:gap-3 has-[>[data-slot=radio-group]]:gap-3",
        className
      )}
      {...props}
    />
  )
}

function FieldLegend({
  className,
  variant = "legend",
  ...props
}: React.ComponentProps<"legend"> & { variant?: "legend" | "label" }) {
  return (
    <legend
      data-slot="field-legend"
      data-variant={variant}
      className={cn(
        "mb-1.5 font-medium data-[variant=label]:text-sm data-[variant=legend]:text-base",
        className
      )}
      {...props}
    />
  )
}

function FieldGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="field-group"
      className={cn(
        "group/field-group @container/field-group flex w-full flex-col gap-5 data-[slot=checkbox-group]:gap-3 *:data-[slot=field-group]:gap-4",
        className
      )}
      {...props}
    />
  )
}

const fieldVariants = cva(
  "group/field flex w-full gap-2 data-[invalid=true]:text-destructive",
  {
    variants: {
      orientation: {
        vertical: "flex-col *:w-full [&>.sr-only]:w-auto",
        horizontal:
          "flex-row items-center has-[>[data-slot=field-content]]:items-start *:data-[slot=field-label]:flex-auto has-[>[data-slot=field-content]]:[&>[role=checkbox],[role=radio]]:mt-px",
        responsive:
          "flex-col *:w-full @md/field-group:flex-row @md/field-group:items-center @md/field-group:*:w-auto @md/field-group:has-[>[data-slot=field-content]]:items-start @md/field-group:*:data-[slot=fi
      },
    },
    defaultVariants: {
      orientation: "vertical",
    },
  }
)
```

```
function Field({
  className,
  orientation = "vertical",
  ...props
}: React.ComponentProps<"div"> & VariantProps<typeof fieldVariants>) {
  return (
    <div
      role="group"
      data-slot="field"
      data-orientation={orientation}
      className={cn(fieldVariants({ orientation }), className)}
      {...props}
    />
  )
}

function FieldContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="field-content"
      className={cn(
        "group/field-content flex flex-1 flex-col gap-0.5 leading-snug",
        className
      )}
      {...props}
    />
  )
}

function FieldLabel({
  className,
  ...props
}: React.ComponentProps<typeof Label>) {
  return (
    <Label
      data-slot="field-label"
      className={cn(
        "group/field-label peer/field-label flex w-fit gap-2 leading-snug group-data-[disabled=true]/field:opacity-50 has-data-checked:border-primary/30 has-data-checked:bg-primary/5 has-[>[data-slot=
        "has-[>[data-slot=field]]:w-full has-[>[data-slot=field]]:flex-col",
        className
      )}
      {...props}
    />
  )
}

function FieldTitle({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="field-label"
      className={cn(
        "flex w-fit items-center gap-2 text-sm font-medium group-data-[disabled=true]/field:opacity-50",
        className
      )}
      {...props}
    />
  )
}

function FieldDescription({ className, ...props }: React.ComponentProps<"p">) {
  return (
    <p
      data-slot="field-description"
      className={cn(
        "text-left text-sm leading-normal font-normal text-muted-foreground group-has-data-horizontal/field:text-balance [[data-variant=legend]+&]:-mt-1.5",
        "last:mt-0 nth-last-2:-mt-1",
        "[&>a]:underline [&>a]:underline-offset-4 [&>a:hover]:text-primary",
        className
      )}
      {...props}
```

```
    />
  )
}

function FieldSeparator({
  children,
  className,
  ...props
}: React.ComponentProps<"div"> & {
  children?: React.ReactNode
}) {
  return (
    <div
      data-slot="field-separator"
      data-content={!!children}
      className={cn(
        "relative -my-2 h-5 text-sm group-data-[variant=outline]/field-group:-mb-2",
        className
      )}
      {...props}
    >
      <Separator className="absolute inset-0 top-1/2" />
      {children && (
        <span
          className="relative mx-auto block w-fit bg-background px-2 text-muted-foreground"
          data-slot="field-separator-content"
        >
          {children}
        </span>
      )}
    </div>
  )
}

function FieldError({
  className,
  children,
  errors,
  ...props
}: React.ComponentProps<"div"> & {
  errors?: Array<{ message?: string } | undefined>
}) {
  const content = useMemo(() => {
    if (children) {
      return children
    }

    if (!errors?.length) {
      return null
    }

    const uniqueErrors = [
      ...new Map(errors.map((error) => [error?.message, error])).values(),
    ]

    if (uniqueErrors?.length == 1) {
      return uniqueErrors[0]?.message
    }

    return (
      <ul className="ml-4 flex list-disc flex-col gap-1">
        {uniqueErrors.map(
          (error, index) =>
            error?.message && <li key={index}>{error.message}</li>
        )}
      </ul>
    )
  }, [children, errors])

  if (!content) {
```

```
    return null
  }

  return (
    <div
      role="alert"
      data-slot="field-error"
      className={cn("text-sm font-normal text-destructive", className)}
      {...props}
    >
      {content}
    </div>
  )
}

export {
  Field,
  FieldLabel,
  FieldDescription,
  FieldError,
  FieldGroup,
  FieldLegend,
  FieldSeparator,
  FieldSet,
  FieldContent,
  FieldTitle,
}
```

app/src/components/ui/hover-card.tsx

```
import * as React from "react"
import { HoverCard as HoverCardPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function HoverCard({
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Root>) {
  return <HoverCardPrimitive.Root data-slot="hover-card" {...props} />
}

function HoverCardTrigger({
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Trigger>) {
  return (
    <HoverCardPrimitive.Trigger data-slot="hover-card-trigger" {...props} />
  )
}

function HoverCardContent({
  className,
  align = "center",
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof HoverCardPrimitive.Content>) {
  return (
    <HoverCardPrimitive.Portal data-slot="hover-card-portal">
      <HoverCardPrimitive.Content
        data-slot="hover-card-content"
        align={align}
        sideOffset={sideOffset}
        className={cn(
          "z-50 w-64 origin-(--radix-hover-card-content-transform-origin) rounded-lg bg-popover p-2.5 text-sm text-popover-foreground shadow-md ring-1 ring-foreground/10 outline-hidden duration-100 da
          className
        )}
        {...props}
      />
    </HoverCardPrimitive.Portal>
  )
}

export { HoverCard, HoverCardTrigger, HoverCardContent }
```

app/src/components/ui/input-group.tsx

```
"use client"

import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Textarea } from "@/components/ui/textarea"

function InputGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="input-group"
      role="group"
      className={cn(
        "group/input-group relative flex h-8 w-full min-w-0 items-center rounded-lg border border-input transition-colors outline-none in-data-[slot=combobox-content]:focus-within:border-inherit in-da
        className
      )}
      {...props}
    />
  )
}

const inputGroupAddonVariants = cva(
  "flex h-auto cursor-text items-center justify-center gap-2 py-1.5 text-sm font-medium text-muted-foreground select-none group-data-[disabled=true]/input-group:opacity-50 [&>kbd]:rounded-[calc(var(--
  {
    variants: {
      align: {
        "inline-start":
          "order-first pl-2 has-[>button]:ml-[-0.3rem] has-[>kbd]:ml-[-0.15rem]",
        "inline-end":
          "order-last pr-2 has-[>button]:mr-[-0.3rem] has-[>kbd]:mr-[-0.15rem]",
        "block-start":
          "order-first w-full justify-start px-2.5 pt-2 group-has-[>input]/input-group:pt-2 [.border-b]:pb-2",
        "block-end":
          "order-last w-full justify-start px-2.5 pb-2 group-has-[>input]/input-group:pb-2 [.border-t]:pt-2",
      },
    },
    defaultVariants: {
      align: "inline-start",
    },
  }
)

function InputGroupAddon({
  className,
  align = "inline-start",
  ...props
}: React.ComponentProps<"div"> & VariantProps<typeof inputGroupAddonVariants>) {
  return (
    <div
      role="group"
      data-slot="input-group-addon"
      data-align={align}
      className={cn(inputGroupAddonVariants({ align }), className)}
      onClick={(e) => {
        if ((e.target as HTMLElement).closest("button")) {
          return
        }
        e.currentTarget.parentElement?.querySelector("input")?.focus()
      }}
      {...props}
    />
  )
}

const inputGroupButtonVariants = cva(
```

```
  "flex items-center gap-2 text-sm shadow-none",
  {
    variants: {
      size: {
        xs: "h-6 gap-1 rounded-[calc(var(--radius)-3px)] px-1.5 [&>svg:not([class*='size-'])]:size-3.5",
        sm: "",
        "icon-xs":
          "size-6 rounded-[calc(var(--radius)-3px)] p-0 has-[>svg]:p-0",
        "icon-sm": "size-8 p-0 has-[>svg]:p-0",
      },
    },
    defaultVariants: {
      size: "xs",
    },
  }
)

function InputGroupButton({
  className,
  type = "button",
  variant = "ghost",
  size = "xs",
  ...props
}: Omit<React.ComponentProps<typeof Button>, "size"> &
  VariantProps<typeof inputGroupButtonVariants>) {
  return (
    <Button
      type={type}
      data-size={size}
      variant={variant}
      className={cn(inputGroupButtonVariants({ size }), className)}
      {...props}
    />
  )
}

function InputGroupText({ className, ...props }: React.ComponentProps<"span">) {
  return (
    <span
      className={cn(
        "flex items-center gap-2 text-sm text-muted-foreground [&_svg]:pointer-events-none [&_svg:not([class*='size-'])]:size-4",
        className
      )}
      {...props}
    />
  )
}

function InputGroupInput({
  className,
  ...props
}: React.ComponentProps<"input">) {
  return (
    <Input
      data-slot="input-group-control"
      className={cn(
        "flex-1 rounded-none border-0 bg-transparent shadow-none ring-0 focus-visible:ring-0 disabled:bg-transparent aria-invalid:ring-0 dark:bg-transparent dark:disabled:bg-transparent",
        className
      )}
      {...props}
    />
  )
}

function InputGroupTextarea({
  className,
  ...props
}: React.ComponentProps<"textarea">) {
  return (
    <Textarea
```

```
      data-slot="input-group-control"
      className={cn(
        "flex-1 resize-none rounded-none border-0 bg-transparent py-2 shadow-none ring-0 focus-visible:ring-0 disabled:bg-transparent aria-invalid:ring-0 dark:bg-transparent dark:disabled:bg-transpare
        className
      )}
      {...props}
    />
  )
}

export {
  InputGroup,
  InputGroupAddon,
  InputGroupButton,
  InputGroupText,
  InputGroupInput,
  InputGroupTextarea,
}
```

app/src/components/ui/input-otp.tsx

```
"use client"

import * as React from "react"
import { OTPInput, OTPInputContext } from "input-otp"

import { cn } from "@/lib/utils"
import { MinusIcon } from "lucide-react"

function InputOTP({
  className,
  containerClassName,
  ...props
}: React.ComponentProps<typeof OTPInput> & {
  containerClassName?: string
}) {
  return (
    <OTPInput
      data-slot="input-otp"
      containerClassName={cn(
        "cn-input-otp flex items-center has-disabled:opacity-50",
        containerClassName
      )}
      spellCheck={false}
      className={cn("disabled:cursor-not-allowed", className)}
      {...props}
    />
  )
}

function InputOTPGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="input-otp-group"
      className={cn(
        "flex items-center rounded-lg has-aria-invalid:border-destructive has-aria-invalid:ring-3 has-aria-invalid:ring-destructive/20 dark:has-aria-invalid:ring-destructive/40",
        className
      )}
      {...props}
    />
  )
}

function InputOTPSlot({
  index,
  className,
  ...props
}: React.ComponentProps<"div"> & {
  index: number
}) {
  const inputOTPContext = React.useContext(OTPInputContext)
  const { char, hasFakeCaret, isActive } = inputOTPContext?.slots[index] ?? {}

  return (
    <div
      data-slot="input-otp-slot"
      data-active={isActive}
      className={cn(
        "relative flex size-8 items-center justify-center border-y border-r border-input text-sm transition-all outline-none first:rounded-l-lg first:border-l last:rounded-r-lg aria-invalid:border-des
        className
      )}
      {...props}
    >
      {char}
      {hasFakeCaret && (
        <div className="pointer-events-none absolute inset-0 flex items-center justify-center">
          <div className="h-4 w-px animate-caret-blink bg-foreground duration-1000" />
        </div>
      )}
```

```
    </div>
  )
}

function InputOTPSeparator({ ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="input-otp-separator"
      className="flex items-center [&_svg:not([class*='size-'])]:size-4"
      role="separator"
      {...props}
    >
      <MinusIcon
      />
    </div>
  )
}

export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator }
```

app/src/components/ui/input.tsx

```
import * as React from "react"

import { cn } from "@/lib/utils"

function Input({ className, type, ...props }: React.ComponentProps<"input">) {
  return (
    <input
      type={type}
      data-slot="input"
      className={cn(
        /* Figma TextField (node 342:1354): white/5 fill, radius 10, 1.5px lime focus border */
        "h-10 w-full min-w-0 rounded-[10px] border-[1.5px] border-transparent bg-white/5 px-3 py-2 text-base transition-colors outline-none file:inline-flex file:h-6 file:border-0 file:bg-transparent
        className
      )}
      {...props}
    />
  )
}

export { Input }
```

app/src/components/ui/item.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Slot } from "radix-ui"

import { cn } from "@/lib/utils"
import { Separator } from "@/components/ui/separator"

function ItemGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      role="list"
      data-slot="item-group"
      className={cn(
        "group/item-group flex w-full flex-col gap-4 has-data-[size=sm]:gap-2.5 has-data-[size=xs]:gap-2",
        className
      )}
      {...props}
    />
  )
}

function ItemSeparator({
  className,
  ...props
}: React.ComponentProps<typeof Separator>) {
  return (
    <Separator
      data-slot="item-separator"
      orientation="horizontal"
      className={cn("my-2", className)}
      {...props}
    />
  )
}

const itemVariants = cva(
  "group/item flex w-full flex-wrap items-center rounded-lg border text-sm transition-colors duration-100 outline-none focus-visible:border-ring focus-visible:ring-[3px] focus-visible:ring-ring/50 [a]
  {
    variants: {
      variant: {
        default: "border-transparent",
        outline: "border-border",
        muted: "border-transparent bg-muted/50",
      },
      size: {
        default: "gap-2.5 px-3 py-2.5",
        sm: "gap-2.5 px-3 py-2.5",
        xs: "gap-2 px-2.5 py-2 in-data-[slot=dropdown-menu-content]:p-0",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

function Item({
  className,
  variant = "default",
  size = "default",
  asChild = false,
  ...props
}: React.ComponentProps<"div"> &
  VariantProps<typeof itemVariants> & { asChild?: boolean }) {
  const Comp = asChild ? Slot.Root : "div"
  return (
    <Comp
```

```
      data-slot="item"
      data-variant={variant}
      data-size={size}
      className={cn(itemVariants({ variant, size, className }))}
      {...props}
    />
  )
}

const itemMediaVariants = cva(
  "flex shrink-0 items-center justify-center gap-2 group-has-data-[slot=item-description]/item:translate-y-0.5 group-has-data-[slot=item-description]/item:self-start [&_svg]:pointer-events-none",
  {
    variants: {
      variant: {
        default: "bg-transparent",
        icon: "[&_svg:not([class*='size-'])]:size-4",
        image:
          "size-10 overflow-hidden rounded-sm group-data-[size=sm]/item:size-8 group-data-[size=xs]/item:size-6 [&_img]:size-full [&_img]:object-cover",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)

function ItemMedia({
  className,
  variant = "default",
  ...props
}: React.ComponentProps<"div"> & VariantProps<typeof itemMediaVariants>) {
  return (
    <div
      data-slot="item-media"
      data-variant={variant}
      className={cn(itemMediaVariants({ variant, className }))}
      {...props}
    />
  )
}

function ItemContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="item-content"
      className={cn(
        "flex flex-1 flex-col gap-1 group-data-[size=xs]/item:gap-0 [&+[data-slot=item-content]]:flex-none",
        className
      )}
      {...props}
    />
  )
}

function ItemTitle({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="item-title"
      className={cn(
        "line-clamp-1 flex w-fit items-center gap-2 text-sm leading-snug font-medium underline-offset-4",
        className
      )}
      {...props}
    />
  )
}

function ItemDescription({ className, ...props }: React.ComponentProps<"p">) {
  return (
    <p
```

```
      data-slot="item-description"
      className={cn(
        "line-clamp-2 text-left text-sm leading-normal font-normal text-muted-foreground group-data-[size=xs]/item:text-xs [&>a]:underline [&>a]:underline-offset-4 [&>a:hover]:text-primary",
        className
      )}
      {...props}
    />
  )
}

function ItemActions({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="item-actions"
      className={cn("flex items-center gap-2", className)}
      {...props}
    />
  )
}

function ItemHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="item-header"
      className={cn(
        "flex basis-full items-center justify-between gap-2",
        className
      )}
      {...props}
    />
  )
}

function ItemFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="item-footer"
      className={cn(
        "flex basis-full items-center justify-between gap-2",
        className
      )}
      {...props}
    />
  )
}

export {
  Item,
  ItemMedia,
  ItemContent,
  ItemActions,
  ItemGroup,
  ItemSeparator,
  ItemTitle,
  ItemDescription,
  ItemHeader,
  ItemFooter,
}
```

app/src/components/ui/kbd.tsx

```
import { cn } from "@/lib/utils"

function Kbd({ className, ...props }: React.ComponentProps<"kbd">) {
  return (
    <kbd
      data-slot="kbd"
      className={cn(
        "pointer-events-none inline-flex h-5 w-fit min-w-5 items-center justify-center gap-1 rounded-sm bg-muted px-1 font-sans text-xs font-medium text-muted-foreground select-none in-data-[slot=tool
        className
      )}
      {...props}
    />
  )
}

function KbdGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <kbd
      data-slot="kbd-group"
      className={cn("inline-flex items-center gap-1", className)}
      {...props}
    />
  )
}

export { Kbd, KbdGroup }
```

app/src/components/ui/label.tsx

```
import * as React from "react"
import { Label as LabelPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function Label({
  className,
  ...props
}: React.ComponentProps<typeof LabelPrimitive.Root>) {
  return (
    <LabelPrimitive.Root
      data-slot="label"
      className={cn(
        /* Figma TextField label: caption 12/16 medium */
        "flex items-center gap-2 text-xs leading-4 font-medium select-none group-data-[disabled=true]:pointer-events-none group-data-[disabled=true]:opacity-50 peer-disabled:cursor-not-allowed peer-di
        className
      )}
      {...props}
    />
  )
}

export { Label }
```

app/src/components/ui/marker.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Slot } from "radix-ui"

import { cn } from "@/lib/utils"

const markerVariants = cva(
  "group/marker relative flex min-h-4 w-full items-center gap-2 text-left text-sm text-muted-foreground [&_svg:not([class*='size-'])]:size-4 [a]:underline [a]:underline-offset-3 [a]:hover:text-foregro
  {
    variants: {
      variant: {
        default: "",
        separator:
          "before:mr-1 before:h-px before:min-w-0 before:flex-1 before:bg-border after:ml-1 after:h-px after:min-w-0 after:flex-1 after:bg-border",
        border: "border-b border-border pb-2",
      },
    },
  }
)

function Marker({
  className,
  variant = "default",
  asChild = false,
  ...props
}: React.ComponentProps<"div"> &
  VariantProps<typeof markerVariants> & {
    asChild?: boolean
  }) {
  const Comp = asChild ? Slot.Root : "div"

  return (
    <Comp
      data-slot="marker"
      data-variant={variant}
      className={cn(markerVariants({ variant, className }))}
      {...props}
    />
  )
}

function MarkerIcon({ className, ...props }: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="marker-icon"
      aria-hidden="true"
      className={cn(
        "size-4 shrink-0 [&_svg:not([class*='size-'])]:size-4",
        className
      )}
      {...props}
    />
  )
}

function MarkerContent({ className, ...props }: React.ComponentProps<"span">) {
  return (
    <span
      data-slot="marker-content"
      className={cn(
        "min-w-0 wrap-break-word group-data-[variant=separator]/marker:flex-none group-data-[variant=separator]/marker:text-center *:[a]:underline *:[a]:underline-offset-3 *:[a]:hover:text-foreground'
        className
      )}
      {...props}
    />
  )
}
```

```
export { Marker, MarkerIcon, MarkerContent, markerVariants }
```

app/src/components/ui/marketing-button.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Slot } from "radix-ui"

import { cn } from "@/lib/utils"
import { Spinner } from "@/components/ui/spinner"

/*
 * Figma Marketing Button (node 1533:1490): glossy CTA buttons.
 * Geometry matches the regular Button; the visual layer differs per variant.
 * Hover = white/20 lighten (black/10 darken on the white variant), applied via
 * the ::after overlay. Focus = 2px background gap + 2px lime ring, prepended
 * to each variant's shadow stack. Class strings stay fully literal so the
 * Tailwind scanner can see them.
 */
const marketingButtonVariants = cva(
  "relative inline-flex shrink-0 items-center justify-center overflow-clip font-semibold whitespace-nowrap transition-all outline-none select-none after:pointer-events-none after:absolute after:inset-
  {
    variants: {
      variant: {
        primary:
          "gloss-lime text-[#1a1a1a] shadow-[inset_0_-3px_0_0_#829b19,0_6px_4px_0_rgba(0,0,0,0.25),0_32px_24px_0_rgba(0,0,0,0.15)] hover:after:bg-white/20 focus-visible:shadow-[0_0_0_2px_#131416,0_0_0
        secondary:
          "bg-white text-[#1a1a1a] shadow-[inset_0_-3px_0_0_rgba(0,0,0,0.32),0_6px_4px_0_rgba(0,0,0,0.25),0_32px_24px_0_rgba(0,0,0,0.15)] hover:after:bg-black/10 focus-visible:shadow-[0_0_0_2px_#13141
        red: "gloss-pink text-white shadow-[inset_0_-3px_0_0_rgba(0,0,0,0.32),0_6px_4px_0_rgba(0,0,0,0.25),0_32px_24px_0_rgba(0,0,0,0.15)] hover:after:bg-white/20 focus-visible:shadow-[0_0_0_2px_#1314
        tertiary:
          "border border-white/5 bg-white/5 text-white backdrop-blur-xl shadow-[inset_0_2px_3px_0_rgba(255,255,255,0.05),0_2px_4px_-0.5px_rgba(0,0,0,0.12)] hover:after:bg-white/20 focus-visible:shadow
        ghost:
          "bg-white/5 text-white backdrop-blur-xl shadow-[inset_0_2px_3px_0_rgba(255,255,255,0.05),0_2px_4px_-0.5px_rgba(0,0,0,0.12)] hover:after:bg-white/20 focus-visible:shadow-[0_0_0_2px_#131416,0_
      },
      /* px = container padding + 6px label inset; svg edge margins restore icon padding */
      size: {
        xs: "h-6 min-w-8 gap-1.5 rounded-md px-2.5 text-xs leading-4 [&_svg:first-child]:-ml-1.5 [&_svg:last-child]:-mr-1.5 [&_svg:not([class*='size-'])]:size-3",
        sm: "h-8 min-w-8 gap-1.5 rounded-lg px-3.5 text-sm leading-4 [&_svg:first-child]:-ml-1.5 [&_svg:last-child]:-mr-1.5 [&_svg:not([class*='size-'])]:size-4",
        md: "h-10 min-w-10 gap-1.5 rounded-[10px] px-[18px] text-sm leading-4 [&_svg:first-child]:-ml-1.5 [&_svg:last-child]:-mr-1.5 [&_svg:not([class*='size-'])]:size-5",
        lg: "h-12 min-w-12 gap-1.5 rounded-xl px-[22px] text-base leading-5 [&_svg:first-child]:-ml-1.5 [&_svg:last-child]:-mr-1.5 [&_svg:not([class*='size-'])]:size-6",
        xl: "h-14 min-w-12 gap-1.5 rounded-xl px-[26px] text-lg leading-6 tracking-[-0.4px] [&_svg:first-child]:-ml-1.5 [&_svg:last-child]:-mr-1.5 [&_svg:not([class*='size-'])]:size-6",
      },
    },
    defaultVariants: {
      variant: "primary",
      size: "md",
    },
  }
)

function MarketingButton({
  className,
  variant = "primary",
  size = "md",
  asChild = false,
  loading = false,
  disabled,
  children,
  ...props
}: React.ComponentProps<"button"> &
  VariantProps<typeof marketingButtonVariants> & {
    asChild?: boolean
    loading?: boolean
  }) {
  const Comp = asChild ? Slot.Root : "button"

  return (
    <Comp
      data-slot="marketing-button"
      data-variant={variant}
      data-size={size}
      aria-busy={loading || undefined}
```

```
      data-loading={loading || undefined}
      disabled={disabled || loading}
      className={cn(
        marketingButtonVariants({ variant, size, className }),
        /* loading = "working", not "unavailable": keep full opacity, keep width */
        loading && "pointer-events-none opacity-100!"
      )}
      {...props}
    >
      {/* ponytail: asChild expects a single child, so it skips the spinner overlay */}
      {loading && !asChild ? (
        <>
          <span className="absolute inset-0 z-10 grid place-items-center">
            <Spinner />
          </span>
          <span aria-hidden className="invisible contents">
            {children}
          </span>
        </>
      ) : (
        children
      )}
    </Comp>
  )
}

/* Figma Marketing Button / Special (node 2100:1478): stacked 112px CTA */
const marketingButtonSpecialVariants = cva(
  "relative inline-flex w-28 shrink-0 flex-col items-center justify-center gap-1.5 overflow-clip rounded-xl px-5 py-6 transition-all outline-none select-none after:pointer-events-none after:absolute a
  {
    variants: {
      variant: {
        primary:
          "gloss-lime text-[#1a1a1a] shadow-[inset_0_-3px_0_0_rgba(0,0,0,0.32),0_6px_4px_0_rgba(0,0,0,0.25),0_32px_24px_0_rgba(0,0,0,0.15)] focus-visible:shadow-[0_0_0_2px_#131416,0_0_0_4px_#d1fe17,ir
        red: "gloss-pink text-white shadow-[inset_0_-3px_0_0_rgba(0,0,0,0.32),0_6px_4px_0_rgba(0,0,0,0.25),0_32px_24px_0_rgba(0,0,0,0.15)] focus-visible:shadow-[0_0_0_2px_#131416,0_0_0_4px_#d1fe17,ins
      },
    },
    defaultVariants: {
      variant: "primary",
    },
  }
)

function MarketingButtonSpecial({
  className,
  variant = "primary",
  loading = false,
  disabled,
  children,
  ...props
}: React.ComponentProps<"button"> &
  VariantProps<typeof marketingButtonSpecialVariants> & {
    loading?: boolean
  }) {
  return (
    <button
      data-slot="marketing-button-special"
      data-variant={variant}
      aria-busy={loading || undefined}
      data-loading={loading || undefined}
      disabled={disabled || loading}
      className={cn(
        marketingButtonSpecialVariants({ variant, className }),
        loading && "pointer-events-none opacity-100!"
      )}
      {...props}
    >
      {loading ? (
        <>
          <span className="absolute inset-0 z-10 grid place-items-center">
```

```
            <Spinner className="size-5" />
          </span>
          <span aria-hidden className="invisible contents">
            {children}
          </span>
        </>
      ) : (
        children
      )}
    </button>
  )
}

export {
  MarketingButton,
  MarketingButtonSpecial,
  marketingButtonVariants,
  marketingButtonSpecialVariants,
}
```

app/src/components/ui/menubar.tsx

```
"use client"

import * as React from "react"
import { Menubar as MenubarPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { CheckIcon, ChevronRightIcon } from "lucide-react"

function Menubar({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Root>) {
  return (
    <MenubarPrimitive.Root
      data-slot="menubar"
      className={cn(
        "flex h-8 items-center gap-0.5 rounded-lg border p-[3px]",
        className
      )}
      {...props}
    />
  )
}

function MenubarMenu({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Menu>) {
  return <MenubarPrimitive.Menu data-slot="menubar-menu" {...props} />
}

function MenubarGroup({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Group>) {
  return <MenubarPrimitive.Group data-slot="menubar-group" {...props} />
}

function MenubarPortal({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Portal>) {
  return <MenubarPrimitive.Portal data-slot="menubar-portal" {...props} />
}

function MenubarRadioGroup({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.RadioGroup>) {
  return (
    <MenubarPrimitive.RadioGroup data-slot="menubar-radio-group" {...props} />
  )
}

function MenubarTrigger({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Trigger>) {
  return (
    <MenubarPrimitive.Trigger
      data-slot="menubar-trigger"
      className={cn(
        "flex items-center rounded-sm px-1.5 py-[2px] text-sm font-medium outline-hidden select-none hover:bg-muted aria-expanded:bg-muted",
        className
      )}
      {...props}
    />
  )
}

function MenubarContent({
  className,
```

```
  align = "start",
  alignOffset = -4,
  sideOffset = 8,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Content>) {
  return (
    <MenubarPortal>
      <MenubarPrimitive.Content
        data-slot="menubar-content"
        align={align}
        alignOffset={alignOffset}
        sideOffset={sideOffset}
        className={cn("z-50 min-w-36 origin-(--radix-menubar-content-transform-origin) overflow-hidden rounded-lg bg-popover p-1 text-popover-foreground shadow-md ring-1 ring-foreground/10 duration-10
        {...props}
      />
    </MenubarPortal>
  )
}

function MenubarItem({
  className,
  inset,
  variant = "default",
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Item> & {
  inset?: boolean
  variant?: "default" | "destructive"
}) {
  return (
    <MenubarPrimitive.Item
      data-slot="menubar-item"
      data-inset={inset}
      data-variant={variant}
      className={cn(
        "group/menubar-item relative flex cursor-default items-center gap-1.5 rounded-md px-1.5 py-1 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground not-data-[variant=c
        className
      )}
      {...props}
    />
  )
}

function MenubarCheckboxItem({
  className,
  children,
  checked,
  inset,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.CheckboxItem> & {
  inset?: boolean
}) {
  return (
    <MenubarPrimitive.CheckboxItem
      data-slot="menubar-checkbox-item"
      data-inset={inset}
      className={cn(
        "relative flex cursor-default items-center gap-1.5 rounded-md py-1 pr-1.5 pl-7 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground focus:**:text-accent-foreground d
        className
      )}
      checked={checked}
      {...props}
    >
      <span className="pointer-events-none absolute left-1.5 flex size-4 items-center justify-center [&_svg:not([class*='size-'])]:size-4">
        <MenubarPrimitive.ItemIndicator>
          <CheckIcon
          />
        </MenubarPrimitive.ItemIndicator>
      </span>
      {children}
    </MenubarPrimitive.CheckboxItem>
```

```
  )
}

function MenubarRadioItem({
  className,
  children,
  inset,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.RadioItem> & {
  inset?: boolean
}) {
  return (
    <MenubarPrimitive.RadioItem
      data-slot="menubar-radio-item"
      data-inset={inset}
      className={cn(
        "relative flex cursor-default items-center gap-1.5 rounded-md py-1 pr-1.5 pl-7 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground focus:**:text-accent-foreground d
        className
      )}
      {...props}
    >
      <span className="pointer-events-none absolute left-1.5 flex size-4 items-center justify-center [&_svg:not([class*='size-'])]:size-4">
        <MenubarPrimitive.ItemIndicator>
          <CheckIcon
          />
        </MenubarPrimitive.ItemIndicator>
      </span>
      {children}
    </MenubarPrimitive.RadioItem>
  )
}

function MenubarLabel({
  className,
  inset,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Label> & {
  inset?: boolean
}) {
  return (
    <MenubarPrimitive.Label
      data-slot="menubar-label"
      data-inset={inset}
      className={cn(
        "px-1.5 py-1 text-sm font-medium data-inset:pl-7",
        className
      )}
      {...props}
    />
  )
}

function MenubarSeparator({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Separator>) {
  return (
    <MenubarPrimitive.Separator
      data-slot="menubar-separator"
      className={cn("-mx-1 my-1 h-px bg-border", className)}
      {...props}
    />
  )
}

function MenubarShortcut({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
```

```
    <span
      data-slot="menubar-shortcut"
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground group-focus/menubar-item:text-accent-foreground",
        className
      )}
      {...props}
    />
  )
}

function MenubarSub({
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.Sub>) {
  return <MenubarPrimitive.Sub data-slot="menubar-sub" {...props} />
}

function MenubarSubTrigger({
  className,
  inset,
  children,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.SubTrigger> & {
  inset?: boolean
}) {
  return (
    <MenubarPrimitive.SubTrigger
      data-slot="menubar-sub-trigger"
      data-inset={inset}
      className={cn(
        "flex cursor-default items-center gap-1.5 rounded-md px-1.5 py-1 text-sm outline-none select-none focus:bg-accent focus:text-accent-foreground data-inset:pl-7 data-open:bg-accent data-open:tex
        className
      )}
      {...props}
    >
      {children}
      <ChevronRightIcon className="ml-auto size-4" />
    </MenubarPrimitive.SubTrigger>
  )
}

function MenubarSubContent({
  className,
  ...props
}: React.ComponentProps<typeof MenubarPrimitive.SubContent>) {
  return (
    <MenubarPrimitive.SubContent
      data-slot="menubar-sub-content"
      className={cn("z-50 min-w-32 origin-(--radix-menubar-content-transform-origin) overflow-hidden rounded-lg bg-popover p-1 text-popover-foreground shadow-lg ring-1 ring-foreground/10 duration-100
      {...props}
    />
  )
}

export {
  Menubar,
  MenubarPortal,
  MenubarMenu,
  MenubarTrigger,
  MenubarContent,
  MenubarGroup,
  MenubarSeparator,
  MenubarLabel,
  MenubarItem,
  MenubarShortcut,
  MenubarCheckboxItem,
  MenubarRadioGroup,
  MenubarRadioItem,
  MenubarSub,
  MenubarSubTrigger,
```

```
  MenubarSubContent,
}
```

app/src/components/ui/message-scroller.tsx

```
import * as React from "react"
import {
  MessageScroller as MessageScrollerPrimitive,
  useMessageScroller,
  useMessageScrollerScrollable,
  useMessageScrollerVisibility,
} from "@shadcn/react/message-scroller"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"
import { ArrowDownIcon } from "lucide-react"

function MessageScrollerProvider(
  props: React.ComponentProps<typeof MessageScrollerPrimitive.Provider>
) {
  return <MessageScrollerPrimitive.Provider {...props} />
}

function MessageScroller({
  className,
  ...props
}: React.ComponentProps<typeof MessageScrollerPrimitive.Root>) {
  return (
    <MessageScrollerPrimitive.Root
      data-slot="message-scroller"
      className={cn(
        "group/message-scroller relative flex size-full min-h-0 flex-col overflow-hidden",
        className
      )}
      {...props}
    />
  )
}

function MessageScrollerViewport({
  className,
  ...props
}: React.ComponentProps<typeof MessageScrollerPrimitive.Viewport>) {
  return (
    <MessageScrollerPrimitive.Viewport
      data-slot="message-scroller-viewport"
      className={cn(
        "size-full min-h-0 min-w-0 scroll-fade-b scrollbar-thin scrollbar-gutter-stable overflow-y-auto overscroll-contain contain-content data-autoscrolling:scrollbar-thumb-transparent data-autoscrol
        className
      )}
      {...props}
    />
  )
}

function MessageScrollerContent({
  className,
  ...props
}: React.ComponentProps<typeof MessageScrollerPrimitive.Content>) {
  return (
    <MessageScrollerPrimitive.Content
      data-slot="message-scroller-content"
      className={cn("flex h-max min-h-full flex-col gap-6", className)}
      {...props}
    />
  )
}

function MessageScrollerItem({
  className,
  scrollAnchor = false,
  ...props
}: React.ComponentProps<typeof MessageScrollerPrimitive.Item>) {
```

```
  return (
    <MessageScrollerPrimitive.Item
      data-slot="message-scroller-item"
      scrollAnchor={scrollAnchor}
      className={cn(
        "min-w-0 shrink-0 [contain-intrinsic-size:auto_10rem] [content-visibility:auto]",
        className
      )}
      {...props}
    />
  )
}

function MessageScrollerButton({
  direction = "end",
  className,
  children,
  render,
  variant = "secondary",
  size = "icon-sm",
  ...props
}: React.ComponentProps<typeof MessageScrollerPrimitive.Button> &
  Pick<React.ComponentProps<typeof Button>, "variant" | "size">) {
  return (
    <MessageScrollerPrimitive.Button
      data-slot="message-scroller-button"
      data-direction={direction}
      data-variant={variant}
      data-size={size}
      direction={direction}
      className={cn(
        "absolute inset-s-1/2 -translate-x-1/2 border-border bg-background text-foreground transition-[translate,scale,opacity] duration-200 hover:bg-muted hover:text-foreground data-[active=false]:po
        className
      )}
      render={render ?? <Button variant={variant} size={size} />}
      {...props}
    >
      {children ?? (
        <>
          <ArrowDownIcon
          />
          <span className="sr-only">
            {direction === "end" ? "Scroll to end" : "Scroll to start"}
          </span>
        </>
      )}
    </MessageScrollerPrimitive.Button>
  )
}

export {
  MessageScrollerProvider,
  MessageScroller,
  MessageScrollerViewport,
  MessageScrollerContent,
  MessageScrollerItem,
  MessageScrollerButton,
  useMessageScroller,
  useMessageScrollerScrollable,
  useMessageScrollerVisibility,
}
```

app/src/components/ui/message.tsx

```
import * as React from "react"

import { cn } from "@/lib/utils"

function MessageGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="message-group"
      className={cn("flex min-w-0 flex-col gap-2", className)}
      {...props}
    />
  )
}

function Message({
  className,
  align = "start",
  ...props
}: React.ComponentProps<"div"> & { align?: "start" | "end" }) {
  return (
    <div
      data-slot="message"
      data-align={align}
      className={cn(
        "group/message relative flex w-full min-w-0 gap-2 text-sm data-[align=end]:flex-row-reverse",
        className
      )}
      {...props}
    />
  )
}

function MessageAvatar({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="message-avatar"
      className={cn(
        "flex w-fit min-w-8 shrink-0 items-center justify-center self-end overflow-hidden rounded-full bg-muted group-has-data-[slot=message-footer]/message:-translate-y-8",
        className
      )}
      {...props}
    />
  )
}

function MessageContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="message-content"
      className={cn(
        "flex w-full min-w-0 flex-col gap-2.5 wrap-break-word group-data-[align=end]/message:*:data-slot:self-end",
        className
      )}
      {...props}
    />
  )
}

function MessageHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="message-header"
      className={cn(
        "flex max-w-full min-w-0 items-center px-3 text-xs font-medium text-muted-foreground group-has-data-[variant=ghost]/message:px-0",
        className
      )}
      {...props}
    />
```

```
  )
}

function MessageFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="message-footer"
      className={cn(
        "flex max-w-full min-w-0 items-center px-3 text-xs font-medium text-muted-foreground group-has-data-[variant=ghost]/message:px-0 group-data-[align=end]/message:justify-end",
        className
      )}
      {...props}
    />
  )
}

export {
  MessageGroup,
  Message,
  MessageAvatar,
  MessageContent,
  MessageFooter,
  MessageHeader,
}
```

app/src/components/ui/native-select.tsx

```
import * as React from "react"

import { cn } from "@/lib/utils"
import { ChevronDownIcon } from "lucide-react"

type NativeSelectProps = Omit<React.ComponentProps<"select">, "size"> & {
  size?: "sm" | "default"
}

function NativeSelect({
  className,
  size = "default",
  ...props
}: NativeSelectProps) {
  return (
    <div
      className={cn(
        "group/native-select relative w-fit has-[select:disabled]:opacity-50",
        className
      )}
      data-slot="native-select-wrapper"
      data-size={size}
    >
      <select
        data-slot="native-select"
        data-size={size}
        className="h-8 w-full min-w-0 appearance-none rounded-lg border border-input bg-transparent py-1 pr-8 pl-2.5 text-sm transition-colors outline-none select-none selection:bg-primary selection:t
        {...props}
      />
      <ChevronDownIcon className="pointer-events-none absolute top-1/2 right-2.5 size-4 -translate-y-1/2 text-muted-foreground select-none" aria-hidden="true" data-slot="native-select-icon" />
    </div>
  )
}

function NativeSelectOption({
  className,
  ...props
}: React.ComponentProps<"option">) {
  return (
    <option
      data-slot="native-select-option"
      className={cn("bg-[Canvas] text-[CanvasText]", className)}
      {...props}
    />
  )
}

function NativeSelectOptGroup({
  className,
  ...props
}: React.ComponentProps<"optgroup">) {
  return (
    <optgroup
      data-slot="native-select-optgroup"
      className={cn("bg-[Canvas] text-[CanvasText]", className)}
      {...props}
    />
  )
}

export { NativeSelect, NativeSelectOptGroup, NativeSelectOption }
```

app/src/components/ui/navigation-menu.tsx

```
import * as React from "react"
import { cva } from "class-variance-authority"
import { NavigationMenu as NavigationMenuPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { ChevronDownIcon } from "lucide-react"

function NavigationMenu({
  className,
  children,
  viewport = true,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Root> & {
  viewport?: boolean
}) {
  return (
    <NavigationMenuPrimitive.Root
      data-slot="navigation-menu"
      data-viewport={viewport}
      className={cn(
        "group/navigation-menu relative flex max-w-max flex-1 items-center justify-center",
        className
      )}
      {...props}
    >
      {children}
      {viewport && <NavigationMenuViewport />}
    </NavigationMenuPrimitive.Root>
  )
}

function NavigationMenuList({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.List>) {
  return (
    <NavigationMenuPrimitive.List
      data-slot="navigation-menu-list"
      className={cn(
        "group flex flex-1 list-none items-center justify-center gap-0",
        className
      )}
      {...props}
    />
  )
}

function NavigationMenuItem({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Item>) {
  return (
    <NavigationMenuPrimitive.Item
      data-slot="navigation-menu-item"
      className={cn("relative", className)}
      {...props}
    />
  )
}

const navigationMenuTriggerStyle = cva(
  "group/navigation-menu-trigger inline-flex h-9 w-max items-center justify-center rounded-lg px-2.5 py-1.5 text-sm font-medium transition-all outline-none hover:bg-muted focus:bg-muted focus-visible:
)

function NavigationMenuTrigger({
  className,
  children,
  ...props
```

```
}: React.ComponentProps<typeof NavigationMenuPrimitive.Trigger>) {
  return (
    <NavigationMenuPrimitive.Trigger
      data-slot="navigation-menu-trigger"
      className={cn(navigationMenuTriggerStyle(), "group", className)}
      {...props}
    >
      {children}{" "}
      <ChevronDownIcon className="relative top-px ml-1 size-3 transition duration-300 group-data-popup-open/navigation-menu-trigger:rotate-180 group-data-open/navigation-menu-trigger:rotate-180" aria-
    </NavigationMenuPrimitive.Trigger>
  )
}

function NavigationMenuContent({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Content>) {
  return (
    <NavigationMenuPrimitive.Content
      data-slot="navigation-menu-content"
      className={cn(
        "top-0 left-0 w-full p-1 ease-[cubic-bezier(0.22,1,0.36,1)] group-data-[viewport=false]/navigation-menu:top-full group-data-[viewport=false]/navigation-menu:mt-1.5 group-data-[viewport=false]/
        className
      )}
      {...props}
    />
  )
}

function NavigationMenuViewport({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Viewport>) {
  return (
    <div
      className={cn(
        "absolute top-full left-0 isolate z-50 flex justify-center"
      )}
    >
      <NavigationMenuPrimitive.Viewport
        data-slot="navigation-menu-viewport"
        className={cn(
          "origin-top-center relative mt-1.5 h-(--radix-navigation-menu-viewport-height) w-full overflow-hidden rounded-lg bg-popover text-popover-foreground shadow ring-1 ring-foreground/10 duration-
          className
        )}
        {...props}
      />
    </div>
  )
}

function NavigationMenuLink({
  className,
  ...props
}: React.ComponentProps<typeof NavigationMenuPrimitive.Link>) {
  return (
    <NavigationMenuPrimitive.Link
      data-slot="navigation-menu-link"
      className={cn(
        "flex items-center gap-2 rounded-lg p-2 text-sm transition-all outline-none hover:bg-muted focus:bg-muted focus-visible:ring-3 focus-visible:ring-ring/50 focus-visible:outline-1 in-data-[slot=
        className
      )}
      {...props}
    />
  )
}

function NavigationMenuIndicator({
  className,
  ...props
```

```
}: React.ComponentProps<typeof NavigationMenuPrimitive.Indicator>) {
  return (
    <NavigationMenuPrimitive.Indicator
      data-slot="navigation-menu-indicator"
      className={cn(
        "top-full z-1 flex h-1.5 items-end justify-center overflow-hidden data-[state=hidden]:animate-out data-[state=hidden]:fade-out data-[state=visible]:animate-in data-[state=visible]:fade-in",
        className
      )}
      {...props}
    >
      <div className="relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm bg-border shadow-md" />
    </NavigationMenuPrimitive.Indicator>
  )
}

export {
  NavigationMenu,
  NavigationMenuList,
  NavigationMenuItem,
  NavigationMenuContent,
  NavigationMenuTrigger,
  NavigationMenuLink,
  NavigationMenuIndicator,
  NavigationMenuViewport,
  navigationMenuTriggerStyle,
}
```

app/src/components/ui/pagination.tsx

```
import * as React from "react"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"
import { ChevronLeftIcon, ChevronRightIcon, MoreHorizontalIcon } from "lucide-react"

function Pagination({ className, ...props }: React.ComponentProps<"nav">) {
  return (
    <nav
      role="navigation"
      aria-label="pagination"
      data-slot="pagination"
      className={cn("mx-auto flex w-full justify-center", className)}
      {...props}
    />
  )
}

function PaginationContent({
  className,
  ...props
}: React.ComponentProps<"ul">) {
  return (
    <ul
      data-slot="pagination-content"
      className={cn("flex items-center gap-0.5", className)}
      {...props}
    />
  )
}

function PaginationItem({ ...props }: React.ComponentProps<"li">) {
  return <li data-slot="pagination-item" {...props} />
}

type PaginationLinkProps = {
  isActive?: boolean
} & Pick<React.ComponentProps<typeof Button>, "size"> &
  React.ComponentProps<"a">

function PaginationLink({
  className,
  isActive,
  size = "icon",
  ...props
}: PaginationLinkProps) {
  return (
    <Button
      asChild
      variant={isActive ? "outline" : "ghost"}
      size={size}
      className={cn(className)}
    >
      <a
        aria-current={isActive ? "page" : undefined}
        data-slot="pagination-link"
        data-active={isActive}
        {...props}
      />
    </Button>
  )
}

function PaginationPrevious({
  className,
  text = "Previous",
  ...props
}: React.ComponentProps<typeof PaginationLink> & { text?: string }) {
```

```
  return (
    <PaginationLink
      aria-label="Go to previous page"
      size="default"
      className={cn("pl-1.5!", className)}
      {...props}
    >
      <ChevronLeftIcon data-icon="inline-start" />
      <span className="hidden sm:block">{text}</span>
    </PaginationLink>
  )
}

function PaginationNext({
  className,
  text = "Next",
  ...props
}: React.ComponentProps<typeof PaginationLink> & { text?: string }) {
  return (
    <PaginationLink
      aria-label="Go to next page"
      size="default"
      className={cn("pr-1.5!", className)}
      {...props}
    >
      <span className="hidden sm:block">{text}</span>
      <ChevronRightIcon data-icon="inline-end" />
    </PaginationLink>
  )
}

function PaginationEllipsis({
  className,
  ...props
}: React.ComponentProps<"span">) {
  return (
    <span
      aria-hidden
      data-slot="pagination-ellipsis"
      className={cn(
        "flex size-8 items-center justify-center [&_svg:not([class*='size-'])]:size-4",
        className
      )}
      {...props}
    >
      <MoreHorizontalIcon
      />
      <span className="sr-only">More pages</span>
    </span>
  )
}

export {
  Pagination,
  PaginationContent,
  PaginationEllipsis,
  PaginationItem,
  PaginationLink,
  PaginationNext,
  PaginationPrevious,
}
```

app/src/components/ui/popover.tsx

```
import * as React from "react"
import { Popover as PopoverPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function Popover({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Root>) {
  return <PopoverPrimitive.Root data-slot="popover" {...props} />
}

function PopoverTrigger({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Trigger>) {
  return <PopoverPrimitive.Trigger data-slot="popover-trigger" {...props} />
}

function PopoverContent({
  className,
  align = "center",
  sideOffset = 4,
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Content>) {
  return (
    <PopoverPrimitive.Portal>
      <PopoverPrimitive.Content
        data-slot="popover-content"
        align={align}
        sideOffset={sideOffset}
        className={cn(
          "z-50 flex w-72 origin-(--radix-popover-content-transform-origin) flex-col gap-2.5 rounded-lg bg-popover p-2.5 text-sm text-popover-foreground shadow-md ring-1 ring-foreground/10 outline-hid
          className
        )}
        {...props}
      />
    </PopoverPrimitive.Portal>
  )
}

function PopoverAnchor({
  ...props
}: React.ComponentProps<typeof PopoverPrimitive.Anchor>) {
  return <PopoverPrimitive.Anchor data-slot="popover-anchor" {...props} />
}

function PopoverHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="popover-header"
      className={cn("flex flex-col gap-0.5 text-sm", className)}
      {...props}
    />
  )
}

function PopoverTitle({ className, ...props }: React.ComponentProps<"h2">) {
  return (
    <div
      data-slot="popover-title"
      className={cn("font-medium", className)}
      {...props}
    />
  )
}

function PopoverDescription({
  className,
  ...props
```

```
}: React.ComponentProps<"p">) {
  return (
    <p
      data-slot="popover-description"
      className={cn("text-muted-foreground", className)}
      {...props}
    />
  )
}

export {
  Popover,
  PopoverAnchor,
  PopoverContent,
  PopoverDescription,
  PopoverHeader,
  PopoverTitle,
  PopoverTrigger,
}
```

app/src/components/ui/progress.tsx

```
"use client"

import * as React from "react"
import { Progress as ProgressPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function Progress({
  className,
  value,
  ...props
}: React.ComponentProps<typeof ProgressPrimitive.Root>) {
  return (
    <ProgressPrimitive.Root
      data-slot="progress"
      className={cn(
        "relative flex h-1 w-full items-center overflow-x-hidden rounded-full bg-muted",
        className
      )}
      {...props}
    >
      <ProgressPrimitive.Indicator
        data-slot="progress-indicator"
        className="size-full flex-1 bg-primary transition-all"
        style={{ transform: `translateX(-${100 - (value || 0)}%)` }}
      />
    </ProgressPrimitive.Root>
  )
}

export { Progress }
```

app/src/components/ui/radio-group.tsx

```
import * as React from "react"
import { RadioGroup as RadioGroupPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function RadioGroup({
  className,
  ...props
}: React.ComponentProps<typeof RadioGroupPrimitive.Root>) {
  return (
    <RadioGroupPrimitive.Root
      data-slot="radio-group"
      className={cn("grid w-full gap-2", className)}
      {...props}
    />
  )
}

function RadioGroupItem({
  className,
  ...props
}: React.ComponentProps<typeof RadioGroupPrimitive.Item>) {
  return (
    <RadioGroupPrimitive.Item
      data-slot="radio-group-item"
      className={cn(
        "group/radio-group-item peer relative flex aspect-square size-4 shrink-0 rounded-full border border-input outline-none after:absolute after:-inset-x-3 after:-inset-y-2 focus-visible:border-rin
        className
      )}
      {...props}
    >
      <RadioGroupPrimitive.Indicator
        data-slot="radio-group-indicator"
        className="flex size-4 items-center justify-center"
      >
        <span className="absolute top-1/2 left-1/2 size-2 -translate-x-1/2 -translate-y-1/2 rounded-full bg-primary-foreground" />
      </RadioGroupPrimitive.Indicator>
    </RadioGroupPrimitive.Item>
  )
}

export { RadioGroup, RadioGroupItem }
```

app/src/components/ui/resizable.tsx

```
"use client"

import * as ResizablePrimitive from "react-resizable-panels"

import { cn } from "@/lib/utils"

function ResizablePanelGroup({
  className,
  ...props
}: ResizablePrimitive.GroupProps) {
  return (
    <ResizablePrimitive.Group
      data-slot="resizable-panel-group"
      className={cn(
        "flex h-full w-full aria-[orientation=vertical]:flex-col",
        className
      )}
      {...props}
    />
  )
}

function ResizablePanel({ ...props }: ResizablePrimitive.PanelProps) {
  return <ResizablePrimitive.Panel data-slot="resizable-panel" {...props} />
}

function ResizableHandle({
  withHandle,
  className,
  ...props
}: ResizablePrimitive.SeparatorProps & {
  withHandle?: boolean
}) {
  return (
    <ResizablePrimitive.Separator
      data-slot="resizable-handle"
      className={cn(
        "relative flex w-px items-center justify-center bg-border ring-offset-background after:absolute after:inset-y-0 after:left-1/2 after:w-1 after:-translate-x-1/2 focus-visible:ring-1 focus-visib
        className
      )}
      {...props}
    >
      {withHandle && (
        <div className="z-10 flex h-6 w-1 shrink-0 rounded-lg bg-border" />
      )}
    </ResizablePrimitive.Separator>
  )
}

export { ResizableHandle, ResizablePanel, ResizablePanelGroup }
```

app/src/components/ui/scroll-area.tsx

```
import * as React from "react"
import { ScrollArea as ScrollAreaPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function ScrollArea({
  className,
  children,
  ...props
}: React.ComponentProps<typeof ScrollAreaPrimitive.Root>) {
  return (
    <ScrollAreaPrimitive.Root
      data-slot="scroll-area"
      className={cn("relative", className)}
      {...props}
    >
      <ScrollAreaPrimitive.Viewport
        data-slot="scroll-area-viewport"
        className="size-full rounded-[inherit] transition-[color,box-shadow] outline-none focus-visible:ring-[3px] focus-visible:ring-ring/50 focus-visible:outline-1"
      >
        {children}
      </ScrollAreaPrimitive.Viewport>
      <ScrollBar />
      <ScrollAreaPrimitive.Corner />
    </ScrollAreaPrimitive.Root>
  )
}

function ScrollBar({
  className,
  orientation = "vertical",
  ...props
}: React.ComponentProps<typeof ScrollAreaPrimitive.ScrollAreaScrollbar>) {
  return (
    <ScrollAreaPrimitive.ScrollAreaScrollbar
      data-slot="scroll-area-scrollbar"
      data-orientation={orientation}
      orientation={orientation}
      className={cn(
        "flex touch-none p-px transition-colors select-none data-horizontal:h-2.5 data-horizontal:flex-col data-horizontal:border-t data-horizontal:border-t-transparent data-vertical:h-full data-verti
        className
      )}
      {...props}
    >
      <ScrollAreaPrimitive.ScrollAreaThumb
        data-slot="scroll-area-thumb"
        className="relative flex-1 rounded-full bg-border"
      />
    </ScrollAreaPrimitive.ScrollAreaScrollbar>
  )
}

export { ScrollArea, ScrollBar }
```

app/src/components/ui/select.tsx

```
"use client"

import * as React from "react"
import { Select as SelectPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { ChevronDownIcon, CheckIcon, ChevronUpIcon } from "lucide-react"

function Select({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Root>) {
  return <SelectPrimitive.Root data-slot="select" {...props} />
}

function SelectGroup({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Group>) {
  return (
    <SelectPrimitive.Group
      data-slot="select-group"
      className={cn("scroll-my-1 p-1", className)}
      {...props}
    />
  )
}

function SelectValue({
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Value>) {
  return <SelectPrimitive.Value data-slot="select-value" {...props} />
}

function SelectTrigger({
  className,
  size = "default",
  children,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Trigger> & {
  size?: "sm" | "default"
}) {
  return (
    <SelectPrimitive.Trigger
      data-slot="select-trigger"
      data-size={size}
      className={cn(
        "flex w-fit items-center justify-between gap-1.5 rounded-lg border border-input bg-transparent py-2 pr-2 pl-2.5 text-sm whitespace-nowrap transition-colors outline-none select-none focus-visib
        className
      )}
      {...props}
    >
      {children}
      <SelectPrimitive.Icon asChild>
        <ChevronDownIcon className="pointer-events-none size-4 text-muted-foreground" />
      </SelectPrimitive.Icon>
    </SelectPrimitive.Trigger>
  )
}

function SelectContent({
  className,
  children,
  position = "item-aligned",
  align = "center",
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Content>) {
  return (
    <SelectPrimitive.Portal>
```

```
      <SelectPrimitive.Content
        data-slot="select-content"
        data-align-trigger={position === "item-aligned"}
        className={cn("relative z-50 max-h-(--radix-select-content-available-height) min-w-36 origin-(--radix-select-content-transform-origin) overflow-x-hidden overflow-y-auto rounded-lg bg-popover t
        position={position}
        align={align}
        {...props}
      >
        <SelectScrollUpButton />
        <SelectPrimitive.Viewport
          data-position={position}
          className={cn(
            "data-[position=popper]:h-(--radix-select-trigger-height) data-[position=popper]:w-full data-[position=popper]:min-w-(--radix-select-trigger-width)",
            position === "popper" && ""
          )}
        >
          {children}
        </SelectPrimitive.Viewport>
        <SelectScrollDownButton />
      </SelectPrimitive.Content>
    </SelectPrimitive.Portal>
  )
}

function SelectLabel({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Label>) {
  return (
    <SelectPrimitive.Label
      data-slot="select-label"
      className={cn("px-1.5 py-1 text-xs text-muted-foreground", className)}
      {...props}
    />
  )
}

function SelectItem({
  className,
  children,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Item>) {
  return (
    <SelectPrimitive.Item
      data-slot="select-item"
      className={cn(
        "relative flex w-full cursor-default items-center gap-1.5 rounded-md py-1 pr-8 pl-1.5 text-sm outline-hidden select-none focus:bg-accent focus:text-accent-foreground not-data-[variant=destruct
        className
      )}
      {...props}
    >
      <span className="pointer-events-none absolute right-2 flex size-4 items-center justify-center">
        <SelectPrimitive.ItemIndicator>
          <CheckIcon className="pointer-events-none" />
        </SelectPrimitive.ItemIndicator>
      </span>
      <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
    </SelectPrimitive.Item>
  )
}

function SelectSeparator({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.Separator>) {
  return (
    <SelectPrimitive.Separator
      data-slot="select-separator"
      className={cn("pointer-events-none -mx-1 my-1 h-px bg-border", className)}
      {...props}
```

```
    />
  )
}

function SelectScrollUpButton({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.ScrollUpButton>) {
  return (
    <SelectPrimitive.ScrollUpButton
      data-slot="select-scroll-up-button"
      className={cn(
        "z-10 flex cursor-default items-center justify-center bg-popover py-1 [&_svg:not([class*='size-'])]:size-4",
        className
      )}
      {...props}
    >
      <ChevronUpIcon
      />
    </SelectPrimitive.ScrollUpButton>
  )
}

function SelectScrollDownButton({
  className,
  ...props
}: React.ComponentProps<typeof SelectPrimitive.ScrollDownButton>) {
  return (
    <SelectPrimitive.ScrollDownButton
      data-slot="select-scroll-down-button"
      className={cn(
        "z-10 flex cursor-default items-center justify-center bg-popover py-1 [&_svg:not([class*='size-'])]:size-4",
        className
      )}
      {...props}
    >
      <ChevronDownIcon
      />
    </SelectPrimitive.ScrollDownButton>
  )
}

export {
  Select,
  SelectContent,
  SelectGroup,
  SelectItem,
  SelectLabel,
  SelectScrollDownButton,
  SelectScrollUpButton,
  SelectSeparator,
  SelectTrigger,
  SelectValue,
}
```

app/src/components/ui/separator.tsx

```
import * as React from "react"
import { Separator as SeparatorPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function Separator({
  className,
  orientation = "horizontal",
  decorative = true,
  ...props
}: React.ComponentProps<typeof SeparatorPrimitive.Root>) {
  return (
    <SeparatorPrimitive.Root
      data-slot="separator"
      decorative={decorative}
      orientation={orientation}
      className={cn(
        "shrink-0 bg-border data-horizontal:h-px data-horizontal:w-full data-vertical:w-px data-vertical:self-stretch",
        className
      )}
      {...props}
    />
  )
}

export { Separator }
```

app/src/components/ui/sheet.tsx

```
"use client"

import * as React from "react"
import { Dialog as SheetPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"
import { XIcon } from "lucide-react"

function Sheet({ ...props }: React.ComponentProps<typeof SheetPrimitive.Root>) {
  return <SheetPrimitive.Root data-slot="sheet" {...props} />
}

function SheetTrigger({
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Trigger>) {
  return <SheetPrimitive.Trigger data-slot="sheet-trigger" {...props} />
}

function SheetClose({
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Close>) {
  return <SheetPrimitive.Close data-slot="sheet-close" {...props} />
}

function SheetPortal({
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Portal>) {
  return <SheetPrimitive.Portal data-slot="sheet-portal" {...props} />
}

function SheetOverlay({
  className,
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Overlay>) {
  return (
    <SheetPrimitive.Overlay
      data-slot="sheet-overlay"
      className={cn(
        "fixed inset-0 z-50 bg-black/10 duration-100 supports-backdrop-filter:backdrop-blur-xs data-open:animate-in data-open:fade-in-0 data-closed:animate-out data-closed:fade-out-0",
        className
      )}
      {...props}
    />
  )
}

function SheetContent({
  className,
  children,
  side = "right",
  showCloseButton = true,
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Content> & {
  side?: "top" | "right" | "bottom" | "left"
  showCloseButton?: boolean
}) {
  return (
    <SheetPortal>
      <SheetOverlay />
      <SheetPrimitive.Content
        data-slot="sheet-content"
        data-side={side}
        className={cn(
          "fixed z-50 flex flex-col gap-4 bg-popover bg-clip-padding text-sm text-popover-foreground shadow-lg transition duration-200 ease-in-out data-[side=bottom]:inset-x-0 data-[side=bottom]:botto
          className
        )}
        {...props}
```

```
      >
        {children}
        {showCloseButton && (
          <SheetPrimitive.Close data-slot="sheet-close" asChild>
            <Button
              variant="ghost"
              className="absolute top-3 right-3"
              size="icon-sm"
            >
              <XIcon
              />
              <span className="sr-only">Close</span>
            </Button>
          </SheetPrimitive.Close>
        )}
      </SheetPrimitive.Content>
    </SheetPortal>
  )
}

function SheetHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sheet-header"
      className={cn("flex flex-col gap-0.5 p-4", className)}
      {...props}
    />
  )
}

function SheetFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sheet-footer"
      className={cn("mt-auto flex flex-col gap-2 p-4", className)}
      {...props}
    />
  )
}

function SheetTitle({
  className,
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Title>) {
  return (
    <SheetPrimitive.Title
      data-slot="sheet-title"
      className={cn(
        "font-heading text-base font-medium text-foreground",
        className
      )}
      {...props}
    />
  )
}

function SheetDescription({
  className,
  ...props
}: React.ComponentProps<typeof SheetPrimitive.Description>) {
  return (
    <SheetPrimitive.Description
      data-slot="sheet-description"
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  )
}

export {
```

```
  Sheet,
  SheetTrigger,
  SheetClose,
  SheetContent,
  SheetHeader,
  SheetFooter,
  SheetTitle,
  SheetDescription,
}
```

app/src/components/ui/sidebar.tsx

```
"use client"

import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Slot } from "radix-ui"

import { useIsMobile } from "@/hooks/use-mobile"
import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Separator } from "@/components/ui/separator"
import {
  Sheet,
  SheetContent,
  SheetDescription,
  SheetHeader,
  SheetTitle,
} from "@/components/ui/sheet"
import { Skeleton } from "@/components/ui/skeleton"
import {
  Tooltip,
  TooltipContent,
  TooltipTrigger,
} from "@/components/ui/tooltip"
import { PanelLeftIcon } from "lucide-react"

const SIDEBAR_COOKIE_NAME = "sidebar_state"
const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7
const SIDEBAR_WIDTH = "16rem"
const SIDEBAR_WIDTH_MOBILE = "18rem"
const SIDEBAR_WIDTH_ICON = "3rem"
const SIDEBAR_KEYBOARD_SHORTCUT = "b"

type SidebarContextProps = {
  state: "expanded" | "collapsed"
  open: boolean
  setOpen: (open: boolean) => void
  openMobile: boolean
  setOpenMobile: (open: boolean) => void
  isMobile: boolean
  toggleSidebar: () => void
}

const SidebarContext = React.createContext<SidebarContextProps | null>(null)

function useSidebar() {
  const context = React.useContext(SidebarContext)
  if (!context) {
    throw new Error("useSidebar must be used within a SidebarProvider.")
  }

  return context
}

function SidebarProvider({
  defaultOpen = true,
  open: openProp,
  onOpenChange: setOpenProp,
  className,
  style,
  children,
  ...props
}: React.ComponentProps<"div"> & {
  defaultOpen?: boolean
  open?: boolean
  onOpenChange?: (open: boolean) => void
}) {
  const isMobile = useIsMobile()
```

```
const [openMobile, setOpenMobile] = React.useState(false)

// This is the internal state of the sidebar.
// We use openProp and setOpenProp for control from outside the component.
const [_open, _setOpen] = React.useState(defaultOpen)
const open = openProp ?? _open
const setOpen = React.useCallback(
  (value: boolean | ((value: boolean) => boolean)) => {
    const openState = typeof value === "function" ? value(open) : value
    if (setOpenProp) {
      setOpenProp(openState)
    } else {
      _setOpen(openState)
    }

    // This sets the cookie to keep the sidebar state.
    document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/; max-age=${SIDEBAR_COOKIE_MAX_AGE}`
  },
  [setOpenProp, open]
)

// Helper to toggle the sidebar.
const toggleSidebar = React.useCallback(() => {
  return isMobile ? setOpenMobile((open) => !open) : setOpen((open) => !open)
}, [isMobile, setOpen, setOpenMobile])

// Adds a keyboard shortcut to toggle the sidebar.
React.useEffect(() => {
  const handleKeyDown = (event: KeyboardEvent) => {
    if (
      event.key === SIDEBAR_KEYBOARD_SHORTCUT &&
      (event.metaKey || event.ctrlKey)
    ) {
      event.preventDefault()
      toggleSidebar()
    }
  }

  window.addEventListener("keydown", handleKeyDown)
  return () => window.removeEventListener("keydown", handleKeyDown)
}, [toggleSidebar])

// We add a state so that we can do data-state="expanded" or "collapsed".
// This makes it easier to style the sidebar with Tailwind classes.
const state = open ? "expanded" : "collapsed"

const contextValue = React.useMemo<SidebarContextProps>(
  () => ({
    state,
    open,
    setOpen,
    isMobile,
    openMobile,
    setOpenMobile,
    toggleSidebar,
  }),
  [state, open, setOpen, isMobile, openMobile, setOpenMobile, toggleSidebar]
)

return (
  <SidebarContext.Provider value={contextValue}>
    <div
      data-slot="sidebar-wrapper"
      style={
        {
          "--sidebar-width": SIDEBAR_WIDTH,
          "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,
          ...style,
        } as React.CSSProperties
      }
```

```
        className={cn(
          "group/sidebar-wrapper flex min-h-svh w-full has-data-[variant=inset]:bg-sidebar",
          className
        )}
        {...props}
      >
        {children}
      </div>
    </SidebarContext.Provider>
  )
}

function Sidebar({
  side = "left",
  variant = "sidebar",
  collapsible = "offcanvas",
  className,
  children,
  dir,
  ...props
}: React.ComponentProps<"div"> & {
  side?: "left" | "right"
  variant?: "sidebar" | "floating" | "inset"
  collapsible?: "offcanvas" | "icon" | "none"
}) {
  const { isMobile, state, openMobile, setOpenMobile } = useSidebar()

  if (collapsible === "none") {
    return (
      <div
        data-slot="sidebar"
        className={cn(
          "flex h-full w-(--sidebar-width) flex-col bg-sidebar text-sidebar-foreground",
          className
        )}
        {...props}
      >
        {children}
      </div>
    )
  }

  if (isMobile) {
    return (
      <Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}>
        <SheetContent
          dir={dir}
          data-sidebar="sidebar"
          data-slot="sidebar"
          data-mobile="true"
          className="w-(--sidebar-width) bg-sidebar p-0 text-sidebar-foreground [&>button]:hidden"
          style={
            {
              "--sidebar-width": SIDEBAR_WIDTH_MOBILE,
            } as React.CSSProperties
          }
          side={side}
        >
          <SheetHeader className="sr-only">
            <SheetTitle>Sidebar</SheetTitle>
            <SheetDescription>Displays the mobile sidebar.</SheetDescription>
          </SheetHeader>
          <div className="flex h-full w-full flex-col">{children}</div>
        </SheetContent>
      </Sheet>
    )
  }

  return (
    <div
```

```
      className="group peer hidden text-sidebar-foreground md:block"
      data-state={state}
      data-collapsible={state === "collapsed" ? collapsible : ""}
      data-variant={variant}
      data-side={side}
      data-slot="sidebar"
    >
      {/* This is what handles the sidebar gap on desktop */}
      <div
        data-slot="sidebar-gap"
        className={cn(
          "relative w-(--sidebar-width) bg-transparent transition-[width] duration-200 ease-linear",
          "group-data-[collapsible=offcanvas]:w-0",
          "group-data-[side=right]:rotate-180",
          variant === "floating" || variant === "inset"
            ? "group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)+(--spacing(4)))]"
            : "group-data-[collapsible=icon]:w-(--sidebar-width-icon)"
        )}
      />
      <div
        data-slot="sidebar-container"
        data-side={side}
        className={cn(
          "fixed inset-y-0 z-10 hidden h-svh w-(--sidebar-width) transition-[left,right,width] duration-200 ease-linear data-[side=left]:left-0 data-[side=left]:group-data-[collapsible=offcanvas]:left
          // Adjust the padding for floating and inset variants.
          variant === "floating" || variant === "inset"
            ? "p-2 group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)+(--spacing(4))+2px)]"
            : "group-data-[collapsible=icon]:w-(--sidebar-width-icon) group-data-[side=left]:border-r group-data-[side=right]:border-l",
          className
        )}
        {...props}
      >
        <div
          data-sidebar="sidebar"
          data-slot="sidebar-inner"
          className="flex size-full flex-col bg-sidebar group-data-[variant=floating]:rounded-lg group-data-[variant=floating]:shadow-sm group-data-[variant=floating]:ring-1 group-data-[variant=floati
        >
          {children}
        </div>
      </div>
    </div>
  )
}

function SidebarTrigger({
  className,
  onClick,
  ...props
}: React.ComponentProps<typeof Button>) {
  const { toggleSidebar } = useSidebar()

  return (
    <Button
      data-sidebar="trigger"
      data-slot="sidebar-trigger"
      variant="ghost"
      size="icon-sm"
      className={cn(className)}
      onClick={(event) => {
        onClick?.(event)
        toggleSidebar()
      }}
      {...props}
    >
      <PanelLeftIcon />
      <span className="sr-only">Toggle Sidebar</span>
    </Button>
  )
}
```

```
function SidebarRail({ className, ...props }: React.ComponentProps<"button">) {
  const { toggleSidebar } = useSidebar()

  return (
    <button
      data-sidebar="rail"
      data-slot="sidebar-rail"
      aria-label="Toggle Sidebar"
      tabIndex={-1}
      onClick={toggleSidebar}
      title="Toggle Sidebar"
      className={cn(
        "absolute inset-y-0 z-20 hidden w-4 transition-all ease-linear group-data-[side=left]:-right-4 group-data-[side=right]:left-0 after:absolute after:inset-y-0 after:start-1/2 after:w-[2px] hover
        "in-data-[side=left]:cursor-w-resize in-data-[side=right]:cursor-e-resize",
        "[[data-side=left][data-state=collapsed]_&]:cursor-e-resize [[data-side=right][data-state=collapsed]_&]:cursor-w-resize",
        "group-data-[collapsible=offcanvas]:translate-x-0 group-data-[collapsible=offcanvas]:after:left-full hover:group-data-[collapsible=offcanvas]:bg-sidebar",
        "[[data-side=left][data-collapsible=offcanvas]_&]:-right-2",
        "[[data-side=right][data-collapsible=offcanvas]_&]:-left-2",
        className
      )}
      {...props}
    />
  )
}

function SidebarInset({ className, ...props }: React.ComponentProps<"main">) {
  return (
    <main
      data-slot="sidebar-inset"
      className={cn(
        "relative flex w-full flex-1 flex-col bg-background md:peer-data-[variant=inset]:m-2 md:peer-data-[variant=inset]:ml-0 md:peer-data-[variant=inset]:rounded-xl md:peer-data-[variant=inset]:shad
        className
      )}
      {...props}
    />
  )
}

function SidebarInput({
  className,
  ...props
}: React.ComponentProps<typeof Input>) {
  return (
    <Input
      data-slot="sidebar-input"
      data-sidebar="input"
      className={cn("h-8 w-full bg-background shadow-none", className)}
      {...props}
    />
  )
}

function SidebarHeader({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-header"
      data-sidebar="header"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props}
    />
  )
}

function SidebarFooter({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-footer"
      data-sidebar="footer"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props}
```

```
    />
  )
}

function SidebarSeparator({
  className,
  ...props
}: React.ComponentProps<typeof Separator>) {
  return (
    <Separator
      data-slot="sidebar-separator"
      data-sidebar="separator"
      className={cn("mx-2 w-auto bg-sidebar-border", className)}
      {...props}
    />
  )
}

function SidebarContent({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-content"
      data-sidebar="content"
      className={cn(
        "no-scrollbar flex min-h-0 flex-1 flex-col gap-0 overflow-auto group-data-[collapsible=icon]:overflow-hidden",
        className
      )}
      {...props}
    />
  )
}

function SidebarGroup({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-group"
      data-sidebar="group"
      className={cn("relative flex w-full min-w-0 flex-col p-2", className)}
      {...props}
    />
  )
}

function SidebarGroupLabel({
  className,
  asChild = false,
  ...props
}: React.ComponentProps<"div"> & { asChild?: boolean }) {
  const Comp = asChild ? Slot.Root : "div"

  return (
    <Comp
      data-slot="sidebar-group-label"
      data-sidebar="group-label"
      className={cn(
        "flex h-8 shrink-0 items-center rounded-md px-2 text-xs font-medium text-sidebar-foreground/70 ring-sidebar-ring outline-hidden transition-[margin,opacity] duration-200 ease-linear group-data-
        className
      )}
      {...props}
    />
  )
}

function SidebarGroupAction({
  className,
  asChild = false,
  ...props
}: React.ComponentProps<"button"> & { asChild?: boolean }) {
  const Comp = asChild ? Slot.Root : "button"
```

```
  return (
    <Comp
      data-slot="sidebar-group-action"
      data-sidebar="group-action"
      className={cn(
        "absolute top-3.5 right-3 flex aspect-square w-5 items-center justify-center rounded-md p-0 text-sidebar-foreground ring-sidebar-ring outline-hidden transition-transform group-data-[collapsibl
        className
      )}
      {...props}
    />
  )
}

function SidebarGroupContent({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-group-content"
      data-sidebar="group-content"
      className={cn("w-full text-sm", className)}
      {...props}
    />
  )
}

function SidebarMenu({ className, ...props }: React.ComponentProps<"ul">) {
  return (
    <ul
      data-slot="sidebar-menu"
      data-sidebar="menu"
      className={cn("flex w-full min-w-0 flex-col gap-0", className)}
      {...props}
    />
  )
}

function SidebarMenuItem({ className, ...props }: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="sidebar-menu-item"
      data-sidebar="menu-item"
      className={cn("group/menu-item relative", className)}
      {...props}
    />
  )
}

const sidebarMenuButtonVariants = cva(
  "peer/menu-button group/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md p-2 text-left text-sm ring-sidebar-ring outline-hidden transition-[width,height,padding] group-has-data-
  {
    variants: {
      variant: {
        default: "hover:bg-sidebar-accent hover:text-sidebar-accent-foreground",
        outline:
          "bg-background shadow-[0_0_0_1px_var(--sidebar-border)] hover:bg-sidebar-accent hover:text-sidebar-accent-foreground hover:shadow-[0_0_0_1px_var(--sidebar-accent)]",
      },
      size: {
        default: "h-8 text-sm",
        sm: "h-7 text-xs",
        lg: "h-12 text-sm group-data-[collapsible=icon]:p-0!",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)
```

```
function SidebarMenuButton({
  asChild = false,
  isActive = false,
  variant = "default",
  size = "default",
  tooltip,
  className,
  ...props
}: React.ComponentProps<"button"> & {
  asChild?: boolean
  isActive?: boolean
  tooltip?: string | React.ComponentProps<typeof TooltipContent>
} & VariantProps<typeof sidebarMenuButtonVariants>) {
  const Comp = asChild ? Slot.Root : "button"
  const { isMobile, state } = useSidebar()

  const button = (
    <Comp
      data-slot="sidebar-menu-button"
      data-sidebar="menu-button"
      data-size={size}
      data-active={isActive}
      className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
      {...props}
    />
  )

  if (!tooltip) {
    return button
  }

  if (typeof tooltip === "string") {
    tooltip = {
      children: tooltip,
    }
  }

  return (
    <Tooltip>
      <TooltipTrigger asChild>{button}</TooltipTrigger>
      <TooltipContent
        side="right"
        align="center"
        hidden={state !== "collapsed" || isMobile}
        {...tooltip}
      />
    </Tooltip>
  )
}

function SidebarMenuAction({
  className,
  asChild = false,
  showOnHover = false,
  ...props
}: React.ComponentProps<"button"> & {
  asChild?: boolean
  showOnHover?: boolean
}) {
  const Comp = asChild ? Slot.Root : "button"

  return (
    <Comp
      data-slot="sidebar-menu-action"
      data-sidebar="menu-action"
      className={cn(
        "absolute top-1.5 right-1 flex aspect-square w-5 items-center justify-center rounded-md p-0 text-sidebar-foreground ring-sidebar-ring outline-hidden transition-transform group-data-[collapsibl
        showOnHover &&
          "group-focus-within/menu-item:opacity-100 group-hover/menu-item:opacity-100 peer-data-active/menu-button:text-sidebar-accent-foreground aria-expanded:opacity-100 md:opacity-0",
        className
```

```
      )}
      {...props}
    />
  )
}

function SidebarMenuBadge({
  className,
  ...props
}: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="sidebar-menu-badge"
      data-sidebar="menu-badge"
      className={cn(
        "pointer-events-none absolute right-1 flex h-5 min-w-5 items-center justify-center rounded-md px-1 text-xs font-medium text-sidebar-foreground tabular-nums select-none group-data-[collapsible=
        className
      )}
      {...props}
    />
  )
}

function SidebarMenuSkeleton({
  className,
  showIcon = false,
  ...props
}: React.ComponentProps<"div"> & {
  showIcon?: boolean
}) {
  // Random width between 50 to 90%.
  const [width] = React.useState(() => {
    return `${Math.floor(Math.random() * 40) + 50}%`
  })

  return (
    <div
      data-slot="sidebar-menu-skeleton"
      data-sidebar="menu-skeleton"
      className={cn("flex h-8 items-center gap-2 rounded-md px-2", className)}
      {...props}
    >
      {showIcon && (
        <Skeleton
          className="size-4 rounded-md"
          data-sidebar="menu-skeleton-icon"
        />
      )}
      <Skeleton
        className="h-4 max-w-(--skeleton-width) flex-1"
        data-sidebar="menu-skeleton-text"
        style={{
          {
            "--skeleton-width": width,
          } as React.CSSProperties
        }
      />
    </div>
  )
}

function SidebarMenuSub({ className, ...props }: React.ComponentProps<"ul">) {
  return (
    <ul
      data-slot="sidebar-menu-sub"
      data-sidebar="menu-sub"
      className={cn(
        "mx-3.5 flex min-w-0 translate-x-px flex-col gap-1 border-l border-sidebar-border px-2.5 py-0.5 group-data-[collapsible=icon]:hidden",
        className
      )}
```

```
      {...props}
    />
  )
}

function SidebarMenuSubItem({
  className,
  ...props
}: React.ComponentProps<"li">) {
  return (
    <li
      data-slot="sidebar-menu-sub-item"
      data-sidebar="menu-sub-item"
      className={cn("group/menu-sub-item relative", className)}
      {...props}
    />
  )
}

function SidebarMenuSubButton({
  asChild = false,
  size = "md",
  isActive = false,
  className,
  ...props
}: React.ComponentProps<"a"> & {
  asChild?: boolean
  size?: "sm" | "md"
  isActive?: boolean
}) {
  const Comp = asChild ? Slot.Root : "a"

  return (
    <Comp
      data-slot="sidebar-menu-sub-button"
      data-sidebar="menu-sub-button"
      data-size={size}
      data-active={isActive}
      className={cn(
        "flex h-7 min-w-0 -translate-x-px items-center gap-2 overflow-hidden rounded-md px-2 text-sidebar-foreground ring-sidebar-ring outline-hidden group-data-[collapsible=icon]:hidden hover:bg-side
        className
      )}
      {...props}
    />
  )
}

export {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupAction,
  SidebarGroupContent,
  SidebarGroupLabel,
  SidebarHeader,
  SidebarInput,
  SidebarInset,
  SidebarMenu,
  SidebarMenuAction,
  SidebarMenuBadge,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarMenuSkeleton,
  SidebarMenuSub,
  SidebarMenuSubButton,
  SidebarMenuSubItem,
  SidebarProvider,
  SidebarRail,
  SidebarSeparator,
```

```
  SidebarTrigger,
  useSidebar,
}
```

app/src/components/ui/skeleton.tsx

```
import { cn } from "@/lib/utils"

function Skeleton({ className, ...props }: React.ComponentProps<"div">) {
  return (
    <div
      data-slot="skeleton"
      className={cn("animate-pulse rounded-md bg-muted", className)}
      {...props}
    />
  )
}

export { Skeleton }
```

app/src/components/ui/slider.tsx

```
"use client"

import * as React from "react"
import { Slider as SliderPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function Slider({
  className,
  defaultValue,
  value,
  min = 0,
  max = 100,
  ...props
}: React.ComponentProps<typeof SliderPrimitive.Root>) {
  const _values = React.useMemo(
    () =>
      Array.isArray(value)
        ? value
        : Array.isArray(defaultValue)
          ? defaultValue
          : [min, max],
    [value, defaultValue, min, max]
  )

  return (
    <SliderPrimitive.Root
      data-slot="slider"
      defaultValue={defaultValue}
      value={value}
      min={min}
      max={max}
      className={cn(
        "relative flex w-full touch-none items-center select-none data-disabled:opacity-50 data-vertical:h-full data-vertical:min-h-40 data-vertical:w-auto data-vertical:flex-col",
        className
      )}
      {...props}
    >
      <SliderPrimitive.Track
        data-slot="slider-track"
        className="relative grow overflow-hidden rounded-full bg-muted data-horizontal:h-1 data-horizontal:w-full data-vertical:h-full data-vertical:w-1"
      >
        <SliderPrimitive.Range
          data-slot="slider-range"
          className="absolute bg-primary select-none data-horizontal:h-full data-vertical:w-full"
        />
      </SliderPrimitive.Track>
      {Array.from({ length: _values.length }, (_, index) => (
        <SliderPrimitive.Thumb
          data-slot="slider-thumb"
          key={index}
          className="relative block size-3 shrink-0 rounded-full border border-ring bg-white ring-ring/50 transition-[color,box-shadow] select-none after:absolute after:-inset-2 hover:ring-3 focus-vis
        />
      ))}
    </SliderPrimitive.Root>
  )
}

export { Slider }
```

app/src/components/ui/sonner.tsx

```
import { useTheme } from "next-themes"
import { Toaster as Sonner, type ToasterProps } from "sonner"
import { CircleCheckIcon, InfoIcon, TriangleAlertIcon, OctagonXIcon, Loader2Icon } from "lucide-react"

const Toaster = ({ ...props }: ToasterProps) => {
  const { theme = "system" } = useTheme()

  return (
    <Sonner
      theme={theme as ToasterProps["theme"]}
      className="toaster group"
      closeButton
      icons={{
        success: (
          <CircleCheckIcon className="size-4" />
        ),
        info: (
          <InfoIcon className="size-4" />
        ),
        warning: (
          <TriangleAlertIcon className="size-4" />
        ),
        error: (
          <OctagonXIcon className="size-4" />
        ),
        loading: (
          <Loader2Icon className="size-4 animate-spin" />
        ),
      }}
      style={
        {
          "--normal-bg": "var(--popover)",
          "--normal-text": "var(--popover-foreground)",
          "--normal-border": "var(--border)",
          "--border-radius": "var(--radius)",
        } as React.CSSProperties
      }
      toastOptions={{
        classNames: {
          toast: "cn-toast",
        },
      }}
      {...props}
    />
  )
}

export { Toaster }
```

app/src/components/ui/spinner.tsx

```
import { cn } from "@/lib/utils"
import { Loader2Icon } from "lucide-react"

function Spinner({ className, ...props }: React.ComponentProps<"svg">) {
  return (
    <Loader2Icon data-slot="spinner" role="status" aria-label="Loading" className={cn("size-4 animate-spin", className)} {...props} />
  )
}

export { Spinner }
```

app/src/components/ui/switch.tsx

```
"use client"

import * as React from "react"
import { Switch as SwitchPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

/* Figma Switch (node 285:266): sm 28×16 / md 40×20 / default 44×24, 2px thumb inset */
function Switch({
  className,
  size = "default",
  ...props
}: React.ComponentProps<typeof SwitchPrimitive.Root> & {
  size?: "sm" | "md" | "default"
}) {
  return (
    <SwitchPrimitive.Root
      data-slot="switch"
      data-size={size}
      className={cn(
        "peer group/switch relative inline-flex shrink-0 items-center rounded-full transition-all outline-none after:absolute after:-inset-x-3 after:-inset-y-2 focus-visible:ring-3 focus-visible:ring-
        "data-[size=default]:h-6 data-[size=default]:w-11 data-[size=md]:h-5 data-[size=md]:w-10 data-[size=sm]:h-4 data-[size=sm]:w-7",
        className
      )}
      {...props}
    >
      <SwitchPrimitive.Thumb
        data-slot="switch-thumb"
        className={cn(
          "pointer-events-none block translate-x-0.5 rounded-full bg-white shadow-[0_1px_2px_0_rgba(0,0,0,0.15)] transition-transform data-checked:border data-checked:border-black/12 data-checked:shad
          "group-data-[size=default]/switch:size-5 group-data-[size=default]/switch:data-checked:translate-x-[22px] group-data-[size=md]/switch:size-4 group-data-[size=md]/switch:data-checked:translat
        )}
      />
    </SwitchPrimitive.Root>
  )
}

export { Switch }
```

app/src/components/ui/table.tsx

```
import * as React from "react"

import { cn } from "@/lib/utils"

function Table({ className, ...props }: React.ComponentProps<"table">) {
  return (
    <div
      data-slot="table-container"
      className="relative w-full overflow-x-auto"
    >
      <table
        data-slot="table"
        className={cn("w-full caption-bottom text-sm", className)}
        {...props}
      />
    </div>
  )
}

function TableHeader({ className, ...props }: React.ComponentProps<"thead">) {
  return (
    <thead
      data-slot="table-header"
      className={cn("[&_tr]:border-b", className)}
      {...props}
    />
  )
}

function TableBody({ className, ...props }: React.ComponentProps<"tbody">) {
  return (
    <tbody
      data-slot="table-body"
      className={cn("[&_tr:last-child]:border-0", className)}
      {...props}
    />
  )
}

function TableFooter({ className, ...props }: React.ComponentProps<"tfoot">) {
  return (
    <tfoot
      data-slot="table-footer"
      className={cn(
        "border-t bg-muted/50 font-medium [&>tr]:last:border-b-0",
        className
      )}
      {...props}
    />
  )
}

function TableRow({ className, ...props }: React.ComponentProps<"tr">) {
  return (
    <tr
      data-slot="table-row"
      className={cn(
        "border-b transition-colors hover:bg-muted/50 has-aria-expanded:bg-muted/50 data-[state=selected]:bg-muted",
        className
      )}
      {...props}
    />
  )
}

function TableHead({ className, ...props }: React.ComponentProps<"th">) {
  return (
    <th
```

```
      data-slot="table-head"
      className={cn(
        "h-10 px-2 text-left align-middle font-medium whitespace-nowrap text-foreground [&:has([role=checkbox])]:pr-0",
        className
      )}
      {...props}
    />
  )
}

function TableCell({ className, ...props }: React.ComponentProps<"td">) {
  return (
    <td
      data-slot="table-cell"
      className={cn(
        "p-2 align-middle whitespace-nowrap [&:has([role=checkbox])]:pr-0",
        className
      )}
      {...props}
    />
  )
}

function TableCaption({
  className,
  ...props
}: React.ComponentProps<"caption">) {
  return (
    <caption
      data-slot="table-caption"
      className={cn("mt-4 text-sm text-muted-foreground", className)}
      {...props}
    />
  )
}

export {
  Table,
  TableHeader,
  TableBody,
  TableFooter,
  TableHead,
  TableRow,
  TableCell,
  TableCaption,
}
```

app/src/components/ui/tabs.tsx

```
"use client"

import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Tabs as TabsPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function Tabs({
  className,
  orientation = "horizontal",
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Root>) {
  return (
    <TabsPrimitive.Root
      data-slot="tabs"
      data-orientation={orientation}
      className={cn(
        "group/tabs flex gap-2 data-horizontal:flex-col",
        className
      )}
      {...props}
    />
  )
}

/*
 * Figma Segmented Control (node 3330:741) + Tabs (node 3448:624).
 * variant: glass|flat|bare = segmented surfaces, underline = tabs with
 * baseline divider. size/shape/tone are forwarded to triggers as data attrs.
 */
const tabsListVariants = cva(
  "group/tabs-list inline-flex w-fit items-center justify-center rounded-lg p-[3px] text-muted-foreground group-data-horizontal/tabs:h-8 group-data-vertical/tabs:h-fit group-data-vertical/tabs:flex-co
  {
    variants: {
      variant: {
        default: "bg-muted",
        line: "gap-1 bg-transparent",
        underline:
          "rounded-none border-b border-white/5 bg-transparent p-0 group-data-horizontal/tabs:h-auto",
        glass:
          "border border-white/5 bg-white/5 shadow-[0_2px_4px_-0.5px_rgba(0,0,0,0.12),inset_0_2px_3px_0_rgba(255,255,255,0.05)] backdrop-blur-xl group-data-horizontal/tabs:h-auto",
        flat: "border border-white/5 bg-white/5 group-data-horizontal/tabs:h-auto",
        bare: "bg-transparent p-0 group-data-horizontal/tabs:h-auto",
      },
      size: {
        sm: "",
        md: "",
      },
      shape: {
        rounded: "",
        pill: "",
      },
      tone: {
        subtle: "",
        solid: "",
        "brand-soft": "",
        brand: "",
      },
    },
    compoundVariants: [
      { variant: ["glass", "flat"], size: "md", className: "gap-1.5 rounded-2xl p-1.5" },
      { variant: ["glass", "flat"], size: "sm", className: "gap-1 rounded-xl p-1" },
      { variant: ["glass", "flat"], shape: "pill", className: "rounded-full" },
      { variant: ["bare", "underline"], size: "md", className: "gap-1.5" },
      { variant: ["bare", "underline"], size: "sm", className: "gap-1" },
    ],
    defaultVariants: {
```

```
      variant: "default",
      size: "md",
      shape: "rounded",
      tone: "subtle",
    },
  }
)

const SEGMENTED = ["glass", "flat", "bare"]

function TabsList({
  className,
  variant = "default",
  size = "md",
  shape = "rounded",
  tone = "subtle",
  ...props
}: React.ComponentProps<typeof TabsPrimitive.List> &
  VariantProps<typeof tabsListVariants>) {
  return (
    <TabsPrimitive.List
      data-slot="tabs-list"
      data-variant={variant}
      data-size={size}
      data-shape={shape}
      data-tone={tone}
      {...(SEGMENTED.includes(variant!) && { "data-segmented": "" })}
      className={cn(tabsListVariants({ variant, size, shape, tone }), className)}
      {...props}
    />
  )
}

function TabsTrigger({
  className,
  ...props
}: React.ComponentProps<typeof TabsPrimitive.Trigger>) {
  return (
    <TabsPrimitive.Trigger
      data-slot="tabs-trigger"
      className={cn(
        "relative inline-flex h-[calc(100%-1px)] flex-1 items-center justify-center gap-1.5 rounded-md border border-transparent px-1.5 py-0.5 text-sm font-medium whitespace-nowrap text-foreground/60
        "group-data-[variant=line]/tabs-list:bg-transparent group-data-[variant=line]/tabs-list:data-active:bg-transparent dark:group-data-[variant=line]/tabs-list:data-active:border-transparent dark:
        "data-active:bg-background data-active:text-foreground dark:data-active:border-input dark:data-active:bg-input/30 dark:data-active:text-foreground",
        "after:absolute after:bg-foreground after:opacity-0 after:transition-opacity group-data-horizontal/tabs:after:inset-x-0 group-data-horizontal/tabs:after:bottom-[-5px] group-data-horizontal/tab
        /* segmented (glass|flat|bare): Figma _Segment (node 3326:699) */
        "group-data-segmented/tabs-list:flex-none group-data-segmented/tabs-list:text-white/50 group-data-segmented/tabs-list:hover:text-white group-data-segmented/tabs-list:data-active:border-transpa
        "group-data-segmented/tabs-list:group-data-[size=md]/tabs-list:h-10 group-data-segmented/tabs-list:group-data-[size=md]/tabs-list:rounded-[10px] group-data-segmented/tabs-list:group-data-[size
        "group-data-segmented/tabs-list:group-data-[size=sm]/tabs-list:h-8 group-data-segmented/tabs-list:group-data-[size=sm]/tabs-list:rounded-lg group-data-segmented/tabs-list:group-data-[size=sm]/
        "group-data-segmented/tabs-list:group-data-[shape=pill]/tabs-list:rounded-full",
        /* selected fills by tone (segmented only) */
        "group-data-segmented/tabs-list:group-data-[tone=subtle]/tabs-list:data-active:glass-sheen group-data-segmented/tabs-list:group-data-[tone=subtle]/tabs-list:data-active:border-white/5 group-da
        "group-data-segmented/tabs-list:group-data-[tone=solid]/tabs-list:data-active:bg-white group-data-segmented/tabs-list:group-data-[tone=solid]/tabs-list:data-active:text-[#131517]",
        "group-data-segmented/tabs-list:group-data-[tone=brand-soft]/tabs-list:data-active:bg-primary/15 group-data-segmented/tabs-list:group-data-[tone=brand-soft]/tabs-list:data-active:text-primary"
        "group-data-segmented/tabs-list:group-data-[tone=brand]/tabs-list:data-active:bg-primary group-data-segmented/tabs-list:group-data-[tone=brand]/tabs-list:data-active:text-primary-foreground",
        /* underline: Figma Tabs (node 3448:624) + _Tab (node 3437:354) */
        "group-data-[variant=underline]/tabs-list:h-auto group-data-[variant=underline]/tabs-list:flex-none group-data-[variant=underline]/tabs-list:rounded-none group-data-[variant=underline]/tabs-li
        "group-data-[variant=underline]/tabs-list:group-data-[size=md]/tabs-list:px-2 group-data-[variant=underline]/tabs-list:group-data-[size=md]/tabs-list:py-5 group-data-[variant=underline]/tabs-l
        "group-data-[variant=underline]/tabs-list:group-data-[size=sm]/tabs-list:px-1.5 group-data-[variant=underline]/tabs-list:group-data-[size=sm]/tabs-list:py-4 group-data-[variant=underline]/tabs
        "group-data-[variant=underline]/tabs-list:group-data-[tone=brand]/tabs-list:data-active:bg-transparent group-data-[variant=underline]/tabs-list:group-data-[tone=brand]/tabs-list:data-active:te
        className
      )}
      {...props}
    />
  )
}

function TabsContent({
  className,
  ...props
```

```
}: React.ComponentProps<typeof TabsPrimitive.Content>) {
  return (
    <TabsPrimitive.Content
      data-slot="tabs-content"
      className={cn("flex-1 text-sm outline-none", className)}
      {...props}
    />
  )
}

export { Tabs, TabsList, TabsTrigger, TabsContent, tabsListVariants }
```

app/src/components/ui/textarea.tsx

```
import * as React from "react"

import { cn } from "@/lib/utils"

function Textarea({ className, ...props }: React.ComponentProps<"textarea">) {
  return (
    <textarea
      data-slot="textarea"
      className={cn(
        /* Figma TextArea (node 2134:76): white/5 fill, radius 12, min-h 80 */
        "flex field-sizing-content min-h-20 w-full rounded-xl border-[1.5px] border-transparent bg-white/5 px-3 py-2.5 text-base transition-colors outline-none placeholder:text-white/50 hover:bg-white
        className
      )}
      {...props}
    />
  )
}

export { Textarea }
```

app/src/components/ui/toggle-group.tsx

```
import * as React from "react"
import { type VariantProps } from "class-variance-authority"
import { ToggleGroup as ToggleGroupPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"
import { toggleVariants } from "@/components/ui/toggle"

const ToggleGroupContext = React.createContext<
  VariantProps<typeof toggleVariants> & {
    spacing?: number
    orientation?: "horizontal" | "vertical"
  }
>({
  size: "default",
  variant: "default",
  spacing: 2,
  orientation: "horizontal",
})

function ToggleGroup({
  className,
  variant,
  size,
  spacing = 2,
  orientation = "horizontal",
  children,
  ...props
}: React.ComponentProps<typeof ToggleGroupPrimitive.Root> &
  VariantProps<typeof toggleVariants> & {
    spacing?: number
    orientation?: "horizontal" | "vertical"
  }) {
  return (
    <ToggleGroupPrimitive.Root
      data-slot="toggle-group"
      data-variant={variant}
      data-size={size}
      data-spacing={spacing}
      data-orientation={orientation}
      style={{ "--gap": spacing } as React.CSSProperties}
      className={cn(
        "group/toggle-group flex w-fit flex-row items-center gap-[--spacing(var(--gap))] rounded-lg data-[size=sm]:rounded-[min(var(--radius-md),10px)] data-vertical:flex-col data-vertical:items-stret
        className
      )}
      {...props}
    >
      <ToggleGroupContext.Provider
        value={{ variant, size, spacing, orientation }}
      >
        {children}
      </ToggleGroupContext.Provider>
    </ToggleGroupPrimitive.Root>
  )
}

function ToggleGroupItem({
  className,
  children,
  variant = "default",
  size = "default",
  ...props
}: React.ComponentProps<typeof ToggleGroupPrimitive.Item> &
  VariantProps<typeof toggleVariants>) {
  const context = React.useContext(ToggleGroupContext)

  return (
    <ToggleGroupPrimitive.Item
      data-slot="toggle-group-item"
```

```
      data-variant={context.variant || variant}
      data-size={context.size || size}
      data-spacing={context.spacing}
      className={cn(
        "shrink-0 group-data-[spacing=0]/toggle-group:rounded-none group-data-[spacing=0]/toggle-group:px-2 focus:z-10 focus-visible:z-10 group-data-[spacing=0]/toggle-group:has-data-[icon=inline-end]
        toggleVariants({
          variant: context.variant || variant,
          size: context.size || size,
        }),
        className
      )}
      {...props}
    >
      {children}
    </ToggleGroupPrimitive.Item>
  )
}

export { ToggleGroup, ToggleGroupItem }
```

app/src/components/ui/toggle.tsx

```
import * as React from "react"
import { cva, type VariantProps } from "class-variance-authority"
import { Toggle as TogglePrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

const toggleVariants = cva(
  "group/toggle inline-flex items-center justify-center gap-1 rounded-lg text-sm font-medium whitespace-nowrap transition-all outline-none hover:bg-muted hover:text-foreground focus-visible:border-rin
  {
    variants: {
      variant: {
        default: "bg-transparent",
        outline: "border border-input bg-transparent hover:bg-muted",
      },
      size: {
        default:
          "h-8 min-w-8 px-2.5 has-data-[icon=inline-end]:pr-2 has-data-[icon=inline-start]:pl-2",
        sm: "h-7 min-w-7 rounded-[min(var(--radius-md),12px)] px-2.5 text-[0.8rem] has-data-[icon=inline-end]:pr-1.5 has-data-[icon=inline-start]:pl-1.5 [&_svg:not([class*='size-'])]:size-3.5",
        lg: "h-9 min-w-9 px-2.5 has-data-[icon=inline-end]:pr-2 has-data-[icon=inline-start]:pl-2",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

function Toggle({
  className,
  variant = "default",
  size = "default",
  ...props
}: React.ComponentProps<typeof TogglePrimitive.Root> &
  VariantProps<typeof toggleVariants>) {
  return (
    <TogglePrimitive.Root
      data-slot="toggle"
      className={cn(toggleVariants({ variant, size, className }))}
      {...props}
    />
  )
}

export { Toggle, toggleVariants }
```

app/src/components/ui/tooltip.tsx

```
"use client"

import * as React from "react"
import { Tooltip as TooltipPrimitive } from "radix-ui"

import { cn } from "@/lib/utils"

function TooltipProvider({
  delayDuration = 0,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Provider>) {
  return (
    <TooltipPrimitive.Provider
      data-slot="tooltip-provider"
      delayDuration={delayDuration}
      {...props}
    />
  )
}

function Tooltip({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Root>) {
  return <TooltipPrimitive.Root data-slot="tooltip" {...props} />
}

function TooltipTrigger({
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Trigger>) {
  return <TooltipPrimitive.Trigger data-slot="tooltip-trigger" {...props} />
}

function TooltipContent({
  className,
  sideOffset = 0,
  children,
  ...props
}: React.ComponentProps<typeof TooltipPrimitive.Content>) {
  return (
    <TooltipPrimitive.Portal>
      <TooltipPrimitive.Content
        data-slot="tooltip-content"
        sideOffset={sideOffset}
        className={cn(
          "z-50 inline-flex w-fit max-w-xs origin-(--radix-tooltip-content-transform-origin) items-center gap-1.5 rounded-md bg-foreground px-3 py-1.5 text-xs text-background has-data-[slot=kbd]:pr-1.
          className
        )}
        {...props}
      >
        {children}
        <TooltipPrimitive.Arrow className="z-50 size-2.5 translate-y-[calc(-50%_-_2px)] rotate-45 rounded-[2px] bg-foreground fill-foreground" />
      </TooltipPrimitive.Content>
    </TooltipPrimitive.Portal>
  )
}

export { Tooltip, TooltipContent, TooltipProvider, TooltipTrigger }
```

app/src/components/upload-field/index.ts

```
export { UploadField } from "./upload-field.tsx";
export type { UploadFieldProps } from "./upload-field.tsx";
```

app/src/components/upload-field/upload-field.tsx

```
"use client";

import type { ComponentPropsWithRef, ReactElement, ReactNode, Ref } from "react";
import { useRender } from "@base-ui/react/use-render";
import { X as IconCloseOutlined } from "lucide-react";
import { Icon, type IconGlyph } from "@higgsfield/quanta/icon";
import { Media } from "@higgsfield/quanta/media";
import { Typography } from "@higgsfield/quanta/typography";
import { cn } from "@/lib/utils";

/**
 * UploadField — the upload / picker tile shared across every template (Figma SC
 * App Builder image-field, node 3313:51149). Its empty state is an
 * icon-in-a-round-chip over a bold title + muted description; its filled state
 * is a white-ringed media preview with a floating remove (X) button. Quanta
 * primitives (`Icon`, `Media`, `Typography`) + `q-` tokens only.
 *
 * The canonical surface is frosted white-5% glass without an outline, with
 * subtle drop/inset shadows and a raised circular icon chip. Keep all template
 * copies of this file identical.
 *
 * Like every upload surface in the template, the field is ONLY the trigger UI — it
 * must open `AssetLibraryModal` from `@/components/asset-library` (never a custom
 * picker). Pass the empty field as the modal `trigger`, and render the filled
 * field with `preview` + `onRemove` once an asset is picked:
 *
 * ```tsx
 * import { AssetLibraryModal } from '@/components/asset-library'
 * import { UploadField } from '@/components/upload-field'
 * import { Image as IconImageOutlined } from "lucide-react";
 *
 * const [asset, setAsset] = useState<{ src: string, name: string } | null>(null)
 *
 * {asset == null
 *   ? (
 *       <AssetLibraryModal
 *         onSelect={item => setAsset(item)}
 *         trigger={(
 *           <UploadField
 *             render={<button type="button" />}
 *             icon={IconImageOutlined}
 *             title="Upload a reference"
 *             subtitle="PNG or JPG, up to 20MB"
 *           />
 *         )}
 *       />
 *     )
 *   : (
 *       <UploadField
 *         preview={asset.src}
 *         previewAlt={asset.name}
 *         onRemove={() => setAsset(null)}
 *       />
 *     )}
 * ```
 *
 * The host swaps via `render` — keep the default `<div>` for the passive filled
 * tile, or render a `<button>` for the empty trigger (it gains hover + focus
 * affordances from the classes).
 */

export type UploadFieldProps = Omit<ComponentPropsWithRef<"div">, "title"> & {
  /** Leading glyph shown in the round chip above the text (empty state). */
  icon?: IconGlyph;
  /** Bold primary line (empty state). */
  title?: ReactNode;
  /** Muted helper line under the title (empty state). */
  subtitle?: ReactNode;
```

```
  /**
   * Filled state. Pass a picked image `src` (rendered as a white-ringed `Media`
   * preview) or a custom node. When set, the icon / title / subtitle empty state
   * is replaced by the preview.
   */
  preview?: ReactNode;
  /** String previews render with the matching Quanta media primitive. */
  previewType?: "image" | "video";
  /** Alt text for the string `preview` image. */
  previewAlt?: string;
  /**
   * Show the floating remove (X) button over the filled tile. Fired when it is
   * clicked. Only rendered together with `preview`.
   */
  onRemove?: () => void;
  /** Swap the host element (e.g. an interactive `<button>` for the modal trigger). */
  render?: ReactElement;
};

export function UploadField({
  icon,
  title,
  subtitle,
  preview,
  previewType = "image",
  previewAlt = "",
  onRemove,
  className,
  render,
  ref,
  ...props
}: UploadFieldProps) {
  const interactive = render != null;
  const filled = preview != null;

  const previewNode =
    typeof preview === "string" ? (
      <div className="overflow-hidden rounded-q-300 border-2 border-white shadow-q-raised">
        <Media ratio="video" rounded="none" className="w-44 max-w-full">
          {previewType === "video" ? (
            <Media.Video src={preview} controls playsInline preload="metadata" />
          ) : (
            <Media.Image src={preview} alt={previewAlt} />
          )}
        </Media>
      </div>
    ) : (
      preview
    );

  return useRender({
    render,
    defaultTagName: "div",
    ref: ref as Ref<Element> | undefined,
    props: {
      className: cn(
        "relative isolate flex min-h-36 flex-1 flex-col items-center justify-center gap-3 overflow-hidden rounded-q-400 px-4 pb-5 pt-6 text-center shadow-[0_2px_4px_-0.5px_rgba(0,0,0,0.12)] transitior
        interactive &&
          "cursor-pointer hover:brightness-110 focus-visible:outline-2 focus-visible:outline-q-border-focus",
        className,
      ),
      children: (
        <>
          <span
            aria-hidden
            className="pointer-events-none absolute inset-0 -z-[1] rounded-q-400 bg-q-transparent-light-05 backdrop-blur-xl"
          />
          {filled ? (
            <>
              <div className="relative flex items-center justify-center">{previewNode}</div>
```

```
            {onRemove != null ? (
              <button
                type="button"
                aria-label="Remove"
                onClick={onRemove}
                className="absolute right-2 top-2 flex items-center justify-center rounded-q-200 bg-q-transparent-dark-40 p-1.5 backdrop-blur-md transition-colors hover:bg-q-transparent-dark-60 focu
              >
                <Icon as={IconCloseOutlined} size="sm" color="primary" />
              </button>
            ) : null}
          </>
        ) : (
          <>
            {icon != null ? (
              <span className="relative flex items-center justify-center rounded-q-full border border-[rgba(197,197,197,0.3)] p-2.5 text-q-icon-primary shadow-[0_20.5px_10.3px_rgba(0,0,0,0.09),0_5px>
                <span
                  aria-hidden
                  className="pointer-events-none absolute inset-0 rounded-q-full bg-q-transparent-light-05"
                />
                <Icon as={icon} size="md" className="relative" />
              </span>
            ) : null}
            <div className="relative flex w-full flex-col items-center gap-1">
              {title != null ? (
                <Typography as="span" variant="body-md-medium" color="primary">
                  {title}
                </Typography>
              ) : null}
              {subtitle != null ? (
                <Typography as="span" variant="body-sm-medium" color="secondary">
                  {subtitle}
                </Typography>
              ) : null}
            </div>
          </>
        )}
        <span
          aria-hidden
          className="pointer-events-none absolute inset-0 rounded-[inherit] shadow-[inset_0_2px_3px_rgba(255,255,255,0.05)]"
        />
      </>
    ),
    ...props,
  },
 });
}
```

app/src/components/user-generations/index.ts

```
export { UserGenerations } from "./user-generations.tsx";
export type { UserGenerationsProps } from "./user-generations.tsx";
```

app/src/components/user-generations/user-generations.tsx

```
import { JustifiedGallery } from "@/components/gallery";
import type { GalleryItem } from "@/components/gallery";

/**
 * UserGenerations — THE single canonical component for any screen that shows a
 * BROWSABLE FEED of the current user's own generations (a History tab/page, an
 * "All generations" view, an after-generate results feed, …).
 *
 * It is personal-only: it renders the CURRENT user's work in this app — never
 * other users' work, never a public/community feed.
 *
 * Everything the feed needs lives INSIDE it and is reached ONLY through this
 * component — the virtualized justified-masonry gallery, the per-tile hover
 * actions (`CardActions`), the `GenerationDetailModal` lightbox, the pulsing
 * `generating` `GenerationCard`, density control, hover-to-play video and
 * infinite scroll. The `@/components/gallery` folder is the private
 * implementation of this component; screens must NOT import it (or hand-roll a
 * grid of `GenerationCard`/`Media`) directly — they render `<UserGenerations />`.
 *
 * The one thing that is NOT this component: a finite, in-flow result/variation
 * grid inside a wizard or generator (e.g. "pick the best of these 6") — that is
 * a distinct selectable pattern, a small Quanta `Grid` of `GenerationCard`s, not
 * a browsable personal feed.
 */
/** Live feeds require real items; seeded history is available only by explicit opt-in. */
export type UserGenerationsProps =
  | { demo: true; items?: never; hasMore?: never; loadingMore?: never; onLoadMore?: never }
  | {
      demo?: false;
      items: GalleryItem[];
      hasMore: boolean;
      loadingMore: boolean;
      onLoadMore: () => void | Promise<unknown>;
    };

export function UserGenerations(props: UserGenerationsProps) {
  return props.demo ? (
    <JustifiedGallery demo grouped={false} />
  ) : (
    <JustifiedGallery
      items={props.items}
      grouped={false}
      hasMore={props.hasMore}
      loadingMore={props.loadingMore}
      onLoadMore={props.onLoadMore}
    />
  );
}
```

app/src/hooks/use-mobile.ts

```
import * as React from "react"

const MOBILE_BREAKPOINT = 768

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState<boolean | undefined>(undefined)

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`)
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
    }
    mql.addEventListener("change", onChange)
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
    return () => mql.removeEventListener("change", onChange)
  }, [])

  return !!isMobile
}
```

app/src/landing-content.ts

```
import { parseLandingContent } from "@higgsfield/app-landing";

const TEMPLATE_PREVIEW = "/assets/landing/template-preview.svg";

// TEMPLATE DEMO CONTENT. Replace it from the product AppBrief together with
// product-specific step UI and owned result/showcase media before shipping.
export const landingContent = parseLandingContent({
  hero: {
    eyebrow: "Your focused workspace",
    title: "Turn complex work into a clear, repeatable flow",
    description:
      "Organize the important work, keep progress visible and move from idea to done without losing context.",
    primaryCta: { label: "Open app", href: "/app" },
    secondaryCta: { label: "See how it works", href: "#how-it-works" },
  },
  preview: {
    kind: "route",
    title: "Interactive app preview",
    src: "/app?preview=1",
    openHref: "/app",
    openLabel: "Open full app",
  },
  steps: {
    title: "Move forward in 3 easy steps",
    description: "A focused workflow that keeps the next action obvious.",
    items: [
      {
        title: "Capture the work",
        description: "Add the items, files or ideas that need your attention.",
        preview: {
          kind: "instruction",
          icon: "layers",
          title: "Open your workspace",
          description: "Add the first item, file or idea",
        },
      },
      {
        title: "Shape the workflow",
        description: "Organize priorities and choose the controls that fit your process.",
        preview: {
          kind: "action",
          label: "Open app",
        },
      },
      {
        title: "Finish with confidence",
        description: "Track progress, review the result and keep the next step moving.",
        preview: {
          kind: "result",
          media: { kind: "image", src: TEMPLATE_PREVIEW, alt: "Completed workflow" },
        },
      },
    ],
  },
  features: {
    title: "Built for seamless progress",
    description:
      "The essential controls stay visible while secondary complexity stays out of the way.",
    items: [
      {
        icon: "layers",
        title: "Clear structure",
        description: "Keep related work together in a workspace that is easy to scan.",
      },
      {
        icon: "sliders",
        title: "Flexible controls",
        description: "Adapt views and settings to the task without crowding the main surface.",
```

```
      },
      {
        icon: "check",
        title: "Visible progress",
        description: "Understand what changed, what is active and what needs attention next.",
      },
    ],
  },
  showcase: {
    title: "See the workspace in action",
    description: "Explore the product's most important flows before opening the full app.",
    items: [
      {
        label: "Overview",
        media: { kind: "image", src: TEMPLATE_PREVIEW, alt: "Workspace overview" },
      },
      {
        label: "Focused workflow",
        media: { kind: "image", src: TEMPLATE_PREVIEW, alt: "Focused workflow view" },
      },
      {
        label: "Progress review",
        media: { kind: "image", src: TEMPLATE_PREVIEW, alt: "Progress review view" },
      },
    ],
  },
  finalCta: {
    title: "Ready to move your work forward?",
    description: "Open the full workspace and start with the task that matters most.",
    action: { label: "Open app", href: "/app" },
  },
});
```

app/src/layouts/AGENTS.md

```
# Custom layout — constrained fallback, not a blank canvas

Use this template only when `studio`, `preset`, and `app-detail` cannot express
the requested product without breaking their layout contracts. Custom does not
mean freeform: adapt `src/layouts/custom.tsx` in place and keep the shared shell
and recipes from `@/components/custom-ui`.

## Mandatory structure

0. `src/routes/index.tsx` owns the generated public landing; this layout is
   rendered full-screen by `src/routes/app.tsx`. Keep those responsibilities
   separate. `CustomTemplate` must continue accepting `previewMode` so the
   landing can show `/app?preview=1` without triggering product side effects.
1. `CustomAppShell` owns responsive navigation, page background, overflow, and
   the desktop/mobile shell. Its desktop navigation is the canonical Quanta
   `Sidebar` shared with Studio. It also owns Studio's subtle top dot-grid +
   radial-glow atmosphere; keep it anchored while content scrolls independently.
   Do not replace it with a hand-built app frame or route-local backdrop.
   Navigation may sit left or right via `navigationSide`, or be omitted. Each
   side may contain one structural rail: the navigation `Sidebar` or the
   preset-style `SettingsRail` passed through `leftRail` / `rightRail`. Never
   position side rails manually in route/layout markup. Every desktop-only
   settings rail needs a mobile Modal/Drawer or dedicated-view counterpart.
2. Every center view uses `WorkspaceContent`, but its inside is product-shaped:
   - `page`: `Page` → centered `PageHeader` → `Section` → `Panel`.
   - `canvas`: any focused domain workspace — 3D editor, node graph, timeline,
     map, code editor, media editor, or another custom interaction surface.
   - `generations`: canonical `UserGenerations` with the shared Studio
     `PromptBox` dock; cards retain their built-in generation detail opening.
     Never hand-build the grid.
   Do not force a 3D/editor/canvas product into dashboard cards or `Page`.
3. In `page` mode, `PageHeader` is the centered first-focus block with generous
   vertical padding. Repeated summary values use `MetricCard`; zero-data panels
   use `EmptyState`. These recipes are optional in `canvas`/`generations` modes.
4. Treat the shell as three visually distinct zones: left rail, center workspace,
   right rail. A rail may be empty and the center may expand, but boundaries,
   background, spacing, and ownership must remain clear.
5. Use one obvious primary action per page. Secondary controls must be quieter.
6. Default pages are focus-first: after `PageHeader`, show at most two primary
   sections before the user navigates or asks for more. Do not optimize for
   fitting everything above the fold; scrolling is healthier than compression.
7. Never duplicate a full feature on Overview when it already has a navigation
   destination. Overview shows a short summary and a `View all` path.
8. Move creation forms, advanced settings, filters, and destructive actions
   behind `Modal`, `Dropdown`, `Accordion`, or a dedicated view. Render them
   inline only when they are the page's single primary task.
9. A card, panel, toolbar group, settings section, or menu exposes at most 3–4
   primary children at once. More controls must be nested under Tabs, Accordion,
   Dropdown, Modal, a “More” action, or a dedicated sub-view. Split complicated
   elements instead of producing dense all-in-one surfaces.

## Visual contract

- Permanently dark; no theme toggle and no `dark:` classes.
- Quanta components and semantic `q-` color/type/radius/shadow tokens only.
- Normal Tailwind spacing/layout utilities are allowed.
- No raw hex/rgb/hsl colors, arbitrary color utilities, gradients, or bespoke
  shadows in app code.
- No third-party component library and no copied shadcn components.
- Lucide icons go through Quanta `Icon`. Sidebar gradient `IconTile` is the one
  exception: pass Phosphor glyphs and always render their real `weight="fill"`
  artwork. Never fake fill on a Lucide outline or use emoji as UI icons.
- Buttons default to `size="sm"`; dense navigation/toolbars use `xs`. Use `md`
  only for a deliberately prominent primary action in a spacious composition,
  never as the routine page/form default.
- Use Inter/title/body typography for product UI. Accent/display typography is
  reserved for a deliberate marketing moment, never routine headings.
- Motion is subtle, functional, and respects reduced motion.
```

```
- Preserve generous rhythm: `Page` owns large section gaps, `Section` owns
  heading-to-content spacing, and `Panel` owns internal padding. Do not tighten
  these globally to squeeze another feature onto the screen.

## Adaptation requirements

Everything shipped in `custom.tsx` is MOCK: brand, navigation, metrics, activity,
forms, labels, empty states, and actions. Replace all of it with the real app's
information architecture and working behavior. Delete recipes the app does not
need; do not leave showcase sections in the finished product.
Everything shipped in `src/landing-content.ts` and
`public/assets/landing/template-preview.svg` is also MOCK. Generate concise
product-specific copy from the same AppBrief. Keep the step previews as
instruction UI → primary action → result media, generate owned result/showcase
media under `public/assets/landing/`, then remove the template preview asset.
Every landing asset must be distinct from the app preview, marketplace cover,
and every other section asset.
Never render agent instructions, implementation reminders, adaptation warnings,
or phrases such as “replace mock content” inside the product UI.

Before finishing:

1. Run `bun run check:ui`.
2. Run `bun run check:adapted` and remove every mock marker it reports.
3. Run `bun run typecheck`.
4. Check desktop and mobile screenshots.
5. Verify empty, loading, error, disabled, and success states.
```

app/src/layouts/custom.tsx

```
"use client";

import type { FormEvent } from "react";
import { useState } from "react";
import {
  Activity,
  BarChart3,
  CheckCircle2,
  Clock3,
  FolderOpen,
  Plus,
  Sparkles,
} from "lucide-react";
import {
  GearSix as NavigationSettings,
  Pulse as NavigationActivity,
  Sparkle as NavigationGenerations,
  SquaresFour as NavigationOverview,
} from "@phosphor-icons/react";
import { Button } from "@higgsfield/quanta/button";
import { Icon } from "@higgsfield/quanta/icon";
import type { IconGlyph } from "@higgsfield/quanta/icon";
import { Input } from "@higgsfield/quanta/input";
import { Modal } from "@higgsfield/quanta/modal";
import { Typography } from "@higgsfield/quanta/typography";
import { PromptBox } from "@/components/prompt-box";
import { UserGenerations } from "@/components/user-generations";
import {
  CustomAppShell,
  EmptyState,
  MetricCard,
  Page,
  PageHeader,
  Panel,
  Section,
  VolumetricIconTile,
  WorkspaceContent,
} from "@/components/custom-ui";
import type { NavigationItem } from "@/components/custom-ui";

const NAVIGATION = [
  { id: "overview", label: "Overview", icon: NavigationOverview, gradient: "blue" },
  {
    id: "generations",
    label: "Generations",
    icon: NavigationGenerations,
    gradient: "green",
  },
  { id: "activity", label: "Activity", icon: NavigationActivity, gradient: "purple" },
  { id: "settings", label: "Settings", icon: NavigationSettings, gradient: "orange" },
] satisfies NavigationItem[];

interface ActivityItem {
  id: string;
  title: string;
  detail: string;
  icon: IconGlyph;
}

const INITIAL_ACTIVITY: ActivityItem[] = [
  { id: "1", title: "First workflow completed", detail: "A few minutes ago", icon: CheckCircle2 },
  { id: "2", title: "New item added", detail: "Today", icon: Plus },
  { id: "3", title: "Workspace updated", detail: "Yesterday", icon: Sparkles },
];

function ActivityList({ items }: { items: ActivityItem[] }) {
  if (items.length === 0) {
    return (
```

```
      <EmptyState
        icon={Activity}
        title="No activity yet"
        description="Meaningful changes will appear here when work begins."
      />
    );
  }

  return (
    <div className="divide-y divide-q-border-subtle">
      {items.map((item) => (
        <div key={item.id} className="flex items-center gap-3 py-3 first:pt-0 last:pb-0">
          <VolumetricIconTile icon={item.icon} size="sm" />
          <div className="flex min-w-0 flex-1 flex-col gap-0.5">
            <Typography as="span" variant="body-sm-medium" color="primary" truncate>
              {item.title}
            </Typography>
            <Typography as="span" variant="caption-sm-regular" color="tertiary">
              {item.detail}
            </Typography>
          </div>
        </div>
      ))}
    </div>
  );
}

function NewItemModal({ onCreate }: { onCreate: (name: string) => void }) {
  const [open, setOpen] = useState(false);
  const [name, setName] = useState("");
  const trimmedName = name.trim();

  const handleSubmit = (event: FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    if (!trimmedName) return;
    onCreate(trimmedName);
    setName("");
    setOpen(false);
  };

  return (
    <Modal.Root open={open} onOpenChange={setOpen}>
      <Modal.Trigger
        render={
          <Button variant="secondary" size="sm" start={<Icon as={Plus} size="sm" />}>
            New item
          </Button>
        }
      />
      <Modal.Content size="sm">
        <Modal.Header>
          <Modal.Title>Create item</Modal.Title>
          <Modal.CloseButton />
        </Modal.Header>
        <Modal.Body>
          <form id="new-item-form" onSubmit={handleSubmit}>
            <Input
              required
              autoFocus
              label="Name"
              placeholder="Give it a clear name"
              value={name}
              onChange={(event) => setName(event.target.value)}
            />
          </form>
        </Modal.Body>
        <Modal.Footer>
          <Modal.FooterActions>
            <Modal.Close render={<Button variant="ghost" size="sm" />}>Cancel</Modal.Close>
            <Button
```

```
              type="submit"
              form="new-item-form"
              variant="secondary"
              size="sm"
              disabled={!trimmedName}
            >
              Create
            </Button>
          </Modal.FooterActions>
        </Modal.Footer>
      </Modal.Content>
    </Modal.Root>
  );
}

function Overview({
  activity,
  onCreate,
  onOpenActivity,
}: {
  activity: ActivityItem[];
  onCreate: (name: string) => void;
  onOpenActivity: () => void;
}) {
  return (
    <Page>
      <PageHeader
        eyebrow="Workspace overview"
        title="Keep your workspace moving"
        description="Track progress and move the most important work forward."
        actions={<NewItemModal onCreate={onCreate} />}
      />

      <Section title="At a glance" description="A concise view of current performance.">
        <div className="grid gap-4 sm:grid-cols-2 xl:grid-cols-3">
          <MetricCard icon={BarChart3} label="Completed" value="128" badge="+12" />
          <MetricCard icon={Clock3} label="In progress" value="8" badge="3 due" />
          <MetricCard icon={FolderOpen} label="Collections" value="24" badge="Today" />
        </div>
      </Section>

      <Section
        title="Recent activity"
        description="Only the latest changes appear here."
        actions={
          <Button variant="ghost" size="xs" onClick={onOpenActivity}>
            View all
          </Button>
        }
      >
        <Panel>
          <ActivityList items={activity.slice(0, 3)} />
        </Panel>
      </Section>
    </Page>
  );
}

function ActivityView({
  activity,
  onCreate,
}: {
  activity: ActivityItem[];
  onCreate: (name: string) => void;
}) {
  return (
    <Page>
      <PageHeader
        title="Activity"
        description="Review recent changes across your workspace."
```

```
        actions={<NewItemModal onCreate={onCreate} />}
      />
      <Section title="Latest changes">
        <Panel>
          <ActivityList items={activity} />
        </Panel>
      </Section>
    </Page>
  );
}

function SettingsFields({
  workspaceName,
  onWorkspaceNameChange,
}: {
  workspaceName: string;
  onWorkspaceNameChange: (value: string) => void;
}) {
  return (
    <Input
      label="Workspace name"
      value={workspaceName}
      onChange={(event) => onWorkspaceNameChange(event.target.value)}
    />
  );
}

function SettingsView({
  workspaceName,
  onWorkspaceNameChange,
  onSave,
  saved,
}: {
  workspaceName: string;
  onWorkspaceNameChange: (value: string) => void;
  onSave: () => void;
  saved: boolean;
}) {
  return (
    <Page>
      <PageHeader title="Settings" description="Manage the details that shape this workspace." />
      <Section title="Workspace">
        <Panel className="flex flex-col gap-4">
          <SettingsFields
            workspaceName={workspaceName}
            onWorkspaceNameChange={onWorkspaceNameChange}
          />
          <div className="flex justify-end">
            <Button variant="secondary" size="sm" disabled={saved} onClick={onSave}>
              Save changes
            </Button>
          </div>
        </Panel>
      </Section>
    </Page>
  );
}

function GenerationsWorkspace({
  prompt,
  generating,
  onPromptChange,
  onGenerateToggle,
}: {
  prompt: string;
  generating: boolean;
  onPromptChange: (value: string) => void;
  onGenerateToggle: () => void;
}) {
  return (
```

```
    <WorkspaceContent mode="generations" className="pb-36">
      <UserGenerations demo />
      <div className="pointer-events-none absolute inset-x-4 bottom-4 flex justify-center">
        <PromptBox.Root surface="glass" className="pointer-events-auto w-[900px] max-w-full">
          <PromptBox.Body>
            <PromptBox.Field
              aria-label="Generation prompt"
              placeholder="Describe what you want to generate..."
              value={prompt}
              onChange={(event) => onPromptChange(event.target.value)}
            />
          </PromptBox.Body>
          <PromptBox.Generate
            disabled={!generating && prompt.trim().length === 0}
            onClick={onGenerateToggle}
          >
            {generating ? "Cancel" : "Generate"}
          </PromptBox.Generate>
        </PromptBox.Root>
      </div>
    </WorkspaceContent>
  );
}

export function CustomTemplate({ previewMode = false }: { previewMode?: boolean }) {
  const [activeId, setActiveId] = useState("overview");
  const [activity, setActivity] = useState<ActivityItem[]>(INITIAL_ACTIVITY);
  const [prompt, setPrompt] = useState("");
  const [generating, setGenerating] = useState(false);
  const [workspaceName, setWorkspaceName] = useState("Custom workspace");
  const [savedWorkspaceName, setSavedWorkspaceName] = useState(workspaceName);

  const handleCreate = (name: string) => {
    setActivity((current) => [
      {
        id: crypto.randomUUID(),
        title: `${name} created`,
        detail: "Just now",
        icon: Plus,
      },
      ...current,
    ]);
  };

  return (
    <div
      className="h-dvh"
      data-app-preview={previewMode ? "true" : undefined}
      inert={previewMode ? true : undefined}
    >
      <CustomAppShell
        brand="Custom App"
        navigation={NAVIGATION}
        activeId={activeId}
        onNavigate={setActiveId}
      >
        {activeId === "generations" ? (
          <GenerationsWorkspace
            prompt={prompt}
            generating={generating}
            onPromptChange={setPrompt}
            onGenerateToggle={() => setGenerating((current) => !current)}
          />
        ) : (
          <WorkspaceContent mode="page">
            {activeId === "overview" ? (
              <Overview
                activity={activity}
                onCreate={handleCreate}
                onOpenActivity={() => setActiveId("activity")}
```

```
              />
            ) : activeId === "activity" ? (
              <ActivityView activity={activity} onCreate={handleCreate} />
            ) : (
              <SettingsView
                workspaceName={workspaceName}
                onWorkspaceNameChange={setWorkspaceName}
                onSave={() => setSavedWorkspaceName(workspaceName)}
                saved={!workspaceName.trim() || workspaceName === savedWorkspaceName}
              />
            )}
          </WorkspaceContent>
        )}
      </CustomAppShell>
    </div>
  );
}
```

app/src/lib/api/example.functions.ts

```
import { createServerFn } from "@tanstack/react-start";
import { z } from "zod";

import { bindings } from "../bindings.server";

// Example server function that touches the app's D1 binding (server-only).
export const getGreeting = createServerFn({ method: "POST" })
  .validator(z.object({ name: z.string().min(1) }))
  .handler(async ({ data }) => {
    const { DB, HF_ENV } = bindings();
    let count = 0;
    if (DB) {
      const row = await DB.prepare("SELECT 1 AS n").first<{ n: number }>();
      count = row?.n ?? 0;
    }
    return { greeting: `Hello, ${data.name}!`, env: HF_ENV ?? "unknown", count };
  });
```

app/src/lib/bindings.server.ts

```
// Server-only access to this app's Cloudflare bindings. Each is present ONLY if
// opted into via app.manifest.json (D1 `DB`, R2 `STORAGE`, KV `KV`, and the
// container `CONTAINER`) — so the accessors are optional; guard before use.
// `cloudflare:workers` is the Workers-runtime module that exposes the Worker
// env (bindings) — usable inside any server-side code (server functions,
// server routes). It is NOT bundled; the runtime provides it.
import { env } from "cloudflare:workers";
// Import the binding types directly — NOT via the global tsconfig `types` list,
// which would clobber the DOM globals the client/SSR React code relies on.
import type {
  D1Database,
  DurableObjectNamespace,
  KVNamespace,
  R2Bucket,
} from "@cloudflare/workers-types";

type AppEnv = {
  DB?: D1Database;
  STORAGE?: R2Bucket;
  KV?: KVNamespace;
  // The container's Durable Object — present only when "container" is set in
  // the manifest. Reach an instance with env.CONTAINER.getByName(id), then
  // .fetch(). See skills/containers.md.
  CONTAINER?: DurableObjectNamespace;
  HF_ENV?: string;
  APP_SLUG?: string;
};

export function bindings(): AppEnv {
  return env as unknown as AppEnv;
}
```

app/src/lib/config.server.ts

```
import process from "node:process";

// Server-only config. The .server.ts suffix prevents Vite from bundling
// this file into the client — values here never reach the browser.
//
// On Cloudflare Workers, env binds at REQUEST time. Module-scope reads
// (e.g. `const x = process.env.X`) resolve to undefined — always read
// process.env INSIDE a function or handler.
//
// When to use which env-access pattern:
//   - .server.ts module (this file): server-only helpers reused across
//     handlers. Wrap reads in a function so they run per-request.
//   - inline process.env inside a createServerFn handler: one-off reads
//     not reused elsewhere.
//   - import.meta.env.VITE_FOO: PUBLIC config readable from both client
//     and server (analytics IDs, public URLs). Define in .env with the
//     VITE_ prefix. Never put secrets here — they ship to the browser.

export function getServerConfig() {
  return {
    nodeEnv: process.env.NODE_ENV,
    // Add server-only values here, e.g.:
    //   databaseUrl: process.env.DATABASE_URL,
    //   stripeSecretKey: process.env.STRIPE_SECRET_KEY,
  };
}
```

app/src/lib/download-media.ts

```
const EXTENSION_BY_TYPE: Record<string, string> = {
  "image/jpeg": "jpg",
  "image/png": "png",
  "image/webp": "webp",
  "image/gif": "gif",
  "video/mp4": "mp4",
  "video/webm": "webm",
};

function filenameFromUrl(url: string, contentType?: string | null): string {
  try {
    const name = new URL(url, window.location.href).pathname.split("/").pop();
    if (name && name.includes(".")) return decodeURIComponent(name);
  } catch {}

  const extension = contentType ? EXTENSION_BY_TYPE[contentType.split(";")[0] ?? ""] : undefined;
  return `generation.${extension ?? "bin"}`;
}

function clickDownload(href: string, filename: string, openInNewTab = false) {
  const anchor = document.createElement("a");
  anchor.href = href;
  anchor.download = filename;
  anchor.rel = "noopener";
  if (openInNewTab) anchor.target = "_blank";
  document.body.appendChild(anchor);
  anchor.click();
  anchor.remove();
}

/**
 * Download a generated image or video. Fetching to a blob makes `download`
 * reliable for same-origin and CORS-enabled media; the direct-link fallback
 * still handles storage URLs that intentionally disallow fetch access.
 */
export async function downloadMedia(url: string, filename?: string): Promise<void> {
  if (typeof document === "undefined" || !url) return;

  try {
    const response = await fetch(url);
    if (!response.ok) throw new Error(`Download failed with ${response.status}`);

    const blob = await response.blob();
    const objectUrl = URL.createObjectURL(blob);
    clickDownload(objectUrl, filename ?? filenameFromUrl(url, blob.type));
    window.setTimeout(() => URL.revokeObjectURL(objectUrl), 1000);
  } catch {
    clickDownload(url, filename ?? filenameFromUrl(url), true);
  }
}
```

app/src/lib/error-capture.ts

```
// Captures the original Error out-of-band so server.ts can recover the stack
// when h3 has already swallowed the throw into a generic 500 Response.

let lastCapturedError: { error: unknown; at: number } | undefined;
const TTL_MS = 5_000;

function record(error: unknown) {
  lastCapturedError = { error, at: Date.now() };
}

if (typeof globalThis.addEventListener === "function") {
  globalThis.addEventListener("error", (event) => record((event as ErrorEvent).error ?? event));
  globalThis.addEventListener("unhandledrejection", (event) =>
    record((event as PromiseRejectionEvent).reason),
  );
}

export function consumeLastCapturedError(): unknown {
  if (!lastCapturedError) return undefined;
  if (Date.now() - lastCapturedError.at > TTL_MS) {
    lastCapturedError = undefined;
    return undefined;
  }
  const { error } = lastCapturedError;
  lastCapturedError = undefined;
  return error;
}
```

app/src/lib/error-page.ts

```
export function renderErrorPage(): string {
  return `<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <title>This page didn't load</title>
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <style>
      body { font: 15px/1.5 system-ui, -apple-system, sans-serif; background: #fafafa; color: #111; display: grid; place-items: center; min-height: 100vh; margin: 0; padding: 1.5rem; }
      .card { max-width: 28rem; width: 100%; text-align: center; padding: 2rem; }
      h1 { font-size: 1.25rem; margin: 0 0 0.5rem; }
      p { color: #4b5563; margin: 0 0 1.5rem; }
      .actions { display: flex; gap: 0.5rem; justify-content: center; flex-wrap: wrap; }
      a, button { padding: 0.5rem 1rem; border-radius: 0.375rem; font: inherit; cursor: pointer; text-decoration: none; border: 1px solid transparent; }
      .primary { background: #111; color: #fff; }
      .secondary { background: #fff; color: #111; border-color: #d1d5db; }
    </style>
  </head>
  <body>
    <div class="card">
      <h1>This page didn't load</h1>
      <p>Something went wrong on our end. You can try refreshing or head back home.</p>
      <div class="actions">
        <button class="primary" onclick="location.reload()">Try again</button>
        <a class="secondary" href="/">Go home</a>
      </div>
    </div>
  </body>
</html>`;
}
```

app/src/lib/higgsfield-error-reporting.ts

```
type HiggsfieldErrorOptions = {
  mechanism?: "manual" | "onerror" | "unhandledrejection" | "react_error_boundary";
  handled?: boolean;
  severity?: "error" | "warning" | "info";
};

type HiggsfieldEvents = {
  captureException?: (
    error: unknown,
    context?: Record<string, unknown>,
    options?: HiggsfieldErrorOptions,
  ) => void;
};

declare global {
  interface Window {
    __higgsfieldEvents?: HiggsfieldEvents;
  }
}

export function reportHiggsfieldError(error: unknown, context: Record<string, unknown> = {}) {
  if (typeof window === "undefined") return;
  window.__higgsfieldEvents?.captureException?.(
    error,
    {
      source: "react_error_boundary",
      route: window.location.pathname,
      ...context,
    },
    {
      mechanism: "react_error_boundary",
      handled: false,
      severity: "error",
    },
  );
}
```

app/src/lib/higgsfield-generation-results.ts

```
import type { Generation, JobPhase, OutputType } from "@higgsfield/fnf/client";
import {
  getJobPhase,
  getMediaType,
  getPreviewUrl,
  getRawUrl,
  hasResult,
  isTerminalJobStatus,
} from "@higgsfield/fnf/client";

export type GenerationMediaPreview =
  | {
      kind: "image";
      phase: JobPhase;
      previewUrl: string;
      rawUrl: string;
    }
  | {
      kind: "video";
      phase: JobPhase;
      rawUrl: string;
      posterUrl?: string;
      previewUrl?: string;
    }
  | {
      kind: "empty";
      phase: JobPhase;
      outputType: OutputType;
      terminal: boolean;
      reason: "pending" | "preview_unavailable" | "failed";
    };

export function selectGenerationMedia(generation: Generation): GenerationMediaPreview {
  const phase = getJobPhase(generation);
  const outputType = getMediaType(generation) ?? generation.type;

  if (!hasResult(generation)) {
    return {
      kind: "empty",
      phase,
      outputType,
      terminal: isTerminalJobStatus(generation.status),
      reason:
        phase === "failed"
          ? "failed"
          : generation.status === "completed"
            ? "preview_unavailable"
            : "pending",
    };
  }

  const rawUrl = getRawUrl(generation);
  const previewUrl = getPreviewUrl(generation);

  if (!rawUrl) {
    return {
      kind: "empty",
      phase,
      outputType,
      terminal: isTerminalJobStatus(generation.status),
      reason: generation.status === "completed" ? "preview_unavailable" : "pending",
    };
  }

  if (outputType === "video") {
    const posterUrl =
      generation.results.thumbnailUrl ??
      (previewUrl && getMediaType(previewUrl) === "image" ? previewUrl : undefined);
```

```
    return {
      kind: "video",
      phase,
      rawUrl,
      ...(previewUrl ? { previewUrl } : {}),
      ...(posterUrl ? { posterUrl } : {}),
    };
  }

  return {
    kind: "image",
    phase,
    rawUrl,
    previewUrl: previewUrl ?? rawUrl,
  };
}

export function getGenerationPrompt(generation: Generation): string | undefined {
  const prompt = generation.input.prompt?.instruction?.trim();
  return prompt && prompt.length > 0 ? prompt : undefined;
}

export function getGenerationStatusLabel(generation: Generation): string {
  return generation.status.replaceAll("_", " ");
}

export function getGenerationCreatedLabel(generation: Generation): string | undefined {
  if (generation.createdAt === undefined) return undefined;

  const ms =
    generation.createdAt > 10_000_000_000 ? generation.createdAt : generation.createdAt * 1000;
  return new Intl.DateTimeFormat(undefined, {
    month: "short",
    day: "numeric",
    hour: "2-digit",
    minute: "2-digit",
  }).format(new Date(ms));
}

export function getGenerationFailureLabel(generation: Generation): string | undefined {
  return (
    generation.failReason?.trim() ||
    (getJobPhase(generation) === "failed" ? getGenerationStatusLabel(generation) : undefined)
  );
}
```

app/src/lib/quanta-icons.ts

```
/**
 * Lucide replacement for `@higgsfield-ai/icons` (template-only).
 *
 * The vendored `@higgsfield/quanta` components import their glyphs from the
 * private, Nexus-only `@higgsfield-ai/icons` package. Generated websites build on
 * the PUBLIC npm registry and must not depend on the internal registry, so
 * `@higgsfield-ai/icons/*` is aliased to THIS module (see `vite.config.ts`
 * `resolve.alias` + `tsconfig.json` paths). Each export mirrors the
 * `@higgsfield-ai/icons/<Name>` subpath a quanta component imports, mapped to its
 * closest lucide glyph (`lucide-react`, ISC). Lucide icons are typed React
 * components driven by `currentColor`, so they drop straight into quanta's
 * `<Icon as={…}>` (which sizes + colors the glyph via CSS tokens on the svg
 * element).
 *
 * The app icon set is lucide — one icon family everywhere; import icons by name
 * (`import { Search } from "lucide-react"`), which typechecks against the real
 * export list. Only the generic UI glyphs the SHIPPED quanta components use are
 * mapped here; if a future quanta sync adds a new glyph to a component, add it
 * (the build fails with an unresolved export otherwise).
 */
import type { IconGlyph } from "@higgsfield/quanta/icon";
import {
  Check,
  ChevronDown,
  ChevronLeft,
  ChevronRight,
  ChevronsUpDown,
  Circle,
  CircleCheck,
  CircleX,
  Folder,
  Info,
  Pin,
  Plus,
  Search,
  TriangleAlert,
  X,
} from "lucide-react";

// TYPE BRIDGE, not a runtime change: lucide components are typed against the
// app's @types/react while quanta types glyphs against its own vendored copy
// (`IconGlyph`). Runtime is unaffected — the glyphs just spread props onto
// the <svg> — so re-export each icon cast to quanta's IconGlyph.
const glyph = (icon: unknown): IconGlyph => icon as IconGlyph;

export const IconMagnifyingGlass2Outlined = glyph(Search);
export const IconMagnifyingGlassOutlined = glyph(Search);
export const IconFolder1Outlined = glyph(Folder);
export const IconCheckmark2MediumOutlined = glyph(Check);
export const IconPlusMediumOutlined = glyph(Plus);
export const IconPinFilledThin = glyph(Pin);
export const IconExclamationTriangleOutlined = glyph(TriangleAlert);
export const IconCrossMediumOutlined = glyph(X);
export const IconCircleXOutlined = glyph(CircleX);
export const IconCircleOutlined = glyph(Circle);
export const IconCircleInfoOutlined = glyph(Info);
export const IconCircleCheckOutlined = glyph(CircleCheck);
export const IconChevronRightMediumOutlined = glyph(ChevronRight);
export const IconChevronLeftMediumOutlined = glyph(ChevronLeft);
export const IconChevronGrabberVerticalOutlined = glyph(ChevronsUpDown);
export const IconChevronDownMediumOutlined = glyph(ChevronDown);
export const IconChevronBottomOutlined = glyph(ChevronDown);
```

app/src/lib/security-headers.server.ts

```
/**
 * Security headers applied to every Worker response. Import in app/src/server.ts
 * and wrap the final response: `return applySecurityHeaders(response)`.
 */
export function applySecurityHeaders(response: Response): Response {
  const headers = new Headers(response.headers);
  // The deployment platform owns `frame-ancestors`; setting it here would add
  // a second, intersecting policy that can block the host preview.
  headers.set(
    "Content-Security-Policy",
    "default-src 'self'; " +
      "script-src 'self' 'unsafe-inline'; " +
      "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; " +
      "font-src 'self' https://fonts.gstatic.com; " +
      "img-src 'self' data: https:; media-src 'self' https:; " +
      "connect-src 'self' https:; " +
      "frame-src 'self' https://auth.higgsfield.app https://auth.higgsfield-dev.app; " +
      "base-uri 'self'; form-action 'self'",
  );
  headers.set("Strict-Transport-Security", "max-age=63072000; includeSubDomains; preload");
  headers.set("X-Content-Type-Options", "nosniff");
  headers.set("Referrer-Policy", "strict-origin-when-cross-origin");
  headers.set("Permissions-Policy", "camera=(), microphone=(), geolocation=()");
  headers.set("X-XSS-Protection", "0");
  return new Response(response.body, {
    status: response.status,
    statusText: response.statusText,
    headers,
  });
}
```

app/src/lib/utils.ts

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}
```

app/src/module/design-inspector/protocol.ts

```
export const DESIGN_MODE_PROTOCOL = "higgsfield.supercomputer.design-mode";
export const DESIGN_MODE_PROTOCOL_VERSION = 1;

export type DesignModeSetMessage = {
  protocol: typeof DESIGN_MODE_PROTOCOL;
  version: typeof DESIGN_MODE_PROTOCOL_VERSION;
  type: "HF_DESIGN_MODE_SET";
  sessionId: string;
  appId?: string;
  enabled: boolean;
};

export type DesignModePreviewCropMessage = {
  protocol: typeof DESIGN_MODE_PROTOCOL;
  version: typeof DESIGN_MODE_PROTOCOL_VERSION;
  type: "HF_DESIGN_PREVIEW_CROP_SET";
  sessionId: string;
  appId?: string;
  selection: DesignSelectionContext;
};

export type DesignSelectionRect = {
  x: number;
  y: number;
  width: number;
  height: number;
};

export type DesignSelectionPoint = {
  x: number;
  y: number;
};

export type DesignSelectionModifiers = {
  metaKey?: boolean;
  ctrlKey?: boolean;
  shiftKey?: boolean;
  altKey?: boolean;
};

export type DesignSelectionElement = {
  nodeId?: string;
  componentName?: string;
  sourceFile?: string;
  sourceLine?: number;
  sourceColumn?: number;
  exportName?: string;
  routeFile?: string;
  tagName: string;
  role?: string;
  ariaLabel?: string;
  textSnippet?: string;
  classList?: string[];
  attributes?: Record<string, string>;
  computedStyle?: Record<string, string>;
};

export type DesignSelectionContext = {
  protocol: typeof DESIGN_MODE_PROTOCOL;
  version: typeof DESIGN_MODE_PROTOCOL_VERSION;
  sessionId: string;
  appId?: string;
  previewOrigin?: string;
  route?: string;
  path?: string;
  timestamp: number;
  viewport: {
    width: number;
```

```
    height: number;
    scrollX: number;
    scrollY: number;
  };
  pointer?: DesignSelectionPoint;
  rect: DesignSelectionRect;
  element: DesignSelectionElement;
  ancestors: DesignSelectionElement[];
  source: "instrumented" | "heuristic";
};
```

app/src/module/design-inspector/README.md

```
# Design inspector module

Template-local child runtime for Supercomputer Design mode.

This module is not an SDK package. The Supercomputer editor owns the parent iframe UI and
postMessage validation; this template module owns what runs inside the generated
app iframe during editable preview builds.

## Files

- `registry.ts` — WeakMap source metadata registry and React ref helpers used by
  compile-time instrumentation.
- `runtime.ts` — browser runtime installed by the app root only when
  `__HF_DESIGN_INSPECTOR__` is true.
- `vite.ts` — Vite/React-Babel helpers that inject design-only callback refs into
  intrinsic JSX DOM tags and ref-capable component usages.

## Rules

- Never manually add `data-hf-*` attributes or source metadata in app code.
- Metadata lives in `WeakMap<Element, SourceMeta>`, not in DOM markup.
- The design build instruments intrinsic tags plus PascalCase/member component
  usages such as `Sparkles`, `Icon.Edit`, or `NavigationMenu.Item`. This gives
  small icon components and wrapper components their own use-site metadata when
  they forward refs to a DOM element.
- Components that swallow refs cannot expose use-site metadata. In that case,
  the runtime falls back to the nearest instrumented DOM child/ancestor or
  heuristic metadata.
- `bun run build` is production-clean by default: no inspector runtime and no
  metadata. Setting `HF_DESIGN_INSPECTOR=1` in the env turns the same build into
  the editable Supercomputer preview build — this is what deploys ship (apps-ci
  / apps-ci-dev export the flag before `bun run build`). `bun run dev:design`
  is the local inspector-on dev server.
- The only DOM mutation allowed while active is global selector state such as
  `body[data-selector-active="true"]`.
- While active, the runtime injects a temporary global cursor style so the
  iframe document uses `cursor: crosshair`; remove it when Design mode is off.
- Hover messages must include live `pointer: { x, y }` viewport coordinates and
  should be sent through `requestAnimationFrame`, even when the hovered element
  did not change. The parent tooltip follows this pointer.
- During scroll, emit `HF_DESIGN_SCROLL` with the live viewport scroll offsets.
  The parent hides transient hover chrome, keeps selected overlays visible by
  applying the scroll delta, then recomputes the hovered element at the last
  pointer position after scroll stops.
- Escape must exit Design mode. The runtime should disable local selector state
  immediately and post `HF_DESIGN_EXIT` so the parent toolbar turns off too.
- Do not log or post cookies, tokens, auth headers, storage values, input values,
  raw HTML, uploaded bytes, or raw media URLs.
```

app/src/module/design-inspector/registry.ts

```
import type { Ref, RefCallback } from "react";

export interface HiggsfieldDesignSourceMeta {
  nodeId: string;
  componentName?: string;
  sourceFile: string;
  sourceLine?: number;
  sourceColumn?: number;
  routeFile?: string;
  tagName?: string;
}

const designMetaByElement = new WeakMap<Element, HiggsfieldDesignSourceMeta>();

function assignRef<T>(ref: Ref<T> | undefined | null, value: T | null) {
  if (!ref) {
    return;
  }

  if (typeof ref === "function") {
    ref(value);
    return;
  }

  try {
    (ref as { current: T | null }).current = value;
  } catch {
    // Some refs are intentionally immutable. Design instrumentation is a
    // preview-only helper and must never break the app's original ref behavior.
  }
}

export function registerHiggsfieldDesignElement(
  element: Element | null,
  meta: HiggsfieldDesignSourceMeta,
) {
  if (!element) {
    return;
  }

  designMetaByElement.set(element, {
    ...meta,
    tagName: meta.tagName ?? element.tagName.toLowerCase(),
  });
}

export function createHiggsfieldDesignRef<T extends Element>(
  meta: HiggsfieldDesignSourceMeta,
): RefCallback<T> {
  return (element) => registerHiggsfieldDesignElement(element, meta);
}

export function composeHiggsfieldDesignRefs<T extends Element>(
  ref: Ref<T> | undefined | null,
  designRef: RefCallback<T>,
): RefCallback<T> {
  return (element) => {
    assignRef(ref, element);
    designRef(element);
  };
}

export function getHiggsfieldDesignMeta(element: Element) {
  return designMetaByElement.get(element);
}

export function findHiggsfieldDesignElement(target: Element) {
  let current: Element | null = target;
```

```
  while (current && current !== document.documentElement && current !== document.body) {
    const meta = designMetaByElement.get(current);
    if (meta) {
      return { element: current, meta };
    }
    current = current.parentElement;
  }
  return null;
}
```

app/src/module/design-inspector/runtime.ts

```
import { findHiggsfieldDesignElement, getHiggsfieldDesignMeta } from "./registry";
import {
  DESIGN_MODE_PROTOCOL,
  DESIGN_MODE_PROTOCOL_VERSION,
  type DesignModePreviewCropMessage,
  type DesignModeSetMessage,
  type DesignSelectionContext,
  type DesignSelectionElement,
  type DesignSelectionPoint,
} from "./protocol";

type InspectorState = {
  enabled: boolean;
  sessionId: string | null;
  appId: string | null;
  parentOrigin: string | null;
  lastHoverElement: Element | null;
  lastPointer: DesignSelectionPoint | null;
  pendingHover: {
    element: Element;
    pointer: DesignSelectionPoint;
  } | null;
  raf: number | null;
  scrollStopTimer: number | null;
};

declare global {
  interface Window {
    __higgsfieldDesignInspectorInstalled?: boolean;
  }
}

const SAFE_ATTRIBUTE_NAMES = new Set([
  "alt",
  "aria-label",
  "href",
  "id",
  "name",
  "placeholder",
  "src",
  "title",
  "type",
]);

const SAFE_STYLE_NAMES = [
  "backgroundColor",
  "borderColor",
  "borderRadius",
  "color",
  "display",
  "fontFamily",
  "fontSize",
  "fontWeight",
  "gap",
  "height",
  "margin",
  "padding",
  "position",
  "width",
] as const;

const state: InspectorState = {
  enabled: false,
  sessionId: null,
  appId: null,
  parentOrigin: null,
  lastHoverElement: null,
  lastPointer: null,
```

```
  pendingHover: null,
  raf: null,
  scrollStopTimer: null,
};

const CURSOR_STYLE_ID = "higgsfield-design-inspector-cursor-style";

function isRecord(value: unknown): value is Record<string, unknown> {
  return typeof value === "object" && value !== null && !Array.isArray(value);
}

function isSetMessage(value: unknown): value is DesignModeSetMessage {
  return (
    isRecord(value) &&
    value.protocol === DESIGN_MODE_PROTOCOL &&
    value.version === DESIGN_MODE_PROTOCOL_VERSION &&
    value.type === "HF_DESIGN_MODE_SET" &&
    typeof value.sessionId === "string" &&
    typeof value.enabled === "boolean"
  );
}

function isPreviewCropMessage(value: unknown): value is DesignModePreviewCropMessage {
  return (
    isRecord(value) &&
    value.protocol === DESIGN_MODE_PROTOCOL &&
    value.version === DESIGN_MODE_PROTOCOL_VERSION &&
    value.type === "HF_DESIGN_PREVIEW_CROP_SET" &&
    typeof value.sessionId === "string" &&
    isRecord(value.selection) &&
    isRecord(value.selection.viewport)
  );
}

function truncate(value: string, max = 180): string | undefined {
  const normalized = value.replace(/\s+/g, " ").trim();
  if (!normalized) {
    return undefined;
  }
  return normalized.slice(0, max);
}

function sanitizeUrlish(value: string): string | undefined {
  const trimmed = value.trim();
  if (!trimmed || /^(javascript|data|blob):/i.test(trimmed)) {
    return undefined;
  }

  try {
    const url = new URL(trimmed, window.location.origin);
    url.search = "";
    url.hash = "";
    return truncate(url.href, 180);
  } catch {
    return truncate(trimmed.split(/[?#]/)[0] ?? "", 180);
  }
}

function safeAttributes(element: Element): Record<string, string> | undefined {
  const attrs: Record<string, string> = {};
  for (const attribute of Array.from(element.attributes)) {
    if (!SAFE_ATTRIBUTE_NAMES.has(attribute.name)) {
      continue;
    }

    const value =
      attribute.name === "href" || attribute.name === "src"
        ? sanitizeUrlish(attribute.value)
        : truncate(attribute.value, 160);
```

```
    if (value) {
      attrs[attribute.name] = value;
    }
  }

  return Object.keys(attrs).length > 0 ? attrs : undefined;
}

function safeClassList(element: Element): string[] | undefined {
  const classes = Array.from(element.classList)
    .map((item) => item.trim())
    .filter(Boolean)
    .filter((item) => item.length <= 120)
    .slice(0, 12);

  return classes.length > 0 ? classes : undefined;
}

function safeTextSnippet(element: Element): string | undefined {
  const tagName = element.tagName.toLowerCase();
  if (["input", "textarea", "select", "option"].includes(tagName)) {
    return undefined;
  }
  return truncate(element.textContent ?? "", 220);
}

function safeComputedStyle(element: Element): Record<string, string> | undefined {
  const computed = window.getComputedStyle(element);
  const style: Record<string, string> = {};
  for (const key of SAFE_STYLE_NAMES) {
    const value = computed[key];
    if (value) {
      style[key] = value;
    }
  }
  return Object.keys(style).length > 0 ? style : undefined;
}

function elementMeta(element: Element): DesignSelectionElement {
  const sourceMeta = getHiggsfieldDesignMeta(element);
  const attrs = safeAttributes(element);
  const classes = safeClassList(element);
  const computedStyle = safeComputedStyle(element);

  return {
    tagName: element.tagName.toLowerCase(),
    ...(sourceMeta?.nodeId ? { nodeId: sourceMeta.nodeId } : {}),
    ...(sourceMeta?.componentName ? { componentName: sourceMeta.componentName } : {}),
    ...(sourceMeta?.sourceFile ? { sourceFile: sourceMeta.sourceFile } : {}),
    ...(sourceMeta?.sourceLine ? { sourceLine: sourceMeta.sourceLine } : {}),
    ...(sourceMeta?.sourceColumn ? { sourceColumn: sourceMeta.sourceColumn } : {}),
    ...(sourceMeta?.routeFile ? { routeFile: sourceMeta.routeFile } : {}),
    ...(element.getAttribute("role") ? { role: element.getAttribute("role") ?? undefined } : {}),
    ...(element.getAttribute("aria-label")
      ? { ariaLabel: truncate(element.getAttribute("aria-label") ?? "", 160) }
      : {}),
    ...(safeTextSnippet(element) ? { textSnippet: safeTextSnippet(element) } : {}),
    ...(classes ? { classList: classes } : {}),
    ...(attrs ? { attributes: attrs } : {}),
    ...(computedStyle ? { computedStyle } : {}),
  };
}

function findSelectableElement(target: EventTarget | null): Element | null {
  if (!(target instanceof Element)) {
    return null;
  }

  const instrumented = findHiggsfieldDesignElement(target);
  if (
```

```
    instrumented &&
    instrumented.element !== document.documentElement &&
    instrumented.element !== document.body
  ) {
    return instrumented.element;
  }

  const preferred = target.closest<HTMLElement>("button, a, input, textarea, select, [role]");
  if (preferred && preferred !== document.documentElement && preferred !== document.body) {
    return preferred;
  }

  let current: Element | null = target;
  while (current && current !== document.documentElement && current !== document.body) {
    const rect = current.getBoundingClientRect();
    if (rect.width > 1 && rect.height > 1) {
      return current;
    }
    current = current.parentElement;
  }

  return null;
}

function selectionForElement(
  element: Element,
  pointer?: DesignSelectionPoint,
): DesignSelectionContext {
  const rect = element.getBoundingClientRect();
  const sourceMeta = getHiggsfieldDesignMeta(element);
  const ancestors: DesignSelectionElement[] = [];
  let current = element.parentElement;
  while (current && current !== document.body && ancestors.length < 5) {
    ancestors.push(elementMeta(current));
    current = current.parentElement;
  }

  return {
    protocol: DESIGN_MODE_PROTOCOL,
    version: DESIGN_MODE_PROTOCOL_VERSION,
    sessionId: state.sessionId ?? "",
    ...(state.appId ? { appId: state.appId } : {}),
    previewOrigin: window.location.origin,
    route: sourceMeta?.routeFile ?? window.location.pathname,
    path: window.location.pathname,
    timestamp: Date.now(),
    viewport: {
      width: window.innerWidth,
      height: window.innerHeight,
      scrollX: window.scrollX,
      scrollY: window.scrollY,
    },
    ...(pointer ? { pointer } : {}),
    rect: {
      x: rect.x,
      y: rect.y,
      width: rect.width,
      height: rect.height,
    },
    element: elementMeta(element),
    ancestors,
    source: sourceMeta ? "instrumented" : "heuristic",
  };
}

function post(
  type:
    | "HF_DESIGN_READY"
    | "HF_DESIGN_HOVER"
    | "HF_DESIGN_SELECT"
```

```
    | "HF_DESIGN_SCROLL"
    | "HF_DESIGN_EXIT"
    | "HF_DESIGN_ERROR",
  payload: Record<string, unknown> = {},
) {
  if (!state.parentOrigin || !state.sessionId) {
    return;
  }

  window.parent.postMessage(
    {
      protocol: DESIGN_MODE_PROTOCOL,
      version: DESIGN_MODE_PROTOCOL_VERSION,
      type,
      sessionId: state.sessionId,
      ...(state.appId ? { appId: state.appId } : {}),
      ...payload,
    },
    state.parentOrigin,
  );
}

function currentViewport() {
  return {
    width: window.innerWidth,
    height: window.innerHeight,
    scrollX: window.scrollX,
    scrollY: window.scrollY,
  };
}

function sendHover(element: Element, pointer?: DesignSelectionPoint) {
  post("HF_DESIGN_HOVER", { selection: selectionForElement(element, pointer) });
}

function pointFromEvent(event: PointerEvent | MouseEvent): DesignSelectionPoint {
  return { x: event.clientX, y: event.clientY };
}

function modifiersFromEvent(event: MouseEvent) {
  return {
    metaKey: event.metaKey,
    ctrlKey: event.ctrlKey,
    shiftKey: event.shiftKey,
    altKey: event.altKey,
  };
}

function scheduleHover(element: Element, pointer: DesignSelectionPoint) {
  state.pendingHover = { element, pointer };

  if (state.raf !== null) {
    return;
  }

  state.raf = window.requestAnimationFrame(() => {
    state.raf = null;
    const pending = state.pendingHover;
    state.pendingHover = null;
    if (!state.enabled || !pending) {
      return;
    }
    sendHover(pending.element, pending.pointer);
  });
}

function onPointerMove(event: PointerEvent) {
  if (!state.enabled) {
    return;
  }
```

```
  const element = findSelectableElement(event.target);
  if (!element) {
    return;
  }
  const pointer = pointFromEvent(event);
  state.lastPointer = pointer;
  state.lastHoverElement = element;
  scheduleHover(element, pointer);
}

function onClick(event: MouseEvent) {
  if (!state.enabled) {
    return;
  }
  const element = findSelectableElement(event.target);
  if (!element) {
    return;
  }
  event.preventDefault();
  event.stopPropagation();
  post("HF_DESIGN_SELECT", {
    selection: selectionForElement(element, pointFromEvent(event)),
    modifiers: modifiersFromEvent(event),
  });
}

function setInspectorCursor(enabled: boolean) {
  document.getElementById(CURSOR_STYLE_ID)?.remove();

  if (!enabled) {
    return;
  }

  const style = document.createElement("style");
  style.id = CURSOR_STYLE_ID;
  style.textContent = `
    body[data-selector-active="true"],
    body[data-selector-active="true"] * {
      cursor: crosshair !important;
    }
  `;
  document.head.appendChild(style);
}

function setSelectorState(enabled: boolean) {
  const target = document.body ?? document.documentElement;
  target.toggleAttribute("data-selector-active", enabled);
  if (enabled) {
    target.setAttribute("data-selector-interaction-mode", "selection");
  } else {
    target.removeAttribute("data-selector-interaction-mode");
  }
  setInspectorCursor(enabled);
}

function flushPendingWork() {
  if (state.raf !== null) {
    window.cancelAnimationFrame(state.raf);
    state.raf = null;
  }
  if (state.scrollStopTimer !== null) {
    window.clearTimeout(state.scrollStopTimer);
    state.scrollStopTimer = null;
  }
  state.pendingHover = null;
}

function sendHoverAtLastPointer() {
  const pointer = state.lastPointer;
  if (!state.enabled || !pointer) {
```

```
    return;
  }

  const hit = document.elementFromPoint(pointer.x, pointer.y);
  const element = findSelectableElement(hit);
  if (!element) {
    state.lastHoverElement = null;
    return;
  }

  state.lastHoverElement = element;
  sendHover(element, pointer);
}

function onScroll() {
  if (!state.enabled) {
    return;
  }

  post("HF_DESIGN_SCROLL", { viewport: currentViewport() });
  state.lastHoverElement = null;
  state.pendingHover = null;

  if (state.raf !== null) {
    window.cancelAnimationFrame(state.raf);
    state.raf = null;
  }

  if (state.scrollStopTimer !== null) {
    window.clearTimeout(state.scrollStopTimer);
  }

  state.scrollStopTimer = window.setTimeout(() => {
    state.scrollStopTimer = null;
    sendHoverAtLastPointer();
  }, 180);
}

function exitDesignMode() {
  if (!state.enabled) {
    return;
  }

  state.enabled = false;
  state.lastHoverElement = null;
  state.lastPointer = null;
  flushPendingWork();
  setSelectorState(false);
  post("HF_DESIGN_EXIT");
}

function onKeyDown(event: KeyboardEvent) {
  if (!state.enabled || event.key !== "Escape") {
    return;
  }

  event.preventDefault();
  event.stopPropagation();
  exitDesignMode();
}

function setEnabled(message: DesignModeSetMessage, parentOrigin: string) {
  state.enabled = message.enabled;
  state.sessionId = message.sessionId;
  state.appId = message.appId ?? null;
  state.parentOrigin = parentOrigin;
  state.lastHoverElement = null;
  state.lastPointer = null;
  flushPendingWork();
```

```
  setSelectorState(message.enabled);

  if (message.enabled) {
    post("HF_DESIGN_READY");
  }
}

function scrollToPreviewCrop(message: DesignModePreviewCropMessage) {
  const { viewport } = message.selection;
  window.scrollTo({
    left: viewport.scrollX,
    top: viewport.scrollY,
    behavior: "instant",
  });
}

export function installHiggsfieldDesignInspector() {
  if (typeof window === "undefined" || window.__higgsfieldDesignInspectorInstalled) {
    return;
  }

  window.__higgsfieldDesignInspectorInstalled = true;

  window.addEventListener("message", (event) => {
    if (event.source !== window.parent) {
      return;
    }
    if (isPreviewCropMessage(event.data)) {
      scrollToPreviewCrop(event.data);
      return;
    }
    if (isSetMessage(event.data)) {
      setEnabled(event.data, event.origin);
    }
  });

  window.addEventListener("pointermove", onPointerMove, true);
  window.addEventListener("click", onClick, true);
  window.addEventListener("scroll", onScroll, true);
  window.addEventListener("keydown", onKeyDown, true);
}
```

app/src/module/design-inspector/vite.ts

```
import type { PluginOption } from "vite";

const DESIGN_REGISTRY_MODULE = "@/module/design-inspector/registry";
const CREATE_DESIGN_REF_LOCAL = "__hfCreateDesignRef";
const COMPOSE_DESIGN_REFS_LOCAL = "__hfComposeDesignRefs";

type BabelPluginApi = {
  types: Record<string, (...args: unknown[]) => unknown>;
};

type BabelPath = {
  node: Record<string, unknown> & {
    name?: unknown;
    loc?: { start?: { line?: number; column?: number } } | null;
    attributes?: unknown[];
    id?: { name?: string };
  };
  parent?: Record<string, unknown> & { id?: { name?: string } };
  hub?: { file?: { opts?: { filename?: string } } };
  findParent?: (predicate: (path: BabelPath) => boolean) => BabelPath | null;
  isProgram?: () => boolean;
};

export function stableHiggsfieldDesignHash(input: string): string {
  let hash = 0;
  for (let index = 0; index < input.length; index += 1) {
    hash = (Math.imul(31, hash) + input.charCodeAt(index)) | 0;
  }
  return Math.abs(hash).toString(36);
}

function normalizeFileName(filename: string | undefined): string | null {
  if (!filename) {
    return null;
  }
  const normalized = filename.replace(/\\/g, "/");
  const marker = "/app/";
  const index = normalized.lastIndexOf(marker);
  return index >= 0 ? normalized.slice(index + marker.length) : normalized;
}

function routeForFile(file: string): string | null {
  const marker = "src/routes/";
  const index = file.indexOf(marker);
  if (index < 0) {
    return null;
  }
  return file.slice(index + marker.length).replace(/\.(tsx|ts|jsx|js)$/, "");
}

function isIntrinsicTagName(name: string): boolean {
  return /^[a-z]/.test(name) && !name.includes(".");
}

function isComponentTagName(name: string): boolean {
  return /^[A-Z]/.test(name);
}

function jsxNameToString(value: unknown): string | null {
  if (!value || typeof value !== "object") {
    return null;
  }

  const node = value as {
    type?: unknown;
    name?: unknown;
    object?: unknown;
    property?: unknown;
```

```
  };

  if (node.type === "JSXIdentifier" && typeof node.name === "string") {
    return node.name;
  }

  if (node.type === "JSXMemberExpression") {
    const objectName = jsxNameToString(node.object);
    const propertyName = jsxNameToString(node.property);
    return objectName && propertyName ? `${objectName}.${propertyName}` : null;
  }

  return null;
}

function shouldSkipTagName(name: string): boolean {
  return name === "Fragment" || name === "React.Fragment";
}

function getAttribute(path: BabelPath, name: string) {
  const attributes = path.node.attributes ?? [];
  for (let index = 0; index < attributes.length; index += 1) {
    const attribute = attributes[index];
    if (!attribute || typeof attribute !== "object" || !("name" in attribute)) {
      continue;
    }
    const attrName = (attribute as { name?: { name?: string } }).name;
    if (attrName?.name === name) {
      return { attribute: attribute as Record<string, unknown>, index };
    }
  }
  return null;
}

function refExpression(attribute: Record<string, unknown>) {
  const value = attribute.value as Record<string, unknown> | null | undefined;
  if (!value || value.type === "StringLiteral") {
    return null;
  }
  if (value.type !== "JSXExpressionContainer") {
    return null;
  }
  const expression = value.expression as Record<string, unknown> | null | undefined;
  if (
    !expression ||
    expression.type === "JSXEmptyExpression" ||
    expression.type === "StringLiteral"
  ) {
    return null;
  }
  return expression;
}

function ensureNamedImport(
  api: BabelPluginApi,
  programPath: BabelPath,
  importedName: string,
  localName: string,
) {
  const t = api.types;
  const body = (programPath.node.body ?? []) as Record<string, unknown>[];
  const existing = body.find((node) => {
    const source = node.source as { value?: unknown } | undefined;
    return node.type === "ImportDeclaration" && source?.value === DESIGN_REGISTRY_MODULE;
  });
  const specifier = t.importSpecifier(t.identifier(localName), t.identifier(importedName));

  if (existing) {
    const specifiers = (existing.specifiers ?? []) as Record<string, { name?: string }>[];
    const alreadyImported = specifiers.some((item) => item.local?.name === localName);
```

```
    if (!alreadyImported) {
      specifiers.push(specifier as Record<string, { name?: string }>);
    }
    return;
  }

  body.unshift(
    t.importDeclaration([specifier], t.stringLiteral(DESIGN_REGISTRY_MODULE)) as Record<
      string,
      unknown
    >,
  );
}

function programForPath(path: BabelPath): BabelPath | null {
  return path.findParent?.((parent) => Boolean(parent.isProgram?.())) ?? null;
}

function componentNameForPath(path: BabelPath, fallback: string): string {
  const owner = path.findParent?.((parent) => {
    const type = parent.node.type;
    return type === "FunctionDeclaration" || type === "VariableDeclarator";
  });

  const nodeName = owner?.node.id?.name;
  if (nodeName) {
    return nodeName;
  }

  const parentName = owner?.parent?.id?.name;
  return parentName ?? fallback;
}

function metaObject(
  api: BabelPluginApi,
  input: {
    nodeId: string;
    componentName: string;
    sourceFile: string;
    sourceLine?: number;
    sourceColumn?: number;
    routeFile?: string | null;
    tagName: string;
  },
) {
  const t = api.types;
  const properties = [
    t.objectProperty(t.identifier("nodeId"), t.stringLiteral(input.nodeId)),
    t.objectProperty(t.identifier("componentName"), t.stringLiteral(input.componentName)),
    t.objectProperty(t.identifier("sourceFile"), t.stringLiteral(input.sourceFile)),
    t.objectProperty(t.identifier("tagName"), t.stringLiteral(input.tagName)),
  ];

  if (typeof input.sourceLine === "number") {
    properties.push(
      t.objectProperty(t.identifier("sourceLine"), t.numericLiteral(input.sourceLine)),
    );
  }
  if (typeof input.sourceColumn === "number") {
    properties.push(
      t.objectProperty(t.identifier("sourceColumn"), t.numericLiteral(input.sourceColumn)),
    );
  }
  if (input.routeFile) {
    properties.push(t.objectProperty(t.identifier("routeFile"), t.stringLiteral(input.routeFile)));
  }

  return t.objectExpression(properties);
}
```

```
export function higgsfieldDesignSourceBabelPlugin(api: BabelPluginApi) {
  return {
    name: "higgsfield-design-source-registry",
    visitor: {
      JSXOpeningElement(path: BabelPath) {
        const file = normalizeFileName(path.hub?.file?.opts?.filename);
        if (!file || file.includes("packages/") || file.includes("routeTree.gen")) {
          return;
        }
        const tagName = jsxNameToString(path.node.name);
        if (!tagName || shouldSkipTagName(tagName)) {
          return;
        }

        const isIntrinsic = isIntrinsicTagName(tagName);
        const isComponent = isComponentTagName(tagName);
        if (!isIntrinsic && !isComponent) {
          return;
        }

        const programPath = programForPath(path);
        if (!programPath) {
          return;
        }

        const line = path.node.loc?.start?.line;
        const column = path.node.loc?.start?.column;
        const sourceColumn = typeof column === "number" ? column + 1 : undefined;
        const nodeSeed = `${file}:${line ?? 0}:${column ?? 0}:${tagName}`;
        const createRefCall = api.types.callExpression(
          api.types.identifier(CREATE_DESIGN_REF_LOCAL),
          [
            metaObject(api, {
              nodeId: stableHiggsfieldDesignHash(nodeSeed),
              componentName: isIntrinsic ? componentNameForPath(path, tagName) : tagName,
              sourceFile: file,
              sourceLine: line,
              sourceColumn,
              routeFile: routeForFile(file),
              tagName,
            }),
          ],
        );

        ensureNamedImport(api, programPath, "createHiggsfieldDesignRef", CREATE_DESIGN_REF_LOCAL);

        const ref = getAttribute(path, "ref");
        if (!ref) {
          path.node.attributes?.push(
            api.types.jsxAttribute(
              api.types.jsxIdentifier("ref"),
              api.types.jsxExpressionContainer(createRefCall),
            ),
          );
          return;
        }

        const existingRef = refExpression(ref.attribute);
        if (!existingRef) {
          return;
        }

        ensureNamedImport(
          api,
          programPath,
          "composeHiggsfieldDesignRefs",
          COMPOSE_DESIGN_REFS_LOCAL,
        );
        ref.attribute.value = api.types.jsxExpressionContainer(
          api.types.callExpression(api.types.identifier(COMPOSE_DESIGN_REFS_LOCAL), [
```

```
            existingRef,
            createRefCall,
          ]),
        );
      },
    },
  };
}

export function higgsfieldDesignInspectorDefine(enabled: boolean): Record<string, string> {
  return {
    __HF_DESIGN_INSPECTOR__: JSON.stringify(enabled),
  };
}

export function higgsfieldDesignInspectorVitePlugin(enabled: boolean): PluginOption {
  return {
    name: "higgsfield-design-inspector-define",
    config() {
      return {
        define: higgsfieldDesignInspectorDefine(enabled),
      };
    },
  };
}
```

app/src/router.tsx

```
import { QueryClient } from "@tanstack/react-query";
import { createRouter } from "@tanstack/react-router";
import { routeTree } from "./routeTree.gen";

export const getRouter = () => {
  const queryClient = new QueryClient();

  const router = createRouter({
    routeTree,
    context: { queryClient },
    scrollRestoration: true,
    defaultPreloadStaleTime: 0,
  });

  return router;
};
```

app/src/routes/__root.tsx

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import {
  Outlet,
  Link,
  createRootRouteWithContext,
  useRouter,
  HeadContent,
  Scripts,
} from "@tanstack/react-router";
import { useEffect, type ReactNode } from "react";
import { button } from "@higgsfield/quanta/button";
import { NotFound } from "@higgsfield/quanta/not-found";

import appCss from "../styles.css?url";
import { reportHiggsfieldError } from "../lib/higgsfield-error-reporting";
// Page metadata (browser <title>/favicon + social og: tags) committed into the
// repo by the marketplace meta API and read at BUILD time — no runtime fetch.
// Editing it via the app settings UI rewrites this file and redeploys the app.
import appMetaJson from "../app-meta.json";

declare const __HF_DESIGN_INSPECTOR__: boolean;

// Built-in defaults for any field that isn't set in app-meta.json.
const DEFAULT_TITLE = "Higgsfield App";
const DEFAULT_DESCRIPTION = "Higgsfield Generated Project";

type AppMeta = {
  og_title?: string | null;
  og_description?: string | null;
  og_image_url?: string | null;
  favicon_url?: string | null;
  og_video_url?: string | null;
  // Read by the Higgsfield platform (marketplace feed card), never by the
  // app itself — keep it in this file, don't render it.
  marketplace_cover_url?: string | null;
};

const appMeta = appMetaJson as AppMeta;

// Build the document head (title / description / og: / twitter: / favicon) from
// app-meta.json, falling back to the defaults above for any unset field.
// og_title/og_description double as the browser <title> and meta description;
// og_image_url (when set) also drives the twitter card + image. Built from
// inline tag literals (conditional spreads for the optional image/favicon) so
// it matches the head() shape TanStack expects.
// favicon/og images live in THIS app's own /assets, so the host is never
// inherent. app-meta.json may carry an absolute higgsfield-app URL with a STALE
// host — baked from the app this one was copied/remixed/renamed from — which would
// serve the wrong app's favicon/og. Strip any higgsfield-app host (prod
// higgsfield.app + dev higgsfield-dev.app) down to a root-relative path so it
// always resolves against whoever serves THIS page (preview / prod / custom
// domain). Genuinely external URLs (a CDN image the owner set) are left absolute.
const APP_HOST_ZONES = ["higgsfield.app", "higgsfield-dev.app"];

function toOwnAssetUrl(value: string | null | undefined): string | null {
  if (!value) return null;
  if (value.startsWith("/")) return value; // already root-relative
  try {
    const u = new URL(value);
    const isAppHost = APP_HOST_ZONES.some(
      (zone) => u.hostname === zone || u.hostname.endsWith(`.${zone}`),
    );
    if (isAppHost) return u.pathname + u.search;
    return value; // external host (CDN, etc.) — keep absolute
  } catch {
    return value; // not a parseable URL — leave as-is
  }
}
```

```
function buildHead(meta: AppMeta) {
  const title = meta.og_title ?? DEFAULT_TITLE;
  const description = meta.og_description ?? DEFAULT_DESCRIPTION;
  const ogImage = toOwnAssetUrl(meta.og_image_url);
  const favicon = toOwnAssetUrl(meta.favicon_url);
  const ogVideo = toOwnAssetUrl(meta.og_video_url);

  return {
    meta: [
      { charSet: "utf-8" },
      { name: "viewport", content: "width=device-width, initial-scale=1" },
      { title },
      { name: "description", content: description },
      { name: "author", content: "Higgsfield" },
      { property: "og:title", content: title },
      { property: "og:description", content: description },
      { property: "og:type", content: "website" },
      { name: "twitter:card", content: ogImage ? "summary_large_image" : "summary" },
      { name: "twitter:site", content: "@Higgsfield" },
      ...(ogImage
        ? [
            { property: "og:image", content: ogImage },
            { name: "twitter:image", content: ogImage },
          ]
        : []),
      // Cover video (og:video) — the animated counterpart of og:image; the
      // Higgsfield feed cards also play it on hover.
      ...(ogVideo ? [{ property: "og:video", content: ogVideo }] : []),
    ],
    links: [
      { rel: "stylesheet", href: appCss },
      ...(favicon ? [{ rel: "icon", href: favicon }] : []),
    ],
  };
}

function NotFoundComponent() {
  return (
    <div className="flex min-h-dvh items-center justify-center bg-q-background-primary px-4">
      <NotFound
        className="mx-auto max-w-md"
        icon={<span className="text-q-title-md-semi-bold text-q-text-primary">404</span>}
        title="Page not found"
        subtitle="The page you're looking for doesn't exist or has been moved."
      >
        <Link to="/" className={button({ variant: "primary", size: "md" }, "mt-3")}>
          Go home
        </Link>
      </NotFound>
    </div>
  );
}

function ErrorComponent({ error, reset }: { error: Error; reset: () => void }) {
  console.error(error);
  const router = useRouter();
  useEffect(() => {
    reportHiggsfieldError(error, { boundary: "tanstack_root_error_component" });
  }, [error]);

  return (
    <div className="flex min-h-dvh items-center justify-center bg-q-background-primary px-4">
      <div className="max-w-md text-center">
        <h1 className="text-q-title-lg-semi-bold text-q-text-primary">This page didn't load</h1>
        <p className="mt-2 text-q-body-sm-regular text-q-text-secondary">
          Something went wrong on our end. You can try refreshing or head back home.
        </p>
        <div className="mt-4 flex flex-wrap justify-center gap-2">
          <button
            onClick={() => {
```

```
              router.invalidate();
              reset();
            }}
            className={button({ variant: "primary", size: "md" })}
          >
            Try again
          </button>
          <a href="/" className={button({ variant: "outline", size: "md" })}>
            Go home
          </a>
        </div>
      </div>
    </div>
  );
}

export const Route = createRootRouteWithContext<{ queryClient: QueryClient }>()({
  // Read the committed page metadata at build time (no runtime fetch).
  head: () => buildHead(appMeta),
  shellComponent: RootShell,
  component: RootComponent,
  notFoundComponent: NotFoundComponent,
  errorComponent: ErrorComponent,
});

function RootShell({ children }: { children: ReactNode }) {
  return (
    <html lang="en" data-theme="default-dark" style={{ colorScheme: "dark" }}>
      {/* Marketplace apps are permanently dark: data-theme is pinned on <html>
          above. Do not add quanta's bootstrapScript/ThemeController, a theme
          toggle, or a light mode. */}
      <head>
        <HeadContent />
      </head>
      <body className="bg-q-background-primary text-q-text-primary">
        {children}
        <Scripts />
      </body>
    </html>
  );
}

function RootComponent() {
  const { queryClient } = Route.useRouteContext();

  useEffect(() => {
    if (!__HF_DESIGN_INSPECTOR__) {
      return;
    }

    void import("../module/design-inspector/runtime")
      .then(({ installHiggsfieldDesignInspector }) => {
        installHiggsfieldDesignInspector();
      })
      .catch((error) => {
        reportHiggsfieldError(
          error instanceof Error ? error : new Error("Failed to load design inspector"),
          {
            boundary: "higgsfield_design_inspector_import",
          },
        );
      });
  }, []);

  return (
    <QueryClientProvider client={queryClient}>
      {/* Required: nested routes render here. Removing <Outlet /> breaks all child routes. */}
      <Outlet />
    </QueryClientProvider>
  );
```

```
}
```

app/src/routes/app.tsx

```
import { createFileRoute } from "@tanstack/react-router";
import { CustomTemplate } from "@/layouts/custom";

interface AppSearch {
  preview: boolean;
}

export const Route = createFileRoute("/app")({
  validateSearch: (search: Record<string, unknown>): AppSearch => ({
    preview: search.preview === "1" || search.preview === true,
  }),
  component: AppRoute,
});

function AppRoute() {
  const { preview } = Route.useSearch();
  return <CustomTemplate previewMode={preview} />;
}
```

app/src/routes/index.tsx

```
import { createFileRoute } from "@tanstack/react-router";
import { lazy, Suspense, useEffect, useMemo, useState } from "react";
import { ArrowDownRight, ArrowRight, ArrowUpRight, BarChart3, Check, CloudRain, CloudSun, Droplets, Globe2, Leaf, Menu, RotateCcw, Sprout, Sun, TrendingUp, Waves, X, Zap } from "lucide-react";
export const Route = createFileRoute("/")({ component: FarmVerse });
type Weather = "Typical" | "Dry spell" | "Heavy rain";
type Nutrients = "Balanced" | "Reduced input" | "Conventional";
const ThreeFarmScene = lazy(() => import("@/components/three-farm-scene"));
function FarmArt({ water, weather }: { water: number; weather: Weather }) {
 return <div className="farm-art"><div className="art-sun"/><div className="art-cloud cloud-a"/><div className="art-cloud cloud-b"/><div className="art-hill hill-a"/><div className="art-hill hill-b"/>
}
function Metric({label,value}:{label:string;value:string}) { return <div className="metric-row"><span>{label}</span><b>{value}</b></div>; }
function FarmVerse() {
 const [menu,setMenu]=useState(false),[water,setWater]=useState(65),[nutrients,setNutrients]=useState<Nutrients>("Balanced"),[weather,setWeather]=useState<Weather>("Typical"),[crop,setCrop]=useState('
 useEffect(()=>{const reduced=window.matchMedia("(prefers-reduced-motion: reduce)").matches;const targets=document.querySelectorAll<HTMLElement>(".manifesto-grid,.principle-strip,.farm-workspace,.weat
 useEffect(()=>{let cancelled=false;fetch("https://api.open-meteo.com/v1/forecast?latitude=18.5204&longitude=73.8567&current=temperature_2m,relative_humidity_2m,precipitation,weather_code,wind_speed_1
 const result=useMemo(()=>{const wf=water<35?-.22:water<55?-.09:water>85?.015:0,weatherF=weather==="Dry spell"?-.13:weather==="Heavy rain"?-.09:0,base=crop==="Chickpea"?1600:crop==="Rice"?4500:4200,ir
 const baseline=useMemo(()=>{const y=crop==="Chickpea"?1600:crop==="Rice"?4500:4200,p=crop==="Chickpea"?64:crop==="Rice"?23:25;const cost=crop==="Chickpea"?26000:crop==="Rice"?42000:32000,water=crop==
 const ask=()=>{const q=question.toLowerCase();let a=`For your ${crop.toLowerCase()} scenario, the model uses ${water}% irrigation allocation, ${weather.toLowerCase()} conditions and a ${nutrients.toL
 const reset=()=>{setWater(65);setWeather("Typical");setNutrients("Balanced");setCrop("Wheat");setSaved(false);};
 const tabs=["Overview","Resources","Economics"];
 return <main className="fv-site"><div className="announcement"><i/> HACK ERA 1.0 PROJECT <span>/</span> A NEW WAY TO THINK ABOUT FARMING <ArrowUpRight size={13}/></div><header className="fv-nav"><a c
 <section className="hero" id="top"><div className="hero-copy"><div className="eyebrow"><i/> A VIRTUAL FARM. REAL-WORLD THINKING.</div><h1>Before you<br/>grow it <em>out there,</em><br/>try it <span>i
 <section className="manifesto section-pad" id="idea"><div className="section-meta"><span>01 / THE IDEA</span><span>THINK BEFORE YOU PLANT</span></div><div className="manifesto-grid"><h2>Farming is fu
 <section className="farm-section section-pad" id="farm"><div className="section-meta light-meta"><span>02 / YOUR VIRTUAL LAND</span><span>INTERACTIVE FARM MODEL</span></div><div className="section-he
 <section className="weather-section section-pad" id="weather"><div className="section-meta"><span>LIVE WEATHER / PUNE</span><span>FORECAST CONTEXT, NOT CERTAINTY</span></div><div className="section-h
 <section className="health-section section-pad" id="crop-health"><div className="section-meta"><span>CROP HEALTH / SUSTAINABLE CARE</span><span>OBSERVE · VERIFY · RESPOND</span></div><div className="
 <section className="future-section section-pad" id="future-lab"><div className="section-meta"><span>03 / THE FUTURE LAB</span><span>YOUR CHOICES, SIDE BY SIDE</span></div><div className="section-heac
 <section className="advisor-section" id="advisor"><div className="advisor-orbit orbit-left"/><div className="advisor-content"><div className="advisor-label">✻   THE FARMVERSE ADVISOR</div><h2>Nu
 <section className="method-section section-pad" id="how-it-works"><div className="section-meta"><span>04 / THE METHOD</span><span>FROM QUESTION TO CHOICE</span></div><div className="section-heading">
 <section className="team-section section-pad" id="team"><div className="section-meta light-meta"><span>05 / THE BUILD</span><span>HACK ERA 1.0</span></div><div className="section-heading"><div><div c
 <section className="stack-section section-pad"><div className="section-meta"><span>THE TOOLKIT</span><span>FAST TO PROTOTYPE · READY TO EXTEND</span></div><div className="stack-layout"><div><h2>A pra
 <section className="docs-section section-pad" id="documentation"><div className="section-meta"><span>PROJECT DOCUMENTATION</span><span>THE IDEA, ARCHITECTURE & PITCH</span></div><div className="secti
 <section className="demo-section section-pad" id="demo"><div className="demo-card"><div><div className="eyebrow"><i/> PROJECT DEMONSTRATION</div><h2>Make the future<br/><em>visible.</em></h2><p>The s
 <section className="closing-section"><div className="closing-icon"><Sprout size={28}/></div><div className="eyebrow"><i/> THE FARMVERSE IDEA</div><h2>Don't just predict<br/>a harvest. <em>Explore it.
 <footer className="footer"><a className="brand" href="#top"><span className="brand-icon"><Sprout size={21}/></span><span>farm<span>verse</span><small>GROW WITH FORESIGHT</small></span></a><p>AI-power
}
```

app/src/routes/README.md

```
# Routes

TanStack Start uses **file-based routing**. Every `.tsx` file in this directory
is a route. Do **not** create `src/pages/`, `src/routes/_app/index.tsx`, or
`app/layout.tsx` — those are Next.js / Remix conventions. The only root layout
is `src/routes/__root.tsx`.

## Conventions

| File                     | URL                                                      |
| ------------------------ | -------------------------------------------------------- |
| `index.tsx`              | `/`                                                      |
| `about.tsx`              | `/about`                                                 |
| `users/index.tsx`        | `/users`                                                 |
| `users/$id.tsx`          | `/users/:id` (dynamic — bare `$`, no curly braces)       |
| `posts/{-$category}.tsx` | `/posts/:category?` (optional segment)                   |
| `files/$.tsx`            | `/files/*` (splat — read via `_splat` param, never `*`)  |
| `_layout.tsx`            | layout route (renders children via `<Outlet />`)         |
| `__root.tsx`             | app shell — wraps every page; preserve `<Outlet />`      |

`routeTree.gen.ts` is auto-generated. Don't edit it by hand. `bun run
typecheck` and `bun run build` regenerate it first (`tsr generate`), so newly
added routes typecheck cleanly right away.
```

app/src/routes/robots[.]txt.ts

```
import { createFileRoute } from "@tanstack/react-router";

export const Route = createFileRoute("/robots.txt")({
  server: {
    handlers: {
      GET: async ({ request }) => {
        const origin = new URL(request.url).origin;
        const body = ["User-agent: *", "Allow: /", "", `Sitemap: ${origin}/sitemap.xml`].join("\n");
        return new Response(body, {
          headers: {
            "Content-Type": "text/plain; charset=utf-8",
            "Cache-Control": "public, max-age=86400",
          },
        });
      },
    },
  },
});
```

app/src/routes/sitemap[.]xml.ts

```
import { createFileRoute } from "@tanstack/react-router";

export const Route = createFileRoute("/sitemap.xml")({
  server: {
    handlers: {
      GET: async ({ request }) => {
        const origin = new URL(request.url).origin;
        const today = new Date().toISOString().split("T")[0];
        const xml = [
          '<?xml version="1.0" encoding="UTF-8"?>',
          '<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">',
          "  <url>",
          `    <loc>${origin}/</loc>`,
          `    <lastmod>${today}</lastmod>`,
          "    <changefreq>weekly</changefreq>",
          "    <priority>1.0</priority>",
          "  </url>",
          "</urlset>",
        ].join("\n");
        return new Response(xml, {
          headers: {
            "Content-Type": "application/xml; charset=utf-8",
            "Cache-Control": "public, max-age=3600",
          },
        });
      },
    },
  },
});
```

app/src/routeTree.gen.ts

```
/* eslint-disable */

// @ts-nocheck

// noinspection JSUnusedGlobalSymbols

// This file was automatically generated by TanStack Router.
// You should NOT make any changes in this file as it will be overwritten.
// Additionally, you should also exclude this file from your linter and/or formatter to prevent it from being checked or modified.

import { Route as rootRouteImport } from './routes/__root'
import { Route as SitemapDotxmlRouteImport } from './routes/sitemap[.]xml'
import { Route as RobotsDottxtRouteImport } from './routes/robots[.]txt'
import { Route as AppRouteImport } from './routes/app'
import { Route as IndexRouteImport } from './routes/index'

const SitemapDotxmlRoute = SitemapDotxmlRouteImport.update({
  id: '/sitemap.xml',
  path: '/sitemap.xml',
  getParentRoute: () => rootRouteImport,
} as any)
const RobotsDottxtRoute = RobotsDottxtRouteImport.update({
  id: '/robots.txt',
  path: '/robots.txt',
  getParentRoute: () => rootRouteImport,
} as any)
const AppRoute = AppRouteImport.update({
  id: '/app',
  path: '/app',
  getParentRoute: () => rootRouteImport,
} as any)
const IndexRoute = IndexRouteImport.update({
  id: '/',
  path: '/',
  getParentRoute: () => rootRouteImport,
} as any)

export interface FileRoutesByFullPath {
  '/': typeof IndexRoute
  '/app': typeof AppRoute
  '/robots.txt': typeof RobotsDottxtRoute
  '/sitemap.xml': typeof SitemapDotxmlRoute
}
export interface FileRoutesByTo {
  '/': typeof IndexRoute
  '/app': typeof AppRoute
  '/robots.txt': typeof RobotsDottxtRoute
  '/sitemap.xml': typeof SitemapDotxmlRoute
}
export interface FileRoutesById {
  __root__: typeof rootRouteImport
  '/': typeof IndexRoute
  '/app': typeof AppRoute
  '/robots.txt': typeof RobotsDottxtRoute
  '/sitemap.xml': typeof SitemapDotxmlRoute
}
export interface FileRouteTypes {
  fileRoutesByFullPath: FileRoutesByFullPath
  fullPaths: '/' | '/app' | '/robots.txt' | '/sitemap.xml'
  fileRoutesByTo: FileRoutesByTo
  to: '/' | '/app' | '/robots.txt' | '/sitemap.xml'
  id: '__root__' | '/' | '/app' | '/robots.txt' | '/sitemap.xml'
  fileRoutesById: FileRoutesById
}
export interface RootRouteChildren {
  IndexRoute: typeof IndexRoute
  AppRoute: typeof AppRoute
  RobotsDottxtRoute: typeof RobotsDottxtRoute
```

```
  SitemapDotxmlRoute: typeof SitemapDotxmlRoute
}

declare module '@tanstack/react-router' {
  interface FileRoutesByPath {
    '/sitemap.xml': {
      id: '/sitemap.xml'
      path: '/sitemap.xml'
      fullPath: '/sitemap.xml'
      preLoaderRoute: typeof SitemapDotxmlRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/robots.txt': {
      id: '/robots.txt'
      path: '/robots.txt'
      fullPath: '/robots.txt'
      preLoaderRoute: typeof RobotsDottxtRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/app': {
      id: '/app'
      path: '/app'
      fullPath: '/app'
      preLoaderRoute: typeof AppRouteImport
      parentRoute: typeof rootRouteImport
    }
    '/': {
      id: '/'
      path: '/'
      fullPath: '/'
      preLoaderRoute: typeof IndexRouteImport
      parentRoute: typeof rootRouteImport
    }
  }
}

const rootRouteChildren: RootRouteChildren = {
  IndexRoute: IndexRoute,
  AppRoute: AppRoute,
  RobotsDottxtRoute: RobotsDottxtRoute,
  SitemapDotxmlRoute: SitemapDotxmlRoute,
}
export const routeTree = rootRouteImport
  ._addFileChildren(rootRouteChildren)
  ._addFileTypes<FileRouteTypes>()

import type { getRouter } from './router.tsx'
import type { startInstance } from './start.ts'
declare module '@tanstack/react-start' {
  interface Register {
    ssr: true
    router: Awaited<ReturnType<typeof getRouter>>
    config: Awaited<ReturnType<typeof startInstance.getOptions>>
  }
}
```

app/src/scroll-scrub-scenes.ts

```
/**
 * Scene data for the scroll-scrub journey — THE file you fill in per build.
 *
 * Single-shot (the default): ONE entry in `scenes`, whose `clip` is the single
 * continuous film. Chapter copy still comes from `chapters` below, rendered as
 * semantic sections over that one clip.
 *
 * Multi-leg (opt-in): one entry per seam-locked leg, in journey order. Every
 * `poster` MUST be the exact first frame of the encoded clip beside it — never
 * a design board or an imagined destination still.
 *
 * Keep this array a module constant. Changing its identity on every render
 * intentionally rebuilds the media controller.
 */
import type {
  ScrollScrubScene,
  ScrollScrubTheme,
} from "@/components/scroll-scrub/scroll-scrub";

/** Brand tokens for the journey layer. Set these from the design brief. */
export const scrollScrubTheme: ScrollScrubTheme = {
  accent: "<accent hex>",
  background: "<background hex>",
  ink: "<ink hex>",
  muted: "<muted ink hex>",
};

export const scrollScrubScenes: ScrollScrubScene[] = [
  {
    body: "<one sentence that earns the next scroll>",
    clip: "/assets/world/scene-01.mp4",
    id: "scene-01",
    kicker: "<kicker>",
    label: "<nav label>",
    mobileClip: "/assets/world/scene-01-mobile.mp4",
    mobilePoster: "/assets/world/scene-01-mobile-poster.png",
    poster: "/assets/world/scene-01-poster.png",
    tags: ["<proof tag>"],
    title: "<scene headline>",
  },
];
```

app/src/server.ts

```
import "./lib/error-capture";

import { consumeLastCapturedError } from "./lib/error-capture";
import { renderErrorPage } from "./lib/error-page";
import { applySecurityHeaders } from "./lib/security-headers.server";

type ServerEntry = {
  fetch: (request: Request, env: unknown, ctx: unknown) => Promise<Response> | Response;
};

let serverEntryPromise: Promise<ServerEntry> | undefined;

async function getServerEntry(): Promise<ServerEntry> {
  if (!serverEntryPromise) {
    serverEntryPromise = import("@tanstack/react-start/server-entry").then(
      (m) => (m.default ?? m) as ServerEntry,
    );
  }
  return serverEntryPromise;
}

// h3 swallows in-handler throws into a normal 500 Response with body
// {"unhandled":true,"message":"HTTPError"} — try/catch alone never fires for those.
async function normalizeCatastrophicSsrResponse(response: Response): Promise<Response> {
  if (response.status < 500) return response;
  const contentType = response.headers.get("content-type") ?? "";
  if (!contentType.includes("application/json")) return response;

  const body = await response.clone().text();
  if (!body.includes('"unhandled":true') || !body.includes('"message":"HTTPError"')) {
    return response;
  }

  console.error(consumeLastCapturedError() ?? new Error(`h3 swallowed SSR error: ${body}`));
  return new Response(renderErrorPage(), {
    status: 500,
    headers: { "content-type": "text/html; charset=utf-8" },
  });
}

export default {
  async fetch(request: Request, env: unknown, ctx: unknown) {
    try {
      const handler = await getServerEntry();
      const response = await handler.fetch(request, env, ctx);
      return applySecurityHeaders(await normalizeCatastrophicSsrResponse(response));
    } catch (error) {
      console.error(error);
      return applySecurityHeaders(
        new Response(renderErrorPage(), {
          status: 500,
          headers: { "content-type": "text/html; charset=utf-8" },
        }),
      );
    }
  },
};
```

app/src/start.ts

```
import { createCsrfMiddleware, createMiddleware, createStart } from "@tanstack/react-start";

import { renderErrorPage } from "./lib/error-page";

const errorMiddleware = createMiddleware().server(async ({ next }) => {
  try {
    return await next();
  } catch (error) {
    if (error != null && typeof error === "object" && "statusCode" in error) {
      throw error;
    }
    console.error(error);
    return new Response(renderErrorPage(), {
      status: 500,
      headers: { "content-type": "text/html; charset=utf-8" },
    });
  }
});

const csrfMiddleware = createCsrfMiddleware({
  filter: (context) => context.handlerType === "serverFn",
});

export const startInstance = createStart(() => ({
  requestMiddleware: [csrfMiddleware, errorMiddleware],
}));
```

app/src/styles.css

```
/* Cascade layer order: reset lowest, then Tailwind/Quanta base, then
 * components and finally utilities — so Quanta's q-* utilities (e.g.
 * q-button-*) always win over global element resets. */
@layer reset, theme, base, components, utilities;

@import "tailwindcss" source(none);
@source "../src";
@source "../packages/app-landing/src";
@source "../packages/quanta/src";
@import "@higgsfield/quanta/tailwind.css";

/* New in-app Quanta-style components (moved out of the quanta package). Their
 * CSS uses Tailwind v4 @utility/@source, so it must load through this tailwind
 * entry. Ordering mirrors quanta's components.css (composer after button;
 * media-card + generation-card after media). */
@import "./components/composer/composer.css";
@import "./components/media-card/media-card.css";
@import "./components/generation-card/generation-card.css";
@import "./components/setting-trigger/setting-trigger.css";
@import "./components/prompt-box/prompt-box.css";
@import "./components/icon-tile/icon-tile.css";

@import "tw-animate-css";
/* shadcn v4 base (scrollbar/scroll-fade utilities, toast base) for src/components/ui */
@import "shadcn/tailwind.css";

@custom-variant dark (&:where([data-theme="default-dark"], [data-theme="default-dark"] *));

/*
 * Quanta is the visual source of truth. This file imports Quanta's Tailwind
 * entry once; that entry brings in primitives, semantic theme variables,
 * typography, z-index, border-width, and q-* component utilities.
 *
 * Generated app layout should use normal Tailwind spacing (`p-4`, `gap-2`,
 * `mt-6`, `h-10`) and Quanta semantic utilities for color/type
 * (`bg-q-background-primary`, `text-q-body-md-regular`). Do not use old numeric
 * token classes such as `px-400` / `gap-200`, and do not reach for raw
 * q-spacing vars in app code. Quanta component internals own their q-* classes.
 *
 * The `--spacing: 0.25rem` restoration keeps native Tailwind layout utilities
 * stable for generated apps and legacy scaffold components. The inline theme
 * aliases below keep shadcn-style imports rendering while new UI migrates to
 * Quanta components.
 */
@theme {
  --spacing: 0.25rem;
}

@theme inline {
  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
  --radius-2xl: calc(var(--radius) + 8px);
  --radius-3xl: calc(var(--radius) + 12px);
  --radius-4xl: calc(var(--radius) + 16px);

  --color-background: var(--hf-color-background-primary);
  --color-foreground: var(--hf-color-text-primary);
  --color-card: var(--hf-color-background-secondary);
  --color-card-foreground: var(--hf-color-text-primary);
  --color-popover: var(--hf-color-background-secondary);
  --color-popover-foreground: var(--hf-color-text-primary);
  --color-primary: var(--hf-color-button-brand);
  --color-primary-foreground: var(--hf-color-button-label-brand);
  --color-secondary: var(--hf-color-background-secondary);
  --color-secondary-foreground: var(--hf-color-text-primary);
  --color-muted: var(--hf-color-background-tertiary);
```

```
  --color-muted-foreground: var(--hf-color-text-secondary);
  --color-accent: var(--hf-color-overlay-hover);
  --color-accent-foreground: var(--hf-color-text-primary);
  /* shadcn v4 components use text-destructive on tinted fills — needs the
   * bright brand red, not the dark error surface */
  --color-destructive: var(--hf-color-button-destructive);
  --color-destructive-foreground: var(--hf-color-button-label-destructive);
  --color-border: var(--hf-color-border-default);
  --color-input: var(--hf-color-border-default);
  --color-ring: var(--hf-color-border-focus);
  --color-ring-offset-background: var(--hf-color-background-primary);

  --color-chart-1: var(--hf-color-lime-500);
  --color-chart-2: var(--hf-color-pink-500);
  --color-chart-3: var(--hf-color-blue-500);
  --color-chart-4: var(--hf-color-yellow-500);
  --color-chart-5: var(--hf-color-purple-500);

  --color-sidebar: var(--hf-color-background-secondary);
  --color-sidebar-foreground: var(--hf-color-text-primary);
  --color-sidebar-primary: var(--hf-color-button-brand);
  --color-sidebar-primary-foreground: var(--hf-color-button-label-brand);
  --color-sidebar-accent: var(--hf-color-overlay-hover);
  --color-sidebar-accent-foreground: var(--hf-color-text-primary);
  --color-sidebar-border: var(--hf-color-border-default);
  --color-sidebar-ring: var(--hf-color-border-focus);

  --font-grotesk: var(--hf-type-family-grotesk);
}

/* Fills used by src/components/ui (badge, tabs, marketing-button), mapped to
 * Quanta's --hf-gradient-* tokens. */
@utility badge-gradient-blue {
  background-image: var(--hf-gradient-badge-blue);
}
@utility badge-gradient-pink {
  background-image: var(--hf-gradient-badge-pink);
}
@utility badge-gradient-purple {
  background-image: var(--hf-gradient-badge-purple);
}
@utility glass-sheen {
  background-image: var(--hf-gradient-glass-sheen);
}
@utility gloss-lime {
  background-image:
    var(--hf-gradient-special-gloss-lime-angled),
    linear-gradient(90deg, var(--hf-color-lime-500), var(--hf-color-lime-500));
}
@utility gloss-pink {
  background-image:
    var(--hf-gradient-special-gloss-pink),
    linear-gradient(90deg, var(--hf-color-pink-500), var(--hf-color-pink-500));
}

/* Toast skin (Figma node 4085:2671): compact glass pill, kind-colored fills */
[data-sonner-toast][data-styled="true"] {
  width: fit-content;
  align-items: center;
  gap: 8px;
  padding: 10px 12px;
  border: 1px solid var(--hf-color-transparent-light-05);
  border-radius: var(--hf-radius-300, 12px);
  background: var(--hf-color-glass-dark);
  backdrop-filter: blur(40px);
  box-shadow: 0 6px 6px rgb(0 0 0 / 0.25);
  color: var(--hf-color-text-primary);
  font-size: 14px;
  line-height: 20px;
  font-weight: 500;
```

```
}
[data-sonner-toast][data-type="success"] {
  border-color: var(--hf-color-transparent-light-15);
  background: #68b922; /* toast/success — raw value in Figma, no hf token */
  backdrop-filter: none;
}
[data-sonner-toast].toast-new {
  border-color: var(--hf-color-transparent-light-15);
  background: #7a58ff; /* toast/new — raw value in Figma, no hf token */
  backdrop-filter: none;
}
[data-sonner-toast] [data-close-button] {
  position: static;
  order: 9;
  margin-left: 4px;
  border: none;
  background: transparent;
  color: inherit;
  transform: none;
}
[data-sonner-toast] [data-close-button]:hover {
  background: var(--hf-color-overlay-hover);
}

:root {
  --radius: 0.625rem;
}

@layer base {
  html {
    background-color: var(--hf-color-background-primary);
    color: var(--hf-color-text-primary);
    color-scheme: dark;
  }

  body {
    min-height: 100vh;
    background-color: var(--hf-color-background-primary);
    color: var(--hf-color-text-primary);
    font-family: var(--hf-type-family-secondary);
  }

  * {
    border-color: var(--hf-color-border-default);
  }
}

/* FarmVerse identity and cinematic landing experience */
:root{--fv-forest:#173e32;--fv-deep:#102f27;--fv-leaf:#4c795b;--fv-sage:#c9d5bd;--fv-mist:#edf0e7;--fv-paper:#f5f3eb;--fv-ink:#1b3027;--fv-muted:#6c7a70;--fv-coral:#d37d60;--fv-line:rgba(27,48,39,.15)
html{scroll-behavior:smooth;scroll-padding-top:88px}body{margin:0!important;background:var(--fv-paper)!important;color:var(--fv-ink)!important;font-family:Inter,ui-sans-serif,system-ui,sans-serif!impo
.hero{position:relative;min-height:670px;padding:88px 8% 82px;display:grid;grid-template-columns: .9fr 1.1fr;align-items:center;gap:3%;background:radial-gradient(ellipse at 78% 48%,#dfe7d5 0,transpare
.farm-art{position:relative;width:100%;min-height:420px;isolation:isolate;display:flex;align-items:center;justify-content:center;overflow:hidden}.farm-art svg{position:relative;z-index:2;width:100%;ma
@keyframes rise-in{from{opacity:0;transform:translateY(22px)}to{opacity:1;transform:translateY(0)}}@keyframes farm-float{0%,100%{transform:translateY(0) rotate(-.3deg)}50%{transform:translateY(-10px)
.section-pad{padding:88px 8.2%}.section-meta{display:flex;align-items:center;justify-content:space-between;gap:20px;padding-bottom:17px;border-bottom:1px solid var(--fv-line);font-size:8px;letter-spac
.farm-section,.team-section{background:var(--fv-forest);color:#f1f1e7}.light-meta{border-color:#ffffff26;color:#a8b9a8}.section-heading{display:flex;align-items:flex-end;justify-content:space-between;
.future-section{background:var(--fv-paper)}.compare-shell{border:1px solid #dfe3d8;background:#fbfaf5;border-radius:5px;padding:27px 29px 22px}.compare-head{display:flex;align-items:center;justify-cor
.advisor-section{position:relative;overflow:hidden;padding:93px 24px 90px;background:#e6ebdf;text-align:center}.advisor-content{position:relative;z-index:2;max-width:760px;margin:auto}.advisor-label{c
.method-section{background:#fbfaf4}.method-grid{display:grid;grid-template-columns:repeat(4,1fr);border-top:1px solid var(--fv-line);border-bottom:1px solid var(--fv-line);margin-top:45px}.method-card
.team-section{position:relative;background:radial-gradient(ellipse at 100% 0,#28533f 0,transparent 40%),var(--fv-forest)}.team-section .section-heading{margin-bottom:43px}.team-section .section-headin
.stack-section{background:var(--fv-paper)}.stack-layout{display:grid;grid-template-columns:.75fr 1.25fr;gap:8%;padding-top:45px}.stack-layout h2{font:normal clamp(40px,4.6vw,58px)/1.03 Georgia,serif;l
@media(min-width:1500px){.hero{padding-left:max(8%,calc((100% - 1300px)/2));padding-right:max(8%,calc((100% - 1300px)/2))}.section-pad{padding-left:max(8.2%,calc((100% - 1280px)/2));padding-right:max(
@media(max-width:600px){.announcement{font-size:7px;letter-spacing:.1em;gap:6px;height:28px}.fv-nav{padding:0 5%;gap:10px}.brand{font-size:22px;gap:8px}.brand-icon{width:34px;height:34px}.nav-cta{padd
.weather-section{background:#e6ebdf}.weather-dashboard{display:grid;grid-template-columns:1.05fr .95fr;gap:15px}.weather-current,.forecast-panel{padding:24px;background:#fbfaf4;border:1px solid #d9e1d
.docs-section{background:#fbfaf4}.docs-grid{display:grid;grid-template-columns:1fr 1fr;gap:12px;margin-top:34px}.doc-card{position:relative;min-height:230px;padding:24px;background:#f1f3eb;border:1px
@media(max-width:820px){.weather-dashboard{grid-template-columns:1fr}.weather-current{padding:20px}.docs-grid{gap:9px}.doc-card{padding:19px}.pitch-outline{padding:19px}.pitch-slides{gap:6px}.pitch-sl
@media(max-width:600px){.weather-dashboard{gap:10px}.weather-current,.forecast-panel{padding:17px}.weather-now{padding:20px 0}.weather-now>div:first-child b{font-size:54px}.weather-big-icon{width:80px
.farm-svg-image{position:relative;z-index:2;width:100%;max-width:690px;height:auto;overflow:visible;filter:drop-shadow(0 15px 12px #163e2c10);animation:farm-float 7s ease-in-out infinite}.farm-stage .
.health-section{background:var(--fv-paper)}.health-layout{display:grid;grid-template-columns:.78fr 1.22fr;gap:18px;margin-top:34px}.health-options{display:grid;align-content:start;gap:8px}.health-opti
@media(max-width:820px){.health-layout{grid-template-columns:1fr 1.15fr;gap:10px}.health-option{grid-template-columns:35px minmax(0,1fr) 12px;padding:10px;gap:8px}.health-symbol{width:35px;height:35px
@media(max-width:600px){.health-layout{grid-template-columns:1fr}.health-options{gap:7px}.health-option{grid-template-columns:42px minmax(0,1fr) 16px;padding:12px}.health-symbol{width:40px;height:40px
```

```
@media(prefers-reduced-motion:reduce){.fv-reveal{opacity:1;transform:none;transition:none}}
.three-farm-viewport{position:relative;width:100%;height:100%;min-height:380px;max-width:690px;isolation:isolate}.three-farm-viewport .farm-svg-image{position:absolute;inset:0;width:100%;height:100%;m
```

app/src/svg.d.ts

```
declare module "*.svg?react" {
  import type { FunctionComponent, SVGProps } from "react";
  const Component: FunctionComponent<SVGProps<SVGSVGElement>>;
  export default Component;
}
```

app/tests/history-performance.test.ts

```
import { expect, test } from "bun:test";
import { readFileSync } from "node:fs";

const hookSource = readFileSync(
  new URL("../src/components/gallery/use-justified-gallery.ts", import.meta.url),
  "utf8",
);
const tileSource = readFileSync(
  new URL("../src/components/gallery/gallery-tile.tsx", import.meta.url),
  "utf8",
);
const styleSource = readFileSync(
  new URL("../src/components/gallery/gallery.css", import.meta.url),
  "utf8",
);
const assetLibrarySource = readFileSync(
  new URL("../src/components/asset-library.tsx", import.meta.url),
  "utf8",
);

test("history windowing keeps scroll, paging, and media work bounded", () => {
  expect(hookSource).not.toContain("setScrollTop");
  expect(hookSource).toContain("lastAutoLoadKeyRef");
  expect(tileSource).toContain('preload="none"');
  expect(tileSource).not.toContain('preload="auto"');
  expect(tileSource).not.toContain("fastScroll");
  expect(styleSource).not.toContain("data-shimmer");
});

test("asset paging re-arms when a fetched page adds no selectable cards", () => {
  expect(assetLibrarySource).toContain("page?.loading !== true");
});
```

app/tests/landing-contract.test.ts

```
import { describe, expect, test } from "bun:test";
import { readFileSync } from "node:fs";
import { landingContentSchema } from "@higgsfield/app-landing";
import { landingContent } from "../src/landing-content";

describe("scroll-scrub website landing contract", () => {
  test("uses a safe full-app route preview", () => {
    expect(() => landingContentSchema.parse(landingContent)).not.toThrow();
    expect(landingContent.preview).toMatchObject({
      kind: "route",
      src: "/app?preview=1",
      openHref: "/app",
    });
    expect(landingContent.steps.items).toHaveLength(3);
    expect(landingContent.steps.items.map((item) => item.preview.kind)).toEqual([
      "instruction",
      "action",
      "result",
    ]);
    expect(landingContent.features.items).toHaveLength(3);
  });

  test("keeps public landing and full app routes separate", () => {
    const landingRoute = readFileSync(new URL("../src/routes/index.tsx", import.meta.url), "utf8");
    const appRoute = readFileSync(new URL("../src/routes/app.tsx", import.meta.url), "utf8");

    // scroll-scrub's home IS the site: "/" renders the journey instead of the
    // stock LandingPage. Everything else about the split is unchanged.
    expect(landingRoute).toContain("ScrollScrub");
    expect(appRoute).toContain('createFileRoute("/app")');
    expect(appRoute).toContain("previewMode");
  });

  test("ships the canonical generations workspace recipe", () => {
    const layout = readFileSync(new URL("../src/layouts/custom.tsx", import.meta.url), "utf8");

    expect(layout).toContain('id: "generations"');
    expect(layout).toContain('mode="generations"');
    expect(layout).toContain("<UserGenerations demo");
    expect(layout).toContain("<PromptBox.Root");
  });
});
```

app/tests/security-headers.test.ts

```
import { expect, test } from "bun:test";
import { applySecurityHeaders } from "../src/lib/security-headers.server";

test("allows the host approval iframe and preserves the response", async () => {
  const response = applySecurityHeaders(new Response("ok", { status: 201 }));
  expect(response.status).toBe(201);
  expect(await response.text()).toBe("ok");
  expect(response.headers.get("content-security-policy")).toContain(
    "frame-src 'self' https://auth.higgsfield.app https://auth.higgsfield-dev.app;",
  );
  expect(response.headers.get("content-security-policy")).not.toContain("frame-ancestors");
});
```

app/tsconfig.json

```
{
  "include": ["src/**/*.ts", "src/**/*.tsx", "vite.config.ts", "eslint.config.js"],
  "compilerOptions": {
    "target": "ES2022",
    "jsx": "react-jsx",
    "module": "ESNext",
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "types": ["vite/client"],

    /* Bundler mode */
    "moduleResolution": "Bundler",
    "allowImportingTsExtensions": true,
    "verbatimModuleSyntax": false,
    "noEmit": true,
    // Lets the root route import app-meta.json (page metadata committed by the
    // marketplace meta API) at build time.
    "resolveJsonModule": true,

    /* Linting */
    "skipLibCheck": true,
    "strict": true,
    "noUnusedLocals": false,
    "noUnusedParameters": false,
    "noFallthroughCasesInSwitch": true,
    "noUncheckedSideEffectImports": true,
    "paths": {
      "@/*": ["./src/*"],
      "@higgsfield-ai/icons/*": ["./src/lib/quanta-icons.ts"]
    }
  }
}
```

app/tsr.config.json

```
{
  "routesDirectory": "./src/routes",
  "generatedRouteTree": "./src/routeTree.gen.ts",
  "routeTreeFileFooter": [
    "import type { getRouter } from './router.tsx'\nimport type { startInstance } from './start.ts'\ndeclare module '@tanstack/react-start' {\n  interface Register {\n    ssr: true\n    router: Awaite
  ]
}
```

app/vite.config.ts

```
import { tanstackStart } from "@tanstack/react-start/plugin/vite";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import {
  higgsfieldDesignInspectorVitePlugin,
  higgsfieldDesignSourceBabelPlugin,
} from "./src/module/design-inspector/vite";
import svgr from "vite-plugin-svgr";
import { defaultServerConditions, defineConfig } from "vite";
import { fileURLToPath } from "node:url";

// The vendored @higgsfield/quanta components import their glyphs from the private
// Nexus-only `@higgsfield-ai/icons`. Generated sites build on the PUBLIC npm
// registry, so we redirect every `@higgsfield-ai/icons/*` import to a lucide
// shim instead (see src/lib/quanta-icons.ts). tsconfig.json has
// the matching `paths` entry so type-checking resolves it too.
const QUANTA_ICONS_SHIM = fileURLToPath(
  new URL("./src/lib/quanta-icons.ts", import.meta.url),
);

export default defineConfig(({ command, mode }) => {
  const designInspectorEnabled = process.env.HF_DESIGN_INSPECTOR === "1" || mode === "design";

  return {
    // fsevents can miss edits under some setups (bun-launched dev, synced/virtual
    // dirs), leaving HMR dead so changes only appear after a manual restart.
    // Polling the watcher makes file changes reliably trigger HMR / SSR reload.
    server: {
      watch: { usePolling: true, interval: 150 },
    },
    resolve: {
      tsconfigPaths: true,
      alias: [{ find: /^@higgsfield-ai\/icons(\/.*)?$/, replacement: QUANTA_ICONS_SHIM }],
    },
    // The server bundle runs as a Cloudflare Worker — there is no node_modules
    // at runtime. Vite's default SSR build leaves npm deps as bare external
    // imports (h3, react, @tanstack/*, seroval, …), which resolve on a Node
    // server but throw "No such module" in a Worker. Bundle them all in.
    // (node: builtins stay external — nodejs_compat provides them.)
    // BUILD ONLY: `vite dev` SSR runs in Node where externalized deps are
    // correct — noExternal there makes the dev module runner evaluate CJS
    // deps (react) as ESM and crash with "module is not defined".
    ssr: {
      // BUILD ONLY: the SSR bundle runs on workerd (Cloudflare Workers), not
      // Node. Target a worker runtime and resolve bundled deps through the
      // edge export conditions (workerd/worker/browser) so packages that ship
      // both variants bundle their edge build (react-dom's web-streams server,
      // etc.) instead of the Node variant leaning on nodejs_compat shims.
      // `vite dev` SSR runs in Node, where default node resolution is correct.
      ...(command === "build"
        ? {
            target: "webworker" as const,
            resolve: {
              conditions: [
                "workerd",
                "worker",
                "browser",
                ...defaultServerConditions.filter((c) => c !== "node"),
              ],
            },
          }
        : {}),
      noExternal: command === "build" ? true : undefined,
      // `cloudflare:workers` is a workerd runtime built-in that exposes the Worker
      // env / bindings (D1 `DB`, R2 `STORAGE`). Like node: builtins it must NOT be
      // bundled; the runtime provides it. (`ssr.external` is typed string[].)
      external: ["cloudflare:workers"],
    },
```

```
    build: {
      // Keep `cloudflare:*` external in the SSR rollup pass too — `noExternal`
      // above would otherwise try to resolve+bundle it and fail.
      rollupOptions: { external: [/^cloudflare:/] },
    },
    plugins: [
      // Local SVG assets (e.g. the branded generate-button sparkle) import as
      // React components via `?react`. `icon: true` sizes them 1em; fill is
      // forced to currentColor so they color like text. Keep the viewBox so
      // CSS sizing scales the glyph.
      svgr({
        svgrOptions: {
          icon: true,
          svgProps: { fill: "currentColor" },
          svgoConfig: {
            plugins: [{ name: "preset-default", params: { overrides: { removeViewBox: false } } }],
          },
        },
      }),
      // TanStack Start plugin must run before React's plugin.
      //
      // SSR build: `vite build` emits a Workers-shaped server bundle
      // (dist/server/server.js — `export default { fetch }`) plus dist/client
      // (hashed static assets). The platform publishes that as a per-tenant
      // Worker on Workers for Platforms, served at <sub>.higgsfield.app/ (host
      // root, so Vite's default base "/" — no base-path juggling).
      //
      // Rendering happens on the server per request, so site code must be
      // SSR-safe: never touch browser-only globals (window, document,
      // localStorage, navigator) during render or at module top level — only
      // inside effects/handlers, or guarded with `typeof window !== "undefined"`.
      tanstackStart({
        server: { entry: "server" },
      }),
      higgsfieldDesignInspectorVitePlugin(designInspectorEnabled),
      react({
        babel: {
          plugins: designInspectorEnabled ? [higgsfieldDesignSourceBabelPlugin] : [],
        },
      }),
      tailwindcss(),
    ],
  };
});
```

app/wrangler.jsonc

```
{
  "$schema": "node_modules/wrangler/config-schema.json",

  // ■■ BUILD / DEV CONFIG — NOT the deploy config ■■■■■■■■■■■■■■■■■■■■
  // Used for local `wrangler dev`. It is NEVER trusted at deploy: the deploy CI
  // GENERATES a fresh, authoritative wrangler.jsonc from trusted marketplace
  // metadata (your app.manifest.json + the app row) and ignores everything here.
  // So `name` and any binding ids below are just BUILD-VALID PLACEHOLDERS — the
  // real values are injected only at deploy.
  //
  // NOTHING is provisioned by default. Declare the infra you want in
  // app.manifest.json (db / r2 / kv / durableObject); the platform provisions it
  // and binds it at deploy. For LOCAL `wrangler dev` of an opted-in resource,
  // uncomment the matching block in the OPT-IN INFRA section below.
  //
  // `name` must stay a valid Cloudflare name (lowercase + dashes, 3–63 chars).
  "name": "app-slug-placeholder",

  // The Worker entry is the TanStack Start SSR server bundle — `vite build`
  // emits dist/server/server.js (`export default { fetch }`). The deploy CI
  // rediscovers the real built entry and points `main` at it.
  "main": "dist/server/server.js",
  "compatibility_date": "2025-05-01",
  "compatibility_flags": ["nodejs_compat"],
  "observability": { "enabled": true },

  // SSR asset routing: static files in dist/client serve directly (asset-first);
  // any non-asset path — incl. "/" — falls through to the SSR Worker (no
  // index.html ships, so "/" never matches an asset). Do NOT set
  // run_worker_first:true — that makes the Worker intercept /assets/* and 404
  // them (the SSR handler has no asset route).
  "assets": {
    "directory": "./dist/client",
    "binding": "ASSETS",
    "not_found_handling": "none",
  },

  // Non-secret identity for local dev/build; the deploy sets the real values.
  // SECRETS (e.g. the fnf token) are NEVER here and NEVER in the Worker — the
  // platform Outbound Worker injects them on server-side egress.
  "vars": { "HF_ENV": "dev", "APP_SLUG": "app-slug-placeholder" },

  // ■■ OPT-IN INFRA (off by default) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  // Declare the service in app.manifest.json; the deploy injects the real
  // binding. For local `wrangler dev` of an opted-in resource, uncomment the
  // matching block here with a placeholder id (the deploy overwrites it anyway).
  //
  // D1 — app.manifest.json "db": true → env.DB. ONE database SHARED by preview +
  // prod (HF_ENV splits CODE, not DATA — a destructive preview migration hits
  // prod data). Schema in migrations/000N_*.sql (additive).
  //   "d1_databases": [
  //     { "binding": "DB", "database_name": "app-slug-placeholder-db",
  //       "database_id": "00000000-0000-0000-0000-000000000000" }
  //   ],
  //
  // R2 — app.manifest.json "r2": true → env.STORAGE. Shared by preview + prod.
  //   "r2_buckets": [
  //     { "binding": "STORAGE", "bucket_name": "app-slug-placeholder-assets" }
  //   ],
  //
  // KV — app.manifest.json "kv": true → env.KV. ONE namespace shared by preview +
  // prod (like D1). Eventually consistent (NOT Redis): great for config + cached
  // reads, NOT for counters/locks/read-after-write (use a Durable Object there).
  //   "kv_namespaces": [
  //     { "binding": "KV", "id": "00000000000000000000000000000000" }
  //   ],
  //
  // Durable Object — app.manifest.json "durableObject": "Rooms" → env.ROOMS; ALSO
```

```
  // `export class Rooms extends DurableObject {…}` from src/server.ts so the class
  // ships. (Containers / code-sandboxes are not wired through the deploy path yet.)
}
```